

前端面试题汇总

第一部分：基础题

【HTML、HTTP、WEB综合问题】

1. 前端需要注意哪些SEO

前端做SEO，核心是让搜索引擎能抓到内容、看懂页面，并愿意在结果中展示。页面应设置准确且各不相同的title和description，关键词自然靠前，keywords只保留少量相关词即可。结构上使用语义化HTML，把重要内容靠前放置，避免依赖JavaScript输出或藏在iframe中。非装饰图片要写有意义的alt，同时提高网站速度，因为可访问性、抓取效率和加载体验都会影响效果。合理的title、description、keywords：搜索引擎对这些标签的权重逐渐减小。在title中，强调重点即可，重要关键词不要超过2次，并且要靠前。每个页面的title应该有所不同。description应该高度概括页面内容，长度适当，避免过度堆砌关键词。每个页面的description也应该有所不同。keywords标签应列举出重要关键词即可。针对title标签，可以使用重要关键词、品牌词或描述页面内容的短语。确保标题简洁、准确地概括页面的主题，并吸引用户点击。在编写description标签时，应尽量使用简洁、具有吸引力的语句来概括页面的内容，吸引用户点击搜索结果。避免堆砌关键词，以自然流畅的方式描述页面keywords标签已经不再是搜索引擎排名的重要因素，但仍然可以列举出与页面内容相关的几个重要关键词，以便搜索引擎了解页面的主题。语义化的HTML代码，符合W3C规范：使用语义化的HTML代码可以让搜索引擎更容易理解网页的结构和内容。遵循W3C规范可以提高网页的可读性和可访问性，对SEO也有好处。重要内容HTML代码放在最前：搜索引擎抓取HTML的顺序是从上到下，有些搜索引擎对抓取长度有限制。因此，将重要的内容放在HTML的前面，确保重要内容一定会被抓取。重要内容不要用js输出：爬虫不会执行JavaScript，所以重要的内容不应该依赖于通过JavaScript动态生成。确保重要内容在HTML中静态存在。少用iframe：搜索引擎通常不会抓取iframe中的内容，因此应该尽量减少iframe的使用，特别是对于重要的内容。非装饰性图片必须加alt：为非装饰性图片添加alt属性，可以为搜索引擎提供关于图片内容的描述，同时也有助于可访问性。提高网站速度：网站速度是搜索引擎排序的一个重要指标。针对title标签，可以使用重要关键词、品牌词或描述页面内容的短语。确保标题简洁、准确地概括页面的主题，并吸引用户点击。在编写description标签时，应尽量使用简洁、具有吸引力的语句来概括页面的内容，吸引用户点击搜索结果。避免堆砌关键词，以自然流畅的方式描述页面keywords标签已经不再是搜索引擎排名的重要因素，但仍然可以列举出与页面内容相关的几个重要关键词，以便搜索引擎了解页面的主题。

1) 商品详情页视觉完整，但查看初始HTML只有一个根节点，标题、价格和正文都由JavaScript请求后生成，你会担心什么？

答：重要内容不应只依赖JavaScript输出，因为部分爬虫不会执行脚本，可能只抓到空壳页面。应让标题、正文等核心信息静态存在于可抓取的HTML中，并保持语义结构清晰；具体采用服务端输出还是

构建期生成，要结合内容更新方式决定。

2) 一个站点的五千个产品页共用同一套title和description，运营又要求批量堆入十几个关键词，你会怎样调整？

答：每个页面应生成能够准确概括自身内容的独立title和description，重要关键词在标题中靠前且避免重复堆砌。description应简洁自然并服务于搜索结果点击，keywords只列少量相关词即可；模板化仍需保留产品差异，不能生成近乎相同的元信息。

3) 首页改版把品牌介绍、核心产品和主标题都放进首屏iframe，外层只保留导航，你会要求怎么改？

答：核心内容应回到主文档的语义化HTML中，因为搜索引擎通常不会把iframe内内容当作当前页面的重要正文抓取。非核心的第三方模块可继续使用iframe，但页面主题、标题与主要信息必须由外层文档独立表达，否则抓取和可访问性都会受影响。

4) 内容页收录下降，页面源码里正文排在大量弹窗、推荐位和脚本模板之后，你会从前端侧怎么排查？

答：先检查重要正文是否真实存在于初始HTML，再确认语义标签和标题层级能清楚表达页面结构，并尽量把核心内容的代码位置前移。部分搜索引擎按从上到下抓取且可能限制抓取长度，冗长的非核心结构会增加风险；同时还要检查加载速度是否恶化。

5) 媒体站为了提速准备删除所有图片的alt，理由是属性会增加HTML体积，你会同意吗？

答：不同意一刀切删除；承载内容的图片必须提供准确的alt，既帮助搜索引擎理解图像，也改善无法看到图片时的可访问性。纯装饰图片不需要描述性文本，可使用空`alt`；优化速度应优先处理资源尺寸和加载方式，而不是牺牲内容语义。

6) 研发只盯着元标签做SEO，性能负责人认为页面速度和HTML结构更重要，你会如何划分优先级？

答：两者都要处理：独立且准确的title、description能表达页面主题，语义化HTML、核心内容前置和非脚本依赖则决定爬虫能否理解内容。网站速度也是搜索排序的重要指标，因此不能用元标签掩盖慢加载；应先保证可抓取内容，再持续优化资源与渲染速度。

2. 的title和alt有什么区别

alt是图片内容的替代描述，title则是用户悬停时看到的补充提示。图片加载失败或由读屏器访问时，真正代替图片传递信息的是alt，搜索引擎也会重点分析它。title属于通用属性，可以补充来源、作者等额外信息，但不是图片必需属性。除纯装饰图片外，我一般都会写简洁准确的alt，同时避免为了SEO堆砌关键词。

title属性：title属性是HTML元素通用的属性，适用于各种元素，不仅仅是标签。当鼠标滑动到元素上时，浏览器会显示title属性的内容，提供额外的信息或解释，帮助用户了解元素的用途或含义。对于标签，鼠标悬停在图片上时会显示title属性的内容。

alt属性：alt属性是标签的特有属性，用于提供图片的替代文本描述。当图片无法加载时，浏览器会显示alt属性的内容，或者在可访问性场景中，读屏器会读取alt属性的内容。alt属性的主要目的是提高图片的可访问性，使无法查看图片的用户也能了解图片的内容或含义。除了纯装饰性图片外，所有标签都应该设置有意义的alt属性值。

补充答案：title属性主要用于提供额外的信息或提示，是对图片的补充描述，可以用于提供更详细的说明，如图片的来源、作者、相关信息等。它不是必需的，但可以增强用户体验，特别是在需要显示更多信息时。

alt属性是图片内容的等价描述，应该简洁明了地描述图片所表达的信息。它对于可访问性至关重要，确保无障碍用户能够理解图片的含义，同时也是搜索引擎重点分析的内容。在设置alt属性时，应该避免过度堆砌关键词，而是提供准确、有意义的描述。

title属性：title属性是HTML元素通用的属性，适用于各种元素，不仅仅是标签。当鼠标滑动到元素上时，浏览器会显示title属性的内容，提供额外的信息或解释，帮助用户了解元素的用途或含义。对于标签，鼠标悬停

在图片上时会显示title属性的内容。 alt属性：alt属性是标签的特有属性，用于提供图片的替代文本描述。当图片无法加载时，浏览器会显示alt属性的内容，或者在可访问性场景中，读屏器会读取alt属性的内容。alt属性的主要目的是提高图片的可访问性，使无法查看图片的用户也能了解图片的内容或含义。除了纯装饰性图片外，所有标签都应该设置有意义的alt属性值。 补充答案： title属性主要用于提供额外的信息或提示，是对图片的补充描述，可以用于提供更详细的说明，如图片的来源、作者、相关信息等。它不是必需的，但可以增强用户体验，特别是在需要显示更多信息时。alt属性是图片内容的等价描述，应该简洁明了地描述图片所表达的信息。它对于可访问性至关重要，确保无障碍用户能够理解图片的含义，同时也是搜索引擎重点分析的内容。在设置alt属性时，应该避免过度堆砌关键词，而是提供准确、有意义的描述。

1) 商品列表里一张主图同时写了alt="红色连衣裙"和title="红色连衣裙"，鼠标悬停正常、读屏却听到重复信息，你会保留哪一个？

答：应优先保留准确的alt，因为它承担图片内容的等价描述，并在图片加载失败或读屏场景中发挥作用。title只是补充提示，不应复制alt；没有额外来源、作者等信息时可以去掉，且不能依赖悬停提示传递关键信息。

2) 新闻详情页的图片加载失败后只剩一个破图标，代码是，为什么title不能替代alt？

答：应补上能表达图片信息的alt，因为它才是图片无法显示时的替代文本，也是读屏器理解图片的主要依据。title属于通用补充属性，通常依赖鼠标悬停展示；触屏、键盘和辅助技术环境下都不能保证获得同等信息。

3) 设计师要求每张轮播图都显示摄影师和拍摄地点，产品又要求读屏用户只听到图片主题，你会怎样拆分alt与title？

答：alt应简洁描述轮播图表达的主题，使无法查看图片的用户获得等价信息；摄影师、地点等补充资料可放入title或页面可见文字。若这些资料也是理解内容所必需，就不应只藏在悬停提示中，而应直接呈现在正文或图片说明里。

4) 无障碍巡检发现首页二百张缩略图都没有alt，但多数图片只是卡片装饰，你会如何判断并修复？

答：先按图片是否传递信息分类：内容图片补充准确且有意义的alt，纯装饰图片使用空替代文本，避免制造无关朗读。不能批量把文件名或统一关键词填进去，否则既降低可访问性，也可能形成搜索关键词堆砌；边界应由图片在当前页面的用途决定。

5) 运营认为给所有图片堆满商品关键词能提升搜索排名，无障碍负责人要求描述自然准确，你会支持哪种方案？

答：应支持自然、准确且简洁的alt，它既服务无法查看图片的用户，也是搜索引擎重点分析的信息。堆砌关键词会偏离图片的等价描述，可能让读屏内容难以理解；营销补充信息应放在标题、正文或结构化内容中，而不是污染alt。

3. HTTP的几种请求方法用途

HTTP请求方法用来表达对资源的操作意图，常见的是获取、提交、替换和删除。GET用于获取资源，参数通常放在URL中，属于安全且幂等的方法；POST把数据放进请求体提交，重复请求可能产生不同结果。PUT通常替换指定位置的资源并具有幂等性，DELETE用于删除资源，HEAD则只获取响应首部。OPTIONS可查询服务端支持的方法，TRACE用于请求消息回路，CONNECT用于建立透明的TCP/IP通道。 GET方法：用途：发送一个请求来获取服务器上的某一资源。面试可能涉及的问题：GET方法的特点是什么？GET方法是HTTP的一种请求方法，用于从服务器获取资源。它是一种幂等的方法，多次发送相同的GET请求会返回相同的结果。GET请求和POST请求的区别是什么？GET请求将参数附加在URL的查询字符串中，而POST请求将参数放在请求体中。GET请求的数据会显示在URL中，而POST请求的数据不会显示在URL中。GET请求一般用

于获取数据，而POST请求一般用于提交数据。GET请求可以有请求体吗？根据HTTP规范，GET请求不应该有请求体，参数应该通过URL的查询字符串传递。GET请求的参数如何传递？GET请求的参数可以通过URL的查询字符串传递，例如：`/api/users?id=123&name=poetry`。GET请求的安全性和幂等性如何保证？GET请求不会对服务器端的资源产生副作用，因此被视为安全的。GET请求是幂等的，多次发送相同的GET请求不会对服务器端产生影响。POST方法：用途：向URL指定的资源提交数据或附加新的数据。面试可能涉及的问题：POST方法的特点是什么？POST方法是HTTP的一种请求方法，用于向服务器提交数据。它不是幂等的，多次发送相同的POST请求可能会产生不同的结果。POST请求和GET请求的区别是什么？POST请求将参数放在请求体中，而GET请求将参数附加在URL的查询字符串中。POST请求的数据不会显示在URL中，而GET请求的数据会显示在URL中。POST请求一般用于提交数据，而GET请求一般用于获取数据。POST请求的请求体如何传递数据？POST请求的数据可以通过请求体以表单形式传递，或者以JSON等格式传递。POST请求的安全性和幂等性如何保证？POST请求可能对服务器端的资源产生副作用，因此被视为不安全的。POST请求不是幂等的，多次发送相同的POST请求可能会对服务器端产生影响。PUT方法：用途：将数据发送给服务器，并将其存储在指定的URL位置。与POST方法不同的是，PUT方法指定了资源在服务器上的位置。面试可能涉及的问题：PUT方法的特点是什么？PUT方法是HTTP的一种请求方法，用于将数据发送给服务器并存储在指定的URL位置。它是一种幂等的方法，多次发送相同的PUT请求会对服务器端产生相同的结果。PUT请求和POST请求有什么区别？PUT请求用于指定资源在服务器上的位置，而POST请求没有指定位置。PUT请求一般用于更新或替换资源，而POST请求一般用于新增资源或提交数据。PUT请求的幂等性如何保证？PUT请求的幂等性保证是由服务器端实现的。服务器端应该根据请求中的资源位置来处理请求，多次发送相同的PUT请求会对该位置上的资源进行相同的更新或替换操作。HEAD方法只请求页面的首部DELETE方法删除服务器上的某资源OPTIONS方法它用于获取当前URL所支持的方法。如果请求成功，会有一个Allow的头包含类似“GET,POST”这样的信息TRACE方法TRACE方法被用于激发一个远程的，应用层的请求消息回路CONNECT方法把请求连接转换到透明的TCP/IP通道 用途：发送一个请求来获取服务器上的某一资源。面试可能涉及的问题：GET方法的特点是什么？GET方法是HTTP的一种请求方法，用于从服务器获取资源。它是一种幂等的方法，多次发送相同的GET请求会返回相同的结果。GET请求和POST请求的区别是什么？GET请求将参数附加在URL的查询字符串中，而POST请求将参数放在请求体中。GET请求的数据会显示在URL中，而POST请求的数据不会显示在URL中。GET请求一般用于获取数据，而POST请求一般用于提交数据。GET请求可以有请求体吗？根据HTTP规范，GET请求不应该有请求体，参数应该通过URL的查询字符串传递。GET请求的参数如何传递？GET请求的参数可以通过URL的查询字符串传递，例如：`/api/users?id=123&name=poetry`。GET请求的安全性和幂等性如何保证？GET请求不会对服务器端的资源产生副作用，因此被视为安全的。GET请求是幂等的，多次发送相同的GET请求不会对服务器端产生影响。GET方法的特点是什么？GET方法是HTTP的一种请求方...

1) 订单查询接口用了GET `/orders?id=123`，后端却顺便把订单标记为已读；刷新页面就改变服务器状态，这仍算安全、幂等的GET吗？

答：这种设计不符合GET应无副作用的安全语义，刷新、预取或缓存重放都可能意外修改状态。查询与标记已读应拆开，查询继续使用GET，状态变更改用提交类方法；即使重复标记结果相同，也不能据此把有副作用的操作称为安全。

2) 文件管理页要把固定地址`/files/report.pdf`的内容整体替换，客户端断线后可能自动重试，你选PUT还是POST？

答：更适合使用PUT `/files/report.pdf`，因为资源位置由请求明确指定，重复发送相同内容应得到相同的服务器状态。POST通常用于向目标提交数据或新增资源，重复调用可能产生不同结果；不过PUT的幂等性仍需服务端按资源位置正确实现。

3) 搜索团队想把包含复杂筛选条件的请求改成带请求体的GET，网关团队只处理查询字符串，你会怎样定接口？

答：按给定规范，应把GET参数放入URL查询字符串，例如/api/users?id=123&name=poetry，不依赖请求体传参。若筛选数据无法合理编码进URL，可改用POST提交查询条件，但会改变缓存、语义和幂等预期，需要客户端、网关与服务端共同约定。

4) 线上跨域预检返回成功，但浏览器随后提示目标接口不允许DELETE，你会先检查哪个响应头和服务端配置？

答：先检查OPTIONS响应及其声明的允许方法，确认其中是否包含DELETE，并核对目标URL的路由配置。OPTIONS用于获知当前URL支持的方法，成功响应通常通过允许方法相关响应头表达能力；只放行预检却未开放实际方法，后续请求仍会失败。

5) 运维要探测一个十兆字节下载地址是否可用，又不希望传输文件正文，接口在GET与HEAD之间怎么选？

答：应优先使用HEAD，它只请求资源的首部，可检查状态和相关元信息而不获取页面实体。GET会取得完整资源，带来不必要的带宽与传输时间；但前提是服务器正确实现HEAD，否则仍需用受控的GET作为兼容方案。

6) 企业代理要求浏览器通过代理服务器建立加密隧道，评审会上有人建议用TRACE，这里真正相关的方法是什么？

答：真正相关的是CONNECT，它用于把请求连接转换为透明的TCP/IP通道，常见于经代理建立隧道的场景。TRACE用于触发应用层请求消息回路，目的不是建立隧道；两者都不是普通页面资源的增删改查方法，部署时还受代理和安全策略约束。

4. 从浏览器地址栏输入url到显示页面的步骤

从输入URL到页面显示，本质上经历资源定位、网络请求、服务端响应和浏览器渲染。浏览器先检查缓存并解析URL，再通过缓存、hosts或DNS获得IP，建立TCP连接后发送HTTP请求。服务端处理请求并返回响应，浏览器根据状态码处理结果，必要时解压、缓存或复用连接。拿到HTML后会构建DOM，解析样式生成CSSOM，再形成渲染树并逐步显示页面。普通脚本可能阻塞解析，而async和defer的下载、执行时机不同，最终还会触发DOMContentLoaded和load。基础版本 浏览器根据请求的URL交给DNS域名解析，找到真实IP，向服务器发起请求；服务器交给后台处理完成后返回数据，浏览器接收文件（HTML、JS、CSS、图像等）；浏览器对加载到的资源（HTML、JS、CSS等）进行语法解析，建立相应的内部数据结构（如HTML的DOM）；载入解析到的资源文件，渲染页面，完成。详细版 在浏览器地址栏输入URL浏览器查看缓存，如果请求资源在缓存中并且新鲜，跳转到转码步骤如果资源未缓存，发起新请求如果已缓存，检验是否足够新鲜，足够新鲜直接提供给客户端，否则与服务器进行验证。检验新鲜通常有两个HTTP头进行控制Expires和Cache-Control：HTTP1.0提供Expires，值为一个绝对时间表示缓存新鲜日期HTTP1.1增加了Cache-Control: max-age=,值为以秒为单位的最大新鲜时间浏览器解析URL获取协议，主机，端口，path浏览器组装一个HTTP（GET）请求报文浏览器获取主机ip地址，过程如下：浏览器缓存本机缓存hosts文件路由器缓存ISP DNS缓存DNS递归查询（可能存在负载均衡导致每次IP不一样）打开一个socket与目标IP地址，端口建立TCP链接，三次握手如下：客户端发送一个TCP的SYN=1，Seq=X的包到服务器端口服务器发回SYN=1，ACK=X+1，Seq=Y的响应包客户端发送ACK=Y+1，Seq=ZTCP链接建立后发送HTTP请求服务器接受请求并解析，将请求转发到服务程序，如虚拟主机使用HTTP Host头部判断请求的服务程序服务器检查HTTP请求头是否包含缓存验证信息如果验证缓存新鲜，返回304等对应状态码处理程序读取完整请求并准备HTTP响应，可能需要查询数据库等操作服务器将响应报文通过TCP连接发送回浏览器浏览器接收HTTP响应，然后根据情况选择关闭TCP连接或者保留重用，关闭TCP连接的四次握手如下：主动方发送Fin=1，Ack=Z，Seq= X报文被动方发送ACK=X+1，Seq=Z报文被动方发送Fin=1，ACK=X，Seq=Y报文主动方发送ACK=Y，Seq=X报文浏览器检查响应状态码：是否为1XX，3XX，4XX，5XX，这些情况处理与2XX不同如果资源可缓存，进行缓存对响应进行解码（例如gzip压缩）根据资源类型决定如何

处理（假设资源为HTML文档）解析HTML文档，构件DOM树，下载资源，构造CSSOM树，执行js脚本，这些操作没有严格的先后顺序，以下分别解释构建DOM树：Tokenizing：根据HTML规范将字符流解析为标记Lexing：词法分析将标记转换为对象并定义属性和规则DOM construction：根据HTML标记关系将对象组成DOM树解析过程中遇到图片、样式表、js文件，启动下载构建CSSOM树：Tokenizing：字符流转换为标记流Node：根据标记创建节点CSSOM：节点创建CSSOM树根据DOM树和CSSOM树构建渲染树(opens new window):从DOM树的根节点遍历所有可见节点，不可见节点包括：1) script,meta这样本身不可见的标签。2)被css隐藏的节点，如display: none对每一个可见节点，找到恰当的CSSOM规则并应用发布可视节点的内容和计算样式js解析如下：浏览器创建Document对象并解析HTML，将解析到的元素和文本节点添加到文档中，此时document.readyState为loadingHTML解析器遇到没有async和defer的script时，将他们添加到文档中，然后执行行内或外部脚本。这些脚本会同步执行，并且在脚本下载和执行时解析器会暂停。这样就可以用document.write()把文本插入到输入流中。同步脚本经常简单定义函数和注册事件处理程序，他们可以遍历和操作script和他们之前的文档内容当解析器遇到设置了async属性的script时，开始下载脚本并继续解析文档。脚本会在它下载完成后尽快执行，但是解析器不会停下来等它下载。异步脚本禁止使用document.write()，它们可以访问自己script和之前的文档元素当文档完成解析，document...

1) 用户第二次打开商品页时命中了磁盘缓存，就有人说浏览器不会再解析URL、构建DOM，这句话哪里不对？

答：缓存命中只可能省去或缩短网络获取与服务器处理，并不会让HTML文档跳过解析和渲染。浏览器仍需识别资源类型、解码内容、解析HTML构建DOM，结合CSSOM生成渲染树并显示页面；缓存内容失效时还要携带验证信息与服务器协商。

2) 一个页面首屏依赖三个域名的静态资源，地址栏回车后长时间没有请求正文，你会沿哪条链路定位卡点？

答：先按URL解析、缓存检查、DNS查询、连接建立和请求发送的顺序查看网络瀑布。DNS阶段可涉及浏览器、本机、hosts、路由器和ISP缓存，之后才会向目标IP与端口建立连接；若卡在其中一层，继续分析DOM或脚本执行没有意义。

3) 安全团队把站点从HTTP切到加密连接，但业务同学仍只描述“三次握手后发送请求”，你会要求补充什么边界？

答：三次握手描述的是建立TCP连接，不能完整代表加密连接准备过程；在发送受保护的HTTP数据前，还存在与加密协议相关的协商。详细步骤会随实际协议和连接复用方式变化，因此面试中应先声明讨论的是基础HTTP/TCP流程，避免把所有访问都简化成同一链路。

4) 线上首页HTML很快返回，但同步外链脚本迟迟下载不完，DOMContentLoaded一直不触发；解析器此时发生了什么？

答：没有async或defer的外链脚本会暂停HTML解析器，等待脚本下载并同步执行，因此后续DOM尚未完成构建。文档解析完成且defer脚本按顺序执行后才触发DOMContentLoaded；图片等资源通常可继续影响load，不能把两个事件混为一谈。

5) 页面出现样式错乱后又恢复，开发者只检查DOM，你会怎样串起CSSOM、渲染树和显示过程排查？

答：除DOM外还要确认样式表是否及时下载并正确构建CSSOM，因为渲染树需要为可见节点匹配计算后的样式。随后检查布局、绘制和合成阶段是否反复发生；被display: none隐藏或本身不可见的节点不会按普通可见节点进入渲染树。

6) 接口返回304后页面仍显示旧内容，前端、网关和服务端互相推责，你会先核对哪些环节？

答：先核对请求是否携带缓存验证信息、服务器为何判定资源未修改，以及浏览器实际复用了哪份本地缓存。304表示客户端应继续使用缓存实体，而不是返回新正文；若内容本应更新，就要追查验证标识、缓存规则或发布链路，而不是从HTML解析阶段寻找原因。

5. 如何进行网站性能优化

网站性能优化本质上是减少传输和请求成本，同时降低页面渲染与脚本执行的开销。我一般会先处理收益较明确的部分，例如启用CDN、缓存和Gzip，压缩JavaScript、CSS与图片，并按需加载非首屏资源。页面侧要减少不必要的DOM操作，缓存节点查询结果，批量改样式时优先切换className。具体方案需要结合性能工具定位，不能为了减少请求而盲目合并所有资源。

content方面减少HTTP请求：合并文件、CSS精灵、inline Image减少DNS查询：DNS缓存、将资源分布到恰当数量的主机名减少DOM元素数量Server方面使用CDN配置ETag对组件使用Gzip压缩Cookie方面减小cookie大小css方面将样式表放到页面顶部不使用CSS表达式使用不使用@import Javascript方面将脚本放到页面底部将javascript和css从外部引入压缩javascript和css删除不需要的脚本减少DOM访问图片方面优化图片：根据实际颜色需要选择色深、压缩优化css精灵不要在HTML中拉伸图片

减少HTTP请求：合并文件、CSS精灵、inline

Image减少DNS查询：DNS缓存、将资源分布到恰当数量的主机名减少DOM元素数量

使用CDN配置ETag对组件使用Gzip压缩

减小cookie大小

将样式表放到页面顶部不使用CSS表达式使用不使用@import

将脚本放到页面底部将javascript和css从外部引入压缩javascript和css删除不需要的脚本减少DOM访问

优化图片：根据实际颜色需要选择色深、压缩优化css精灵不要在HTML中拉伸图片

你有用过哪些前端性能优化的方法？

减少http请求次数：CSS

Sprites、JS、CSS源码压缩、图片大小控制合适；网页Gzip，CDN托管，data缓存

，图片服务器。前端模板 JS+数据，减少由于HTML标签导致的带宽浪费，前端用变量保存AJAX请求结果，每次操作本地变量，不用请求，减少请求次数用innerHTML代替DOM操作，减少DOM操作次数，优化javascript性能。当需要设置的样式很多时设置className而不是直接操作style少用全局变量、缓存DOM节点查找的结果。减少IO读取操作避免使用CSS

Expression (css表达式)又称Dynamic

properties(动态属性)图片预加载，将样式表放在顶部，将脚本放在底部 加上时间戳避免在页面的主体布局

中使用table，table要等其中的内容完全下载之后才会显示出来，显示比div+css布局慢 谈谈性能优化问题

代码层面：避免使用css表达式，避免使用高级选择器，通配选择器缓存利用：缓存Ajax，使用CDN，使用

外部js和css文件以便缓存，添加Expires头，服务端配置ETag，减少DNS查找等请求数量：合并样式和脚本

，使用css图片精灵，初始首屏之外的图片资源按需加载，静态资源延迟加载请求带宽：压缩文件，开启GZ

IP 前端性能优化最佳实践？ 性能评级工具（PageSpeed或YSlow）合理设置HTTP缓存：Expires与Cache-

control静态资源打包，开启 Gzip 压缩（节省响应流量）CSS3模拟图像，图标base64（降低请求数）模块

延迟(defer)加载/异步(async)加载Cookie隔离（节省请求流量）localStorage（本地存储）使用CDN加速

（访问最近服务器）启用HTTP/2（多路复用，并行加载）前端自动化（gulp/webpack）

1) 一个HTTP/2页面仍把二十个脚本强行合成单个大文件，团队理由是“减少请求一定更快”，你会如何评审？

答：减少请求是常见优化方向，但启用HTTP/2后可利用多路复用，不能只按请求数量判断收益。大文件可能让首屏下载并执行不需要的代码，失去按需加载能力；应结合首屏依赖、缓存复用和传输体积评估，不能机械沿用合并策略。

2) 图片站首屏有五十张原图，HTML又把它们缩成缩略图展示，带宽持续升高，你会先改哪两处？

答：先按实际展示尺寸和颜色需求生成、压缩合适的图片，避免依赖HTML拉伸原图；首屏之外的图片再按需或延迟加载。这样同时减少响应流量和初始请求压力，但预加载只应留给确实马上需要的资源，否则会抢占首屏网络带宽。

3) 广告业务要求所有静态资源走六个子域名以并行下载，基础设施已启用HTTP/2，你会如何处理域名数量与CDN的冲突？

答：CDN仍可用于让用户访问较近的服务器，但资源域名数量应控制在恰当范围，因为每个新主机名都可能增加DNS查询与连接成本。HTTP/2已支持同一连接多路复用，盲目拆域未必有利；最终要依据实际网络链路和缓存策略验证。

4) 线上页面脚本执行时频繁查找并修改上千个节点，网络面板却很健康，你会把优化重点放在哪里？

答：重点应转向JavaScript与DOM操作，缓存节点查找结果、减少重复DOM访问，并在批量设置样式时优先切换className。若大量字符串内容适合一次更新，可评估innerHTML，但它可能覆盖节点和事件状态，不能替代所有精细DOM操作。

5) 缓存负责人主张给所有接口无限期本地缓存，业务负责人要求数据实时更新，你会怎样划分策略？

答：静态资源可通过Expires、Cache-Control和ETag提高复用率，稳定的AJAX结果也可按业务容忍度缓存。实时数据不能仅为性能长期复用旧值，应设置合理新鲜期或验证机制；缓存降低请求与带宽，却会引入失效和一致性成本。

6) 页面传输体积很大，服务端已启用CDN，但每个请求仍携带较大的登录Cookie，你会怎样继续优化？

答：应减小Cookie内容，并评估静态资源是否需要携带业务身份信息，避免在大量请求中重复传输无关数据。CDN解决的是分发距离和缓存能力，不能自动消除请求头开销；隔离时还要保证认证范围、域配置和业务访问权限不被破坏。

6. HTTP状态码及其含义

HTTP状态码用于告诉客户端请求处于什么结果，以及接下来应该如何处理。2XX表示请求成功或已被接受，3XX通常涉及重定向或缓存，例如301是永久移动，302是临时跳转，304表示资源没有修改。4XX是客户端侧问题，其中401表示未授权，403表示禁止访问，404表示资源不存在。5XX则说明服务端处理失败或暂时不可用，例如500和503。

1XX：信息状态码100 Continue继续，一般在发送post请求时，已发送了http header之后服务端将返回此信息，表示确认，之后发送具体参数信息2XX：成功状态码200

OK正常返回信息201 Created请求成功并且服务器创建了新的资源202

Accepted服务器已接受请求，但尚未处理3XX：重定向301 Moved

Permanently请求的网页已永久移动到新位置。302 Found临时性重定向。303 See

Other临时性重定向，且总是使用GET请求新的URI。304 Not

Modified自从上次请求后，请求的网页未修改过。4XX：客户端错误400 Bad

Request服务器无法理解请求的格式，客户端不应当尝试再次使用相同的内容发起请求。401

Unauthorized请求未授权。403 Forbidden禁止访问。404 Not

Found找不到如何与URI相匹配的资源。5XX:服务器错误500 Internal Server

Error最常见的服务器端错误。503 Service Unavailable服务器端暂时无法处理请求（可能是过载或维护）

100 Continue继续，一般在发送post请求时，已发送了http

header之后服务端将返回此信息，表示确认，之后发送具体参数信息 200 OK正常返回信息201

Created请求成功并且服务器创建了新的资源202 Accepted服务器已接受请求，但尚未处理 301 Moved

Permanently请求的网页已永久移动到新位置。302 Found临时性重定向。303 See

Other临时性重定向，且总是使用GET请求新的URI。304 Not
Modified自从上次请求后，请求的网页未修改过。400 Bad
Request服务器无法理解请求的格式，客户端不应当尝试再次使用相同的内容发起请求。401
Unauthorized请求未授权。403 Forbidden禁止访问。404 Not Found找不到如何与URI相匹配的资源。
500 Internal Server Error最常见的服务器端错误。503 Service
Unavailable服务器端暂时无法处理请求（可能是过载或维护）

1) 创建用户接口成功后返回200，客户端却要猜资源是否真的创建；评审时你会建议改成哪个状态码？

答：若请求成功并创建了新资源，更明确的状态码是201 Created，它能直接表达创建结果。200 OK只表示请求正常成功，语义较宽；若服务器只是接收任务但尚未完成处理，则应考虑202 Accepted，不能提前宣称资源已创建。

2) 批量导出报表需要后台处理几分钟，接口收到任务后立刻返回，产品坚持用201，你会怎样判断？

答：若服务器只是接受请求、尚未完成导出，应返回202 Accepted，让客户端按约定查询后续状态。只有已经成功创建相应资源时，201才贴合语义；202也不保证最终处理成功，因此还需要任务状态与失败反馈机制。

3) 登录用户能访问项目列表，但打开管理员审计页时权限不足，接口应该返回401还是403？

答：用户身份已确认但没有访问该页面的权限，应返回403 Forbidden。401 Unauthorized更适合请求未获得有效授权的情况；若系统出于资源隐藏策略统一返回404，那属于额外安全约定，不能把三个状态码的基本语义混用。

4) CDN回源突然大量出现304，但浏览器仍重复下载完整资源，你会检查响应链路里的什么现象？

答：先确认浏览器收到的确实是304 Not Modified，并且本地仍保存着可复用的资源实体；304本身表示资源自上次请求后未修改。若完整正文仍被传输，应排查中间缓存、状态码改写或客户端缓存丢失，不能把正常的协商响应当成新内容。

5) 旧域名迁移尚未验证完成，SEO要求直接上301，运维希望先用302，你支持哪一方？

答：迁移尚未确认稳定时应先使用302

Found表达临时重定向，验证目标地址和链路后再决定是否改为301 Moved Permanently。301明确表示永久移动，语义和缓存影响更强；临时阶段贸然使用会提高错误迁移的修复成本。

6) 大促期间接口一会儿成功、一会儿因过载失败，监控只记成统一的500，你会要求区分哪个状态码？

答：由过载或维护导致的暂时不可处理应区分为503 Service Unavailable，而500 Internal Server Error表示一般性的服务器内部错误。区分后监控和客户端才能判断故障性质；但是否重试仍需结合请求是否幂等及服务端策略，不能看到503就无限重放。

7. 语义化的理解

语义化就是用合适的标签和属性准确描述页面内容，而不是把所有结构都堆成。这样浏览器、搜索引擎和屏幕阅读器更容易理解页面层级，也能改善SEO、可访问性以及代码维护体验。实际开发中，我会用、
、
表达结构，为图片提供alt，并让表单控件与标签正确关联。语义标签不能代替合理的层级和嵌套，选错标签同样会让结构变得含糊。语义化是指在编写HTML和CSS代码时，通过恰当的选择标签和属性，使得代码更具有语义性和可读性，使得页面结构和内容更加清晰明了。语义化的目的是让页面具备良好的可访问性、可维护性和可扩展性。语义化的重要性体现在以下几个方面：可访问性（Accessibility）：通过使用恰当

的标签和属性，可以提高页面的可访问性，使得辅助技术（如屏幕阅读器）能够更好地理解和解析页面内容，使得残障用户能够正常浏览和使用网页。搜索引擎优化（SEO）：搜索引擎更喜欢能够理解和解析的页面内容，语义化的HTML结构可以提高页面在搜索引擎结果中的排名，增加网页的曝光和访问量。代码可读性和可维护性：使用语义化的标签和属性，可以让代码更易于阅读和理解，提高代码的可维护性。开发人员可以更快速地定位和修改特定功能或内容。设备兼容性：不同设备和平台对于网页的渲染和解析方式有所不同，语义化的代码可以增加网页在各种设备上的兼容性，确保页面在不同环境中的正确显示和使用。语义化在前端开发中的具体表现和实践包括以下几个方面：

- 选择合适的HTML标签：在构建页面结构时，选择恰当的HTML标签来描述内容的含义。例如，使用表示页面的页眉，表示导航栏，表示独立的文章内容等。
- 使用有意义的标签和属性：避免滥用标签，而是选择更具语义的标签来表达内容的含义。同时，合理使用标签的属性，如alt属性用于图像的替代文本，title属性用于提供额外的信息等。
- 结构和层次化：通过正确嵌套和组织HTML元素，构建清晰的页面结构和层次关系。使用语义化的父子关系，让内容的层级关系更加明确，便于样式和脚本的编写和维护。
- 文本格式化：使用合适的标签和属性来标记文本的格式和语义。例如，使用标签表示重要文本，标签表示强调文本，标签表示引用文本等。
- 无障碍支持：考虑到残障用户的需求，使用语义化的标签和属性可以提高页面的可访问性。例如，为表格添加适当的表头和描述信息，为表元素关联标签等。
- CSS选择器的语义化：在编写CSS样式时，尽量使用具有语义的类名和ID，避免过于依赖元素标签选择器，以增强样式的可读性和可维护性。

通过遵循语义化的原则，我们能够构建出更具有可读性、可访问性和可维护性的前端代码，提高用户体验和开发效率。同时，也能够使网页在不同的环境和设备上保持一致的表现，增强网站的可持续性和可扩展性。

总结 用正确的标签做正确的事情！HTML语义化就是让页面的内容结构化，便于对浏览器、搜索引擎解析；在没有样式CSS情况下也以一种文档格式显示，并且是容易阅读的。搜索引擎的爬虫依赖于标记来确定上下文和各个关键字的权重，利于SEO。使阅读源代码的人对网站更容易将网站分块，便于阅读维护理解

1) 商品详情页把价格、库存和购买按钮全部写成带类名的div，视觉效果正常，关闭CSS后却无法辨认结构，这能算语义化吗？

答：不能，视觉样式相同不代表内容含义相同，应使用标题、段落、列表和按钮等匹配职责的元素。语义结构能让无样式文档仍可阅读，也便于浏览器、搜索引擎和辅助技术解析；若只为布局包裹内容，保留div反而更准确。

2) 后台系统有一个用div模拟的“提交”控件，鼠标能点击，但键盘用户无法操作，你会要求怎么改？

答：应优先替换为原生button，让控件直接获得按钮语义和浏览器提供的基础交互能力。若受组件框架限制只能保留div，就必须补齐可聚焦、键盘操作和状态表达，但这些手工逻辑更容易遗漏，也增加维护成本。

3) 资讯页每个广告容器都用了section，但其中既没有标题，也不能作为独立内容理解，你会保留这些标签吗？

答：不会，仅用于样式或栅格布局的容器更适合使用div。section应表达页面中的一个内容章节，并通过清晰的标题和嵌套关系体现层次；滥用语义元素会让文档结构变得含混，不能仅凭标签名称提升语义。

4) 表单上线后，读屏器只播报输入框类型，不播报“手机号”和“验证码”，但页面上肉眼能看到文字，你先排查哪里？

答：先检查可见文本是否通过label与对应表单控件建立了明确关联，而不是仅靠位置或占位符暗示含义。随后核对控件嵌套、标识符和错误提示的结构；只增加视觉文字无法保证辅助技术理解，额外属性也不能弥补错误的基础结构。

5) 产品认为把所有页面改成header、nav、article和footer就能直接提高搜索排名，研发该怎样判断这项改造？

答：这项改造的直接价值是帮助搜索引擎识别正文、导航和内容层级，而不能保证排名必然提升。应先修正标题结构、正文边界和错误嵌套，再结合内容质量评估收益；若只是机械替换div，可能改善可读性，却未必产生可观的搜索结果变化。

8. 介绍一下你对浏览器内核的理解？

浏览器内核可以理解为负责页面渲染的引擎，以及负责解析和执行JavaScript的引擎。渲染引擎会解析HTML、样式和图像，完成样式计算、布局与绘制；常见实现包括WebKit、Blink、Gecko和旧版IE使用的Trident。脚本执行则由V8、SpiderMonkey、JavaScriptCore等引擎负责。不同内核对页面的解释和渲染可能存在差异，因此涉及布局、样式或交互时仍要做兼容性测试。

浏览器内核是浏览器的核心组成部分，主要分为两个部分：渲染引擎（也称为布局引擎或渲染引擎）和JavaScript引擎。渲染引擎：渲染引擎负责解析网页的HTML、XML、图像等内容，并将其转换为可视化的网页形式展示给用户。它负责处理网页的布局、样式计算、绘制等任务。不同浏览器的内核对网页的解释和渲染方式可能会有差异，因此不同浏览器的渲染效果也会有所不同。常见的渲染引擎包括：WebKit：主要用于Safari和Chrome浏览器。Gecko：主要用于Firefox浏览器。Trident：主要用于旧版本的Internet Explorer浏览器。Blink：基于WebKit，用于Chrome、Opera和部分Chromium浏览器。JavaScript引擎：JavaScript引擎负责解析和执行网页中的JavaScript代码，实现网页的动态交互和功能。不同浏览器的JavaScript引擎性能和特性也可能存在差异。常见的JavaScript引擎包括：V8：用于Chrome和Opera浏览器，具有高性能和快速执行速度。SpiderMonkey：用于Firefox浏览器。JavaScriptCore：用于Safari浏览器。WebKit：主要用于Safari和Chrome浏览器。Gecko：主要用于Firefox浏览器。Trident：主要用于旧版本的Internet Explorer浏览器。Blink：基于WebKit，用于Chrome、Opera和部分Chromium浏览器。V8：用于Chrome和Opera浏览器，具有高性能和快速执行速度。SpiderMonkey：用于Firefox浏览器。JavaScriptCore：用于Safari浏览器。在早期，渲染引擎和JavaScript引擎没有明确的分离，它们在同一个内核中工作。随着时间的推移，JavaScript引擎逐渐独立出来，使内核更专注于页面渲染和布局方面的任务。理解浏览器内核对于前端开发人员非常重要，因为不同的内核可能会对网页的解释和渲染产生影响，从而影响页面的布局、样式和交互效果。在开发过程中，需要考虑不同浏览器内核的差异，并进行兼容性测试和优化，以确保网页在不同浏览器上都能正确显示和运行。

1) 测试同学说“页面能执行JavaScript，说明浏览器内核渲染没有问题”，但实际首屏布局全部错位，你同意这个判断吗？

答：不同意，脚本执行和页面渲染由职责不同的部分承担，前者正常不能证明样式计算、布局与绘制正常。应分别检查JavaScript引擎报错和渲染引擎对HTML、CSS的解析结果；把二者统称为一个故障点会误导定位。

2) 同一套活动页在Chrome、Firefox和Safari上出现边距及交互差异，研发排期只允许做一轮兼容处理，你会怎样落地？

答：应按各浏览器所用渲染引擎和脚本引擎分别验证布局、样式与交互，而不是只看页面能否打开。先锁定影响核心流程的差异，再通过兼容写法和跨浏览器测试收敛；只在单一浏览器调通，无法证明其他内核解释结果一致。

3) 企业旧终端仍使用基于Trident的浏览器，新系统却大量采用现代页面能力，业务要求与新版浏览器表现完全一致，你会直接承诺吗？

答：不会直接承诺，因为旧渲染引擎对页面结构、样式和脚本特性的解释能力可能与现代内核存在明显差异。应先划定必须支持的功能并进行兼容性验证，必要时提供降级页面；完全一致往往意味着更

高实现和测试成本，部分能力还可能无法补齐。

4) 线上页面只有Firefox出现按钮位置偏移，但控制台没有脚本异常，排查时你会先看V8相关问题还是布局链路？

答：应先查布局链路，因为Firefox使用的脚本引擎不是V8，且没有脚本异常时更应关注样式计算、元素尺寸和绘制差异。通过最小复现逐步移除样式并对比其他渲染引擎；若交互状态会动态改样式，再回到脚本执行链路验证。

5) 架构评审中有人建议用“换一个更快的JavaScript引擎”解决图片解码后页面闪烁，你会接受这个选型吗？

答：不会把它作为首选，因为图片、布局和绘制主要属于渲染链路，脚本引擎只负责解析与执行JavaScript。应先确认闪烁是否来自资源加载、样式重算或绘制时序；只有证据表明脚本计算阻塞了渲染时，脚本执行性能才是相关因素。

9. html5有哪些新特性、移除了那些元素？

HTML5增加了语义化、多媒体、客户端存储和通信等能力，同时移除了纯表现元素与frame系列。常见的能力有video、audio、canvas、localStorage、WebSocket，以及header、nav、section等语义标签。像font、center、frameset这类元素被淘汰，是因为样式应交给CSS，框架页也会影响可用性。区分HTML与HTML5时，我一般看DOCTYPE、新增结构元素和功能元素。HTML5现在已经不是SGML的子集，主要是关于图像，位置，存储，多任务等功能的增加新增选择器document.querySelector、document.querySelectorAll拖拽释放(Drag and drop) API媒体播放的video和audio本地存储localStorage和sessionStorage离线应用manifest桌面通知Notifications语意化标签article、footer、header、nav、section增强表单控件calendar、date、time、email、url、search地理位置Geolocation多任务webworker全双工通信协议websocket历史管理history跨域资源共享(CORS)Access-Control-Allow-Origin页面可见性改变事件visibilitychange跨窗口通信PostMessageForm Data对象绘画canvas移除的元素：纯表现的元素：basefont、big、center、font、s、strike、tt、u对可用性产生负面影响的元素：frame、frameset、noframes支持HTML5新标签：IE8/IE7/IE6支持通过document.createElement方法产生的标签可以利用这一特性让这些浏览器支持HTML5新标签浏览器支持新标签后，还需要添加标签默认的风格当然也可以直接使用成熟的框架、比如html5shim

新增选择器document.querySelector、document.querySelectorAll拖拽释放(Drag and drop) API媒体播放的video和audio本地存储localStorage和sessionStorage离线应用manifest桌面通知Notifications语意化标签article、footer、header、nav、section增强表单控件calendar、date、time、email、url、search地理位置Geolocation多任务webworker全双工通信协议websocket历史管理history跨域资源共享(CORS)Access-Control-Allow-Origin页面可见性改变事件visibilitychange跨窗口通信PostMessageForm Data对象绘画canvas 纯表现的元素：basefont、big、center、font、s、strike、tt、u对可用性产生负面影响的元素：frame、frameset、noframes IE8/IE7/IE6支持通过document.createElement方法产生的标签可以利用这一特性让这些浏览器支持HTML5新标签浏览器支持新标签后，还需要添加标签默认的风格 如何区分 HTML 和 HTML5 DOCTYPE声明、新增的结构元素、功能元素

1) 老页面用center、font和big控制文章样式，改版负责人认为它们能显示就不必迁移，你会怎样处理？

答：应将这些纯表现元素迁移为有语义的结构配合CSS，因为它们不负责表达内容含义，并不属于推荐的页面结构。迁移时要先保留视觉基线，再替换标签和样式；若旧浏览器仍在支持范围，还需单独回归显示差异。

2) 产品要在课程页加入视频、拖拽排序、日期输入和历史记录管理，你会把它们都交给同一种HTML5能力实现吗？

答：不会，这些需求分别对应video、拖放API、增强表单控件和history等不同能力。实现前应逐项验证浏览器支持并设置降级路径；“属于HTML5”只是宽泛分类，不能代表它们共享相同接口或兼容边界。

3) 运营工具要在主线程一次处理大量图片，同时要求输入框始终流畅，候选方案是canvas加普通循环，你会补充什么约束？

答：canvas负责绘制，不会自动把计算移出主线程；耗时任务应评估交给Web Worker，再把结果传回页面绘制。拆分后仍要控制数据传递和渲染频率，因为工作线程不能直接替代所有页面操作，绘制压力也不会凭空消失。

4) 移动端注册页使用type="date"和type="email"，某些环境展示或校验行为与设计稿不同，你先认定是业务代码错误吗？

答：不能直接认定，增强表单控件的界面和行为可能受到浏览器实现差异影响。应先核对元素类型、实际支持情况和提交值，再决定保留原生控件还是提供自定义降级；自定义方案能统一外观，但会增加校验、交互和可访问性成本。

5) 离线看板计划把所有接口数据都塞进localStorage，并把它与WebSocket、CORS一起称为存储方案，你会如何纠正选型？

答：应拆开职责：localStorage和sessionStorage用于客户端存储，WebSocket处理全双工通信，CORS约束跨源访问。离线数据量和结构超出简单键值存储能力时需另评估存储技术；混用概念会掩盖容量、同步读写及网络安全边界。

10. HTML5的离线储存怎么使用，工作原理能不能解释一下？

传统的AppCache通过页面的manifest属性读取资源清单并缓存文件，但它已从标准移除，新项目应使用Service Worker和Cache API。当年浏览器会解析.appcache文件，在线时下载清单资源，离线时直接使用本地副本。页面还可以通过window.applicationCache处理相关状态，不过这种方式的更新和失败控制都比较僵硬。现在通常由Service Worker拦截fetch，再按场景选择缓存优先、网络优先或后台更新。内容已过时 下面描述的AppCache (manifest+.appcache) 已从 HTML 标准中移除，Chrome 85+、Firefox 84+ 均已彻底下线，新项目一律用Service Worker + Cache API。保留这段是因为面试官偶尔仍会问"HTML5 离线存储"，答题时要主动点出它已废弃并给出替代方案——这恰恰是加分点。 在用户没有与因特网连接时，可以正常访问站点或应用，在用户与因特网连接时，更新用户机器上的缓存文件原理：HTML5的离线存储是基于一个新建的.appcache文件的缓存机制(不是存储技术)，通过这个文件上的解析清单离线存储资源，这些资源就会像cookie一样被存储了下来。之后当网络在处于离线状态下时，浏览器会通过被离线存储的数据进行页面展示如何使用：页面头部像下面一样加入一个manifest的属性；在cache.manifest文件的编写离线存储的资源在离线状态时，操作window.applicationCache进行需求实现 页面头部像下面一样加入一个manifest的属性；在cache.manifest文件的编写离线存储的资源在离线状态时，操作window.applicationCache进行需求实现 CACHE MANIFEST #v0.11CACHE.js/app.js css/style.css NETWORK:resource/logo.png FALLBACK:/offline.html

1) 维护人员拿出旧项目的manifest="cache.appcache"，要求新建的离线页面照搬这套配置，你会同意吗？

答：不会，AppCache已从标准移除，现代项目应采用Service Worker配合Cache API。旧清单中的CACHE、NETWORK和FALLBACK可作为梳理资源策略的输入，但不能继续作为新项目实现基础，否则会面临浏览器不可用的硬边界。

2) 一个巡检页面需要断网后仍能打开外壳和离线提示，你会怎样把旧AppCache需求迁移为可落地方案？

答：应注册Service Worker，在其生命周期内缓存必要的页面、脚本和样式，并在请求处理中决定读取网络还是缓存。离线兜底页可在网络请求失败时返回；缓存范围和更新策略必须明确，否则资源更新后可能出现新旧文件不一致。

3) 业务从“只读离线页面”升级为“断网可编辑数百条巡检记录，联网后再提交”，只增加Cache API足够吗？

答：不够，Cache API适合管理请求与响应，业务记录仍需要合适的客户端数据存储，并设计重新联网后的同步流程。还要定义重复提交和冲突处理规则；没有服务端配合时，离线编辑只能保存本地状态，不能保证最终数据一致。

4) 发布后部分用户一直看到旧版脚本，即使刷新也不更新，站点使用了Service Worker，你会从哪里开始排查？

答：先核对Service Worker的注册范围、缓存命中策略和缓存版本，确认页面请求是否始终优先返回旧响应。再检查新工作线程是否完成安装和激活，并清理不再使用的缓存；直接让所有请求永久走缓存会增强离线能力，却可能阻断正常更新。

5) 新闻站评审时，一方要求所有内容采用缓存优先以保证秒开，另一方要求用户永远看到最新稿件，你会怎样取舍？

答：不能对全部资源使用同一策略，稳定的静态资源可更积极缓存，而时效性强的正文应优先验证网络或设置明确更新机制。离线时再返回缓存内容并标注状态更稳妥；越偏向缓存，可用性越强，但内容陈旧风险也越高。

11. 浏览器是怎么对HTML5的离线储存资源进行管理和加载的呢

使用AppCache时，浏览器在线检查manifest清单并管理缓存，离线时则直接加载已经保存的资源。第一次访问会按清单下载文件；之后访问先使用本地资源，再比较新旧manifest，只有清单发生变化才重新下载。也就是说，只改资源却不改清单，缓存可能不会更新，这也是它的明显边界。由于AppCache已被淘汰，现在应采用可编程控制请求和缓存的Service Worker。在线的情况下，浏览器发现html头部有manifest属性，它会请求manifest文件，如果是第一次访问app，那么浏览器就会根据manifest文件的内容下载相应的资源并且进行离线存储。如果已经访问过app并且资源已经离线存储了，那么浏览器就会使用离线的资源加载页面，然后浏览器会对比新的manifest文件与旧的manifest文件，如果文件没有发生改变，就不做任何操作，如果文件改变了，那么就会重新下载文件中的资源并进行离线存储。离线的环境下，浏览器就直接使用离线存储的资源。

12. 请描述一下cookies，sessionStorage和localStorage的区别？

cookie主要用于随请求传给服务端，而sessionStorage和localStorage只保存在客户端，不会自动上传。cookie通常约4KB，可通过过期时间、Domain和Path控制有效期与范围；另外两者一般可存约5MB或更多。sessionStorage跟当前窗口或标签页会话走，关闭后清除；localStorage在同源页面间共享，除非主动删除，否则会长期保留。登录状态等少量服务端必需信息适合cookie，临时表单数据适合sessionStorage，长期配置更适合localStorage。cookies、sessionStorage和localStorage都是在Web开发中用于在客户端存储数据的技术，但它们在以下方面存在区别：

存储容量	cookies	sessionStorage	localStorage
cookies	通常存储容量较小，一般限制在4KB左右。	存储容量比cookies大，不同浏览器的限制有所不同，但一般可以存储5MB或更多的数据。	存储容量也较大，通常和sessionStorage相近，能满足大多数Web应用的存储需求。
数据有效期	可以设置过期时间，若不设置，默认在浏览器关闭时失效。如果设置了过期时间，那么在过期时间之前，cookies会一直存在于客户端，即使浏览器关闭后重新打		

开也依然有效。sessionStorage：数据仅在当前会话期间有效，当浏览器窗口关闭时，sessionStorage中的数据会被自动清除。localStorage：数据没有自动过期时间，除非主动删除，否则数据会一直保存在客户端，即使浏览器关闭、电脑重启，数据依然存在。

作用域 cookies：在同域名下的不同页面之间可以共享，但如果设置了Path属性，那么只有在指定路径下的页面才能访问该cookie。此外，cookies还可以通过设置Domain属性在不同子域名之间共享。sessionStorage：具有严格的页面级作用域，即只有在同一个浏览器窗口（或标签页）中打开的页面才能共享sessionStorage中的数据。不同浏览器窗口或标签页之间无法访问彼此的sessionStorage数据，即使它们来自同一个域名。localStorage：在同域名下的所有页面都可以共享localStorage中的数据，只要是在同一个浏览器中访问该域名，无论在哪个页面存储的数据，其他页面都可以读取和修改。

与服务器的交互 cookies：每次向服务器发送请求时，浏览器会自动将同域名下的cookies发送到服务器，这可能会增加不必要的网络传输开销，尤其是当cookies数据量较大时。sessionStorage和localStorage：数据仅存储在客户端，不会自动发送到服务器。如果需要与服务器进行数据交互，需要通过 Ajax 等方式手动将数据发送到服务器。

应用场景 cookies：常用于存储用户登录状态、购物车信息、用户偏好设置等。由于其会在每次请求时发送到服务器，所以也可以用于在客户端和服务端之间传递一些少量的必要信息。sessionStorage：适合存储一些临时数据，例如在一个多步骤的表单填写过程中，存储用户在当前页面填写的数据，当用户完成整个流程或关闭页面时，这些数据就不再需要了。localStorage：用于长期存储用户相关的数据，如用户的配置信息、离线缓存数据等。例如，一个 Web 应用可以将一些常用的静态资源存储在localStorage中，以便在离线状态下仍然能够访问应用的部分功能。

1) 电商页把完整购物车JSON放进cookie，每次图片请求都携带它，开发认为这和放进localStorage没区别，你认可吗？

答：不认可，cookie容量通常只有约4KB，并会按作用域自动随请求发送；localStorage容量更大且不会自动上传服务器。购物车若无需服务端在每次请求中读取，更适合客户端存储并按需同步，但长期数据仍需考虑过期与一致性。

2) 多步骤投保表单要求刷新后保留填写内容，关闭当前标签页后自动清除，你会选择哪种存储？

答：应优先选择sessionStorage，它适合保存当前标签页会话中的临时表单状态，刷新和同页导航时仍可读取。实现时还要按同源和标签页范围验证流程；若用户关闭标签页或换到独立标签页继续填写，数据通常不能按原方案共享。

3) 账号设置从单一子域扩展到多个子域，产品要求所有页面共享偏好数据，原先的localStorage方案还能直接沿用吗？

答：不能直接沿用，localStorage受源范围约束，不会因为属于同一主域就自动跨子域共享。若偏好需要随服务端请求使用，可评估设置合适Domain的cookie；扩大共享范围也会扩大数据暴露面，不能为了便利无限放宽作用域。

4) 用户反馈关闭浏览器后仍保持登录，研发认定“会话cookie一定没有被清除”，你会怎样排查？

答：先检查该cookie是否设置了过期时间，以及服务端是否通过其他凭据恢复登录状态。未设置过期时间的cookie通常按会话处理，设置后则可跨浏览器重启保留；还要结合实际浏览器会话恢复行为验证，不能只凭页面现象下结论。

5) 安全评审在cookie和localStorage之间争论登录凭据存放位置，你会依据哪些职责边界做决定？

答：先确认凭据是否需要浏览器自动随请求发送：需要时cookie更贴近客户端与服务器传递场景，不需要时客户端存储可由代码按需读取。选型还必须结合脚本访问、请求作用域和过期策略评估风险；仅比较容量或持久时间，无法得出安全结论。

6) 运营页要求两个同源标签页同步筛选条件，但每个标签页的未提交表单必须彼此隔离，你会如何组合存储？

答：共享的筛选条件可放入localStorage，因为同源页面能够共同读取和修改；未提交表单放入sessionStorage，以保持标签页级隔离。两类数据仍应使用不同键和清理策略，否则长期共享数据与临时会话数据会互相污染。

13. iframe有那些缺点？

iframe的主要缺点是可能拖慢页面加载、影响SEO，多个实例还会挤占并行连接。它加载时会阻塞主页面的onload，而且搜索引擎通常不能很好地理解其中的内容。由于它和主页面共享连接池，数量多时并行加载能力也可能下降。我一般不会把核心内容放进iframe；必须使用时，可以用JavaScript动态设置src，或者根据场景考虑AJAX等替代方案。iframe是一种在网页中嵌入其他网页或文档的标签，虽然它在某些情况下可以提供一些便利，但也存在一些缺点需要考虑：阻塞主页面的onload事件：当页面中存在iframe时，iframe的加载会阻塞主页面的onload事件触发。这可能会导致页面加载速度变慢，影响用户体验。不利于SEO：搜索引擎的爬虫程序通常不能很好地解读iframe内部的内容。因此，如果重要的页面内容被放置在iframe中，搜索引擎可能无法正确地索引和收录这些内容，从而影响网页的搜索引擎优化（SEO）。连接限制和并行加载：iframe和主页面共享连接池，而大多数浏览器对相同域的连接数有限制。这意味着当页面中包含多个iframe时，浏览器需要同时处理这些连接，可能会影响页面的并行加载能力，导致页面加载速度变慢。为了避免以上问题，可以考虑以下解决方案：尽量避免使用iframe，特别是在主要内容部分。如果必须使用iframe，可以通过JavaScript动态地给iframe添加src属性值，而不是在静态HTML中指定。这样可以绕开阻塞主页面的onload事件。对于需要被搜索引擎索引的重要内容，避免将其放置在iframe中。考虑使用其他替代方案，如AJAX加载内容或使用现代的前端框架来实现类似的效果。总之，iframe在某些场景下可以提供方便，但在使用时需要注意其缺点，并根据具体情况进行权衡和选择。

1) 内容团队把商品详情正文放进同域iframe，页面肉眼可见且交互正常，却发现搜索引擎长期只收录外层壳壳；“用户能看到就一定能被索引”为什么不成立？

答：页面可见不等于搜索引擎会把嵌入内容归入父页面，重要正文放在iframe中存在无法被正确解读和收录的风险。应让核心内容直接进入主文档结构，只把非核心或确需隔离的页面留在iframe；否则需要接受父页面SEO能力下降的代价。

2) 门户首页接入三个统计和客服页面后，首屏组件已经渲染完成，但window.onload迟迟不触发，依赖它关闭加载动画的代码一直卡住；你会怎么改？

答：先检查这些iframe的资源加载是否拖延了主页面onload，再把加载动画和首屏逻辑从该事件中解耦。非首屏嵌入内容可由JavaScript在主内容就绪后再设置src；这样能减轻阻塞，但会推迟子页面可用时间并增加状态管理。

3) 一个后台从单个报表扩展到同域十几个iframe，主页面静态资源和各报表同时加载时明显变慢；即使服务器没有报错，为什么嵌入数量仍可能成为瓶颈？

答：这些iframe与主页面可能共享同域连接资源，数量增加后会争用有限的并行连接，主页面资源因而可能排队。应压缩同时加载的嵌入数量，并对非当前区域做延迟加载；若全部报表必须立即可用，就无法彻底避免并发竞争。

4) 线上页面偶发出现“主体已显示但初始化逻辑未执行”，监控显示主文档脚本无异常，只是某个第三方iframe响应很慢；排查顺序是什么？

答：先确认初始化是否绑定在window.onload，再逐个禁用或延迟设置iframe.src，观察事件触发时间是否恢复。若定位到慢嵌入，应把关键初始化提前到不依赖全部子资源完成的阶段；第三方加载仍不可控，保留超时和降级界面更稳妥。

5) 架构评审中，一方主张继续用iframe快速接入旧报表，另一方要求改成AJAX或前端组件以改善加载和SEO；你怎么划分适用边界？

答：非核心、无需搜索引擎收录且改造成本高的独立页面，可以保留iframe，但要控制数量和加载时机。主要内容或强依赖首屏性能的区域更适合直接渲染、AJAX加载或组件化；替代方案会增加整合成本，也需自行处理样式与脚本协作。

14. WEB标准以及W3C标准是什么？

Web标准和W3C标准是一组用于统一网页结构、样式与交互行为的技术规范。它们覆盖HTML、CSS、DOM、SVG等领域，目的是让网页在不同浏览器和设备上尽量保持一致。落到编码上，就是正确闭合和嵌套标签、规范书写，并通过外链CSS和JavaScript分离结构、表现与行为。遵循这些规范也有利于可访问性、可维护性和后续扩展。WEB标准是指由万维网联盟（World Wide Web Consortium，简称W3C）制定的一系列技术规范和指南，旨在确保网页在不同的浏览器和设备上具有一致的表现和行为。这些标准涵盖了HTML、CSS、JavaScript等前端技术，并规定了它们的语法、结构、样式以及交互行为等方面的规范。W3C标准是由W3C组织制定和推广的一系列技术标准，旨在推动网络技术和互操作性。W3C是一个国际性的标准化组织，由互联网行业的各大公司、研究机构和个人组成，致力于制定并推广互联网的开放标准。W3C标准包括HTML、CSS、XML、DOM、SVG等多个技术领域，并且不断更新和演进，以适应新的需求和技术发展。具体来说，WEB标准和W3C标准强调以下几个方面： 标签闭合：HTML标签必须按照规定的格式正确闭合，以确保页面结构的准确性和一致性。标签小写：HTML标签和属性应该使用小写字母，以避免浏览器解析错误。不乱嵌套：HTML标签应该按照正确的嵌套规则进行使用，不应该出现乱七八糟的嵌套结构，以确保页面结构的清晰和可维护性。使用外链CSS和JS：将CSS样式和JavaScript代码尽可能地放在外部文件中，并通过链接的方式引入，以实现结构、行为和表现的分离，提高代码的可重用性和可维护性。通过遵循这些标准和规范，开发人员可以编写出更加规范、可靠和跨平台的网页，确保网页在不同的浏览器和设备上得到一致的显示和行为，提供更好的用户体验。此外，遵循WEB标准和W3C标准还有助于网页的可访问性、可维护性和可扩展性，同时推动互联网技术的进步和发展。

1) 代码评审里有人认为“Chrome能显示就算符合Web标准”，但同一页面在移动设备上结构错乱；这个判断漏掉了什么？

答：单一浏览器能够容错显示，并不能证明页面遵循了Web标准，标准关注不同浏览器和设备之间的互操作性与一致行为。应检查标签闭合、嵌套、大小写习惯及结构、样式、行为的组织；完全一致仍可能受实现差异影响，需配合跨端测试。

2) 一个多人维护的活动页把样式、脚本和大量内联事件都塞进HTML，两周后改版频繁产生重复代码；你会怎样按标准思想重构？

答：应把文档结构留在HTML，将可复用样式和交互逻辑分别迁移到外链CSS与JavaScript，形成结构、表现、行为的分离。这样更便于复用和维护，但外链文件的加载、缓存与依赖顺序也需要明确管理。

3) 团队从桌面官网扩展到手机和其他终端后，负责人要求所有标签都改成小写并正确闭合；这些书写约束能否单独保证跨平台一致？

答：规范书写能减少解析歧义和维护成本，却不能单独保证所有设备呈现完全一致。还要遵循正确嵌套、标准化HTML、CSS和交互行为，并在目标浏览器与设备上验证；终端能力和实现差异仍可能要求兼容处理。

4) 线上只有某类页面出现DOM层级异常，模板中能看到未闭合标签和列表元素交叉嵌套，但浏览器仍勉强渲染；你会先修脚本还是先修结构？

答：应先修复标签闭合与非法嵌套，因为错误结构会让解析后的DOM偏离源码预期，随后脚本选择器和样式规则都可能被连带影响。修正后再比较实际DOM与模板；若异常仍在，才继续排查CSS或JavaScript。

5) 产品要求快速上线一次性专题页，开发者想全部内联以少建文件，架构师坚持外链CSS和JS；你如何处理这组选型冲突？

答：若页面极小且一次性，内联并非必然不可运行，但会削弱结构、表现、行为分离带来的复用和维护收益。只要专题可能迭代或共享样式脚本，就应优先外链；外链会增加资源组织工作，不能把形式本身当成质量保证。

15. xhtml和html有什么区别？

XHTML和HTML的核心区别在于，XHTML对文档书写格式的要求更严格。XHTML元素必须正确嵌套和闭合，名称区分大小写，而且文档必须存在根元素。功能层面上，原文强调它可兼容浏览器、手机和PDA，也便于浏览器快速、正确地编译网页。简单来说，编写XHTML时不能依赖浏览器替你容忍不规范的标记。一个是功能上的差别主要是XHTML可兼容各大浏览器、手机以及PDA，并且浏览器也能快速正确地编译网页另外是书写习惯的差别XHTML元素必须被正确地嵌套，闭合，区分大小写，文档必须拥有根元素主要是XHTML可兼容各大浏览器、手机以及PDA，并且浏览器也能快速正确地编译网页XHTML元素必须被正确地嵌套，闭合，区分大小写，文档必须拥有根元素

1) 旧模板在普通HTML页面里混用了、未闭合段落和大小写属性，浏览器仍能显示；把文件直接声明为XHTML后，为什么不能期待它继续照常工作？

答：XHTML要求元素正确嵌套和闭合，并区分大小写，文档还必须具有根元素，原模板不满足这些书写约束。迁移前应先规范标签结构与大小写；只改声明而不改内容，可能导致解析失败或页面结构不符合预期。

2) 团队要把一套页面同时交付给桌面浏览器、手机和较旧的移动终端，负责人以“兼容面广”为由选择XHTML；落地时最先建立哪些代码检查？

答：应优先检查所有元素是否正确嵌套与闭合、名称大小写是否一致，以及文档是否只有合规的根元素。严格书写有利于页面被快速、正确地处理；但兼容性仍要通过目标终端验证，不能只依赖文档类型判断。

3) 服务端模板原本输出完整XHTML，后来改为拼接多个业务片段，某个片段带来第二个根节点；其他标签都闭合时，文档仍违反了什么约束？

答：它仍违反XHTML文档必须拥有根元素的约束，多个并列顶层节点不能组成合规的完整文档。模板层应由统一外壳提供唯一根元素，业务片段只输出其内部内容；若片段可独立返回，则需分别生成完整文档。

4) 线上某批XHTML页面只有新增组件无法正常处理，差异对比发现的结束标签写成，而普通HTML测试页看起来正常；你如何定位？

答：应先按大小写敏感规则检查起止标签是否完全匹配，再验证新增组件的嵌套和闭合关系。这里与不能视为同一名称，修正模板生成逻辑比增加浏览器兼容分支更直接；其他结构错误也需一并排除。

5) 一个内容站已用宽松HTML稳定运行，架构师希望全量迁到XHTML以获得更严格的代码约束，而业务方担心改造风险；你怎么权衡？

答：若目标终端确实需要XHTML的兼容与严格处理特性，迁移才有明确收益，并应先完成嵌套、闭合、大小写和根元素检查。仅为了统一书写习惯而全量迁移，可能放大存量模板问题；可先用代码规范和校验逐步治理。

16. Doctype作用? 严格模式与混杂模式如何区分? 它们有何意义?

DOCTYPE用来告诉浏览器按哪种HTML或XHTML规范解析文档，并影响页面进入严格模式还是混杂模式。声明正确时，浏览器通常按规范解析和渲染；声明缺失或格式不正确时，则可能进入用于兼容旧网站的混杂模式。严格模式有助于减少跨浏览器差异，混杂模式会更宽松地容忍不规范的HTML和CSS。工程中应提供正确的DOCTYPE，只有维护旧页面时才需要关注混杂模式的兼容行为。DOCTYPE（文档类型声明）的作用是告知浏览器的解析器使用哪种HTML或XHTML规范来解析文档。严格模式（标准模式）是指浏览器按照HTML或XHTML的规范严格解析和渲染页面，以确保页面在不同浏览器中具有一致的展示效果和行为。在严格模式下，浏览器会按照规范要求的方式处理HTML和CSS代码。混杂模式（怪异模式或兼容模式）是指浏览器使用较宽松的解析方式来渲染页面，以模拟旧式浏览器的行为，以保证旧版网站的兼容性。在混杂模式下，浏览器可能会容忍一些不符合规范的HTML和CSS代码，导致页面展示和行为在不同浏览器中有差异。通过DOCTYPE声明的类型来区分严格模式和混杂模式。当DOCTYPE声明为严格的HTML或XHTML规范时，浏览器会进入严格模式；当DOCTYPE声明缺失或格式不正确时，浏览器会进入混杂模式。严格模式和混杂模式的意义在于确保页面在不同浏览器中的一致性和兼容性。严格模式使开发者能够使用更规范的HTML和CSS代码，减少兼容性问题，提高网页的可靠性和可维护性。混杂模式则用于支持旧版网站，以确保这些网站在新版浏览器中能够正确显示和运行。

1) 两个页面引用完全相同的CSS，其中一个顶部缺少有效DOCTYPE，结果旧式布局规则表现不同；“样式文件相同，渲染模式就相同”错在哪里？

答：渲染模式不只由CSS决定，有效的DOCTYPE会告知浏览器按相应HTML或XHTML规范解析文档。声明缺失或格式错误时可能进入混杂模式并模拟旧浏览器行为，因此应先统一文档声明，再比较具体样式差异。

2) 你接手一个多模板站点，准备把页面统一到标准模式，但不能一次重写所有样式；上线前会怎样控制改造范围？

答：先为模板补充正确且一致的DOCTYPE，再对依赖旧式渲染行为的页面做分批回归，重点比较布局和交互。标准模式能提升跨浏览器一致性，但切换会暴露历史代码对混杂模式的依赖，不能只改声明后直接全量发布。

3) 公司仍需维护十年前的内部系统，新页面要求标准模式，旧页面却依赖浏览器模拟旧行为；是否应该强制所有页面立即使用同一种模式？

答：新页面应采用有效DOCTYPE进入标准模式，以规范的HTML和CSS降低后续兼容成本。旧页面若依赖混杂模式，可暂时隔离并逐步改造；立即切换虽然形式统一，却可能破坏现有展示和业务行为。

4) 线上只有一个营销页在不同浏览器中布局差异明显，源码里也存在不规范标签和CSS；你如何判断是代码错误还是渲染模式导致的？

答：先核对页面开头的DOCTYPE是否缺失、拼写错误或与文档类型不符，再与正常模板做最小差异对比。确认进入预期模式后，再修复不合规范的HTML与CSS；混杂模式会容忍部分错误，容易掩盖真正的结构问题。

5) 业务方认为混杂模式“兼容性更强”，希望新项目也使用它来减少浏览器问题；你会接受吗？

答：不应把混杂模式作为新项目的通用兼容方案，它主要用于模拟旧式浏览器行为以维持旧网站。新项目采用标准模式更有利于获得一致、可靠且可维护的渲染结果；代价是历史代码若被复用，必须先消除对旧行为的依赖。

17. 行内元素有哪些？块级元素有哪些？

空(void)元素有那些？行内元素和块级元素有什么区别？

常见行内元素有a、span、img、input，块级元素有div、ul、li、h1、p，空元素则包括br、hr、img、input、link、meta。行内元素通常不独占一行，按从左到右排列并按内容占用空间；块级元素会独占一行、自动换行，通常占满父元素宽度，而且可以设置宽高。空元素本身没有内容，因此不需要成对的结束标签。本质上，行内和块级描述的是元素默认的display表现，实际开发中可以通过修改display让它们互换。行内元素有：abspaceinputselectstrong等。块级元素有：divulolliidtdh1h2h3h4等标题标签、p段落标签等。空元素（void元素）是指没有内容的HTML元素。常见的空元素包括：换行元素、水平线元素、图片元素、输入框元素、样式表引用元素、元数据元素等。行内元素不可以设置宽高，不独占一行，它们会按照从左到右的顺序排列，并尽可能占据内容所需的宽度。块级元素可以设置宽高，独占一行，会自动换行。块级元素会在页面上以独立的块形式展现，并占据其父元素的整个宽度。

1) 设计稿要求一个span形式的状态标签固定为120px × 40px，开发直接写width和height后尺寸没有按预期变化；为什么同样写法放在div上却通常有效？

答：span默认属于行内元素，不独占一行，并主要按内容所需空间排列，因此直接设置宽高通常不符合预期。div属于块级元素，可以设置宽高且默认独占一行；若保留行内排布，需要调整其布局方式，同时评估换行影响。

2) 商品列表要让十几个链接从左到右排列，而每个商品卡片又要占满父容器一行；你会分别选择哪类元素作为布局起点？

答：连续链接可从a、span等行内元素出发，它们会从左到右排列并尽量只占内容空间。整行卡片可用div、li等块级元素，便于设置宽高并形成独立块；最终视觉仍应由CSS控制，元素选择还要符合内容语义。

3) 模板生成器把、和都按普通容器输出了成对结束标签，构建没有报错但代码审查认为模型设计有问题；问题出在哪里？

答：这些标签属于常见空元素，本身不承载子内容，不应被当作可包裹子节点的普通容器。生成器应为br、hr、img、input、link、meta等建立空元素规则；否则模板抽象会误导调用方，并可能产生不规范标记。

4) 线上文章页的目录项原本使用li，改版时被统一替换成span后，条目不再自然换行并挤在同一行；你会先检查什么？

答：先检查替换是否把默认块级排列改成了行内排列：li通常形成独立块，而span会从左到右连续排布。恢复合适元素或明确设置布局即可修正；若目录依赖列表结构，还应保留ul、ol与li的正确嵌套。

5) 组件库评审中，一方想用div实现所有文本和链接以方便设置尺寸，另一方坚持全部使用行内标签以减少换行；你如何裁决？

答：应按内容结构和布局需求选择起点：独立区域适合块级元素，连续文本或链接适合行内元素，不能只为宽高或换行统一标签。布局差异可由CSS调整，但滥用元素会降低结构清晰度；空元素也不能被拿来承载内容。

18. HTML全局属性(global attribute)有哪些

HTML全局属性是各类元素都可以使用的通用属性，常见的有class、id、style、title、lang、data-*和draggable。class与id主要用于标识元素，其中id在文档内应保持唯一，style则用于编写行内CSS样式。需要给元素附加自定义数据时可以使用data-*，涉及语言说明或拖拽能力时再分别设置lang和draggable。title适合提供与元素相关的建议信息，不应把所有业务数据都塞进这个属性。class:为元素设置类标识data-*:为元素增加自定义属性draggable: 设置元素是否可拖拽id: 元素id，文档内唯一lang: 元素内容的语言style: 行内css样式title: 元素相关的建议信息

19. Canvas和SVG有什么区别？

Canvas本质上是基于像素的一次性绘制，而SVG是基于XML的矢量图形，并且会保留独立的DOM节点。因此Canvas放大后可能失真或出现锯齿，绘制完成后也不能直接操作其中的图形对象。SVG可以无损缩放，还能修改、删除元素或直接绑定事件。像素级图像处理、游戏和动画通常更适合Canvas，图表、数据可视化及需要交互和缩放的场景则更适合SVG。svg绘制出来的每一个图形的元素都是独立的DOM节点，能够方便的绑定事件或用来修改。canvas输出的是一整幅画布svg输出的图形是矢量图形，后期可以修改参数来自由放大缩小，不会失真和锯齿。而canvas输出标量画布，就像一张图片一样，放大会失真或者锯齿

Canvas：Canvas 是一个HTML5元素，用于在网页上绘制图形、动画和图像。通过使用JavaScript绘制图形，Canvas提供了一个像素级的绘图环境。Canvas

绘制的是位图，它是由一系列的像素组成的，所以在放大时会出现像素失真或锯齿效应。Canvas不会保留绘图的对象，绘制完成后，图形将被保存为一张图片。由于绘制是基于像素的，Canvas更适合处理像素级的图像处理、游戏开发等场景。Canvas

不支持事件绑定，需要通过监听鼠标、键盘等事件来实现交互。

SVG：

SVG

是一种基于XML的矢量图形格式，用于在网页上绘制图形和图像。SVG

使用XML描述图形，它由一系列的矢量对象组成，可以方便地修改和操作。SVG

绘制的是矢量图形，它基于数学描述，可以自由缩放和变换而不会失真或产生锯齿效应。SVG

保留了绘图的对象，可以对其进行修改、删除和动态操作。由于是矢量图形，SVG

更适合处理图表、数据可视化和可缩放的图形场景。SVG

支持事件绑定，可以方便地为图形元素添加交互行为。综上所述，Canvas适用于像素级绘图和动画，而SVG适用于矢量图形和可缩放的图像。选择使用Canvas还是SVG取决于具体的需求和场景。

1) 一个流程图用Canvas绘制后，产品要求点击任意节点改颜色；你能像操作SVG一样直接找到对应图形节点吗？

答：不能直接找到，因为Canvas绘制完成后只保留整块位图，不会为矩形、连线等图形保留独立DOM节点。需要自行维护图形对象、坐标和命中检测，再按新状态重绘画布；交互对象较多时，这套管理逻辑的复杂度会明显上升。

2) 后台要展示几百个可点击的数据点，并支持悬停提示和动态修改样式，你会先选SVG还是Canvas？

答：这类强调逐元素交互的页面会优先考虑SVG，每个数据点都是可查询、可修改并可绑定事件的独立元素。实现时可直接更新对应节点属性；但当图形数量继续增大、DOM管理成为负担时，需要重新评估Canvas的像素绘制方案。

3) 同一张品牌图既要显示在手机卡片中，又要放大到桌面大屏，设计师要求边缘始终清晰，选型会怎样变化？

答：应优先使用SVG，因为图形由矢量对象和数学参数描述，缩放或变换时不会像位图那样出现像素失真和锯齿。Canvas若要适应更大显示尺寸，通常需要按目标像素重新绘制；涉及照片或像素级处理时，选择才可能反过来。

4) 一个Canvas小游戏能收到画布的点击事件，但点击某个角色时总是触发错对象，你会从哪里排查？

答：先排查角色坐标、绘制顺序以及点击位置到画布坐标的换算，因为Canvas不会替内部图形分发事件。应保存每个对象的范围并执行命中检测，必要时按层级反向查找；缩放、位移或高分辨率画布会让坐标不一致的问题更隐蔽。

5) 团队在可编辑图表上争论：动画同学偏向Canvas，交互同学偏向SVG，你用什么标准做决定？

答：应根据主要矛盾决定：大量像素绘制、图像处理或连续动画更偏向Canvas，频繁选中、修改和绑定事件则更偏向SVG。还要同时评估图形数量、缩放清晰度与对象管理成本；两者并非互相替代，复杂页面也可分层组合。

20. HTML5 为什么只需要写

因为HTML5不再基于SGML，无需通过外部DTD描述文档规则，所以只写就够了。这段声明本质上是告诉浏览器按标准模式解析和渲染页面，避免浏览器采用其他兼容方式。相比之下，HTML4.01基于SGML，必须在声明中指定对应的DTD。实际编写HTML5页面时保留这行简短声明即可，不需要再复制旧版那种很长的文档类型定义。HTML5不基于SGML，因此不需要对DTD进行引用，但是需要doctype来规范浏览器的行为而HTML4.01基于SGML,所以需要对DTD进行引用，才能告知浏览器文档所使用的文档类型

下面是对此进行更详细的解释：在HTML5中，不再基于SGML (Standard Generalized Markup Language) 标准，而是定义了自己的独立规范。由于不再使用SGML，因此不需要引用外部的DTD（文档类型定义）来验证文档的结构和规则。因此，HTML5

只需要简单地使用声明，它是一个标准模式的声明，告诉浏览器当前文档遵循的是HTML5

规范。这样，浏览器就可以根据HTML5规范来解析和渲染文档，而无需引用外部的DTD。相比之下，HTML4.01基于SGML标准，需要通过声明来指定所使用的DTD，例如。这个DTD提供了规范的文档结构和规则，以确保浏览器正确解析和显示文档。HTML5

的简化声明的设计，使得创建和编写HTML

文档更加简单和直观。此外，这也有助于提高浏览器的兼容性，因为所有的浏览器都会将文档解析为

HTML5，无需根据DTD进行选择 and 适配。总结起来，HTML5不再依赖SGML，因此不需要引用外部DTD，只需使用简单的声明来指定文档类型，规范浏览器的行为。

21. 如何在页面上实现一个圆形的可点击区域？

圆形可点击区域可以用SVG、CSS的border-radius，或者用JavaScript判断点击坐标来实现。如果只是做普通圆形按钮，我一般会给等宽高元素设置border-radius: 50%，写法最简单。需要矢量图形时可用绑定事件；如果必须精确判断点击位置是否落在圆内，就计算点击点到圆心的距离，并与半径比较。

有几种方法可以实现一个圆形的可点击区域：使用SVG（可缩放矢量图形）：可以使用SVG元素创建一个圆形，并通过添加事件监听器实现点击功能。使用

CSSborder-radius：通过设置一个具有相等宽度和高度的元素，并将border-radius属性设置为50%可以创建一个圆形区域。circle{width:100px;height:100px;border-radius:50%;}

使用纯JavaScript实现：通过计算鼠标点击的坐标与圆心的距离，判断点击位置是否在圆形区域内。

```
circle{width:100px;height:100px;background-color:red;border-radius:50%;} functionhandleClick(event){varcircle=document.getElementById("circle");varcircleRect=circle.getBoundingClientRect();varcircleCenterX=circleRect.left+circleRect.width/2;varcircleCenterY=circleRect.top+circleRect.height/2;varclickX=event.clientX;varclickY=event.clientY;vardistance=Math.sqrt(Math.pow(clickX-circleCenterX,2)+Math.pow(clickY-circleCenterY,2));if(distance<=circleRect.width/2){//
```

点击在圆形区域内// 执行相应操作}}

1) 一个宽高都是100px、设置了border-radius: 50%的元素，用户点击外接矩形的角落也触发了业务逻辑；你会怎样确认圆形命中是否真的成立？

答：不能只凭视觉上变圆就断定命中范围符合需求，应在目标浏览器中实际验证四个角的点击结果。若命中仍按矩形计算，就根据点击坐标到圆心的距离做判断，满足距离不大于半径时才执行操作；元素发生缩放后还要重新取得实际尺寸。

2) 地图上要放一个可点击的圆形热点，后续还要修改半径和颜色，你会用SVG的还是普通元素？

答：这种图形属性需要持续调整的场景适合使用SVG的，通过cx、cy和r明确描述圆，并在该元素上监听点击。它比手工计算形状更直观；若页面只是一个固定圆形入口，等宽高元素配合border-radius: 50%会更轻量。

3) 设计把圆形头像入口从正圆改成宽120px、高80px，但代码仍写border-radius: 50%，点击区域和视觉结果会发生什么变化？

答：它会形成椭圆而不是正圆，因为元素宽高已经不相等，百分比圆角会分别依据两个方向计算。若需求仍是圆形，应保持宽高一致，并以较短边或明确半径约束尺寸；仅调整圆角值无法补偿不一致的盒子比例。

4) 一个通过坐标计算命中的圆形按钮，页面滚动后点击位置整体偏移，你会检查pageX还是clientX与哪组边界数据混用了？

答：应先统一坐标系：getBoundingClientRect()返回的是相对视口的位置，因此应与event.clientX、event.clientY配对。若改用页面坐标，则必须把滚动偏移纳入计算；混用两套坐标会让页面未滚动时正常、滚动后稳定偏移。

5) 画布里有上千个圆形粒子需要点击，而运营页只有一个圆形按钮，这两个页面为什么不该共用同一种实现？

答：运营页的单一入口用等宽高元素加border-radius: 50%即可，结构简单且无需维护坐标。画布中的大量粒子没有独立DOM节点，需要按圆心距离进行命中检测并自行管理对象；粒子越多，逐个检测与重绘的成本越值得关注。

22. 网页验证码是干嘛的，是为了解决什么安全问题

网页验证码本质上是用来区分真人和自动化程序，降低网站被机器批量操作的风险。它常用于登录、注册、投票和发帖等场景，避免程序持续尝试账号密码、批量注册、刷票或灌水。验证码通常要求用户识别随机字符、数字或图形，让自动化操作更难直接通过，从而减少网站资源被恶意滥用。网页验证码（CAPTCHA）的作用是用于区分用户是计算机还是人的公共全自动程序。它主要解决以下安全问题：防止恶意破解：通过要求用户输入验证码，可以防止恶意用户使用自动化程序（如暴力破解工具）对密码、账号进行不断的尝试，提高系统的安全性。防止刷票和论坛灌水：验证码可以阻止自动化程序大规模注册账号、刷票或在论坛上大量无意义的发帖，保护网站资源免受滥用。通过要求用户正确地输入验证码，可以验证用户的身份，确保其为真实的人类用户，而不是自动化程序或恶意攻击者。验证码通常会显示一张包含随机字符、数字或图形的图片，用户需要根据图片中的内容进行识别并输入正确的答案。这样，网页验证码有效地提高了网站和应用程序的安全性，防止了各种恶意行为的发生。

23. viewport

viewport用来控制移动端页面的视口宽高、初始缩放比例以及是否允许用户缩放。常见写法会设置width=device-width和initial-scale=1.0，让布局视口与设备宽度对应，并按正常比例显示。minimum-scale、maximum-scale和user-scalable可继续限制缩放；处理移动端细线时，也可以结合transform: scale(0.5)做局部调整。

```
// width 设置viewport宽度，为一个正整数，或字符串 'device-width' //
device-width                                设备宽度//                                height
设置viewport高度，一般设置了宽度，会自动解析出高度，可以不用设置//                                initial-scale
默认缩放比例（初始缩放比例），为一个数字，可以带小数//                                minimum-scale
允许用户最小缩放比例，为一个数字，可以带小数//                                maximum-scale
允许用户最大缩放比例，为一个数字，可以带小数// user-scalable 是否允许手动缩放 延伸提问怎样处理
```

移动端1px被渲染成2px问题？ 怎样处理 移动端1px被渲染成2px问题？ 局部处理
meta标签中的viewport属性
，initial-scale设置为1rem按照设计稿标准走，外加利用transform的scale(0.5)缩小一倍即可； 全局处理
meta标签中的viewport属性 ，initial-scale设置为0.5rem按照设计稿标准走即可

1) 移动页已经写了width=device-width，但设计稿仍按固定桌面宽度排版；你会认为加上viewport就自动完成适配吗？

答：不会，viewport只让布局视口宽度按设备宽度解析，并设置初始缩放比例，不会自动改造固定宽度布局。页面仍需使用合适的相对单位、弹性布局或断点规则；如果内容本身写死宽度，窄屏依旧会溢出。

2) 活动页希望首次打开按设备宽度以1.0比例展示，你会怎样配置最小必要的viewport内容？

答：可使用width=device-width, initial-scale=1.0，前者让视口匹配设备宽度，后者指定初始缩放比例。height通常无需单独设置；继续加入minimum-scale、maximum-scale或user-scalable前，应确认确有产品约束。

3) 产品为了防止页面被放大，要求写maximum-scale=1.0,user-scalable=no，但页面包含密集表单和小字号说明，你会直接接受吗？

答：不会把禁止缩放当作默认配置，因为它会限制用户手动放大页面，密集内容可能因此更难阅读。应先通过响应式布局和可读字号解决展示问题，再判断业务是否确实必须锁定缩放；即便语法有效，也不代表体验合理。

4) 一条设计稿中的细线在部分移动设备上看起来变粗，团队准备把全站initial-scale改成0.5，你会先排查什么？

答：先检查当前viewport、设备像素比、线条实现方式和页面是否已按设计稿做过缩放，避免重复缩放。局部细线可考虑在保持initial-scale=1的前提下单独用transform: scale(0.5)处理；全局改为0.5会影响整个页面尺寸与缩放语义。

5) 适配方案里有人主张用全局initial-scale=0.5解决所有细线，有人只处理组件边框，你倾向哪一种？

答：通常更倾向局部处理，因为它把影响限制在确实需要变细的元素，不会改变整页的初始缩放。全局0.5只有在尺寸体系和rem计算都按该策略统一设计时才应考虑；迁移现有页面时容易造成布局整体缩小或比例错乱。

24. 渲染优化

渲染优化的核心是减少阻塞、网络与DOM开销，让资源加载和页面更新更高效。我一般会把样式放在顶部、脚本放在底部，并结合gzip、CDN、缓存和图片预加载改善加载过程。更新页面时尽量批量操作DOM，多项样式通过className统一切换，动画优先考虑CSS。小图标可用base64减少请求，但大图会增加CPU开销，需要权衡。禁止使用iframe（阻塞父文档onload事件）iframe会阻塞主页面的Onload事件搜索引擎的检索程序无法解读这种页面，不利于SEOiframe和主页面共享连接池，而浏览器对相同域的连接有限制，所以会影响页面的并行加载使用iframe之前需要考虑这两个缺点。如果需要使用iframe，最好是通过javascript动态给iframe添加src属性值，这样可以绕开以上两个问题禁止使用gif图片实现loading效果（降低CPU消耗，提升渲染性能）使用CSS3代码代替JS动画（尽可能避免重绘重排以及回流）对于一些小图标，可以使用base64位编码，以减少网络请求。但不建议大图使用，比较耗费CPU小图标优势在于减少HTTP请求避免文件跨域修改及时生效页面头部的会阻塞页面；（因为Renderer进程中JS线程和渲染线程是互斥的）页面中空的href和src会阻塞页面其他资源的加载（阻塞下载进程）网页gzip，CDN托管，data缓存，图片服务器前端模板 JS+数据，减少由于HTML标签导致的带宽浪费，前端用变量保存AJAX请求结果，每

次操作本地变量，不用请求，减少请求次数用innerHTML代替DOM操作，减少DOM操作次数，优化javascript性能当需要设置的样式很多时设置className而不是直接操作style少用全局变量、缓存DOM节点查找的结果。减少IO读取操作图片预加载，将样式表放在顶部，将脚本放在底部

加上时间戳对普通的网站有一个统一的思路，就是尽量向前端优化、减少数据库操作、减少磁盘IO。iframe会阻塞主页面的Onload事件搜索引擎的检索程序无法解读这种页面，不利于SEO。iframe和主页面共享连接池，而浏览器对相同域的连接有限制，所以会影响页面的并行加载。使用iframe之前需要考虑这两个缺点。

如果需要使用iframe，最好是通过javascript动态给iframe添加src属性值，这样可以避开以上两个问题。

小图标优势在于减少HTTP请求避免文件跨域修改及时生效。减少HTTP请求避免文件跨域修改及时生效。

1) 有人为了“减少请求”把一张大背景图也转成base64塞进样式表，你为什么不会把它当成无条件优化？

答：base64更适合体积较小的图标，优势是减少独立请求并避开图片文件的跨域问题。大图编码会扩大样式资源并增加解码和处理成本，还无法像独立文件那样灵活加载；是否采用应结合资源大小和缓存方式判断。

2) 商品列表需要一次更新几十个节点的多个样式，你会逐条写element.style，还是切换一个className？

答：会优先切换预先定义好的className，把多项样式变化集中交给样式规则处理，减少零散的DOM样式操作。若某些值必须由运行时计算，可只把动态部分单独写入；类名过度组合也会增加样式维护成本。

3) 第三方内容必须通过iframe接入，业务又要求主页面尽快完成加载，你会怎样降低它对加载流程的影响？

答：不能简单禁止iframe，但可以先渲染主页面，再由JavaScript按需设置它的src，避免一开始就参与关键加载。还应关注它与主页面共享连接资源以及搜索引擎难以解读内部内容的限制；核心正文不宜只放在其中。

4) 线上页面偶发重复请求，排查时发现若干和，这些空属性为什么值得优先清理？

答：空的src或href可能被浏览器解析成对当前页面或相关资源的请求，从而干扰其他资源加载并制造无意义流量。应在没有有效地址时移除属性或延后赋值，同时检查模板默认值；仅做网络层优化无法消除这类错误标记。

5) 首屏同时包含内联脚本、外部样式、预加载图片和动画，你会怎样安排关键资源，而不是只喊“减少重绘”？

答：样式表应尽量放在顶部，非关键脚本放到底部或避免阻塞首屏，并对确实要展示的图片做合理预加载。动画优先评估能否用CSS3完成，同时结合gzip、CDN和缓存减少传输等待；过度预加载也会争抢关键资源。

25. meta viewport相关

新项目通常保留charset、viewport、title和description，再按需求增加theme-color或robots。viewport用来控制移动端页面的初始缩放和可视区域，是适配手机页面的关键配置。X-UA-Compatible、renderer、HandheldFriendly以及UC、QQ的私有标签大多属于历史兼容方案，现在不应整段照抄。实际配置时还要谨慎限制user-scalable，避免影响用户缩放页面。下面这份清单是2015年前后的历史资料，其中X-UA-Compatible、apple-mobile-web-app-*、renderer、HandheldFriendly、MobileOptimized、UC/QQ的x5-*私有标签绝大多数已经失效或被厂商废弃，新项目不要照抄。

今天真正需要写的只有四个：charset、viewport、title、description，再按需加theme-color和robots。

26. 你做的页面在哪些浏览器测试过？这些浏览器的内核分别是什么？

我一般按渲染内核和真实使用环境确定测试范围，覆盖Chrome、Safari、Firefox，以及国内业务常见的微信内置浏览器。Chrome和新版Edge使用Blink，Safari使用WebKit，Firefox使用Gecko。iOS上的浏览器都使用WebKit，所以移动端还需要用iOS真机验证，不能只测桌面版Chrome。兼容性通常交给Browserslist、Autoprefixer和按需polyfill保障，不依赖UA判断。

现在还活着的三个渲染引擎
Blink：Chrome、Edge、Opera、以及国内几乎所有“壳”浏览器（360、QQ、UC的桌面版）。JS引擎是V8
WebKit：Safari，以及iOS上的全部浏览器。JS引擎是JavaScriptCore
Gecko：Firefox。JS引擎是SpiderMonkey
已经退场的（只当历史背景，别当答案主体）
Trident：IE系列，微软已于2022年终止IE11支持EdgeHTML；旧版Edge，2020年起Edge改用Blink
Presto：旧版Opera，2013年起Opera改用Blink
这题的加分点
苹果在iOS上强制所有浏览器使用WebKit，所以“iOS上的Chrome”本质是套壳Safari。移动端遇到“安卓正常、iOS挂了”的兼容问题，多半就出在这里——能说出这一点，比背内核名字有用得多。实际测试矩阵怎么讲：Chrome + Safari（含iOS真机）+ Firefox + 微信内置浏览器（国内业务必测），配Browserslist + Autoprefixer + core-js 按需polyfill 做保障。

1) 候选人说桌面端测过Chrome、新版Edge和Opera，所以已经覆盖三个不同内核；你会怎样追问这个判断？

答：这三者当前主要都使用Blink，不能按浏览器品牌数量等同于内核覆盖数量，相关JavaScript引擎也都是V8。若要覆盖主要差异，还应加入使用WebKit的Safari和使用Gecko的Firefox。

2) 一个面向国内用户的响应式站点准备上线，你会要求怎样的最小测试矩阵，而不是只给一份浏览器名称清单？

答：至少覆盖Chrome、Safari、Firefox、iOS真机和微信内置浏览器，并围绕业务核心流程执行测试。构建侧再用Browserslist明确目标，配合Autoprefixer和按需polyfill；工具配置不能替代真实环境验证。

3) 同一个页面在安卓Chrome正常，在iOS

Chrome上布局异常，开发坚持“两边都是Chrome，不可能是内核问题”，你怎么判断？

答：应把它视为WebKit兼容问题继续排查，因为苹果要求iOS上的浏览器使用WebKit，iOS Chrome并不是桌面或安卓上的Blink实现。先用同版本系统的Safari对照复现；仅按浏览器图标归类会误判测试覆盖。

4) 兼容性报告里写着“旧版Edge用EdgeHTML，现在的Edge也一样”，你会如何纠正并处理测试范围？

答：需要按版本阶段拆开：旧版Edge使用EdgeHTML，从改用Chromium架构后则使用Blink。是否继续测试旧引擎取决于项目明确支持范围，不能把历史内核当成当前答案；同理，Trident和Presto主要作为历史背景。

5) 产品要求支持Safari，开发认为测过iOS

Chrome就足够；在桌面和移动端两个范围里，这个结论分别成立吗？

答：在iOS上两者都基于WebKit，测试结果有参考价值，但浏览器外壳和集成功能仍可能不同，核心流程仍应在目标应用中确认。桌面端Chrome使用Blink、Safari使用WebKit，因此桌面Safari不能被Chrome测试替代。

27. div+css的布局较table布局有什么优点？

相比用table做页面布局，div配合CSS能让结构和表现分离，改版与维护都更方便。样式集中在CSS中，调整布局时通常不必改动大量页面结构，文档也会更清晰简洁。语义合理的结构对搜索引擎更友好，页面加

载也更轻便。现在实际开发会优先使用Flex或Grid，而table应保留给真正的二维表格数据。

改版的时候更方便 只要改css文件。页面加载速度更快、结构化清晰、页面显示简洁。表现与结构相分离。易于优化（seo）搜索引擎更友好，排名更容易靠前。

28. a：img的alt与title有何异同？b：strong与em的异同？

alt是图片无法呈现时的替代内容，title是补充提示；strong表示内容重要，em表示语气强调。图片加载失败或用户代理不能显示图片时，会使用alt传达原本的信息，而title通常在鼠标悬停时提供额外说明。strong和em都是语义化标签，浏览器常分别显示为粗体和斜体，但不能只把它们当作样式工具。如果只是视觉上加粗或倾斜，更适合使用b、i或CSS。alt(alt text): 用于为不能显示图像、窗体或applets的用户代理（UA）提供替代文字。它由lang属性指定替代文字的语言。在某些浏览器中，当没有title属性时，会将alt属性作为工具提示（tooltip）显示。title(tool tip): 用于为元素提供额外的提示信息，当鼠标悬停在元素上时显示。它提供了一种向用户解释元素用途或提供有关元素的补充信息的方式。

b：strong与em的异同？strong: 是表示文本的重要性或紧急性的标签，通常呈现为加粗的文本样式。它用于强调内容的重要性，可以为内容赋予更大的权重。em: 是表示文本的强调或重要性的标签，通常呈现为斜体的文本样式。它用于更强烈地强调内容，使其在阅读时更具有突出性，但并不改变内容的含义。

注意：strong和em都是语义化标签，用于表示文本的语义和重要性，而不仅仅是样式上的改变。

1) 商品详情页的主图加载失败后只剩一个图标，但鼠标悬停还能看到title="黑色运动鞋"，为什么这仍不算提供了可用的替代内容？

答：title只是补充提示，不能替代alt承担图片不可见时的文本替代职责。应根据图片用途填写alt="黑色运动鞋"，而非依赖悬停提示；触屏设备、键盘操作和部分辅助技术都未必能稳定获得title。

2) 内容编辑把文章中二十张装饰分隔图都写成alt="装饰图片"，读屏用户需要逐张听完，前端应怎样落地修正？

答：纯装饰图片应使用空替代文本alt=""，让辅助技术跳过无信息价值的内容；承载信息的图片才需要描述其功能或含义。不能简单删除alt，因为缺失属性时，用户代理可能改为朗读文件名等无意义内容。

3) 设计师要求所有加粗文字都改成，编辑又把语气重读全部写成，这两种语义应怎样划界？

答：表示内容的重要性或紧急性，表示语气上的强调，默认粗体或斜体只是常见呈现。若需求只是视觉加粗或倾斜，应交给CSS；否则样式变化会被误写成语义，后续阅读模式和辅助技术也难以理解作者意图。

4) 线上无障碍验收时，测试人员发现读屏器朗读和普通文本的语调几乎相同，是否应把它全部换成？

答：不应仅凭听感替换，因为读屏器是否改变语调取决于产品及用户设置，语义价值并不等于固定的声音效果。应检查重要内容是否确实使用、强调语气是否使用；若只是装饰性粗体，再改用或CSS。

5) CMS同时要求图片有简短替代文本和较长的操作说明，产品想把两段内容都塞进alt，前端该怎样取舍？

答：alt应保留图片不可见时仍必须传达的等价信息，额外说明可放在邻近正文或可访问的说明区域中。title可作为补充提示，但不能承载唯一的关键操作信息；过长的alt会干扰连续阅读，也无法解决悬停不可用的问题。

29. 你能描述一下渐进增强和优雅降级之间的不同吗

渐进增强是从基本可用开始逐步加能力，优雅降级则是先做完整体验，再为低版本浏览器提供兼容。前者面向未来扩充效果和交互，同时保证基础功能始终可靠；后者从复杂形态往回收缩，允许部分体验衰减。实际开发中，我一般会优先保证核心内容和功能可用，再根据浏览器能力增强体验。 渐进增强：针对低版本浏览器进行构建页面，保证最基本的功能，然后再针对高级浏览器进行效果、交互等改进和追加功能达到更好的用户体验。优雅降级：一开始就构建完整的功能，然后再针对低版本浏览器进行兼容。 区别：优雅降级是从复杂的现状开始，并试图减少用户体验的供给，而渐进增强则是从一个非常基础的，能够起作用的版本开始，并不断扩充，以适应未来环境的需要。降级（功能衰减）意味着往回看；而渐进增强则意味着朝前看，同时保证其根基处于安全地带

30. 为什么利用多个域名来存储网站资源会更有效？

多域名存储资源主要是HTTP/1.1时代的优化，可以绕过同域并发限制、减少主域连接占用，并避免静态请求携带主站cookie。它也便于通过CDN缓存和分发资源，让静态内容与主站隔离。到了HTTP/2或HTTP/3，单连接可以多路复用，继续拆域名会增加DNS、TCP和TLS建连成本。现在通常收敛到一两个静态域名，并用p reconnect提前建连；其中独立的无cookie域名仍有价值。

利用多个域名来存储网站资源可以带来以下好处：CDN缓存更方便：内容分发网络（CDN）可以更轻松地缓存和分发位于不同域名下的资源，提高资源的访问速度和可用性。突破浏览器并发限制：大多数浏览器对同一域名下的并发请求数量有限制，通过将资源分布在多个域名下，可以突破这一限制，同时发送更多的并发请求，加快页面加载速度。节约cookie带宽：浏览器在每个请求中都会携带相应域名下的cookie信息，通过将资源分布在不同的域名下，可以减少对cookie的传输，节约带宽和提高性能。节约主域名的连接数：浏览器对同一域名下的连接数也有限制，通过将资源请求分散到多个域名下，可以减少对主域名的连接数占用，提高页面的响应速度和并发处理能力。防止不必要的安全问题：将静态资源与主要网站内容分离到不同的域名下，可以降低恶意攻击者利用资源加载过程中的安全漏洞对主站点进行攻击的风险。

这题现在要反着答 上面那套结论属于HTTP/1.1 时代。HTTP/2

之后单条连接就能多路复用，同域并发限制已经不存在，再拆域名反而是负优化——

每多一个域名就多一次 DNS 解析 + TCP 握手 + TLS 握手，还破坏了连接复用。

今天的做法：静态资源收敛到 1~2 个域名走 HTTP/2 或 HTTP/3，用提前建连。唯一还成立的是 cookie-free 域名 —— 独立静态域名可以避免每个静态资源请求都背上主站的 cookie。

面试时能主动指出"这条优化已经被 HTTP/2 推翻"，比复述老结论加分得多。

1) 一个使用HTTP/2的活动页把 120 个静态资源平均拆到 8 个域名，团队认为域名越多并发越高，你会接受这个结论吗？

答：不会，突破同域并发限制主要是HTTP/1.1时代的收益；HTTP/2已能在单条连接上多路复用。拆成8个域名会增加DNS、TCP和TLS建连成本，还会破坏连接复用，因此现代站点通常应收敛到少量静态域名。

2) 主站登录后带有较大的业务cookie，而每个字体、图片和脚本请求都携带它，静态资源域名应怎样规划？

答：可将静态资源放到不携带主站业务cookie的独立域名，并配置对应的CDN缓存与回源策略。这样能减少重复头部带宽，但必须控制cookie的Domain范围；若设置成覆盖所有子域，独立静态域名也无法获得预期收益。

3) 老旧内网页面仍通过HTTP/1.1加载大量小图，而公网新版已升级到HTTP/2，两套环境是否应该沿用同一套域名分片规则？

答：不应机械共用：HTTP/1.1下适度分片可能缓解同域连接限制，HTTP/2下则更适合集中资源复用连接。应先确认实际协商协议和资源结构，再决定是否保留旧分片；迁移时还要评估缓存地址变化与额外建连成本。

4) 改成三个静态域名后，监控显示首屏资源等待时间反而增加，DevTools中每组请求前都有建连阶段，你会先查什么？

答：先在Network的Protocol列确认是否已使用h2或h3，再检查各域名的DNS、连接和TLS耗时。若资源被分散后无法复用连接，应合并到1~2个域名；确需保留的关键域名可用提前建连。

5) 安全团队要求静态资源与主站彻底隔离，性能团队要求所有资源同域复用连接，这场选型冲突该怎样裁决？

答：应按资源信任边界与实际协议权衡，而不是单看域名数量。独立域名有利于隔离主站cookie、缓存策略及部分资源加载风险，但会产生额外建连；在HTTP/2或HTTP/3下通常以少量独立静态域名折中，并配合严格的域名权限配置。

31. 简述一下src与href的区别

src表示把外部资源嵌入当前元素，href表示在当前文档与目标资源之间建立引用关系。像script、img和iframe通常使用src，资源会成为元素内容的一部分；a和link则通常通过href指向页面或样式资源。浏览器处理src资源时可能暂停当前文档的加载和解析，而处理href引用时通常可以并行进行。

src和href是HTML中两个常见的属性，它们有以下区别：src属性（source）用于指定要嵌入到当前文档中的外部资源的位置。例如，用于引入一个外部的JavaScript文件，或者用于显示一个外部的图像文件。浏览器在解析到带有src属性的元素时，会暂停当前文档的加载和解析，去下载并执行或显示指定的资源。href属性（hypertext reference）用于建立当前文档和引用资源之间的关联。它通常用于链接到其他文档或外部资源，例如Link用于创建一个指向外部网页的链接，或者用于引入外部的CSS样式表。浏览器在解析到带有href属性的元素时，会同时进行当前文档和引用资源的加载和处理，而不会阻塞当前文档的解析。

总结来说：src用于替换当前元素，指向的资源会嵌入到文档中，例如脚本、图像、框架等。href用于建立文档与引用资源之间的链接，例如链接到其他文档或引入外部样式表。注意：尽管它们的用途不同，但在实际使用时，需要根据元素的类型和需求正确地选择使用src或href属性。

1) 新人说“凡是会发起网络请求的属性都该叫src”，但样式表使用，你怎样指出这个反例的关键？

答：是否发起请求不是划分标准，关键是元素表达嵌入还是关联。src指向替代或嵌入当前位置的资源，如图片和脚本；link始终声明文档与外部资源的关系，关系类型由rel指定，所以即使下载样式表也使用href。

2) 首屏需要提前获取一份稍后执行的脚本，工程师写了，为什么预加载资源仍使用href？

答：preload建立的是当前文档与待预取资源之间的关系，并没有用该资源替换link元素，因此使用href。rel="preload"只负责提示浏览器提前获取，真正执行仍需对应的；两处地址或资源类型不匹配会浪费下载。

3) 一个页面同时遇到图片、外链、样式和脚本四类需求，组件API想统一暴露名为url的属性，渲染层应怎样映射到src与href？

答：渲染层应按元素语义映射：img、script等嵌入资源使用src，a和link等建立关联或导航的元素使用href。统一业务字段可以减少上层差异，但底层不能把两者互换；还需分别处理替代文本、rel等配套语义。

4) 线上页面的外部脚本下载完成却没有执行，代码是，排查时为什么不能只盯着网络状态？

答：script加载外部脚本要求使用src，href不会建立脚本嵌入关系，因此网络面板没有预期请求时应先检查元素和属性是否匹配。改成后再查状态码与执行错误；若标签同时存在src和内联代码，内联内容会被忽略。

5) 性能评审中有人断言“所有src都会阻塞HTML解析，所有href都不会”，面对图片和样式表两个反例应怎样修正？

答：不能仅由属性名判断阻塞行为，阻塞取决于元素类型、资源用途及加载方式。图片的src通常不会像经典同步脚本那样暂停解析，而样式表虽使用href，仍会影响渲染并可能间接影响脚本执行；应结合具体元素分析加载、解析和渲染阶段。

32. 知道的网页制作会用到的图片格式有哪些？

网页图片常见格式可以按内容归类：照片通常用JPEG，透明图或高质量位图用PNG，简单动画可用GIF，需要无损缩放的图形用SVG。PNG-8最多支持256种颜色，体积较小；PNG-24色彩更多，但文件通常也更大。新格式里，WebP兼顾较高压缩率和图像质量，APNG则支持帧动画。具体选择要看画面类型、透明度、动画需求和浏览器兼容性，不能只追求格式新。在网页制作中，常用的图片格式包括：JPEG（Joint Photographic Experts Group）：适用于存储照片和复杂的图像，具有较高的压缩比，但会有一定的图像质量损失。PNG（Portable Network Graphics）：适用于图标、透明背景的图像以及需要保留较高图像质量的场景。可以选择使用PNG-8或PNG-24，前者支持最多256种颜色，后者支持更多颜色但文件体积更大。GIF（Graphics Interchange Format）：适用于简单动画和图标，支持透明背景和基本的透明度。SVG（Scalable Vector Graphics）：矢量图形格式，使用XML描述图形，具有无损缩放和可编辑性。

除了上述常见的图片格式，还有一些新兴的图片格式：WebP：由Google开发的一种旨在提高图片加载速度的格式，具有较高的压缩率和图像质量，逐渐被主流浏览器支持。APNG（Animated Portable Network Graphics）：是PNG的位图动画扩展，支持帧动画效果，但浏览器兼容性较差。请注意，选择合适的图片格式应根据具体需求，如图像内容、透明度要求、动画效果等。新的图片格式如WebP和APNG可以根据项目需求和兼容性考虑是否使用。

1) 摄影频道把高色彩照片导出为PNG-24，而品牌线稿图导出为JPEG，上线后体积和边缘质量都不理想，你会怎样调整格式？

答：照片和复杂连续色彩图像通常优先选JPEG，接受有损压缩以控制体积；线稿、图标或需要透明背景的位图更适合PNG。若图形本身可用矢量描述，应优先考虑SVG，但复杂照片不适合强行矢量化。

2) 设计系统有 300

个单色图标，需要在不同主题和尺寸下变色，前端在PNG与SVG之间应怎样选择？

答：这类可缩放、需要动态改色的图标更适合SVG，它使用矢量描述，无损缩放且便于编辑。PNG适合固定像素效果或复杂透明位图，但多尺寸会产生多份资源；使用外部SVG时还需根据嵌入方式规划样式控制。

3) 产品要求一张带透明背景的加载动画同时兼顾颜色层次和帧动画，设计师只提供GIF，你会考虑哪些替代方案？

答：GIF适合简单动画，但其颜色和透明能力有限；需要更丰富透明效果时可评估APNG，现代压缩方案也可纳入候选。最终仍应按目标浏览器验证支持情况并准备回退，因为来源资料明确提示APNG的兼容性需要项目侧评估。

4) 图片组件升级为AVIF、WebP和JPEG三套资源后，部分用户只看到破图，你会怎样定位格式协商或回退链路？

答：先检查中各、资源地址及最终回退是否齐全，再在目标浏览器确认实际选择的候选。浏览器会选择自己支持的格式；若类型声明错误、文件不可达或缺少JPEG等兜底，格式支持差异就会直接表现为破图。

5) 移动端商品卡在不同视口和高像素密度屏幕上总是下载同一张大图，格式已经换成WebP，为什么仍不算完整优化？

答：格式压缩只解决编码效率，不能替代尺寸选择；应使用srcset与sizes提供适合不同展示宽度或像素密度的候选资源。浏览器据此选择文件，同时保留可用的默认src；候选尺寸声明不准确仍可能下载过大的图片或造成清晰度不足。

33. 在CSS/JS代码上线之后，开发人员经常会优化性能。从用户刷新网页开始，一次JS请求一般情况下有哪些地方会有缓存处理？

一次JS请求通常会经过DNS、CDN、浏览器和服务器这几层缓存处理。DNS缓存复用域名解析结果，CDN则让用户从更近的节点取得静态文件，减少回源和传输时间。浏览器依据Cache-Control、Expires等响应头决定是否复用本地资源，服务器也可以缓存动态生成的内容。具体保留多久不能一刀切，需要结合文件更新频率和业务场景设置。

在进行JS请求时，可以在以下几个地方进行缓存处理，以提高性能和减少资源加载时间：DNS缓存：浏览器会缓存已解析的域名和对应的IP地址，这样在下次请求同一域名时可以直接使用缓存的IP地址，避免重新进行DNS解析。CDN缓存：如果使用了内容分发网络（CDN），CDN会缓存静态资源文件，如CSS和JS文件，以便快速地分发给用户。当用户再次请求同一资源时，可以从CDN缓存中获取，减少向源服务器的请求次数。浏览器缓存：浏览器会缓存已请求的静态资源文件，如CSS和JS文件。可以通过设置HTTP响应头中的Cache-Control和Expires字段来控制浏览器缓存的行为。如果设置了适当的缓存策略，浏览器在下次请求同一资源时可以直接从本地缓存中获取，而不需要再次向服务器请求。服务器缓存：服务器可以对动态生成的JS文件进行缓存，以避免重复生成相同的响应。服务器可以通过设置响应头中的Cache-Control和Expires字段，或者使用缓存代理服务器来进行缓存处理。需要注意的是，缓存的有效期限和缓存策略的设置需要根据具体的需求和业务场景来确定。合理地利用缓存可以显著提高网页加载速度和用户体验。

34. 一个页面上有大量的图片（大型电商网站），加载很慢，你有哪些方法优化这些图片的加载，给用户更好的体验。

我会从减少图片体积、控制加载时机和提高传输效率三个方向优化。图片先按展示尺寸压缩，并根据内容选择JPEG、PNG、WebP或SVG，避免小屏设备下载过大的资源。非可视区域使用懒加载，可能马上访问的下一组图片再按需预加载，同时借助CDN和浏览器缓存减少传输与重复请求。大量小图标还可以合并为精灵图，从而降低请求数量。使用图像压缩技术：通过使用图像压缩工具，如PhotoShop、TinyPNG等，将图片文件的大小减小，以减少加载时间。使用适当的图像格式：根据图像的特性选择合适的图像格式，如JPEG、PNG、WebP等。JPEG适用于照片和复杂图像，而PNG适用于简单的图标和透明图像。WebP是一种现代的图像格式，可以在保持良好质量的同时减小文件大小。图片CDN加速：使用内容分发网络（CDN）来加速图片的传输，将图片文件缓存到离用户更近的服务器，减少传输时间。图片延迟加载：采用图片懒加载技术，将页面上不可见区域的图片暂时不加载，当用户滚动页面至可见区域时再进行加载，以减少初始加载时间。使用CSS精灵图：将多个小图标或背景图片合并为一张大图，并利用CSS的background-position来定位显示需要的部分，减少HTTP请求的数量。使用矢量图形：使用矢量图形（如SVG）代替位图，以减小文件大小并保持清晰度，适用于简单的图形和图标。响应式图片：针对不同的设备和屏幕尺寸提供适当大小的图片，以避免在小屏幕设备上加载过大的图片。图片懒加载、预加载：根据用户的浏览行为，提前加载下一页或下一组图片，以提高用户体验和流畅度。图片缓存：设置适当的缓存策略，让浏览器在首次加载后对图片进行缓存，减少重复加载的次数。

综合应用这些优化技术可以减小图片的加载大小和加载时间，提升网页的加载速度，给用户更好的体验。

35. 常见排序算法的时间复杂度,空间复杂度

常见排序中，冒泡、插入和选择排序通常是 $O(n^2)$ 时间、 $O(1)$ 空间，而快排、归并和堆排通常能做到 $O(n \log n)$ 时间。冒泡和插入在最好情况下可到 $O(n)$ ，选择排序仍是 $O(n^2)$ 。快排最坏会退化到 $O(n^2)$ ，空间复杂度最好为 $O(\log n)$ 、平均和最坏为 $O(n)$ ；归并需要 $O(n)$ 额外空间，堆排只需 $O(1)$ 。计数排序适合计数范围明确的场景，时间为 $O(n+k)$ 、空间为 $O(k)$ 。

下面是一些常见的排序算法及其时间复杂度和空间复杂度的概述：

冒泡排序 (Bubble Sort) :	时间复杂度：最好情况下 $O(n)$ ，平均和最坏情况下 $O(n^2)$	空间复杂度： $O(1)$
插入排序 (Insertion Sort) :	时间复杂度：最好情况下 $O(n)$ ，平均和最坏情况下 $O(n^2)$	空间复杂度： $O(1)$
选择排序 (Selection Sort) :	时间复杂度：最好情况下 $O(n^2)$ ，平均和最坏情况下 $O(n^2)$	空间复杂度： $O(1)$
快速排序 (Quick Sort) :	时间复杂度：最好情况下 $O(n \log n)$ ，平均情况下 $O(n \log n)$ ，最坏情况下 $O(n^2)$	空间复杂度：最好情况下 $O(\log n)$ ，平均和最坏情况下 $O(n)$
归并排序 (Merge Sort) :	时间复杂度：最好情况下 $O(n \log n)$ ，平均情况下 $O(n \log n)$ ，最坏情况下 $O(n \log n)$	空间复杂度： $O(n)$
堆排序 (Heap Sort) :	时间复杂度：最好情况下 $O(n \log n)$ ，平均情况下 $O(n \log n)$ ，最坏情况下 $O(n \log n)$	空间复杂度： $O(1)$
希尔排序 (Shell Sort) :	时间复杂度：取决于所选的间隔序列，最好情况下 $O(n \log n)$ ，平均和最坏情况下根据间隔序列的选择而不同	空间复杂度： $O(1)$
计数排序 (Counting Sort) :	时间复杂度：最好情况下 $O(n+k)$ ，平均和最坏情况下 $O(n+k)$	空间复杂度： $O(k)$ ，其中 k 是计数范围
桶排序 (Bucket Sort) :	时间复杂度：最好情况下 $O(n+k)$ ，平均和最坏情况下根据桶的数量和排序算法的选择而不同	空间复杂度： $O(n+k)$
基数排序 (Radix Sort) :	时间复杂度：最好情况下 $O(nk)$ ，平均和最坏情况下 $O(nk)$	空间复杂度： $O(n+k)$

36. web开发中会话跟踪的方法有哪些

会话跟踪常用Cookie配合Session，也可以通过URL重写、隐藏表单字段或客户端IP传递和识别会话。Cookie会由浏览器自动携带会话标识，服务器再用该标识关联保存在服务端的会话数据。禁用Cookie时可以把会话标识放进URL或隐藏input，但前者会拉长地址并暴露会话信息，后者主要适用于表单提交。单靠IP并不可靠，因为多个用户可能共享同一个出口地址。Cookie: 使用Cookie是最常见的会话跟踪方法之一。服务器在响应中设置一个包含会话ID的Cookie，然后在后续的请求中，浏览器会自动将该Cookie发送回服务器，以标识用户的会话。Session: 服务器使用会话来跟踪用户的状态。每个会话都会分配一个唯一的会话ID，该ID通常存储在Cookie中或通过URL重写传递给服务器。服务器使用会话ID来关联用户的请求，并在服务器端存储会话数据。URL重写: 将会话ID作为查询参数添加到URL中，以便在每个请求中传递会话信息。这种方法不需要依赖Cookie，适用于禁用Cookie的情况，但会增加URL的长度并暴露会话信息。隐藏input: 在表单中添加一个隐藏的input字段，将会话ID作为其值传递给服务器。服务器接收到请求时可以通过解析请求参数获取会话ID，以进行会话跟踪。IP地址: 使用客户端的IP地址作为会话跟踪的依据。服务器根据不同的IP地址来区分不同的用户，并跟踪他们的会话状态。然而，由于多个用户可能共享相同的IP地址（如在同一局域网内），这种方法可能不准确。

这些方法可以单独或结合使用，根据实际需求和安全考虑选择适当的会话跟踪方法。

37. HTTP request报文结构是怎样的

HTTP/1.1请求报文由请求行、请求头、空行和可选的请求体组成。请求行包含请求方法、请求URI和协议版本，并以CRLF结束。后面可以有多行请求头，每行同样以CRLF结束，再用一个空行与请求体分隔。像GET请求可能没有请求体，而提交数据时可以携带消息实体。首行是Request-Line包括：请求方法，请求URI，协议版本，CRLF首行之后是若干行请求头，包括general-header，request-header或者entity-header，每个一行以CRLF结束请求头和消息实体之间有一个CRLF分隔根据实际请求需要可能包含一个消息实体一个请求报文例子如下： GET /Protocols/rfc2616/rfc2616-sec5.html HTTP/1.1 Host: www.w3.org

Connection: keep-alive Cache-Control: max-age=0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8 User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/35.0.1916.153 Safari/537.36 Referer: https://www.google.com.hk/ Accept-Encoding: gzip,deflate,sdch Accept-Language: zh-CN,zh;q=0.8,en;q=0.6 Cookie: authorstyle=yes If-None-Match: "2cc8-3e3073913b100" If-Modified-Since: Wed, 01 Sep 2004 13:24:52 GMT name=qiu&age=25

1) 网关抓到一段HTTP/1.1请求，首行后直接出现JSON，没有空行分隔请求头和实体，后端为什么可能把它判成畸形报文？

答：HTTP/1.1请求先是请求行，再是逐行的请求头，头部与消息实体之间必须用一个空行分隔，也就是连续的CRLF。缺少该分隔时，解析器可能把JSON当成头字段或等待更多数据；修复时应由成熟客户端生成报文，避免手工拼接。

2) 你在终端手工构造POST /users

HTTP/1.1，请求行只写了路径和协议版本，却收到400，最先应核对哪些结构字段？

答：请求行必须依次包含请求方法、请求目标和协议版本，并以CRLF结束；HTTP/1.1请求还应提供合法的Host头。随后检查每个请求头的行结束符、头体空行及实体长度是否一致，任一结构错误都可能让服务器拒绝解析。

3) 同一个上传接口有时发送空实体、有时发送表单内容，团队认为所有请求都必须包含消息体，这会造成什么误判？

答：消息实体是按请求需要可选的，请求方法和接口语义决定是否携带，而不是报文固定组成。即使没有实体，请求行、请求头和头部终止空行仍需正确；有实体时还要让接收端能确定边界，否则可能出现截断或持续等待。

4) 线上登录偶发返回431或400，应用日志还没进入业务处理，浏览器请求里积累了大量Cookie，你会怎样排查？

答：先检查反向代理和应用服务器的请求头大小限制，并统计Cookie及其他头字段的实际体积。规范未统一规定实现上限，超限可能在业务代码前被拒绝；应清理无效Cookie、收窄作用域，谨慎调大限制以免放大资源消耗风险。

5) 抓包工具显示HTTP/2使用二进制帧，但DevTools仍按请求行、请求头和正文展示，排障时该以哪个视图理解报文？

答：DevTools展示的是便于阅读的逻辑请求信息，不代表线上仍按HTTP/1.1文本格式传输。HTTP/2会把信息编码为二进制帧并压缩头部；排查字段语义可看开发者工具，分析分帧、复用或协议错误则应查看实际协议与抓包结果。

38. HTTP response报文结构是怎样的

HTTP/1.1响应报文由状态行、响应头、空行和可选的响应体组成。状态行包含协议版本、状态码和状态描述，并以CRLF结束。随后是若干响应头，再通过一个CRLF空行与响应体分开。响应体可以承载HTML、JSON等内容，但是否存在要看具体响应。首行是状态行包括：HTTP版本，状态码，状态描述，后面跟一个CRLF首行之后是若干行响应头，包括：通用头部，响应头部，实体头部响应头部和响应实体之间用一个CRLF空行分隔最后是一个可能的消息实体 响应报文例子如下： HTTP/1.1 200 OK Date: Tue, 08 Jul 2014 05:28:43 GMT Server: Apache/2 Last-Modified: Wed, 01 Sep 2004 13:24:52 GMT ETag: "40d7-3e3073913b100" Accept-Ranges: bytes Content-Length: 16599 Cache-Control: max-age=21600 Expires: Tue, 08 Jul 2014 11:28:43 GMT P3P:

39. title与h1的区别、b与strong的区别、i与em的区别

表示页面标题，表示内容主标题；、偏展示，、带有强调语义。参与文档层级，搜索引擎通常会把它视为页面主要标题，而用于页面本身的标题。和主要改变粗体、斜体外观，不表达强调。需要让辅助工具识别重要性或强调语气时，应选或。title属性用于提供元素的额外信息，通常以工具提示的形式显示。它没有语义化的意义，仅表示一个标题或描述性文本。它在SEO中没有直接影响，但可以提供更好的用户体验和辅助工具提示。是HTML中的标题元素，用于表示页面的主标题。它具有层次结构，表示文档的结构和内容。搜索引擎通常会将标签中的文本作为页面的主要标题，并根据其重要性进行权重分配。是用于粗体显示文本的HTML元素，仅仅表示展示的效果，没有语义上的强调意义。在使用屏幕阅读器等辅助工具阅读网页时，不会改变读取方式，仅仅呈现粗体效果。是表示文本的强调元素，具有语义化的含义，用于强调重要内容。在屏幕阅读器等辅助工具中，会以更加强调的方式读取标签中的文本，传达给用户更强的语气。用于将文本显示为斜体，仅表示展示的效果，没有语义上的强调意义。表示强调的文本，具有语义化的含义，用于强调某些内容。在屏幕阅读器等辅助工具中，会以更加强调的方式读取标签中的文本，传达给用户更强的强调效果。总结：和在语义化和SEO方面有区别，一个用于页面标题，一个用于内容标题。和都可以用于加粗文本，但具有语义化的强调效果。和都可以用于斜体文本，但具有语义化的强调效果。

1) 商品详情页为了统一视觉，把页面名称只写进，正文顶部用普通模拟大标题；搜索结果正常，但读屏用户进入页面后难以定位主题，你会判定哪里有问题？

答：问题在于负责页面级标题信息，不能替代正文中的。正文主标题应使用建立文档层次，样式再交给CSS；只用会丢失标题语义，辅助工具和搜索引擎都更难识别内容结构。

2) 运营要求文章里所有红色警告都用加粗，组件库则坚持改成；如果这些文字表示付款风险，你支持哪一方？

答：付款风险属于重要内容，应使用，再通过CSS控制颜色和字重。主要带来视觉上的粗体，不表达重要性；若只是商品名或关键词需要醒目而不存在强调语义，使用反而更贴切。

3) 设计稿把术语、引用语气和必须立即处理的提示都画成斜体，开发准备统一输出；面对这三类内容，你会怎样拆分？

答：不能因视觉效果相同就统一使用。普通斜体展示可用，真正需要强调语气的提示应使用；最终是否倾斜仍由样式决定，否则改版取消斜体时，错误的语义结构仍会保留下来。

4) 线上文章页肉眼看起来标题、加粗和斜体都正常，但无障碍测试反馈重点信息没有被强调；你会优先检查哪些代码现象？

答：我会先检查是否用、承担了主标题和语义强调，再核对页面是否存在明确的、与。视觉正常只能证明样式生效，不能证明文档结构正确；辅助工具的具体朗读方式还会受实现差异影响。

5) 一个内容管理系统只能让编辑选择“粗体”和“斜体”，却无法判断文字是装饰还是强调；你会直接把它们映射成和吗？

答：不应把所有视觉格式直接映射为语义强调，因为编辑选择粗体或斜体未必表示内容重要。可保留、作为展示选项，并另设“重要”和“强调”能力输出、；代价是编辑器模型和迁移规则会更复杂。

40. 请你谈谈Cookie的弊端

Cookie的主要弊端是容量和数量受限、会随请求传输，并且存在安全与跨域限制。单个Cookie通常只有约4096字节，各浏览器对同域数量也有限制，因此不适合存放大量数据。它会增加同域请求的传输开销，存

储在客户端还可能因拦截而泄露session等信息。较大的客户端数据可考虑localStorage、sessionStorage或服务器端存储，但要按使用场景取舍。cookie虽然在存储客户端数据方面提供了方便，并减轻了服务器的负担，但它也存在一些弊端和限制，包括：

- 数量限制：每个特定域名下的cookie数量有限。例如，旧版的IE6最多允许20个cookie，而IE7及更高版本允许50个cookie，其他浏览器也有类似的限制。
- 大小限制：每个cookie的大小也有限制，通常为约4096字节（不同浏览器可能有差异），为了兼容性，一般建议将cookie大小控制在4095字节以内。
- 清理策略：一些浏览器会根据策略清理过期或不常使用的cookie，这可能会导致某些数据丢失或需要重新设置。
- 安全性问题：cookie存储在客户端，如果被恶意拦截，攻击者可以获取其中的数据，包括session信息，可能导致安全隐患。
- 跨域限制：cookie在同源策略下工作，无法跨域访问。每个域名下的cookie只能被同域名的页面访问和修改。
- 对网络性能的影响：cookie会增加每个请求的数据量，从而增加了网络传输的开销，尤其在请求大量静态资源的网页时，会对加载速度产生一定的影响。

要解决这些问题，可以使用其他存储方式，如localStorage或sessionStorage，使用服务器端存储来替代部分或全部cookie，或者通过其他技术手段来优化和管理cookie的使用。

1) 登录系统准备把完整用户资料序列化进一个cookie，单条数据接近4

KB，研发说浏览器能存就没有问题；你会接受这个设计吗？

答：不会，单个cookie通常只有约4096字节的容量，兼容设计还应留出属性和编码带来的余量。用户资料应主要保存在服务端，客户端只保留必要标识；继续膨胀可能导致写入失败、截断或不同浏览器表现不一致。

2) 电商首页有上百个静态资源请求，而主域名下积累了多个体积较大的cookie；性能排查发现请求头持续偏大，你会如何落地优化？

答：应先删减无用字段、缩短必要值，并避免让静态资源请求携带业务cookie。可将静态资源放到不需要这些凭据的域名或路径策略下，同时把大块状态迁到服务端或适合的客户端存储；迁移前要确认数据是否必须随请求发送。

3) 产品要求购物车状态永不丢失，计划只写长期cookie；但浏览器可能按策略清理过期或长期不用的数据，你会怎样调整承诺和实现？

答：不能把客户端cookie当作永久可靠的数据源，因为它可能被用户或浏览器策略清理。已登录用户的购物车应以服务端存储为准，客户端仅用于标识或临时恢复；匿名场景仍需接受数据可能丢失，并向产品明确这一边界。

4) 线上出现少量用户会话被冒用，排查发现会话标识直接保存在客户端cookie中；仅把值改成加密文本，能否解决风险？

答：仅加密不能消除被窃取后重放会话的风险，因为攻击者未必需要读懂标识。应避免存放敏感明文，采用安全传输并缩短会话有效期，同时在服务端支持失效和异常校验；客户端凭据一旦泄露，仍需按会话泄露处理。

5) 两个独立业务域名要共享登录态，架构师提议前端直接读取彼此的cookie；你会怎样判断并给出替代边界？

答：前端不能直接跨域读取和修改另一个域名的cookie，这是其作用域限制。共享登录应通过统一认证服务和服务端交换结果设计，而不是扩大客户端数据暴露；若只是同一域名下的子路径或子域，还需按实际作用域谨慎配置。

41. git fetch和git pull的区别

git fetch只下载远程更新，不改当前分支；git pull会下载并直接合并到当前分支。本质上，git pull相当于连续执行git fetch和git merge，所以用起来更省步骤，但也可能马上进入冲突处理。想先比较本地与远程的差异时，我一般用git

fetch；确认更新可以安全合入后，再手动决定如何合并。git pull：执行git pull命令时，Git会自动从远程仓库下载最新的提交并将其合并到当前分支。它是git fetch和git merge两个操作的组合。它会自动将远程仓库的更新合并到当前分支，并自动解决可能的冲突。一般情况下，使用git pull可以快速获取远程最新代码并合并到本地分支。git fetch：执行git fetch命令时，Git会从远程仓库下载最新的提交，但不会自动将其合并到当前分支。它只是将远程仓库的最新代码下载到本地，并更新本地仓库中远程分支的指针位置。这样，你可以在本地查看远程仓库的更新情况，进行代码比较或其他操作。但它不会修改你当前所在的分支。总结：git pull是直接从远程仓库获取最新代码并合并到当前分支，而git fetch只是获取最新代码到本地，并不会自动合并。使用git pull可以更方便地获取最新代码并更新本地分支，而git fetch适合查看远程仓库的更新情况，进行代码比较或其他操作。

42. http2.0 做了哪些改进 http3.0 呢

HTTP/2重点优化一个连接内的传输效率，HTTP/3则改用基于UDP的QUIC继续解决传输层问题。HTTP/2通过二进制分帧和多路复用并行传输请求，再用HPACK压缩重复头部，也支持服务器推送。它仍基于TCP，丢包时可能出现队头阻塞。HTTP/3让不同流互不拖累，并通过连接ID支持网络切换时的连接迁移，拥塞控制也更灵活。HTTP/2 的特性包括：

1. 二进制分帧 HTTP/1.x：基于文本格式，报文以明文形式传输，解析过程复杂且容易出错，效率较低。HTTP/2：采用二进制分帧层，将所有传输的信息分割为更小的帧，并对这些帧进行二进制编码。帧是 HTTP/2 数据传输的最小单位，包括头帧和数据帧等。这种二进制分帧的方式使得协议的解析更加高效、准确，提高了数据传输的性能。
2. 多路复用 HTTP/1.x：同一时间一个连接只能处理一个请求，当有多个请求时，需要依次排队等待处理，容易出现“队头阻塞”问题，即一个请求阻塞会影响后续请求的处理。HTTP/2：通过多路复用机制，允许在一个连接上同时并行处理多个请求和响应。每个请求和响应被拆分成多个独立的帧，这些帧可以在连接上乱序发送和接收，然后在另一端根据帧的标识进行重新组装。这样可以充分利用网络带宽，提高连接的利用率，避免了“队头阻塞”问题。
3. 头部压缩 HTTP/1.x：请求和响应的头部信息通常以明文形式重复传输，包含了很多重复的字段，如User-Agent、Cookie等，会占用大量的带宽。HTTP/2：采用 HPACK 算法对头部信息进行压缩。HPACK 会维护一个静态表和一个动态表，对于重复出现的头部字段，只需要在表中存储一次，后续传输时只需传输对应的索引，从而大大减少了头部信息的传输量，降低了带宽消耗。
4. 服务器推送 HTTP/1.x：客户端需要明确请求服务器上的资源，服务器只能被动响应客户端的请求，无法主动向客户端推送资源。HTTP/2：支持服务器推送功能，服务器可以在客户端请求某个资源时，主动将客户端可能需要的其他资源一起推送给客户端。例如，当客户端请求一个 HTML 页面时，服务器可以同时推送该页面所需的 CSS、JavaScript 等资源，减少了客户端的请求次数，提高了页面的加载速度。

而 HTTP/3 则是基于 QUIC 协议的新一代 HTTP 协议。QUIC 是一个基于 UDP 的传输协议，具有以下特性：

1. 基于 QUIC 协议 HTTP/2：仍然基于 TCP 协议，TCP 协议在处理丢包重传、拥塞控制等方面存在一些固有的问题，可能会导致“队头阻塞”问题，影响数据传输的性能。HTTP/3：采用 QUIC（快速 UDP 互联网连接）协议作为传输层协议。QUIC 基于 UDP 实现，但在 UDP 的基础上增加了可靠传输、拥塞控制、加密等功能。QUIC 协议的连接建立速度更快，并且在丢包情况下不会影响其他流的传输，避免了 TCP 协议中的“队头阻塞”问题，进一步提高了数据传输的性能和可靠性。
2. 连接迁移 HTTP/2：基于 TCP 协议，TCP 连接依赖于源 IP 地址、目的 IP 地址、源端口和目的端口，当设备的网络环境发生变化（如从 Wi-Fi 切换到移动数据网络）时，TCP 连接会中断，需要重新建立连接，这可能会导致数据传输中断和延迟。HTTP/3：QUIC 协议通过连接 ID 来标识连接，而不是依赖于 IP 地址和端口。当设备的网络环境发生变化时，只要连接 ID 不变，QUIC 连接可以保持不变，实现无缝的连接迁移，提高了用户体验。
3. 更灵活的拥塞控制 HTTP/2：TCP 协议的拥塞控制算法是固定的，不同的网络环境可能需要不同的拥塞控制策略，TCP 难以快速适应网络变化。HTTP/3：QUIC 协议的拥塞控制更加灵活，可以根据不同的网络情况动态调整拥塞

控制策略，更好地适应各种复杂的网络环境，提高数据传输的效率。 总结：HTTP/2和HTTP/3都是在传输层进行的协议改进，HTTP/2在TCP上引入了二进制分帧传输、多路复用、头部压缩和服务器推送等特性，而HTTP/3则是基于UDP的QUIC协议，引入了连接迁移、无队头阻塞、自定义拥塞控制和前向安全和前向纠错等新特性。

1) 监控显示站点已启用HTTP/2，但一次大文件传输丢包后，同一连接里的多个接口仍一起变慢；同事说多路复用已经彻底消除了队头阻塞，你怎么纠正？

答：HTTP/2解决的是应用层请求按顺序占用连接的问题，但它仍运行在TCP上。底层字节流发生丢包时，后续数据仍可能等待重传，因此连接内多个流会受影响；要进一步隔离流之间的丢包影响，需要评估基于QUIC的HTTP/3。

2) 一个首屏会并发请求几十个接口和资源，团队仍按HTTP/1.x的经验为每个请求新建连接；迁移到HTTP/2后，你会怎样调整连接策略？

答：应优先复用同一连接，让不同请求和响应拆成帧后并行传输，并依据流标识在接收端重组。这样能提升连接利用率并减少应用层排队；若底层TCP丢包，连接级阻塞仍存在，不能把连接数简单压到一个就视为完成优化。

3) 某接口每次都携带大量重复的Cookie和User-Agent，后端准备删除必要请求头来省流量；如果链路使用HTTP/2，你会提出什么替代方案？

答：不应为了节省带宽删除协议和业务必需的头部，而应利用HTTP/2的HPACK压缩。静态表和动态表可让重复字段后续主要传输索引；若头部本身包含不断变化的大值，压缩收益会受限，仍要从业务上控制其体积。

4) 移动端从公司Wi-Fi切到蜂窝网络后，长连接频繁中断并重新建立；服务端计划升级协议解决，你会如何比较HTTP/2与HTTP/3？

答：HTTP/2基于依赖地址和端口的TCP连接，网络切换通常需要重连。HTTP/3的QUIC使用连接ID标识连接，具备连接迁移能力，更适合该场景；实际效果仍取决于客户端、服务端和网络链路是否完整支持。

5) 安全评审质疑HTTP/3基于UDP，认为它既不可靠也没有拥塞控制；你会如何回应这个选型冲突？

答：HTTP/3并非直接把应用数据裸放在UDP上，而是由QUIC补充可靠传输、拥塞控制和加密等能力。它还能让不同流在丢包时减少相互阻塞，并更灵活地调整拥塞策略；代价是部署和链路兼容性需要单独验证。

【CSS相关】

43. css sprite是什么,有什么优缺点

CSS Sprite就是把多张小图合成一张大图，再用background-position和尺寸显示其中需要的区域。这样能把多个图片请求合成一次，减少网络传输的次数和延迟，合并后的图片也可以统一压缩。它适合图标或背景图案较多、需要集中管理资源的页面。代价是拼图本身有工作量，修改其中一块时可能要重新排版大图，并同步调整对应的CSS。 CSS Sprite（CSS精灵）是一种将多个小图片合并到一张大图中的技术。通过在页面中引用这张大图，并设置合适的background-position和尺寸，可以显示出所需的小图标或背景图案。

。 优点： 减少HTTP请求数：将多个小图片合并成一张大图，减少了浏览器与服务器之间的请求次数，提高

了页面加载速度。提高性能：由于减少了请求数，减少了网络传输时间和延迟，加快了页面加载速度，提升了用户体验。减小图片大小：合并后的大图可以使用更高效的压缩算法进行压缩，减小了图片的文件大小。方便更换风格：只需要替换或修改一张大图中的小图标或背景图案，就可以改变整个页面的样式，维护和更换风格更加方便。缺点：图片合并麻烦：合并图片需要手动调整和拼接小图标或背景图案，需要一定的工作量。维护麻烦：如果需要修改其中一个小图标或背景图案，可能需要重新布局整个大图，并且需要更新相应的CSS样式。总结：CSS Sprite通过将多个小图片合并成一张大图，减少了HTTP请求，提高了页面加载速度和性能。它的优点包括减少请求数、提高性能、减小图片大小和方便更换风格。然而，它的缺点在于图片合并和维护的麻烦。

44. display: none;与visibility: hidden;的区别

display: none会让元素退出渲染树且不占空间，visibility: hidden只是隐藏内容，原来的位置还在。前者通常会引起文档重排，而且子孙元素无法单独恢复显示；后者一般只触发重绘，子孙元素可设置visibility: visible重新显示。display不是继承属性，而visibility会继承。读屏器不会读取display: none的内容，但会读取visibility: hidden的内容，因此选择时还要考虑页面布局和辅助访问需求。display: none和visibility: hidden都可以使元素不可见，但它们在实现上有一些区别。区别：display: none会使元素完全从渲染树中消失，不占据任何空间，而visibility: hidden不会使元素从渲染树中消失，仍然占据空间，只是内容不可见。display: none是非继承属性，子孙节点消失是因为元素本身从渲染树中消失，修改子孙节点的属性无法使其显示。而visibility: hidden是继承属性，子孙节点消失是因为继承了hidden属性，通过设置visibility: visible可以使子孙节点显示。修改具有常规流的元素的display属性通常会导致文档重排（重新计算元素的位置和大小）。而修改visibility属性只会导致本元素的重绘（重新绘制元素的可见部分）。读屏器（屏幕阅读软件）不会读取display: none元素的内容，但会读取visibility: hidden元素的内容。综上所述，display: none和visibility: hidden虽然都可以使元素不可见，但在元素在渲染树中的位置、对子孙节点的影响、性能方面有所不同。选择使用哪种方式取决于具体的需求和场景。

45. link与@import的区别

是HTML标签，@import是写在样式表规则之前的CSS语法。浏览器解析页面时可以并行下载多个样式表，而@import要等所属样式表加载后再按顺序处理，更容易拖慢样式生效。还支持rel="alternate stylesheet"等属性，兼容性和使用方式也更灵活。实际项目里我一般用引入外部样式，避免运行时依赖@import。是HTML方式，@import是CSS方式。标签在HTML文档的部分中使用，用于引入外部CSS文件；@import是在CSS文件中使用，用于引入其他CSS文件。标签最大限度地支持并行下载，浏览器会同时下载多个外部CSS文件；而@import引入的CSS文件会导致串行下载，浏览器会按照顺序逐个下载CSS文件，这可能导致页面加载速度变慢，出现FOUC（Flash of Unstyled Content）问题。标签可以通过rel="alternate stylesheet"指定候选样式表，用户可以在浏览器中切换样式；而@import不支持rel属性，无法提供候选样式表功能。浏览器对标签的支持早于@import，一些古老的浏览器可能不支持@import方式引入CSS文件，而可以正确解析标签。@import必须出现在样式规则之前，而且只能在CSS文件的顶部引用其他文件；而标签可以放置在文档的任何位置。总体来说，标签在性能、兼容性和灵活性方面优于@import。因此，在实际使用中，推荐使用标签来引入外部CSS文件。

46. 什么是FOUC?如何避免

FOUC是页面先显示无样式内容，等外部CSS生效后又突然切换样式的闪烁现象。本质上是内容已经开始渲染，但样式表下载或解析得更晚，所以应把放进，让浏览器尽早处理。首屏关键样式可以内联，也可以预加载外部样式，并通过减少文件数量和体积缩短等待时间。媒体查询则适合按条件加载样式，避免下载当前场景用不到的内容。FOUC（Flash Of Unstyled Content）指的是在页面加载过程中，由于外部样式表

(CSS) 加载较慢或延迟，导致页面先以无样式的方式显示，然后突然闪烁出样式的现象。

为了避免FOUC，可以采取以下方法：

- 将样式表放置在文档的标签中：通过将样式表放在文档头部，确保浏览器在渲染页面内容之前先加载和解析样式表，从而避免了页面一开始的无样式状态。
- 使用内联样式：将关键的样式直接写在HTML标签的style属性中，这样即使外部样式表加载延迟，页面仍然可以有基本的样式展示，避免出现完全无样式的情况。
- 使用样式预加载：在HTML的中使用标签，将样式表提前预加载，以确保在页面渲染之前样式表已经下载完毕。
- 避免过多的样式表和样式文件：减少页面中使用的样式表数量和样式文件大小，优化样式表的结构和规则，从而加快样式表的加载速度。
- 使用媒体查询避免不必要的样式加载：通过媒体查询(@media)在适当的条件下加载特定的样式，避免在不需要的情况下加载不必要的样式。

综上所述，通过优化样式加载顺序、使用内联样式、样式预加载和合理使用媒体查询等方法，可以有效避免FOUC的出现，提供更好的用户体验。

47. 如何创建块级格式化上下文(block formatting context),BFC有什么用

BFC是相对独立的块级布局区域，常用来包含浮动、避开浮动元素，以及控制外边距折叠。常见触发方式包括float非none、overflow非visible、position: absolute/fixed，以及display: flow-root。父元素创建BFC后会把浮动子元素计入高度，兄弟元素创建BFC后也不会覆盖浮动区域。实际开发更推荐display:

flow-root，因为overflow: hidden还可能裁剪溢出内容。 BFC(Block Formatting Context)，块级格式化上下文，是一个独立的渲染区域，让处于

内部的元素与外部的元素相互隔离，使内外元素的定位不会相互影响

要创建一个块级格式化上下文（BFC），可以应用以下方法：

- 使用float属性：将元素的float属性设置为除none以外的值，可以创建一个BFC。
- 使用overflow属性：将元素的overflow属性设置为除visible以外的值，例如auto或hidden，可以创建一个BFC。
- 使用display属性：将元素的display属性设置为inline-block、table-cell、table-caption等特定的值，可以创建一个BFC。
- 使用position属性：将元素的position属性设置为absolute、fixed、relative或sticky，可以创建一个BFC。
- 使用contain属性：将元素的contain属性设置为layout，可以创建一个BFC（仅适用于部分浏览器）。

在IE下,Layout,可通过zoom:1触发

BFC布局与普通文档流布局区别 普通文档流布局: 浮动的元素是不会被父级计算高度非浮动元素会覆盖浮动元素的位置margin会传递给父级元素两个相邻元素上下的margin会重叠 BFC布局规则: 浮动的元素会被父级计算高度(父级元素触发了BFC)非浮动元素不会覆盖浮动元素的位置(非浮动元素触发了BFC)margin不会传递给父级(父级触发BFC)属于同一个BFC的两个相邻元素上下margin会重叠 开发中的应用 阻止margin重叠可以包含浮动元素 —— 清除内部浮动(清除浮动的原理是两个div都位于同一个BFC区域之中)自适应两栏布局可以阻止元素被浮动元素覆盖

48. display、float、position的关系

position、float会影响display的最终计算结果，其中绝对定位的处理优先于浮动。当display: none时元素不生成盒子，另外两个属性自然不起作用；当position为absolute或fixed时，float会计算为none，盒子同时被块级化。没有绝对定位但float非none时，盒子同样会块级化，例如inline会按block处理。根元素也会调整display，其余情况下才按声明值正常生效。如果display取值为none，那么position和float都不起作用，这种情况下元素不产生框否则，如果position取值为absolute或者fixed，框就是绝对定位的，float的计算值为none，display根据下面的表格进行调整。否则，如果float不是none，框是浮动的，display根据下表进行调整否则，如果元素是根元素，display根据下表进行调整其他情况下display的值为指定值总结起来：绝对定位、浮动、根元素都需要调整display 综上所述，display、float和position之间存在一定的关系，它们的取值会相互影响元素的布局和显示方式。根据不同的取值组合，元素的display值可能会被调整。

49. 清除浮动的几种方式，各自的优缺点

清除浮动常用伪元素、父级设置`overflow:hidden`，也可以追加带`clear:both`的空元素。伪元素或`clearfix`不增加多余标签，清除效果较好，但兼容老浏览器时要配合`zoom:1`。`overflow:hidden`写法简单，却可能把溢出内容裁掉；空`div`或`br`容易理解，但会污染结构。父级定高或一起浮动也能解决，不过前者要求高度已知，后者还要定义宽度，灵活性较差。以下是清除浮动的几种常见方式以及它们的优缺点：

- 父级`div`定义`height`：将父级容器的高度设置为已浮动元素的高度。优点是简单易实现，缺点是需要提前知道浮动元素的高度，如果高度发生变化，需要手动调整。
- 结尾处加空`div`标签`clear:both`：在浮动元素后面添加一个空的`div`标签，并设置`clear:both`。优点是简单易实现，缺点是需要添加多余的空标签，不符合语义化。
- 父级`div`定义伪类`:after`和`zoom`：父级容器使用伪元素`:after`清除浮动，并设置`zoom:1`触发`hasLayout`。优点是不需要额外添加多余的标签，清除浮动效果好，缺点是对老版本浏览器的兼容性需要考虑。
- 父级`div`定义`overflow:hidden`：将父级容器的`overflow`属性设置为`hidden`。优点是简单易实现，不需要添加额外的标签，缺点是可能会造成内容溢出隐藏。
- 父级`div`也浮动，需要定义宽度：将父级容器也设置为浮动，并定义宽度。优点是清除浮动效果好，缺点是需要定义宽度，不够灵活。
- 结尾处加`br`标签`clear:both`：在浮动元素后面添加`br`标签，并设置`clear:both`。和第2种方式类似，优缺点也相似。

使用`clearfix`类：在父级容器上应用`clearfix`类，该类包含伪元素清除浮动。优点是代码简洁易懂，不需要额外添加标签，缺点是需要定义并引用`clearfix`类。总体而言，使用伪类`:after`和`zoom`的方式是较为常见和推荐的清除浮动的方法，它可以避免添加多余的标签，并具有良好的兼容性。然而，不同场景下适合使用不同的清除浮动方式，需要根据实际情况选择合适的方法。

50. 为什么要初始化CSS样式？

初始化CSS是为了抹平浏览器默认样式的差异，让页面从统一、可控的基础样式开始。不同浏览器对元素的默认外观可能不同，不处理就容易出现显示偏差，也会增加后续覆盖和调整样式的成本。统一基础样式后，界面风格更容易保持一致，代码的可读性和维护性也会更好。实际使用时要控制初始化范围，可以选择现成的初始化库或自行定义，避免过度清理带来代码冗余。

初始化 CSS 样式的目的主要有以下几点：

- 浏览器兼容性：不同浏览器对于 HTML 元素的默认样式存在差异，通过初始化 CSS 样式，可以尽量消除不同浏览器之间的显示差异，使页面在各个浏览器中更加一致。
- 统一样式：通过初始化 CSS 样式，可以为各个元素提供一个统一的基础样式，避免默认样式的影响。这有助于开发者在项目中构建一致的界面风格，提高开发效率。
- 提高可维护性：初始化 CSS 样式可以避免在编写具体样式时受到浏览器默认样式的干扰，减少不必要的样式覆盖和调整，从而提高代码的可维护性和可读性。
- 优化性能：通过初始化 CSS 样式，可以避免不必要的样式计算和渲染，减少浏览器的工作量，提升页面加载和渲染性能。

需要注意的是，在进行 CSS 样式初始化时，应该注意选择合适的方式和范围，避免过度初始化造成不必要的代码冗余和性能损耗。同时，针对具体项目和需求，可以选择使用已有的 CSS 初始化库或者自定义初始化样式。

51. css3有哪些新特性

CSS3的新特性可以归为选择器与响应式布局、视觉效果、变换和动画这几类。选择器增加了`:nth-child()`、`:not()`等伪类，配合`@media`、`flex`和多列布局，可以更灵活地选择元素并适配不同屏幕。视觉层面提供了`border-radius`、渐变、阴影和`rgba()`，很多效果不再依赖图片。交互效果则主要靠`transform`、`transition`、`@keyframes`和`animation`，分别处理变换、过渡与完整动画。

CSS3引入了许多新特性，以下是其中一些常见的特性：

- 新增选择器：例如`:nth-child()`、`:first-of-type`、`:last-of-type`等，可以根据元素在父元素中的位置进行选择。
- 弹性盒模型：通过`display: flex`可以创建弹性布局，简化了元素的排列和对齐方式。
- 多列布局：使用`column-count`和`column-width`等属性可以实现将内容分为多列显示。
- 媒体查询：通过`@media`可以根据设备的特性和屏幕大小应用不同的样式规则。
- 个性化字体：使用`@font-face`可以引入自定义字体，并在网页中使用。
- 颜色透明度：通过`rgba()`可以设置颜色的透明度。
- 圆角：使用`border-radius`可以给元素添加圆角效果。
- 渐变：使用`linear-gradient()`可以创建线

性渐变背景效果。阴影：使用box-shadow可以为元素添加阴影效果。倒影：使用box-reflect可以为元素添加倒影效果。文字装饰：使用text-stroke-color可以设置文字描边的颜色。文字溢出：使用text-overflow可以处理文字溢出的情况。背景效果：使用background-size可以控制背景图片的大小。边框效果：使用border-image可以为边框使用图片来创建特殊效果。转换：使用transform可以实现元素的旋转、倾斜、位移和缩放等变换效果。平滑过渡：使用transition可以为元素的属性变化添加过渡效果。动画：通过@keyframes和animation可以创建元素的动画效果。CSS3引入了许多新的伪类，以下是一些常见的新增伪类：
:nth-child(n)：选择父元素下的第n个子元素。:first-child：选择父元素下的第一个子元素。:last-child：选择父元素下的最后一个子元素。:nth-of-type(n)：选择父元素下特定类型的第n个子元素。:first-of-type：选择父元素下特定类型的第一个子元素。:last-of-type：选择父元素下特定类型的最后一个子元素。:only-child：选择父元素下仅有的一个子元素。:only-of-type：选择父元素下特定类型的唯一一个子元素。:empty：选择没有任何子元素或者文本内容的元素。:target：选择当前活动的目标元素。:enabled：选择可用的表单元素。:disabled：选择禁用的表单元素。:checked：选择被选中的单选框或复选框。:focus：选择当前获取焦点的元素。:hover：选择鼠标悬停在上方的元素。:visited：选择已访问过的链接。:not(selector)：选择不符合给定选择器的元素。这些新增的伪类为选择元素提供了更多的灵活性和精确性，使得开发者能够更好地控制和样式化文档中的元素。

52. display有哪些值？说明他们的作用

display常见值有block、inline、inline-block、none、list-item、table和inherit，用于决定元素生成什么类型的盒子。block独占一行并可设置宽高，inline随内容排列且不能按块级方式设置宽高，inline-block则兼顾两者特点。none会让元素完全不参与渲染，也不占空间；list-item和table分别按列表项、块级表格显示。需要沿用父元素的框类型时可以使用inherit，具体选择取决于页面的布局需求。

display属性用于定义元素应该生成的框类型。以下是常见的display属性值及其作用：
block：将元素转换为块状元素，独占一行，可设置宽度、高度、边距等属性。
inline：将元素转换为行内元素，不独占一行，只占据内容所需的空间，无法设置宽度、高度等块级属性。
none：设置元素不可见，在渲染时将其完全隐藏，不占据任何空间。
inline-block：使元素既具有行内元素的特性（不独占一行），又具有块级元素的特性（可设置宽度、高度等属性），可以看作是行内块状元素。
list-item：将元素作为列表项显示，常用于有序列表（`<ol>`）和无序列表（`<ul>`）中，会添加列表标记。
table：将元素作为块级表格显示，常用于构建表格布局，类似于

53. 介绍一下标准的CSS的盒子模型？低版本IE的盒子模型有什么不同的？

盒模型从内到外由content、padding、border和margin组成，标准盒模型与低版本IE怪异盒模型的区别在于宽高计算范围。标准的content-box中，声明的width只表示内容区，元素总宽度还要加上内边距、边框和外边距。怪异模型的width已包含内容、内边距和边框，总占位只需再算外边距。现代浏览器可用box-sizing：
border-box获得这种计算方式，我一般在响应式布局或网格系统中使用它，让尺寸更容易控制。有两种，IE盒子模型、W3C盒子模型；
盒模型：内容(content)、填充(padding)、边界(margin)、边框(border)；区别：
IE的content部分把border和padding`计算了进去；
盒子模型构成：内容(content)、内填充(padding)、边框(border)、外边距(margin)
IE8及其以下版本浏览器，未声明DOCTYPE，内容宽高会包含内填充和边框，称为怪异盒模型(IE盒模型)
标准(W3C)盒模型：
元素宽度 = width + padding + border + margin
怪异(IE)盒模型：元素宽度 = width + margin
标准浏览器通过设置css3的box-sizing: border-box属性，触发“怪异模式”解析计算宽高
box-sizing常用的属性有哪些？分别有什么作用
box-sizing属性用于控制元素的盒模型类型，常用的属性值有：
content-box：默认值，使用标准的W3C盒模型，元素的宽度和高度仅包括内容区域（content），不包括填充、边框和外边距。
border-box：使

用怪异的IE盒模型，元素的宽度和高度包括内容区域（content）、填充（padding）和边框（border），但不包括外边距（margin）。即元素的宽度和高度指定的是内容区域加上填充和边框的总宽度和高度。inherit：继承父元素的box-sizing属性值。通过设置不同的box-sizing属性值，可以控制元素的盒模型类型，进而影响元素的布局和尺寸计算。使用border-box可以更方便地处理元素的宽度和高度，特别适合响应式布局和网格系统的设计。

54. CSS优先级算法如何计算？

CSS优先级通常按important、内联样式、ID选择器、类或属性及伪类选择器、元素及伪元素选择器的顺序比较。同一层级中，选择器权重更高的规则会生效；如果权重相同，则后出现的声明覆盖前面的声明。可继承属性还可能从父元素传给子元素，但继承值不会直接胜过命中当前元素的规则。工程中应谨慎使用important，因为它虽然能快速覆盖样式，却会提高后续维护和再次覆盖的难度。CSS优先级是用于确定当多个样式规则应用到同一个元素时，哪个样式规则会被应用的一种规则。优先级的计算基于选择器的权重。以下是CSS优先级计算的一般规则：
important：样式规则使用了important标记，具有最高优先级，无论其位置在哪里。
内联样式：直接应用在元素上的style属性具有较高的优先级。
ID选择器：使用ID选择器的样式规则具有较高的优先级。例如，#myElement。
类选择器、属性选择器和伪类选择器：使用类选择器（例如.myClass）、属性选择器（例如[type="text"]）和伪类选择器（例如:hover）的样式规则的优先级低于ID选择器。
元素选择器和伪元素选择器：使用元素选择器（例如div）和伪元素选择器（例如::before）的样式规则的优先级低于类选择器、属性选择器和伪类选择器。

当存在多个样式规则具有相同的优先级时，会根据以下规则进行决定：
就近原则：当同一元素上存在多个具有相同优先级的样式规则时，最后出现的样式规则将被应用。
继承：某些样式属性可以被子元素继承，如果父元素具有样式规则，子元素将继承该样式。
需要注意的是，以上规则仅适用于一般情况，有些情况下可能存在更复杂的优先级计算。同时，使用important应该谨慎，过度使用important可能导致样式管理困难和维护问题。

55. 对BFC规范的理解？

BFC是一个独立的块级格式化上下文，用来约束内部盒子的布局，并隔离它们对外部布局的影响。它可以由非none的float、absolute或fixed定位、部分特殊display值，以及特定的overflow设置触发。BFC不会与浮动元素重叠，也能把内部浮动元素计入自身高度；其边界还可阻止内外边距合并，不过内部相邻块的垂直外边距仍可能合并。实际布局中，它常用于处理父元素高度塌陷、自适应两栏布局或边距重叠问题。
一个页面是由很多个Box组成的,元素的类型和display属性,决定了这个Box的类型不同类型的Box,会参与不同的Formatting Context（决定如何渲染文档的容器）,因此Box内的元素会以不同的方式渲染,也就是说BFC内部的元素和外部的元素不会互相影响
BFC（Block Formatting Context）是CSS中的一种渲染规范，用于决定和控制元素在文档中的布局和渲染方式。BFC定义了一个独立的渲染区域，使得处于不同BFC内部的元素相互隔离，互不影响。
以下是对BFC规范的一些理解：
BFC的创建条件：触发BFC的条件包括元素的float属性不为none、position属性为absolute或fixed、display属性为inline-block、table-cell、table-caption等，以及通过特定的CSS属性（如overflow）进行触发。
BFC的特性：内部的块级盒子会在垂直方向上一个接一个地放置。相邻的两个块级盒子的垂直外边距会发生合并。BFC的区域不会与浮动元素重叠。BFC在页面布局时会考虑浮动元素。BFC可以包含浮动元素，并计算其高度。BFC的边界会阻止边距重叠。
BFC的应用：
清除浮动：创建一个父级元素成为BFC，可以清除其内部浮动的影响，避免父元素塌陷。
创建自适应的两栏布局：通过将两个列容器设置为BFC，可以避免它们相互影响。
阻止边距重叠：当两个相邻元素的边距发生重叠时，将其中一个元素设置为BFC，可以解决边距重叠问题。
内部的块级盒子会在垂直方向上一个接一个地放置。相邻的两个块级盒子的垂直外边距会发生合并。BFC的区域不会与浮动元素重叠。BFC在页面布局时会考虑浮动元素。BFC可以包含浮动元素，并计算其高度。BFC的边界会阻止边距重叠。
清除浮动：创建一个父级元素成为BFC，可以清除其内部浮动的影响，避免父元素塌陷。创建自适应的两栏

布局：通过将两个列容器设置为BFC，可以避免它们相互影响。阻止边距重叠：当两个相邻元素的边距发生重叠时，将其中一个元素设置为BFC，可以解决边距重叠问题。总的来说，BFC规范通过创建独立的渲染上下文，使得元素的布局和渲染更加可控，避免了一些常见的布局问题和冲突。它在清除浮动、解决边距重叠等方面具有重要的应用价值。

56. 谈谈浮动和清除浮动

浮动通过`float: left`或`float: right`让元素脱离正常文档流并靠向包含块一侧，周围内容会围绕它排布。因为浮动元素不再占据常规流空间，父元素可能无法获得其高度，后续元素也可能出现重叠等布局问题。清除影响时，可以在后面放置设置`clear: both`的块元素，也可以让父元素通过`overflow: auto`、`overflow: hidden`触发BFC。工程中更常用`::after`配合`clearfix`，既能包含浮动，又不必额外添加无语义的空元素。浮动（float）是CSS中的一种布局方式，它允许元素向左或向右浮动并脱离文档的正常流，其他元素会围绕浮动元素进行布局。浮动的特点和应用：元素浮动后，其原位置会被其他元素填充，不再占据文档流中的空间。浮动元素会尽可能地靠近其包含块的左侧或右侧，直到遇到另一个浮动元素或包含块的边界。浮动元素可以通过设置`float`属性为`left`或`right`进行左浮动或右浮动。常见应用包括实现多列布局、文字环绕图片等。清除浮动（clear float）是为了解决浮动元素带来的影响和布局问题而采取的措施。浮动元素会导致其父元素的高度塌陷（父元素无法检测到浮动元素的高度），以及其他元素可能与浮动元素重叠。为了解决这些问题，可以使用清除浮动的方法：空元素清除浮动：在浮动元素后面添加一个空的块级元素，并设置其`clear`属性为`both`，使其在浮动元素下方换行，达到清除浮动的效果。父级元素使用`overflow`属性：给包含浮动元素的父元素设置`overflow`属性为`auto`或`hidden`，可以触发BFC（块格式化上下文），从而包含浮动元素。使用伪元素清除浮动：使用`::after`伪元素给包含浮动元素的父元素添加一个清除浮动的样式，例如设置`content`为空字符串、`display`为`table`等。使用`clearfix`类：给包含浮动元素的父元素添加一个`clearfix`类，该类定义了清除浮动的样式，例如设置`clearfix`类的`::after`伪元素清除浮动。需要注意的是，清除浮动的方法应当适用于具体的布局需求和兼容性考虑。同时，清除浮动可能会影响到其他样式的布局，因此需要综合考虑和测试。

57. position的值，relative和absolute定位原点是

`position`常用值有`static`、`relative`、`absolute`、`fixed`和`sticky`，其中`relative`相对自身原位置定位，`absolute`相对最近的已定位祖先定位。`relative`偏移后仍保留原来的空间，不会脱离文档流。`absolute`会脱离文档流；找不到`position`非`static`的祖先时，就以初始包含块为基准。`fixed`通常相对视口，而`sticky`需要配合`top`或`bottom`等阈值使用。`position`属性用于控制元素的定位方式，常用的取值包括：`static`：默认值，表示元素在文档流中正常定位，不会受到`top`、`right`、`bottom`、`left`属性的影响。`relative`：生成相对定位的元素，相对于其正常位置进行定位，通过设置`top`、`right`、`bottom`、`left`属性来调整元素的位置，不会脱离文档流，周围的元素仍然会按照正常布局进行排列。`absolute`：生成绝对定位的元素，相对于最近的非`static`定位的父元素进行定位，如果没有非`static`定位的父元素，则相对于文档根元素（即浏览器窗口）进行定位。绝对定位的元素会脱离文档流，不占据空间，可以通过设置`top`、`right`、`bottom`、`left`属性来精确控制元素的位置。`fixed`：生成绝对定位的元素，相对于浏览器窗口进行定位，不会随着页面的滚动而改变位置。可以通过设置`top`、`right`、`bottom`、`left`属性来指定元素的位置。`inherit`：规定从父元素继承`position`属性的值。对于`relative`和`absolute`定位，其原点（坐标基准点）是元素在正常文档流中的位置。通过调整`top`、`right`、`bottom`、`left`属性，可以相对于原点在水平和垂直方向上进行偏移，实现元素的精确定位。

58. display:inline-block 什么时候不会显示间隙？

相邻的`inline-block`标签之间没有空白字符时，通常就不会显示那段间隙。本质上是源码里的空格或换行被当作行内空白渲染，并不是元素自动生成了外边距。工程中可以删除标签间空白，或者把父元素设为`font-size: 0`，再给子元素恢复字号。负`margin`、`letter-spacing`或`word-spacing`也能抵消，但数值依赖字体和布

局，维护起来更脆弱。display: inline-block元素在默认情况下会产生间隙，这是因为它们被视为行内元素，会保留默认的行框高度和基线对齐。然而，可以采取一些方法来消除这些间隙，使元素紧密排列，例如在携程网站中的布局。以下是一些消除间隙的常见方法：移除空格：在HTML代码中，将inline-block元素之间的空格删除，以消除间隙。使用负值margin：通过设置负值的左右外边距（margin）来抵消间隙。例如，可以使用margin-right: -4px;来消除间隙。使用font-size: 0;：将inline-block元素的父元素的字体大小设置为0，然后在inline-block元素上重新设置所需的字体大小。这样可以消除间隙，因为元素内部没有文字导致的间隙。使用letter-spacing：在inline-block元素的父元素上设置负值的letter-spacing，例如letter-spacing: -4px;，可以消除间隙。使用word-spacing：在inline-block元素的父元素上设置负值的word-spacing，例如word-spacing: -4px;，可以消除间隙。这些方法都是通过调整元素的布局或字体属性来实现消除间隙的效果。具体的方法选择取决于实际需求和布局要求。

59. PNG\GIF\JPG的区别及如何选

照片和复杂图像通常选JPG，需要透明背景时选PNG，需要简单动画时可以选GIF。JPG支持丰富颜色并采用有损压缩，可以在画质和体积之间取舍，但不支持透明。PNG使用无损压缩，PNG-8最多支持256色，真彩色PNG支持完整的alpha透明，不过文件通常更大。GIF同样最多支持256色，支持简单循环动画和有限透明，更适合颜色较少的动态图。PNG、GIF、和JPG是常见的图像文件格式，它们在以下方面有所区别：GIF (Graphics Interchange Format)使用8位像素，最多支持256种颜色。采用无损压缩算法，不会损失图像质量。支持简单的动画功能，可以创建循环播放的图像。支持二进制透明和索引透明，可以实现简单的透明效果。适用于图标、简单的动画和带有透明背景的图像。JPEG (Joint Photographic Experts Group)支持高达16.7百万种颜色，适合存储照片和复杂图像。使用有损压缩算法，可以调整压缩质量以平衡图像质量和文件大小。不支持透明效果，背景会被默认填充为白色。适合摄影、艺术作品等需要保留高质量细节的图像。PNG (Portable Network Graphics)有两种类型：PNG-8和真彩色PNG。PNG-8类似于GIF，支持最多256种颜色，文件较小，可以实现透明效果。真彩色PNG支持高分辨率的真彩色图像，文件较大，支持完全的alpha透明度。不支持动画功能。适合图标、背景、按钮等需要透明度的图像。GIF (Graphics Interchange Format)使用8位像素，最多支持256种颜色。采用无损压缩算法，不会损失图像质量。支持简单的动画功能，可以创建循环播放的图像。支持二进制透明和索引透明，可以实现简单的透明效果。适用于图标、简单的动画和带有透明背景的图像。JPEG (Joint Photographic Experts Group)支持高达16.7百万种颜色，适合存储照片和复杂图像。使用有损压缩算法，可以调整压缩质量以平衡图像质量和文件大小。不支持透明效果，背景会被默认填充为白色。适合摄影、艺术作品等需要保留高质量细节的图像。PNG (Portable Network Graphics)有两种类型：PNG-8和真彩色PNG。PNG-8类似于GIF，支持最多256种颜色，文件较小，可以实现透明效果。真彩色PNG支持高分辨率的真彩色图像，文件较大，支持完全的alpha透明度。不支持动画功能。适合图标、背景、按钮等需要透明度的图像。选择使用哪种图像格式取决于图像的特点和用途：如果需要动画效果，可以选择GIF格式。如果是照片或复杂图像，需要高质量和丰富的颜色，可以选择JPG格式。如果需要透明背景或简单的透明效果，可以选择PNG格式，根据图像的复杂性选择PNG-8或真彩色PNG。

60. 行内元素float:left后是否变为块级元素？

行内元素设置float: left后，不是简单改成普通块级元素，而是会表现出类似inline-block的尺寸特性。它会脱离正常文档流并向左浮动，此时可以设置width、height以及垂直方向的padding。它的宽度默认仍按内容计算，不会像普通块元素那样自动占满父容器，同时周围内容还可能产生环绕效果。需要铺满父容器时，可以再明确设置width: 100%。当行内元素设置了float: left;后，并非直接变为块级元素，而是表现出类似行内块级元素 (inline-block) 的特性。行内元素设置了float: left;后会产生以下效果：行内元素会脱离文档流，并根据设置的浮动方向向左浮动。

其宽度不再受到文本内容的限制，而是根据内容的宽度来确定。可以设置padding-top、padding-bottom、width、height等属性，并产生相应的效果。相邻的行内元素会环绕在其周围，形成类似于文本环绕的效果。需要注意的是，设置了浮动的行内元素不会自动填充父元素的宽度，而是根据内容的宽度进行布局。如果希望行内元素具有块级元素的宽度特性，可以设置width: 100%; 总结：行内元素设置了float: left;后，它的表现类似于行内块级元素，但仍然属于行内元素的性质，只是在布局和尺寸上有所改变。

61. 在网页中的应该使用奇数还是偶数的字体？为什么呢？

网页中通常更建议使用偶数字号，例如12px、14px或16px。偶数字号更容易保持整数像素对齐，减少半像素渲染可能带来的文字模糊。它也方便和标题、段落及间距建立统一的比例，让页面看起来更平衡，并减少不同浏览器间的细微显示差异。不过这不是硬性规定，如果奇数字号更符合整体设计和视觉效果，也可以使用。在网页中，通常建议使用偶数字号的字体，即字号为偶数（如12px、14px、16px等）。这是因为偶数字号相对更容易与网页设计的其他部分构成比例关系，具有更好的视觉平衡和一致性。

以下是一些原因和考虑因素：

- 整数像素对齐：偶数字号的字体大小通常是整数像素，而在网页渲染中，整数像素对齐可以提供更锐利和清晰的显示效果。当字号为奇数时，可能需要进行半像素渲染，这可能会导致字体显示模糊或模糊。
- 比例和对称：使用偶数字号的字体可以更容易与其他设计元素形成比例和对称。网页设计通常依赖于一致的比例和对称性，而使用偶数字号的字体可以更好地与网页中的其他元素（如标题、段落、间距等）形成和谐的视觉关系。
- 浏览器兼容性：一些浏览器对于奇数字号字体的渲染效果可能与偶数字号字体略有不同，可能会导致细微的差异。使用偶数字号字体可以减少在不同浏览器上的显示差异。

需要注意的是，这只是一些建议，并不意味着绝对规定。在实际设计中，根据具体情况和个人审美偏好，也可以使用奇数字号字体。最重要的是确保字体大小与整体设计风格和一致性相匹配。

62. ::before 和 :after中双冒号和单冒号 有什么区别？解释一下这2个伪元素的作用

单冒号通常表示伪类，双冒号表示伪元素，::before和::after分别在元素内容的前后生成内容。像:hover、:focus描述的是元素状态，而::first-line、::before处理的是元素的某个部分或生成内容。使用时可以通过content配合定位和样式，实现图标、角标或其他装饰效果。本质上是不用修改实际HTML结构，也能在指定位置补充视觉内容。在CSS中，单冒号(:)和双冒号(::)有不同的含义和用途。

单冒号(:)：单冒号用于表示伪类。伪类是用于选择元素的特定状态或动作的关键字，例如:hover、:active、:focus等。伪类表示元素的某种状态或行为，它们通常用于选择元素的特定状态并应用相应的样式。

双冒号(::)：双冒号用于表示伪元素。伪元素是用于在文档中生成或插入特定内容的关键字，例如::before、::after、::first-line、::first-letter等。伪元素可以创建或修改元素的一部分内容，它们允许开发人员在元素的特定位置添加样式或内容，而无需在文档中实际插入额外的HTML元素。

关于::before和::after伪元素的作用：

- ::before：::before伪元素用于在选中元素的内容前插入生成的内容。通过设置content属性和应用样式，可以在元素的开始位置插入额外的内容，这样就可以实现一些装饰性效果或添加额外的元素内容。
- ::after：::after伪元素用于在选中元素的内容后插入生成的内容。与::before类似，通过设置content属性和应用样式，可以在元素的结束位置插入额外的内容，从而实现装饰性效果或添加其他元素内容。

这两个伪元素可以通过CSS属性进行定位、设置样式、添加内容等，使开发人员可以在不修改实际HTML结构的情况下，实现一些额外的视觉效果或功能。

63. 如果需要手动写动画，你认为最小时间间隔是多久，为什么？（阿里）

手动编写动画时，通常把最小时间间隔设为约16.7ms。因为多数显示器默认刷新率是60Hz，也就是一秒刷新60次，每帧时间约为 $1000 / 60 = 16.7\text{ms}$ 。间隔再小，部分更新也可能赶不上显示器刷新，反而出现跳帧或不连续。实际设备和浏览器性能存在差异，所以还要结合目标环境测试和优化，不能只依赖这个理论值。

多数显示器默认频率是60Hz，即1秒刷新60次，所以理论上最小间隔为 $1/60 * 1000\text{ms} = 16.7\text{ms}$ 。如果需要手动编写动画，建议将最小时间间隔设置为16.7ms，即每帧动画的时间间隔。这是因为大多数显示器的默

刷新频率是60Hz，也就是每秒刷新60次。在这种情况下，将动画的时间间隔设置为16.7ms可以确保每帧动画都能够在显示器刷新之前完成。如果时间间隔小于16.7ms，则会导致某些帧在显示器刷新之后才能呈现，造成不连续的动画效果，也称为“跳帧”现象。因此，将时间间隔设置为16.7ms是一个相对较小的值，可以保证较平滑的动画效果，并且适应大多数显示器的刷新频率。需要注意的是，由于设备和浏览器的差异，实际的刷新频率和性能可能会有所不同。因此，在编写动画时，还应该进行实际测试和优化，确保动画在各种设备和浏览器上都能够获得良好的表现。

64. CSS合并方法

CSS可以通过预处理器、构建工具或服务器端配置合并，并在合并后进行压缩。我一般会让Sass、Less等预处理器组织多个文件，再交给Webpack、Gulp等工具自动产生一个文件，这样兼顾维护性和发布效率。服务器端合并也能减少请求数量，而压缩可以继续去掉空格、注释等无关内容。少量关键样式可以内联，但样式较多时会影响维护，因此要根据项目规模取舍。正确合并CSS的方法可以包括以下几点：

- 内联CSS：将CSS直接写在HTML文件的标签中或者通过style属性添加到具体元素中。这样可以减少HTTP请求并提高页面加载速度，但可维护性较差。
- CSS预处理器：使用CSS预处理器如Sass、Less或Stylus，它们提供了更灵活和可维护的方式来编写CSS，并且可以将多个CSS文件合并为一个。
- 合并工具：使用构建工具如Grunt、Gulp或Webpack，通过配置任务来合并CSS文件。这些工具提供了任务运行、文件合并、压缩等功能，可以自动化合并CSS，并在开发过程中或上线前进行处理。
- HTTP请求合并：如果使用多个CSS文件，可以通过服务器端配置将它们合并成一个文件，并通过单个HTTP请求加载。这样可以减少请求的数量，提高页面加载速度。
- 压缩：对合并后的CSS文件进行压缩，去除空格、注释等无关字符，以减小文件大小，提高加载速度。

需要根据具体的项目和需求选择适合的合并方法，以提高页面性能和开发效率。

65. CSS不同选择器的权重(CSS层叠的规则)

CSS选择器通常按!important、行内样式、ID、类或属性及伪类、元素选择器的顺序比较权重。常见计算方式是行内样式记作1000，每个ID记作100，类、属性或伪类记作10，元素记作1。组合选择器需要把各部分权重相加，例如#id1

div的权重是101。如果最终权重相同，就看声明顺序，后写的规则会覆盖先写的规则。

CSS选择器的权重规则可以总结如下：

- !important规则：具有最高的优先级，优先级为最大。
- 行内样式：通过style属性直接定义的样式具有较高的权重，优先级为1000。
- ID选择器：每个ID选择器的权重为100。
- 类选择器、属性选择器和伪类选择器：每个类选择器、属性选择器或伪类选择器的权重为10。
- 元素选择器：每个元素选择器的权重为1。

当应用多个选择器到同一个元素时，根据上述规则计算各个选择器的权重，具有较高权重的样式将被应用。如果存在权重相同的情况，则根据样式规则的先后顺序来决定哪个样式生效，后声明的样式会覆盖先声明的样式。下面是一个权重计算的示例：

```
/* 权重为 1 */div{/* 权重为 10 */.class1{/* 权重为 100 */#id1{/* 权重为 101 (100 + 1) */#id1 div{/* 权重为 11 (10 + 1) */.class1 div{/* 权重为 21 (10 + 10 + 1) */.class1 .class2 div{
```

根据权重的计算规则，选择器的权重越高，其样式优先级越高，将更有可能应用到对应的元素上。

66. 列出你所知道可以改变页面布局的属性

能改变页面布局的属性主要分为显示与定位、盒模型尺寸，以及现代布局三类。display、position、float和四个方向偏移决定元素怎样参与排版或放在哪里；width、height、margin、padding与box-sizing决定盒子占用的空间。复杂页面通常会优先使用flex或grid，因为它们更适合处理对齐、分栏和自适应。overflow负责内容溢出时的表现，z-index则处理定位元素之间的层叠顺序。

以下是一些可以改变页面布局的常用属性：

- position：控制元素的定位方式，如static、relative、absolute

e、fixed等。display：指定元素的显示方式，如block、inline、inline-block、flex、grid等。float：使元素浮动到指定的位置，常用于创建多列布局。width和height：控制元素的宽度和高度。margin和padding：调整元素的外边距和内边距。top、left、right和bottom：设置元素相对于其定位父元素的偏移位置。z-index：控制元素的层叠顺序。overflow：控制元素内容溢出时的处理方式。box-sizing：指定元素的盒模型类型，如content-box和border-box。flexbox和grid：强大的布局模型，用于创建复杂的网格布局和灵活的盒模型布局。

这些属性可以结合使用，通过调整它们的值和组合，可以实现各种不同的页面布局和排列方式。

67. CSS在性能优化方面的实践

CSS性能优化要同时减少资源传输成本，并降低浏览器匹配和渲染样式的开销。文件侧可以压缩、合理合并并开启Gzip，再配合缓存减少重复下载；样式应通过放在中，避免@import带来的额外请求延迟。代码侧要控制选择器层级，少用复杂选择器、CSS表达式和开销较大的滤镜。合并文件能减少请求，但也会增大单个资源，因此实际使用时要结合页面需要取舍。在性能优化方面，以下是一些CSS的实践方法：压缩和合并CSS：使用CSS压缩工具将CSS文件压缩，并将多个CSS文件合并为一个文件，减少网络请求次数和文件大小。使用Gzip压缩：配置服务器开启Gzip压缩，可以减小CSS文件的大小，加快文件传输速度。将CSS文件放在标签中：将CSS文件的引用放在HTML文档的标签中，以便在页面渲染前加载CSS样式。避免使用@import：避免在CSS中使用@import导入其他CSS文件，因为@import会增加额外的请求延迟，推荐使用标签引入CSS文件。使用缩写属性：尽量使用CSS的缩写属性，如margin、padding、font等，可以减少CSS文件的大小。避免使用滤镜：某些CSS滤镜效果会导致性能下降，特别是在大型页面中使用，尽量避免滤镜的使用。合理使用选择器：选择器的复杂性会影响CSS选择器的匹配速度，尽量避免使用过于复杂的选择器，减少选择器的层级和嵌套。避免使用CSS表达式：CSS表达式会在每次页面重绘时重新计算，影响性能，尽量避免使用。使用缓存：通过设置适当的HTTP响应头，将CSS文件缓存到浏览器中，减少重复请求。使用媒体查询：针对不同设备和屏幕尺寸，使用媒体查询来加载不同的CSS样式，提高响应性能。这些实践方法可以帮助优化CSS在网页加载和渲染过程中的性能，减少加载时间，提升用户体验。

68. CSS3动画（简单动画的实现，如旋转等）

CSS3简单动画通常由transform描述变化结果，再用transition或animation控制变化过程。transition适合元素从一种状态过渡到另一种状态，可以设置属性、时长、速度曲线和延迟；animation配合@keyframes定义每一帧，更适合自动播放或循环动画。比如旋转可以在关键帧中把transform从rotate(0deg)改到rotate(360deg)。再设置animation-duration: 3s、linear和infinite，元素就会匀速持续旋转。依靠CSS3中提出的三个属性：transition、transform、animation。transition：定义了元素在变化过程中是怎么样的，包含transition-property、transition-duration、transition-timing-function、transition-delay。transform：定义元素的变化结果，包含rotate、scale、skew、translate。animation：动画定义了动作的每一帧（@keyframes）有什么效果，包括animation-name、animation-duration、animation-timing-function、animation-delay、animation-iteration-count、animation-direction。

以下是一个使用CSS3动画实现旋转的示例：

```
/* 定义动画关键帧 */
@keyframes rotate {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

/* 应用动画到元素 */
.element {
  animation-name: rotate; /* 指定动画名称 */
  animation-duration: 3s; /* 动画持续时间 */
  animation-timing-function: linear; /* 动画速度曲线 */
  animation-delay: 0s; /* 动画延迟时间 */
  animation-iteration-count: infinite; /* 动画重复次数，这里设置为无限循环 */
  animation-direction: normal; /* 动画播放方向 */
}
```

69. base64的原理及优缺点

Base64本质上是把二进制数据按每 3 字节分组，再转换成 4 个 6 位的可打印字符，不足部分用填充符补齐。它适合在文本环境中传输或存储二进制内容，也能把小图片直接嵌入HTML、CSS或JavaScript，从而减少请求。代价是编码后体积会增大，而且编码、解码还会消耗一定的CPU。另外它是可逆编码，不提供真正的安全性，所以不能当作加密方案使用。Base64是一种将二进制数据编码为ASCII字符的方法，通过将二进制数据转换为由64个字符组成的可打印字符序列，实现二进制数据的传输和存储。

Base64编码的原理如下： 将待编码的数据按照每3个字节一组进行分组。将每组3个字节转换为4个6位的Base64字符。如果最后一组不足3个字节，根据需要进行填充。将转换后的Base64字符拼接在一起，形成最终的Base64编码结果。 优点： 可以将二进制数据转换为文本数据，方便在文本环境中传输和存储。减少了HTTP请求，可以将小的图片或其他资源直接嵌入到HTML、CSS或JavaScript代码中，减少了对服务器的请求次数。 缺点： Base64编码会使数据的大小增加，因为每3个字节的原始数据会转换为4个字节的Base64编码数据。Base64编码是一种可逆的编码方法，虽然可以加密数据，但并不提供真正的安全性。编码和解码过程涉及到字符转换和处理，消耗了一定的CPU资源。 总的来说，Base64编码适用于在文本环境中传输和存储二进制数据，并且可以减少HTTP请求。但在需要考虑数据大小和性能的情况下，需要权衡使用Base64编码的优缺点。

70. 几种常见的CSS布局

常见的CSS布局包括流体布局、圣杯布局和双飞翼布局，它们都能实现两侧定宽、中间自适应的三栏结构。流体布局通过左右浮动和中间区域的外边距预留空间，但要注意浮动可能造成父容器高度塌陷，需要清除浮动。圣杯布局用浮动、负边距和相对定位调整两侧位置，并让主体内容在文档中靠前。双飞翼布局则在主体内部设置左右外边距，继续用浮动和负边距归位两翼，减少了对相对定位的依赖。

71. stylus/sass/less区别

Sass、Less和Stylus都是CSS预处理器，核心能力相近，差别主要在语法形式和具体写法。三者都支持变量、嵌套、混合和样式复用，但Sass更强调缩进层级，Less和Stylus的写法更接近普通CSS，可以使用大括号。它们在继承、颜色混合等功能上的语法也不完全相同，例如Sass使用@extend，颜色处理函数也各有区别。实际选择时一般看团队习惯、现有工具链和项目要求，不必只凭功能多少决定。Sass, Less, 和 Stylus 是三种常用的 CSS 预处理器，它们在功能和语法上有一些区别： 语法差异：Sass 使用类似 Ruby 的缩进语法，而 Less 和 Stylus 使用类似 CSS 的语法。Sass 使用严格的缩进表示层次与嵌套关系，而 Less 和 Stylus 可以使用大括号{}表示层次和嵌套关系。变量：Sass、Less 和 Stylus 都支持变量，用于存储和复用值。在 Sass 和 Less 中，变量以\$符号开头，而在 Stylus 中，变量以\$或者@符号开头。混合：混合（Mixins）允许将一组 CSS 规则集合存储为一个可重用的样式块。Sass 和 Less 使用@mixin定义混合，而 Stylus 使用mixin关键字定义混合。嵌套：Sass、Less 和 Stylus 都支持嵌套规则，可以在父级规则内部定义子级规则。Sass 使用缩进表示嵌套关系，Less 和 Stylus 使用大括号{}表示嵌套关系。继承：继承（Inheritance）允许一个选择器继承另一个选择器的样式。在 Sass 中，使用@extend实现继承，而在 Less 和 Stylus 中，使用&符号实现继承。颜色混合：颜色混合（Color blending）允许将两个颜色混合生成一个新的颜色。Sass 使用mix()函数实现颜色混合，Less 使用blend()函数，而 Stylus 使用mix()函数。环境和工具支持：Sass 是基于 Ruby 的，需要安装Ruby环境来编译，而 Less 和Stylus可以通过 Node.js 的 NPM 安装相应的库来编译。总的来说，Sass、Less和Stylus在功能上大致相似，它们都提供了类似的特性，如变量、混合、嵌套、继承等，但在语法和一些细节上有一些差异。选择哪种预处理器取决于个人偏好、团队需求和项目要求。

72. postcss的作用

PostCSS本质上是一个用插件转换和优化CSS的处理平台，而不是单独提供固定语法的工具。它先把CSS解析成抽象语法树，再交给插件修改，因此能力取决于接入了哪些插件。常见场景是用Autoprefixer自动补浏览器前缀，或者进行代码压缩和新语法转换。它也可以接在Sass编译之后继续处理生成的CSS，两者并不冲突。可以直观的理解为：它就是一个平台。为什么说它是一个平台呢？因为我们直接用它，感觉不能干什么事情，但是如果让一些插件在它上面跑，那么将会很强大PostCSS提供了一个解析器，它能够将CSS解析成抽象语法树通过PostCSS这个平台上，我们能够开发一些插件，来处理我们的CSS，比如热门的：autoprefixerpostcss可以对sass处理过后的css再处理 最常见的就是autoprefixer PostCSS是一个用于转换CSS的工具，它提供了一个插件化的架构，可以通过加载各种插件来处理CSS。主要作用包括：转换 CSS：PostCSS可以将CSS解析成抽象语法树（AST），并允许开发者编写插件来修改和转换CSS。这使得开发者可以自定义和扩展CSS的功能，从而提供更灵活的编写样式的能力。自动添加浏览器前缀：PostCSS的插件生态系统中最常用的插件之一是autoprefixer。它可以根据配置和浏览器兼容性自动为样式属性添加浏览器前缀，以确保在不同浏览器中正确显示样式。代码优化和压缩：PostCSS的插件可以用于优化和压缩CSS代码，删除不必要的空格、注释、重复规则等，以减小文件大小并提高加载速度。使用未来的CSS语法：PostCSS可以支持使用未来的CSS语法和功能，例如使用CSS Variables、CSS Modules、CSS Grid等。通过一些插件，可以在现有浏览器中使用这些新特性，而无需等待浏览器的更新。总之，PostCSS提供了一个灵活的平台和插件生态系统，可以对CSS进行各种转换和优化，使开发者能够更好地编写和管理样式代码，并兼容不同的浏览器和未来的CSS标准。

73. css样式（选择器）的优先级

CSS样式最终由重要性、选择器权重和书写顺序共同决定，通常可理解为!important、内联样式、普通规则依次比较。普通规则之间会计算选择器权重，权重更高的直接生效，而不是靠后面的低权重规则覆盖。权重和重要性都相同时，后写的规则才会覆盖先写的规则。继承样式的优先级很低，我一般不会靠堆选择器或滥用!important解决冲突，以免后续难以维护。CSS样式的优先级可以根据以下规则进行计算：内联样式（Inline Styles）具有最高的优先级。通过在HTML元素的style属性中直接定义的样式将覆盖其他样式规则。权重（Specificity）是确定样式优先级的重要因素。每个选择器都有一个权重，权重由选择器的组成部分决定，通常根据选择器的类型、类、ID和内联样式等进行计算。选择器的权重值越高，其样式优先级越高。重要性（Importance）可以通过!important关键字来设置。使用!important的样式具有最高的优先级，将覆盖任何其他样式规则。然而，滥用!important可能导致样式管理的困难，应该慎用。层叠顺序（Cascade Order）也会影响样式的优先级。当存在多个具有相同权重和重要性的样式规则时，后写的样式规则将覆盖先写的样式规则。

综上所述，通过计算选择器的权重、考虑重要性以及根据样式规则的层叠顺序，可以确定CSS样式的优先级，并最终确定应用在元素上的样式。小结 计算权重确定!important内联样式后写的优先级高

74. 自定义字体的使用场景

自定义字体适合品牌宣传、Banner、固定标题文案以及字体图标等需要强化视觉识别的场景。它能让文字风格与品牌形象保持一致，也能把标题、标语或特殊提示从普通正文中突出出来。字体图标还可以通过CSS统一控制颜色和大小，比单独维护多张图更灵活。实际使用时要考虑字体文件的加载成本，尤其中文字体体积可能较大，通常需要做子集化并准备系统字体作为降级方案。自定义字体可以在以下场景中使用：宣传和品牌：使用自定义字体可以增加品牌的独特性和识别度。品牌标语、宣传资料、广告等场合可以使用自定义字体来传达品牌的风格和形象。Banner设计：在网站或移动应用的横幅广告（Banner）设计中，自定义字体可以使文字内容更加吸引人，与整体设计风格相匹配，并帮助吸引用户的注意力。固定文案：如果有一些特定的文案需要在网站或应用中固定展示，如标题、标语、特殊提示等，使用自定义字体可以使这些文案与其他普通文本区分开来，突出其重要性或特殊性。字体图标：自定义字体可以用于创建字体图标集合，如使用字体图标库（如Font Awesome）来代替传统的图标文件。这样可以提供灵活性和可扩展

展性，通过CSS样式来控制图标的颜色、大小和样式，而无需使用单独的图像文件。总之，自定义字体在需要强调品牌形象、设计吸引人的文案或创建字体图标集合等场景中非常有用。它们为设计师和开发人员提供了更多的创意和自定义选项，以满足特定的设计需求。

75. 如何美化CheckBox

美化CheckBox的常见做法是保留原生复选框的交互能力，再通过关联的label和自定义样式绘制外观。label的for要对应input的id，这样点击文字也能切换选中状态。样式上可以用:checked + label区分默认态和选中态，并通过伪元素绘制边框、背景或勾选图标。隐藏原生控件时不建议使用display:none，否则会影响聚焦和键盘操作；需求简单时也可以直接用accent-color修改主题色。

属性for和id隐藏原生的:checked + 要美化复选框（CheckBox），可以按照以下步骤进行操作：使用标签和关联的元素：将复选框包裹在标签中，并使用for属性与对应的元素的id进行关联，确保点击标签时也可以触发复选框的选择状态。

Checkbox Label 隐藏原生的元素：使用 CSS 样式将原生的元素隐藏起来，使其不可见。

input[type="checkbox"]{display:none;} 使用 CSS 样式美化复选框：利用 CSS 样式为复选框创建自定义的外观效果。

input[type="checkbox"] + label{/* 定义复选框的样式 */}input[type="checkbox"]:checked + label{/* 定义选中时复选框的样式 */}

在上述样式中，:checked选择器用于选中状态下的样式，+选择器用于选择紧接在元素后的兄弟元素。

通过调整这些样式，您可以改变复选框的外观，如更改背景颜色、添加边框、改变图标等。可以使用 CSS

属性如background-color、border、color、content等来设置样式。

例如，以下示例将复选框的背景颜色设置为蓝色，并在选中时添加一个打勾的图标：

input[type="checkbox"] + label{display:inline-block;padding-left:25px;position:relative;cursor:pointer;}input[type="checkbox"]

label:before{content:"";display:inline-block;width:16px;height:16px;border:2px solid #999;border-radius:3px;margin-right:10px;position:absolute;left:0;top:2px;}input[type="checkbox"]:checked + label:before{background-color:blue;border-color:blue;content:"\2713";text-align:center;font-size:

14px;line-height:16px;color:#fff;} 通过使用类似上述的 CSS 样式，您可以根据需要定制复选框的外观和样式，以实现美化效果。

76. 伪类和伪元素的区别

伪类用于描述并选择元素的特定状态，伪元素则用于选中元素的某个部分或创建可供样式处理的虚拟内容。例如:hover、:checked表示交互或选中状态，而::before、::after常用于插入装饰内容，::first-line用于选中首行。写法上，伪类使用单冒号，伪元素推荐使用双冒号；部分旧伪元素因兼容历史写成单冒号也能工作。简单来说，前者改变“什么时候选中”，后者解决“选中元素的哪一部分”。

伪类表状态伪元素是真的有元素前者单冒号，后者双冒号 以下是对伪类和伪元素的更详细解释：伪类（Pseudo-classes）是用于选择元素的特定状态或条件的关键词。它们以单冒号（:）作为前缀，并应用于选择器的末尾。伪类选择器可以选择处于特定状态的元素，例如鼠标悬停、被点击、被选中等。常见的伪类包括:hover、:active、:visited、:nth-child等。伪元素（Pseudo-elements）则是用于在文档中创建虚拟的元素，这些元素可以在选中的元素中添加额外的样式和内容。伪元素以双冒号（::）作为前缀，并应用于选择器的末尾。伪元素可以在选中的元素中插入新的内容或样式，如在元素前、后插入内容或改变选中元素的某个部分的样式。常见的伪元素包括::before、::after、::first-line、::first-letter等。总结来说，伪类用于选择特定状态的元素，而伪元素则用于在选中的元素中创建虚拟的元素。伪类以单冒号（:）作为前缀，伪元素以双冒号（::）作为前缀。在实际使用中，由于历史原因，有些伪元素也可以使用单冒号（:）来表示，但为了遵循最新的规范，推荐使用双冒号（::）表示伪元素。

77. base64的使用

base64是把二进制数据编码成ASCII字符串，常用于把小图片或字体直接嵌入HTML、CSS和JavaScript。资源跟随代码加载，可以少发一些HTTP请求，对小图标这类文件比较合适。代价是编码后体积通常增加约三分之一，而且代码会变长、维护起来更麻烦。文件较大时，我一般保留独立资源请求，避免额外增加传输量。用于减少HTTP请求适用于小图片base64的体积约为原图的3/4。base64是一种将二进制数据编码为ASCII字符串的方法，它常被用于将小文件（如图片、字体文件等）嵌入到HTML、CSS或JavaScript中，从而减少对服务器的请求次数。使用base64编码可以将二进制文件转换为文本字符串，这样可以直接将字符串嵌入到代码中，而无需单独请求文件。这样做的好处是可以减少HTTP请求的数量，提升页面加载速度，尤其适用于小图片或者一些图标字体等。然而，使用base64编码也有一些注意事项。由于base64编码后的文本字符串比原始二进制数据体积大约增加了1/3，因此对于大文件来说，使用base64会导致数据传输量增加，可能会影响网页的加载速度。此外，由于嵌入了文件内容，导致代码体积增大，也会对可维护性产生一定的影响。因此，通常建议将base64使用在小文件上，如小图标、小图片等，而对于大文件，仍然应该以原始文件的形式进行请求和传输，以获得更好的性能和可维护性。

78. 自适应布局

自适应布局就是让页面元素根据可用空间调整尺寸和位置，常见做法有浮动配合外边距、负外边距，以及Flexbox。左侧宽度固定时，可以让它浮动或绝对定位，再用右侧的margin留出位置，但宽度和间距需要手动计算。复杂一些的布局可以借助容器和负margin，不过规则不够直观。实际开发中我更倾向使用display: flex，固定左栏并让右栏通过flex-grow: 1占满剩余空间。思路：左侧浮动或者绝对定位，然后右侧margin撑开使用包含，然后靠负margin形成bfc使用flex。自适应布局是指能够根据不同设备或窗口尺寸自动调整布局的一种设计方式。下面是几种常见的自适应布局方法：使用浮动或绝对定位：左侧元素使用浮动或绝对定位，右侧元素通过设置左侧元素的margin来撑开布局。这种方法需要手动计算和设置宽度和间距，适用于简单的布局需求。
.left{float:left;width:200px;}.right{margin-left:220px;} 使用包含元素和负边距：将左右两个元素放在一个容器内，设置容器的overflow属性为hidden，然后通过负边距将左侧元素向左移动，右侧元素则会自动占据剩余空间。这种方法需要使用负边距，适用于复杂布局需求。
.container{overflow:hidden;}.left{float:left;width:200px;margin-left:-100%;}.right{float:left;} 使用Flexbox 布局：使用 Flexbox 弹性布局可以轻松实现自适应布局。通过设置容器的display属性为flex，并使用flex-grow属性来控制元素的伸缩性，可以自动调整元素的宽度和布局。
.container{display:flex;}.left{flex-grow:0;flex-shrink:0;width:200px;}.right{flex-grow:1;}
以上是一些常见的自适应布局方法，根据具体的需求和场景，可以选择合适的方法来实现自适应布局。

79. 请用CSS写一个简单的幻灯片效果页面

简单的纯CSS幻灯片可以用animation配合@keyframes，按时间切换容器的背景图片。容器先设置固定宽高和overflow: hidden，再让动画无限循环，就能形成连续播放效果。关键帧可在0%、25%、50%、75%和100%分别指定不同的background。这种方案适合不需要交互的简单轮播，如果要支持按钮、分页或动态内容，单靠这段动画就不够了。知道是要用CSS3。使用animation动画实现一个简单的幻灯片效果
/**css**/.ani{width:480px;height:320px;margin:50px auto;overflow:hidden;box-shadow:0 0 5px rgba(0,0,0,1);background-size:cover;background-position:center;-webkit-animation-name:"loops";-webkit-animation-duration:20s;-webkit-animation-iteration-count:infinite;}@-webkit-keyframes"loops"{0%{background:url(http://d.hiphotos.baidu.com/image/w%3D400/sign=c01e6adca964034f0fcdc3069fc27980/e824b899a9014c08e5e38ca4087b02087af4f4d3.jpg)no-repeat;}25%{background:url(http://b.hiphotos.baidu.com/image/w%3D400/sign=edee1572e9f81a4c2632edc9e72b6029/30adcbef76094b364d72bceba1cc7cd98c109dd0.jpg)no-repeat;}50%{background:url(http://b.hiphotos.baidu.com/image/w%3D400/sign=937dace2552c11dfded1be2353266255/d8f9d72a605925

```
2d258e7605369b033b5bb5b912.jpg)no-repeat;}75%{background:url(http://g.hiphotos.baidu.com/image/w%3D400/sign=7d37500b8544ebf86d71653fe9f9d736/0df431adcbef76095d61f0972cdda3cc7cd99e4b.jpg)no-repeat;}100%{background:url(http://c.hiphotos.baidu.com/image/w%3D400/sign=cfb239ceb0fb43161a1f7b7a10a54642/3b87e950352ac65ce2e73f76f9f2b21192138ad1.jpg)no-repeat;}}
```

80. 什么是外边距重叠？重叠的结果是什么？

外边距重叠，也叫margin collapse，是相邻盒子的垂直外边距合并成一个外边距。两个值都为正数时取较大值，都为负数时取绝对值较大的那个，一正一负则相加。它常出现在相邻兄弟元素之间，也可能发生在父元素与第一个或最后一个子元素之间。需要避免时，可以用padding或border隔开，也可以通过浮动或绝对定位改变原来的布局关系。外边距重叠就是margin-collapse 在CSS当中，相邻的两个盒子（可能是兄弟关系也可能是祖先关系）的外边距可以结合成一个单独的外边距。这种合并外边距的方式被称为折叠，并且因而所结合成的外边距称为折叠外边距。折叠结果遵循下列计算规则：两个相邻的外边距都是正数时，折叠结果是它们两者之间较大的值。两个相邻的外边距都是负数时，折叠结果是两者绝对值的较大值。两个外边距一正一负时，折叠结果是两者的相加的和。下面是详细分析 外边距重叠（margin collapse）指的是在某些情况下，相邻的两个元素之间的外边距会发生合并，并且取两者之间的较大值作为最终的外边距值。外边距重叠主要发生在垂直方向上，而水平方向上的外边距不会发生重叠。外边距重叠的结果是两个相邻元素的外边距被合并成一个单独的外边距，这可能会导致布局上的一些意外效果，比如元素之间的间距变得比预期的要大。下面是一些常见情况下外边距重叠的例子：相邻的兄弟元素的外边距重叠：`.box{margin-top:20px;margin-bottom:30px;}` 在这个例子中，两个相邻的兄弟元素之间的上下外边距会发生重叠，最终的外边距值为30px，而不是预期的50px。

父元素与第一个/最后一个子元素的外边距重叠：

`.parent{margin-bottom:20px;}.child{margin-top:30px;}` 在这个例子中，父元素的下外边距和子元素的上外边距发生重叠，最终的外边距值为30px，而不是预期的50px。了解外边距重叠的规则可以帮助我们更好地控制元素的布局，避免意外的外边距重叠效果。在需要避免外边距重叠的情况下，可以使用一些方法，如使用内边距（padding）或边框（border）来隔离外边距、使用浮动（float）或绝对定位（position）等。

81. rgba()和opacity的透明效果有什么不同？

rgba()只改变指定颜色的透明度，而opacity会让整个元素及其内部内容一起透明。比如给背景设置半透明的rgba()，文字和子元素仍可保持不透明；设置opacity: 0.5后，文字、图片等内容也会一起变淡。两者的透明度都用0到1表示，0是完全透明，1是完全不透明。只想做半透明背景时用rgba()，需要整体淡化元素时再用opacity。rgba()是一种CSS颜色值表示方法，可以在其中指定红、绿、蓝三个通道的颜色值以及透明度。通过调整透明度值来实现元素的透明效果，仅影响元素的颜色或背景色，不影响元素内的其他内容的透明度。`.element{background-color:rgba(255,0,0,0.5);/* 半透明红色背景 */}` opacity是CSS属性，用于设置元素的整体透明度。它会影响元素以及元素内的所有内容的透明度，包括文本、图像等。设置元素的透明度会影响整个元素及其内容的可见性。`.element{opacity:0.5;/* 元素及其内容半透明 */}` 需要注意的是，opacity的值是一个0到1之间的数字，0表示完全透明，1表示完全不透明。而rgba()中的透明度值是一个介于0到1之间的数字，0表示完全透明，1表示完全不透明。另外，需要注意的是，opacity的透明度是继承的，子元素会继承父元素的透明度效果，而rgba()设置的透明度不会继承给子元素。综上所述，rgba()和opacity在实现透明效果上有一些不同，需要根据具体的需求和效果来选择使用哪种方式。

小结 rgba()和opacity都能实现透明效果，但最大的不同是opacity作用于元素，以及元素内的所有内容的透明度，而rgba()只作用于元素的颜色或其背景色。（设置rgba透明的元素的子元素不会继承透明效果！

）

82. css中可以让文字在垂直和水平方向上重叠的两个属性是什么？

垂直方向可以用line-height控制，水平方向可以用letter-spacing控制。line-height决定行与行之间的距离，把行高调整到合适的较小值时，多行文字会在垂直方向靠近甚至重叠。letter-spacing决定字符间距，设置负值会让相邻字符靠近，数值足够小时便会出现水平重叠。这两个属性都要结合字号调整，避免文字重叠过度而影响阅读。

垂直方向：line-height 水平方向：letter-spacing
在CSS中，可以使用以下两个属性实现文字在垂直和水平方向上的重叠： 垂直方向：line-height通过设置line-height属性，可以控制行高，从而实现文字在垂直方向上的重叠。将line-height的值设置为大于文字大小的值，可以使文字垂直居中或与其他文字重叠。`.text{line-height:1.5; /* 行高为文字大小的1.5倍 */}` 水平方向：letter-spacing通过设置letter-spacing属性，可以控制字符之间的间距，从而实现文字在水平方向上的重叠。将letter-spacing的值设置为负数，可以让字符紧密排列，产生重叠效果。`.text{letter-spacing:-2px; /* 字符间距为负数，产生重叠效果 */}` 垂直方向：line-height 通过设置line-height属性，可以控制行高，从而实现文字在垂直方向上的重叠。将line-height的值设置为大于文字大小的值，可以使文字垂直居中或与其他文字重叠。`.text{line-height:1.5; /* 行高为文字大小的1.5倍 */}` 水平方向：letter-spacing 通过设置letter-spacing属性，可以控制字符之间的间距，从而实现文字在水平方向上的重叠。将letter-spacing的值设置为负数，可以让字符紧密排列，产生重叠效果。`.text{letter-spacing:-2px; /* 字符间距为负数，产生重叠效果 */}`

这两个属性可以根据具体的需求来调整，实现文字的垂直和水平方向上的重叠效果。

83. 如何垂直居中一个浮动元素？

可以通过绝对定位配合偏移量实现垂直居中，也可以直接让父容器使用Flex布局。元素尺寸已知时，可设置top: 50%，再用负的半高margin-top把元素向上拉回自身高度的一半。另一种定位写法是同时设置top: 0、bottom: 0和margin:

auto，让上下剩余空间自动均分。如果是图片等普通内容，我一般会在父容器设置display:

flex和align-items: center，表达更直接。 垂直居中一个浮动元素可以使用以下两种方法：

方法一（已知元素的高宽）：

```
div1{background-color:#6699FF;width:200px;height:200px;position:absolute; /* 父元素需要相对定位 */top:50%;left:50%;margin-top:-100px; /* 二分之一的height */margin-left:-100px; /* 二分之一的width */}
```

方法二：
`div1{width:200px;height:200px;background-color:#6699FF;margin:auto;position:absolute; /* 父元素需要相对定位 */left:0;top:0;right:0;bottom:0;}`

对于垂直居中一个，可以使用更简便的方法：
`container{display:flex;justify-content:center;align-items:center;}`

上述代码将元素放置在一个容器中（#container），通过使用 Flex 布局的justify-content:center;和align-items:center;属性，实现了图片在容器中的垂直居中效果。

84. px和em的区别

px是大小固定的绝对长度单位，em是根据字体大小计算的相对长度单位。父元素字号为16px时，子元素的1em通常对应16px，2em则对应32px；父级字号变化后，使用em的尺寸也会按比例变化。没有额外设置字号时，浏览器默认字号通常是16px。需要固定、容易计算的尺寸时可用px，希望页面随字体设置灵活缩放时更适合用em。 px和em是两种不同的长度单位，它们的区别如下： px（像素）是一个绝对单位，表示固定的像素大小。无论父元素的字体大小如何，px的值都不会改变，它是一个固定的长度单位。em（倍数）是一个相对单位，它相对于父元素的字体大小来确定自身的大小。如果没有设置字体大小，则1em等于浏览器默认的字体大小（通常是16px）。如果父元素的字体大小是16px，那么1em就等于16px，2em就等于32px，以此类推。 由于em是相对单位，它具有一定的灵活性和可扩展性。当需要调整整个页面

的字体大小时，只需更改根元素的字体大小，其他使用em作为单位的元素会自动按比例调整大小，从而实现页面的整体缩放效果。

相对于px来说，em更适用于实现弹性布局、响应式设计以及根据用户偏好进行字体大小调整等场景。小结px和em都是长度单位，区别是，px的值是固定的，指定是多少就是多少，计算比较容易。em得值不是固定的，并且em会继承父级元素的字体大小。浏览器的默认字体高都是16px。所以未经调整的浏览器都符合:1em=16px。那么12px=0.75em,10px=0.625em。

相对于显示器屏幕分辨率，无法用浏览器字体放大功能em 值并不是固定的，会继承父级的字体大小： $em = \text{像素值} / \text{父级font-size}$

85. Sass、LESS是什么？大家为什么要使用他们？

Sass和Less都是CSS预处理器，使用扩展语法编写样式，再编译成浏览器能识别的CSS。它们补充了变量、计算和嵌套等能力，修改颜色或间距时可以统一调整，也能减少重复选择器。通过混合、继承和模块导入，公共样式更容易复用，大型样式表也更便于拆分维护。实际使用时仍要经过编译，其中Less既可以在客户端运行，也可以借助Node.js在服务端处理。他们是CSS预处理器。他是CSS上的一种抽象层。他们是一种特殊的语法/语言编译成CSS。例如Less是一种动态样式语言。

将CSS赋予了动态语言的特性，如变量，继承，运算，函数。LESS既可以在客户端上运行（支持IE 6+,Webkit,Firefox），也可一在服务端运行（借助Node.js）

以下是为什么人们选择使用Sass和Less的一些原因：变量和计算：Sass和Less都支持变量，可以定义和重用各种值，如颜色、字体、边距等。它们还允许进行数学计算，简化了样式表的编写和维护。嵌套规则：Sass和Less允许在样式规则中嵌套其他规则，提高了样式表的可读性和可维护性。通过嵌套，可以更清晰地表示元素的层次结构，减少了样式选择器的重复。混合（Mixins）：混合是一种可以在多个选择器中重复使用的样式块。通过定义和调用混合，可以避免样式的重复编写，并且使样式表更加模块化和可复用。继承：继承允许一个选择器继承另一个选择器的样式规则，减少了样式的冗余。当多个选择器具有相同的样式时，可以通过继承来避免重复编写样式。模块化和导入：Sass和Less支持将样式表拆分为多个模块，并通过导入机制进行组合。这使得样式表的组织和管理更加灵活和可扩展。自定义函数：Sass和Less都允许定义自定义函数，可以用于处理样式值，进行复杂的计算和操作。强大的工具和生态系统：Sass和Less都有丰富的工具和插件生态系统，提供了许多辅助工具、编译器和构建工具，如预处理器编译器、自动刷新、自动前缀添加等，极大地提升了前端开发的效率。总而言之，Sass和Less使得CSS的编写更加简洁、模块化和可维护，提供了一些高级功能和工具，使前端开发更加高效和灵活。

86. 知道css有个content属性吗？有什么作用？有什么应用？

content主要配合::before和::after伪元素生成内容，用来补充文本或实现装饰效果。它可以放字符串、图标，也可以通过attr()读取元素属性，或结合counter()生成自动编号。常见场景有清除浮动、添加标识和展示序号，例如给::after设置内容后再配合clear:

both。它不适合直接用于普通元素，生成内容的布局通常还要结合合适的display。css的content属性专门应用在before/after伪元素上，用于来插入生成内容。最常见的应用是利用伪类清除浮动。/**一种常见利用伪类清除浮动的代码*/.clearfix:after{content:"";//这里利用到了content属性display:block;height:0;visibility:hidden;clear:both;}.clearfix{*zoom:1;} content属性主要用于在::before和::after伪元素中插入生成的内容。它通常与伪元素一起使用，以在文档中插入额外的内容或修饰样式。content属性可以接受多种类型的值，包括文本字符串、URL、计数器、计数器符号和引用标签等。通过设置不同的content值，可以实现一些常见的效果，例如：插入文本内容：可以在伪元素中使用content属性插入自定义的文本内容，用于装饰样式或添加额外的标识。插入图标和符号：通过设置content属性为某个图标字体或Unicode编码，可以在伪元素中插入图标和特殊符号。计数器：结合使用content属性和counter函数，可以在伪元素中显示自动生成的计数器，用于标记序号或计数。引用标签：通过设置content属性为attr()函数，可以

在伪元素中引用元素的属性值，用于显示元素的属性内容。上述代码示例中，通过设置content属性为一个点字符，在::after伪元素中插入一个看不见的点，配合其他样式属性实现了清除浮动的效果。这是一种常见的清除浮动的技巧之一。需要注意的是，content属性只对::before和::after伪元素起作用，对于其他元素并没有效果。此外，content属性必须与display属性一起使用，通常为block或inline-block，以确保生成的内容具有正确的布局。

87. 水平居中的方法

水平居中需要看元素和布局类型，行内内容用text-align:

center，块级元素通常用自动外边距、弹性布局或定位。定宽块级元素可以设置margin-left:

auto和margin-right: auto，因为剩余水平空间会分配到两侧。父元素使用display:

flex时，设置justify-content: center更适合常规布局。元素宽度未知且需要脱离文档流时，可以用left:

50%配合transform: translateX(-50%)，旧式表格布局则可用table-cell和文本居中。元素为行内元素，设

置父元素text-align:center如果元素宽度固定，可以设置左右margin为auto;绝对定位和移动:absolute + transform使用flex-box布局，指定justify-content属性为centerdisplay设置为table-cell

下面进行详细说明：文本居中：如果元素为行内元素，可以将父元素的text-align属性设置为center，这样子元素就会水平居中对齐。
.parent{text-align:center;}

固定宽度的居中：如果元素宽度已知并固定，可以通过将左右margin设置为auto来实现水平居中。

.element{margin-left:auto;margin-right:auto;} 绝对定位和移动：可以使用绝对定位和transform来实现水平居中。首先将元素的左边距和右边距都设置为auto，然后使用transform属性将元素向左平移50%。

.element{position:absolute;left:50%;transform:translateX(-50%);} Flexbox布局：使用display:

flex将父元素设置为弹性容器，然后使用justify-content属性将子元素水平居中。

.parent{display:flex;justify-content:center;}

表格布局：将父元素的display属性设置为table-cell，并将text-align属性设置为center。

.parent{display:table-cell;text-align:center;}

这些方法可以根据具体的布局需求和浏览器兼容性进行选择和使用。

88. 垂直居中的方法

垂直居中同样取决于内容和布局，常规容器优先考虑Flexbox，特殊场景再用定位、行高或表格布局。父元素设置display: flex和align-items:

center，不要求提前知道子元素高度，维护起来也比较直接。需要绝对定位时，可用top:

50%配合transform: translateY(-50%);如果高度已知，也能使用负的margin-top。单行文本可以让line-height等于容器高度，而旧式未知高度布局可用table-cell配合vertical-align:

middle，但要考虑兼容性和布局限制。将显示方式设置为表格，display:table-cell,同时设置vertical-align

: middle使用flex布局，设置为align-item: center绝对定位中设置bottom:0,top:0,并设置margin:auto

绝对定位中固定高度时设置top:50%, margin-top值为高度一半的负值文本垂直居中设置line-height为height值inline-block兄弟元素：通过在父元素中插入一个inline-block元素，并设置其垂直对齐方式为middle

来实现垂直居中 表格布局：将父元素的display属性设置为table，并将子元素的display属性设置为table-cell，然后使用vertical-align属性将子元素垂直居中

未知高度的块级父子元素居中，模拟表格布局缺点：IE67不兼容，父级overflow: hidden失效

.parent{display:table;}.child{display:table-cell;vertical-align:middle;}

Flexbox布局：将父元素的display属性设置为flex，并使用align-items属性将子元素垂直居中。

.parent{display:flex;align-items:center;} 绝对定位和负边距：对于已知高度的子元素，将父元素设置为相对定位，子元素设置为绝对定位，并使用top:

50%将其垂直居中，然后通过负边距的方式将子元素向上移动一半的高度

```
.parent{position:relative;}.child{position:absolute;top:50%;margin-top:-50px;/*
```

假设子元素高度为100px的一半

```
*/}
```

文本垂直居中：对于单行文本，可以设置父元素的line-height属性和高度相等，从而实现文本的垂直居中

```
.parent{height:100px;line-height:100px;}
```

CSS3位移：使用CSS3的transform属性的translateY函数将子元素向上位移一半的高度实现垂直居中

```
.child{position:absolute;top:50%;transform:translateY(-50%);} inline-block兄弟元素：通过在父元素中  
插入一个inline-block元素，并设置其垂直对齐方式为middle来实现垂直居中 .parent{height:100%;}.extra{display:inline-block;vertical-align:middle;}.child{display:inline-block;vertical-align:middle;}
```

89. 如何使用CSS实现硬件加速？

使用CSS实现硬件加速，本质上是把元素提升为独立的复合图层，让GPU参与渲染。常见做法是给动画元素设置transform: translateZ(0)，也可以优先让动画只修改transform、opacity或filter。这样能减少二维动画开始和结束时可能出现的重绘操作，尤其适合动画较多或渲染较复杂的元素。不过复合图层并非越多越好，使用过度也可能带来性能问题，因此需要结合实际页面测试。

硬件加速是指通过创建独立的复合图层，让GPU来渲染这个图层，从而提高性能，一般触发硬件加速的CSS属性有transform、opacity、filter，为了避免2D动画在开始和结束的时候的repaint操作，一般使用transform:translateZ(0) 使用CSS实现硬件加速可以通过以下方法： 使用3D变换：通过应用3D变换，如translateZ(0)，来触发硬件加速。这会将元素视为3D空间中的一个对象，使浏览器使用GPU进行渲染。

```
.element{transform:translateZ(0);} 使用CSS动画：使用CSS动画属性（如transform、opacity、filter）  
来触发硬件加速。这可以通过创建一个动画并将其应用于元素来实现。
```

```
.element{animation:myAnimation 1s linear infinite;}@keyframesmyAnimation{0%{transform:translateZ(0);}100%{transform:translateZ(0);} 使用  
CSS过渡：通过使用CSS过渡属性（如transform、opacity、filter）来触发硬件加速。这可以通过设置过渡效果来实现。 .element{transition:transform 0.3s ease;}.element:hover{transform:translateZ(0);} 请注  
意，硬件加速并不是适用于所有情况的解决方案，它对于涉及大量动画或复杂渲染的元素特别有效。但是  
，在某些情况下，过多地使用硬件加速可能会导致性能问题，因此需要在实际使用时进行评估和测试。
```

90. 重绘和回流（重排）是什么，如何避免？

回流是浏览器重新计算元素的尺寸和位置，重绘则是在布局不变时重新绘制颜色、背景等外观。回流通常会连带触发重绘，而单纯重绘不一定引起回流，所以布局变化的成本一般更高。工程中我会用transform和opacity做动画，通过修改class、DocumentFragment或离线DOM合并多次操作。读取offsetTop、clientWidth等布局属性时也要集中处理并缓存结果，避免边写样式边读取布局造成强制同步计算。

重绘（Repaint）和回流（Reflow）是浏览器在渲染页面时的两个关键过程。 重绘（Repaint）是指当元素的外观属性（如颜色、背景等）发生改变，但不影响布局时的重新绘制过程。重绘不会影响元素的几何尺寸和位置。回流（Reflow）是指当元素的布局属性（如尺寸、位置、隐藏/显示等）发生改变，导致浏览器重新计算元素的几何属性，重新构建渲染树的过程。回流会导致其他相关元素的回流和重绘。回流必将引起重绘，而重绘不一定会引起回流

避免重绘和回流对于提高页面性能和响应速度至关重要。以下是一些减少重绘和回流的方法： 使用 CSS3 动画：使用 CSS3 的transform和opacity等属性来创建动画效果，因为它们会触发硬件加速，减少重绘和回流的影响。批量修改样式：避免频繁修改单个元素的样式，尽可能将修改合并为一次操作，可以使用class或修改style属性的方式。使用文档片段：当需要添加多个 DOM 元素到文档中时，可以先创建一个文档片段（DocumentFragment），将元素添加到片段中，然后再将片段一次性添加到文档中，减少回流次数。使用离线 DOM：将元素从文档中移除（display: none），进行复杂的操作（如修改样式、添加子元素等），完成后再将元素放回文档，以减少回流和重绘的影响。缓存布局属性值：如果需要多次访问某个元素

的布局属性（如位置、尺寸等），可以将其值缓存起来，避免多次触发回流计算。避免强制同步布局：避免在 JavaScript 中获取布局属性（如使用offsetTop、clientWidth等），因为它会强制同步计算布局信息，触发回流。如果需要获取布局信息，最好将获取操作放在一起，或使用getBoundingClientRect()方法。通过合理的设计和优化，可以最小化重绘和回流的次数，提高页面性能和用户体验。

91. 说一说css3的animation

CSS3的animation通过@keyframes定义关键帧，再用动画属性控制播放过程。常用配置可以归为时间、次数和状态三类，例如duration控制时长，iteration-count控制循环次数，play-state控制暂停或运行。它比transition更适合自动播放、多关键帧和循环动画，fill-mode: forwards还能让元素停留在最后一帧。实际使用时通常优先对transform和opacity做动画，以减少对布局和绘制的影响。CSS3的animation属性是用于创建动画效果的一种方式。它可以通过关键帧（@keyframes）来定义动画的每一帧，以实现元素的平滑过渡和动态效果。使用animation属性时，需要设置以下几个关键的子属性：animation-name：定义动画的名称，对应@keyframes中的动画名。animation-duration：定义动画的持续时间，可以设置为具体的时间值，如2s表示2秒，或者使用关键词infinite表示无限循环。animation-timing-function：定义动画的时间函数，控制动画在不同时间点的速度变化，常见的取值有linear（线性）、ease（缓入缓出）、ease-in（缓入）、ease-out（缓出）、ease-in-out（缓入缓出）等。animation-delay：定义动画的延迟时间，即动画开始之前的等待时间。animation-iteration-count：定义动画的循环次数，可以设置为具体的次数，或者使用关键词infinite表示无限循环。animation-direction：定义动画的播放方向，包括正常播放（normal）、反向播放（reverse）、交替反向播放（alternate）等。animation-fill-mode：定义动画播放之前和之后的样式状态，包括保持初始状态（none）、保持最后状态（forwards）、保持初始和最后状态（both）等。animation-play-state：定义动画的播放状态，可以控制动画的暂停和继续播放，包括paused（暂停）和running（运行）。通过调整这些子属性的取值，可以创建各种不同的动画效果，使元素在页面上实现平滑的过渡和动态的效果。CSS3的animation提供了一种简洁、易用且高性能的方式来实现动画效果，减少了对JavaScript的依赖。

92. 左边宽度固定，右边自适应

左边固定、右边自适应的两列布局，我一般优先使用Flex：左侧定宽，右侧设置flex:

1。父容器加上display:

flex后，右侧会自动占满剩余空间，结构简单，也便于后续调整。传统方案也可以让左侧float: left，或使用绝对定位，再给右侧设置对应的margin-left、left和right。如果页面不受旧布局方式限制，Flex通常更直观；需要兼容既有代码时，再根据原来的浮动或定位结构选择方案。

左侧固定宽度，右侧自适应宽度的两列布局实现 html结构 固定宽度自适应宽度 在外层div（类名为outer）的div中，有两个子div，类名分别为left和right，其中left为固定宽度，而right为自适应宽度

方法1：左侧div设置成浮动：float: left，右侧div宽度会自动适应 .outer{width:100%;height:500px;background-color:yellow;}.left{width:200px;height:200px;background-color:red;float:left;}.right{height:200px;background-color:blue;}

方法2：对右侧div进行绝对定位，然后再设置right=0，即可以实现宽度自适应 绝对定位元素的第一个高级特性就是其具有自动伸缩的功能，当我们将width设置为auto的时候（或者不设置，默认为auto），绝对定位元素会根据其left和right自动伸缩其大小 .outer{width:100%;height:500px;background-color:yellow;position:relative;}.left{width:200px;height:200px;background-color:red;}.right{height:200px;background-color:blue;position:absolute;left:200px;top:0;right:0;}

方法3：将左侧div进行绝对定位，然后右侧div设置margin-left: 200px .outer{width:100%;height:500px;background-color:yellow;position:relative;}.left{width:200px;height:200px;background-color:red;position:absolute;}.right{height:200px;background-color:blue;margin-left:200px;}

方法4：使用flex布局 .outer{width:100%;height:500px;background-color:yellow;display:flex;flex-direction:row;}.left{width:200px;height:200px;background-color:red;}.right{height:200px;background-color:blue;flex:1;}

93. 两种以上方式实现已知或者未知宽度的垂直水平居中

宽高未知时可以用Flex或绝对定位配合transform，宽高已知时还可以使用负margin。Flex通过justify-content:center和align-items:center

center在两个方向居中，通常最直观。绝对定位方案把top、left设为50%，再用translate(-50%, -50%)抵消元素自身尺寸，因此不必提前知道宽高。若尺寸固定，也可以用一半宽高的负margin；旧场景还可利用table-cell配合vertical-align: middle。 3
/** 1 */.wrapper{position:relative;.box{position:absolute;top:50%;left:50%;width:100px;height:100px;margin:-50px 0 0 -50px;}}/** 2 */.wrapper{position:relative;.box{position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);}}/** 3
*/.wrapper{.box{display:flex;justify-content:center;align-items:center;height:100px;}}/** 4
*/.wrapper{display:table;.box{display:table-cell;vertical-align:middle;}}

94. 如何实现小于12px的字体效果

可以把文字元素转成inline-block，再通过transform:

scale()实现小于12px的视觉效果。行内元素默认没有可直接控制的宽高，因此通常先设置display:

inline-block，让缩放行为更容易控制。再用transform-origin: left top固定缩放基准点，避免文字从中心缩放后出现明显偏移。需要注意，缩放可能降低文字清晰度，也不会自然解决原有布局占位问题，所以应优先保证可读性。实现小于12px的字体效果可以使用CSS的transform: scale()属性。但需要注意的是，该属性只能应用于具有宽度和高度的元素，而行内元素默认是没有宽度和高度的。

为了在行内元素上应用缩放效果，可以将其转换为具有宽度和高度的块级元素，例如使用display:

inline-block。

下面是一个示例代码：

```
.small-text{display:inline-block;transform:scale(0.7);transform-origin:left top;}
```

在上述代码中，我们将目标元素的display属性设置为inline-block，然后应用了transform:

scale(0.7)来缩小元素的大小，并通过transform-origin: left top设置缩放的基准点为左上角。使用以上代码，可以实现小于12px的字体效果，但需要注意，缩放可能会导致文本的清晰度下降，因此在使用此技术时需要谨慎权衡效果和可读性。

95. css hack原理及常用hack

CSS hack本质上是利用不同浏览器解析CSS的差异，让某条样式只在特定浏览器或版本中生效。常见做法可以归为属性写法、特殊选择器和IE条件注释，例如color: red\9、* html .element以及针对IE 7加载独立样式表。它能处理特定兼容问题，但会降低代码的可读性和维护性，浏览器更新后也可能产生副作用。我一般只把它当作兼容旧环境的兜底，正常开发更倾向使用前缀、特性检测和渐进增强。原理：利用不同浏览器对CSS的支持和解析结果不一样编写针对特定浏览器样式。常见的hack有属性hack选择器hackIE条件注释 属性hack选择器hackIE条件注释 CSS hack是一种技术手段，用于在不同的浏览器或特定的浏览器版本上应用不同的样式规则。它通过利用浏览器对CSS解析的差异来实现。 以下是一些常见的CSS

hack及其应用： 属性hack：针对特定浏览器或浏览器版本设置不同的属性值。 /*

仅适用于IE10及以下版本 */.element{color:red\9;}

选择器hack：使用特定的选择器来针对特定浏览器或浏览器版本。 /* 仅适用于IE6 */ html

.element{color:red;} IE条件注释：通过IE条件注释来针对不同的IE版本应用不同的样式。

上述代码中，和之间的样式表链接只会在IE7浏览器中生效。 需要注意的是，CSS hack是一种针对特定浏览器或浏览器版本的解决方案，但它可能存在以下问题： 可能会导致代码的可读性

和维护性降低。在不同的浏览器更新版本或新的浏览器出现时，需要更新和调整hack。可能会导致一些意外的副作用或兼容性问题。因此，在使用CSS hack时，需要权衡利弊，并尽量考虑更加稳定和标准的解决方案，例如使用CSS前缀、特性检测和渐进增强等方法来实现跨浏览器兼容性。

96. CSS有哪些继承属性

CSS中可继承的属性大多与文字呈现有关，包括字体、文本排版、行高、颜色、可见性和光标等类别。常见的有font-family、font-size、line-height、color、text-align、white-space、visibility和cursor。父元素设置这些属性后，子元素通常会自动获得相同效果，除非子元素声明了自己的值进行覆盖。简单来说，判断时可以优先看它是否描述文字表现，但具体属性仍应以其继承规则为准。CSS中有一些属性是可以继承的，这意味着父元素的某些样式属性会自动应用到子元素上。以下是一些常见的继承属性：

字体相关属性：font-family、font-size、font-weight、font-style

文本排版属性：word-break、letter-spacing、text-align、text-rendering、word-spacing、white-space、text-indent、text-transform、text-shadow

行高属性：line-height

颜色相关属性：color

可见性属性：visibility

光标属性：cursor

font-family、font-size、font-weight、font-style、word-break、letter-spacing、text-align、text-rendering、word-spacing、white-space、text-indent、text-transform、text-shadow、line-height、color、visibility、cursor 这些属性在父元素上设置后，会自动应用到其子元素上，除非子元素有自己的特定样式覆盖了继承的属性。需要注意的是，并非所有的CSS属性都是可继承的，有些属性是不会传递给子元素的。在使用CSS样式时，需要注意属性的继承特性，以便正确地应用样式到子元素。

97. 外边距折叠(collapsing margins)

外边距折叠只发生在常规流块级元素的垂直方向，相邻的margin会合并，而不是简单相加。同为正值时取较大值，同为负值时取绝对值较大的值，一正一负则相加。常见场景包括相邻兄弟、父元素与首尾子元素，以及空块自身的上下外边距。需要阻断时，我一般用padding、border或创建BFC，也可以改成Flex、Grid布局。外边距折叠(collapsing margins)是CSS中一种特定的行为，它会导致一些相邻元素的外边距合并成一个更大的外边距。以下是关于外边距折叠的几个规则：

相邻的普通流中的块级元素的垂直外边距会折叠。只有垂直方向上的外边距才会发生折叠，水平方向上的外边距不会折叠。浮动元素、绝对定位元素以及行内块元素的外边距不会和其他元素的外边距折叠。创建了块级格式化上下文(BFC)的元素的的外边距不会和其子元素的外边距折叠。常见的创建BFC的方式包括设置overflow为除visible之外的值、使用float属性、使用position: absolute等。相邻元素自身的margin-bottom和margin-top会折叠。当一个元素的margin-bottom和下一个元素的margin-top相邻时，它们会合并成一个外边距。外边距折叠在布局中可能会导致一些意外的效果，因此在需要避免外边距折叠的情况下，可以采用以下方法：

设置padding、border或者使用inline-block等方式来阻止外边距折叠。使用浮动(float)或绝对定位(position: absolute)等方式创建块级格式化上下文(BFC)，从而阻止外边距折叠。

了解和掌握外边距折叠的规则可以帮助我们更好地处理布局和样式相关的问题。

98. CSS选择符有哪些？哪些属性可以继承

CSS选择器可以按基础、关系和状态来理解；继承则主要发生在文字排版属性上，盒模型、定位和背景属性通常不继承。常用选择器有标签、类、id、属性选择器，以及后代、子代和兄弟关系选择器，交互状态可用:hover、:checked等伪类表达。像color、font-size、line-height通常会继承，而margin、padding、width、position不会。特殊情况下可以给属性显式设置inherit，强制采用父元素的值。

你列举的CSS选择器是正确的，以下是它们的详细说明：

id选择器(#myid)：通过元素的id属性选择元素，id应该是唯一的。

类选择器(.myclassname)：通过元素的class属性选择元素，一个元素可以有多个类名。

标签选择器(div、h1、p等)：通过元素的标签名选择元素。

相邻选择器(h1

p) : 选择紧接在指定元素后面的兄弟元素。子选择器 (ul > li) : 选择指定元素的直接子元素。后代选择器 (li a) : 选择指定元素的后代元素，可以是子元素、孙子元素等。通配符选择器 (*) : 匹配所有元素。属性选择器 (a[rel="external"]) : 通过元素的属性值选择元素。伪类选择器 (a:hover、li:nth-child等) : 选择特定状态或位置的元素。这些选择器可以单独使用，也可以组合使用，以选择特定的元素或元素组。通过选择器，可以实现对页面中的不同元素进行样式控制和操作。 CSS哪些属性可以继承？哪些属性不可以继承 实际上，继承性并不是根据某个属性是可继承属性还是不可继承属性来决定的。继承性是根据具体的属性规定来决定的。以下是一些常见的可继承属性和不可继承属性的示例： 可继承属性： 文本相关属性：font-family、font-size、font-weight、font-style、line-height、color链接相关属性：text-decoration、cursor盒模型相关属性：margin、padding列表相关属性：list-style-type、list-style-image、list-style-position表格相关属性：border-collapse、border-spacing 不可继承属性： 盒模型相关属性：width、height、border、border-radius、padding、margin定位属性：position、top、right、bottom、left、float背景相关属性：background-color、background-image、background-repeat、background-position、background-size 需要注意的是，即使某个属性是可继承属性，但也可以通过显式设置子元素的样式来覆盖继承的值。另外，有些属性可以通过使用inherit关键字来强制继承父元素的值，即使它本身不具备继承性。 需要注意的是，以上列举的可继承属性和不可继承属性并非全部，具体的属性是否具有继承性需要查阅相关文档进行确认。

99. CSS3新增伪类有那些

CSS3新增伪类主要用于按元素结构和状态选取节点，常见的有:nth-child()、:not()、:checked、:disabled、:target和:empty。结构伪类适合处理列表位置，例如:nth-child(2n)选偶数项，:nth-child(2n+1)选奇数项。状态伪类更适合表单和锚点交互，例如用:checked匹配已选控件。要注意::before、::after、::selection属于伪元素，不应和伪类混为一谈。

你列举的是CSS3新增的一些伪类，它们可以用于选择特定的元素状态或位置。以下是它们的详细说明： :root : 选择文档的根元素，等同于html元素。:empty : 选择没有子元素的元素。:target : 选取当前活动的目标元素。:not(selector) : 选择除selector元素以外的元素。:enabled : 选择可用的表单元素。:disabled : 选择禁用的表单元素。:checked : 选择被选中的表单元素。:after : 在元素内部最前添加内容。:before : 在元素内部最后添加内容。:nth-child(n) : 匹配父元素下指定子元素中排序第n个的元素。:nth-last-child(n) : 匹配父元素下指定子元素中从后向前数排序第n个的元素。:nth-child(odd) : 匹配父元素下指定子元素中奇数位置的元素。:nth-child(even) : 匹配父元素下指定子元素中偶数位置的元素。:nth-child(3n+1) : 匹配父元素下指定子元素中满足3n+1位置的元素。:first-child : 选择父元素下的第一个子元素。:last-child : 选择父元素下的最后一个子元素。:only-child : 选择父元素下唯一的子元素。:nth-of-type(n) : 匹配父元素下指定类型的子元素中排序第n个的元素。:nth-last-of-type(n) : 匹配父元素下指定类型的子元素中从后向前数排序第n个的元素。:nth-of-type(odd) : 匹配父元素下指定类型的子元素中奇数位置的元素。:nth-of-type(even) : 匹配父元素下指定类型的子元素中偶数位置的元素。:nth-of-type(3n+1) : 匹配父元素下指定类型的子元素中满足3n+1位置的元素。:first-of-type : 选择父元素下指定类型的第一个子元素。:last-of-type : 选择父元素下指定类型的最后一个子元素。:only-of-type : 选择父元素下指定类型的唯一子元素。::selection : 选择被用户选取的元素部分。:first-line : 选择元素中的第一行。:first-letter : 选择元素中的第一个字符

100. 如何居中div？如何居中一个浮动元素？如何让绝对定位的div居中

普通div可用定宽配合margin:

0

auto水平居中；浮动和绝对定位元素则需要结合定位偏移处理。浮动元素可以设置position: relative、left: 50%，再用transform: translateX(-50%)拉回自身宽度的一半。绝对定位元素尺寸明确时，可将top、right、bottom、left全设为0，再配合margin: auto居中。如果尺寸未知，通常用top: 50%、left:

50%加transform: translate(-50%, -50%)更合适。要居中一个div，可以使用以下方法：使用margin: 0 auto;将div水平居中。前提是给div设置一个固定的宽度。`div{width:200px;margin:0 auto;}`对于浮动元素，可以使用相对定位和负边距的方式进行居中。`.container{position:relative;}.float-element{float:left;position:relative;left:50%;transform:translateX(-50%);}`对于绝对定位的div，可以通过设置top,left,right,bottom属性为0，并将margin设置为auto来实现居中。`.absolute-element{position:absolute;top:0;left:0;right:0;bottom:0;margin:auto;}`以上是一些常用的居中方法，具体选择哪种方法取决于你的布局需求和元素的类型。

101. 用纯CSS创建一个三角形的原理是什么

纯CSS三角形本质上是利用零宽高元素的边框交界形状，只显示其中一条边框。元素设为width:0和height:0后，把四条边框设得较宽，交界处会形成斜边。再将三条边框设为transparent，保留一条颜色，就能得到对应方向的等腰三角形。这种写法常用于提示框箭头等简单装饰，方向和大小可通过边框颜色、宽度调整。使用纯CSS创建三角形的原理是利用元素的边框属性来实现。具体步骤如下：创建一个具有宽度和高度为0的元素。设置元素的边框宽度为一个较大的值，例如20px。设置元素的边框样式为solid，表示实线边框。通过调整元素的边框颜色，使得三条边中的一条边有颜色，其余两条边颜色为透明，从而形成三角形的形状。在给定的示例中，#demo元素的宽度和高度为0，边框宽度为20px，边框颜色设置为透明、透明、红色和透明，从而形成一个红色的等腰三角形。/* 把上、右、左、三条边隐藏掉（颜色设为transparent）

```
*/#demo{width:0;height:0;border-width:20px;border-style:solid;border-color:transparent transparent red transparent;}
```

102. 一个满屏品字布局如何设计？

满屏“品”字布局可以让容器占满视口，再把三个区域都设为半屏宽、半屏高并依次排列。外层使用width: 100%和height: 100vh，子元素使用width: 50%、height: 50vh。子元素通过float: left或display: inline-block排在同一行，空间不足时会自然换行。具体采用哪种方式，要结合页面是否需要清除浮动以及行内空白处理。要实现一个满屏的“品”字布局，可以采用以下方法：HTML 结构：CSS 样式：`.container{width:100%;height:100vh; /* 或者使用 height: 100% */}.top, .middle, .bottom{width:50%;height:50vh; /* 或者使用 height: 50% */float:left; /* 或者使用 display: inline-block; */}.top{background-color:red; /* 设置上方区域的背景色 */}.middle{background-color:white; /* 设置中间区域的背景色 */}.bottom{background-color:blue; /* 设置下方区域的背景色 */}`这样设置后，上方区域（红色）占据整个屏幕宽度的一半，中间区域（白色）和下方区域（蓝色）各占据屏幕宽度的一半。可以使用float: left;或者display: inline-block;来让它们在同一行显示，不换行。

103. li与li之间有看不见的空白间隔是什么原因引起的？有什么解决办法

当li使用行内排列方式时，标签之间的换行和空格会被解析为文本节点，因此视觉上会出现间隔。这不是元素自身的边距，而是源码中的空白字符参与了渲染。我一般优先把ul设为display:

flex，既能消除这类间隔，也更方便控制排列；旧式布局也可以让li使用float:

left。负margin虽然能抵消空隙，但数值依赖实际样式，维护时要更谨慎。li 与 li

之间的看不见的空白间隔是由于HTML 中的换行符和空格字符造成的。这些空白字符会被解析为文本节点，而文本节点默认会应用一定的样式，导致li 之间产生间隔。

解决这个问题的方法有多种，以下是一些常见的解决办法：使用负margin：可以将li 的margin 设置为负值，例如margin-right: -4px;。这样可以抵消掉li 之间的间隔。

ul{margin:0;padding:0;}li{margin-right:-4px;padding-right:4px;list-style:none;} 使用浮动：给li 添加float: left;属性，使其浮动在一行内，消除间隔。

`ul{margin:0;padding:0;overflow:auto;}li{float:left;list-style:none;}` 使用 Flexbox 布局：将 `ul` 设置为 Flex 容器，使 `li` 自动排列在一行内，间隔消失。`ul{margin:0;padding:0;display:flex;}li{list-style:none;}` 这些方法可以根据具体情况选择使用，根据不同的布局需求选择最合适的解决办法。

104. 请列举几种隐藏元素的方法

隐藏元素常用display: none、visibility: hidden、opacity: 0和移出可视区域几类方式，区别主要在于是否占据布局空间。display: none不占位，而visibility: hidden与opacity: 0都会保留原位置。还可以用绝对定位配合left: -9999px将元素移出屏幕，或用transform: scale(0)、height: 0实现视觉隐藏。实际选择时要先确认后续是否还需要保留该元素的布局位置。visibility: hidden::设置元素为隐藏状态，但仍然占据空间。opacity: 0::将元素的透明度设置为0，使其完全透明。position: absolute;或left: -9999px::将元素定位到屏幕可见区域之外，实现隐藏效果。display: none::将元素设为不可见，并且不占据页面布局空间。transform: scale(0)::将元素的缩放比例设置为0，元素将不可见，但原始位置仍保留。：使用hidden属性来隐藏元素，与display: none效果相同，但用于记录元素的状态。height: 0::将元素的高度设为0，消除边框，使其不可见。filter: blur(0)::使用filter属性将元素的模糊度设置为0，从视觉上使元素“消失”。这些方法可以根据实际需求选择适合的隐藏方式，以达到隐藏元素的效果。

105. rgba() 和 opacity 的透明效果有什么不同

opacity会让元素及其内部内容一起透明，而rgba()只调整指定颜色本身的透明度。比如容器设置opacity: 0.5后，里面的文字和图片也会变成半透明，取值范围是0到1。如果只给背景设置rgba(255, 0, 0, 0.5)，受影响的只是红色背景，子元素内容不会继承这个透明效果。需要背景半透明但文字保持清晰时，我会选择rgba()。opacity作用于元素以及元素内的所有内容（包括文字）的透明度rgba()只作用于元素自身的颜色或其背景色，子元素不会继承透明效果opacity和rgba()在透明效果上有以下不同：opacity：设置元素及其内容的透明度。透明度值范围为 0（完全透明）到 1（完全不透明）。透明度会应用于元素及其内部所有内容，包括文本、图像等。这意味着如果将一个元素的透明度设置为 0.5，那么元素及其内容都会以半透明的形式显示。.element{opacity:0.5;} rgba()：用于设置元素自身的颜色或背景色的透明度。这个函数接受四个参数，前三个参数表示颜色的红、绿、蓝通道的数值（取值范围 0-255），第四个参数表示透明度（取值范围 0-1）。透明度值为 0 表示完全透明，为 1 表示完全不透明。这意味着只有元素自身的颜色或背景色会受到透明度的影响，而元素内部的内容不会继承透明效果。.element{background-color:rgba(255,0,0,0.5);/* 红色背景，透明度为 0.5 */} 总结来说，opacity会影响元素及其内部所有内容的透明度，而rgba()只会影响元素自身的颜色或背景色的透明度，不会影响元素内部的内容。

106. css 属性 content 有什么作用

content主要配合::before和::after伪元素，在元素前后插入生成内容。它既能放字符串或图标，也能通过attr()读取元素属性，还可以用于显示计数等内容。比如给链接前面加箭头，可以写成content: "→"，不用额外增加真实的HTML节点。使用时还要为伪元素设置合适的display，如inline或inline-block。content属性主要用于在::before和::after伪元素中插入额外的内容或样式。通过设置content属性，可以在元素的前面或后面插入指定的内容，包括文本、图标、计数等。它可以接受各种值，如字符串、引用、计数器等。例如，可以使用content属性在元素前面插入一个文本字符：.element::before{content:"→";} 上述代码将在.element元素的前面插入一个箭头字符。此外，content属性还可以与其他 CSS

属性一起使用，例如结合attr()函数来获取元素的属性值并插入到内容中：

```
.element::before{content:attr(data-text);}
```

上述代码会将元素的data-text属性值插入到.element元素的前面作为内容。需要注意的是，content属性只能用于::before和::after伪元素中，并且必须配合display属性设置为inline或inline-block才能生效。

107. 请解释一下 CSS3 的 Flexbox（弹性盒布局模型）以及适用场景

Flexbox是一种弹性盒布局模型，用来灵活地对齐、分布和调整容器中的项目。父容器可以根据可用空间改变子项目的宽高与排列方式，因此内容多少不一致时，也比较容易做出等高或自适应布局。它还支持调整项目顺序，并能方便地实现水平居中、垂直居中和平均分布。实际开发中，它很适合响应式页面和移动端布局，也可以结合媒体查询适配不同屏幕。Flexbox是 CSS3 中引入的一种弹性盒布局模型，用于在容器内创建灵活的、自适应的布局。它提供了一种强大的方式来对齐、分布和调整容器中的项目。

Flexbox的适用场景包括但不限于以下情况：

- 等高布局：Flexbox可以轻松实现容器内多个项目等高的布局，无论项目的内容多少。
- 自适应布局：Flexbox可以根据容器的可用空间自动调整项目的大小，以适应不同尺寸的屏幕或容器。
- 项目排序：Flexbox可以通过改变项目的顺序来实现在不同屏幕尺寸下的布局调整。
- 对齐和分布：Flexbox提供了多种对齐和分布项目的方式，如水平居中、垂直居中、平均分布等。
- 响应式设计：Flexbox可以与媒体查询结合使用，根据屏幕尺寸调整容器中项目的布局方式。

Flexbox的特点包括：

- 父容器具有弹性，可以自动调整项目的大小和顺序。
- 子项目可以具有灵活的宽度、高度和顺序。
- 可以轻松实现响应式设计，适应不同的屏幕尺寸和设备。
- 提供了多种对齐和分布项目的属性和方法，使布局更加灵活和易于控制。

总而言之，Flexbox提供了一种简单、直观且强大的布局方式，适用于构建各种自适应和灵活的布局，特别适合用于构建响应式设计和移动端布局。

108. 经常遇到的浏览器的JS兼容性有哪些？解决方法是什么

常见的JavaScript兼容问题集中在样式读取、事件对象、鼠标坐标、按键码和文本内容这些浏览器接口上。例如读取样式可优先用getComputedStyle()，旧版IE则可能需要currentStyle；事件对象也存在参数e与window.event的差异。处理这类问题时，我一般先做特性检测，再选择对应实现，也可以借助Modernizr、jQuery等兼容性库。浏览器嗅探可以作为识别手段，但代码仍应尽量采用一致的标准和通用方法。

经常遇到的浏览器的 JavaScript 兼容性问题及解决方法如下：

当前样式：获取元素的当前样式属性值
解决方法：使用getComputedStyle(el,

null)方法获取当前样式属性值，或者使用el.currentStyle属性（仅适用于 IE 浏览器）。

事件对象：处理事件时获取事件对象

解决方法：使用e参数获取事件对象，或者使用window.event（仅适用于 IE 浏览器）。

鼠标坐标：获取鼠标在页面上的坐标位置
解决方法：使用e.pageX和e.pageY获取鼠标坐标位置，或者使用window.event.x和window.event.y（仅适用于 IE 浏览器）。

按键码：处理键盘事件时获取按下的键盘按键码

解决方法：使用e.which获取按键码，或者使用event.keyCode（仅适用于 IE 浏览器）。

文本节点：获取元素中的文本内容

解决方法：使用el.textContent获取文本内容，或者使用el.innerText（仅适用于 IE 浏览器）。

为了解决浏览器的兼容性问题，可以采取以下方法：

- 特性检测：通过检测浏览器是否支持某个特定的方法或属性，从而决定使用哪种方式来处理。
- 浏览器嗅探：根据浏览器的用户代理字符串来识别浏览器类型，从而采取相应的处理方式。
- 使用兼容性库：如Modernizr、jQuery等，它们封装了许多兼容性处理的方法，可以简化开发过程。
- 编写跨浏览器的代码：使用一致的标准和通用的 JavaScript 方法，避免依赖特定浏览器的特性。

综上所述，通过特性检测、浏览器嗅探、使用兼容性库以及编写跨浏览器的代码，可以解决常见的浏览器 JavaScript 兼容性问题。 以下是对每个兼容性问题的示例代码：

1. 当前样式：

```

vare=document.getElementById('myElement');// 获取当前样式属性值varcurrentStyle;if(window.ge
tComputedStyle){currentStyle=window.getComputedStyle(el,null).getPropertyValue('color');}elsei
f(el.currentStyle){currentStyle=el.currentStyle.color;}console.log(currentStyle); 2. 事件对象： //
处理点击事件functionhandleClick(e){//
获取事件对象varevent=e||window.event;console.log(event);}绑定点击事件varelement=docume
nt.getElementById('myElement');if(element.addEventListener){element.addEventListener('click',h
andleClick,false);}elseif(element.attachEvent){element.attachEvent('onclick',handleClick);} 3.
鼠标坐标： // 处理鼠标移动事件functionhandleMouseMove(e){// 获取鼠标坐标位置varmouseX,mous
eY;if(e.pageX||e.pageY){mouseX=e.pageX;mouseY=e.pageY;}elseif(e.clientX||e.clientY){mouse
X=e.clientX+document.body.scrollLeft+document.documentElement.scrollLeft;mouseY=e.clientY
+document.body.scrollTop+document.documentElement.scrollTop;}console.log('Mouse position:
'+mouseX+', '+mouseY);}绑定鼠标移动事件varelement=document.getElementById('myElement');i
f(element.addEventListener){element.addEventListe...

```

109. 请写出多种等高布局

等高布局可以用Flexbox、CSS Grid、表格布局、伪元素或脚本计算来实现。常规场景我会优先用display: flex配合align-items:

stretch，代码少，子项会自动拉伸到同一高度。网格布局适合行列关系明确的页面，表格法则通过display: table-cell获得等高效果。遇到特殊结构还可以用绝对定位的伪元素补齐背景，或由脚本取最大高度，但要权衡兼容性和代码复杂度。使用 JavaScript/jQuery：通过脚本计算元素高度，并将最大高度应用于其他元素。使用伪元素和绝对定位：在容器元素中添加一个伪元素，使用绝对定位将其拉伸到与最高元素相同的高度。使用表格布局：将容器元素设置为display: table，将子元素设置为display:

table-cell，然后使用vertical-align: top或其他对齐方式来实现等高效果。使用网格布局（CSS Grid）：使用grid-template-rows属性将所有行设置为相同的高度。使用flexbox嵌套：将容器元素设置为display: flex，然后在每个子元素中再嵌套一个flexbox容器，将子元素的高度设置为100%，从而实现等高布局。css 3 flexbox布局：.container{display: flex; align-items: stretch;}

需要根据具体情况选择适合的等高布局方法，考虑兼容性、布局需求和代码复杂度等因素。

110. 浮动元素引起的问题

浮动元素脱离普通文档流后，容易造成父元素高度塌陷、内容重叠和文字环绕。父级没有计算浮动子项的高度时，后续布局以及父级的背景、边框都可能表现异常；剩余空间不足时，其他元素也可能与它挤在一起。常见处理是在父级末尾用::after生成块级内容，并设置clear:

both。如果某个元素不应该环绕浮动内容，也可以给它设置clear，或改用更合适的布局方式。

浮动元素可能引起以下问题：父元素高度塌陷：当父元素包含了浮动元素时，如果没有清除浮动，父元素的高度将无法被正确计算，导致父元素的高度塌陷，影响页面布局。

解决方法：可以在父元素的末尾添加一个空的块级元素，并设置其清除浮动的属性。例如：

.parent::after{content:"";display:block;clear:both;} 元素重叠：浮动元素脱离了正常的文档流，可能导致其他元素与其重叠。这通常发生在没有足够空间容纳浮动元素的情况下。

解决方法：可以通过在需要避免重叠的元素上应用clear属性或使用适当的布局技术来解决重叠问题。

文字环绕：浮动元素会导致周围的文本环绕在其周围，可能影响页面的可读性和布局。解决方法：可以使用clear属性或使用适当的布局技术来控制浮动元素周围的文本布局，确保页面的可读性和整体布局。

父元素高度不准确：如果父元素包含浮动元素而没有设置清除浮动，父元素的高度可能不准确，可能会导致布局问题。

解决方法：可以在父元素上应用清除浮动的样式，或使用其他布局技术来解决父元素高度不准确的问题。

总的来说，使用浮动元素时，应注意处理相关的布局问题，避免出现高度塌陷、重叠和布局混乱等问题。可以使用清除浮动、适当的布局技术或其他CSS属性来解决这些问题。

111. CSS优化、提高性能的方法有哪些

优化CSS，核心是减少传输和请求成本、降低选择器匹配复杂度，并避免无谓的重排与重绘。加载方面可以合并和压缩样式文件、删除空规则，并把样式放在页面头部，让浏览器尽早开始渲染。代码方面我会控制选择器嵌套深度，利用继承、公共样式和简写属性减少重复。小图标较多时可考虑雪碧图，但具体是否合并资源，还要结合页面结构和维护成本判断。 以下是一些优化和提高CSS性能的方法：

- 合并CSS文件：将多个CSS文件合并为一个文件，减少HTTP请求次数。将CSS文件放在页面最上面：确保CSS文件在HTML文件中的头部，以便浏览器尽早加载和渲染样式。
- 移除空的CSS规则：删除不需要的空的CSS规则，减少文件大小。
- 避免使用CSS表达式：CSS表达式在每次页面重绘时都会执行，对性能有一定影响，应避免使用。
- 优化选择器嵌套：避免过深的选择器嵌套，减少选择器的复杂度，以提高选择器匹配速度。
- 最大限度地利用CSS继承：合理利用CSS属性的继承特性，减少重复的代码量。
- 抽象和提取公共样式：将重复的样式抽象出来，提取为公共样式，减少重复的代码量。
- 使用简写属性：合理使用CSS的简写属性，减少代码量，例如使用margin代替margin-top、margin-right等单独属性。
- 避免不必要的浏览器重绘和重排：在进行DOM操作时，尽量减少对页面布局的影响，以避免不必要的浏览器重绘和重排。
- 使用CSS雪碧图：将多个小的图标合并到一个图像文件中，通过CSS的背景定位来显示不同的图标，减少HTTP请求次数。
- 使用压缩和缩小CSS文件：使用CSS压缩工具，将CSS文件进行压缩和缩小，减少文件大小，提高加载速度。

这些方法可以帮助优化和提高CSS性能，减少文件大小和HTTP请求次数，加快页面加载速度，提升用户体验。

112. 浏览器是怎样解析CSS选择器的

浏览器通常从选择器最右侧开始匹配，再逐步向左检查祖先元素是否满足条件。例如.nav li a会先找到符合a的元素，再向上验证是否存在匹配的li和.nav。这样能较早排除不匹配的节点，面对复杂选择器和大量元素时，可以减少后续检查范围。属性选择器、伪类等情况可能采用不同的优化方式，所以工程上更重要的是避免选择器过深、过于复杂。 浏览器解析CSS选择器的过程是从右到左进行的。当浏览器遇到一个CSS规则时，它会从选择器的最右边开始解析，然后逐步向左匹配元素。 具体的解析过程如下：

- 浏览器从右向左选择器的最右边开始解析，找到与该选择器匹配的元素。
- 然后，浏览器向左继续解析选择器的下一个部分，检查该元素的父元素是否满足选择器的条件。
- 如果满足选择器的条件，则继续向左解析选择器的下一个部分，继续检查父元素的父元素是否满足选择器的条件。
- 浏览器重复这个过程，直到解析完整整个选择器或者找不到与选择器匹配的元素。

这种从右到左的解析方式可以提高选择器的性能，特别是在面对复杂的选择器和大量元素的情况下。因为从右到左的解析可以尽早过滤掉不匹配的元素，减少了需要遍历的元素数量，提高了选择器的匹配速度。 需要注意的是，并非所有的CSS选择器都适用于从右到左的解析方式，例如属性选择器、伪类选择器等可能需要从左到右进行解析。浏览器在解析CSS选择器时会根据具体的情况选择最优的解析方式。

113. 在网页中的应该使用奇数还是偶数的字体

网页字号一般优先使用偶数，但不必把它当成硬性规则。偶数字号更容易和页面里的尺寸形成稳定的比例关系，文本对齐时也更方便。部分浏览器和操作系统对奇数字号的呈现可能存在差异，因此中文排版常见12px、14px。最终仍要结合设计需求、内容密度和可读性选择，视觉效果合适时也可以使用奇数字号。

在网页中，一般推荐使用偶数号的字体大小。以下是一些原因：

- 比例关系：偶数字号相对更容易与网页设计的其他部分形成比例关系，使整个页面看起来更加协调和平衡。
- 对齐：使用偶数字号字体可以使文本段落对齐更加方便。如果使用奇数字号字体，文本段落的对齐可能会受到影响，因为奇数字号字体的基线位置可能会导致对齐不准确。
- 兼容性：在某些情况下，奇数字号字体可能在不同浏览器和操作系统中显示不一致。使用偶数字号字体可以减少这种显示差异的可能性。

对于中文网页排版，常用的字体大小是12号和14号。

4号。这两个偶数字号字体大小在中文排版中使用较多，可以提供良好的可读性和视觉效果。
需要注意的是，具体选择何种字体大小还要根据设计需求、内容呈现和用户体验等因素进行综合考虑。

114. margin和padding分别适合什么场景使用

margin适合控制元素与外部的距离，padding适合控制内容与元素边框的距离。想拉开两个模块，或者调整元素相对周围内容的位置时，我一般使用margin。如果要给文字和边框留出空间，或让背景色、背景图覆盖这块空白区域，就使用padding。两者有时看起来可以互换，但是否需要背景填充、是否要影响元素自身尺寸，通常是选择的关键。 margin和padding在布局和样式设计中有不同的用途和适用场景。适合使用 margin 的场景： 创建元素之间的空白间距，用于调整元素之间的间隔。添加外部空白，使元素与周围的元素或容器之间产生间距。用于调整元素的定位和对齐。用于调整元素的外边距折叠效果（当相邻元素的外边距相遇时）。 适合使用 padding 的场景： 在元素的内部添加空白区域，用于调整元素内部内容与边框之间的距离。用于创建元素的背景色或背景图的填充区域。用于调整元素的内边距，影响元素内容与边框的距离。用于控制元素的尺寸和布局。 需要根据具体的设计需求和布局目标来决定使用margin还是padding。在一些情况下，它们可以互相替代使用，但在其他情况下，选择正确的属性可以更好地控制布局和样式效果。

115. 抽离样式模块怎么写，说出思路

抽离样式模块的核心，是把公共规则集中复用，再让各业务模块只维护自己的样式。我会先提取颜色、字体、按钮等公共样式，然后按页面结构拆成header.css、sidebar.css这类独立文件。命名上统一采用BEM等约定，并控制文件引入顺序，减少选择器冲突和意外覆盖。项目构建时还可以通过webpack或gulp合并、压缩和打包，后续需求变化也只需更新对应模块。 抽离样式模块的思路可以按照以下步骤进行： 识别公共样式：分析页面中多个模块或页面之间共享的样式，例如颜色、字体、按钮样式等，将其提取为公共样式。创建公共样式表：将公共样式抽离到单独的样式表文件中，例如common.css，以便在多个页面或模块中引用。定义命名规范：为业务样式定义统一的命名规范，例如使用 BEM（Block-Element-Modifier）或其他命名约定，确保样式模块的可维护性和可扩展性。拆分业务样式：根据页面或模块的功能和结构，将样式拆分为多个模块，并命名为相应的类名或选择器。每个模块应只关注自身的样式，并尽量避免与其他模块产生冲突。创建样式模块文件：为每个样式模块创建单独的样式文件，例如header.css、sidebar.css等，并按需引入到页面中。构建样式层级结构：根据页面或模块的结构和层级关系，合理组织样式文件的引入顺序，确保样式的层叠顺序和继承关系正确。统一管理样式文件：根据项目需要，可以使用构建工具（如 webpack、gulp 等）进行样式文件的合并、压缩和打包，减少网络请求和提升页面加载速度。维护和更新：在日常开发中，根据需求的变化和新功能的添加，及时更新和维护样式模块，保持样式的一致性和可复用性。 通过以上步骤，可以将样式按照模块化的方式进行抽离和管理，提高代码的可读性、可维护性和重用性。同时，样式模块的拆分也有助于团队协作和并行开发。

116. 元素竖向的百分比设定是相对于容器的高度吗

如果说的是元素的百分比高度，那么它通常相对于包含它的父元素高度计算。例如设置height: 50%，表示元素高度取父元素高度的一半。关键边界是父元素必须有可确定的高度；如果父元素保持height: auto，百分比高度可能无法生效。作为对比，百分比宽度按照父元素宽度计算，所以横向和竖向使用的参照尺寸并不相同。 实际上，元素竖向的百分比设定是相对于包含它的父元素的高度，而不是宽度。当给一个元素设置竖向的百分比高度时，它会根据其父元素的高度进行计算。例如，如果一个元素的高度设置为50%，则表示该元素的高度将是其父元素高度的50%。

请注意，如果父元素没有明确设置高度，或者父元素的高度是由其内容决定的（如默认的height: auto），那么百分比高度可能无效，因为无法确定相对的基准高度。 需要注意的是，元素的宽度百分比设定是相对于包含它的父元素的宽度。因此，元素的百分比设定在竖向和横向方向上是不同的。

117. 全屏滚动的原理是什么？用到了CSS的那些属性

全屏滚动本质上是让每次滚动切换一整屏内容，而不是让页面连续移动。页面会监听滚轮或触摸事件，再根据滚动距离决定显示哪一屏。样式上通常用overflow: hidden隐藏视口外内容，并通过transform: translate()移动页面，配合display:

none控制内容显隐。具体采用整体位移还是逐屏切换，要根据页面结构和动画需求决定。全屏滚动（Full Page Scroll）的原理是通过设置页面的高度为视口高度，并使用滚动事件来控制页面的滚动效果。通过监听滚动事件，当用户滚动页面时，根据滚动的距离来切换页面的显示内容，实现页面的切换效果。

在实现全屏滚动时，可能会用到以下一些CSS属性：

overflow: hidden; 用于隐藏超出视口范围的内容，以实现滚动效果。transform: translate(100%, 100%); 通过translate属性将页面移动到视口之外，从而隐藏页面的初始位置。display:

none; 可以将页面的初始状态设置为隐藏，待滚动到相应位置时再显示。这些属性可以结合使用，根据滚动事件的触发来控制页面的显示和隐藏，从而实现全屏滚动的效果。具体的实现方式可能因应用场景而有所差异。

118. 什么是响应式设计？响应式设计的基本原理是什么？如何兼容低版本的IE

响应式设计是让同一套网站适配不同尺寸的终端，不必为每种设备单独开发一个版本。它主要依靠CSS3媒体查询，在不同屏幕宽度下应用对应的布局和样式。低版本IE不支持相关能力时，可以用JavaScript获取窗口宽度，并在resize事件中切换body的类名。这个方案需要自己维护宽度区间，因此断点较多时会增加样式和脚本的维护成本。响应式设计就是网站能够兼容多个终端，而不是为每个终端做一个特定的版本基本原理是利用CSS3媒体查询，为不同尺寸的设备适配不同样式对于低版本的IE，可采用JS获取屏幕宽度，然后通过resize方法来实现兼容：

```
$(window).resize(function(){screenRespond();});screenRespond();functionscreenRespond(){varscreenWidth=$(window).width();if(screenWidth<800){$("body").attr("class","");}}
```

119. 什么是视差滚动效果，如何给每页做不同的动画

视差滚动是让背景层、内容层等元素随页面滚动以不同速度移动，从而形成层次感和立体效果。实现时会监听滚轮或触摸事件，把滚动距离和方向换算成各层的位移或缩放值，再通过transform或动画库更新画面。每页可以分别配置淡入淡出、移动、旋转等效果，并按照滚动进度控制播放顺序和速度。动画过多会影响滚动流畅度，所以需要控制复杂度，并结合节流、防抖或硬件加速做优化。视差滚动效果是一种在网页中使用的动画效果，通过在滚动页面时，不同层级的元素以不同的速度移动，形成立体的运动效果。这种效果可以给用户带来更加丰富、生动的视觉体验。实现视差滚动效果时，可以将页面划分为背景层、内容层和悬浮层等不同的层级。通过设置不同层级的元素以不同的速度移动，可以形成层次感和立体效果。具体实现视差滚动效果的方法如下：使用HTML和CSS创建页面的不同层级，如背景层、内容层和悬浮层。监听滚动事件（如鼠标滚轮事件或触摸滑动事件）。当滚动事件触发时，根据滚动的距离和速度，计算出不同层级元素的位移值。使用CSS的transform属性或JavaScript的动画库（如GSAP、ScrollMagic等）来实现元素的平移或缩放效果。根据需求，为每个页面或元素设置不同的动画效果，如淡入淡出、旋转、缩放等。根据滚动的进度和方向，控制元素的动画播放顺序和速度，以达到预期的视差滚动效果。需要注意的是，在实现视差滚动效果时，要考虑到性能和用户体验。过多或复杂的动画效果可能会影响页面加载和滚动的流畅性，因此需要谨慎选择和设计动画效果，并进行性能优化，如合理使用硬件加速、使用节流和防抖等技术手段。另外，为每个页面或元素设置不同的动画效果可以根据具体的设计需求来确定。可以根据页面的主题、内容或目的来决定使用何种动画效果，如淡入淡出效果、元素的移动或旋转效果等，以增加页面的吸引力和交互性。可以通过CSS的animation属性或JavaScript的动画库来实现这些动画效果，并根据滚动进度或其他触发条件来控制动画的播放。总结 视差滚动是指多层背景以不同的速度移动，形成立

体的运动效果，具有非常出色的视觉体验一般把网页解剖为：背景层、内容层和悬浮层。当滚动鼠标滚轮时，各图层以不同速度移动，形成视差的实现原理以“页面滚动条”作为“视差动画进度条”以“滚轮刻度”当作“动画帧度”去播放动画的监听mousewheel事件，事件被触发即播放动画，实现“翻页”效果以“页面滚动条”作为“视差动画进度条”以“滚轮刻度”当作“动画帧度”去播放动画的监听mousewheel事件，事件被触发即播放动画，实现“翻页”效果

120. a标签上四个伪类的执行顺序是怎么样的

a标签四个伪类通常按:link、:visited、:hover、:active的顺序书写。它们分别对应未访问、已访问、鼠标悬停和正在激活的链接状态，可以用L-V-H-A，也就是“LoveHate”来记。保持这个顺序，后面的交互状态才能在需要时覆盖前面的基础状态。实际编写时还要注意选择器权重，避免其他样式干扰覆盖结果。伪类在a标签上的执行顺序是link（未访问链接）->visited（已访问链接）->hover（鼠标悬停）->active（激活状态）。执行顺序可以用记忆口诀“L-V-H-A”（Love Hate）来记忆，表示喜欢和讨厌的顺序。首先应用link样式，然后是visited样式，接着是hover样式，最后是active样式。这个顺序也是CSS解析和应用伪类样式的规定顺序。

121. 伪元素和伪类的区别和作用

伪类给已有元素添加特定状态下的样式，伪元素则用于表现文档中并不存在的额外内容或局部结构。比如a: hover和p:first-child都是在匹配现有元素，不会创建新元素。p::before、p::after可以在内容前后插入展示内容，p::first-line、p::first-letter则能单独修饰首行或首字母。简单来说，需要描述元素状态时用伪类，需要补充视觉内容或选中局部结构时用伪元素。伪元素 -- 在内容元素的前后插入额外的元素或样式，但是这些元素实际上并不在文档中生成。它们只在外部显示可见，但不会在文档的源代码中找到它们，因此，称为“伪”元素。例如：

```
p::before{content:"第一章:"};p::after{content:"Hot!"};p::first-line{background:red};p::first-letter{font-size:30px;}
```

伪类 -- 将特殊的效果添加到特定选择器上。它是已有元素上添加类别的，不会产生新的元素。例如：

```
a:hover{color:#FF00FF}p:first-child{color:red}
```

122. ::before 和 :after 中双冒号和单冒号有什么区别

双冒号是CSS3为伪元素规定的新写法，单冒号则是早期CSS沿用下来的旧语法。伪类一直使用单冒号，例如:hover和:active；改用::before、::after，是为了从语法上把伪元素与伪类区分开。由于低版本IE不兼容双冒号，老代码常用:before、:after来照顾兼容性。实际开发中是否保留单冒号，主要取决于项目需要支持的浏览器范围。在CSS中伪类一直用:表示，如:hover,:active等伪元素在CSS1中已存在，当时语法是用:表示，如:before和:after后来在CSS3中修订，伪元素用::表示，如::before和::after，以此区分伪元素和伪类由于低版本IE对双冒号不兼容，开发者为了兼容性各浏览器，继续使用:after这种老语法表示伪元素总结起来，::before是CSS3中写伪元素的新语法，而:after是早期版本CSS中存在的、兼容IE的旧语法，用于表示伪元素。在实际开发中，为了兼容性考虑，可以选择使用单冒号的写法。

123. 如何修改Chrome记住密码后自动填充表单的黄色背景

可以针对Chrome自动填充输入框时添加的input:-webkit-autofill状态覆盖背景，或者直接关闭表单自动填充。黄色背景来自浏览器为自动填充表单附加的私有样式，因此普通输入框规则未必能覆盖它。需要保留自动填充时，可设置{-webkit-autofill { background-color: transparent; }}；不需要该能力时，可在form上设置autocomplete="off"。前者保留使用便利，后者处理更直接，应按表单场景取舍。产生原因：由于Chrome默认会给自动填充的input表单加上input:-webkit-autofill私有属性造成的解决方案1：在form

标签上直接关闭了表单的自动填充：autocomplete="off"解决方案2：input:-webkit-autofill { background-color: transparent; } input [type=search] 搜索框右侧小图标如何美化？ input[type="search"]::-webkit-search-cancel-button{-webkit-appearance:none;height:15px;width:15px;border-radius:8px;background:url("images/searchicon.png")no-repeat 0 0;background-size:15px 15px;}

124. 网站图片文件，如何点击下载？而非点击预览

可以给指向图片的标签添加download属性，让浏览器点击后优先下载而不是预览。例如下载会下载该图片，也可以写成download="网站LOGO"来指定保存时使用的文件名。这个方案依赖浏览器对download的支持，部分浏览器存在限制，点击后仍可能打开预览。如果网站启用了内容安全策略CSP，还要确认相关配置允许文件下载。可以通过在标签中添加href属性来指定图片文件的路径，并使用download属性来指示浏览器下载该文件而非预览。

上述代码会在网页中显示一个链接，点击该链接会下载名为logo.jpg的图片文件。

您还可以通过添加download属性的值来指定下载文件的名称，如下所示：

上述代码会下载名为网站LOGO.jpg的图片文件。请注意，download属性在某些浏览器中可能不被支持或具有限制。在不支持该属性的浏览器中，点击链接可能仍然会打开预览。此外，如果您在网站中使用了内容安全策略（Content Security Policy），可能需要额外配置才能允许下载文件。确保文件路径正确，并根据需要设置适当的下载文件名称。

125. 你对 line-height 是如何理解的

line-height表示一行文字占用的高度，它同时包含字体实际高度和上下行间距。容器没有显式设置height时，文本产生的行框会把容器撑开，因此调整行高也会影响整体布局。单行文本场景下，把line-height设成与容器高度相同，通常可以实现垂直居中。多行文本则更适合用它控制行间距，不能简单套用单行居中的做法。line-height是一行字的高度，包括了字体的实际高度以及行间距（字间距）。当一个元素没有显式设置height属性时，它的高度会由line-height决定。如果一个容器没有设置高度，并且容器内部有文本内容，那么容器的高度会由line-height撑开。如果将line-height设置为与容器的height相同的值，可以实现单行文本的垂直居中。注意，line-height和height都可以撑开元素的高度，但是设置height属性会触发元素的haslayout（仅适用于部分浏览器），而line-height不会触发该特性。需要注意的是，line-height还可以影响多行文本的行间距和垂直居中，它是一个很常用的属性用于调整文本的排版和布局。

126. line-height 三种赋值方式有何区别？（带单位、纯数字、百分比）

三种写法的核心区别，在于后代继承的是具体行高，还是可以按自身字号计算的比例。px直接指定固定值，em和百分比会先结合当前元素的font-size算出行高，再把计算值传给后代。纯数字传递的是比例，例如子元素字号为18px、继承1.5时，行高就是27px。因此字号可能变化的正文区域，我一般更倾向纯数字，固定对齐时再考虑带单位的值。对于line-height的三种赋值方式，如下所述：带单位：使用像素（px）或其他单位（如em）进行赋值。当使用固定值（如px）时，line-height会直接采用该固定值作为行高。而当使用相对单位（如em）时，line-height会根据元素的父元素的字体大小（font-size）来计算行高，即乘以相应的倍数。纯数字：直接使用数字进行赋值。这种情况下，数字会被传递给后代元素，作为其行高的比例因子。例如，如果父元素的行高为1.5，而子元素的字体大小为18px，那么子元素的行高就会被计算为1.5 * 18 = 27px。百分比：使用百分比进行赋值。百分比值会相对于父元素的字体大小进行计算，并将计算后的值传递给后代元素作为其行高。总的来说，带单位的方式是直接指定具体的行高值，纯数字和百分比的方式会将计算后的行高值传递给后代元素。这些不同的赋值方式可以根据具体的需求和设计效果来选择使用。

127. 设置元素浮动后，该元素的 display 值会如何变化

元素设置浮动后，不能简单理解为所有display声明都会被改成block，但它会生成块级框并脱离正常文档流。对span、a这类默认内联元素来说，浮动后会表现为块级元素，可以设置宽高。其他元素仍要结合原本或显式声明的display判断，不能一概而论。浮动后的内容还会允许周围内容环绕，因此布局时通常也要关注它对兄弟元素的影响。设置元素浮动后，元素的display值并不会自动变成block，而是保持原有的display值。浮动元素的display值仍然是根据元素的默认样式或通过CSS显式设置的值。然而，浮动元素会生成一个块级框，并且脱离了正常的文档流，会影响其他元素的布局。常见的浮动值为left或right，使元素向左或向右浮动，并允许其他内容环绕在其周围。需要注意的是，对于一些内联元素，如span、a等，默认的display值是inline，当设置这些内联元素为浮动时，会自动转换为block，但这并不适用于所有元素。

128. 让页面里的字体变清晰，变细用CSS怎么做？（iOS手机浏览器字体齿轮设置）

在iOS Safari中，可以设置-webkit-font-smoothing: antialiased，让字体显示得更清晰、细腻一些。这个属性控制的是字体平滑效果，可以写在body上统一应用，也可以只加到需要调整的局部元素上。它属于特定浏览器相关的非标准属性，并不能保证在其他浏览器中产生相同效果。因此我会把它当作iOS端的显示优化，而不是跨平台字体粗细控制方案。通过设置-webkit-font-smoothing属性为antialiased可以在iOS手机浏览器中使字体显示更清晰、更细腻。这个属性是针对iOS Safari浏览器的特定设置。

```
body{-webkit-font-smoothing:antialiased;}
```

将上述代码应用于你的CSS文件中，或者将其添加到你想要应用字体设置的元素的样式中，可以改善字体在iOS手机浏览器中的显示效果。注意，该属性只在iOS Safari浏览器中生效，在其他浏览器中可能没有效果。

129. font-style 属性 oblique 是什么意思

font-style: oblique表示让正常字形向右倾斜，以模拟斜体效果。它和italic的区别是，italic通常使用字体设计师专门制作的斜体字形，而oblique只是倾斜原有字形。当字体没有提供专门的斜体时，可以用它获得类似效果。因为这种机械倾斜不一定自然，排版要求较高时应优先选择带真正斜体字形的字体。font-style: oblique;是用来设置字体样式为倾斜（oblique）的。当字体本身没有提供italic斜体样式时，可以使用oblique属性来模拟倾斜效果。倾斜样式是一种字体样式，使字体呈现为向右倾斜的外观，类似于斜体（italic）样式。不同之处在于，斜体样式是由字体设计师提供的专门的斜体字形，而倾斜样式是通过将正常字形倾斜来模拟出来的。使用font-style: oblique;可以在没有专门提供斜体字形的字体上实现倾斜效果，但需要注意的是，由于是通过倾斜正常字形来模拟，所以结果可能不如专门设计的斜体字形效果好。

130. display:inline-block 什么时候会显示间隙

相邻的inline-block元素之间只要保留了换行或空格，页面上通常就会出现水平间隙。本质上这些源码空白会作为文本空白参与行内排版，间隙大小还会受到父元素字号影响。常见处理是让标签紧挨着写，或者把父元素设为font-size: 0，再给予元素恢复字号。另外它默认按基线对齐，底部可能留下垂直空隙，此时可设置vertical-align: top。

display: inline-block;可能会在元素之间产生间隙的情况包括：相邻的inline-block元素之间有换行或空格分隔时，会产生间隙。非inline-block的水平元素设置为inline-block时，会有默认的水平间距。默认情况下，inline-block元素的默认对齐方式是基线对齐，而不是顶部对齐，因此可能会产生垂直间隙。可以通过设置vertical-align: top;将元素顶部对齐来消除垂直间隙。父元素的字体大小会影响inline-block元素的间隙。如果父元素设置了字体大小，可以将其设置为font-size: 0;，然后在inline-block元素内部设置所需的字体大小，以消除垂直间隙。将多个li标签写在同一行，可以消除垂直间隙，但这会导致代码可读性差。这些方法可以用来解决inline-block元素之间产生的间隙问题。

131.

一个高度自适应的div，里面有两个div，一个高度100px，希望另一个填满剩下的高度

我一般会把父容器设为纵向Flex布局，固定第一个子元素为100px，再给另一个设置flex:

1 填满剩余高度。这种写法会由布局系统自动分配空间，不需要手算尺寸。也可以用height: calc(100% - 100px)，但百分比生效需要父容器具备可计算的高度。如果布局依赖定位，则可以让父元素相对定位，并给第二个元素设置position: absolute、top: 100px和bottom: 0。方案1: .sub { height: calc(100%-100px); }方案2: .container { position:relative; }.sub { position: absolute; top: 100px; bottom: 0; }方案3: .container { display:flex; flex-direction:column; }.sub { flex:1; } .sub { height: calc(100%-100px); }.container { position:relative; }.sub { position: absolute; top: 100px; bottom: 0; } .container { display:flex; flex-direction:column; }.sub { flex:1; } 这三种方案都可以实现一个高度自适应的div，其中一个子div高度为100px，另一个子div填满剩下的高度。

方案1使用了calc()函数来计算剩余高度，.sub的高度设置为calc(100% - 100px)，即剩余高度。方案2使用了相对定位和绝对定位，父容器.container设置为相对定位，子div.sub设置为绝对定位，设置top:

100px让其距离顶部100px，设置bottom:

0让其底部与父容器底部对齐。方案3使用了弹性布局（Flexbox），父容器.container设置为display:

flex，并指定flex-direction:

column使其垂直排列。子div.sub使用flex:

1来填充剩余的空间，自动调整高度。

这些方案都可以实现相应的效果，具体选择哪种方案取决于实际需求和布局结构。

132. css 的渲染层合成是什么 浏览器如何创建新的渲染层

渲染层合成是浏览器把处于不同层叠空间的渲染对象分层，再按正确顺序组合显示的过程。这样才能正确处理元素重叠、透明度和混合效果，合成顺序不对就可能出现显示异常。浏览器可能因定位属性、透明度或混合模式，以及3D

transform、filter等视觉效果创建新层。animation、transition、遮罩或backface-visibility:

hidden等也可能触发分层，具体是否创建仍由浏览器结合页面情况决定。在 DOM

树中每个节点都会对应一个渲染对象（RenderObject），当它们的渲染对象处于相同的坐标空间（z轴空间）时，就会形成一个 RenderLayers，也就是渲染层。渲染层将保证页面元素以正确的顺序堆叠，这时候就会出现层合成（composite），从而正确处理透明元素和重叠元素的显示。对于有位置重叠的元素的页面，这个过程尤其重要，因为一旦图层的合并顺序出错，将会导致元素显示异常在 CSS中，渲染层合成（Layer

Composition）是浏览器中用于处理页面元素的显示和堆叠顺序的机制。它通过创建新的渲染层（Render Layer）来管理页面中的元素，并确保它们以正确的顺序进行渲染和呈现。

浏览器创建新的渲染层的条件包括但不限于以下情况：定位属性：元素具有position: fixed、position: relative、position:

absolute等定位属性时，会创建新的渲染层。透明度和混合模式：元素的透明度设置小于 1（opacity: 0.5）或具有非normal的混合模式（mix-blend-mode: multiply）时，会创建新的渲染层。3D

变换：元素具有 3D 变换（transform:

translateZ(0)）时，会创建新的渲染层。滤镜效果：元素应用了滤镜效果（filter:

blur(5px)）时，会创建新的渲染层。遮罩效果：元素应用了遮罩效果（mask-image:

url(mask.png)）时，会创建新的渲染层。CSS 动画：元素正在进行 CSS 动画（animation、transition）

时，会创建新的渲染层。其他因素：还有一些其他因素，如元素应用了backface-visibility:

hidden、column-count不为auto等，也可能触发创建新的渲染层。创建新的渲染层有助于确保页面元素

以正确的顺序进行堆叠和渲染，并处理透明元素和重叠元素的显示。这对于具有位置重叠的页面元素尤为重要，因为合成层的顺序错误可能导致元素显示异常。通过创建新的渲染层，浏览器可以更好地管理元素

的渲染和呈现，提高页面的性能和用户体验。---

【JavaScript相关】

133. 闭包

闭包是函数与其定义时词法环境的组合，让函数离开原作用域后仍能访问外层变量。比如防抖函数返回的回调会一直引用timer，从而在多次调用之间保存状态。它也适合封装私有变量、实现缓存；循环异步回调则可用let创建每轮独立绑定，ES5中通常借助IIFE。闭包会延长被引用变量的生命周期，因此不用时应解除引用，避免不必要的内存占用。闭包就是能够读取其他函数内部变量的函数闭包是指有权访问另一个函数作用域中变量的函数，创建闭包的最常见的方式就是在一个函数内创建另一个函数，通过另一个函数访问这个函数的局部变量,利用闭包可以突破作用链域闭包的特性：函数内再嵌套函数内部函数可以引用外层的参数和变量参数和变量不会被垃圾回收机制回收闭包就是能够读取其他函数内部变量的函数闭包是指有权访问另一个函数作用域中变量的函数，创建闭包的最常见的方式就是在一个函数内创建另一个函数，通过另一个函数访问这个函数的局部变量,利用闭包可以突破作用链域闭包的特性：

函数内再嵌套函数内部函数可以引用外层的参数和变量参数和变量不会被垃圾回收机制回收

说说你对闭包的理解 使用闭包主要是为了设计私有的方法和变量。闭包的优点是可以避免全局变量的污染，缺点是闭包会常驻内存，会增大内存使用量，使用不当很容易造成内存泄露。在js中，函数即闭包，只有函数才会产生作用域的概念闭包的最大用处有两个，一个是可以读取函数内部的变量，另一个就是让这些变量始终保持在内存中闭包的另一个用处，是封装对象的私有属性和私有方法好处：能够实现封装和缓存等；坏处：就是消耗内存、不正当使用会造成内存溢出的问题 使用闭包主要是为了设计私有的方法和变量。闭包的优点是可以避免全局变量的污染，缺点是闭包会常驻内存，会增大内存使用量，使用不当很容易造成内存泄露。在js中，函数即闭包，只有函数才会产生作用域的概念闭包

的最大用处有两个，一个是可以读取函数内部的变量，另一个就是让这些变量始终保持在内存中

闭包的另一个用处，是封装对象的私有属性和私有方法

好处：能够实现封装和缓存等；

坏处：就是消耗内存、不正当使用会造成内存溢出的问题 使用闭包的注意点 由于闭包会使得函数中的变量都被保存在内存中，内存消耗很大，所以不能滥用闭包，否则会造成网页的性能问题，在IE中可能导致内存泄露解决方法是，在退出函数之前，将不使用的局部变量全部删除 举出闭包实际场景运用的例子

比如常见的防抖节流 // 防抖functiondebounce(fn,delay=300){lettimer;//闭包引用的外界变量returnfunction(){constargs=arguments;if(timer){clearTimeout(timer);}timer=setTimeout(()=>{fn.apply(this,args);},delay);}} 使用闭包可以在JavaScript中模拟块级作用域 functionoutputNumbers(count){(function(){for(vari=0;i<count;i++){alert(i);})();alert(i);//导致一个错误！} 闭包可以用于在对象中创建私有变量 varaaa=(function(){vara=1;functionbbb(){a++;console.log(a);}functionccc(){a++;console.log(a);}return{b:bbb,//json结构c:ccc;}})();console.log(aaa.a);//undefinedaaa.b();//2aaa.c();//3

134. 说说你对作用域链的理解

作用域链就是变量和函数的查找路径：先查当前作用域，找不到再逐层向外，直到全局作用域。它由函数定义位置决定，调用函数时会创建新的执行环境并放到查找链前端，所以内部函数能访问外层变量，外层却不能反向访问内部变量。闭包本质上保留了这条链，使外层变量在函数执行结束后仍可能存活。要注意，作用域链解决变量可见性，不能和查找对象属性的原型链混为一谈。作用域链是一种用于查找变量和函数的机制，它是由当前执行环境和其所有父级执行环境的变量对象组成的链式结构。当在一个执行环境中访问变量或函数时，会首先在当前执行环境的变量对象中查找，如果找不到，则会沿着作用域链向上查找，直到找到对应的变量或函数，或者达到最外层的全局对象（如window）。作用域链的创建是在函数定义时确定的，它与函数的定义位置有关。当函数被调用时，会创建一个新的执行环境，其中包含一个新的变量对象，并将其添加到作用域链的前端。这样，函数内部就可以访问其所在作用域以及其外部作用域中的

变量和函数，形成了一个作用域链。

```
function outer(){var outerVar='Outer variable'; console.log(innerVar); //
```

外部作用域的变量 console.log(globalVar); //

在上述示例中，函数 inner() 内部可以访问到其外部函数 outer() 中定义的变量 outerVar，这是因为 inner() 的作用域链中包含了外部函数的变量对象。同样，inner() 也可以访问全局作用域中的变量 globalVar，因为全局作用域也在作用域链中。通过作用域链的机制，函数可以访问外部作用域中的变量，但外部作用域不能访问函数内部的变量，这就实现了变量的封装和保护。值得注意的是，当函数执行完毕后，其执行环境会被销毁，对应的变量对象也会被释放，因此作用域链也随之消失。这也是闭包的概念中所提到的保持变量的生命周期的特性。总结 作用域链的作用是保证执行环境里有权访问的变量和函数是有序的，作用域链的变量只能向上访问，变量访问到 window 对象即被终止，作用域链向下访问变量是不被允许的简单的说，作用域就是变量与函数的可访问范围，即作用域控制着变量与函数的可见性和生命周期

以下是一个示例，展示了作用域链的工作原理：

```
variable'; function inner(){var innerVar='Inner
```

```
内部作用域的变量 console.log(outerVar); //
```

```
全局作用域的变量} inner(); } var globalVar='Global
```

135. JavaScript 原型，原型链？有什么特点？

原型是对象共享属性和方法的关联对象，原型链则是属性不存在时沿原型逐层查找的路径。构造函数的 prototype 会成为其所创建实例的原型，因此通常有 instance.__proto__ === Constructor.prototype，而原型上的 constructor 又指回构造函数。查找会从对象自身开始，一直向上直到找到目标；到链尾仍未找到时得到 undefined。由于原型通过引用共享，修改原型会影响所有关联实例，这既方便复用，也需要避免意外污染。每个对象都会在其内部初始化一个属性，就是 __proto__，当我们访问一个对象的属性时如果这个对象内部不存在这个属性，那么他就会去 __proto__ 里找这个属性，这个 __proto__ 又会有自己的 __proto__，于是就这样一直找下去，也就是我们平时所说的原型链的概念。按照标准，__proto__ 是不对外公开的，也就是说是个私有属性关系：instance.constructor.prototype == instance.__proto__ //

eg. var a={}; a.constructor.prototype==a.__proto__ 特点：JavaScript 对象是通过引用来传递的，我们创建的每个新对象实体中并没有一份属于自己的原型副本。当我们修改原型时，与之相关的对象也会继承这一改变

当我们需要一个属性的时，JavaScript 引擎会先看当前对象中是否有这个属性，如果没有的就会查找他的 Prototype 对象是否有这个属性，如此递推下去，一直检索到 Object 内建对象原型：JavaScript 的所有对象中都包含了一个 [__proto__] 内部属性，这个属性所对应的就是该对象的原型 JavaScript 的函数对象，除了原型 [__proto__] 之外，还预置了 prototype 属性当函数对象作为构造函数创建实例时，该 prototype 属性值将被作为实例对象的原型 [__proto__]。原型链：当一个对象调用的属性/方法自身不存在时，就会去自己 [__proto__] 关联的前辈 prototype 对象上去找如果没找到，就会去该 prototype 原型 [__proto__] 关联的前辈 prototype 去找。依次类推，直到找到属性/方法或 undefined 为止。从而形成了所谓的“原型链”

原型特点：JavaScript 对象是通过引用来传递的，当修改原型时，与之相关的对象也会继承这一改变

特点：JavaScript 对象是通过引用来传递的，我们创建的每个新对象实体中并没有一份属于自己的原型副本。当我们修改原型时，与之相关的对象也会继承这一改变

当我们需要一个属性的时，JavaScript 引擎会先看当前对象中是否有这个属性，如果没有的

就会查找他的 Prototype 对象是否有这个属性，如此递推下去，一直检索到 Object 内建对象 原型：JavaScript 的所有对象中都包含了一个 [__proto__] 内部属性，这个属性所对应的就是该对象的原型 JavaScript 的

函数对象，除了原型 [__proto__] 之外，还预置了 prototype 属性当函数对象作为构造函数创建实例时，该 prototype 属性值将被作为实例对象的原型 [__proto__]。原型链：当一个对象调用的属性/方法自身不存在时，就会去自己 [__proto__] 关联的前辈 prototype 对象上去找如果没找到，就会去该 prototype 原型 [__proto__] 关联的前辈 prototype 去找。依次类推，直到找到属性/方法或 undefined 为止。从而形成了所谓的

“原型链”

原型特点：

JavaScript 对象是通过引用来传递的，当修改原型时，与之相关的对象也会继承这一改变

136. 请解释什么是事件代理

事件代理也叫事件委托，本质上是利用事件冒泡，把多个子元素的监听统一交给父元素处理。父元素收到事件后，通过event.target判断实际触发者，因此能减少重复注册和内存占用，动态新增的子元素也无需重新绑定。实际处理嵌套内容时，我一般用event.target.closest('.item')，避免点击图标等内部节点时匹配失败。它依赖事件冒泡，像focus、blur这类事件要改用focusin、focusout等可冒泡方案。

事件代理（Event Delegation），又称之为事件委托。是JavaScript中常用绑定事件的常用技巧。顾名思义，“事件代理”即是把原本需要绑定的事件委托给父元素，让父元素担当事件监听的职务。事件代理的原理是DOM元素的事件冒泡。使用事件代理的好处是可以提高性能可以大量节省内存占用，减少事件注册，比如在table上代理所有td的click事件就非常棒可以实现当新增子对象时无需再次对其绑定

下面是一个简单的事件代理的示例代码：Item 1Item 2Item 3 // 使用事件代理绑定点击事件varmyList=document.getElementById('myList');myList.addEventListener('click',function(event){if(event.target.tagName==='LI'){console.log('Clicked on:',event.target.textContent);}});在上述示例中，我们将点击事件绑定到父元素myList上，当点击子元素li时，事件会冒泡到父元素，父元素上的事件处理函数会捕获到事件，并根据event.target来判断点击的具体元素。这样就实现了对子元素点击事件的代理处理。使用事件代理的优势是可以减少事件处理程序的数量，尤其适用于大量的子元素或动态添加的元素，避免了为每个子元素都绑定事件处理程序的麻烦。同时，对于新增的子元素，无需再次绑定事件，它们会自动继承父元素上的事件处理。需要注意的是，在事件代理中，我们需要通过event.target来判断具体触发事件的元素，从而执行相应的逻辑。

137. Javascript如何实现继承？

JavaScript可以通过原型链实现继承，常见做法是让子构造函数的prototype指向父构造函数的实例。这样创建子类实例时，就能沿着原型链访问父类提供的属性和方法。替换原型后要把Child.prototype.constructor改回Child，否则实例通过constructor识别构造函数时会得到错误结果。在JavaScript中，实现继承的方式有多种，包括构造继承、原型继承、实例继承和拷贝继承等。其中，使用构造函数与原型混合方式是较常用和推荐的方式。

以下是使用构造函数与原型混合方式实现继承的示例代码：
functionParent(){this.name='poetry';}functionChild(){this.age=28;}// 使用构造函数继承Child.prototype=newParent();Child.prototype.constructor=Child;vardemo=newChild();console.log(demo.age);// 输出: 28console.log(demo.name);// 输出: poetryconsole.log(demo.constructor);// 输出: Child
通过将Child.prototype设置为new Parent()，子类Child继承了父类Parent的属性和方法。然后，通过手动将Child.prototype.constructor设置为Child，确保子类的构造函数指向自身。这样，demo.constructor的输出将是Child，表示demo实例的构造函数是Child，以确保子类的实例通过constructor属性可以准确地识别其构造函数。

138. 谈谈This对象的理解

this表示函数当前的调用上下文，普通函数中的指向由调用方式决定。作为对象方法调用时，它指向该对象；通过call、apply或bind调用时，则指向显式指定的对象。使用new时，this指向新创建的实例；普通调用在非严格模式下指向全局对象，在严格模式下是undefined。箭头函数不会创建自己的this，而是直接沿用外层作用域的值。在全局作用域中，this指向全局对象（在浏览器环境中通常是window对象）。在函数中，this的值取决于函数的调用方式。如果函数是作为对象的方法调用，this指向调用该方法的对象。如果函数是作为普通函数调用，this指向全局对象（非严格模式下）或undefined（严格模式下）。如果函数是通过call、apply或bind方法调用，this指向call、apply或bind方法的第一个参数所指定的对象。如果函数是作为构造函数调用（使用new关键字），this指向新创建的对象。在箭头函数中，this的值是继承自外部作用域的，它不会因为调用方式的改变而改变。如果函数是作为对象的方法调用，this指向调用该方法的对

象。如果函数是作为普通函数调用，this指向全局对象（非严格模式下）或undefined（严格模式下）。如果函数是通过call、apply或bind方法调用，this指向call、apply或bind方法的第一个参数所指定的对象。如果函数是作为构造函数调用（使用new关键字），this指向新创建的对象。

下面是一些示例代码，以说明this的不同情况：

```
// 全局作用域中的 this
console.log(this); // 输出: Window
// 对象方法中的 this
const obj = {name: 'poetry', sayHello: function() { console.log(`Hello, ${this.name}!`); }};
obj.sayHello(); // 输出: Hello, poetry!
// 普通函数调用中的 this
function greeting() { console.log(`Hello, ${this.name}!`); }
greeting(); // 输出: Hello, undefined
(非严格模式下输出: Hello, [全局对象的某个属性值])
// 使用 call/apply/bind 改变 this
const person = {name: 'poetry'};
greeting.call(person); // 输出: Hello, poetry!
greeting.apply(person); // 输出: Hello, poetry!
const boundGreeting = greeting.bind(person);
boundGreeting(); // 输出: Hello, poetry!
// 构造函数中的 this
function Person(name) { this.name = name; }
const poetry = new Person('poetry');
console.log(poetry.name); // 输出: poetry
// 箭头函数中的 this
const arrowFunc = () => { console.log(this); };
arrowFunc(); // 输出: Window
```

139. 事件模型

浏览器事件模型包含捕获、目标和冒泡三个阶段，事件先从外层向目标传递，再从目标向外层返回。addEventListener的第三个参数为true时监听捕获阶段，为false或省略时监听冒泡阶段。比如点击嵌套按钮，外层的捕获处理函数会先执行，随后执行按钮自身和内层元素的冒泡处理函数。多个处理函数绑定在同一目标上时，通常按添加顺序执行。

事件流分为三个阶段：捕获阶段、目标阶段和冒泡阶段。

捕获阶段（Capture Phase）：事件从最外层的父节点开始向下传递，直到达到目标元素的父节点。在捕获阶段，事件会经过父节点、祖父节点等，但不会触发任何事件处理程序。**目标阶段（Target Phase）：**事件到达目标元素本身，触发目标元素上的事件处理程序。如果事件有多个处理程序绑定在目标元素上，它们会按照添加的顺序依次执行。**冒泡阶段（Bubble Phase）：**事件从目标元素开始向上冒泡，传递到父节点，直到传递到最外层的父节点或根节点。在冒泡阶段，事件会依次触发父节点、祖父节点等的事件处理程序。事件流的默认顺序是从目标元素的最外层父节点开始的捕获阶段，然后是目标阶段，最后是冒泡阶段。但是可以通过事件处理程序的绑定顺序来改变事件处理的执行顺序。

例如，以下代码演示了事件流的执行顺序：

```
Click me
var outer = document.getElementById('outer');
var inner = document.getElementById('inner');
var btn = document.getElementById('btn');
outer.addEventListener('click', function() { console.log('Outer div clicked'); }, true); // 使用捕获阶段进行事件监听
inner.addEventListener('click', function() { console.log('Inner div clicked'); }, false); // 使用冒泡阶段进行事件监听
```

当点击按钮时，事件的执行顺序如下：

- 捕获阶段：触发外层div的捕获事件处理程序。
- 目标阶段：触发按钮的事件处理程序。
- 冒泡阶段：触发内层div的冒泡事件处理程序。

输出结果为：Outer div clicked Button clicked Inner div clicked

这个示例展示了事件流中捕获阶段、目标阶段和冒泡阶段的执行顺序。可以通过addEventListener方法的第三个参数来控制事件处理函数在捕获阶段或冒泡阶段执行，true表示捕获阶段，false或不传表示冒泡阶段。

140. new操作符具体干了什么呢？

new会创建一个新对象，把它关联到构造函数的prototype，再以该对象作为this执行构造函数。构造函数中的属性和方法因此会被添加到新对象上，并且这个对象默认会被返回。手写时可以用Object.setPrototypeOf建立原型关系，再用apply传入参数并执行构造函数。如果构造函数显式返回了非null对象，就使用该

返回值，否则仍返回新创建的对象。 创建一个空对象，并且this变量引用该对象，同时还继承了该函数的原型属性和方法被加入到this引用的对象中新创建的对象由this所引用，并且最后隐式的返回this

实现一个简单的new方法，可以按照以下步骤进行操作： 创建一个新的空对象。将新对象的原型链接到构造函数的原型对象。将构造函数的作用域赋给新对象，以便在构造函数中使用this引用新对象。执行构造函数，并将参数传递给构造函数。如果构造函数没有显式返回一个对象，则返回新对象。

```
function myNew(constructor,...args){// 创建一个新的空对象 const newObj={};//
将新对象的原型链接到构造函数的原型对象 Object.setPrototypeOf(newObj,constructor.prototype);//
将构造函数的作用域赋给新对象，并执行构造函数 const result=constructor.apply(newObj,args);// 如果
构造函数有显式返回一个对象，则返回该对象；否则返回新对象 return typeof result==='object'&&result!=
=null?result:newObj;} 使用上述自定义的myNew方法，可以实现与new操作符类似的效果，如下所示： f
unction Person(name,age){this.name=name;this.age=age;} Person.prototype.sayHello=function(){
console.log('Hello, my name is '+this.name);}; var poetry=myNew(Person,'poetry',25); console.log(poetry.name);// 输出:
poetry console.log(poetry.age);// 输出: 25 poetry.sayHello();// 输出: Hello, my name is poetry 注意
，这只是一个简化的实现，不考虑一些复杂的情况，例如原型链的继承和构造函数返回对象的情况。在实际应用中，建议使用内置的new操作符来创建对象实例，因为它处理了更多的细节和边界情况。
```

141. Ajax原理

Ajax本质上是浏览器通过JavaScript异步请求服务器，拿到数据后局部更新DOM，不用刷新整个页面。核心对象是XMLHttpRequest，创建实例后用open()配置请求、send()发送，再通过onreadystatechange等待结果。readyState为4代表请求完成，此时还要检查status，成功后才能处理响应。它能减少不必要的数据往返、改善交互体验，但也会暴露接口细节，对搜索引擎支持较弱，调试起来相对麻烦。 Ajax的原理简单来说是在用户和服务器之间加了一个中间层(AJAX引擎)，通过XMLHttpRequest对象来向服务器发异步请求，从服务器获得数据，然后用javascript来操作DOM而更新页面。使用户操作与服务器响应异步化。

这其中最关键的一步就是从服务器获得请求数据Ajax的过程只涉及JavaScript、XMLHttpRequest和DOM。XMLHttpRequest是ajax的核心机制 // 手写简易ajax/** 1. 创建连接
/var xhr=null; xhr=new XMLHttpRequest(); / 2. 连接服务器 **/xhr.open('get',url,true); /** 3. 发送请求
/xhr.send(null); / 4. 接受请求 **/xhr.onreadystatechange=function(){if(xhr.readyState===4){if(xhr.status===200){success(xhr.responseText);}else{/** false **/fail&&fail(xhr.status);}} } // promise封装
function ajax(url){const p=new Promise((resolve,reject)=>{const xhr=new XMLHttpRequest(); xhr.open('GET',url,true); xhr.onreadystatechange=function(){if(xhr.readyState===4){if(xhr.status===200){resolve(JSON.parse(xhr.responseText));}elseif(xhr.status===404||xhr.status===500){reject(new Error('404 not found'))}} } xhr.send(null); } }) return p; } //

测试 const url='/data/test.json'; ajax(url).then(res=>console.log(res)).catch(err=>console.error(err))
ajax 有那些优缺点？ 优点：通过异步模式，提升了用户体验。优化了浏览器和服务器的传输，减少不必要的数据往返，减少了带宽占用。Ajax在客户端运行，承担了一部分本来由服务器承担的工作，减少了大用户量下的服务器负载。Ajax可以实现动态不刷新（局部刷新） 缺点：安全问题AJAX暴露了与服务器交互的细节。对搜索引擎的支持比较弱。不容易调试。 通过异步模式，提升了用户体验。优化了浏览器和服务器的传输，减少不必要的数据往返，减少了带宽占用。Ajax在客户端运行，承担了一部分本来由服务器承担的工作，减少了大用户量下的服务器负载。Ajax可以实现动态不刷新（局部刷新）
安全问题AJAX暴露了与服务器交互的细节。对搜索引擎的支持比较弱。不容易调试。

142. 如何解决跨域问题？

跨域通常用服务端配置CORS或部署代理解决，本质是处理浏览器同源策略对跨源访问的限制。同源要求协议、域名和端口都相同；如果接口可改，我一般让服务端返回Access-Control-Allow-Origin等响应头。开发环境可以用webpack-dev-server代理，生产环境则可由Nginx或Node.js中间层转发，让浏览器只访问同源地址。JSONP仅支持GET，document.domain也有场景和版本限制，因此通常不作为现代项目的首选。首先了解下浏览器的同源策略 同源策略/SOP（Same origin policy）是一种约定，由Netscape公司1995年引入浏览器，它是浏览器最核心也最基本的安全功能，如果缺少了同源策略，浏览器很容易受到XSS、CSRF等攻击。所谓同源是指"协议+域名+端口"三者相同，即便两个不同的域名指向同一个ip地址，也非同源

1. 通过jsonp跨域 封装一个可用的 JSONP 方法，可以参考以下示例代码：

```
functionjsonp(url,params,callback){//
```

```
生成唯一的回调函数名constcallbackName='jsonp_'+Date.now();// 将参数拼接到 URL 中constqueryString=Object.keys(params).map(key=>encodeURIComponent(key)+'='+encodeURIComponent(params[key])).join('&');// 创建 script 元素constscript=document.createElement('script');script.src=url+'?'+queryString+'&callback='+callbackName;//
```

```
定义回调函数window[callbackName]=function(data){// 调用回调函数callback(data);// 删除 script 元素和回调函数document.head.removeChild(script);deletewindow[callbackName];// 将 script 元素添加到页面中document.head.appendChild(script);}
```

使用示例：

```
jsonp('http://www.example.com/api',{user:'admin'},function(data){console.log(data);}); 这个jsonp函数接受三个参数：URL、参数对象和回调函数。它会生成一个唯一的回调函数名，并将参数拼接到 URL 中。然后创建一个元素，并将 URL 设置为带有回调函数名的 URL。定义一个全局的回调函数，当响应返回时调用该回调函数，并将数据传递给回调函数。最后将元素添加到页面中，触发跨域请求。当请求完成后，删除元素和回调函数。这样，你就可以通过封装的 JSONP 方法来实现跨域请求并获取响应数据了。
```

2. document.domain + iframe跨域 自Chrome 101版本开始，document.domain将变为可读属性，也就是意味着上述这种跨域的方式被禁用了

此方案仅限主域相同，子域不同的跨域应用场景 1.) 父窗口：(http://www.domain.com/a.html)

document.domain='domain.com';varuser='admin'; 2.) 子窗口：(http://child.domain.com/b.html)

document.domain='domain.com';// 获取父窗口中的变量alert('get js data from parent --->'+window.parent.user);

3. nginx代理跨域 通过 Nginx 配置反向代理，将跨域请求转发到同源接口，从而避免浏览器的同源策略限制。

下面是一个示例配置，展示了如何通过 Nginx 实现跨域代理：

```
server{listen80;server_nameyour-domain.com;location/api{#
```

```
设置代理目标地址proxy_passhttp://api.example.com;# 设置允许的跨域请求头add_headerAccess-Control-Allow-Origin$http_origin;add_headerAccess-Control-Allow-Methods"GET, POST, OPTIONS";add_headerAccess-Control-Allow-Credentials true;add_headerAccess-Contr...
```

143. 模块化开发怎么做？

模块化开发就是把功能拆成职责清晰的独立文件，通过明确的导入和导出组织依赖。现代浏览器和工具链通常采用ES Modules，用export暴露能力、用import引入依赖；Node.js项目也常见通过require和module.exports使用CommonJS。较早的浏览器代码还会用立即执行函数隔离私有变量，或借助AMD异步加载模块。实际选择要看运行环境，现代前端优先使用ES Modules，维护旧项目时再兼容其他规范。

当涉及模块化开发时，有多种方法可供选择： 1. 立即执行函数模式： 使用立即执行函数来创建模块，将私有成员放在函数作用域内，不直接暴露给外部。通过返回一个包含公共方法的对象，使这些方法可以在外部访问。

```
varmodule=(function(){varprivateVar='Private Variable';functionprivateMethod(){console.log('This is a private method');}functionpublicMethod(){console.log('This is a public
```

```

method');return{publicMethod:publicMethod;}}(function(){module.publicMethod();// Output: This is a
public          method          2.          CommonJS :
使用require导入模块，使用module.exports或exports导出模块。适用于 Node.js 环境。 // math.jsfunc
ionadd(a,b){returna+b;}functionsubtract(a,b){returna-b;}module.exports={add,subtract}; //
app.jsconstmath=require('./math');console.log(math.add(2,3));//          Output:
5console.log(math.subtract(5,2));//          Output: 3 3. ES Modules :
使用import导入模块，使用export导出模块。适用于现代浏览器环境和支持 ES6 模块的工具链。 //
math.jsexportfunctionadd(a,b){returna+b;}exportfunctionsubtract(a,b){returna-b;} //
app.jsimport{add,subtract}from'./math';console.log(add(2,3));//          Output:
5console.log(subtract(5,2));//          Output: 3 4. AMD (Asynchronous Module Definition) :
使用define定义模块，通过异步加载模块。适用于浏览器环境和需要按需加载模块的场景。 // math.jsdefi
ne([],function(){functionadd(a,b){returna+b;}functionsubtract(a,b){returna-b;}return{add,subtrac
t;}); // app.jsrequire(['math'],function(math){console.log(math.add(2,3));//          Output:
5console.log(math.subtract(5,2));//          Output: 3}); 以上是常见的模块化开发方式，每种方式都有自己的
特点和使用场景，可以根据具体需求选择适合的模块化规范。

```

144. 异步加载JS的方式有哪些？

异步加载JavaScript常用async、defer、动态创建script和动态import()。async会并行下载并在完成后尽快执行，不保证多个脚本的执行顺序，适合彼此独立的脚本；defer同样并行下载，但会等文档解析完成并按出现顺序执行。需要运行时按需加载时，可以动态插入script，模块化项目则更适合使用返回Promise的import()。旧方案还包括XMLHttpRequest配合eval()、LABjs或Sea.js，但选择时应优先考虑代码依赖关系和执行时机。你提到的异步加载JS的方式都是常见且有效的方法。以下是对每种方式的简要介绍：

1. 设置属性async="async"：通过将async属性设置为"async"，脚本将异步加载并立即执行，不会阻塞页面的解析和渲染。脚本加载完成后，将在页面中的任何位置立即执行。
2. 动态创建script DOM：使用JavaScript动态创建元素，并将其添加到文档中。通过设置src属性指定脚本的URL，异步加载脚本。

```
var script=document.createElement('script');script.src='script.js';document.head.appendChild(script);
```
3. XMLHttpRequest脚本注入：使用XMLHttpRequest对象加载脚本内容，并将其注入到页面中。通过异步请求获取脚本内容后，使用eval()函数执行脚本。

```
var xhr=new XMLHttpRequest();xhr.open('GET','script.js',true);xhr.onreadystatechange=function(){if(xhr.readyState===4&&xhr.status===200){eval(xhr.responseText);}};xhr.send();
```
4. 异步加载库LABjs：LABjs是一个用于异步加载JavaScript的库，可以管理和控制加载顺序。它提供了简洁的API来定义和加载依赖关系，以及控制脚本加载的时机。

```
$LAB.script('script1.js').wait().script('script2.js');
```
5. 模块加载器Sea.js：Sea.js是一个用于Web端模块化开发的加载器，可以异步加载和管理模块依赖关系。它支持异步加载JavaScript模块，并在模块加载完成后执行回调函数。
6. sea.js.use(['module1','module2'],function(module1,module2){// 执行依赖模块加载完成后的逻辑});
7. Deferred Scripts（延迟脚本）：使用元素的defer属性可以将脚本延迟到文档解析完成后再执行。延迟脚本会按照它们在文档中出现的顺序执行，但在DOMContentLoaded事件触发之前执行。
8. Dynamic Import（动态导入）：使用动态导入语法import()可以异步加载JavaScript模块。这种方式返回一个Promise对象，可以通过then()方法处理模块加载完成后的逻辑。

```
import('module.js').then(module=>{// 执行模块加载完成后的逻辑}).catch(error=>{// 处理加载失败的情况});
```
9. Web Workers（Web 工作者）：Web Workers是运行在后台线程中的JavaScript脚本，可以进行耗时操作而不会阻塞主线程。可以使用Web Workers异步加载和执行JavaScript脚本，以提高页面的响应性。

```
var worker = new Worker('worker.js'); worker.onmessage = function(event) { // 处理从 Worker 返回的消息 }; worker.postMessage('start');
```

145. 那些操作会造成内存泄漏？

对象已经不用了却仍被引用，就可能造成内存泄漏。常见情况包括意外创建全局变量、闭包长期持有外部数据，以及调试控制台保留了对象引用。老版本IE中，移除绑定事件的DOM元素却不解绑，也可能无法释放内存；把字符串传给setTimeout同样不推荐。循环引用本身不一定泄漏，如果外部已经没有引用，垃圾回收器仍可能将其回收。 JavaScript

内存泄露指对象在不需要使用它时仍然存在，导致占用的内存不能使用或回收。未使用var声明的全局变量、闭包函数(Closures)、循环引用(两个对象相互引用)、控制台日志(console.log)移除存在绑定事件的DOM元素(IE)、setTimeout的第一个参数使用字符串而非函数的话，会引发内存泄漏。垃圾回收器定期扫描对象，并计算引用了每个对象的其他对象的数量。如果一个对象的引用数量为0（没有其他对象引用过该对象），或对该对象的惟一引用是循环的，那么该对象的内存即可回收。下面是一些常见操作可能导致内存泄漏的示例代码：

未使用var声明的全局变量：`function foo(){bar='global variable';// 没有使用 var 声明}foo();`

闭包函数(Closures)：`function outer(){var data='sensitive data'; return function(){}// 内部函数形成了闭包 console.log(data);}; var inner=outer(); inner();//`

闭包引用了外部函数的变量，导致变量无法被释放

循环引用：

```
function createObjects(){var obj1={}; var obj2={}; obj1.ref=obj2; obj2.ref=obj1; //
```

对象之间形成循环引用，导致无法被垃圾回收}createObjects(); // 控制台日志(console.log)：

```
function processData(data){console.log(data); // 控制台日志可能会引用数据，阻止垃圾回收//
```

处理数据的逻辑} // 移除存在绑定事件的 DOM 元素(IE)：

```
var element=document.getElementById('myElement'); element.onclick=function() { //
```

处理点击事件}; // 移除元素时没有显式地解绑事件处理程序，可能导致内存泄漏（在 IE

浏览器中）element.parentNode.removeChild(element); 使用字符串作为setTimeout的第一个参数：

```
setTimeout('console.log("timeout");',1000); // 使用字符串作为参数，会导致内存泄漏（不推荐） 注意：
```

以上示例只是为了说明可能导致内存泄漏的操作，并非一定会发生内存泄漏。在实际开发中，需要注意避免这些操作或及时进行相应的内存管理和资源释放。

146. XML和JSON的区别？

JSON更轻量、解析更快，XML则更强调数据的描述性和扩展性。JSON使用简洁的键值对，体积通常更小，而且能被JavaScript直接解析和处理。XML通过标签描述结构，含义更直观，但一般需要借助DOM解析，处理过程相对复杂。简单的数据交互通常选JSON，更看重结构描述或跨系统交换时可以考虑XML。XML（可扩展标记语言）和JSON（JavaScript对象表示法）是两种常用的数据格式，它们在以下几个方面有一些区别： 数据体积方面：

JSON相对于XML来说，数据的体积小，因为JSON使用了较简洁的语法，所以传输的速度更快。

数据交互方面：JSON与JavaScript的交互更加方便，因为JSON数据可以直接被JavaScript解析和处理，无需额外的转换步骤。XML需要使用DOM操作来解析和处理数据，相对而言更复杂一些。数据描述方面：

XML对数据的描述性较强，它使用标签来标识数据的结构和含义，可以自定义标签名，使数据更具有可读性和可扩展性。JSON的描述性较弱，它使用简洁的键值对表示数据，适合于简单的数据结构和传递。

传输速度方面：JSON的解析速度要快于XML，因为JSON的语法更接近JavaScript对象的表示，JavaScript引擎能够更高效地解析JSON数据。 需要根据具体的需求和使用场景选择合适的数据格式，一般来说，如果需要简单、轻量级的数据交互，并且与JavaScript紧密集成，可以选择JSON。而如果需要较强的数据描述性和扩展性，或者需要与其他系统进行数据交互，可以选择XML。

147. 谈谈你对webpack的看法

Webpack本质上是一个功能强大、扩展性很高的模块打包工具。它把JavaScript、CSS、图片等资源统一视为模块，再根据依赖关系输出一个或多个静态文件，方便组织和维护代码。通过loader和插件，还能完成资源转换、代码压缩、文件合并及按需加载等工作。开发阶段配合开发服务器和热模块替换，可以提高调试效率；它的优势也在于插件生态和社区资源比较丰富。

Webpack是一个功能强大的模块打包工具，它在现代Web开发中扮演着重要的角色。以下是对Webpack的看法：

- 模块化开发：**Webpack以模块化的方式管理项目中的各种资源，包括JavaScript、CSS、图片、字体等。它能够将这些资源视为模块，并根据模块之间的依赖关系进行打包，使代码结构更清晰、可维护性更高。
- 强大的打包能力：**Webpack具有强大的打包能力，能够将项目中的多个模块打包成一个或多个静态资源文件。它支持各种模块加载器和插件，可以处理各种类型的资源文件，并且能够进行代码压缩、文件合并、按需加载等优化操作，以提高应用的性能和加载速度。
- 生态系统丰富：**Webpack拥有一个庞大的插件生态系统，可以满足各种项目的需求。通过使用各种插件，我们可以实现代码的优化、资源的压缩、自动化部署等功能，大大提升了开发效率。
- 开发工具支持：**Webpack提供了开发工具和开发服务器，支持热模块替换（Hot Module Replacement）等功能，使开发过程更加高效和便捷。它能够实时监听文件的变化并自动重新编译和刷新页面，极大地提升了开发体验。
- 社区活跃：**Webpack拥有一个庞大的社区，开发者们积极分享各种有用的插件和工具，提供了大量的学习资源和解决方案。通过与社区的交流和学习，我们可以更好地了解Webpack的使用技巧和最佳实践。

总的来说，Webpack是一个非常强大和灵活的模块打包工具，它在现代Web开发中发挥着重要作用。通过Webpack，我们可以更好地组织和管理项目代码，提高开发效率和代码质量，同时也能够享受到丰富的插件和工具支持。

148. 说说你对AMD和Commonjs的理解

AMD和CommonJS都是模块规范，核心区别是前者偏浏览器异步加载，后者偏服务端同步加载。AMD通常通过define声明模块和依赖，模块加载完成后执行回调，典型实现是RequireJS。CommonJS使用require加载模块，并通过module.exports或exports暴露接口，Node.js内置支持。浏览器需要避免模块加载阻塞时更适合AMD，服务端读取本地模块时则适合CommonJS。

对于AMD（Asynchronous Module Definition）和CommonJS的理解如下：

- AMD（异步模块定义）：**AMD是一种用于浏览器端的模块定义规范。它支持异步加载模块，允许在模块加载完成后执行回调函数。AMD推荐的风格是通过define函数定义模块，并通过返回一个对象来暴露模块的接口。典型的AMD实现是RequireJS。
- CommonJS：**CommonJS是一种用于服务器端的模块定义规范，Node.js采用了这个规范。它使用同步加载模块的方式，即只有模块加载完成后才能执行后续操作。CommonJS的风格是通过module.exports或exports的属性赋值来暴露模块的接口。CommonJS适用于服务器端的模块加载，因为在服务器端文件的读取是同步的，不会影响性能。

总结：AMD和CommonJS是两种不同的模块定义规范，分别适用于浏览器端和服务器端的模块加载。AMD采用异步加载模块的方式，适用于浏览器环境，允许并行加载多个模块，适用于复杂的模块依赖关系。CommonJS采用同步加载模块的方式，适用于服务器环境，因为在服务器端文件的读取是同步的。在实际开发中，可以根据项目的需求和运行环境选择使用AMD或CommonJS规范来组织和加载模块。

AMD 示例代码：

```
// 模块定义define(['moduleA','moduleB'],function(moduleA,moduleB){// 模块代码varfoo=moduleA.foo();varbar=moduleB.bar();return{baz:function(){console.log(foo+bar);}}});  
// 模块加载require(['myModule'],function(myModule){myModule.baz();// 调用模块方法});
```

CommonJS 示例代码：

```
// 模块定义// moduleA.jsexports.foo=function(){return'Hello';}  
// moduleB.jsexports.bar=function(){return'World';}  
// 主程序// main.jsvarmoduleA=require('./moduleA');varmoduleB=require('./moduleB');varfoo=moduleA.foo();varbar=moduleB.bar();console.log(foo+' '+bar);
```

在浏览器环境下，可以使用RequireJS作为AMD规范的实现库。在Node.js环境下，CommonJS模块加载是内置的，无需使用额外的库。以上示例代码是在浏览器端和Node.js环境中分别使用AMD和CommonJS规范加载模块的简单示例。

149. 常见web安全及防护原理

常见Web安全问题的本质，是不可信输入被执行，或用户身份、浏览器权限被冒用。XSS要靠输出转义、安全模板和CSP，CSRF则用Token、来源校验阻止攻击者借登录态操作。服务端还要用参数化查询防SQL注入，并校验上传文件的类型、大小和存储位置。会话场景中，我会配合HTTPS、HttpOnly、Secure和定期更新会话ID，避免只依赖单一措施。

常见Web安全问题及对应的防护原理如下所示，并附上相应的示例代码：1. SQL注入 就是通过把SQL命令插入到Web表单递交或输入域名或页面请求的查询字符串，最终达到欺骗服务器执行恶意的SQL命令 总的来说有以下几点永远不要信任用户的输入，要对用户的输入进行校验，可以通过正则表达式，或限制长度，对单引号和双"-"进行转换等永远不要使用动态拼装SQL，可以使用参数化的SQL或者直接使用存储过程进行数据查询存取永远不要使用管理员权限的数据库连接，为每个应用使用单独的权限有限的数据库连接不要把机密信息明文存放，请加密或者hash掉密码和敏感的信息防护原理：使用参数化查询或预编译语句使用ORM框架或查询构建器对用户输入进行输入验证和过滤 示例代码：//使用参数化查询const sql='SELECT * FROM users WHERE username = ? AND password = ?';db.query(sql,[username,password],[err,result])=>{// 处理查询结果};//使用预编译语句const sql='SELECT * FROM users WHERE username = ? AND password = ?';const stmt=db.prepare(sql);stmt.run(username,password,[err,result])=>{// 处理查询结果}; 2. 跨站脚本攻击 (XSS) Xss(cross-site scripting)攻击指的是攻击者往Web页面里插入恶意html标签或者javascript代码。比如：攻击者在论坛中放一个看似安全的链接，骗取用户点击后，窃取cookie中的用户私密信息；或者攻击者在论坛中加一个恶意表单，当用户提交表单的时候，却把信息传送到攻击者的服务器中，而不是用户原本以为的信任站点 防护原理：对用户输入进行合适的转义和过滤使用安全的模板引擎或自动转义函数使用HTTP头部中的Content Security Policy (CSP)

对用户输入进行合适的转义和过滤使用安全的模板引擎或自动转义函数使用HTTP头部中的Content Security Policy (CSP) 示例代码：// 对用户输入进行转义function escapeHTML(input){return input.replace(/&g>';};// 使用安全的模板引擎const template=Handlebars.compile('{{data}}');const html=template({data:userInput});// 使用Content Security Policy (CSP)res.setHeader('Content-Security-Policy','script-src \'self\''); 3. 跨站请求伪造 (CSRF) 防护原理：使用CSRF Token进行验证验证请求来源验证HTTP Referer头 使用CSRF Token进行验证验证请求来源验证HTTP Referer头 示例代码：// 使用CSRF Token进行验证app.use((req,res,next)=>{res.locals.csrfToken=generateCSRFToken();next();});// 验证请求来源if(req.headers.origin!=='https://example.com'){// 请求不是来自预期的来源，拒绝处理}// 验证HTTP Referer头if(req.headers.referer!=='https://example.com/'){// 请求不是来自预期的来源，拒绝处理} XSS与...

150. 用过哪些设计模式？

前端里我更关注真正能解决耦合问题的设计模式，常见的是工厂、单例、观察者或发布订阅，以及装饰者。工厂适合按条件创建不同对象，单例适合日志器、全局配置这类唯一实例，但全局状态会影响扩展和测试。观察者由主题直接维护观察者，发布订阅则通过事件总线转发，双方耦合更低。需要兼容第三方或旧接口时，还可以用适配器统一调用方式。当被问到你用过哪些设计模式时，你可以列举出你在前端开发中常使用的设计模式。以下是几个常见的设计模式，以及它们的优缺点、适用场景和示例代码：1.

工厂模式 (Factory Pattern) : 优点: 封装了对象的创建过程, 降低了耦合性, 提供了灵活性和可扩展性。缺点: 增加了代码的复杂性, 需要创建工厂类。适用场景: 当需要根据不同条件创建不同对象时, 或者需要隐藏对象创建的细节时, 可以使用工厂模式。 示例代码:

```
class Button { constructor(text) { this.text = text; } render() { console.log(`Rendering button with text: ${this.text}`); } } class ButtonFactory { createButton(text) { return new Button(text); } } const factory = new ButtonFactory(); const button = factory.createButton('Submit'); button.render(); // Output: Rendering button with text: Submit
```

2. 单例模式 (Singleton Pattern) : 优点: 确保一个类只有一个实例, 节省系统资源, 提供全局访问点。缺点: 可能引入全局状态, 不利于扩展和测试。适用场景: 当需要全局唯一的对象实例时, 例如日志记录器、全局配置对象等, 可以使用单例模式。 示例代码: `class Logger { constructor() { if (Logger.instance) { return Logger.instance; } Logger.instance = this; } log(message) { console.log(`Logging: ${message}`); } } const logger1 = new Logger(); const logger2 = new Logger(); console.log(logger1 === logger2); // Output: true`

3. 观察者模式 (Observer Pattern) : 优点: 实现了对象之间的松耦合, 支持广播通信, 当一个对象状态改变时, 可以通知依赖它的其他对象进行更新。缺点: 可能导致性能问题和内存泄漏, 需要合理管理观察者列表。适用场景: 当需要实现对象之间的一对多关系, 一个对象的改变需要通知其他多个对象时, 可以使用观察者模式。 示例代码: `class Subject { constructor() { this.observers = []; } addObserver(observer) { this.observers.push(observer); } removeObserver(observer) { const index = this.observers.indexOf(observer); if (index !== -1) { this.observers.splice(index, 1); } } notify(message) { this.observers.forEach((observer) => observer.update(message)); } } class Observer { update(message) { console.log(`Received message: ${message}`); } } const subject = new Subject(); const observer1 = new Observer(); const observer2 = new Observer(); subject.addObserver(observer1); subject.addObserver(observer2); subject.notify('Hello, observers!'); // Output`

4. 发布订阅模式 (Publish-Subscribe Pattern) : 优点: 解耦了发布者和订阅者, 使它们可以独立变化。增加了代码的灵活性和可维护性。缺点: 可能会导致发布者过度发布消息, 造成性能问题。订阅者需要订阅和取消订阅相关的逻辑。适用场景: 当存在一对多的关系, 一个对象的状态变化需要通知多个其他对象时, 可以使用发布订阅模式。 示例代码: `class PubSub { constructor() { this.subscribers = {}; } subscribe(event, callback) { if (!this.subscribers[event]) { this.subscribers[event] = []; } this.subscribers[event].push(callback); } publish(event, data) { if (this.subscribers[event]) { this.subscribers[event].forEach(callback => callback(data)); } } }`

151. 为什么要有同源限制?

同源限制是浏览器用来隔离不同站点数据和操作权限的安全边界, 同源要求协议、域名和端口都相同。如果没有这层限制, 恶意页面就可能通过脚本读取其他站点的页面、表单和敏感数据, 甚至操作用户账户。需要跨源读取资源时, 应由服务端通过CORS明确授权, 也可以在特定场景使用JSONP。不过同源策略不是全部安全措施, 站点仍要单独防护XSS和CSRF。 同源策略指的是: 协议, 域名, 端口相同, 同源策略是一种安全协议举例说明: 比如一个黑客程序, 他利用iframe把真正的银行登录页面嵌到他的页面上, 当你使用真实的用户名, 密码登录时, 他的页面就可以通过Javascript读取到你的表单中input中的内容, 这样用户名, 密码就轻松到手了。 同源限制是为了保护用户的隐私和安全而存在的。它的主要目的是防止恶意网站利用客户端脚本对其他网站的信息进行读取和操作, 从而避免信息泄露和恶意攻击。 同源策略通过限制来自不同源的网页之间的交互, 确保只有同源的网页可以相互访问彼此的资源。同源策略要求协议、域名和端口必须完全相同才能实现同源。如果不满足同源条件, 浏览器会禁止跨域请求和操作。

同源限制的作用包括但不限于: 防止跨站点脚本攻击 (XSS): 同源限制可以防止恶意网站通过跨域脚本注入攻击来获取用户敏感信息或操作用户的账户。防止跨站请求伪造 (CSRF): 同源限制可以防止恶意网站伪造用户请求, 以用户的身份执行非法操作。保护用户隐私: 同源限制可以防止其他网站通过跨域方式获取用户在当前网站的敏感信息。 同源限制通过浏览器的安全策略实现, 确保在不同源的网页之间存在一定的隔离性, 提高用户的安全性和隐私保护。但同时也给一些特定的跨域场景带来了限制, 因此在需要跨域访问的情况下, 可以使用跨域技术 (如跨域资源共享CORS、JSONP等) 来解决问题。 示例代码中提到的黑客程序利用了跨域嵌套iframe的方式, 通过读取用户输入的信息来进行攻击。同源限制可以防止这种攻

击，因为该黑客程序的域名与银行登录页面的域名不同，无法通过跨域访问获取用户输入的敏感信息。

152.

offsetWidth/offsetHeight,clientWidth/clientHeight与scrollWidth/scrollHeight的区别

offsetWidth/offsetHeight表示元素包含内容、内边距和边框的完整尺寸，clientWidth/clientHeight则不包含边框和滚动条。scrollWidth/scrollHeight反映内容实际占用的尺寸，连不可见的溢出部分也会计算进去。简单来说，前者看元素整体，中间一组看内部可视区，后一组看内容总量。判断内容是否溢出时，可以比较scrollWidth与clientWidth，没有溢出时两者通常相同。offsetWidth/offsetHeight：返回元素的总宽度/高度，包括内容宽度、内边距和边框宽度。该值包含了元素的完整尺寸，包括隐藏的部分和滚动条占用的空间。clientWidth/clientHeight：返回元素的可视区域宽度/高度，即内容区域加上内边距，但不包括滚动条的宽度。该值表示元素内部可见的部分尺寸。scrollWidth/scrollHeight：返回元素内容的实际宽度/高度，包括内容区域的尺寸以及溢出内容的尺寸。如果内容没有溢出，则与clientWidth/clientHeight的值相同。区别总结：offsetWidth/offsetHeight包含了元素的边框和滚动条占用的空间，提供了元素的完整尺寸。clientWidth/clientHeight只包含元素的内容区域和内边距，不包括滚动条，表示了元素内部可见的部分尺寸。scrollWidth/scrollHeight包含了元素内容的实际宽度/高度，包括溢出内容的尺寸。

示例代码：

```
#box{width:200px;height:200px;padding:20px;border:2px solid black;overflow:scroll;}#content{width:400px;height:400px;}var box=document.getElementById('box');console.log('offsetWidth:',box.offsetWidth);// 224 (200 + 20 + 2 + 2)console.log('offsetHeight:',box.offsetHeight);// 224 (200 + 20 + 2 + 2)console.log('clientWidth:',box.clientWidth);// 200 (200 + 20 + 20)console.log('clientHeight:',box.clientHeight);// 200 (200 + 20 + 20)console.log('scrollWidth:',box.scrollWidth);// 400 (content的宽度)console.log('scrollHeight:',box.scrollHeight);// 400 (content的高度)
```

在上面的示例中，box元素的尺寸为200px

200px，有20px的内边距和2px的边框。内部的content元素的尺寸为400px

400px，超出了父元素的尺寸。通过不同的属性获取到的值可以看到它们的差异。

offsetWidth/offsetHeight返回值包含content

+

padding

+

border，效果与e.getBoundingClientRect()相同clientWidth/clientHeight返回值只包含content

+

padding，如果有滚动条，也不包含滚动条scrollWidth/scrollHeight返回值包含content + padding + 溢出内容的尺寸

153. javascript有哪些方法定义对象

JavaScript定义对象常用对象字面量、构造函数、Object.create()、class和工厂函数。字面量最直接，适合创建结构明确的单个对象；构造函数和class更适合批量生成同类实例。需要精确控制原型时可以用Object.create()，传入null还能得到无原型对象。对象也可通过原型扩展共享方法，或用Object.assign()合并多个源对象的属性。对象字面量：var obj = {}；原型是Object.prototype构造函数：var obj = new Object();Object.create():var obj = Object.create(Object.prototype);Object.create(null)没有原型Object.create({...})可指定原型Object.create(null)没有原型Object.create({...})可指定原型 1. 字面量表示法 (Literal Notation)：使用对象字面量{}直接创建对象，并在其中定义属性和方法。

```
const person = {name: 'poetry', age: 30, sayHello: function() {console.log('Hello!')}};
```

2.

构造函数 (Constructor)：

使用构造函数创建对象，可以定义一个构造函数，然后使用new关键字实例化对象。function Person(name, age) {this.name = name; this.age = age; this.sayHello = function() {console.log('Hello!')}}; const person = new Person('poetry', 30); 3. Object.create()方法：使用Object.create()方法创建一个新对象，并将

指定的原型对象设置为新对象的原型。可以传入一个原型对象作为参数，也可以传入null作为参数来创建没有原型的对象。 `const personPrototype={sayHello:function(){console.log('Hello!');}};const person=Object.create(personPrototype);person.name='poetry';person.age=30;` 4.class关键字（ES6引入）：使用class关键字可以定义类，并通过new关键字实例化对象。 `class Person{constructor(name,age){this.name=name;this.age=age;}sayHello(){console.log('Hello!');}}const person=new Person('poetry',30);` 5. 工厂函数（Factory Function）：使用一个函数来封装创建对象的逻辑，并返回新创建的对象。 `function createPerson(name,age){const person={};person.name=name;person.age=age;person.sayHello=function(){console.log('Hello!');};return person;}const person=createPerson('poetry',30);` 6. 原型（Prototype）：在 JavaScript 中，每个对象都有一个原型（prototype），可以通过原型链来继承属性和方法。 `function Person(name,age){this.name=name;this.age=age;}Person.prototype.sayHello=function(){console.log('Hello!');};const person=new Person('poetry',30);` 7.Object.assign()方法：使用Object.assign()方法可以将一个或多个源对象的属性复制到目标对象中，从而创建一个新对象。 `const person1={name:'poetry',age:30};const person2={sayHello:function(){console.log('Hello!');}};const person=Object.assign({},person1,person2);`

154. 常见兼容性问题？

常见兼容性问题主要来自不同浏览器对盒模型、样式、JavaScript API、事件和图片格式的支持差异。比如元素尺寸不一致时，可以通过box-sizing统一计算方式；新样式不被支持时，则要补浏览器前缀或提供替代方案。处理事件对象、鼠标坐标和计算样式时，也要先做能力检测，再选择对应实现。实际开发不能只看页面是否显示，还要在目标浏览器中测试交互和异常场景。

常见的兼容性问题有很多，以下列举一些常见的问题：

- 浏览器的盒模型差异：不同浏览器对盒模型的解析存在差异，导致元素的尺寸计算不一致。可以使用CSS盒模型属性（box-sizing）来进行控制。
- 浏览器对CSS属性的支持差异：不同浏览器对CSS属性的支持程度不同，某些属性在某些浏览器中可能不起作用或解析不正确。需要使用CSS前缀（Vendor Prefix）或使用兼容性方案来处理。
- JavaScript API的差异：不同浏览器对JavaScript API的支持存在差异，某些方法、属性或事件在某些浏览器中可能不可用或行为不同。需要进行兼容性检测并使用替代方案或进行特定的处理。
- 样式的兼容性：不同浏览器对样式的解析存在差异，可能导致页面显示不一致。需要针对不同浏览器进行样式的调整和优化。
- 图片格式的兼容性：不同浏览器对图片格式的支持存在差异，某些格式在某些浏览器中可能不被支持或显示异常。需要根据需求选择合适的图片格式，并进行兼容性处理。
- 事件处理的差异：不同浏览器对事件的处理存在差异，例如事件对象的属性、方法、坐标获取等方面。需要进行兼容性处理，使用合适的方法来获取事件相关信息。

示例代码：// 获取鼠标坐标

```
function getMousePosition(event){var x,y;if(event.pageX||event.pageY){x=event.pageX;y=event.pageY;}else{x=event.clientX+document.body.scrollLeft+document.documentElement.scrollLeft;y=event.clientY+document.body.scrollTop+document.documentElement.scrollTop;}return{x:x,y:y};}//
```

兼容性处理

```
var event=event||window.event;var mousePosition=getMousePosition(event);// 获取元素样式function getComputedStyle(element){if(window.getComputedStyle){return window.getComputedStyle(element,null);}else{return element.currentStyle;}}//
```

兼容性处理

```
var elementStyle=getComputedStyle(element);//
```

图片格式兼容性处理

```
var img=new Image();img.src='image.png';img.onerror=function(){//
```

图片加载失败，处理兼容性}; 以上示例代码展示了对常见兼容性问题的处理方法，包括事件对象的属性获取、样式获取和图片加载的兼容性处理。在实际开发中，需要根据具体的兼容性问题选择合适的解决方案，并进行兼容性测试和调整。

155. 说说你对promise的了解

Promise用来描述异步操作的最终结果，初始为pending，之后只能变成fulfilled或rejected。状态一旦确定就不能再次改变，而settled表示它已成功或失败，并不是还能继续转换的新阶段。调用resolve或reject后，可以分别通过then和catch处理结果与错误。finally无论成功还是失败都会执行，适合放清理或收尾逻辑。

依照Promise/A+的定义，Promise有四种状态：pending（进行中）：初始状态，表示异步操作尚未完成。当创建一个Promise对象时，它的初始状态就是pending。fulfilled（已成功）：表示异步操作已成功完成，并且返回了一个结果值。一旦Promise的状态转为fulfilled，就会调用onFulfilled回调函数。rejected（已失败）：表示异步操作执行过程中出现了错误或失败。一旦Promise的状态转为rejected，就会调用onRejected回调函数。settled（已结束）：表示Promise已经被resolved（fulfilled或rejected）。在settled状态下，Promise的状态已经确定，不会再发生变化。需要注意的是，Promise的状态转换是单向的，一旦状态确定后就不可再改变。一开始是pending，然后可以转为fulfilled或rejected，一旦转换为其中一种状态，就会保持在那个状态，无法再次改变。 以下是一个示例代码，演示Promise的不同状态和状态转换过程：

```
// 创建一个Promise对象
var promise = new Promise(function(resolve, reject){
  // 异步操作
  setTimeout(function(){
    var randomNum = Math.random();
    if(randomNum > 0.5){
      resolve('Operation succeeded');
    } else {
      reject('Operation failed');
    }
  }, 1000);
});
// 使用Promise对象
promise.then(function(result){
  console.log('Success:', result);
}).catch(function(error){
  console.log('Error:', error);
}).finally(function(){
  console.log('Promise settled');
});
```

在上面的示例中，我们创建了一个Promise对象，并在内部定义了一个异步操作。根据异步操作的结果，调用了resolve或reject方法来改变Promise的状态。然后使用then方法注册了成功时的回调函数，使用catch方法捕获了错误。最后，使用finally方法来注册一个在Promise完成后必定会执行的回调函数。这样，我们就可以通过对Promise的状态进行判断和处理，来执行相应的操作。

156. 你觉得jQuery源码有哪些写的好的地方

jQuery源码比较出色的地方，是把易用性、扩展能力和浏览器兼容封装在了一套清晰结构里。它用自执行函数隔离作用域，只向外暴露jQuery和\$，减少了全局污染。jQuery()内部通过jQuery.fn.init创建实例，再共享jQuery.fn上的方法，因此既不用手写new，又能复用原型能力。配合链式调用、事件委托和插件机制，常见的DOM操作与功能扩展会更统一。jQuery的源码结构清晰，模块化的设计使得各个功能模块之间相互独立，易于维护和扩展。jQuery采用了很多优化技巧，例如使用惰性函数、缓存DOM查询结果、事件委托等，以提高性能和效率。jQuery提供了一致而强大的选择器功能，支持多种选择器语法，使得操作DOM元素更加灵活方便。jQuery提供了丰富的插件生态系统，使得开发者可以轻松扩展功能，且插件之间可以很好地兼容和组合使用。jQuery封装了跨浏览器的解决方案，解决了浏览器兼容性问题，使开发者可以更专注于业务逻辑而不用关心底层实现细节。jQuery提供了丰富的DOM操作方法和动画效果，使得开发者可以轻松实现复杂的交互效果。jQuery文档详细且易于理解，提供了丰富的示例和用法说明，方便开发者学习和使用。

总的来说，jQuery源码在设计和实现上具有很多优秀的地方，使得它成为广泛应用的前端库之一。

以下是一个简单的示例代码，展示了jQuery源码中的一些优秀设计和实现：

```
// 定义一个自执行的匿名函数，将window对象作为局部变量传入，避免作用域链查找
(function(window, undefined){
  // 定义jQuery构造函数
  var jQuery = function(selector){
    return new jQuery.fn.init(selector);
  };
  // 将原型对象简写为fn，提高代码效率
  jQuery.fn = jQuery.prototype = {
    // 初始化方法
    init: function(selector){
      // ...
    },
    // 扩展的实例方法
    // ...
  };
  // 将jQuery的原型对象赋值给fn，实现链式调用
  jQuery.fn.init.prototype = jQuery.fn;
  // ...
})(window, undefined);
```

将jQuery绑定到全局对象window上，提供全局访问window.jQuery=window.\$=jQuery;})(window);

这段示例代码展示了jQuery源码中的一些优秀设计和实现：使用自执行的匿名函数，将window对象作为局部变量传入，提高访问window对象的效率，同时避免全局污染。通过将原型对象简写为fn，提高代码效率，同时利用init方法作为构造函数，实现链式调用。使用原型继承机制，将jQuery的原型对象赋值给fn，使得实例对象可以直接访问jQuery的方法。将jQuery绑定到全局对象window上，使得可以通过jQuery或\$全局变量访问jQuery的功能。

这些设计和实现使得jQuery具有清晰的结构、高效的代码和易于使用的特性，成为广泛应用的前端库。

157. 谈谈你对vue、react、angular的理解

Vue、React和Angular都以组件化、声明式开发为核心，但定位和约束程度不同。Vue渐进灵活，模板和响应式数据绑定容易上手，适合快速开发，也能按需扩展。React用JSX和虚拟DOM组织可复用组件，生态更大，但需要自行选择不少配套方案。Angular内置路由、依赖注入等完整能力，适合大型复杂应用，不过概念更多、学习成本也更高。继续对比Vue、React和Angular的优缺点对比：Vue.js: 优点：简单易学：Vue.js具有简单易学的特点，可以快速上手，适合小型项目或初学者。响应式数据绑定：Vue.js使用双向数据绑定机制，能够实现数据的自动更新和同步，提高开发效率。轻量灵活：Vue.js的核心库很小，可以根据需要逐渐引入插件，具有灵活性和可扩展性。生态系统丰富：Vue.js拥有庞大的社区和生态系统，有大量的第三方库和组件可供使用。缺点：生态系统相对较小：相对于React和Angular，Vue.js的生态系统规模相对较小，可能在某些方面的资源和支持较少。简单易学：Vue.js具有简单易学的特点，可以快速上手，适合小型项目或初学者。响应式数据绑定：Vue.js使用双向数据绑定机制，能够实现数据的自动更新和同步，提高开发效率。轻量灵活：Vue.js的核心库很小，可以根据需要逐渐引入插件，具有灵活性和可扩展性。生态系统丰富：Vue.js拥有庞大的社区和生态系统，有大量的第三方库和组件可供使用。生态系统相对较小：相对于React和Angular，Vue.js的生态系统规模相对较小，可能在某些方面的资源和支持较少。

Angular: 优点：完整的功能集：Angular是一个完整的前端框架，提供了路由、模块化、依赖注入等功能，适用于大型和复杂的应用程序开发。强大的模板系统：Angular具有强大的模板系统，支持丰富的指令和组件，使开发者可以更轻松地构建复杂的用户界面。强大的工具支持：Angular提供了强大的开发工具和调试工具，使开发和调试更加便捷。缺点：学习曲线较陡峭：相对于Vue.js和React，Angular的学习曲线较陡峭，需要掌握更多的概念和技术。复杂性较高：由于Angular是一个完整的框架，它的复杂性较高，对于简单项目可能会显得过于臃肿。优点：完整的功能集：Angular是一个完整的前端框架，提供了路由、模块化、依赖注入等功能，适用于大型和复杂的应用程序开发。强大的模板系统：Angular具有强大的模板系统，支持丰富的指令和组件，使开发者可以更轻松地构建复杂的用户界面。强大的工具支持：Angular提供了强大的开发工具和调试工具，使开发和调试更加便捷。缺点：学习曲线较陡峭：相对于Vue.js和React，Angular的学习曲线较陡峭，需要掌握更多的概念和技术。复杂性较高：由于Angular是一个完整的框架，它的复杂性较高，对于简单项目可能会显得过于臃肿。React: 优点：高性能：React使用虚拟DOM技术，能够提高应用程序的性能，减少DOM操作。组件化开发：React倡导组件化开发，使得代码更加模块化、可维护和可复用。大而活跃的社区：React拥有庞大而活跃的社区，有大量的第三方库和组件可供使用。前后端通用：React可以进行服务器端渲染，使得应用程序具有更好的性能和搜索引擎优化。缺点：学习曲线较陡峭：React使用了JSX语法和一些独特的概念，对于新手来说可能需要一定的学习成本。高性能：React使用虚拟DOM技术，能够提高应用程序的性能，减少DOM操作。组件化开发：React倡导组件化开发，使得代码更加模块化、可维护和可复用。大而活跃的社区：React拥有庞大而活跃的社区，有大量的第三方库和组件可供使用。前后端通用：React可以进行服务器端渲染，使得应用程序具有更好的性能和搜索引擎优化。

学习曲线较陡峭：React使用了JSX语法和一些独特的概念，对于新手来说可能需要一定的学习成本。

158. Node的应用场景

Node.js更适合高并发、I/O密集和实时通信场景，例如接口服务、代理转发与聊天应用。它基于事件驱动和非阻塞I/O，可以高效处理大量连接，并借助V8执行JavaScript。前端工程中也常用它编写CLI、构建工具和自动化脚本，因为前后端能够共用语言和模块生态。边界是CPU密集任务会阻塞单线程，应拆分计算或交给更合适的服务处理。

特点： 基于事件驱动和非阻塞I/O：Node.js采用事件驱动的编程范式，通过异步非阻塞的I/O模型实现高效的并发处理，能够处理大量的并发连接。单线程：Node.js使用单线程处理请求，避免了传统多线程模型中线程切换的开销，提高了处理请求的效率。基于V8引擎：Node.js使用Google Chrome浏览器中的V8引擎解释执行JavaScript代码，具有高性能和高效的特点。跨平台：Node.js可以在多个操作系统上运行，如Windows、Linux、Mac等。

优点： 高并发性能：Node.js的事件驱动和非阻塞I/O模型使其能够处理大量并发请求，适用于构建高性能的网络应用。快速开发：Node.js使用JavaScript语言，具有统一的开发语言，使得前端开发人员可以轻松上手进行服务器端开发。丰富的模块生态系统：Node.js拥有庞大的模块生态系统，提供了丰富的第三方模块和工具，可以快速构建复杂的应用程序。轻量和高效：Node.js具有较小的内存占用和快速的启动时间，适合部署在云环境或资源有限的设备上。

缺点： 单线程限制：Node.js使用单线程处理请求，如果有长时间运行的计算密集型任务或阻塞操作，会导致整个应用程序的性能下降。可靠性低：Node.js在处理错误和异常方面相对较弱，一旦代码某个环节崩溃，整个应用程序都可能崩溃，需要仔细处理错误和异常情况。不适合CPU密集型任务：由于Node.js的单线程特性，不适合处理需要大量计算的CPU密集型任务，这类任务可能会阻塞事件循环，影响整个应用程序的性能。

Node.js的应用场景主要包括以下几个方面：

- 服务器端开发：**Node.js在服务器端开发中表现出色。由于其事件驱动、非阻塞的特性，适合处理高并发的网络请求，可以快速构建高性能的网络应用程序，如Web服务器、API服务器、实时聊天应用等。
- 实时应用程序：**基于Node.js的实时应用程序能够实现双向通信，例如实时聊天应用、协作工具、多人游戏等。Node.js的事件驱动模型和非阻塞I/O使得处理大量并发连接变得更加高效。
- 命令行工具：**Node.js提供了丰富的API和模块，使得开发命令行工具变得简单和高效。通过Node.js可以编写自定义的命令行工具，用于执行各种任务、自动化流程和脚本处理等。
- 构建工具：**Node.js可以用于构建前端的构建工具和任务执行器，如Grunt和Gulp。这些工具利用Node.js的模块化和文件操作能力，帮助开发者自动化地处理代码的编译、压缩、打包等任务。
- 代理服务器：**基于Node.js可以构建高性能的代理服务器，用于代理请求、路由转发、负载均衡等。Node.js的非阻塞I/O使得代理服务器能够同时处理大量的并发请求。

总的来说，Node.js适用于需要处理高并发、实时性要求高、需要构建高性能网络应用的场景。它在Web开发、实时应用、命令行工具等领域都有广泛的应用。

159. 谈谈你对AMD、CMD的理解

AMD和CMD都是浏览器端的异步模块规范，核心区别是声明依赖和加载依赖的时机。AMD由RequireJS提出，通常在define时把依赖前置声明，再通过回调使用模块。CMD由SeaJS提出，依赖可以写在模块内部，通过require在用到的位置加载，更强调依赖就近。两者解决的是早期浏览器模块化问题，选择时主要看项目原有规范和团队习惯。

AMD (Asynchronous Module Definition) 和CMD (Common Module Definition) 是用于浏览器端的模块加载规范。它们的目标都是解决模块化开发的问题，提供了异步加载模块的机制，以提高网页的性能和加载速度。

AMD (Asynchronous Module Definition) 是由RequireJS提出的一种模块加载规范。AMD规范采用异步加载模块的方式，在使用模块之前，需要先定义模块的依赖关系，然后通过回调函数来使用模块。这种方式适用于浏览器环境，可以避免阻塞页面的加载。AMD规范使用define函数来定义模块，可以指定模块的依赖关系和回调函数。在回调函数中可以获取依赖模块，并进行相应的操作。示例代码：

```
define(['module1','module2'],function(module1,module2){//使用module1和module2进行操作});
```

CMD (Common Module Definition) 是由SeaJS提出的一种模块加载规范。CMD规范与AMD规范类似，也采用异步加载模块的方式。但与AMD不同的是，CMD规范在使用模块之前不需要先定义依赖关系，而是在使用时才进行模块的加载。CMD规范使用define函数来定义模块，可以在回调函数中使用require函数来加载依赖模块。示例代码：

```
define(function(require){var module1=require('module1');var module2=require('module2');//
```

使用module1和module2进行操作}); 总体来说, AMD和CMD都是用于浏览器端的模块加载规范, 目的是解决模块化开发的问题。它们的区别在于模块定义和加载的时机不同, AMD在定义时就指定依赖关系并加载模块, 而CMD在使用时才加载模块。根据具体的项目需求和团队的开发习惯, 可以选择适合的规范进行模块化开发。

160. 那些操作会造成内存泄漏

内存泄漏本质上是无用对象仍被引用, 常见来源包括未清理的定时器、事件监听器、闭包和脱离页面的DOM。长期存在的全局变量或无限增长的缓存也会持续占用内存, 资源、第三方实例不用后同样应关闭或销毁。异步任务和回调需要妥善管理状态, 避免相关引用一直无法释放。我一般会在组件卸载时集中清理监听和定时器, 并通过内存快照检查残留对象。

除了之前提到的操作, 以下是更多可能导致内存泄漏的操作, 并附带示例代码:

1. 定时器未清理:
function startTimer(){setInterval(()=>{// 定时操作},1000);} // 没有清理定时器, 导致内存泄漏
解决方法: 在不需要定时器时, 使用clearInterval或clearTimeout清理定时器。

2. 异步操作未完成导致回调函数未执行: function fetchData(callback){// 异步操作, 例如 AJAX 请求或数据库查询// 忘记调用回调函数, 导致内存泄漏// 示例中没有调用 fetchData 的回调函数
解决方法: 确保异步操作完成后, 调用相应的回调函数, 或使用Promise或async/await等方式管理异步操作的状态。

3. DOM 元素未正确移除:
function createDOMElement(){const element=document.createElement('div');// 在页面中插入 element, 但没有移除// 该函数可能被多次调用, 导致大量无用的 DOM 元素存在于内存中}
解决方法: 在不需要的时候, 使用removeChild或其他方法将 DOM 元素从页面中移除。

4. 未释放闭包中的引用: function createClosure(){const data='sensitive data';setTimeout(()=>{console.log(data);},1000);} // 闭包中引用了外部的 data 变量, 导致 data 无法被垃圾回收
解决方法: 在不需要使用闭包中的外部变量时, 确保取消引用, 例如将闭包中的引用设置为null。

除了之前提到的操作, 以下是更多可能导致内存泄漏的操作, 并附带示例代码:

5. 未正确释放事件监听器: function addEventListener(){const element=document.getElementById('myElement');element.addEventListener('click',()=>{// 事件处理程序})// 没有移除事件监听器, 导致内存泄漏
解决方法: 在不需要监听事件时, 使用removeEventListener方法将事件监听器移除。

6. 大量数据缓存导致内存占用过高: function cacheData(){const data=fetchData();// 获取大量数据// 将数据存储在全局变量或其他长久存在的对象中// 数据缓存过多, 占用大量内存资源
解决方法: 及时清理不再需要的数据缓存, 或使用适当的数据存储方案, 例如使用数据库等。

7. 循环引用: function createCircularReference(){const obj1={};const obj2={};obj1.ref=obj2;obj2.ref=obj1; // obj1 和 obj2 彼此引用, 导致无法被垃圾回收
解决方法: 确保循环引用的对象在不再需要时被解除引用, 例如将相应的属性设置为null。

8. 未正确释放资源: function openResource(){const resource=openSomeResource();// 忘记关闭或释放 resource, 导致资源泄漏}
解决方法: 在不再需要使用资源时, 确保关闭、释放或销毁相应的资源, 例如关闭数据库连接、释放文件句柄等。

161. web开发中会话跟踪的方法有哪些

会话跟踪常见做法有Cookie、服务端Session、URL重写、隐藏表单域, 也可以按客户端IP辅助识别。Session把数据留在服务端, 客户端通常只携带会话ID, 因此比直接把状态放进Cookie更容易集中管理。URL和隐藏域适合在请求或表单间传递标识, 但使用范围有限; IP还会受共享网络和代理影响, 不能作为可靠的唯

一依据。在Web开发中，常见的会话跟踪方法包括：

1. Cookie：使用HTTP Cookie来跟踪会话状态，将会话信息存储在客户端。
 示例代码：
 设置Cookie document.cookie="sessionID=abc123; expires=Sat, 31 Dec 2023 23:59:59 GMT; path=/";
 读取Cookie var sessionID=document.cookie;
2. Session：使用服务器端的会话管理机制，在服务器端存储会话数据，客户端通过会话ID来进行访问。
 示例代码（使用Express.js框架）：
 在服务器端设置Session app.use(session({secret:'secretKey',resave:false,saveUninitialized:true}));
 在路由处理程序中存储和访问Session数据 req.session.username='poetry'; var username=req.session.username;
3. URL重写：在URL中附加会话标识符来进行会话跟踪。
 示例代码：
 https://example.com/page?sessionID=abc123
4. 隐藏Input：在HTML表单中使用隐藏的输入字段来存储会话信息。
 示例代码：
5. IP地址：根据客户端的IP地址进行会话跟踪，但这种方法可能受到共享IP、代理服务器等因素的影响。
 示例代码（使用Node.js）：
 var clientIP=req.headers['x-forwarded-for']||req.connection.remoteAddress;

162. JS的基本数据类型和引用数据类型

JavaScript的基本类型包括undefined、null、boolean、number、string、symbol和bigint，引用类型则以Object及数组、函数等派生对象为主。基本类型赋值时复制的是值，改动新变量不会影响原变量；对象赋值复制的是引用，两个变量可能指向同一份数据。比较时也有类似区别，例如两个独立的{}并不严格相等；另外typeof null返回object，判断数组应使用Array.isArray()。

基本数据类型：

- undefined: 表示未定义或未初始化的值。
- null: 表示空值或不存在的对象。
- boolean: 表示逻辑上的true或false。
- number: 表示数值，包括整数和浮点数。
- string: 表示字符串。
- symbol: 表示唯一的、不可变的值，通常用作对象的属性键。

引用数据类型：

- object: 表示一个复杂的数据结构，可以包含多个键值对。
- array: 表示一个有序的、可变长度的集合。
- function: 表示可执行的代码块，可以被调用执行。

基本数据类型在赋值时是按值传递的，每个变量都有自己的存储空间，修改一个变量不会影响其他变量。而引用数据类型在赋值时是按引用传递的，多个变量引用同一个对象，修改一个变量会影响其他变量。需要注意的是，null和undefined既是基本数据类型，也是特殊的值，表示不同的含义。

163. 介绍js有哪些内置对象

JavaScript内置对象可以按用途理解为基础对象、数据处理对象、集合对象和异步或错误处理对象。基础能力主要由Object、Function、String和Number提供，日期、计算及文本匹配通常交给Date、Math和RegExp。处理结构化数据和集合时常用JSON、Array、Map与Set，异步流程和异常则分别使用Promise、Error。实际开发没必要死记完整名单，知道对象负责什么、需要时能找到对应方法更重要。

JavaScript中有许多内置对象，用于提供各种功能和方法，常见的内置对象包括：

- Object: 所有对象的基类。
- Array: 用于表示和操作数组的对象。
- Boolean: 代表布尔值true或false。
- Number: 代表数字，用于执行数值操作和计算。
- String: 代表字符串，用于处理和操作文本数据。
- Date: 用于处理日期和时间。
- RegExp: 用于进行正则表达式匹配。
- Function: 用于定义和调用函数。
- Math: 提供数学计算相关的方法和常量。
- JSON: 用于解析和序列化JSON数据。
- Error: 用于处理错误。

用于表示和处理错误。Map: 一种键值对的集合，其中键可以是任意类型。Set: 一种集合数据结构，存储唯一的值。Promise: 用于处理异步操作和编写更优雅的异步代码。Symbol: 代表唯一的标识符。这些内置对象提供了丰富的功能和方法，可以满足不同的编程需求。开发人员可以利用这些对象来处理数据、执行操作、处理错误等。

164. 说几条写JavaScript的基本规范

写JavaScript时应保持命名、作用域、比较方式和代码格式一致，让代码更容易阅读和维护。我一般优先使用const，确实需要重新赋值时才用let，并通过函数或模块避免污染全局作用域。判断相等使用===或!==，条件与循环始终写花括号，数组和对象优先采用字面量语法。缩进、引号和分号等风格可以交给Prettier与ESLint统一，同时避免eval()、with和不必要的隐式类型转换。

下面是几条常见的写JavaScript的基本规范：使用驼峰命名法（camel case）命名变量、函数和对象属性，例如：firstName,getUserData(),myObject.property使用大写字母开头的驼峰命名法（Pascal case）命名构造函数或类，例如：Person,UserModel使用全大写字母和下划线命名常量，例如：MAX_VALUE,API_KEY使用单行注释（//）或块注释（/* */）对代码进行注释，解释代码的用途和实现思路使用缩进（通常是四个空格或一个制表符）来表示代码块的层次结构，增加代码的可读性使用严格模式（"use strict";）来提高代码的安全性和效率，避免使用隐式全局变量尽量避免使用全局变量，封装代码到函数或模块中，使用局部变量来限制作用域，减少命名冲突在声明变量时，使用let或const来代替var，避免变量提升和作用域问题尽量避免使用隐式类型转换，使用严格相等运算符（===和!==）进行比较，避免类型不匹配的问题在使用条件语句（if、else）和循环语句（for、while）时，始终使用花括号来明确代码块的范围，避免歧义和错误使用单引号或双引号来表示字符串，保持一致性，推荐使用单引号尽量使用模板字符串来拼接字符串，避免使用字符串连接符（+）或复杂的字符串拼接操作使用数组和对象的字面量语法（[]和{}）来创建数组和对象，而不是使用构造函数，例如：let arr = [1, 2, 3],let obj = {name: 'poetry', age: 25}对于长的逻辑语句或表达式，可以使用合适的换行和缩进来增加可读性，或者使用括号将其分成多行避免使用eval()函数和with语句，它们可能引起安全问题和性能问题 这些规范旨在提高代码的可读性、可维护性和一致性，促进团队协作和代码质量的提升。在编写JavaScript代码时，遵循这些规范可以帮助开发人员写出更优雅、健壮和易于

165. JavaScript有几种类型的值

JavaScript的值分为原始类型和引用类型两大类。原始类型包括undefined、null、boolean、number、string、symbol和bigint，赋值或传参时复制的是值。引用类型以Object为基础，数组、函数、日期、正则和错误对象都属于这一类，传递时复制的是指向堆中对象的引用地址。判断类型时，基础值可先用typeof，数组等对象则可结合专用方法判断。JavaScript有以下几种类型的值：原始数据类型：Undefined：表示未定义的值。Null：表示空值。Boolean：表示布尔值，只有两个取值：true和false。Number：表示数字，包括整数和浮点数。String：表示字符串，用于表示文本数据。Symbol（ES6新增）：表示唯一的、不可变的值。引用数据类型：Object：表示对象，是一种复合值，可以包含多个键值对。Array：表示数组，是一种有序的、可变的集合。Function：表示函数，可以执行特定的任务。Date：表示日期和时间。RegExp：表示正则表达式，用于匹配和处理字符串。Error：表示错误对象，用于捕获和处理异常情况。原始数据类型存储在栈中，通过值的复制来进行赋值和传递。而引用数据类型存储在堆中，通过引用的方式进行赋值和传递，实际上传递的是指向堆中对象的引用地址。注意：ES6新增的Symbol类型是一种唯一的、不可变的数据类型，用于创建唯一的标识符，主要用于对象属性的键值。

166. eval是做什么的

eval()会把传入的字符串当作JavaScript代码解析并执行，也可以返回字符串表达式的计算结果。它虽然能动态生成代码，但传入内容一旦不可信，就可能执行恶意代码，同时还会带来性能方面的问题。比如解析JS

ON时理论上可以使用eval(), 但更合适的做法是使用JSON.parse(), 因为它不会把任意内容当代码执行。实际开发中如果有其他实现方式, 我一般不会选择eval()。eval()是JavaScript的一个全局函数, 用于将传入的字符串作为JavaScript代码进行解析和执行。其主要功能有以下几个方面: 动态执行代码: eval()可以将字符串作为JavaScript代码进行执行, 将字符串解析为可执行的JavaScript代码。这样可以动态生成和执行代码, 灵活性较高。计算字符串表达式: eval()可以计算传入的字符串表达式并返回结果。解析JSON: 在某些情况下, 可以使用eval()将JSON字符串解析为JavaScript对象。但是需要注意, 使用eval()解析JSON字符串存在安全风险, 因为它会执行传入的任意代码, 可能导致恶意代码的注入。需要注意的是, 由于eval()执行的字符串会被解析和执行, 因此在使用eval()时要格外小心, 避免执行不可信的代码, 以防止安全漏洞和性能问题。在大多数情况下, 可以通过其他方式实现相同的功能, 而不必使用eval()。

167. null, undefined 的区别

undefined通常表示值尚未定义, 而null表示开发者明确把一个值设为空。变量声明后没有赋值, 或者访问不存在的对象属性时, 得到的通常是undefined; 需要表达“这里目前没有对象”时, 则可以主动赋值为null。两者做宽松比较时可能发生类型转换, 因此判断具体是哪一种空值时, 应使用===。简单来说, 一个更像系统默认的缺失状态, 另一个更像代码主动表达的空状态。null和undefined是JavaScript中表示空值或缺失值的两个特殊值。区别如下: undefined表示变量声明了但没有被赋值, 或者访问对象属性不存在时的默认返回值。当变量被声明但未被赋值时, 默认值为undefined。当访问对象的不存在属性时, 返回值为undefined。null表示变量被赋予了一个空值, 表示有一个对象, 但该对象为空。当想要明确表示一个变量为空对象时, 可以将其赋值为null。null是一个特殊的对象值, 表示对象为空, 即不指向任何内存地址。总结: undefined表示缺少值或未定义的值, 常见于变量声明但未赋值的情况。null表示空对象, 常见于显式地将对象赋值为空。在使用条件判断时, 要注意区分它们的差异。对于严格相等比较, 推荐使用===来避免类型转换, 以准确判断两者是否相等。

168. ["1", "2", "3"].map(parseInt) 答案是多少

["1", "2", "3"].map(parseInt)的结果是[1, NaN, NaN]。map()调用回调时会依次传入元素、索引和数组, 而parseInt()会把前两个参数当作待解析字符串和进制基数。于是实际执行的是parseInt("1", 0)、parseInt("2", 1)和parseInt("3", 2): 前者得到1, 后两者分别因基数非法和数字不符合二进制规则得到NaN。需要按十进制转换时, 应写成map(s => parseInt(s, 10))。parseInt(str, radix) 解析一个字符串, 并返回10进制整数第一个参数str, 即要解析的字符串第二个参数radix, 基数(进制), 范围2-36, 以radix进制的规则去解析str字符串。不合法导致解析失败如果没有传radix当str以0开头, 则按照16进制处理当str以0开头, 则按照8进制处理(但是ES5取消了, 可能还有一些老的浏览器使用)会按照10进制处理其他情况按照10进制处理eslint会建议parseInt写第二个参数(是因为0开始的那个8进制写法不确定(如078), 会按照10进制处理) 当str以0开头, 则按照16进制处理当str以0开头, 则按照8进制处理(但是ES5取消了, 可能还有一些老的浏览器使用)会按照10进制处理其他情况按照10进制处理 // 拆解const arr=["1","2","3"]const res=arr.map((item,index,array)=>{// item: '1', index: 0// item: '2', index: 1// item: '3', index: 2return parseInt(item,index)// parseInt('1', 0) // 0相当没有传, 按照10进制处理返回1 等价于parseInt('1')// parseInt('2', 1) // NaN 1不符合radix 2-36的一个范围// parseInt('3', 2) // 2进制没有3 返回NaN})// 答案 [1, NaN, NaN]

169. javascript 代码中的"use strict";是什么意思

"use strict"表示开启严格模式，让JavaScript引擎用更严格的规则解析和执行代码。它会把部分静默错误变成显式报错，例如禁止使用未声明变量、重复参数、with和八进制字面量，独立调用函数时this也是undefined。写在脚本顶部会影响整个脚本，写在函数体顶部则只影响该函数。ES

module和class默认处于严格模式，不需要重复声明。

"use

strict"是一种特定的指令（directive），用于告诉

JavaScript

解析器在解析代码时采用严格模式。它可以出现在

JavaScript

代码的顶部（全局严格模式）或函数体的顶部（函数级严格模式）。

使用严格模式的好处包括：

消除了一些 JavaScript 的不安全操作，使代码更加安全。阻止使用一些不推荐或已废弃的语法和特性。强制执行更严格的语法和错误检查，减少潜在的错误。提高性能，某些优化措施只在严格模式下生效。

严格模式对一些错误和不合理的行为进行了修正，例如：未声明的变量不能被使用。不能对只读属性进行赋值。函数的参数不能有重复的名称。不能删除变量或函数。不能使用八进制字面量（例如0123）。不能使用with语句。要注意的是，启用严格模式可能会导致一些代码在非严格模式下不起作用，因为严格模式对语法和行为有更高的要求。因此，在使用严格模式之前，需要仔细测试和检查代码，确保代码在严格模式下正常运行。示例：`"use strict";function myFunction(){// 函数级严格模式// ...}// 全局严格模式`上述代码中的"use strict"指令告诉 JavaScript 解析器在解析函数或全局代码时应该采用严格模式。

170. JSON 的了解

JSON是一种轻量、跨语言的数据交换格式，常用于前后端接口传输结构化数据。它看起来像JavaScript对象，但语法更严格，例如键必须使用双引号，也不能写注释和尾逗号。JavaScript通过JSON.stringify()序列化，通过JSON.parse()反序列化。实际使用要注意函数、undefined等值可能丢失，循环引用会报错，所以不适合拿来做通用深拷贝。

JSON (JavaScript

Object

Notation) 是一种轻量级的数据交换格式，以文本形式表示结构化的数据。它采用类似于

JavaScript

对象的键值对的方式来描述数据，易于阅读和编写，同时也便于机器解析和生成。

JSON具有以下特点：

数据格式简单明确：JSON使用键值对（key-value pairs）的形式表示数据，使用大括号{}定义对象，使用方括号[]定义数组，键和值之间使用冒号:分隔。支持多种数据类型：JSON支持包括字符串、数字、布尔值、对象、数组和null在内的基本数据类型。跨平台和语言：JSON是一种通用的数据交换格式，不依赖于特定的编程语言或平台，可以被各种编程语言解析和生成。

在

JavaScript

中，可以使用内置的JSON对象进行 JSON 字符串与 JavaScript 对象之间的转换。常用的方法有：

JSON.parse()：将 JSON 字符串解析为 JavaScript 对象。JSON.stringify()：将 JavaScript 对象转换为 JSON 字符串。示例：`// JSON字符串转换为JSON对象var jsonString='{"name": "poetry", "age": 28, "city": "shenzhen"}';var jsonObj=JSON.parse(jsonString);// JSON对象转换为JSON字符串var obj={name:"poetry",age:28,city:"shenzhen"};var jsonString=JSON.stringify(obj);`

通过使用

JSON，我们可以方便地在不同的系统和平台之间传递和处理数据。它在

Web

开发中被广泛应用于前后端数据交互、API 接口设计等场景。

171. js延迟加载的方式有哪些

JavaScript延迟加载常用defer、动态插入script，也可以根据场景使用async或动态import()。defer会并行下载脚本，并在HTML解析完成后按顺序执行，通常适合有依赖关系的业务脚本。async下载完成就执行，顺序不确定，更适合彼此独立的第三方脚本。需要按用户操作或业务条件加载时，我一般会动态创建script，或者用import()配合打包工具按需分包。延迟加载（Deferred Loading）是一种优化网页性能的技术，可以延迟加载页面中的资源（如脚本、样式表、图片等），从而加快页面的初始加载速度。

以下是几种常见的

JavaScript

延迟加载的方式：

defer

属性：将脚本标签的defer属性设置为"defer"，使得脚本在页面完成解析时执行。例如：动态创建

script

DOM：通过

JavaScript

动态创建元素，并将其插入到页面中。这样可以控制脚本的加载时机，例如在页面加载完毕后再加载脚本。`var script=document.createElement('script');script.src='script.js';document.body.appendChild(script);`
XMLHttpRequest 脚本注入：使用 XMLHttpRequest 对象加载 JavaScript 脚本，并将其注入到页面中。这种方式可以在页面加载过程中异步加载脚本。示例代码如下：

```
var xhr=new XMLHttpRequest();xhr.open('GET','script.js',true);xhr.onload=function(){if(xhr.status===200){var script=document.createElement('script');script.textContent=xhr.responseText;document.body.appendChild(script);}};xhr.send();
```


延迟加载工具：使用第三方的延迟加载工具库，如LazyLoad，可以更方便地管理和控制页面中的延迟加载资源。这些工具通常提供了更多的功能和配置选项，例如按需加载、懒加载、预加载等。

这些延迟加载的方式可以根据具体的需求和场景选择合适的方式来优化页面加载性能，提升用户体验。

172. 同步和异步的区别

同步会等待当前任务完成再继续执行，异步则先发起任务，完成后再通过回调、事件或Promise通知结果。因此同步耗时操作容易阻塞后续代码，而网络请求、定时器等异步任务不会一直占住主执行流程。JavaScript虽然是单线程，但浏览器会处理这些耗时操作，完成后再把回调放进任务队列，由事件循环调度。`async/await`只是让异步代码更像同步写法，本质仍然基于Promise。
同步：浏览器访问服务器请求，用户看到页面刷新，重新发请求，等请求完，页面刷新，新内容出现，用户看到新内容,进行下一步操作
异步：浏览器访问服务器请求，用户正常操作，浏览器后端进行请求。等请求完，页面不刷新，新内容也会出现，用户看到新内容
常见的异步操作包括网络请求（Ajax）、定时器（`setTimeout`、`setInterval`）、事件处理等。在这些异步操作中，任务的执行不会阻塞程序的其他部分，而是在后台进行，当任务完成时，会通过回调函数或事件来通知程序进行下一步操作。
总结：同步操作是按照顺序依次执行任务，阻塞程序的执行；异步操作是通过回调函数或事件触发来执行任务，不会阻塞程序的执行，提高了程序的并发性和响应性。在实际开发中，异步操作通常用于处理耗时操作和需要等待结果的任务，以提高程序的性能和用户体验。

173. defer和async

defer和async都会并行下载外部脚本，核心区别在于执行时机和顺序。defer会等文档解析完成后再执行，并且多个脚本按标签书写顺序运行，适合存在依赖或需要操作DOM的代码。async下载完成就立即执行，可能打断文档解析，多个脚本的执行顺序也无法保证，更适合统计、广告这类彼此独立的脚本。需要注意，这两个属性只对带src的外部脚本生效，写在内联脚本上没有作用。
defer并行加载js文件，会按照页面上script标签的顺序执行
async并行加载js文件，下载完成立即执行，不会按照页面上script标签的顺序执行
下面是更详细的解释：
defer和async是用于控制标签加载和执行的属性。
defer属性用于延迟脚本的执行，即脚本会被并行下载，但会等到整个文档解析完成后再执行。多个带有defer属性的脚本会按照它们在文档中的顺序执行。这样可以确保脚本在操作DOM之前加载，避免阻塞页面的渲染。需要注意的是，只有外部脚本（通过src属性引入的脚本）才能使用defer属性。
async属性用于异步加载脚本，即脚本会被并行下载，并在下载完成后立即执行。多个带有async属性的脚本的执行顺序是不确定的，哪个脚本先下载完成就先执行。这样可以提高脚本的加载性能，但可能会导致脚本之间的依赖关系出现问题。同样，只有外部脚本才能使用async属性。
需要注意的是，defer和async属性只在外部脚本中生效，即通过src属性引入的脚本。如果脚本直接嵌入在标签中，这两个属性不起作用。
选择使用defer还是async取决于脚本的加载和执行顺序的重要性。如果脚本之间有依赖关系，并且需要按照顺序执行，应使用defer。如果脚本之间没有依赖关系，且可以并行加载和执行，可以使用async来提高加载性能。

174. 说说严格模式的限制

严格模式会禁用一些容易产生隐蔽问题的写法，让错误更早暴露出来。例如变量必须先声明，函数参数不能重名，不能使用with，也不能给只读属性赋值或删除不可删除的属性。它还会改变部分运行行为，比如普

通函数里的this不再自动指向全局对象，arguments也不会与形参联动。使用时可以在脚本或函数体开头写'use strict'，但要留意旧代码是否依赖这些宽松行为。严格模式（Strict Mode）是 ECMAScript 5 引入的一种特殊模式，用于限制 JavaScript 代码中的一些不安全或不规范的语法，提供更严格的语法检查，减少一些怪异行为，并改善代码质量和可维护性。严格模式的一些限制包括但不限于：变量必须先声明再使用，禁止隐式全局变量。函数的参数不能有同名属性，否则会报错。禁止使用with语句。不能对只读属性赋值，否则会报错。不能使用前缀0表示八进制数，否则会报错。不能删除不可删除的属性，不能删除变量，只能删除对象属性。eval函数在其内部引入的变量不会影响外部作用域。eval和arguments不能被重新赋值。arguments不会自动反映函数参数的变化。不能使用arguments.callee和arguments.caller。禁止this指向全局对象。不能使用fn.caller和fn.arguments获取函数调用的堆栈。增加了一些保留字，如protected、static和interface。使用严格模式可以提高代码的可靠性，减少意外错误和怪异行为。要启用严格模式，可以在脚本文件或函数体的开头加上'use strict'来指示 JavaScript 解析器以严格模式解析代码。

175. attribute和property的区别是什么

attribute是写在HTML标签上的特性，而property是该元素对应的DOM对象属性。前者通常通过getAttribute、setAttribute访问，值以字符串形式保存；后者可以直接用点语法读写，并且能保存布尔值、数字等具体类型。标准特性通常会映射到对应属性并相互影响，但自定义特性和对象属性不会自动同步。实际开发中，我一般用property操作元素状态，用attribute处理标签声明信息或自定义数据。attribute是dom元素在文档中作为html标签拥有的属性；property就是dom元素在js中作为对象拥有的属性。对于html的标准属性来说，attribute和property是同步的，是会自动更新的但是对于自定义的属性来说，他们是不同步的 attribute和property是用于描述DOM元素的特性和属性的两个概念。区别如下：Attribute（属性）是DOM元素在HTML文档中定义的特性，它可以在HTML标签上声明并存储相关信息。例如，中的class就是一个属性。在JavaScript中，可以通过getAttribute和setAttribute方法来获取和设置属性的值。Property（属性）是DOM元素作为对象的属性，用于访问和操作元素的状态和行为。例如，document.getElementById('myElement').className中的className就是DOM对象的属性。在JavaScript中，可以直接通过.运算符来访问和修改对象的属性。Attribute（属性）是DOM元素在HTML文档中定义的特性，它可以在HTML标签上声明并存储相关信息。例如，中的class就是一个属性。在JavaScript中，可以通过getAttribute和setAttribute方法来获取和设置属性的值。Property（属性）是DOM元素作为对象的属性，用于访问和操作元素的状态和行为。例如，document.getElementById('myElement').className中的className就是DOM对象的属性。在JavaScript中，可以直接通过.运算符来访问和修改对象的属性。主要区别：同步性：对于HTML标准属性来说，属性和特性是同步的，它们会相互影响和更新。但是对于自定义的属性，特性和属性之间是不同步的。值的类型：属性值是具体的数据类型，例如字符串、布尔值、数字等。而特性值始终是字符串。访问方式：属性可以通过直接访问对象的属性来获取和设置，而特性需要使用相关的方法（例如getAttribute和setAttribute）来访问和操作。需要注意的是，大多数情况下，我们更常使用属性来操作DOM元素，因为它们更直观和方便。而特性主要用于处理自定义属性或一些特殊情况下的操作。

176. 谈谈你对ES6的理解

ES6是一次重要的语言升级，它让JavaScript在作用域、异步编程和代码组织上更适合工程化开发。日常最常用的是let、const、箭头函数、模板字符串和解构，因为它们能减少作用域污染，也能让数据处理和函数书写更直接。面对异步流程可以使用Promise，模块之间则通过import、export明确依赖关系；Map、Set等数据结构也补足了常见的数据处理能力。实际使用时还要结合目标环境判断兼容性，必要时做相应处理。ES6（ECMAScript 2015）是JavaScript的第六个主要版本，引入了许多新的语言特性和改进，以提升开发人员的效率和代码质量。以下是ES6的一些重要特性：块级作用域：引入let和const关键字，允许在块级作用域中声明变量，解决了变量提升和作用域污染的问题。箭头函数：使用箭头(=>)定义函数，简化了函数的书写，并且自动绑定了this。模板字符串：使用反引号(`)包裹字符串，可以在字符串中使用变量和表

达式，实现更灵活的字符串拼接和格式化。解构赋值：通过解构赋值语法，可以从数组或对象中提取值，并赋给对应的变量，简化了变量赋值的操作。默认参数：函数可以定义默认参数值，简化了函数调用时传参的操作。扩展运算符：使用三个点 (...) 进行数组和对象的展开操作，可以将一个数组或对象拆分为独立的元素，或者将多个数组或对象合并为一个。Promise：引入了Promise对象，用于更好地处理异步操作，解决了回调地狱的问题，并提供了更清晰的异步编程模式。类和模块化：ES6引入了类的概念，可以使用class关键字定义类，实现了更接近传统面向对象编程的方式。同时，ES6还提供了模块化的支持，可以使用import和export语法导入和导出模块。模块化：引入了模块化的概念，可以使用import和export语法导入和导出模块，提供了更好的代码组织和模块复用的方式。迭代器和生成器**：引入了迭代器和生成器的概念，可以通过自定义迭代器来遍历数据集合，并使用生成器函数来生成迭代器。管道操作符：提案阶段的特性，引入了管道操作符(|>)，可以将表达式的结果作为参数传递给下一个表达式，简化了函数调用和方法链的写法。这些特性只是ES6的一部分，还有其他许多特性，如Promise.all、Map、Set、Proxy、Reflect等。ES6的引入使得JavaScript语言更加现代化和强大，提供了更多的编程工具和语法

当然，以下是对ES6特性的一些示例代码： 箭头函数： // 传统函数function sum(a,b){return a+b;}// 箭头函数const sum=(a,b)=>a+b;// 使用箭头函数作为回调函数const numbers=[1,2,3,4,5];const squaredNumbers=numbers.map(num=>num*num);console.log(squaredNumbers);// [1, 4, 9, 16, 25] 模板字符串： const name="Alice";const age=25;// 使用模板字符串进行字符串拼接和变量插值const message=`My name is \${name} and I am \${age} years old.`;console.log(message);// My name is Alice and I am 25 years old. 解构赋值： // 数组解构赋值const numbers=[1,2,3,4,5];const [first,second,...rest]=numbers;console.log(first);// 1 console.log(second);// 2 console.log(rest);// [3, 4, 5]// 对象解构赋值const person={name:"Alice",age:25};const {name,age}=person;console.log(name);// Alice console.log(age);// 25 默认参数： // 函数默认参数function greet(name="Anonymous"){console.log(`Hello,\${name}!`);greet();} // Hello, Anonymous! greet("Alice");// Hello, Alice! 扩展运算符： // 数组展开const numbers=[1,2,3];const combined=[...numbers,4,5];console.log(combined);// [1, 2, 3, 4, 5]// 对象展开const person={name:"Alice",age:25};const copied...

177. 什么是面向对象编程及面向过程编程，它们的异同和优缺点

面向过程以执行步骤和函数为中心，面向对象则把数据与操作封装成对象，通过对象协作完成需求。前者更直观、执行效率通常较高，但数据和函数相对分离，需求复杂后容易出现重复代码，维护和扩展也更困难。后者借助封装、继承和多态提高复用性与灵活性，更适合组织复杂系统，不过抽象设计和学习成本更高，也可能带来额外性能开销。实际选择要看场景：流程简单时面向过程足够清晰，业务关系复杂、变化频繁时面向对象通常更容易管理。

面向过程编程（Procedural Programming）和面向对象编程（Object-Oriented Programming）是两种不同的编程范式。面向过程编程是一种以过程（函数、方法）为中心的编程方式，将程序看作是一系列的步骤，通过顺序执行这些步骤来解决问题。面向过程编程强调的是解决问题的步骤和算法，将问题划分为不同的子任务，通过函数的调用和数据的传递来实现任务之间的协作。面向过程编程的特点：程序以过程为单位进行组织，以函数为基本单位。数据和函数是分离的，函数对于数据的操作是通过参数进行传递。强调算法和步骤，对问题进行分解和抽象，通过顺序、选择、循环等基本结构来控制程序的流程。程序的可重用性较低，容易产生大量的重复代码。面向对象编程是一种以对象为中心的编程方式，将程序看作是一系列相互关联的对象，通过对象之间的交互和消息传递来解决问题。面向对象编程强调的是事物（对象）的抽象、封装、继承和多态性。面向对象编程的特点：程序以对象为单位进行组织，对象是数据和方法的封装体。数据和函数（方法）是紧密关联的，对象通过方法来操作自己的数据。强调对象之间的关系和交互，通过继承、封装和多态性来实现代码的复用和灵活性。程序的可重用性较高，易于维护和扩展。 异同和优缺点： 异同：异同点

在于编程的思维方式和组织代码的方式不同，面向过程注重解决问题的步骤和算法，面向对象注重对象和对象之间的关系和交互。面向过程将问题分解为不同的函数来实现，而面向对象将问题抽象为对象，通过对象的方法来实现功能。优点：面向过程编程的优点包括：简单、直观、执行效率高。面向对象编程的优点包括：可重用性高、易于扩展和维护、代码更加模块化和灵活。缺点：面向过程编程的缺点包括：代码重复性高、维护困难、扩展性差。面向对象编程的缺点包括：复杂性高、学习曲线陡。异同点在于编程的思维方式和组织代码的方式不同，面向过程注重解决问题的步骤和算法，面向对象注重对象和对象之间的关系和交互。面向过程将问题分解为不同的函数来实现，而面向对象将问题抽象为对象，通过对象的方法来实现功能。面向过程编程的优点包括：简单、直观、执行效率高。面向对象编程的优点包括：可重用性高、易于扩展和维护、代码更加模块化和灵活。面向过程编程的缺点包括：代码重复性高、维护困难、扩展性差。面向对象编程的缺点包括：复杂性高、学习曲线陡。

178. 面向对象编程思想

面向对象编程的核心，是把相关数据和操作封装成对象，再通过对象之间的交互来组织程序。封装负责隐藏实现、控制对外接口，继承用于复用和扩展已有能力，多态则让同一种调用在不同对象上表现出不同的行为。这样设计通常能提高代码的可维护性、复用性和可测试性，也更容易适应需求变化。它并非适合所有场景，因为抽象层次过多会增加设计复杂度、学习成本和性能开销，是否采用仍要结合项目规模与需求判断。面向对象编程（Object-Oriented Programming，简称OOP）是一种编程思想和方法论，其基本思想是以对象为中心，通过封装、继承和多态等机制来组织和管理代码。以下是面向对象编程的基本思想：

- 封装（Encapsulation）：将数据和对数据的操作封装在一起，形成对象。对象对外暴露有限的接口，隐藏内部实现细节，提供更好的模块化和抽象性。
- 继承（Inheritance）：通过继承机制，一个类（子类）可以继承另一个类（父类）的属性和方法，并可以在此基础上添加、修改或扩展功能。继承可以实现代码的重用性和层次化的组织。
- 多态（Polymorphism）：多态是指同一个方法名可以在不同的对象上产生不同的行为。通过多态，可以以统一的方式处理不同类型的对象，提高代码的灵活性和可扩展性。

面向对象编程的优点包括：

- 易维护性：对象的封装性和模块化使得代码易于维护和理解。对修改封闭、对扩展开放的原则可以减少对已有代码的影响。
- 代码重用性：通过继承和组合等机制，可以重用已有的类和代码，减少重复编写代码的工作量。
- 灵活性和可扩展性：面向对象编程提供了灵活的结构和抽象层次，使得代码易于扩展和修改，适应需求的变化。
- 可靠性：封装和继承等机制可以提高代码的可靠性和可测试性，减少错误的发生和影响范围。
- 可理解性：面向对象的代码通常具有良好的可读性和可理解性，对象和类的设计使得代码更加直观和自然。

面向对象编程可以提供更高层次的抽象和组织，适用于复杂的系统和大型项目的开发。但同时，面向对象编程也有一些限制和挑战，如学习曲线较陡、设计复杂度高、性能开销等。适合选择何种编程思想取决于具体的项目需求和开发场景。

179. 对web标准、可用性、可访问性的理解

Web标准保证页面在不同浏览器和设备上的一致性与互操作性，可用性关注用户是否好用，可访问性关注包括残障用户在内的所有人是否能用。遵循HTML、CSS、JavaScript等规范，通常也能改善代码的可维护性和扩展性。可用性更强调界面直观、一致、反馈明确，减少学习和操作成本；可访问性则需要兼顾读屏器、放大器和辅助键盘等使用方式。工程中这三者应一起考虑，因为标准化实现是基础，清晰交互和无障碍支持才能让更多用户顺利获取内容。Web标准（Web Standards）是指由W3C（World Wide Web Consortium）等组织制定的用于开发和实现Web内容的一系列规范和标准。它包括HTML、CSS、JavaScript等技术规范，旨在确保不同浏览器和设备在呈现网页时的一致性和互操作性。遵循Web标准可以提高网站的可维护性、可访问性和可扩展性，同时提升用户体验。可用性（Usability）是指一个产品或系统在使用过程中的易用性和用户满意度。一个具有良好可用性的产品能够满足用户的需求，提供直观、简洁、一致的用户界面，减少用户的学习成本和操作复杂度，提供明确的反馈和帮助信息，从而提升用户的效率和满意度。可访问性（Accessibility）是指Web内容对于所有用户，包括残障用户，的可阅读和可理解。

性。它涉及到使用无障碍技术和遵循无障碍设计原则，以确保残障用户能够平等地获取和使用Web内容。这包括为视觉、听觉、运动和认知等方面的残障用户提供适当的辅助功能和支持，如屏幕阅读器、放大器、辅助键盘等。可维护性（Maintainability）是指一个系统或代码的易维护性和可理解性。一个具有良好可维护性的系统能够快速定位和修复问题，容易进行功能扩展和修改。为了提高可维护性，代码应具有良好的结构和组织，遵循设计模式和编程规范，提供清晰的注释和文档，同时采用合适的工具和方法进行版本控制和测试。这三个概念在Web开发中都非常重要。遵循Web标准可以提高网站的可访问性和可用性，从而更好地服务于用户的需求。同时，考虑可维护性可以降低代码的维护成本和风险，使开发团队能够更加高效地进行开发和迭代。

180. 如何通过JS判断一个数组

判断数组优先使用`Array.isArray(value)`，它会直接返回布尔值，是ES5提供的标准方法。如果需要另一种通用写法，可以判断`Object.prototype.toString.call(value) === '[object Array]'`。`value instanceof Array`和`value.constructor === Array`也能用于判断，但后者依赖可被访问或修改的`constructor`属性，因此不够稳妥。工程中一般选`Array.isArray()`，只有兼容特定旧环境或处理特殊边界时，再考虑其他方案。

instanceof方法：使用`instanceof`运算符判断对象是否为数组，返回布尔值。例如：`arr instanceof Array`。

constructor方法：使用`constructor`属性返回对象的构造函数，并判断该构造函数是否为数组构造函数。例如：`arr.constructor === Array`。

使用`Object.prototype.toString.call()`方法：利用`Object.prototype.toString.call(value)`方法，将要判断的变量作为参数传入，并判断返回的字符串是否为`"[object Array]"`。例如：`Object.prototype.toString.call(arr) === '[object Array]'`。

ES5新增的`isArray()`方法：使用`Array.isArray()`方法判断一个值是否为数组，返回布尔值。例如：`Array.isArray(arr)`。

181. 谈一谈let与var的区别

`let`是块级作用域，`var`是函数作用域或全局作用域，这是两者最核心的区别。`var`的声明会提升，声明前读取会得到`undefined`；`let`在声明前不可访问，否则会抛出`ReferenceError`。同一作用域内，`var`可以重复声明，而`let`不允许。比如在`for`循环里，`let`会为每次迭代创建新绑定，异步回调能依次拿到0、1、2，`var`则会共享同一个变量，最后都拿到3。

1. 块级作用域：`let`声明的变量具有块级作用域，在块级作用域内定义的变量只在该块内有效。`var`声明的变量没有块级作用域，它的作用域是函数级的或全局的。 示例代码：
// 使用 `let` 声明变量
`function example1(){let x=10;if(true){let x=20;console.log(x);// 输出 20}console.log(x);// 输出 10}`
// 使用 `var` 声明变量
`function example2(){var y=30;if(true){var y=40;console.log(y);// 输出 40}console.log(y);// 输出 40}`
`example2();`

2. 变量提升：`let`声明的变量不存在变量提升，必须在声明后使用。使用`var`声明的变量会存在变量提升，可以在声明之前使用。 示例代码：
// 使用 `let` 声明变量
`function example3(){console.log(x);// 报错：ReferenceError: x is not defined}let x=10;example3();`
// 使用 `var` 声明变量
`function example4(){console.log(y);// 输出 undefined}var y=20;example4();`

3. 重复声明：`let`声明的变量不允许重复声明，重复声明会导致报错。使用`var`声明的变量允许重复声明，不会报错，后面的声明会覆盖前面的声明。 示例代码：
// 使用 `let` 声明变量
`let z=30;let z=40;// 报错：SyntaxError: Identifier 'z' has already been declared`
// 使用 `var` 声明变量
`var w=50;var w=60;console.log(w);// 输出 60`

4. 循环中的区别 在`for`循环中，使用`var`声明的变量具有函数作用域，因此在循环结束后仍然可以访问到循环变量；而使用`let`声明的变量具有块级作用域，因此在每次循环迭代时会创建一个新的变量实例，避免了常见的循环中的问题。 使用`let`声明的变量在循环体内部具有块级作用域，每次迭代都会创建一个新的变量。使用`var`声明的变量在循环体内部没有块级作用域，变量是函数级的或全局的。 示例代码：
// 使用 `let` 声明变量的循环
`for(let i=0;i<3;i++){setTimeout(function(){console.log(i);// 输出 0, 1, 2},1000);}`
// 使用 `var` 声明变量的循环
`for(var j=0;j<3;j++){setTimeout(function(){console.log(j);// 输出 3, 3, 3},1000);}`

182. map与forEach的区别

map用来把数组转换成一个等长的新数组，forEach则更适合单纯遍历和执行操作。map会把每次回调的返回值依次放入新数组，而forEach本身返回undefined。因为map有数组结果，所以后面还能继续调用filter、reduce等方法；forEach无法这样串联。实际写代码时，需要生成新数据就用map，只是打印、记录或修改外部状态时用forEach更直观。forEach方法是无法中断的，即使在遍历过程中使用return语句也无法停止遍历。而map方法可以使用return语句中断遍历。map方法会生成一个新的数组，并将每次遍历的返回值按顺序放入新数组中。而forEach方法没有返回值，仅用于遍历数组。map方法可以链式调用其他数组方法，比如filter、reduce等。而forEach方法不能链式调用其他数组方法。

示例代码：

```
const numbers = [1, 2, 3, 4, 5]; // 使用 forEach 方法遍历数组
numbers.forEach(function(item, index, array) { console.log(item); });
// 输出数组元素 console.log(index); // 输出索引值 console.log(array); // 输出原数组
// 使用 map 方法遍历数组并生成新数组
const doubledNumbers = numbers.map(function(item, index, array) { return item * 2; });
console.log(doubledNumbers); // 输出 [2, 4, 6, 8, 10]
// 在上面的示例中，使用 forEach 方法遍历数组并输出元素、索引和原数组。
// 而使用 map 方法遍历数组并返回每个元素的两倍值，生成一个新的数组 doubledNumbers。
// 注意，在 map 的回调函数中使用了 return 语句来指定返回值。
// 总结：forEach 方法用于遍历数组，没有返回值；map 方法也用于遍历数组，返回一个新的数组，并且可以通过在回调函数中使用 return 语句来指定每次遍历的返回值。
```

183. 谈一谈你理解的函数式编程

函数式编程本质上是把计算拆成函数求值，再通过函数组合完成业务逻辑。它强调纯函数和不可变数据，相同输入应得到相同输出，并尽量不修改参数或外部状态，因此代码更容易预测和测试。函数也可以作为参数或返回值，所以map、filter、reduce这类高阶函数很适合做抽象和组合。惰性计算则是在真正需要结果时才执行，不过实际开发中不必追求完全纯粹，应根据可读性和业务场景取舍。

简单说，"函数式编程"是一种"编程范式"（programming paradigm），也就是如何编写程序的方法论它具有以下特性：闭包和高阶函数、惰性计算、递归、函数是"第一等公民"、只用"表达式"

函数式编程（Functional Programming）是一种编程范式，它强调将计算过程视为函数求值的数学模型，通过组合和应用函数来进行程序开发。函数式编程具有以下特点：

- 纯函数（Pure Functions）：函数的输出只由输入决定，不会产生副作用，即对同样的输入始终返回相同的输出。纯函数不会修改传入的参数，也不会改变外部状态，使得代码更加可预测和易于测试。
- 不可变性（Immutability）：数据一旦创建就不能被修改，任何对数据的改变都会创建一个新的数据副本。这种不可变性使得代码更加安全，避免了一些潜在的错误。
- 高阶函数（Higher-Order Functions）：函数可以作为参数传递给其他函数，也可以作为返回值返回。这种高阶函数的能力可以用来进行函数的组合、封装和抽象，提高代码的复用性和可读性。
- 函数组合（Function Composition）：通过将多个函数组合成一个新的函数，可以实现更复杂的逻辑。函数组合可以通过函数的返回值作为参数传递给另一个函数，将多个函数连接起来形成一个函数链。
- 惰性计算（Lazy Evaluation）：只在需要的时候才进行计算，避免不必要的计算。这种惰性计算可以提高程序的性能和效率。

下面是一个简单的函数式编程的示例代码：

```
// 纯函数示例：计算一个数组中所有偶数的平均值
function calculateAverage(numbers) {
  const evenNumbers = numbers.filter((num) => num % 2 === 0);
  const sum = evenNumbers.reduce((acc, curr) => acc + curr, 0);
  return sum / evenNumbers.length;
}
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const average = calculateAverage(numbers);
console.log(average); // 输出 6

// 高阶函数示例：使用 map 和 reduce 计算数组中每个元素的平方和
function calculateSquareSum(numbers) {
  return numbers.map((num) => num * num).reduce((acc, curr) => acc + curr, 0);
}
const numbers = [1, 2, 3, 4, 5];
const squareSum = calculateSquareSum(numbers);
console.log(squareSum); // 输出 55

// 函数组合示例：组合两个函数计算数组中偶数的平方和
const evenNumbersSquareSum = calculateSquareSum(numbers.filter((num) => num % 2 === 0));
console.log(evenNumbersSquareSum); // 输出 20
```

在上

面的示例代码中，我们使用纯函数的方式编写了两个函数calculateAverage和calculateSquareSum，它们接收一个数组作为参数，并根据函数式编程的原则

184. 谈一谈箭头函数与普通函数的区别？

箭头函数和普通函数最关键的差别，是箭头函数没有自己的this，而普通函数的this由调用方式决定。箭头函数会捕获定义位置外层的this，使用call、apply或bind也不能改变它。它还不能通过new实例化，没有自己的arguments，需要用剩余参数...args，同时也不能作为Generator使用yield。我一般在回调或需要保留外层this时用箭头函数，需要动态this或构造实例时使用普通函数。 this指向：箭头函数没有自己的this，它会捕获所在上下文的this值。而普通函数的this是在运行时确定的，根据调用方式决定。 // 普通函数中的this指向调用者functiongreet(){console.log(`Hello,\${this.name}!`);}constperson={name:'Alice'};greet.call(person);// 输出：Hello, Alice!// 箭头函数中的this指向定义时的上下文constgreetArrow=()=>{console.log(`Hello,\${this.name}!`);}greetArrow.call(person);// 输出：Hello, undefined! 不可作为构造函数：箭头函数不能使用new关键字来创建实例，它没有自己的prototype属性，无法进行实例化。

```
constPerson=(name)=>{this.name=name;}
```

错误，箭头函数不能作为构造函数;constperson=newPerson('Alice');// 错误，无法实例化箭头函数无arguments对象：箭头函数没有自己的arguments对象，可以使用Rest参数来代替。

```
functionsum(){console.log(arguments);}sum(1,2,3);// 输出：Arguments(3) [1, 2, 3]constsumArrow=(...args)=>{console.log(args);}sumArrow(1,2,3);// 输出：[1, 2, 3]
```

无yield命令：箭头函数不能用作Generator函数，无法使用yield命令进行函数的暂停和恢复。 function*generatorFunc(){yield1;yield2;}constgen=generatorFunc();console.log(gen.next().value);//

输出：1constarrowGen=()=>{yield1;}// 错误，箭头函数不能使用yield命令; 综上所述，箭头函数与普通函数在this指向、构造函数能力、arguments对象和yield命令等方面有明显的区别。根据具体的使用场景和需求，选择适合的函数类型进行编程。 总结 函数体内的this对象，就是定义时所在的对象，而不是使用时所在的对象不可以当作构造函数，也就是说，不可以使用new命令，否则会抛出一个错误不可以使用arguments对象，该对象在函数体内不存在。如果要使用，可以用Rest参数代替不可以使用yield命令，因此箭头函数不能用作Generator函数

185. 谈一谈函数中this的指向

普通函数的this由调用方式决定，箭头函数的this则继承定义位置外层的绑定。对普通函数来说，优先级通常是new、call/apply/bind、对象方法调用、独立调用。独立调用在非严格模式下指向全局对象，严格模式下是undefined；方法赋给变量后再调用，也会丢失原来的对象绑定。箭头函数没有自己的this，所以不能用call或bind改变，也不能通过new调用。

函数中的this指向是根据函数的调用方式而确定的，有以下几种常见的情况：

方法调用模式：当函数作为对象的方法被调用时，this指向调用该方法的对象。

```
constperson={name:'Alice',greet:function(){console.log(`Hello,${this.name}!`);}};person.greet();// 输出：Hello, Alice! 函数调用模式：当函数独立调用时，this指向全局对象（在浏览器环境中通常指向window对象）或undefined（在严格模式下）。
```

```
functiongreet(){console.log(`Hello,${this.name}!`);}constname='Alice';greet();// 输出：Hello, undefined!// 在严格模式下'use strict';greet();// 输出：Hello, undefined!
```

构造器调用模式：当函数用作构造器（使用new关键字）创建对象时，this指向新创建的对象。 functionPerson(name){this.name=name;this.greet=function(){console.log(`Hello,\${this.name}!`);}}constperson=newPerson('Alice');person.greet();// 输出：Hello, Alice!

apply/call调用模式：使用apply或call方法来调用函数时，可以手动指定this的值。 functiongreet(){con

```
sole.log(`Hello,${this.name}!`);const person={name:'Alice'};greet.call(person);// 输出：Hello, Alice!greet.apply(person);// 输出：Hello, Alice!
```

总结来说，函数中的this指向是根据函数的调用方式确定的，可以是调用函数的对象、全局对象、新创建的对象，或者通过apply/call方法手动指定。了解函数的调用方式可以帮助理解和正确使用this关键字。

总结 1. this 指向有哪几种 默认绑定：全局环境中，this默认绑定到window隐式绑定：一般地，被直接对象所包含的函数调用时，也称为方法调用，this隐式绑定到该直接对象隐式丢失：隐式丢失是指被隐式绑定的函数丢失绑定对象，从而默认绑定到window。显式绑定：通过call()、apply()、bind()方法把对象绑定到this上，叫做显式绑定new绑定：如果函数或者方法调用之前带有关键字new，它就构成构造函数调用。对于this绑定来说，称为new绑定构造函数通常不使用return关键字，它们通常初始化新对象，当构造函数的函数体执行完毕时，它会显式返回。在这种情况下，构造函数调用表达式的计算结果就是这个新对象的值如果构造函数使用return语句但没有指定返回值，或者返回一个原始值，那么这时将忽略返回值，同时使用这个新对象作为调用结果如果构造函数显式地使用return语句返回一个对象，那么调用表达式的值就是这个对象 构造函数通常不使用return关键字，它们通常初始化新对象，当构造函数的函数体执行完毕时，它会显式返回。在这种情况下，构造函数调用表达式的计算结果就是这个新对象的值如果构造函数使用return语句但没有指定返回值，或者返回一个原始值，那么这时将忽略返回值，同时使用这个新对象作为调用结果如果构造函数显式地使用return语句返回一个对象，那么调用表达式的值就是这个对象

2. 改变函数内部 this 指针的指向函数 (bind, apply, call的区别)

apply：调用一个对象的一个方法，用另一个对象替换当前对象。例如：B.apply(A, arguments);即A对象应用B对象的方法call：调用一个对象的一个方法，用另一个对象替换当前对象。例如：B.call(A, args1,args2);即A对象调用B对象的方法bind除了返回是函数以外，它的参数和call一样

3. 箭头函数 箭头函数没有this，所以需要通过查找作用域链来确定this的值，这就意味着如果箭头函数被非箭头函数包含，this绑定的就是最近一层非...

186. 异步编程的实现方式

异步编程常见实现有回调、事件或发布订阅、Promise、Generator以及async/await。回调最直接，但层级多后耦合较高；事件和发布订阅能解耦调用双方，不过执行流程不够直观。Promise用链式调用统一处理结果和错误，Generator能借助yield暂停和恢复执行，但需要额外管理流程。我一般优先用返回Promise的async/await，因为代码结构接近同步写法，错误可以通过try/catch处理。

回调函数：在异步操作完成后，通过回调函数来处理结果

优点：简单、容易理解缺点：不利于维护，代码耦合高

优点：简单、容易理解缺点：不利于维护，代码耦合高

```
function fetchData(callback){setTimeout(()=>{const data='Hello, world!';callback(data);},1000);}fetchData(()=>{console.log(data);// 输出：Hello, world!});
```

事件监听：通过事件的发布和订阅来实现异步操作

优点：容易理解，可以绑定多个事件，每个事件可以指定多个回调函数

缺点：事件驱动型，流程不够清晰

优点：容易理解，可以绑定多个事件，每个事件可以指定多个回调函数

缺点：事件驱动型，流程不够清晰

```
function fetchData(){setTimeout(()=>{const data='Hello, world!';eventEmitter.emit('dataReceived',data);},1000);}eventEmitter.on('dataReceived',(data)=>{console.log(data);// 输出：Hello, world!});fetchData();
```

发布/订阅(观察者模式)：类似于事件监听，但是可以通过消息中心来管理发布者和订阅者

类似于事件监听，但是可以通过‘消息中心’，了解现在有多少发布者，多少订阅者

类似于事件监听，但是可以通过‘消息中心’，了解现在有多少发布者，多少订阅者

```
function fetchData(){setTimeout(()=>{const data='Hello, world!';messageCenter.publish('dataReceived',data);},1000);}messageCenter.subscribe('dataReceived',(data)=>{console.log(data);// 输出：Hello, world!});fetchData();
```

Promise对象：使用Promise对象可以更方便地处理异步操作的结果和错误

优点：可以利用then方法，进行链式写法；可以书写错误时的回调函数；缺点：编写和理解，相对比较难

优点：可以利用then方法，进行链式写法；可以书写错误时的回调函数；缺点：编写和理解，相对比较难

```
function fetchData(){return new Promise((resolve,reject)=>{setTimeout(()=>{const data='Hello, world!';resolve(data);},1000);});}fetchData().then((data)=>{console.log(data);// 输出：Hello, world!}).catch((error)=>{console.error(error);});
```

Generator函数：使用Generator函数可以实现函数体内外的数据交换和错误处理

优点：函数体内外的数据交换、错误处理机制

缺点：流程管理不方便

```
function* fetchData(){try{const data=yield new Promise((resolve,reject)=>{setTimeout(()=>{resolve('Hello, world!');},1000);});console.log(data);// 输出：Hello, world!}catch(error){console.error(error);}}const generator=fetchData();const promise=generator.next().value;promise.then((data)=>{generator.next(data);}).catch((error)=>{generator.throw(error);});
```

async函数：async函数是Generator函数的语法糖，可以更方便地编写和理解异步代码...

187. 谈谈你对原生JavaScript了解程度

我对原生JavaScript的理解可以归为语言基础、浏览器能力和工程应用三个层面。语言基础包括数据类型、作用域与闭包、函数、原型链和异步机制，它们决定代码为什么这样运行。浏览器层面主要是事件、DOM、BOM、网络请求和存储，用来完成页面交互与数据通信。工程中还要关注模块化、跨域、资源异步加载和内存泄漏；真要判断掌握程度，我更愿意结合具体代码说明原理与边界，而不是只罗列名词。

数据类型：JavaScript具有多种数据类型，包括字符串、数字、布尔值、对象、数组、函数等2.

运算：JavaScript支持常见的算术运算、逻辑运算和比较运算，也支持位运算和三元运算符

```
let sum=5+3; let isTrue=true&&false; let isEqual=10===5; let bitwiseOr=3|5; let result=(num>0)?"Positive":"Negative";
```

对象：JavaScript中的对象是键值对的集合，可以通过字面量形式或构造函数创建对象

```
let person={name:"poetry",age:25}; let car=new Object(); car.brand="Toyota"; car.color="Blue";
```

Function：JavaScript中的函数是一等公民，可以作为变量、参数或返回值进行操作

```
function add(a,b){return a+b;} let multiply=function(a,b){return a*b;} let result=multiply(2,3);
```

继承：JavaScript使用原型链实现对象之间的继承关系

```
function Animal(name){this.name=name;} Animal.prototype.sayHello=function(){console.log("Hello, I'm "+this.name);}; function Dog(name,breed){Animal.call(this,name);this.breed=breed;} Dog.prototype=Object.create(Animal.prototype); Dog.prototype.constructor=Dog; let dog=new Dog("Max","Labrador"); dog.sayHello();
```

闭包：闭包是指函数能够访问其词法作用域外的变量，通过闭包可以实现数据的私有化和封装

```
function outerFunction(){let count=0; return function(){count++; console.log(count);};} let increment=outerFunction(); increment(); // 输出：1 increment(); // 输出：2
```

作用域：JavaScript具有函数作用域和块级作用域，在不同的作用域中变量的可访问性不同

```
function example(){let x=10;if(true){let y=20; console.log(x); // 输出：10 console.log(y); // 输出：20}}
```

原型链：原型链是JavaScript中实现对象继承的机制，每个对象都有一个原型对象，形成一个链式结构

```
function Animal(name){this.name=name;} Animal.prototype.sayHello=function(){console.log("Hello, I'm "+this.name);}; function Dog(name,breed){this.breed=breed;} Dog.prototype=Object.create(Animal.prototype); Dog.prototype.constructor=Dog; let dog=new Dog("Max","Labrador"); dog.sayHello();
```

事件：JavaScript通过事件来响应用户的操作，可以通过事件监听和事件处理函数来实现

```
let button=document.getElementById("myButton"); button.addEventListener("click",function(){console.log("Button clicked");});
```

10. RegExp：正则表达式是一种用于匹配和操作字符串的...

188. Js动画与CSS动画区别及相应实现

CSS动画更适合简单、固定的过渡效果，JavaScript动画则适合需要精确控制和复杂交互的场景。CSS通过过渡或关键帧声明动画，代码较少，浏览器也更容易进行硬件加速，但运行过程中的控制能力有限。JavaScript可以逐帧计算状态并响应实时数据，通常用requestAnimationFrame，复杂效果也可以交给GSAP或基于WebGL的Pixi.js。我一般优先用CSS完成基础位移和渐变，只有动画逻辑或交互控制较复杂时才切到JavaScript。在JavaScript中实现动画可以通过以下方式：使用setTimeout或setInterval函数结合DOM操作来实现逐帧动画。这种方式需要手动计算和控制每一帧的变化，并且需要注意处理动画的性能问题。let element=document.getElementById("animate");let position=0;function animate(){position+=1;element.style.left=position+"px";if(position<100){setTimeout(animate,16);}}然后通过上述示例代码来创建Pixi.js应用程序，并实现所需的动画效果。

相比之下，CSS动画具有以下优点：性能优化：浏览器可以对CSS动画进行硬件加速，以提高动画的性能和流畅度。简单易用：使用CSS关键帧动画可以通过简单的CSS样式声明来定义动画，代码相对简单。兼容性：CSS动画在现代浏览器中得到很好的支持，并且在某些情况下可以更好地处理动画效果。

然而，CSS动画也有一些限制：控制能力受限：CSS动画通常只能实现简单的线性或简单的缓动效果，对于复杂的动画效果和交互控制，可能需要使用JavaScript来实现。兼容性局限：某些老版本的浏览器可能不支持某些CSS动画属性和效果。因此，根据实际需求和性能考虑，选择合适的动画实现方式是很重要的。在简单的动画效果和性能要求较高时，可以优先考虑使用CSS动画；而在复杂的动画控制和交互需求时，使用JavaScript来实现动画更为灵活。

189. JS 数组和对象的遍历方式，以及几种方式的比较

数组遍历常用for、forEach、for...of和map，对象遍历常用for...in、Object.keys或Object.entries。for能操作索引但写法稍繁琐，forEach更简洁但不能用break和continue，需要转换并生成新数组时才适合map。for...of适合数组等可迭代对象，但不能直接取得索引。遍历对象时，我一般优先用Object.keys或Object.entries，因为for...in还可能遍历原型链上的属性，不适合直接遍历数组。数组的遍历方式：1. for循环：可以使用普通的for循环来遍历数组元素。优点：灵活性高，可以根据索引进行操作。缺点：代码相对繁琐，需要手动管理索引。const array=[1,2,3];for(let i=0;i<array.length;i++){console.log(array[i]);}3. for...of循环：使用for...of循环来遍历数组。优点：语法简洁，无需手动管理索引，可以遍历任何可迭代对象。缺点：无法获取当前元素的索引。

const array=[1,2,3];for(const element of array){console.log(element);}4. map方法：使用数组的map方法进行遍历并返回新数组。优点：可以同时遍历和转换数组的元素，返回一个新数组。缺点：不适合只需要遍历而不需要返回新数组的情况。const array=[1,2,3];const mappedArray=array.map((element)=>element*2);console.log(mappedArray);对象的遍历方式：1. for...in循环：for...in循环是用于遍历对象属性的，但也可用于遍历数组。优点：可以遍历数组的索引或属性。缺点：会遍历数组的原型链，不稳定且性能较差，不推荐在数组上使用。

const obj={a:1,b:2,c:3};for(const key in obj){console.log(key,obj[key]);}2. Object.keys方法结合forEach方法：

const obj={a:1,b:2,c:3};Object.keys(obj).forEach((key)=>{console.log(key,obj[key]);});3. Object.entries方法结合forEach方法：

const obj={a:1,b:2,c:3};Object.entries(obj).forEach(([key,value])=>{console.log(key,value);});

比较总结：for循环是最基本的遍历方式，适用于所有情况，但代码较为繁琐。forEach方法是数组专用的遍历方法，代码简洁，但无法使用break和continue跳出循环。for...of循环适用于遍历可迭代对象，如数组、字符串等，语法简单，但无法获取索引。map方法适用于对数组进行映射转换，返回新数组。for...in循环适用于遍历对象的属性，但会遍历原型链上的属性。Object.keys方法结合forEach方法适用于遍历对象的属性，不遍历原型链。Object.entries方法结合forEach方法适用于遍历对象的键值对。根据不同的需求 and 数据结构，选择合适的遍历方式可以提高代码的可读性和性能。使用基本的for循环可以处理各种情况

，forEach和map方法提供了简洁的数组遍历方式，for...of循环适用于遍历可迭代对象，for...in循环和Object.keys/Object.entries结合forEach方法适用于遍历对象的属性和键值对。

190. gulp是什么

Gulp是基于Node.js流的前端自动化构建工具，用来把资源处理和重复任务串成可执行的流水线。它强调代码优于配置，通常从gulp.src读取文件，经多个pipe完成压缩等处理，再由gulp.dest输出结果。因为数据通过流传递，可以减少频繁的IO操作，API也比较少。比如压缩CSS时，可以定义一个任务，并用gulp.series把它纳入默认执行流程。gulp`是前端开发过程中一种基于流的代码构建工具，是自动化项目的构建利器；它不仅能对网站资源进行优化，而且在开发过程中很多重复的任务能够使用正确的工具自动完成。Gulp的核心概念：流，简单来说就是建立在面向对象基础上的一种抽象的处理数据的工具。在流中，定义了一些处理数据的基本操作，如读取数据，写入数据等，程序员是对流进行所有操作的，而不用关心流的另一头数据的真正流向。gulp正是通过流和代码优于配置的策略来尽量简化任务编写的工作。Gulp的特点：易于使用：通过代码优于配置的策略，gulp 让简单的任务简单，复杂的任务可管理。构建快速利用Node.js流的威力，你可以快速构建项目并减少频繁的IO操作。易于学习通过最少的API，掌握gulp毫不费力，构建工作尽在掌握：如同一系列流管道。易于使用：通过代码优于配置的策略，gulp 让简单的任务简单，复杂的任务可管理。构建快速利用Node.js流的威力，你可以快速构建项目并减少频繁的IO操作。易于学习通过最少的API，掌握gulp毫不费力，构建工作尽在掌握：如同一系列流管道。

```
const gulp = require('gulp');
const cleanCSS = require('gulp-clean-css'); // 压缩CSS任务
gulp.task('minify-css', () => {
  return gulp.src('src/css/*.css')
    .pipe(cleanCSS())
    .pipe(gulp.dest('dist/css'));
});
```

默认任务gulp.task('default', gulp.series('minify-css')); 上述示例定义了一个名为minify-css的任务，用于压缩CSS文件。通过使用gulp.src选择要处理的文件，然后通过cleanCSS插件进行压缩操作，最后将压缩后的文件保存到dist/css目录下。通过gulp.task定义任务，最后通过gulp.series定义默认任务，将minify-css任务作为默认任务执行。总结：Gulp的特点在于其简单的API和基于流的处理方式。通过使用Gulp，开发者可以轻松地定义和执行各种任务，提高开发效率。它的易用性、快速构建和易学性使得Gulp成为前端开发中常用的自动化构建工具之一。

191. 说一下Vue的双向绑定数据的原理

Vue的双向绑定本质上是数据劫持配合发布者-订阅者模式，让数据变化能够自动通知视图更新。它通过Object.defineProperty()给属性设置getter和setter，读取时触发getter，赋新值时触发setter。在setter中会通知依赖该属性的订阅者，再执行对应的视图更新。简单来说，数据变化负责发布消息，视图等依赖负责订阅并响应，因此两边能够保持同步。vue.js则是采用数据劫持结合发布者-订阅者模式的方式，通过Object.defineProperty()来劫持各个属性的setter，getter，在数据变动时发布消息给订阅者，触发相应的监听回调。Vue的双向绑定数据的原理是基于数据劫持和发布者-订阅者模式的组合。具体步骤如下：Vue通过Object.defineProperty()方法对数据对象进行劫持。在劫持过程中，为每个属性添加了getter和setter。当访问属性时，会触发getter函数，而当属性值发生变化时，会触发setter函数。在setter函数中，Vue会通知相关的订阅者，即依赖于该属性的视图或其他数据。订阅者收到通知后，会执行相应的更新操作，将新的数据反映到视图上。

这样，当数据发生变化时，Vue能够自动更新相关的视图，实现了双向绑定的效果。这种原理结合了数据劫持和发布者-订阅者模式的特点，实现了数据与视图之间的自动同步。通过数据劫持，Vue能够捕获数据的变化，而发布者-订阅者模式则确保了数据变化时的及时通知和更新。

示例代码：// 定义一个数据对象const data = { message: 'Hello Vue!' }; // 通过Object.defineProperty()劫持数据对象Object.defineProperty(data, 'message', { get() { console.log('访问数据'); return this._message; }, set(new Value) { console.log('更新数据'); this._message = new Value; } }); // 通知订阅者，执行更新操作notifySubscribers(); // 定义一个订阅者列表const subscribers = [];

订阅者订阅数据 `function subscribe(callback){subscribers.push(callback);}` // 通知订阅者，执行更新操作
`function notifySubscribers(){subscribers.forEach((callback)=>{callback();});}` //
 订阅者更新视图 `function updateView(){console.log('视图更新：',data.message);}` //
 订阅数据变化 `subscribe(updateView);` // 修改数据，触发更新 `data.message='Hello VueJS!'`；在上述示例中，我们通过 `Object.defineProperty()` 对 `data` 对象的 `message` 属性进行劫持，并在 `getter` 和 `setter` 中添加了相应的日志和更新操作。订阅者通过 `subscribe` 方法订阅数据变化，并在 `updateView` 方法中更新视图。当我们修改 `data.message` 的值时，会触发 `setter` 函数，从而通知订阅者执行更新操作，最终更新了视图。通过这种方式，Vue 实现了双向绑定的效果，使得数据的变化能够自动反映到视图上。

192. let var const 区别

`var` 是函数级作用域并存在变量提升，`let` 和 `const` 是块级作用域且存在暂存死区，而 `const` 还要求声明后不能重新赋值。`var` 在代码执行前会先处理声明，所以同一函数内的代码块不能隔离它；`let` 则只在所在代码块中有效。`const` 创建的是只读引用，基本类型不能改值，但对对象或数组内部的属性、元素仍然可以修改。我一般在值无需重新赋值时用 `const`，需要变化时再用 `let`。`let` 允许你声明一个作用域被限制在块级中的变量、语句或者表达式 `let` 绑定不受变量提升的约束，这意味着 `let` 声明不会被提升到当前该变量处于从块开始到初始化处理的“暂存死区”。`var` 声明变量的作用域限制在其声明位置的上下文中，而非声明变量总是全局的。由于变量声明（以及其他声明）总是在任意代码执行之前处理的，所以在代码中的任意位置声明变量总是等效于在代码开头声明。`const` 声明创建一个值的只读引用（即指针）基本数据当值发生改变时，那么其对应的指针也将发生改变，故造成 `const` 申明基本数据类型时再将其值改变时，将会造成报错，例如 `const a = 3; a = 5` 时将会报错。如果是复合类型时，如果只改变复合类型的其中某个 `Value` 项时，将还是正常使用。示例代码：
`let: function example(){let x=10;if(true){let x=20;console.log(x);}` // Output: 20
`console.log(x);` // Output: 10
`example();`
`var: function example(){var x=10;if(true){var x=20;console.log(x);}` // Output: 20
`console.log(x);` // Output: 20
`example();`
`const: function example(){const x=10;if(true){const x=20;console.log(x);}` // Output: 20
`console.log(x);` // Output: 10
`example();`
 在上述示例中，使用 `let` 关键字声明的变量 `x` 具有块级作用域，它的作用范围仅限于 `if` 语句块内部。而使用 `var` 关键字声明的变量 `x` 则具有函数级作用域，它的作用范围在整个函数内部都可见。

对于 `const` 关键字声明的变量 `x`，它创建了一个只读的引用，也就是说它的值不能被修改。在示例中，`const x = 10` 声明了一个常量 `x`，而在 `if` 语句块内部再次使用 `const x = 20` 声明了一个新的常量 `x`，它的作用范围也仅限于 `if` 语句块内部。总结：`let` 关键字声明的变量具有块级作用域，不会被提升，存在暂存死区。`var` 关键字声明的变量具有函数级作用域，会被提升到当前作用域的顶部。`const` 关键字声明的变量创建一个只读的引用，其值不可修改，但对于复合类型的变量，可以修改其属性或元素的值。

193. 快速的让一个数组乱序

我会优先用 Fisher-Yates 洗牌：从数组末尾向前遍历，每次与 `[0, i]` 范围内的随机元素交换。它只需遍历一次，时间复杂度为 $O(n)$ ，得到的排列也更均匀。`arr.sort(() => Math.random() - 0.5)` 虽然写法短，但随机比较破坏了排序函数的稳定规则，结果还会受排序实现影响。若项目已使用 `lodash`，也可以直接调用 `shuffle`；需要密码学级随机性时，应改用 `crypto.getRandomValues`。

方法1：使用数组的 `sort` 方法结合随机数
`var arr=[1,2,3,4,5,6,7,8,9,10];arr.sort(function(){return Math.random()-0.5;});console.log(arr);`

方法2：使用 Fisher-Yates 算法
`function shuffleArray(array){var currentIndex=array.length,temporaryValue,randomIndex;while(currentIndex!==0){randomIndex=Math.floor(Math.random()*currentIndex);currentIndex-=1;temporaryValue=array[currentIndex];array[currentIndex]=array[randomIndex];array[randomIndex]=temporaryValue;}`


```
;array[randomIndex]=temporaryValue;}returnarray;}vararr=[1,2,3,4,5,6,7,8,9,10];console.log(shuffleArray(arr));
```

方法3：使用lodash库的shuffle方法
`import shuffle from 'lodash/shuffle'`
`var arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]; console.log(shuffle(arr));`
对比总结：方法1使用数组的sort方法结合随机数，是一种简单快速的方式，但并不是真正意义上的乱序，因为它通过排序来实现的。方法2使用Fisher-Yates算法，通过交换数组中的元素来实现乱序，是一种更可靠的乱序方式。方法3使用lodash库的shuffle方法，提供了一个方便的工具函数来实现数组的乱序，不需要自己编写乱序算法。总体而言，如果只是需要简单的乱序，方法一已经足够。但如果对于乱序的质量和随机性有较高的要求，可以使用方法二的Fisher-Yates算法或者借助第三方库来实现。

194. 如何渲染几万条数据并不卡住界面

我一般首选虚拟列表，只渲染可视区及少量缓冲数据，避免把几万个节点同时塞进DOM。固定高度的列表可以直接按滚动距离计算索引，不定高列表则需要测量并缓存元素高度。若业务确实要逐步展示全部数据，可以用requestAnimationFrame分批渲染，并通过DocumentFragment合并每批插入。数据计算很重时可放进Web Worker，分页或无限滚动也能从数据量上减轻压力。

方式1：使用requestAnimationFrame 这道题考察了如何在不卡住页面的情况下渲染数据，也就是说不能一次性将几万条都渲染出来，而应该一次渲染部分DOM，那么就可以通过requestAnimationFrame来每16 ms刷新一次 在渲染大量数据时，避免一次性将所有数据都渲染出来可以提高性能，以保持界面的流畅性。以下是一个示例代码，演示如何使用requestAnimationFrame来分批渲染大量数据，避免卡住界面：

```
Document控件setTimeout(()=>{// 插入十万条数据const total=100000// 一次插入 20条，可以根据实际性能调整const once=20// 渲染数据总共需要几次const loopCount=total/oncelet countOfRender=0let ul=document.querySelector("ul");function add(){// 优化性能，使用文档片段插入，减少回流const fragment=document.createDocumentFragment();for(let i=0;i<once;i++){const li=document.createElement("li");li.textContent=`${i+1}`;fragment.appendChild(li);}ul.appendChild(fragment);requestAnimationFrame(add);}add();}
```

上述代码会将十万条数据分批插入到ul列表中，每次插入20条数据，并通过requestAnimationFrame在每一帧中执行渲染，保证不卡住界面。这样用户可以逐步看到数据的渲染过程，而不是等待所有数据都渲染完毕后才显示。这种方式可以提高用户体验并避免界面卡顿。 方式2：使用虚拟滚动 使用虚拟滚动（Virtual Scrolling）可以在渲染大量数据时提高性能，只渲染可见区域的数据，而不是将所有数据都插入到DOM中。这样可以减少DOM操作和内存占用，从而提升性能和响应速度。以下是使用虚拟滚动完成这道题的示例代码：

```
Document.container{height:400px;overflow:auto;}.item{height:30px;line-height:30px;border-bottom:1px solid #ccc;}setTimeout(()=>{// 插入十万条数据const total=100000// 单个元素的高度const itemHeight=30// 计算可见区域能容纳的元素数量const visibleItemCount=Math.ceil(visibleHeight/itemHeight)// 当前滚动位置对应的元素索引let startIndex=0;let endIndex=visibleItemCount;let container=document.querySelector(".container");let content=document.querySelector(".content");function renderItems(){content.innerHTML="";for(let i=startIndex;i<endIndex;i++){const li=document.createElement("li");li.textContent=`${i+1}`;content.appendChild(li);}renderItems();}
```

195. 希望获取到页面中所有的checkbox怎么做？

直接使用document.querySelectorAll('input[type="checkbox"]')，就能获取页面中所有由input实现的复选框。它返回静态NodeList，需要数组方法时可用Array.from()或展开语法转换。只取已选中的元素，可以把选择器改为input[type="checkbox"]:checked。也可以先用getElementsByTagName('input')获取动态集合再按type筛选，但循环中增删节点时要留意集合会同步变化。 var domList=document.getElem

entsByTagName('input')varcheckboxList=[];varlen=domList.length;//缓存到局部变量while(len--){
//使用while的效率会比for循环更高if(domList[len].type== 'checkbox'){checkboxList.push(domList[l
en]);}} 这段代码使用document.getElementsByTagName('input')获取到页面中所有的input元素，并通过遍历筛选出type为checkbox的元素，然后将它们存储在checkboxList数组中请注意，这段代码假设所有的复选框都是通过元素实现的，如果你的页面中还有其他方式创建的复选框，可能无法正确获取到。另外，建议将domList.length缓存到局部变量中，可以提高代码的性能。

196. 怎样添加、移除、移动、复制、创建和查找节点

节点操作可以按创建、查找和修改来处理：用createElement创建，用querySelector查找，再用append、remove等方法调整结构。添加已有节点时，直接append到新位置就是移动，不需要先手动删除。复制可用cloneNode(true)做深拷贝，但事件监听器不会随节点一起复制。兼容老写法时，也可以使用appendChild、removeChild、insertBefore和replaceChild。

下面是一些用于添加、移除、移动、复制、创建和查找节点的常用方法：

创建新节点

```
document.createElement(tagName);//
```

```
创建一个指定标签名的元素节点document.createTextNode(text);//
```

```
创建一个包含指定文本的文本节点document.createDocumentFragment();//
```

```
创建一个空的文档片段节点 添加、移除、替换、插入节点 parentNode.appendChild(node);//
```

```
在父节点的末尾添加一个子节点parentNode.removeChild(node);//
```

```
从父节点中移除指定的子节点parentNode.replaceChild(newNode,oldNode);//
```

```
用新节点替换指定的旧节点parentNode.insertBefore(newNode,referenceNode);//
```

```
在参考节点之前插入一个新节点 查找节点 document.getElementsByTagName(tagName);//
```

```
返回指定标签名的元素节点集合document.getElementsByName(name);//
```

```
返回具有指定名称的元素节点集合document.getElementById(id);// 返回具有指定 id 的元素节点 注意，
```

以上方法都是基于document对象进行操作的，如果需要在特定的节点上执行这些操作，可以使用相应节点的方法，例如parentNode.appendChild(node)。示例代码： // 创建新节点varnewElement=document.createElement('div');varnewText=document.createTextNode('Hello, world!');varfragment=document.createDocumentFragment();//

```
添加节点document.body.appendChild(newElement);newElement.appendChild(newText);//
```

```
移除节点document.body.removeChild(newElement);// 替换节点varoldElement=document.getElementById('old');varnewElement=document.createElement('div');document.body.replaceChild(newElement,oldElement);// 插入节点varreferenceElement=document.getElementById('reference');varnewNode=document.createElement('p');document.body.insertBefore(newNode,referenceElement);// 查找节点varelementsByTagName=document.getElementsByTagName('div');varelementsByName=document.getElementsByName('name');varelementById=document.getElementById('id');
```

以上代码演示了如何创建新节点、添加节点、移除节点、替换节点、插入节点以及查找节点的方法。请根据实际情况调整代码并操作相应的节点。

197. 正则表达式

正则表达式本质上是用字符类、量词和边界描述字符串的匹配规则，可用于校验、提取和替换。字面量写法更简洁；模式来自变量时要用RegExp()，并注意字符串中的反斜杠需要再次转义。实际开发常配合test()、match()、replace()使用，量词后加?可切换为非贪婪匹配。像(a+)+这样的嵌套量词可能造成灾难性回溯，处理用户输入时要留意ReDoS风险。

正则表达式构造函数RegExp()和正则表达式字面量的主要区别在于语法和使用方式。

正则表达式构造函数RegExp() 使用字符串作为参数，需要进行双重转义，即需要使用双反斜杠来表示特殊

字符，如\d表示数字，\w表示字母数字下划线等。构造函数的参数可以是一个字符串，也可以是两个字符串，第一个字符串是正则表达式模式，第二个字符串是修饰符。如果正则表达式模式是一个变量，只能使用构造函数的方式创建正则表达式。正则表达式字面量// 使用两个斜杠//将正则表达式包围起来。字面量的方式更简洁，不需要进行双重转义，直接使用特殊字符即可。正则表达式字面量的模式和修饰符直接写在斜杠之间。

在前端面试中，正则表达式是一个常见的考点。以下是一些与正则表达式相关的重要知识点总结：1.

基本语法 正则表达式是由字符和特殊字符组成的模式，用于匹配字符串中的文本。常见的特殊字符包括元字符（如.、*、+、?等）和字符类（如[...]、[^\...]、\d、\w等）。2. 匹配模式 使用正则表达式可以进行文本匹配、查找、替换等操作。匹配模式可以包括固定文本和通配符，用于定义要匹配的模式。量词（如*、+、?、{n}、{n,m}等）用于指定匹配的次数。

3. 常见的正则表达式应用场景 邮箱验证：匹配邮箱的正则表达式可以验证邮箱的合法性。密码验证：通过正则表达式可以验证密码的复杂度要求。手机号验证：使用正则表达式可以验证手机号码的格式是否正确。URL 提取：通过正则表达式可以从文本中提取出符合 URL 格式的链接。HTML 标签处理：正则表达式可以用于匹配和处理 HTML 标签。

4. 常见的正则表达式方法 test()：测试字符串是否匹配正则表达式。exec()：在字符串中查找匹配的文本，并返回匹配结果。match()：在字符串中查找匹配的文本，并返回所有匹配结果的数组。search()：在字符串中查找匹配的文本，并返回第一个匹配结果的索引。replace()：将匹配的文本替换为指定的字符串。split()：根据正则表达式将字符串拆分为数组。

```
// 示例字符串const str='Hello, World! This is a test string.';
// test(): 测试字符串是否匹配正则表达式const regex1=/test/; console.log(regex1.test(str)); // true
// exec(): 在字符串中查找匹配的文本，并返回匹配结果const regex2=/is/g; let result; while((result=regex2.exec(str))!=null){ console.log(result[0]); // "is" (每次循环匹配的结果) console.log(result.index); // 匹配的起始索引 }
// match(): 在字符串中查找匹配的文本，并返回所有匹配结果的数组const regex3=/o/g; console.log(str.match(regex3)); // ["o", "o", "o"]
// search(): 在字符串中查找匹配的文本，并返回第一个匹配结果的索引const regex4=/World/; console.log(str.search(regex4)); // 7
// replace(): 将匹配的文本替换为指定的字符串const regex5=/test/; const newStr=str.replace(regex5,'replacement'); console.log(newStr);
```

```
"Hello, World! This is a replacement string."
// split(): 根据正则表达式将字符串拆分为数组const regex6=/[.!\s]/; const arr=str.split(regex6); console.log(arr);
// ["Hello", "World", "This", "is", "a", "test", "string"]
5. 贪婪匹配和非贪婪匹配 贪婪匹配是指正则表达式默认匹配尽可能长的字符串。非贪婪匹配是指正则表达式匹配尽可能短的字符串，在量词后加上?实现非贪婪匹配。以下是一些常见的正则表达式面试题考点及其答案总结...
```

198. Javascript中callee和caller的作用？

arguments.callee指向当前执行的函数，fn.caller指向调用当前函数的函数，但两者都已经不推荐使用。callee过去常用于匿名递归，caller则能追踪调用来源，不过它们会带来安全性和性能问题，在严格模式下也不可用。递归场景我一般改用具名函数表达式，让函数通过名称调用自身。若业务需要记录调用来源，更稳妥的方式是显式传参，而不是读取调用栈。callee和caller是两个旧的非标准属性，它们在现代的JavaScript中已经不推荐使用，且在严格模式下不可用。caller属性：caller属性用于获取调用当前函数的函数的引用。它返回一个函数对象或者null。这个属性主要用于追踪函数的调用来源。但由于其潜在的安全性和性能问题，它已经被废弃了，并且在严格模式下无法使用。callee属性：callee属性返回当前正在执行的函数的引用。它通常在匿名函数或递归函数中使用，允许函数引用自身。但同样由于其潜在的安全性和性能问题，它也被废弃了，并且在严格模式下无法使用。

应该尽量避免使用caller和callee属性，而使用更现代的 JavaScript 特性和技术来完成相应的任务。例如，可以使用具名函数表达式来递归调用函数，使用函数引用作为参数传递来追踪函数调用的来源。

以下是一个示例，如果一对兔子每月生一对兔子；一对新生兔，从第二个月起就开始生兔子；假定每对兔

子都是一雌一雄，试问一对兔子，第n个月能繁殖成多少对兔子？（使用callee完成）
`var result=[];function fn(n){//典型的斐波那契数列if(n==1){return 1;}elseif(n==2){return 1;}else{if(result[n]){return result[n];}else{//argument.callee()表示fn()result[n]=arguments.callee(n-1)+arguments.callee(n-2);return result[n];}}}` 以下是一个示例，展示了如何使用具名函数表达式进行递归调用：
`const factorial=function calculateFactorial(n){if(n<=1){return 1;}return n*calculateFactorial(n-1);};console.log(factorial(5));// 输出: 120` 在这个示例中，使用具名函数表达式calculateFactorial来递归调用自身，计算阶乘的结果。这种方式比使用callee属性更清晰和可维护。 总结 caller是返回一个对函数的引用，该函数调用了当前函数；callee是返回正在被执行的function函数，也就是所指定的function对象的正文

199. window.onload和\$(document).ready

window.onload要等页面中的图片等资源全部加载完成，\$(document).ready()在DOM结构准备好后就会执行。因为等待范围不同，ready通常触发得更早，更适合做元素查询、事件绑定这类只依赖DOM的初始化。原生实现可以监听DOMContentLoaded，不需要为了这件事专门依赖jQuery。如果逻辑必须读取图片加载后的尺寸，再选择window.onload更合适。 原生JS的window.onload与Jquery的\$(document).ready(function(){}))有什么不同？如何用原生JS实现Jq的ready方法？ window.onload()方法是必须等到页面内包括图片的所有元素加载完毕后才能执行。\$(document).ready()是DOM结构绘制完毕后就执行，不必等到加载完毕
`function ready(fn){if(document.addEventListener){//标准浏览器document.addEventListener('DOMContentLoaded',function(){//注销事件，避免反复触发document.removeEventListener('DOMContentLoaded',arguments.callee,false);fn();//执行函数},false);}elseif(document.attachEvent){//IEdocument.attachEvent('onreadystatechange',function(){if(document.readyState=='complete'){document.detachEvent('onreadystatechange',arguments.callee);fn();//函数执行}}});}`

200. addEventListener()和attachEvent()的区别

addEventListener()是标准事件绑定方式，attachEvent()是早期IE的非标准实现，现代开发应使用前者。addEventListener()的事件名不带on，还能通过第三个参数选择捕获或冒泡；attachEvent()的事件名要带on，并且只在冒泡阶段处理。两者回调里的this也不同：前者指向触发事件的元素，后者指向window。 获取事件信息时，前者使用回调参数，后者通常依赖window.event。 addEventListener()是DOM Level 2 标准定义的方法，而attachEvent()是早期IE浏览器的非标准方法。addEventListener()可以为同一个元素的同一个事件类型添加多个事件处理函数，而attachEvent()只能绑定一个事件处理函数。addEventListener()使用事件捕获阶段或冒泡阶段来处理事件，可以通过第三个参数来指定是在捕获阶段处理还是在冒泡阶段处理。而attachEvent()只能在冒泡阶段处理事件。addEventListener()的事件处理函数中的this指向触发事件的元素，而attachEvent()的事件处理函数中的this指向window对象。addEventListener()中的事件处理函数可以通过event参数来获取事件信息，而attachEvent()的事件处理函数需要通过window.event来获取事件信息。 由于attachEvent()是早期IE浏览器的非标准方法，且在现代浏览器中已经被废弃，推荐使用addEventListener()来绑定事件。

201. 获取页面所有的checkbox

获取页面全部复选框，直接用document.querySelectorAll("input[type='checkbox']")即可。如果只要已选中的元素，可以再加上:checked伪类。返回值是静态NodeList，需要使用数组方法时，可通过扩展运算符或Array.from转成数组；也可以遍历所有input，再按type === 'checkbox'筛选。
`var resultArr=[];var input=document.querySelectorAll('input');for(var i=0;i<input.length;i++){if(input[i].type==='checkbox'){resultArr.push(input[i]);}}//resultArr即中获取到了页面中的所有checkbox`

202. 数组去重方法总结

普通数组去重我一般优先用[...new Set(arr)]，写法短，遍历规模下通常也是O(n)。兼容旧写法时，可以用循环配合indexOf、includes或相邻元素比较，但前两类可能达到O(n²)，排序还会改变元素顺序。需要哈希去重时，Map比普通对象更稳妥，因为对象键会转成字符串。还要注意indexOf无法识别NaN，而对象之间比较的是引用，所以内容相同也未必会被去掉。

方法一、利用ES6 Set去重（ES6中最常用）

```
function unique(arr){return Array.from(new Set(arr))}var arr=[1,1,'true','true',true,true,15,15,false,false,undefined,undefined,null,null,NaN,NaN,'NaN',0,0,'a','a',{},{},{}];console.log(unique(arr))//[1, "true", true, 15, false, undefined, null, NaN, "NaN", 0, "a", {}, {}]
```

方法二、利用for嵌套for，然后splice去重（ES5中最常用）

```
function unique(arr){for(var i=0;i<arr.length;i++){if(arr.indexOf(arr[i])!=i){arr.splice(i,1);i--}}return arr;}var arr=[1,1,'true','true',true,true,15,15,false,false,undefined,undefined,null,null,NaN,NaN,'NaN',0,0,'a','a',{},{},{}];console.log(unique(arr))//[1, "a", "true", true, 15, false, 1, {...}, null, NaN, NaN, "NaN", 0, "a", {...}, undefined]
```

方法十、利用Map数据结构去重

```
function arrayNonRepeatfy(arr){let map=new Map();let array=new Array();// 数组用于返回结果for(let i=0;i<arr.length;i++){if(!map.has(arr[i])){map.set(arr[i],1);array.push(arr[i])}}return array;}var arr=[1,1,'true','true',true,true,15,15,false,false,undefined,undefined,null,null,NaN,NaN,'NaN',0,0,'a','a',{},{},{}];console.log(arrayNonRepeatfy(arr))//[1, "true", true, 15, false, undefined, null, NaN, "NaN", 0, "a", {...}, {...}]
```

方法十二、[...new Set(arr)] [...new Set(arr)]

代码就是这么少----（其实，严格来说并不算是一种，相对于第一种方法来说只是简化了代码）

203. （设计题）想实现一个对页面某个节点的拖曳？如何做？（使用原生JS）

原生拖拽本质上是：mousedown记录鼠标和元素的初始位置，mousemove计算偏移并更新坐标，mouseup结束拖拽。mousemove和mouseup通常绑定在document上，否则鼠标移出元素后可能收不到事件。

位置可以更新left、top，也可以用transform: translate()，高频移动时再配合requestAnimationFrame。

实际使用还要限制浏览器边界；移动端则需处理touchstart、touchmove和touchend。

给需要拖拽的节点绑定mousedown,mousemove,mouseup事件，mousedown事件触发后，开始拖拽mousemove时，需要通过event.clientX和clientY获取拖拽位置，并实时更新位置mouseup时，拖拽结束需要注意浏览器边界的情况。

下面是一个使用原生JavaScript实现页面节点拖拽的示例代码：

```
// 获取需要拖拽的节点var draggableNode=document.getElementById("draggable");// 初始化拖拽状态var isDragging=false;var offset={x:0,y:0};// 绑定mousedown事件draggableNode.addEventListener("mousedown",function(event){// 设置拖拽状态为trueisDragging=true;// 计算鼠标相对于节点的偏移量offset.x=event.clientX-draggableNode.offsetLeft;offset.y=event.clientY-draggableNode.offsetTop;});// 绑定mousemove事件document.addEventListener("mousemove",function(event){// 如果处于拖拽状态if(isDragging){// 计算节点的新位置var left=event.clientX-offset.x;var top=event.clientY-offset.y;// 更新节点的位置draggableNode.style.left=left+"px";draggableNode.style.top=top+"px;}});// 绑定mouseup事件document.addEventListener("mouseup",function(){// 设置拖拽状态为falseisDragging=false;});
```

在上面的代码中，首先获取需要拖拽的节点draggableNode，然后初始化拖拽状态和偏移量。在mousedown事件中，设置拖拽状态为true，并计算鼠标相对于节点的偏移量。在mousemove事件中，如果处于拖拽状态，根据鼠标位置和偏移量计算节点的新位置，并更新节点的left和top样式。最后，在mouseup事件中，设置拖拽状态为false，表示拖拽结束。

需要注意的是，上述代码只实现了简单的拖拽功能，如果需要考虑边界情况、拖拽限制等，还需要进行适当的处理。

204. Javascript全局函数和全局变量

JavaScript常见全局属性有Infinity、NaN、undefined，常用全局函数包括parseInt、isNaN、encodeURIComponent、decodeURI和eval。浏览器里还能直接访问window、document、console，以及定时器和弹窗相关函数。编码完整地址时用encodeURIComponent，编码参数值时更适合encodeURIComponent，因为两者保留的URL结构字符不同。isNaN会发生隐式转换，判断时可优先用Number.isNaN，parseInt也建议明确传入进制。全局变量 Infinity代表正的无穷大的数值。NaN指示某个值是不是数字值。undefined指示未定义的值。Date表示日期和时间的对象。Math包含了数学相关的函数和常量。JSON用于解析和序列化JSON数据的对象。console用于在控制台输出信息的对象。document表示当前HTML文档的对象。window表示浏览器窗口的对象。全局函数 decodeURI()解码某个编码的URI。decodeURIComponent()解码一个编码的URI组件。encodeURIComponent()把字符串编码为URI。encodeURIComponent()把字符串编码为URI组件。escape()对字符串进行编码。eval()计算JavaScript字符串，并把它作为脚本代码来执行。isFinite()检查某个值是否为有穷大的数。isNaN()检查某个值是否是数字。Number()把对象的值转换为数字。parseInt()解析一个字符串并返回一个整数。String()把对象的值转换为字符串。unescape()对由escape()编码的字符串进行解码setTimeout()在指定的延迟时间后执行一次函数。setInterval()每隔指定的时间间隔重复执行函数。clearTimeout()取消使用setTimeout()创建的延迟执行。clearInterval()取消使用setInterval()创建的重复执行。alert()在浏览器中显示一个警告框。confirm()在浏览器中显示一个确认框。prompt()在浏览器中显示一个提示框，接收用户输入

205. 使用js实现一个持续的动画效果

持续动画一般用requestAnimationFrame递归调用，在每一帧更新状态并修改元素样式。它由浏览器配合屏幕刷新节奏调度，比固定频率的setInterval更适合页面动画；旧环境也可以回退到setTimeout。如果动画速度不能受刷新率影响，位移应根据时间差计算，而不是每帧固定加一。动画结束或组件卸载时要保存并取消任务；纯样式变化则优先考虑无限循环的CSS animation。定时器思路

```
var e=document.getElementById('e');var flag=true;var left=0;setInterval(()=>{left==0?flag=true:left==100?flag=false:"flag?e.style.left=`${left++}px`:e.style.left=`${left--}px`",1000/60})requestAnimationFrame //兼容性处理window.requestAnimationFrame=(function(){return window.requestAnimationFrame||window.webkitRequestAnimationFrame||window.mozRequestAnimationFrame||function(callback){window.setTimeout(callback,1000/60);}})();var e=document.getElementById("e");var flag=true;var left=0;function render(){left==0?flag=true:left==100?flag=false:"";flag?e.style.left=`${left++}px`:e.style.left=`${left--}px`;}(function animloop(){render();requestAnimationFrame(animloop);})();
```

使用css实现一个持续的动画效果

```
animation:mymove 5s infinite;@keyframes mymove{from{top:0px;}to{top:200px;}}
```

animation-name规定需要绑定到选择器的keyframe名称。animation-duration规定完成动画所花费的时间，以秒或毫秒计。animation-timing-function规定动画的速度曲线。animation-delay规定在动画开始之前的延迟。animation-iteration-count规定动画应该播放的次数。animation-direction规定是否应该轮流反向播放动画

206. 封装一个函数，参数是定时器的时间，.then执行回调函数

这个函数返回一个Promise，并在setTimeout到期后调用resolve，这样外部就能接着写.then()。例如sleep(3000).then(callback)，本质上是把定时器的回调形式包装成链式异步调用。这个封装只表示等待结束，不会阻塞主线程；如果业务还要求取消定时器，就需要额外保存任务标识并调用clearTimeout。

```
function sleep(time){return new Promise((resolve)=>setTimeout(resolve,time));} // 测试sleep(3000).then(()=>{console.log('定时器结束，执行回调函数');});
```

207. 怎么判断两个对象相等？

判断两个对象是否相等，要先明确比较的是同一个引用，还是内部属性和值都一致。===只能判断引用；简单对象可以比较序列化结果，但JSON.stringify()会受属性顺序和特殊值影响，不适合作为通用深比较。需

要比较嵌套对象或数组时，可以递归检查键数量、键是否存在以及对应值是否相等。实际项目若边界类型较多，我一般会直接使用lodash的isEqual()。obj={a:1,b:2}obj2={a:1,b:2}obj3={a:1,b:'2'}使用JSON.stringify()：将对象转换为字符串，然后进行比较。这种方式适用于对象中的属性值都是基本数据类型，并且属性的顺序对比较结果无影响。

```
JSON.stringify(obj)===JSON.stringify(obj2);// true
JSON.stringify(obj)===JSON.stringify(obj3);// false
```

使用循环遍历：逐个比较对象的属性值。这种方式适用于对象中的属性值类型复杂，包括嵌套对象或数组。

```
function deepEqual(obj1,obj2){// 比较基本数据类型的值if(obj1===obj2){return true;}//
```

```
比较对象的属性个数if(Object.keys(obj1).length!==Object.keys(obj2).length){return false;}// 逐个比较对象的属性值for(let key in obj1){if(!obj2.hasOwnProperty(key)||!deepEqual(obj1[key],obj2[key])){return false;}}return true;}deepEqual(obj,obj2);// true
deepEqual(obj,obj3);// false
```

使用lodash库的isEqual()方法：lodash是一个流行的JavaScript工具库，其中的isEqual()方法可以比较两个对象是否相等，包括深度比较。首先，需要通过npm install lodash命令安装lodash库，然后在代码中引入isEqual()方法：

```
const _=require('lodash');_isEqual(obj,obj2);// true
_isEqual(obj,obj3);// false
```

208. 项目做过哪些性能优化？

项目性能优化要针对实际瓶颈处理，通常会从资源加载、页面渲染和框架运行三个方向入手。加载侧可做代码分割、路由与图片懒加载、资源压缩和缓存，目的是减少首屏需要下载的内容；静态资源也可以交给CDN。渲染侧会减少频繁的DOM操作，大列表使用虚拟化，并避免没有必要的组件更新。框架层再结合Vue的keep-alive、合理使用v-if，或React.memo()、React

Profiler等手段，但最终要通过分析工具确认瓶颈，不能把所有优化机械地堆上去。这是一些常见的性能优化措施，可以在项目中采取来提升网页的加载速度和性能。以下是一些常见的性能优化措施：减少 HTTP 请求数：合并和压缩 CSS、JavaScript

文件，使用雪碧图、字体图标等减少图片请求，减少不必要的资源请求。减少 DNS

查询：减少使用不同的域名，以减少 DNS 查询次数。使用 CDN：将静态资源部署到 CDN 上，提供更快的访问速度。避免重定向：确保网页没有多余的重定向，减少额外的网络请求。图片懒加载：延迟加载图片

，只有当图片进入可视区域时再进行加载。减少 DOM 元素数量：优化页面结构，减少 DOM 元素的数量，提升渲染性能。减少 DOM 操作：避免频繁的 DOM 操作，合并操作或使用 DocumentFragment 进行批量操作。

```
var container=document.getElementById("container");var fragment=document.createDocumentFragment();for(var i=0;i<data.length;i++){var div=document.createElement("div");div.innerHTML=template(data[i]);container.appendChild(div);fragment.appendChild(div);}container.appendChild(fragment);
```

压缩文件：压缩

JavaScript、CSS、字体、图片等静态资源文件，减小文件大小。优化 CSS

Sprite：将多个小图标合并为一个大图，并通过 CSS 进行定位，减少图片请求。压缩文件：压缩

JavaScript、CSS、字体、图片等静态资源文件，减小文件大小。优化 CSS

Sprite：将多个小图标合并为一个大图，并通过 CSS 进行定位，减少图片请求。

```
.icon{background-image:url("sprite.png");background-position:-10px
```

```
-20px;width:20px;height:20px;}使用
```

iconfont：将图标字体作为替代图像，减少图片请求并提高渲染性能。；字体裁剪：只加载页面上实际使用的字体字符，减少字体文件的大小。需要使用字体工具（如Fontello、IcoMoon等）进行裁剪

多域名分发：将网站的内容划分到不同的域名下，以提高并发请求的能力。需要在项目中配置不同的域名

或子域名减少使用 iframe：避免频繁使用

iframe，因为它们会增加额外的网络请求和页面加载时间。避免图片 src 为空：确保 img 标签的 src 属性不为空，避免浏览器发送不必要的请求。把样式表放在 link 中：避免使用内联样式，将样式表放在 link 标签中，使浏览器可以并行加载样式和内容。把 JavaScript 放在页面底部：将 JavaScript

脚本放在页面底部，使页面内容可以先加载完毕，提升用户体验。webpack性能优化

使用生产模式（production mode）：在Webpack配置中设置mode为production，这将启用许多内置的优化功能，例如代码压缩、作用域提升等。代码分割（Code Splitting）：使用Webpack的代码分割功能，将代码拆分为多个小块，按需加载，避免打包一个巨大的文件。懒加载（Lazy Loading）：使用动态导入（Dynamic Import）或import()函数，按需加载模块，在需要时才加载相关代码。Tree Shaking：通过配置Webpack的optimization选项，启用sideEffects和usedExports，以消除未使用的代码（dead code）。缓存：使用Webpack的chunkhash或contenthash生成文件名，实现缓存机制，利用浏览器缓存已经加载的文件。并行处理（Parallel Processing）：使用thread-loader或HappyPack插件，将Webpack的构建过程多线程化，加速构建速度。使用缩小作用域（Narrowing the Scope）：通过配置Webpack的resolve选项，缩小模块解析的范围，减少不必要的查找。使用外部依赖（External Dependencies）：将一些稳定的、不经常修改的库或框架通过externals配置排除，使用CDN引入，减少打包体积。使用插件和加载器（Plugins and Loaders）：选择高效的插件和加载器，合理配置它们的选项，以优化构建过程和资...

209. 浏览器缓存

浏览器缓存分为强缓存和协商缓存，前者直接读取本地资源，后者会向服务器确认资源是否有变化。强缓存常用Cache-Control: max-age或Expires；失效后再通过ETag或Last-Modified协商，未变化就返回304。两者命中时服务器都不返回资源正文，区别在于是否发送请求。普通刷新会检查协商缓存，强制刷新则会跳过两类缓存。

浏览器缓存分为强缓存和协商缓存。当客户端请求某个资源时，获取缓存的流程如下：先根据这个资源的一些http header判断它是否命中强缓存，如果命中，则直接从本地获取缓存资源，不会发请求到服务器；当强缓存没有命中时，客户端会发送请求到服务器，服务器通过另一些request header验证这个资源是否命中协商缓存，称为http再验证，如果命中，服务器将请求返回，但不返回资源，而是告诉客户端直接从缓存中获取，客户端收到返回后就会从缓存中获取资源；强缓存和协商缓存共同之处在于，如果命中缓存，服务器都不会返回资源；区别是，强缓存不对发送请求到服务器，但协商缓存会。当协商缓存也没命中时，服务器就会将资源发送回客户端。当ctrl+f5强制刷新网页时，直接从服务器加载，跳过强缓存和协商缓存；当f5刷新网页时，跳过强缓存，但是会检查协商缓存；强缓存 Expires（该字段是http 1.0时的规范，值为一个绝对时间的GMT格式的时间字符串，代表缓存资源的过期时间）Cache-Control: max-age（该字段是http1.1的规范，强缓存利用其max-age值来判断缓存资源的最大生命周期，它的值单位为秒）

```
const http = require('http');
const fs = require('fs');
const path = require('path');
http.createServer((req, res) => {
  const filePath = path.join(__dirname, 'public', req.url);
  fs.readFile(filePath, (err, data) => {
    if (err) {
      res.writeHead(404);
      res.end('File not found');
      return;
    }
    const stat = fs.statSync(filePath);
    const expires = new Date(Date.now() + 3600000);
    // 设置缓存过期时间为1小时
    res.setHeader('Expires', expires.toUTCString());
    res.setHeader('Cache-Control', 'max-age=3600');
    res.writeHead(200);
    res.end(data);
  });
}).listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

协商缓存 Last-Modified（值为资源最后更新时间，随服务器response返回）If-Modified-Since（通过比较两个时间来判断资源在两次请求期间是否有过修改，如果没有修改，则命中协商缓存）ETag（表示资源内容的唯一标识，随服务器response返回）If-None-Match（服务器通过比较请求头部的If-None-Match与当前资源的ETag是否一致来判断资源是否在两次请求之间有过修改，如果没有修改，则命中协商缓存）

```
const http = require('http');
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
http.createServer((req, res) => {
  const filePath = path.join(__dirname, 'public', req.url);
  fs.readFile(filePath, (err, data) => {
    if (err) {
      res.writeHead(404);
      res.end('File not found');
      return;
    }
    const stat = fs.statSync(filePath);
    const lastModified = stat.mtime.toUTCString();
    const ifModifiedSince = req.headers['if-modified-since'];
    const fileHash = crypto.createHash('md5').update(data).digest('hex');
    const etag = `"${fileHash}"`;
    const ifNoneMatch = req.headers['if-none-match'];
```


210. 谈谈你对WebSocket的理解

WebSocket是建立在持久连接上的双向通信协议，客户端和服务端都能主动发送消息。相比反复轮询，它的延迟和控制开销更低，而且既能传文本，也能传二进制数据。连接地址使用ws://或加密的wss://，浏览器通过new WebSocket(url)建连，再用事件监听收发和关闭状态。它适合在线聊天、实时数据展示和多人协同等持续推送场景。由于http存在一个明显的弊端（消息只能有客户端推送到服务器端，而服务器端不能主动推送到客户端），导致如果服务器如果有连续的变化，这时只能使用轮询，而轮询效率过低，并不适合。于是WebSocket被发明出来

WebSocket 是一种在 Web 应用程序中实现双向通信的协议。与传统的 HTTP 请求-响应模式不同，WebSocket 提供了持久连接，使服务器能够主动向客户端推送数据，而不需要客户端发起请求。以下是对 WebSocket 的理解：

双向通信：WebSocket 允许客户端和服务端之间建立持久连接，并通过这个连接进行双向通信。客户端和服务端可以随时发送消息给对方，实现实时的数据传输。

实时性：相比传统的 HTTP 请求-响应模式，WebSocket 具有更低的延迟和更高的实时性。服务器可以立即将数据推送给客户端，而不需要等待客户端的请求。

协议标识符：WebSocket 使用ws://（非加密）或wss://（加密）作为协议标识符，用于建立与服务器的连接。

较少的控制开销：WebSocket 的协议控制数据包头部较小，不需要携带完整的头部信息，减少了数据传输的开销。

支持文本和二进制数据：WebSocket 不仅可以传输文本数据，还可以传输二进制数据，使得它适用于各种类型的应用场景。

支持扩展：WebSocket 协议定义了扩展机制，允许用户自定义扩展或实现自定义的子协议，例如压缩算法、认证机制等。

无跨域问题：WebSocket 协议不存在跨域限制，可以轻松地在不同域名下进行通信。

简单实现：实现 WebSocket 相对简单，服务器端和客户端都有相应的库或 API 可以使用，例如 Node.js 中的 socket.io、ws 等，客户端则可以使用浏览器提供的 WebSocket API。

总的来说，WebSocket 提供了一种高效、实时的双向通信机制，使得 Web 应用程序可以实现实时更新、即时通信等功能。它具有较低的延迟、支持文本和二进制数据传输、无跨域限制等优势，可以广泛应用于在线聊天、实时数据展示、多人协同编辑等领域。

1. WebSocket 示例代码： 以下是一个简单的使用 WebSocket 的示例代码，包括客户端和服务端的实现：

客户端代码（JavaScript）：

```
// 创建 WebSocket 连接
const socket = new WebSocket('ws://localhost:3000');

// 监听连接建立事件
socket.addEventListener('open', () => {
  console.log('Connected to server');
});

// 发送消息给服务器
socket.send('Hello server!');

// 监听接收到消息事件
socket.addEventListener('message', (event) => {
  const message = event.data;
  console.log('Received message:', message);
});

// 监听连接关闭事件
socket.addEventListener('close', () => {
  console.log('Disconnected from server');
});
```

服务器端代码（Node.js）：

```
const WebSocket = require('ws');

// 创建 WebSocket 服务器
const wss = new WebSocket.Server({ port: 3000 });

// 监听连接建立事件
wss.on('connection', (socket) => {
  console.log('Client connected');
});

// 监听接收到消息事件
socket.on('message', (message) => {
  console.log('Received message:', message);
  // 发送消息给客户端
  socket.send('Hello client!');
});

// 监听连接关闭事件
socket.on('close', () => {
  console.log('Client disconnected');
});
```

上述示例中，客户端通过new WebSocket(url)创建一个 WebSocket 连接，监听连接建立、接收到消息和...

211. 尽可能多的说出你对 Electron 的理解

Electron是把Chromium和Node.js结合起来，用HTML、CSS、JavaScript开发跨平台桌面应用的框架。Chromium负责页面渲染，Node.js则让应用能够访问系统资源并执行本地操作。它还能提供菜单、对话框、托盘、通知和自动更新等桌面能力，调试时也能使用开发者工具。它适合编辑器、聊天工具和播放器等需要复用Web技术栈的应用。Electron 是一个用于构建跨平台桌面应用程序的开源框架。它将 Chromium 嵌入到一个 Node.js 运行时环境中，使开发者可以使用 Web 技术（HTML、CSS 和

JavaScript) 来构建桌面应用程序。 以下是对 Electron 的理解： 跨平台开发：Electron 提供了一种使用 Web 技术来构建跨平台桌面应用程序的方式。开发者可以使用 HTML、CSS 和 JavaScript 来构建应用程序界面，而不必关心不同操作系统的差异。基于 Chromium 和 Node.js：Electron 是在 Chromium 渲染引擎的基础上构建的，因此可以充分利用 Chrome 提供的强大功能和先进的 Web 标准支持。同时，它还集成了 Node.js，使开发者可以使用 JavaScript 访问底层系统资源和执行本地操作。原生体验：Electron 应用程序可以获得与本机桌面应用程序相似的用户体验，包括菜单、对话框、系统托盘、通知等。通过使用 Electron 的 API，开发者可以轻松地创建原生风格的界面和交互。丰富的生态系统：Electron 拥有庞大的开发者社区和丰富的插件生态系统，可以方便地集成第三方库和工具，以扩展应用程序的功能和性能。自动更新：Electron 提供了自动更新机制，使得应用程序能够自动下载和安装最新的版本，提供给用户更好的体验和功能。调试和开发工具：Electron 集成了开发工具，包括 Chrome 开发工具和 Node.js 调试器，方便开发者进行调试和性能优化。广泛的应用领域：Electron 被广泛应用于构建桌面应用程序，包括编辑器、IDE、聊天工具、音乐播放器、视频播放器、游戏等。总的来说，Electron 提供了一种便捷的方式来使用 Web 技术构建跨平台桌面应用程序，并且具有与本机应用程序相似的用户体验和功能。它融合了 Chromium 和 Node.js 的强大特性，拥有活跃的社区和丰富的插件生态系统，使得开发者能够高效地构建功能丰富、易于维护的桌面应用程序。

212. 深浅拷贝

浅拷贝只复制对象的第一层，深拷贝则继续复制嵌套对象，让副本修改后不影响原对象。Object.assign() 和展开运算符都属于浅拷贝，因此遇到嵌套引用时仍会共享数据。深拷贝可以递归实现，也可以使用 lodash 的 cloneDeep()；简单数据还可使用 JSON.parse(JSON.stringify())。不过 JSON 方案会丢失函数和 undefined，也不能正确处理循环引用及部分特殊对象。

浅拷贝 使用 Object.assign() 方法

```
let obj1 = {a: 1, b: 2}; let obj2 = Object.assign({}, obj1); console.log(obj2); // { a: 1, b: 2 } 使用展开运算符
let obj1 = {a: 1, b: 2}; let obj2 = {...obj1}; console.log(obj2); // { a: 1, b: 2 } 深拷贝 1. 使用递归实现深拷贝函数
function deepClone(obj) { if (obj === null || typeof obj !== 'object') { return obj; } let clone = Array.isArray(obj) ? [] : {}; for (let key in obj) { if (obj.hasOwnProperty(key)) { clone[key] = deepClone(obj[key]); } } return clone; }
let obj1 = {a: 1, b: {c: 2}}; let obj2 = deepClone(obj1); obj2.b.c = 3; console.log(obj1.b.c); // 2 console.log(obj2.b.c); // 3
2. 使用第三方库 lodash 的 cloneDeep() 方法 const _ = require('lodash'); let obj1 = {a: 1, b: {c: 2}}; let obj2 = _.cloneDeep(obj1); obj2.b.c = 3; console.log(obj1.b.c); // 2 console.log(obj2.b.c); // 3
3. 使用 JSON.parse(JSON.stringify()) 实现深拷贝
let obj1 = {a: 1, b: {c: 2}}; let obj2 = JSON.parse(JSON.stringify(obj1)); obj2.b.c = 3; console.log(obj1.b.c); // 2 console.log(obj2.b.c); // 3
该方法也是有局限性的
```

会忽略 undefined 不能序列化函数 不能解决循环引用的对象

JSON.stringify() 方法在实现深拷贝时有一些局限性，包括：

无法处理函数：JSON.stringify() 方法在序列化对象时会忽略函数属性，因为函数不符合 JSON 格式的数据类型。经过序列化和反序列化后，函数属性会丢失。 let obj = {name: 'poetry', sayHello: function() { console.log('Hello!'); }}; let serializedObj = JSON.stringify(obj); let clonedObj = JSON.parse(serializedObj); console.log(clonedObj.name); // 'poetry' console.log(typeof clonedObj.sayHello); // 'undefined' 无法处理循环引用：如果对象存在循环引用，即对象内部包含对自身的引用，JSON.stringify() 方法无法正确处理，会导致循环引用的属性被序列化为 null。 let obj = {name: 'poetry'}; obj.self = obj; let serializedObj = JSON.stringify(obj); console.log(serializedObj); // {"name": "poetry", "self": null} 无法处理特殊对象：JSON.stringify() 方法无法序列化某些特殊对象，如 Date 对象、正则表达式、Map、Set 等，它们在序列化过程中会转换成空对象。 let obj = {now: new Date(), regex: /[a-z]+/, set: new Set([1, 2, 3]), map: new

```
Map([[1,'one'],[2,'two']]);letserializedObj=JSON.stringify(obj);console.log(serializedObj);//
{"now":{},"regex":{},"set":{},"map":{}} 无法处理undefined属性：JSON.stringify()方法在序列化对象时会忽略undefined属性，序列化后的结果不包含该属性...
```

213. 防抖/节流

防抖是连续触发结束后只执行一次，节流是持续触发时按固定间隔执行。防抖会在每次触发时重置计时器，所以搜索联想、输入校验这类场景通常只处理最后一次操作。节流限制的是执行频率，更适合scroll、mousemove和拖拽等需要持续反馈的事件。时间戳节流首次会立即执行但可能丢掉末次，定时器节流首次延迟执行却能保留末次。

1. 防抖 防抖函数原理：把触发非常频繁的事件合并成一次去执行在指定时间内只执行一次回调函数，如果在指定的时间内又触发了该事件，则回调函数的执行时间会基于此刻重新开始计算防抖动和节流本质是不一样的。防抖是将多次执行变为最后一次执行，节流是将多次执行变成每隔一段时间执行 eg. 像百度搜索，就应该用防抖，当我连续不断输入时，不会发送请求；当我一段时间内不输入了，才会发送一次请求；如果小于这段时间继续输入的话，时间会重新计算，也不会发送请求。

手写简化版：// func是用户传入需要防抖的函数// wait是等待时间constdebounce=(func,wait=50)=>{// 缓存一个定时器idlettimer=0// 这里返回的函数是每次用户实际调用的防抖函数// 如果已经设定过定时器了就清空上一次的定时器// 开始一个新的定时器，延迟执行用户传入的方法returnfunction(...args){if(timer)clearTimeout(timer)timer=setTimeout(()=>{func.apply(this,args)},wait)}}}

适用场景： 文本输入的验证，连续输入文字后发送 AJAX 请求进行验证，验证一次就好按钮提交场景：防止多次提交按钮，只执行最后提交的一次服务端验证场景：表单验证需要服务端配合，只执行一段连续的输入事件的最后一次，还有搜索联想词功能类似 2. 节流 节流函数原理:指频繁触发事件时，只会在指定的时间段内执行事件回调，即触发事件间隔大于等于指定的时间才会执行回调函数。总结起来就是：事件，按照一段时间的间隔来进行触发。 像dom的拖拽，如果用消抖的话，就会出现卡顿的感觉，因为只在停止的时候执行了一次，这个时候就应该用节流，在一定时间内多次执行，会流畅很多 手写简版 使用时间戳的节流函数会在第一次触发事件时立即执行，以后每过wait秒之后才执行一次，并且最后一次触发事件不会被执行

时间戳方式： // func是用户传入需要防抖的函数// wait是等待时间constthrottle=(func,wait=50)=>{//

上一次执行该函数的时间letlastTime=0returnfunction(...args){// 当前时间letnow=+newDate()// 将当前时间和上一次执行函数时间对比// 如果差值大于设置的等待时间就执行函数if(now-lastTime>wait){lastTime=nowfunc.apply(this,args)}}setInterval(throttle(()=>{console.log(1)},500),1) 定时器方式：使用定时器的节流函数在第一次触发时不会执行，而是在 delay 秒之后才执行，当最后一次停止触发后，还会再执行一次函数 functionthrottle(func,delay){vartimer=0;returnfunction(){varcontext=this;varargs=arguments;if(timer)return//

当前有任务了，直接返回timer=setTimeout(function(){func.apply(context,args);timer=0;},delay);}}}

适用场景： 拖拽场景：固定时间内只执行一次，防止超高频次触发位置变动。DOM元素的拖拽功能实现（mousemove） 缩放场景：监控浏览器resize滚动场景：监听滚动scroll事件判断是否到页面底部自动加载更多动画场景：避免短时间内多次触发动画引起性能问题 总结 函数防抖：限制执行次数，多次密集的触发只执行一次将几次操作合并为一次操作进行。原理是维护一个计时器，规定在delay时间后触发函数，但是在delay时间内再次触发的话，就会取消之前的计时器而重新设置。这样一来，只有最后一次操作能被触发。函数节流：限制执行的频率，按照一定的时间间隔有节奏的执行使得一定时间内只触发一次函数。原理是通过判断是否到达一定时间来触发函数。 将几次操作合并为一次操作进行。原理是维护一个计时器，规定在delay时间后触发函数，但是在delay时间内再次触发的话，就会取消之前的计时器而重新设置。这样一来，只有最后一次操作能被触发。

使得一定时间内只触发一次函数。原理是通过判断是否到达一定时间来触...

214. 谈谈变量提升？

变量提升是执行环境创建时预先处理声明，但赋值仍留在原来的代码位置。var声明会被初始化为undefined，因此声明前读取不会报错，只是拿不到后面的赋值。函数声明会连同函数体一起进入内存，而且同名函数中后声明的会覆盖前面的，处理优先级也高于变量声明。let和const同样会被处理，但声明前处于暂时性死区，访问会抛出ReferenceError。变量提升是 JavaScript 中的一种行为，它指的是在代码执行过程中，变量和函数的声明会在其所在作用域的顶部被提升到执行环境中的过程。这意味着可以在变量或函数声明之前使用它们，而不会引发错误。在 JavaScript 中，使用 var 声明变量时会发生变量提升。具体来说，变量声明会在代码执行前的编译阶段被解析并添加到执行环境中，但是变量的赋值操作会保留在原来的位置。这就导致了以下的行为：当执行 JS 代码时，会生成执行环境，只要代码不是写在函数中的，就是在全局执行环境中，函数中的代码会产生函数执行环境，只此两种执行环境

1. 变量声明会被提升，但赋值操作不会被提升：`console.log(a);// undefined`
`vara=10;`
上述代码在执行时会被解析为：`vara;console.log(a);// undefined`
a=10; 变量提升 这是因为函数和变量提升的原因。通常提升的解释是说将声明的代码移动到了顶部，这其实没有什么错误，便于大家理解。但是更准确的解释应该是：在生成执行环境时，会有两个阶段。第一个阶段是创建的阶段，JS 解释器会找出需要提升的变量和函数，并且给他们提前在内存中开辟好空间，函数的话会将整个函数存入内存中，变量只声明并且赋值为undefined，所以在第二个阶段，也就是代码执行阶段，我们可以直接提前使用

2. 函数优先于变量提升 在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升

```
b()// call  
b  
secondfunctionb(){console.log('call b fist')}functionb(){console.log('call b second')}varb='Hello world'  
var会产生很多错误，所以在ES6中引入了let。let不能在声明前使用，但是这并不是常说的let不会提升，let提升了，在第一阶段内存也已经为他开辟好了空间，但是因为这个声明的特性导致了并不能在声明前使用
```

总结 变量提升是 JavaScript 的一种行为，将变量和函数声明提升到作用域的顶部。使用var声明的变量会被提升，但赋值操作保留在原来的位置。在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升使用let和const声明的变量也存在变量提升，但在声明前访问会引发暂时性死区错误。

215. 什么是单线程，和异步的关系

JavaScript单线程是指主线程同一时刻只能按顺序执行一段代码，而异步用于避免耗时操作一直阻塞它。网络请求或文件读取可以交给其他线程或进程处理，让主线程继续响应页面和执行后续任务。操作完成后，再通过回调、Promise或async/await通知主线程处理结果。本质上异步没有把主线程变成多线程，只是把等待过程挪开了；如果任务本身长期占用主线程，页面仍会失去响应。在 JavaScript 中，单线程指的是 JavaScript 引擎在执行代码时只有一个主线程，也就是说一次只能执行一条指令。这意味着 JavaScript 代码是按照顺序执行的，前一段代码执行完成后才会执行下一段代码。

异步是一种编程模型，用于处理非阻塞的操作。在 JavaScript 中，异步编程可以通过回调函数、Promise、async/await等方式来实现。异步操作不会阻塞主线程的执行，从而提高了程序的响应性能和用户体验。异步的关系与单线程密切相关，因为 JavaScript 是单线程的，如果所有的操作都是同步的，那么一旦遇到一个耗时的操作，比如网络请求或文件读取，整个程序都会被阻塞，用户界面也会停止响应，导致用户体验差。通过使用异步编程模型，可以将耗时的操作委托给其他线程或进程来处理，使得主线程可以继续执行其他任务，提高了程序的并发性和响应性。当异步操作完成后，通过回调函数或 Promise 的方式通知主线程，主线程再执行相应的回调逻辑。

总结一下：JavaScript是单线程的，只有一个主线程用于执行代码。异步编程是一种处理非阻塞操作的方式，提高程序的响应性能和用户体验。异步操作可以将耗时的任务委托给其他线程或进程处理，主线程继续执行其他任务。异步操作完成后通过回调函数或Promise的方式通知主线程。

216. 前端面试之hybrid

Hybrid是把原生应用与Web技术结合起来，在原生容器中嵌入WebView运行页面。业务界面可以用HTML、CSS和JavaScript跨平台开发，并通过桥接能力调用相机、定位或推送等原生功能。它还能借助在线更新更快地修复问题和发布功能，开发效率通常比完全原生更高。取舍是复杂图形和动画可能受WebView性能限制，因此对原生体验要求很高的功能需要谨慎评估。

<http://blog.poetries.top/2018/10/21/fe-interview-hybrid/>(opens new window) Hybrid（混合应用）是指结合了原生应用和Web技术开发的程序。它通常在移动应用开发中使用，允许开发人员使用Web技术（如HTML、CSS和JavaScript）来构建跨平台的移动应用，并在原生应用中嵌入Web视图。

以下是我对Hybrid的理解：跨平台开发：Hybrid应用具有跨平台的优势，通过使用Web技术开发一次，可以在多个平台上运行，如iOS和Android。这样可以节省开发时间和成本，并且能够更快地推出产品。原生功能访问：Hybrid应用可以利用原生应用提供的功能和特性，如相机、地理定位、推送通知等。通过使用桥接技术，可以在Web视图中调用原生代码，实现对原生功能的访问和调用。Web技术栈：Hybrid应用使用Web技术栈进行开发，包括HTML、CSS和JavaScript。开发人员可以使用熟悉的Web开发工具和框架来构建应用程序，并且可以利用丰富的Web生态系统中的第三方库和工具。在线更新：Hybrid应用可以通过Web进行在线更新，不需要用户手动更新应用程序。这使得开发人员能够快速修复错误、添加新功能，并将这些变更推送给用户，提供更好的用户体验。性能权衡：与原生应用相比，Hybrid应用在性能方面可能存在一些权衡。由于在Web视图中运行，Hybrid应用的性能可能受到一些限制，特别是在处理复杂的图形和动画效果时。然而，随着Web技术的不断发展，这些性能限制正在逐渐减小。总的来说，Hybrid应用是一种将Web技术与原生应用相结合的开发模式，提供了跨平台开发、访问原生功能、在线更新等优势。它在移动应用开发中具有一定的灵活性和便利性，可以满足开发人员快速开发和发布应用程序的需求。

以下是一个简单的示例代码，展示了如何使用Hybrid开发框架（例如Ionic）创建一个基本的Hybrid应用：Hybrid App
Hybrid App
Welcome to Hybrid App!
This is a hybrid application developed using Ionic framework.
// 在Ionic的Angular控制器中编写业务逻辑
angular.module('starter',['ionic']).controller('AppController',function(\$scope){\$scope.message="Hello, Hybrid App!";});
// 启动Ionic应用
ionic.Platform.ready(function(){angular.bootstrap(document,['starter']));};
上述示例中使用了Ionic框架，通过引入Ionic的CSS和JavaScript库，我们可以使用Ionic提供的UI组件和工具来构建Hybrid应用。在应用的脚本代码中，使用AngularJS来编写业务逻辑，控制器中定义了一个message变量，可以在视图中显示。最后，通过ionic.Platform.ready来启动Ionic应用。请注意，这只是一个简单的示例，实际的Hybrid应用可能包含更多的功能和复杂性。不同的Hybrid开发框架可能有不同的用法和特性，具体的开发代码和结构会根据所选框架而有所不同。

217. 前端面试之组件化

组件化就是把复杂的UI拆成职责单一、能复用和组合的小单元，从而降低维护成本。组件之间最好保持单向数据流，父组件通过props传入数据，子组件通过事件回调通知变化，避免状态互相修改。复用逻辑可以优先考虑Hooks或Composition API，它们比Mixin、HOC更容易追踪来源。工程中我会区分基础组件、业务组件和页面组件，只有基础组件需要追求高度通用，并配合TypeScript和Storybook约束接口、维护文档。
<http://blog.poetries.top/2018/10/21/fe-interview-component/>(opens new window)

218. 前端面试之MVVM浅析

MVVM通过ViewModel连接Model和View，让数据与视图能够自动同步。相比MVC由Controller操作视图、MVP由Presenter调用视图接口，MVVM把这部分工作交给数据绑定。它本质上是让开发者少写手动DOM操作，把注意力放在数据变化上。实现时通常依赖响应式系统完成依赖收集和更新派发，再由模板编译建立绑定；Vue接近这一模式，而React更强调单向数据流。

<http://blog.poetries.top/2018/10/21/fe-interview-mvvm/>(opens new window)

219. 实现效果，点击容器内的图标，图标边框变成border 1px solid red，点击空白处重置

可以用事件委托在容器上监听click，点中图标时切换选中类，点到容器外时清除选中状态。容器内通过event.target.closest('.icon')判断目标，命中后先移除其他图标的选中类，再给当前图标添加。文档级监听里可用container.contains(event.target)判断是否点在外部，因此不一定必须调用stopPropagation。样式建议通过class管理；若border会引发布局跳动，可以预留透明边框或改用outline。const box=document.getElementById('box');function isIcon(target){return target.className.includes('icon');}box.onClick=function(e){e.stopPropagation();const target=e.target;if(isIcon(target)){target.style.border='1px solid red;'}}const doc=document;doc.onclick=function(e){const children=box.children;for(let i;<children.length;i++){if(isIcon(children[i])){children[i].style.border='none';}}}

220. 请简单实现双向数据绑定MVVM

简单的双向绑定可以用Object.defineProperty劫持数据属性，再监听输入事件，把数据和输入框的值互相同步。设置data.text时在setter中更新input.value，用户输入时则把event.target.value回写到data.text。这两条链路分别解决“数据变、视图变”和“视图变、数据变”，本质上是两个方向的同步。这个写法只适合演示；完整的MVVM还要处理依赖收集、更新派发和模板编译，也可以用Proxy实现对象代理。const data={};const input=document.getElementById('input');Object.defineProperty(data,'text',{get(){return this.value;},set(value){input.value=value;this.value=value;}});input.addEventListener('change',function(e){data.text=e.target.value;});当你修改输入框的值时，data.text会更新，而当你设置data.text的值时，输入框的值也会更新。这实现了简单的双向数据绑定。请注意，这只是一个基础示例，实际的MVVM框架会更复杂且功能更强大

221. 实现Storage，使得该对象为单例，并对localStorage进行封装设置setItem(key, value)和getItem(key)

可以在类外保存唯一实例，并通过静态方法getInstance统一获取，再封装localStorage的读写。第一次调用时创建Storage，后续直接返回已有实例，因此整个运行环境只会使用同一个对象。setItem和getItem分别转调原生同名方法，调用方不用直接接触localStorage。需要注意，原生存储返回的是字符串，键不存在时会得到null。var instance=null;class Storage{static getInstance(){if(!instance){instance=new Storage();}return instance;}setItem(key,value){localStorage.setItem(key,value);}getItem(key){return localStorage.getItem(key);}}现在，你可以使用Storage.getInstance()来获取Storage的单例对象，并使用setItem和getItem方法来设置和获取localStorage中的值。// 使用示例const storage=Storage.getInstance();storage.setItem('name','poetry');const name=storage.getItem('name');console.log(name);// 输出: poetry

222. 谈谈你对Event Loop的理解

Event Loop的核心是同步代码执行完后清空微任务，再进入下一轮宏任务。JavaScript是单线程的，异步回调会先进入对应队列，等执行栈空闲后才有机会运行。像Promise.then属于微任务，setTimeout、setInterval属于宏任务，所以前者通常会先执行。浏览器必要时会在两轮任务之间渲染界面，而Node.js的任务类型和调度细节并不完全相同。前端面试中关于事件循环(Event Loop)的考点主要包括以下内容：事件循环的基本原理：介绍JavaScript的单线程特性，事件循环的概念和工作原理，以及任务队列(Task Queue)的概念。宏任务和微任务：区分宏任务(Macrotask)和微任务(Microtask)的概念，理解它们在事件循环中的执行顺序。常见的宏任务和微任务：了解常见的宏任务和微任务的类型，如setTimeout、setInterval、Promise、MutationObserver等。异步操作的执行顺序：理解异步操作的执行顺序，如何

在事件循环中处理异步代码，微任务优先于宏任务执行等。宏任务中的异步操作：了解在宏任务中的异步操作（例如setTimeout）是如何被添加到任务队列中的，以及它们的执行时机。浏览器中的事件循环和Node.js 中的事件循环：了解浏览器环境和 Node.js 环境下事件循环的差异，如setImmediate的区别等。了解和掌握事件循环的原理和机制对于理解 JavaScript 异步编程非常重要。在面试中，常常会通过让求职者解释事件循环的执行顺序、分析代码的输出结果等方式来考察他们对事件循环的理解。首先，js是单线程的，主要的任务是处理用户的交互，而用户的交互无非就是响应DOM的增删改，使用事件队列的形式，一次事件循环只处理一个事件响应，使得脚本执行相对连续，所以有了事件队列，用来储存待执行的事件，那么事件队列的事件从哪里被push进来的呢。那就是另外一个线程叫事件触发线程做的事情了，他的作用主要是在定时触发器线程、异步HTTP请求线程满足特定条件下的回调函数push到事件队列中，等待js引擎空闲的时候去执行，当然js引擎执行过程中有优先级之分，首先js引擎在一次事件循环中，会先执行js线程的主任务，然后会去查找是否有微任务microtask（promise），如果有那就优先执行微任务，如果没有，在去查找宏任务macrotask（setTimeout、setInterval）进行执行 众所周知JS是门非阻塞单线程语言，因为在最初JS就是为了和浏览器交互而诞生的。如果JS是门多线程的语言话，我们在多个线程中处理DOM就可能发生问题（一个线程中新加节点，另一个线程中删除节点） JS在执行的过程中会产生执行环境，这些执行环境会被顺序的加入到执行栈中。如果遇到异步的代码，会被挂起并加入到Task（有多种task） 队列中。一旦执行栈为空，EventLoop就会从Task队列中拿出需要执行的代码并放入执行栈中执行，所以本质上来说JS中的异步还是同步行为

```
console.log('script start');setTimeout(function(){console.log('setTimeout');},0);console.log('script end');
不同的任务源会被分配到不同的Task队列中，任务源可以分为 微任务（microtask） 和 宏任务（macrotask）。在ES6规范中，microtask称为jobs，macrotask称为task console.log('script start');setTimeout(function(){console.log('setTimeout');},0);new Promise((resolve)=>{console.log('Promise')resolve()}).then(function(){console.log('promise1');}).then(function(){console.log('promise2');});console.log('script end');// script start => Promise => script end => promise1 => promise2 => setTimeout
```

以上代码虽然setTimeout写在Promise之前，但是因为Promise属于微任务而setTimeout属于宏任务

微任务 process.nextTick promiseObject.observe MutationObserver 宏任务
script setTimeout setInterval setImmediate...

223. JavaScript 对象生命周期的理解

JavaScript对象的生命周期可以理解为创建、使用和被垃圾回收三个阶段。创建对象时引擎会分配内存，使用期间变量、函数或其他对象可能直接或间接持有它的引用。对象不再可达后，垃圾回收器会在合适的时机释放内存，现代引擎通常采用标记清除，并结合分代回收等方式提高效率。开发者不用手动释放内存，但及时解除无用引用或设为null，能避免对象被长期保留。 JavaScript

对象的生命周期可以概括为以下几个阶段：

创建阶段：当使用new关键字或对象字面量语法创建一个对象时，JavaScript 引擎会为该对象分配内存，并将其初始化为一个空对象。使用阶段：在对象创建后，可以对其进行属性的读取、修改和方法的调用等操作。对象被使用时，它可能会被传递给其他函数或存储在变量中，以供后续操作使用。引用阶段：在对象的使用过程中，其他变量或函数可能会引用该对象，形成对该对象的引用关系。这些引用关系可以是直接的，也可以是通过其他对象的属性或方法间接引用的。回收阶段：当一个对象不再被引用时，或者所有引用都被循环引用时，垃圾回收机制会将其标记为可回收，并在适当的时候回收该对象所占用的内存。垃圾回收器定期扫描内存中的对象，检查它们的引用情况，并释放那些不再被引用的对象。

需要注意的是，JavaScript 使用自动垃圾回收机制来管理内存，开发者不需要显式地释放对象占用的内存。垃圾回收器会自动跟踪对象的引用关系，并在适当的时候回收无用的对象。开发者可以通过将对象的引用置为null来显式地解除对对象的引用，以帮助垃圾回收器更早地回收对象。在浏览器环境中，垃圾回收器

通常使用标记-清除算法来判断对象是否可回收。当一个对象不再可达时，即没有任何引用指向该对象，垃圾回收器会将其标记为可回收，并在垃圾回收的过程中将其释放。一些现代的浏览器还使用了更高级的垃圾回收算法，如分代回收和增量标记等，以提高垃圾回收的效率和性能。总结来说，JavaScript 对象的生命周期包括创建、使用和回收三个阶段。开发者无需显式地管理对象的内存，而是通过使用对象和及时解除对象的引用来帮助垃圾回收器自动回收不再使用的对象。

224. 我现在有一个canvas，上面随机布着一些黑块，请实现方法，计算canvas上有多少个黑块

可以用getImageData读取画布像素，遍历并统计RGB都为0的像素数量。像素数组按RGBA每四项表示一个像素，因此循环时每次递增4，命中黑色就把计数加一。这个实现统计的是黑色像素数；如果题目里的“黑块”指彼此分离的区域，还需要额外定义区域如何判定。画布较大时逐像素扫描会有性能成本，跨域图片还可能因同源限制导致读取失败。要计算canvas上有多少个黑块，需要遍历canvas上的每个像素，并检查该像素的颜色是否为黑色。以下是一种实现方法：

```
function countBlackBlocks(canvas) {
  const context = canvas.getContext('2d');
  const imageData = context.getImageData(0, 0, canvas.width, canvas.height);
  const pixels = imageData.data;
  let blackCount = 0;
  for (let i = 0; i < pixels.length; i += 4) {
    // 获取像素的RGB颜色值
    const red = pixels[i];
    const green = pixels[i + 1];
    const blue = pixels[i + 2];
    // 判断颜色是否为黑色
    if (red === 0 && green === 0 && blue === 0) {
      blackCount++;
    }
  }
  return blackCount;
}
```

使用该方法，传入canvas元素作为参数，即可计算canvas上有多少个黑块。该方法通过获取canvas上的像素数据，遍历每个像素并判断其颜色是否为黑色（RGB 值为 [0, 0, 0]），累计黑色像素的数量，并最后返回计数结果。请注意，为了避免跨域问题，确保canvas的源与脚本执行的域相同，或者将canvas的图片源设为与脚本执行的域相同。另外，如果canvas的大小较大，遍历像素的操作可能会比较耗时，可能会影响性能。因此，在处理较大的canvas时，需要注意性能优化。

225. 现在要你完成一个Dialog组件，说说你设计的思路？它应该有什么功能？

我会把Dialog设计成受控组件，由外部通过visible、onOk和onCancel管理状态。头部、正文和底部用插槽开放定制，并支持点击蒙层关闭、内容超高滚动以及自定义渲染位置。显示可以用v-if或v-show控制，打开时还要锁定页面滚动，避免底层内容跟着移动。实际使用时可配合v-model和ok、cancel事件，让业务状态始终由父组件掌握。基于上述需求，以下是设计思路和Vue实现示例：

设计思路：创建一个Dialog组件，它是一个可控的组件，接收visible、onOk和onCancel等属性。使用v-if或者v-show来控制Dialog组件的显示与隐藏。在组件内部，使用插槽（slot）来允许自定义头部和底部内容。在Dialog组件的模板中，设置一个蒙层（mask），用于遮盖底层内容。点击蒙层时触发onCancel事件关闭Dialog。在Dialog组件的内容区域，设置一个滚动条容器，当内容超出容器高度时显示滚动条。根据需要，可以在组件外部指定渲染位置、设置外层样式等。Vue实现示例：

```
Default Header Confirm Cancel export default {
  props: {
    visible: { type: Boolean, default: false },
    onOk: { type: Function, default: () => {} },
    onCancel: { type: Function, default: () => {} },
  },
  watch: {
    visible(newVal) {
      if (newVal) {
        document.body.style.overflow = 'hidden';
      } else {
        document.body.style.overflow = '';
      }
    },
  },
  methods: {
    onOk() { this.onOk(); },
    onCancel() { this.onCancel(); },
  },
  dialog-container {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    display: flex;
    justify-content: center;
    align-items: center;
  },
  dialog-mask {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: rgba(0, 0, 0, 0.5);
  },
  dialog-content {
    background-color: white;
    width: 400px;
    border-radius: 4px;
    overflow: hidden;
  },
  dialog-header {
    padding: 16px;
    border-bottom: 1px solid #ccc;
  },
  dialog-body {
    padding: 16px;
    max-height: 400px;
    overflow-y: auto;
  },
  dialog-footer {
    padding: 16px;
    border-top: 1px solid #ccc;
    text-align: right;
  },
  dialog-footer button {
    margin-left: 8px;
  }
}
```

在使用示例中，我们可以在父组件中通过v-model指令来控制visible属性的值，从而控制Dialog组件的显示和隐藏。同时，可以通过@ok和@cancel事件监听来处理确定和取消按钮的点击事件。

Show Dialog Custom Header This is the dialog


```

content.Custom Actionimport Dialogfrom './Dialog';export default {components: {Dialog},data() {return {dialogVisible:false};},methods: {showDialog() {this.dialogVisible=true;},handleOk() { // Handle ok button clickconsole.log('Ok button clicked');this.dialogVisible=false;},handleCancel() { // Handle cancel button clickconsole.log('Cancel button clicked');this.dialogVisible=false;},handleCustomAction() { // Handle custom action button clickconsole.log('Custom action button clicked');}}};

```

注意，在使用示例中，我们引入了自定义的Dialog组件，并在父组件中进行注册。然后使用v-mode...

226. ajax、axios、fetch区别

Ajax是异步通信技术的统称，fetch是浏览器原生请求接口，axios则是功能更完整的第三方请求库。传统Ajax通常基于XHR，配置和回调写法相对繁琐；fetch基于Promise，更轻量，但需要自行处理超时、错误码和请求进度等问题。axios在浏览器端使用XHR，在Node.js中发送http请求，并提供拦截器、数据转换和取消请求等能力。简单场景可以选fetch，需要统一处理请求和响应时，我一般会选axios。

总结一下ajax、axios和fetch的区别： ajax是一种技术统称，基于原生的XHR开发，已经有了fetch的替代方案。fetch是一个原生的API，用于进行网络请求，支持Promise API，但在某些方面功能较为简单，需要进行封装来处理错误、超时等情况。axios是一个第三方库，可以用于浏览器和Node.js环境中发出HTTP请求，支持Promise API，提供了更多的功能和选项，如拦截请求和响应、转换数据、取消请求等。

下面是它们的一些主要区别和特点：

- 代码简洁性：fetch和axios相比，ajax的代码较为冗长，需要手动配置各项参数；fetch和axios使用更简洁，支持链式调用和配置对象参数。
- 浏览器兼容性：fetch是基于原生的Fetch API，较新的API，不支持低版本的浏览器，需要进行兼容性处理；ajax和axios对各种浏览器有较好的兼容性。
- 功能丰富性：axios提供了更多的功能和选项，如拦截请求和响应、转换数据、取消请求等，而fetch较为简单，需要进行封装来实现这些功能。
- 错误处理：axios和fetch支持Promise API，可以使用catch方法捕获错误；ajax需要通过error回调函数来处理错误。
- 请求取消：axios和fetch支持请求的取消操作，可以提前终止请求；ajax没有原生的取消请求的方法。
- 默认带cookie：axios和ajax默认会自动携带请求的cookie信息，而fetch默认不会携带，需要手动配置。
- 请求进度监测：axios和ajax支持原生监测请求的进度，如上传和下载的进度；fetch没有原生的请求进度监测方法。

jQuery ajax
\$.ajax({type:'POST',url:url,data:data,dataType:dataType,success:function({}),error:function({})});

优缺点： 本身是针对MVC的编程,不符合现在前端MVVM的浪潮基于原生的XHR开发，XHR本身的架构不清晰，已经有了fetch的替代方案jQuery整个项目太大，单纯使用ajax却要引入整个jQuery非常的不合理

（采取个性化打包的方案又不能享受CDN服务） axios axios({method:'post',url:'/user/12345',data:{firstName:'Fred',lastName:'Flintstone'}}).then(function(response){console.log(response);}).catch(function(error){console.log(error);});

优缺点：

从浏览器中创建XMLHttpRequest从node.js发出http请求支持Promise

API拦截请求和响应转换请求和响应数据取消请求自动转换JSON数据客户端支持防止CSRF/XSRF fetch try{let response=await fetch(url);let data=response.json();console.log(data);}catch(e){console.log("Oops, error",e);} 优缺点： fetch只对网络请求报错，对400，500都当做成功的请求，需要封装去处理fetch默认不会带cookie，需要添加配置项fetch不支持abort，不支持超时控制，使用setTimeout及Promise.reject的实现的超时控制并不能阻止请求过程继续在后台运行，造成了量的浪费fetch没有办法原生监测请求的进度，而XHR可以

227. JavaScript的组成

JavaScript通常由ECMAScript、DOM和BOM三部分组成。ECMAScript负责语言本身，包括语法、数据类型、函数、运算符和内置对象等核心能力。DOM把HTML或XML文档表示成树，让代码能够增删改查页面节点；BOM则提供window、location、history等浏览器交互接口。简单来说，前者解决怎么写程序，

后两者解决怎么操作页面和浏览器。JavaScript由三部分组成：ECMAScript（核心）：ECMAScript是JavaScript的基础，定义了语言的语法、类型、语句、关键字等。它规定了JavaScript的基本语法、数据类型、函数、运算符、控制流等核心特性，并提供了对数组、对象、字符串、正则表达式等的操作方法和功能。ECMAScript的版本以ES6（ES2015）为基准，随着时间的推移，新版本的ECMAScript引入了更多的语言特性和功能。DOM（文档对象模型）：DOM是一种表示和操作HTML、XML文档的接口。它定义了文档的结构、属性和方法，允许开发者通过JavaScript来访问和修改网页的内容、结构和样式。DOM将文档表示为一个树形结构，其中每个节点代表文档中的一个元素、属性、文本等。开发者可以使用DOM提供的API对这些节点进行增删改查操作，实现动态更新和交互效果。BOM（浏览器对象模型）：BOM是一种提供了与浏览器窗口进行交互的接口。它提供了访问浏览器窗口、处理窗口尺寸、导航历史、处理Cookie、发送HTTP请求等功能。BOM中的对象包括window、navigator、location、history、screen等，开发者可以使用这些对象来控制浏览器的行为和获取相关信息。这三部分共同构成了JavaScript的整体，使其成为一种强大的编程语言，能够在网页中实现丰富的交互和动态效果。

228. 检测浏览器版本有哪些方式？

可以解析navigator.userAgent、检查特定的window成员，或者使用Browser、Platform.js这类检测库。userAgent最常见，但它可能被伪造或篡改，因此结果不能当成绝对可靠的依据。检查某个对象或成员是否存在，更适合判断浏览器是否具备目标能力，例如通过'ActiveXObject' in window辅助识别IE。工程中如果只是决定能否使用某项功能，我会优先做特性检测；只有确实需要浏览器名称和版本时，才结合多种方式判断。检测浏览器版本可以使用以下几种方式：使用navigator.userAgent：通过检查navigator.userAgent属性，可以获取包含了浏览器相关信息的用户代理字符串。可以根据用户代理字符串中的特定关键字或标识符来确定浏览器的类型和版本。例如，使用UA.toLowerCase().indexOf('chrome')可以检测是否为Chrome浏览器。使用window对象的成员：根据浏览器的不同，window对象的成员可能会有所差异。可以通过检查特定的window对象成员是否存在来确定浏览器的类型和版本。例如，通过检查'ActiveXObject' in window可以判断是否为IE浏览器。使用现成的JavaScript库或框架：有一些专门用于检测浏览器类型和版本的JavaScript库或框架，如Browser、Platform.js等。这些库提供了简单易用的API，可以方便地获取浏览器信息。需要注意的是，检测浏览器版本可能会受到用户代理字符串的伪造或篡改，因此并不是一种绝对可靠的方法。在实际应用中，最好结合多种方式进行浏览器版本的检测，以增加准确性和可靠性。

229. 介绍JS有哪些内置对象

JavaScript内置对象可以按基础类型、功能工具、集合和异步元编程来理解。基础类型常见的有Object、Array、String和Number，功能工具则包括Date、RegExp、Math与JSON。处理集合时会用到Map、Set及其弱引用版本，异步或对象拦截场景常用Promise、Proxy和Reflect。需要注意，window、document这类对象由浏览器宿主环境提供，不属于ECMAScript内置对象。

当提到JavaScript的内置对象时，以下是一些常见的和全面的内置对象的列表：基本数据类型封装对象：Object：用于创建对象的基类。Array：用于创建和操作数组的对象。Boolean：表示布尔值的对象，包括true和false。Number：表示数字的对象，可以进行数值操作和转换。String：表示字符串的对象，提供了字符串操作和处理的方法。BigInt：表示任意精度整数的对象，用于处理超出Number范围的整数。

功能类对象：Function：用于定义和调用函数的对象。Date：用于处理日期和时间的对象。RegExp：用于进行正则表达式匹配和操作的对象。Error：表示错误的对象，用于抛出和处理异常。Math：提供了各种数学运算的方法和常量。集合类对象：Set：表示一组唯一值的集合。Map：提供了键值对的数据结构，可以使用任何数据类型作为键。WeakSet：类似于Set，但只能存储对象引用，并且不会阻止垃圾回收。WeakMap：类似于Map，但只能使用对象作为键，并且不会阻止垃圾回收。其他对象：Symbol：表示唯一标识符的数据类型，用于创建对象的属性键。Promise：用于处理异步操作的对象，提供了更好的处理异步任务的方式。Proxy：用于创建对象的代理，可以拦截并自定义对象的操作。Reflect：提供了一组静态方

法，用于操作对象的属性和方法。JSON：用于解析和序列化JSON数据的对象。这只是一些常见的内置对象，JavaScript还有许多其他内置对象和全局函数，用于处理各种类型的数据和操作。根据不同的需求和场景，可以选择适合的内置对象来使用。

230. 说几条写JavaScript的基本规范

写JavaScript最重要的是保持风格一致、命名清楚，并避免制造隐式全局变量。缩进、花括号、分号和单双引号采用哪种方案都可以，但团队内必须统一，否则会增加阅读和维护成本。变量与函数应先声明再使用，普通标识符采用驼峰命名，构造函数首字母大写，常量通常使用全大写。代码应尽量通过模块和局部作用域组织，注释重点解释用途或关键思路，而不是重复代码本身。

以下是一些常见的JavaScript编码规范：

- 使用一致的缩进：推荐使用四个空格进行缩进，而不是制表符。
- 使用一致的代码风格：在代码中使用一致的花括号（大括号）风格，可以是"K&R"风格（花括号放在行尾）或"Allman"风格（花括号独占一行）。
- 使用分号结束语句：在每条语句的末尾使用分号结束，这有助于避免意外的错误。
- 声明变量和函数：在使用之前，先声明变量和函数，避免隐式的全局变量。
- 命名约定：使用有意义且符合约定的变量和函数命名，采用驼峰命名法，首字母小写，构造函数使用大写字母开头，常量全大写。
- 使用严格模式：在脚本或函数的开头使用严格模式（`'use strict'`），可以帮助捕获潜在的错误并使代码更加安全。
- 编写清晰的注释：使用注释来解释代码的用途、思路或重要细节，有助于他人理解代码。
- 格式化对象和数组：使用花括号`{}`来声明对象，使用方括号`[]`来声明数组，并且按照一定的格式排列其中的元素，提高可读性。
- 使用单引号或双引号：可以选择使用单引号或双引号来表示字符串，但要保持一致性。
- 避免使用全局变量：尽量避免使用全局变量，使用模块化的方式组织代码，减少命名冲突和意外的副作用。

这些规范有助于提高代码的可读性、可维护性和一致性，使团队协作更加顺畅，并降低代码出错的风险。在编写JavaScript代码时，遵循一致的编码规范是一个良好的实践。

231. 如何编写高性能的JavaScript

高性能JavaScript的核心是减少加载、计算和页面渲染过程中的重复工作。加载阶段可以压缩打包脚本，并通过`async`或`defer`避免阻塞页面解析；执行阶段则应缓存重复计算结果和DOM查询。页面交互中要批量修改DOM，滚动等高频事件可用节流或防抖，多子元素监听适合使用事件委托，从而减少回流、重绘和监听器数量。优化前我一般会先用浏览器性能工具定位瓶颈，避免为了局部速度牺牲代码的可读性和可维护性。以下是一些编写高性能JavaScript的技巧：

- 使用严格模式：在JavaScript代码中使用严格模式，可以帮助检测潜在的错误，并提高代码性能。
- 将脚本放在底部：将JavaScript脚本放在HTML页面的底部，这样可以避免阻塞页面的渲染，提高页面加载速度。
- 打包和压缩代码：将JavaScript脚本进行打包和压缩，减少网络请求和文件大小，提高加载速度。
- 非阻塞下载：使用异步加载的方式下载JavaScript脚本，通过将脚本放在标签的`async`或`defer`属性中，避免阻塞页面的渲染。
- 减少全局变量的使用：尽量避免过多使用全局变量，使用局部变量来保存数据，减少作用域链的查找时间。
- 优化循环和迭代：在循环和迭代过程中，尽量减少重复的计算和操作，将需要重复使用的值存储在局部变量中，避免重复访问对象成员。
- 缓存DOM访问：在访问DOM节点时，尽量将访问结果缓存起来，避免重复查询DOM树，提高代码执行效率。
- 避免使用`eval()`和`Function()`构造器：这些方法会动态编译和执行代码，对性能有一定的影响，尽量避免使用它们。
- 使用直接量创建对象和数组：在创建对象和数组时，尽量使用直接量的方式，避免使用构造函数，这样可以减少额外的函数调用和内存分配。
- 最小化重绘和回流：重绘（repaint）和回流（reflow）是页面渲染的过程，它们会消耗大量的计算资源，尽量避免频繁的重绘和回流，可以通过合并操作、使用CSS动画等方式来优化。

当涉及到编写高性能的JavaScript代码时，还有一些其他的技巧可以考虑：

- 减少对象成员嵌套：在访问对象的成员时，尽量减少多层嵌套，这样可以提高访问速度。例如，将`obj1.obj2.prop`改为`obj1Prop`。
- 避免频繁的字符串操作：字符串操作比较耗费性能，尤其是在循环中频繁拼接字符串。可以使用数组或模板字符串来优化字符串操作。
- 使用事件委托：对于需要监听多个子元素事件的情况，可以将事件监听器添加到它们的父元素上，通过事件冒泡机制来处理事件。这样可以减少事件监听器的数量，提高性能。

。避免频繁的重绘：如果需要对DOM进行多次修改，最好将这些修改操作放在一起，而不是分散在多个地方，这样可以减少重绘次数。使用局部作用域：将代码封装在函数或模块中，利用局部作用域来限制变量的作用范围，避免命名冲突和全局变量污染。使用合适的数据结构和算法：在处理大量数据或复杂逻辑时，选择合适的数据结构和算法可以提高代码的效率。了解不同数据结构和算法的特点，选择最佳的方案。节流和防抖：对于一些频繁触发的事件（如滚动、调整窗口大小等），可以使用节流和防抖的技术来限制事件的触发频率，减少不必要的计算和操作。使用性能分析工具：利用浏览器提供的性能分析工具（如Chrome的开发者工具）来检测和分析代码的性能瓶颈，找到需要优化的地方。避免使用过时的方法和特性：某些方法和特性可能已经过时或存在性能问题，尽量避免使用它们，使用最新的标准和API来编写代码。定期进行代码优化和重构：不断优化和重构代码，去除冗余和低效的部分，使代码保持简洁、高效和易于维护。综合使用这些技巧，可以显著提高JavaScript代码的性能和执行效率。但需要注意的是，优化代码时应该根据具体情况进行评估和测试，避免过度优化导致代码可读性和可维护性的降低。

232. 描述浏览器的渲染过程，DOM树和渲染树的区别

浏览器会把HTML解析成DOM树，把CSS解析成CSSOM树，再合并生成渲染树，经过布局和绘制显示页面。DOM树描述文档的完整结构，包括标签、文本和注释；渲染树面向实际显示，只保留需要参与渲染的节点及其样式信息。布局阶段负责计算节点的位置和大小，绘制阶段再根据这些结果呈现外观，所以频繁修改布局相关属性可能触发回流或重绘。比如head中的节点和设置为display:

none的元素仍可存在于DOM树中，但不会进入渲染树。浏览器的渲染过程：解析HTML构建DOM（文档对象模型）树：浏览器将接收到的HTML文档解析成一个树状结构，该结构被称为DOM树。DOM树表示了HTML文档的结构和内容。解析CSS构建CSSOM（CSS对象模型）树：浏览器将接收到的CSS文件解析成一个树状结构，该结构被称为CSSOM树。CSSOM树表示了CSS样式规则的层级和规则。合并DOM树和CSSOM树生成渲染树（Render Tree）：浏览器将DOM树和CSSOM树合并，生成一个渲染树（Render Tree）。渲染树只包含需要显示在页面上的节点，隐藏的节点（如head）和不可见的节点（如display: none）不包含在渲染树中。布局（Layout）：渲染树中的每个节点都有对应的布局信息，浏览器根据这些布局信息计算节点在屏幕中的位置和大小，这个过程称为布局或回流（reflow）。绘制（Painting）：浏览器根据渲染树的布局信息和样式信息，将节点绘制到屏幕上，这个过程称为绘制或重绘（repaint）。

DOM树和渲染树的区别：DOM树（文档对象模型树）是由HTML文档解析而来，它反映了文档的结构和内容，包括HTML标签、文本节点和注释等。DOM树中的每个节点都有其对应的CSS样式规则。渲染树（Render Tree）是由DOM树和CSSOM树合并而成，它是用于显示在浏览器中的树状结构。渲染树只包含需要显示在页面上的节点，不包含隐藏的节点和不可见的节点。渲染树中的每个节点都有其对应的布局信息和样式信息，用于计算节点在屏幕中的位置和大小，并将节点绘制到屏幕上。总结：DOM树表示了HTML文档的结构和内容，而渲染树是为了将文档在浏览器中显示而构建的树结构。渲染树只包含需要显示的节点，并且每个节点都有对应的布局 and 样式信息，用于计算和绘制节点在屏幕上的位置和外观。

233. script 的位置是否会影响首屏显示时间

script的位置会影响首屏显示时间，尤其是未加异步或延迟属性的脚本。浏览器遇到普通script时，会暂停解析HTML，等脚本下载并执行完成后再继续，因此放在中更容易推迟页面呈现。放到底部，可以让主要内容先完成解析和显示。我一般还会根据依赖关系使用defer或async，减少脚本对解析和渲染的阻塞。

script的位置对首屏显示时间有影响。虽然浏览器在解析HTML生成DOM过程中，js文件的下载是并行的，不需要DOM处理到script节点，但是脚本的执行会阻塞页面的解析和渲染。当浏览器遇到script标签时，会暂停解析HTML，开始下载并执行脚本。只有脚本执行完毕后，浏览器才会继续解析和渲染页面。如果script标签放在标

签中，那么脚本的下载和执行会先于页面的渲染，这样会延迟首屏显示的开始时间。为了提高首屏显示时间，一般建议将script标签放在标签底部，在大部分内容都已经显示出来后再加载和执行脚本，这样可以让页面尽快呈现给用户，提升用户体验。另外，可以使用异步加载的方式（如将script标签添加async属性）或延迟加载的方式（如将script标签添加defer属性），来减少脚本对页面加载的阻塞影响。这样可以在不阻塞页面渲染的情况下加载和执行脚本，加快首屏显示的完成时间。

234. 介绍 DOM 的发展

DOM的发展，本质上是从浏览器各自实现的接口，逐步演变成统一且能力更完整的文档操作标准。DOM Level 0没有形成正式标准，到了DOM Level 1，W3C才规范了对文档结构的访问和操作。DOM Level 2重点补充事件和CSS支持，DOM Level 3又加入XPath以及文档加载、保存能力。后续标准继续扩展，例如MutationObserver监听节点变化，或通过Shadow DOM隔离子树。

DOM：文档对象模型（Document Object Model），定义了访问HTML和XML文档的标准，与编程语言及平台无关DOM Level

0：提供了查询和操作Web文档的内容API。未形成标准，实现混乱。如：`document.forms['login']`DOM Level 1：W3C提出标准化的DOM，简化了对文档中任意部分的访问和操作。如：JavaScript中的Document对象DOM Level

2：原来DOM基础上扩充了鼠标事件等细分模块，增加了对CSS的支持。如：`getComputedStyle(elem, pseudo)`DOM Level 3：增加了XPath模块和加载与保存（Load and Save）模块。如：`XPathEvaluator`DOM Level

4：继续扩展了DOM标准，引入了一些新的接口和功能，如MutationObserver用于监听DOM变动、Shadow DOM用于创建独立的DOM子树等

235. 介绍DOM0，DOM2，DOM3事件处理方式区别

DOM0、DOM2和DOM3的主要区别，在于事件绑定能力以及可处理的事件类型不同。DOM0直接给onclick等属性赋函数，同类事件只能保留一个处理器，后赋值会覆盖前一个，解绑时设为null。DOM2使用addEventListener和removeEventListener，既能绑定多个函数，也能通过useCapture选择捕获或冒泡阶段。DOM3延续这套机制，并扩展了文本输入等事件类型及自定义事件处理能力。

DOM0级事件处理方式：通过直接给事件属性赋值的方式进行事件处理，例如`element.onclick = func`。这种方式只能为同一个事件属性赋一个处理函数，且无法进行事件捕获阶段的处理。取消事件处理需要将事件属性赋值为null`btn.onclick = func; btn.onclick = null`；DOM2级事件处理方式：引入了addEventListener和removeEventListener方法来注册和移除事件处理函数。通过使用该方式，可以为同一个事件属性添加多个处理函数，且可以在事件的捕获阶段或冒泡阶段进行处理。使用addEventListener注册事件处理函数，使用removeEventListener移除事件处理函数`btn.addEventListener('click', func, false); btn.removeEventListener('click', func, false); btn.attachEvent("onclick", func); btn.detachEvent("onclick", func)`；DOM3级事件处理方式：引入了新的事件类型和接口，提供更多的事件处理选项。可以使用自定义的事件类型，并通过eventUtil等自定义的工具对象来添加和移除事件处理函数。DOM3级事件处理方式还引入了事件的命名空间概念，允许对特定命名空间的事件进行处理`eventUtil.addListener(input, "textInput", func)`；eventUtil是自定义对象，textInput是DOM3级事件

`btn.onclick = func; btn.onclick = null; btn.addEventListener('click', func, false); btn.removeEventListener('click', func, false); btn.attachEvent("onclick", func); btn.detachEvent("onclick", func); eventUtil.addListener(input, "textInput", func)`；eventUtil是自定义对象，textInput是DOM3级事件。在事件处理过程中，事件会经历捕获阶段、目标阶段和冒泡阶段。捕获阶段从文档根节点开始，向下传递到触发事件的目标元素，然后进入目标阶段，最后冒泡阶段从目标元素向上冒泡到文档根节点。DOM2和DOM3级事件处理方式都支持捕获和冒泡阶段的

处理，可以通过第三个参数useCapture来控制事件是在捕获阶段还是冒泡阶段触发。需要注意的是，DOM2和DOM3级事件处理方式的兼容性较好，而DOM0级事件处理方式在现代的开发中很少使用，推荐使用DOM2级或DOM3级事件处理方式。

236. 区分什么是“客户区坐标”、“页面坐标”、“屏幕坐标”

客户区、页面和屏幕坐标的区别，关键在于它们采用的坐标原点不同。clientX、clientY相对浏览器可视区左上角，页面滚动后数值不会包含滚动距离；pageX、pageY相对整个页面左上角，会把滚动距离计算进去。screenX、screenY则相对设备物理屏幕左上角，适合描述指针在屏幕中的位置。获取元素位置时，offsetLeft、offsetTop相对定位父级，而getBoundingClientRect()返回相对可视区的边框信息。客户区坐标：鼠标指针在可视区中的水平坐标(clientX)和垂直坐标(clientY) 页面坐标：鼠标指针在页面布局中的水平坐标(pageX)和垂直坐标(pageY) 屏幕坐标：设备物理屏幕的水平坐标(screenX)和垂直坐标(screenY)

```
body{margin:0;height:2000px;}#box{width:200px;height:200px;background-color:red;position:absolute;left:100px;top:100px;}document.addEventListener('mousemove',function(event){console.log('客户区坐标:',event.clientX,event.clientY);console.log('页面坐标:',event.pageX,event.pageY);console.log('屏幕坐标:',event.screenX,event.screenY)});
```

如何获得一个DOM元素的绝对位置？

```
elem.offsetLeft: 返回元素相对于其定位父级左侧的距离elem.offsetTop: 返回元素相对于其定位父级顶部的距离elem.getBoundingClientRect(): 返回一个DOMRect对象，包含一组描述边框的只读属性，单位像素
```

```
body{margin:0;padding:0;}#container{width:500px;height:500px;position:relative;border:1px solid black;}#box{width:100px;height:100px;background-color:red;position:absolute;left:200px;top:200px;}var box=document.getElementById('box');var offsetLeft=box.offsetLeft;var offsetTop=box.offsetTop;console.log('offsetLeft:',offsetLeft);console.log('offsetTop:',offsetTop);var rect=box.getBoundingClientRect();console.log('rect:',rect);console.log('left:',rect.left);console.log('top:',rect.top);
```

237. Javascript垃圾回收方法

现代JavaScript引擎主要使用标记清除法，老旧引擎还可能采用引用计数法。标记清除会从根对象递归遍历引用，可达对象保留，其余对象回收，所以能处理已经与外部断开的循环引用。引用计数则在计数归零时回收对象，但对象互相引用时计数无法归零，容易造成内存泄漏。实际引擎还会结合分代回收、增量标记等优化，具体实现比这两种基础模型更复杂。

正常情况下，现代的JavaScript引擎会使用标记清除（mark and sweep）算法作为主要的垃圾回收方法。引用计数（reference counting）在某些老旧的JavaScript引擎中可能会被使用。标记清除（mark and sweep）是JavaScript中最常见的垃圾回收算法，其工作原理如下：垃圾回收器会在运行时给存储在内存中的所有变量加上标记。垃圾回收器会从根对象开始，递归遍历所有的引用，标记它们为“进入环境”。在遍历完成后，垃圾回收器会对未被标记的变量进行清除，即将其回收内存空间。被清除的内存空间将被重新分配给后续的变量使用。

```
function foo(){var x={name:'poetry'};var y={name:'Jane'}; // 循环引用，x 引用了 y，y 引用了 x
x.ref=y;y.ref=x; // x 和 y 不再被使用，将被标记为垃圾
x=null;y=null; // 垃圾回收器在适当的时机清理循环引用的对象 // 调用函数触发垃圾回收foo();
```

引用计数（reference counting）是一种简单的垃圾回收算法，其工作原理如下：对于每个对象，引擎会维护一个引用计数器，用于记录当前有多少个引用指向该对象。当一个引用指向对象时，引用计数器加一；当一个引用不再指向对象时，引用计数器减一。当引用计数器为零时，说明该对象没有被引用，可以将其回收内存空间。引用计数算法容易出现循环引用的问题，即两个或多个对象互相引用，但没有被其他对象引用，导致引用计数器无法归零，造成内存泄漏。值得注意的是，现代的JavaScript引擎往往会采用更高级的垃圾回收算法，如基于分代的垃圾回收和增量标记等，以提高垃圾回收的效率和性能。以上所述的标记清除和引用计数仅是简单的介绍，实际的垃圾回收算法比较复杂，并涉及到更多的优化和细节。

// 引用计数无法处理循环引用问题，这里只作演示

```
function foo(){var x={name:'poetry'};var y={name:'Jane'}; // x 和 y 引用计数均为
```

```
1varrefCountX=1;varrefCountY=1;// 循环引用，x 引用了 y，y 引用了 xx.ref=y;y.ref=x;// x 和 y
不再被使用，引用计数减一refCountX--;refCountY--;//
当引用计数为零时，垃圾回收器可以清理对象if(refCountX===0){// 清理 x
对象的内存x=null;}if(refCountY===0){// 清理 y 对象的内存y=null;}}// 调用函数触发垃圾回收foo(); 请注意，上述示例中的引用计数示例仅为演示目的，并未解决循环引用导致的内存泄漏问题。在实际开发中，为了避免内存泄漏，需要使用更高级的垃圾回收算法和技术，或者手动解除循环引用。
```

238. 请解释一下 JavaScript 的同源策略

同源策略是浏览器的安全机制，只有协议、域名和端口都相同，两个页面才属于同源。它主要限制脚本读取跨源响应、访问跨源页面的DOM，以及获取其他源的Cookie、LocalStorage和IndexedDB数据。本质上是避免恶意网站借助用户身份读取其他站点的敏感信息。确实需要跨域时，可以使用CORS、JSONP或代理服务器；服务器之间的通信不受浏览器同源策略限制。同源策略（Same-Origin Policy）是浏览器中一种重要的安全机制，用于限制来自不同源（协议、域名、端口）的脚本对当前文档的访问权限。同源策略的作用是保护用户的信息安全，防止恶意网站获取敏感数据或进行跨站攻击。同源策略限制了以下行为：脚本访问跨源文档的 DOM：通过脚本在页面中嵌入的 `iframe` 元素加载的跨源文档无法通过脚本访问其DOM，除非目标文档明确允许。脚本读取跨源文档的内容：通过脚本在页面中嵌入的 `iframe` 元素加载的跨源文档无法通过脚本读取其内容，包括读取属性、执行方法等。脚本发送跨源 `AJAX` 请求：脚本无法直接发送跨源的 `AJAX` 请求，只能向同源的服务器发送请求。Cookie、LocalStorage和IndexedDB的限制：跨源的脚本无法访问其他源的Cookie、LocalStorage或IndexedDB数据。同源策略的存在使得浏览器可以更好地保护用户的隐私和安全。然而，也有一些场景需要进行跨域访问，例如使用JSONP、CORS、代理服务器等方式来实现跨域请求。需要注意的是，同源策略仅在浏览器中执行，不会限制服务器之间的通信，服务器可以自由地进行跨域访问。

239. 如何删除一个cookie

删除Cookie的常见做法，是把同名Cookie的过期时间设为过去，或者设置max-age=0。例如可以执行`document.cookie = "cookieName=; expires=Thu, 01 Jan 1970 00:00:00 UTC"`，让浏览器将它视为已过期。也可以使用`document.cookie = "cookieName=; max-age=0"`，效果相同。需要注意，删除时的path和domain必须与设置时一致，否则可能只是创建了另一个已过期的同名Cookie。删除一个Cookie可以通过以下几种方式实现：

1. 将 Cookie 的过期时间设置为过去的时间：
`var date=new Date();date.setDate(date.getDate()-1);document.cookie="cookieName=; expires="+date.toUTCString();` 将cookieName替换为要删除的Cookie的名称。
2. 使用expires参数设置过期时间：
`document.cookie="cookieName=; expires=Thu, 01 Jan 1970 00:00:00 UTC";` 同样，将cookieName替换为要删除的Cookie的名称。
3. 使用max-age参数设置过期时间：
`document.cookie="cookieName=; max-age=0";` 同样，将cookieName替换为要删除的Cookie的名称。 请注意，删除Cookie时需要确保path和domain参数与要删除的Cookie的设置一致，以确保正确删除指定的Cookie

240. 页面编码和被请求的资源编码如果不一致如何处理

页面编码和资源编码不一致时，应把两边统一为相同字符集，通常由响应头和页面声明共同保证。服务端可以通过Content-Type: text/html; charset=utf-8告诉浏览器按UTF-8解析返回内容，页面则在head中声明。如果两处声明与资源实际编码不一致，浏览器解析和显示时就可能出现乱码。工程上要同时检查资源本身的编码，以及字符集是否在请求和响应环节被正确传递。

如果页面编码和被请求的资源编码不一致，可以采取以下处理方式：

后端响应头设置charset：在服务器端返回资源（例如 `HTML` 页面、`CSS` 文件、`JavaScript`

文件) 时, 在响应头中设置正确的字符编码, 确保与页面编码一致。例如, 在 HTTP 头部中添加以下内容:
Content-Type: text/html; charset=utf-8 这样可以告诉浏览器使用UTF-8编码解析返回的资源。
前端页面设置charset: 在 HTML

页面的部分添加标签, 并设置正确的字符编码, 确保与被请求的资源编码一致。例如:

这样可以告诉浏览器使用UTF-8编码解析当前页面。通过上述方式设置正确的字符编码, 可以确保页面和被请求的资源在解析和显示时使用一致的编码, 避免乱码等问题。需要注意的是, 确保页面和资源的编码设置一致, 并且字符编码在各个环节中正确传递和解析。

241. 把放在之前和之后有什么区别? 浏览器会如何解析它们?

放在之前符合HTML规范, 放在它之后属于语法错误, 但现代浏览器通常仍会解析并执行脚本。浏览器遇到后一种写法时, 会通过容错机制忽略前面的, 把这个视为仍在body内部, 所以实际功能通常没有区别。少数场景会省略和来减少输出字节, 不过收益有限。工程代码里我会把脚本放在之前, 避免依赖浏览器容错, 也更利于校验和维护。

将放在之前和之后的区别主要是在符合HTML标准的语法规则和浏览器的容错机制上, 具体如下: 符合HTML标: 按照HTML标准规定, 标签应该放在标签内, 通常是放在之前。将放在之后是不符合HTML标准的, 属于语法错误。但是, 现代浏览器通常会自动容错并解析这样的语法, 不会出现明显的错误。浏览器解析: 浏览器会解析并执行标签中的JavaScript代码。无论放在之前还是之后, 浏览器都会执行其中的代码。浏览器的容错机制会忽略之前的, 视作仍然在内部。因此, 从功能和效果上来说, 两者没有区别。服务器输出优化: 在一些情况下, 省略和闭合标签可以减少服务器输出的内容, 因为浏览器会自动补全这些标签。对于大型网站或需要优化响应速度的场景, 这种优化可以略微减少传输的字节数。需要注意的是, 虽然现代浏览器对放置标签的位置比较宽容, 但为了遵循HTML标准和保持代码的可读性和可维护性, 推荐将标签放在之前, 符合语义和结构的要求。

242. JavaScript 中, 调用函数有哪几种方式

JavaScript函数常见的调用方式有直接调用、对象方法调用、构造器调用, 以及通过call、apply或bind调用。func()没有明确接收者, 而obj.method()会以对象的方法形式执行; new Func()则把函数当作构造器创建实例。需要指定函数内部的this时, 可以用call或apply立即调用, 区别主要在参数传递形式。bind不会立刻执行, 而是先返回绑定好this的新函数, 适合稍后再调用。

在JavaScript中, 调用函数有以下几种方式:

方法调用模式: 将函数作为对象的方法调用, 使用点运算符来调用函数。 obj.method(arg1,arg2);

函数调用模式: 直接调用函数, 没有明确的接收者对象。 func(arg1,arg2);

构造器调用模式: 使用new关键字调用函数作为构造器来创建对象实例。 newFunc(arg1,arg2);

call/apply调用模式: 使用call或apply方法来调用函数, 并指定函数内部的this值, 以及参数列表。

func.call(obj,arg1,arg2);func.apply(obj,[arg1,arg2]);

bind调用模式: 使用bind方法创建一个新函数, 并指定新函数的this值, 然后调用新函数。

varnewFunc=func.bind(obj);newFunc(arg1,arg2);

这些不同的调用方式提供了灵活性和适用性, 可以根据不同的场景选择合适的方式来调用函数。

243. 列举一下JavaScript数组和对象有哪些原生方法?

数组原生方法可以按增删、遍历、查找和转换来记, 对象方法则主要处理属性、原型及对象状态。数组增删常用push、pop、splice, 遍历转换常用forEach、map、filter, 查找可用find、includes。对象操作里, Object.keys、Object.values和Object.entries用于读取自身可枚举属性, Object.assign用于复制属性。需要控制属性或对象状态时, 再使用Object.defineProperty、Object.freeze或Object.seal, 按场景选择比死记完整名单更实用。

数组方法: arr.concat(arr1, arr2,

arrn)：连接多个数组并返回新数组。arr.copyWithin(target, start, end)：将数组的一部分复制到同一数组中的另一个位置。arr.entries()：返回一个包含数组键值对的迭代器对象。arr.every(callbackFn, thisArg)：测试数组中的所有元素是否都通过了指定函数的测试。arr.fill(value, start, end)：用静态值填充数组的一部分。arr.filter(callbackFn, thisArg)：创建一个新数组，其中包含通过指定函数筛选的所有元素。arr.find(callbackFn, thisArg)：返回数组中第一个满足测试函数的元素的值。arr.findIndex(callbackFn, thisArg)：返回数组中第一个满足测试函数的元素的索引。arr.flat(depth)：将多维数组展平为一维数组。arr.flatMap(callbackFn, thisArg)：首先使用映射函数映射每个元素，然后将结果展平为一维数组。arr.forEach(callbackFn, thisArg)：对数组中的每个元素执行指定函数。arr.includes(searchElement, fromIndex)：判断数组中是否包含指定元素。arr.indexOf(searchElement, fromIndex)：返回指定元素在数组中首次出现的索引。arr.join(separator)：将数组元素连接为一个字符串，并使用指定的分隔符。arr.keys()：返回一个包含数组键的迭代器对象。arr.lastIndexOf(searchElement, fromIndex)：返回指定元素在数组中最后一次出现的索引。arr.map(callbackFn, thisArg)：创建一个新数组，其中包含通过指定函数对每个元素进行处理后的结果。arr.pop()：移除并返回数组的最后一个元素。arr.push(element1, element2, ..., elementN)：向数组末尾添加一个或多个元素，并返回新的长度。arr.reduce(callbackFn, initialValue)：对数组中的所有元素执行指定的累积函数，返回累积结果。arr.reduceRight(callbackFn, initialValue)：对数组中的所有元素执行指定的累积函数（从右到左），返回累积结果。arr.reverse()：反转数组中元素的顺序。arr.shift()：移除并返回数组的第一个元素。arr.slice(start, end)：从数组中提取指定范围的元素，并返回一个新数组。arr.some(callbackFn, thisArg)：测试数组中的至少一个元素是否通过了指定函数的测试。arr.sort(compareFunction)：对数组元素进行排序，可以传入自定义的比较函数。arr.splice(start, deleteCount, item1, item2, ...)：从数组中添加/删除元素，并返回被删除的元素。arr.toLocaleString()：将数组中的元素转换为字符串，并返回该字符串。arr.toString()：将数组中的元素转换为字符串，并返回该字符串。arr.unshift(element1, element2, ..., elementN)：向数组开头添加一个或多个元素，并返回新的长度。arr.values()：返回一个包含数组值的迭代器对象。

对象方法：Object.assign(target, ...sources)：将一个或多个源对象的属性复制到目标对象，并返回目标对象。Object.create(proto, [propertiesObject])：使用指定的原型对象和属性创建一个新对象。Object.defineProperties(obj, props)：定义一个或多个对象的新属性或修改现有属性的配置。Object.defineProperty(obj...

244. Array.slice() 与 Array.splice() 的区别？

slice()用于截取并返回新数组，不修改原数组；splice()用于删除、替换或插入元素，会直接修改原数组。slice(start, end)包含起始位置但不包含结束位置，返回的是指定范围的元素。splice(start, deleteCount, ..items)从起始位置处理指定数量的元素，并返回被删除元素组成的数组。只想读取一段数据时用slice()，需要在原数组中间增删或替换内容时才用splice()，因为后者还可能改变数组长度。slice()方法返回一个新数组，包含从原数组中指定的开始位置到结束位置（不包括结束位置）的元素，不会修改原数组。splice()方法通过删除或替换现有元素或者添加新元素来修改原数组。它会返回被删除的元素组成的数组。

```
const fruits = ['apple', 'banana', 'orange', 'mango', 'kiwi']; // 从索引 1 开始删除 2 个元素，并插入 'grape' 和 'pear'
const deletedFruits = fruits.splice(1, 2, 'grape', 'pear'); console.log(deletedFruits); // 输出: ['banana', 'orange']
console.log(fruits); // 输出: ['apple', 'grape', 'pear', 'mango', 'kiwi']
const fruits = ['apple', 'banana', 'orange']; // 在索引 1 的位置插入 'grape' 和 'kiwi'
fruits.splice(1, 0, 'grape', 'kiwi'); console.log(fruits); // 输出: ['apple', 'grape', 'kiwi', 'banana', 'orange']
```

主要区别如下：slice()是纯粹的读取操作，不会对原数组进行修改，而splice()是对数组进行操作，会修改原数组。slice()的参数是起始位置和结束位置，返回选定的元素组成的新数组。splice()的参数

是起始位置、删除的元素个数以及可选的插入元素，返回被删除的元素组成的新数组。slice()的结束位置是不包括在选取范围内的，而splice()中的删除元素个数是包括在操作范围内的。slice()不会改变原数组的长度，而splice()可以改变原数组的长度。总的来说，slice()是用来提取数组中的一部分元素，不改变原数组，而splice()是用来操作数组，可以删除、替换或插入元素，会改变原数组。

```
const fruits = ['apple', 'banana', 'orange', 'mango', 'kiwi']; // 从索引 1 开始（包括索引 1），到索引 3 结束（不包括索引 3）
const slicedFruits = fruits.slice(1, 3); console.log(slicedFruits); // 输出: ['banana', 'orange']
console.log(fruits); // 输出: ['apple', 'banana', 'orange', 'mango', 'kiwi']
```

245. MVVM

MVVM通过ViewModel连接Model和View，利用数据绑定让数据与界面自动同步。Model管数据和业务规则，View负责展示与交互，ViewModel则转换数据并响应用户输入。本质上，响应式系统会在读取数据时收集依赖，在数据变化时通知订阅者更新视图。Object.defineProperty()能拦截已有属性的读写，而Proxy对数组变化和动态新增属性的处理更灵活，但也要考虑旧浏览器兼容性。MVVM（Model-View-View Model）是一种软件架构模式，用于实现用户界面（UI）和业务逻辑的分离。它的设计目标是将界面的开发与后端的业务逻辑分离，使代码更易于理解、维护和测试。在MVVM中，各个组成部分的职责如下：Model（模型）：表示应用程序的数据和业务逻辑。它负责数据的存储、检索和更新，并封装了与数据相关的操作和规则。View（视图）：展示用户界面，通常是由UI元素组成的。它是用户与应用程序进行交互的界面，负责将数据呈现给用户，并接收用户的输入。ViewModel（视图模型）：连接View和Model，负责处理业务逻辑和数据的交互。它从Model中获取数据，并将数据转换为View可以理解和展示的格式。ViewModel还负责监听View的变化，并根据用户的输入更新Model中的数据。MVVM的核心思想是数据绑定，通过双向绑定机制将View和ViewModel中的数据保持同步。当ViewModel中的数据发生变化时，View会自动更新，反之亦然。这种数据驱动的方式使得开发者可以专注于业务逻辑的实现，而无需手动操作DOM元素来更新界面。MVVM的优势包括：可维护性：将界面逻辑与业务逻辑分离，使代码更易于理解和维护。可测试性：由于视图逻辑与业务逻辑解耦，可以更容易地编写单元测试来验证ViewModel的行为。可复用性：ViewModel可以独立于具体的View，可以复用在不同的界面上，提高代码的重用性。团队协作：MVVM模式将界面开发与后端逻辑分离，使得前端和后端开发人员可以并行工作，提高团队的协作效率。总而言之，MVVM是一种能够将界面逻辑与业务逻辑分离的软件架构模式，通过数据绑定实现了View和ViewModel的自动同步，提高了代码的可维护性、可测试性和可复用性。在Vue中，ViewModel由Vue实例扮演。Vue通过数据绑定机制建立了View和ViewModel之间的连接，当ViewModel中的数据发生变化时，View会自动更新，反之亦然。这种双向数据绑定使得开发者能够以一种声明式的方式编写代码，而不需要手动操作DOM来更新界面。总结来说，MVVM是一种将数据驱动视图的设计模式，通过ViewModel作为中间层来实现数据和视图之间的解耦。Vue作为一种流行的MVVM框架，提供了强大的数据绑定和响应式系统，使开发者能够更轻松地构建交互性强的Web应用程序。数据劫持Vue内部使用了Object.defineProperty()来实现双向绑定，通过这个函数可以监听到set和get的事件

```
var data = { name: 'poetry' };
observe(data);
let name = data.name; // -> get value
data.name = 'yyy'; // -> change value
function observe(obj) { // 判断类型
  if (!obj || typeof obj !== 'object') { return }
  Object.keys(data).forEach(key => {
    defineReactive(data, key, data[key]);
  });
  function defineReactive(obj, key, val) { // 递归子属性
    observe(val);
    Object.defineProperty(obj, key, {
      enumerable: true,
      configurable: true,
      get: function reactiveGetter() {
        console.log('get value');
        return val;
      },
      set: function reactiveSetter(newVal) {
        console.log('change value');
        val = newVal;
      }
    });
  }
}
```

以上代码简单的实现了如何监听数据的 set 和 get 的事件，但是仅仅如此是不够的，还需要在适当的时候给属性添加发布订阅 {{name}}

在解析如上模板代码时，遇到{name}就会给属性name添加发布订阅 // 通过 Dep 解耦

```
class Dep {
  constructor() {
    this.subs = [];
  }
  addSub(sub) { // sub 是 Watcher
```

实例this.subs.push(sub))not...

246. WEB应用从服务器主动推送Data到客户端有那些方式

常见方案是WebSocket、SSE和长轮询，选择时主要看通信方向、实时性和服务端资源成本。WebSocket建立持久的双向连接，适合客户端与服务端都要频繁发消息的实时场景。SSE复用持久的HTTP连接做单向推送，更适合新闻、行情这类服务端持续下发数据的需求。长轮询兼容传统请求模型，但连接需要反复续接，实现更复杂，对服务端资源的消耗也更大。

WebSocket：WebSocket是一种双向通信协议，通过建立持久连接，服务器可以主动向客户端推送数据，而不需要客户端发送请求。WebSocket提供了实时性更好的数据推送能力，适用于需要实时更新数据的场景。

Server-Sent Events (SSE)：SSE是HTML5中定义的一种服务器推送技术，通过建立一个持久的HTTP连接，服务器可以向客户端推送数据，客户端通过监听事件来接收推送的数据。SSE适用于需要实现单向实时数据推送的场景，例如实时新闻、实时股票行情等。

Long Polling：长轮询是一种通过客户端不断发送请求，服务器在有数据更新时立即响应的方式。客户端发送一个请求到服务器，服务器一直保持连接打开，直到有新的数据可用或超时，然后将响应返回给客户端，客户端再立即发送下一个请求。长轮询可以模拟实时的数据推送，但相比WebSocket和SSE，它的实现相对复杂，并且对服务器资源的消耗较大。

以上这些方式都可以实现服务器主动推送数据到客户端，选择哪种方式取决于具体的需求和技术栈的选择。WebSocket和SSE是现代Web应用中较为常用的服务器推送技术，它们提供了更好的实时性和效率。

示例代码如下：

WebSocket 示例：客户端代码：

```
const socket = new WebSocket('ws://your-server-url');
socket.addEventListener('open', () => { console.log('WebSocket 连接已建立'); });
socket.addEventListener('message', (event) => { const data = event.data; console.log('收到服务器推送的数据:', data); });
socket.addEventListener('close', () => { console.log('WebSocket 连接已关闭'); });
```

服务器端代码（使用Node.js和WebSocket库ws）：

```
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });
wss.on('connection', (ws) => { console.log('新的WebSocket 连接已建立'); // 模拟推送数据给客户端
setInterval(() => { ws.send('服务器主动推送的数据'); }, 5000);
ws.on('close', () => { console.log('WebSocket 连接已关闭'); }); });
```

Server-Sent Events 示例：客户端代码：

```
const eventSource = new EventSource('your-server-url');
eventSource.addEventListener('message', (event) => { const data = event.data; console.log('收到服务器推送的数据:', data); });
eventSource.addEventListener('error', (event) => { console.error('发生错误:', event); });
```

服务器端代码（使用Node.js和Express框架）：

```
const express = require('express');
const app = express();
app.get('/stream', (req, res) => { res.setHeader('Content-Type', 'text/event-stream');
res.setHeader('Cache-Control', 'no-cache');
res.setHeader('Connection', 'keep-alive'); //
```

模拟每5秒发送一次数据给客户端

```
setInterval(() => { res.write(`data:
服务器主动推送的数据\n\n`); }, 5000);
app.listen(8080, () => { console.log('服务器已启动'); });
```

服务器主动推送的数据\n\n`); }, 5000);

app.listen(8080, () => { console.log('服务器已启动'); });

请注意，以上示例仅为简单示例，实际使用时需要根据具体的需求和技术栈进行适当的调整。

247. 继承

JavaScript继承主要依靠原型链和构造函数调用，工程上更完整的做法是寄生组合继承。单用原型链虽然简单、能复用原型方法，但引用属性会被实例共享，也不能给父构造函数传参。单用Parent.call(this)可以传参并复制实例属性，却拿不到父类原型上的方法，每个实例还可能保存自己的函数副本。寄生组合继承用Object.create(Parent.prototype)连接原型，再修正constructor，同时在子构造函数中调用父构造函数，兼顾属性隔离和方法复用。

原型链继承，将父类的实例作为子类的原型，他的特点是实例是子类的实例也是父类的实例，父类新增的原型方法/属性，子类都能够访问，并且原型链继承简单易于实现，缺点是来自原型对象的所有属性被所有实例共享，无法实现多继承，无法向父类构造函数传参。

```
function Parent() { this.name = 'Parent'; }
Parent.prototype.sayHello = function () { console.log('Hello, I am ' + this.name); };
function Child() {}
Child.prototype = new Parent();
var child = new Child();
child.sayHello();
```

lo();// Output: Hello, I am Parent 构造继承，使用父类的构造函数来增强子类实例，即复制父类的实例属性给子类，构造继承可以向父类传递参数，可以实现多继承，通过call多个父类对象。但是构造继承只能继承父类的实例属性和方法，不能继承原型属性和方法，无法实现函数服用，每个子类都有父类实例函数的副本，影响性能 functionParent(name){this.name=name;}Parent.prototype.sayHello=function(){console.log('Hello, I am '+this.name);};functionChild(name){Parent.call(this,name);}varchild=newChild('Child');child.sayHello();// Output: Hello, I am Child 实例继承，为父类实例添加新特性，作为子类实例返回，实例继承的特点是不限制调用方法，不管是new子类()还是子类()返回的对象具有相同的效果，缺点是实例是父类的实例，不是子类的实例，不支持多继承 functioncreateParent(){varparent={name:'Parent',sayHello:function(){console.log('Hello, I am '+this.name);}};returnparent;}functioncreateChild(){varchild=Object.create(createParent());child.name='Child';returnchild;}varchild=createChild();child.sayHello();// Output: Hello, I am Child 拷贝继承：特点：支持多继承，缺点：效率较低，内存占用高（因为要拷贝父类的属性）无法获取父类不可枚举的方法（不可枚举方法，不能使用for in访问到） functioncopyProperties(target,source){for(varkeyinsource){if(source.hasOwnProperty(key)){target[key]=source[key];}}}functionParent(){this.name='Parent';}Parent.prototype.sayHello=function(){console.log('Hello, I am '+this.name);};functionChild(){Parent.call(this);this.name='Child';}copyProperties(Child.prototype,Parent.prototype);varchild=newChild();child.sayHello();// Output: Hello, I am Child 组合继承：通过调用父类构造，继承父类的属性并保留传参的优点，然后通过将父类实例作为子类原型，实现函数复用 functionParent(name){this.name=name;}Parent.prototype.sayHello=function(){console.log('Hello, I am '+this.name);};functionChild(name){Parent.call(...

248. 有四个操作会忽略enumerable为false的属性

for...in、Object.keys()、JSON.stringify()和Object.assign()都会忽略enumerable为false的属性。区别在于，for...in会遍历自身及原型链上的可枚举属性，而另外三种操作只处理对象自身的可枚举属性。实际使用时要注意，属性明明存在却没出现在遍历结果、序列化文本或拷贝对象里，通常就需要检查它的enumerable描述符。 for...in循环：只遍历对象自身的和继承的可枚举的属性。 varobj={prop1:'value1',prop2:'value2'};Object.defineProperty(obj,'prop3',{value:'value3',enumerable:false});for(varkeyinobj){console.log(key);// Output: prop1, prop2} Object.keys()：返回对象自身的所有可枚举的属性的键名。 varobj={prop1:'value1',prop2:'value2'};Object.defineProperty(obj,'prop3',{value:'value3',enumerable:false});varkeys=Object.keys(obj);console.log(keys);// Output: ["prop1", "prop2"] JSON.stringify()：只串行化对象自身的可枚举的属性。 varobj={prop1:'value1',prop2:'value2'};Object.defineProperty(obj,'prop3',{value:'value3',enumerable:false});varjson=JSON.stringify(obj);console.log(json);// Output: '{"prop1":"value1","prop2":"value2"}" Object.assign()：忽略enumerable为false的属性，只拷贝对象自身的可枚举的属性。 varsource={prop1:'value1',prop2:'value2'};Object.defineProperty(source,'prop3',{value:'value3',enumerable:false});vartarget={};Object.assign(target,source);console.log(target);// Output: { prop1: 'value1', prop2: 'value2' } 在以上示例中，enumerable属性为false的属性prop3在遍历、获取键名、序列化、拷贝等操作中都被忽略了。

249. 属性的遍历

对象属性的遍历方式，核心区别是是否包含原型链、不可枚举属性和Symbol键。for...in会遍历自身及继承的可枚举字符串键，Object.keys()只返回自身的可枚举字符串键。需要查看不可枚举键时用Object.getOwnPropertyNames()，只取Symbol键用Object.getOwnPropertySymbols()，最完整的是Reflect.ownKeys()。这些方式都按数值键升序、字符串键加入顺序、Symbol键加入顺序遍历。 ES6提供了5种方法用于遍历对象的属性，它们分别是for...in、Object.keys()、Object.getOwnPropertyNames()、Object.get

OwnPropertySymbols()和Reflect.ownKeys()。这些方法在遍历对象属性时都遵循相同的次序规则 for...infor...in循环遍历对象自身的和继承的可枚举属性（不含 Symbol 属性）。Object.keys(obj)Object.keys返回一个数组，包括对象自身的（不含继承的）所有可枚举属性（不含 Symbol 属性）的键名。Object.getOwnPropertyNames(obj)Object.getOwnPropertyNames返回一个数组，包含对象自身的属性（不含 Symbol 属性，但是包括不可枚举属性）的键名。Object.getOwnPropertySymbols(obj)Object.getOwnPropertySymbols返回一个数组，包含对象自身的属性所有 Symbol 属性的键名。Reflect.ownKeys(obj)Reflect.ownKeys返回一个数组，包含对象自身的（不含继承的）所有键名，不管键名是 Symbol 或字符串，也不管是否可枚举。以上的 5 种方法遍历对象的键名，都遵守同样的属性遍历的次序规则。首先遍历所有数值键，按照数值升序排列。其次遍历所有字符串键，按照加入时间升序排列。最后遍历所有 Symbol 键，按照加入时间升序排列。

```
const obj = {[Symbol()]: 'symbol', b: 'b', 10: '10', 2: '2', a: 'a'};
for...infor (let key in obj) { console.log(key); } // Output: 2, 10, b, a
Object.keys()const keys = Object.keys(obj); console.log(keys); // Output: ["2", "10", "b", "a"]
Object.getOwnPropertyNames()const propertyNames = Object.getOwnPropertyNames(obj); console.log(propertyNames); // Output: ["2", "10", "b", "a"]
Object.getOwnPropertySymbols()const symbols = Object.getOwnPropertySymbols(obj); console.log(symbols); // Output: [Symbol()]
Reflect.ownKeys()const allKeys = Reflect.ownKeys(obj); console.log(allKeys); // Output: ["2", "10", "b", "a", Symbol()]
```

在上述示例中，对象obj的属性遍历次序为首先数值属性2和10，然后是字符串属性b和a，最后是Symbol属性。需要注意的是，for...in只会遍历可枚举属性且会包括继承的属性，而Object.keys()、Object.getOwnPropertyNames()、Object.getOwnPropertySymbols()和Reflect.ownKeys()只会遍历对象自身的属性，不包括继承的属性

250. 为什么通常在发送数据埋点请求的时候使用的是 1x1 像素的透明 gif 图片

埋点使用1x1透明GIF，是因为它能用很小的成本发起完整的跨域HTTP GET请求。图片加载天然支持跨域，不需要处理复杂配置，页面也不用解析返回数据。请求是异步、非阻塞的，不会因为上报而卡住页面后续执行。选择透明的1x1 GIF还能避免影响界面，并把图片本身体积控制得很小，适合大量统计请求。完成整个HTTP请求+响应：使用GIF图片可以触发完整的HTTP请求+响应流程，尽管在埋点请求中不需要获取和处理响应内容。这样可以确保埋点请求按照正常的HTTP流程发送，并且服务器也能正常地接收和处理请求无需获取和处理数据：GIF图片作为埋点请求，不需要获取和处理返回的数据。它只是简单地发送一个GET请求，不需要等待响应或处理响应内容，因此能够快速地完成请求并继续执行后续的代码跨域友好：由于GIF图片是通过标签加载的，而标签在浏览器中天然支持跨域请求，因此使用1x1像素的透明GIF图片发送埋点请求可以轻松地实现跨域请求，无需关注跨域限制和复杂的配置无阻塞执行：埋点请求通常是为了收集用户行为或统计数据，对于页面的性能和用户体验来说，不应该影响页面的加载和执行速度。由于GIF图片请求是异步的且无阻塞的，页面可以继续加载和执行其他代码，不会因为发送埋点请求而产生阻塞性能优化：相比使用XMLHttpRequest对象发送GET请求，使用1x1像素的透明GIF图片能够在性能上更加高效。GIF图片的体积最小，仅需要43个字节（最小的BMP文件需要74个字节，PNG需要67个字节，而合法的GIF，只需要43个字节），而且在网络传输中通常会进行gzip压缩，进一步减小传输的数据量，这对于大规模的数据埋点和统计是非常有利的 综上所述，使用1x1像素的透明GIF图片作为数据埋点请求具有简单、快速、跨域友好、无阻塞等优势，使得它成为常用的数据埋点方式之一

251. 在输入框中如何判断输入的是一个正确的网址

判断输入是否为正确网址，我一般优先用URL对象解析，解析失败则认为格式不合法。实现时用try...catch包住new URL(url)，成功返回true，抛出异常则返回false。正则表达式适合限制必须以http或https开头、域名结构等业务格式，但规则写得过严容易误判。要求更严格时，可以先用正则做格式筛选，再交给URL对象解析。判断输入的内容是否为正确的网址有多种方式，以下是几种常见的方式： 1. 使用正则表达式判断

```
function isUrlUsingRegex(url){const regex=/^(http|https):\/\/[^\w-]+(\.[^\w-]+)+([\w-\.~,@?^=%&\/~\+]*[^\w-~@?^=%&\/~\+])?$/;return regex.test(url);} 2. 使用URL对象进行解析
function isUrlUsingURL(url){try{new URL(url);return true;}catch(error){return false;}} 3.
使用正则表达式和URL对象的结合判断 function isUrlUsingRegexAndURL(url){const regex=/^(http|https):\/\/[^\w-]+(\.[^\w-]+)+([\w-\.~,@?^=%&\/~\+]*[^\w-~@?^=%&\/~\+])?$/;return regex.test(url)&&isUrlUsingURL(url);} 综上所述，判断输入的内容是否为正确的网址可以通过正则表达式匹配、URL对象进行解析，或者两者结合使用。使用正则表达式可以简单地验证网址的格式是否正确，而使用URL对象可以更严谨地验证网址的各个组成部分是否有效。选择哪种方式取决于具体的需求和使用场景。
```

252. 常用设计模式有哪些并举例使用场景

前端常用设计模式可以归为对象创建、消息通信、行为切换和功能增强几类。工厂模式适合按参数创建不同支付对象，单例模式适合全局配置这类只需一个实例的对象。发布订阅或观察者模式常用于事件总线 and 响应式数据，但要注意及时取消订阅，避免对象一直被保留。策略模式可以切换排序算法，装饰模式则能在不修改原对象的前提下增加日志或性能监测能力。

工厂模式:使用场景：当需要根据不同的参数创建不同类型的对象时，可以使用工厂模式。例如，根据用户的选择创建不同类型的支付方式对象。优点：封装了对象的创建过程，客户端只需关注传入参数即可获取所需对象，降低了耦合度。缺点：增加了代码的复杂性，需要额外编写工厂方法。

```
class PaymentFactory{
  createPayment(type){
    switch(type){
      case 'credit': return new CreditPayment();
      case 'debit': return new DebitPayment();
      default: throw new Error('Invalid payment type');
    }
  }
}
const paymentFactory = new PaymentFactory();
const creditPayment = paymentFactory.createPayment('credit');
const debitPayment = paymentFactory.createPayment('debit');
```

单例模式:使用场景：当整个系统中只需要一个实例时，可以使用单例模式。例如，全局的系统配置对象。优点：确保只有一个实例存在，提供了全局访问点，避免了重复创建实例。缺点：对扩展不友好，单例的实例化和使用耦合在一起。

```
class SystemConfig{
  constructor(){// Initialize system configuration}
  static getInstance(){
    if(!SystemConfig.instance){
      SystemConfig.instance = new SystemConfig();
    }
    return SystemConfig.instance;
  }
}
const config = SystemConfig.getInstance();
```

发布-订阅模式:使用场景：当存在多个对象之间需要进行解耦的消息通信时，可以使用发布-订阅模式。例如，实现一个事件总线用于组件间的通信。优点：解耦了对象之间的通信，订阅者只需关注自己感兴趣的事件，发布者不需要关心具体的订阅者。缺点：容易造成内存泄漏，需要手动取消订阅，否则订阅者会一直存在。

```
const EventBus = {
  events: {},
  subscribe(event, callback){
    if(!this.events[event]){
      this.events[event] = [];
    }
    this.events[event].push(callback);
  },
  publish(event, data){
    if(this.events[event]){
      this.events[event].forEach(cb => cb(data));
    }
  },
  unsubscribe(event, callback){
    if(this.events[event]){
      this.events[event] = this.events[event].filter(cb => cb !== callback);
    }
  }
};
// 订阅事件
EventBus.subscribe('userLoggedIn', handleUserLoggedIn);
// 发布事件
EventBus.publish('userLoggedIn', {username: 'poetry'});
// 取消订阅
EventBus.unsubscribe('userLoggedIn', handleUserLoggedIn);
```

253. 原型链判断

这道题目深入考察了原型链相关知识点

自定义事件通常先用`new Event()`创建，再通过`element.dispatchEvent()`分发，并用`addEventListener()`监听。如果使用旧写法，则由`document.createEvent()`创建事件，再调用`initEvent()`设置名称、是否冒泡和是否可取消。现代构造函数写法更简单直观，通常不需要再调用`initEvent()`。需要注意，事件只会在被分发后触发，是否能向上层传播取决于冒泡配置。

三要素：`document.createEvent()``event.initEvent()``element.dispatchEvent()` 在 JavaScript 中，可以使用以下三个要素来创建和触发自定义事件：`document.createEvent()`：这个方法用于创建一个新的事件对象。可以使用不同的方法根据需要创建不同类型的事件对象，例如`createEvent('Event')`、`createEvent('CustomEvent')`等。这个方法已经过时，推荐使用更现代的方式创建事件对象，如下文所示。`Event`构造函数：这是现代的方式来创建事件对象。可以使用`new Event(eventName)`创建一个新的事件对象，其中`eventName`是自定义事件的名称。`event.initEvent()`：对于使用`document.createEvent()`创建的事件对象，可以调用`initEvent(eventName, bubbles, cancelable)`方法进行初始化。其中`eventName`是事件名称，`bubbles`是一个布尔值，表示事件是否冒泡，`cancelable`是一个布尔值，表示事件是否可以被取消。`element.dispatchEvent()`：这个方法用于触发自定义事件。可以将创建好的事件对象通过调用`dispatchEvent(event)`方法分派到指定的 DOM 元素上，从而触发相应的事件处理程序。

下面是一个示例，演示如何使用这三个要素来创建和触发自定义事件：

```
创建自定义事件对象const event=newEvent('customEvent');//
```

```
初始化事件对象（可选）event.initEvent('customEvent',true,true);//
```

```
获取要触发事件的元素const element=document.getElementById('myElement');//
```

触发自定义事件`element.dispatchEvent(event)`；在上述示例中，首先使用`Event`构造函数创建了一个名为`'customEvent'`的自定义事件对象。然后，可以选择使用`initEvent()`方法对事件对象进行初始化，指定事件名称、冒泡和取消属性。最后，通过`getElementById()`方法获取要触发事件的元素，并调用`dispatchEvent()`方法将自定义事件对象分派到该元素上，从而触发自定义事件。 请注意，这里使用的是现代的事件创建和触发方法，而不是使用过时的`createEvent()`方法。这是因为现代的方法更加简单直观，并且具有更好的性能。

256. 前端性能定位、优化指标以及计算方法

前端性能定位要把真实监控、分析工具和关键指标结合起来，不能只列优化手段。我一般用Performance看时间轴、网络和资源，再用Lighthouse获取整体评估；线上则通过埋点或`window.performance`收集数据。指标保留LCP、FCP、FID和CLS，分别观察主要内容呈现、首次内容绘制、首次输入延迟和布局稳定性。采集时可用PerformanceObserver，也可用web-vitals简化处理，但真实环境仍需持续监测。

前端性能优化 已经是老生常谈的一项技术了 很多人说起性能优化方案的时候头头是道但是真正的对于性能分析定位和性能指标这块却一知半解 所以这道题虽然和性能相关

但是考察点在于平常项目如何进行性能定位和分析 我们可以从 前端性能监控-埋点以及`window.performance`相关的api去回答也可以从性能分析工具Performance和Lighthouse还可以从性能指标LCPFCPFIDCLS等去着手 下面是关于前端性能定位、优化指标以及计算方法的一些信息： 前端性能监控和埋点：通过在关键点上埋点，可以监控网页的加载时间、资源请求、错误等关键性能指标。常用的前端性能监控工具包括自定义的日志记录、第三方服务（如Google Analytics、Sentry等）和开源工具（如Fundebug、Tongji.js等）。此外，`window.performance`API提供了性能数据，可以通过它获取更详细的性能指标，如页面加载时间、资源加载时间等。性能分析工具：使用性能分析工具可以深入分析网站的性能瓶颈，并提供有针对性的优化建议。其中两个常用的工具是： 前端性能监控和埋点：通过在关键点上埋点，可以监控网页的加载时间、资源请求、错误等关键性能指标。常用的前端性能监控工具包括自定义的日志记录、第三方服务（如Google Analytics、Sentry等）和开源工具（如Fundebug、Tongji.js等）。此外，`window.performance`API提供了性能数据，可以通过它获取更详细的性能指标，如页面加载时间、资源加载时间等。性能分析工具：使用性能分析工具可以深入分析网站的性能瓶颈，并提供有针对性的优化建议。其中两个常用的工具是： Performance：现代浏览器提供的内置性能分析工具，可通过浏览器开发者工具访问。它提供了时间轴记录、CPU、内存和网络分析等功能，帮助开发者找到性能瓶颈并进行优化。Lighthouse：由Google开发的开源工具，可用于自动化测试网页性能，并提供综合的性能报告。它评估网页在多个方面

的性能表现，并给出相应的优化建议。

性能指标：性能指标是用于衡量网站性能的关键指标，常用的指标包括：LCP (Largest Contentful Paint)：标识页面上最大的可见内容加载完成的时间，衡量用户可见内容的加载速度。FCP (First Contentful

Paint)：表示页面上第一个内容元素（如文字、图片）呈现的时间，标识页面加载的起点。FID (First

Input Delay)：测量从用户首次与页面交互（点击链接、按钮等）到浏览器实际响应交互的时间。CLS (Cumulative

Layout Shift)：测量页面上元素布局的稳定性，即元素在页面加载过程中发生的意外移动的累积量。TTFB (Time

To First Byte)：表示从发起请求到接收到第一个字节的时间，衡量服务器响应速度。TTI (Time To

Interactive)：表示页面变得可交互的时间，即用户可以进行操作和与页面进行交互的时间点。TBT (Total

Blocking Time)：衡量页面在加载过程中存在的阻塞时间总和，即浏览器忙于处理 JavaScript

执行而导致无法响应用户输入的时间。这些指标可以通过性能分析工具或浏览器开发者工具来获得。优化这些指标有助于提升页面加载速度、响应性和用户体验。以下是这些指标的计算方法和示例代码：1.

LCP (Largest Contentful Paint)：

计算方法：监测到页面上的最大可见元素（如图片、视频等）加载完成的时间点。const observer=new PerformanceObserver((list)=>{for(const entry of list.getEntries()){if(entry.entryType==='largest-contentful-paint'){console.log('LCP:',entry.renderTime||entry.loadTime);}}});observer.observe({type:'largest-contentful-paint',buffered:true}); 2....

257. 谈谈你对函数是一等公民的理解

函数是一等公民，意思是它和数字、字符串、对象一样，都可以作为普通的值来使用。因此，函数既能赋给变量、放进数组或对象，也能作为参数传入另一个函数，或者作为返回值返回。这样的设计让高阶函数、函数组合和柯里化成为可能，也是JavaScript支持函数式编程的重要基础。函数还能通过闭包保留对外部变量的引用，适合封装状态，但也要留意这些引用带来的生命周期问题。

JavaScript中的函数被称为一等公民 (First-class Citizens)，这意味着函数在语言中被视为普通的值，可以像其他数据类型（例如数字、字符串、对象）一样被传递、赋值、存储和返回。以下是对 JavaScript

函数作为一等公民的几个重要特性和理解：可以赋值给变量：函数可以像其他数据类型一样赋值给变量。

你可以将函数定义存储在变量中，并在需要时将其作为值传递给其他函数或存储在数据结构中。可以作为

参数传递：函数可以作为参数传递给其他函数。这使得函数能够接受其他函数作为输入，并根据需要执行

或处理。可以作为返回值：函数可以作为另一个函数的返回值。你可以在一个函数内部定义并返回另一个

函数，这使得函数能够灵活地生成和返回其他函数。可以存储在数据结构中：函数可以存储在数组、对象

或其他数据结构中。这使得你可以在需要时使用函数，并根据需求对其进行组合、迭代或操作。可以通过

字面量或表达式定义：函数可以通过函数字面量（函数表达式）或函数声明来定义。这为我们提供了灵活

性，可以根据需要选择不同的方式来定义函数。可以通过闭包捕获状态：由于 JavaScript 中的函数形成了

闭包，函数可以访问其所在作用域中的变量。这意味着函数可以捕获并保持对外部变量的引用，即使在函

数外部不可访问的情况下也可以使用。这些特性使得 JavaScript 中的函数非常强大和灵活。函数作为一等

公民使得我们可以使用函数式编程的思想和技术，如高阶函数、函数组合、柯里化等，以更加优雅和灵活

地编写代码。---

【jQuery相关】

258. 你觉得jQuery源码有哪些写的好的地方

我觉得jQuery源码比较值得学习的是封装边界、原型扩展和兼容性处理。它用立即执行函数隔离内部变量，只把\$和jQuery暴露到全局，减少了命名污染。功能通过jQuery.fn扩展到原型上，既方便复用，也形成了清晰的插件机制；同时，它统一封装不同浏览器的差异，让调用方不必反复写兼容代码。源码里还会使用局部变量缓存和原生DOM操作兼顾效率，不过阅读时更应关注这些设计思想，而不是只记具体API。jQuery源码采用模块化的设计，将不同功能的代码模块化，并通过jQuery.fn扩展原型链，使得可以灵活地使用各种功能和方法。这样的设计使得代码结构清晰，易于维护和扩展。jQuery源码中考虑了跨浏览器兼容性，通过封装和处理不同浏览器的差异，使得开发者可以方便地编写跨浏览器兼容的代码。jQuery源码中使用了許多优化技巧，如缓存变量、使用局部作用域、使用原生DOM操作等，以提高性能和执行效率。jQuery源码注重代码的可读性和可维护性，采用了语义化的命名和良好的代码风格，使得代码易于理解和维护。jQuery源码提供了丰富的插件系统，使得开发者可以根据自己的需求扩展和定制jQuery的功能，且插件之间可以互相独立运行，提高了代码的可扩展性和重用性。

这些优点使得jQuery成为一个广泛使用的JavaScript库，并且为众多开发者提供了便利和效率。

以下是一个简单的示例，展示了jQuery源码中的一些写法和优点：

```
(function(window,undefined){  
在匿名函数中封装代码，避免全局污染varjQuery=function(selector,context){  
jQuery的构造函数//  
创建并返回一个jQuery对象，通过jQuery.fn扩展原型链returnnewjQuery.fn.init(selector,context);  
将jQuery原型指向jQuery.fn，方便使用jQuery.fn的方法jQuery.fn=jQuery.prototype=  
jQuery的原型对象constructor:jQuery,扩展的方法和属性init:function(selector,context){  
初始化函数// ...returnthis;},更多方法...};在jQuery原型上扩展方法jQuery.fn.extend({  
扩展的方法// ...});将jQuery暴露到全局作用域window.jQuery=window.$=jQuery;})(window);
```

这段代码展示了jQuery源码中的一些优点，包括使用匿名函数封装代码、通过原型链扩展方法、使用局部变量和缓存、考虑跨浏览器兼容性等。这些设计和写法使得jQuery成为一个功能强大、性能优秀、易于使用和扩展的JavaScript库。

259. jQuery 的实现原理

jQuery的实现本质上是对原生JavaScript和DOM能力做统一封装，再通过一个对象提供简洁的调用方式。源码放在立即执行函数中隔离作用域，并把jQuery和\$挂到window上供外部使用。它借助构造函数和原型链共享选择、遍历、事件等方法，方法返回当前jQuery对象时就能继续链式调用。除此之外，事件、AJAX和动画也被封装成统一接口，价值在于减少重复代码并屏蔽底层使用差异。

jQuery的实现原理可以总结如下：

使用立即调用表达式(IIFE)：jQuery的源码被包裹在一个匿名的立即调用函数表达式中(function(){ /* jQuery code */ })();，这样可以创建一个独立的函数作用域，避免变量污染全局命名空间。创建一个全局变量：通过window.jQuery = window.\$ = jQuery;将jQuery对象赋值给window对象的属性，从而使得jQuery和\$在全局作用域下可访问，方便其他代码使用。构造函数和原型链：jQuery使用function jQuery() { /* constructor code */ }定义了一个构造函数，使用jQuery.prototype扩展了原型链，从而在构造函数的基础上拥有了一系列方法和属性。DOM操作和选择器：jQuery封装了一系列DOM操作和选择器的方法，使得开发者可以通过简洁的语法来操作和遍历DOM元素。链式调用：jQuery的方法通常返回

对象本身，使得可以通过链式调用的方式进行连续的操作和修改。事件处理：jQuery

提供了强大的事件处理机制，可以方便地绑定和解绑事件，并提供了一系列事件处理方法。AJAX

请求：jQuery提供了简化的AJAX方法，使得进行异步数据请求变得更加便捷。动画效果：jQuery内置了一些常用的动画效果，如淡入淡出、滑动等，可以通过简单的方法调用来实现动画效果。总的来说，jQuery的实现原理是通过封装和扩展原生JavaScript功能，提供了便捷的DOM操作、事件处理、动画效果、AJAX请求等功能，使得开发者可以更快速、高效地开发和操作网页应用。

260. jQuery.fn的init方法返回的this指的是什么对象

jQuery.fn.init返回的this，指的是调用init初始化出来的当前jQuery对象。用户执行\$('div')后拿到的就是这个对象，后续可以在它上面调用addClass、css、text等原型方法。相关方法继续返回this，就能让多次操作始终围绕同一个对象进行，从而形成链式调用。需要注意，this不是原生DOM元素本身，而是封装了元素集合和操作方法的jQuery对象。jQuery.fn的init方法返回的this指的是jQuery对象本身。当用户使用jQuery()或\$('')初始化jQuery对象时，实际上是调用了init方法，而这个方法返回的就是一个jQuery对象，也就是this。通过返回this，jQuery实现了链式调用的特性，可以连续对同一个jQuery对象进行操作和调用方法。例如：

```
var $div = $('div'); // 初始化一个 jQuery 对象
$div.addClass('highlight') // 对该 jQuery 对象调用 addClass 方法
.css('color','red') // 继续调用 css 方法
.text('Hello, World!'); // 继续调用 text 方法
```

上述操作可以链式调用，连续对同一个jQuery对象进行多个方法的操作。在这个例子中，\$div是一个jQuery对象，通过调用addClass、css和text方法，并在每次方法调用后返回this，实现了链式调用的效果。这样的链式调用可以简化代码，提高可读性和开发效率。

261. jQuery.extend 与 jQuery.fn.extend 的区别

jQuery.extend()用来扩展jQuery本身或合并对象，jQuery.fn.extend()则用来扩展实例方法。前者添加的方法通过\$.method()调用，后者添加到jQuery.fn后，要通过\$(selector).method()调用。\$.extend(target, ...sources)会修改并返回目标对象，首个参数传true时还能递归合并。写通用工具适合用前者，写作用于选中元素的插件则用后者。

jQuery.extend()和jQuery.fn.extend()是jQuery提供的两个方法用于扩展功能。

jQuery.extend(object)：这个方法用于向jQuery添加静态方法，也称为工具方法。通过传入一个对象，可以将对象中的方法和属性添加到jQuery对象上，从而可以通过\$.method()的形式来调用这些静态方法。例如，\$.min()和\$.max()是通过\$.extend()添加的静态方法，可以直接通过\$.min()和\$.max()来调用。jQuery.extend([deep,] target, object1 [, objectN])：这个方法用于扩展目标对象(target)，将一个或多个对象的属性和方法合并到目标对象中。它可以实现对象的深度拷贝，还可以控制是否进行递归合并。第一个参数deep是可选的，用于控制是否进行深度拷贝，默认为浅拷贝。目标对象将被修改，同时返回目标对象。这个方法主要用于对象的合并和扩展。

jQuery.fn.extend(json)：这个方法用于向jQuery原型(jQuery.fn)添加成员函数，也称为实例方法。通过传入一个对象，可以将对象中的方法添加到jQuery原型上，从而可以通过\$(selector).method()的形式来调用这些实例方法。例如，\$.fn.alertValue()是通过\$.fn.extend()添加的实例方法，可以通过\$("#email").alertValue()来调用。综上所述，\$.extend()用于添加静态方法和进行对象的合并，而\$.fn.extend()用于添加实例方法。它们都是为了扩展jQuery的功能，提供更多的方法和功能供开发者使用。

当使用\$.extend()方法时，可以通过以下示例来理解其用法：

```
// 添加静态方法
$.extend({min:function(a,b){return a<b?a:b;}});
console.log($.min(2,3)); // 输出: 2
console.log($.max(4,5)); // 输出: 5

// 合并对象
var settings={validate:false,limit:5};
var options={validate:true,name:"bar"};
$.extend(settings,options);
console.log(settings); // 输出: { validate: true, limit: 5, name: "bar" }
```

当使用\$.fn.extend()方法时，可以通过以下示例来理解其用法：

```
// 添加实例方法
$.fn.extend({alertValue:function(){
    $(this).click(function(){
        alert($(this).val());
    });
}});
$("#email").alertValue();
```

点击元素时，弹出其值。在上述示例中，\$.extend()用于添加静态方法min()和max()，可以通过\$.min()和\$.max()来调用。另外，\$.extend()也用于将options对象的属性合并到settings对象中，实现对象的合并。而\$.fn.extend()用于添加实例方法alertValue()，通过选取具有id为"email"的元素，并调用.alertValue()方法，当点击该元素时会弹出其值。

262. jQuery 的属性拷贝(extend)的实现原理是什么，如何实现深拷贝

`$.extend()`本质上是遍历源对象的属性并写入目标对象，传入`true`后会递归复制引用类型。默认浅拷贝只复制对象或数组的引用，所以修改拷贝结果中的嵌套数据，原对象也可能跟着变化。使用`$.extend(true, {}, source)`时，嵌套对象和数组会继续向下复制，修改结果通常不会影响源对象。是否需要深拷贝，要看业务是否会修改嵌套数据。jQuery 的属性拷贝（`extend`）实现原理如下：浅拷贝：当使用`$.extend(target, obj1, obj2, ...)`进行属性拷贝时，它会将`obj1`、`obj2`等对象的属性复制到`target`对象中。如果属性值是对象或数组，那么复制的是对象或数组的引用，即浅拷贝。这意味着修改复制后的对象中的引用类型属性时，原始对象和拷贝后的对象会同时受到影响。深拷贝：如果需要进行深拷贝，即复制对象及其引用类型属性的值而不是引用，可以通过使用`$.extend(true, target, obj1, obj2, ...)`来实现。这样，在拷贝过程中，会递归遍历对象的属性，对引用类型属性进行深度拷贝。

浅拷贝：当使用`$.extend(target, obj1, obj2, ...)`进行属性拷贝时，它会将`obj1`、`obj2`等对象的属性复制到`target`对象中。如果属性值是对象或数组，那么复制的是对象或数组的引用，即浅拷贝。这意味着修改复制后的对象中的引用类型属性时，原始对象和拷贝后的对象会同时受到影响。深拷贝：如果需要进行深拷贝，即复制对象及其引用类型属性的值而不是引用，可以通过使用`$.extend(true, target, obj1, obj2, ...)`来实现。这样，在拷贝过程中，会递归遍历对象的属性，对引用类型属性进行深度拷贝。

以下是一个示例，展示浅拷贝和深拷贝的区别：

```
var obj1 = {name: "poetry", age: 30, hobbies: ["reading", "playing"], address: {city: "New York", country: "USA"}}; // 浅拷贝
var shallowCopy = $.extend({}, obj1);
shallowCopy.name = "Jane";
shallowCopy.hobbies.push("swimming");
shallowCopy.address.city = "San Francisco";
console.log(obj1); // 原始对象受到影响
console.log(shallowCopy); // 深拷贝
var deepCopy = $.extend(true, {}, obj1);
deepCopy.name = "Mike";
deepCopy.hobbies.push("traveling");
deepCopy.address.city = "Chicago";
console.log(obj1); // 原始对象不受影响
console.log(deepCopy);
```

在上述示例中，浅拷贝的结果是`shallowCopy`，它复制了`obj1`的属性，包括引用类型的数组`hobbies`和对象`address`。当修改`shallowCopy`的属性值时，原始对象`obj1`也会受到影响。而深拷贝的结果是`deepCopy`，它同样复制了`obj1`的属性，但是在拷贝过程中，对于引用类型的属性值进行了深度拷贝。因此，修改`deepCopy`的属性值不会影响到原始对象`obj1`。

263. jQuery 的队列是如何实现的

jQuery队列通过`queue()`保存函数，并由`dequeue()`按先进先出的顺序逐个取出执行。队列函数完成后调用传入的`next()`，才会继续处理下一项，因此异步任务也能按指定顺序衔接。`clearQueue()`可以直接清空尚未执行的函数，也可以用数组替换已有队列。连续动画就是典型场景，多段`animate()`会借助队列依次播放。jQuery 的队列是通过使用`queue()`、`dequeue()`和`clearQueue()`方法来实现的。`queue()`方法用于将一个或多个函数添加到指定元素的队列中。可以为每个队列命名，以便后续使用。函数将按照它们被添加到队列的顺序执行。`dequeue()`方法用于从队列中取出并执行队列中的下一个函数。它按照先进先出的顺序执行队列中的函数，并返回执行的结果。`clearQueue()`方法用于清空队列中的所有函数，使队列为空。以下是一个示例，展示了如何使用队列控制方法：

```
var func1 = function() { console.log('事件1'); };
var func2 = function() { console.log('事件2'); };
var func3 = function() { console.log('事件3'); };
var func4 = function() { console.log('事件4'); };
// 入栈队列事件
$('#box').queue("queue1", func1); // 将 func1 添加到 queue1 队列
$('#box').queue("queue1", func2); // 将 func2 添加到 queue1 队列
// 替换队列事件
$('#box').queue("queue1", []); // 清空 queue1 队列
$('#box').queue("queue1", [func3, func4]); // 替换 queue1 队列为 func3 和 func4
// 获取队列事件（返回一个函数数组）
var queue1 = $('#box').queue("queue1");
console.log(queue1); // [func3(), func4()]
// 出栈队列事件并执行
$('#box').dequeue("queue1"); // 执行 func3
$('#box').dequeue("queue1"); // 执行 func4
// 清空整个队列
$('#box').clearQueue("queue1"); // 清空 queue1 队列
```

在上述示例中，使用`queue()`方法将函数添加到名为“queue1”的队列中。然后，使用

dequeue()方法从队列中取出并执行函数。使用clearQueue()方法可以清空整个队列。 请注意，上述示例中使用了console.log()函数来输出结果。你可以根据需要将其替换为适合你的代码的逻辑。 队列在jQuery

中的主要应用场景包括：

动画效果：通过将多个动画函数添加到队列中，按照顺序依次执行，实现动画的连续效果。 \$('#element').animate({property1:value1},duration1).animate({property2:value2},duration2).animate({property3:value3},duration3); 异步操作：当需要按照特定的顺序执行一系列异步操作时，可以将这些操作添加到队列中，确保它们按照期望的顺序执行。

```
$('#button').click(function(){$('#loading').fadeIn().delay(2000).fadeOut();// 显示加载动画$('#result').queue(function(next){$.ajax({url:'example.com',success:function(data){$(this).html(data);//
```

```
异步请求完成后更新页面内容next();// 执行下一个队列函数}});}); 队列控制：通过队列控制方法，可以按照需要添加、移除、替换或清空队列中的函数，从而灵活地控制函数的执行顺序和流程。
```

```
$('#element').queue("myQueue",function(next){// 队列函数的逻辑next();// 执行下一个队列函数});$('#element').dequeue("myQueue");//
```

```
执行队列函数$('#element').clearQueue("myQueue");// 清空队列 总之，队列在 jQuery中用于管理和控制一系列函数的顺序执行，特别适用于动画效果、异步操作和队列控制的场景。
```

264. jQuery 中的 bind(), live(), delegate(), on() 的区别

bind()是直接绑定，live()和delegate()属于事件代理，而on()是现在统一使用的绑定入口。bind()只作用于当前匹配元素，动态新增元素不会自动获得处理函数；live()把代理放在document上，范围较大且已被弃用。delegate()能把代理限制在指定父元素上，但也推荐改用on()。面对动态子元素时，可以写成\$(parent).on('click', '.child', handler)。 示例代码： // bind()\$('#myButton').bind('click',function(){// 点击事件处理逻辑});// live()\$('#myButton').live('click',function(){// 点击事件处理逻辑});// delegate()\$('#myContainer').delegate('#myButton','click',function(){// 点击事件处理逻辑});// on()\$('#myButton').on('click',function(){// 点击事件处理逻辑}); 解释： bind()方法直接在目标元素上绑定事件处理程序，适用于静态元素。live()方法通过事件冒泡在document上捕获事件，可以处理动态添加的元素。但在 jQuery 1.7+ 版本中已被弃用，推荐使用on()方法代替。delegate()方法通过事件冒泡在指定的父元素上进行事件代理，可以精确指定事件的范围，适用于动态添加的元素。在 jQuery 1.7+ 版本中，推荐使用on()方法代替。on()方法是最新的事件绑定机制，可以替代bind()、live()和delegate()。它可以在目标元素上直接绑定事件，也可以通过事件冒泡或事件代理进行处理。使用on()方法更加灵活和统一，是推荐的事件绑定方式。 请注意，以上示例是基于jQuery 1.7+版本的使用法，如果使用的是旧版本的jQuery，部分方法可能已被弃用或行为有所不同。建议查阅对应版本的官方文档以获取详细信息。

265. 是否知道自定义事件

自定义事件就是利用事件机制实现发布与订阅，让不同模块不用直接互相调用。原生写法是用CustomEvent创建事件，通过addEventListener监听，再用dispatchEvent触发，业务数据可以放在detail中。需要事件冒泡时，要显式把bubbles设为true，因为它默认不冒泡。jQuery中则用.on()监听、.trigger()触发，适合组件通知或模块解耦。 自定义事件是一种在JavaScript中实现发布/订阅模式的方式，通过自定义事件可以实现模块间的解耦和更灵活的事件处理。 在原生JavaScript中，可以使用CustomEvent对象来创建自定义事件，然后使用addEventListener方法来监听事件，使用dispatchEvent方法来触发事件。

示例代码如下：

//

```
创建自定义事件var customEvent=new CustomEvent('myEvent',{detail:{data:'example'}});//
```

```
监听事件document.addEventListener('myEvent',function(event){console.log('Event triggered: '+event.detail.data);});// 触发事件document.dispatchEvent(customEvent);
```

在jQuery中，可以使用on方法来绑定自定义事件的处理函数，使用trigger方法来触发自定义事件。

示例代码如下：

```
//
绑定自定义事件处理函数$(document).on('myEvent',function(event,data){console.log('Event
triggered:      '+data);});//      触发自定义事件$(document).trigger('myEvent','example');
通过自定义事件，可以实现模块之间的松耦合，使代码更加可维护和可扩展。
```

266. jQuery 通过哪个方法和 Sizzle 选择器结合的

jQuery主要通过\$.find()与Sizzle选择器引擎结合。调用时把选择器交给Sizzle解析和执行，并把当前元素作为查找上下文，所以既能从整个文档查找，也能把范围限制在某个容器内。比如\$('.parent').find('.child')只筛选该父元素内部的匹配节点，这样查找范围更明确。

在jQuery中，通过\$.find()方法与Sizzle选择器结合使用来进行元素的查找和筛选。 示例代码如下： //
在整个文档中查找符合条件的元素varelements=\$('body').find('.my-class');//

在特定元素内部查找符合条件的子元素varchildren=\$('.parent-element').find('.child-class'); 在上面的示例中，\$.find()方法用于在指定的上下文中查找符合条件的元素。可以传递一个选择器作为参数，由Sizzle选择器引擎解析和执行。

通过\$.find()方法，jQuery与Sizzle选择器结合，可以实现灵活的元素查找和筛选操作。

267. jQuery 中如何将数组转化为 JSON 字符串，然后再转化回来

数组转成JSON字符串用JSON.stringify()，需要还原时再用JSON.parse()。这两个都是原生方法，通常没必要依赖jQuery再封装一套；旧代码里的\$.parseJSON()从jQuery 3.0开始已经过时。实际使用时可以封装成\$.array2json和\$.json2array，方便统一调用。还要注意输入必须是合法的JSON字符串，否则解析会直接报错。 // 通过原生 JSON.stringify/JSON.parse 扩展 jQuery
实现\$.array2json=function(array){returnJSON.stringify(array);}\$.json2array=function(array){//
\$.parseJSON(array); // 3.0 开始，已过时returnJSON.parse(array);}
调用varjson=\$.array2json(['a','b','c']);vararray=\$.json2array(json);

268. jQuery 一个对象可以同时绑定多个事件，这是如何实现的

jQuery通过.on()接收多个事件名，或者接收“事件名到处理函数”的对象，从而给同一个对象绑定多个事件。如果多个事件共用一个逻辑，可以写成\$('#btn').on('mouseover mouseout', func)，触发任意一个事件都会调用func。如果处理逻辑不同，就传入对象，分别为mouseover、mouseout和click指定函数。两种写法本质相同，选择时主要看这些事件是否需要共享处理逻辑。

在jQuery中，可以使用.on()方法为一个对象同时绑定多个事件。 示例代码1： \$("#btn").on("mouseover mouseout",func); 上述代码中，通过.on()方法为#btn元素同时绑定了mouseover和mouseout两个事件，它们都会调用同一个处理函数func。 示例代码2：

\$("#btn").on({mouseover:func1,mouseout:func2,click:func3}); 上述代码中，通过.on()方法为#btn元素同时绑定了mouseover、mouseout和click三个事件，分别指定了不同的处理函数func1、func2和func3。这样，当这些事件触发时，相应的处理函数将被调用。通过将多个事件名称和处理函数作为参数传递给.on()方法，或者使用一个包含事件名称和处理函数的对象作为参数，可以实现为一个对象同时绑定多个事件。这样做既简洁又方便，可以提高代码的可读性和维护性。

269. 针对 jQuery 的优化方法

优化jQuery的重点，是减少重复查询、缩小选择范围，并控制对象和事件处理的开销。我一般会缓存频繁使用的DOM对象，从父级#id开始查找，避免反复遍历页面。多个操作可用链式调用，列表事件则通过on做事件委托，减少绑定数量。元素关联的数据可以交给data()管理，同时应选择较新的jQuery版本，以获

得已有的性能和稳定性改进。 缓存频繁操作 DOM 对象： 当需要频繁操作某个 DOM 对象时，将其缓存到一个变量中可以避免不必要的 DOM 查询和遍历，提高代码的执行效率。 尽量使用 id 选择器代替 class 选择器： 使用 id 选择器的性能比 class 选择器更高，因为 id 在页面中是唯一的，而 class 可能存在多个匹配项。 总是从 #id 选择器来继承： 通过指定父级元素的 id 选择器来限定查找范围，这样可以减少选择器的搜索范围，提高查找速度。 尽量使用链式操作： 链式操作可以减少不必要的 jQuery 对象的创建和查找，简化代码结构，提高代码的可读性和性能。 使用事件委托 on 绑定事件： 通过将事件绑定在父元素上，利用事件冒泡机制，将事件处理委托给父元素处理，可以减少事件绑定的数量，提高性能。 采用 jQuery 的内部函数 data() 来存储数据： 使用 data() 函数可以将数据附加到 DOM 元素上，避免在元素上存储额外的数据属性，减少内存占用和提高性能。 使用最新版本的 jQuery： 每个版本的 jQuery 都在性能方面进行了优化和改进，使用最新版本可以获得更好的性能和稳定性。 通过应用这些优化方法，可以有效提升 jQuery 代码的执行效率和性能，提升用户体验。 下面给出一些示例代码来演示上述优化方法的应用： 缓存频繁操作 DOM 对象： // 不优化的写法for(var i=0;i<\$(".item").length;i++){\$(".item").eq(i).addClass("active");} // 优化后的写法var \$items=\$(".item");for(var i=0;i<\$items.length;i++){\$items.eq(i).addClass("active");} 尽量使用 id 选择器代替 class 选择器： // 不优化的写法\$(".container .item").addClass("active");// 优化后的写法\$("#container #item").addClass("active"); 总是从 #id 选择器来继承： // 不优化的写法\$(".container .item").find(".sub-item").addClass("active");// 优化后的写法\$("#container").find(".item").find(".sub-item").addClass("active"); 尽量使用链式操作： // 不优化的写法\$(".item").addClass("active");\$(".item").removeClass("hidden");\$(".item").show();// 优化后的写法\$(".item").addClass("active").removeClass("hidden").show(); 使用事件委托 on 绑定事件： // 不优化的写法\$(".button").click(function(){// 处理点击事件}); // 优化后的写法\$("#container").on("click",".button",function(){// 处理点击事件}); 采用 jQuery 的内部函数 data() 来存储数据： // 不优化的写法\$(".item").attr("data-value","123");// 优化后的写法\$(".item").data("value","123"); 这些示例代码展示了如何应用上述优化方法，根据具体的使用场景选择相应的优化方法可以提高 jQuery 代码的性能和执行效率。

270. jQuery 的 slideUp 动画，当鼠标快速连续触发，动画会滞后反复执行，该如何处理呢

可以在执行slideUp()前调用stop()，终止正在进行的动画，再启动本次动画。鼠标连续触发时，每次事件都会再次执行动画，因此前面的动画还没结束，后面的调用就会不断累积。比如在mouseover中使用\$(".element").stop().slideUp()，可以避免动画反复滞后。也可以用setTimeout延迟执行，但延迟时间需要结合实际交互效果来取舍。 当鼠标快速连续触发导致动画滞后反复执行时，可以采取以下两种处理方式： 设置延迟处理： 在触发元素上的事件处理函数中，使用setTimeout方法来延迟执行动画，从而避免快速连续触发造成动画滞后反复执行的问题。

```
$(".trigger").on("mouseover",function(){setTimeout(function(){$(".element").slideUp();},200);});
```

上述代码中，通过设置 200 毫秒的延迟，在每次触发事件时先延迟一段时间后再执行动画，从而确保动画能够顺利完成。

停止所有动画并执行相应动画： 在触发元素的事件处理函数中，可以先使用stop方法停止所有正在进行的动画，然后再执行相应的动画事件。

```
$(".trigger").on("mouseover",function(){$(".element").stop().slideUp();});
```

通过调用stop方法，可以立即停止所有正在进行的动画，并清除动画队列，然后再执行新的动画事件。这样可以避免动画滞后反复执行的问题。以上两种处理方式可以根据具体的情况选择使用，以确保动画的正常执行。

271. jQuery UI 如何自定义组件

自定义jQuery UI组件可以调用`$.widget("ns.widgetName", [baseWidget], widgetPrototype)`，把组件名称、可选基类和原型对象传进去。原型对象里通常定义`options`、`_create()`、`_refresh()`和`_setOption()`，分别负责默认配置、初始化、刷新外观与响应配置变化。需要复用现有能力时可以传入基础组件进行扩展，否则可以省略。组件创建后，既能直接初始化，也能通过`option`修改具体配置。

通过向`$.widget()`方法传递组件名称和一个原型对象，可以自定义 jQuery UI 组件。具体的使用方法如下：`$.widget("ns.widgetName",[baseWidget],widgetPrototype)`；其中：`"ns.widgetName"`是组件的名称，命名空间（ns）和组件名称（widgetName）可以根据实际情况进行替换。`[baseWidget]`是可选参数，表示组件的基础组件，可以是其他已定义的 jQuery UI 组件，用于扩展和继承基础组件的功能。`widgetPrototype`是一个包含组件方法和属性的原型对象，用于定义自定义组件的行为和外观。

以下是一个示例代码，演示如何自定义一个简单的 jQuery UI 组件：
`$.widget("custom.customWidget",{// 组件的默认选项options:{color:"blue",fontSize:12},// 组件的创建函数，初始化组件_create:function(){this.element.addClass("custom-widget").text("Custom Widget");this._refresh();},// 刷新组件，根据选项更新外观_refresh:function(){this.element.css({color:this.options.color,fontSize:this.options.fontSize+"px"});},//`

`设置选项的方法_setOption:function(key,value){this._super(key,value);this._refresh();});` 上述示例中，定义了一个名为`customWidget`的自定义组件。组件具有默认选项`color`和`fontSize`，在创建函数`_create`中初始化组件，并在`_refresh`方法中根据选项更新外观。使用`_setOption`方法来设置选项并触发刷新。

使用自定义组件时，可以通过以下方式进行初始化和设置选项：
`$("#myElement").customWidget();` // 初始化自定义组件
`$("#myElement").customWidget("option","color","red");` // 设置选项
通过以上方式，可以自定义 jQuery UI 组件并进行初始化和配置。

272. jQuery 与 jQuery UI、jQuery Mobile 区别

jQuery是基础JavaScript库，jQuery UI是基于它的界面组件库，而jQuery Mobile面向移动Web应用。jQuery主要简化DOM操作、事件、动画和AJAX，解决的是通用网页开发问题。需要对话框、拖拽、排序或主题时，可以使用jQuery UI。如果场景侧重移动设备的页面导航、表单和触摸交互，则jQuery Mobile的定位更合适，选择时要看目标设备和界面需求。jQuery是一个功能强大的 JavaScript 库，主要用于简化和增强 JavaScript 开发中的 DOM 操作、事件处理、动画效果和 AJAX 请求等。它主要面向 PC 浏览器环境，使开发者能够更方便地操作和操控网页元素。jQuery UI则是建立在jQuery库的基础上的一个用户界面库。它提供了一系列交互性的特效、小部件和主题，用于创建丰富而美观的用户界面。jQuery UI的组件包括对话框、拖拽、排序、自动完成等，开发者可以根据自己的需要选择和配置相应的组件。jQuery

Mobile是一个专门针对移动设备的框架，也是基于jQuery的。它提供了一套用于构建移动 Web 应用的工具和组件，使开发者能够快速创建适配移动设备的界面和交互效果。jQuery Mobile的组件包括页面导航、按钮、表单元素、触摸事件等，可以帮助开发者构建具有响应式和触摸友好的移动应用程序。

综上所述，jQuery主要用于 PC 浏览器环境下的 Web 开发，jQuery UI则提供了丰富的用户界面组件和主题，而jQuery Mobile则专注于移动设备上的 Web 应用开发。每个库都有自己的特点和应用场景，可以根据具体的需求选择使用。

273. jQuery 和 Zepto 的区别？各自的使用场景

jQuery更偏向功能完整和浏览器兼容，Zepto则面向移动端，强调轻量和性能。两者的API比较接近，都能处理DOM、事件和动画，但Zepto对桌面浏览器的兼容性不如jQuery。需要兼顾多种浏览器的传统PC项目，我会选jQuery；资源敏感的移动页面更适合Zepto。如果是新项目，也要结合原生API和现代框架再做取

舍。你的总结很准确。jQuery是一个功能强大的JavaScript库，主要面向PC网页开发，并兼容主流的浏览器。它提供了丰富的功能和跨浏览器的兼容性，可以方便地进行DOM操作、事件处理、动画效果和AJAX请求等。在移动设备方面，jQuery单独推出了jQuery Mobile，用于移动端的Web应用开发。Zepto是一个专注于移动设备的JavaScript库，它的设计目标是提供轻量级的库并具有良好的性能。Zepto的API基本兼容jQuery，可以方便地进行DOM操作、事件处理和动画效果等。由于它专注于移动设备，所以在移动端的性能和兼容性方面表现更好。然而，在PC浏览器上的兼容性并不如jQuery。因此，jQuery主要用于PC网页开发，适用于需要广泛兼容各类浏览器的项目。而Zepto则更适用于移动设备的开发，尤其是在对性能和轻量级要求较高的移动应用中。根据具体的项目需求和目标平台选择合适的库是很重要的。

274. jQuery对象的特点

jQuery对象本质上是包装了一组原生DOM元素的类数组对象。它有数字索引和length，但不是真正的数组，元素遍历通常使用.each()等jQuery方法。它还支持隐式迭代和链式调用，例如对集合执行.hide().fadeIn()，每个元素都会被处理。需要原生节点时，可以通过[0]或.get(0)取出。jQuery对象是由jQuery构造函数创建的对象，它是一个类数组对象。jQuery对象是通过选择器选择的一组元素，可以对这组元素进行操作和处理。jQuery对象具有jQuery库提供的一系列方法，可以方便地进行DOM操作、事件处理、动画效果和数据交互等。jQuery对象可以使用jQuery的方法进行链式调用，提高代码的简洁性和可读性。需要注意的是，jQuery对象并不是真正的数组，虽然它类似于数组，但不具备数组的所有特性。但可以使用类数组对象的一些方法，如.each()和.length。示例：

```
var $divs = $('div'); // 选择所有的 div 元素，返回一个 jQuery 对象
console.log($divs.length); // 输出 div 元素的数量
$divs.hide().fadeIn(); // 链式调用方法，隐藏 div 元素然后淡入显示
$divs.each(function() { // 遍历每个 div 元素并执行回调函数
    console.log($(this).text()); // 输出每个 div 元素的文本内容
});
```

总结起来，jQuery对象是一个特殊的对象，它代表了一组元素，并提供了方便的方法和链式调用，以简化对这组元素的操作。

275. Zepto的实现原理

Zepto的实现思路是基于原生浏览器能力做一层轻量封装，提供接近jQuery的调用方式。它通过选择器引擎完成元素查询，再用原生DOM API封装增删改查；事件部分则建立在原生事件机制之上，提供.on()、.off()等接口。动画通过修改CSS属性并配合定时器完成，异步请求基于XMLHttpRequest。它也支持插件扩展，但并不保证兼容jQuery的全部功能。Zepto是一个轻量级的JavaScript库，旨在提供类似于jQuery的功能，但专注于移动设备的Web开发。它的实现原理如下：

核心选择器功能：Zepto使用CSS选择器引擎Sizzle来实现元素的选择和遍历。Sizzle是一个独立的选择器库，它使用了CSS3选择器语法和一些扩展来支持复杂的选择操作。

DOM操作：Zepto封装了一系列DOM操作方法，如添加、删除、修改和查询元素等。它通过原生DOM API来实现这些方法，并对其进行了封装和优化，以提供更便捷的操作接口。

事件处理：Zepto提供了一套事件处理机制，通过绑定和触发事件来实现交互功能。它使用原生的DOM事件来处理事件，并提供了一些简化和增强的方法，如.on()、.off()、.trigger()等。

动画效果：Zepto支持基本的动画效果，如淡入淡出、滑动、渐变等。它通过改变元素的CSS属性值和使用定时器来实现动画效果，同时对动画进行了优化，以提高性能和流畅度。

AJAX请求：Zepto提供了简单的AJAX功能，可以发送异步请求并处理服务器返回的数据。它使用原生的XMLHttpRequest对象来实现AJAX请求，并提供了一些便捷的方法，如\$.ajax()、\$.get()、\$.post()等。

扩展插件：Zepto支持通过插件扩展其功能。开发者可以根据需要编写自定义插件，并将其集成到Zepto中，以提供更多的功能和特性。

总而言之，Zepto的实现原理是基于原生的DOM API和一些辅助库，通过封装和优化，提供了类似于jQuery的功能，并专注于移动设备的Web开发。它的目标是提供简洁、高效的代码，以满足移动设备上的性能和资源要求。以下是Zepto

的一些常见用法示例代码：选择器和 DOM 操作：// 通过选择器获取元素并操作

```
var $element = $('my-element');
$element.addClass('highlight');
$element.text('Hello, Zepto!');
// 创建新元素并插入到页面中
var $newElement = $('New Element');
$('body').append($newElement);
// 事件处理：// 绑定事件处理函数
$('my-button').on('click', function() { console.log('Button clicked!'); });
// 解绑事件处理函数
$('my-button').off('click');
// 触发自定义事件
$('my-element').trigger('customEvent');
// 动画效果：// 淡入淡出效果
$('my-element').fadeIn();
$('my-element').fadeOut();
// 滑动效果
$('my-element').slideDown();
$('my-element').slideUp();
// 渐变效果
$('my-element').animate({ opacity: 0.5 }, 500);
// AJAX 请求：// 发送 GET 请求
$.get('https://api.example.com/data', function(response) { console.log('Response:', response); });
// 发送 POST 请求
$.post('https://api.example.com/data', { name: 'poetry', age: 25 }, function(response) { console.log('Response:', response); });
```

这些示例代码展示了 Zepto 的一些常见用法，包括选择器和 DOM 操作、事件处理、动画效果以及 AJAX 请求。请注意，Zepto 的用法与 jQuery 类似，但并不完全兼容 jQuery 的所有功能和方法。

276. 是否用过 jQuery 的 Deferred

Deferred是jQuery用来组织异步流程的一套方案，作用和后来的Promise比较接近。创建对象后，可以用`resolve()`或`reject()`改变状态，并通过`done()`、`fail()`、`always()`分别处理成功、失败和结束逻辑。对外只暴露`promise()`，可以避免调用方随意修改状态。维护老的jQuery项目时经常会遇到它，新代码通常直接使用原生Promise或`async/await`。---

【Bootstrap相关】

277. 什么是Bootstrap？以及为什么要使用Bootstrap？

Bootstrap是一个基于HTML、CSS和JavaScript的开源前端框架，用来快速搭建响应式、移动优先的网页。它把常见布局、样式和交互封装成统一组件，能减少重复开发，让页面更快成形。它还处理了不同屏幕的适配和主流浏览器兼容问题，并提供了完善的文档与社区支持。需要快速交付后台或通用网站时很合适，但具体界面仍要结合业务需求选择和调整组件。Bootstrap是一个流行的开源前端框架，用于快速构建响应式和移动优先的网站和Web应用程序。它基于HTML、CSS和JavaScript，并提供了一套预定义的样式和组件，使开发人员能够快速构建现代化、美观和功能丰富的界面。使用Bootstrap的原因如下：

- 快速开发：Bootstrap提供了大量的预定义样式和组件，开发人员可以通过简单地应用这些样式和组件来快速构建页面和布局。这样可以节省大量的时间和精力，加快项目的开发速度。
- 响应式设计：Bootstrap具有响应式设计的特性，能够适应不同屏幕大小和设备类型的展示。无论是在桌面电脑、平板还是手机上，页面都能够自动适配和呈现最佳的用户体验。
- 浏览器兼容性：Bootstrap经过广泛的测试和优化，能够在各种主流浏览器中保持一致的显示效果和功能。开发人员不需要为不同浏览器的差异而担心，可以放心地使用Bootstrap来开发跨浏览器兼容的网站和应用程序。
- 组件丰富：Bootstrap提供了大量的UI组件，例如导航菜单、按钮、表格、表单、模态框等，这些组件具有统一的样式和交互效果，可以快速构建出现代化的用户界面。
- 社区支持和文档丰富：Bootstrap拥有庞大的开发者社区，有许多开发者在使用和贡献Bootstrap，因此可以轻松获取到各种问题的解答和技术支持。此外，官方提供了详细的文档和示例，开发人员可以方便地学习和使用Bootstrap。

总而言之，使用Bootstrap可以使开发人员更加高效地构建现代化的网站和Web应用程序。

，减少重复工作，提高开发速度，并确保页面具有良好的用户体验和浏览器兼容性。

278. 使用Bootstrap时，要声明的文档类型是什么？以及为什么要这样声明？

使用Bootstrap时，应在HTML文件开头声明HTML5文档类型，也就是。这个声明会告诉浏览器按照现代标准解析和渲染页面，避免因文档类型不同而进入其他解析模式，影响样式和组件的一致性。它也让页面可以正常使用HTML5的语义化元素、属性和相关API，并更容易通过W3C标准验证。简单来说，这既是Bootstrap正常工作的基础，也有利于页面在不同浏览器及未来Web环境中的兼容性。在使用Bootstrap时，需要在HTML文件的开头声明HTML5文档类型（Doctype），即。这是因为Bootstrap使用了一些HTML5的新特性、元素和CSS属性，而HTML5文档类型的声明能够告诉浏览器使用HTML5的解析方式来解析页面内容。

声明HTML5文档类型的重要性在于：

- 使用HTML5特性：声明HTML5文档类型后，开发人员可以在页面中使用HTML5新增的元素、属性和API，如、、等。这些新特性可以提供更好的语义化结构，以及更丰富和灵活的功能。
- 浏览器解析方式：声明HTML5文档类型可以让浏览器按照HTML5的规范来解析和渲染页面，确保页面在各种浏览器中一致地显示。不同的文档类型会触发不同的浏览器解析模式，而HTML5的解析模式更符合现代化的标准和规范。
- 代码验证：使用HTML5文档类型声明可以使代码通过W3C标准的验证，符合Web开发的最佳实践。W3C验证能够帮助开发人员发现和修复代码中的错误和潜在问题，提高代码的质量和可靠性。
- 未来兼容性：HTML5是Web发展的趋势和标准，使用HTML5文档类型声明可以确保页面在未来的Web环境中保持良好的兼容性。同时，它也是使用其他HTML5相关技术和框架的前提条件，如Canvas、Web Components等。

总之，声明HTML5文档类型是使用Bootstrap时的必要步骤，它能够确保页面能够正常使用Bootstrap提供的样式和组件，并且能够充分利用HTML5的新特性和浏览器解析方式，以获得更好的开发和用户体验。

279. 什么是Bootstrap网格系统

Bootstrap网格系统是一套基于 12 列栅格、用于构建响应式页面布局的组件。它通过container、row和列来组织内容，开发者只需指定元素占用的列数，就能灵活组合不同宽度。比如col-md-6表示在对应屏幕尺寸下占据一半宽度，视口变化时列会自动调整或重新排列。它适合快速统一桌面、平板和移动端布局，具体列宽也可以按不同尺寸分别设置并按需扩展。Bootstrap网格系统是Bootstrap框架中的一个核心组件，用于创建响应式的网页布局。它基于一个12列的栅格系统，可以根据设备或视口的大小，将页面水平分割为不同的列。关键特点如下：

- 响应式设计：Bootstrap的网格系统是响应式的，可以自动适应不同屏幕尺寸，从大屏幕到小屏幕，如桌面、平板和移动设备。网格系统的列会自动调整和重新排列，以适应不同的视口大小，从而提供更好的用户体验。
- 12列栅格系统：网格系统将页面水平分为12列，并使用CSS的类来定义每个元素所占的列数。开发人员可以根据需要将元素放置在这些列中，从而创建灵活的布局。列数可以是整数，也可以是小数，以实现更精细的布局。
- 容器和行：网格系统由容器（container）和行（row）组成。容器是网格系统的最外层包裹元素，用于限制网格内容的宽度，并提供水平的内边距。行是容器内的一行元素，用于包含网格列。每一行被分割为12个等宽的列。
- 列（Column）：列是网格系统的基本单位，位于行内，用于放置内容。开发人员可以通过为列添加CSS类来定义其所占的列数，如col-md-6表示该列占据父容器的一半宽度。可以根据需要在不同的屏幕尺寸上指定不同的列宽。通过使用Bootstrap的网格系统，开发人员可以轻松创建具有灵活布局的响应式网页。网格系统提供了简单易用的类，使得网页元素可以自适应不同的屏幕尺寸，从而提供了一致的用户体验，并且可以根据需要进行自定义和扩展。

280. Bootstrap 网格系统（Grid System）的工作原理

Bootstrap网格系统本质上是把页面横向分成 12 份，通过容器、行和列组合出不同宽度的布局。row放在.container中，列再作为row的直接子元素，这样容器负责宽度、对齐和内边距，行负责组织水平方向的列。列宽由预定义类控制，例如.col-xs-4占 4 份，也就是三分之一；列的padding形成间距，行的负margin再抵消外侧多出的空白。比如要排三个等宽区域，可以放三个.col-xs-4，更复杂的布局则通过调整各列占用的

份数来实现。Bootstrap网格系统的工作原理如下：容器（Container）：在使用Bootstrap网格系统之前，需要将行（Row）放置在.container类内。容器提供适当的对齐和内边距，并将网格内容限制在特定宽度范围内。行（Row）：行用于创建列的水平组。行必须是容器的直接子元素。通过将元素放置在行内，可以将其划分为网格列，并确定它们在水平方向上的排列方式。列（Column）：列是网格系统的基本单位，用于放置内容。列必须是行的直接子元素。使用预定义的网格类（例如.col-xs-4）可以快速创建网格布局。通过指定要横跨的列数，可以将内容放置在相应的列中，例如.col-xs-4表示占据12个列中的4个列，即1/3的宽度。列间距（Gutter）：列之间的间距是通过列内的内边距（padding）来创建的。网格系统使用负外边距（margin）在行上来抵消列的内边距，从而实现列之间的间隔。这样可以确保列之间没有额外的空白间隙。列宽度：Bootstrap网格系统将页面水平分为12列。通过指定要横跨的列数，可以控制列的宽度。例如，如果想要创建三个相等宽度的列，可以使用三个.col-xs-4，每个列占据12个列中的4个列，即1/3的宽度。通过遵循以上原则，可以利用Bootstrap网格系统快速创建具有灵活布局的网页。网格类提供了简单且易于使用的方式来定义列的宽度和排列方式，而预定义的样式和混合类则可用于更高级的布局需求。

281. 对于各类尺寸的设备，Bootstrap设置的class前缀分别是什么

Bootstrap 4的网格前缀依次是.col-、.col-sm-、.col-md-、.col-lg-和.col-xl-。它们对应=576px）：.col-sm-中型设备台式电脑（>=768px）：.col-md-大型设备台式电脑（>=992px）：.col-lg-超大型设备台式电脑（>=1200px）：.col-xl- 请注意，Bootstrap 4中的class前缀在移动优先的设计下发生了变化。超小设备手机没有前缀，而其他尺寸的设备都使用相应的前缀进行类名设置。这些前缀用于指定列的宽度和响应式行为。

282. Bootstrap 网格系统列与列之间的间隙宽度是多少

Bootstrap网格系统默认的列间隙是30px。这个间隙并不是单独插入一个空白元素，而是由每列左右各15px的padding共同形成。相邻两列靠在一起时，两侧内边距相加正好得到30px，本质上是用列内边距实现视觉分隔。调整列样式时要注意这层结构，避免重复添加间距导致布局变宽。在Bootstrap中，默认的列与列之间的间隙宽度是30px。这个间隙宽度是通过列的内边距（padding）来实现的，每个列的左右内边距都是15px。这样可以在列与列之间创建一个空白间隔，用于提供视觉上的分隔和排列。

283. 如果需要在标题的旁边创建副标题，可以怎样操作

最直接的做法是在标题内部加入，例如主标题 副标题。这样主标题和副标题处在同一个标题元素中，同时可以分别控制副标题的字号和样式。如果设计只需要较小的文字效果，也可以使用.small类并在CSS中定义样式。实际选择时，我一般看副标题是否需要独立标记：需要就用，只调整外观则可以用类。

要在一个标题的旁边创建副标题，可以使用以下两种方式进行操作：

使用元素：在标题元素内部的旁边添加一个元素，并在该元素中放置副标题的内容。示例代码如下：

主标题副标题 在上述示例中，元素表示主标题，而元素表示副标题。

使用.small类：给标题元素添加一个.small的类，然后在CSS中定义该类的样式。示例代码如下：主标题在上述示例中，通过给元素添加.small类，可以在CSS中为该元素定义副标题的样式。无论是使用元素还是添加.small类，都可以在标题的旁边创建副标题，并通过样式进行调整。选择哪种方式取决于具体的需求和设计风格。

284. 用Bootstrap，如何设置文字的对齐方式？

在Bootstrap中，可以分别使用text-left、text-center和text-right设置文字左对齐、居中和右对齐。这些工具类可以直接加到、或等文本元素上，因为它们负责的是元素中文字的对齐方式。比如居中文本就会让段落内容居中。简单来说，根据设计需要选择对应类即可，不必再单独编写一套对齐样式。在Bootstrap

中，可以使用以下类来设置文字的对齐方式：

text-center：用于将文本居中对齐。text-right：用于将文本向右对齐。text-left：用于将文本向左对齐。

这些类可以应用于任何文本元素，例如、`<p>`等。示例代码如下：

居中对齐的文本 向右对齐的标题 向左对齐的文本 通过给相应的元素添加适当的类，即可设置文本的对齐方式。请根据需要选择相应的类来实现所需的对齐效果。

285. Bootstrap如何设置响应式表格？

给表格添加table类，再用带table-responsive类的容器把它包起来，就能创建响应式表格。本质上是让容器在宽度不足时出现横向滚动，避免表格把小屏页面撑破。还可以按需叠加table-striped、table-bordered等样式类，但这些只控制外观，不负责响应式行为。要创建响应式的表格，可以将表格包裹在具有table-responsive类的父元素中。这样可以使表格在小屏幕设备上水平滚动，并保持适当的显示。

下面是设置响应式表格的示例代码： 通过将表格包裹在具有table-responsive类的元素中，即可实现响应式的表格。当表格在小屏幕设备上无法适应屏幕宽度时，用户可以通过水平滚动来查看表格的内容。 请注意，响应式表格需要使用table类来设置基本的表格样式。可以根据需要添加其他Bootstrap提供的表格类，如表格样式类（table-striped、table-bordered等）和表格颜色类（table-primary、table-success等）等。

286. 使用Bootstrap创建垂直表单的基本步骤？

创建一个，把每组标签和控件放进form-group容器，再给输入框、文本域或下拉框添加form-control类。这样各表单项会按垂直方向排列，并获得Bootstrap提供的间距和控件样式。label的for应与控件的id对应，点击标签时才能定位到相应控件；需要更多字段时，继续按相同结构添加即可。 使用 Bootstrap 创建垂直表单的基本步骤如下： 创建一个元素，并添加role="form"属性，用于标识该表单的角色。 将每个表单项（标签和控件）放置在一个带有class="form-group"的元素中，这样可以获得最佳的间距和样式。 将文本输入框、文本域和下拉菜单元素添加class="form-control"类，以应用 Bootstrap 提供的样式和布局。 姓名： 邮箱： 留言： 通过以上步骤，你可以使用 Bootstrap 创建一个简单的垂直表单。你可以根据需要添加更多的表单项，并根据 Bootstrap 提供的文档进行样式和布局的调整。

287. 使用Bootstrap创建水平表单的基本步骤？

给添加form-horizontal，每个表单项用form-group包裹，再为标签设置control-label、为控件设置form-control。标签和输入区域可以配合col-sm-8这类栅格宽度，让它们在同一行内排布。简单来说，水平效果依赖表单类与栅格布局共同完成；同时要让标签的for和控件的id保持一致。 使用 Bootstrap 创建水平表单的基本步骤如下：

创建一个元素，并添加class="form-horizontal"类，以便应用水平表单的样式。

将每个表单项（标签和控件）放置在一个带有class="form-group"的元素中，以获得最佳的间距和样式。

在标签元素中添加class="control-label"类，以标识其为表单项的标签。 姓名： 邮箱：

通过以上步骤，你可以使用 Bootstrap

创建一个简单的水平表单。你可以根据需要添加更多的表单项，并根据 Bootstrap 提供的文档进行样式和布局的调整。

288. 使用Bootstrap如何创建表单控件的帮助文本？

把帮助说明放在对应表单控件的form-group内，并给承载文本的或添加help-block类。因为帮助文本和输入控件位于同一组中，用户能直观看出这段提示针对哪个字段，Bootstrap也会应用相应样式。通常将它紧

跟在控件后面；如果默认外观不满足页面需求，再基于该类补充自定义样式。要在 Bootstrap 中创建表单控件的帮助文本，可以按照以下步骤进行操作：

将帮助文本放置在与表单控件相关联的或元素内。姓名

在帮助文本的标签（通常是或）中添加class="help-block"。姓名这里是帮助文本

通过以上步骤，你可以在 Bootstrap 的表单控件中添加帮助文本。帮助文本将根据 Bootstrap 的样式进行显示，并提供相关的提示或指导信息。你可以根据需要自定义帮助文本的样式。

289. 使用Bootstrap激活或禁用按钮要如何操作？

激活按钮给元素加active类，禁用原生按钮则加disabled属性。active主要呈现按下状态，通常还会配合aria-pressed="true"表达语义。链接没有原生disabled属性，需要结合disabled类、aria-disabled="true"、tabindex="-1"和事件拦截。只把按钮视觉变灰并不能真正阻止点击，也不利于读屏器识别。要在 Bootstrap 中激活或禁用按钮，可以按照以下步骤进行操作：

激活按钮：给按钮添加.active的class。

禁用按钮：给按钮添加disabled="disabled"的属性。禁用按钮

通过以上步骤，你可以在 Bootstrap 中激活或禁用按钮。激活按钮将应用活动状态的样式，禁用按钮将禁用按钮的交互性并应用禁用状态的样式。

290. Bootstrap有哪些关于img的class？

图片常用类可以归为三类：响应式用img-fluid，缩略图用img-thumbnail，圆角或圆形用rounded、rounded-circle。在Bootstrap 3中，对应名称是img-responsive、img-rounded和img-circle。这些类负责缩放或外观，不负责预留图片空间。我一般还会设置width、height或aspect-ratio，避免图片加载后顶动布局。在 Bootstrap 中，有以下关于元素的类（class）可用：

.img-rounded：为图片添加圆角效果

.img-circle：将图片呈现为圆形。

.img-thumbnail：为图片添加缩略图样式。

.img-responsive：使图片具有响应式特性，可以根据父元素的大小自动调整图片的尺寸。

通过使用这些类，可以为图片添加不同的样式和功能，以满足项目的需求。

291. Bootstrap中有关元素浮动及清除浮动的class？

Bootstrap 5用float-start、float-end和float-none控制浮动，用clearfix清除浮动。响应式场景可以写成float-md-end，而旧版本常见的是pull-left、pull-right或float-left、float-right。clearfix一般加在父元素上，用来处理子元素浮动造成的高度塌陷。现代布局更常用Flex或Grid，浮动类主要留给特定排版场景。在 Bootstrap 中，有以下关于元素浮动及清除浮动的类（class）可用：

.pull-left：将元素浮动到左边。

.pull-right：将元素浮动到右边。

.clearfix：清除浮动，用于解决浮动元素导致的父元素高度塌陷的问题。一般在包含浮动元素的父元素上应用此类。

左浮动元素右浮动元素

通过使用这些类，可以实现元素的浮动和清除浮动的效果，使布局更加灵活和符合设计要求。

292. 除了屏幕阅读器外，其他设备上隐藏元素的class？

Bootstrap 5使用visually-hidden，可让元素在视觉设备上隐藏，但仍供屏幕阅读器读取。在Bootstrap 3/4中，这个类叫sr-only。它本质上通过定位、裁剪和极小尺寸隐藏内容，而不是使用display:none，所以不会离开无障碍树。需要键盘聚焦时显示可用visually-hidden-focusable，例如“跳到主内容”链接；若视觉和读屏都要隐藏，则用d-none。

除了屏幕阅读器外，可以使用.sr-only类来隐藏元素，这个类可以在其他设备上隐藏元素，但在屏幕阅读器中仍然可见。这对于提供辅助功能和可访问性非常有用。

例如，可以将.sr-only类应用于某个元素，以隐藏其内容：这段文本在其他设备上隐藏 这样，在常规的可视设备上，这段文本是隐藏的，而在屏幕阅读器中，用户仍然可以访问和阅读这段文本的内容。

293. Bootstrap如何制作下拉菜单？

用外层dropdown包住触发按钮和dropdown-menu，再通过按钮的data-toggle="dropdown"激活下拉行为。按钮通常配合dropdown-toggle、唯一id和aria-expanded，菜单则用aria-labelledby关联按钮。菜单项可设置role="presentation"，链接使用role="menuitem"和tabindex="-1"，标题使用dropdown-header。如果使用Bootstrap5，需要把属性改成带bs前缀的data-bs-toggle。

将下拉菜单包裹在class="dropdown"的中；在触发下拉菜单的按钮中添加：class="btn dropdown-toggle" id="dropdownMenu1" data-toggle="dropdown"在包裹下拉菜单的ul中添加：class="dropdown-menu" role="menu" aria-labelledby="dropdownMenu1"在下拉菜单的列表项中添加：role="presentation"。其中，下拉菜单的标题要添加class="dropdown-header"，选项部分要添加tabindex="-1"。

正确，您提供的步骤是制作下拉菜单的基本步骤。下面是一个示例代码：

下拉菜单选项1 选项2选项3标题选项4 这是一个简单的下拉菜单示例，当点击按钮时，下拉菜单会展开显示选项。您可以根据自己的需求修改按钮文本、菜单项内容和样式。

294. Bootstrap如何制作按钮组？以及水平按钮组和垂直按钮组的优先级？

水平按钮组用btn-group包裹按钮，垂直按钮组则使用btn-group-vertical；两者同时出现时，btn-group的样式优先，按钮会水平排列。本质上，这两个类分别控制按钮的横向和纵向布局。实际使用时应根据界面场景只选择其中一个，避免同时声明造成样式含义冲突。按钮组容器还可以补上role="group"和aria-label，让辅助技术知道这些按钮属于同一组。

制作按钮组的基本步骤如下：

使用class="btn-group"的包裹按钮组。这将把一组按钮组合在一起。

按钮1按钮2按钮3

如果想要创建垂直按钮组，可以使用class="btn-group-vertical"。按钮1按钮2按钮3 需要注意的是，btn-group的优先级高于btn-group-vertical的优先级。因此，如果使用了btn-group的包裹按钮组，并且同时使用了btn-group-vertical的，则以btn-group的样式为准，按钮组将水平显示。

295. Bootstrap如何设置按钮的下拉菜单？

把普通按钮、下拉触发按钮和菜单一起放进.btn-group，就能组成按钮下拉菜单。触发按钮需要添加.dropdown-toggle和data-toggle="dropdown"，点击后Bootstrap才会切换菜单的显示状态。菜单使用.dropdown-menu，内部再放具体的菜单项，同时通过aria-haspopup、aria-expanded和sr-only补充无障碍信息。需要保留主按钮独立操作时，可以采用这种拆分式结构，让旁边的箭头按钮只负责展开菜单。

要设置按钮的下拉菜单，可以将按钮和下拉菜单放置在一个.btn-group中。下面是示例代码：

按钮下拉菜单菜单项1菜单项2菜单项3 在上面的代码中，.btn-group包含了一个按钮和一个带有.dropdown-toggle类的下拉菜单触发器按钮。通过添加.dropdown-toggle类和相关的data-toggle属性，可以实现按钮点击时展示下拉菜单。下拉菜单则是使用.dropdown-menu类定义的。

296. Bootstrap中的输入框组如何制作？

用一个带input-group类的div包住输入框，再把带input-group-addon类的span放在输入框前面或后面，就能组成输入框组。输入框本身通常使用form-control，这样前后缀和输入区域会按Bootstrap的样式结合起来。比如在用户名输入框前放一个@，可以直观提示用户输入内容的含义。需要更丰富的交互时，也可以按场景加入按钮或下拉菜单。要制作Bootstrap中的输入框组，可以按照以下步骤进行操作：创建一个带有class="input-group"的，将其作为输入框组的容器。在中，使用标签并添加class="input-group-addon"，在其中放置额外的内容，例如前缀或后缀元素。将标签放在元素的前面或后面，以实现前缀或后缀元素与输入框的结合。以下是一个示例代码：@ 在上面的代码中，元素具有class="input-group"，元素具有class="input-group-addon"，并包含了一个前缀元素@。输入框使用标签，并添加了class="form-control"

用于样式设置。

你还可以根据需要添加其他的输入框组元素，如按钮、下拉菜单等，以创建更丰富的输入框组。

297. Bootstrap中的导航都有哪些？

Bootstrap常用导航包括标签页nav-tabs、胶囊导航nav-pills、导航栏navbar和面包屑breadcrumb。标签页适合切换同级内容，胶囊导航更像选项分类，而导航栏负责承载网站的主要入口。页面层级较深时用面包屑，列表内容较多时还可以搭配pagination。实际选择要看信息关系，不建议为了样式把不同用途的导航混在一起。

Bootstrap提供了多种导航组件，以下是其中几种常用的导航：

标签页导航（Tabs）：使用class="nav nav-tabs"创建标签页导航，用于在不同的内容页之间进行切换。

胶囊式标签页导航（Pills）：使用class="nav nav-pills"创建胶囊式标签页导航，通常用于表示不同的选项或分类。

导航栏（Navbar）：使用class="navbar navbar-default"和role="navigation"创建导航栏，用于显示网站的主要导航链接。

面包屑导航（Breadcrumb）：使用class="breadcrumb"创建面包屑导航，用于显示当前页面在网站结构中的位置。

除了上述导航组件外，Bootstrap还提供了其他导航相关的组件，如导航工具栏（Navbar Toolbar）、下拉菜单（Dropdown）、分页导航（Pagination）等，可以根据具体的需求选择合适的导航组件来构建网站的导航部分。

以下是几个常见导航组件的示例代码：

标签页导航（Tabs）：`HomeProfileMessages`

胶囊式标签页导航（Pills）：

`HomeProfileMessages`

导航栏（Navbar）：

`LogoHomeAboutServicesContact`

面包屑导航（Breadcrumb）：`HomeCategoryCurrent Page`

这些示例代码仅为演示Bootstrap导航组件的基本结构，具体样式和功能可以根据需要进行进一步定制和扩展。

298. Bootstrap中设置分页的class？

分页使用pagination，通常写在上；翻页式导航可以使用pager。pagination适合展示具体页码，让用户直接跳到目标页；pager只提供上一页和下一页，更适合线性浏览。具体项目里还可以基于这些类调整布局和样式。如果需要完整页码导航，我一般优先选pagination。

在Bootstrap中，可以使用以下class来设置分页和翻页样式：

默认分页样式：class="pagination"

`Previous123Next`

默认翻页样式：class="pager"

`PreviousNext`

这些class可以帮助你创建具有分页和翻页功能的导航组件，你可以根据需要进行样式和布局的调整。

299. Bootstrap中显示标签的class？

显示标签可以使用label，并搭配label-primary等状态类改变外观。单独写会得到默认标签，再增加颜色类就能表达主要、提示等不同语义。它适合放在标题或文本旁，用来标记分类和状态。样式可以继续定制，但最好让颜色与标签含义保持一致，避免只靠视觉装饰。

在Bootstrap中，可以使用class="label"来显示标签样式。以下是一个示例代码：

`Default`

上述代码将创建一个默认样式的标签，你可以根据需要添加更多的class来自定义标签的样式。例如，你可以使用class="label label-primary"来创建一个带有主要颜色的标签。

`Primary`

通过添加不同的class，你可以改变标签的颜色、背景色和其他样式，以适应你的设计需求。

300. Bootstrap中如何制作徽章？

在元素中加入badge类即可制作徽章，例如26。徽章本质上是一段醒目的提示信息，常用来显示消息数量或待处理计数。需要区分不同状态时，可以再搭配badge-primary等样式类调整背景色和文字颜色。它适合做简短提示，不宜放入过长内容，否则会削弱徽章的识别效果。

在Bootstrap中，可以使用class="badge"来创建徽章。徽章用于在元素上显示一些提示信息或计数。

以下是一个示例代码： 26 上述代码将创建一个默认样式的徽章，并在徽章上显示数字"26"。你可以根据需要调整徽章的样式，例如改变背景颜色、文本颜色等。

26

通过添加不同的class，你可以改变徽章的外观，使其适应你的设计需求。

301. Bootstrap中超大屏幕的作用是什么？

Bootstrap的超大屏幕通过class="jumbotron"创建，通常放在页面顶部，用来突出重要信息并吸引用户注意。它会用更大的标题字号和更多留白，让内容比普通区块更醒目。实际使用时可以放标题、副标题、说明文字和按钮，例如作为网站欢迎区或核心功能入口。背景颜色和文本样式也能按设计需求自定义，但不适合所有内容都使用，否则会削弱重点。在Bootstrap中，class="jumbotron"用于创建超大屏幕（Jumbotron）组件。超大屏幕常用于页面的顶部，用于突出显示重要的信息或引导用户的注意力。

超大屏幕组件具有以下特点：标题的字体大小较大，以引起用户的注意。提供更多的外边距，使内容在页面上更突出。可以容纳大量的文本或其他内容，如标题、副标题、按钮等。可以通过自定义样式来改变背景颜色、文本样式等。以下是一个示例代码：Welcome to our website!We provide high-quality products and excellent services.Learn more 上述代码将创建一个超大屏幕组件，其中包含一个标题、副标题和一个按钮。你可以根据需要自定义超大屏幕的内容和样式，使其符合你的设计需求。---

【微信小程序相关】

302. 微信小程序有几个文件

微信小程序常见文件可分为页面文件和全局文件，单个页面通常由wxml、wxss、js和json四类文件组成。wxml负责结构，wxss负责样式，js处理数据和交互，json保存页面配置。项目根目录还有app.js、app.json和app.wxss，分别承载全局逻辑、全局配置与全局样式。实际项目还可能包含project.config.json、sitemap.json或分包目录，所以不能简单理解为整个项目只有固定数量的文件。

微信小程序主要包含以下几个文件：

.wxml（WeiXin

Markup

Language）：用于描述小程序的页面结构，类似于HTML。在这个文件中，可以使用小程序提供的组件和自定义的组件，以及使用数据绑定和逻辑控制语句来构建页面的结构。

.wxss（WeiXin

Style

Sheets）：是一套样式语言，用于描述.wxml文件中组件的样式。类似于CSS，可以设置元素的样式、布局、动画效果等。

.js：小程序的逻辑处理文件，包含了页面的逻辑代码。可以在这个文件中定义页面的事件处理函数、数据的处理和操作等。通过JavaScript的逻辑控制，可以实现小程序的交互功能。

app.json：小程序的全局配置文件，必须存在于小程序项目中。在这个文件中，可以进行页面的注册，设置小程序的窗口样式，配置底部导航栏（tabBar），以及设置网络请求等。

app.js：小程序的全局JavaScript文件，用于监听和处理小程序的生命周期函数、声明全局变量等。可以在这个文件中编写全局的逻辑代码。

app.wxss：小程序的全局样式文件，用于设置小程序全局的样式。可以在这个文件中定义小程序的全局样式，如页面的背景色、字体样式、导航栏样式等。

以上是微信小程序中常见的文件类型，每个文件都有其特定的作用和功能，通过不同类型的文件协同工作，可以构建出完整的微信小程序应用。

303. 微信小程序怎样跟事件传值

微信小程序里，我一般用data-*把参数挂到绑定事件的元素上，再从event.currentTarget.dataset中取值。例如给列表项写data-id="{{item.id}}"，点击回调里通过event.currentTarget.dataset.id获取对应的id；元素自身的id则可以从event.currentTarget.id读取。表单提交这类事件有所不同，输入值由组件放在ev

ent.detail.value中，所以应按事件类型读取。需要注意，存在子元素时应使用currentTarget定位绑定事件的元素，避免拿到实际触发事件的子元素。在微信小程序中，可以通过以下方式实现事件传值：使用data-*属性：给HTML元素添加自定义的data-*属性来传递需要的值。例如，可以在某个元素上添加data-id属性来传递对应的ID值。在事件处理函数中，可以通过event.currentTarget.dataset来获取这些属性值。点击我Page({handleTap(event){constid=event.currentTarget.dataset.id;console.log(id);// 输出：123}})使用自定义属性：在小程序中可以给HTML元素添加自定义属性，然后在事件处理函数中通过event.currentTarget.dataset或event.currentTarget.id来获取这些属性值。点击我Page({handleTap(event){constid=event.currentTarget.dataset.id;constelementId=event.currentTarget.id;console.log(id);// 输出：123console.log(elementId);// 输出：my-element}})使用event.detail：对于一些特定的事件，例如表单的提交事件bindsubmit，可以通过event.detail来获取额外的参数值。提交Page({handleSubmit(event){const{username,password}=event.detail.value;console.log(username,password);// 输出：输入的用户名和密码}})通过以上方法，可以在微信小程序中实现事件传值，并在事件处理函数中获取传递的值，以实现更灵活的交互和数据处理。

304. 小程序的 wxss 和 css 有哪些不一样的地方？

WXSS基于CSS，但增加了小程序专用能力，同时对资源、选择器和组件样式做了限制。它提供rpx，把屏幕宽度统一为750rpx并按设备换算；导入样式时写@import "path.wxss"，而不是CSS常见的url()形式。选择器只支持常用的一部分，属性选择器和复杂伪类不能直接用，本地图片也不能作为background-image，通常要换成网络地址或base64。自定义组件默认存在样式隔离，需要通过styleIsolation调整；日常布局仍可使用大部分CSS属性，并以Flex为主。小程序的WXSS (WeiXin Style Sheets) 和CSS (Cascading Style Sheets) 在语法和功能上有一些不同之处：图片引入方式：在WXSS中，图片引入需要使用外链地址，即通过网络地址加载图片，而不能使用相对路径或本地文件路径。/* WXSS */.image{background-image:url("https://example.com/image.jpg");}缺少body元素：小程序中没有body元素，样式可以直接写在页面的组件选择器上。Hello, Mini Program!/* WXSS */.container{background-color:#f5f5f5;.text{color:#333;font-size:14px;}样式导入方式：小程序中的样式文件可以使用@import导入其他样式文件。/* WXSS */@import"common.wxss";.container{/* styles */}不支持部分CSS属性和选择器：小程序的WXSS并不完全支持所有的CSS属性和选择器，例如不支持浮动(float)和定位(position)属性，也不支持伪类选择器(:hover、:before等)和部分伪元素选择器。/* WXSS */.container{/* 不支持的属性 *//* 不支持的伪类选择器 */}需要注意的是，虽然WXSS和CSS在一些语法和功能上存在差异，但在基本的样式定义和样式属性上，它们仍然具有相似性，并可以使用类似的语法和规则进行样式的定义和控制。

305. 小程序关联微信公众号如何确定用户的唯一性

小程序关联微信公众号时，应使用UnionID作为同一用户的统一标识。OpenID只能保证用户在单个应用内唯一，而同一微信开放平台账号下，用户在小程序和公众号中的UnionID可以保持一致。小程序通过wx.login获取code，服务端换取openid和session_key，再结合用户信息中的encryptedData、iv解密得到UnionID。前提是小程序与公众号已绑定到同一微信开放平台账号，并且用户满足相应的授权或关联条件。在小程序中，如果要确定用户的唯一性并与微信公众号关联，可以使用以下步骤：在小程序中调用wx.login方法获取用户的临时登录凭证code。将获取到的code发送到后端服务器。后端服务器通过code调用微信开放平台的接口，如https://api.weixin.qq.com/sns/jscode2session，获取用户的openid和session_key。后端服务器将openid返回给小程序前端，并存储在客户端。在小程序中调用wx.getUserInfo方法获取用户信息，包括encryptedData和iv。将获取到的encryptedData和iv发送到后端服务器。后端服务器使用用户的session_key对encryptedData进行解密，获取用户的unionId。将用户的unionId与用户的其他

信息一起存储在后端服务器，用于唯一标识用户并与微信公众号关联。

需要注意的是，获取用户的unionId需要满足以下条件：小程序需要通过微信开放平台的方式进行接入，而不是独立的小程序账号。小程序和微信公众号需要在同一个微信开放平台账号下，且已经完成了关联。用户在小程序和微信公众号之间需要存在关联关系，例如用户曾经在微信公众号中授权过。通过以上步骤，可以确定用户的唯一性并与微信公众号进行关联。后续的操作中，可以根据用户的unionId进行个性化的业务逻辑处理。

306. 微信小程序与vue区别

微信小程序和Vue都是数据驱动、组件化开发，但运行机制和具体语法不同。小程序用`{{}}`、`wx:if`和`bindtap`，修改数据通常要调用`setData`；Vue则使用`v-if`、`v-on`和`v-model`，表单双向绑定更直接。两者的生命周期也不同，小程序页面常用`onLoad`、`onShow`、`onUnload`，Vue则有`created`、`mounted`、`updated`等钩子。实际迁移时不能只替换模板语法，还要重新处理事件、数据回写和平台生命周期。

微信小程序和Vue在一些方面确实存在一些差异，以下是它们之间的一些区别：

- 生命周期：微信小程序的生命周期相对简单，包括`onLoad`、`onShow`、`onReady`、`onHide`、`onUnload`等几个基本生命周期函数。而Vue拥有更为丰富的生命周期钩子函数，例如`created`、`mounted`、`updated`、`destroyed`等。
- 数据绑定语法：微信小程序使用`{{}}`来进行数据绑定，而Vue使用简洁的`v-bind`和`v-model`指令来实现数据绑定。
- 元素显示和隐藏：在微信小程序中，可以使用`wx-if`和`hidden`来控制元素的显示和隐藏；而在Vue中，可以使用`v-if`和`v-show`来实现相同的功能。
- 事件处理：微信小程序中使用`bindtap`或`catchtap`来绑定事件处理函数，而Vue使用`v-on`或`@`来绑定事件处理函数。
- 数据双向绑定：在Vue中，可以通过使用`v-model`指令实现表单元素与数据的双向绑定，而微信小程序需要手动获取表单元素的值并将其赋给对应的数据变量。

除了上述的区别之外，微信小程序和Vue在一些基本概念和思想上也存在差异，例如组件化开发的方式、数据状态管理的实现方式等。然而，它们都可以用于构建具有交互性和可复用性的前端应用程序，并且都有着广泛的应用和社区支持。 ---

【webpack相关】

307. 优化 webpack 打包体积的思路

优化webpack打包体积，核心是减少无用代码、拆分非首屏资源，并压缩最终产物。我一般先用`webpack-bundle-analyzer`找出体积来源，再通过`Tree Shaking`、按需引入和动态`import()`处理真正占空间的模块。图片、字体、JavaScript和CSS也要分别压缩，生产环境不要打入调试插件或不合适的`source-map`。大依赖可以通过`externals`配合CDN引入，但要权衡额外请求和连接开销；服务端Gzip与缓存主要减少传输和重复加载，并不会缩小原始构建产物。

优化 webpack 打包体积的思路包括：

- 提取第三方库或通过引用外部文件的方式引入第三方库：将第三方库单独打包，并通过CDN引入，减少打包体积。
- 使用代码压缩插件：例如`UglifyJsPlugin`，可以压缩JavaScript代码，减小文件体积。
- 启用服务器端的Gzip压缩：通过服务器端配置Gzip压缩，减少传输体积。
- 按需加载资源文件：使用`require.ensure`或动态导入(`import()`)的方式按需加载资源文件，避免一次性加载所有资源，优化加载速度和体积。
- 优化 devtool 中的 source-map：选择合适的 devtool 配置，确保在开发阶段能够提供足够的错误追踪信息，但不会增加过多的打包体积。
- 剥离 CSS 文件：将CSS文件单独打包，通过标签引入，利用浏览器的并行加载能力。
- 去除不必要的插件：检查 webpack 配置中的插件，移除不必要的插件或根据环境区分开发环境和生产环境的配置，避免将开发环境的调试工具

打包到生产环境中。除了上述优化思路，还可以考虑以下几点：

- 使用 Tree Shaking：通过配置 webpack，将未使用的代码在打包过程中消除，减少打包体积。
- 使用模块化引入：合理使用 ES6 模块化语法或其他模块化方案，按需引入模块，避免不必要的全局引入。
- 按需加载第三方库：对于较大的第三方库，可以考虑按需加载，而不是一次性全部引入。
- 优化图片资源：压缩图片，使用适当的图片格式，尽量减小图片体积。
- 优化字体文件：如果使用了大量的字体文件，可以考虑只引入需要的字体文件，避免全部引入。
- 使用缓存：通过配置合适的缓存策略，利用浏览器缓存机制，减少重复加载资源。

综合以上优化思路，可以有效减小 webpack 打包生成的文件体积，提升应用性能和加载速度。需要根据具体项目情况和需求，选择合适的优化策略和配置。

308. 优化 webpack 打包效率的方法

提升 webpack 打包效率，本质上是减少重复构建和不必要的解析工作。可以启用文件系统或 loader 缓存，并通过 include、exclude 和精确的搜索路径缩小处理范围，让未改动或无关文件少参与构建。开发环境再配合增量构建、热更新和较轻量的 devtool，同时关闭提取 CSS、计算文件 hash 等非必要工作。耗时的转换可以选择更高效的 loader 或多进程处理，但小项目要考虑线程调度和配置复杂度，未必越多越快。

- 使用增量构建和热更新：在开发环境下，使用增量构建和热更新功能，只重新构建修改过的模块，减少整体构建时间。
- 避免无意义的工作：在开发环境中，避免执行无意义的工作，如提取 CSS、计算文件 hash 等，以减少构建时间。
- 配置合适的 devtool：选择适当的 devtool 配置，提供足够的调试信息，但不会对构建性能产生太大影响。
- 选择合适的 loader：根据需要的资源类型选择高效的 loader，避免不必要的解析和处理过程。
- 启用 loader 缓存：对于耗时较长的 loader，如 babel-loader，可以启用缓存功能，避免重复处理同一文件。
- 采用引入方式引入第三方库：对于第三方库，可以通过直接引入的方式（如 CDN 引入）来减少打包时间。
- 提取公共代码：通过配置 webpack 的 SplitChunks 插件，提取公共代码，避免重复打包相同的代码，提高打包效率。
- 优化构建时的搜索路径：指定需要构建的目录和不需要构建的目录，减少搜索范围，加快构建速度。
- 模块化引入需要的部分：使用按需引入的方式，只引入需要的模块或组件，避免加载不必要的代码，提高构建效率。

通过以上优化措施，可以有效提升 webpack 的打包效率，减少开发和构建时间，提升开发效率和用户体验。根据具体项目需求和场景，选择适合的优化方法进行配置和调整。

309. 编写 Loader

编写 Loader，本质上就是导出一个接收源文件内容并返回转换结果的函数。例如处理文本文件时，可以把 source 反转，再拼成 module.exports 对应的 JavaScript 模块代码。然后在 webpack 的 module.rules 中用 test 匹配 .txt，并通过 loader 指定该文件的实际路径。这个例子适合说明基本机制，真实项目还要留意内容转义，避免生成无效代码。

编写一个名为 reverse-txt-loader 的 Loader，实现对文本内容进行反转处理的功能。

```
reverse-txt-loader.js
module.exports=function(source){//
  对源代码进行处理，这里是将字符串反转
  const reversedSource=source.split('').reverse().join('');//
  返回处理后的 JavaScript 代码作为模块输出
  return `module.exports = '${reversedSource}';`;
};
上述代码定义了一个函数，该函数接收一个参数 source，即原始的文本内容。在函数内部，我们将源代码进行反转处理，并将处理后的结果拼接成一个字符串，再通过 module.exports 输出为一个 JavaScript 模块。
要使用这个 Loader，需要在 webpack 配置中指定该 Loader 的路径：
webpack.config.js
module.exports={//
  ...module:{rules:[{test:/\.txt$/,use:[{loader:'./path/reverse-txt-loader'}]}]}// ...};
上述配置将该 Loader 应用于所有以 .txt 结尾的文件。在构建过程中，当遇到需要加载的 .txt 文件时，会调用 reverse-txt-loader 对文件内容进行反转处理，并将处理后的结果作为模块的输出。
```

请注意，在实际使用中，需要根据实际路径修改loader配置的路径，并将该 Loader 安装在项目中。

310. 编写plugin

编写Plugin，核心是创建一个实现apply(compiler)方法的类，并在其中订阅webpack的生命周期钩子。

比如通过compiler.hooks.emit.tap拿到compilation，就能在产物输出阶段遍历资源并添加注释。插件配置可以由构造函数接收，使用时再通过plugins数组传入实例。它能监听多个钩子，但应根据操作发生的时机选择合适钩子，避免职责混乱。编写一个自定义的 Webpack 插件需要创建一个 JavaScript 类，并在类中实现指定的生命周期方法。下面是一个简单的示例，展示如何编写一个自定义的 Webpack 插件：

```
class MyPlugin { constructor(options) { //
  // 在构造函数中可以接收插件的配置参数
  this.options = options; // Webpack 在安装插件时会自动调用
  // apply 方法，并将 compiler 对象传递进来
  apply(compiler) { // 在适当的生命周期钩子中挂载插件的功能 //
    // 示例：在
    emit
```

```
    // 生命周期钩子中添加自定义的功能
    compiler.hooks.emit.tap('MyPlugin', (compilation) => { //
      // compilation 对象包含了当前构建过程的各种信息 //
      // 可以在这里执行一些自定义的操作 //
    });
  }
}
```

示例：向输出的文件中添加自定义的注释

```
const comment = this.options.comment || 'Custom comment';
for (const asset in compilation.assets) {
  if (compilation.assets.hasOwnProperty(asset)) {
    compilation.assets[asset].source = () => {
      return `/*${comment}*/\n${compilation.assets[asset].source()};`;
    };
  }
}
```

以上是一个简单的插件示例，它在构建过程中的emit生命周期钩子中向输出的文件添加了自定义的注释。你可以根据实际需求在其他生命周期钩子中实现不同的功能。

要使用该插件，在 webpack 的配置文件中进行如下配置：

```
const MyPlugin = require('./path/to/MyPlugin');
module.exports = {
  // ...
  plugins: [new MyPlugin({ comment: 'Custom comment' })],
};
```

这样，当你运行 webpack 构建时，该插件就会被应用，并执行指定的功能。需要注意的是，Webpack 的插件机制非常灵活，可以根据实际需求编写各种各样的插件。插件可以监听多个生命周期钩子，并在每个生命周期钩子中实现自定义的功能。详细的插件开发文档可以参考 Webpack 官方文档。

311. 说一下webpack的一些plugin，怎么使用webpack对项目进行优化

我会把webpack优化分成减小产物和加快构建两类，再按实际瓶颈选择插件或配置。体积方面可用Ignore Plugin减少无用模块，通过Tree Shaking、splitChunks和动态import()删除或拆分代码。速度方面可用thread-loader并行处理，并结合缓存减少重复工作；devtool也要在调试体验和构建速度之间取舍。具体方案不能堆插件，应该根据项目场景组合使用。Webpack

提供了许多插件（Plugins）来帮助优化项目构建和性能。下面列举一些常用的插件以及它们的作用：

构建优化插件：

- ContextReplacementPlugin：用于限制某些模块的上下文，可以减少编译体积。
- Ignore Plugin：用于忽略特定的模块，减少打包体积。
- babel-plugin-import：用于按需加载和使用模块，减少打包体积。
- babel-plugin-transform-runtime：将代码中的公共部分提取到一个单独的模块中，减少打包体积。
- happypack、thread-loader：实现并行编译，加快构建速度。
- uglifyjs-webpack-plugin：通过并行压缩和缓存来加快代码压缩的速度。

性能优化插件：

- Tree-shaking：通过静态分析代码，去除未使用的代码，减少打包体积。
- Scope Hoisting：将模块之间的关系进行静态分析，减少打包后的模块数量，提升代码执行速度。
- webpack-md5-plugin：根据文件内容生成 hash，实现缓存的更新机制。
- splitChunksPlugin：根据配置将代码拆分成多个块，实现按需加载和并行加载的效果。
- import()、require.ensure：动态导入模块，实现按需加载，提升页面加载速度。

除了使用这些插件，还可以通过配置 webpack 的其他参数来进一步优化项目，例如：

- 配置devtool：选择合适的 Source Map 类型，既满足调试需求又不影响构建速度。
- 配置output：使用chunkhash或contenthash生成文件名，实现长期缓存。
- 使用cache-loader、hard-source-webpack-plugin、uglifyjs-webpack-plugin等插件开启缓存，加速再次构建。
- 使用DllWebpackPlugin和DllReference

Plugin预编译公共模块，减少重复构建时间。综合使用这些插件和优化策略，可以显著提升 webpack 项目的构建效率和性能。但是需要根据具体的项目需求和场景选择合适的插件和优化方法。

312. webpack Plugin 和 Loader 的区别

Loader负责转换单个模块的源码，Plugin则通过构建事件扩展整个webpack流程。前者通常在module.rules中匹配文件，例如把Sass转成CSS，或者把ES6转成ES5。后者监听特定生命周期，可以处理打包优化、资源管理或环境变量注入等更广泛的任务。简单来说，文件内容怎么转换交给Loader，需要改变构建行为时再用Plugin。Loader用于对模块源码进行转换，将非 JavaScript 模块转换为 JavaScript 模块，或对模块进行预处理。它描述了webpack如何处理不同类型的文件，比如将 Sass 文件转换为 CSS 文件，或将ES6代码转换为ES5代码。Loader是针对单个文件的转换操作，通过配置rules来匹配文件并指定相应的LoaderPlugin用于扩展 webpack 的功能，解决Loader无法解决的问题。Plugin可以监听webpack构建过程中的事件，并在特定的时机执行相应的操作。它可以在打包优化、资源管理、环境变量注入等方面提供额外的功能。Plugin

的功能范围更广泛，可以修改webpack的内部行为，从而实现更复杂的构建需求。总的来说，Loader是用于处理模块源码的转换工具，而Plugin则是用于扩展webpack的功能，通过监听webpack构建过程中的事件来执行相应的操作。它们各自的作用和功能不同，但都可以用于优化和定制webpack的构建过程。在配置webpack时，我们可以通过配置Loader和Plugin来满足不同的需求，并实现对模块的转换和构建过程的定制化

313. tree shaking 的原理是什么

Tree Shaking本质上是利用ESM的静态结构，在构建阶段找出并删除没有被引用的代码。因为import和export的依赖关系在编译期就能确定，打包器可以先标记无用导出，再由压缩工具完成删除。它通常要求保留ESM语法并正确配置sideEffects，否则打包器不敢贸然移除文件。像引入样式或顶层注册全局变量这类操作存在副作用，即使没有使用导出也可能需要保留。Tree shaking 的原理主要是基于静态分析的方式来实现无用代码的消除，从而减小最终打包生成的文件体积。它的工作原理可以简要概括如下：采用ES6 Module语法：Tree shaking只对ES6 Module语法进行静态分析和优化。ES6 Module的特点是可以进行静态分析，这意味着在编译阶段就能够确定模块之间的依赖关系。静态分析模块依赖：在编译过程中，通过静态分析可以确定每个模块的依赖关系，以及模块中导出的函数、变量等信息。标记未被引用的代码：在静态分析的过程中，会标记出那些未被其他模块引用的函数、变量和代码块。消除未被引用的代码：在构建过程中，根据静态分析得到的标记信息，可以对未被引用的代码进行消除。这样，在最终生成的打包文件中，未被引用的代码将不会包含在内。总结来说，Tree shaking 的核心思想是通过静态分析模块依赖关系，并标记和消除未被引用的代码。这样可以大大减小打包后的文件体积，提升应用的性能和加载速度。需要注意的是，Tree shaking 只对 ES6 Module 语法起作用，而对于 CommonJS 等其他模块系统则无法进行静态分析和优化。

314. common.js 和 es6 中模块引入的区别

CommonJS是运行时加载并导出值的拷贝，ESM则会静态解析依赖并提供实时绑定。require()是普通函数，可以按条件动态调用；静态import通常写在模块顶层，因此构建工具能提前分析依赖。模块内部的值变化时，ESM的导入方能看到更新，而CommonJS已取得值通常不会随之变化。也正因为静态结构更明确，需要做Tree Shaking时一般会优先保留ESM。CommonJS 是一种模块规范，最初被应用于Nodejs，成为 Nodejs 的模块规范。运行在浏览器端的 JavaScript 由于也缺少类似的规范，在 ES6 出来之前，前端也实现了一套相同的模块规范 (例如:AMD)，用来对前端模块进行管理。自 ES6 起，引入了一套新的ES6 Module规范，在语言标准的层面上实现了模块功能，而且实现得相当简单，有望成为浏览器和服务端通用的模块解决方案。但目前浏览器对ES6 Module兼容还不太好，我们平时在Webp

ack中使用的export和import，会经过Babel转换为CommonJS规范 CommonJS 和 ES6 Module 在模块引入的方式和特性上有一些区别，主要包括以下几个方面：

输出方式：CommonJS输出的是一个值的拷贝，而ES6 Module输出的是值的引用。在CommonJS中，模块导出的值是被复制的，即使导出模块后修改了模块内部的值，也不会影响导入模块的值。而在ES6 Module中，模块导出的值是引用关系，如果导出模块后修改了模块内部的值，会影响到导入模块的值加载时机：

CommonJS模块是运行时加载，也就是在代码执行到导入模块的位置时才会加载模块并执行。而ES6 Module是编译时输出接口，也就是在代码编译阶段就会确定模块的依赖关系，并在运行前静态地解析模块的导入和导出导出方式：CommonJS采用的是module.exports导出，可以导出任意类型的值。ES6 Module采用的是export导出，只能导出具名的变量、函数、类等，而不能直接导出任意值导入方式：CommonJS使用require()来导入模块，可以使用动态语法，允许在条件语句中使用。ES6

Module使用import来导入模块，它是静态语法，只能写在模块的顶层，不能写在条件语句中this

指向：CommonJS模块中的this指向当前模块的exports对象，而不是全局对象。ES6

Module中的this默认是undefined，在模块中直接使用this会报错

总的来说，CommonJS主要用于服务器端的模块化开发，运行时加载，更适合动态加载模块，而ES6 Module是在语言层面上实现的模块化方案，静态编译，更适合在构建时进行模块依赖的静态分析和优化。在前端开发中，通常使用打包工具（如webpack）将ES6

Module转换为CommonJS或其他模块规范，以实现在浏览器环境中的兼容性。

315. babel原理

Babel的原理是把源码解析成AST，通过插件改写这棵树，再生成兼容目标环境的新代码。解析阶段包含词法和语法分析，转换阶段则由插件遍历节点并完成语法替换，这是整个流程的核心。最后，生成器会把新的AST还原成代码，并可同时生成source

map。要注意，箭头函数这类语法能直接转译，而Promise等运行时能力通常还需要polyfill。 Babel

是一个 JavaScript 编译器。他把最新版的 javascript 编译成当下可以执行的版本，简言之，利用 babel 就可以让我们在当前的项目中随意的使用这些新最新的 es6，甚至 es7 的语法 ES6、7代码输入 ->babel进行解析 -> 得到AST（抽象语法树）->plugin用babel-traverse对AST树进行遍历转译 ->得到新的AST树->用babel-generator通过AST树生成ES5代码

它的工作流程包括解析（parse）、转换（transform）和生成（generate）三个主要步骤

解析（parse）：Babel 使用解析器（如Babylon）将输入的 JavaScript

代码解析成抽象语法树（AST）。解析器将代码分析成语法结构，并生成对应的 AST，表示代码的抽象语法结构。这个阶段包括词法分析和语法分析。词法分析将源代码转换为一个个标记（tokens）的流，而语法分析则将这个标记流转换为 AST 的形式。转换（transform）：在转换阶段，Babel

使用插件（plugins）对 AST 进行遍历和转换。插件可以对 AST 进行增删改查的操作，可以根据需求对语法进行转换、代码优化等。Babel 的插件系统非常灵活，可以根据需要自定义插件或使用现有插件来进行代码转换。生成（generate）：在生成阶段，Babel

使用生成器（如babel-generator）将经过转换的 AST 转换回字符串形式的 JavaScript 代码。生成器会深度优先遍历 AST，并根据

的节点类型生成对应的代码字符串，最终将代码字符串输出。 通过以上三个步骤，Babel

实现了将最新版本的 JavaScript 代码转换为向后兼容的代码，使得开发者可以在当前环境中使用较新的 JavaScript 特性和语法。同时，Babel

还提供了一些常用的插件和预设（presets），以便开发者快速配置和使用常见的转换规则，如转换

ES6、ES7 语法、处理模块化、转换 JSX 等。 总的来说，Babel

的原理是通过解析、转换和生成的过程，将新版本的 JavaScript

代码转换为兼容旧环境的代码，使开发者能够在当前环境中使用较新的 JavaScript 特性和语法。---

【框架相关】

316. Vue 响应式原理

Vue响应式的核心是数据劫持配合观察者模式：读取时收集依赖，修改时通知相关视图更新。Vue 2用Object.defineProperty拦截已有属性，并通过Dep保存对应的Watcher；数组则需要重写变异方法。它无法直接感知属性新增和删除，而且初始化时递归观测深层对象会有额外开销。Vue 3改用Proxy代理对象，能覆盖新增、删除和数组索引等操作，并在访问嵌套对象时再继续代理。

整体思路是数据劫持+观察者模式 对象内部通过defineReactive方法，使用Object.defineProperty将属性进行劫持（只会劫持已经存在的属性），数组则是通过重写数组方法来实现。当页面使用对应属性时，每个属性都拥有自己的dep属性，存放他所依赖的watcher（依赖收集），当属性变化后会通知自己对应的watcher去更新（派发更新）。

```
class Observer{//
  观测值 constructor(value){this.walk(value);}walk(data){// 对象上的所有属性依次进行观测
    let keys=Object.keys(data);for(let i=0;i<keys.length;i++){
      性能会受影响 Object.defineProperty(data,key,{get(){console.log("获取值");
      //需要做依赖收集过程 这里代码没写出来 return value;},set(newValue){if(newValue===value)return;
      console.log("设置值");//需要做派发更新过程
      这里代码没写出来 value=newValue;});});
    export function observe(value){//
      如果传过来的是对象或者数组 进行属性劫持 if(Object.prototype.toString.call(value)=== "[object Object]" || Array.isArray(value)){return new Observer(value);}}
```

317. Vue nextTick 原理

Vue的nextTick会把回调延迟到本轮异步DOM更新完成后执行。数据变化时，渲染任务 and 用户回调会进入队列，同一轮只触发一次异步刷新，避免重复更新。它优先使用Promise.then，再依次降级到MutationObserver、setImmediate和setTimeout。所以修改数据后要读取新DOM，可以调用this.\$nextTick()，也可以await nextTick()。nextTick中的回调是在下次DOM更新循环结束之后执行的延迟回调。在修改数据之后立即使用这个方法，获取更新后的DOM。主要思路就是采用微任务优先的方式调用异步方法去执行nextTick包装的方法

```
let callbacks=[];let pending=false;function flushCallbacks(){pending=false;//把标志还原为false//
  依次执行回调 for(let i=0;i<callbacks.length;i++){callbacks[i].then(flushCallbacks);}
  }else if(typeof MutationObserver !== "undefined"){// MutationObserver 主要是监听dom变化 也是一个异步方法
    let counter=1;const observer=new MutationObserver(flushCallbacks);const textNode=document.createTextNode(String(counter));observer.observe(textNode,{characterData:true,});timerFunc={()=>{counter=(counter+1)%2;textNode.data=String(counter);}};
  }else if(typeof setImmediate !== "undefined"){// 如果前面都不支持
    判断 setImmediate timerFunc={()=>{setImmediate(flushCallbacks);}};
  }else{// 最后降级采用 setTimeout
    timerFunc={()=>{setTimeout(flushCallbacks,0);}};
  }
  export function nextTick(cb){// 除了渲染 watcher 还有用户自己手动调用的nextTick 一起被收集到数组 callbacks.push(cb);if(!pending){//
    如果多次调用nextTick 只会执行一次异步
    等异步队列清空之后再吧标志变为false pending=true;timerFunc();}}
```

318. Vue diff 原理

Vue diff的作用是比较新旧虚拟DOM，尽量用较少的真实DOM操作完成视图更新。它主要比较同层节点，节点类型不同就替换，并通过key判断列表节点能否复用。Vue 2使用双端指针匹配列表头尾，Vue 3会处理相同首尾，并借助最长递增子序列减少中间节点移动。列表可能插入或重排时，不建议用索引作为key，否则容易出现错误复用。

319. 路由原理 history 和 hash 两种路由方式的特点

hash和history都能在不刷新页面的情况下切换前端视图，区别主要在地址形式和服务端要求。hash通过location.hash和hashchange工作，#后的内容不会进入HTTP请求，因此刷新通常不会导致后端404。history借助pushState()、replaceState()修改地址，链接更美观，但刷新时服务器会收到真实路径请求。使用history模式时，我一般会让后端把未知路由回退到index.html。

hash 模式
location.hash的值实际就是URL中#后面的东西

它的特点在于：hash虽然出现URL中，但不会被包含在HTTP请求中，对后端完全没有影响，因此改变 hash 不会重新加载页面可以为 hash 的改变添加监听事件
window.addEventListener("hashchange",funcRef,false); 每一次改变hash (window.location.hash) 都会在浏览器的访问历史中增加一个记录利用hash的以上特点，就可以来实现前端路由“更新视图但不重新请求页面”的功能了。特点：兼容性好但是不美观 history 模式 利用了HTML5 History Interface中新增的pushState()和replaceState()方法 这两个方法应用于浏览器的历史记录站，在当前已有的back、forward、go的基础之上，它们提供了对历史记录进行修改的功能。这两个方法有个共同的特点：当调用他们修改浏览器历史记录栈后，虽然当前URL改变了，但浏览器不会刷新页面，这就为单页应用前端路由“更新视图但不重新请求页面”提供了基础特点：虽然美观，但是刷新会出现404需要后端进行配置 ---

【编程题相关】

320. 写一个通用的事件侦听器函数

通用事件侦听器要统一封装事件绑定、移除、阻止传播、取消默认行为和获取事件目标。现代浏览器可以直接使用addEventListener和removeEventListener，老环境则兼容attachEvent或on事件名。需要注意，移除监听时必须拿到绑定时的同一个处理函数，否则无法正确解绑。兼容旧版IE时，还要分别处理cancelBubble、returnValue和srcElement。

event(事件)工具集，来源：github.com/markyun/markyun.Event={//

```
视能力分别使用dom0||dom2||IE方式 来绑定事件// 参数： 操作的元素,事件名称,事件处理程序
addEvent:function(element,type,handler){if(element.addEventListener){//事件类型、需要执行的函数、是否捕捉
element.addEventListener(type,handler,false);}elseif(element.attachEvent){element.attachEvent('on'+type,function(){handler.call(element);});}else{element['on'+type]=handler;}}// 移除事件
removeEvent:function(element,type,handler){if(element.removeEventListener){element.removeEventListener(type,handler,false);}elseif(element.detachEvent){element.detachEvent('on'+type,handler);}else{element['on'+type]=null;}}// 阻止事件（主要是事件冒泡，因为IE不支持事件捕获）stopPropagation:function(ev){if(ev.stopPropagation){ev.stopPropagation();}else{ev.cancelBubble=true;}}// 取消事件的默认行为preventDefault:function(event){if(event.preventDefault){event.preventDefault();}else{event.returnValue=false;}}//
```

获取事件目标getTarget:function(event){returnevent.target||event.srcElement;}

321. 如何判断一个对象是否为数组

判断对象是不是数组，优先使用Array.isArray(arg)，必要时再用Object.prototype.toString.call(arg)兜底。前者语义直接，后者可以通过结果是否为[object Array]判断类型。两种方式组合后，数组返回true，普通对象和字符串都会返回false。不要依赖typeof，因为数组得到的也是object，无法和普通对象区分。function isArray(arg){return Array.isArray(arg)|| (typeof arg==='object' && Object.prototype.toString.call(arg) === '[object Array]');} 首先使用Array.isArray方法判断arg是否为数组。如果是数组，则直接返回true。否则，执行后面的类型判断逻辑。

这样，你可以使用isArray函数来判断一个对象是否为数组。例如：
trueconsole.log(isArray([])); // true
falseconsole.log(isArray({})); // false
falseconsole.log(isArray('')); // false

322. 冒泡排序

冒泡排序就是反复比较相邻元素，如果前一个更大就交换，直到数组从小到大排列。每完成一轮，当前未排序部分中较大的值就会逐步移动到后面，所以内层循环的比较范围可以不断缩小。我一般会加一个swapped标志，如果某一轮完全没有交换，说明数组已经有序，可以直接结束。对于[3, 1, 4, 6, 5, 7, 2]，最终结果是[1, 2, 3, 4, 5, 6, 7]。它通过比较相邻的两个数，如果后一个数比前一个数小，则交换它们的位置。重复这个过程，直到所有的数都按照从小到大的顺序排列。

代码中使用了两层嵌套的循环。外层循环控制比较的轮数，内层循环用于比较相邻的两个数并交换位置。下面代码添加了一个标志位来判断是否发生了交换，如果某一轮比较中没有发生交换，说明数组已经有序，可以提前结束循环。 var arr=[3,1,4,6,5,7,2];function bubbleSort(arr){var len=arr.length;for(var i=0;i<len-1;i++){var temp=arr[i];arr[i]=arr[i+1];arr[i+1]=temp;swapped=true; // 发生了交换}}if(!swapped){break; // 没有发生交换，提前结束循环}}return arr;}console.log(bubbleSort(arr)); 我们在内层循环中添加了一个标志位swapped，初始值为false。如果发生了交换，将swapped设置为true。在每一轮外层循环结束后，检查swapped的值，如果为false，说明数组已经有序，提前结束循环。

这样可以避免在已经排序完成的数组上进行不必要的比较，提高了冒泡排序的效率。 运行结果应为[1, 2, 3, 4, 5, 6, 7]。

323. 快速排序

快速排序本质上是选一个基准值，把较小和较大的元素分到两边，再递归处理两部分。示例中取中间元素作为基准，并用左右两个数组保存比较结果，最后通过concat合并。它采用分治方式，通常时间复杂度写作O(nlogn)，适合处理较多数据。需要注意，代码里的splice会改动传入数组，而且建立左右数组会占用额外空间。

采用二分法，取出中间数，数组每次和中间数比较，小的放到左边，大的放到右边。快速排序的思想很简单，整个排序过程只需要三步： 在数据集之中，找一个基准点建立两个数组，分别存储左边和右边的数组利用递归进行下次比较快速排序的时间复杂度为O(nlogn)，是一种高效的排序算法。 var arr=[3,1,4,6,5,7,2];function quickSort(arr){if(arr.length==0){return [];} // 返回空数组var cIndex=Math.floor(arr.length/2);var c=arr.splice(cIndex,1);var l=[];var r=[];for(var i=0;i<arr.length;i++){if(arr[i]<c){l.push(arr[i]);}else{r.push(arr[i]);}}return quickSort(l).concat(c,quickSort(r));}console.log(quickSort(arr));

324. 编写一个方法 求一个字符串的字节长度

在题目约定英文字符占一个字节、中文字符占两个字节时，可以遍历字符串并根据字符编码累加长度。先把字节数设为str.length，遇到charCodeAt(i) > 255的字符就再加一。比如你好,as中两个中文字符各按两个字节计算，其余字符各按一个字节计算。这个方法依赖题目的简化假设，不代表所有实际字符编码下的真实字节数。假设：一个英文字符占用一个字节，一个中文字符占用两个字节

```
functionGetBytes(str){varlen=str.length;varbytes=len;for(vari=0;i<255)bytes++;returnbytes;}alert(GetBytes("你好,as"));
```

325. bind的用法，以及如何实现bind的函数和需要注意的点

bind用来创建一个绑定了指定this的新函数，它不会像call、apply那样立即执行。手写时可以用闭包保存原函数、上下文和预置参数，调用新函数时再通过apply合并并传入后续参数。简单实现只处理this，完整一些还要支持额外参数。实际开发不建议直接改写Function.prototype.bind，因为修改内置对象原型可能与其他代码冲突。bind的作用与call和apply相同，区别是call和apply是立即调用函数，而bind是返回了一个函数，需要调用的时候再执行。

一个简单的bind函数实现如下

```
Function.prototype.bind=function(ctx){varfn=this;returnfunction(){fn.apply(ctx,arguments);}}; bind方法用于创建一个新函数，并将其中的this值绑定到指定的对象。与call和apply不同，bind方法不会立即调用函数，而是返回一个绑定了指定this值的新函数，供以后调用。它将原函数保存在fn变量中，然后返回了一个匿名函数。当新函数被调用时，它会使用fn.apply来设置函数的上下文（this值）为传入的ctx对象，并将参数通过arguments对象传递进去。需要注意的是，bind方法还可以接受额外的参数，这些参数会在调用新函数时作为参数传递进去。修改代码，使其支持传递额外参数的实现如下：
```

```
Function.prototype.bind=function(ctx){varfn=this;varargs=Array.prototype.slice.call(arguments,1);// 获取额外参数returnfunction(){varcombinedArgs=args.concat(Array.prototype.slice.call(arguments));//
```

```
合并额外参数和新函数调用时的参数fn.apply(ctx,combinedArgs);}}; 另外，需要注意的是，使用原型链修改内置对象的方法可能会与其他代码发生冲突或不兼容。因此，在实际开发中，最好避免修改内置对象的原型方法，以免引起意想不到的问题。
```

326. 实现一个函数clone

实现clone的核心是区分基本类型、数组和对象，并递归复制引用类型中的每个成员。基本类型直接返回；数组创建新数组后逐项克隆，对象则创建新对象并遍历属性继续克隆。本质上既考察值与引用的区别，也考察类型判断和递归设计。还要留意循环引用，否则递归可能无法结束；复杂场景可以选择lodash、underscore等工具库。

可以对JavaScript中的5种主要的数据类型,包括Number、String、Object、Array、Boolean) 进行值复考察点1：对于基本数据类型和引用数据类型在内存中存放的是值还是指针这一区别是否清楚考察点2：是否知道如何判断一个变量是什么类型的考察点3：递归算法的设计

```
// 方法一：Object.prototype.clone=function(){varo=this.constructor===Array?[]:{};for(vareinthis){o[e]=typeofthis[e]=== "object"?this[e].clone():this[e];}returno;} //方法二：/** * 克隆一个对象 * @param Obj * @returns */functionclone(Obj){varbuf;if(ObjinstanceofArray){buf=[];//创建一个空的数组vari=Obj.length;while(i--){buf[i]=clone(Obj[i]);}returnbuf;}elseif(ObjinstanceofObject){buf={};//创建一个空对象for(varkinObj){//为这个对象添加新的属性buf[k]=clone(Obj[k]);}returnbuf;}else{//普通变量直接赋值returnObj;}}
```

这里提供了两种方法来实现对象的克隆（clone）。方法一使用了原型链的方式，在Object.prototype上添加了一个名为clone的方法。该方法可以克隆一个对象，对于数组类型则创建一个空数组，对于对象类型则创建一个空对象，并递归地复制属性值。方法二是一个独立的函数clone，通过判断对象的类型来进行不同的处理。如果是数组类型，则创建一个空数组，并递归地克隆数组的每个元素；如果是对象类型，则创建一个空对象，并递归地克隆对象的每个属性；对于其他类型的变量，直接返回该变量。这两种方法都使用了递归算法，通过遍历对象的属性，并根据属性的类型进行复制操作，从而实现对象的克隆。需要注意的是，在使用递归算法时，要注意处理循环引用的情况，以避免进入无限循环。在实际开发中，可以根据

需要选择适合的方法来实现对象的克隆。同时，还可以使用现代的深拷贝工具库，如lodash、underscore等，来实现更复杂的对象克隆操作。

327. 下面这个ul，如何点击每一列的时候alert其index

可以在遍历每个li时保存当前索引，再在点击回调中通过alert输出它。一种做法是把索引挂到元素的index属性上，回调里读取this.index。如果不想给元素增加属性，也可以用立即执行函数把当前的i保存在闭包中。这里不能让所有回调直接共享循环变量，否则点击时拿到的可能都是循环结束后的值。考察闭包这是第一条这是第二条这是第三条 // 方法一：var lis=document.getElementById('2223').getElementsByTagName('li');for(var i=0;i<3;i++){lis[i].index=i;lis[i].onclick=function(){alert(this.index);}}//方法二：var lis=document.getElementById('2223').getElementsByTagName('li');for(var i=0;i<3;i++){(function(a){return function(){alert(a);}})(i);lis[i].onclick=(function(a){return function(){alert(a);}})(i);}

328. 定义一个log方法，让它可以代理console.log的方法

可以定义一个log函数，在内部通过console.log.apply(console, arguments)代理原来的输出。如果只写console.log(msg)，它只能方便地接收一个参数，不能完整模拟console.log的多参数调用。使用apply后，传给log的所有实参都会原样转交，并把调用上下文设为console。因此log("hello", 123)这类调用也能按预期输出。 // 可行的方法一：function log(msg){console.log(msg);}log("hello world");// hello world! 如果要传入多个参数呢？显然上面的方法不能满足要求，所以更好的方法是：function log(){console.log.apply(console,arguments);};

329. 输出今天的日期

可以用Date获取年月日，补齐两位后拼成YYYY-MM-DD。其中年份取getFullYear()，月份取getMonth()+1，日期取getDate()；月份必须加一，因为它从0开始计数。月或日小于10时在前面补0，再用短横线连接。也可以用toLocaleDateString()配合格式选项，但输出形式会受到地区参数影响。

以YYYY-MM-DD的方式，比如今天是2014年9月26日，则输出2014-09-26 var d=new Date();// 获取年，getFullYear()返回4位的数字 var year=d.getFullYear();// 获取月，月份比较特殊，0是1月，11是12月 var month=d.getMonth()+1;// 变成两位 month=month<10?'0'+month:month;// 获取日 var day=d.getDate();day=day<10?'0'+day:day;alert(year+'-'+month+'-'+day);

除了上述代码中使用Date对象的方法外，还有其他方式可以获取今天的日期并输出。一种常见的方式是使用toLocaleDateString()方法，该方法可以返回表示日期的字符串。可以通过传递适当的选项来指定所需的日期格式。以下是使用toLocaleDateString()方法获取今天的日期的示例代码： var today=new Date();var options={year:'numeric',month:'2-digit',day:'2-digit'};var formattedDate=today.toLocaleDateString('en-US',options);console.log(formattedDate);

在上述代码中，首先创建一个Date对象表示今天的日期。

然后，定义一个选项对象options，其中指定了年份、月份和日期的格式。最后，使用toLocaleDateString()方法将日期对象转换为指定格式的字符串，并将其赋值给formattedDate变量。

通过console.log()函数输出formattedDate，即可得到以YYYY-MM-DD的格式表示的今天的日期。

这种方法的优点是可以根据需求更灵活地定制日期的格式，适用于不同的地区和语言设置。

330. 用js实现随机选取10-100之间的10个数字，存入一个数组，并排序

先生成10到100之间的随机整数，循环写入数组10次，再调用sort((a, b) => a - b)排序。随机数公式可写成Math.floor(Math.random() * (100 - 10 + 1)) + 10，这里加1是为了让100也有

机会被取到。排序时要传数值比较函数，否则sort()默认按字符串规则比较。题目没有要求去重，因此生成的数字允许重复。 `var iArray=[];function getRandom(istart,iend){var iChoice=istart-iend+1;return Math.floor(Math.random()*iChoice+istart);}for(var i=0;i<10;i++){iArray.push(getRandom(10,100));}iArray.sort();`

331. 写一段JS程序提取URL中的各个GET参数

可以截取URL中问号后的查询串，按&拆成参数，再按第一个=组装为键值对象。例如a=1&b=2&c=&e应得到a: '1'、b: '2'、c: ''，而没有等号的e对应undefined。实际写法中不要反复调用split('=')，保存拆分结果会更清楚。若参数值可能包含编码字符，还应先做相应的解码处理。 有这样一个URL：http://item.taobao.com/item.htm?a=1&b=2&c=&d=xxx&e，请写一段JS程序提取URL中的各个GET参数(参数名和参数个数不确定)，将其按key-value形式返回到一个json结构中，如{a:'1', b:'2', c:'', d:'xxx', e:undefined} `function serializeUrl(url){var result={};url=url.split("?")[1];var map=url.split("&");for(var i=0,len=map.length;i<len;i++){result[map[i].split("=")[0]]=map[i].split("=")[1];}return result;}`

332. 写一个function，清除字符串前后的空格

直接返回str.trim()就能清除字符串首尾的空白字符。如果需要兼容不支持该方法的环境，可以给String.prototype.trim增加降级实现，用/^\\s+/和/\\s+\$/分别替换开头、结尾的空白。本质上只处理两端，因此字符串中间原有的空格不会被改动。使用前也要确认传入的是字符串，避免在null或undefined上直接调用。使用自带接口trim()，考虑兼容性：`if(!String.prototype.trim){String.prototype.trim=function(){return this.replace(/^\\s+/, "").replace(/\\s+$/, "");}} // test the function var str=" \\t\\n test string ".trim();alert(str=="test string");// alerts "true"`

333. 实现每隔一秒钟输出1,2,3...数字

可以在循环里用立即执行函数保存当前序号，再通过setTimeout按序延迟输出。每次循环把i传给参数j，形成独立闭包，避免回调执行时拿到循环结束后的i。延迟时间设为j * 1000毫秒，并输出j + 1，就能依次得到1、2、3...。这里第一次会立即输出；如果要求一秒后才输出1，延迟应改为(j + 1) * 1000。 `for(var i=0;i<10;i++){(function(j){setTimeout(function(){console.log(j+1)},j*1000)})(i)}` 在循环中，立即执行函数被用作一个闭包，用于保存每次循环中的i的值。这是为了避免在setTimeout函数中使用的回调函数在执行时捕获到的是循环结束后的i的值。 setTimeout函数用于设置一个定时器，它接受两个参数：回调函数和延迟时间（以毫秒为单位）。在每次循环中，通过将j*1000作为延迟时间，实现每隔一秒钟输出数字的效果。回调函数输出的数字为j+1，因为j从0开始。 通过这种方式，可以确保每隔一秒钟输出1, 2, 3...的数字。每个数字的输出时间间隔为一秒。

334. 实现一个函数，判断输入是不是回文字符串

先确认输入是字符串，再比较原字符串与反转后的字符串是否相等，就能判断它是不是回文。可以用split("")拆成字符数组，调用reverse()反转，最后通过join("")重新拼接。两边完全一致时返回true，否则返回false。对于非字符串输入直接返回false，避免数组方法调用出错。`function run(input){if(typeof input !== 'string')return false;return input.split("").reverse().join("")===input;}`

335. 数组扁平化处理

数组扁平化可以递归遍历嵌套元素，把所有内容合并到一个一维数组中。常见写法是用reduce()累积结果，遇到数组就继续调用flatten()，普通元素则直接通过concat()合并。也可以配合扩展运算符反复展开，只要数组中仍存在子数组就继续处理。递归写法更直观，但嵌套层级较多时会产生更多函数调用。 实现一

个flatten方法，使得输入一个数组，该数组里面的元素也可以是数组，该方法会输出一个扁平化的数组

```
function flatten(arr){return arr.reduce(function(prev,item){return prev.concat(Array.isArray(item)?flatten(item):item);},[]);}
```

除了使用reduce方法，还可以使用递归和ES6的扩展运算符等方式来实现数组的扁平化处理。递归方式：

```
function flatten(arr){var result=[];arr.forEach(function(item){if(Array.isArray(item)){result=result.concat(flatten(item));}else{result.push(item);}});return result;}
```

使用ES6的扩展运算符：

```
function flatten(arr){while(arr.some(Array.isArray)){arr=[];arr.concat(...arr);}return arr;}
```

这些方法都可以将多层嵌套的数组扁平化成一个一维数组。使用递归方法时，通过遍历数组的每个元素，如果元素是数组，则递归调用扁平化函数；如果元素不是数组，则直接添加到结果数组中。使用ES6的扩展运算符时，通过不断地展开数组中的每个元素，直到所有元素都不再是数组为止。

336. 实现一个函数clone，可以对JavaScript中的5种主要的数据类型（包括Number、String、Object、Array、Boolean）进行值复制

可以根据当前值的构造类型创建数组或对象，再逐项复制属性，遇到对象时递归调用clone()。Number、String和Boolean这类非对象属性可以直接赋值，因为复制的是它们的值。对于Array，通过this.constructor === Array创建新数组；其他对象则创建普通对象。该写法使用for...in遍历属性，适合题目限定的五种主要数据类型。Object.prototype.clone=function(){var o=this.constructor===Array?[]:{};for(var e in this){o[e]=typeof this[e]=== "object"?this[e].clone():this[e];}return o;}

337. 手写 promise.all 和 race（京东）

Promise.all要按输入顺序收集全部结果，Promise.race则把最先落定的结果直接返回。实现all时用索引写入结果数组并计数，不能直接push，因为各任务完成顺序不确定；任意一项失败就立即reject。两者都应先用Promise.resolve包装输入项，以兼容普通值。还要注意，all接收空数组时应返回空数组，而race接收空数组会一直处于pending。

```
//静态方法staticall(promiseArr){let result=[];//声明一个计数器 每一个promise返回就加一let count=0;return new MyPromise((resolve,reject)=>{for(let i=0;i<promiseArr.length;i++){let p=promiseArr[i];p.then(res=>{result[i]=res;count++;if(count===promiseArr.length){resolve(result);}}).catch(err=>{reject(err);});}})}//静态方法staticrace(promiseArr){return new MyPromise((resolve,reject)=>{for(let i=0;i<promiseArr.length;i++){let p=promiseArr[i];p.then(res=>{resolve(res);}).catch(err=>{reject(err);});}})}
```

338. 手写-实现一个寄生组合继承

寄生组合继承就是在子构造函数中调用父构造函数，再用Object.create接上父类原型。Parent.call(this,...args)负责复制实例属性，Children.prototype = Object.create(Parent.prototype)负责继承原型方法。这样不会通过new Parent()额外执行一次父构造函数，最后还要把constructor修正为Children。如果还需要继承静态成员，可以再用Object.setPrototypeOf(Children, Parent)。function Parent(name){this.name=name;this.say=()=>{console.log(111);}}Parent.prototype.play=()=>{console.log(222);};function Children(name){Parent.call(this);this.name=name;}Children.prototype=Object.create(Parent.prototype);Children.prototype.constructor=Children;let child=new Children("111");console.log(child.name);child.say();child.play();

339. 手写-new 操作符

模拟new的核心是创建一个指向构造函数原型的新对象，并让构造函数以它作为this执行。可以先用Object.create(fn.prototype)创建对象，再通过fn.call(obj, ...args)传入参数。若构造函数显式返回对象或函数

，就使用该返回值；返回原始值或没有返回值时，仍然返回新对象。实际封装时还应校验fn是否可作为构造函数，箭头函数就不能被new。 `function myNew(fn,...args){let obj=Object.create(fn.prototype);let res=fn.call(obj,...args);if(res&&(typeof res==="object"||typeof res==="function")){return res;}return obj;} // 用法如下：function Person(name,age){this.name=name;this.age=age;}Person.prototype.say=function(){console.log(this.age);};let p1=myNew(Person,"lihua",18);console.log(p1.name);console.log(p1);p1.say();`

340. 手写-setTimeout 模拟实现 setInterval（阿里）

用setTimeout模拟setInterval，就是每次回调执行完后再安排下一次定时任务。我一般会保存当前的定时标识，并返回一个取消函数；取消时设置停止标记，必要时再调用clearTimeout。这种写法保证上一轮回调完成后才开始等待下一轮，不会按固定间隔持续入队。代价是回调执行时间会叠加到间隔中，因此长时间运行可能产生累计误差。 `function mySetInterval(fn,time=1000){let timer=null,isClear=false;function interval(){if(isClear){isClear=false;clearTimeout(timer);return;}fn();timer=setTimeout(interval,time);}timer=setTimeout(interval,time);return ()=>{isClear=true;}} 函数内部定义了一个timer变量和一个isClear变量，timer用于保存setTimeout的返回值，isClear用于标记是否需要清除定时器。interval函数是核心的定时执行函数，它会在每个时间间隔内执行一次回调函数fn。在每次执行回调函数之前，会检查isClear的值，如果为true，表示需要清除定时器，此时会调用clearTimeout清除定时器并直接返回。否则，会执行回调函数fn，然后通过setTimeout设置下一个定时执行。在调用mySetInterval函数时，会立即执行一次interval函数，并通过setTimeout设置下一次的定时执行。同时，返回一个函数，调用该函数可以手动清除定时器。需要注意的是，模拟实现的mySetInterval函数在执行回调函数时是通过setTimeout实现的，因此会存在一定的延迟。实际上，使用setInterval能更准确地控制时间间隔，因为setInterval会尽可能保持固定的间隔时间。而使用setTimeout实现的mySetInterval函数可能会存在一些累积的误差。 // 测试let a=mySetTimeout(()=>{console.log(111);},1000)let cancel=mySetTimeout(()=>{console.log(222);},1000)cancel();`

341. 手写-发布订阅模式（字节）

发布订阅模式可以用一个events对象或Map实现，事件名对应一组回调，并提供on、emit、off和once。on负责追加回调，emit遍历执行并透传参数，off过滤指定回调；不传回调时也可以约定清空该事件。once本质上是包装原回调，执行后自动取消订阅，同时保留原函数引用，方便off正确匹配。还要处理事件不存在时不报错，这种结构常用于EventEmitter或事件总线。

```
class EventEmitter{
  constructor(){
    this.events={};
  }
  // 实现订阅on(type,callback){
    if(!this.events[type]){
      this.events[type]=[callback];
    }else{
      this.events[type].push(callback);
    }
  }
  // 删除订阅off(type,callback){
    if(!this.events[type])return;
    this.events[type]=this.events[type].filter((item)=>{return item!==callback;});
  }
  // 只执行一次订阅事件once(type,callback){
    function fn(){
      callback();
      this.off(type,fn);
    }
    this.on(type,fn);
  }
  // 触发事件emit(type,...rest){
    this.events[type]&&this.events[type].forEach((fn)=>fn.apply(this,rest));
  }
  // 使用如下
  const event=new EventEmitter();
  const handle=(...rest)=>{console.log(rest);};
  event.on("click",handle);
  event.emit("click",1,2,3,4);
  event.off("click",handle);
  event.emit("click",1,2);
  event.once("dbClick",()=>{console.log(123456);});
  event.emit("dbClick");
  event.emit("dbClick");
}
```

342. 手写-防抖节流（京东）

防抖是每次触发都重新计时，停止触发满wait后才执行；节流则保证一个wait窗口内最多执行一次。实现时要用apply透传调用时的this和参数，否则回调上下文可能丢失。防抖可以增加immediate支持首次立即执行；节流中，时间戳方案首次立即但可能丢末次，定时器方案首次延迟但能保留末次。实际在滚动监听

```

或输入搜索中使用，我一般还会提供cancel，方便组件卸载时清掉待执行任务。 // 防抖functiondebounce(fn,delay=300){//默认300毫秒lettimer;returnfunction(){constargs=arguments;if(timer){clearTimeout(timer);}timer=setTimeout( ()=>{fn.apply(this,args);// 改变this指向为调用debounce所指的对象},delay);}} // 测试window.addEventListener("scroll",debounce( ()=>{console.log(111);},1000));// 节流// 设置一个标志functionthrottle(fn,delay){letflag=true;return( ()=>{if(!flag)return;flag=false;timer=setTimeout( ()=>{fn();flag=true;},delay);});window.addEventListener("scroll",throttle( ()=>{console.log(111);},1000));

```

把虚拟DOM转成真实节点，本质上是根据tag创建元素、设置属性，再递归渲染children并追加进去。字符串和数字不能按元素处理，需要先转换并通过document.createTextNode创建文本节点。属性处理不能全部依赖setAttribute，像样式、类名和onClick事件应分别处理，事件要使用addEventListener。批量渲染时可以先放进DocumentFragment再统一插入，后续还可延伸到diff和key的作用。

```
{tag:'DIV',attrs:{id:'app'},children:[{tag:'SPAN',children:[{tag:'A',children:[]}]}, {tag:'SPAN',children:[{tag:'A',children:[]}, {tag:'A',children:[]}]}]}
```

把上面的虚拟Dom转化成下方真实Dom

答案

```
// 真正的渲染函数function _render(vnode){// 如果是数字类型转化为字符串if(typeof vnode === "number"){vnode = String(vnode);}// 字符串类型直接就是文本节点if(typeof vnode === "string"){return document.createTextNode(vnode);}// 普通DOMconst dom = document.createElement(vnode.tag);if(vnode.attrs){// 遍历属性Object.keys(vnode.attrs).forEach((key) => {const value = vnode.attrs[key];dom.setAttribute(key, value);});}// 子数组进行递归操作// 这一步是关键vnode.children.forEach((child) => dom.appendChild(_render(child)));return dom;}
```

对象扁平化就是递归遍历嵌套值，把访问路径拼成键，例如{ a: { b: 1 } }变成{ 'a.b': 1 }。遇到普通对象时用点号继续拼接，遇到数组时用[index]，到基本类型后再把值写入结果对象。判断可递归值时要排除null，因为typeof null也是object。空对象和空数组没有子项可遍历，若业务需要保留结构就应直接记录原值；这种方法常用于表单字段、埋点参数或配置数据打平。

题目描述

```
const obj = {a: {b: 1, c: 2, d: {e: 5}}, b: [1, 3, {a: 2, b: 3}], c: 3}
flatten(obj) // 结果返回如下 // { 'a.b': 1, 'a.c': 2, 'a.d.e': 5, 'b[0]': 1, 'b[1]': 3, 'b[2].a': 2, 'b[2].b': 3, c: 3 }
```

答案

```
function isObject(val) { return typeof val === "object" && val !== null; }
function flatten(obj) {
  if (!isObject(obj)) return;
  let res = {};
  const dfs = (cur, prefix) => {
    if (isObject(cur)) {
      if (Array.isArray(cur)) {
        cur.forEach((item, index) => {
          dfs(item, `${prefix}${index}`);
        });
      } else {
        for (let k in cur) {
          dfs(cur[k], `${prefix}${k}`);
        }
      }
    } else {
      res[prefix] = cur;
    }
  };
  dfs(obj, "");
  return res;
}
flatten(obj);
```

这道题用栈处理最合适：左括号入栈，右括号与栈顶配对，类型不一致就直接返回false。我会用映射表保存三组括号关系，这样匹配逻辑更集中，也不需要写很多条件分支。遍历结束后栈必须为空，否则说明还有左括号没有闭合；字符串长度为奇数时也可以提前判错。整体只遍历一次，时间复杂度是 $O(n)$ ，最坏空间复杂度也是 $O(n)$ 。题目描述 给定一个只包括 '(', ')', '{', '}', '[', ']' 的字符串 s，判断字符串是否有效。有效字符串需满足：

- 左括号必须用相同类型的右括号闭合。
- 左括号必须以正确的顺序闭合。

示例 1：输入：s = "()" 输出：true 示例 2：输入：s = "()[]{}" 输出：true 示例 3：输入：s = "(]" 输出：false 答案 c

```
onstisValid=function(s){if(s.length%2===1){returnfalse;}constregObj={"{":"}", "(":")", "[":"]"};letstack=[];for(leti=0;i<s.length;i++){if(s[i]=== "{" || s[i]=== "(" || s[i]=== "[" ){stack.push(s[i]);}else{constcur=stack.pop(
```



```
);if(s[i]!==regObj[cur]){returnfalse;}}if(stack.length){returnfalse;}returntrue;};
```

346. 手写-查找数组公共前缀（美团）

我一般用纵向扫描：以第一个字符串为基准，逐个位置比较所有字符串，发现字符不同或已经越界，就返回此前得到的前缀。这种写法比横向合并、分治或二分更直观，面试时也更容易把边界处理完整。空数组应返回空字符串，只有一个元素时直接返回它，任意字符串为空时结果也必然为空。设最短字符串长度为 m 、数组长度为 n ，时间复杂度是 $O(m \times n)$ 。 题目描述 编写一个函数来查找字符串数组中的最长公共前缀。如果不存在公共前缀，返回空字符串""。 示例 1：输入：strs = ["flower","flow","flight"] 输出："fl" 示例 2：输入：strs = ["dog","racecar","car"] 输出：" " 解释：输入不存在公共前缀。 答案 `const longestCommonPrefix=function(strs){const str=strs[0];let index=0;while(index<str.length){const strCur=str.slice(0,index+1);for(let i=0;i<strs.length;i++){if(!strs[i]||!strs[i].startsWith(strCur)){return str.slice(0,index);}}index++;}return str;};`

347. 手写-字符串最长的不重复子串

这道题本质上是滑动窗口：右指针不断扩张，遇到重复字符时，把左指针移动到该字符上次出现位置的下一位。我会用Map记录字符最后出现的索引，因此整个字符串只需遍历一次。关键是写成`left = Math.max(left, map.get(c) + 1)`，避免窗口外的重复字符让左指针倒退，并在每轮更新最大长度。时间复杂度为 $O(n)$ ，空间复杂度为 $O(\min(n, \text{字符集大小}))$ 。 题目描述 给定一个字符串 s ，请你找出其中不含重复字符的最长子串 的长度。 示例 1：输入：s = "abcabcbb" 输出：3 解释：因为无重复字符的最长子串是 "abc"，所以其长度为 3。 示例 2：输入：s = "bbbbb" 输出：1 解释：因为无重复字符的最长子串是 "b"，所以其长度为 1。 示例 3：输入：s = "pwwkew" 输出：3 解释：因为无重复字符的最长子串是 "wke"，所以其长度为 3。 请注意，你的答案必须是 子串 的长度，"pwke" 是一个子序列，不是子串。 示例 4：输入：s = "" 输出：0 答案 `const lengthOfLongestSubstring=function(s){if(s.length===0){return 0;}let left=0;let right=1;let max=0;while(right-1){left=index+left+1;}else{lr=s.slice(left,right+1);max=Math.max(max,lr.length);}right++;}return max;}; ---`

【前端综合问题】

348. 谈谈你对重构的理解

重构是在不改变外部可观察行为的前提下，调整代码或页面结构，让系统更容易维护、扩展，并改善性能和用户体验。前端重构既可能是提炼函数、拆分组件，也可能涉及语义化结构、浏览器兼容、移动端适配和SEO优化。实际处理时我会围绕当前问题选择少量改动，例如功能扩展前整理结构，或修复问题时清理相关坏味道。重构应当小步进行并保持可运行，只有收益明确时才值得做。 重构是指在不改变外部行为的前提下，对代码、结构、布局或者设计进行优化和改进的过程。在前端开发中，重构通常指对网站或应用程序的前端部分进行优化和改造，以提升性能、可维护性和用户体验。

对于传统的网站来说，重构可以包括以下方面的优化： 结构优化：将使用表格布局的页面改为使用DIV+CSS布局，使页面结构更加语义化、清晰，并提高页面的可读性和维护性。兼容性优化：将网站的前端代码和样式进行调整，使其能够兼容现代浏览器，并修复一些在旧版浏览器中存在的兼容性问题，提升用户体验和页面的可访问性。移动优化：针对移动设备的特点和要求，对网站进行优化，如响应式设计或者针对移动设备的单独样式和布局，以提供更好的移动端用户体验。SEO优化：通过优化网站的HTML结构、标签使

用、关键词等方面，提升网站在搜索引擎中的排名和可见性，从而增加网站的流量和曝光度。 重构的目标是改进现有代码和结构，使其更加高效、可维护和可扩展，同时保持网站的功能和外观不变。通过重构，可以提升网站的性能和用户体验，减少代码冗余，改善代码质量，并为后续的功能扩展和维护提供更好的基础。

349. 什么样的前端代码是好的

好的前端代码应该让人容易读懂、方便修改和复用，同时兼顾性能、可靠性、兼容性与安全性。结构清晰、命名准确，能减少理解和调试成本；模块低耦合，改动时也不容易牵连其他功能。抽象并非越多越好，组件、函数和样式应在确有复用价值时再拆分。面对不同浏览器、设备和异常输入，代码还要稳定运行，并通过测试防住常见问题。 好的前端代码具有以下特点： 可读性高：代码结构清晰，命名规范，注释清晰明了，易于理解和维护。高复用性：代码组织良好，模块化设计，可通过复用组件、函数和样式来减少代码的冗余。低耦合性：模块之间相互独立，减少模块之间的依赖，修改一个模块时不会对其他模块造成影响。高性能：代码优化，减少不必要的计算和请求，合理使用缓存机制，提高页面加载速度和响应性能。可维护性：代码结构清晰，逻辑简洁，易于调试和修改，便于团队协作和后续的功能迭代。可靠性：代码经过严格的测试，处理各种异常情况，确保系统的稳定性和可靠性。宽容性：能够适应不同浏览器和设备的差异，具有良好的兼容性，确保在各种环境下都能正常运行和展示。安全性：防范常见的前端安全漏洞，保护用户信息和系统数据的安全。 此外，好的前端代码还应遵循设计模式的六大原则，包括单一职责原则、开放封闭原则、里氏替换原则、依赖倒置原则、接口隔离原则和迪米特法则，以提高代码的可扩展性、可维护性和可重用性。 综上所述，好的前端代码是可读性高、复用性强、低耦合性、高性能、可维护性和可靠性强的代码，同时符合设计模式的原则。

350. 对前端工程师这个职位是怎么样理解的？它的前景会怎么样

前端工程师本质上是把视觉设计和产品需求，落成用户能直接操作的界面。除了用HTML、CSS和JavaScript实现交互，还要关注加载速度、响应体验、可访问性以及后续维护。工作中需要和设计、产品、后端持续协作，也可能借助Node.js扩展到移动端或桌面端。这个岗位仍有发展空间，但技术变化很快，前景取决于能否持续学习并提升产品体验。 前端工程师是负责开发和实现用户界面的程序员。他们在网站和应用程序开发过程中扮演着至关重要的角色，负责将设计师提供的视觉设计转化为具有交互性和响应性的前端界面。以下是我对前端工程师职位的理解： 实现界面交互：前端工程师通过编写HTML、CSS和JavaScript代码来实现用户界面的交互功能，包括按钮点击、表单提交、页面切换等。提升用户体验：前端工程师致力于提供优秀的用户体验，通过优化页面加载速度、响应性能和用户界面设计，提升用户对产品的满意度和使用体验。跨平台开发：随着Node.js的发展，前端工程师可以使用JavaScript开发跨平台的应用程序，如桌面应用、移动应用等，扩展了前端工程师的技术领域。与团队合作：前端工程师需要与UI设计师、产品经理、后端工程师等团队成员密切合作，理解需求并与他们进行沟通，确保产品的需求能够得到有效地实现。页面结构和重构：前端工程师负责设计和构建页面的结构，使其具备良好的可读性和可维护性。他们还会进行页面重构，以提高网站的性能和可访问性。 前端工程师的前景非常广阔。随着移动互联网和Web技术的快速发展，前端技术变得越来越重要。随着新的前端技术和框架的涌现，前端工程师需要不断学习和更新自己的技能，以适应行业的变化。同时，前端工程师对于提高用户体验和产品质量起着关键作用，因此在各个行业都有很高的需求。 随着移动应用、响应式设计和用户体验的重要性不断增加，前端工程师的职业前景将会持续看好。他们将继续在设计和开发过程中扮演重要角色，并为产品的成功做出贡献。

351. 你觉得前端工程的价值体现在哪

前端工程的价值，是让产品在不同设备和浏览器上都能稳定、快速且易用地呈现给用户。它既要处理兼容性和跨平台差异，也要通过优化代码、资源与缓存减少加载和响应时间。前端还负责连接后端接口，把数据准确、安全地处理后清晰展示出来。实际取舍时不能只追求视觉一致，还要兼顾性能、可访问性和维护

成本。前端工程的价值在于以下几个方面：用户体验优化：前端工程师通过优化用户界面和交互设计，提供良好的用户体验，使用户能够轻松、高效地使用产品。他们关注用户界面的可用性、易用性和可访问性，确保用户能够舒适地与产品进行交互。浏览器兼容性支持：不同的浏览器和设备有不同的特性和标准支持，前端工程师需要处理这些差异，确保产品在各种浏览器和设备上都能正常运行和呈现一致的用户体验。他们会进行浏览器兼容性测试和适配，以确保产品在不同环境下都能够良好运行。性能优化：前端工程师负责优化前端代码和资源加载，以提高网页的加载速度和响应性能。他们会优化代码结构、压缩和合并文件、使用缓存机制等手段，从而减少页面加载时间，提高用户体验。跨平台支持：随着移动设备和多平台应用的普及，前端工程师需要适应不同平台和设备的需求，开发跨平台应用或者响应式设计，使产品能够在不同平台上都具备良好的用户体验。数据展示和接口支持：前端工程师负责展示后端数据，并通过与后端接口的交互实现数据的获取和处理。他们需要熟悉数据接口的调用和处理，确保数据的准确性和安全性，同时展示给用户的数据呈现清晰、易懂。总之，前端工程的价值体现在提供良好的用户体验、保证产品在不同浏览器和平台上的兼容性、优化网页性能以及实现数据展示和接口支持等方面。他们为产品的成功和用户满意度做出了重要贡献。

352. 平时如何管理你的项目

我一般会从统一约定、文件组织和版本管理三个方面管理前端项目。团队先明确样式、命名、编码和注释规范，是为了降低协作时的理解成本，并让代码保持一致。文件和静态资源按统一目录及功能命名，图片也会选择合适格式并做必要优化，方便查找和控制体积。每次提交Git时写清改动内容与目的，但注释和规范也要适度，避免增加无效维护负担。

项目管理在前端开发中非常重要，以下是一些常见的项目管理实践：确定项目全局约定：团队需要确定全局样式、编码规范、命名约定等，以保证代码的一致性和可维护性。统一编码风格：团队成员需要遵循统一的编码风格，例如使用继承式的写法、单样式一行等，以提高代码的可读性和可理解性。标注样式和模块：及时在代码中标注关键样式的编写人以及调用的地方，同时对页面进行标注，方便团队成员的理解和协作。文件组织结构：将CSS和HTML文件分别放在不同的文件夹中，并统一命名规则，例如使用style.css作为CSS文件的命名。JS文件管理：将JS文件放在独立的文件夹中，并根据功能进行命名，使用英文翻译来描述其功能。图片整合与优化：采用整合的方式使用图片，并使用适当的格式和优化策略，以减少文件大小和提升加载速度。规范代码注释：对代码进行详细的注释，包括HTML、JS、CSS等，以提高代码的可读性和可维护性。严格要求静态资源存放路径：规定静态资源的存放路径，遵循统一的文件夹结构，方便团队成员查找和管理资源文件。Git提交说明：在每次提交代码到版本控制系统时，要求团队成员填写清晰的提交说明，描述提交的内容和目的。这些项目管理实践可以提高团队的协作效率、代码质量和项目可维护性，确保项目的顺利进行和成功交付。

353. 组件封装

组件封装的核心是单一职责和可复用，同时通过清晰、适量的接口降低使用成本。我一般会先分析布局与交互，再完成基础结构和样式，之后删除冗余代码并提取通用能力。命名、文档、示例和测试也要跟上，否则组件虽然能复用，却很难维护。基础组件可以更通用，业务组件则应贴近实际需求，不必为了覆盖所有场景堆很多配置。组件封装是前端开发中常用的技术手段，它的目的是为了实现代码的重用、提高开发效率和代码质量。在组件封装过程中，需要注意以下几个方面：分析布局：首先需要对布局进行分析，确定组件的结构和样式。了解组件在不同场景下的表现形式和行为。初步开发：根据布局的分析，开始进行组件的初步开发，包括HTML结构、CSS样式和基本的交互行为。化繁为简：在初步开发的基础上，对组件进行优化和简化。去除冗余的代码，提取通用的样式和功能，确保组件的精简和高效。组件抽象：在封装组件时，需要将组件的功能进行抽象，使其具有单一的职责和可复用性。将组件的各个部分分离，并提供适当的接口和配置项，使组件的使用更加灵活。常用的操作包括：使用合适的命名规范，保证组件的易读性和可维护性。提供必要的文档和示例，方便其他开发人员使用和理解组件。尽量降低组件与外部环境的

耦合度，使其在不同的项目中都能够灵活使用。考虑组件的可扩展性，使其能够适应未来的需求变化。在开发过程中进行测试，确保组件的功能和性能达到预期。通过良好的组件封装实践，可以提高代码的可维护性、可复用性和可测试性，加快开发速度，减少重复劳动，并提升整体的代码质量。

354. Web 前端开发的注意事项

Web前端开发要同时兼顾适配、兼容性、性能、安全和代码可维护性。移动端应正确设置viewport，结合百分比布局、box-sizing: border-box、rem和媒体查询适配不同尺寸。性能上可通过缓存、资源压缩和减少请求改善加载速度，安全上要防范XSS、CSRF等常见问题。上线前还要覆盖主流浏览器、不同机型和系统测试，不能只在开发机上验证。在进行Web前端开发时，有一些注意事项可以帮助你提高开发效率和代码质量，以下是一些常见的注意事项：

- 特别设置meta标签viewport：通过设置viewport来适配不同设备的屏幕大小，确保网页在移动设备上有良好的显示效果。
- 使用百分比布局宽度：使用百分比来设置元素的宽度，结合box-sizing: border-box;可以更好地适应不同屏幕大小。
- 使用rem作为计算单位：使用rem作为字体大小和元素尺寸的计算单位，rem相对于根节点html的字体大小，可以实现响应式的布局。
- 使用CSS3新特性：充分利用CSS3提供的新特性，如弹性盒模型、多列布局、媒体查询等，来简化布局和样式的编写，并提升用户体验。
- 多机型、多尺寸、多系统覆盖测试：在开发过程中，要充分考虑不同设备、不同屏幕尺寸和不同操作系统的兼容性，进行全面的测试，确保网页在各种环境下都能正常显示和工作。

此外，还有一些其他的注意事项：

- 优化网页性能：合理使用缓存、压缩资源、减少HTTP请求等方式来提升网页的加载速度和响应性能。
- 代码规范和可维护性：遵循一致的命名规范，使用合适的注释，拆分模块，保持代码的可读性和可维护性。
- 跨浏览器兼容性：测试并确保网页在主流浏览器（如Chrome、Firefox、Safari、Edge等）上都能正常运行。
- 安全性：注意防止常见的Web安全漏洞，如跨站脚本攻击（XSS）、跨站请求伪造（CSRF）等。

综上所述，注意这些事项可以帮助你开发出更高质量、更兼容性好的Web前端项目。

355. 在设计 Web APP 时，应当遵循以下几点

设计Web App时应保持界面和操作简单，让用户更快找到重点并完成任务。动画、图形和文字样式只在确实有帮助时使用，过度装饰反而会干扰内容。手势也要克制，既要避免操作复杂，也要防止与浏览器原有手势冲突。内容较多时可用局部刷新或异步加载减少页面跳转，并用合适的加载提示和清晰层级缓解等待感。在设计Web APP时，可以遵循以下几点来提升用户体验和页面效果：

- 简化不重要的动画/动效/图形文字样式：避免过度使用动画效果和视觉元素，保持页面简洁和清晰。只在必要的地方使用动画，确保其对用户体验有积极影响。
- 少用手势，避免与浏览器手势冲突：在设计交互时，尽量减少需要用户进行复杂手势操作的情况，避免与浏览器自带手势产生冲突。保持用户操作的简洁性和一致性。
- 减少页面内容和跳转次数：优化页面内容，保持简洁明了，尽量在当前页面展示所需的信息和功能，避免频繁的页面跳转。通过局部刷新、异步加载等技术手段来提升用户体验和页面加载速度。
- 增强Loading趣味性，增强页面主次关系：在加载过程中，可以设计有趣的加载动画或提示，提高用户的等待体验。同时，通过合适的页面布局和元素设计，突出页面的主要内容和功能，帮助用户更快地获取所需信息。

综上所述，遵循以上设计原则可以使Web APP在用户体验和页面效果方面更出色，提供更好的用户体验和用户参与感。

356. 你怎么看待 Web App/hybrid App/Native App？（移动端前端 和 Web 前端区别？）

Web	App、Hybrid	App和Native
App没有绝对优劣，关键是平衡开发成本、更新方式、体验和设备能力。Web App跨平台且更新方便，但相机、GPS、推送和图形效果会受限制；Hybrid		

App用WebView兼顾部分原生体验与离线能力，但性能通常较慢。Native App体验和本地资源访问更强，却要承担多系统开发、版本更新和商店审核成本。移动端开发还要额外关注触摸手势、屏幕适配和设备能力，而普通Web前端更侧重响应式与浏览器兼容。

Web App(HTML5)：采用HTML5生存在浏览器中的应用，不需要下载安装优点：开发成本低，迭代更新容易，不需用户升级，跨多个平台和终端缺点：消息推送不够及时，支持图形和动画效果较差，功能使用限制（相机、GPS等）

Hybrid App(混合开发)：UI WebView，需要下载安装优点：接近 Native App的体验，部分支持离线功能缺点：性能速度较慢，未知的部署时间，受限于技术尚不成熟

Native App(原生开发)：依托于操作系统，有很强的交互，需要用户下载安装使用优点：用户体验完美，支持离线工作，可访问本地资源（通讯录，相册）缺点：开发成本高（多系统），开发成本高（版本更新），需要应用商店的审核

优点：开发成本低，迭代更新容易，不需用户升级，跨多个平台和终端缺点：消息推送不够及时，支持图形和动画效果较差，功能使用限制（相机、GPS等）

优点：接近 Native App的体验，部分支持离线功能缺点：性能速度较慢，未知的部署时间，受限于技术尚不成熟

优点：用户体验完美，支持离线工作，可访问本地资源（通讯录，相册）缺点：开发成本高（多系统），开发成本高（版本更新），需要应用商店的审核

移动端前端和Web前端在某些方面存在区别，如下所示：

开发环境和技术：移动端前端主要面向移动设备，需要熟悉移动端的开发环境和相关技术，如React Native、Flutter、Ionic等。而Web前端则主要面向浏览器，使用HTML、CSS、JavaScript等技术进行开发。

用户体验和交互：移动端前端需要更关注移动设备的特性和用户体验，例如触摸操作、手势识别、屏幕适配等，以提供更好的移动端交互体验。而Web前端则更注重页面的响应式设计和跨浏览器兼容性，以确保在不同浏览器和设备上的良好表现。

功能支持和限制：移动端前端开发可能会受到一些限制，如对设备资源的访问（相机、GPS等）、消息推送的限制等。而Web前端在功能上较为灵活，可以通过浏览器提供的API和插件来实现一些高级功能。

部署和更新：移动端前端需要通过应用商店进行发布和更新，需要经过审核和下载安装。而Web前端则可以直接通过URL访问，无需下载安装，便于部署和更新。

综上所述，移动端前端和Web前端在开发环境、用户体验、功能支持和部署等方面存在一些区别，开发者需要根据具体需求和平台选择合适的开发方式。同时，随着技术的不断发展，Hybrid App和Web App的出现使得移动端前端在一定程度上能够接近Native App的体验。

357. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。

页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。

设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。

CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。

JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。

响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。

性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。

部署和上线：将重构

后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

358. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

359. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构

后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

360. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

361. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构

后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

362. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

363. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构

后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

364. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

365. 页面重构怎么操作

页面重构要先评估现状并规划目标，再分别处理结构、样式、脚本和资源，最后经过测试后上线。我一般会先找出加载速度、响应性能和代码结构上的问题，因为没有明确目标，重构很容易变成单纯重写。实施时重点优化语义化HTML、可复用的CSS和模块化JavaScript，同时压缩图片、减少请求并做好响应式适配。完成后还要覆盖不同浏览器和设备，部署后结合性能与用户反馈继续调整。 页面重构是对网站或应用程序进行优化和改进的过程，旨在提高页面性能、用户体验和代码可维护性。以下是页面重构的一般操作步骤：

分析和评估：首先，对现有页面进行全面的分析和评估。确定需要改进的方面，如页面加载速度、响应性能、代码结构等。设计和规划：根据分析的结果，设计新的页面结构和布局，并规划重构的步骤和优化策略。确保重构后的页面能够提供更好的用户体验和性能。

HTML重构：根据设计和规划，对HTML代码进行重构。优化标签的结构和语义化，使用合适的标签和属性，去除冗余代码和不必要的嵌套。CSS重构：对CSS样式进行重构，优化样式的选择器和属性的使用。遵循CSS规范，减少样式冗余，提高样式的复用性和可维护性。JavaScript重构：对JavaScript代码进行重构，优化代码的结构和逻辑。使用模块化开发和设计模式，减少全局变量的使用，提高代码的可读性和可维护性。

图片和资源优化：对页面中的图片和其他资源进行优化，压缩文件大小，提高加载速度。使用合适的图片格式，使用雪碧图或矢量图标减少HTTP请求。响应式设计：考虑不同设备和屏幕尺寸的适配，使用响应式布局和媒体查询，确保页面在不同设备上具有良好的展示效果。性能优化：优化页面的性能，包括减少HTTP请求，使用缓存机制，延迟加载和异步加载资源，合理使用CSS和JavaScript动画效果等。

测试和调试：在重构完成后，进行全面的测试和调试，确保页面在不同浏览器和设备上正常运行，并验证重构的效果和改进是否符合预期。部署和上线：将重构

后的页面部署到生产环境，并监控其性能和用户反馈。根据反馈和数据进行进一步的优化和改进。 页面重构是一个综合性的工作，需要综合考虑页面结构、样式、脚本和性能等方面的优化。通过合理的规划和操作，可以提升页面的质量、性能和用户体验。 ---

【JS基础】

366. 类型及检测方式

JavaScript有 7 种基本类型和Object引用类型，检测时要根据目标选择typeof、instanceof或Object.prototype.toString.call()。typeof适合基本类型和函数，但typeof null、数组及普通对象都会得到object。instanceof本质上检查原型链，适合引用类型，却不能正确判断基本类型，而且constructor可能被修改。需要精确区分数组、日期、正则等对象时，我一般会用Object.prototype.toString.call()。 1. JS内置类型 JavaScript 的数据类型有下图所示 其中，前 7 种类型为基础类型，最后1 种（Object）为引用类型，也是你需要重点关注的，因为它在日常工作中是使用得最频繁，也是需要关注最多技术细节的数据类型 JavaScript一共有8种数据类型，其中有7种基本数据类型：Undefined、Null、Boolean、Number、String、Symbol（es6新增，表示独一无二的值）和BigInt（es10新增）；1种引用数据类型——Object（Object本质上是由一组无序的名值对组成的）。里面包含function、Array、Date等。JavaScript不支持任何创建自定义类型的机制，而所有值最终都将是上述 8 种数据类型之一。引用数据类型:对象Object（包含普通对象-Object，数组对象-Array，正则对象-RegExp，日期对象-Date，数学函数-Math，函数对象-Function） 引用数据类型:对象Object（包含普通对象-Object，数组对象-Array，正则对象-RegExp，日期对象-Date，数学函数-Math，函数对象-Function） 在这里，我想先请你重点了解下面两点，因为各种 JavaScript 的数据类型最后都会初始化的放在不同的内存中，因此上面的数据类型大致可以分成两类来进行存储：

原始数据类型：基础类型存储在栈内存，被引用或拷贝时，会创建一个完全相等的变量；占据空间小、大小固定，属于被频繁使用数据，所以放入栈中存储。引用数据类型：引用类型存储在堆内存，存储的是地址，多个引用指向同一个地址，这里会涉及一个“共享”的概念；占据空间大、大小不固定。引用数据类型在栈中存储了指针，该指针指向堆中该实体的起始地址。当解释器寻找引用值时，会首先检索其在栈中的地址，取得地址后从堆中获得实体。 JavaScript 中的数据是如何存储在内存中的？ 在 JavaScript 中，原始类型的赋值会完整复制变量值，而引用类型的赋值是复制引用地址。 在 JavaScript 的执行过程中，主要有三种类型内存空间，分别是代码空间、栈空间、堆空间。其中的代码空间主要是存储可执行代码的，原始类型(Number、String、Null、Undefined、Boolean、Symbol、BigInt)的数据值都是直接保存在“栈”中的，引用类型(Object)的值是存放在“堆”中的。因此在栈空间中(执行上下文)，原始类型存储的是变量的值，而引用类型存储的是其在“堆空间”中的地址，当 JavaScript 需要访问该数据的时候，是通过栈中的引用地址来访问的，相当于多了一道转手流程。

在编译过程中，如果 JavaScript 引擎判断到一个闭包，也会在堆空间创建换一个“closure(fn)”的对象（这是一个内部对象，JavaScript 是无法访问的），用来保存闭包中的变量。所以闭包中的变量是存储在“堆空间”中的。 JavaScript 引擎需要用栈来维护程序执行期间上下文的状态，如果栈空间大了话，所有的数据都存放在栈空间里面，那么会影响到上下文切换的效率，进而又影响到整个程序的执行效率。通常情况下，栈空间都不会设置太大，主要用来存放一些原始类型的小数据。而引用类型的数据占用的空间都比较大，所以这一类数据会被存放到堆中，堆空间很大，能存放很多大的数据，不过缺点是分配内存和回收内存都会占用一定的时间。因此需要“栈”和“堆”两种空间。 题目一：初出茅庐 let a={name:'lee',age:18};let b=a;console.log(a.name);//第一个console b.name='son';console.log(a.name);//第二个console console.log(b.name);//

第三个console 这道题比较简单，我们可以看到第一个 console 打出来 name 是 'lee'，这应该没什么疑问；但是在执行了 b.name='son' 之后，结果你会发现 a 和 b 的属性 name 都是 'son'，第二个和第三个打印结果是一样的，这里就体现了引用...

1) 埋点SDK用typeof payload === 'object'判断可展开数据，线上收到null后在Object.keys(payload)报错，判断条件该怎样收紧？

答：应先排除null，至少写成payload !== null && typeof payload === 'object'，因为typeof null的结果也是object。若只接受普通对象，还要继续排除数组、日期等引用类型；仅靠typeof无法完成这种细分。

2) 表单校验要区分数组、日期、正则和普通对象，数据量不大但来源复杂，你会统一使用哪种检测方式，返回值要注意什么？

答：可使用Object.prototype.toString.call(value)获取形如[object Array]、[object Date]的标签，再统一提取类型名。它比typeof更适合区分这些内置引用类型；返回值的大小写和格式必须固定，否则调用方容易把Array与array混用。

3) 微前端页面从另一个iframe传来数组，当前代码用value instanceof Array却返回false，为什么，应该换什么判断？

答：instanceof检查右侧构造函数的prototype是否出现在对象原型链上，不同iframe拥有不同的内建造函数和原型。判断数组应使用Array.isArray(value)，或在统一类型工具中使用Object.prototype.toString.call；跨执行环境时不要依赖当前窗口的Array.prototype。

4) 线上有个对象被库代码替换了原型，随后value.constructor === Model判断失效，你会按什么顺序排查？

答：先查看对象当前原型链以及constructor属性来自哪里，因为替换prototype或覆盖constructor都会让该判断失真。若要判断原型关系应改用合适的instanceof，若要识别数据形状则校验字段；两者语义不同，也都不能仅凭构造器名称兜底。

5) 接口字段可能是数字0、字符串"0"、null或NaN，产品要求只有后两者显示“数据异常”，能不能用if (!value)？

答：不能，!value会同时把数字0等假值判为异常，破坏合法数据。应明确写成value === null || Number.isNaN(value)；不要用全局isNaN代替，后者会进行类型转换，使非数字字符串也可能被判为NaN。

367. This

普通函数的this在调用时确定：直接调用通常走默认绑定，obj.fn()指向obj，构造调用则指向new创建的对象。多种规则同时出现时，优先级是new、显式绑定、隐式绑定、默认绑定。箭头函数没有自己的this，它会沿用最近外层普通函数的this，因此用bind也改不了。对普通函数多次调用bind时，最终绑定对象由第一次bind决定。不同情况的调用，this指向分别如何。顺带可以提一下es6中箭头函数没有this,arguments,super等，这些只依赖包含箭头函数最接近的函数

我们先来看几个函数调用的场景
function foo(){console.log(this.a)}var a=1foo()const obj={a:2,foo:foo}obj.foo()const c=new foo()
对于直接调用foo来说，不管foo函数被放在了什么地方，this一定是window对于obj.foo()来说，我们只需要记住，谁调用了函数，谁就是this，所以在这个场景下foo函数中的this就是obj对象对于new的方式来说，this被永远绑定在了c上面，不会被任何方式改变this

说完了以上几种情况，其实很多代码中的this应该就没什么问题了，下面让我们看看箭头函数中的this
function a(){return ()=>{return ()=>{console.log(this)}}}console.log(a())() 首先箭头函数其实是没有this的，箭头函数中的this只取决包裹箭头函数的第一个普通函数的this。在这个例子中，因为包裹箭头函数的

第一个普通函数是a，所以此时的this是window。另外对箭头函数使用bind这类函数是无效的。最后种情况也就是bind这些改变上下文的API了，对于这些函数来说，this取决于第一个参数，如果第一个参数为空，那么就是window。那么说到bind，不知道大家是否考虑过，如果对一个函数进行多次bind，那么上下文会是什么呢？

```

let a = {}
let fn = function() { console.log(this) }
fn.bind(a)() // => ?

```

如果你认为输出结果是a，那么你就错了，其实我们可以把上述代码转换成另一种形式

```

// fn.bind().bind(a) 等于 let fn2 = function fn1() { return function() { return fn.apply() }.apply(a) }
fn2()

```

可以从上述代码中发现，不管我们给函数bind几次，fn中的this永远由第一次bind决定，所以结果永远是window

```

let a = { name: 'poetries' }
function foo() { console.log(this.name) }
foo.bind(a)() // => 'poetries'

```

以上就是this的规则了，但是可能会发生多个规则同时出现的情况，这时候不同的规则之间会根据优先级最高的来决定this最终指向哪里。首先，new的方式优先级最高，接下来是bind这些函数，然后是obj.foo()这种调用方式，最后是foo这种调用方式，同时，箭头函数的this一旦被绑定，就不会再被任何方式所改变。

函数执行改变this 由于 JS 的设计原理：在函数中，可以引用运行环境中的变量。因此就需要一个机制来让我们可以在函数体内部获取当前的运行环境，这便是this。

因此要明白this指向，其实就是要搞清楚函数的运行环境，说人话就是，谁调用了函数。例如 obj.fn()，便是obj调用了函数，既函数中的this === objfn()，这里可以看成window.fn()，因此this === window

但这种机制并不完全能满足我们的业务需求，因此提供了三种方式可以手动修改this的指向：

```

call: fn.call(target, 1, 2)
apply: fn.apply(target, [1, 2])
bind: fn.bind(target)(1, 2)

```

1) 管理后台启用严格模式后，工具函数foo()内访问this.user报错，而旧页面里曾经指向window，你怎么解释这个差异？

答：普通函数直接调用时没有接收者，严格模式下this是undefined，因此读取this.user会报错；非严格浏览器脚本中才可能回退到window。业务代码不应依赖这种全局回退，应显式传入对象或以obj.foo()的形式调用。

2) React类组件把this.handleSave直接交给按钮，点击后this丢失；团队在构造器bind和class字段箭头函数之间怎么选？

答：两种方式都能让回调保留组件实例：构造器中执行一次bind，或把处理器定义为捕获外层this的箭头函数。选型应服从项目编译配置和代码规范；不要在每次渲染时临时bind，否则会持续产生新的函数引用。

3) 事件系统原来用handler.call(context, event)注入上下文，现在有人把处理器改成箭头函数，为什么context不再生效？

答：箭头函数没有自己的this，它从定义位置捕获外层this，所以call的第一个参数无法改写它。若事件系统契约依赖动态上下文，应保留普通函数；若处理器只依赖词法作用域，箭头函数更明确，但会放弃运行时注入能力。

4) 一段代码执行fn.bind(user).bind(admin)()，日志仍打印user.name，排查时你会如何判断是哪次绑定生效？

答：第一次bind(user)已返回一个绑定函数，后续再对该绑定函数执行bind(admin)不会替换原目标函数的绑定上下文。因此应沿函数创建链找到最早的bind；多次绑定会增加理解成本，若需要动态切换对象，应改用call、apply或显式参数。

5) 对象方法obj.run既先被bind(config)，又被new调用，代码评审中有人认为实例的this仍是config，你会怎么判断？

答：以new调用绑定函数时，构造调用创建的新实例应成为原函数的this，绑定对象不会作为实例上下文。实现手写bind时必须识别构造调用并转交原型关系；若原函数返回对象，还要遵循构造函数显式返回对象的语义，简单apply并不完整。

368. apply/call/bind 原理

call、apply和bind都能指定函数执行时的this，区别在传参形式和是否立即执行。call逐个接收参数，apply接收参数数组，两者会马上调用函数；bind则返回新函数，还能预先保存部分参数。手写call或apply时，可以把原函数临时挂到目标对象上执行，再删除该属性，使用Symbol能避免属性冲突。实现bind还要处理构造调用：通过new执行返回函数时，应让新实例成为this。

call、apply和bind是挂在Function对象上的三个方法，调用这三个方法的必须是一个函数。

func.call(thisArg, param1, param2, ...) func.apply(thisArg, [param1,param2,...]) func.bind(thisArg, param1, param2, ...) 在浏览器里，在全局范围内this 指向window对象；在函数中，this永远指向最后调用他的那个对象；构造函数中，this指向new出来的那个新的对象；call、apply、bind中的this被强绑定在指定的那个对象上；箭头函数中this比较特殊,箭头函数this为父作用域的this，不是调用时的this.要知道前四种方式,都是调用时确定,也就是动态的,而箭头函数的this指向是静态的,声明的时候就确定了下来；apply、call、bind都是js给函数内置的一些API，调用他们可以为函数指定this的执行,同时也可以传参。 let a={value:1}function getValue(name,age){console.log(name)console.log(age)console.log(this.value)}getValue.call(a,'poe','24')getValue.apply(a,['poe','24'])

bind和其他两个方法作用也是一致的，只是该方法会返回一个函数。并且我们可以通过bind实现柯里化方法的应用场景

下面几种应用场景，你多加体会就可以发现它们的理念都是“借用”方法的思路。我们来看看都有哪些。

判断数据类型

用Object.prototype.toString来判断类型是最合适的，借用它我们几乎可以判断所有类型的数据 function getType(obj){let type=typeof obj;if(type!=="object"){return type;}return Object.prototype.toString.call(obj).replace(/^\\$/,'\$1');} 类数组借用方法 类数组因为不是真正的数组，所有没有数组类型上自带的种种方法，所以我们可以利用一些方法去借用数组的方法，比如借用数组的push方法，看下面的一段代码。

```
var arrayLike={0:'java',1:'script',length:2}Array.prototype.push.call(arrayLike,'jack','lily');console.log(typeof arrayLike);// 'object'console.log(arrayLike);// {0: "java", 1: "script", 2: "jack", 3: "lily", length: 4}
```

用call的方法来借用Array 原型链上的 push方法，可以实现一个类数组的push方法，给arrayLike添加新的元素 获取数组的最大 / 最小值 我们可以用 apply 来实现数组中判断最大 / 最小值，apply直接传递数组作为调用方法的参数，也可以减少一步展开数组，可以直接使用Math.max、Math.min来获取数组的最大值 / 最小值，请看下面这段代码。 let arr=[13,6,10,11,16];const max=Math.max.apply(Math,arr);const min=Math.min.apply(Math,arr);console.log(max);// 16console.log(min);// 6

实现一个 bind 函数 对于实现以下几个函数，可以从几个方面思考 不传入第一个参数，那么默认为window改变了this指向，让新的对象可以执行该函数。那么思路是否可以变成给新的对象添加一个函数，然后在执行完以后删除？ Function.prototype.myBind=function(context){if(typeof this !== 'function'){throw new TypeError('Error')}}var _this=thisvar args=[...arguments].slice(1)// 返回一个函数return functionF...

1) 代码评审里有人说call、apply、bind都只是参数写法不同，但保存const saved = fn.bind(user, 1)后没有立即执行，你怎么反驳？

答：call和apply会立即调用函数，只是前者逐个传参、后者接收参数集合；bind返回新的绑定函数，调用可以延后并支持预置参数。把三者当成同一种立即调用API，会导致副作用时机和返回值判断错误。

2) 你手写的myCall用固定属性context.fn =

this，线上对象本来就有fn字段，调用后字段丢了，怎么修？

答：固定属性名会覆盖对象已有字段，随后delete context.fn又把原值删除，应改用临时Symbol作为键并在调用后清理。清理最好放进finally，否则目标函数抛错时临时属性会残留；此外还要处理不可扩

展对象，这种挂载模拟存在天然边界。

3) 旧工具的myCall写成context = context ||

window，传入数字0或空字符串时却把this指向全局对象，问题在哪里？

答：||会把0、空字符串和false都当成缺省值，因而错误替换合法的thisArg。实现应只按目标语义处理null和undefined，并考虑原始值装箱及严格模式差异；简单的对象挂载法无法完全复刻原生行为。

4) 批量统计页面对一个超大数组执行Math.max.apply(null, values)后抛出参数数量相关异常，你会如何改造并验证？

答：apply会把数组元素作为独立实参传入，数组足够大时可能超过引擎允许的参数数量，不能把具体上限当成稳定契约。应使用reduce逐项比较，或分块求局部最大值再合并；同时明确空数组结果，避免默认得到不符合业务预期的值。

5) 手写bind只做return (...args) => fn.apply(context, args)，测试普通调用通过，但new Bound()的实例关系失败，缺了哪些语义？

答：这种箭头包装既不能作为构造函数，也没有处理new对绑定上下文的覆盖，因此无法模拟原生bind的构造调用。返回值应使用普通函数，构造时让新实例作为this并衔接原函数原型；完整复制原生函数的所有内部行为仍不现实，应标明实现范围。

369. 变量提升

变量提升不是把代码搬到顶部，而是在创建执行环境时提前处理声明。var会先被初始化为undefined，所以声明前读取不会报错，但赋值仍留在原来的位置。let和const同样会建立绑定，只是初始化前处于TDZ，提前访问会报错。函数声明会连同函数体一起提升；同名声明出现时，后面的函数会覆盖前面的函数，而且函数声明优先于变量声明。当执行JS代码时，会生成执行环境，只要代码不是写在函数中的，就是在全局执行环境中，函数中的代码会产生函数执行环境，只此两种执行环境。 b()// call bconsole.log(a)// undefinedvara='Hello world'functionb(){console.log('call b')}想必以上的输出大家肯定都已经明白了，这是因为函数和变量提升的原因。通常提升的解释是说将声明的代码移动到了顶部，这其实没有什么错误，便于大家理解。但是更准确的解释应该是：在生成执行环境时，会有两个阶段。第一个阶段是创建的阶段，JS解释器会找出需要提升的变量和函数，并且给他们提前在内存中开辟好空间，函数的话会将整个函数存入内存中，变量只声明并且赋值为undefined，所以在第二个阶段，也就是代码执行阶段，我们可以直接提前使用 在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升 b()// call b secondfunctionb(){console.log('call b fist')}functionb(){console.log('call b second')}varb='Hello world' var会产生很多错误，所以在 ES6中引入了let。let不能在声明前使用，但是这并不是常说的let不会提升，let提升了，在第一阶段内存也已经为他开辟好了空间，但是因为这个声明的特性导致了并不能在声明前使用

1) 你接手的活动页里先执行console.log(price)，下一行才写var price = 99，页面没有报错却打印了undefined；如果同事坚持说“引擎把代码搬到了顶部”，你怎么纠正？

答：结果是undefined，但引擎并没有真的重排源码。执行环境创建阶段会登记var price并初始化为undefined，执行阶段到赋值语句才写入99；“搬到顶部”只能作为表象解释，遇到函数声明冲突时容易误判。

2) 一个旧版结算页同时声明了两个function submit()，后面又写var submit = 'disabled'，但在赋值前调用submit()执行的是第二个函数；你如何解释这段初始化过程？

答：创建阶段会处理函数声明与变量声明，同名的后一个函数声明覆盖前一个，而同名var声明不会把已登记的函数改成undefined。执行到var submit = 'disabled'时才发生赋值，此后submit()才会因值不再是函数而失败。

3) 团队把配置读取从var token改为let token，监控从静默拿到undefined变成了启动时抛出ReferenceError；这是回归还是更严格的约束？

答：这是声明语义改变带来的主动失败，不代表let完全没有提升。绑定在块级作用域创建时已经存在，但声明执行前处于暂时性死区，任何读取都会抛出ReferenceError；代价是旧代码会暴露初始化顺序问题，却能避免错误值继续传播。

4) 线上首屏偶发ReferenceError: Cannot access 'config' before initialization，同一代码块里确实写了let config；你会沿着什么方向定位，而不是把它当成变量未声明？

答：先查报错读取点是否位于let config的声明执行之前，包括条件分支、初始化表达式和间接函数调用。该名称已绑定到当前块，外层同名变量也会被遮蔽，因此不能继续沿作用域链取外层值；修复应调整初始化顺序或拆分作用域。

5) 报表页的循环用var

i注册五个延迟回调，评审中有人主张只改let，另一人要求兼容现有写法；你会怎样选择？

答：能使用块级绑定优先改为let，因为循环每轮都会形成独立绑定，回调自然取得当轮值。若必须保留var，可用IIFE把i作为参数封入独立作用域，或用setTimeout(fn, 0, i)透传实参；后两种更啰嗦，也更依赖调用形式。

370. 执行上下文

执行上下文就是JS执行一段代码时所需的运行环境，核心包含变量对象、作用域链和this。创建阶段会登记变量、函数声明和形参，因此会出现变量提升；执行阶段才逐行完成赋值和调用。全局代码、函数和eval会形成不同上下文，函数调用时，新上下文会压入执行栈，结束后再弹出。函数内部实际通过活动对象AO管理局部变量及arguments，箭头函数没有自己的arguments。

当执行JS代码时，会产生三种执行上下文

全局执行上下文 函数执行上下文 eval执行上下文

每个执行上下文中都有三个重要的属性 变量对象（VO），包含变量、函数声明和函数的形参，该属性只能在全局上下文中访问作用域链（JS采用词法作用域，也就是说变量的作用域是在定义时就决定了）this

对于上述代码，执行栈中有两个上下文：全局上下文和函数上下文。

stack=[globalContext,fooContext] 对于全局上下文来说，VO大概是这样的

globalContext.VO===globe globalContext.VO={a:undefined,foo:;}

对于函数foo来说，VO不能访问，只能访问到活动对象（AO）

fooContext.VO===foo.AOfooContext.AO{i:undefined,b:undefined,arguments:<>}// arguments

是函数独有的对象(箭头函数没有)// 该对象是一个伪数组，有`length`属性且可以通过下标访问元素//

该对象中的`callee`属性代表函数本身//`caller`属性代表函数的调用者 对于作用域链，可以把它理解成

包含自身变量对象和上级变量对象的列表，通过[[Scope]]属性查找上级变量 fooContext. [[Scope]]=[globalContext.VO]fooContext.Scope=fooContext. [[Scope]]+fooContext.VOfooContext.Scope=[foo

Context.VO,globalContext.VO] 接下来让我们看一个老生常谈的例子，var b()// call

bconsole.log(a)// undefinedvara='Hello world'functionb(){console.log('call b')}想必以上的输出大家

肯定都已经明白了，这是因为函数和变量提升的原因。通常提升的解释是说将声明的代码移动到了顶部

，这其实没有什么错误，便于大家理解。但是更准确的解释应该是：在生成执行上下文时，会有两个阶段

。第一个阶段是创建的阶段（具体步骤是创建VO），JS解释器会找出需要提升的变量和函数，并且给他们

提前在内存中开辟好空间，函数的话会将整个函数存入内存中，变量只声明并且赋值为undefined，所以在

第二个阶段，也就是代码执行阶段，我们可以直接提前使用。

在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升 b()// call b

secondfunctionb(){console.log('call b fist')}functionb(){console.log('call b second')}varb='Hello

world' var会产生很多错误，所以在ES6中引入了let。let不能在声明前使用，但是这并不是常说的let不会提升，let提升了声明但没有赋值，因为临时死区导致了并不能在声明前使用。

对于非匿名的立即执行函数需要注意以下一点 `var foo=1(function foo(){foo=10console.log(foo)}())// -> 1`
`foo() { foo = 10; console.log(foo) }` 因为当JS解释器在遇到非匿名的立即执行函数时，会创建一个辅助的特定对象，然后将函数名称作为这个对象的属性，因此函数内部才可以访问到foo，但是这个值又是只读的，所以对它的赋值并不生效，所以打印的结果还是这个函数，并且外部的值也没有发生更改。
`specialObject={};Scope=specialObject+Scope;foo=new FunctionExpression;foo.[[Scope]]=Scope;specialObject.foo=foo;// {DontDelete}, {ReadOnly}delete Scope[0];// remove specialObject from the front of scope chain` 总结 执行上下文可以简单理解为一个对象： 它包含三个部分：变量对象(VO)作用域链(词法...

1) 订单页运行foo()时，调用栈里既有全局代码又有foo，同事却说整个页面只有一个“作用域对象”；你会拿这次调用怎样反驳？

答：页面启动会建立全局执行上下文，调用foo()时还会把新的函数执行上下文压入执行栈，两者不是同一个对象。foo执行期间位于栈顶，全局上下文暂停；函数返回后其上下文出栈，控制权才交回调用方。

2) 你在调试器里检查function foo(i){ var b = 20 }，产品要求解释为什么函数内既能找到i、b，又能读取页面级变量；你会怎么说明查找路径？

答：函数上下文的活动对象会包含形参i、局部变量b和函数独有的arguments，标识符先在这里查找。未命中时再沿词法作用域链访问上级变量对象，直到全局对象；这条链由函数定义位置决定，不会因调用位置改变。

3) 一段遗留脚本把普通函数改成箭头函数后，读取arguments[0]直接失败；在不改变调用参数数量的前提下，根因是什么？

答：普通函数执行上下文具有自己的arguments伪数组，可通过下标和length读取实参；箭头函数没有自己的arguments。若业务仍需收集参数，应显式使用剩余参数，不能假定箭头函数会复制外层普通函数的活动对象。

4) 生产环境出现Maximum call stack size exceeded，调用栈显示renderNode上百层重复，而每层只创建少量局部变量；你如何把故障和执行上下文联系起来排查？

答：每次调用renderNode都会创建函数执行上下文并压栈，递归没有及时结束就会持续占用调用栈，最终溢出。先核对终止条件和输入树是否存在异常回路，再考虑改成显式栈迭代；仅减少局部变量不能消除调用层数。

5) 插件系统允许传入一段eval配置脚本，架构师认为它与普通函数执行完全相同；在执行上下文模型里，你会指出什么差异和代价？

答：eval会产生独立类型的执行上下文，不能简单等同于一次普通函数调用。它还让标识符和作用域关系在运行时才更难确定，增加理解与排查成本；若配置只需要数据，应优先采用结构化输入，避免引入动态代码执行边界。

371. 作用域

作用域决定变量在哪里可访问，而作用域链决定当前找不到变量时去哪里继续找。JS使用词法作用域，链路在函数定义时就确定，调用位置不会把它改掉；查找会从当前作用域逐层向外，最后到全局作用域。常见范围可以归为全局、函数和块级作用域，其中let、const能形成块级作用域并存在TDZ。工程中应尽量缩小变量范围，避免全局变量过多造成命名冲突。作用域：作用域是定义变量的区域，它有一套访问变量的规则，这套规则来管理浏览器引擎如何在当前作用域以及嵌套的作用域中根据变量（标识符）进行变量查找

作用域链：作用域链的作用是保证对执行环境有权访问的所有变量和函数的有序访问，通过作用域链，我们可以访问到外层环境的变量和函数。作用域链的本质是一个指向变量对象的指针列表。变量对象是一个包含了执行环境中所有变量和函数的对象。作用域链的前端始终都是当前执行上下文的变量对象。全局执行上下文的变量对象（也就是全局对象）始终是作用域链的最后一个对象。当我们查找一个变量时，如果当前执行环境中没有找到，我们可以沿着作用域链向后查找作用域链的创建过程跟执行上下文的建立有关....

作用域可以理解为变量的可访问性，总共分为三种类型，分别为：全局作用域、函数作用域、块级作用域，ES6 中的let、const就可以产生该作用域。其实看完前面的闭包、this这部分内部的话，应该基本能了解作用域的一些应用。

一旦我们将这些作用域嵌套起来，就变成了另外一个重要的知识点「作用域链」，也就是 JS 到底是如何访问需要的变量或者函数的。首先作用域链是在定义时就被确定下来的，和箭头函数里的 this 一样，后续不会改变，JS 会一层层往上寻找需要的内容。其实作用域链这个东西我们在闭包小结中已经看到过它的实体了：[[Scopes]]

图中的[[Scopes]]是个数组，作用域的一层层往上寻找就等同于遍历[[Scopes]]。1. 全局作用域
全局变量是挂载在 window

对象下的变量，所以在网页中的任何位置你都可以使用并且访问到这个全局变量

```
varglobalName='global';functiongetName(){console.log(globalName)}//  
globalvarname='inner'console.log(name)// inner}getName();console.log(name);//console.log(globalName);//globalfunctionsetName(){vName='setName';setName();console.log(vName);//  
setName
```

从这段代码中我们可以看到，globalName 这个变量无论在什么地方都是可以被访问到的，所以它就是全局变量。而在 getName 函数中作为局部变量的 name 变量是不具备这种能力的当然全局作用域有相应的缺点，我们定义很多全局变量的时候，会容易引起变量命名的冲突，所以在定义变量的时候应该注意作用域的问题。2. 函数作用域
函数中定义的变量叫作函数变量，这个时候只能在函数内部才能访问到它，所以它的作用域也就是函数的内部，称为函数作用域

```
functiongetName(){varname='inner';console.log(name);//inner}getName();console.log(name);
```

除了这个函数内部，其他地方都是不能访问到它的。同时，当这个函数被执行完之后，这个局部变量也相应会被销毁。所以你会看到在 getName 函数外面的 name 是访问不到的

3. 块级作用域 ES6 中新增了块级作用域，最直接的表现就是新增的 let 关键词，使用 let 关键词定义的变量只能在块级作用域中被访问，有“暂时性死区”的特点，也就是说这个变量在定义之前是不能被使用的。在 JS 编码过程中 if 语句及 for 语句后面 {...} 这里面所包括的，就是块级作用域

```
console.log(a)//a is not definedif(true){leta='123'; console.log(a); // 123}console.log(a)//a is not defined
```

从这段代码可以看出，变量 a 是在 if 语句 {...} 中由 let 关键词进行定义的变量，所以它的作用域是 if 语句括号中的那部分，而在外面进行访问 a 变量是会报错的，因为这里不是它的作用域。所以在 if 代码块的前后输出 a 这个变量的结果，控制台会显示 a 并没有定义

1) 运营页在 if (enabled) { let banner = 'A'

} 后执行 console.log(banner)，同事认为条件为真就应该能读取；你如何解释这个反例？

答：即使条件为真，banner 的可访问范围也只在 if 的花括号内，块执行结束后外部读取仍会报未定义。是否执行过赋值不会扩大词法作用域；若外部确实需要该值，应在外层声明并在块内赋值。

2) 两个通过普通接入的营销组件都声明了顶层 var

status，后加载的组件覆盖前者；在不引入模块系统的旧页面里你会怎样隔离？

答：可把每个组件包进 IIFE，让内部变量落在各自的函数作用域中，只显式暴露必要接口。这样能减少顶层 var 挂到全局对象造成的命名冲突；代价是依赖和导出需要手工组织，不能获得模块系统的静态边界。

3) 代码评审里有人把函数从模块顶部移动到点击处理器内部，理由是“调用位置没变，查到的变量也不会变”；若内外层都有theme，结果会怎样变化？

答：函数的作用域链由定义位置决定，移动定义位置可能改变它捕获的theme，即使调用语句完全不变。查找会从函数自身环境逐层向外，命中最近的同名绑定即停止；因此这种重构必须核对变量遮蔽，而不能只比较调用点。

4) 线上日志出现vName is not defined，代码只在某个开关分支里执行vName = 'setName'，另一个分支随后读取它；你会先检查哪些作用域条件？

答：先确认赋值分支是否真正执行，以及脚本是否处于允许隐式创建全局变量的运行环境。未声明赋值不能作为可靠的跨分支通信方式，严格约束下还可能直接报错；应在明确的外层作用域声明状态，并保证所有读取路径都有初始化。

5) 一个页面需要让计数器只被组件内部使用，团队在“顶层全局变量”和“函数作用域封装”之间争论；你会如何取舍？

答：应选函数作用域封装，因为全局变量位于作用域链末端，页面各处都可能访问或覆盖，命名冲突面更大。封装后只返回必要操作函数，让内部计数保持不可直接访问；代价是外部调试和修改必须经过暴露的接口。

372. 闭包

闭包本质上是函数保留了对其定义时外层作用域的引用，因此外层函数结束后仍能访问相关变量。它常用于封装私有变量，定时器、事件监听和作为参数传递的回调也会自然形成这种引用关系。比如循环里用var配合setTimeout，回调执行时共享的i已变成6，所以会连续输出6。改用let、IIFE，或把当前值作为定时器参数传入，都能让每次回调拿到对应的值。闭包其实就是一个可以访问其他函数内部变量的函数。创建闭包的最常见的方式就是在一个函数内创建另一个函数，创建的函数可以访问到当前函数的局部变量。因为通常情况下，函数内部变量是无法在外部访问的（即全局变量和局部变量的区别），因此使用闭包的作用，就具备实现了能在外部访问某个函数内部变量的功能，让这些内部变量的值始终可以保存在内存中。下面我们通过代码先来看一个简单的例子

```
function fun1(){var a=1;return function(){console.log(a);}};fun1();var result=fun1();result();// 1//
```

结合闭包的概念，我们把这段代码放到控制台执行一下，就可以发现最后输出的结果是 1（即 a 变量的值）。那么可以很清楚地发现，a 变量作为一个 fun1 函数的内部变量，正常情况下作为函数内的局部变量，是无法被外部访问到的。但是通过闭包，我们最后还是可以拿到 a 变量的值。闭包有两个常用的用途。闭包的第一个用途是使我们在函数外部能够访问到函数内部的变量。通过使用闭包，我们可以通过在外部调用闭包函数，从而在外部访问到函数内部的变量，可以使用这种方法来创建私有变量。函数的另一个用途是使已经运行结束的函数上下文中的变量对象继续留在内存中，因为闭包函数保留了这个变量对象的引用，所以这个变量对象不会被回收。其实闭包的本质就是作用域链的一个特殊的应用，只要了解了作用域链的创建过程，就能够理解闭包的实现原理。

```
let a=1// fn
是闭包function fn(){console.log(a);}function fn1(){let a=1//
```

```
这里也是闭包return()=>{console.log(a);}const fn2=fn1()fn2()
```

大家都知道闭包其中一个作用是访问私有变量，就比如上述代码中的 fn2 访问到了 fn1 函数中的变量 a。但是此时 fn1 早已销毁，我们是如何访问到变量 a 的呢？不是都说原始类型是存放在栈上的么，为什么此时却没有被销毁掉？接下来笔者会根据浏览器的表现来重新理解关于原始类型存放位置的说法。先来说下数据存放的正确规则是：局部、占用空间确定的数据，一般会存放在栈中，否则就在堆中（也有例外）。

那么接下来我们可以通过 Chrome 来帮助我们验证这个说法。上图中画红框的位置我们能看到一个内部的对象[[Scopes]]，其中存放着变量 a，该对象是被存放在堆上的，其中包含了闭包、全局对象等等内容，因此我们能通过闭包访问到本该销毁的变量。另外最开始我们对

于闭包的定位是：假如一个函数能访问外部的变量，那么这个函数它就是一个闭包，因此接下来我们看看在全局下的表现是怎么样。 `leta=1varb=2// fn` 是闭包 `functionfn(){console.log(a,b);}` 从上图我们能发现全局下声明的变量，如果是 `var` 的话就直接被挂到 `globe` 上，如果是其他关键字声明的话就被挂到 `Script` 上。虽然这些内容同样还是存在 `[[Scopes]]`，但是全局变量应该是存放在静态区域的，因为全局变量无需进行垃圾回收，等需要回收的时候整个应用都没了。只有在下图的场景中，原始类型才可能是被存储在栈上。这里为什么要说可能，是因为 JS 是门动态类型语言，一个变量声明时可以是原始类型，马上又可以赋值为对象类型，然后又回到原始类型。这样频繁的在堆栈上切换存储位置，内部引擎是不是也会有什么优化手段，或者干脆全部都丢堆上？只有 `const` 声明的原始类型才一定存在栈上？当然这只是笔者的一个推测，暂时没有深究，读者可以忽略这段瞎想。因此笔者对于原始类型存储位置的理解为：局部变量才是被存储在栈上，全局变量存在静态区域上，其它都存储在堆上。当然这个理解是建立在 `Chrome` 的表现之上的，在不同的浏览器上因为引擎的不同，可能存储的方式还是有所变化的。闭包产生的原因我们在前面介绍了作用域的概念，那么你还需明白作用域链的基本概念。其实很简单，当访问一个变量时，代码解释器会首先在当前的作用域查找，如果没找到，就去父级作用域去查找，直到找到该变量或者不存在父级作用域中，这样的链路就是作用域链。需要注意的是，每一个子函数都会拷贝上级作用域，形成一个作用域的链条。那么我们还是通过下面的代码来详细说明一下作用域链 `var...`

1) 搜索页的工厂函数已经返回，但它返回的 `getKeyword` 仍能打印函数内的 `keyword`；同事据此断言“父函数根本没有结束”，你怎么纠正？

答：父函数的调用已经结束并从执行栈退出，保留下来的是内部函数对其词法环境的引用。只要 `getKeyword` 仍可达，它需要的 `keyword` 就不会被回收；这不等于整个调用过程仍在执行，也不意味着所有局部数据都必须保留。

2) 你要为支付组件保存一个不能被外部直接改写的重试次数，团队不允许把它挂到全局对象；闭包方案应怎样设计？

答：可在工厂函数内声明计数变量，并只返回读取、递增或重置它的函数，外部通过这些闭包访问状态。变量不暴露到全局，调用结束后仍因闭包引用而保留；接口一旦泄露给长期存活对象，该状态的生命周期也会随之延长。

3) 通知中心用 `var i` 给五个按钮注册延迟回调，点击后全部显示 6；如果暂时不能改循环声明，怎样修复且保持输出 1 到 5？

答：用 IIFE 在每轮把当前 `i` 作为参数传入，使每个回调引用不同的局部绑定；也可通过 `setTimeout` 的第三个参数把当轮值传给回调。两者都避开共享的全局 `i`，但前者增加一层函数，后者依赖定时器的传参接口。

4) 单页应用切换路由十次后内存持续增加，旧页面的按钮节点和定时器回调仍能在堆快照中找到；你会怎样验证是否由闭包引用造成？

答：检查未解绑的事件监听器、未清理的定时器及其回调，因为这些闭包可能继续引用页面函数作用域中的变量。沿保留引用链确认旧节点或大对象是否被回调的作用域链持有，再在卸载阶段解除监听和清除定时器；闭包本身并不等于泄漏，关键是引用是否仍有业务必要。

5) 数据面板既可用全局对象保存筛选状态，也可返回一组闭包函数维护私有状态；多人协作下你会选哪一种？

答：若状态只服务单个面板，闭包更适合限制访问范围，并能让状态在创建函数结束后继续存在。全局对象便于直接观察和跨模块共享，却更容易被任意代码覆盖；闭包的代价是生命周期不直观，需要控制返回函数被谁长期持有。

6) 异步请求回调能读取发起请求时函数里的requestId，但回调是在函数返回后才执行；这与作用域链有什么关系？

答：回调函数在定义时就形成了对外层词法环境的引用，因此稍后执行仍能沿作用域链找到requestId。异步机制只改变回调执行时机，不会把标识符改为按调用位置查找；若外层绑定被多个回调共享，读取到的也会是该共享绑定当时的值。

373. New的原理

new本质上会创建对象、连接构造函数原型、绑定this并决定最终返回值。构造函数执行时，新增的属性会写到这个实例上，使实例既能访问自身属性，也能沿原型链访问共享方法。若构造函数显式返回对象，结果会被该对象替代；返回字符串等非对象值时，仍返回原实例。手写时可用Object.create()建立原型关系，再通过apply()执行构造函数并处理返回值。 常见考点

new做了那些事？new返回不同的类型时会有什么表现？手写 new 的实现过程 new 关键词的主要作用就是执行一个构造函数、返回一个实例对象，在 new 的过程中，根据构造函数的情况，来确定是否可以接受参数的传递。下面我们通过一段代码来看一个简单的 new 的例子

```
function Person(){this.name='Jack';}var p=new Person();console.log(p.name)// Jack
```

这段代码比较容易理解，从输出结果可以看出，p 是一个通过 person 这个构造函数生成的一个实例对象，这个应该很容易理解。

new操作符可以帮助我们构建出一个实例，并且绑定上 this，内部执行步骤可大概分为以下几步：创建一个新对象对象连接到构造函数原型上，并绑定this（this 指向新对象）执行构造函数代码（为这个新对象添加属性）返回新对象

在第四步返回新对象这边有一个情况会例外：那么问题来了，如果不用new这个关键词，结合上面的代码改造一下，去掉new，会发生什么样的变化呢？我们再来看下面这段代码

```
function Person(){this.name='Jack';}var p=Person();console.log(p)// undefined
console.log(name)// Jack
console.log(p.name)// 'name' of undefined
```

从上面的代码中可以看到，我们没有使用new这个关键词，返回的结果就是undefined。其中由于JavaScript代码在默认情况下this的指向是window，那么name的输出结果就为Jack，这是一种不存在new关键词的情况。那么当构造函数中有return一个对象的操作，结果又会是什么样子呢？我们再来看一段在上面的基础上改造过的代码。

```
function Person(){this.name='Jack';return{age:18};}var p=new Person();console.log(p){age:18}
console.log(p.name)// undefined
console.log(p.age)// 18
```

通过这段代码又可以看出，当构造函数最后return出来的是一个和this无关的对象时，new 命令会直接返回这个新对象，而不是通过 new 执行步骤生成的 this 对象。但是这里要求构造函数必须是返回一个对象，如果返回的不是对象，那么还是会按照 new 的实现步骤，返回新生成的对象。接下来还是在这段代码的基础之上稍微改动一下

```
function Person(){this.name='Jack';return'tom';}var p=new Person();console.log(p){name:'Jack'}
console.log(p.name)// Jack
```

可以看出，当构造函数中return的不是一个对象时，那么它还是会根据new关键词的执行逻辑，生成一个新的对象（绑定了最新this），最后返回出来。因此我们总结一下：new 关键词执行之后总是会返回一个对象，要么是实例对象，要么是 return 语句指定的对象

手工实现New的过程

```
function create(fn,...args){if(typeof fn !== 'function'){throw 'fn must be a function'}}
1、用new Object() 的方式新建了一个对象obj
var obj = new Object()
2、给该对象的__proto__赋值为fn.prototype，即设置原型链
obj.__proto__ = fn.prototype
1、2步骤合并
创建一个空对象，且这个空对象继承构造函数的 prototype 属性
即实现 obj.__proto__ === constructor.prototype
var obj=Object.create(fn.prototype);
```

3、执行fn，并将obj作为内部this。使用 apply，改变构造函数 this 的指向到新建的对象，这样 obj 就可以访问到构造函数中的属性

```
var res=fn.apply(obj,args);
4、如果fn有返回值，则将其...
```

1) 用户中心执行new Person()后能读到实例属性和Person.prototype上的方法，同事却说new只是调用了一次函数；你会用对象关系怎样反驳？

答：new不只是调用函数，它还创建新对象，并把该对象连接到Person.prototype，再以新对象作为this执行构造函数。构造函数写入的是实例自身属性，原型方法则通过原型链访问；普通调用不会自动完成这些关系。

2) 遗留代码里Person()漏写new，返回值变成undefined，同时页面出现了全局name；你会怎样阻止同类污染？

答：在该非严格示例中，普通调用没有创建实例，构造函数默认返回undefined，而this.name写到了全局对象。工程上应统一构造入口并检查调用方式，或使用必须通过new调用的构造形式；仅依靠调用者记忆仍可能再次遗漏。

3) 缓存模块的构造函数执行了this.key = 'A'，随后return { key: 'B' }；调用方要求实例必须保留原型方法，实际会发生什么？

答：显式返回对象会取代原本创建的实例，因此调用方拿到{ key: 'B' }，也不会自动保留构造函数原型链上的方法。若必须返回标准实例，就不要返回无关对象，或确保返回对象具有所需原型关系；返回原始值则会被忽略。

4) 团队手写的create(Constructor, ...args)生成对象后，实例读不到原型方法；你会按new的哪几个环节定位？

答：先确认是否用Object.create(Constructor.prototype)建立了原型连接，再检查是否以新对象为this调用了构造函数并传入参数。最后核对返回逻辑：构造函数返回对象时采用该对象，否则返回创建的实例；缺少任一环节都会偏离new的行为。

5) 评审手写new时，一版用obj.__proto__ = fn.prototype，另一版用Object.create(fn.prototype)；你会选择哪版并说明边界？

答：应优先使用Object.create(fn.prototype)，它在创建对象时直接建立所需的原型关系，表达也更贴近目标。随后仍需用apply绑定this并处理构造函数的显式返回值；只完成原型连接并不能等价实现完整的新。

6) 构造函数返回null、字符串或普通对象时，测试负责人要求列出三种调用结果；你的断言应该是什么？

答：返回普通对象时，new的结果被该对象替代；返回字符串等原始值时，结果仍是新创建的实例。null虽然历史上typeof表现特殊，但不是可替代实例的对象返回值，因此也应保留实例；手写实现不宜只靠含糊的类型判断。

374. 原型/原型链

原型链就是对象通过原型逐级连接起来的属性查找链，核心关系是实例.__proto__ === 构造函数.prototype。prototype用来保存实例共享的属性和方法，原型上的constructor通常再指回构造函数。访问属性时会先查对象自身，再沿原型向上查找，直到Object.prototype，仍未找到就得到undefined。工程中获取原型优先使用Object.getPrototypeOf()，不建议直接依赖非标准的__proto__。

__proto__和prototype关系：__proto__和constructor是对象独有的。2prototype属性是函数独有的在js中我们使用构造函数来新建一个对象的，每一个构造函数的内部都有一个prototype属性值，这个属性值是一个对象，这个对象包含了可以由该构造函数的所有实例共享的属性和方法。当我们使用构造函数新建一个对象后，在这个对象的内部将包含一个指针，这个指针指向构造函数的prototype属性对应的值，在ES5

中这个指针被称为对象的原型。一般来说我们是不应该能够获取到这个值的，但是现在浏览器中都实现了

proto 属性来让我们访问这个属性，但是我们最好不要使用这个属性，因为它不是规范中规定的。ES5 中新增了一个Object.getPrototypeOf()方法，我们可以通过这个方法来获取对象的原型。当我们访问一个对象的属性时，如果这个对象内部不存在这个属性，那么它就会去它的原型对象里找这个属性，这个原型对象又会有自己的原型，于是就这样一直找下去，也就是原型链的概念。原型链的尽头一般来说都是Object.prototype所以这就是我们新建的对象为什么能够使用toString()等方法的原因。特点：JavaScript 对象是通过引用来传递的，我们创建的每个新对象实体中并没有一份属于自己的原型副本。当我们修改原型时，与之相关的对象也会继承这一改变 原型(prototype): 一个简单的对象，用于实现对象的 属性继承。可以简单的理解成对象的爹。在Firefox和Chrome中，每个JavaScript对象中都包含一个__proto__(非标准)的属性指向它爹(该对象的原型)，可obj.__proto__进行访问。构造函数: 可以通过new来 新建一个对象的函数。实例: 通过构造函数和new创建出来的对象，便是实例。

实例通过__proto__指向原型，通过constructor指向构造函数。

以Object为例，我们常用的Object便是一个构造函数，因此我们可以通过它构建实例。 // 实例const instance=new Object() 则此时， 实例为instance, 构造函数为Object，我们知道，构造函数拥有一个prototype的属性指向原型，因此原型为: // 原型const prototype=Object.prototype 这里我们可以来看出三者的关系: 实例.__proto__ === 原型原型.constructor === 构造函数构造函数.prototype === 原型 // 这条线其实是基于原型进行获取的，可以理解成一条基于原型的映射线// 例如:// const o = new Object()// o.constructor === Object --> true// o.__proto__ = null// o.constructor === Object --> false实例.constructor===构造函数 原型链 原型链是由原型对象组成，每个对象都有__proto__属性，指向了创建该对象的构造函数的原型，__proto__将对象连接起来组成了原型链。是一个用来实现继承和共享属性的有限的对象链 属性查找机制: 当查找对象的属性时，如果实例对象自身不存在该属性，则沿着原型链往上一级查找，找到时则输出，不存在时，则继续沿着原型链往上一级查找，直至最顶级的原型对象Object.prototype，如还是没找到，则输出undefined；属性修改机制: 只会修改实例对象本身的属性，如果不存在，则进行添加该属性，如果需要修改原型的属性时，则可以用:b.prototype.x = 2；但是这样会造成所有继承于该对象的实例的属性发生改变。 js 获取原型的方法 p.prototype.constructor.prototypeObject.getPrototypeOf(p) 总结 每个函数都有prototype属性，除了Function.prototype.bind()，该属性指向原型。每个对象都有__proto__属性，指向了创建该对象的构造函数的原型。其实这个属性指向了[[pr...

1) 控制台里user.name能读到值，但Object.keys(user)和接口序列化都没有name，你会怎么解释这个反例？

答：name很可能不在user自身，而是沿原型链从原型对象上找到的。用Object.hasOwn(user, 'name')判断自有属性，再用Object.getPrototypeOf(user)逐层定位；依赖继承属性做序列化会产生遗漏。

2) 组件库想让所有实例共享一份格式化方法，你会把方法放在构造函数里，还是放到prototype上？

答：共享且无实例状态的方法应放到构造函数的prototype上，实例通过原型链复用同一个函数。若在构造函数中逐个创建方法，每个实例都会持有独立函数；但会被实例改写的方法不适合直接共享。

3) 线上热修复把Widget.prototype.render换成了新函数，已创建的几百个组件会立即使用新版吗？

答：只要实例自身没有同名render，后续查找就会命中新原型方法，因此已有实例也会受到影响。若某个实例已定义自身的render，它会遮蔽原型实现；修改共享原型属于全局影响操作，需要评估兼容性。

4) 调试时看到item.constructor !==

Item，同事据此认定对象不是Item实例，你会怎么排查？

答：不能只依赖constructor，因为它通常也是沿原型链取得的普通属性，可能被覆盖或因重设原型而丢失。应同时检查Object.getPrototypeOf(item)、Item.prototype和item instanceof Item；原型链被人为改写时，这些结果也会随之变化。

5) 旧代码大量使用obj.__proto__改继承关系，新模块要继续沿用还是改成标准API？

答：新模块应使用Object.getPrototypeOf读取原型，用Object.create建立所需原型关系，避免继续依赖__proto__。__proto__暴露的是内部[[Prototype]]的访问入口，但直接改链会让属性来源和实例判断更难追踪。

375. 继承

JavaScript继承的核心是连接原型并初始化父类属性，常用结果是寄生组合继承或class extends。单独借用构造函数无法继承原型方法，单独使用原型链又会让多个实例共享引用类型属性。组合继承解决了这两个问题，但父构造函数会执行两次；用Object.create(Parent.prototype)可避免这次多余调用，并修正constructor。class本质上仍基于原型，子类构造函数需要调用super()，静态继承还涉及构造函数本身的原型关系。

涉及面试题：原型如何实现继承？Class如何实现继承？Class本质是什么？

首先先来讲下class，其实在JS中并不存在类，class只是语法糖，本质还是函数

class Person {} Person instanceof Function // true 组合继承 组合继承是最常用的继承方式 function Parent(value) { this.val = value } Parent.prototype.getValue = function () { console.log(this.val) } function Child(value) { Parent.call(this, value) } Child.prototype = new Parent() const child = new Child(1) child.getValue() // 1 child instanceof Parent // true 以上继承的方式核心是在子类的构造函数中通过Parent.call(this)继承父类的属性，然后改变子类的原型为new Parent()来继承父类的函数。这种继承方式优点在于构造函数可以传参，不会与父类引用属性共享，可以复用父类的函数，但是也存在一个缺点就是在继承父类函数的时候调用了父类构造函数，导致子类的原型上多了不需要的父类属性，存在内存上的浪费 寄生组合继承 这种继承方式对组合继承进行了优化，组合继承缺点在于继承父类函数时调用了构造函数，我们只需要优化掉这点就行了 function Parent(value) { this.val = value } Parent.prototype.getValue = function () { console.log(this.val) } function Child(value) { Parent.call(this, value) } Child.prototype = Object.create(Parent.prototype, { constructor: { value: Child, enumerable: false, writable: true, configurable: true } }) const child = new Child(1) child.getValue() // 1 child instanceof Parent // true 以上继承实现的核心就是将父类的原型赋值给了子类，并且将构造函数设置为子类，这样既解决了无用的父类属性问题，还能正确的找到子类的构造函数。 Class 继承 以上两种继承方式都是通过原型去解决的，在 ES6 中，我们可以使用 class 去实现继承，并且实现起来很简单 class Parent { constructor(value) { this.val = value } getValue() { console.log(this.val) } } class Child extends Parent { constructor(value) { super(value) this.val = value } } let child = new Child(1) child.getValue() // 1 child instanceof Parent // true class实现继承的核心在于使用extends表明继承自哪个父类，并且在子类构造函数中必须调用super，因为这段代码可以看成Parent.call(this, value)。 ES5 和 ES6 继承的区别： ES6 继承的子类需要调用super()才能拿到子类，ES5的话是通过apply这种绑定的方式类声明不会提升，和let这些一致 function Super() {} Super.prototype.getNumber = function () { return 1 } function Sub() {} Sub.prototype = Object.create(Super.prototype, { constructor: { value: Sub, enumerable: false, writable: true, configurable: true } }) let s = new Sub() s.getNumber() 以下详细讲解几种常见的继承方式 1. 方式1: 借助call function Parent1() { this.name = 'paren...

1) 有人只在Child里写Parent.call(this)，实例属性都有了，却调用child.getValue()报错，缺的是哪条继承关系？

答：Parent.call(this)只把父构造函数中的实例属性初始化到子实例上，不会让Child.prototype继承Parent.prototype。应再用Object.create(Parent.prototype)建立原型链并修正constructor，否则父类原型方法不可见。

2) 表单模型有数组字段，多个Child实例修改其中一个的选项后全部联动，原型链代码该从哪里查？

答：先检查是否写了Child.prototype = new Parent()，从而把父构造函数创建的数组留在共享原型上。子构造函数中调用Parent.call(this)可让每个实例获得独立引用，再用寄生组合继承复用父类方法。

3) 父构造函数新增了日志注册逻辑后，每创建一个子实例却出现额外初始化记录，组合继承为什么会放大这个副作用？

答：传统组合继承在设置Child.prototype = new Parent()时执行一次父构造函数，创建子实例时又通过Parent.call(this)执行一次。改为Object.create(Parent.prototype)可避免为建原型而调用构造函数，也不会在子类原型上残留无用实例属性。

4) 把Child.prototype直接赋成Parent.prototype后，修复子类constructor却连父类实例的constructor也变了，原因是什么？

答：两者此时引用同一个原型对象，修改Child.prototype.constructor等同于修改Parent.prototype.constructor。应让子类拥有以父原型为原型的新对象，即Child.prototype = Object.create(Parent.prototype)，再单独恢复不可枚举的constructor。

5) 把ES5继承迁成class Child extends

Parent后，父类静态校验方法也能从Child调用，原来的寄生组合代码为什么做不到？

答：传统寄生组合继承通常只连接Child.prototype与Parent.prototype，因此只覆盖实例侧继承。extends还会建立构造函数之间的继承关系，使子类可访问父类静态成员；手写方案需要额外设置Child的原型，但会增加兼容和维护成本。

376. 面向对象

面向对象是用对象组织状态和行为，并通过封装、继承、多态与抽象控制代码关系的一种编程思想。封装可以借助闭包和局部变量隐藏实现，只向外暴露必要方法；继承则可通过原型链与父构造函数调用复用属性和行为。多态强调同一父级下的不同子对象可以呈现不同形态，抽象类还可在构造器或待实现方法中抛错，避免被直接使用。对象拷贝也要注意边界：浅拷贝会共享引用类型，确需隔离时才使用递归深拷贝。

编程思想 基本思想是使用对象，类，继承，封装等基本概念来进行程序设计优点易维护采用面向对象思想设计的结构，可读性高，由于继承的存在，即使改变需求，那么维护也只是在局部模块，所以维护起来是非常方便和较低成本的易扩展开发工作的重用性、继承性高，降低重复工作量。缩短了开发周期 易维护采用面向对象思想设计的结构，可读性高，由于继承的存在，即使改变需求，那么维护也只是在局部模块，所以维护起来是非常方便和较低成本的易扩展开发工作的重用性、继承性高，降低重复工作量。缩短了开发周期 采用面向对象思想设计的结构，可读性高，由于继承的存在，即使改变需求，那么维护也只是在局部模块，所以维护起来是非常方便和较低成本的

一般面向对象包含：继承，封装，多态，抽象

1.

对象形式的继承 浅拷贝

```
var Person = {name: 'poetry', age: 18, address: {home: 'home', office: 'office'}, schools: ['x', 'z']};
var programmer = {language: 'js'};
function extend(p, c) {
  var c = c || {};
  for (var prop in p) {
    c[prop] = p[prop];
  }
  extend(Person, programmer);
  programmer.name = 'poetry';
  programmer.address.home = 'house';
  // house
  Person.address.home = 'house';
  // house
}
```

从上面的结果看出，浅拷贝的缺陷在于修改了子对象中引用类型的值，会影响到父对象中的值，因为在浅拷贝中对引用类型的拷贝只是拷贝了地址，指向了内存中同一个副本

深拷贝

```
function extendDeeply(p, c) {
  var c = c || {};
  for (var prop in p) {
    if (typeof p[prop] === 'object') {
      c[prop] = (p[prop].constructor === Array) ? [] : {};
      extendDeeply(p[prop], c[prop]);
    } else {
      c[prop] = p[prop];
    }
  }
}
```


利用递归进行深拷贝，这样子对象的修改就不会影响到父对象

```
extendDeeply(Person,programer);programer.address.home='poetry';Person.address.home;//
home 利用call和apply继承 functionParent(){this.name="abc";this.address={home:"home"}};function
Child(){Parent.call(this);this.language="js";} ES5中的Object.create()
varp={name:'poetry'};varobj=Object.create(p);obj.name;// poetry
Object.create()作为new操作符的替代方案是ES5之后才出来的。我们也可以自己模拟该方法： //模拟O
bject.create()方法functionmyCreate(o){functionF(){}F.prototype=o;o=newF();returno;}varp={na
me:'poetry'};varobj=myCreate(p);obj.name;// poetry 目前，各大浏览器的最新版本（包括IE9）都部
署了这个方法。如果遇到老式浏览器，可以用下面的代码自行部署
if(!Object.create){Object.create=function(o){functionF(){}F.prototype=o;returnnewF();}} 2.
类的继承 Object.create() functionPerson(name,age){Person.prototype.headCount=1;Person.pro
totype.eat=function(){console.log('eating...');}functionProgrammer(name,age,title){Programmer
.prototype=Object.create(P...
```

1) 商品模板用浅拷贝生成两个编辑对象，修改其中一个的address.home后另一个也变了，为什么顶层字段却正常？

答：浅拷贝只复制顶层值，嵌套对象仍是同一引用，所以修改address.home会反映到所有副本。需要隔离嵌套状态时应做与数据结构匹配的深拷贝；简单递归还要谨慎处理null、特殊对象和循环引用。

2) 插件系统要求外部只能调用start，不能直接改内部令牌，ES5环境下你会怎样封装？

答：可把令牌保存在构造函数或立即执行函数的局部变量中，只通过闭包暴露start等受控方法。这样私有数据不挂在实例上，外部无法直接读写；代价是每个实例可能创建自己的闭包方法，也不便于原型复用。

3) 一个页面要创建上万个轻量模型，当前构造函数为每个实例定义save方法，你会怎样调整对象设计？

答：若save不依赖私有闭包变量，应把它移到构造函数的prototype上，让所有实例共享同一个方法。若它必须访问局部私有状态，则不能直接迁移而不改变封装方式，需要在共享方法与私有性之间取舍。

4) 重构后programmer.work()能执行，但内部调用父类eat时this.name变成了undefined，你会检查哪一行？

答：应检查父类方法是否以Person.prototype.eat.apply(this, arguments)或等价方式调用。若把方法取出后直接执行，调用上下文可能丢失，父类方法便读不到当前子实例属性；显式绑定会增加耦合，但能保持预期接收者。

5) 团队为三种检测器设计继承树，但业务只要求它们都实现detect和stop，你会坚持抽象父类吗？

答：只有共享实现或需要统一约束时，抽象父类才有明显价值，可在基类构造器或未实现的方法中抛错以阻止误用。若只是能力相同而状态与实现无关，深继承会增加耦合，组合独立对象或约定统一接口通常更易扩展。

377. 事件机制

DOM事件触发时会依次经历捕获、目标和冒泡三个阶段。捕获从根节点向目标元素传播，冒泡则从目标元素向外返回，addEventListener的第三个参数决定监听器注册在哪个阶段。事件委托本质上是利用冒泡，把多个子元素同类事件交给父元素处理，适合列表这类节点较多或动态增删的场景。需要中断传播时用stopPropagation()，若还要阻止同一元素上的其他监听器，则用stopImmediatePropagation()；取消默

认行为用preventDefault()。涉及面试题：事件的触发过程是怎么样的？知道什么是事件代理嘛？1. 简介
事件流是一个事件沿着特定数据结构传播的过程。冒泡和捕获是事件流在DOM中两种不同的传播方法
事件流有三个阶段 事件捕获阶段处于目标阶段事件冒泡阶段 事件捕获 事件捕获（event capturing）：通俗的理解就是，当鼠标点击或者触发dom事件时，浏览器会从根节点开始由外到内进行事件传播，即点击了子元素，如果父元素通过事件捕获方式注册了对应的事件的话，会先触发父元素绑定的事件 事件冒泡
事件冒泡（dubbed

bubbling）：与事件捕获恰恰相反，事件冒泡顺序是由内到外进行事件传播，直到根节点

无论是事件捕获还是事件冒泡，它们都有一个共同的行为，就是事件传播 2. 捕获和冒泡 letdiv1=document.getElementById('div1');letdiv2=document.getElementById('div2');div1.onClick=function(){alert('1')}div2.onClick=function(){alert('2');} 当点击div2时，会弹出两个弹出框。在ie8/9/10、chrome浏览器，会先弹出“2”再弹出“1”，这就是事件冒泡：事件从最底层的节点向上冒泡传播。事件捕获则跟事件冒泡相反 W3C的标准是先捕获再冒泡，addEventListener的第三个参数决定把事件注册在捕获（true）还是冒泡(false) 3. 事件对象 4. 事件流阻止 在一些情况下需要阻止事件流的传播，阻止默认动作的发生 event.preventDefault()：取消事件对象的默认动作以及继续传播。event.stopPropagation()/

event.cancelBubble = true：阻止事件冒泡。 事件的阻止在不同浏览器有不同处理 在IE下使用event.returnValue= false，在非IE下则使用event.preventDefault()进行阻止

preventDefault与stopPropagation的区别 preventDefault告诉浏览器不用执行与事件相关联的默认动作（如表单提交）stopPropagation是停止事件继续冒泡，但是对IE9以下的浏览器无效 5. 事件注册 通常我们使用addEventListener注册事件，该函数的第三个参数可以是布尔值，也可以是对象。对于布尔值useCapture参数来说，该参数默认值为false。useCapture决定了注册的事件是捕获事件还是冒泡事件一般来说，我们只希望事件只触发在目标上，这时候可以使用stopPropagation来阻止事件的进一步传播。通常我们认为stopPropagation是用来阻止事件冒泡的，其实该函数也可以阻止捕获事件。stopImmediatePropagation（DOM3级新增事件）

同样也能实现阻止事件冒泡和捕获，但是还能阻止该事件目标执行别的注册事件

stopImmediatePropagation()和stopPropagation()的区别在哪儿呢？后者只会阻止冒泡或者是捕获。

但是前者除此之外还会阻止该元素的其他事件发生，但是后者就不会阻止其他事件的发生。 node.addEventListener('click',(event)=>{event.stopImmediatePropagation()console.log('冒泡')},false);node.addEventListener('click',(event)=>{console.log('冒泡2')},false) 6. 事件委托

在js中性能优化的其中一个主要思想是减少dom操作。节省内存不需要给子节点注销事件 假设有100个li，每个li有相同的点击事件。如果为每个li都添加事件，则会造成dom访问次数过多，引起浏览器重绘与重排的次数过多，性能则会降低。使用事件委托则可以解决这样的问题 原理 实现事件委托是利用了事件的冒泡原理实现的。当我们为最外层的节点添加点击事件，那么里面的ul、li、a的点击事件都会冒泡到最外层节点上，委托...

1) 弹窗里的链接点击后既跳转页面又触发外层关闭逻辑，同事只调用了preventDefault()，为什么弹窗仍会关闭？

答：preventDefault()取消的是链接跳转等默认动作，不会停止事件沿DOM传播，因此外层监听仍会执行。若确实不应触发外层逻辑，需要调整外层判断或调用stopPropagation()；随意停止传播会破坏依赖冒泡的其他功能。

2) 商品列表有数百个动态增删的条目，每项都有同样的收藏按钮，你会把监听器挂在哪里？

答：可在稳定的列表容器上注册一个冒泡阶段监听器，根据事件目标定位对应条目并执行收藏逻辑。事件委托减少逐项绑定和注销，也能覆盖后来插入的节点；不冒泡的事件或传播被中途阻止时不能直接套用。

3) 埋点要求在子组件调用stopPropagation()后仍尽量观察点击，你会选择捕获还是冒泡阶段注册？

答：可在更外层以addEventListener的捕获选项注册，因为事件会先从根向目标传播，再进入目标和冒泡阶段。若事件在更早的捕获节点已被停止，监听仍可能收不到；捕获监听也不能替代对业务事件边界的明确设计。

4) 同一个按钮注册了两个click监听，第一个执行了stopPropagation()，第二个仍然打印日志，这是浏览器异常吗？

答：不是，stopPropagation()只阻止事件继续传播到其他节点，不会取消当前节点上其余监听器。若必须阻止同一目标的后续监听，应使用stopImmediatePropagation()；这会让监听器之间产生强顺序依赖，排查成本较高。

5) 线上委托监听偶尔拿到内部图标节点，导致用nodeName === 'li'的分支失效，你会怎样修复并验证传播链？

答：点击嵌套元素时，事件目标是实际命中的最深节点，不保证就是承载业务数据的li。应从目标向上寻找符合条件且仍位于委托容器内的节点，并检查沿途是否调用了传播阻止方法；选择器过宽还可能误匹配容器外结构。

378. 模块化

现在谈JavaScript模块化，核心就是比较ESM和CommonJS，AMD、CMD作为历史背景了解即可。ESM的依赖关系可在编译期静态分析，因此构建工具能做Tree Shaking，导入值也是实时绑定；CommonJS则通过require在运行时加载，拿到的是module.exports导出的值。加载方式上，CommonJS更适合服务端的同步读取，ESM更适合浏览器和现代构建工具。维护老项目时可能遇到require.js或sea.js，但新项目一般优先使用import、export。

版本说明（面试主线）：下面列的四种方案里，现在的主线只有两个——浏览器和构建工具用ES Module（ESM，import/export），Node

端历史上用CommonJS（require/module.exports）、现也全面支持ESM。AMD（require.js）和CMD（sea.js）是ES Module出现之前、为解决浏览器异步加载模块而生的社区方案，如今已基本退出历史舞台，了解它们主要是为理解「模块化的演进」和偶尔维护老项目。面试回答模块化，应以「ESM vs CommonJS 的区别」为主线（ESM 编译期静态分析、可 Tree Shaking、实时绑定、异步；CJS 运行时加载、值拷贝、同步），把AMD/CMD作为历史背景一笔带过即可。

js中现在比较成熟的有四种模块加载方案：第一种是CommonJS方案，它通过require来引入模块，通过module.exports定义模块的输出接口。这种模块加载方案是服务器端的解决方案，它是以同步的方式来引入模块的，因为在服务端文件都存储在本地磁盘，所以读取非常快，所以以同步的方式加载没有问题。但如果是在浏览器端，由于模块的加载是使用网络请求，因此使用异步加载的方式更加合适。第二种是AMD方案，这种方案采用异步加载的方式来加载模块，模块的加载不影响后面语句的执行，所有依赖这个模块的语句都定义在一个回调函数里，等到加载完成后再执行回调函数。require.js实现了AMD规范第三种是CMD方案，这种方案和AMD方案都是为了解决异步模块加载的问题，sea.js实现了CMD规范。它和require.js的区别在于模块定义时对依赖的处理不同和对依赖模块的执行时机的处理不同。第四种方案是ES6提出的方案，使用import和export的形式来导入导出模块

在有Babel的情况下，我们可以直接使用ES6的模块化 // file

```
a.js export function a() {} export function b() {} // file
b.js export default function() {} import {a,b} from './a.js' import XXX from './b.js' CommonJS
```

CommonJs是Node独有的规范，浏览器中使用就需要用到Browserify解析了。 //
a.js module.exports={a:1} // oreports.a=1 // b.js var module=require('./a.js') module.a // -> log 1

在上述代码中，`module.exports`和`exports`很容易混淆，让我们来看看大致内部实现

```
var module = require('./a.js'); module.a //
```

这里其实就是包装了一层立即执行函数，这样就不会污染全局变量了，// 重要的是 `module`

这里，`module` 是 Node 独有的一个变量 `module.exports={a:1}`// 基本实现 `var module={exports:{}}`//

`exports` 就是个空对象// 这个是什么 `exports` 和 `module.exports`

用法相似的原因 `var exports=module.exports` `var load=function(module){//`

导出的东西 `var a=1` `module.exports=a` `return module.exports`}; 再来说说 `module.exports`和`exports`，用法其实是相似的，但是不能对`exports`直接赋值，不会有任何效果。

对于CommonJS和ES6中的模块化的两者区别是：前者支持动态导入，也就是`require(`${path}/xx.js`)`，后者目前不支持，但是已有提案。前者是同步导入，因为用于服务端，文件都在本地，同步导入即使卡住主线程影响也不大。而后者是异步导入，因为用于浏览器，需要下载文件，如果也采用同步导入会对渲染有很大影响。前者在导出时都是值拷贝，就算导出的值变了，导入的值也不会改变，所以如果想更新值，必须重新导入一次。但是后者...

1) 计数模块导出`count`后持续自增，`require`方保存的值不再变化，而`import`方观察到更新，你怎么解释？

答：CommonJS导出通常表现为运行时取得导出值，解构或保存后不会自动形成ESM的实时绑定。ES Module的导入绑定指向导出接口，源模块更新时导入侧可观察到变化；业务代码仍不应尝试给导入绑定重新赋值。

2) 浏览器首屏直接使用同步`require`风格加载几十个远程模块，架构评审时你会否决吗？

答：浏览器模块需要经过网络获取，同步加载会阻塞后续执行，不适合作为现代页面的主线方案。应采用原生或构建工具支持的ES Module；AMD、CMD主要用于理解旧项目中的异步模块演进。

3) 插件入口路径要由租户配置在运行时决定，同事坚持所有依赖都写成静态`import`，冲突点在哪里？

答：静态`import`的接口和依赖可在解析阶段确定，利于构建工具分析，但不能直接用任意运行时字符串替代模块说明符。确需按条件加载时应采用环境支持的动态导入机制；路径范围过于开放会削弱构建可分析性并增加加载失败风险。

4) 一个CommonJS文件先写`exports.a = 1`，随后又写`exports = { b: 2 }`，调用方为什么只能拿到`a`？

答：初始时`exports`只是`module.exports`的引用，给`exports.a`赋值会修改真正的导出对象。把`exports`重新赋成新对象只改变局部变量指向，不会替换`module.exports`；要整体导出必须直接给`module.exports`赋值。

5) 老后台同时存在`require.js`、`sea.js`和新写的ESM，迁移时要不要把三者当成同等长期方案维护？

答：不应同等投入，现代浏览器与构建工具的主线应收敛到ES Module，Node侧则根据现有包体系处理CommonJS与ESM。AMD强调依赖前置，CMD强调就近依赖和不同执行时机，保留它们通常只为兼容尚未迁移的旧模块。

379. Iterator迭代器

一个对象能否被`for...of`遍历，关键看它是否实现了`[Symbol.iterator]`。这个方法需要返回迭代器对象，而迭代器通过`next()`逐项返回`{ value, done }`，两者分别负责提供遍历入口和推进遍历过程。数组、类数组对象、`Set`、`Map`已经部署了该接口，普通对象默认没有，所以不能直接使用`for...of`。如果确实需要遍历对象，可以手动规定属性顺序并实现接口，也可以用Generator简化实现，但顺序规则要由开发者明确。Iterat

or（迭代器）是一种接口，也可以说是一种规范。为各种不同的数据结构提供统一的访问机制。任何数据结构只要部署Iterator接口，就可以完成遍历操作（即依次处理该数据结构的所有成员）。 Iterator语法：
const obj = {[Symbol.iterator]: function() {}}

[Symbol.iterator]属性名是固定的写法，只要拥有了该属性的对象，就能够用迭代器的方式进行遍历。

迭代器的遍历方法是首先获得一个迭代器的指针，初始时该指针指向第一条数据之前，接着通过调用 next 方法，改变指针的指向，让其指向下一条数据。每一次的next都会返回一个对象，该对象有两个属性value代表想要获取的数据done布尔值，false表示当前指针指向的数据有值，true表示遍历已经结束

value代表想要获取的数据done布尔值，false表示当前指针指向的数据有值，true表示遍历已经结束

Iterator 的作用有三个： 创建一个指针对象，指向当前数据结构的起始位置。也就是说，遍历器对象本质上，就是一个指针对象。第一次调用指针对象的next方法，可以将指针指向数据结构的第一个成员。第二次调用指针对象的next方法，指针就指向数据结构的第二个成员。不断调用指针对象的next方法，直到它指向数据结构的结束位置。 每一次调用next方法，都会返回数据结构的当前成员的信息。具体来说，就是返回一个包含value和done两个属性的对象。其中，value属性是当前成员的值，done属性是一个布尔值，表示遍历是否结束。

```
let arr = [{ num: 1 }, 2, 3]; let it = arr[Symbol.iterator]()//  
获取数组中的迭代器 console.log(it.next())// { value: Object { num: 1 }, done: false }  
console.log(it.next())// { value: 2, done: false } console.log(it.next())// { value: 3, done: false }  
console.log(it.next())// { value: undefined, done: true } 对象没有布局Iterator接口，无法使用for of遍历。  
下面使得对象具备Iterator接口 一个数据结构只要有Symbol.iterator属性，就可以认为是“可遍历的”  
原型部署了Iterator接口的数据结构有三种，具体包含四种，分别是数组，类似数组的对象，Set和Map结构  
为什么对象（Object）没有部署Iterator接口呢？ 一是因为对象的哪个属性先遍历，哪个属性后遍历是不确定的，需要开发者手动指定。然而遍历遍历器是一种线性处理，对于非线性的数据结构，部署遍历器接口，就等于要部署一种线性转换对对象部署Iterator接口并不是很必要，因为Map弥补了它的缺陷，又正好有Iteraotr接口  
let obj = {id: '123', name: '张三', age: 18, gender: '男', hobby: '睡觉'}; obj[Symbol.iterator] = function() {  
let keyArr = Object.keys(obj); let index = 0; return {next() {return index < keyArr.length ? {value: {key: keyArr[index], val: obj[keyArr[index++]}} : {done: true}}}}; for (let key of obj) {console.log(key)}
```

1) 订单接口返回{0:'A',1:'B',length:2}，列表页执行[...data]却报is not iterable，明明它能按下标读取，问题出在哪？

答：能按下标读取只说明它是类数组，不代表实现了可迭代协议；展开语法会查找data[Symbol.iterator]，缺失时就会抛错。若只需转成数组可用Array.from(data)，若要支持统一遍历，则应显式部署迭代器。

2) 组件库有个树形结果对象，团队希望它既能保留原结构，又能被for...of按业务指定顺序遍历，你会怎样设计[Symbol.iterator]？

答：应让[Symbol.iterator]()返回独立的迭代器对象，并由闭包保存当前位置；每次next()返回{value, done}，顺序由业务规则明确决定。不要直接依赖普通对象属性的自然排列，否则遍历语义会隐含在数据结构中，后续调整字段容易改变结果。

3) 同一个分页集合需要同时启动两次for...of，如果[Symbol.iterator]()总是返回缓存的同一个迭代器，会出现什么现象？

答：两次遍历会共享指针，一个循环调用next()后会推进另一个循环的位置，最终产生跳项、错序或提前结束。每次调用[Symbol.iterator]()都应创建新的状态和指针；只有明确设计为一次性消费的数据流，才适合共享迭代状态。

4) 线上导出任务偶尔多出一条undefined，自定义迭代器结束分支返回了{value: undefined, done: false}，你会怎么定位？

答：应直接检查连续调用next()的返回序列，因为done:

false表示当前值仍是合法成员，即使value为undefined也会被消费。结束分支必须返回{done: true}，通常同时令value为undefined；还要核对索引递增是否发生在取值之前。

5) 接口字段对象要依次输出键和值，同事主张给所有普通对象的原型统一加[Symbol.iterator]，你会接受吗？

答：不建议修改所有普通对象的原型，因为对象需要先约定按键、按值还是按键值对遍历，统一实现会强加一种线性语义。更稳妥的是在具体数据结构上实现迭代器，或先用Object.keys明确顺序；通用键值集合也可考虑使用原生可迭代的Map。

380. Promise

Promise是保存异步结果的状态容器，状态只能从pending变为fulfilled或rejected，一旦改变就不可逆。每次调用then()都会返回新的Promise，因此可以把异步步骤串起来，避免多层回调嵌套；相关回调会作为微任务参与事件循环。并发任务全部成功才继续时用Promise.all()，需要保留每项结果时用Promise.allSettled()。只取首个成功结果可用Promise.any()，而超时竞争这类场景更适合Promise.race()，因为它采用最先改变的状态。这里你谈promise的时候，除了将他解决的痛点以及常用的API之外，最好进行拓展把eventloop带进来好好讲一下，microtask(微任务)、macrotask(任务)

的执行顺序，如果看过promise源码，最好可以谈一谈 原生Promise是如何实现的。Promise的关键点在于callback的两个参数，一个是resolve，一个是reject。还有就是Promise的链式调用（Promise.then()，每一个then都是一个责任人） Promise是ES6新增的语法，解决了回调地狱的问题。可以把Promise看成一个状态机。初始是pending状态，可以通过函数resolve和reject，将状态转变为resolved或者rejected状态，状态一旦改变就不能再次变化。then函数会返回一个Promise实例，并且该返回值是一个新的实例而不是之前的实例。因为Promise规范规定除了pending状态，其他状态是不可以改变的，如果返回的是一个相同实例的话，多个then调用就失去意义了。对于then来说，本质上可以把它看成是flatMap 1. Promise的基本情况 简单来说它就是一个容器，里面保存着某个未来才会结束的事件（通常是异步操作）的结果。从语法上说，Promise 是一个对象，从它可以获取异步操作的消息 一般 Promise 在执行过程中，必然会处于以下几种状态之一。 待定（pending）：初始状态，既没有被完成，也没有被拒绝。已完成（fulfilled）：操作成功完成。已拒绝（rejected）：操作失败。 待定状态的Promise对象执行的话，最后要么会通过一个值完成，要么会通过一个原因被拒绝。当其中一种情况发生时，我们用Promise的then方法排列起来的相关处理程序就会被调用。因为最后Promise.prototype.then和Promise.prototype.catch方法返回的是一个Promise，所以它们可以继续被链式调用 关于 Promise 的状态流转情况，有一点值得注意的是，内部状态改变之后不可逆，你需要在编程过程中加以注意。文字描述比较晦涩，我们直接通过一张图就能很清晰地看出 Promise 内部状态流转的情况

从上图可以看出，我们最开始创建一个新的Promise返回给p1，然后开始执行，状态是pending，当执行resolve之后状态就切换为fulfilled，执行reject之后就变为rejected的状态 2. Promise的静态方法 all 方法语法：Promise.all (iterable) 参数： 一个可迭代对象，如Array。描述：此方法对于汇总多个promise的结果很有用，在 ES6 中可以将多个Promise.all异步请求并行操作，返回结果一般有下面两种情况。当所有结果成功返回时按照请求顺序返回成功结果。当其中有一个失败方法时，则进入失败方法我们来看下业务的场景，对于下面这个业务场景页面的加载，将多个请求合并到一起，用 all 来实现可能效果会更好，请看代码片段 语法：Promise.all (iterable) 参数：一个可迭代对象，如Array。描述： 此方法对于汇总多个promise的结果很有用，在 ES6 中可以将多个Promise.all异步请求并行操作，返回结果一般有下面两种情况。当所有结果成功返回时按照请求顺序返回成功结果。当其中有一个失败方法时，则进入失败方法

当所有结果成功返回时按照请求顺序返回成功结果。当其中有一个失败方法时，则进入失败方法 // 在一个页面中需要加载获取轮播列表、获取店铺列表、获取分类列表这三个操作，页面需要同时发出请求进行页

面渲染，这样用`Promise.all`来实现，看起来更清晰、一目了然。//1.获取轮播数据列表function getBannerList(){return new Promise((resolve, reject) => {setTimeout(function(){resolve('轮播数据')}, 300)})} //2. 获取店铺列表function getStoreList(){re...

1) 支付页创建一个Promise后先调用resolve('ok')，异常分支又调用reject(error)，监控却始终看到成功，这是实现缺陷还是预期行为？

答：这是预期行为，Promise只能从pending转为fulfilled或rejected，第一次状态变化后便不可逆。排查时应找出哪个异步分支最先落定，并避免多个回调竞争；后续调用不会覆盖既有结果，也不能用来撤销已完成状态。

2) 首页要并行加载轮播、店铺和分类数据，三块全部到齐才允许渲染，团队在串行await和Promise.all之间争论，你会选哪个？

答：三个请求彼此无依赖且必须全部成功时，应使用Promise.all并行汇总，结果会按传入顺序返回，而不是按完成顺序排列。任意一个请求拒绝都会使汇总结果进入拒绝分支，因此还需在页面层明确整体失败时的降级或重试策略。

3) 批量上传 50 个附件时，产品要求成功项保留、失败项单独展示原因，原来的Promise.all一遇失败就进入catch，该怎么改？

答：应改用Promise.allSettled，等待全部任务落定后读取每项的status，成功项取value，失败项取reason。它解决的是结果汇总不短路，并不负责重试、取消或并发控制；这些策略仍需由上传层单独实现。

4) 图片详情页用Promise.race([loadImage(), timeout()])做五秒超时，界面已经提示失败，稍后图片的onload却仍然触发，你如何解释？

答：Promise.race只让最先改变状态的任务决定外层结果，不会停止其余异步操作，因此图片加载仍可能继续并触发原回调。排查时要区分“外层已拒绝”和“底层任务已取消”；若业务要求真正终止，还需要底层操作自身提供取消能力。

5) 同事实现了一个简化版Promise，只在调用then时检查当前状态；异步resolve后回调始终不执行，缺了哪部分机制？

答：调用then时若状态仍为pending，实现必须保存成功和失败处理函数，待resolve或reject改变状态后再触发。仅处理已完成状态只能覆盖同步落定场景，也无法支撑可靠链式调用；完整实现还要让每次then返回新的Promise。

6) 搜索页同时请求三个镜像源，只需要第一个成功结果；有人选择race，有人选择any，哪个更符合需求？

答：应选择Promise.any，因为它忽略先到达的拒绝，直到某个任务成功才完成；只有全部任务都拒绝时，外层才拒绝。Promise.race采用最先落定的结果，最快返回的若是失败，就会提前结束，不符合“首个成功”的约束。

381. Generator

Generator是一种可以暂停并恢复执行的函数，本质上也是Iterator接口的一种实现。调用生成器函数不会立即跑完代码，而是返回迭代器；每次调用next()，代码会执行到下一个yield并交出执行权。再次调用next(value)时，传入值会成为上一个yield表达式的结果，但第一次传参会被忽略。它适合描述分段执行流程，也是async/await演化所依赖的思路之一。Generator是ES6中新增的语法，和Promise一样，都可以用来异步编程。Generator函数可以说是Iterator接口的具体实现方式。Generator

最大的特点就是可以控制函数的执行。function*用来声明一个函数是生成器函数，它比普通的函数声明多了一个*,*的位置比较随意可以挨着function关键字，也可以挨着函数名yield产出的意思，这个关键字只能

出现在生成器函数体内，但是生成器中也可以没有yield关键字，函数遇到yield的时候会暂停，并把yield后面的表达式结果抛出去next作用是将代码的控制权交还给生成器函数

```
function*foo(x){lety=2*(yield(x+1))letz=yield(y/3)return(x+y+z)}letit=foo(5)console.log(it.next())//=> {value: 6, done: false}console.log(it.next(12))// => {value: 8, done: false}console.log(it.next(13))// => {value: 42, done: true}
```

上面这个示例就是一个Generator函数，我们来分析其执行过程：首先 Generator函数调用时它会返回一个迭代器当执行第一次 next 时，传参会被忽略，并且函数暂停在 yield (x + 1) 处，所以返回 5 + 1 = 6当执行第二次 next 时，传入的参数等于上一个 yield 的返回值，如果你不传参，yield 永远返回 undefined。此时 let y = 2 * 12，所以第二个 yield 等于 2 * 12 / 3 = 8当执行第三次 next 时，传入的参数会传递给 z，所以 z = 13, x = 5, y = 24，相加等于 42 yield实际就是暂缓执行的标示，每执行一次next()，相当于指针移动到下一个yield位置 总结一下，Generator函数是ES6提供的一种异步编程解决方案。通过yield标识位和next()方法调用，实现函数的分段执行 遍历器对象生成函数，最大的特点是可以交出函数的执行权 function关键字与函数名之间有一个星号；函数体内部使用yield表达式，定义不同的内部状态；next指针移向下一个状态 这里你可以说说Generator的异步编程，以及它的语法糖async和await，传统的异步编程。ES6之前，异步编程大致如下

回调函数事件监听发布/订阅 传统异步编程方案之一：协程，多个线程互相协作，完成异步任务。 // 使用 * 表示这是一个 Generator 函数// 内部可以通过 yield 暂停代码// 通过调用 next 恢复执行function*test(){leta=1+2;yield2;yield3;}letb=test();console.log(b.next());// > { value: 2, done: false }console.log(b.next());// > { value: 3, done: false }console.log(b.next());// > { value: undefined, done: true } 从以上代码可以发现，加上*的函数执行后拥有了next函数，也就是说函数执行后返回了一个对象。每次调用next函数可以继续执行被暂停的代码。以下是Generator函数的简单实现 // cb 也就是编译过的 test 函数functiongenerator(cb){return(function(){varobject={next:0,stop:function(){};return{next:function(){varret=cb(object);if(ret===undefined)return{value:undefined,done:true};return{value:ret,done:false};}}})();// 如果你使用 babel 编译后可以发现 test 函数变成了这样functiontest(){vara;returngenerator(function...

1) 报表代码调用buildRows()后没有生成任何数据，直到执行iterator.next()才出现第一行；同事认为函数调用已经执行了函数体，错在哪里？

答：调用Generator函数只会返回迭代器，不会像普通函数那样立即完整执行函数体；第一次next()才会运行到首个yield并暂停。若调用方忘记驱动迭代器，内部逻辑就不会推进，这也是把控制权交给外部调度者的代价。

2) 面试现场这段代码const it=foo(5); it.next(); it.next(12); it.next(13)最终返回42，第二次传入的12到底进入了哪里？

答：第二次next(12)的参数会成为上一个yield表达式的结果，因此y被计算为2 * 12，随后函数在下一个yield再次暂停。第一次next()的参数会被忽略，因为当时还没有暂停中的yield可以接收它。

3) 十万条记录需要分批格式化，主线程不能一次跑完整个循环，你会怎样利用Generator让调度器分段推进？

答：可以把处理过程写成Generator，每完成一批记录便yield当前进度，外部调度器在合适时机调用next()继续。这样执行权会在批次之间交还给调用方，但Generator本身不会自动安排时间片，批量大小和再次调度仍需额外设计。

4) 异步流水线第二步依赖第一步的请求结果，但调用方只连续执行next()没有传值，后续计算得到NaN，你会查什么？

答：应检查调用方是否把上一步完成值传入下一次next(result)，因为暂停点左侧拿到的是下一次next的参数；省略时该yield表达式结果就是undefined。同时确认每次返回的done，避免在生成器已经结束仍继续传值。

5) 团队想用Generator替代async/await编排三个串行请求，理由是它能暂停函数；这两种方案的关键取舍是什么？

答：Generator只提供通过yield暂停、由next()恢复的控制机制，异步结果何时完成以及如何继续都要由执行器负责。async/await相当于Generator配合自动执行器，语义更直接且返回Promise；只需要显式控制推进过程时，手动生成器才更有价值。

382. async/await

async/await是Generator加自动执行器思路的语法糖，让基于Promise的异步流程写起来更像同步代码。async函数会返回Promise，执行到await时会暂时交还控制权，等结果完成后再恢复后续逻辑。它比连续的then调用更直观，也更容易组织错误处理。多个任务互不依赖时，我一般不会逐个await，因为这样会失去并发性，更适合用Promise.all()。

Generator函数的语法糖。有更好的语义、更好的适用性、返回值是Promise。await和promise一样，更多的是考笔试题，当然偶尔也会问到和promise的一些区别。await相比直接使用Promise来说，优势在于处理then的调用链，能够更清晰准确的写出代码。缺点在于滥用await可能会导致性能问题，因为await会阻塞代码，也许之后的异步代码并不依赖于前者，但仍然需要等待前者完成，导致代码失去了并发性，此时更应该使用Promise.all。一个函数如果加上async

，那么该函数就会返回一个Promise async => *await => yield // 基本用法asyncfunctiontimeout(ms){awaitnewPromise((resolve)=>{setTimeout(resolve,ms)})}asyncfunctionasyncConsole(value,ms){awaittimeout(ms)console.log(value)}asyncConsole('hello async and await',1000)
下面来看一个使用await的代码。 var a=0 var b=async()=>{a=a+await 10 console.log('2',a)} // -> '2'
10 a=(await 10)+a console.log('3',a) // -> '3' 20 } b() a++ console.log('1',a) // -> '1' 1
首先函数b先执行，在执行到await

10之前变量a还是0，因为在await内部实现了generators，generators会保留堆栈中东西，所以这时候a = 0被保存了下来因为await是异步操作，遇到await就会立即返回一个pending状态的Promise对象，暂时返回执行代码的控制权，使得函数外的代码得以继续执行，所以会先执行console.log('1',

a)这时候同步代码执行完毕，开始执行异步代码，将保存下来的值拿出来使用，这时候a = 10
10然后后面就是常规执行代码了 优缺点： async/await的优势在于处理 then

的调用链，能够更清晰准确的写出代码，并且也能优雅地解决回调地狱问题。当然也存在一些缺点，因为await将异步代码改造成了同步代码，如果多个异步代码没有依赖性却使用了await

会导致性能上的降低。 async原理 async/await语法糖就是使用Generator函数+自动执行器来运作的 // 定义了一个promise，用来模拟异步请求，作用是传入参数++functiongetNum(num){returnnewPromise((resolve,reject)=>{setTimeout(()=>{resolve(num+1)},1000)})} // 自动执行器，如果一个Generator函数没有执行完，则递归调用functionasyncFun(func){var gen=func();functionnext(data){var result=gen.next(data);if(result.done)returnresult.value;result.value.then(function(data){next(data);});}next();}
// 所需要执行的Generator函数，内部的数据在执行完成一步的promise之后，再调用下一步var func=function*(){var f1=yield getNum(1);var f2=yield getNum(f1);console.log(f2);}; asyncFun(func); 在运行的过程中，判断一个函数的promise是否完成，如果已经完成，将结果传入下一个函数，继续重复此步骤每一个next()方法返回值的value属性为一个Promise对象，所以我们为其添加then方法，在then方法里面接着运行next方法挪移遍历器指针，直到Generator函数运行完成

1) 用户资料页调用loadUser()后直接打印返回值，看到的是Promise而不是用户对象；函数里明明写了return user，为什么？

答：只要函数声明为async，调用结果就会被包装成Promise，函数体中的return user对应其完成值。调用方需要使用await loadUser()或注册then获取对象；把返回值当普通对象读取，会在异步结果落定前访问错误的数据形态。

2) 首屏循环请求 20 个互不依赖的商品详情，每个都写成await fetchOne(id)，页面等待时间明显累加，你会怎样改？

答：这些请求没有前后依赖，应先用ids.map(fetchOne)创建任务，再通过await Promise.all(...)汇总，使它们保持并发。若任一请求失败，Promise.all会进入拒绝路径；页面需要部分成功时，应改用能保留每项状态的汇总策略。

3) 订单创建必须先拿用户信息，再用用户编号查询优惠券；评审者仍要求全部放进Promise.all，你会同意吗？

答：不会，因为优惠券请求的参数依赖用户请求结果，两个操作无法在输入尚未产生时真正并行。这里顺序await更能表达依赖关系；除非接口或数据结构被重新设计，否则强行使用Promise.all只会制造无效请求或复杂的占位逻辑。

4) 线上保存按钮点击后只执行到第一个await，后续日志和成功提示都没出现，控制台显示拒绝未处理，你会从哪里排查？

答：先检查被等待的Promise是否进入rejected，因为拒绝会使当前async函数中断，并让它返回的Promise同样拒绝。应在能决定补救策略的位置使用try/catch，或由调用方接住拒绝；只加成功日志无法覆盖异常路径。

5) 同事写了一个基于Generator的asyncFun，请求成功时能串行执行，但任一步拒绝后流程悬空；自动执行器还缺什么？

答：执行器不能只对result.value注册成功回调，还要处理拒绝并把错误交回生成器或让外层Promise拒绝。每次推进也应检查done，完成时解析最终返回值；否则它只是演示版递归驱动器，达不到async/await的错误传播语义。

6) 搜索页有两个独立请求，代码为了写起来像同步流程而连续使用两个await；可读性和并发性冲突时你怎么选？

答：若后一个请求不依赖前一个，应先同时启动两个任务，再用一次Promise.all等待结果，既保留清晰的赋值结构，也避免无意义串行。await的价值是表达异步步骤，不意味着每一步都必须顺序启动；错误处理仍要覆盖汇总后的拒绝。

383. 事件循环

事件循环的核心是执行一个宏任务后清空全部微任务，再进入下一轮宏任务。同步脚本本身属于宏任务，执行期间遇到Promise.then等会登记微任务，遇到setTimeout、交互或I/O回调则进入相应的宏任务队列。当前执行栈清空后，微任务会按登记顺序持续执行，新产生的微任务也要在这一轮清空，之后浏览器才可能渲染并取下一个宏任务。Node.js还会细分事件循环阶段，并且process.nextTick的优先级高于Promise微任务。默认代码从上到下执行，执行环境通过script来执行（宏任务）在代码执行过程中，调用定时器promiseclick事件...不会立即执行，需要等待当前代码全部执行完毕给异步方法划分队列，分别存放到微任务（立即存放）和宏任务（时间到了或事情发生了才存放）到队列中script执行完毕后，会清空所有的微任务微任务执行完毕后，会渲染页面（不是每次都调用）再去宏任务队列中看有没有到达时间的，拿出来其中一个执行执行完毕后，按照上述步骤不停的循环 例子 自动执行的情况 会输出 listener1 listener2 task1 task2 如果手动点击click

会一个宏任务取出来一个个执行，先执行click的宏任务，取出微任务去执行。会输出 listener1 task1 listener2 task2 console.log(1) async function asyncFunc() { console.log(2) // await xx ==> promise.resolve(()=>{ console.log(3) }).then(() // console.log(3) 放到 promise.resolve 或立即执行 await console.log(3) // 相当于把 console.log(4) 放到了 then promise.resolve(()=>{ console.log(3) }).then(()=>{ // console.log(4) // }) // 微任务谁先注册谁先执行 console.log(4) } setTimeout(()=>{ console.log(5) }) const promise = new Promise((resolve, reject) => { console.log(6) resolve(7) }) promise.then(d=>{ console.log(d) }) asyncFunc() console.log(8) // 输出 1 6 2 3 8 7 4 5

1. 浏览器事件循环 涉及面试题：异步代码执行顺序？解释一下什么是Event Loop？JavaScript的单线程，与它的用途有关。作为浏览器脚本语言，JavaScript的主要用途是与用户互动，以及操作DOM。这决定了它只能是单线程，否则会带来很复杂的同步问题。比如，假定JavaScript同时有两个线程，一个线程在某个DOM节点上添加内容，另一个线程删除了这个节点，这时浏览器应该以哪个线程为准？所以，为了避免复杂性，从一诞生，JavaScript就是单线程，这已经成了这门语言的核心特征，将来也不会改变

js代码执行过程中会有很多任务，这些任务总的分成两类：同步任务异步任务 当我们打开网站时，网页的渲染过程就是一大堆同步任务，比如页面骨架和页面元素的渲染。而像加载图片音乐之类占用资源大耗时久的任务，就是异步任务。我们，我们用导图来说明：我们解释一下这张图：同步和异步任务分别进入不同的执行"场所"，同步的进入主线程，异步的进入Event Table并注册函数。当指定的事情完成时，Event Table会将这个函数移入Event Queue。主线程内的任务执行完毕为空，会去Event Queue读取对应的函数，进入主线程执行。上述过程会不断重复，也就是常说的Event Loop(事件循环)。那主线程执行栈何时为空呢？js引擎存在monitoring process进程，会持续不断的检查主线程执行栈是否为空，一旦为空，就会去Event Queue那里检查是否有等待被调用的函数 以上就是js运行的整体流程 面试中该如何回答呢？

下面是我个人推荐的答案：首先js 是单线程运行的，在代码执行的时候，通过将不同函数的执行上下文压入执行栈中来保证代码的有序执行在执行同步代码的时候，如果遇到了异步事件，js 引擎并不会一直等待其返回结果，而是会将这个事件挂起，继续执行执行栈中的其他任务当同步事件执行完毕后，再将异步事件对应的回调加入到与当前执行栈中不同的另一个任务队列中等待执行任务队列可以分为宏任务对列和微任务对列，当当前执行栈中的事件执行完毕后，js引擎首先会判断微任务对列中是否有任务可以执行，如果有就将微任务队首的事件压入栈中执...

1) 控制台代码先注册setTimeout(fn,0)，后注册Promise.resolve().then(task)，结果task先执行；定时器写在前面为什么没有优先？

答：当前script本身先作为一个宏任务执行，期间定时器回调进入宏任务队列，then回调进入微任务队列。同步代码结束后会先清空全部微任务，再进入下一轮取宏任务，所以注册先后不能跨越任务类型决定执行次序。

2) 按钮点击后先把状态改成loading，随后同步计算两秒，页面却直到计算结束才更新，你会怎样解释并调整？

答：点击回调作为当前任务占用主线程，同步计算未结束时，浏览器没有机会进入必要的UI渲染阶段，因此状态虽已修改却尚未绘制。应拆分长任务并主动让出执行权，或把重计算移出主线程；仅把计算包装进微任务仍可能阻塞渲染。

3) 消息列表一次追加数据后递归创建Promise.then微任务，页面长时间不重绘，明明每次回调都很短，问题在哪里？

答：一次宏任务结束后，事件循环会持续清空微任务，包括执行过程中新增的微任务；递归产生微任务可能让队列迟迟无法清空。渲染通常位于微任务检查之后，因此应限制递归链，并把后续批次交给新的任务或其他调度机制。

4) 线上日志显示同步代码正常结束，但某个`setTimeout(...,0)`偶尔延迟很久才执行，你会先排查哪些阻塞来源？

答：0只表示满足计时条件后可进入候选队列，不保证立即执行；当前执行栈、此前积压的微任务以及更早待处理的任务都会推迟回调。应检查长同步计算、连续微任务和高频事件处理，而不能仅凭定时参数判断调度异常。

5) Node服务里同时安排`process.nextTick`、`Promise.then`、`setTimeout`和`setImmediate`，同事套用浏览器顺序直接下结论，你会接受吗？

答：不能直接套用，因为Node的事件循环包含`timers`、`poll`、`check`等阶段，`setImmediate`在`check`阶段执行。`process.nextTick`的优先级还高于普通`Promise`微任务；定时器与`setImmediate`的先后需结合注册位置和所处阶段判断。

6) 数据看板想把大量DOM更新全部塞进`Promise.then`，理由是微任务比定时器更早执行，这个选型有什么风险？

答：微任务确实会在当前任务结束后优先清空，但大量DOM操作会占满微任务检查阶段，反而推迟浏览器渲染和下一轮事件处理。应按可见更新边界分批执行，并给渲染留下机会；“更早进入执行栈”不等于页面能更早呈现。

384. 垃圾回收

JavaScript垃圾回收会识别并释放程序不再使用的对象，V8则通过分代机制降低回收成本。新生代对象多数存活时间短，所以使用分成From和To空间的Scavenge算法，存活对象会被复制，满足条件时晋升到老年代。老年代对象存活更久，主要采用标记清除，并用标记压缩缓解内存碎片。由于回收会暂停应用逻辑，V8还通过增量标记把较长的老年代标记过程拆开执行。对于在JavaScript中的字符串，对象，数组是没有固定大小的，只有当对他们进行动态分配存储时，解释器就会分配内存来存储这些数据，当JavaScript的解释器消耗完系统中所有可用的内存时，就会造成系统崩溃。内存泄漏，在某些情况下，不再使用到的变量所占用内存没有及时释放，导致程序运行中，内存越占越大，极端情况下可以导致系统崩溃，服务器宕机。JavaScript有自己的一套垃圾回收机制，JavaScript的解释器可以检测到什么时候程序不再使用这个对象了（数据），就会把它所占用的内存释放掉。针对JavaScript的来及回收机制有以下两种方法（常用）：标记清除，引用计数标记清除 v8 的垃圾回收机制基于分代回收机制，这个机制又基于世代假说，这个假说有两个特点，一是新生的对象容易早死，另一个是不死的对象会活得更久。基于这个假说，v8引擎将内存分为了新生代和老年代。新创建的对象或者只经历过一次的垃圾回收的对象被称为新生代。经历过多次垃圾回收的对象被称为老年代。新生代被分为 From 和 To 两个空间，To 一般是闲置的。当 From 空间满了的时候会执行 Scavenge 算法进行垃圾回收。当我们执行垃圾回收算法的时候应用逻辑将会停止，等垃圾回收结束后再继续执行。这个算法分为三步：首先检查 From 空间的存活对象，如果对象存活则判断对象是否满足晋升到老年代的条件，如果满足条件则晋升到老年代。如果不满足条件则移动 To 空间。如果对象不存活，则释放对象的空间。最后将 From 空间和 To 空间角色进行交换。新生代对象晋升到老年代有两个条件：第一个是判断对象是否已经经过一次 Scavenge 回收。若经历过，则将对象从 From 空间复制到老年代中；若没有经历，则复制到 To 空间。第二个是 To 空间的内存使用占比是否超过限制。当对象从 From 空间复制到 To 空间时，若 To 空间使用超过 25%，则对象直接晋升到老年代中。设置 25% 的原因主要是因为算法结束后，两个空间结束后会交换位置，如果 To 空间的内存太小，会影响后续的内存分配。老年代采用了标记清除法和标记压缩法。标记清除法首先会对内存中存活的对象进行标记，标记结束后清除掉那些没有标记的对象。由于标记清除后会造成很多的内存碎片，不便于后面的内存分配。所以了解决内存碎片的问题引入了标记压缩法。由于在进行垃圾回收的时

候会暂停应用的逻辑，对于新生代方法由于内存小，每次停顿的时间不会太长，但对于老生代来说每次垃圾回收的时间长，停顿会造成很大的影响。为了解决这个问题 V8 引入了增量标记的方法，将一次停顿进行的过程分为了多步，每次执行完一小步就让运行逻辑执行一会，就这样交替运行

1) 商品瀑布流每次滚动都创建一批临时对象，内存呈锯齿状上涨后又回落；既然最终都释放了，为什么页面仍可能周期性卡顿？

答：内存能回落只说明对象最终可回收，不代表回收没有成本。新生代From空间满后会执行Scavenge，检查存活对象并复制或晋升，期间应用逻辑会暂停；高频分配会增加回收频率，因此还要减少滚动热路径中的临时对象。

2) 编辑器把一个仍被全局缓存引用的文档对象标记成“已关闭”，随后主动调用垃圾回收；引擎会因为业务上不用它而释放内存吗？

答：不会，垃圾回收依据对象是否仍可被程序引用，而不是业务状态字段。全局缓存仍保存引用时，该对象会被判断为存活；应删除缓存项或清空引用，单纯改成“已关闭”以及等待回收都不能解决占用。

3) 列表页持续创建短命数据对象，候选人建议把所有对象都长期缓存，避免频繁触发新生代回收；这个方案为什么可能适得其反？

答：长期缓存会让原本容易早死的对象继续存活，并可能在经历回收后晋升到老生代。老生代回收需要标记存活对象，还可能通过标记压缩处理碎片，停顿影响通常更明显；缓存只应保留确有复用价值且能及时淘汰的数据。

4) 监控显示一次明显长停顿发生在老生代回收阶段，同事认为标记清除结束后已有连续可用空间；你会怎样纠正并推进排查？

答：标记清除会释放未标记对象，但可能留下分散的内存碎片，并不保证形成连续空间。引擎还可能执行标记压缩来整理存活对象，因此应结合对象存活量和晋升情况排查长期引用，而不是只盯着回收后的空闲总量。

5) 动画团队要求彻底消除老生代垃圾回收停顿，并把增量标记当成保证；你会如何评估这个目标？

答：增量标记把一次较长的标记过程拆成多步，让应用逻辑在步骤之间获得执行机会，但并不等于垃圾回收零停顿。工程目标应改为控制单次影响和垃圾产生量，同时减少无意义的长期存活对象；不能把增量机制当作实时性承诺。

385. 内存泄露

内存泄露本质上是对象已经不再使用，却仍被某个引用链持有，导致垃圾回收无法释放它。常见原因可以归为两类：意外全局变量和闭包长期保留引用，以及未关闭的定时器、事件监听或已删除DOM的残留引用。排查时我一般用Chrome Timeline标记并观察内存变化，定位持续增长的异常点。开发中还要留意console.log保留的对象，避免它干扰内存回收和判断。

意外的全局变量：无法被回收定时器：未被正确关闭，导致所引用的外部变量无法被释放事件监听：没有正确销毁（低版本浏览器可能出现）闭包第一种情况是我们由于使用未声明的变量，而意外的创建了一个全局变量，而使这个变量一直留在内存中无法被回收。第二种情况是我们设置了setInterval定时器，而忘记取消它，如果循环函数有对外部变量的引用的话，那么这个变量会被一直留在内存中，而无法被回收。第三种情况是我们获取一个DOM元素的引用，而后面这个元素被删除，由于我们一直保留了对这个元素的引用，所以它也无法被回收。第四种情况是不合理的使用闭包，从而导致某些变量一直被留在内存当中。dom引用:dom元素被删除时，内存中的引用未被正确清空控制台console.log打印的东西 第一种情况是我们由于使用未声明的变量，而意外的创建了一个全局变量，而使这个变量一直留在内存中无法被回收。第二种情况是我们设置了setInterval定时器，而忘记

取消它，如果循环函数有对外部变量的引用的话，那么这个变量会被一直留在内存中，而无法被回收。第三种情况是我们获取一个DOM元素的引用，而后面这个元素被删除，由于我们一直保留了对这个元素的引用，所以它也无法被回收。第四种情况是不合理的使用闭包，从而导致某些变量一直被留在内存当中。可用chrome中的timeline进行内存标记，可视化查看内存的变化情况，找出异常点。

内存泄露排查方法(opens new window)

1) 单页后台关闭一个弹窗后，节点已经从页面移除，但堆里仍能看到整棵弹窗子树；前端同事说“删了DOM就一定会回收”，你怎么判断？

答：从文档树移除不等于失去全部引用，代码中的变量、缓存或闭包仍可能持有该节点。应沿引用关系找到保留它的对象并清空对应引用；只执行节点删除无法保证释放，残留引用还可能连带保留子节点和关联数据。

2) 行情组件每次进入页面都会启动一个setInterval，离开页面只删除界面节点；连续切换后内存上涨且旧回调仍在执行，应该改哪里？

答：组件销毁时必须取消对应的setInterval，并解除回调对外部状态的持续引用。定时器未关闭时，循环函数及其引用的变量会一直存活；只删除界面节点既阻止不了回调，也无法释放被闭包保留的数据。

3) 埋点脚本为了调试，把每次请求的完整响应都赋给未声明变量；数据量持续增加后页面占用不降，开启严格模式或改成局部变量能解决什么？

答：未声明赋值可能意外形成全局变量，使响应对象长期处于可达状态；改为受控的局部变量并在生命周期结束后解除引用，才能让数据具备回收条件。严格模式有助于尽早暴露此类赋值，但已有的全局缓存仍需显式清理。

4) 路由页反复进入退出后内存曲线持续抬升，代码里同时存在事件监听、定时器、闭包和已删除节点引用；你会怎样缩小故障范围？

答：先用Chrome的Timeline记录多轮进入、退出过程并标记内存变化，确认异常增长与哪段生命周期重合。随后逐项检查监听是否销毁、定时器是否取消、闭包和变量是否仍持有节点；单看某一次峰值不足以证明泄漏。

5) 团队想把所有闭包都改写成全局函数来治理泄漏，这个取舍合理吗？

答：不合理，闭包本身不是必然泄漏，风险来自闭包被长生命周期对象持续引用，同时又捕获了不再需要的数据。应缩小捕获范围并在定时器、监听或缓存结束时断开引用；改成全局函数反而可能引入更长生命周期的全局状态。

386. 深浅拷贝

浅拷贝只复制一层，深层对象仍共享引用；深拷贝则要为嵌套数据创建相互独立的副本。对象可用Object.assign或展开运算符浅拷贝，数组也可用slice、concat，但修改嵌套属性仍会互相影响。JSON.parse(JSON.stringify(obj))虽然简单，却会忽略undefined、函数和Symbol，还会改变Date、RegExp等类型，并且无法处理循环引用。要求完整时可递归结合Reflect.ownKeys、属性描述符和WeakMap，兼顾特殊对象、原型及循环引用。

1. 浅拷贝的原理和实现 自己创建一个新的对象，来接受你要重新复制或引用的对象值。如果对象属性是基本的数据类型，复制的就是基本类型的值给新对象；但如果属性是引用数据类型，复制的就是内存中的地址，如果其中一个对象改变了这个内存中的地址，肯定会影响到另一个对象

方法一：object.assign object.assign是ES6中object的一个方法，该方法可以用于JS对象的合并等多个用途，其中一个用途就是可以进行浅拷贝。该方法的第一个参数是拷贝的目标对象，后面的参数是拷贝的来源对象（也可以是多个来源）。object.assign的语法为：Object.assign(target, ...sources)的示例代码如下：

```

let target = {}; let source = {a: {b: 1}}; Object.assign(target, source); console.log(target); // { a: { b: 1 } };
但是使用 object.assign 方法有几点需要注意
它不会拷贝对象的继承属性；它不会拷贝对象的不可枚举的属性；可以拷贝Symbol类型的属性。 let obj1 =
{a: {b: 1}, sym: Symbol(1)}; Object.defineProperty(obj1, 'innumerable', {value: '不可枚举属性', enumerable: false});
let obj2 = {}; Object.assign(obj2, obj1); obj1.a.b = 2; console.log('obj1', obj1); console.log('obj2', obj2);
从上面的样例代码中可以看到，利用Object.assign也可以拷贝Symbol类型的对象，但是如果到了对象的第二层属性 obj1.a.b 这里的时候，前者值的改变也会影响后者的第二层属性的值，说明其中依旧存在着访问共同堆内存的问题，也就是说这种方法还不能进一步复制，而只是完成了浅拷贝的功能
方法二：扩展运算符方式 我们也可以利用 JS
的扩展运算符，在构造对象的同时完成浅拷贝的功能。扩展运算符的语法为：let cloneObj = { ...obj }; /*
对象的拷贝 */ let obj = {a: 1, b: {c: 1}}; let obj2 = {...obj}; obj.a = 2; console.log(obj); // {a: 2, b: {c: 1}}
console.log(obj2); // {a: 1, b: {c: 1}}; obj.b.c = 2; console.log(obj); // {a: 2, b: {c: 2}} console.log(obj2);
// {a: 1, b: {c: 2}} /* 数组的拷贝 */ let arr = [1, 2, 3]; let newArr = [...arr]; //跟arr.slice()是一样的效果 扩展运算符
和Object.assign有同样的缺陷，也就是实现的浅拷贝的功能差不多，但是如果属性都是基本类型的值，使用扩展运算符进行浅拷贝会更加方便
方法三：concat 拷贝数组 数组的concat方法其实也是浅拷贝，所以连接一个含有引用类型的数组时，需要注意修改原数组中的元素的属性，因为它会影响拷贝之后连接的数组。不过concat只能用于数组的浅拷贝，使用场景比较局限。代码如下所示。
let arr = [1, 2, 3]; let newArr = arr.concat(); newArr[1] = 100; console.log(arr); // [ 1, 2, 3 ]
console.log(newArr); // [ 1, 100, 3 ] 方法四：slice 拷贝数组 slice方法也比较有局限性，因为它仅仅针对数组类型。slice方法会返回一个新的数组对象，这一对象由该方法的前两个参数来决定原数组截取的开始和结束时间，是不会影响和改变原始数组的。 slice 的语法为：arr.slice(begin, end);
let arr = [1, 2, {val: 4}]; let newArr = arr.slice(); newArr[2].val = 1000; console.log(arr); // [ 1, 2, ...

```

1) 表单状态执行{...state}后，修改副本里的address.city，原表单也跟着变化；代码评审里有人坚持扩展运算符已经完成复制，你怎么解释？

答：扩展运算符只创建新的顶层对象，address这类嵌套引用仍指向同一块数据，因此修改其内部属性会同时影响两边。若只改顶层基本类型字段，浅拷贝足够；要求嵌套状态完全隔离时才需要继续复制对应层级或执行深拷贝。

2) 数据合并用Object.assign({}, source)，结果继承属性和不可枚举配置没有进入副本，但Symbol键被保留；这是实现故障还是方法边界？

答：这是Object.assign的边界：它不会复制继承属性和不可枚举属性，但能够复制可枚举的Symbol属性。若业务要求保留原型和完整属性特性，应结合Object.getOwnPropertyDescriptors与Object.create；代价是实现复杂度明显增加。

3) 日志快照包含Date、RegExp、undefined、NaN和循环引用，团队仍准备使用JSON.parse(JSON.stringify(data))；上线前你会拦住哪些风险？

答：这条路径会让Date变成字符串、RegExp变成空对象，忽略undefined，并把NaN与无穷值变成null；遇到循环引用还会直接失败。它只适合可安全表示为JSON的数据，不能当作通用深拷贝方案。

4) 候选人写的递归深拷贝遇到数组得到普通对象，遇到循环引用则无限递归；你要求在现有思路修正，会看哪些关键动作？

答：应按具体类型创建数组、Date、RegExp等对应实例，并在递归前用WeakMap记录“原对象到副本”的映射。再次遇到同一对象时直接返回已有副本即可闭合环；若只用普通对象承接所有引用类型，类型语义仍会丢失。

5) 权限模型依赖不可枚举字段、Symbol键和自定义原型，业务方又要求副本与原对象完全分离；普通递归赋值为什么不够？

答：普通for...in递归拿不到完整的自有键和属性描述，也不会自动保留原型链。可用Reflect.ownKeys枚举键，以Object.getOwnPropertyDescriptors获取特性，再通过Object.create保留原型；函数通常仍只能保留引用，不能假定得到独立副本。

6) 评审中一方主张所有接口结果都深拷贝，另一方只愿意用slice、concat或扩展运算符；你会依据什么决定？

答：先看调用方是否会修改嵌套引用：只需隔离数组容器或对象顶层时，slice、concat和扩展运算符成本更低，但它们都是浅拷贝。只有确实要求多层数据互不影响时才采用深拷贝，并明确特殊类型、属性描述和循环引用的支持范围。

387. 节流与防抖

防抖是在停止触发一段时间后执行，节流是在固定时间窗口内最多执行一次。防抖每次收到新事件都会清除旧定时器并重新计时，适合避免连续点击产生多次请求。节流则比较当前时间与上次执行时间，常用于降低scroll监听的调用频率。手写时返回函数不能用箭头函数替代外层普通函数，还要通过apply透传调用时的this和参数。函数防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内事件又被触发，则重新计时。这可以使用在一些点击请求的事件上，避免因为用户的多次点击向后端发送多次请求。函数节流是指规定一个单位时间，在这个单位时间内，只能有一次触发事件的回调函数执行，如果在同一个单位时间内某事件被触发多次，只有一次能生效。节流可以使用在scroll函数的事件监听上，通过事件节流来降低事件调用的频率。// 函数防抖的实现functiondebounce(fn,wait){var timer=null;return function(){var context=this,args=arguments;// 如果此时存在定时器的话，则取消之前的定时器重新计时if(timer){clearTimeout(timer);timer=null;}// 设置定时器，使事件间隔指定事件后执行timer=setTimeout(()=>{fn.apply(context,args);},wait);}}// 函数节流的实现functionthrottle(fn,delay){var preTime=Date.now();return function(){var context=this,args=arguments,nowTime=Date.now();// 如果两次时间间隔超过了指定时间，则执行函数。if(nowTime-preTime>=delay){preTime=Date.now();returnfn.apply(context,args);}}

1) 搜索框输入“javascript”时连续触发十次input，产品要求用户停下后只查询一次；同事却用每300ms执行一次的节流，你会怎么改？

答：这里应使用防抖：每次输入都取消旧定时器并重新计时，只有连续一段时间没有新输入才执行查询。节流会在持续输入期间按时间窗口放行回调，仍可能产生多次请求；等待时长过大还会增加结果出现的延迟。

2) 长列表监听scroll更新吸顶位置，用户持续滚动五秒时页面必须周期性刷新；若改用只在停止滚动后执行的防抖，会出现什么现象？

答：防抖会不断重新计时，持续滚动期间回调可能一直不执行，吸顶状态因而滞后到操作结束。此处更适合节流，在规定时间内最多放行一次更新以降低频率；时间窗口过大仍会让视觉反馈不够连续。

3) 支付按钮在一秒内被连点多次，接口不能重复提交；只套用示例里的尾触发防抖，业务上还缺少什么约束？

答：尾触发防抖能把密集点击合并为一次延后调用，但它不能证明请求尚未完成时不会再次提交。按钮还应在请求期间进入不可重复触发状态，并由业务层校验提交状态；防抖只是降低调用次数，不能替代接口幂等或状态控制。

4) 组件卸载后，防抖封装里的定时器仍触发旧回调并访问已销毁状态；你会怎样定位和修复这类线上报错？

答：先确认卸载前最后一次事件是否留下待执行的setTimeout，以及回调闭包捕获了哪些组件数据。封装应暴露取消能力，并在组件销毁时清除定时器；仅移除事件监听不会自动取消已经排队的尾触发任务。

5) 同一套拖拽逻辑既要按固定频率更新坐标，又要在停止后保存最终位置，只允许选纯节流或纯防抖吗？

答：不必二选一：移动过程中用节流限制坐标更新频率，停止操作后再用防抖触发最终保存，更符合两个阶段的目标。若只用节流，最终一次状态可能未及时持久化；只用防抖则持续拖动时缺少中间反馈。

388. Proxy代理

Proxy是在目标对象外层建立统一拦截，让读取、赋值、删除等操作先经过自定义的handler。创建时传入target和handler，例如访问属性触发get，修改属性触发set，因此既能监视外部访问，也能在复杂操作前做校验或资源管理。它还支持原型、属性描述符、函数调用和构造等多类操作，不只是简单的属性读写。由于这些能力无法由Object.defineProperty完整模拟，所以现有polyfill只能覆盖部分功能。

proxy在目标对象的外层搭建了一层拦截，外界对目标对象的某些操作，必须通过这层拦截

```
var proxy = new Proxy(target, handler);
```

new Proxy()表示生成一个Proxy实例，target参数表示所要拦截的目标对象，handler参数也是一个对象，用来定制拦截行为

```
var target = {name: 'poetries'};
var logHandler = {
  get: function(target, key) {
    console.log(`${key}被读取`);
    return target[key];
  },
  set: function(target, key, value) {
    console.log(`${key}被设置为${value}`);
    target[key] = value;
  }
};
var targetWithLog = new Proxy(target, logHandler);
targetWithLog.name; // 控制台输出：name 被读取
targetWithLog.name = 'others'; // 控制台输出：name 被设置为 others
console.log(target.name); // 控制台输出：others
targetWithLog读取属性的值时，实际上执行的是logHandler.get：在控制台输出信息，并且读取被代理对象target的属性。在targetWithLog设置属性值时，实际上执行的是logHandler.set：在控制台输出信息，并且设置被代理对象target的属性的值 // 由于拦截函数总是返回35，所以访问任何属性都得到35
var proxy = new Proxy({}, {
  get: function(target, property) {
    return 35;
  }
});
proxy.time; // 35
proxy.name; // 35
proxy.title; // 35
Proxy实例也可以作为其他对象的原型对象
var proxy = new Proxy({}, {
  get: function(target, property) {
    return 35;
  }
});
let obj = Object.create(proxy);
obj.time; // 35
proxy对象是obj对象的原型，obj对象本身并没有time属性，所以根据原型链，会在proxy对象上读取该属性，导致被拦截
```

Proxy的作用

对于代理模式Proxy的作用主要体现在三个方面

拦截和监视外部对对象的访问降低函数或类的复杂度在复杂操作前对操作进行校验或对所需资源进行管理

Proxy所能代理的范围--handler 实际上 handler 本身就是ES6所新设计的一个对象.它的作用就是用来自定义代理对象的各种可代理操作

。它本身一共有13中方法,每种方法都可以代理一种操作.其13种方法如下

//

在读取代理对象的原型时触发该操作，比如在执行

Object.getPrototypeOf(proxy)

时。handler.getPrototypeOf()//

在设置代理对象的原型时触发该操作，比如在执行

Object.setPrototypeOf(proxy, null)

时。handler.setPrototypeOf()//

在判断一个代理对象是否是可扩展时触发该操作，比如在执行

Object.isExtensible(proxy)

时。handler.isExtensible()//

在让一个代理对象不可扩展时触发该操作，比如在执行

Object.preventExtensions(proxy)

时。handler.preventExtensions()//

在获取代理对象某个属性的属性描述时触发该操作，比如在执行

Object.getOwnPropertyDescriptor(proxy, "foo")

时。handler.getOwnPropertyDescriptor()//

在定义代理对象某个属性时的属性描述时触发该操作，比如在执行 Object.defineProperty(proxy, "foo", {})

时。andler.defineProperty()// 在判断代理对象是否拥有某个属性时触发该操作，比如在执行 "foo" in proxy...

1) 权限对象套上Proxy后，开发者保留了原始target，随后直接执行target.role = 'admin'，日志里没有任何拦截记录；代理失效了吗？

答：代理没有失效，只有经由代理对象发起的操作才会进入handler。直接修改原始target会绕过外层拦截，但代理读取时仍能看到目标对象的新值；若审计必须完整，就不能继续向业务代码暴露原始对象引用。

2) 配置中心希望读取任意不存在的字段都返回35，但代理同时被设为另一个对象的原型；访问子对象缺失字段时为什么也会得到35？

答：子对象自身找不到属性时会沿原型链读取代理对象，从而触发代理的get拦截。由于该拦截无条件返回35，缺失字段也得到同一结果；这种默认值策略会掩盖拼写错误，生产配置通常需要区分真实属性与兜底值。

3) 一个校验层既要拦截属性赋值和删除，又要监控in判断及键枚举；只实现get、set为什么覆盖不了需求？

答：不同对象操作对应不同陷阱：赋值使用set，删除使用deleteProperty，in判断使用has，键枚举则涉及ownKeys。只实现get、set只能监控读写属性值；选择Proxy前应列出必须覆盖的操作并配置相应handler。

4) 线上发现代理对象的普通读写有日志，但Object.getPrototypeOf(proxy)、Object.defineProperty(proxy, 'x', desc)完全没进入现有埋点；你会检查哪里？

答：应检查handler是否实现了对应的getPrototypeOf和defineProperty陷阱，而不是继续修改get、set。Proxy会按操作类型分派拦截，普通属性读写正常不能证明其他操作已覆盖；遗漏陷阱时相关行为会直接落到目标对象。

5) 项目必须运行在无法原生支持Proxy的旧环境，架构师提出用Object.defineProperty做完整垫片；你会如何做选型判断？

答：不能把Object.defineProperty视为完整的Proxy替代，因为它无法模拟全部代理语义，现有垫片也只能覆盖部分能力，例如get、set。若业务依赖删除、枚举、原型或函数调用等拦截，应调整兼容目标或改写方案，而不是承诺透明降级。

6) 安全团队想用Proxy同时包装普通对象、函数和构造器，要求分别审计读取、调用与new；应该如何映射这些行为？

答：普通属性读取进入get，直接调用函数代理进入apply，通过new构造实例则进入construct。三类行为需要分别实现和校验，单靠属性读取陷阱无法审计调用；代理层还能集中管理资源，但会增加行为边界和调试复杂度。

389. Ajax

Ajax是浏览器通过JavaScript异步发起HTTP请求，并局部更新页面而不必整页刷新的通信方式。原生实现通常用XMLHttpRequest：创建对象、配置地址、发送请求，再通过readyState和status判断响应。实际使用时可以封装成Promise，成功时resolve数据，请求失败或网络异常时reject，让调用链更清晰。

它是一种异步通信的方法，通过直接由js脚本向服务器发起http通信，然后根据服务器返回的数据，更新网页的相应部分，而不用刷新整个页面的一种方法。

面试手写（原生）：
//1：创建Ajax对象var xhr = window.XMLHttpRequest ? new XMLHttpRequest() : new ActiveXObject('Microsoft.XMLHTTP');// 兼容IE6及以下版本
//2：配置

Ajax请求地址xhr.open('get', 'index.xml', true);
//3：发送请求xhr.send(null);
// 严谨写法
//4：监听请求，接受响应xhr.onreadystatechange = function() { if (xhr.readyState == 4 && xhr.status == 200 || xhr.status == 304) console.log(xhr.responseText); }
jQuery写法 \$.ajax({ type: 'post', url: '', async: true, // async 异步 sync

```

同步 data:data, //针对post请求 dataType:'jsonp', success:function (msg) { }, error:function (error) { }
}) promise 封装实现： // promise 封装实现：functiongetJSON(url){// 创建一个 promise
对象letpromise=newPromise(function(resolve,reject){letxhr=newXMLHttpRequest();// 新建一个 http
请求xhr.open("GET",url,true);//
设置状态的监听函数xhr.onreadystatechange=function(){if(this.readyState!==4)return;//
当请求成功或失败时，改变 promise
的状态if(this.status===200){resolve(this.response);}else{reject(newError(this.statusText));};//
设置错误监听函数xhr.onerror=function(){reject(newError(this.statusText));};//
设置响应的数据类型xhr.responseType="json";//
设置请求头信息xhr.setRequestHeader("Accept","application/json");// 发送 http
请求xhr.send(null);};returnpromise;}

```

1) 订单页发起原生XHR后，服务端已经返回200，但页面偶尔在响应尚未完整时就开始渲染，你会检查哪段判断？

答：先检查回调是否同时判断了readyState === 4和成功状态码，不能只看到status === 200就消费响应。readyState === 4表示请求过程已经完成，再读取response或responseXML；条件组合还应加括号，避免&&、||优先级造成误判。

2) 老项目要把十几个读取配置的XHR调用改成Promise，调用方希望直接拿到JSON，你会怎样封装成功、失败和解析过程？

答：在封装内创建XMLHttpRequest，调用open("GET", url, true)，并把responseType设为json。状态完成且为200时用resolve(xhr.response)，其他状态和onerror都调用reject；若服务端返回的内容不是合法JSON，解析行为仍可能失败，需要由调用方处理拒绝。

3) 接口网关把成功响应从200调整为304，原来的请求封装却统一进入失败分支，你会改哪条判定？

答：当前封装若只接受status === 200，自然会把304当作失败，需要按接口约定补充允许的状态。修改前应确认304对该请求是否确实代表可用结果，并检查客户端是否有可复用内容；机械放宽为所有状态都成功，会掩盖真实的服务端错误。

4) 商品页请求一直没有进入成功回调，控制台也看不到明确异常；代码里写着xhr.onreadystatechange和xhr.readyState，你怎么定位？

答：这里首先是属性名拼写错误，正确写法是onreadystatechange和readyState，错误属性不会建立预期的状态监听。修正后再记录readyState、status和statusText，区分请求未完成、HTTP状态失败与网络错误；只盯着响应内容无法定位监听器根本没执行的情况。

5) 数据看板每分钟请求一次JSON，团队争论继续写回调还是统一封装成Promise；在不引入第三方库的前提下你怎么选？

答：保留原生XHR作为通信能力，但对外返回Promise，让调用方用统一的成功和失败链路组织刷新逻辑。封装中集中设置Accept、responseType、状态判断和onerror，可减少各页面重复代码；代价是仍要明确哪些HTTP状态算成功，不能把协议约定藏进模糊的通用封装。

390. 深入数组

理解数组方法时，我一般先看它是否修改原数组、返回什么，以及能处理哪类数据。push、splice、sort等会改变自身，concat、slice、filter等不会；map、reduce、find则分别适合映射、累计和查找。类数组只有索引和length，像arguments、HTMLCollection、NodeList，需要时可用Array.from或slice.call转成数组。一、梳理数组 API 1.Array.of Array.of用于将参数依次转化为数组中的一项，然后返回这个新数组

，而不管这个参数是数字还是其他。它基本上与 `Array` 构造器功能一致，唯一的区别就在单个数字参数的处理上 `Array.of(8.0);` // `[8]Array(8.0);` // `[empty × 8]Array.of(8.0,5);` // `[8, 5]Array(8.0,5);` // `[8, 5]Array.of('8');` // `["8"]Array('8');` // `["8"]` 2. `Array.from` 从语法上看，`Array.from` 拥有 3 个参数：类似数组的对象，必选；加工函数，新生成的数组会经过该函数的加工再返回；`this`作用域，表示加工函数执行时`this`的值。这三个参数里面第一个参数是必选的，后两个参数都是可选的。我们通过一段代码来看看它的用法。

```
var obj={0:'a',1:'b',2:'c',length:3};Array.from(obj,function(value,index){console.log(value,index,this,arguments.length);return value.repeat(3);//必须指定返回值，否则返回 undefined},obj);// return 的 value 重复了三遍，最后返回的数组为 ["aaa","bbb","ccc"]// 如果这里不指定 this 的话，加工函数完全可以是一个箭头函数。上述代码可以简写为如下形式。Array.from(obj,(value)=>value.repeat(3));// 控制台返回 (3) ["aaa", "bbb", "ccc"] 除了上述obj对象以外，拥有迭代器的对象还包括String、Set、Map等，Array.from 统统可以处理，请看下面的代码。 // StringArray.from('abc');// ["a", "b", "c"]// SetArray.from(new Set(['abc','def']));// ["abc", "def"]// MapArray.from(new Map([[1,'ab'],[2,'de']]));// [[1, 'ab'], [2, 'de']] 3.Array 的判断 在 ES5 提供该方法之前，我们至少有如下 5 种方式去判断一个变量是否为数组。 var a=[];
1.基于instanceofinstanceofArray;// 2.基于constructora.constructor===Array;//
3.基于Object.prototype.isPrototypeOfArray.prototype.isPrototypeOf(a);//
4.基于getPrototypeOfObject.getPrototypeOf(a)===Array.prototype;//
5.基于Object.prototype.toStringObject.prototype.toString.apply(a)==='[object Array]'; ES6 之后新增了一个Array.isArray方法，能直接判断数据类型是否为数组，但是如果 isArray 不存在，那么Array.isArray的 polyfill 通常可以这样写：
if(!Array.isArray){Array.isArray=function(arg){return Object.prototype.toString.call(arg)==='[object Array]';}} 4. 改变自身的方法 基于 ES6，会改变自身值的方法一共有9个，分别为pop、push、reverse、shift、sort、splice、unshift，以及两个 ES6 新增的方法 copyWithin 和 fill // pop方法var array=["cat","dog","cow","chicken","mouse"];var item=array.pop();console.log(array);// ["cat", "dog", "cow", "chicken"]console.log(item);// m...
```

1) 报表代码写了`Array(8)`想得到`[8]`，结果页面生成了 8 个空槽；如果参数来自用户输入，你会怎么修？

答：单个数字传给`Array`会被解释为数组长度，因此结果是包含 8 个空槽的数组，而不是`[8]`。需要把参数作为一个元素时应使用`Array.of(value)`；继续依赖构造器会让数字参数与字符串参数产生不同语义，容易形成数据类型相关的隐患。

2) 接口返回`{0:"a",1:"b",2:"c",length:3}`，列表组件既要转成数组又要把每项重复三次，你会怎样写而不再遍历一轮？

答：可以直接使用`Array.from(data, value => value.repeat(3))`，转换类数组的同时完成映射，得到`['aaa','bbb','ccc']`。映射函数必须显式返回加工后的值，否则对应位置会变成`undefined`；若回调依赖普通函数的`this`，还可通过第三个参数传入作用域。

3) 微前端页面从另一个执行环境传来一个数组，旧代码用`value instanceof Array`判断后走错分支，你会替换成什么？

答：应优先改用`Array.isArray(value)`，它直接表达数组判断，也避免把判断绑定到当前环境的`Array`构造器。需要兼容缺少该方法的旧环境时，可回退到`Object.prototype.toString.call(value) === '[object Array]'`；依赖`constructor`或原型相等会受到对象来源和原型变化影响。

4) 表格排序后发现上游缓存里的原始顺序也被改了，代码只有`const result = rows.sort()`；为什么两个变量一起变化？

答：`sort`会直接修改原数组，并返回同一个数组引用，所以`result`与`rows`指向同一对象。若上游数据必须保持不变，应先用`rows.slice()`或`rows.concat()`复制，再对副本排序；浅复制只能隔离数组结构，元素对象内部的修改仍会共享。

5) 批量删除标签时，开发者以为`splice(1,`

2)返回删除后的原数组，结果后续保存的数据只剩两个标签；你如何解释返回值约定？

答：`splice`会修改原数组，但返回的是被删除元素组成的新数组，而不是修改后的原数组。保存剩余列表应继续使用原变量，若要记录删除项再接收`splice`的返回值；这类接口同时具有副作用和返回值，混用时尤其容易覆盖业务数据。

6) 页面用`querySelectorAll`和`getElementsByClassName`各拿到一批节点，DOM更新后两边数量表现不同，你会怎样处理遍历快照？

答：两者得到的都是类数组，但`HTMLCollection`会随文档结构及时更新，而`querySelectorAll`返回的`NodeList`在该场景下适合按查询结果遍历。需要稳定快照时可用`Array.from(collection)`转成真正数组，再使用`map`、`filter`等数组方法；转换后的数组不会继续自动反映DOM变化。

【HTML】

391. meta 标签：自动刷新/跳转

`meta refresh`可以让页面在指定时间后自动刷新，或者跳转到另一个地址。例如`content="5; URL=page2.html"`表示五秒后跳转，去掉URL后，`content="60"`就表示每隔六十秒刷新。它适合简单的页面轮播或监控大屏刷新；如果还要根据业务状态决定是否跳转，单靠这个标签就不够灵活。假设要实现一个类似 PPT 自动播放的效果，你很可能会想到使用 JavaScript 定时器控制页面跳转来实现。但其实有更加简洁的实现方法，比如通过 `meta` 标签来实现：上面的代码会在 5s 之后自动跳转到同域下的 `page2.html` 页面。我们要实现 PPT 自动播放的功能，只需要在每个页面的 `meta` 标签内设置好下一个页面的地址即可。另一种场景，比如每隔一分钟就需要刷新页面的大屏幕监控，也可以通过 `meta` 标签来实现，只需去掉后面的 URL 即可：`meta viewport`相关

1) 产品把展厅页面做成多页PPT，要求每页停留 5

秒后进入同域的下一页，不想维护定时器代码，你会在页面头部写什么？

答：可以在当前页的中设置，浏览器会在 5 秒后跳到指定页面。每一页都要配置自己的下一页地址；任何一页地址错误或缺失都会中断播放链路，而且这种方式不负责保存页面运行状态。

2) 监控大屏要求每 60

秒整页刷新一次，值班同事却复制了带URL的模板，结果页面不断跳走；正确配置是什么？

答：定时刷新当前页面时应去掉目标地址，使用。带上URL表示延时跳转而非单纯刷新，因此排查时先检查`content`中是否包含分号后的地址；整页刷新会重建页面，不适合必须保留前端临时状态的界面。

3) 同一套展示页部署到不同域名，`content="5;`

`URL=page2.html"`是否需要为每个环境重写完整域名？

答：示例中的page2.html是同源相对地址，因此同样的页面结构通常不必写死完整域名。部署时仍要保证目标文件与当前路径关系一致，否则相对地址会解析到错误位置；跨域跳转或目录结构不同的环境不能直接套用这一配置。

4) 下线页设置了Refresh，测试反馈等待 5 秒后仍停在原页，你会按什么顺序检查？

答：先确认标签位于文档内，并核对http-equiv="Refresh"、秒数及URL的拼写和引号。随后直接访问目标地址，排除文件不存在或相对路径错误；如果页面还有其他跳转逻辑，也要避免多套机制同时控制导航而让现象难以判断。

5) 搜索落地页既要自动轮播，又希望搜索结果准确展示页面摘要，只有Refresh标签够吗？

答：不够，Refresh只负责延时刷新或跳转，不提供页面描述信息。还应设置语义清晰的，供搜索引擎提取和展示摘要；描述应反映当前页面内容，不能把导航控制与搜索信息混成一个标签。

392. viewport

viewport用来控制移动端页面的视口宽度、初始缩放比例，以及用户能否手动缩放。常见写法是设置width=device-width和initial-scale=1.0，让页面按设备宽度布局。minimum-scale、maximum-scale、user-scalable可以继续限制缩放范围，但要结合交互需求取舍。处理移动端细线时，也可以配合rem和transform: scale(0.5)做局部缩放。 // width 设置viewport宽度，为一个正整数，或字符串 'device-width' // device-width 设备宽度// height 设置viewport高度，一般设置了宽度，会自动解析出高度，可以不用设置// initial-scale 默认缩放比例（初始缩放比例），为一个数字，可以带小数// minimum-scale 允许用户最小缩放比例，为一个数字，可以带小数// maximum-scale 允许用户最大缩放比例，为一个数字，可以带小数// user-scalable 是否允许手动缩放 延伸提问怎样处理移动端1px被渲染成2px问题 怎样处理 移动端1px被渲染成2px问题 局部处理 meta标签中的viewport属性，initial-scale设置为1rem按照设计稿标准走，外加利用transform的scale(0.5)缩小一倍即可； 全局处理 meta标签中的viewport属性，initial-scale设置为0.5rem按照设计稿标准走即可

1) 设计稿标注页面宽度跟随手机屏幕，开发却把viewport的width写成固定数字，横竖屏下布局都不对；你会怎么改？

答：应将width设置为device-width，让布局视口宽度跟随设备宽度，并通常配合initial-scale=1.0。固定宽度会让浏览器按同一个布局视口缩放不同设备上的页面；改完仍需检查CSS是否另有固定宽度，否则仅调整meta不能解决全部溢出。

2) 活动页要求初始按原尺寸展示，同时允许用户在1到3倍之间缩放，你会怎样调整现有的user-scalable=no？

答：应改为允许手动缩放，并设置initial-scale=1、minimum-scale=1、maximum-scale=3。这几个值分别控制初始、最小和最大缩放比例，user-scalable=no与允许缩放的需求直接冲突；范围仍要结合页面布局验证，避免放大后关键操作区域被遮挡。

3) 移动端边框按设计稿写成1px，在高像素密度设备上视觉却像2px；只改这个组件时你会怎么处理？

答：局部处理可保持的initial-scale=1，页面的rem继续按设计稿标准计算，再对细线元素使用transform: scale(0.5)缩小。缩放会同时影响元素视觉尺寸和定位关系，因此应把变换限制在边框或专用伪元素上，避免整个业务组件一起缩小。

4) 团队想用initial-scale=0.5一次解决全站的移动端细线问题，但站内已有大量按现行rem编写的页面，你会直接合并吗？

答：不会直接合并，因为把initial-scale调为0.5属于全局视口策略，所有页面的初始缩放都会受到影响。只有在整站rem规则都按对应设计标准统一时才适合采用，并需要完整回归布局；已有页面单位体系不一致时，局部transform的影响范围更可控。

5) 线上页面在部分手机上显得异常放大，代码同时设置了initial-scale=1、minimum-scale=1和maximum-scale=1；你会先核对哪些配置含义？

答：先确认width是否为device-width，再检查三个缩放参数是否确实为合法数字，以及user-scalable是否符合产品要求。initial-scale决定初始比例，另外两个只限定允许的缩放边界，不能替代正确的视口宽度；若配置无误，还要继续检查页面自身的固定尺寸和rem基准。

393. 性能优化

HTML层面的性能优化，本质上是减少脚本阻塞，并提前完成关键资源的连接或加载。普通script会暂停解析，async在下载完成后立即执行，defer和默认的类型="module"则等文档解析完成后执行。跨域资源可按需要使用dns-prefetch、preconnect、prefetch或preload，其中preload的加载确定性更高。搜索场景还应设置meta description，方便搜索引擎提取页面描述。性能优化是前端开发中避不开的问题，性能问题无外乎两方面原因：渲染速度慢、请求时间长。性能优化虽然涉及很多复杂的原因和解决方案，但其实只要通过合理地使用标签，就可以在在一定程度上提升渲染速度以及减少请求时间

1. script 标签：调整加载顺序提升渲染速度 由于浏览器的底层运行机制，渲染引擎在解析 HTML 时，若遇到 script 标签引用文件，则会暂停解析过程，同时通知网络线程加载文件，文件加载后会切换至 JavaScript 引擎来执行对应代码，代码执行完成之后切换至渲染引擎继续渲染页面。在这一过程中可以看到，页面渲染过程中包含了请求文件以及执行文件的时间，但页面的首次渲染可能并不依赖这些文件，这些请求和执行文件的动作反而延长了用户看到页面的时间，从而降低了用户体验。为了减少这些时间损耗，可以借助 script 标签的

3 个属性来实现。 async 属性。立即请求文件，但不阻塞渲染引擎，而是文件加载完毕后阻塞渲染引擎并立即执行文件内容 defer 属性。立即请求文件，但不阻塞渲染引擎，等到解析完 HTML 之后再执行文件内容HTML5 标准 type 属性，对应值为“module”。让浏览器按照 ECMA Script 6

标准将文件当作模块进行解析，默认阻塞效果同 defer，也可以配合 async 在请求完成后立即执行。绿色的线表示执行解析 HTML，蓝色的线表示请求文件，红色的线表示执行文件 当渲染引擎解析 HTML 遇到 script 标签引入文件时，会立即进行一次渲染。所以这也就是为什么构建工具会把编译好的引用 JavaScript 代码的 script 标签放入到 body 标签底部，因为当渲染引擎执行到 body 底部时会先将已解析的内容渲染出来，然后再去请求相应的 JavaScript 文件

2. link 标签：通过预处理提升渲染速度 在我们对大型单页应用进行性能优化时，也许会用到按需懒加载的方式，来加载对应的模块，但如果能合理利用link标签的rel属性值来进行预加载，就能进一步提升渲染速度。

dns-prefetch。当link标签的rel属性值为“dns-prefetch”时，浏览器会对某个域名预先进行 DNS 解析并缓存。这样，当浏览器在请求同域名资源的时候，能省去从域名查询 IP 的过程，从而减少时间损耗。下图是淘宝网设置的 DNS 预解析preconnect。让浏览器在一个 HTTP 请求正式发给服务器前预先执行一些操作，这包括DNS 解析、TLS 协商、TCP 握手，通过消除往返延迟来为用户节省时间prefetch/preload。两个值都是让浏览器预先下载并缓存某个资源，但不同的是，prefetch 可能会在浏览器忙时被忽略，而preload

则是一定会被预先下载。prerender。浏览器不仅会加载资源，还会解析执行页面，进行预渲染 这几个属性值恰好反映了浏览器获取资源文件的过程，在这里我绘制了一个流程简图，方便你记忆。

3. 搜索优化 meta 标签：提取关键信息通过 meta

标签可以设置页面的描述信息，从而让搜索引擎更好地展示搜索结果。示例 通过 meta

标签可以设置页面的描述信息，从而让搜索引擎更好地展示搜索结果。示例

1) 首页把统计脚本直接放在，网络稍慢时正文长期不出现；浏览器遇到普通外链script时发生了什么？

答：渲染引擎解析到普通外链script时会暂停HTML解析，等待文件请求完成并切换到JavaScript引擎执行，之后才继续渲染。若首屏不依赖该脚本，这段下载和执行时间就会推迟内容出现；脚本若必须同步生成后续DOM，则不能只为提速而随意改成异步。

2) 页面有两个存在依赖关系的初始化脚本，要求下载期间不阻塞HTML，并在文档解析完成后按顺序执行，你选async还是defer？

答：这里应选defer，它会立即请求文件但不阻塞HTML解析，并在文档解析完成后执行。async会在文件下载完毕后立即打断解析并执行，多个文件的完成先后也可能不符合依赖顺序；依赖脚本使用async容易出现偶发初始化失败。

3) 团队把入口改成后又额外加了async，结果模块在HTML尚未解析完时访问节点失败；差异在哪里？

答：模块脚本默认具有类似defer的阻塞效果，会在HTML解析完成后执行；加入async后则会在请求完成时尽快执行。入口代码依赖页面节点时不应贸然添加async，或必须让代码自行等待DOM就绪；模块化并不意味着执行时机可以忽略。

4) 首屏字体来自独立CDN，性能记录显示请求前耗时主要落在域名解析、连接和TLS协商，你会配置dns-prefetch还是preconnect？

答：这种场景更适合对该CDN使用preconnect，因为它可以提前完成DNS解析、TCP握手和TLS协商。dns-prefetch只预先解析域名，覆盖不了后续连接成本；预连接应留给确定很快会访问的关键域名，过度配置会产生没有收益的连接工作。

5) 路由懒加载的下一页资源有时根本没被提前下载，代码使用的是prefetch；产品要求进入下一页前必须准备好，你会怎么取舍？

答：prefetch在浏览器繁忙时可能被忽略，因此不能把它当作必须完成的下载保证；确定当前流程必需的资源可考虑改用preload。preload会更积极地预先下载，但也会占用当前页面的网络资源，若下一页并非高概率访问，反而可能挤压首屏关键请求。

6)

搜索结果中的页面摘要混乱，开发准备靠增加预加载标签修复；你会把排查方向转到哪里？

答：预加载标签解决的是资源获取和渲染时机，不负责搜索结果摘要，应检查页面是否设置了准确的。描述内容需要提炼当前页面的关键信息，帮助搜索引擎展示结果；它与preload、preconnect属于不同优化目标，混用不会修复摘要质量。

394. 如何高效操作DOM

高效操作DOM的核心是减少频繁访问和修改，因为它既会带来引擎切换，也可能触发重排与重绘。循环中反复读取节点时，我一般先把节点引用缓存到循环外，避免重复访问document.body。批量创建元素时，可以先拼好完整的HTML，再通过一次innerHTML更新，减少多次插入。修改尺寸、边距或增删节点可能引发重排，而颜色、背景等变化通常只需重绘，成本并不完全相同。

1. 为什么说 DOM 操作耗时

1.1 线程切换 浏览器为了避免两个引擎同时修改页面而造成渲染结果不一致的情况，增加了另外一个机制，这两个引擎具有互斥性，也就是说在某个时刻只有一个引擎在运行，另一个引擎会被阻塞。操作系统在进行线程切换的时候需要保存上一个线程执行时的状态信息并读取下一个线程的状态信息，俗称上下文切换。而这个操作相对而言是比较耗时的每次 DOM 操作就会引发线程的上下文切换——从 JavaScript 引擎切换到渲染引擎执行对应操作，然后再切换回 JavaScript 引擎继续执行，这就带来了性能损耗。单次切换消耗的时间是非常少的，但是如果频繁地大量切换，那么就会产生性能问题

比如下面的测试代码，循环读取一百万次 DOM 中的 body 元素的耗时是读取 JSON 对象耗时的 10 倍。//
测试次数：一百万次const times=1000000//
缓存body元素console.time('object')let body=document.body//
循环赋值对象作为对照参考for(let i=0;i<times;i++){document.body.innerHTML+=html//
31.919921875msconsole.timeEnd('innerHTML')}

1) 商品详情页只循环读取一万次document.body，既没改样式也没增删节点，为什么仍可能明显慢于读取已缓存的普通对象？

答：瓶颈不一定来自重排，频繁访问DOM本身也需要在JavaScript执行与渲染相关能力之间反复交互。应在循环外缓存document.body或目标节点，循环内复用引用；但缓存节点只能减少重复查询，无法消除后续样式修改引起的渲染成本。

2) 数据看板循环处理一千个卡片，每轮先读offsetHeight，再写style.height，页面滚动时持续卡顿，你会怎样重构？

答：交替读写会让前一次修改影响布局，随后的尺寸读取可能迫使浏览器立即完成样式和布局计算。先批量读取高度并保存，再集中写入样式，必要时把写入安排到requestAnimationFrame；若每帧仍处理大量节点，还要减少参与渲染的元素数量。

3) 后台表格一次追加一万个，开发者在循环里反复appendChild，另一位坚持拼字符串后一次写入innerHTML，你如何判断？

答：两种写法的关键差异是提交给页面的次数，先在内存中完成构造再一次更新通常更符合批量操作原则。纯静态且内容可信时可一次赋值innerHTML，需要保留节点引用或逐项绑定行为时更适合离线创建后统一挂载；用户输入未经处理时不能直接拼入HTML。

4) 线上筛选器只改了文字颜色却出现闪烁，团队直接认定发生了重排，你会从哪里纠正并排查？

答：文字颜色变化通常只要求重绘，不能仅凭视觉闪烁断定发生重排；尺寸、边距、节点增删等才更可能改变元素排布。应在性能面板查看样式计算、布局与绘制记录，并定位同一交互中是否还读取了布局属性或修改了几何样式。

5) 消息列表从一千条增长到十万条，已经把节点创建改成一次批量插入，首屏仍然很慢，继续优化innerHTML还有意义吗？

答：批量插入只能减少操作次数，十万个节点仍会扩大样式计算、排布和绘制的工作量，继续微调字符串拼接通常不是核心矛盾。应分页或虚拟化，只保留视口附近节点；代价是滚动定位、动态高度和可访问性处理会更复杂。

【CSS基础】

395. 盒模型

盒模型从内到外由content、padding、border和margin组成，元素最终占用多大空间取决于box-sizing。默认的content-box中，声明的width只表示内容宽度，总宽度还要加上内边距、边框和外边距。使用border-box时，width已包含padding与border，因此总宽度只需再计算margin。如果设置为inherit，元素会沿用父元素的盒模型规则。 content（元素内容） + padding（内边距） + border（边框） + margin（外边距） 延伸：box-sizing content-box：默认值，总宽度

=margin+border+padding+width
border-box：盒子宽度包含padding和border，总宽度 = margin + width
inherit：从父元素继承box-sizing属性

1) 登录页输入框写了width: 200px; padding: 0 12px; border: 1px solid，验收量到的占用宽度却是226px，是哪一层计算错了？

答：默认content-box中的200px只表示内容区，左右内边距增加24px，左右边框再增加2px，所以边框盒宽度是226px。改为box-sizing: border-box后，内边距和边框被计入声明宽度，但外边距仍在盒子之外。

2) 组件库要求所有表单控件按设计稿总宽度布局，你会怎样设置box-sizing，又为什么要覆盖伪元素？

答：可在根元素声明box-sizing: border-box，再让普通元素及::before、::after通过inherit继承，统一宽度口径。伪元素也可能带内边距和边框，遗漏后会出现局部计算差异；接入依赖content-box的旧组件时需为其作用域显式恢复。

3) 一个width: 100%的侧栏在窄屏仍横向溢出，它还有左右padding、border和margin，切成border-box就一定能修好吗？

答：border-box只把内边距和边框纳入100%，左右外边距依然额外占据空间，因此仍可能超过父容器。应同时检查margin、父级可用宽度和内容最小尺寸；若内容本身不可收缩，单改盒模型也不会消除溢出。

4) 线上卡片声明width:

320px，开发者工具显示内容区不足320px，团队怀疑浏览器算错，你会核对哪些属性？

答：先看计算后的box-sizing；若为border-box，声明宽度代表边框盒，内容区会扣除左右padding与border。再检查规则是否由全局继承、组件样式或伪元素覆盖，并分别读取内容盒与边框盒尺寸，避免把不同测量口径混为一谈。

5) 设计系统在content-box和border-box之间选全局默认值，表单、栅格和第三方富文本组件同时存在，你怎么取舍？

答：自有组件按设计稿总宽度布局时，border-box能减少增加内边距或边框后尺寸意外膨胀，通常更易维护。切换全局默认值会改变既有组件的尺寸语义，应先做视觉回归，并给依赖content-box的第三方子树设置明确边界；盒模型统一不等于可以忽略外边距。

396. BFC

BFC是独立的块级布局区域，内部元素的布局不会直接影响外部元素。它可由根元素、浮动元素、绝对或固定定位，以及特定的display、非visible的overflow等方式触发。它不会与外部浮动区域重叠，计算高度时也会包含浮动子元素，所以常用于清除内部浮动或实现自适应两栏布局。需要注意，同一BFC内相邻块的垂直margin仍会重叠，要隔离它们需建立不同的BFC。块级格式化上下文，是一个独立的渲染区域，让处于BFC内部的元素与外部的元素相互隔离，使内外元素的定位不会相互影响。

IE下为Layout，可通过zoom:1触发 触发条件：根元素position: absolute/fixeddisplay: inline-block / tablefloat元素overflow != visible 规则：属于同一个BFC的两个相邻Box垂直排列属于同一个BFC的两个相邻Box的margin会发生重叠BFC中子元素的margin box的左边，与包含块 (BFC)border box的左边相接触 (子元素absolute除外)BFC的区域不会与float的元素区域重叠计算BFC的高度时，浮动子元素也参与计算文字层不会被浮动层覆盖，环绕于周围 应用：阻止margin重叠可以包含浮动元素——清除内部浮动(清除浮动的原理是两个div都位于同一个BFC区域之中)自适应两栏布局可以阻止元素被浮动元素覆盖

1) 订单页两个垂直卡片分别有margin-bottom: 20px和margin-top: 30px，实际间距只有30px，为什么简单相加不成立？

答：两个卡片处于同一BFC且垂直相邻时，上下外边距会发生重叠，因此不能按50px计算。工程上优先让间距只由一侧负责，或用父级布局统一控制；额外创建BFC虽能隔离重叠，却会改变布局边界。

2) 资讯流容器内部全是浮动卡片，父容器高度变成零，后面的页脚顶了上来，你会如何让父级重新包住内容？

答：让父容器建立BFC后，计算其高度时会把浮动子元素纳入，从而恢复对页脚的正常推开。可根据约束选择display:

flow-root，旧代码也常见非visible的overflow；后者可能裁剪阴影、菜单或其他溢出内容。

3) 两栏页面左侧头像float:

left，右侧正文背景却伸到头像下面，产品要求正文区域始终不与头像重叠，你会改哪一层？

答：可让右侧正文建立新的BFC，其区域不会与浮动元素的区域重叠，从而形成自适应剩余宽度的两栏效果。触发方式会附带不同布局语义，例如overflow: hidden可能裁剪内容，position:

absolute又会脱离常规文档流，不能只为触发而盲选。

4) 线上下拉菜单被截断，排查发现祖先为清除内部浮动设置了overflow: hidden，怎样确认并修复而不让容器再次塌陷？

答：先临时关闭该祖先的overflow，确认菜单恢复且父容器高度随之丢失，就能验证裁剪与包含浮动来自同一规则。可改用display: flow-root，或使用清除浮动的伪元素方案；若必须兼容特定旧环境，则需结合兼容范围选择替代实现。

5) 同事提议给所有模块都加overflow: hidden来隔离布局，你会接受这种全局策略吗？

答：不应把overflow: hidden当作无成本的布局重置，它虽然能建立BFC，也会改变溢出内容的可见性。只在确实需要包含浮动、阻止外边距重叠或避开浮动区域的局部容器使用，并优先选择副作用符合需求的触发方式。

397. 层叠上下文

z-index只在同一个层叠上下文中比较，并不是数值越大全局层级就越高。根元素会创建层叠上下文，定位元素以及带有transform、opacity、filter等属性的元素也可能形成新的上下文。子元素的z-index再大，也无法直接越过父级上下文与外部元素竞争。遇到弹窗被遮挡时，我一般先检查祖先元素是否创建了独立层叠上下文。元素提升为一个比较特殊的图层，在三维空间中（z轴）高出普通元素一等。触发条件根层叠上下文(html)positioncss3属性flextransformopacityfilterwill-changewebkit-overflow-scrolling flextransformopacityfilterwill-changewebkit-overflow-scrolling 层叠等级：层叠上下文在z轴上的排序在同一层叠上下文中，层叠等级才有意义z-index的优先级最高

1) 支付弹窗设置了position: fixed; z-index: 9999，仍被导航栏的z-index: 10盖住，为什么数值更大却输了？

答：z-index只在所属层叠上下文内比较，弹窗若被包在较低层级的祖先上下文中，9999无法越过祖先与导航栏竞争。应检查祖先是否由transform、opacity、filter或will-change等创建上下文，通常把弹窗挂到顶层容器更稳妥。

2) 动画组件为了启用位移给外壳加了transform:

translateZ(0)，结果内部提示气泡被相邻卡片遮住，你会怎样落地修复？

答：外壳的transform会创建新的层叠上下文，气泡的层级被限制在该上下文内，提高气泡自身的z-index未必有效。可在动画结束后移除不再需要的transform，或把气泡通过Portal挂到更高层；迁移后还要重新处理定位参照和滚动跟随。

3) 设计师要求半透明容器使用opacity:

.9, 同时要求里面的下拉层越过页面顶栏, 这两个约束冲突时怎么处理?

答: 容器的opacity会形成层叠上下文, 内部下拉层无法仅靠增大z-index跨越外部顶栏。可把透明效果放到独立背景层, 避免作用于承载下拉层的祖先, 或将下拉层移到顶层; 后者会增加坐标换算、焦点管理和事件边界成本。

4) 线上只有某个活动页的浮层被遮挡, 代码里已经写了z-index, 你会如何快速找到真正限制它的祖先?

答: 从浮层开始逐级检查祖先的计算样式, 重点查看定位与z-index, 以及transform、opacity、filter、will-change等可能创建层叠上下文的属性。再对照兄弟区域所属的上下文逐层比较, 而不是只看两个元素的数值; 临时关闭可疑规则可辅助验证。

5) 团队准备建立全站z-index数值表, 有人主张弹窗统一用999999, 这套方案能彻底避免遮挡吗?

答: 统一层级令牌只能规范同一层叠上下文内的排序, 无法突破祖先上下文的边界, 因此极大数值不能根治遮挡。更可靠的方案是规定弹层挂载位置和顶层上下文结构, 再为遮罩、弹窗、提示等分配有限层级; 集中挂载也会带来定位与可访问性管理责任。

398. 左右居中方案

左右居中要根据元素类型选择: 行内元素用text-align: center, 定宽块用margin: 0 auto, 不定宽元素可用定位配合位移。text-align应写在父元素上, 因为它控制的是内部行内内容的对齐。块级元素宽度确定后, 左右margin设为auto就能平分剩余空间。如果元素宽度未知, 可以用left: 50%加transform:

translateX(-50%), 避免手算负边距。

行内元素:	text-align:	center	定宽块状元素:
左右margin值为auto	不定宽块状元素:	table布局, position	+ transform /* 方案1
/.wrap{text-align:center}.center{display:inline;/ or *//* display: inline-block; *//* 方案2			
/.center{width:100px;margin:0 auto;}/ 方案2			
*/.wrap{position:relative;}.center{position:absolute;left:50%;transform:translateX(-50%);}			

1) 工具栏里的提示文字是行内元素, 开发者给它写了margin: 0 auto却没有居中, 页面结构不允许改, 你会改哪条规则?

答: 行内内容应由包含块设置text-align: center, margin: 0 auto不是这种场景的居中机制。该规则会同时影响容器内其他行内内容, 因此若工具栏还有左对齐按钮, 需要增加独立包装层或缩小样式作用范围。

2) 设置页有一个明确为320px宽的块级表单, 需要在父容器中水平居中, 团队在flex和自动外边距之间争论, 你选哪个?

答: 仅需居中一个定宽块时, 给表单设置width: 320px; margin: 0 auto就足够, 语义直接且不改变父级布局模式。若父级还要统一排列多个子项, flex才更有价值; 固定宽度在窄屏可能溢出, 还需配合响应式宽度约束。

3) 搜索页的按钮组宽度随文案和语言变化, 不能预先写死, 父容器又不能改成flex, 怎样保证它始终水平居中?

答: 可让按钮组采用收缩到内容宽度的布局方式, 再使用自动外边距; 也可在相对定位父级中用left: 50%配合transform: translateX(-50%)。绝对定位方案会脱离常规文档流, 父级高度和相邻内容不能再依赖它自然撑开。

4) 角标使用position: absolute; left:

50%后总是向右偏半个自身宽度, 设计稿宽度还会动态变化, 你会怎么修?

答：left: 50%只把角标左边缘放到父容器中线，因此视觉中心必然偏右。补上transform: translateX(-50%)，位移百分比按元素自身宽度计算，可适配动态宽度；同时要确认父级提供了正确的定位包含块。

5) 弹层标题用position + transform做水平居中后，内部提示层的z-index突然压不过相邻区域，你如何权衡居中方案？

答：用于居中的transform会创建层叠上下文，内部提示层可能因此无法跨越外部上下文。若标题只是普通流内内容，可改用text-align、自动外边距或父级flex居中；若必须绝对定位，则应调整提示层挂载位置，而不是无限增大其z-index。

399. 上下垂直居中方案

垂直居中主要看元素高度是否已知：定高可用外边距或定位加负边距，不定高更适合transform、flex或vertical-align。已知高度时，把元素移动到top: 50%后再回退自身高度的一半即可。高度未知时，transform: translateY(-50%)不依赖具体尺寸，父元素使用display:

flex和align-items: center也更直接。兼容传统行内布局时，还可以借助inline-block和vertical-align: middle。

定高：margin，position + margin(负值)不定高：position+transform，flex，IFC +

vertical-align:middle /* 定高方案1 */.center{height:100px;margin:50px 0;}//* 定高方案2

/.center{height:100px;position:absolute;top:50%;margin-top:-25px;}// 不定高方案1

/.center{position:absolute;top:50%;transform:translateY(-50%);}// 不定高方案2

/.wrap{display:flex;align-items:center;}.center{width:100%;}// 不定高方案3 *//* 设置 inline-block

则会在外层产生 IFC，高度设为 100% 撑开 wrap 的高度 */.wrap::before{content:"";height:100%;display:inline-block;vertical-align:middle;}.wrap{text-align:center;}.center{display:inline-block;vertical-align:middle;}

1) 活动页把一段可能换成三行的文案写成固定height，再用上下margin居中；运营改字后文案偏到下方，你会保留这套写法吗？

答：不会保留依赖定高的margin方案，因为内容高度变化后，原先计算的上下间距不再成立。应改用父容器display: flex; align-items: center，或用绝对定位配合top: 50%、transform: translateY(-50%)；前者更适合正常文档流。

2) 弹窗主体高度由接口返回内容决定，现有代码是position: absolute; top: 50%; margin-top: -150px，短内容正常、长内容上移，你会怎样改？

答：把固定负margin改成transform: translateY(-50%)，位移比例会依据元素自身实际高度计算，不需要预先知道高度。若弹窗还要水平居中，可同时使用left: 50%和translate(-50%, -50%)；内容超过视口时仍需另设最大高度和滚动策略。

3) 一个老专题页不能改浮动和定位结构，父容器高度固定，要求让未知高度的提示块垂直居中，你会怎样用inline-block与vertical-align落地？

答：可让父容器的::before成为高度100%的inline-block，再让它和提示块都设置vertical-align: middle。这个伪元素负责撑出可对齐的行内格式化上下文，提示块也必须是inline-block；空白字符和基线间隙可能影响细节，需要检查模板换行。

4) 列表空状态用了display: flex; align-items:

center，但上线后仍贴着容器顶部；检查发现父节点没有可用高度，你会从哪里排查？

答：先确认执行居中的.wrap是否真的具有明确高度或被上级布局拉伸，align-items: center只能在已有的交叉轴剩余空间内分配位置。再检查样式是否被覆盖以及空状态是否为该容器的直接子项；父容器高度随内容收缩时，换任何居中语法都看不出效果。

5) 表单行高度固定为100px，设计坚持用负margin居中，开发希望统一换成flex；你会如何取舍？

答：固定高度且子项高度也稳定时，position配合负margin能成立，但尺寸一改就必须同步更新偏移量。组件会复用或内容可能变化时更适合flex，只需在父项设置align-items:center；若绝对定位是为了脱离文档流，则保留定位并改用transform更稳妥。

400. 选择器权重计算方式

选择器权重可以按ID、类及属性、元素三个层级比较，先比较高层级，完全相同时后写的规则覆盖先写的。!important的优先级高于普通声明，元素的内联style也高于一般的外部规则。通配符以及后代、兄弟等关系符本身不会带来更高权重，继承样式通常也排在直接指定的规则之后。选择器匹配通常从右向左解析，这样能更早过滤不符合条件的元素。!important > 内联样式 = 外联样式 > ID选择器 > 类选择器 = 伪类选择器 = 属性选择器 > 元素选择器 = 伪元素选择器 > 通配选择器 = 后代选择器 = 兄弟选择器 属性后面加!import会覆盖页面内任何位置定义的元素样式作为style属性写在元素内的样式id选择器类选择器标签选择器通配符选择器(*) 浏览器自定义或继承 同一级别：后写的会覆盖先写的 css选择器的解析原则：选择器定位DOM元素是从右往左的方向，这样可以尽早的过滤掉一些不必要的样式规则和元素

1) 组件页中.dialog .title写在文件前面，后面的h2仍然覆盖不了字号；同事认为“后写一定赢”，你怎么指出判断漏洞？

答：后写覆盖先写只发生在同一级权重下，.dialog .title含两个类选择器，权重高于单个元素选择器h2。应先比较!important、内联样式、ID、类或伪类或属性、元素或伪元素等层级，再比较源码顺序；继承值也不能击败直接命中的声明。

2) 线上主题样式盖不住节点上的style="color:red"，维护者准备给主题选择器再加三个类名，你会接受吗？

答：继续堆类名通常仍盖不过内联样式，因为内联声明的优先级高于普通的ID、类和元素选择器。应优先移除或改造内联样式的生成位置；若确实无法控制且业务必须覆盖，只能谨慎使用!important，代价是后续覆盖会更困难。

3) 表格有一万行，样式规则写成.page .content table tbody tr td.warning；评审既担心权重失控，也担心匹配路径过长，你会怎样调整？

答：把规则收敛为靠近目标节点的稳定类名，例如.warning-cell，既减少无意义的祖先约束，也避免累积过高的类和元素权重。浏览器定位选择器时按从右向左匹配，右端应尽量有明确目标；是否形成可见性能问题仍应以页面测量为准，不能仅凭行数断言。

4) 灰度发布后只有一部分按钮变色，开发工具里两条规则都是.panel .button，一条来自主题包、一条来自业务包；你怎么定位是谁赢了？

答：两条选择器权重相同，应查看最终加载后的源码顺序，后出现的声明会覆盖先出现的声明。还要核对属性是否带!important、元素是否存在内联样式，以及选择器是否确实命中同一节点；构建产物的样式注入顺序变化也会改变结果。

5) 设计系统想让业务方容易覆盖默认样式，但业务团队又要求禁止!important；你会怎样制定选择器约束？

答：默认规则应使用短而稳定的类选择器，避免叠加ID、多层祖先和标签来制造不必要的权重。业务覆盖时保持同级或略高权重，并依靠明确的加载顺序；这样符合层叠规则，也降低双方不断加选择器的对抗成本，但仍需统一样式入口顺序。

401. 清除浮动

清除浮动的目的，是让父元素能够正常包住浮动的子元素。可以在浮动元素后添加带有clear:

both的空div，但这会引入没有实际语义的标签。也可以给父元素设置overflow:

hidden或auto来触发BFC。我一般更倾向于使用clearfix伪元素，因为它同样通过clear:

both生效，又能保持文档结构清晰；兼容IE6时再补zoom:

1。

在浮动元素后面添加clear:both的空div元素 给父元素添加overflow:hidden或者auto样式，触发BFC
.container{width:300px;background-color:#aaa;overflow:hidden;zoom:1;/*IE6*/}

使用伪元素，也是在元素末尾添加一个点并带有clear: both属性的元素实现的。 .clearfix{zoom:1;/*IE6*/}
.clearfix:after{content:".";height:0;clear:both;display:block;visibility:hidden;}

推荐使用第三种方法，不会在页面新增div，文档结构更加清晰

1) 资讯页左右两栏都设了float，父容器背景高度却变成0，有人只给父容器补一个固定高度；你会同意吗？

答：不同意用固定高度掩盖塌陷，因为浮动子项脱离普通文档流，内容增加后仍会溢出。应在末尾清除浮动，或让父元素通过overflow:

hidden、auto形成BFC；固定高度只适合内容尺寸被严格约束的少数场景。

2) 老后台几十个列表都存在父级塌陷，模板团队不允许在每个列表末尾插入空div，你会怎样统一修复？

答：给这些父容器统一挂.clearfix，使用::after生成块级伪元素并设置clear: both，即可在末尾清除浮动。它不新增无语义的真实节点，文档结构更清晰；若仍需兼容非常旧的环境，源码中的zoom:

1属于针对旧版IE的兼容处理。

3) 卡片父级用overflow:

hidden清浮动后，子元素阴影被截断；结构不能加真实节点时，你会怎么改？

答：应撤掉用于清浮动的overflow: hidden，改用父元素的::after配合content、display:

block和clear: both。overflow方案通过形成BFC包含浮动，但同时会裁剪超出边界的视觉内容；伪元素法更适合需要保留阴影或外溢效果的容器。

4) 线上页脚压到浮动商品列表上方，开发工具显示.clearfix:after存在却没有撑开父级；你会逐项检查哪些声明？

答：先检查伪元素是否实际生成，content缺失时::after不会产生可用于清除的盒子。再确认它具有display: block和clear: both，并排除规则被覆盖；若清除节点不在浮动元素所属的同一父容器末尾，也无法修复目标父级的高度。

5) 维护者在“空div清除”“父级overflow: auto”“伪元素清除”之间争论，页面还要求结构简洁且不能出现额外滚动条，你会选哪一个？

答：优先选伪元素清除，因为它不增加真实div，也不会像overflow: auto那样可能引入滚动或改变溢出表现。空节点法直观但污染结构，overflow法代码短却附带裁剪或滚动语义；只有这些副作用符合容器需求时才应采用后两者。

402. 左边定宽，右边自适应方案

左边定宽、右边自适应，可以让左侧设置固定宽度并左浮动，再由右侧占据剩余空间。常见写法是左侧width:

120px、float:

left，右侧通过margin-left:

120px留出位置。另一种方式是让两边都浮动，并把右侧宽度写成calc(100%

-

120px)。前一种写法更直接，后一种适合确实需要明确计算右侧宽度的场景。 float + margin, float +

calc /* 方案1 */.left{width:120px;float:left;}.right{margin-left:120px;}/* 方案2

*/.left{width:120px;float:left;}.right{width:calc(100% - 120px);float:left;}

1) 管理台左栏固定120px并浮动，右栏也写了width:

100%，结果被挤到下一行；同事说右栏本来就是“占满剩余空间”，错在哪里？

答：width: 100%按包含块宽度计算，并不会自动扣除左侧浮动的120px，两栏总宽度因此可能超过容器。可让右栏不设宽度而使用margin-left: 120px，或把右栏宽度明确写成calc(100% - 120px)并配合浮动。

2) 老门户页必须保留float，侧栏固定120px，右侧文章要随窗口伸缩；你会怎样写最少且稳定的布局规则？

答：让左栏设置width: 120px; float: left，右栏设置margin-left: 120px，即可把右侧可用区域留出来。右栏宽度由普通块级布局自动填充，通常不必再计算；若左右间还要留缝，应把间距一并计入margin-left。

3) 产品把侧栏从120px改成可配置值，但现有float + calc在多处各写了一遍宽度；你会怎样避免两边配置失配？

答：左栏宽度和右栏扣减值必须来自同一个尺寸约束，否则任一处漏改都会产生空隙或换行。可统一维护该值，再用于左栏width与右栏的calc(100% - 固定值)；如果无需显式控制右栏宽度，改用等值margin-left会减少一处计算。

4) 发布后右栏偶尔掉到左栏下方，只有容器加了边框和内边距的页面复现；你会怎样排查calc(100% - 120px)？

答：先检查两栏最终占用宽度是否还包含边框、内边距或额外外边距，因为这些尺寸可能让总和超过父容器。再核对两栏是否都成功浮动、calc是否被覆盖，以及父级可用宽度；必要时调整盒模型或把额外间距纳入计算，避免临界宽度换行。

5) 代码评审中，一方主张右栏用margin-left: 120px，另一方坚持float + calc；右栏只是普通文章流，你会选哪套？

答：普通文章流优先用左栏浮动、右栏等值margin-left，它不要求右栏也浮动，规则更少且能自然占据剩余宽度。float + calc适合确实需要明确右栏宽度或两栏都参与浮动的结构，但尺寸、间距和盒模型必须同步计算，维护成本更高。

403. 左右两边定宽，中间自适应

左右两边定宽、中间自适应，可以用浮动配合外边距，也可以直接用flex。浮动方案让左右分别靠两侧排列，中间通过margin: 0 120px留出空间；需要明确宽度时，也能用calc(100% - 240px)计算。更简洁的写法是父容器设置display: flex，左右固定为120px，中间设置flex: 1。具体选择取决于现有布局方式，维护旧浮动页面时不必强行改写。

圣杯布局（设置BFC，margin负值法），flex .wrap{width:100%;height:200px;}.wrap > div{height:100%;}/*

/.left{width:120px;float:left;}.right{float:right;width:120px;}.center{margin:0 120px;}/ 方案2

/.left{width:120px;float:left;}.right{float:right;width:120px;}.center{width:calc(100% - 240px);margin-left:120px;}/ 方案3

*/.wrap{display:flex;}.left{width:120px;}.right{width:120px;}.center{flex:1;}

1) 直播页左右栏各120px，中间栏写了width:

100%，上线后被挤到下一行；负责人认为左右浮动不会占普通流宽度，你怎么解释并修正？

答：浮动虽脱离普通文档流，但中间栏的100%仍按父容器完整宽度计算，无法与两侧共同容纳。可让左右分别float: left、float: right，中间使用margin: 0 120px；也可显式计算中间宽度，但必须把两侧总宽扣除。

2) 新闻详情页要求左右广告栏固定120px，中间正文自适应，现有float结构不能重写；你会给出什么落地代码？

答：左栏设width: 120px; float: left，右栏设width: 120px; float: right，中间正文设margin: 0 120px。中间不必固定宽度，会由块级布局占据扣除外边距后的空间；还要保证页面中的排列和清浮动规则与现有结构相容。

3) 设计把右栏改为160px、左栏仍为120px，原来的.center { margin: 0 120px }出现遮挡，你会怎样调整而不是继续套对称值？

答：中间栏应改为margin-left: 120px; margin-right: 160px，分别为两侧保留实际宽度。若采用calc，中间宽度也要扣除总计280px，不能沿用100% - 240px；侧栏还有额外间距时同样必须纳入约束。

4) 一个三栏页面在窄窗口下中间栏消失，开发工具显示.center { width: calc(100% - 240px); margin-left: 120px }，你会先查哪些尺寸关系？

答：先确认父容器实际可用宽度是否小于两侧合计240px，以及边框、内边距、外边距是否使三栏总占用继续增加。再检查左右浮动方向和中间宽度是否被其他规则覆盖；该方案没有为极窄容器定义降级行为，必要时必须另设最小宽度或重排布局。

5) 新建后台的三栏框架不受旧浏览器约束，团队在float + margin、float + calc和flex之间选型，你会推荐哪一种？

答：推荐父容器使用display: flex，左右栏固定width: 120px，中间栏设flex: 1，约束关系直接且无需负担浮动清除。旧结构难改时，float + margin比显式calc少一层宽度计算；若中间内容存在不可收缩项，仍需另外处理溢出风险。

6) 三栏容器改成flex后，中间长表格把右侧固定栏顶出屏幕；这和“中间flex: 1自适应”是否矛盾，你怎么排查？

答：不矛盾，flex: 1只声明中间项分配剩余空间，内部内容仍可能提出更大的最小尺寸需求。应检查长表格或不换行内容的溢出约束，并确保左右固定栏没有被允许意外收缩；浮动方案同样需要面对超长内容，只是故障表现可能不同。

404. CSS动画和过渡

transition适合属性状态变化时的平滑过渡，animation配合@keyframes则能描述完整的多阶段动画。transition主要控制目标属性、持续时间、速度曲线和延迟；animation还可以设置播放次数、方向及结束后的状态，并通过animationend监听完成。关键帧既能写成from到to，也能用百分比安排多个阶段。实现位移、旋转或缩放时，我会优先使用transform，简单显隐可用opacity，通常更适合连续动画。animation / keyframes animation-name: 动画名称，对应@keyframes animation-duration: 间隔 animation-timing-function: 曲线 animation-delay: 延迟 animation-iteration-count: 次数 infinite: 循环动画 animation-direction: 方向 alternate: 反向播放 animation-fill-mode: 静止模式 forwards: 停止时，保留最后一帧 backwards: 停止时，回到第一帧 both: 同时运用 forwards / backwards 常用钩子: animationend infinite: 循环动画 alternate: 反向播放 forwards: 停止时，保留最后一帧 backwards: 停止时，回到第一帧 both: 同时运用 forwards / backwards 动画属性: 尽量使用动画属性进行动画，能拥有较好的性能表现 translate scale rotate skew opacity color transform 位移属性 translate(x, y) 旋转属性 rotate() 缩放属性 scale() 倾斜属性 skew() transition transition-property (过渡的属性的名称)。transition-duration (定义过渡效果花费的时间,默认是0)。transition-timing-function: linear(匀速) ease(慢速开始，然后变快，然后慢速结束) (规定过渡效果的时间曲线，最常用的是这两个)。transition-delay (规定过渡效果何时开始。默认是 0) 一般情况下，我们都是写一起的，比如：transition: width 2s ease 1s 关键帧动画 animation

一个关键帧动画，最少包含两部分，animation

属性及属性值（动画的名称和运行方式运行时间等）。@keyframes（规定动画的具体实现过程）

animation 属性可以拆分为 animation-name规定@keyframes 动画的名称。animation-duration规定动画完成一个周期所花费的秒或毫秒。默认是0。animation-timing-function规定动画的速度曲线。默认是

“ease”，常用的还有linear，同transition。animation-delay规定动画何时开始。默认是

0。animation-iteration-count规定动画被播放的次数。默认是 1，但我们一般用infinite，一直播放 而@keyframes的使用方法，可以是from->to（等同于0%和100%），也可以是从0%->100%之间任意个的分层设置。我们通过下面一个稍微复杂点的demo来看一下，基本上用到了上面说到的大部分知识

eg:@keyframesmymove{from{top:0px;}to{top:200px;}}/* 等同于： */@keyframesmymove{0%{top:0px;}25%{top:200px;}50%{top:100px;}75%{top:200px;}100%{top:0px;}} 用css3动画使一个图片旋转
loader{display:block;position:relative;-webkit-animation:spin 2s linear infinite;animation:spin 2s linear infinite;}@-webkit-keyframesspin{0%{-webkit-transform:rotate(0deg);-ms-transform:rotate(0deg);transform:rotate(0deg);}100%{-webkit-transform:rotate(360deg);-ms-transform:rotate(360deg);transform:rotate(360deg);}}@keyframesspin{0%{-webkit-transform:rotate(0deg);...

1) 商品卡片从列表左侧滑入时，开发写了transition: left 300ms

ease，滚动过程中位移不连贯；只依据这些样式，你会先改哪一处？

答：先把位移从left改为transform: translateX()，并让transition-property指向transform。资料明确把translate、scale、rotate、skew、opacity列为更适合动画的属性，但仅凭样式不能断言卡顿一定来自布局计算，仍需结合页面负载排查。

2) 设计稿要求加载图标持续匀速旋转，另一位开发却用悬停触发的transition实现；这套方案为什么无法直接满足要求，你会怎么写？

答：持续旋转应使用animation配合@keyframes，例如从transform:

rotate(0deg)变化到rotate(360deg)，并设置linear infinite。transition依赖属性状态发生变化，本身不负责循环播放；代价是无限动画会持续运行，离开可见区域后是否暂停需要另行设计。

3) 弹窗打开后要分三段完成上移、轻微回落和停稳，产品又要求关闭时反向播放；你会怎样组织关键帧和播放方向？

答：用百分比关键帧描述三个阶段，并通过animation-duration与animation-timing-function控制整体节奏；需要往返播放时可设置animation-direction: alternate。alternate会按迭代轮次反向执行，不等于于任意时刻点击关闭就从当前进度倒放，交互中断仍需额外状态控制。

4) 线上反馈横幅动画结束后突然跳回初始位置，代码里@keyframes的100%已经是目标位置；你会优先检查哪个属性？

答：优先检查animation-fill-mode，若要结束后保留最后一帧，应使用forwards，需要同时处理开始前与结束后的样式可用both。它只影响动画执行区间外采用哪一帧，不会把关键帧结果永久写回元素原始样式，后续状态切换仍可能覆盖视觉结果。

5) 表单校验提示只需在错误出现时淡入，开发在transition和animation之间争执；你会依据什么选型？

答：若提示只是从既有状态平滑过渡到错误状态，使用transition配置属性、时长、曲线和延迟更直接。若要安排多个阶段或独立循环，则animation与@keyframes更合适；两者都应优先选择资料列出的动画属性，避免为了统一写法引入不必要的关键帧。

405. CSS3的新特性

CSS3常见的新特性可以归为视觉效果、盒模型与响应式能力、变换动画三类。视觉上有圆角、阴影和渐变，布局相关可用box-sizing，响应式页面则依赖@media。交互效果通常由transform、transition和animation完成，其中transition需要状态变化触发，animation可以按关键帧自行运行。文本场景还可以用word-break和text-overflow处理换行与溢出。transition：过渡transform: 旋转、缩放、移动或倾斜animation: 动画gradient: 渐变box-shadow: 阴影border-radius: 圆角word-break:normal|break-all|keep-all; 文字换行(默认规则|单词也可以换行|只在半角空格或连字符换行)text-overflow: 文字超出部分处理text-shadow: 水平阴影，垂直阴影，模糊的距离，以及阴影的颜色。box-sizing:content-box|border-box盒模型媒体查询@media screen and (max-width: 960px) {}还有打印print

1) 资讯卡片标题限定一行，内容超出后产品要求显示省略号；开发只加了text-overflow: ellipsis，页面仍把整段文字画出来，你会怎么判断？

答：text-overflow只规定溢出内容的呈现方式，单独设置通常不足以形成可见截断，还要确保元素确实产生受限宽度和溢出状态。资料只列出了该特性的用途，未给出完整组合，因此不能把失败归因于浏览器版本，应继续核对换行与溢出相关样式。

2) 英文订单号把移动端表格撑出屏幕，运营又要求普通英文句子尽量按单词换行；word-break应怎样选，风险是什么？

答：普通文本优先保留normal的默认换行规则；若订单号这类长串必须在任意字符处断开，可针对该字段使用break-all。keep-all只允许在半角空格或连字符等位置换行，不适合无分隔符长串；全局使用break-all则可能破坏普通单词的可读性。

3) 设计系统规定按钮视觉宽度为120px，组件同时有边框和内边距，接入方发现最终占位超过120px；你会改哪个盒模型配置？

答：可将组件设为box-sizing: border-box，让声明宽度包含内容区、内边距和边框，更容易维持设计稿给出的外部尺寸。content-box下声明宽度只约束内容区，额外边框和内边距会扩大盒子；切换盒模型后，依赖原内容区宽度的子元素可能需要重新校验。

4) 打印订单页时，网页导航和操作按钮也出现在纸面上；团队想再复制一套打印页面，你会先提出什么方案？

答：先使用@media print为打印介质覆盖样式，隐藏导航与操作区，并调整正文布局，而不是立即维护第二套页面。媒体查询也能按屏幕条件应用规则，例如max-width；但资料没有覆盖所有打印兼容细节，分页、背景和纸张尺寸仍应在目标浏览器实际验证。

5) 营销页需要圆角卡片、阴影、渐变背景和悬停放大，开发准备全部导出成图片；你会保留哪些为CSS，实现时注意什么？

答：圆角、盒阴影、渐变以及悬停缩放都可分别用border-radius、box-shadow、gradient和transform: scale()表达，通常无需把整块视觉固化成图片。这样便于尺寸和状态变化，但复杂视觉是否能完全还原仍取决于设计要求，不能仅凭特性清单承诺一致效果。

406. 列举几个css中可继承和不可继承的元素

通常，控制文字表现的属性更容易继承，而控制盒子尺寸、位置和布局的属性通常不会继承。例如color、font-family、line-height、text-align可以从父元素传给后代，visibility和cursor也可继承。相对地，display、margin、padding、width、position等不会自动继承，否则父级布局很容易意外影响子元素。遇到边界情况我会查看属性定义，而不是只靠这条经验判断。不可继承的：display、margin、border、padding、background、height、min-height、max-height、width、min-width、max-width、overflow、position、left、right、top、bottom、z-index、float、clear、table-layout、vertical-align所有元素可继承：visibility和cursor。内联元素可继承：letter-spacing、word-spacing、white-space、line-height

、color、font、font-family、font-size、font-style、font-variant、font-weight、text-decoration、text-transform、direction。终端块状元素可继承：text-indent和text-align。列表元素可继承：list-style、list-style-type、list-style-position、list-style-image。transition和animation的区别 Animation和transition大部分属性是相同的，他们都是随时间改变元素的属性值，他们的主要区别是transition需要触发一个事件才能改变属性，而animation不需要触发任何事件的情况下才会随时间改变属性值，并且transition为2帧，从from to，而animation可以一帧一帧的 ---

【浏览器】

407. 浏览器架构

现代浏览器采用多进程架构，把浏览器界面、页面渲染、网络、GPU和插件等职责隔离开。浏览器主进程负责界面、交互和子进程管理，渲染进程运行Blink与V8，把HTML、CSS和JavaScript转成可交互页面。进程隔离后，某个页面或插件崩溃通常不会拖垮整个浏览器，沙箱也能限制恶意程序获取系统权限。代价是进程会占用独立资源，但能换来更好的稳定性、安全性和多核利用。

单进程浏览器时代 单进程浏览器是指浏览器的所有功能模块都是运行在同一个进程里，这些模块包含了网络、插件、JavaScript运行环境、渲染引擎和页面等。其实早在2007年之前，市面上浏览器都是单进程的。缺点不稳定：一个插件的意外崩溃会引起整个浏览器的崩溃不流畅：所有页面的渲染模块、JavaScript执行环境以及插件都是运行在同一个线程中的，这就意味着同一时刻只能有一个模块可以执行不安全：可以通过浏览器的漏洞来获取系统权限，这些脚本获取系统权限之后也可以对你的电脑做一些恶意的事情，同样也会引发安全问题以上这些就是当时浏览器的特点，不稳定，不流畅，而且不安全。

不稳定：一个插件的意外崩溃会引起整个浏览器的崩溃不流畅：所有页面的渲染模块、JavaScript执行环境以及插件都是运行在同一个线程中的，这就意味着同一时刻只能有一个模块可以执行不安全：可以通过浏览器的漏洞来获取系统权限，这些脚本获取系统权限之后也可以对你的电脑做一些恶意的事情，同样也会引发安全问题。

多进程浏览器时代 由于进程是相互隔离的，所以当页面或者插件崩溃时，影响到的仅仅是当前的页面进程或者插件进程，并不会影响到浏览器和其他页面，这就完美地解决了页面或者插件的崩溃会导致整个浏览器崩溃，也就是不稳定的问题JavaScript也是运行在渲染进程中的，所以即使JavaScript阻塞了渲染进程，影响到的也只是当前的渲染页面，而并不会影响浏览器和其他页面，因为其他页面的脚本是运行在它们自己的渲染进程中的Chrome把插件进程和渲染进程锁在沙箱里面，这样即使在渲染进程或者插件进程里面执行了恶意程序，恶意程序也无法突破沙箱去获取系统权限。

最新的Chrome浏览器包括：1个浏览器（Browser）主进程、1个GPU进程、1个网络（NetWork）进程、多个渲染进程和多个插件进程。

浏览器进程。主要负责界面显示、用户交互、子进程管理，同时提供存储等功能。

渲染进程。核心任务是将HTML、CSS和JavaScript转换为用户可以与之交互的网页，排版引擎Blink和JavaScript引擎V8都是运行在该进程中，默认情况下，Chrome会为每个Tab标签创建一个渲染进程。出于安全考虑，渲染进程都是运行在沙箱模式下。

GPU进程。其实，Chrome刚开始发布的时候是没有GPU进程的。而GPU的使用初衷是为了实现3D CSS的效果，只是随后网页、Chrome的UI界面都选择采用GPU来绘制，这使得GPU成为浏览器普遍的需求。最后，Chrome在其多进程架构上也引入了GPU进程。

网络进程。主要负责页面的网络资源加载，之前是作为一个模块运行在浏览器进程里面的，直至最近才独立出来，成为一个单独的进程。

插件进程。主要是负责插件的运行，因插件易崩溃，所以需要通过插件进程来隔离，以保证插件进程崩溃不会对浏览器和页面造成影响。

1) 客服说一个页面里的插件崩溃后整个浏览器都应该退出，因为“插件属于浏览器”；结合现代多进程架构，你会怎样反驳？

答：现代浏览器会把插件和页面等职责放入相互隔离的进程，插件崩溃通常只影响对应插件进程，不应必然带走浏览器和其他页面。单进程时代才容易因一个模块异常导致整体崩溃；进程隔离降低故障范围，但不能据此保证所有异常都绝不扩散。

2) 后台报表执行一段很长的JavaScript后当前标签页失去响应，但地址栏和其他页面仍可操作；这与哪些进程职责吻合？

答：JavaScript引擎V8与排版引擎Blink都运行在渲染进程中，长脚本阻塞该进程时，主要影响当前页面的执行和渲染。浏览器主进程负责界面、交互和子进程管理，其他页面通常还有各自的渲染进程；具体进程分配策略不宜由现象单独推断。

3) 安全评审认为把渲染进程拆开只是为了防崩溃，与恶意网页获取系统权限无关；你会指出哪处判断有问题？

答：多进程设计还配合沙箱限制渲染进程和插件进程访问系统资源，即使其中执行恶意程序，也更难直接突破边界取得系统权限。进程隔离解决的是影响范围，沙箱解决的是权限约束，两者目标相关但并不等价；沙箱也不是可以忽略浏览器漏洞的绝对安全保证。

4) 页面能显示但图片和脚本长期加载失败，另一个标签页的界面交互仍正常；排查时你会优先区分哪些进程？

答：应优先区分负责资源加载的网络进程、负责页面转换与交互的渲染进程，以及管理界面和子进程的浏览器主进程。资源普遍加载异常更应关注网络链路或网络进程，单页解析执行异常则偏向渲染进程；仅凭一个标签页现象仍不足以锁定故障点。

5) 团队讨论是否把所有浏览器能力塞回单进程以节省开销，稳定性、安全性和流畅性之间该怎么取舍？

答：单进程减少了进程边界，却会让插件、页面、网络和脚本故障更容易牵连整个浏览器，也难以借助隔离与沙箱控制影响范围。多进程的价值是稳定性、安全性及利用多核能力，代价是架构和资源管理更复杂；资料没有给出内存数据，不能虚构量化收益。

408. JavaScript单线程模型

JavaScript单线程指同一时刻只有一个调用链在执行，并不代表浏览器只有一个线程。这样设计能避免多个线程同时修改DOM时产生冲突，但长时间运行的脚本也会阻塞页面渲染。异步任务会进入任务队列，等当前任务完成、执行线程空闲后再按事件循环取出执行。计算密集型工作可以交给Web

Worker，不过它不能直接操作DOM，而且通信成本高于计算收益时并不划算。JavaScript语言的一大特点就是单线程，也就是说，同一时间只能做一件事，前面的任务没做完，后面的任务只能等着。

1. 为什么JavaScript是单线程的呢？这主要与JavaScript用途有关。它的主要用途是与用户互动，以及操作DOM。如果JavaScript是多线程的，会带来很多复杂的问题，假如JavaScript有A和B两个线程，A线程在DOM节点上添加了内容，B线程删除了这个节点，应该是哪个为准呢？

所以，为了避免复杂性，所以设计成了单线程。虽然HTML5提出了Web Worker标准。Web Worker的作用，就是为JavaScript创造多线程环境，允许主线程创建Worker线程，将一些任务分配给后者运行。但是子线程完全受主线程控制，且不得操作DOM。所以这个并没有改变JavaScript单线程的本质。一般使用Web Worker的场景是代码中有很多计算密集型或高延迟的任务，可以考虑分配给Worker线程。但是使用的时候一定要注意，worker线程是为了让你的程序跑的更快，但是如果worker线程和主线程之间通信的时间大于了你不使用worker线程的时间，结果就得不偿失了。

2. 浏览器内核中线程之间的关系 GUI渲染线程和JS引擎线程互斥js是可以操作DOM的，如果在修改这些元素的同时渲染页面（js线程和ui线程同时运行），那么渲染线程前后获得的元素数据可能就不一致了。JS阻塞页面加载js如果执行时间过长就会阻塞页面 js是可以操作DOM的，如果在修改这些元素的同时渲染页面（js线程和ui线程同时运行），那么渲染线程前后获得的元素数据可能就不一致了。

js如果执行时间过长就会阻塞页面 3. 浏览器是多进程的优点 默认新开 一个 tab 页面 新建一个进程,所以单个 tab 页面崩溃不会影响到整个浏览器。第三方插件崩溃也不会影响到整个浏览器。多进程可以充分利用现代CPU 多核的优势。方便使用沙盒模型隔离插件等进程,提高浏览器的稳定性。 4. 进程和线程又是什么呢 进程(process)和线程(thread)是操作系统的基本概念。 进程是 CPU 资源分配的最小单位(是能拥有资源和独立运行的最小单位)。线程是 CPU 调度的最小单位(是建立在进程基础上的一次程序运行单位)。 由于每个进程至少要做一件事,所以一个进程至少有一个线程。系统会给每个进程分配独立的内存,因此进程有它独立的资源。同一进程内的各个线程之间共享该进程的内存空间(包括代码段,数据集,堆等)。 进程可以理解为一个工厂不同车间,相互独立。线程是车间里的工人,可以自己做事,也可以相互配合做同一件事情。 5. 任务队列 单线程就意味着,所有任务都要排队执行,前一个任务结束,才会执行后一个任务。如果一个任务需要执行,但此时JavaScript引擎正在执行其他任务,那么这个任务就需要放到一个队列中进行等待。等到线程空闲时,就可以从这个队列中取出最早加入的任务进行执行(类似于我们去银行排队办理业务,单线程相当于说这家银行只有一个服务窗口,一次只能为一个人服务,后面到的就需要排队,而任务队列就是排队区,先到的就优先服务) 注意:如果当前线程空闲,并且队列为空,那每次加入队列的函数将立即执行。 为什么会有任务队列? 由于 JS 是单线程的,同步执行任务会造成浏览器的阻塞,所以我们将 JS 分成一个又一个的任务,通过不停的循环来执行事件队列中的任务。

1) 同事说浏览器能同时下载图片,所以页面里的JavaScript也会自动并行修改同一个DOM;你会怎样指出概念混淆?

答:浏览器内部可以有多个进程和线程,但页面主线程上的JavaScript仍按单线程模型执行,不会自动由两个线程同时修改同一DOM。这种设计避免添加节点与删除节点并发发生时的结果冲突;网络并行能力不能直接推出脚本执行模型也相同。

2) 数据看板需要在前端执行大量计算,输入期间明显卡顿;开发打算把计算搬到Web Worker,你会先确认哪些边界?

答:计算密集型或高延迟任务适合交给Web Worker,从而避免长时间占用页面主线程。Worker受主线程控制且不能操作DOM,结果必须回传后再由主线程更新界面;若任务很小或通信成本超过计算收益,迁移反而可能得不偿失。

3) 动画播放时,一段同步脚本持续处理大批数据,结果点击和页面绘制一起停住;为什么渲染线程没有绕过脚本继续工作?

答:资料指出GUI渲染线程与JS引擎线程互斥,脚本执行时间过长会阻塞页面加载和渲染。这样可避免脚本修改DOM的同时渲染线程读取到前后不一致的数据;拆分任务或移出重计算能缓解阻塞,但不能让主线程上的DOM操作并行执行。

4) 线上按钮点击后偶尔很晚才响应,监控显示事件已经进入任务队列;你会先排查事件是否丢失,还是主线程是否被占用?

答:应先检查主线程是否正在执行耗时任务,因为队列中的任务只有在线程空闲后才能按顺序取出执行。事件进入队列说明它不一定丢失,更可能是在等待前面的同步工作完成;若线程空闲且队列原本为空,新增任务才可能立即得到执行。

5) 两个标签页中,一个页面脚本死循环,另一个页面仍正常;有人据此断言JavaScript已经变成多线程,你会如何解释?

答:这反映的是浏览器多进程隔离,而不是单个页面里的JavaScript改成了多线程。不同标签页默认可由不同渲染进程承载,各自运行脚本,因此一个页面阻塞通常不影响其他页面;具体标签与进程映射可能受实现策略影响,不应把“默认”理解为绝对一一对应。

6) 架构师要在主线程分片执行和Web

Worker之间选方案，任务还需要频繁更新DOM；你会偏向哪边？

答：若核心工作必须频繁读写DOM，Worker不能直接完成这些操作，只能承担可独立计算的部分，再把结果交回主线程。计算量大且通信较少时Worker更有价值，交互步骤多且每次计算很轻时，主线程分任务可能更简单；最终取舍应比较阻塞时间与通信成本。

409. Chrome 打开一个页面需要启动多少进程？分别有哪些进程？

Chrome打开一个页面至少需要四个进程：浏览器主进程、网络进程、GPU进程和渲染进程。主进程负责界面、交互和子进程管理，网络进程加载资源，GPU进程负责绘制与合成，渲染进程则运行Blink和V8，把页面变成可交互内容。实际数量通常不止四个，因为还可能存在多个渲染进程和插件进程。渲染与插件独立运行，本质上是为了通过沙箱和进程隔离控制安全风险及崩溃影响。打开 1 个页面至少需要 1 个网络进程、1 个浏览器进程、1 个 GPU 进程以及 1 个渲染进程，共 4 个；最新的 Chrome 浏览器包括：1 个浏览器（Browser）主进程、1 个 GPU 进程、1 个网络（NetWork）进程、多个渲染进程和多个插件进程。浏览器进程：主要负责界面显示、用户交互、子进程管理，同时提供存储等功能。渲染进程：核心任务是将 HTML、CSS 和 JavaScript 转换为用户可以与之交互的网页，排版引擎 Blink 和 JavaScript 引擎 V8 都是运行在该进程中，默认情况下，Chrome 会为每个 Tab 标签创建一个渲染进程。出于安全考虑，渲染进程都是运行在沙箱模式下。GPU 进程：其实，Chrome 刚开始发布的时候是没有 GPU 进程的。而 GPU 的使用初衷是为了实现 3D CSS 的效果，只是随后网页、Chrome 的 UI 界面都选择采用 GPU 来绘制，这使得 GPU 成为浏览器普遍的需求。最后，Chrome 在其多进程架构上也引入了 GPU 进程。网络进程：主要负责页面的网络资源加载，之前是作为一个模块运行在浏览器进程里面的，直至最近才独立出来，成为一个单独的进程。插件进程：主要是负责插件的运行，因插件易崩溃，所以需要通过插件进程来隔离，以保证插件进程崩溃不会对浏览器和页面造成影响。

1) 测试同学说打开一个空白标签页只需要一个进程，因为页面里没有业务代码；你会怎么纠正这个判断？

答：即使是空白页，也不能把整个Chrome页面理解成单进程运行；按基础架构至少涉及浏览器、网络、GPU和渲染进程。它们分别承担界面与子进程管理、资源加载、绘制合成和页面解析执行，是否空白只影响工作量，不改变职责隔离。

2) 一个后台管理页同时加载HTML、样式、脚本和接口数据，负责人要求你说明这些任务分别落在哪些进程，你会怎么划分？

答：网络资源加载主要由网络进程负责，HTML、CSS和JavaScript转成交互页面的核心工作由渲染进程完成，其中包含Blink与V8。浏览器进程负责界面、用户交互和子进程管理，最终绘制与图层合成还会使用GPU进程。

3) 产品从单标签页改成 20 个标签页，并安装了多个插件，却要求进程数仍固定为 4 个，这个约束为什么不成立？

答：4 个只是打开一个页面时所涉及的基础进程类别和最低数量描述，并不是整个浏览器会长期固定为 4 个进程。Chrome默认可能为各个Tab创建渲染进程，插件也可能运行在独立插件进程中，因此标签页和插件增加后，实际进程数通常会随之变化。

4) 线上页面突然白屏，但浏览器地址栏、菜单和其他标签页仍能操作，你会优先怀疑哪个进程，并怎样缩小范围？

答：应优先检查该页面对应的渲染进程，因为Blink、V8以及页面内容生成都运行在那里，而浏览器界面仍正常说明主进程未必失效。可用Chrome任务管理器观察对应任务是否崩溃或资源异常，再区分是脚本执行、页面渲染还是插件影响。

5) 架构评审时有人建议把插件直接放进浏览器主进程，以减少进程数量和通信成本，你会接受吗？

答：通常不应接受，因为插件更容易发生崩溃，独立插件进程的价值正是隔离故障，避免连带破坏浏览器和页面。合并可能减少部分进程与通信开销，却会扩大故障影响面；除非运行环境和插件风险受到严格控制，否则不值得牺牲隔离性。

6) 页面请求已经返回，但内容迟迟没有显示；后端认定网络进程有问题，你会怎样串联网络进程与渲染进程判断责任边界？

答：请求返回只能说明网络进程完成了资源加载，不代表页面已经完成解析、布局、绘制和合成。应继续检查渲染进程中的Blink、V8是否被脚本或渲染工作阻塞，并观察GPU绘制合成环节；仅凭接口成功或失败不能覆盖整条显示链路。

410. 渲染机制

浏览器渲染是一条从DOM、CSSOM到渲染树，再经过布局、绘制和图层合成的流水线。修改元素尺寸或位置会触发布局，也就是回流，后续通常还要重绘；只改颜色、背景等外观，一般只需重绘，所以回流成本更高。像transform、opacity这类动画可以利用独立图层，尽量减少对布局的影响，但图层也不能创建过多。我一般还会把频繁更新放进requestAnimationFrame，并避免循环读取offsetWidth、offsetHeight后又立即改样式。

1. 浏览器如何渲染网页 概述：浏览器渲染一共有五步 处理HTML并构建DOM树。处理CSS构建CSSOM树。将DOM与CSSOM合并成一个渲染树。根据渲染树来布局，计算每个节点的位置。调用GPU绘制，合成图层，显示在屏幕上

第四步和第五步是最耗时的部分，这两步合起来，就是我们通常所说的渲染 具体如下图过程如下图所示渲染

网页生成的时候，至少会渲染一次在用户访问的过程中，还会不断重新渲染重新渲染需要重复之前的第四步(重新生成布局)+第五步(重新绘制)或者只有第五步(重新绘制) 在构建CSSOM树时，会阻塞渲染，直至CSSOM树构建完成。并且构建CSSOM树是一个十分消耗性能的过程，所以应该尽量保证层级扁平，减少过度层叠，越是具体的CSS选择器，执行速度越慢当HTML解析到script标签时，会暂停构建DOM，完成后才会从暂停的地方重新开始。也就是说，如果你想首屏渲染的越快，就越不应该在首屏就加载JS文件。并且CSS也会影响JS的执行，只有当解析完样式表才会执行JS，所以也可以认为这种情况下，CSS也会暂停构建DOM

2. 浏览器渲染五个阶段 2.1 第一步：解析HTML标签，构建DOM树 在这个阶段，引擎开始解析html，解析出来的结果会成为一棵dom树dom的目的至少有2个

作为下个阶段渲染树状图的输入成为网页和脚本的交互界面。(最常用的就是getElementById等等)

当解析器到达script标签的时候，发生下面四件事情 html解析器停止解析,如果是外部脚本，就从外部网络获取脚本代码将控制权交给js引擎，执行js代码恢复html解析器的控制权 由此可以得到第一个结论1 由于标签是阻塞解析的，将脚本放在网页尾部会加速代码渲染。defer和async属性也能有助于加载外部脚本。defer使得脚本会在dom完整构建之后执行；async标签使得脚本只有在完全available才执行，并且是以非阻塞的方式进行的 2.2 第二步：解析CSS标签，构建CSSOM树 我们已经看到html解析器碰到脚本后会做的事情，接下来我们看下html解析器碰到样式表会发生的情况js会阻塞解析，因为它会修改文档(document)。css不会修改文档的结构，如果这样的话，似乎看起来css样式不会阻塞浏览器html解析。但是事实上css样式表是阻塞的。阻塞是指当cssom树建立好之后才会进行下一步的解析渲染

通过以下手段可以减轻cssom带来的影响

将script脚本放在页面底部尽可能快的加载css样式表将样式表按照media type和media query区分，这样有助于我们将css资源标记成非阻塞渲染的资源。非阻塞的资源还是会被浏览器下载，只是优先级较低

第三步：把DOM和CSSOM组合成渲染树 (render tree)

2.3
2.4

第四步：在渲染树的基础上进行布局，计算每个节点的几何结构

布局(layout)：定位坐标和大小，是否换行，各种position,overflow,z-index属性 2.5 调用 GPU

绘制，合成图层，显示在屏幕上 将渲染树的各个节点绘制到屏幕上，这一步被称为绘制painting 3.

渲染优化相关 3.1 Load 和 DOMContentLoaded 区别 Load事件触发代表页面中的DOM，CSS，JS，图片已经全部加载完毕。DOMContentLoaded事件触发代表初始的HTML被完全加载和解析，不需要等待C

SS，JS，图片加载 3.2 图层 一般来说，可以把普通文档流看成一个图层。特定的属性可以生成一个新的图层。不同的图层渲染互不影响，所以对于某些频繁需要渲染的建议单独生成一个新图层，提高性能。但也

不能生成过多的图层，会引起反作用。通过以下几个常用属性可以生成新图层 3D变换：translate3d、translateZwill-changevideo、iframe标签通过动画实现的opacity动画转换position: fixed 3.3

重绘 (Repaint) 和回流 (Reflow)

重绘和回流是渲染步骤中的一小节，但是这两个步骤对于性能影响很大

重绘是当节点需要更改外观而不会影响布局的，比如改变color就叫称为重绘回流是布局或者几何属性需...

1) 一个卡片组件只把文字颜色从黑色改成红色，开发却说一定会重新计算整页布局；你会怎样反驳？

答：单纯修改color通常只改变外观而不改变元素几何结构，因此会触发重绘，不必然触发布局回流。只有宽高、边距、字体或内容等影响位置和尺寸的变化才需要重新布局；不过重绘仍有成本，影响区域过大时也可能造成卡顿。

2) 一个 1000 行列表在循环中反复读取offsetHeight，同时逐项修改style，滚动时明显卡顿，你会怎样改造？

答：这类读写交替可能迫使浏览器反复获得最新布局结果，应先批量读取所需尺寸并缓存，再集中修改样式，避免在循环中持续查询几何属性。大量节点还可先离线或隐藏后统一修改，但display:none的切换本身会产生回流，需要权衡修改次数。

3) 动效设计要求悬浮按钮每帧修改top，视觉同学又不接受改变运动轨迹；你会选择什么实现，并说明限制？

答：应保持相同轨迹但改用transform: translate(...)表达位移，避免持续修改top引发几何布局和后续重绘。频繁动画还可使用requestAnimationFrame对齐浏览器更新节奏；若动画必须真实改变文档流位置，就无法完全绕开回流。

4) 首屏的HTML很小，但头部同步加载大型脚本和多份样式表后长时间空白，你会沿渲染链路排查什么？

答：先检查同步script是否暂停了HTML解析，以及样式表是否迟迟未完成CSSOM构建并影响后续渲染和脚本执行。可将非关键脚本后移或使用defer、async，并尽快加载必要样式、按media条件拆分；依赖执行顺序的脚本不能盲目改为async。

5) 一个长列表滚动时大量区域持续重绘，团队准备给每个列表项都加will-change，你会批准这个方案吗？

答：不应直接给所有列表项提层；独立图层可隔离频繁渲染区域，但图层过多也会产生反作用。应先确认真正高频变化的节点，只对动画或重绘热点使用will-change、translate3d等手段，并在性能面板中验证布局与绘制是否确实减少。

6) 监控显示DOMContentLoaded很早触发，但首屏图片仍未出现，产品却认为页面已经完全加载；你会如何解释并设置判断口径？

答：DOMContentLoaded只表示初始HTML已完成加载和解析，不要求图片等资源全部完成，因此不能作为完整页面就绪的唯一口径。若关注全部资源完成应观察load，若关注可交互首屏则还要结合脚本执行和实际渲染状态；两者代表的业务体验并不相同。

411. 缓存机制

浏览器缓存的关键区别是是否发送请求：强缓存命中时不请求服务器，协商缓存则要向服务器确认资源有没有变化。首次响应会连同缓存头一起保存，后续浏览器根据Cache-Control或Expires判断缓存是否仍然有效；失效后再进入协商阶段。no-cache并不是完全不缓存，而是使用前必须向服务器校验，资源未变化时可以避免重新下载。真正禁止保存的是no-store，而public和private则用于控制代理缓存能否保存响应。

1. 首先得明确 http 缓存的好处 减少了冗余的数据传输，减少网费减少服务器端的压力Web缓存能够减少延迟与网络阻塞，进而减少显示某个资源所用的时间加快客户端加载网页的速度 2. 常见 http 缓存的类型 私有缓存（一般为本地浏览器缓存）代理缓存 3. 然后谈谈本地缓存 本地缓存是指浏览器请求资源时命中了浏览器本地的缓存资源，浏览器并不会发送真正的请求给服务器了。它的执行过程是 第一次浏览器发送请求给服务器时，此时浏览器还没有本地缓存副本，服务器返回资源给浏览器，响应码是200 OK，浏览器收到资源后，把资源和对应的响应头一起缓存下来第二次浏览器准备发送请求给服务器时候，浏览器会先检查上一次服务端返回的响应头信息中的Cache-Control，它的值是一个相对值，单位为秒，表示资源在客户端缓存的最大有效期，过期时间为第一次请求的时间减去Cache-Control的值，过期时间跟当前的请求时间比较，如果本地缓存资源没过期，那么命中缓存，不再请求服务器如果没有命中，浏览器就会把请求发送给服务器，进入缓存协商阶段。与本地缓存相关的头有：Cache-Control、Expires，Cache-Control有多个可选值代表不同的意义，而Expires就是一个日期格式的绝对值。 3.1 Cache-Control

Cache-Control是HTTP缓存策略中最重要的头，它是HTTP/1.1中出现的，它由如下几个值 no-cache：不使用本地缓存。需要使用缓存协商，先与服务器确认返回的响应是否被更改，如果之前的响应中存在ETag，那么请求的时候会与服务端验证，如果资源未被更改，则可以避免重新下载no-store：直接禁止浏览器缓存数据，每次用户请求该资源，都会向服务器发送一个请求，每次都会下载完整的资源public：可以被所有的用户缓存，包括终端用户和CDN等中间代理服务器。private：只能被终端用户的浏览器缓存，不允许CDN等中继缓存服务器对其缓存。max-age：从当前请求开始，允许获取的响应被重用的最长时间（秒）。must-revalidate，当缓存过期时，需要去服务端校验缓存的有效性。 例如：Cache-Control: public, max-age=1000# 表示资源可以被所有用户以及代理服务器缓存，最长时间为1000秒。

注意，虽然你可能在其他资料中看到可以使用 meta 标签来设置缓存，比如像下面的形式：<meta http-equiv="expires" content="Wed, 20 Jun 2021 22:33:00 GMT" 但在 HTML5 规范中，并不支持这种方式，所以尽量不要使用 meta 标签来设置缓存。 3.2 Expires Expires是HTTP/1.0出现的头信息，同样是用于决定本地缓存策略的头，它是一个绝对时间，时间格式是如Mon, 10 Jun 2015 21:31:12 GMT，只要发送请求时间是在Expires之前，那么本地缓存始终有效，否则就会去服务器发送请求获取新的资源。如果同时出现Cache-Control：max-age和Expires，那么max-age优先级更高。他们可以这样组合使用 Cache-Control: public Expires: Wed, Jan 10 2018 00:27:04 GMT 3.3 所谓的缓存协商当第一次请求时服务器返回的响应头中存在以下情况时 没有Cache-Control和ExpiresCache-Control和Expires过期了Cache-Control的属性设置为no-cache时 那么浏览器第二次请求时就会与服务器进行协商，询问浏览器中的缓存资源是不是旧版本，需不需要更新，此时，服务器就会做出判断，如果缓存和服务端资源的最新版本是一致的，那么就无需再次下载该资源，服务端直接返回304 Not Modified状态码，如果服务器发现浏览器中的缓存已经是旧版本了，那么服务器就会把最新资源的完整内容返回给浏览器，状态码就是200 Ok，那么服务端是根据什么来判断浏览器的缓存是不是最新的呢？其实是根据HTTP的另外两组头信息，分别是：Last-Modified/If-Modified-Since与...

1) 商品详情页第二次打开时，Network显示请求携带If-None-Match，但响应是200而不是304；有人据此断言浏览器没有使用缓存，你怎么判断？

答：不能仅凭200判断浏览器未使用缓存，因为携带If-None-Match已说明客户端正在用上次保存的ETag发起再验证。若服务端当前ETag不一致，就应返回新资源和200；只有标识一致时才返回不带资源实体的304。

2) 发版后的首页HTML已配置协商缓存，第二次请求带着If-None-Match；服务端需要怎样处理，浏览器收到响应后又怎样取资源？

答：服务端应比较请求中的If-None-Match与当前资源的ETag，一致时返回304，不再传输资源内容。浏览器随后从本地缓存读取实体；若两者不一致，则服务端返回更新后的资源，并随响应提供新的校验标识。

3) 一个高频访问的小图标和一个体积较大的JS文件都命中了本地缓存，前端同事认为它们必然进入同一种缓存位置，你会怎样纠正？

答：两者不一定进入相同位置，浏览器还会在Memory Cache与Disk Cache之间进行选择。内存缓存读取更快但随渲染进程结束而消失；较大的JS、CSS通常更倾向磁盘，而具体归属仍受浏览器策略影响。

4) 用户按普通F5后接口返回304，按Ctrl+F5却下载了完整资源；值班同学怀疑服务端随机失效缓存，你怎么解释并排查？

答：这种差异符合两种刷新方式的缓存流程：普通F5会跳过强缓存，但仍进行协商校验，所以可能收到304。Ctrl+F5会跳过强缓存和协商缓存，直接从服务端加载；排查时应对比请求头、状态码及实际传输内容。

5) 运营页更新频繁，架构师主张所有资源都只用Cache-Control: max-age，后端则主张全部依赖ETag；你如何说明两种方案的请求成本？

答：max-age命中强缓存时不会向服务器发请求，适合生命周期内无需重新确认的资源；ETag属于协商缓存，每次再验证仍会产生请求，但未变化时只返回304。选择取决于更新及时性要求，不能把免请求与免传资源视为同一件事。

6) 一个PWA页面断网后仍展示旧数据，清理普通磁盘缓存也没有恢复；同时DevTools标记资源来自Service Worker，下一步查哪里？

答：应优先检查Service Worker的离线缓存和网络代理逻辑，因为它的缓存位置优先于Memory Cache、Disk Cache和Push Cache。仅清理磁盘缓存可能绕不开其响应；还要核对注册状态与缓存更新策略，但不能直接归因于HTTP强缓存。

412. 浏览器存储

浏览器端常用的持久化方案有cookie、localStorage、sessionStorage和IndexedDB，选择时主要看数据用途、生命周期与容量。cookie通常保存身份或登录状态，约4K，请求时会自动携带；另外三者不随请求发送，更适合客户端数据。localStorage会一直保留到被清理，sessionStorage通常随页面会话结束而清理，大量结构化数据可以考虑IndexedDB。涉及登录态时还要关注HttpOnly、Secure和SameSite，分别用于限制脚本访问、安全传输以及降低跨站请求风险。

我们经常需要对业务中的一些数据进行存储，通常可以分为 短暂性存储 和 持久性存储。短暂性的时候，我们只需要将数据存在内存中，只在运行时可用持久性存储，可以分为 浏览器端 与 服务器端

存储方案	持久性	容量限制	携带方式	其他特性
cookie	否	4K	自动携带	可用于身份、登录状态
localStorage	是	5M	不携带	永久保存，窗口关闭不删
sessionStorage	是	5M	不携带	会话结束即删，窗口关闭删
IndexedDB	是	无限制	不携带	结构化数据，异步API

从上表可以看到，cookie已经不建议用于存储。如果

没有大量数据存储需求的话，可以使用localStorage和sessionStorage。对于不怎么改变的数据尽量使用localStorage存储，否则可以用sessionStorage存储。对于cookie，我们还需要注意安全性 Name，即该Cookie的名称。Cookie一旦创建，名称便不可更改。Value，即该Cookie的值。如果值为Unicode字符，需要为字符编码。如果值为二进制数据，则需要使用BASE64编码。Max Age，即该Cookie失效的时间，单位秒，也常和Expires一起使用，通过它可以计算出其有效时间。Max Age如果为正数，则该Cookie在Max Age秒之后失效。如果为负数，则关闭浏览器时Cookie即失效，浏览器也不会以任何形式保存该Cookie。Path，即该Cookie的使用路径。如果设置为/path/，则只有路径为/path/的页面可以访问该Cookie。如果设置为/，则本域名下的所有页面都可以访问该Cookie。Domain，即可以访问该Cookie的域名。例如如果设置为.zhihu.com，则所有以zhihu.com，结尾的域名都可以访问该Cookie。Size字段，即此Cookie的大小。Http字段，即Cookie的httponly属性。若此属性为true，则只有在HTTP Headers中会带有此 Cookie 的信息，而不能通过document.cookie来访问此 Cookie。Secure，即该Cookie是否仅被使用安全协议传输。安全协议。安全协议有HTTPS、SSL等，在网络上传输数据之前先将数据加密。默认为false。

1) 登录页把明文用户编号写进Cookie，开发认为加上HttpOnly后就绝对安全；作为安全评审你会指出什么？

答：HttpOnly只限制脚本通过document.cookie读取，并不等于明文值本身安全。登录态相关的value不应直接使用明文用户标识，还应结合Secure限制安全协议传输，并用SameSite降低跨站请求携带带来的风险。

2) 一个后台系统要保存登录态、当前窗口的筛选条件和较大规模的结构化离线数据，你会分别落到哪类存储？

答：登录态通常由Cookie承载，筛选条件若只需随当前窗口存在可放sessionStorage，较大且需持久化的数据更适合IndexedDB。Cookie约受4K限制且随请求进入请求头，localStorage、sessionStorage通常只有数兆容量。

3) 客服要求关闭页面后筛选条件消失，但产品又要求重新打开浏览器仍保留用户偏好；只允许在localStorage和sessionStorage间选，你怎么拆分？

答：筛选条件应放入sessionStorage，因为其生命周期与页面窗口相关，关闭后会被清理；用户偏好应放入localStorage，除非主动清理否则会持续存在。两者都不会自动参与服务端通信，但都不适合超出其容量边界的数据。

4) 线上接口请求头突然膨胀，排查发现同一域名下写入了大量业务Cookie；业务方却说这些数据只在个人中心页面使用，你怎么处理？

答：应把不需要服务端识别的业务数据移出Cookie，按生命周期改放localStorage、sessionStorage或更大容量的IndexedDB。Cookie会在匹配其作用范围的请求中自动进入请求头，即使接口本身不用这些值，也会持续产生传输负担。

5) 多子域系统把登录Cookie的Domain设为顶级业务域、Path设为/，便利性与隔离性发生冲突，你会怎么评审？

答：该配置会扩大Cookie的可用范围，使匹配该域及路径的更多页面和请求都可能携带它。若登录态只服务某个子域或路径，应尽量收窄Domain与Path；范围缩小会增加多系统共享登录态的设计成本。

6) 安全团队要求登录Cookie只能走HTTPS、脚本不可读取，并尽量减少跨站请求携带；你会配置哪些属性，还要提醒什么边界？

答：应配置Secure、HttpOnly与合适的SameSite：分别限制安全协议传输、阻止脚本读取并约束跨站携带。它们针对的是不同风险面，不能互相替代；同时仍需妥善处理登录态value，不能把敏感身份

直接作为明文保存。

413. 跨域方案

跨域的核心是绕开或满足浏览器同源策略，常见方案可分为服务端放行、代理转发和标签请求。接口能改服务端时优先配置CORS；开发环境可用webpack devServer.proxy或http-proxy-middleware做反向代理。只需发起GET请求时可以用JSONP，图片上报则适合img ping，但它拿不到响应正文。页面间通信可考虑iframe或postMessage，同时要留意交互安全和数据传递限制。

很多种方法，但万变不离其宗，都是为了搞定同源策略。重用的有jsonp、iframe、cors、img、HTML5 postMessage等等。其中用到html标签进行跨域的原理就是html不受同源策略影响。但只是接受Get的请求方式，这个得清楚。延伸1：img iframe script 来发送跨域请求有什么优缺点？1.iframe 优点：跨域完毕之后DOM操作和互相之间的JavaScript调用都是没有问题的缺点：1.若结果要以URL参数传递，这就意味着在结果数据量很大的时候需要分割传递，巨烦。2.还有一个是iframe本身带来的，母页面和iframe本身的交互本身就有安全性限制。

2. script 优点：可以直接返回json格式的数据，方便处理缺点：只接受GET请求方式 3. 图片ping 优点：可以访问任何url，一般用来进行点击追踪，做页面分析常用的方法缺点：不能访问响应文本，只能监听是否响应

延伸2：配合 webpack 进行反向代理？

webpack在devServer选项里面提供了一个proxy的参数供开发人员进行反向代理

```
'/api':{target:'http://www.example.com',// your target hostchangeOrigin:true,// needed for virtual hosted sitespathRewrite:{'^/api:':// rewrite path}},
```

然后再配合http-proxy-middleware插件对api请求地址进行代理

```
const express=require('express');const proxy=require('http-proxy-middleware');// proxy api requestsconst exampleProxy=proxy(options);// 这里的 options 就是 webpack 里面的 proxy 选项对应的每个选项// mount `exampleProxy` in web serverconst app=express();app.use('/api',exampleProxy);app.listen(3000);
```

然后再用nginx把允许跨域的源地地址添加到报头里面即可 说到nginx，可以再谈谈CORS配置，大致如下 location/{if(\$request_method='OPTIONS'){add_header'Access-Control-Allow-Origin'*';add_header'Access-Control-Allow-Methods'GET, POST, OPTIONS';add_header'Access-Control-Allow-Credentials'true';add_header'Access-Control-Allow-Headers'DNT, X-Mx-ReqToken, Keep-Alive, User-Agent, X-Requested-With, If-Modified-Since, Cache-Control, Content-Type';add_header'Access-Control-Max-Age'86400;add_header'Content-Type"text/plain charset=UTF-8';add_header'Content-Length'0;return200;}}

1) 埋点页面用new Image().src上报成功，产品却要求读取服务端返回的用户标签；沿用图片ping为什么行不通？

答：图片ping可以向跨域URL发起请求，但脚本不能读取响应正文，因此无法取得用户标签。它更适合点击追踪和页面分析，只能借助加载或失败事件判断是否响应；若业务依赖响应数据，就必须改用允许读取结果的受控方案。

2) 开发环境把/api配到devServer.proxy后请求正常，静态文件部署到测试环境却开始跨域；你会把代理落在哪一层？

答：devServer.proxy只属于本地开发服务器，打包后的静态资源不会携带这段转发逻辑。测试环境应由实际承载流量的服务处理，例如让Nginx按/api反向代理，或由接口端配置CORS；两边规则不一致仍会造成环境差异。

3) 旧页面用script加载跨域数据，接口负责人现在要求改成POST并提交较大的筛选条件；原方案还能继续吗？

答：依赖script的跨域方案只能发起GET，无法直接满足新的POST约束。把大量数据塞进查询参数还会受到URL长度、缓存和日志暴露等边界影响，应改由代理或CORS承载请求；迁移成本是服务端需要同步调整接口策略。

4) 线上跨域接口连真实业务日志都没有，只看到浏览器先发出OPTIONS；你会先核对Nginx的哪些响应内容？

答：真实请求未到业务服务时，应先检查预检响应是否允许当前源、方法和请求头，并确认OPTIONS能直接返回成功状态。还要核对Access-Control-Allow-Credentials与来源策略是否匹配，以及配置是否覆盖目标路径；任一项不匹配，浏览器都可能拦住后续请求。

5) 安全负责人反对放开整站CORS，前端又希望所有环境都使用统一的/api地址；你会如何取舍？

答：更稳妥的选择是让页面始终请求同域/api，再由各环境的Nginx转发到真实接口，从浏览器视角规避跨域。若必须让第三方源直接访问，再按明确来源、方法和请求头配置CORS；代理增加运维配置，开放CORS则扩大暴露边界。

6) 微前端容器需要和跨域iframe交换筛选状态，数据量持续增长；你会继续拼接URL参数还是改用什么机制？

答：大量状态通过URL参数传递会面临长度限制、拆分复杂和信息暴露，不适合作为持续通信通道。应使用postMessage传递结构化消息，并在接收端校验origin、消息类型和数据格式；它解决通信问题，但不会自动解除跨域DOM的安全限制。

414. XSS 和 CSRF

XSS是把恶意代码注入页面并执行，CSRF是借用用户已有登录态发送非本人意愿的请求。XSS分持久型和非持久型，防护重点是不信任用户输入：普通内容做转义，富文本采用白名单过滤，并用CSP限制资源加载和执行。CSRF利用了浏览器自动携带目标域名Cookie的机制，但攻击站点通常拿不到该Cookie。工程上应避免用GET修改数据，并结合验证token、验证码及SameSite等措施。

涉及面试题：什么是XSS攻击？如何防范XSS攻击？什么是CSP？XSS简单点来说，就是攻击者想尽一切办法将可以执行的代码注入到网页中。XSS可以分为多种类型，但是总体上我认为分为两类：持久型和非持久型。持久型也就是攻击的代码被服务端写入进数据库中，这种攻击危害性很大，因为如果网站访问量很大的话，就会导致大量正常访问页面的用户都受到攻击。

举个例子，对于评论功能来说，就得防范持久型XSS攻击，因为我可以在评论中输入以下内容 这种情况如果前后端没有做好防御的话，这段评论就会被存储到数据库中，这样每个打开该页面的用户都会被攻击到。非持久型相比于前者危害就小的多了，一般通过修改URL参数的方式加入攻击代码，诱导用户访问链接从而进行攻击。

举个例子，如果页面需要从URL中获取某些参数作为内容的话，不经过过滤就会导致攻击代码被执行 `alert(1) --> {{name}}` 但是对于这种攻击方式来说，如果用户使用Chrome这类浏览器的话，浏览器就能自动帮助用户防御攻击。但是我们不能因此就不防御此类攻击了，因为我不能确保用户都使用了该类浏览器。对于XSS攻击来说，通常有两种方式可以用来防御。转义字符 首先，对于用户的输入应该是永远不信任的。最普遍的做法就是转义输入输出的内容，对于引号、尖括号、斜杠进行转义 `function escape(str){str=str.replace(/&/g,'&');str=str.replace(/>/g,'>');str=str.replace(/"/g,'"');str=str.replace(/'/g,''');str=str.replace(/\n/g,'\\n');str=str.replace(/\\r/g,'\\r');return str}`

通过转义可以将攻击代码 `alert(1)` 变成 `// -> <script>alert(1)</script>` 但是对于显示富文本来讲，显然不能通过上面的办法来转义所有字符，因为这样会把需要的格式也过滤掉。对于这种情况，通常采用白名单过滤的办法，当然也可以通过黑名单过滤，但是考虑到需要过滤的标签和标签属性实在太多，更加推荐使用白名单的方式

```
const xss=require('xss') let html=xss('XSS')
```

Demoalert("xss");')// -> XSS Demo<script>alert("xss");</script>console.log(html)

以上示例使用了js-xss来实现，可以看到在输出中保留了h1标签且过滤了script标签 CSP CSP本质上就是建立白名单，开发者明确告诉浏览器哪些外部资源可以加载和执行。我们只需要配置规则，如何拦截是由浏览器自己实现的。我们可以通过这种方式来尽量减少XSS攻击。 通常可以通过两种方式来开启 CSP：设置HTTP Header中的Content-Security-Policy设置meta标签的方式 这里以设置HTTP Header来举例 只允许加载本站资源 Content-Security-Policy: default-src 'self' 只允许加载 HTTPS 协议图片 Content-Security-Policy: img-src https://* 允许加载任何来源框架 Content-Security-Policy: child-src 'none' 当然可以设置的属性远不止这些，你可以通过查阅文档(opens new window)的方式来学习，这里就不过多赘述其他的属性了。 对于这种方式来说，只要开发者配置了正确的规则，那么即使网站存在漏洞，攻击者也不能执行它的攻击代码，并且CSP的兼容性也不错。 2 CSRF 跨站请求伪造（英语：Cross-site request forgery），也被称为one-click attack或者session riding，通常缩写为CSRF或者XSRF，是一种挟制用户在当前已...

1) 评论页已经把全部转义了，安全同学却用一个带恶意参数的分享链接触发了脚本；团队因此认定“转义防不了XSS”，你怎么判断？

答：不能据此否定转义，先确认参数最终进入的是文本、属性、脚本还是其他上下文，以及是否存在绕过转义的直接插入。普通文本应按输出上下文转义；富文本则需白名单净化，单靠一套字符替换无法覆盖所有注入位置。

2) 内容平台允许作者提交含标题、链接和图片的HTML，前端负责人主张提交前用正则删掉，后端认为原样入库即可；你会怎么落地？

答：不能把前端正则作为安全边界，调用接口即可绕过，而且危险标签和属性很多，黑名单容易遗漏。服务端应使用成熟净化方案按允许的标签、属性和资源规则处理，前端可同步净化改善预览，并用严格CSP限制可加载、可执行的资源。

3) 业务为了接入外部图片和统计脚本，要求把CSP放宽为允许任意来源；富文本白名单仍然保留，这时你接受吗？

答：不应直接允许任意来源，否则CSP对遗漏注入点的兜底能力会明显下降。应按资源类型分别配置可信来源，例如限制脚本、图片和子页面，并优先通过响应头下发；若业务必须放宽某类资源，要明确它不会连带放宽脚本执行。

4) 用户反馈打开一条历史评论后账号出现异常操作，数据库里发现了可执行片段，但安全人员抓不到攻击者读取Cookie的请求；你如何区分是持久型`XSS还是CSRF`？

答：评论中的代码被入库并在访问者页面执行，首先按持久型XSS排查，不以是否成功读取Cookie作为必要条件。检查渲染入口、净化结果和CSP报告；若只是外站借浏览器自动携带目标域Cookie发请求、外站拿不到其内容，才更符合CSRF。

5) 支付团队已给会话Cookie设置SameSite=Strict，因此想删除所有CSRF token；接口里还保留一个通过GET修改订阅状态的入口，你同意吗？

答：不同意把SameSite当成唯一措施，更不能保留有副作用的GET，因为图片等元素就能自动发起这类请求。应先把修改操作改为合适的方法，再附带服务端可验证的token或必要的用户验证；SameSite、接口访问限制和验证信息属于纵深防御。

415. Service Worker

Service Worker本质上是运行在网页与浏览器、网络之间的代理，可拦截请求并提供离线缓存能力。它通常在install事件中预先缓存文件，再通过fetch事件决定返回缓存还是访问网络，因此常用于缓存静态文件和改善首屏加载。它不能直接操作DOM，需要通过postMessage与受控页面通信。使用时还要注意只能运

行在本地环境或HTTPS下，并且作用范围受脚本所在路径限制。Service workers本质上充当Web应用程序与浏览器之间的代理服务器，也可以在网络可用时作为浏览器和网络间的代理。它们旨在（除其他之外）使得能够创建有效的离线体验，拦截网络请求并基于网络是否可用以及更新的资源是否驻留在服务器上采取适当的动作。他们还允许访问推送通知和后台同步API。浏览器对ServiceWorker做了很多限制。在ServiceWorker中无法直接访问DOM，但可以通过postMessage接口发送的消息来与其控制的页面进行通信。ServiceWorker只能在本地环境下或HTTPS网站中使用。ServiceWorker有作用域的限制，一个ServiceWorker脚本只能作用于当前路径及其子路径；目前该技术通常用来做缓存文件，提高首屏速度。

```
// index.jsif(navigator.serviceWorker){navigator.serviceWorker.register("sw.js").then(function(registration){console.log("service worker 注册成功");}).catch(function(err){console.log("service worker 注册失败");});}// sw.js// 监听 `install` 事件，回调中缓存所需文件self.addEventListener("install",e=>{e.waitUntil(caches.open("my-cache").then(function(cache){returncache.addAll(["./index.html","./index.js"]);});});// 拦截所有请求事件// 如果缓存中已经有请求的数据就直接用缓存，否则去请求数据self.addEventListener("fetch",e=>{e.respondWith(caches.match(e.request).then(function(response){if(response){returnresponse;}console.log("fetch source");});});};
```

打开页面，可以在开发者工具中的Application看到ServiceWorker已经启动了。在Cache中也可以发现我们所需的文件已被缓存。当我们重新刷新页面可以发现我们缓存的数据是从ServiceWorker中读取的。

1) 新闻站把资源写入Cache

API后就宣称“已经支持离线”，但断网刷新时HTML仍然白屏；这说明团队混淆了什么？

答：写入缓存不等于请求会自动命中缓存，必须由已激活且处于作用域内的Service Worker在fetch事件中返回匹配响应。还要确认入口文档确实被缓存并有离线回退；首次访问、注册失败或尚未被控制的页面仍可能无法离线。

2) 商城准备缓存index.html和index.js来提升再次访问速度，代码只在install中执行cache.addAll()；你会怎样补全最小可用链路？

答：注册脚本后，在install的event.waitUntil()中等待预缓存完成，并在fetch中用event.respondWith()返回caches.match(request)的结果。缓存未命中时必须继续请求网络或返回明确回退，否则处理器可能得不到有效响应；同时要保证站点使用HTTPS或本地环境。

3) 主站从根路径注册sw.js后缓存正常，迁到/shop/sw.js后首页请求不再被拦截，但/shop/cart正常；运维认为是浏览器缓存损坏，你怎么看？

答：更可能是作用域变化，而不是缓存损坏，因为Service Worker默认只能控制脚本所在路径及其子路径。应检查注册记录、当前页面的控制器与注册作用域；若确需覆盖首页，应调整脚本位置或在受控配置下声明合适作用域。

4) 线上显示Service Worker注册成功，Application面板也能看到它，但刷新后静态文件仍全部走网络；你按什么顺序排查？

答：先确认当前页面已被该注册实例控制，并核对页面路径是否落在其作用域内，再检查fetch事件是否触发。随后验证缓存名称、请求键和caches.match()是否一致，以及respondWith()是否真正返回了响应；只完成注册或安装并不代表页面已经从缓存读取。

5) 运营后台要求直接在Service Worker中读取表单DOM，以便断网时保存编辑内容；架构师则建议页面把数据发给它，你支持哪一方？

答：应由页面读取表单，再通过postMessage与Service Worker通信，因为后者不能直接访问DOM。离线缓存、请求拦截、推送和后台同步适合放在该代理层；页面状态采集仍属于受控页面职责，消息协议也要处理版本与校验。

416. DOM 节点操作

DOM节点操作主要包括创建、增删替换、查找和属性修改，关键是按场景选对接口。创建元素或文本可用`createElement`、`createTextNode`，批量组织节点时可先放进`createDocumentFragment`。插入、删除和替换分别可以用`appendChild`、`removeChild`、`replaceChild`或`insertBefore`。查找时精确id可用`getElementById`，复杂选择条件更适合`querySelector`或`querySelectorAll`，属性则通过`getAttribute`、`setAttribute`等接口处理。

(1) 创建新节点 `createDocumentFragment()`//创建一个DOM片段`createElement()`//创建一个具体的元素`createTextNode()`//创建一个文本节点

(2) 添加、移除、替换、插入
`appendChild(node)``removeChild(node)``replaceChild(new,old)``insertBefore(new,old)`

(3) 查找 `getElementById()``getElementsByName()``getElementsByTagName()``getElementsByClassName()``querySelector()``querySelectorAll()`;

(4) 属性操作
`getAttribute(key)``setAttribute(key,value)``hasAttribute(key)``removeAttribute(key)`;

1) 表格里有一个，同事说`querySelector('#row-7')`返回的是“动态集合”，删掉该行后变量会自动变空；你怎么纠正？

答：`querySelector()`返回首个匹配元素或空值，不是会随文档变化的集合，已保存的变量也不会因节点移除而自动清空。若要获取多个匹配项应使用`querySelectorAll()`；代码还持有节点引用时，该引用依然存在，不能把查询结果与文档结构变化混为一谈。

2) 评论列表一次插入 300 个新条目，代码在循环里逐个`appendChild()`到可见容器；只允许使用题目列出的原生节点能力，你会怎么改？

答：先用`createDocumentFragment()`作为临时容器，把`createElement()`和`createTextNode()`创建的条目依次追加进去，最后一次性挂到列表。这样组织也便于在入树前完成内容和属性设置；片段只减少零散插入，仍需控制总节点规模。

3) 设计稿要求把现有占位节点换成真实卡片，并保证它仍位于原来的兄弟节点之间；有人提议先`removeChild()`再`appendChild()`，你选什么？

答：应使用父节点的`replaceChild(newNode, oldNode)`，它直接在原位置完成替换，不会把新卡片移动到容器末尾。若只是把节点插到某个兄弟节点前，则用`insertBefore(newNode, referenceNode)`；操作前仍要确保旧节点确实属于该父节点。

4) 线上筛选器偶发报错，日志显示`removeChild`的目标节点并不在当前列表中；页面同时会重新查询并替换整块列表，你怎么定位？

答：先核对删除时使用的父节点和目标节点是否来自同一轮查询，整块替换后旧引用可能已不属于新容器。删除前检查父子关系或重新按稳定标识查找，再调用`removeChild()`；若目标可能已被其他流程删除，应把并发更新设计成幂等。

5) 组件要判断按钮是否配置`disabled`，评审中一方读取`getAttribute('disabled')`，另一方用`hasAttribute('disabled')`；这里只关心属性是否存在，该怎么选？

答：应使用`hasAttribute('disabled')`，因为需求判断的是属性存在性，而不是取得其字符串值。需要读取具体属性内容时才用`getAttribute()`，写入和清理分别用`setAttribute()`、`removeAttribute()`；不要把缺失属性和值为空混作同一业务状态。

417. 掌握页面的加载过程

页面加载就是浏览器获取并从上到下解析HTML，遇到外部资源按需下载，解析到后逐步渲染页面。通常会暂停文档解析，等脚本下载、解析和执行完再继续，所以放在里可能拖慢内容展示。工程中可以把脚本放到文档底部，或用`defer`、`async`控制加载和执行时机；前者通常等解析完成后执行，后者下载完成就执行

且顺序不可预测。判断完成状态时，DOMContentLoaded表示初始HTML已解析，load则还要等待DOM、CSS、JS和图片全部加载。

网页加载流程 当我们打开网址的时候，浏览器会从服务器中获取到 HTML 内容浏览器获取到 HTML 内容后，就开始从上到下解析 HTML 的元素内容会先被解析，此时浏览器还没开始渲染页面我们看到元素里有用于描述页面元数据的元素，还有一些元素涉及外部资源（如图片、CSS 样式等），此时浏览器会去获取这些外部资源。除此之外，我们还能看到元素中还包含着不少的元素，这些元素通过src属性指向外部资源当浏览器解析到这里时（步骤 3），会暂停解析并下载 JavaScript 脚本当 JavaScript 脚本下载完成后，浏览器的控制权转交给 JavaScript 引擎。当脚本执行完成后，控制权会交回给渲染引擎，渲染引擎继续往下解析HTML页面此时元素内容开始被解析，浏览器开始渲染页面

我们看到元素里有用于描述页面元数据的元素，还有一些元素涉及外部资源（如图片、CSS 样式等），此时浏览器会去获取这些外部资源。除此之外，我们还能看到元素中还包含着不少的元素，这些元素通过src属性指向外部资源

在这个过程中，我们看到中放置的元素会阻塞页面的渲染过程：把 JavaScript 放在里，意味着必须把所有 JavaScript 代码都下载、解析和解释完成后，才能开始渲染页面。如果外部脚本加载时间很长（比如一直无法完成下载），就会造成网页长时间失去响应，浏览器就会呈现“假死”状态，用户体验会变得很糟糕因此，对于对性能要求较高、需要快速将内容呈现给用户的网页，常常会将 JavaScript 脚本放在的最后面。这样可以避免资源阻塞，页面得以迅速展示。我们还可以使用defer/async/preload等属性来标记标签，来控制 JavaScript 的加载顺序

延迟加载的方式有哪些

js 的加载、解析和执行会阻塞页面的渲染过程，因此我们希望

js 脚本能够尽可能的延迟加载，提高页面的渲染速度。 几种方式是：

将 js 脚本放在文档的底部，来使 js 脚本尽可能的在后来加载执行给 js 脚本添加defer属性，这个属性会让脚本的加载与文档的解析同步解析，然后在文档解析完成后再执行这个脚本文件，这样的话就能使页面的渲染不被阻塞。多个设置了defer属性的脚本按规范来说最后是顺序执行的，但是在一些浏览器中可能不是这样给 js 脚本添加async属性，这个属性会使脚本异步加载，不会阻塞页面的解析过程，但是当脚本加载完成后立即执行 js脚本，这个时候如果文档没有解析完成的话同样会阻塞。多个async属性的脚本的执行顺序是不可预测的，一般不会按照代码的顺序依次执行动态创建DOM标签的方式，我们可以对文档的加载事件进行监听，当文档加载完成后再动态的创建script标签来引入 js 脚本

怎么判断页面是否加载完成 Load事件触发代表页面中的DOM，CSS，JS，图片已经全部加载完毕。DOMContentLoaded事件触发代表初始的HTML被完全加载和解析，不需要等待CSS，JS，图片加载

1) 活动页把同步放在，网络慢时长时间不出现；后端说HTML已经返回，所以不该白屏，你怎么解释？

答：浏览器自上而下解析到该脚本时会暂停HTML解析，等待脚本下载并交给JavaScript引擎执行，之后才继续处理。因此响应到达不等于页面可渲染；可把非关键脚本移到文档末尾，或按依赖关系选择defer、async。

2) 页面有三个存在依赖顺序的脚本，又不希望它们阻塞文档解析；一位开发选async，另一位选defer，你支持谁？

答：支持defer，它让脚本下载与文档解析并行，并在文档解析完成后执行，多个脚本按规范保持顺序。async下载完成就立即执行，执行时仍可能打断尚未完成的解析，而且多个脚本的先后不可预测，不适合存在依赖的链路。

3) 第三方统计脚本完全独立，但产品要求数据尽早上报，前端又不能让它持续阻塞首屏；你会在async、defer和动态插入之间怎么选？

答：独立且希望尽早执行时可选async，它不会等待文档解析完成，但下载完成后的执行仍可能短暂阻塞解析。若必须等文档完成再加载，可监听加载事件后动态创建script；选择取决于上报时效与渲染影响，不能假定async保证执行顺序。

4) 监控显示DOMContentLoaded已触发，但load又过了很久才发生，页面包含大图、样式和多个外部脚本；你如何判断卡在哪个资源？

答：这表示初始HTML已完成加载和解析，但页面依赖的其他资源尚未全部结束。结合网络请求检查仍在下载或失败重试的CSS、JS、图片等资源；load要等页面资源完成，不能只凭两事件间隔断定某一种资源。

5) 旧项目把所有脚本都移到末尾，新负责人要求统一改成async，理由是“两者都不会阻塞首屏”；你会接受统一替换吗？

答：不接受统一替换，末尾脚本按文档顺序遇到并执行，而async的执行顺序不可预测，下载完成时还可能打断解析。独立脚本适合async，有依赖顺序且需延后执行的脚本更适合defer；动态插入则适用于明确等待某个加载事件的场景。

418. 从输入URL到页面展示过程

输入URL后，浏览器会完成域名解析、建立连接、收发HTTP响应，再把资源渲染成可见页面。域名先经过多级缓存或DNS查询得到IP，随后通过三次握手建立TCP连接；如果是HTTPS，还要进行TLS握手来加密传输。服务器处理请求时会校验if-none-match、if-modified-since等缓存信息，缓存有效返回304，否则以200返回资源。浏览器拿到内容后构建DOM、计算样式和布局，再分层、栅格化并合成显示；大页面通常优先处理视口附近的图块，避免无意义的绘制开销。

1. DNS域名解析 根 DNS 服务器：返回顶级域 DNS 服务器的 IP 地址 顶级域 DNS 服务器：返回权威 DNS 服务器的 IP 地址 权威 DNS 服务器：返回相应主机的 IP 地址 DNS的域名查找，在客户端和浏览器，本地DNS之间的查询方式是递归查询；在本地DNS服务器与根域及其子域之间的查询方式是迭代查询；在客户端输入 URL 后，会有一个递归查找的过程，从浏览器缓存中查找->本地的hosts文件查找->找本地DNS解析器缓存查找->本地DNS服务器查找，这个过程中任何一步找到了都会结束查找流程。如果本地DNS服务器无法查询到，则根据本地DNS服务器设置的转发器进行查询。若未用转发模式，则迭代查找过程如下图：

结合起来的图表示：在查找过程中，有以下优化点：DNS存在着多级缓存，从离浏览器的距离排序的话，有以下几种：浏览器缓存，系统缓存，路由器缓存，IPS服务器缓存，根域名服务器缓存，顶级域名服务器缓存，主域名服务器缓存。在域名和IP的映射过程中，给了应用基于域名做负载均衡的机会，可以是简单的负载均衡，也可以根据地址和运营商做全局的负载均衡。

2. 建立TCP连接 首先，判断是不是https的，如果是，则HTTPS其实是HTTP + SSL / TLS 两部分组成，也就是在HTTP上又加了一层处理加密信息的模块。服务端和客户端的信息传输都会通过TLS进行加密，所以传输的数据都是加密后的数据 进行三次握手，建立TCP连接。第一次握手：建立连接。客户端发送连接请求报文段第二次握手：服务器收到SYN报文段。服务器收到客户端的SYN报文段，需要对这个SYN报文段进行确认第三次握手：客户端收到服务器的SYN+ACK报文段，向服务器发送ACK报文段 SSL握手过程 第一阶段 建立安全能力包括协议版本 会话Id 密码构件 压缩方法和初始随机数第二阶段 服务器发送证书 密钥交换数据和证书请求，最后发送请求-相应阶段的结束信号第三阶段 如果有证书请求客户端发送此证书之后客户端发送密钥交换数据 也可以发送证书验证消息第四阶段 变更密码构件和结束握手协议 完成了之后，客户端和服务端就可以开始传送数据 发送HTTP请求，服务器处理请求，返回响应结果 TCP连接建立后，浏览器就可以利用HTTP/HTTPS协议向服务器发送请求了。服务器接受到请求，就解析请求头，如果头部有缓存相关信息如if-none-match与if-modified-since，则验证缓存是否有效，若有效则返回状态码为304，若无效则重新返回资源，状态码为200

这里有发生的一个过程是HTTP缓存，是一个常考的考点，大致过程如图： 3. 关闭TCP连接 4. 浏览器渲染 按照渲染的时间顺序，流水线可分为如下几个子阶段：构建 DOM 树、样式计算、布局阶段、分层、栅格化和显示。如图：渲染进程将 HTML 内容转换为能够读懂DOM树结构。渲染引擎将 CSS 样式表转化为浏览器可以理解的styleSheets，计算出DOM节点的样式。创建布局树，并计算元素的布局信息。对布局树进行分层，并生成分层树。为每个图层生成绘制列表，并将其提

交到合成线程。合成线程将图层分图块，并栅格化将图块转换成位图。合成线程发送绘制图块命令给浏览器进程。浏览器进程根据指令生成页面，并显示到显示器上。 构建 DOM 树 转码 (Bytes -> Characters) —— 读取接收到的 HTML 二进制数据，按指定编码格式将字节转换为 HTML 字符串Tokens 化 (Characters -> Tokens) —— 解析 HTML，将 HTML 字符串转换为结构清晰的 Tokens，每个 Token 都有特殊的含义同时有自己的一套规则构建 Nodes (Tokens -> Nodes) —— 每个 Node 都添加特定的属性（或属性访问器），通过指针能够确定 Node 的父、子、兄弟关系和所属 treeScope（例如：iframe 的 treeScope 与外层页面的 treeScope 不同）构建 DOM 树 (Nodes -> DOM Tree) —— 最重要...

1) 用户输入域名后很快拿到IP，同事据此说“接下来浏览器只要发HTTP请求，不需要再建立连接”；如果地址是HTTPS，你怎么纠正？

答：拿到IP只完成了域名解析，通常还需通过三次握手建立TCP连接，并在其上完成TLS安全协商，之后才能传输加密的HTTP数据。缓存或连接复用可能缩短部分阶段，但不能把DNS命中等同于请求已经可发送。

2) 跨地区用户首次打开页面时域名解析明显偏慢，架构组准备调整权威解析和负载均衡；前端排查报告应把完整查询链路写到什么程度？

答：应先记录浏览器缓存、hosts、本地解析器缓存到本地DNS的递归查找，再说明本地DNS未命中时会向根域、顶级域和权威DNS迭代查询。域名到IP的映射还能承载按地区或运营商的负载均衡，但实际慢点需用观测数据确认。

3) 接口请求带上If-None-Match和If-Modified-Since后返回304，开发却仍等待一个完整响应体再渲染；这里的缓存流程应怎样理解？

答：304表示服务器验证缓存仍有效，浏览器应复用本地资源，而不是等待服务器重新发送完整实体。缓存无效或资源已变化时才重新返回资源并通常使用200；条件请求仍发生了网络往返，因此它与完全不发请求的本地命中不同。

4) 线上偶发“接口响应很快但页面仍白屏”，性能记录显示时间集中在样式计算和布局之后；你会沿渲染流水线怎么查？

答：响应完成后仍要经历构建DOM、样式计算、生成布局树、分层、生成绘制列表、栅格化和显示。应定位是复杂样式与布局计算、过大的图层，还是视口附近图块迟迟未完成栅格化；只看接口耗时无法解释渲染进程和合成线程的开销。

5) 长列表页面高达数十屏，产品要求首次打开就栅格化全部内容以保证滚动绝对流畅，浏览器工程师反对；你站哪边？

答：支持按视口附近图块优先栅格化，因为用户首次只能看到视口内的一小部分，提前处理全部大图层会产生不必要开销。合成线程负责分块并安排栅格化，再把绘制命令交给浏览器进程显示；过度预处理还可能拖慢首屏。

6) 面试中候选人把“构建DOM”直接说成“把字符串变成节点数组”，但线上乱码只发生在某种响应编码下；你会怎样继续追问并判断？

答：构建DOM前先要按指定编码把接收的字节转成字符，编码判断错误就可能在分词前产生乱码。随后才是字符转Token、Token转带属性和关系的节点，最终建立父子兄弟关系形成DOM树；节点数组的说法遗漏了转码和树结构。

419. 渲染引擎什么情况下才会为特定的节点创建新的图层

当节点形成独立层叠上下文，或者内容需要被裁剪时，渲染引擎会为它创建新的图层。常见触发条件可以归为定位与层级、视觉效果和渲染隔离，例如非auto的z-index、非none的transform、小于1的opacity

，以及will-change。本质上，这些属性要求节点独立参与合成或控制绘制范围。比如固定尺寸容器中的内容发生溢出时，文字裁剪区域会单独成层；如果还有滚动条，滚动条也可能成为独立图层。层叠上下文是HTML元素的三维概念，这些HTML元素在一条假想的相对于面向（电脑屏幕的）视窗或者网页的用户的z轴上延伸，HTML元素依据其自身属性按照优先级顺序占用层叠上下文的空间。

拥有层叠上下文属性的元素会被提升为单独的一层。拥有层叠上下文属性：根元素 (HTML), z-index 值不为 "auto" 的绝对/相对定位元素，position, 固定 (fixed) / 沾滞 (sticky) 定位 (沾滞定位适配所有移动设备上的浏览器，但老的桌面浏览器不支持) z-index 值不为 "auto" 的 flex 子项 (flex item)，即：父元素 display: flex | inline-flex, z-index 值不为 "auto" 的 grid 子项，即：父元素 display: grid opacity 属性值小于 1 的元素 (参考 the specification for opacity)，transform 属性值不为 "none" 的元素，mix-blend-mode 属性值不为 "normal" 的元素，filter 值不为 "none" 的元素，perspective 值不为 "none" 的元素，clip-path 值不为 "none" 的元素 mask / mask-image / mask-border 不为 "none" 的元素 isolation 属性被设置为 "isolate" 的元素在 will-change 中指定了任意 CSS 属性 (参考 这篇文章) -webkit-overflow-scrolling 属性被设置 "touch" 的元素 contain 属性值为 "layout", "paint", 或者综合值比如 "strict", "content" 需要剪裁 (clip) 的地方也会被创建为图层。这里的剪裁指的是，假如我们把 div 的大小限定为 200 * 200 像素，而 div 里面的文字内容比较多，文字所显示的区域肯定会超出 200 * 200 的面积，这时候就产生了剪裁，渲染引擎会把裁剪文字内容的一部分用于显示在 div 区域。出现这种裁剪情况的时候，渲染引擎会为文字部分单独创建一个层，如果出现滚动条，滚动条也会被提升为单独的层。

1) 商品卡片只设置了 position:

relative，没有 z-index，同事却断言它一定会生成独立图层；你会怎么纠正这个判断？

答：不能仅凭 position: relative 得出结论，源码列出的条件是绝对或相对定位元素同时满足 z-index 不为 auto。还要区分层叠上下文与渲染引擎实际创建的图层，不能把所有定位元素一概视为已经提层。

2) 长列表里有 500 个卡片，为了准备悬浮动画统一加了 will-change:

transform，上线后资源占用反而增加，你会怎么改？

答：不应给全部卡片长期声明 will-change，它会向渲染引擎提示相关属性即将变化，可能促使元素提前拥有独立层。只在动画临近开始时作用于实际运动的卡片，并在结束后移除；大量常驻图层会增加资源和合成管理成本。

3) 设计师把半透明浮层的 opacity 从 0.99 改成 1，同时去掉了 transform，你会预期它的层级行为发生什么变化？

答：opacity 小于 1 和 transform 不为 none 都属于源码列出的层叠上下文条件，两者移除后，这个浮层不再因它们获得独立层。它仍可能被 filter、isolation、定位与 z-index 等其他属性影响，因此必须结合完整样式判断。

4) 一个 200px × 200px 的滚动容器出现文字残影，现场只检查了 z-index；你会继续查哪些与分层有关的线索？

答：应继续检查容器的裁剪范围、溢出内容和滚动条，因为源码指出需要剪裁时，文字内容与滚动条可能被分别创建为图层。再核对 transform、filter、clip-path、mask 和 contain 等属性，避免把裁剪问题误判成单纯的层叠顺序错误。

5) 弹窗既能用定位配合非 auto 的 z-index，也能在父容器上设置 isolation:

isolate；评审时你会如何取舍？

答：两者都会建立新的层叠上下文，但表达的约束不同：前者直接参与定位元素的层叠排序，后者用于隔离内部元素与外部混合。应按弹窗的层级关系选择，不能只为“提层”叠加属性，否则会制造新的层叠边界并增加排查难度。

420. 定时器与requestAnimationFrame、requestIdleCallback

定时任务用setTimeout，动画更新用requestAnimationFrame，低优先级工作用requestIdleCallback。setTimeout到点只是把回调加入任务队列，主线程忙时仍要等待，并不保证准时执行。requestAnimationFrame会跟随屏幕刷新，在重绘前回调，后台标签页还会暂停，但主线程阻塞时同样可能掉帧。requestIdleCallback适合切割长任务或发送数据，不过只有当前帧有空闲时间才会执行，必要时可设置timeout。1. setTimeout setTimeout的运行机制：执行该语句时，是立即把当前定时器代码推入事件队列，当定时器在事件列表中满足设置的时间值时将传入的函数加入任务队列，之后的执行就交给任务队列负责。但是如果此时任务队列不为空，则需等待，所以执行定时器内代码的时间可能会大于设置的时间

```
setTimeout(()=>{console.log(1);},0)console.log(2);
```

 输出 2, 1;
setTimeout的第二个参数表示在执行代码前等待的毫秒数。上面代码中，设置为0，表面意思是 执行代码前等待的毫秒数为0，即立即执行。但实际上的运行结果我们也看到了，并不是表面上看起来的样子，千万不要被欺骗了。实际上，上面的代码并不是立即执行的，这是因为setTimeout有一个最小执行时间，HTML5标准规定了setTimeout()的第二个参数的最小值（最短间隔）不得低于4毫秒。当指定的时间低于该时间时，浏览器会用最小允许的时间作为setTimeout的时间间隔，也就是说即使我们把setTimeout的延迟时间设置为0，实际上可能为4毫秒后才事件推入任务队列。定时器代码在被推送到任务队列前，会先被推入到事件列表中，当定时器在事件列表中满足设置的时间值时会被推到任务队列，但是如果此时任务队列不为空，则需等待，所以执行定时器内代码的时间可能会大于设置的时间

```
setTimeout(()=>{console.log(111);},100);
```

上面代码表示100ms后执行console.log(111)，但实际上实行的时间肯定是大于100ms后的，100ms只是表示 100ms 后将任务加入到"任务队列"中，必须等到当前代码（执行栈）执行完，主线程才会去执行它指定的回调函数。要是当前代码耗时很长，有可能要等很久，所以并没有办法保证，回调函数一定会在setTimeout()指定的时间执行。2. setTimeout 和 setInterval区别 setTimeout: 指定延期后调用函数，每次setTimeout计时到后就会去执行，然后执行一段时间后才继续setTimeout,中间就多了误差，（误差多少与代码的执行时间有关）。setInterval：以指定周期调用函数，而setInterval则是每次都精确的隔一段时间推入一个事件（但是，事件的执行时间不一定就不准确，还有可能是这个事件还没执行完毕，下一个事件就来了）。btn.onclick=function(){setTimeout(function(){console.log(1);},250);} 点击按钮后，首先将onclick事件处理程序加入队列。该程序执行后才设置定时器，再有250ms后，指定的代码才被添加到队列中等待执行。如果上面代码中的onclick事件处理程序执行了300ms，那么定时器的代码至少要在定时器设置之后的300ms后才会被执行。队列中所有的代码都要等到javascript进程空闲之后才能执行，而不管它们是如何添加到队列中的。如图所示，尽管在255ms处添加了定时器代码，但这时候还不能执行，因为onclick事件处理程序仍在运行。定时器代码最早能执行的时机是在300ms处，即onclick事件处理程序结束之后。3. setInterval存在的一些问题： JavaScript中使用setInterval开启轮询。定时器代码可能在代码再次被添加到队列之前还没有完成执行，结果导致定时器代码连续运行好几次，而之间没有任何停顿。而javascript引擎对这个问题的解决是：当使用setInterval()时，仅当没有该定时器的任何其他代码实例时，才将定时器代码添加到队列中。这确保了定时器代码加入到队列中的最小时间间隔为指定间隔。

但是，这样会导致两个问题：某些间隔被跳过；多个定时器的代码执行之间的间隔可能比预期的小 假设，某个onclick事件处理程序使用setInterval()设置了200ms间隔的定时器。如果事件处理程序花了300ms多一点时间完成，同时定时器代码也花了差不多的时间，就会同时出现跳过某间隔的情况 例子中的第一个定时器是在205ms处添加到队...

421. Vue 响应式原理

Vue响应式本质上是拦截数据读写：读取时收集依赖，修改时通知Watcher更新视图。Vue 2用Object.defineProperty()递归劫持属性，数组则重写push、splice等变异方法，再通过虚拟DOM的对比局部更新真实DOM。它无法直接感知对象属性的新增和删除，需要使用Vue.set()、Vue.delete()。Vue

3改用Proxy代理对象和数组，嵌套对象可在读取时继续交给reactive处理。

Vue

的响应式原理是核心是通过 ES5 的保护对象的Object.defineProperty中的访问器属性中的 get 和 set 方法，data 中声明的属性都被添加了访问器属性，当读取 data 中的数据时自动调用 get 方法，当修改 data 中的数据时，自动调用 set 方法，检测到数据的变化，会通知观察者 Wacher，观察者 Wacher自动触发重新render 当前组件（子组件不会重新渲染），生成新的虚拟 DOM 树，Vue 框架会遍历并对比新虚拟 DOM 树和旧虚拟 DOM 树中每个节点的差别，并记录下来，最后，加载操作，将所有记录的不同点，局部修改到真实 DOM树上。

虚拟DOM (Virtual DOM): 用 js 对象模拟的，保存当前视图内所有 DOM 节点对象基本描述属性和节点间关系的树结构。用js对象，描述每个节点，及其父子关系，形成虚拟 DOM 对象树结构。因为只要在data中声明的基本数据类型的数据，基本不存在数据不响应问题，所以重点介绍数组和对象在vue中的数据响应问题，vue可以检测对象属性的修改，但无法监听数组的所有变动及对象的新增和删除，只能使用数组变异方法及\$set方法。可以看到，arrayMethods首先继承了Array，然后对数组中所有能改变数组自身的方法，如push、pop等这些方法进行重写。重写后的方法会先执行它们本身原有的逻辑，并对能增加数组长度的 3 个方法push、unshift、splice方法做了判断，获取到插入的值，然后把新添加的值变成一个响应式对象，并且再调用ob.dep.notify()手动触发依赖通知，这就很好地解释了用vm.items.splice(newLength) 方法可以检测到变化 总结：Vue

采用数据劫持结合发布—订阅模式的方法，通过Object.defineProperty()来劫持各个属性的

setter, getter，在数据变动时发布消息给订阅者，触发相应的监听回调。 Observer遍历数据对象，给所有属性加上setter和getter，监听数据的变化compile解析模板指令，将模板中的变量替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，

收到通知，更新视图 Watcher订阅者是Observer和Compile之间通信的桥梁，主要做的事情在自身实例化时往属性订阅器 (dep)

里面添加自己待属性变动dep.notice()通知时，调用自身的update()方法，并触发Compile中绑定的回调

Object.defineProperty()，那么它的用法是什么，以及优缺点是什么呢？可以检测对象中数据发生的修改对于复杂的对象，层级很深的话，是不友好的，需要经行深度监听，这样子就需要递归到底，这也是它的缺点。对于一个对象中，如果你新增加属性，删除属性，**Object.defineProperty()**是不能观测到的，

那么应该如何解决呢？可以通过Vue.set()和Vue.delete()来实现。 // 模拟 Vue 中的 data 选项letdata={msg:'hello'}// 模拟 Vue 的实例letvm={}// 数据劫持:当访问或者设置 vm 中的成员的时候，做一些干预操作Object.defineProperty(vm,'msg',{//

可枚举(可遍历)enumerable:true,// 可配置(可以使用 delete 删除，可以通过 defineProperty 重新定义)configurable:true://

当获取值的时候执行get(){console.log('get:',data.msg)returndata.msg},// 当设置值的时候执行set(newValue){console.log('set:',newValue)if(newValue===data.msg){return}data.msg=newValue// 数据更改，更新 DOM 的值document.querySelector('#app').textContent=data.msg}})// 测试vm.msg='Hello World'...

1) Vue 2表单里执行form.newKey =

1后控制台数据已变化，但输入区没有更新；有人说是虚拟DOM的diff漏了，你同意吗？

答：不同意，根因更可能是新增属性没有被Object.defineProperty()转换成getter和setter，变化因而没有进入依赖通知链路。应使用Vue.set()或\$set添加属性；若是删除则使用Vue.delete()，而不是

先怀疑diff。

2) 一个多层配置对象初始化后会频繁替换子对象，你要用Observer、Dep和Watcher解释一次更新链路，会怎么说？

答：Observer会把已声明的多层属性转换为访问器，并在新对象被赋值时继续处理其成员；读取阶段由Dep收集对应的Watcher。写入触发通知后，Watcher更新视图并生成新的虚拟DOM，再比较差异并局部修改真实DOM。

3) 列表页直接执行items[3] = row和items.length =

2后视图不同步，但改成splice就恢复了；你如何解释并修复？

答：在Vue 2中，并非所有数组变动都能被访问器机制观察，而splice等变异方法经过重写，会处理新增值并调用依赖通知。应使用框架支持的数组变异方法或\$set；直接索引赋值和改长度不能假定会触发视图更新。

4) 把同一份深层对象从Vue 2迁到Vue

3后，评审者认为Proxy只代理第一层，所以嵌套字段不会响应；你怎么回应？

答：外层Proxy的确先拦截第一层访问，但读取结果若仍是对象，可以在Reflect.get后继续通过reactive建立代理，从而实现深度观测。它通常是按访问过程继续代理，并不等于所有嵌套对象失去响应性；具体行为仍要检查实际实现。

5) Vue 3数组一次写入触发了多次get、set日志，页面也重复更新；你会在哪些条件上收紧触发逻辑？

答：应检查写入的key是否是目标对象自身属性，并比较旧值与新值，避免没有实际变化时仍调用trigger。数组操作可能经过多个代理陷阱，不能把每次拦截都等同于有效变更；条件过宽会造成多余通知，过窄则可能漏更新。

6) 有人主张在大型对象初始化时递归给所有层级做defineProperty比Proxy更合适，因为两者都能深度监听；你如何权衡？

答：defineProperty需要遍历并改造已有属性，新增和删除还要借助Vue.set()、Vue.delete()；深层对象会放大递归处理成本。Proxy能直接监听对象和数组，并可在读取到嵌套对象时继续代理，但它属于不同的运行机制，迁移不能只替换一个API。

422. 发布订阅模式和观察者模式

两种模式的核心区别是有没有事件中心：观察者模式直接关联双方，发布订阅模式通过中间层转发。在观察者模式中，目标对象保存观察者，状态变化后直接调用其update()，所以双方存在依赖。发布订阅中，发布者只负责\$emit，订阅者只负责\$on，彼此不需要知道对方。像兄弟组件通过eventBus通信，就是典型的发布订阅场景。

1. 发布/订阅模式 发布者发布者信号中心 订阅者发布者信号中心

我们假定，存在一个"信号中心"，某个任务执行完成，就向信号中心"发布"(publish)一个信号，其他任务可以向信号中心"订阅"(subscribe)这个信号，从而知道什么时候自己可以开始执行。这就叫做"发布/订阅模式"(publish-subscribe pattern)

Vue 的自定义事件 let vm=newVue()vm.\$on('dataChange',()=>{console.log('dataChange')})vm.\$on('dataChange',()=>{console.log('dataChange1')})vm.\$emit('dataChange')

兄弟组件通信过程 // eventBus.js//

事件中心leteventHub=newVue()// ComponentA.vue// 发布者addTodo:function(){// 发布消息(事件)eventHub.\$emit('add-todo',{text:this.newTodoText})this.newTodoText=""// ComponentB.vue// 订阅者created:function(){// 订阅消息(事件)eventHub.\$on('add-todo',this.addTodo)} 模拟 Vue 自定义事件的实现

```
class EventEmitter{
  constructor(){
    {
      eventType: [ handler1, handler2 ]
    }
  }
  this.subs={}
  // 订阅通知$on(e
```



```

eventType,fn){this.subs[eventType]=this.subs[eventType]||[]this.subs[eventType].push(fn)}//
发布通知$emit(eventType){if(this.subs[eventType]){this.subs[eventType].forEach(v=>v())}}//
测试varbus=newEventEmitter()// 注册事件bus.$on('click',function(){console.log('click')})bus.$on('cli
ck',function(){console.log('click1')})// 触发事件bus.$emit('click')
2. 观察者模式 观察者(订阅者)
--Watcherupdate():当事件发生时，具体要做的事情目标(发布者) --Depsubs数组:存储所有的观察者add
Sub():添加观察者notify():当事件发生，调用所有观察者的update()方法没有事件中心
update():当事件发生时，具体要做的事情
subs数组:存储所有的观察者addSub():添加观察者notify():当事件发生，调用所有观察者的update()方法
// 目标(发布者)// DependencyclassDep{constructor(){// 存储所有的观察者this.subs=[]}
添加观察者addSub(sub){if(sub&&sub.update){this.subs.push(sub)}}//
通知所有观察者notify(){this.subs.forEach(sub=>sub.update())}
观察者(订阅者)classWatcher{update(){console.log('update')}}
测试letdep=newDep()letwatcher=newWatcher()dep.addSub(watcher)dep.notify()
3. 总结
观察者模式是由具体目标调度，比如当事件触发，Dep就会去调用观察者的方法，所以观察者模 式的订
阅者与发布者之间是存在依赖的发布/订阅模式由统一调度中心调用，因此发布者和订阅者不需要知道对方的
存在

```

1) 订单页里OrderStore直接保存三个视图观察者并逐个调用update()，同事却把它叫作发布订阅；你会怎么追问他？

答：这更符合观察者模式，因为具体目标直接持有观察者，并在状态变化时调用它们的update()，中间没有独立信号中心。应让他指出发布者与订阅者之间的调度媒介；若只能找到目标内部的订阅列表，就不能按发布订阅描述。

2) 兄弟组件通过全局eventHub.\$emit('add-todo')通信，新增页面也要监听同名事件；你会要求怎样组织注册与清理？

答：应把事件名称、载荷约定和订阅位置集中管理，并在组件不再需要监听时解除对应处理函数。发布订阅让双方通过事件中心解耦，但也隐藏了调用关系；监听增长后若缺少生命周期管理，会留下重复执行和难以追踪的订阅。

3) 需求改成一个发布者要通知多个完全不认识的业务模块，而且模块可以独立上下线；你会选直接观察还是信号中心？

答：更适合使用带信号中心的发布订阅，让发布者只发布事件，各模块自行订阅，不需要互相持有引用。代价是执行链路不再直观，必须约束事件命名、载荷和取消订阅；若参与者固定且关系紧密，直接观察反而更简单。

4) 线上一次点击打印了三遍click，代码里只有一次\$emit；你会怎样定位这个EventEmitter故障？

答：先检查同一处理函数是否被多次\$on，以及组件重复创建时旧订阅是否仍保留，因为\$emit会遍历事件类型对应的全部处理函数。再记录注册与销毁路径确认订阅数量；只在触发处防抖会掩盖重复注册，不能消除泄漏。

5) 状态库里的Dep已经能通知Watcher，架构师仍要求再套一层全局事件中心；你认为值得吗？

答：若Dep已直接维护并通知明确的Watcher，再加信号中心会把观察者链路改成更间接的调度，未必带来收益。只有确实需要让互不认识的发布者和订阅者解耦时才值得引入；否则会增加事件契约和故障定位成本。

6) 评审中有人说Watcher既然叫订阅者，Vue响应式就必然是发布订阅模式；你会如何串联两个概念？

答：“订阅者”这个角色名不足以决定模式，关键是通知由谁调度。Dep保存Watcher并直接调用其update()时属于观察者结构；只有发布者和订阅者通过独立信号中心交互，才符合这里的发布订阅模式。

423. 为什么使用 Virtual DOM

使用虚拟DOM，是为了用普通对象描述视图，并在状态变化后统一计算真实DOM应该怎样更新。它会保留上一次视图状态，通过diff找出前后差异，只把必要变化同步到页面，避免开发者手动维护复杂的DOM操作。它在复杂视图下有助于控制渲染成本，但不能简单理解为任何场景都更快。更重要的是它解耦了具体渲染平台，同一套视图描述还能用于SSR、原生应用或小程序。手动操作DOM比较麻烦，还需要考虑浏览器兼容性问题，虽然有jQuery等库简化DOM操作，但是随着项目的复杂DOM操作复杂提升为了简化DOM的复杂操作于是出现了各种MVVM框架，MVVM框架解决了视图和状态的同步问题为了简化视图的操作我们可以使用模板引擎，但是模板引擎没有解决跟踪状态变化的问题，于是Virtual DOM出现了Virtual DOM的好处是当状态改变时不需要立即更新DOM，只需要创建一个虚拟树来描述DOM，Virtual DOM内部将弄清楚如何有效(diff)的更新DOM虚拟DOM可以维护程序的状态，跟踪上一次的状态通过比较前后两次状态的差异更新真实DOM 虚拟DOM的作用 维护视图和状态的关系复杂视图情况下提升渲染性能除了渲染DOM以外，还可以实现SSR(Nuxt.js/Next.js)、原生应用(Weex/React Native)、小程序(mpvue/uni-app)等

1) 活动页只有一个计数器，候选人说引入Virtual DOM后每次更新一定比精准设置textContent更快；你认同吗？

答：不能作出“一定更快”的结论，因为Virtual DOM还要创建虚拟树并比较前后状态，精准的直接更新可能路径更短。它的核心价值是维护视图与状态关系，并由框架确定差异更新；简单节点不应仅凭性能口号选型。

2) 复杂表单有几十处状态联动，团队现在到处手写DOM查询和赋值；引入Virtual DOM能解决哪部分工程问题？

答：它能用虚拟树描述当前视图，在状态变化后比较新旧状态，只把记录到的差异更新到真实DOM。这样状态与视图的同步由统一机制维护，减少散落的手动操作；但业务状态设计不合理时，虚拟DOM本身不会消除逻辑复杂度。

3)

同一套组件既要在浏览器渲染，又计划支持SSR和原生容器；你会怎样评价虚拟树这层抽象？

答：虚拟树不只用于浏览器DOM，还可作为视图描述支持SSR、原生应用或小程序等不同渲染目标。它让状态与具体宿主操作之间多一层抽象，但各目标仍需要对应渲染实现；不能把“能描述”误解为无需适配即可运行。

4) 线上列表更新后出现旧行残留，研发只盯着真实DOM截图；按虚拟DOM链路你会依次核对什么？

答：先确认状态是否真的变化，再检查新虚拟树是否正确描述目标视图，以及新旧树比较是否记录了应有差异，最后核对差异是否应用到真实DOM。若状态源本身错误，继续调DOM更新没有意义；若有人绕过框架直接改DOM，也会破坏前后状态的一致性。

5) 可视化大屏每帧都要更新大量节点，一方坚持全部交给虚拟DOM，另一方要求全部手写；你会如何决策？

答：不能只凭技术立场决定，应先区分高频热点区域与普通状态视图，并验证比较和真实更新分别占据多少成本。复杂视图可借虚拟DOM维护状态关系，高频且更新路径固定的局部则可能需要更直接的策略；混用时必须划清所有权，避免双方同时修改同一节点。

6) 模板引擎已经能根据数据生成HTML，产品经理质疑为何还要虚拟DOM；你会怎么解释两者边界？

答：模板能简化视图生成，但仅生成内容并不等于持续跟踪状态变化。虚拟DOM会保留上一次视图状态，在新状态到来时比较两棵虚拟树并更新差异；代价是增加运行时表示与比较过程，因此静态页面未必需要这层机制。

424. VDOM：三个 part

VDOM可以拆成三个部分：用VNode描述节点、通过diff记录差异、再根据patch更新真实DOM。VNode只是保留必要信息的普通JS对象，创建成本通常低于直接创建属性繁多的真实节点。数据变化先集中反映到虚拟树，再统一修改DOM tree，可以减少零散操作引发的重绘和回流。它的价值也不只在性能，还解耦了对HTML的依赖，方便做AOT、SSR和其他平台渲染。虚拟节点类，将真实DOM节点用js对象的形式进行展示，并提供render方法，将虚拟节点渲染成真实DOM节点diff比较：对虚拟节点进行js层面的计算，并将不同的操作都记录到patch对象re-render：解析patch对象，进行re-render 补充1VDOM的必要性？ 创建真实DOM的代价高：真实的DOM节点node实现的属性很多，而vnode仅仅实现一些必要的属性，相比起来，创建一个vnode的成本比较低。触发多次浏览器重绘及回流：使用vnode，相当于加了一个缓冲，让一次数据变动所带来的所有node变化，先在vnode中进行修改，然后diff之后对所有产生差异的节点集中一次对DOM tree进行修改，以减少浏览器的重绘及回流。 补充2：vue 为什么采用vdom？ 引入Virtual DOM在性能方面的考量仅仅是一方面。性能受场景的影响是非常大的，不同的场景可能造成不同实现方案之间成倍的性能差距，所以依赖细粒度绑定及Virtual DOM哪个的性能更好还真不是一个容易下定论的问题。Vue之所以引入了Virtual DOM，更重要的原因是为了解耦HTML依赖，这带来两个非常重要的好处是：不再依赖HTML解析器进行模版解析，可以进行更多的AOT工作提高运行时效率：通过模版AOT编译，Vue的运行时体积可以进一步压缩，运行时效率可以进一步提升；可以渲染到DOM以外的平台，实现SSR、同构渲染这些高级特性，Weex等框架应用的就是这一特性。 综上，Virtual DOM在性能上的收益并不是最主要的，更重要的是它使得Vue具备了现代框架应有的高级特性。

1) 商品列表更新后页面已经变了，同事却说VDOM就是缓存的DOM字符串；结合一次更新链路，你会怎么纠正他？

答：VDOM不是HTML字符串，而是用精简的JavaScript对象描述节点。数据变化后先生成并比较虚拟节点，把差异记录为patch，再解析patch更新真实DOM；如果直接跳过比较操作真实节点，就不属于这条更新链路。

2) 一个表格组件每次筛选都直接遍历并修改多处真实DOM，评审时你为什么可能要求改成先计算vnode差异再统一更新？

答：多处真实DOM修改可能连续触发重绘和回流，而虚拟节点可充当更新缓冲。应先在JavaScript层完成diff、汇总为patch，再集中修改DOM tree；但更新范围很小且路径固定时，不能仅凭使用VDOM就断言一定更快。

3) 一个画布编辑器每帧只移动单个元素，负责人仍坚持“用了VDOM性能必然最好”，你会怎样判断？

答：不能把VDOM视为所有场景下的性能保证，它还要承担创建虚拟节点和执行diff的成本。应根据更新频率、节点规模和真实DOM操作范围验证收益；对高度可预测的细粒度更新，直接绑定或定向修改可能更合适。

4) 线上出现“状态已变化但局部视图没更新”，你沿着vnode → diff → patch → re-render会怎么定位？

答：先确认新状态是否生成了预期vnode，再检查diff是否识别出对应差异并写入patch。若patch正确，则继续查看解析和真实DOM更新阶段；逐段核对能区分响应数据、差异计算与提交渲染三类故障。

5) 团队要让同一套组件既支持浏览器页面又支持SSR和非DOM平台，为什么VDOM的价值不只在减少回流？

答：虚拟节点把组件描述与浏览器HTML、真实DOM解耦，使渲染结果能交给不同平台的渲染器处理。模板还可提前编译为render function，承载更多AOT优化；代价是增加虚拟节点、diff和平台适配层的复杂度。

425. vue 和 react技术选型

Vue和React都能做数据驱动、组件化和服务端渲染，选型关键是项目复杂度与团队匹配度。Vue提供v-model，写法直观，但底层仍是单向数据流；React更强调不可变数据和单向更新。大规模数据渲染及多人协作的复杂项目，可以考虑React配合Redux；追求快速开发的小型项目，Vue通常更顺手。开发风格上，React常用JSX，而Vue多用单文件组件组织模板、逻辑和样式。

相同点：数据驱动页面，提供响应式的视图组件都有virtual DOM,组件化的开发，通过props参数进行父子之间组件传递数据，都实现了webComponents规范数据流动单向，都支持服务器的渲染SSR都有支持native的方法，react有React native，vue有wexx

不同点：数据绑定：Vue实现了双向的数据绑定v-model，底层本质上还是单向数据流（v-model隐藏了背后:value 和 @input 的细节）数据渲染：大规模的数据渲染，react更快使用场景：React配合Redux架构适合大规模多人协作复杂项目，Vue适合小快的项目开发风格：react推荐做法jsx + inline style把html和css都写在js了 vue是采用webpack +vue-loader单文件组件格式，html, js, css同一个文件

1) 表单页用了v-model，评审同事因此认定Vue是双向数据流、React才是单向数据流，你会怎么解释？

答：v-model提供双向绑定的使用体验，但底层仍可理解为值向下传递、事件向上通知。它隐藏了类似:value与@input的配对细节，并没有推翻单向数据流；复杂表单仍要明确数据所有者，避免状态来源不清。

2) 五十人协作的大型业务平台要在Vue与React间选型，架构负责人只拿模板语法简洁度做决定，你会补充哪些判断？

答：大型多人协作项目应重点评估状态管理、代码约束和团队协作方式，来源材料更倾向React配合Redux。Vue的单文件组件适合集中组织模板、逻辑和样式，但规模结论不是绝对规则，最终仍要结合团队能力与既有生态。

3) 数据看板要渲染大规模数据，团队因为材料写着“React更快”就准备整体迁移，你会批准吗？

答：不会仅凭框架级结论批准迁移，材料只给出了大规模数据渲染中React更快的概括，未限定组件结构和更新模式。应先定位瓶颈并用当前页面验证；迁移会引入重写与协作成本，而且无法替代列表裁剪等针对性优化。

4) 新项目既要浏览器端SSR，半年后又可能扩展原生端，产品要求现在就锁定技术栈，你怎样比较两者？

答：两者都支持组件化、单向数据流和服务端渲染，因此SSR本身不足以拉开差距。原生方向可比较React

Native与Weex相关方案及团队能力；跨端只是选型约束之一，不能忽略生态成熟度和维护成本。

5) 线上页面出现父组件数据已变、子组件显示仍旧值，开发者把原因归结为“Vue和React数据流不同”，你会先查什么？

答：两者都通过props进行父子数据传递，并以单向数据流组织更新，不能先用框架差异解释故障。应检查父组件传值、子组件是否复制并滞留旧状态，以及更新是否实际触发；若使用v-model，还要核对值与事件链路。

426. nextTick

nextTick用来把回调推迟到下一轮DOM更新结束后执行，从而拿到更新后的页面节点。因为Vue不会在每次数据变化时立刻更新DOM，而是把任务放进队列并统一清空，多次调用也会先收集起来。实现时会根据环境依次尝试Promise、MutationObserver、setImmediate，最后才使用setTimeout。所以修改数据后马上读取尺寸或内容时，我一般会放到nextTick回调里。

nextTick可以让我们在下次DOM更新循环结束之后执行延迟回调，用于获得更新后的DOM

nextTick主要使用了宏任务和微任务。根据执行环境分别尝试采用

PromiseMutationObserversetImmediate如果以上都不行则采用setTimeout

定义了一个异步方法，多次调用nextTick会将方法存入队列中，通过这个异步方法清空当前队列

1) 详情页执行this.count++后立刻读取节点文本仍是旧值，同事认为响应式失效了；你会怎样解释并修正？

答：数据变化不代表真实DOM已在当前同步代码中完成更新，因此立即读取可能仍是旧内容。把读取逻辑放进nextTick，让它在下一次DOM更新循环结束后执行；它只保证等待本轮更新，不负责修复错误的数据依赖。

2)

批量导入页在一个同步循环里调用了二十次nextTick，这些回调会各自启动一次定时任务吗？

答：多次调用会先把回调保存到同一队列，再由选定的异步方法清空当前队列，而不是简单理解为每次都独立刷新。回调仍应保持轻量；如果其中执行大量计算，即使合并调度也会阻塞后续渲染。

3) 老旧运行环境没有原生Promise，业务依赖nextTick更新后聚焦输入框，它是否必然失效？

答：不必然失效，nextTick会按环境尝试Promise、MutationObserver、setImmediate，最后可退回setTimeout。不同回退方式属于微任务或宏任务，执行时机可能有差异；业务不应依赖比“更新循环结束后”更细的顺序假设。

4) 弹窗打开后偶发拿不到更新后的输入框，日志显示数据已改且nextTick已调用，你会怎样排查？

答：先确认修改数据和注册nextTick的先后关系，并检查输入框是否受条件渲染控制。随后在回调中验证节点是否已生成，排除组件尚未进入对应更新循环；若节点由后续异步数据决定，一次nextTick并不能等待那次未来更新。

5) 开发者想把所有nextTick都替换成setTimeout(fn, 0)，理由是最终都能异步执行，你会接受吗？

答：不应直接等价替换，nextTick面向的是本轮DOM更新结束后的回调队列，并会选择当前环境可用的异步机制。setTimeout只是最终回退方案，通常属于更晚的宏任务；替换会削弱与框架更新周期之间的语义关联。

427. 生命周期

Vue生命周期描述了组件从初始化、挂载、更新到销毁的完整过程，不同阶段适合处理不同事情。beforeCreate时状态还没初始化，到了created已完成数据响应性，但真实DOM尚未生成；执行渲染和挂载后才进入mounted。数据变化时会经过依赖通知、差异比较，并依次触发beforeUpdate和updated；销毁时则在对应钩子间移除节点、依赖和监听。keep-alive组件不会直接销毁，而是通过激活与失活钩子切换状态。

init initLifecycle/Event，往vm上挂载各种属性callHook: beforeCreated:

实例刚创建initInjection/initState: 初始化注入和data响应性created: 创建完成，属性已经绑定，但还未生成真实dom`进行元素的挂载：\$el / vm.\$mount()是否有template: 解析成render function*.vue文件:vue-loader会将编译成render functionbeforeMount: 模板编译/挂载之前执行render function，生成真实的dom，并替换到dom tree中mounted: 组件已挂载*.vue文件:vue-loader会将编译成render function update

执行diff算法，比对改变是否需要触发UI更新flushScheduleQueue watcher.before: 触发beforeUpdate钩子 -watcher.run(): 执行watcher中的notify，通知所有依赖项更新UI触发updated钩子: 组件已更新activated / deactivated(keep-alive): 不销毁，缓存，组件激活与失活destroybeforeDestroy: 销毁开始销毁自身且递归销毁子组件以及事件监听remove(): 删除节点watcher.teardown(): 清空依赖vm.\$off(): 解绑监听destroyed: 完成后触发钩子 beforeDestroy: 销毁开始销毁自身且递归销毁子组件以及事件监听remove(): 删除节点watcher.teardown(): 清空依赖vm.\$off(): 解绑监听destroyed: 完成后触发钩子 remove(): 删除节点watcher.teardown(): 清空依赖vm.\$off(): 解绑监听

上面是vue的声明周期的简单梳理，接下来我们直接以代码的形式来完成vue的初始化 newVue({})// 初始化Vue实例function _init(){// 挂载属性initLifeCycle(vm)// 初始化事件系统，钩子函数等initEvent(vm)// 编译slot、vnodeinitRender(vm)// 触发钩子callHook(vm,'beforeCreate')// 添加inject功能initInjection(vm)// 完成数据响应性 props/data/watch/computed/methodsinitState(vm)// 添加 provide 功能initProvide(vm)// 触发钩子callHook(vm,'created')// 挂载节点if(vm.\$options.el){vm.\$mount(vm.\$options.el)}// 挂载节点实现function mountComponent(vm){// 获取 render functionif(!this.options.render){// template to render// Vue.compile = compileToFunctionslet{render}=compileToFunctions()this.options.render=render}// 触发钩子callHook('beforeMount')// 初始化观察者// render 渲染 vdom，vdom=vm.render()// update: 根据 diff 出的 patches 挂载成真实的 domvm._update(vdom)// 触发钩子callHook(vm,'mounted')// 更新节点实现function queueWatcher(watcher){nextTick(flushScheduleQueue)}// 清空队列function flushScheduleQueue(){// 遍历队列中所有修改for(){// beforeUpdatewatcher.before()// 依赖局部更新节点watcher.update()callHook('updated')}}// 销毁实例实现Vue.proto...

1) 图表页在created中读取this.\$refs.chart得到undefined，但接口请求能正常发出；这两个现象为什么不矛盾？

答：created时数据、方法和注入等状态已经初始化，因此可以启动不依赖真实节点的逻辑。此时组件尚未挂载并生成真实DOM，所以\$refs不可用；图表初始化应放到mounted，同时考虑卸载时释放实例。

2) 后台页一次修改多项响应式数据，开发者期望每次赋值都立即触发一轮beforeUpdate和updated，你会如何纠正？

答：更新会进入调度队列，由flushScheduleQueue统一处理相关watcher。更新前触发beforeUpdate，依赖完成更新后再触发updated；不要把钩子次数与赋值次数机械对应，也不应在updated中无条件继续改状态。

3) 带keep-alive的搜索页切走后定时器仍在运行，但组件没有触发销毁钩子，你会把清理放在哪里？

答：keep-alive切换时组件只是失活并被缓存，不会走完整销毁流程，因此应在deactivated暂停定时器或其他活跃任务。再次进入时在activated恢复；真正销毁仍需在beforeDestroy或onBeforeUnmount等阶段完成最终清理。

4) 单页应用运行数小时后内存持续上涨，路由离开组件时节点消失了，你会沿销毁阶段检查哪些链路？

答：先确认是否进入beforeDestroy、执行自身及子组件移除，再核对watcher.teardown()和vm.\$off()是否完成。还要检查业务自行注册的全局监听、定时器或第三方实例；框架解绑自身事件，并不必然覆盖所有外部资源。

5) 团队把Vue 2组件迁到Vue

3，准备逐个把created改成onCreated，你会怎么调整方案？

答：Vue 3没有对应的onCreated，beforeCreate和created的初始化职责通常由setup承接。其余阶段使用onBeforeMount、onMounted、onBeforeUpdate、onUpdated、onBeforeUnmount与onUnmounted；迁移时还要重新确认逻辑对真实DOM的依赖。

428. vue-router

vue-router的本质是监听URL变化，匹配路由配置，再通过router-view渲染对应组件。hash模式监听hashchange，history模式监听popstate，区别主要在于感知地址变化的方式。代码跳转可以用this.\$router.push()，模板里则常用router-link，最终都驱动当前路由状态变化。它作为插件通过install接入Vue，注册全局组件，并用响应式的current让匹配组件自动重新渲染。mode hashhistory 跳转 this.\$router.push() 占位 vue-router源码实现 作为一个插件存在:实现VueRouter类和install方法实现两个全局组件:router-view用于显示匹配组件内容，router-link用于跳转监控url变化:监听hashchange或popstate事件响应最新url: 创建一个响应式的属性current，当它改变时获取对应组件并显示 // 我们的插件：//

1.实现一个Router类并挂载期实例//

2.实现两个全局组件router-link和router-viewlet Vue;class VueRouter{// 核心任务：//

1.监听url变化constructor(options){this.\$options=options;// 缓存path和route映射关系// 这样找组件更快this.routeMap={}this.\$options.routes.forEach(route=>{this.routeMap[route.path]=route})//

数据响应式// 定义一个响应式的current，则如果他变了，那么使用它的组件会rerenderVue.util.defineReactive(this,'current','')// 请确保onHashChange中this指向当前实例window.addEventListener('hashchange',this.onHashChange.bind(this))window.addEventListener('load',this.onHashChange.bind(this))onHashChange()//

console.log(window.location.hash);this.current=window.location.hash.slice(1)||'/'}//

插件需要实现install方法//

接收一个参数，Vue构造函数，主要用于数据响应式VueRouter.install=function(_Vue){//

保存Vue构造函数在VueRouter中使用Vue=_Vue//

任务1：使用混入来做router挂载这件事情Vue.mixin({beforeCreate()){//

只有根实例才有router选项if(this.\$options.router){Vue.prototype.\$router=this.\$options.router}}//

任务2：实现两个全局组件// router-link: 生成一个a标签，在url后面添加#// aaaa//

aaaVue.component('router-link',{props:{to:{type:String,required:true},},render(h){// h(tag, props,

```
children)returnh('a',{attrs:{href:'#'+this.to}},this.$slots.default)// 使用jsx// return
{this.$slots.default}}})Vue.component('router-view',{render(h){// 根据current获取组件并render//
current怎么获取?// console.log('render',this.$router.current);// 获取要渲染的组件letcomponent=null
const{routeMap,current}=this.$routerif(routeMap[current]){component=routeMap[current].comp
onent)returnh(component)}}})exportdefaultVueRouter
```

1) 订单页点击后没有整页刷新，同事却说它只是普通链接；路由插件实际还做了什么？

答：router-link负责生成可跳转的链接，router-view根据当前地址渲染匹配组件。插件还要监听hash change或popstate，并把最新地址写入响应式的current；仅生成标签不足以完成视图联动。

2) 你要实现一个最小版hash路由，配置里有二十条静态路径，为什么先构建routeMap，地址变化后又怎样更新页面？

答：初始化时把path与路由配置缓存到routeMap，可在地址变化时直接查找对应组件。监听hashchange和首次load，更新响应式current，依赖它的router-view就会重新渲染；该简化实现未覆盖动态参数等复杂匹配。

3) 产品要求把地址从#/detail改成/detail，开发只删除了#，结果浏览器前进后退不再更新视图，你会检查哪里？

答：无#的历史模式应监听popstate，不能继续只依赖hashchange。还要确认跳转逻辑确实更新浏览器历史，并同步响应式当前地址；仅改变链接外观不会自动补齐历史模式的监听和状态维护。

4) 页面地址已经变成#/about，但仍显示首页，你按源码链路会怎么定位？

答：先检查hashchange是否触发，以及处理函数中的this是否仍指向路由实例。再核对current是否截取为/about、routeMap是否存在该路径，最后检查router-view是否读取同一个路由实例；任一环节断开都会导致视图不更新。

5) 微前端团队争论用还是业务代码统一调用this.\$router.push()，你会如何划分？

答：声明式导航适合模板中明确可点击的页面入口，能让跳转意图直接体现在视图结构里。命令式this.\$router.push()更适合校验、保存或权限判断后的跳转；两者最终都要驱动地址变化，选择代价主要在控制流程和可读性。

429. vuex

Vuex是集中管理组件共享状态的方案，核心是用固定的数据流让状态变化可预测。state保存状态，getter负责派生数据，mutations同步修改状态，actions承载异步或业务逻辑并提交mutation，复杂应用再用modules拆分。其内部会让state响应式，并通过commit、dispatch找到对应处理函数。实际使用时，我只会把跨组件共享、需要统一管理的状态放进去，局部状态仍留在组件内。

集中式存储管理应用的所有组件的状态，并以相应的规则保证状态以可预测的方式发生变化

state: 状态中心 mutations: 更改状态 actions: 异步更改状态 getters: 获取状态 modules: 状态保存应用状态
将state分成多个modules，便于管理 状态- state state保存应用状态
exportdefaultnewVueX.Store({state:{counter:0},}) 状态变更- mutations

mutations用于修改状态，store.js

exportdefaultnewVueX.Store({mutations:{add(state){state.counter++}}}) 派生状态- getters

从state派生出新状态，类似计算属性 exportdefaultnewVueX.Store({getters:{doubleCounter(state){// 计算剩余数量 return state.counter * 2;}}}) 动作- actions 加业务逻辑，类似于controller

exportdefaultnewVueX.Store({actions:{add({commit}){setTimeout(()=>{})}}}) 测试代码: counter: {{store.state.counter}}async counter:

{{store.state.counter}}double:{{store.getters.doubleCounter}} vuex原理解析 实现一个插件:声明Sto

re类，挂载\$storeStore具体实现:创建响应式的state，保存mutations、actions和getters实现commit根据用户传入type执行对应mutation实现dispatch根据用户传入type执行对应action，同时传递上下文实现getters，按照getters定义对state做派生 创建响应式的state，保存mutations、actions和getters实现commit根据用户传入type执行对应mutation实现dispatch根据用户传入type执行对应action，同时传递上下文实现getters，按照getters定义对state做派生 //

目标1：实现Store类，管理state（响应式的），commit方法和dispatch方法//

目标2：封装一个插件，使用更容易使用letVue;classStore{constructor(options){// 定义响应式的state// this.\$store.state.xx// 借鸡生蛋this._vm=newVue({data:{\$\$state:options.state}})this._mutations=options.mutationsthis._actions=options.actions//

绑定this指向this.commit=this.commit.bind(this)this.dispatch=this.dispatch.bind(this)// 只读getstate(){returnthis._vm._data.\$\$state}setstate(val){console.error('不能直接赋值呀，请换别的方式！！天王盖地虎！！');};// 实现commit方法，可以修改statecommit(type,payload){// 拿出mutations中的处理函数执行它constentry=this._mutations[type]if(!entry){console.error('未知mutation类型');return}entry(this.state,payload)}dispatch(type,payload){constentry=this._actions[type]if(!entry){console.error('未知action类型');return}}//

上下文可以传递当前store实例进去即可entry(this,payload)}}functioninstall(_Vue){Vue=_Vue//混入stor...

1) 商品详情页把接口请求直接写进mutation，点击购买时组件用commit触发；评审同事认为“反正最后都要改state”，你会接受吗？

答：不会，mutation应承担同步、可预测的状态变更，异步流程和业务编排应放进action，完成后再commit。把请求塞进mutation会让变更发生时间难以判断，也破坏按规则追踪状态的链路；纯同步修改则没必要额外套一层action。

2) 后台有用户、权限、订单三个域，所有state、mutation和action都堆在一个store.js，新增成员经常提交错同名操作，你会怎样拆？

答：我会按业务域拆成modules，让每个模块管理自己的state、mutations、actions和getters，组件只依赖对应域的公开能力。拆分边界应跟业务职责一致，而不是按文件数量平均切割；模块过细会增加跳转和调用成本。

3) 购物车页要求“一键清空”立即完成，而结算页要求先调用接口、成功后再清空；两处都想复用名为clearCart的逻辑，你会怎样安排commit和dispatch？

答：本地立即清空应由mutation完成，结算流程则由action负责异步调用，成功后再commit同一个清空操作。这样状态写入入口保持明确，业务流程也能复用；若接口失败仍提前提交，就需要额外回滚并可能造成界面与服务端不一致。

4) 线上点击“异步加一”后计数始终不变，控制台提示未知action类型，但同步按钮正常；你会沿着Vuex的哪条执行链排查？

答：先核对dispatch的type是否存在于已注册的actions，再确认该action是否通过上下文调用了正确的commit。Vuex的dispatch只负责找到并执行动作，真正修改state仍要落到mutation；模块注册或命名不一致也会在入口处中断。

5) 团队准备写一个轻量状态库，要求支持响应式state、同步提交和异步动作，但暂时不要模块系统；最小实现需要保留哪些机制？

答：至少要用Vue的响应式能力承载state，保存操作表，并实现按type查找的commit与dispatch，同时把实例注入组件成为\$store。还要绑定方法的this，并拒绝整体替换只读state；省略getters后就不具备统一派生状态能力。

430. vue3带来的新特性/亮点

Vue 3的亮点可以归为响应式升级、代码组织改善和编译运行性能优化。响应式从Object.defineProperty换成Proxy，代理对象本身；Composition API让同一业务逻辑聚合在一起，也更利于复用和类型推导。编译阶段会标记动态节点，使虚拟DOM的更新成本更多取决于动态内容，并配合Tree-shaking减少未使用代码。组件层面还加入了Fragment、Teleport和Suspense，分别覆盖多根节点、跨容器渲染和异步等待场景。

1. 压缩包体积更小 当前最小化并被压缩的 Vue 运行时大小约为 20kB（2.6.10 版为 22.8kB）。Vue 3.0捆绑包的大小大约会减少一半，即只有10kB！

2. Object.defineProperty -> Proxy Object.defineProperty是一个相对比较昂贵的操作，因为它直接操作对象的属性，颗粒度比较小。将它替换为es6的Proxy，在目标对象之上架了一层拦截，代理的是对象而不是对象的属性。这样可以将原本对对象属性的操作变为对整个对象的操作，颗粒度变大。javascript引擎在解析的时候希望对象的结构越稳定越好，如果对象一直在变，可优化性降低，proxy不需要对原始对象做太多操作。

3. Virtual DOM 重构 vdom的本质是一个抽象层，用javascript描述界面渲染成什么样子。react用jsx，没办法检测出可以优化的动态代码，所以做时间分片，vue中足够快的话可以不用时间分片

传统vdom的性能瓶颈：虽然 Vue 能够保证触发更新的组件最小化，但在单个组件内部依然需要遍历该组件的整个 vdom 树。传统 vdom 的性能跟模版大小正相关，跟动态节点的数量无关。在一些组件整个模版内只有少量动态节点的情况下，这些遍历都是性能的浪费。JSX 和手写的 render function 是完全动态的，过度的灵活性导致运行时可以用于优化的信息不足那为什么不直接抛弃vdom呢？高级场景下手写render function获得更强的表达力生成的代码更简洁兼容2.x

传统vdom的性能瓶颈：虽然 Vue 能够保证触发更新的组件最小化，但在单个组件内部依然需要遍历该组件的整个 vdom 树。传统 vdom 的性能跟模版大小正相关，跟动态节点的数量无关。在一些组件整个模版内只有少量动态节点的情况下，这些遍历都是性能的浪费。JSX 和手写的 render function 是完全动态的，过度的灵活性导致运行时可以用于优化的信息不足 那为什么不直接抛弃vdom呢？高级场景下手写render function获得更强的表达力生成的代码更简洁兼容2.x

vue的特点是底层为Virtual DOM，上层包含有大量静态信息的模版。为了兼容手写 render function，最大化利用模版静态信息，vue3.0采用了动静结合的解决方案，将vdom的操作颗粒度变小，每次触发更新不再以组件为单位进行遍历，主要更改如下 将模版基于动态节点指令切割为嵌套的区块每个区块内部的节点结构是固定的每个区块只需要以一个Array追踪自身包含的动态节点

vue3.0将 vdom 更新性能由与模版整体大小相关提升为与动态内容的数量相关

Vue 3.0 动静结合的 Dom diff Vue3.0提出动静结合的 DOM diff 思想，动静结合的 DOM diff其实是在预编译阶段进行了优化。之所以能够做到预编译优化，是因为 Vue core 可以静态分析 template，在解析模版时，整个 parse 的过程是利用正则表达式顺序解析模板，当解析到开始标签、闭合标签和文本的时候都会分别执行对应的回调函数，来达到构造 AST 树的目的。借助预编译过程，Vue 可以做到的预编译优化就很强大了。比如在预编译时标记出模版中可能变化的组件节点，再次进行渲染前 diff

时就可以跳过“永远不会变化的节点”，而只需要对比“可能会变化的动态节点”。这也就是动静结合的 DOM diff 将 diff 成本与模版大小正相关优化到与动态节点正相关的理论依据。

4. Performance vue3在性能方面比vue2快了2倍。

重写了虚拟DOM的实现运行时编译update性能提高SSR速度提高

5. Tree-shaking support vue3中的核心api都支持了tree-shaking，这些api都是通过包引入的方式而不是直接在实例化时就注入，只会对使用到的功能或特性进行打包（按需打包），这意味着更多的功能和更小的体积。

6. Composition API vue2中，我们一般会采用mixin来复用逻辑代码，用倒是挺好用的，不过也存在一些问题：例如代码来源不清晰、方法属性等...

1) 一个展示页只有标题会变化，模板里还有上百个静态节点；同事说Vue3更新时仍必须完整比较整棵Virtual DOM，你怎么判断？

答：这个判断忽略了Vue3 的编译期优化：模板会被分析为结构稳定的区块，并追踪其中可能变化的动态节点。更新成本因此更接近动态内容数量，而非模板总体大小；手写render function或高度动态的结构无法同等利用静态信息。

2) 组件库要同时提供表单、动画和状态能力，但业务项目只用了其中一部分，构建负责人担心Vue3 功能越多包越大；你会关注什么？

答：应关注这些核心API是否以可导入、可摇树的形式使用，让构建工具只保留实际引用的功能。Vue 3 的tree-shaking支持能减少未使用能力进入产物，但最终体积仍受业务代码、第三方依赖和构建配置影响，不能只凭框架特性下结论。

3) 管理后台频繁给响应式对象新增字段，技术负责人认为Vue3 仍要像基于Object.defineProperty的方案那样逐属性处理，你会如何解释选型变化？

答：Vue3 以Proxy在对象外层建立拦截，代理对象操作，而不是预先逐个改造已有属性。它减少了对原对象属性结构的直接操作，也更适合对象结构变化的场景；代价是运行环境必须具备相应的Proxy能力，不能把它当作旧机制的无条件替换。

4) 弹窗放在有overflow: hidden和复杂z-index的卡片组件里，线上始终被裁切；产品又要求弹窗逻辑仍归卡片维护，你会用Vue3 的什么能力？

答：可用Teleport把弹窗实际渲染到卡片外的目标DOM容器，同时保留它在原组件中的声明和逻辑归属。这样能绕开祖先容器的裁切或层叠限制；必须确保目标节点存在，并继续处理焦点、遮罩和关闭等交互职责。

5) 订单页的异步子组件加载前会短暂空白，团队在“父组件手写加载状态”和Vue3 内建组件之间争论，你会怎么取舍？

答：若等待发生在异步组件渲染阶段，可用Suspense提供default与fallback内容，把加载占位集中在边界上。简单请求或需要精细控制错误、重试与局部状态时，显式状态可能更清楚；Suspense解决等待展示，不会自动替代完整的异常处理。

6) 一个大型组件把搜索、分页和鼠标坐标逻辑分散在data、computed、methods与生命周期中，维护者找不到同一业务链；Vue3 哪项变化最直接？

答：Composition API允许按业务逻辑聚合相关状态、计算、监听和生命周期，并把鼠标等逻辑提取为可复用组合函数。它还能改善来源识别与类型推导；简单组件继续使用Options API也成立，强行组合反而可能增加抽象层次。

431. Composition api

Composition API是按业务功能组织和复用组件逻辑的一套组合式接口。Options API会把同一功能拆到data、computed、methods等区域，组件变大后阅读和维护都更费劲；组合式写法则能把相关状态与操作放在一起。相比mixin，它通过显式调用组合函数返回数据，来源更清楚，也降低了属性命名冲突和类型推断困难。它基于Vue响应式系统自动收集依赖，调用也不依赖固定顺序，但中小型、逻辑简单的组件并不一定必须改写。Composition API也叫组合式API，是Vue3.x的新特性。通过创建Vue 组件，我们可以将接口的可重复部分及其功能提取到可重用的代码段中。仅此一项就可以使我们的应用程序在可维护性和灵活性方面走得更远。然而，我们的经验已经证明，光靠这一点可能是不够的，尤其是当你的应用程序变得非常大的时候——想想几百个组件。在处理如此大的应用程序时，共享和重用代码变得尤为重要。Vue2.0中，随着功能的增加，组件变得越来越复杂，越来越难维护，而难以维护的根本原因是Vue的API设计迫使开发者使用watch，computed，methods选项组织代码，而不是实际的业务逻辑。另外Vue2.0缺少一种较为简洁的低成本的机制来完成逻辑复用，虽然可以mixin完成逻辑复用，但是当mixin变多的时候，会使得难以找到对应的data、computed或者method来源于哪个mixin，使得类型推断难以进行。所以Composition API的出现，主要是为了解决Options API带来的问题，第一个是代码组织问题，Composition API可

以让开发者根据业务逻辑组织自己的代码，让代码具备更好的可读性和可扩展性，也就是说当下一个开发者接触这一段不是他自己写的代码时，他可以更好的利用代码的组织反推出实际的业务逻辑，或者根据业务逻辑更好的理解代码。第二个是实现代码的逻辑提取与复用，当然mixin也可以实现逻辑提取与复用，但是像前面所说的，多个mixin作用在同一个组件时，很难看出property是来源于哪个mixin，来源不清楚，另外，多个mixin的property存在变量命名冲突的风险。而Composition API刚好解决了这两个问题。通俗的讲：没有Composition API之前vue相关业务的代码需要配置到option的特定的区域，中小型项目是没有问题的，但是在大型项目中会导致后期的维护性比较复杂，同时代码可复用性不高。Vue3.x中的composition-api就是为了解决这个问题而生的。composition api提供了以下几个函数：setup、ref、reactive、watch、effect、computed、toRefs、生命周期钩子。都说Composition API与React Hook很像，说说区别。从React Hook的实现角度看，React Hook是根据useState调用的顺序来确定下一次重渲染时的state是来源于哪个useState，所以出现了以下限制：不能在循环、条件、嵌套函数中调用Hook，必须确保总是在你的React函数的顶层调用Hook。useEffect、useMemo等函数必须手动确定依赖关系。而Composition API是基于Vue的响应式系统实现的，与React Hook的相比，声明在setup函数内，一次组件实例化只调用一次setup，而React Hook每次重渲染都需要调用Hook，使得React的GC比Vue更有压力，性能也相对于Vue来说也较慢。Composition API的调用不需要顾虑调用顺序，也可以在循环、条件、嵌套函数中使用响应式系统自动实现了依赖收集，进而组件的部分的性能优化由Vue内部自己完成，而React Hook需要手动传入依赖，而且必须保证依赖的顺序，让useEffect、useMemo等函数正确的捕获依赖变量，否则会由于依赖不正确使得组件性能下降。虽然Composition API看起来比React Hook好用，但是其设计思想也是借鉴React Hook的。

1) 一个表格组件用了五个mixin，模板里的load和rows找不到来源，另一个mixin又声明了同名属性；你会怎样重构？

答：应把各业务能力提取为显式调用的组合函数，例如由useTable返回rows和load，组件能直接看出来来源。需要重命名时可在解构或返回结构上主动处理；组合函数过度拆碎也会让调用关系分散，因此边界应围绕完整业务逻辑。

2) 搜索页的筛选、分页、请求取消逻辑分别散在watch、computed、methods和卸载钩子里，新成员改一次要跳四处；setup中你会怎么组织？

答：我会让同一搜索流程的ref、computed、watch和生命周期钩子相邻放置，再把稳定可复用的部分提取成useSearch。这种组织方式按业务关系而非选项类型聚合代码；若组合函数同时承担多个领域，仍会退化成难维护的大函数。

3) 权限组合函数只在管理员分支中需要，评审者套用React规则，禁止在条件分支调用它；在Vue的Composition API下这个限制成立吗？

答：不应机械套用React Hook的调用顺序规则，Vue的setup对每个组件实例执行一次，组合式调用不靠固定序号匹配状态。它可以出现在条件、循环或嵌套函数中；但分支调用产生的生命周期与资源清理仍要清晰，否则工程上依然容易遗漏。

4) 线上筛选条件变化后副作用没有执行，同事刚把相关读取移出watchEffect，却认为响应式系统会自动追踪文件里所有变量；你会怎样排查？

答：先确认响应式值是否在该副作用执行期间被实际读取，因为依赖收集来自运行中的访问关系，而不是静态扫描整个文件。再检查值是否仍由ref或reactive持有；脱离响应式容器或不在执行链中读取，都不会建立预期依赖。

5) 团队要在watch和watchEffect间选型：风控页必须明确监听账户和金额，并记录变更前值；哪一个更合适？

答：这里更适合watch，因为依赖目标需要显式表达，并且业务要使用新旧值进行审计或判断。watchEffect更适合让系统自动收集副作用中读取的依赖；自动收集虽简洁，但依赖边界不够显眼，复杂风控逻辑会更难审查。

6) React开发者说组合式函数就是换名后的Hook，并准备给每个副作用手写依赖数组；你会指出哪些实现差异？

答：Vue的组合式能力建立在响应式依赖收集上，watchEffect等机制可根据实际读取自动建立依赖，并不依靠Hook调用顺序。setup通常按组件实例执行一次，而ReactHook随重渲染再次调用；两者思想相近，但不能照搬依赖数组和顶层调用约束。

432. computed 的实现原理

computed本质上是带缓存的惰性观察者，通过dirty标记决定是否重新求值。首次读取时它会执行计算并缓存结果；依赖变化后，若当前没有订阅者，通常只把dirty设为true，等下次读取再计算。存在订阅者时，它会重新计算并比较新旧结果，最终值确实变化才通知渲染观察者，从而减少无效更新。因此，同一派生结果会被多次读取时适合用computed，单纯执行动作则不必依赖这层缓存。

computed本质是一个惰性求值的观察者computed

watcher。其内部通过this.dirty属性标记计算属性是否需要重新求值。当computed的依赖状态发生改变时，就会通知这个惰性的 watcher，computed watcher通过this.dep.subs.length判断有没有订阅者，有的话，会重新计算，然后对比新旧值，如果变化了，会重新渲染。(Vue

想确保不仅仅是计算属性依赖的值发生变化，而是当计算属性最终计算的值发生变化时才会触发渲染

watcher重新渲染，本质上是一种优化。)没有的话，仅仅把this.dirty = true(当计算属性依赖于其他数据时，属性并不会立即重新计算，只有之后其他地方需要读取属性的时候，它才会真正计算，即具备lazy（懒计算）特性。)

1) 报表页把totalPrice写成普通方法，模板在十几个位置调用，任意按钮触发重渲染都会重新遍历订单；改成computed为什么有意义？

答：computed由惰性的computed watcher管理，依赖未变化时可复用已有结果，不必在每次渲染中重复执行遍历。依赖变化后它才进入需要重新求值的状态；若计算本身极轻且只调用一次，缓存带来的收益可能并不明显。

2) 商品列表有一个昂贵的派生筛选值，但折叠面板关闭时没有任何模板或其他逻辑读取它；源数据更新后会马上重新计算吗？

答：在没有订阅者时，依赖变化通常只会把计算观察者的dirty标记设为true，不会立刻执行筛选。等面板展开并再次读取该值时才真正求值，这就是惰性计算；若其他观察者仍订阅它，就不能假设始终无人触发。

3) 一个computed同时依赖价格和折扣，价格变化后被连续读取三次；面试现场让你描述dirty从变化到复用的过程，你会怎么说？

答：依赖变化会通知对应的情性观察者，使其进入需要求值的状态；首次读取时重新执行计算并产出结果，随后读取可继续使用缓存。直到相关依赖再次变化前，不应重复计算；前提是这些依赖确实在求值过程中被响应式系统收集。

4) 线上日志显示源字段频繁变化，但依赖某个computed的视图并非每次都重渲染；你会先判断这是故障还是优化？

答：先比较计算属性最终的新旧值，而不是只看底层依赖是否变化；最终值未变时，不触发渲染观察者属于预期优化。若最终值已变化仍不更新，再检查依赖是否具有响应性、计算属性是否被订阅，以及读取链路是否真正建立。

5) 库存页需要在数量变化后写日志并请求补货，同事想把这些副作用塞进computed以利用缓存；你会如何取舍？

答：不应把日志和请求作为计算属性的核心职责，computed更适合根据响应式状态产生可缓存的派生值。副作用应放到侦听器或明确的业务动作中，以便控制触发和失败处理；否则惰性读取会让请求时机取决于“是否有人访问”。

433. watch 的理解

watch本质上是观察数据变化，并在变化后执行回调，它本身没有缓存能力。当监听对象内部的属性时，可以开启deep: true，让对象中的每一项都被监听。深度监听覆盖得更全面，但也会带来额外的性能开销。明确知道要观察哪个属性时，我一般会使用字符串形式监听，缩小监听范围。watch没有缓存性，更多的是观察的作用，可以监听某些数据执行回调。当我们需要深度监听对象中的属性时，可以打开deep: true选项，这样便会对对象中的每一项进行监听。这样会带来性能问题，优化的话可以使用字符串形式监听
注意：Watcher：观察者对象，实例分为渲染 watcher(render watcher),计算属性 watcher(computed watcher),侦听器 watcher (user watcher) 三种

1) 个人资料页把完整用户对象做成computed，只为了姓名变化时调用埋点；同事认为computed和watch都能依赖收集，你会接受吗？

答：不接受，埋点属于数据变化后的副作用，应使用watch执行回调，而computed应保留给派生值和缓存。混用会让读取动作夹带外部行为，触发时机也更难判断；若只是展示拼接姓名，才适合计算属性。

2) 配置中心监听一个包含大量嵌套字段的表单，任意字段改动都触发自动保存；直接设置deep: true有什么工程风险？

答：深度监听会遍历对象中的各项以建立监听，结构越大，初始化与变更处理的负担越值得警惕。更稳妥的是只监听确实影响保存的字段，或拆成多个边界明确的对象；若需求就是捕获任意深层修改，精确监听会增加维护清单。

3) 订单页原来只监听status，产品临时要求地址任一层级变化也重新校验；你会继续写单字段监听，还是改成深度监听？

答：若地址结构稳定且只有少数字段参与校验，优先精确监听这些路径，避免无关字段也触发回调。只有规则明确要求捕获整个地址对象的任意深层变化时才考虑deep: true；范围扩大后必须评估回调频率和对象规模。

4) 线上修改profile.contact.phone后校验没有执行，但替换整个profile对象时会执行；当前只监听了对象本身，你会怎么定位？

答：先确认侦听器是否只观察了对象引用，因为深层属性变化不等于引用被替换。随后核对目标字段路径，并选择精确的字符串路径监听或按需求开启deep；开启深度监听虽能覆盖该变化，也可能把无关字段纳入回调。

5) 审计模块既要计算账户风险等级，又要在等级变化时发送告警；架构师想全部交给一个侦听器，你会怎样拆分？

答：风险等级应由computed watcher负责派生和缓存，告警则由user watcher观察等级变化后执行回调。组件渲染仍由render watcher响应最终展示变化；拆分能区分计算、用户副作用和渲染职责，但要避免告警回调反向修改依赖造成循环。

434. vue 渲染过程

Vue的渲染过程是先把模板编译成render函数，再生成VNode，最后通过patch更新真实DOM。编译时，parse把模板转成AST，optimize标记静态节点，generate再生成render函数字符串。数据变化会触发渲染Watcher，重新执行render得到新的VNode。随后通过DOM

diff对比新旧节点，只对真实DOM做必要的添加、修改或删除。调用compile函数生成render函数字符串，编译过程如下:parse

使用大量的正则表达式对template字符串进行解析，将标签、指令、属性等转化为抽象语法树AST。模板 -> AST （最消耗性能）optimize 遍历AST，找到其中的一些静态节点并进行标记，方便在页面重渲染的时候进行diff比较时，直接跳过这一些静态节点，优化runtime的性能generate 将最终的AST转化为render函数字符串调用new

Watcher函数,监听数据的变化,当数据发生变化时，Render 函数执行生成 vnode 对象调用patch方法,对比新旧 vnode 对象,通过 DOM diff 算法,添加、修改、删除真正的 DOM 元素 parse

使用大量的正则表达式对template字符串进行解析，将标签、指令、属性等转化为抽象语法树AST。模板 -> AST （最消耗性能）optimize 遍历AST，找到其中的一些静态节点并进行标记，方便在页面重渲染的时候进行diff比较时，直接跳过这一些静态节点，优化runtime的性能generate 将最终的AST转化为render函数字符串

1) 商品详情页只修改购物车数量，为什么不能理解成Vue会重新创建整棵真实DOM？

答：数据变化会触发对应的Watcher，重新执行render生成新的VNode，但不会直接重建整棵真实DOM。随后patch对比新旧VNode，只对需要变化的真实节点执行添加、修改或删除；若节点结构或标识发生变化，复用范围才会缩小。

2) 后台表格有 3000

行，筛选条件每输入一个字符就明显卡顿，你会把耗时拆到渲染链路的哪些阶段？

答：先区分数据变化后执行render的脚本耗时，以及patch后真实DOM更新带来的布局和绘制耗时。若生成大量VNode已经很慢，应减少参与渲染的数据；若主要慢在真实节点更新，则应分页或缩小一次更新的节点规模。

3) 运营平台允许用户在线修改template，而普通业务组件在构建时已有render，两类组件的渲染链路有什么关键差异？

答：在线模板必须先经过parse、optimize和generate，得到render后才能进入创建VNode与patch的流程。构建阶段已经生成render的组件可跳过运行时编译；动态模板若频繁变化，编译成本会重新出现，不能只优化后续diff。

4) 线上页面数据已更新，控制台也能看到新值，但视图保持旧内容，你会沿着哪条链路定位断点？

答：先确认变化是否触发负责渲染的Watcher，再检查render是否生成了符合预期的新VNode，最后观察patch是否更新了真实DOM。若新旧VNode本身没有差异，应回查渲染依赖；若差异存在但节点未变，再检查节点匹配与补丁过程。

5) 内容团队要求首屏模板高度动态，性能团队要求彻底移除运行时编译，你会如何划分方案边界？

答：固定结构的组件应在构建阶段完成模板编译，运行时直接执行render，避免承担最耗性能的模板到AST转换。真正由用户配置结构的区域才保留动态编译，并限制其范围和触发次数；完全动态与完全预编译无法同时满足同一块模板。

435. 说一说keep-alive实现原理

keep-alive的核心是缓存组件对应的VNode，再次命中时直接从缓存中取出。它通过include和exclude按组件的name控制哪些需要缓存，参数可以是字符串、正则或数组。max用来限制缓存数量，超过上限后会删除最先进入的缓存项。这个淘汰过程采用LRU思路，让最近访问过的内容更有机会继续保留。

keep-alive组件接受三个属性参数：include、exclude、max

include指定需要缓存的组件name集合，参数格式支持String, RegExp, Array。当为字符串的时候，多个组件名称以逗号隔开。exclude指定不需要缓存的组件name集合，参数格式和include一样。max指定最多可缓存组件的数量,超过数量删除第一个。参数格式支持String、Number。原理

keep-alive实例会缓存对应组件的VNode,如果命中缓存，直接从缓存对象返回对应VNode LRU (Least recently used) 算法根据数据的历史访问记录来进行淘汰数据，其核心思想是“如果数据最近被访问过，那么将来被访问的几率也更高”。(墨菲定律：越担心的事情越会发生)

1) 订单列表被keep-alive包裹后，从详情页返回仍保留筛选条件，这是否说明它把页面真实DOM截图后直接恢复了？

答：不是，keep-alive的核心是缓存对应组件的VNode，再次命中时从缓存对象返回该VNode。保留页面状态来自组件实例及其虚拟节点被复用，而不是保存一份页面截图；未命中缓存时仍需按正常组件流程创建。

2) 移动端有商品、订单、消息三类列表，只希望订单页返回时保留状态，你会怎样配置include和组件名称？

答：给订单页设置稳定的组件name，并让include只匹配该名称，商品页和消息页便不会进入缓存。include支持String、RegExp或Array，字符串包含多个名称时用逗号分隔；名称不匹配会导致看似配置成功却始终无法命中。

3) 路由扩展到 20

个详情页，产品要求全部缓存，客户端负责人又要求限制缓存数量，你会怎么处理max？

答：设置max限制缓存组件数量，超出后按最近使用情况淘汰长期未访问的缓存项。max支持String或Number，但它只能控制数量，不能判断业务重要性；关键页面仍应结合include、exclude明确缓存范围。

4) 线上反馈用户返回列表时状态偶尔丢失，但同一路由有时又能恢复，你会优先核对哪些缓存条件？

答：先核对组件name是否稳定且确实命中include，再检查是否被exclude排除或因超过max而被淘汰。随后确认访问顺序，因为LRU会改变淘汰对象；若组件从未命中缓存，问题不在恢复阶段，而在缓存筛选条件。

5) 中后台有 30

个路由，团队争论是全部套keep-alive，还是只缓存高频表单页，你会支持哪种方案？

答：应只缓存确实需要保留编辑状态或页面上下文的组件，并用include或exclude明确名单，同时设置合理的max。全量缓存会让缓存集合持续扩大并触发淘汰，命中结果更依赖访问历史；选择性缓存更可控，但未缓存页面需要自行恢复状态。

436. 为什么访问data属性不需要带data

访问data属性时不需要再写data，是因为Vue把这些属性代理到了实例上。简单来说，读取this.xxx时，代理的get会转发到内部对象对应的属性；给this.xxx赋值时，set也会把新值写回去。这个代理通过Object.defineProperty实现，并将属性设置为可枚举、可配置，因此模板和实例代码都能用更简洁的方式访问数据。vue中访问属性代理this.data.xxx转换this.xxx的实现 /** 将 某一个对象的属性 访问 映射到对象的某一个属性成员上 */function proxy(target, prop, key){Object.defineProperty(target, key, {enum


```
erable:true,configurable:true,get(){return target[prop][key];},set(newVal){target[prop][key]=newVal;}};}
```

1) 组件里写了data: { count: 1

}，模板和方法都能访问this.count，这是否意味着count被复制到实例上了？

答：不是复制，而是在实例的count属性上通过Object.defineProperty建立访问代理。读取this.count时，get返回this.data.count；写入时，set再把新值交给对应的数据属性，因此两处不会维护两份独立状态。

2) 表单组件有 80

个字段，团队想手写this.data.xxx来减少一层代理，你会如何评价这个改动？

答：现有代理让模板和实例方法统一使用this.xxx，手写完整路径会改变组件的访问约定，却不会改变底层数据仍位于data的事实。是否值得改动应以实际测量为依据；仅凭多一次get转发，无法断言它就是表单卡顿来源。

3) 基础组件实例上已经存在status，业务又在data中声明同名字段并希望通过this.status访问，代理实现需要满足什么约束？

答：代理要求在目标实例上为该key定义可配置的访问属性，因此同名成员会产生覆盖或定义冲突风险。初始化阶段应检测并避免名称碰撞，必要时重命名业务字段；否则this.status指向哪一层会变得不可靠。

4) 线上出现this.total = 20后this.data.total仍是旧值，你会怎样验证代理是否被破坏？

答：先用属性描述符检查实例total是否仍具有代理的get和set，再确认它们是否指向预期的data.total。若该属性后来被重新定义，赋值就可能不再转发；若代理完整，则继续核对实际代理容器和字段名是否一致。

5) 代码评审中有人建议用一次性赋值this.count =

this.data.count代替访问器代理，两种方案的行为差异是什么？

答：一次性赋值只同步初始化时的值，之后修改实例属性或data.count都不会自动反映到另一处。访问器代理则让每次读写都落到同一个底层字段；代价是实例属性必须保留这组get、set，不能随意改成普通值属性。

437. template预编译是什么

模板预编译，就是在项目构建阶段把template转换成render function。如果留到运行时处理，组件实例化时需要先完成一次模板编译，之后才能执行渲染，这部分工作会带来运行时损耗。提前编译后，组件运行时可以直接使用生成的渲染函数，从而跳过模板编译。它优化的是组件启动和渲染前的准备过程，并不是让模板在运行时反复编译。对于Vue组件来说，模板编译只会在组件实例化的时候编译一次，生成渲染函数之后在也不会进行编译。因此，编译对组件的 runtime 是一种性能损耗。

而模板编译的目的仅仅是将template转化为render

function，这个过程，正好可以在项目构建的过程中完成，这样可以让实际组件在 runtime 时直接跳过模板渲染，进而提升性能，这个在项目构建的编译template的过程，就是预编译。

1) 一个组件只实例化一次，负责人因此认为运行时编译template没有成本，你会怎样反驳？

答：模板即使只在组件实例化时编译一次，也必须完成从template到render function的转换，这仍是一次运行时性能损耗。预编译把这段工作移到项目构建过程，使实际组件运行时直接执行渲染函数；实例少只能降低影响，不能让成本消失。

2) 营销页有 40 个固定组件模板，构建负责人问是否值得做预编译，你会给出什么落地判断？

答：这些模板在发布前已经确定，适合在构建阶段统一转换为render function，运行时无需再次处理模板。落地时应确认产物包含可直接执行的渲染函数；若仍把原始template留到浏览器处理，就没有获得预编译的核心收益。

3) 低代码平台的模板由运营发布后即时生效，架构组仍要求所有模板构建期预编译，这个约束为何无法直接满足？

答：构建期预编译只能处理构建时已经存在的模板，而运营发布的模板在运行时才确定，无法被先前构建产物完整覆盖。可把固定外壳预编译，将动态区域限制在必要范围；若动态结构也必须即时变化，就仍需承担相应的模板转换成本。

4) 线上首开页面卡顿，火焰图显示组件实例化阶段出现明显模板编译工作，你会检查构建链路的什么结果？

答：先检查发布产物中的组件是否已经携带render function，以及运行时是否仍接收到原始template。若模板没有在构建阶段转换，实例化时就会再次承担编译；若已有渲染函数，则应继续排查render和后续真实节点更新，而非归因于预编译。

5) 插件系统在“构建期预编译”和“运行时接收模板”之间选型，产品强调灵活性，性能团队强调启动速度，你会怎样定边界？

答：核心页面和稳定插件应优先预编译，让运行时直接进入渲染流程；必须动态下发结构的插件才保留运行时模板转换。前者牺牲发布后的即时结构变化，后者增加实例化阶段的编译工作，边界应由模板是否能在构建时确定来划分。

438. 介绍一下Vue中的Diff算法

Vue的Diff算法用于比较新旧虚拟DOM，找出同层节点中真正需要更新的部分。它先判断节点是否相同，不同就删除旧节点并创建新节点，相同则通过patchVnode继续处理子节点。双方都有多个子节点时，才进入updateChildren做核心比较，匹配到相同子节点后再递归处理。它不做跨层级比较，因此把原本可能达到 $O(n^3)$ 的复杂度降低到了 $O(n)$ 。在新老虚拟DOM对比时 首先，对比节点本身，判断是否为同一节点，如果不为相同节点，则删除该节点重新创建节点进行替换如果为相同节点，进行patchVnode，判断如何对该节点的子节点进行处理，先判断一方有子节点一方没有子节点的情况(如果新的children没有子节点，将旧的子节点移除)比较如果都有子节点，则进行updateChildren，判断如何对这些新老节点的子节点进行操作(diff核心)。匹配时，找到相同的子节点，递归比较子节点 在diff中，只对同层的子节点进行比较，放弃跨级的节点比较，使得时间复杂从 $O(n^3)$ 降低值 $O(n)$ ，也就是说，只有当新旧children都为多个子节点时才需要用核心的Diff算法进行同层级比较。

1) 资讯列表把一个卡片从第三层容器移到相邻栏目，内容完全相同，为什么不能期待diff跨层找到并直接复用？

答：该算法只比较同层子节点，主动放弃跨级匹配，因此相同内容移动到不同层级时不会按跨层节点处理。旧位置可能被删除，新位置再创建对应节点；这项约束换来了线性的同层比较范围，代价是跨层重排无法直接复用。

2) 商品列表更新 1000

条数据时，只有价格字段变化，你会如何解释从根节点到真实DOM的处理路径？

答：先比较新旧虚拟节点本身，若判定为同一节点，就进入patchVnode处理其子节点。双方都有多个子节点时进入updateChildren，找到相同子节点后递归比较，最终只把确定的差异落实到真实DOM；若节点不相同，则会删除并重新创建。

3) 空状态页从“无订单”切换为包含 20 条订单的列表，和两个列表各有 20 条数据时，是否都会进入核心updateChildren？

答：不会，只有新旧双方都具有多个子节点时，才需要进入核心的同层子节点比较。由无子节点变为有子节点时可直接添加，新的一方没有子节点时则移除旧子节点；因此应先判断子节点存在形态，再决定是否执行列表diff。

4) 线上更新一条记录后整行输入状态丢失，你怀疑节点被替换，会沿着diff的哪个判断排查？

答：先检查新旧行节点是否被判定为同一节点，因为一旦不相同，旧节点会被删除并创建新节点，内部状态也无法沿用。若节点相同，再进入patchVnode检查子节点处理；排查重点是生成的虚拟节点结构与身份条件，而不只是数据值。

5) 设计器团队希望支持任意节点跨层拖拽，同时要求每次更新都保持现有diff的低复杂度，你会如何说明取舍？

答：现有算法通过只比较同层子节点，把原本可能达到 $O(n^3)$ 的一般树比较收敛到 $O(n)$ 范围。任意跨层匹配会扩大搜索空间，不能直接沿用这一复杂度承诺；可把拖拽结果视为旧层删除、新层创建，或由业务层额外维护映射。

6) 代码评审时有人把patchVnode和updateChildren当成同一层逻辑，你会用“新节点没有子节点”的场景怎样区分？

答：patchVnode负责同一节点内部该如何更新，其中先判断新旧双方是否存在子节点。只有双方都有多个子节点时才调用updateChildren做核心同层比较；若新节点没有子节点，直接移除旧子节点即可，无需进入列表匹配。

439. 说说Vue2.0和Vue3.0有什么区别

Vue 3相比Vue 2，核心变化集中在响应式、代码组织和编译运行效率上。响应式从Object.defineProperty改为Proxy，可以直接监听数组变化和属性增删，也不必预先遍历对象的每个属性。它新增Composition API，让相关逻辑更容易组织和复用；同时重构虚拟DOM，对静态节点、slot和内联事件做编译优化。代码结构也更利于Tree shaking，并使用TypeScript替代Flow。 重构响应式系统，使用Proxy替换Object.defineProperty，使用Proxy优势：可直接监听数组类型的数据变化监听的目标为对象本身，不需要像Object.defineProperty一样遍历每个属性，有一定的性能提升可拦截apply、ownKeys、has等13种方法，而Object.defineProperty不行直接实现对象属性的新增/删除新增Composition API，更好的逻辑复用和代码组织重构Virtual DOM模板编译时的优化，将一些静态节点编译成常量slot优化，将slot编译为lazy函数，将slot的渲染的决定权交给子组件模板中内联事件的提取并重用（原本每次渲染都重新生成内联函数）代码结构调整，更便于Tree shaking，使得体积更小使用Typescript替换Flow 可直接监听数组类型的数据变化监听的目标为对象本身，不需要像Object.defineProperty一样遍历每个属性，有一定的性能提升可拦截apply、ownKeys、has等13种方法，而Object.defineProperty不行直接实现对象属性的新增/删除 模板编译时的优化，将一些静态节点编译成常量slot优化，将slot编译为lazy函数，将slot的渲染的决定权交给子组件模板中内联事件的提取并重用（原本每次渲染都重新生成内联函数）

1) 商品编辑页给一个已有响应式对象动态增加discount字段，Vue 2页面不更新，迁到Vue 3后却正常了；你怎么解释这种差异，又会提醒团队别得出什么错误结论？

答：差异来自响应式实现：Vue 2的Object.defineProperty主要在初始化时逐项劫持属性，后续新增或删除属性不能被直接感知；Vue 3的Proxy代理对象本身，可以拦截这类操作。不能因此认定所有更新都会更快，组件划分、依赖数量和渲染成本仍会决定实际表现。

2) 一个Vue 2后台有几十种动态表单，字段和数组项频繁增删；如果准备迁移到Vue 3，你会优先验证哪些响应式代码，而不是只确认页面能启动？

答：应优先检查依赖Vue.set、Vue.delete、数组变更补丁以及深层对象初始化的代码，因为迁移后这些路径会由Proxy直接跟踪。再用新增字段、删除字段和数组增删等真实操作做回归，确认计算属性

与视图同步；响应式改善并不能替代业务状态和表单边界测试。

3) 一个多人维护的复杂业务组件在Vue 2中把请求、筛选、分页都塞进mixins，升级时有人主张只换响应式实现、不引入Composition API；你会如何取舍？

答：只完成兼容迁移是更低风险的路径，Composition API并不是升级后必须立即采用。若组件长期存在逻辑分散、同名覆盖或复用困难，可按业务能力抽成组合函数并逐段验证；一次性同时改框架、状态结构和组织方式，会扩大回归范围，也更难定位问题。

4) 迁移后列表数据没变，但性能面板显示重复创建了大量模板内联事件函数；你会从Vue 3的编译与虚拟DOM优化角度检查什么？

答：先确认生产构建确实使用了对应的Vue 3模板编译器，并检查模板是否让静态节点、内联事件和slot获得编译期优化。Vue 3可提升静态内容、复用内联事件并将slot编译为惰性函数；若大量内容本身是动态的，这些优化空间会自然缩小。

5) 团队争论升级收益时，一方只谈Proxy，另一方只谈包体积；你会怎样给出更完整的技术判断？

答：应把收益拆成运行时、开发组织和构建产物三类：Proxy改善对象与数组变化追踪，Composition API改善逻辑复用，编译器和虚拟DOM重构减少部分重复工作。代码结构也更利于Tree Shaking，且实现语言由Flow转向TypeScript；最终收益仍受依赖兼容和业务代码形态约束。

【React】

440. 对虚拟DOM的理解

虚拟DOM的核心价值不是比手写DOM操作更快，而是让声明式更新和视图抽象成为可能。它作为局部更新链路的一环，用更轻量的数据结构承载视图，使新旧视图能够执行Diff，避免每次都做全量更新。直接、精准地操作真实DOM通常仍可能更快，因为人为已经省掉了比较过程。工程上的取舍是接受一定抽象成本，换取state = UI的开发方式以及更好的跨平台能力。虚拟dom从来不是用来和直接操作dom对比的，它们俩最终殊途同归。虚拟dom只不过是局部更新的一个环节而已，整个环节的对比对象是全量更新。虚拟dom对于state = UI的意义是，虚拟dom使diff成为可能（理论上也可以直接用dom对象diff，但是太臃肿），促进了新的开发思想，又不至于性能太差。但是性能再好也不可能好过直接操作dom，人脑连diff都省了。还有一个很重要的意义是，对视图抽象，为跨平台助力。其实我最终希望你明白的事情只有一件：虚拟DOM的价值不在性能，而在别处。因此想要从性能角度来把握虚拟DOM的优势，无异于南辕北辙。偏偏在面试场景下，10个人里面有9个都走这条歧路，最后9个人里面自然没有一个能自圆其说，实在让人惋惜。真正理解虚拟DOM(opens new window)

1) 活动页只有一个计数器，工程师说直接写textContent肯定比虚拟DOM快，所以虚拟DOM没价值；你会怎样回应这个反例？

答：单点且更新路径已知时，直接操作真实DOM通常少了一层创建和比较，虚拟DOM并不保证更快。它的价值是让开发者用state = UI描述界面，再通过差异计算完成通用的局部更新；若把比较对象限定成一次精准手工操作，结论本身就失去了公平性。

2) 一个表格页面每次筛选都重新生成整棵虚拟节点树，但最终只改了几个单元格；这层虚拟DOM在工程上解决了什么，又没有解决什么？

答：它提供了轻量的视图描述，使新旧结果能够执行diff，再把必要变更提交到真实DOM，而不必由业务代码维护每个更新路径。它没有消除生成对象和比较树的成本，也不会自动修复不合理的全量状态更新；组件边界和状态粒度仍需工程控制。

3) 同一套业务组件要同时运行在浏览器和原生容器中，产品要求交互逻辑复用但宿主控件完全不同；虚拟DOM在这里承担哪一层职责？

答：虚拟DOM抽象的是视图结构与变化，而不是某一种宿主节点，因此同一套组件描述可以交给不同渲染器落地。浏览器渲染器操作DOM，其他平台可映射到自身视图对象；跨端仍需处理平台能力、布局 and 交互差异，不能理解为业务代码百分之百无需适配。

4) 线上长列表滚动卡顿，监控只显示主线程繁忙，团队立即把责任归到diff；你会怎样验证，而不是直接移除虚拟DOM？

答：先区分耗时发生在状态计算、虚拟树创建与比较，还是最终布局、绘制和真实DOM操作，并用性能轨迹定位长任务。虚拟DOM只是局部更新链路中的一环，卡顿也可能来自全量重渲染或节点规模过大；没有分段证据就改渲染模型，往往只会转移成本。

5) 后台搭建器需要支持大量动态条件和组件插槽，一方主张手写DOM获得极致性能，另一方坚持虚拟DOM；你会如何做选型？

答：动态结构复杂且更新路径难以人工维护时，虚拟DOM能用统一声明式模型换取可组合性和可维护性，通常更适合作为默认方案。只有热点路径稳定、性能瓶颈已被证明确实位于通用比较时，才值得局部采用更直接的更新方式；代价是手工同步状态与视图的复杂度会上升。

441. 谈谈你对React的理解

React本质上是一个组件化的视图层框架，用声明式方式把状态映射成界面。声明式让界面逻辑更直观，组件化便于拆分和复用，也更容易做到高内聚、低耦合。借助虚拟DOM，同一套组件思想还能用于Web、Native和VR等场景。它只负责视图层，大型应用仍要结合社区方案处理路由和状态管理，因此会增加选型与学习成本。

React 是一个网页 UI 框架，通过组件化的方式解决视图层开发复用的问题，本质是一个组件化框架。

它的核心设计思路有三点，分别是声明式、组件化与 通用性。声明式的优势在于直观与组合。组件化的优势在于视图的拆分与模块复用，可以更容易做到高内聚低耦合。通用性在于一次学习，随处编写。比如 React Native，React 360 等，这里主要靠虚拟 DOM 来保证实现。这使得 React 的适用范围变得足够广，无论是 Web、Native、VR，甚至 Shell 应用都可以进行开发。这也是 React 的优势。但作为一个视图层的框架，React 的劣势也十分明显。它并没有提供完整的一揽子解决方案，在开发大型前端应用时，需要向社区寻找并整合解决方案。虽然一定程度上促进了社区的繁荣，但也为开发者在技术选型和学习适用上造成了一定的成本。承接在优势后，可以再谈一下自己对于 React 优化的看法、对虚拟 DOM 的看法

1) 一个静态营销页只有几处按钮交互，候选人说用了React就一定比原生实现更先进；作为技术负责人你会怎么判断？

答：React的优势不在于天然更先进，而在于用声明式和组件化组织会持续变化、需要复用的视图。若页面结构简单且长期稳定，引入组件运行时与配套工具未必带来足够收益；选型应看状态复杂度、复用需求和团队维护成本，而不是框架标签。

2) 订单后台把筛选栏、表格和详情弹窗拆成二十多个组件，但改一个字段仍要跨层修改；为什么“拆成组件”没有自动实现高内聚低耦合？

答：组件化只提供拆分与组合机制，边界若按标签数量而非业务职责划分，依赖仍会散落在多层。应把相关状态、行为和视图收拢到稳定能力中，并明确输入输出；拆得过细会增加数据传递和理解成本

，拆得过粗又会削弱复用与隔离。

3) 同一支团队要把Web端的内容流迁到原生端，管理层认为React Native意味着整套页面直接复制；你会如何校正预期？

答：可复用的核心是声明式组件思维和部分业务逻辑，虚拟DOM所代表的视图抽象让不同渲染器成为可能。浏览器DOM与原生视图的能力、样式和交互并不相同，因此宿主相关代码仍要适配；“一次学习，随处编写”不等于“一份界面代码，所有平台无差别运行”。

4) 大型React项目上线前，路由、数据请求和状态管理分别来自不同社区库，升级时出现版本冲突；这是偶发管理失误，还是框架定位带来的成本？

答：这与React主要聚焦视图层的定位直接相关，它没有为大型应用提供完整的一揽子方案。社区生态带来选择空间，也把选型、集成和升级责任交给团队；应通过统一技术基线、封装边界和升级策略控制成本，但无法把这部分复杂度完全消除。

5) 架构评审中，一方看重声明式与跨端能力，另一方担心生态拼装成本；什么项目条件下你仍会选择React？

答：当界面状态复杂、组件复用价值高，或团队需要在多种渲染目标间共享开发模型时，React的声明式、组件化和通用性更有吸引力。若项目更需要开箱即用的全套约束，团队又缺少整合生态的能力，则其视图层定位可能成为负担；选择必须连同路由、数据和工程规范一起评估。

442. 如何避免React生命周期中的坑

避免生命周期的坑，关键是让渲染阶段保持纯净，把副作用和清理放到合适的提交阶段。Fiber让渲染可以暂停或重来，因此不要在已废弃的`componentWillMount`、`componentWillReceiveProps`和`componentWillUpdate`中请求数据或绑定事件。异步请求一般放在`componentDidMount`，更新前读取DOM可结合`getSnapshotBeforeUpdate`与`componentDidUpdate`。卸载时还要取消定时器和事件监听，并用错误边界避免渲染异常直接白屏。

16.3版本

>=16.4版本

在线查看：<https://projects.wojtekmaj.pl/react-lifecycle-methods-diagram>(opens new window)

避免生命周期中的坑需要做好两件事：不在恰当的时候调用了不该调用的代码；在需要调用时，不要忘了调用。那么主要有这么 7 种情况容易造成生命周期的坑`getDerivedStateFromProps`容易编写反模式代码，使受控组件与非受控组件区分模糊`componentWillMount`在 React 16.3 起被标记弃用、React 19 已移除（仅存`UNSAFE_componentWillMount`），不要在新代码中使用，主要原因是新的异步渲染架构会导致它被多次调用。所以网络请求及事件绑定代码应移至`componentDidMount`中。`componentWillReceiveProps`同样被标记弃用、React 19 已移除，被`getDerivedStateFromProps`所取代，主要原因是性能问题`shouldComponentUpdate`通过返回`true`或者`false`来确定是否需要触发新的渲染。主要用于性能优化`componentWillUpdate`同样是由于新的异步渲染机制而被标记废弃、React 19 已移除，不要在新代码中使用，原先的逻辑可结合`getSnapshotBeforeUpdate`与`componentDidUpdate`改造使用。如果在`componentWillUnmount`函数中忘记解除事件绑定，取消定时器等清理操作，容易引发

bug如果没有添加错误边界处理，当渲染发生异常时，用户将会看到一个无法操作的白屏，所以一定要添加`getDerivedStateFromProps`容易编写反模式代码，使受控组件与非受控组件区分模糊`componentWillMount`在 React 16.3 起被标记弃用、React 19 已移除（仅存`UNSAFE_componentWillMount`），不要在新代码中使用，主要原因是新的异步渲染架构会导致它被多次调用。所以网络请求及事件绑定代码应移至`componentDidMount`中。`componentWillReceiveProps`同样被标记弃用、React 19 已移除，被`getDerivedStateFromProps`所取代，主要原因是性能问题`shouldComponentUpdate`通过返回`true`或者`false`来确定是否需要触发新的渲染。主要用于性能优化`componentWillUpdate`同样是由于新的异步渲染机制而被标记废弃、React 19 已移除，不要在新代码中使用，原先的逻辑可结合`getSnapshotBeforeUpdate`与`componentDidUpdate`改造使用。如果在`componentWillUnmount`函数中忘记解除事件绑定，取消定时器

等清理操作，容易引发

bug如果没有添加错误边界处理，当渲染发生异常时，用户将会看到一个无法操作的白屏，所以一定要添加“React 的请求应该放在哪里，为什么？”这也是经常会被追问的问题。你可以这样回答。对于异步请求，应该放在componentDidMount中去操作。从时间顺序来看，除了componentDidMount还可以有以下选择：
constructor：可以放，但从设计上而言不推荐。constructor 主要用于初始化 state 与函数绑定，并不承载业务逻辑。而且随着类属性的流行，constructor 已经很少使用了
componentWillMount：已被标记废弃、React 19 已移除，在新的异步渲染架构下会触发多次渲染，容易引发 Bug，不利于未来 React 升级后的代码维护。所以 React 的请求放在componentDidMount 里是最好的选择。
透过现象看本质：React 16 缘何两次求变？Fiber 架构简析
Fiber 是 React 16 对 React 核心算法的一次重写。你只需要 get 到这一个点：Fiber 会使原本同步的渲染过程变成异步的。在 React 1...

1) 一个类组件在componentWillMount里发起订单请求，开发环境偶尔出现两笔相同请求；为什么把去重逻辑塞进去仍不是可靠修复？

答：componentWillMount属于可能因异步渲染而重复执行的旧生命周期，且在React 19已移除，仅遗留UNSAFE_形式用于迁移。请求应移到提交后的componentDidMount，并按业务需要处理取消或幂等；仅在旧钩子内部去重会继续绑定不稳定的执行时机。

2) 资料编辑页收到新userId后，用getDerivedStateFromProps把全部表单状态覆盖一遍，结果用户刚输入的内容频繁丢失；你会怎样重构？

答：这里混淆了受控数据和组件内部草稿，任何父组件更新都可能触发错误覆盖。应先明确表单由父级完全控制，还是仅在身份变化时重新初始化，再缩小需要派生的状态；getDerivedStateFromProps必须保持纯净，但纯净并不能消除状态双源带来的反模式。

3) 聊天页卸载后仍持续收到消息并触发更新，线上表现为内存上涨和重复提示；你会从哪个生命周期切入排查？

答：应检查componentWillUnmount是否对事件订阅、定时器及其他长期资源执行了对称清理，并确认绑定与解绑使用的是同一引用。仅阻止卸载后的状态更新不能终止资源本身；若组件反复挂载，遗漏清理还会累积监听器，使故障越来越明显。

4) 发布后某个商品卡片渲染异常导致整个内容区白屏，但网络和接口都正常；生命周期层面应补什么保护，它又保护不了什么？

答：应在合适的组件边界增加错误边界，把渲染异常限制在局部并提供降级界面，避免整棵相关视图不可操作。边界位置要兼顾隔离范围与恢复体验；它不是所有错误的通用捕获器，网络失败和事件回调等场景仍需各自处理。

5) 滚动列表更新前必须记录容器高度，DOM更新后再恢复滚动位置；有人准备把读取和写入都放进componentDidUpdate，你会怎么拆？

答：更新前需要读取旧DOM状态时，应使用getSnapshotBeforeUpdate获取快照，再把结果交给componentDidUpdate完成更新后的DOM操作。这样对应pre-commit可读DOM、commit执行副作用的阶段划分；若读写时机混在提交后，旧布局信息可能已经丢失。

443. React Fiber架构

Fiber是对React核心协调算法的重构，本质上把不可中断的递归遍历改成可暂停、可恢复的任务单元遍历。每个Fiber节点通过child、sibling和return连接，使协调工作能够拆分，并按优先级让高优任务先执行。可中断的reconciliation阶段负责比较并标记变化，commit阶段则同步更新真实DOM。它不一定缩短总计

算时间，主要价值是避免长任务持续占用主线程，让交互保持响应。最主要的思想就是将任务拆分。DOM需要渲染时暂停，空闲时恢复。window.requestIdleCallbackReact内部实现的机制 React 追求的是“快速响应”，那么，“快速响应”的制约因素都有什么呢 CPU的瓶颈：当项目变得庞大、组件数量繁多、遇到大计算量的操作或者设备性能不足使得页面掉帧，导致卡顿。IO的瓶颈：发送网络请求后，由于需要等待数据返回才能进一步操作导致不能快速响应。fiber架构主要就是用来解决CPU和网络的问题，这两个问题一直也是最影响前端开发体验的地方，一个会造成卡顿，一个会造成白屏。为此 react 为前端引入了两个新概念：Time Slicing时间分片和Suspense。

1. React 都做过哪些优化

React渲染页面的两个阶段调度阶段（reconciliation）：在这个阶段 React 会更新数据生成新的Virtual DOM，然后通过Diff算法，快速找出需要更新的元素，放到更新队列中去，得到新的更新队列。渲染阶段（commit）：这个阶段 React 会遍历更新队列，将其所有的变更一次性更新到DOM上React 15 架构React15架构可以分为两层Reconciler（协调器）——

负责找出变化的组件；Renderer（渲染器）—— 负责将变化的组件渲染到页面上；

调度阶段（reconciliation）：在这个阶段 React 会更新数据生成新的Virtual DOM，然后通过Diff算法，快速找出需要更新的元素，放到更新队列中去，得到新的更新队列。渲染阶段（commit）：这个阶段 React 会遍历更新队列，将其所有的变更一次性更新到DOM上

React15架构可以分为两层Reconciler（协调器）—— 负责找出变化的组件；Renderer（渲染器）—— 负责将变化的组件渲染到页面上； Reconciler（协调器）—— 负责找出变化的组件；Renderer（渲染器）—— 负责将变化的组件渲染到页面上；

在React15及以前，Reconciler采用递归的方式创建虚拟DOM，递归过程是不能中断的。如果组件树的层级很深，递归会占用线程很多时间，递归更新时间超过了16ms，用户交互就会卡顿。为了解决这个问题，React16将递归的无法中断的更新重构为异步的可中断更新，由于曾经用于递归的虚拟DOM数据结构已经无法满足需要。于是，全新的Fiber架构应运而生。React 16 架构为了解决同步更新长时间占用线程导致页面卡顿的问题，也为了探索运行时优化的更多可能，React开始重构并一直持续至今。重构的目标是实现Concurrent Mode（并发模式）。从v15到v16，React团队花了两年时间将源码架构中的Stack Reconciler重构为Fiber ReconcilerReact16架构可以分为三层：Scheduler（调度器）——

调度任务的优先级，高优任务优先进入Reconciler；Reconciler（协调器）—— 负责找出变化的组件：更新工作从递归变成了可以中断的循环过程。Reconciler内部采用了Fiber的架构；Renderer（渲染器）—— 负责将变化的组件渲染到页面上。React 17 优化使用Lane来管理任务的优先级。Lane用二进制位表示任务的优先级，方便优先级的计算（位运算），不同优先级占用不同位置的“赛道”，而且存在批的概念，优先级越低，“赛道”越多。高优先级打断低优先级，新建的任务需要赋予什么优先级等问题都是Lane所要解决的问题。Concurrent Mode的目的是实现一套可中断/恢复的更新机制。其由两部分组成：一套协程架构：Fiber Reconciler基于协程架构的启发式更新算法：控制协程架构工作方式的算法 React 16 架构 为了解决同步更新长时间占用线程导致页面卡顿的问题，也为了探索运行时优化的更多可能，React开始重构并一直持续至今。重构的目标是实现Concurrent Mode（并发模式）。从v15到v16，React团队花了两年时间将源码架构中的Stack Reconciler重构为Fiber ReconcilerReact16架构可以分为三层：Scheduler（调度器）——

调度任务的优先级，高优...

1) 搜索页一次更新要遍历上千个组件，接入Fiber后总计算时间没有明显下降，团队据此认为架构改造失败；这个判断哪里有问题？

答：Fiber的主要目标不是减少总工作量，而是把协调阶段拆成可暂停、恢复或放弃的小任务，避免长时间独占主线程。总计算甚至可能没有下降，但输入等高优先级工作能更早响应；如果瓶颈是单个不可分割的同步任务，任务拆分也难以发挥作用。

2) 输入框联动一个大型结果列表，用户连续键入时旧列表还没算完；Scheduler、Reconciler和Renderer分别应承担什么职责？

答：Scheduler根据优先级安排工作，使更紧急的输入更新先进入处理；采用Fiber的Reconciler可中断地找出视图变化，Renderer再把确定的变更提交给宿主环境。若所有更新都被赋予相同优先级，架构具备中断能力也不等于产品自然获得理想调度。

3) 低优先级报表渲染被新的交互更新打断，日志显示前面一部分协调工作后来没有提交；这是数据丢失还是预期行为？

答：这是可中断协调阶段允许出现的预期行为：更高优先级任务到来后，未完成的低优先级工作可能被放弃并在之后重新计算。用户尚未看到这些中间结果，因此不会产生半裸界面；这也要求协调阶段保持纯净，不能把不可重复的副作用放进去。

4) 线上拖拽仍会掉帧，性能记录显示一次commit中有大量真实DOM更新；为什么Fiber没把这段工作切到多帧完成？

答：可拆分的是生成和比较更新结果的reconciliation阶段，commit要同步应用变更，以避免数据与可见UI长时间处于不一致状态。应继续排查本次提交的节点规模与DOM成本，而不是期待提交阶段被任意暂停；过大的commit仍可能造成明显卡顿。

5) 有人建议直接用浏览器的requestIdleCallback实现全部React调度，理由是它能返回剩余时间；你会指出哪些工程约束？

答：requestIdleCallback提供了空闲时间提示，但兼容性和触发频率并不稳定，切换标签页等情况还可能显著降低调用频率。React因此实现自己的Scheduler，并额外支持多种任务优先级；单纯等待空闲时间也可能让持续被高优任务挤压的工作长期得不到执行。

6) 代码评审发现类组件在协调阶段的旧will生命周期里注册事件，而团队正启用可中断更新；这和Fiber的哪条约束直接冲突？

答：协调阶段可能暂停、终止或重新开始，因此其中的生命周期存在重复执行的可能，注册事件会产生不可安全重放的副作用。副作用应移到同步提交阶段，并在卸载时清理；这正是旧componentWillMount、componentWillUpdate等生命周期被弃用并最终移除的重要背景。

444. createElement过程

React.createElement会根据类型、属性和子节点创建一个描述界面的React元素。JSX编译后本质上就是这类调用，第一个参数是标签或组件类型，第二个参数是props，后续参数表示子节点。返回结果只是普通的界面描述，并不会在这一步直接操作真实DOM，实际更新要经过协调和提交。React18

起挂载通常使用createRoot(container).render(element)，但不影响createElement的参数和作用。

React.createElement()：根据指定的第一个参数创建一个React元素

React.createElement(type,[props],[...children]) 第一个参数是必填，传入的是似HTML标签名称，eg: ul, li第二个参数是选填，表示的是属性，eg: className第三个参数是选填，子节点，eg:

要显示的文本内容 下面示例里的ReactDOM.render是历史写法：React 18

起改用ReactDOM.createRoot(container).render(element)，React 19

已移除ReactDOM.render。示例本身讲的是createElement的参数形式，挂载 API

换成createRoot后结论不变。 //写法一：var child1=React.createElement('li',null,'one');var child2=React.createElement('li',null,'two');var content=React.createElement('ul',{className:'teststyle'},child1,child2);// 第三个参数可以分开也可以写成一个数组ReactDOM.render(content,document.getElementById('example'));//写法二：var child1=React.createElement('li',null,'one');var child2=React.createElement('li',null,'two');var content=React.createElement('ul',{className:'teststyle'},[child1,child2]);ReactDOM.render(content,document.getElementById('example'));

1) 商品列表页把`React.createElement('li', { className: 'item' }, sku)`当成已经插入页面的节点，随后立刻查询`item`却找不到，误判在哪里？

答：误判是把创建React元素与挂载到页面混为一谈；这次调用只按`type`、属性和子节点构造待渲染内容。还要把返回结果交给根节点渲染，`ReactDOM.render(element, container)`，否则页面中不会出现对应节点。

2) 后台菜单有 200 项，开发者把子项数组作为第三个参数传给`React.createElement('ul', null, children)`；评审认为必须展开成`...children`，你怎么判断？

答：两种传法在该参数形式下都成立：子节点既能作为后续参数逐个传入，也能以数组形式放在第三个参数。工程上应统一团队写法并保证数组内容确实是合法子节点；不要仅因没有展开数组，就把渲染异常归因于`createElement`。

3) 老管理台仍用`ReactDOM.render(content, container)`展示`createElement`示例，升级到React18后负责人与维护者对是否重写有分歧，你会怎么处理？

答：示例所讲的三个参数形式不需要改变，但挂载代码应迁移为`ReactDOM.createRoot(container).render(content)`。`React18`起旧写法已经是历史用法，`React19`更移除了`ReactDOM.render`；继续保留会把元素创建和版本相关的挂载接口混在一起。

4) 营销页生成的列表没有样式，代码是`React.createElement('ul', { classname: 'teststyle' }, items)`，DOM结构和文本都正常，你先查哪里？

答：先核对第二个参数中的属性名，`React`示例使用的是`className`，而不是`classname`。第三个参数已经能正常产生文本和列表结构，说明排查重点应落在属性传递及样式选择器；改挂载API不能修复属性拼写错误。

5) 低代码渲染器需要同时生成两个，有人主张先各自调用`createElement`，另一人主张直接传配置对象数组；哪种边界必须守住？

答：传给父元素的子项应当是已经创建好的React子节点，例如两个`React.createElement('li', ...)`的结果，再逐个或以数组传入。普通配置对象不能仅凭形状替代子节点；若选择配置驱动，仍需先把配置映射为元素，代价是多一层转换与校验。

445. 调和阶段 `setState`内部干了什么

调用`setState`后，`React`会合并或排队保存状态更新，并启动调和流程。调和时会根据新状态构建新的`React`元素树，再与上一次的元素树做`diff`，找出真正发生变化的部分。这样最终只更新必要的UI，而不是每次都重建整个页面；状态计算和界面更新也不一定在调用处立即完成。当调用`setState`时，`React`会做的第一件事情是将传递给`setState`的对象合并到组件的当前状态这将启动一个称为和解（`reconciliation`）的过程。和解（`reconciliation`）的最终目标是以最有效的方式，根据这个新的状态来更新UI。

为此，`React`将构建一个新的`React`元素树（您可以将其视为UI的对象表示）一旦有了这个树，为了弄清UI如何响应新的状态而改变，`React`会将这个新树与上一个元素树相比较（`diff`）通过这样做，`React`将会知道发生的确切变化，并且通过了解发生什么变化，只需在绝对必要的情况下进行更新即可最小化UI的占用空间

1) 订单页调用`setState({ status: 'paid' })`后，开发者认为`React`会直接把按钮文本改成“已支付”，这个判断漏掉了什么过程？

答：`setState`触发的不是一次面向具体DOM文本的直接修改，而是基于新状态重新构建`React`元素树。`React`还要把新树与上一棵树比较，确定真正发生变化的位置后再更新UI；因此不能把状态对象与某

个DOM操作简单画等号。

2) 看板一次收到 300

条状态变化，产品要求只更新发生变化的卡片；你如何用调和过程解释实现判断？

答：React会依据新状态生成新的元素树，再与上一次的元素树进行比较，从而找出需要更新的部分。组件仍应让状态准确表达每张卡片的结果，不能手工假定只有某个节点变化；错误的状态建模会让生成的新树本身就不符合预期。

3) 同一份组件升级后改用并发能力，负责人仍要求“每次调用setState后立刻完成整棵树比较”，这个约束合理吗？

答：不应把setState理解为调用返回前必须完成全部UI更新；可靠结论是它会发起状态更新与后续调和。具体调度时机不由本节的树比较模型保证，业务若依赖立即可见结果，就把实现绑到了未承诺的执行时序上。

4) 库存页状态已经变成0，页面却仍显示“有货”；值班同学只盯着接口响应，你会沿哪条链路排查？

答：先确认传入setState的新状态是否正确，再检查该状态生成的新元素树是否真的改变了展示分支。随后核对树比较后对应UI是否提交；若新旧树表达相同内容，调和没有可更新之处，问题通常在状态到元素的映射而非接口本身。

5) 团队想绕过React，监听状态后直接改DOM，以避免元素树比较；在频繁更新的仪表盘里你如何取舍？

答：直接改DOM会绕开“新状态生成元素树、与旧树比较、只更新必要部分”的一致更新链路。除非该区域明确由外部系统独占管理，否则React后续调和可能覆盖手工结果；省下一次声明式计算的同时，会增加状态与页面失配的风险。

446. setState

setState的核心是排队和批量更新，不能简单理解成一个异步函数。对象写法连续读取旧state时，同名字段可能互相覆盖；依赖前一次结果时，应使用函数式写法，让更新按顺序计算。React18

的createRoot和React19 会在定时器、原生事件等场景自动批处理，React17

及以前则可能同步刷新。需要在提交后读取结果，可以使用回调；确需强制同步时可用flushSync。

先看版本结论（React 17 / 18 / 19 三档）

本节余下的Transaction（事务）与isBatchingUpdates分析描述的是React 16 / 17 的内部实现，属于历史对照：现在 Fiber 走的是 lane 优先级调度，不再依赖单个全局布尔标志。这些内容仍然值得读——它解释了「为什么当年定时器里会变成同步」——但不要当作 React 18 及以后的现状。

下面凡是出现「在原生事件和 setTimeout 中 setState 是同步的」这类表述，都请补上前提：React 17 及以前，或 React 18 的ReactDOM.render兼容模式。在了解setState之前，我们先来简单了解下 React

一个包装结构：Transaction：事务（Transaction）是 React

中的一个调用结构，用于包装一个方法，结构为：initialize - perform(method) -

close。通过事务，可以统一管理一个方法的开始与结束；处于事务流中，表示进程正在执行一些操作

setState：React 中用于修改状态，更新视图。它具有以下特点：

异步与同步:setState并不是单纯的异步或同步，这其实与调用时的环境相关：

在合成事件和生命周期钩子(除 componentDidMount) 中，setState是"异步"的；原因：

因为在setState的实现中，有一个判断：当更新策略正在事务流的执行中时，该组件更新会被推入dirtyComponents队列中等待执行；否则，开始执行batchedUpdates队列更新；在生命周期钩子调用中，更新策略都处于更新之前，组件仍处于事务流中，而componentDidUpdate是在更新之后，此时组件已经不在事务流中了，因此则会同步执行；在合成事件中，React 是基于 事务流完成的事件委托机制

实现，也是处于事务流中；问题：无法在setState后马上从this.state上获取更新后的值。解决：如果需要马上同步去获取新值，setState其实是可以传入第二个参数的。setState(updater, callback)，在回调中即可获取最新值；在原生事件和setTimeout中，setState是同步的，可以马上获取更新后的值（仅限 React 17 及以前 / React 18 兼容模式；React 18 createRoot起这些场景同样批量合并）；原因：原生事件是浏览器本身的实现，与事务流无关，自然是同步；而setTimeout是放置于定时器线程中延后执行，此时事务流已结束，因此也是同步；批量更新：在合成事件和生命周期钩子中，setState更新队列时，存储的是合并状态(Object.assign)。因此前面设置的 key 值会被后面所覆盖，最终只会执行一次更新；函数式：由于Fiber及合并的问题，官方推荐可以传入函数的形式。setState(fn)，在fn中返回新的state对象即可，例如this.setState((state, props) => newState)；使用函数式，可以用于避免setState的批量更新的逻辑，传入的函数将会被顺序调用；原因：因为在setState的实现中，有一个判断：当更新策略正在事务流的执行中时，该组件更新会被推入dirtyComponents队列中等待执行；否则，开始执行batchedUpdates队列更新；在生命周期钩子调用中，更新策略都处于更新之前，组件仍处于事务流中，而componentDidUpdate是在更新之后，此时组件已经不在事务流中了，因此则会同步执行；在合成事件中，React是基于事务流完成的事件委托机制实现，也是处于事务流中；问题：无法在setState后马上从this.state上获取更新后的值。解决：如果需要马上同步去获取新值，setState其实是可以传入第二个参数的。setState(updater, callback)，在回调中即可获取最新值；在生命周期钩子调用中，更新策略都处于更新之前，组件仍处于事务流中，而componentDidUpdate是在更新之后，此时组件已经不在事务流中了，因此则会同步执行；在...

1) 类组件计数器在一次点击里连续执行两次this.setState({ count: this.state.count + 1 })，两个回调都打印1；候选人说队列丢了一次更新，你同意吗？

答：不同意，两次对象式更新读取的都是当时相同的this.state.count，同一批次里同名字段会被后一次结果覆盖。改成this.setState(state => ({ count: state.count + 1}))才会顺序消费前一次结果；两个提交后回调看到的是合并完成的最终值。

2) React19 的搜索页在Promise.then里连续更新筛选条件和页码，产品担心产生两次渲染；你如何落地并验证？

答：React19 会对该异步回调中的更新进行批处理，不应沿用旧版“异步回调逐次同步刷新”的判断。把相互依赖的状态更新写成函数式形式，并用渲染日志或Profiler验证实际提交；批处理只减少提交次数，不会替你修正错误的状态依赖。

3) 遗留系统一部分用React18的ReactDOM.render，新页面使用createRoot，同样在setTimeout中两次setState却表现不同，怎么解释？

答：React18 的兼容根沿用React17 行为，定时器中的更新会逐次同步刷新；createRoot则启用automaticbatching，将这些更新一起处理。排查时必须先确认根节点创建方式，不能只看React主版本；迁移根节点还可能改变依赖同步读值的旧代码。

4) 线上弹窗执行setState后马上读取this.state.open，在React18createRoot下仍是旧值，开发者准备套一层setTimeout，你会怎么改？

答：不要依赖定时器碰运气，因为automaticbatching已覆盖setTimeout，回调中的读取也未必对应预想的提交边界。类组件可使用setState(updater, callback)在更新完成后读取，或在componentDidUpdate中响应变化；若新值依赖旧值，应同时改用函数式更新。

5) 展开面板后必须立刻测量新DOM高度，架构师禁止所有同步刷新，动画负责人坚持使用flushSync；你支持哪边？

答：该场景确实需要在状态更新后立即读取已提交的DOM，可把最小范围的更新包在flushSync中。它是React18createRoot与React19打破批处理的出口，但会牺牲批处理收益；应限制在必要测量点，不能作为普通状态更新的默认写法。

6) 组件卸载后请求回调仍调用setState，监控出现警告；有人提议每个组件都维护isUnmounted，另一人要把数据提升到父层，你怎么选？

答：先让异步任务在卸载时停止回写；若数据生命周期确实长于组件，再考虑放到父级或外部状态中通过props传入。isUnmounted判断是局部防线，会在大量组件中形成重复状态；无论选哪种方案，都不能让已销毁实例继续进入更新队列。

447. setState原理分析

setState本质上会把状态变更放进更新队列，再由React在合适的时机统一计算和渲染。React16、17的旧实现通过isBatchingUpdates和dirtyComponents控制批处理，React18起则改用Fiber的lane调度 and 自动批处理。直接修改this.state不会进入这套更新流程，因此不能可靠触发重新渲染。若新值依赖旧值，应使用函数式更新；提交后的值可在回调或componentDidUpdate中读取。本节是 React 16 / 17 内部实现的历史对照：下面的pre/post钩子、isBatchingUpdates、dirtyComponents属于旧的Transaction批处理实现。React 18起改为Fiber的lane优先级调度 + automatic batching，「是否在React调用栈里」不再决定批处理与否。版本结论见上一节开头的三档对照表。

1. setState异步更新 我们都知道，React通过this.state来访问state，通过this.setState()方法来更新state。当this.setState()方法被调用的时候，React会重新调用render方法来重新渲染UI首先如果直接在setState后面获取state的值是获取不到的。在React内部机制能检测到的地方，setState就是异步的；在React检测不到的地方，例如setInterval, setTimeout，setState就是同步更新的（React 17 及以前；React 18createRoot起这些场景同样批量合并，React 19 无法退回旧行为）

因为setState是可以接受两个参数的，一个state，一个回调函数。因此我们可以在回调函数里面获取值，setState方法通过一个队列机制实现state更新，当执行setState的时候，会将需要更新的state合并之后放入状态队列，而不会立即更新this.state如果我们不使用setState而是使用this.state.key来修改，将不会触发组件的re-render。如果将this.state赋值给一个新的对象引用，那么其他不在对象上的state将不会被放入状态队列中，当下次调用setState并对状态队列进行合并时，直接造成了state丢失

1.1 setState批量更新的过程 在react生命周期和合成事件执行前后都有相应的钩子，分别是pre钩子和post钩子，pre钩子会调用batchedUpdate方法将isBatchingUpdates变量置为true，开启批量更新，而post钩子会将isBatchingUpdates置为false isBatchingUpdates变量置为true，则会走批量更新分支，setState的更新会被存入队列中，待同步代码执行完后，再执行队列中的state更新。isBatchingUpdates为true，则把当前组件（即调用了setState的组件）放入dirtyComponents数组中；否则batchUpdate所有队列中的更新而在原生事件和异步操作中，不会执行pre钩子，或者生命周期的中的异步操作之前执行了pre钩子，但是pos钩子也在异步操作之前执行完了，isBatchingUpdates必定为false，也就不会进行批量更新 enqueueUpdate包含了React避免重复render的逻辑。mountComponent和updateComponent方法在运行的最开始，会调用到batchedUpdates进行批处理更新，此时会将isBatchingUpdates设置为true，也就是将状态标记为现在正处于更新阶段了。isBatchingUpdates为true，则把当前组件（即调用了setState的组件）放入dirtyComponents数组中；否则batchUpdate所有队列中的更新

1.2 为什么直接修改this.state无效 要知道setState本质是通过一个队列机制实现state更新的。执行setState时，会将需要更新的state合并后放入状态队列，而不会立刻更新state，队列机制可以批量更新state。如果不通过setState而直接修改this.state，那么这个state不会放入状态队列中，下次调用setState时对状态队列进行合并时，会忽略之前直接被修改的state，这样我们就无法合并了，而且实际也没有把你想要的state更新上去

1.3 什么是批量更新 Batch Update 在一些mv*框架中，就是将一段时间内对model的修改批量更新到view的机制。比如那前端比较火的React、vue（next...）

1) 资料编辑页直接执行`this.state.profile.name =`

`value`，调试器里值已变化但页面没重渲染；开发者认为React的diff失效了，你怎么纠正？

答：直接修改`this.state`没有通过`setState`进入状态更新队列，因此不能据此期待React重新调用`render`。应把状态视为不可直接修改的数据，通过`setState`提交新值；否则后续队列合并还可能忽略这次私改，造成页面与内存观察结果不一致。

2) 表格一次编辑三个字段，代码连续调用对象式`setState`，评审担心每次都重绘整页；你会怎样实现？

答：这些更新会先进入状态队列，并在合适时机批量合并后更新视图，而不是把每次调用都等同于一次立即DOM修改。若字段互不覆盖，可提交各自的部分状态；若新值依赖旧状态，则使用函数式更新，避免对象式更新在队列中覆盖结果。

3) 老代码用`isBatchingUpdates`和`dirtyComponents`解释React19

的所有更新时序，面试中你会指出什么版本边界？

答：这套`pre`、`post`、`isBatchingUpdates`与`dirtyComponents`描述的是React16、17的`Transaction`实现，只适合作为历史对照。React18起采用Fiber的lane优先级调度与`automatic batching`，触发点是否位于React调用栈内已不再决定是否批处理。

4)

支付页调用`setState`后日志仍是旧金额，值班同学认定更新失败；你要求他按什么顺序定位？

答：先确认日志是否紧跟在`setState`后，因为状态更新入队并不保证当场改写`this.state`。再把观察点移到`setState`回调或`componentDidUpdate`，确认提交后的值；若那里仍异常，继续检查对象式更新是否被后续同字段更新覆盖。

5) 类组件需要在价格更新完成后上报一次埋点，有人放在`render`，有人放进`setState`第二个回调；你怎么取舍？

答：不要在`render`中上报，因为重新渲染不等于这次价格更新的专属完成通知，容易产生重复副作用。针对单次类组件更新可使用`setState`的第二个回调，跨多次价格变化的统一响应则放在`componentDidUpdate`并比较前后值；两者都要防止回调再次无条件更新状态。

6) 遗留组件在`shouldComponentUpdate`中调用`setState`修正数据，线上出现持续更新和内存上涨；根因与改法是什么？

答：该生命周期本就在判断一次待处理更新，内部再次调用`setState`会继续向待处理队列追加更新，可能形成循环调用。应把数据修正移到事件入口、状态计算阶段或受条件约束的更新位置；仅加日志或扩大内存不能消除循环更新链路。

448. React事务机制

React事务机制是Stack架构时期用来包裹更新流程、实现批处理的一套机制。它会在目标方法执行前运行`initialize`，执行完成后再运行`close`，从而统一开启和结束批量更新。处于事务中的多次`setState`会先被收集，所以调用后看起来没有立即生效。需要注意，这属于旧版实现；Fiber架构下已经由lane优先级调度等机制接替。

1) 遗留后台在React合成事件里连续调用三次`setState`，开发者用“每次调用都立刻刷新一次”解释页面行为；结合事务图你会怎么反驳？

答：旧实现会在受React管理的调用过程外包一层事务，在开始阶段进入批量更新状态，结束阶段再统一处理队列。因而三次调用不能简单等同于三次即时提交；但该解释属于旧版内部机制，不应直接套到React18、19的lane调度上。

2) 维护者准备把旧Transaction图直接写进React19

的故障手册，用它判断setTimeout一定逃离批处理；你批准吗？

答：不批准，这组图更适合解释旧实现中事务边界与批处理的关系，不能证明现代版本的定时器更新仍逐次刷新。React18createRoot起已有automaticbatching，React19也保持批处理；手册必须先标注根节点和版本边界。

3) React17 页面在原生事件回调里两次更新状态，迁移createRoot后日志时序改变；值班人员只检查业务代码，漏了什么？

答：他漏查了根节点迁移带来的批处理边界变化：旧事务机制无法覆盖的原生回调，在React18createRoot下也会进入automaticbatching。应对照迁移前后的根节点、触发来源和提交日志；不能继续用旧图中的事务包裹范围推断新版本时序。

4) 数据大屏要求更新状态后立即读取新DOM，团队想修改事务开关强制提交；你会给出什么工程方案？

答：不要操作内部事务状态，这些标志属于旧实现细节，也不是稳定公共接口。现代React应在确有同步DOM读取需求时，以最小范围使用flushSync包住更新；它会打破批处理并增加同步工作，因此只适合作为受控逃生口。

5) 培训中有人把Transaction、automaticbatching和reconciliation说成同一件事，导致新人认为合并更新就是跳过树比较；你如何串起来？

答：Transaction描述旧版如何包裹一段执行并在边界处理更新，automaticbatching描述现代版本如何合并一批状态更新；它们都不等于reconciliation。批次形成后，React仍需根据新状态构建元素树并与旧树比较，再决定必要的UI更新。

449. React组件和渲染更新过程

React组件的渲染分为计算差异的render阶段和提交变更的commit阶段，前者可中断，后者不可中断。JSX本质上会调用createElement生成虚拟节点，组件再根据props和state执行render()，最终把结果更新到页面。调用setState后，相关组件会被标记为待更新，重新生成新节点并与旧节点比较，因此副作用应留到提交阶段处理。

渲染和更新过程

JSX即createElement函数执行生成vnodepatch(elem,vnode)和patch(vnode,newNode)

组件渲染过程 props staterender()生成vnodepatch(elem, vnode) 组件更新过程 setState-->dirtyComponents(可能有子组件)render生成newVnodepatch(vnode, newVnode)

1) 商品详情页只修改了购物车数量，父组件和多个商品推荐子组件的render都执行了，但DOM几乎没变；你会把这次更新描述成整页重绘吗？

答：不会，组件重新执行render只是在依据新的props、state生成newVnode，不等于重新创建整页真实DOM。随后patch(vnode, newVnode)会找出变化并更新对应节点；若只看到函数执行次数，不能直接判定真实DOM操作过多。

2) 后台表格点击“刷新状态”后调用了setState，面试官要求你从这一行代码一直讲到页面文字变化，中间链路怎么说？

答：setState会使相关组件进入待更新集合，可理解为被标记到dirtyComponents，其中也可能涉及子组件。React随后重新执行组件渲染得到newVnode，再与旧vnode做patch，最终把差异落实到真实DOM；它不是直接修改页面节点。

3) 一个组件首次挂载和后续调用setState时都执行了render，前端同学认为两条链路完全相同，你会指出哪处关键差异？

答：首次挂载是由props、state生成vnode，再执行类似patch(elem, vnode)的过程，把描述落实到容器。更新阶段已有旧vnode，所以重点变成生成newVnode并执行patch(vnode, newVnode)；两者都渲染，但比较基准不同。

4) 订单页调用setState后数据已经变了，界面却仍显示旧金额；你会按哪几层定位，而不是直接怀疑浏览器缓存？

答：先确认更新是否让目标组件进入待更新流程，再检查render生成的newVnode是否真的包含新金额。若虚拟节点正确，再沿patch(vnode, newVnode)检查差异是否落到对应真实节点；若渲染输入仍旧，问题通常在状态或属性传递链路，而非DOM补丁阶段。

5) 团队想绕过虚拟节点，在库存变化时直接找到节点改textContent，理由是只有一个数字变化；你如何评估这种取舍？

答：直接改DOM可能立即显示结果，但会绕开由props、state生成vnode再执行patch的一致更新链路。后续组件再次渲染时，声明结果可能覆盖手工修改，也会增加状态与页面不一致的排查成本；除非明确隔离该节点，否则不宜作为常规方案。

450. 如何解释 React 的渲染流程

React 的渲染流程可以概括为触发更新、调度任务、计算差异和提交结果。React 16以前的Stack Reconciler以递归和事务为核心，执行期间容易持续占用主线程；之后的Fiber Reconciler把节点组织成可遍历的Fiber结构，并支持按优先级让出执行权。render阶段构造workInProgress树，可中断且不应产生副作用；commit阶段同步更新DOM、执行生命周期和处理ref，不可中断。普通管理后台里两者差距未必明显，但动画、画布或手势场景更容易体现Fiber调度的价值。

React 的渲染过程大致一致，但协调并不相同，以React 16为分界线，分为Stack Reconciler和Fiber Reconciler。这里的协调从狭义上来讲，特指 React 的 diff 算法，广义上来讲，有时候也指 React 的reconciler模块，它通常包含了diff算法和一些公共逻辑。回到Stack Reconciler中，Stack Reconciler的核心调度方式是递归。调度的基本处理单位是事务，它的事务基类是Transaction，这里的事务是 React 团队从后端开发中加入的概念。在 React 16 以前，挂载主要通过 ReactDOM 模块完成，更新通过ReactUpdate模块完成，模块之间相互分离，落脚执行点也是事务。在React 16及以后，协调改为了Fiber Reconciler。它的调度方式主要有两个特点，第一个是协作式多任务模式，在这个模式下，线程会定时放弃自己的运行权利，交还给主线程，通过requestIdleCallback实现。第二个特点是策略优先级，调度任务通过标记tag的方式分优先级执行，比如动画，或者标记为high的任务可以优先执行。Fiber Reconciler的基本单位是Fiber，Fiber基于过去的React Element提供了二次封装，提供了指向父、子、兄弟节点的引用，为diff工作的双链表实现提供了基础。在新的架构下，整个生命周期被划分为Render 和 Commit 两个阶段。Render 阶段的执行特点是可中断、可停止、无副作用，主要是通过构造workInProgress树计算出diff。以current树为基础，将每个Fiber作为一个基本单位，自下而上逐个节点检查并构造 workInProgress 树。这个过程不再是递归，而是基于循环来完成在执行上通过requestIdleCallback来调度执行每组任务，每组中的每个计算任务被称为work，每个work完成后确认是否有优先级更高的work需要插入，如果有就让位，没有就继续。优先级通常是标记为动画或者high的会先处理。每完成一组后，将调度权交回主线程，直到下一次requestIdleCallback调用，再继续构建workInProgress树在commit阶段需要处理effect列表，这里的effect列表包含了根据diff 更新 DOM 树、回调生命周期、响应 ref等。但一定要注意，这个阶段是同步执行的，不可中断暂停，所以不要在componentDidMount、componentDidUpdate、componentWillUnmount中去执行重度消耗算力的任务如果只是一般的应用场景，比如管理后台、H5 展示页等，两者性能差距并不大，但在动画、画布及手势等场景下，Stack Reconciler的设计会占用主线程，造成卡顿，而fiber reconciler的设计则能带来高性能的表现

1) 动画面板更新时主线程连续被占用，同事说React只是在递归整棵组件树；放到React 16之后的架构里，这个说法哪里不准确？

答：React 16之后使用Fiber Reconciler，基本处理单位是带父、子、兄弟引用的Fiber，工作过程不再等同于旧架构的连续递归。协调工作可被拆分、暂停和恢复，并结合任务优先级让位；但后续commit仍是同步且不可中断的。

2) 画布编辑器同时收到拖拽更新和低优先级列表刷新，产品要求拖拽优先响应；渲染流程中的哪一层负责体现这种优先级？

答：优先级体现在Fiber Reconciler的调度与render工作阶段，任务会被标记并按策略安排，高优先级工作可插入并获得执行机会。它改善的是协调过程对主线程的占用方式，不代表所有工作都会异步；真正应用变更的commit仍要同步完成。

3) 管理后台只有少量表单和列表，架构评审却把从Stack Reconciler到Fiber Reconciler当成必然的大幅性能收益；你会接受吗？

答：不会直接接受，一般管理后台或H5展示页中，两种协调方式的体验差距未必明显。Fiber的优势更容易在动画、画布、手势或大批量更新等主线程敏感场景体现；是否有收益仍要结合更新规模和交互特征判断。

4) 线上弹窗打开时页面长时间不响应，日志显示节点差异已经算完，随后生命周期回调里执行了重计算；你会把故障定位到哪个阶段？

答：应重点定位commit阶段，因为它会同步处理effect列表，包括更新DOM、生命周期回调和ref。该阶段不可中断，在componentDidMount、componentDidUpdate或卸载回调中执行重计算会直接延长阻塞；把渲染阶段做成可中断并不能抵消这类开销。

5) 面试官让你用current树、workInProgress树和真实DOM串起一次更新，不能只背“渲染与提交”，你会怎么讲？

答：render阶段以current树为基础，逐个处理Fiber并构造workInProgress树，在此期间计算差异且应避免副作用。完成后进入同步commit，按照effect信息更新真实DOM、触发生命周期并处理ref；前一阶段可调度，后一阶段不能暂停。

451. diff算法是怎么运作

React的diff通过只比较同层节点、按组件类型判断复用，并用key识别同层元素，把树比较从 $O(n^3)$ 简化到 $O(n)$ 。节点类型不同会直接删除旧子树并创建新子树，类型相同才继续比较属性和子节点，所以跨层移动通常也会被当成删除与新增。列表更新时会优先处理可复用节点，剩余节点再执行插入、删除或移动。key应保持唯一和稳定，否则在头部插入或顺序变化时，可能让后续节点失去正确的复用关系；用索引作key也有同样风险。每一种节点类型有自己的属性，也就是prop，每次进行diff的时候，react会先比较该节点类型，假如节点类型不一样，那么react会直接删除该节点，然后直接创建新的节点插入到其中，假如节点类型一样，那么会比较prop是否有更新，假如有prop不一样，那么react会判定该节点有更新，那么重渲染该节点，然后在对其子节点进行比较，一层一层往下，直到没有子节点。把树形结构按照层级分解，只比较同级元素。给列表结构的每个单元添加唯一的key属性，方便比较。React只会匹配相同class的component（这里的class指的是组件的名字）合并操作，调用component的setState方法的时候，React将其标记为 -dirty。到每一个事件循环结束，React检查所有标记dirty的component重新绘制。选择性子树渲染。开发人员可以重写shouldComponentUpdate提高diff的性能

优化

为了降低算法复杂度，React的diff会预设三个限制：只对同级元素进行Diff。如果一个DOM节点在前后两次更新中跨越了层级，那么React不会尝试复用他。两个不同类型的元素会产生出不同的树。如果元素由div变为p，React会销毁div及其子孙节点，并新建p及其子孙节点。开发者可以通过key prop来暗示哪些子元素在不同的渲染下能保持稳定。考虑如下例子：

Diff的思路

该如何设计算法呢？如果让我设计一个Diff算法，我首先想到的方案是：判断当前节点的更新属于哪种情况如果是新增，执行新增逻辑如果是删除，执行删除逻辑如果是更新，执行更新逻辑 按这个方案，其实有个隐含的前提——不同操作的优先级是相同的但是React团队发现，在日常开发中，相较于新增和删除，更新组件发生的频率更高。所以Diff会优先判断当前节点是否属于更新。

基于以上原因，Diff算法的整体逻辑会经历两轮遍历：

第一轮遍历：处理更新的节点。第二轮遍历：处理剩下的不属于更新的节点。 diff算法的作用

计算出Virtual DOM中真正变化的部分，并只针对该部分进行原生DOM操作，而非重新渲染整个页面。

传统diff算法 通过循环递归对节点进行依次对比，算法复杂度达到 $O(n^3)$ ，n是树的节点数，这个有多可怕呢？——如果要展示1000个节点，得执行上亿次比较。。即便是CPU快能执行30亿条命令，也很难在一秒

内计算出差异。React的diff算法 什么是调和？将Virtual DOM树转换成actual DOM树的最少操作的过程称为 调和 。 什么是React diff算法？ diff算法是调和的具体实现。 diff策略 React用 三大策略

将 $O(n^3)$ 复杂度 转化为 $O(n)$ 复杂度 策略一（tree diff）： Web UI中DOM节点跨层级的移动操作特别少，可以忽略不计。 策略二（component diff）：

拥有相同类的两个组件 生成相似的树形结构，拥有不同类的两个组件 生成不同的树形结构。

策略三（element diff）： 对于同一层级的一组子节点，通过唯一id区分。 tree diff

React通过updateDepth对Virtual DOM树进行层级控制。对树分层比较，两棵树 只对同一层次节点 进行比较。如果该节点不存在时，则该节点及其子节点会被完全删除，不会再进行进一步比较。只需遍历一次，就能完成整棵DOM树的比较。 那么问题来了，如果DOM节点出现了跨层级操作,diff会咋办呢？

答：diff只简单考虑同层级的节点位置变换，如果是跨层级的话，只有创建节点和删除节点的操作。

如上图所示，以A为根节点的整棵树会被重新创建，而不是移动，因此 官方建议不要进行DOM节点跨层级操作，可以通过CSS隐藏、显示节点，而不是真正地移除、添加DOM节点 component diff

React对不同的组件间的比较，有三种策略 同一类型的两个组件，按原策略（层级比较）继续比较Virtual DOM树即可。同一类型的两个组件，组件A变化为组件B时，可能Virtual

DOM没有任何变化，如果知道这点（变换的过程中，Virtual

DOM没有改变），可节省大量计算时间，所以 用户 可以通过shouldComponentUpdate...

1) 消息列表头部插入一条新消息，开发者没有提供稳定key，结果插入点后的多行都被当成变化；为什么diff没识别出只是新增一项？

答：同层子节点缺少稳定标识时，React主要按位置判断对应关系，头部插入会改变后续节点的位置映射。为每条消息使用唯一且稳定的业务key，才能帮助element diff区分插入、删除和移动；key只在一组兄弟节点中参与匹配。

2) 可拖拽看板把一个带输入状态的卡片从A列移到B列，业务希望组件实例和状态原样保留；仅让两边使用相同key能做到吗？

答：不能保证，因为React只在同一层级、同一组子节点中利用key建立对应关系，不会把跨父节点移动当作可复用的同一次移动。旧子树会被删除，新位置会创建节点，局部状态可能随之丢失；需要保持层级不变并调整展示，或把需保留的状态移到更稳定的位置。

3) 设计系统把列表项根节点从div改成p，内部子组件和属性看起来完全一致；为什么上线后仍出现整段节点重建？

答：根元素类型不同会触发不同树的假设，React会销毁旧div及其子孙节点，再创建新的p子树，而不是继续比较内部结构。只有类型相同时才会比较prop并向下处理子节点；结构相似并不足以跨类型复用。

4) 千行表格更新一行后出现大量节点操作，你拿到新旧虚拟树时会先查哪些破坏复用关系的线索？

答：先核对同层列表是否有唯一、稳定的key，以及更新前后元素或组件类型是否被意外替换。再检查节点是否跨父级移动，因为这会表现为旧子树删除和新子树创建；这些线索比单纯统计组件执行次数更能解释异常补丁规模。

5) 前端负责人建议实现完整的跨层树匹配，认为能减少任何移动场景的节点创建；你如何回应复杂度与收益冲突？

答：完整树比较的传统复杂度可达到 $O(n^3)$ ，React通过只比较同层节点、按组件类型判断、用key区分同组元素，把常见比较约束到 $O(n)$ 。支持任意跨层复用会破坏这些前提并显著增加成本；更稳妥的是调整页面结构，减少跨层移动。

6) 一个类组件渲染代价较高，但其输入在多数更新中不变；shouldComponentUpdate与diff的关系该怎样向评审解释？

答：shouldComponentUpdate可在已知组件输出无需变化时阻止继续计算其子树，从而减少不必要的diff。它依赖开发者正确判断props与state是否影响输出，判断错误会让页面漏更新；因此应针对明确稳定的子树使用，而不是无条件阻断。

452. 合成事件原理

合成事件本质上是React对原生事件的统一封装，事件通常委托到顶层容器，而不是直接绑定在每个子元素上。原生事件冒泡到委托节点后，React会创建SyntheticEvent，找到对应组件，并按组件树分发处理函数。这样既能屏蔽浏览器差异，也便于统一管理事件并减少大量监听器带来的内存开销。需要注意版本边界：React

17以前主要委托到document，之后改为各自的根容器；混用原生事件时也要留意两套传播路径的差别。为了解决跨浏览器兼容性问题，React会将浏览器原生事件（Browser Native Event）封装为合成事件（SyntheticEvent）传入设置的事件处理器中。这里的合成事件提供了与原生事件相同的接口，不过它们屏蔽了底层浏览器的细节差异，保证了行为的一致性。另外有意思的是，React并没有直接将事件附着到子元素上，而是以单一事件监听器的方式将所有的事件发送到顶层进行处理。这样React在更新DOM的时候就不需要考虑如何去处理附着在DOM上的事件监听器，最终达到优化性能的目的

所有的事件挂在document上，DOM事件触发后冒泡到document；React找到对应的组件，造出一个合成事件出来；并按组件树模拟一遍事件冒泡。event不是原生的，是SyntheticEvent合成事件对象和Vue事件不同，和DOM事件也不同

React 17 之前的事件冒泡流程图

所以这就造成了，在一个页面中，只能有一个版本的

React。如果有多个版本，事件就乱套了。值得一提的是，这个问题在React 17中得到了解决，事件委托不再挂在document上，而是挂在DOM容器上，也就是ReactDOM.Render所调用的节点上。

React 17 后的事件冒泡流程图

那到底哪些事件会被捕获生成合成事件呢？可以从React的源码测试文件中一探究竟。下面的测试快照中罗列了大量的事件名，也只有在这份快照中的事件，才会被捕获生成合成事件。// react/packages/react-dom/src/__tests__/__snapshots__/ReactTestUtils-test.js.snapArray["abort","animationEnd","animationIteration","animationStart","auxClick","beforeInput","blur","canPlay","canPlayThrough","cancel","change","click","close","compositionEnd","compositionStart","compositionUpdate","contextMenu","copy","cut","doubleClick","drag","dragEnd","dragEnter","dragExit","dragLeave","dragOver","dragStart","drop","durationChange","emptied","encrypted","ended","error","focus","gotPointerCapture","input","invalid","keyDown","keyPress","keyUp","load","loadStart","loadedData","loadedMetadata","lostPointerCapture","mouseDown","mouseenter","mouseleave","mousemove","mouseout","mouseover","mouseup","paste","pause","play","playing","pointerCancel","pointerDown","pointerEnter","pointerLeave","pointerMove","pointerOut","pointerOver","pointerUp","progress","rateChange","reset","scroll","seeked","seeking","select","stalled","submit","suspend","timeUpdate","toggle","touchCancel",

"touchEnd","touchMove","touchStart","transitionEnd","volumeChange","waiting","wheel",] 如果DOM上绑定了过多的事件处理函数,整个页面响应以及内存占用可能都会受到影响。React为了避免这类DOM事件滥用,同时屏蔽底层不同浏览器之间的事件系统的差异,实现了一个...

1) 列表里有一千个按钮,实习生认为每个onClick都会对应一个直接挂在按钮上的原生监听器;结合React的事件链路,你会如何纠正?

答: React不会按这种理解把每个处理器直接附着到对应子节点,而是统一监听支持的事件,再在事件到达委托层后定位组件并分发。传给业务处理器的是SyntheticEvent,这种中间层减少了逐节点管理监听器的负担,也统一了浏览器差异。

2) 同一页面嵌入两个独立React应用,旧方案都把事件交给document,团队担心事件互相影响;React 17的委托位置变化解决了什么?

答: React 17将事件委托从document下移到各自的React根容器,也就是渲染所使用的容器节点。这样多个应用或版本能在各自边界内处理合成事件,降低顶层事件系统相互干扰的风险;跨容器的原生冒泡规则仍需单独考虑。

3) 埋点SDK用原生addEventListener,业务组件使用onClick,一次点击出现顺序与组件树直觉不一致;排查时为什么不能只看JSX嵌套?

答: 业务onClick经过React的统一监听、合成和dispatchEvent分发,并非直接绑定在JSX对应节点上。应同时确认原生监听器所在节点、捕获或冒泡阶段,以及React版本对应的委托层;混用两套事件系统时,原生传播路径与组件树模拟分发不能混为一谈。

4) 线上只有部分事件能进入组件处理器,click正常但某个自定义事件名称毫无响应;你会怎样判断是不是合成事件系统的问题?

答: 先确认该事件是否属于React支持并会捕获生成SyntheticEvent的事件集合,不能假设任意事件名都会被统一分发。若不在支持范围,应改用合适的原生监听或封装桥接;若属于支持事件,再沿原生冒泡、根容器监听和dispatchEvent定位中断点。

5) 基础设施团队主张全部改用原生事件,理由是SyntheticEvent只是多包了一层;你会保留它的哪些工程价值?

答: 合成事件统一了不同浏览器的事件接口,并让React集中管理监听和分发,避免组件更新时反复处理大量节点监听器。全部改用原生事件会获得更直接的底层控制,但需要自行承担兼容性、解绑和与React委托边界协作的成本;两套体系混用还会增加传播顺序的复杂度。

453. JSX语法糖本质

JSX本质上是JavaScript的语法扩展,也是创建ReactElement的语法糖。经典转换中,Babel会先把它解析成AST,再转换为React.createElement调用,最终得到描述节点类型、属性和子元素的普通对象。这个对象组成虚拟DOM树,后续经过对比和更新渲染到页面。实际开发选择JSX,是因为它比手写嵌套函数更直观,也不需要额外引入模板指令。JSX是语法糖,通过babel转成React.createElement函数,在babel官网上可以在线把JSX转成React的JS语法 首先解析出来的话,就是一个createElement函数然后这个函数执行完后,会返回一个vnode通过vdom的patch或者是其他的一个方法,最后渲染一个页面script标签中不添加text/babel解析jsx语法的情况下 const ele=React.createElement("h2",null,"Hello React!");// 历史写法: React 18 起用 ReactDOM.createRoot(...).render(ele), React 19 已移除 ReactDOM.renderReactDOM.render(ele,document.getElementById("app"));

JSX的本质是React.createElement()函数 createElement函数返回的对象是ReactElement对象。createElement的写法如下 class App extends React.Component{constructor(){super()}this.state={}}render(){return React.createElement("div",null,/*第一个子元素, header*/React.createElement("div",{

```
className:"header"},React.createElement("h1",{title:"\u6807\u9898"},"\u6211\u662F\u6807\u9898"))/*第二个子元素，content*/React.createElement("div",{className:"content"},React.createElement("h2",null,"\u6211\u662F\u9875\u9762\u7684\u5185\u5BB9"),React.createElement("button",null,"\u6309\u94AE"),React.createElement("button",null,"+1"),React.createElement("a",{href:"http://www.baidu.com"},"\u767E\u5EA6\u4E00\u4E0B"))/*第三个子元素，footer*/React.createElement("div",{className:"footer"},React.createElement("p",null,"\u6211\u662F\u5C3E\u90E8\u7684\u5185\u5BB9")));}}// 历史写法：React 18 起用 ReactDOM.createRoot(document.getElementById("app")).render()ReactDOM.render(,document.getElementById("app"));
```

实际开发中不会使用createElement来创建ReactElement的，一般都是使用JSX的形式开发。

ReactElement在程序中打印一下

```
render(){let ele=(我是标题我是页面的内容按钮+1 百度一下我是尾部的内容)console.log(ele);return ele;}
```

react通过babel把JSX转成createElement函数，生成ReactElement对象，然后通过ReactDOM.render函数把ReactElement渲染成真实的DOM元素

为什么 React 使用 JSX 在回答问题之前，我首先解释下什么是 JSX 吧。JSX 是一个JavaScript的语法扩展，结构类似 XML。JSX 主要用于声明React元素，但 React 中并不强制使用JSX。即使使用了JSX，也会在构建过程中，通过Babel 插件编译为React.createElement。所以 JSX 更像是React.createElement的一种语法糖接下来与 以外的三种技术方案进行对比首先是模板，React

团队认为模板不应该是开发过程中的关注点，因为引入了模板语法、模板指令等...

1) 代码评审里有人说会在浏览器里直接创建一个名为UserCard的DOM标签；你会用编译产物怎样反驳？

答：JSX会先经Babel转换为类似React.createElement(UserCard, {name: "Ada"})的调用，其中type是组件本身而不是浏览器标签字符串。该调用组装并返回ReactElement描述，之后才由React的渲染流程把它落实到真实DOM。

2) 低代码平台不能使用JSX，只允许执行普通JavaScript；要生成一个带两个li的ul，最少需要向createElement提供哪些信息？

答：需要提供节点type、保存属性的config，以及从第三个参数开始传入的一个或多个children。例如父节点类型为"ul"，子项分别是嵌套的React.createElement("li", {key: "..."}, "...")；返回值仍是元素描述，不会在调用时直接操作DOM。

3) 调试台打印JSX变量时只看到带type、key、ref、props的普通对象，开发者以为渲染失败了；这个现象正常吗？

答：正常，createElement的职责主要是整理参数并组装ReactElement，所以JSX表达式的结果本来就是JavaScript对象形式的节点描述。真实节点要等后续渲染与补丁流程处理；仅打印元素对象看不到浏览器DOM，不能据此判断失败。

4) 组件传入key、ref、className和多个子节点后，业务代码发现props的形状与原始配置对象不同；createElement做了哪些整理？

答：createElement会单独提取key、ref等保留字段，把普通配置项放入props，并将子节点整理到props.children。一个子节点通常直接保存，多个子节点会组成数组；因此不能把所有JSX属性都假设为组件可读取的普通props。

5) 团队准备自己写Babel插件，把内部XML风格语法转换成React元素；最核心的AST转换动作是什么？

答：插件需要在Babel解析得到AST后访问JSXElement，提取标签、属性和子节点，再把该节点替换为React.createElement调用表达式。难点不只是替换函数名，还要完整保留组件类型、属性表达式和

嵌套子节点；只处理字符串标签会在组件或复杂属性场景失效。

6) 模板方案和JSX方案选型时，有人认为JSX是运行时模板解释器；基于其工作链路，你会怎样说明取舍？

答：JSX是JavaScript的语法扩展，通常在构建阶段转换为React.createElement，并不依赖浏览器把它当模板解释。它减少了额外模板指令概念，并有利于编辑器提示，但仍需要编译工具处理；若运行环境没有对应转换，就必须直接编写普通JavaScript调用。

454. 为什么 React 元素有一个 \$\$typeof 属性

React元素上的\$\$typeof是身份标记，用来区分可信的ReactElement与可能从外部混入的普通对象。它的值使用Symbol，而Symbol不能被JSON序列化，因此服务端或数据库返回的JSON很难直接伪造成合法元素。React检查不到这个标记时会拒绝按元素处理，从而降低旧式元素注入导致XSS的风险。它是一层安全护栏，但不能替代对dangerouslySetInnerHTML内容本身的谨慎处理。目的是为了防止 XSS 攻击。因为Symbol 无法被序列化，所以 React 可以通过有没有 \$\$typeof 属性来断出当前的 element 对象是从数据库来的还是自己生成的。如果没有 \$\$typeof 这个属性，react 会拒绝处理该元素。在 React 的古老版本中，下面的写法会出现 XSS 攻击：

```
// 服务端允许用户存储 JSON let expectedTextButGotJSON = { type: 'div', props: { dangerouslySetInnerHTML: { __html: '/* 把你想的搁着 */' }, }, // ... }; let message = { text: expectedTextButGotJSON }; // React 0.13 中有风险 {message.text}
```

1) 评论页原本只显示后端返回的文本，但接口把一段带type、props和dangerouslySetInnerHTML的JSON对象交给了{message.text}；没有\$\$typeof校验时会发生什么？

答：旧版React可能把这个普通对象误认成元素，并按其中的props创建节点，最终让攻击者控制的HTML进入页面。\$\$typeof相当于元素身份标记；缺少该标记的对象不会作为合法元素处理，但普通文本输出是否安全仍取决于具体渲染入口。

2) 消息中心既接收服务端JSON，又允许业务代码通过React.createElement动态生成提示组件，你会怎样划分两类数据的信任边界？

答：服务端JSON只能作为文本或经过校验的业务数据，不能直接充当待渲染的元素对象；动态组件则应在前端通过JSX或React.createElement创建。这样生成的元素带有React认可的\$\$typeof，但组件类型和危险属性仍不能由用户输入任意决定。

3) 如果运行环境支持把Symbol转成自定义字符串并写入数据库，攻击者能否仅靠伪造同名的"\$\$typeof"字段绕过检查？

答：仅写入同名字符串通常不能等价于React内部使用的Symbol值，因为标准JSON无法表达Symbol。不过若应用在反序列化后主动把外部标记转换为真实标记，就重新打开了信任通道；因此不要为用户数据补造元素身份。

4) 一次发布后，富文本详情页仍出现脚本注入，而监控显示所有可疑对象都没有通过元素身份校验，你会先排查哪些渲染入口？

答：应先搜索dangerouslySetInnerHTML以及把外部内容交给组件类型或属性的路径，确认注入是否绕开了伪造元素这一入口。\$\$typeof只阻止普通JSON冒充React元素，不能替代HTML净化、URL协议校验或服务端输入治理。

5) 安全评审有人主张删掉\$\$typeof，改为递归检查对象是否只有type、props、children，这种方案为什么不可靠？

答：字段白名单只能判断对象的形状，攻击者仍可构造完全符合形状约束的JSON，并把危险内容放进允许的属性中。使用JSON无法序列化的Symbol作为身份标记，能区分前端创建的元素和数据库对象

；代价是它只提供来源识别，不负责内容安全。

455. Virtual DOM 的工作原理是什么

虚拟DOM的工作流程可以概括为用对象描述界面、比较前后差异，再把patch应用到真实DOM。JSX经编译并执行后会生成包含节点类型、props和children的普通对象，这些对象共同组成树。状态变化时，框架对新旧树执行diff，只根据差异更新真实页面，从而减少人为操作DOM的风险。它还便于服务端渲染和跨平台复用，但会占用额外内存，也不意味着所有高性能场景都更合适。虚拟DOM的工作原理是通过JS对象模拟DOM的节点。在Facebook构建React初期时，考虑到要提升代码抽象能力、避免人为的DOM操作、降低代码整体风险等因素，所以引入了虚拟DOM。虚拟DOM在实现上通常是Plain Object，以React为例，在render函数中写的JSX会在Babel插件的作用下，编译为React.createElement执行JSX中的属性参数React.createElement执行后会返回一个Plain Object，它会描述自己的tag类型、props属性以及children情况等。这些Plain Object通过树形结构组成一棵虚拟DOM树。当状态发生变更时，将变更前后的虚拟DOM树进行差异比较，这个过程称为diff，生成的结果称为patch。计算之后，会渲染Patch完成对真实DOM的操作。虚拟DOM的优点主要有三点：改善大规模DOM操作的性能、规避XSS风险、能以较低的成本实现跨平台开发。虚拟DOM的缺点在社区中主要有两点内存占用较高，因为需要模拟整个网页的真实DOM高性能应用场景存在难以优化的情况，类似像Google Earth一类的高性能前端应用在技术选型上往往不会选择React。除了渲染页面，虚拟DOM还有哪些应用场景？这个问题考验面试者的想象力。通常而言，我们只是将虚拟DOM与渲染绑定在一起，但实际上虚拟DOM的应用更为广阔。比如，只要你记录了真实DOM变更，它甚至可以应用于埋点统计与数据记录等。SSR原理借助虚拟dom,服务器中没有dom概念的，react巧妙的借助虚拟dom，然后可以在服务器中nodejs可以运行起来react代码。

1) 商品页只更新购物车角标，有人却说Virtual DOM会直接把整页真实DOM重建一遍；你会怎样纠正这个判断？

答：状态变化后会先得到新的虚拟DOM树，再与旧树执行diff，计算出描述差异的patch，最后把必要变更应用到真实DOM。虚拟树本身通常是描述标签、props和子节点的普通JS对象，但比较和保存这些对象也有计算与内存成本。

2) 你接手一个用JSX写成的活动页，评审要求画出从render到浏览器节点更新的链路，关键步骤应该是什么？

答：JSX会经编译转换为创建元素的调用，执行后产生描述节点的普通对象，并按子节点关系组成虚拟DOM树。状态更新时生成新树，diff得出patch，渲染阶段再据此操作真实DOM；这条链路把声明式描述与具体DOM操作隔开。

3) 同一套资讯卡片既要在浏览器渲染，也计划输出到非DOM宿主环境，虚拟DOM在这个约束变化下提供了什么能力？

答：虚拟DOM是与真实DOM分离的对象描述，因此上层组件逻辑不必直接依赖浏览器节点，底层可以针对不同平台实现对应渲染过程。跨平台并非自动免费获得，各宿主仍需处理属性、事件和生命周期差异，平台特有功能也需要适配。

4) 一个包含大量节点的地图式页面持续掉帧，Profiler显示JS比较和对象分配都很重，但真实DOM更新并不多，你会如何解释并推进排查？

答：瓶颈可能位于虚拟树创建和diff，而不是最终的真实DOM写入；虚拟DOM不保证所有高性能场景都更快。应缩小参与更新的树和更新频率，并评估局部直接绘制或更专用的渲染技术；类似高性能地

图场景可能并不适合由React主导全部渲染。

5) 服务端没有浏览器DOM，却要为用户详情页生成首屏HTML，虚拟DOM为什么仍能参与这条链路？

答：组件先生成与浏览器DOM无关的虚拟节点描述，因此相关代码可以在Node.js环境运行，再由服务端渲染器把描述转换为HTML。它解决的是渲染描述与宿主解耦，浏览器接管后的事件绑定和状态衔接仍需另一阶段完成。

456. React有哪些优化性能的手段

React性能优化的核心是避免无意义的重渲染，并把暂时用不到的代码延后加载。类组件可以用PureComponent或shouldComponentUpdate控制更新，函数组件则常用React.memo；局部计算和回调再按需要交给useMemo、useCallback。列表更新时要使用稳定且唯一的key，避免用数组下标造成错误复用。体积较大的页面还可以通过lazy配合Suspense懒加载，但这些手段应在确有重复渲染或加载成本时使用。

类组件中的优化手段 使用纯组件PureComponent作为基类。使用shouldComponentUpdate生命周期函数来自定义渲染逻辑。 方法组件中的优化手段 使用React.memo高阶函数包装组件，React.memo可以实现类似于shouldComponentUpdate或者PureComponent的效果使用useMemo使用React.useMemo精细化的管控，useMemo 控制的则是是否需要重复执行某一段逻辑，而React.memo控制的是是否需要重渲染一个组件使用useCallback。 使用React.useMemo精细化的管控，useMemo控制的则是是否需要重复执行某一段逻辑，而React.memo 控制的是是否需要重渲染一个组件 其他方式在列表需要频繁变动时，使用唯一 id 作为 key，而不是数组下标。必要时通过改变 CSS 样式隐藏显示组件，而不是通过条件判断显示隐藏组件。使用Suspense和 lazy 进行懒加载，例如：

```
import React, {lazy, Suspense} from "react";
export default class CallingLazyComponent extends React.Component {
  render() {
    var ComponentToLazyLoad = null;
    if (this.props.name === "Mayank") {
      ComponentToLazyLoad = lazy(() => import("./mayankComponent"));
    } else if (this.props.name === "Anshul") {
      ComponentToLazyLoad = lazy(() => import("./anshulComponent"));
    }
    return (
      This is the Base User: {this.state.name} Loading...
    );
  }
}
```

1) 订单列表父组件每次输入筛选词都会创建新数组，团队给所有子组件套了React.memo，但渲染次数没有下降；你会指出哪里判断错了？

答：React.memo控制的是组件在属性可比较为未变化时跳过重渲染，新数组会破坏引用稳定性，使包装失去预期效果。应先确认筛选计算和传入属性是否需要useMemo，但缓存本身也有维护成本，简单计算或持续变化的依赖未必受益。

2) 后台表格有数千条可编辑记录，某个单元格更新会带动整页刷新；在不改业务功能的前提下，你会怎样落地优化？

答：先用React.memo或类组件的PureComponent隔离无关行，并保证列表使用稳定且唯一的业务id作为key。对确实昂贵的派生计算使用useMemo，对需要稳定传递的回调用useCallback；若属性始终变化，这些优化仍会失效。

3) 仪表盘组件切换后必须保留内部输入和滚动位置，产品同时要求隐藏期间减少开销；你会选择条件卸载还是CSS隐藏？

答：需要保留组件实例和内部状态时，可以通过CSS切换显示状态，避免反复卸载与挂载。代价是隐藏组件仍占用DOM和内存，也可能继续响应状态更新；若组件长期不用或资源昂贵，条件卸载通常更合适。

4) 一次重构把列表key从业务id换成数组下标，排序后出现输入框内容跟着错误行移动；你会如何定位并修复？

答：排序改变了下标与业务记录的对应关系，React会按旧key复用节点，从而把局部状态保留在错误条目上。恢复稳定且唯一的业务id，并检查新增、删除和排序链路；只有列表顺序和成员永久不变时，下标风险才相对有限。

5) 首屏包体过大，设置页中的两个低频模块只有用户进入特定分支才需要，你会怎样使用lazy和Suspense，又要防什么问题？

答：可用lazy(() => import(...))延迟加载分支组件，并在外层用Suspense提供加载期间的占位内容。这样把非首屏代码推迟到需要时下载，但首次进入会产生等待；加载边界、失败处理以及组件引用是否有效都需要单独设计。

457. Redux实现原理解析

Redux本质上用单向数据流和唯一的store，把状态变化收口到dispatch(action)与纯函数reducer。createStore内部维护state和监听器，派发后调用reducer生成新状态，再通知订阅者更新视图。异步请求、日志等副作用由中间件拦截action，通过逐层包装dispatch扩展流程。使用时不能直接修改state，否则状态变化会变得不可预测。

在 Redux 的整个工作过程中，数据流是严格单向的。这一点一定要背下来，面试的时候也一定要记得说为什么要用redux。在React中，数据在组件中是单向流动的，数据从一个方向父组件流向子组件（通过props），所以，两个非父子组件之间通信就相对麻烦，redux的出现就是为了解决state里面的数据问题。

Redux设计理念 Redux是将整个应用状态存储到一个地方上称为store,里面保存着一个状态树store tree,组件可以派发(dispatch)行为(action)给store,而不是直接通知其他组件，组件内部通过订阅store中的状态state来刷新自己的视图。如果你想对数据进行修改，只有一种途径：派发 action。action 会被 reducer 读取，进而根据 action 内容的不同对数据进行修改、生成新的 state（状态），这个新的 state 会更新到 store 对象里，进而驱动视图层面做出对应的改变。

Redux三大原则 唯一数据源 整个应用的state都被存储到一个状态树里面，并且这个状态树，只存在于唯一的store中 保持只读状态 state是只读的，唯一改变state的方法就是触发action，action是一个用于描述以发生时间的普通对象

数据改变只能通过纯函数来执行

使用纯函数来执行修改，为了描述action如何改变state的，你需要编写reducers 从编码的角度理解

Redux 工作流程 使用createStore 来完成 store 对象的创建 // 引入 reduximport{createStore}from'redux'// 创建 storeconststore=createStore(reducer,initial_state,applyMiddleware(middleware1,middleware2,...)); createStore 方法是一切的开始，它接收三个入参： reducer；初始状态内容；指定中间件 reducer 的作用是将新的 state 返回给 store 一个 reducer 一定是一个纯函数，它可以有各种各样的内在逻辑，但它最终一定要返回一个 state：

constreducer=(state,action)=>{// 此处是各种样的 state处理逻辑returnnew_state} 当我们基于某个 reducer 去创建 store 的时候，其实就是给这个 store 指定了一套更新规则： // 更新规则全都写在 reducer 里conststore=createStore(reducer) action 的作用是通知 reducer “让改变发生” 要想让 state 发生改变，就必须用正确的 action 来驱动这个改变。

constaction={type:"ADD_ITEM",payload:'text'} action 对象中允许传入的属性有多个，但只有 type 是必传的。type 是 action 的唯一标识，reducer 正是通过不同的 type 来识别出需要更新的不同的 state，由此才能够实现精准的“定向更新”。 派发 action，靠的是 dispatch action 本身只是一个对象，要想让 reducer 感知到 action，还需要“派发 action”这个动作，这个动作是由 store.dispatch 完成的。这里我简单地示范一下：

```
import{createStore}from'redux'// 创建 reducerconstreducer=(state,action)=>{// 此处是各种样的 state处理逻辑returnnew_state}// 基于 reducer 创建 stateconststore=createStore(reducer)// 创建一个 action，这个 action 用 "ADD_ITEM" 来标识constaction={type:"ADD_ITEM",payload:'text'}// 使用 dispatch 派发 action，action 会进入到 reducer 里触发对应的更新store.dispatch(action)
```

以上这段代码，是从编码角度对 Redux ...

1) 购物车页面有人绕过dispatch，直接修改从store.getState()取得的数组，视图偶尔更新、调试记录却没有对应事件；为什么这违背了Redux的工作流？

答：Redux规定状态只读，变更必须由普通action经store.dispatch送入reducer，再生成新的state。直接修改既绕开了订阅通知和行为记录，也破坏了可预测性；即使页面偶然显示正确，时间回溯和问题定位也会失真。

2) 请你在代码评审中补全一个最小createStore：它要支持读取状态、派发行为和订阅更新，dispatch内部的执行顺序应是什么？

答：闭包保存state和监听函数集合，getState返回当前状态，subscribe登记监听者。dispatch(action)先执行reducer(state, action)得到新状态，再依次通知监听者；初始化时还需触发一次reducer，使默认状态得以建立。

3) 支付流程要求dispatch接受函数来发起异步请求，但现有reducer只能消费带type的普通对象；你会把扩展点放在哪里？

答：应通过中间件包装dispatch，让函数类型的action先被类似redux-thunk的逻辑拦截，并在异步完成后派发普通action。reducer继续保持纯函数，只负责根据当前状态和action生成新状态；请求等副作用进入reducer会破坏结果可复现性。

4) 线上接入两个日志与异步中间件后，执行顺序和配置顺序相反，排查applyMiddleware时你会关注哪段函数组合？

答：中间件本质是加工dispatch的工厂，形态可表示为store => next => action，每层把包装后的函数交给下一层。组合时常通过反向构造形成嵌套，最终得到的最外层会最先执行；若某层未调用next(action)，后续中间件和reducer都会被截断。

5) 大型表单拆成多个领域reducer后，团队打算让每个reducer都返回完整状态树；combineReducers应怎样避免这种耦合？

答：combineReducers可遍历reducer映射，把各自对应的state[attr]和同一个action传入，再将返回值组装成新的总状态。每个reducer只管理自己的状态切片，边界更清晰；切片命名或状态结构调整会影响选择器和持久化数据，需要同步迁移。

6) 同事认为action自己会修改状态，因此在actionCreator里直接操作store；你会怎样区分action、reducer和store的职责？

答：action只是描述已发生行为的普通对象，至少需要用于识别行为的type；actionCreator只是生成该对象。store.dispatch负责把它交给reducer，后者计算新状态并由store保存、通知订阅者；把修改逻辑藏进actionCreator会破坏单向数据流。

458. 谈谈你对状态管理的理解

状态管理的核心，是用明确的数据流统一保存和修改跨组件状态，让变化可预测、可追踪。Flux通过Action、Dispatcher、Store到视图的单向流动，解决了数据流向不清的问题；Redux则进一步用单一数据源、只读状态和纯函数reducer约束更新。异步请求等副作用通常交给中间件处理。若更看重少量样板代码和自动响应更新，也可以选择Mobx。

首先介绍 Flux，Flux 是一种使用单向数据流的形式来组合 React 组件的应用架构。Flux 包含了 4 个部分，分别是Dispatcher、Store、View、Action。Store存储了视图层所有的数据，当Store变化后会引起 View 层的更新。如果在视图层触发一个Action，就会使当前的页面数据值发生变化。Action 会被 Dispatcher 进行统一的收发处理，传递给 Store 层，Store 层已经注册过相关 Action 的处理逻辑，处理对应的内部状态变化后，触发 View 层更新。Flux 的优点是单向数据流，解决了 MVC 中

数据流向不清的问题，使开发者可以快速了解应用行为。从项目结构上简化了视图层设计，明确了分工，数据与业务逻辑也统一存放管理，使在大型架构的项目中更容易管理、维护代码。其次是 Redux，Redux 本身是一个 JavaScript 状态容器，提供可预测化状态的管理。社区通常认为 Redux 是 Flux 的一个简化设计版本，它提供的状态管理，简化了一些高级特性的实现成本，比如撤销、重做、实时编辑、时间旅行、服务端同构等。Redux 的核心设计包含了三大原则：单一数据源、纯函数 Reducer、State 是只读的。Redux 中整个数据流的方案与 Flux 大同小异。Redux 中的另一大核心点是处理“副作用”，AJAX 请求等异步工作，或不是纯函数产生的第三方的交互都被认为是“副作用”。这就造成在纯函数设计的 Redux 中，处理副作用变成了一件至关重要的事情。社区通常有两种解决方案：第一类是在 Dispatch 的时候会有一个 middleware 中间件层，拦截分发的 Action 并添加额外的复杂行为，还可以添加副作用。第一类方案的流行框架有 Redux-thunk、Redux-Promise、Redux-Observable、Redux-Saga 等。第二类是允许 Reducer 层中直接处理副作用，采取该方案的有 React Loop，React Loop 在实现中采用了 Elm 中分形的思想，使代码具备更强的组合能力。除此以外，社区还提供了更为工程化的方案，比如 rematch 或 dva，提供了更详细的模块架构能力，提供了拓展插件以支持更多功能。Redux 的优点很多：结果可预测；代码结构严格易维护；模块分离清晰且小函数结构容易编写单元测试；Action 触发的方式，可以在调试器中使用时间回溯，定位问题更简单快捷；单一数据源使服务端同构变得更为容易；社区方案多，生态也更为繁荣。最后是 Mobx，Mobx 通过监听数据的属性变化，可以直接在数据上更改触发 UI 的渲染。在使用上更接近 Vue，比起 Flux 与 Redux 的手动挡的体验，更像开自动挡的汽车。Mobx 的响应式实现原理与 Vue 相同，以 Mobx 5 为分界点，5 以前采用 Object.defineProperty 的方案，5 及以后使用 Proxy 的方案。它的优点是样板代码少、简单粗暴、用户学习快、响应式自动更新数据让开发者的心智负担更低。Mobx 在开发项目时简单快速，但应用 Mobx 的场景，其实完全可以用 Vue 取代。如果纯用 Vue，体积还会更小巧。第一类是在 Dispatch 的时候会有一个 middleware 中间件层，拦截分发的 Action 并添加额外的复杂行为，还可以添加副作用。第一类方案的流行框架有 Redux-thunk、Redux-Promise、Redux-Observable、Redux-Saga 等。第二类是允许 Reducer 层中直接处理副作用，采取该方案的有 React Loop，React Loop 在实现中采用了 Elm 中分形的思想，使代码具备更强的组合能力。除此以外，社区还提供了更为工程化的方案，比如 rematch 或 dva，提供了更详细的模块架构能力，提供了拓展插件以支持更多功能。结果可预测；代码结构严格易维护；模块分离清晰且小函数结构容易编写单元测试；Action 触发的方式，可以在调试器中使用时间回溯，定位问题更简单快捷；单一数据源使服务端同构变得更为容易；社区方案多，生态也更为繁荣。

1) 一个小型资料页只有父子组件传值，架构师却要求所有输入都先经过 Dispatcher 和全局 Store；你会如何评价这种选型？

答：Flux 的价值在于用 Action、Dispatcher、Store 和 View 建立清晰的单向数据流，尤其适合状态关系复杂的应用。简单父子传值强行全局化会增加结构和行为跳转成本；是否引入应由共享范围、变更复杂度和维护需求决定。

2) 协作编辑页面需要统一管理文档、选区和工具栏状态，并希望每次变化都能定位到触发行为；你会怎样组织 Flux 数据流？

答：视图触发 Action，由 Dispatcher 统一分发给已注册处理逻辑的 Store，Store 更新内部状态后通知 View 刷新。这样数据沿单一方向流动，行为入口容易追踪；若把所有局部交互都塞入同一 Store，模块边界仍可能变得模糊。

3) 产品新增撤销、重做和服务端同构要求，现有响应式状态允许任意位置直接赋值；为什么团队可能转向 Redux？

答：Redux 以单一数据源、只读状态和纯函数 Reducer 约束变化，使同一组 action 更容易重放，因此更适合撤销、重做、时间回溯和同构等需求。代价是更新路径更严格，副作用不能随意混入 reducer，

还需额外的中间件或工程封装。

4) 线上偶发状态无法复现，日志中同一个action重放后得到不同结果，而reducer内部读取了时间并发起AJAX；你会怎样修正？

答：应把时间读取和AJAX等副作用移出纯函数reducer，通过dispatch前的中间件处理，再把确定结果封装成普通action。Reducer只依据传入的state与action计算新状态；否则时间旅行、单元测试和故障重放都会失去可信度。

5) 团队在Redux与Mobx之间争论：大型权限后台强调变更审计，原型团队更看重少样板和直接赋值，你会怎样取舍？

答：强调可预测、严格结构和行为追踪时，Redux的action与纯reducer更契合，也便于测试和时间回溯。追求快速开发和自动响应更新时，Mobx的属性监听与直接修改更轻量；代价是隐式依赖增多后，变化来源通常不如显式action清晰。

6) 维护一个旧项目时发现Mobx的响应式层依赖属性拦截，新模块却使用Proxy；这与它和Vue的相邻原理有什么关系？

答：Mobx与Vue的响应式思路相近，都是监听数据属性变化后驱动UI更新；资料所述的分界是Mobx 5以前使用Object.defineProperty，之后使用Proxy。两种机制的能力边界不同，混合维护时应以项目实际版本和构建环境为准，不能只按写法推断。

459. connect组件原理分析

connect是连接React组件与Redux store的高阶函数，负责订阅状态并把数据和派发能力映射成组件的props。Provider先通过context向下传递store，connect再用mapStateToProps和mapDispatchToProps计算组件真正需要的内容。状态或组件自身props变化时，它会重新计算并判断是否需要渲染。组件卸载时还要取消订阅，避免遗留监听。

1. connect用法 作用：连接React组件与Redux store
connect([mapStateToProps],[mapDispatchToProps],[mergeProps],[options])// 这个函数允许我们将store 中的数据作为 props 绑定到组件上const mapStateToProps=(state)=>{return{count:state.count}}这个函数的第一个参数就是Redux的store，我们从中摘取了count属性。你不必将state中的数据原封不动地传入组件，可以根据state中的数据，动态地输出组件需要的（最小）属性函数的第二个参数ownProps，是组件自己的props 当state变化，或者ownProps变化的时候，mapStateToProps都会被调用，计算出一个新的stateProps，（在与ownProps merge后）更新给组件

mapDispatchToProps(dispatch,ownProps):dispatchProps connect的第二个参数是mapDispatchToProps，它的功能是，将action作为props绑定到组件上，也会成为MyComp的 `props`

2. 原理解析 首先connect之所以会成功，是因为Provider组件 在原应用组件上包裹一层，使原来整个应用成为Provider的子组件接收Redux的store作为props，通过context对象传递给子孙组件上的connect

connect做了些什么 它真正连接Redux和React，它包在我们的容器组件的外一层，它接收上面Provider提供的store里面的state和dispatch，传给一个构造函数，返回一个对象，以属性形式传给我们的容器组件

3. 源码 connect是一个高阶函数，首先传入mapStateToProps、mapDispatchToProps，然后返回一个生产Component的函数(wrapWithConnect)，然后再将真正的Component作为参数传入wrapWithConnect，这样就生产出一个经过包裹的Connect组件，该组件具有如下特点

通过props.store获取祖先Component的store props包括stateProps、dispatchProps、parentProps,合并在一起得到nextState，作为props传给真正的ComponentcomponentDidMount时，添加事件this.store.subscribe(this.handleChange)，实现页面交互shouldComponentUpdate时判断是否有避免进行渲染，提升页面性能，并得到nextStatecomponentWillUnmount时移除注册的事件this.handleChange

// 主要逻辑export default function connect(mapStateToProps,mapDispatchToProps,mergeProps,options={}){return function wrapWithConnect(WrappedComponent){class Connect extends Compo

```

nent{constructor(props,context){// 从祖先Component处获得storethis.store=props.store||context.s
torethis.stateProps=computeStateProps(this.store,props)this.dispatchProps=computeDispatchPro
ps(this.store,props)this.state={storeState:null}// 对stateProps、dispatchProps、parentProps进行合
并this.updateState())shouldComponentUpdate(nextProps,nextState){// 进行判断，当数据发生改变
时，Component重新渲染if(propsChanged||mapStateProducedChang...

```

1) 商品列表页把store直接作为一个data属性传给展示组件，团队说这样比写mapStateToProps省事；当无关的登录状态变化时，列表也收到新属性，这种接法的问题在哪？

答：connect的职责不是原样转发整个state，而是通过mapStateToProps摘取组件真正需要的最小数据。传递范围过大会扩大组件与全局状态的耦合，也让无关状态变化更容易进入更新链路；代价是后续拆分、测试和定位渲染来源都会更困难。

2) 订单容器既要读取state.orders，又要根据父组件传入的userId过滤数据，还要暴露reload操作；你会怎样组织connect的几个参数？

答：在mapStateToProps(state, ownProps)中结合userId计算订单属性，在mapDispatchToProps(dispatch, ownProps)中封装需要派发的操作。最后由默认或自定义mergeProps合并stateProps、dispatchProps与父级属性；合并规则越复杂，越要防止同名属性被意外覆盖。

3) 同一个详情组件要挂在两个不同的Provider下，父组件还可能通过props.store显式传入另一个store；此时包装层应该从哪里取状态，更新订阅又跟着谁？

答：按给出的实现，包装层优先读取props.store，否则才从context获取祖先Provider提供的store。状态读取、dispatch和subscribe必须绑定同一个实例，否则页面可能展示一个仓库的数据却监听另一个仓库；显式覆盖因此需要非常谨慎。

4) 线上出现“ReduxDevTools里状态已经变化，但被connect包装的库存组件没有刷新”，你会沿着包装组件的哪条链路排查？

答：先确认Provider传入的store是否正确，再检查挂载时是否执行了store.subscribe，订阅回调有没有通过setState触发包装层更新。随后核对mapStateToProps是否产出变化，以及shouldComponentUpdate是否错误拦截；卸载过早或订阅错误实例也会造成同样现象。

5) 团队想把所有mapStateToProps、mapDispatchToProps和父级属性都手工塞进一个对象再传下去，认为这样更可控；什么时候值得写mergeProps，什么时候应保持默认合并？

答：只有属性需要重命名、组合或根据父级参数形成特定接口时，才值得提供mergeProps。普通场景保留默认合并更容易追踪数据来源，也减少每次更新时的额外计算；自定义合并若持续产生不稳定结果，还会削弱包装层避免无效渲染的能力。

6) 类组件页面不能直接读取context中的Redux数据，架构师建议每个组件都手写订阅；connect相比这种方案补齐了哪些生命周期责任？

答：connect把取store、计算状态与派发属性、订阅变化、判断更新和向真实组件传参集中在高阶组件中。组件挂载时注册订阅，卸载时还应移除订阅，避免遗留监听；手写方案若漏掉清理或更新判断，很容易产生泄漏或冗余渲染。

460. React Hooks

React Hooks让函数组件能够使用状态和副作用，并把相关逻辑抽成可复用的自定义Hook。它解决了类组件中生命周期拆散业务逻辑、this使用复杂，以及HOC或render props容易造成嵌套的问题。由于内部按调用顺序保存Hook，只能在函数组件或自定义Hook顶层调用，不能放进条件、循环或嵌套函数。普通副作用优先用useEffect，需要同步读取或调整DOM、避免闪烁时再用useLayoutEffect。

代码逻辑聚合，逻辑复用HOC嵌套地狱代替class React 中通常使用 类定义 或者 函数定义 创建组件：在类定义中，我们可以使用到许多 React 特性，例如 state、各种组件生命周期钩子等，但是在函数定义中，我们却无能为力，因此 React 16.8 版本推出了一个新功能 (React Hooks)，通过它，可以更好的在函数定义组件中使用 React 特性。

函数组件与类组件的对比：无关“优劣”，只谈“不同” 类组件需要继承 class，函数组件不需要；类组件可以访问生命周期方法，函数组件不能；类组件中可以获取到实例化后的 this，并基于这个 this 做各种各样的事情，而函数组件不可以；类组件中可以定义并维护 state（状态），而函数组件不可以；但是类组件它太重了，对于解决许多问题来说，编写一个类组件实在是一个过于复杂的姿势。复杂的姿势必然带来高昂的理解成本，这也是我们所不想看到的 react hooks的好处：跨组件复用：其实 render props / HOC 也是为了复用，相比于它们，Hooks 作为官方的底层 API，最为轻量，而且改造成本小，不会影响原来的组件层次结构和传说中的嵌套地狱；类定义更为复杂 不同的生命周期会使逻辑变得分散且混乱，不易维护和管理；时刻需要关注this的指向问题；代码复用代价高，高阶组件的使用经常会使整个组件树变得臃肿； 状态与UI隔离： 正是由于 Hooks 的特性，状态逻辑会变成更小的粒度，并且极容易被抽象成一个自定义 Hooks，组件中的状态和 UI 变得更为清晰和隔离。 注意：避免在 循环/条件判断/嵌套函数 中调用 hooks，保证调用顺序的稳定；只有 函数定义组件 和 hooks 可以调用 hooks，避免在 类组件 或者 普通函数 中调用；不能在useEffect中使用useState，React 会报错提示；类组件不会被替换或废弃，不需要强制改造类组件，两种方式能并存； 重要钩子 状态钩子 (useState)： 用于定义组件的 State，其到类定义中this.state的功能； // useState 只接受一个参数：初始状态// 返回的是组件名和更改该组件对应的函数const[flag,setFlag]=useState(true);// 修改状态setFlag(false)// 上面的代码映射到类定义中:this.state={flag:true}constflag=this.state.flagconstsetFlag=(bool)=>{this.setState({flag:bool,})} 生命周期钩子 (useEffect)： 类定义中有许多生命周期函数，而在 React Hooks 中也提供了一个相应的函数 (useEffect)，这里可以看做componentDidMount、componentDidUpdate和componentWillUnmount的结合。

useEffect(callback, [source])接受两个参数 callback： 钩子回调函数；source： 设置触发条件，仅当 source 发生改变时才会触发；useEffect钩子在没有传入[source]参数时，默认在每次 render 时都会优先调用上次保存的回调中返回的函数，后再重新调用回调； useEffect(()=>{// 组件挂载后执行事件绑定console.log('on')addEventListener()// 组件 update 时会执行事件解绑return()=>{console.log('off')removeEventListener()},{source});// 每次 source 发生改变时，执行结果(以类定义的生命周期，便于大家理解):// --- DidMount ---// 'on'// --- DidUpdate ---// 'off'// 'on'// --- DidUpdate ---// 'off'// 'on'// --- WillUnmount ---// 'off' 通过第二个参数，我们便可模拟出几个常用的生命周期: componentDidMount: 传入[]时...

1) 筛选面板只有展开时才执行useEffect订阅快捷键，收起时跳过；测试发现后面的useState偶尔读到别的状态，为什么不能把Hook放进if？

答：Hook依赖每次渲染中稳定的调用顺序，条件分支会使内部链表的取值位置发生错位。应始终调用Hook，把条件放到副作用内部，或把展开区域拆成独立组件；循环、嵌套函数中的条件调用同样存在风险。

2) 一个报表页把订阅、标题更新和请求状态全塞进类组件的多个生命周期，产品又要求三个页面复用；你会怎样用自定义Hook重组？

答：把彼此关联的状态和副作用分别抽成useSubscription、useTitle等自定义Hook，让每段业务逻辑在一个位置维护并跨组件复用。自定义Hook内仍可调用useState、useEffect，但自身也必须遵守顶层调用规则；拆得过碎会增加依赖关系的理解成本。

3) 聊天室的useEffect原来依赖[roomId]，有人为减少请求改成[]；用户在同一页面切换房间后仍收到旧房间消息，你会如何修正执行与清理时机？

答：订阅确实随roomId变化，因此依赖项应保留[roomId]。每次切换时，React会先执行上一次回调返回的退订函数，再建立新订阅，卸载时也会清理；改成[]只适用于整个挂载期间参数不会变化的副作用。

4) 线上弹窗打开时偶发绑定两次滚动监听，关闭后仍触发回调；相关代码用了没有依赖数组的useEffect，你会先查什么？

答：先检查副作用是否每次渲染都重新绑定，以及回调是否返回了与本次绑定对应的移除函数。没有依赖数组时，每次渲染会先清理旧副作用再执行新副作用；若清理缺失、函数引用不匹配或绑定逻辑不对称，就会残留监听。

5) 可视化页面需要读取更新后的DOM尺寸并立即调整样式，useEffect下会短暂闪动；同事提议所有副作用统一换成useLayoutEffect，你怎么取舍？

答：这类必须在DOM变更后同步测量并调整样式的逻辑适合useLayoutEffect，可减少绘制后的视觉闪动。普通订阅、日志和数据副作用仍优先使用useEffect；useLayoutEffect会同步阻塞后续绘制，放入耗时计算会拖慢页面呈现。

6) 编辑器把回调传给多个依赖组件，父组件每次渲染都创建新函数；评审同时提出useCallback、useMemo和useRef，三者在这里分别解决什么？

答：需要稳定回调引用时使用useCallback，需要缓存计算结果或传入属性时使用useMemo，访问真实节点或保存可变引用则使用useRef。它们用途不同，不能互相替代；缓存本身也增加依赖维护成本，应围绕明确的重复渲染或计算问题使用。

461. 受控组件和非受控组件

受控组件的值由React管理，非受控组件的值则由DOM自己保存。同时传入value和onChange时，组件的输入与更新都由外部控制；只传defaultValue，通常就是给非受控组件设置初始值。受控方式更方便统一管理和同步数据，非受控方式脱离了组件自身的state或props控制，一般不建议作为常规方案。// 上面的是受控组件 下面的是非受控组件 当你一个组件同时传递一个value以及onChange事件时，它就是一个受控组件，输入输出都是我控制的。第二个只是传递了默认的初时值，并没有传onChange事件，非受控组件是一种反模式，它的值不受组件自身的state或props控制

1) 登录页给却没有传onChange，测试说输入框怎么敲都不变；开发认为删掉value就算修好，你怎么看？

答：传入value意味着显示值由外部控制，没有对应的onChange更新来源时，用户输入无法形成新的受控值。应补上变更回调并更新状态；若改用defaultValue，则只是交给组件内部维护，父级也就失去持续控制能力。

2) 后台编辑页有姓名、部门和角色三个字段，保存按钮必须随任一字段变化更新可用状态；你会让输入组件接收哪些属性？

答：这类实时联动适合受控方式：父级持有字段值，向每个输入传入value和onChange，变更后统一计算保存条件。数据入口和出口都由父级掌握，校验与提交更直接；代价是父级需要维护完整的状态更新链路。

3) 搜索框原来只需要一个初始关键词，改版后路由参数变化时必须立即覆盖输入内容；继续使用defaultValue={keyword}能满足吗？

答：defaultValue只表达初始值，后续路由参数变化不应依赖它持续驱动输入框。需要外部参数随时覆盖显示内容时，应改为value={keyword}并提供onChange更新来源；切换模式时还要避免同时混

用两套值来源。

4) 线上表单从接口数据未返回时的undefined，切换成返回后的字符串，控制台提示输入组件控制方式发生变化；你会怎样定位？

答：先检查首屏是否没有传value，数据返回后才开始传入，导致组件从内部维护切到外部控制。若决定受控，应从首次渲染就提供稳定值，例如空字符串，并始终配套onChange；若决定非受控，则只传defaultValue，不要中途切换。

5) 内容发布页只在点击提交时读取标题，产品没有实时校验、联动或外部重置要求；受控和非受控怎么选？

答：仅需初始值且提交时再读取的场景，可以采用defaultValue，让输入值不受父级state或props持续控制。若随后加入实时校验、跨字段联动或外部覆盖，就更适合受控方式；选择取决于谁必须成为稳定的数据源。

462. 如何避免ajax数据请求重新获取

避免重复请求的核心是缓存数据并对相同请求去重，通常优先交给React Query或SWR处理。它们能按请求键复用缓存、合并并发请求，并在数据过期后重新验证，比把服务端数据塞进Redux或Zustand更省维护成本。组件里还要正确设置useEffect依赖，并用AbortController取消过期请求；不常变化的接口也可以配合Cache-Control或ETag。核心思路是缓存 +

请求去重——把已经拿到的数据缓存起来，相同的请求不重复发。

用数据请求库管理缓存（现代主线）：React Query / SWR内置了缓存、请求去重（同一时刻多个组件请求同一 key 只发一次）、后台重新验证（stale-while-revalidate）、失效控制。这是现在处理「服务端状态」的标准做法——避免重复请求本就是它们的核心能力，不用自己造轮子。放全局状态（旧做法）：把数据存进 Redux/Zustand，组件先读缓存、没有再请求。这是过去常见的做法，但要自己处理缓存失效、去重、更新，样板多——现在更推荐用 React Query 管服务端状态、全局 store 只放真正的客户端状态。请求去重（in-flight dedup）：对「正在进行中」的相同请求做去重——用一个 Map 缓存进行中的 Promise，相同 key 直接返回同一个 Promise，避免并发的重复请求。HTTP 缓存：对可缓存的接口配Cache-Control/ETag，让浏览器层面命中缓存（适合不常变的数据）。组件层面：用useEffect的依赖数组控制只在必要时请求、用AbortController取消过期请求避免竞态。一句话：优先用 React Query/SWR（自带缓存 + 去重 + 重验证），它们专门解决「服务端状态」的重复获取；全局 store 只放客户端状态；再配合请求去重和 HTTP 缓存。

1) 商品详情页三个区域同时请求/product?id=42，抓包看到首屏并发三次；只把最后一次结果存进缓存，为什么仍挡不住重复请求？

答：结果缓存只有请求完成后才能命中，三个调用在完成前仍会各自发出请求。请求层应按包含参数的key把进行中的Promise放进Map，后续调用复用它；结束后还要按缓存策略清理或转存结果，避免把失败请求永久保留。

2) 列表页返回时产品要求先展示旧数据、再静默校准库存，团队准备自己维护过期时间、重试和并发合并；你会怎么落地？

答：优先使用React Query或SWR，利用缓存先呈现已有数据，再通过后台重新验证更新库存。它们也覆盖请求去重与失效控制，能减少自建状态机；团队仍需定义合理的key和数据失效边界，否则不同筛选条件可能误用同一缓存。

3) 新闻接口从“几乎不变”改成“运营每分钟修订”，后端却设置了长时间Cache-Control；前端继续依赖浏览器缓存会有什么风险，怎么调整？

答：浏览器缓存适合不常变化的数据，更新频率提高后，过长的缓存时间会让用户持续看到旧内容。应由服务端调整Cache-Control，或使用ETag让浏览器重新验证；前端请求库的失效策略也要同步，否则两层缓存会产生不同的新鲜度判断。

4) 用户快速切换status筛选，慢的旧请求最后返回并覆盖了新结果；页面还偶发重复请求，你会按什么顺序排查？

答：先核对useEffect依赖是否准确反映status，避免无关渲染重复触发或漏掉必要请求。再用AbortController取消已经过期的请求，防止旧响应覆盖新状态；请求去重只能合并相同key，不能替代不同筛选条件间的竞态处理。

5) 架构评审争论把所有接口结果放进Redux，还是引入React

Query；订单数据需要缓存、失效、重试，主题和侧栏状态只在客户端存在，你怎么划分？

答：订单数据属于服务端状态，优先交给React Query或SWR管理缓存、去重、重验证与失效。主题和侧栏这类客户端状态再放入全局store；把接口数据统一塞进Redux也能工作，但团队必须自行补齐更新和缓存生命周期。

6) 接口已经支持ETag，前端又用了SWR，同事认为两者重复只能留一个；在一次返回列表页的流程里，它们各自拦截什么？

答：SWR在应用层按key复用数据和进行中的请求，并可先展示旧数据后重新验证；ETag则让重新访问服务器时验证资源是否变化，可能避免重复传输完整响应。两层可以协作，但缓存时效要协调，否则应用层不发请求时不会触发HTTP验证。

463. 组件之间通信

组件通信通常按关系远近选择：父子之间用props和回调，跨多层用Context，复杂公共状态再考虑状态管理库。父组件可以把状态传给子组件，子组件则通过回调把输入或事件交回父组件，兄弟组件一般把状态提升到共同父级。主题、语言这类跨层信息适合Context；组件树两侧共享的复杂数据可用Redux等方案，但会增加学习和结构成本。需要父组件直接调用子组件方法时，也可以通过ref获取实例，不过这是比较特殊的通信方式。

父子组件通信自定义事件redux和context

context如何运用

父组件向其下所有子孙组件传递信息如一些简单的信息：主题、语言复杂的公共信息用redux

在跨层级通信中，主要分为一层或多层的情况 如果只有一层，那么按照 React 的树形结构进行分类的话，主要有以下三种情况：父组件向子组件通信，子组件向父组件通信以及平级的兄弟组件间互相通信。在父与子的情况下，因为 React 的设计实际上就是传递Props即可。那么场景体现在容器组件与展示组件之间，通过Props传递state，让展示组件受控。在子与父的情况下，有两种方式，分别是回调函数与实例函数。

回调函数，比如输入框向父级组件返回输入内容，按钮向父级组件传递点击事件等。实例函数的情况有些特别，主要是在父组件中通过

React

的

ref

API

获取子组件的实例，然后通过实例调用子组件的实例函数。这种方式在过去常见于 Modal 框的显示与隐藏多层级间的数据通信，有两种情况。第一种是一个容器中包含了多层子组件，需要最底部的子组件与顶部组件进行通信。在这种情况下，如果不断透传

Props

或回调函数，不仅代码层级太深，后续也很不好维护。第二种是两个组件不相关，在整个

React

的组件树的两侧，完全不相交。那么基于多层级间的通信一般有三个方案。第一个是使用

React

的Context

API，最常见的用途是做语言包国际化第二个是使用全局变量与事件。第三个是使用状态管理框架，比如Flux、Redux

及

Mobx。优点是由于引入了状态管理，使得项目的开发模式与代码结构得以约束，缺点是学习成本相对较高

第一个是使用

React

的Context

API，最常见的用途是做语言包国际化第二个是使用全局变量与事件。第三个是使用状态管理框架，比如

Mobx。优点是由于引入了状态管理，使得项目的开发模式与代码结构得以约束，缺点是学习成本相对较高

1) 商品卡片的收藏按钮点击后要通知父列表更新计数，开发准备用ref调子组件实例再反向取值；这条只有一层的通信为什么不合适？

答：一层子到父通信应由父组件传入回调，子组件在点击时把结果作为参数回传，数据流更清晰。通过ref获取实例并调用方法属于更特殊的命令式场景，过去常见于弹窗显隐；用它传普通业务数据会增加父子实现耦合。

2) 结算页的优惠金额由父组件计算，价格展示组件只负责渲染，数量组件还要把修改结果传回去；你会怎样设计属性边界？

答：父组件通过props把优惠金额和数量传给展示组件，再把数量变更回调传给数量组件。子组件触发回调后由父级更新状态并重新下发，形成单向数据流；不要让两个子组件私下同步副本，否则容易出现价格与数量不一致。

3) 主题值要从应用根节点传到第六层按钮，中间五层都不用它；设计师又要求切换主题后整棵页面同步，你会选Context还是逐层props？

答：这种面向一批子孙组件的简单公共信息适合Context，根节点提供主题，深层按钮直接消费，避免无意义的逐层透传。若信息演变为复杂且频繁变化的业务状态，则应重新评估状态管理框架；Context不等于所有全局状态的默认容器。

4) 两个位于组件树两侧的面板通过全局事件同步筛选条件，线上偶尔出现页面已卸载但监听仍执行；你会查哪里并考虑怎样替换？

答：先检查事件监听是否在挂载时注册、卸载时解除，以及同一组件是否重复注册。对于跨层级且互不相交的复杂共享状态，可改用Redux、Mobx等状态管理框架，让更新路径受到统一约束；全局事件虽直接，但生命周期和数据来源更难追踪。

5) 小型后台只有语言和主题两个公共值，架构师要求一开始就上Redux，另一位坚持所有层级都传props；你如何裁决？

答：语言和主题属于Context的典型用途，可由上层统一提供并让后代消费，不必为了简单信息引入完整状态管理框架。逐层props在层级很浅时仍最直观；只有公共状态复杂度和协作约束明显上升时，Redux等方案的结构化收益才更突出。

6) 父组件需要控制第三方弹窗的打开和关闭，但该弹窗只暴露实例方法；团队规定所有子到父通信都必须用回调，这里是否需要例外？

答：若第三方弹窗确实只提供命令式实例方法，父组件可通过React的ref获取实例并调用显示或隐藏方法。它适用于这类特殊控制，不应扩展成常规数据通信机制；能通过props和回调表达的状态，仍应优先保持声明式传递。

464. 类组件与函数组件有什么区别呢？

类组件和函数组件在展示及现代浏览器中的性能没有明显差异，真正不同的是开发时的心智模型。类组件围绕面向对象、this和生命周期组织代码，函数组件则更强调不可变数据、减少副作用，并通过Hooks组织和复用逻辑。性能优化时，类组件常用shouldComponentUpdate，函数组件可用React.memo；随着Hooks出现，函数组件已经能覆盖原先依赖生命周期的场景。组件设计也更推崇组合而非继承，因此函数组件通常更符合后续的并发与细粒度逻辑复用方向。作为组件而言，类组件与函数组件在使用与呈现上没有任何不同，性能上在现代浏览器中也不会有明显差异它们在开发时的心智模型上却存在巨大的差异。类组件是基于面向对象编程的，它主打的是继承、生命周期等核心概念；而函数组件内核是函数式编程，主打的是immutable、没有副作用、引用透明等特点。之前，在使用场景上，如果存在需要使用生命周期的组

件，那么主推类组件；设计模式上，如果需要使用继承，那么主推类组件。但现在由于 React Hooks 的推出，生命周期概念的淡出，函数组件可以完全取代类组件。其次继承并不是组件最佳的设计模式，官方更推崇“组合优于继承”的设计概念，所以类组件在这方面的优势也在淡出。性能优化上，类组件主要依靠 `shouldComponentUpdate` 阻断渲染来提升性能，而函数组件依靠 `React.memo` 缓存渲染结果来提升性能。从上手程度而言，类组件更容易上手，从未来趋势上看，由于 React Hooks 的推出，函数组件成了社区未来主推的方案。类组件在未来时间切片与并发模式中，由于生命周期带来的复杂度，并不易于优化。而函数组件本身轻量简单，且在 Hooks 的基础上提供了比原先更细粒度的逻辑组织与复用，更能适应 React 的未来发展。

1) 商品详情页把类组件改成函数组件后，产品认为页面表现和性能都会明显提升，你会怎样纠正这个判断？

答：单纯改写组件形态通常不会带来可感知的展示或性能提升，两者作为组件时的使用结果并无本质差异。迁移价值主要来自 Hooks 带来的细粒度逻辑组织与复用，以及更适合后续并发能力的模型；若现有代码稳定且没有复用诉求，不应把重写包装成性能优化。

2) 一个类组件同时处理请求、订阅和表单校验，多个生命周期里散落着同一业务逻辑，准备迭代时你会怎样落地改造？

答：适合在本次业务迭代中改成函数组件，并按业务关注点拆出自定义 Hook，让请求、订阅和校验各自聚合相关状态与清理逻辑。不要机械地把每个生命周期翻译成一个 `useEffect`，否则只是换了语法；依赖项和副作用边界不清时，迁移后仍会难以维护。

3) 后台管理系统大量使用基类继承权限校验，新模块坚持继续继承，另一位开发主张全面改成组合，你会支持哪一边？

答：新模块应优先使用组合，把权限能力做成包装组件、Context 或自定义 Hook，而不是继续扩大组件继承层级。React 的组件复用更适合“组合优于继承”，函数组件也因此削弱了类组件的传统优势；旧基类可渐进兼容，但一次性替换会扩大回归范围。

4) 函数组件接入实时行情后偶发重复订阅，类组件版本却正常；日志显示每次渲染后订阅数量都增加，你先查哪里？

答：先检查副作用是否写在渲染过程，或 `useEffect` 是否缺少正确依赖与清理函数，确保每次重新订阅前都会解除旧订阅。函数组件强调渲染无副作用，副作用应集中在对应的 Hook 中；若依赖对象每次都重新创建，还需稳定引用，否则清理正确也可能频繁重订阅。

5) 数据表格频繁刷新，类组件方案要写 `shouldComponentUpdate`，函数组件方案准备套 `React.memo`，你会如何决定并验证？

答：两种方案都能阻断不必要的渲染：类组件使用 `shouldComponentUpdate`，函数组件通常使用 `React.memo` 缓存渲染结果。应先确认瓶颈来自重复渲染，再保证传入的对象和回调引用足够稳定；盲目增加比较逻辑也有成本，且无法修复组件内部昂贵计算。

465. 如何设计 React 组件

设计 React 组件的核心是单一职责、接口精简，并根据业务边界合理分层。我一般把纯展示和通用能力放进基础组件，把状态与业务逻辑留给容器组件或页面，避免为了复用而过度抽象。组件之间优先通过 `props`、`children` 等方式组合，接口命名要有语义，也要考虑受控与非受控场景。基础组件还应补齐 TypeScript 类型、无障碍能力、单测和 Storybook 文档。React 组件应从设计与工程实践两个方向进行探讨从设计上而言，社区主流分类的方案是展示组件与灵巧组件 展示组件内部没有状态管理，仅仅用于最简单的展示表达。展示组件中最基础的一类组件称作代理组件。代理组件常用于封装常用属性、减少重复代码。很经典的场景就是引入 `Antd` 的 `Button` 时，你再自己封一层。如果未来需要替换掉 `Antd`

或者需要在所有的 Button 上添加一个属性，都会非常方便。基于代理组件的思想还可以继续分类，分为样式组件与布局组件两种，分别是将样式与布局内聚在自己组件内部。从工程实践而言，通过文件夹划分的方式切分代码。我初步常用的分割方式是将页面单独建立一个目录，将复用性略高的 components 建立一个目录，在下面分别建立 basic、container 和 hoc 三类。这样可以保证无法复用的业务逻辑代码尽量留在 Page 中，而可以抽象复用的部分放入 components 中。其中 basic 文件夹放展示组件，由于展示组件本身与业务关联性较低，所以可以使用 Storybook 进行组件的开发管理，提升项目的工程化管理能力

1) 订单页把请求、权限判断和几十行表格结构都写进一个所谓“展示组件”，评审时你会指出什么设计矛盾？

答：它已经不是纯展示组件，因为数据获取和权限判断属于业务协调或状态管理，应该由页面或容器层承担。展示组件只接收明确数据并表达界面，才能低成本复用和独立验证；如果强行追求绝对无状态，也可能把紧密相关的交互拆得过散。

2) 六个页面都直接使用Antd

Button，现在设计要求统一补埋点并可能更换组件库，你会怎样调整目录和依赖？

答：应增加项目自己的按钮代理组件，集中封装常用属性、样式和埋点，再让业务页面依赖这层稳定接口。通用展示组件可放入components/basic并用Storybook管理；代理层若暴露过多底层专有属性，未来替换组件库仍会产生大面积修改。

3) 团队规定所有组件必须放进公共components，结果两百多个只服务单一页面的组件也被共享，你会如何重新划分？

答：无法复用且高度依赖业务流程的代码应留在对应Page目录，只有抽象稳定、复用性较高的内容才进入公共组件目录。可进一步按basic、container和hoc等职责分类，使依赖方向清楚；过早公共化会固化尚未稳定的接口，并增加跨页面修改风险。

4) 改动会员卡样式后十几个页面错位，排查发现每页都复制了相同布局和间距，你会怎样止损并定位抽象边界？

答：先把重复的样式与布局收敛为样式组件或布局组件，再逐页替换，避免继续复制并分别修补。组件内部应内聚稳定的视觉规则，页面只提供业务内容；若不同页面的结构差异已经超过共同点，硬合并会制造大量条件属性，应保留少量明确变体。

5) 搜索页组件已经拆成三十个文件，阅读一次交互要跨越多层props，但架构师仍要求继续细拆，你会用什么标准取舍？

答：拆分应围绕职责、复用价值和封装边界，而不是文件数量或代码行数。稳定的展示单元可进入basic，负责数据与状态协调的部分留在容器或页面层；粒度过细会让父子关系和数据传递更难理解，粒度过粗则会把业务逻辑与展示再次绑死。

466. 组件的协同及（不）可控组件

组件协同本质上是组织父子关系和复用公共逻辑，而可控与不可控的区别在于状态由谁管理。父组件可以通过嵌套和props统一管理子组件，也可以抽离多个组件共有的方法，减少重复代码，但抽象过多会让逻辑分散、维护难度上升。可控组件的值来自外部状态，符合React数据流，也更方便读取和处理。不可控组件通常用defaultValue初始化，后续需要借助ref获取实际值。为什么要进行组件的协同 我们在实际的开发项目的时候，不会只用几个组件，有时候遇到大型的项目，可能会有成千上百的组件，难免会遇到有功能重复的组件。要进行修改，就会修改大部分的文件。所以我们需要进行组件的协同开发。

什么是组件的协同使用？ 组件的协同本质上是对组件的一种组织、管理的方式。目的：逻辑清晰：这是组件与组件之间的逻辑代码模块化封装细节：像面向对象一样将常用的方法以及数据封装起来提高代码的复

用性：因为是组件，相当于一个封装好的东西，用的时候直接调用 逻辑清晰：这是组件与组件之间的逻辑
代码模块化封装细节：像面向对象一样将常用的方法以及数据封装起来提高代码的复用性：因为是组件，
相当于一个封装好的东西，用的时候直接调用 如何实现组件的协同使用 第一种：增加一个父组件，将其他
的组件进行嵌套，更多的是实现代码的封装第二种：通过一些操作从后台获取数据，React中的Mixin，更
多的是实现代码的复用 组件嵌套的含义 组件嵌套的本质是父子关系 组件嵌套的优缺点 优点：逻辑清晰：
父子关系类似于人类中的父子关系模块化开发：每个模块对应一个功能，不同的模块可以同步开发封装细
节：开发者必须要关注组件的功能，不需要了解细节缺点：编写难度高：父子组件的关系需要经过深思熟
虑，贸然编写可能导致关系混乱，代码难以维护无法掌握所有细节：使用者只知道组件的用法，不知道实
现细节，遇到问题难以修复 逻辑清晰：父子关系类似于人类中的父子关系模块化开发：每个模块对应一个
功能，不同的模块可以同步开发封装细节：开发者必须要关注组件的功能，不需要了解细节 编写难度高：
父子组件的关系需要经过深思熟虑，贸然编写可能导致关系混乱，代码难以维护无法掌握所有细节：使用
者只知道组件的用法，不知道实现细节，遇到问题难以修复 Mixin Mixin的含义
Mixin=一组方法。他的目的是横向抽离出组件的相似代码，把组件的共同作用以及效果的代码提出来
Mixin的优缺点 优点代码复用：抽离出通用的代码，减少开发成本，提高开发效率即插即用：可以使用许多
现有的Mixin来开发自己的代码适应性强：改动一次代码，影响多个组件缺点编写难度高：Mixin可能被用在
各种环境中，想要兼容多种环境就需要更多的 - 码与逻辑，通用的代价是提高复杂度降低代码的可读性：组
件的优势在于将逻辑与是界面直接结合在一起，Mixin本质上会分散逻辑，理解起来难度大 代码复用：抽离
出通用的代码，减少开发成本，提高开发效率即插即用：可以使用许多现有的Mixin来开发自己的代码适应
性强：改动一次代码，影响多个组件
编写难度高：Mixin可能被用在各种环境中，想要兼容多种环境就需要更多的 - 码与逻辑，通用的代价是提
高复杂度降低代码的可读性：组件的优势在于将逻辑与是界面直接结合在一起，Mixin本质上会分散逻辑，
理解起来难度大 不可控组件 上图：defaultValue的值是固定的，这就是一个不可控组件如果要获取input
的value值，只有使用ref获取节点来获取值 可控组件 defaultValue的值是根据状态确定了，只需要拿到thi
s.state.value的值就可以了这里需要注意一下：使用value的值是不可修改的，defaultValue的值是可以
修改的 可控组件的优点 符合React的数据流数据存储在state中，便于获取便于处理数据

1) 注册页的手机号输入框传了defaultValue，用户编辑后父组件更新默认值，界面却没有跟着变化，为什么？

答：defaultValue只负责非受控输入框的初始值，挂载后输入内容由DOM自己维护，父组件随后修改默认值不会持续驱动界面。若业务要求外部状态始终决定内容，应改用value配合onChange；这样也意味着父组件必须及时更新状态，否则输入框会表现为不可编辑。

2) 结算页要求地址输入变化后立即联动运费、校验和提交按钮状态，你会选受控还是非受控组件？

答：应优先选择受控组件，把输入值保存在React的state中，并通过value和变更回调形成单向数据流。这样联动逻辑能读取同一份确定数据，也便于统一校验；字段数量很大或变更非常频繁时，需要控制状态作用域，避免无关区域跟随渲染。

3) 一个文件上传控件需要读取原生节点状态，团队却要求所有表单项都必须用value控制，你会怎样处理这组选型冲突？

答：不必把所有输入都强行设计成受控模式，无法由普通value完整驱动或更适合读取DOM状态的控件可使用非受控方案，并通过ref获取节点值。组件应封装这些实现细节，对外暴露清晰事件；代价是数据不再始终位于React状态中，联动和即时校验会更困难。

4) 筛选面板上线后偶发出现“输入框显示值与提交参数不一致”，代码同时通过ref读DOM，又在state中保存一份值，你先怎么排查？

答：先确认界面的唯一数据源，检查渲染使用的是value、defaultValue还是DOM当前值，并对照提交路径读取了哪一份数据。受控和非受控机制混用容易产生两个不同步的状态源；应统一为state驱动，或明确只在提交时通过ref读取，避免双向同步。

5) 十几个组件都要复用请求和日志逻辑，旧方案准备继续追加Mixin，新方案主张父组件协调，你会如何取舍？

答：组件间的结构协作可由父组件嵌套和组织，使父子关系、数据来源与封装边界保持清晰。Mixin能横向复用一组方法，但会把逻辑分散到组件之外，兼容场景越多复杂度越高；遗留代码可维护，新增逻辑则应优先选择边界更显式的组合方式。

467. React-Router 的实现原理及工作方式分别是什么

React Router的实现原理是监听地址变化并匹配对应组件，工作时则通过上下文连接路由容器、匹配组件和导航组件。Hash模式依赖哈希变化，History模式使用pushState、replaceState等浏览器API，后者还要求服务端配置historyApiFallback。浏览器端使用History API，原生端可以使用基于数组维护记录的内存历史。像Router提供上下文，Route负责匹配，Link则完成平台相关的导航。 React

Router路由的基础实现原理分为两种，如果是切换

Hash的方式，那么依靠浏览器Hash变化即可；如果是切换网址中的Path，就要用到HTML5 History API中的pushState、replaceState等。在使用这个方式时，还需要在服务端完成historyApiFallback配置在React Router内部主要依靠history库完成，这是由React Router自己封装的库，为了实现跨平台运行的特性，内部提供两套基础history，一套是直接使用浏览器的History API，用于支持react-router-dom；另一套是基于内存实现的版本，这是自己做的一个数组，用于支持react-router-native。React Router的工作方式可以分为设计模式与关键模块两个部分。从设计模式的角度出发，在架构上通过Monorepo进行库的管理。Monorepo具有团队间透明、迭代便利的优点。其次在整体的数据通信上使用了 Context API完成上下文传递。在关键模块上，主要分为三类组件：第一类是 Context 容器，比如 Router 与 MemoryRouter；第二类是消费者组件，用以匹配路由，主要有 Route、Redirect、Switch 等；第三类是与平台关联的功能组件，比如Link、NavLink、DeepLinking等。 React router原理分析(opens new window)

1) 运营把商品详情从/product/42改成#/product/42后，认为两种地址只是写法不同，你会怎样解释底层差异？

答：Hash路由依赖地址中散列部分的变化，而普通Path路由依赖HTML5HistoryAPI的pushState、replaceState等能力。两者都能驱动客户端匹配组件，但服务器参与程度不同；选择Path后若没有回退配置，直接访问深层地址可能由服务器返回404。

2) 内容站准备从HashRouter切到正常路径，上线前除了替换路由容器，你会要求服务端完成什么配置？

答：服务端需要配置historyApiFallback或等价回退规则，让未知的前端路径仍返回应用入口，再由ReactRouter完成匹配。还要排除静态资源和真实接口路径，不能把所有请求无条件重写；否则深层链接虽能打开，资源请求或后端错误却可能被错误地返回HTML。

3) 同一套路由配置既要跑浏览器端管理后台，又要跑不具备浏览器HistoryAPI的测试环境，历史记录层怎么选？

答：浏览器环境使用基于HistoryAPI的实现，非浏览器场景可使用内存历史记录，以数组维护位置变化。ReactRouter通过不同的history基础实现支持跨平台，上层匹配逻辑可以保持接近；内存实现不会天然改变真实地址栏，不能把测试结果直接等同于浏览器导航行为。

4)

用户直接打开/orders/1001得到服务器404，但先访问首页再点Link却正常，你会如何定位？

答：现象说明客户端路由匹配大概率正常，故障位于服务器对深层路径的处理，应先检查入口回退配置和部署平台重写规则。Link在应用内通过HistoryAPI切换，因此不会重新请求对应文档；修复时需保留API、资源和真实文件的优先匹配，避免过度回退。

5) 微前端页面需要共享当前位置，团队争论是层层传location，还是让路由组件直接读取，你会怎么设计？

答：路由容器应通过Context API传递位置和导航上下文，负责匹配的消费者组件按需读取，避免每层手工转发。整体可区分Router、MemoryRouter等容器，路由匹配组件，以及Link、NavLink等平台功能组件；多个应用根并存时仍要明确各自上下文边界。

468. React 17 带来了哪些改变

React 17的关键变化是更新JSX转换、重构事件系统，并引入更细致的Lane优先级模型。新转换会从react/jsx-runtime自动引入解析能力，因此使用JSX时不再必须手动导入React，编译输出也有简化空间。事件委托从全局document下移到各自的根容器，让多个React应用之间的影响更可控，同时取消事件池，异步访问事件不再需要e.persist()。源码调度方面，Lane用二进制位表达优先级，比原来的expirationTime能覆盖更多边界情况。最重要的是以下三点：新的JSX转换逻辑事件系统重构Lane模型的引入

1. 重构 JSX 转换逻辑 在过去，如果我们在 React 项目中写入下面这样的代码：

```
function MyComponent() { return 这是我的组件 }
```

React 是会报错的，原因是 React 中对 JSX 代码的转换依赖的是React.createElement这个函数。因此但凡我们在代码中包含了

JSX，那么就必须在文件中引入 React，像下面这样：

```
import React from 'react'; function MyComponent() { return 这是我的组件 }
```

而React 17则允许我们在不引入 React 的情况下直接使用 JSX。这是因为在 React 17 中，编译器会自动帮我们引入 JSX 的解析器，也就是说像下面这样一段逻辑：

```
function MyComponent() { return 这是我的组件 }
```

会被编译器转换成这个样子：

```
import { jsx as _jsx } from 'react/jsx-runtime'; function MyComponent() { return _jsx('p', { children: '这是我的组件' }); }
```

react/jsx-runtime中的 JSX 解析器将取代React.createElement完成JSX的编译工作，这个过程对开发者而言是自动化、无感知的。因此，新的

JSX 转换逻辑带来的最显著的改变就是降低了开发者的学习成本。react/jsx-runtime中的 JSX 解析器看上去似乎在调用姿势上和React.createElement区别不大，那么它是否只是React.createElement换了个马甲呢？

当然不是，它在内部实现了React.createElement无法做到的性能优化和简化。在一定情况下，它可能会略微改善编译输出内容的大小

2. 事件系统重构 事件系统在 React 17 中的重构要从以下两个方面来看：

- 2.1 卸掉历史包袱：放弃利用 document 来做事件的中心化管理 React 16.13.x 版本中的事件系统会通过将所有事件冒泡到 document 来实现对事件的中心化管理 这样的做法虽然看上去已经足够巧妙，但仍然有它不聪明的地方——document

是整个文档树的根节点，操作 document 带来的影响范围实在是太大了，这将会使事情变得更加不可控 在 React 17 中，React 团队终于正面解决了这个问题：事件的中心化管理不会再全部依赖document，管控相关的逻辑被转移到了每个 React 组件自己的容器 DOM 节点中。比如说我们在 ID 为 root 的 DOM 节点下挂载了一个 React 组件，像下面代码这样：

```
const rootElement = document.getElementById("root"); // 这里沿用 React 17 的写法说明事件挂载位置的变化；React 18 起写作 // ReactDOM.createRoot(rootElement).render(), 事件同样挂在这个容器节点上 ReactDOM.render(rootElement);
```

那么事件管控相关的逻辑就会被安装到root 节点上去。这样一来，React 组件就能够自己玩自己的，再也无法对全局的事件流构成威胁了

- 2.2 拥抱新的潮流：放弃事件池 在 React 17 之前，合成事件对象会被放进一个叫作“事件池”的地方统一管理。这样做的目的是能够实现事件对象

的复用，进而提高性能：每当事件处理函数执行完毕后，其对应的合成事件对象内部的所有属性都会被置空，意为为下一次被复用做准备。这也就意味着事件逻辑一旦执行完毕，我们就拿不到事件对象了，React官方给出的这个例子就很可能说明问题，请看下面这个代码 `function handleChange(e){ // This won't work because the event object gets reused. setTimeout(()=>{ console.log(e.target.value); }, ...`

1) 一个只写的文件删掉 `import React from`

'react'后仍能编译，同事认为浏览器已经原生支持JSX，你会怎样纠正？

答：不是浏览器直接执行JSX，而是新的JSX转换由编译器自动引入 `react/jsx-runtime`，再生成相应的函数调用。它因此不再要求每个JSX文件显式导入 `React`，并可能简化编译输出；若构建工具尚未启用新转换，删除导入仍可能导致编译失败。

2) 大型页面里同时挂载两个独立React根节点，升级到React17

后事件隔离为什么比旧版本更容易控制？

答：React17 将事件的中心化管理从全局 `document` 移到各自的根容器节点，因此不同React根对全局事件流的影响更小。工程上仍要检查原生事件监听、冒泡边界和第三方脚本；容器级委托降低了相互干扰，并不代表所有跨根事件冲突都会自动消失。

3) 表单代码在事件回调结束后通过 `setTimeout` 读取 `e.target.value`，React16

需要 `e.persist()`，升级到React17 后要怎么处理？

答：React17 放弃了合成事件池，异步回调可以继续访问事件对象，因此不再需要为此调用 `e.persist()`。迁移时可清理依赖事件复用机制的兼容代码，但仍应注意目标DOM是否已被移除；取消事件池只保证对象不被统一置空，不保证相关节点永远存在。

4) 升级React17 后点击事件完全不触发，控制台也没有业务异常；页面由脚本动态替换过 `root` 容器，你先查什么？

答：先确认React实际挂载的根容器是否仍在文档中，以及动态替换节点后是否重新完成挂载，因为事件委托现在安装在该容器上。再检查容器附近的原生监听器是否阻止传播；继续只排查 `document` 上的React事件监听会偏离方向，但全局第三方监听仍可能造成干扰。

5) 调度模块准备继续用单一过期时间表达所有任务优先级，源码维护者建议切换到Lane模型，你如何说明取舍？

答：React17 引入的Lane模型使用二进制位表示不同优先级，并通过位运算组合和处理任务，比单一 `expirationTime` 能表达更细腻的优先级关系。它为更多调度边界提供了空间，但属于内部复杂度的增加；业务代码不应依赖具体位值，也不能把升级直接等同于页面必然变快。

【性能】

469. DNS 预解析

DNS预解析就是提前把域名解析成对应的IP，减少真正请求资源时等待解析的时间。浏览器访问外部域名之前通常需要先完成解析，因此预先处理可以让后续请求更早发出。比如页面确定会访问 `blog.poetries.top`，可以通过提前解析。DNS解析也是需要时间的，可以通过预解析的方式来预先获得域名所对应的IP

1) 商品页接入地图、客服、埋点和两个CDN，产品要求给全部第三方域名都加 `dns-prefetch`，你会直接照做吗？

答：不会直接给所有域名都加，而是只保留当前页面较可能请求的域名。dns-prefetch只是提前完成域名到IP的解析，若用户根本不会触发客服或地图，请求前的准备工作就可能白做；候选域名过多时，还应通过实际请求链路筛选。

2) 首屏图片来自img.example.com，你准备怎样在页面里落地DNS预解析，并确认写法指向正确域名？

答：可在文档头部加入，让浏览器提前解析该图片域名。配置必须写资源实际访问的主机名，不能只写主站域名；若图片后来改到其他域名，这条提示不会替新域名完成解析。

3) 同一套站点内网环境直接访问固定IP，公网环境则通过多个域名加载资源，平台团队想统一保留dns-prefetch，是否都有收益？

答：固定IP的请求不存在域名到IP的解析过程，因此dns-prefetch对它没有对应收益；公网域名请求才可能减少后续等待DNS的时间。统一模板可以保留条件配置，但不能把该提示当成所有网络环境都有效的加速手段。

4) 活动页已经加了dns-prefetch，线上瀑布图里接口请求仍长时间等待，开发因此认定预解析失效，你会先查什么？

答：先确认中的主机名是否与接口最终请求域名一致，并区分耗时究竟发生在DNS、连接还是服务端响应阶段。dns-prefetch只预先获得域名对应的IP，无法消除连接建立、重定向或接口处理耗时；若DNS本身不慢，整体变化可能并不明显。

5) 支付页只有一个首屏接口域名，但架构师希望用dns-prefetch解决整条请求链路延迟，你如何界定这个方案？

答：它只能针对请求前的DNS解析环节，适合域名确定且稍后会访问的场景，不能替代接口、网络连接或服务端优化。应先按请求阶段定位瓶颈，再决定是否加入预解析；若主要耗时不在DNS，继续堆叠该标签只会掩盖真正原因。

470. 缓存

浏览器缓存主要分为强缓存和协商缓存，区别在于资源复用前是否还要向服务器发请求。强缓存命中时直接使用本地资源，常用Cache-Control，其优先级高于Expires；协商缓存则携带ETag或修改时间验证，未变化时返回304。频繁变化的资源可用no-cache配合ETag，代码文件通常设置长期缓存并通过文件指纹更新，不应缓存的资源则用no-store。缓存对于前端性能优化来说是个很重要的点，良好的缓存策略可以降低资源的重复加载提高网页的整体加载速度通常浏览器缓存策略分为两种：强缓存和协商缓存 强缓存 实现强缓存可以通过两种响应头实现：Expires和Cache-Control。强缓存表示在缓存期间不需要请求，state code为200 Expires: Wed, 22 Oct 2018 08:41:00 GMT Expires是HTTP / 1.0的产物，表示资源会在Wed, 22 Oct 2018 08:41:00 GMT后过期，需要再次请求。并且Expires受限于本地时间，如果修改了本地时间，可能会造成缓存失效 Cache-control: max-age=30 Cache-Control出现于HTTP / 1.1，优先级高于Expires。该属性表示资源会在30秒后过期，需要再次请求 协商缓存 如果缓存过期了，我们就可以使用协商缓存来解决问题。协商缓存需要请求，如果缓存有效会返回304协商缓存需要客户端和服务端共同实现，和强缓存一样，也有两种实现方式 Last-Modified和If-Modified-Since Last-Modified表示本地文件最后修改日期，If-Modified-Since会将Last-Modified的值发送给服务器，询问服务器在该日期后资源是否有更新，有更新的话就会将新的资源发送回来但是如果本地打开缓存文件，就会造成Last-Modified被修改，所以在HTTP / 1.1出现了ETag ETag和If-None-Match ETag类似于文件指纹，If-None-Match会将当前ETag发送给服务器，询问该资源 ETag 是否变动，有变动的话就将新的资源发送回来。并且ETag优先级比Last-Modified高 选择合适的缓存策略 对于大部分的场景都可以使用强缓存配合协商缓存解决，但是在一些特殊的地方可能

需要选择特殊的缓存策略

对于某些不需要缓存的资源，可以使用Cache-control: no-store，表示该资源不需要缓存对于频繁变动的资源，可以使用Cache-Control: no-cache并配合ETag使用，表示该资源已被缓存，但是每次都会发送请求询问资源是否更新。对于代码文件来说，通常使用Cache-Control: max-age=31536000并配合策略缓存使用，然后对文件进行指纹处理，一旦文件名变动就会立刻下载新的文件

1) 运营后台的配置接口每天变更多次，后端却给它设置一年max-age，前端说再配ETag就能保证实时，你认可吗？

答：不认可，因为强缓存有效期内浏览器通常不会发起请求，也就没有机会用ETag校验。频繁变动且仍允许保存副本的资源更适合Cache-Control: no-cache配合ETag；若内容根本不应落入缓存，则应使用no-store，代价是每次都要完整请求。

2) 构建后的脚本文件名带内容指纹，首页文档仍可能更新，你会怎样分别配置两类资源的缓存？

答：带指纹的代码文件可设置Cache-Control: max-age=31536000，内容变化后通过新文件名触发重新下载。入口文档若频繁变化，可采用no-cache并配合ETag做协商校验；若入口也长期强缓存，用户可能持续拿到引用旧资源的文档。

3) 公司代理仍会经过只理解部分旧缓存规则的链路，服务端同时返回Expires和Cache-Control，浏览器应以谁为准？

答：在支持HTTP/1.1缓存语义的客户端中，Cache-Control的优先级高于Expires，应以其指令和max-age为主要依据。Expires依赖绝对时间且受本地时间影响，可作为兼容信息保留；两者冲突会增加排查成本，配置时应保持意图一致。

4) 用户反馈刚发布的app.js仍是旧内容，网络面板显示200且请求没有发到服务器，你会怎样定位？

答：这更像资源命中了未过期的强缓存，应先检查响应中的Cache-Control、Expires、实际URL和文件名是否变化。强缓存期间不会发起校验请求，状态表现仍可能是200；若固定URL配了长max-age，应改用内容指纹，而不是期待ETag立即纠正。

5) 图片接口返回内容很大但一周才变化一次，团队在Last-Modified与ETag之间争论，你会如何选择？

答：两者都能用于协商缓存，但ETag更像资源指纹，且校验优先级高于Last-Modified，更适合需要准确判断内容是否变化的资源。Last-Modified依赖修改时间，本地打开缓存文件还可能改变该时间；采用ETag会增加服务端生成和维护标识的工作。

6) 安全团队要求登录后的敏感响应不能保存，性能团队却想用no-cache减少传输，你如何裁决？

答：若要求是不允许存储响应，应使用Cache-Control: no-store，因为no-cache仍允许缓存，只是每次使用前都要向服务器验证。性能团队可把协商缓存留给允许保存且频繁变化的普通资源；敏感数据场景需要接受重复传输带来的成本。

471. 使用 HTTP / 2.0

HTTP/2的核心改进是多路复用，让多个请求可以共用同一个TCP连接。在HTTP/1.1下，请求受浏览器并发限制，连接建立和断开会消耗多个RTT，大文件还会受到TCP慢启动影响。HTTP/2通过复用连接减少这类等待，同时使用头部压缩降低请求数据量，因此更适合资源请求较多的页面。

因为浏览器会有并发请求限制，在HTTP / 1.1时代，每个请求都需要建立和断开，消耗了好几个RTT时间，并且由于TCP慢启动的原因，加载体积大的文件会需要更多的时间在HTTP / 2.0中引入了多路复用，能够让多

个请求使用同一个TCP链接，极大的加快了网页的加载速度。并且还支持Header压缩，进一步的减少了请求的数据大小

1) 新闻首页把二十个接口迁到HTTP/2后，开发认为每个请求从此都不再有网络成本，你会怎样纠正？

答：HTTP/2的收益在于多个请求可复用同一条TCP连接，并通过头部压缩减少传输数据，并不意味着请求本身没有成本。接口响应体、服务端处理和网络往返仍然存在；若瓶颈不在连接并发或请求头，迁移后的体感提升可能有限。

2) 一个首屏要并发加载十个脚本和六张图片，你会用什么证据确认线上确实按HTTP/2复用了连接？

答：应在浏览器网络面板或抓包结果中核对协议，并观察同一主机的多个请求是否共享连接，而不是只看服务器配置文件。只有客户端与服务端实际使用HTTP/2，多路复用和头部压缩才会生效；若链路中间环节回落，页面仍会表现为旧协议路径。

3) 站点只有一个体积较大的下载文件，没有并发小请求，负责人仍把升级HTTP/2当作唯一优化项，你会同意吗？

答：不会把它视为唯一方案，因为多路复用主要改善多个请求共享连接的场景，单个大文件仍受传输速率和TCP慢启动等因素影响。升级协议可能保留其他收益，但还应检查文件体积与传输链路；否则协议变化不一定解决主要耗时。

4) 网关宣称已启用HTTP/2，但页面仍出现大量连接建立时间和并发等待，你会按什么顺序排查？

答：先核对浏览器实际协商出的协议，再检查资源是否分散在不同主机，因为连接复用依赖实际使用的连接范围。随后区分等待来自旧协议并发限制、连接建立还是服务端响应；仅网关支持不代表每个资源请求都已通过同一条HTTP/2连接。

5) 老项目为绕过HTTP/1.1并发限制，把静态资源拆到多个域名；迁移HTTP/2后是否应立即合并？

答：多域名会形成不同连接，无法让所有资源共享同一条TCP连接，因此应重新评估拆分是否仍有价值。是否合并还要结合部署、缓存和故障隔离约束验证，不能只改域名；若资源仍跨主机，多路复用的覆盖范围自然会受限。

472. 预加载

预加载适合那些暂时不用、但希望浏览器尽早获取的重要资源。它本质上是一种声明式的fetch，通过强制浏览器提前发起请求，而且不会阻塞onload事件。合理使用可以降低首屏加载时间，例如提前获取后续会用到的重要文件；不过需要留意浏览器兼容性，不能把它当成所有环境都可靠的默认方案。在开发中，可能会遇到这样的情况。有些资源不需要马上用到，但是希望尽早获取，这时候就可以使用预加载预加载其实是声明式的fetch，强制浏览器请求资源，并且不会阻塞onload事件，可以使用以下代码开启预加载 预加载可以一定程度上降低首屏的加载时间，因为可以将一些不影响首屏但重要的文件延后加载，唯一缺点就是兼容性不好

1) 详情页有一个用户可能点击后才播放的视频，产品要求首屏用preload强制下载，理由是它不会阻塞onload，你接受吗？

答：不能仅凭不阻塞onload就预加载，因为preload会强制浏览器请求资源，用户不播放时可能产生无效下载。它更适合当前页面确定需要且希望尽早取得的资源；对低概率交互内容，应结合使用概率和资源成本再决定。

2) 首屏确定要执行的脚本发现得很晚，你准备如何用HTML声明预加载，并避免链接指向错误资源？

答：可在文档中加入，让浏览器更早发起该资源请求。落地时必须保证href与页面最终使用的资源一致，并在目标浏览器中验证兼容性；地址不一致会形成额外请求，兼容性不足时也不能依赖它保证效果。

3) 内嵌WebView的浏览器版本分布复杂，业务负责人仍要求把首屏达标完全押在preload上，你如何评估？

答：不应把结果完全押在preload上，因为其兼容性存在限制，部分运行环境可能无法获得预期收益。应先验证目标WebView是否支持并正确执行，再保留正常资源发现与加载路径；预加载只能作为提前请求的优化，不能成为资源可用性的唯一前提。

4)

上线后网络面板出现两次相同脚本请求，页面还加了对应preload，你会先核对哪些地方？

答：先核对预加载的href与脚本实际URL是否完全一致，包括协议、主机、路径和参数，再确认页面最终确实使用该资源。preload是一次声明式请求，若声明目标与实际目标不同，就可能产生两份下载；还应检查是否有重复标签或重定向。

5) 团队把下一页可能使用的模块也全部标成preload，架构师建议改成prefetch，你站哪一边？

答：下一页未来可能使用的资源更符合prefetch，而preload面向当前页面所需的脚本、样式等资源。把大量不确定资源设为preload会强制提前请求，可能浪费传输资源；若下一页并非高概率访问，连prefetch也应谨慎使用。

6) 页面把非首屏但确定重要的文件延后加载，开发担心这会拖慢首次呈现，preload与async、defer分别解决什么？

答：preload解决资源尽早获取的问题，本身不会阻塞onload；async和defer描述脚本异步读取后的执行方式。defer保持脚本顺序依赖，async则加载完成后执行；不能用执行属性替代资源优先发现，也不能用预加载替代脚本执行控制。

473. 预渲染

预渲染就是让浏览器提前在后台渲染用户接下来大概率会访问的页面，从而缩短真正打开时的等待时间。

可以通过指定目标页面，本质上是用当前的网络和计算资源换取后续访问速度。这个优化只适合用户几乎确定会打开的页面，否则提前下载和渲染的内容没有被使用，反而会白白消耗资源。

可以通过预渲染将下载的文件预先在后台渲染，可以使用以下代码开启预渲染。预渲染虽然可以提高页面的加载速度，但是要确保该页面百分百会被用户在之后打开，否则就白白浪费资源去渲染。总结 defer和async在网络读取的过程中都是异步解析defer是有顺序依赖的，async只要脚本加载完后就会执行preload可以对当前页面所需的脚本、样式等资源进行预加载prefetch加载的资源一般不是用于当前页面的，是未来很可能用到的这样一些资源

1) 电商首页列出一百个商品链接，运营想给每个详情页都加，说这样点击一定更快，你会批准吗？

答：不会批准批量预渲染，因为它会在后台提前渲染目标页面，用户未访问的页面会白白消耗资源。预渲染只适合之后几乎确定会打开的页面；面对一百个候选目标，应缩小到确定性极高的路径，否则加速收益建立在大量浪费之上。

2) 支付成功页之后固定进入订单详情页，你会怎样写预渲染声明，并说明它何时值得保留？

答：可在当前文档中加入，让目标页面预先在后台渲染。只有用户后续几乎必然进入该地址时才值得保留；若流程允许关闭页面、跳转其他入口或目标URL不确定，就应重新评估。

3) 同一流程从“付款后必进订单页”改成“付款后可能去抽奖、订单页或首页”，原有prerender还应保留吗？

答：原有预渲染的成立条件已经变弱，不应继续按必达路径处理。它的代价不仅是提前获取，还包括后台渲染目标页面；当三个去向都可能发生时，应根据真实导航确定性缩减或移除，避免为未打开页面消耗资源。

4) 上线预渲染后，低端设备在当前页出现额外资源消耗，用户却很少进入目标页，你会怎样确认根因？

答：先检查页面中的prerender声明及其目标地址，再对照用户实际后续导航，确认目标页是否真的概率打开。若大多数用户没有访问，后台渲染就是直接浪费；应移除或只在路径高度确定时声明，不能用当前页加载完成来证明配置合理。

5) 结算页的下一步几乎确定，但团队在prefetch和prerender之间争论，你如何选择？

答：若只想提前取得未来可能使用的资源，prefetch更贴近需求；若目标页面之后确定会打开，并希望连后台渲染也提前完成，才考虑prerender。后者工作更重，对导航确定性要求更高；一旦用户改走其他路径，已完成的渲染就会浪费。

6) 首屏脚本、下一页资源和下一页完整页面都想提前处理，评审会上有人建议统一使用preload，你会怎么拆分？

答：当前页面所需的脚本或样式使用preload，未来很可能用到的资源对应prefetch，几乎确定会访问的完整页面才适合prerender。三者提前处理的对象和成本不同，统一使用preload会混淆当前页资源与未来导航，也无法等同于后台渲染。

474. 懒执行与懒加载

懒执行是推迟计算逻辑，懒加载则是推迟非关键资源的下载，两者优化的对象并不一样。耗时逻辑如果首屏用不到，可以等定时器或用户事件触发后再执行；图片、视频等资源则可以等进入指定区域时再加载。图片场景里常先展示占位图，把真实地址放在自定义属性中，满足加载条件后再赋给src，但要确保触发逻辑可靠。懒执行 懒执行就是将某些逻辑延迟到使用时再计算。该技术可以用于首屏优化，对于某些耗时逻辑并不需要在首屏就使用的，就可以使用懒执行。懒执行需要唤醒，一般可以通过定时器或者事件的调用来唤醒 懒加载 懒加载就是将不关键的资源延后加载 懒加载的原理就是只加载自定义区域（通常是可视区域，但也可以是即将进入可视区域）内需要加载的东西。对于图片来说，先设置图片标签的src属性为一张占位图，将真实的图片资源放入一个自定义属性中，当进入自定义区域时，就将自定义属性替换为src属性，这样图片就会去下载资源，实现了图片懒加载

懒加载不仅可以用于图片，也可以使用在别的资源上。比如进入可视区域才开始播放视频等

1) 商品详情页把价格计算、推荐列表和首屏主图都改成进入视口后执行，结果页面先出现空壳，用户还没滚动就无法下单；这里把“懒”用错在哪了？

答：首屏成交链路不应被统一延后，价格、主图等当前区域必需内容应立即执行或加载。懒执行适合暂时不用的耗时逻辑，懒加载适合非关键资源；判断标准是当前区域与当前操作是否依赖它，延后关键内容只会把成本变成可感知等待。

2) 一个长列表有500张商品图，接口一次返回了全部图片地址；你会怎样落地图片懒加载，同时避免未加载区域把页面高度撑乱？

答：先为图片保留明确尺寸并展示占位图，把真实地址放在自定义属性中，再用IntersectionObserver监听图片进入或接近可视区域时替换src。加载完成后应取消观察，失败时保留占位或展示重试入口；

没有预留尺寸会在图片出现时引发布局跳动。

3) 资讯页的下一个视频刚进入可视区域才开始下载并播放，用户在弱网下总看到黑屏；产品又不接受进入页面就加载全部视频，你怎么调整触发边界？

答：把触发区域从“已经可见”扩大为“即将进入可视区域”，提前加载下一段视频，但仍只播放当前可见内容。预加载距离应结合滚动速度、资源体积和网络状况控制；范围过大会退化为批量加载，范围过小则无法掩盖下载等待。

4) 图片懒加载上线后，部分图片滚到屏幕内仍保持占位图，而手动刷新又恢复；你会按什么顺序定位？

答：先确认观察器是否绑定到真实图片节点，以及列表更新后新增节点是否被继续观察，再检查真实地址写入和src替换逻辑。随后查看图片请求是否发出、是否失败，并检查容器滚动场景下观察区域是否设置正确；只监听初始节点会让动态列表稳定复现漏加载。

5) 筛选面板包含一段耗时统计逻辑：用setTimeout延后执行，还是等用户点击“展开”再执行？如果结果还要用于首屏角标，你怎么选？

答：若统计只服务于展开后的面板，应由展开事件唤醒，避免定时器在用户从不使用时仍消耗主线程。若首屏角标依赖同一结果，就不能整体懒执行，应拆出角标所需的最小计算，其余逻辑继续延后；拆分会增加状态与缓存一致性的维护成本。

475. 文件优化

文件优化的核心是减少资源体积、避免阻塞渲染，并把静态资源更高效地送到用户手里。图片应按设备尺寸请求裁剪版本，照片可用JPEG，小图可用PNG或base64，图标优先考虑SVG，支持时也可选压缩效果更好的WebP。CSS放在head，脚本按依赖选择defer或async，耗时计算可交给Webworker。服务端还应开启压缩并使用不同于主站的CDN域名，避免静态资源请求携带主站Cookie。

图片优化对于如何优化图片，有2个思路：减少像素点、减少每个像素点能够显示的颜色。图片加载优化：不用图片。很多时候会使用到很多修饰类图片，其实这类修饰图片完全可以用CSS去代替。对于移动端来说，屏幕宽度就那么点，完全没有必要去加载原图浪费带宽。一般图片都用CDN加载，可以计算出适配屏幕的宽度，然后去请求相应裁剪好的图片。小图使用base64格式将多个图标文件整合到一张图片中（雪碧图）。选择正确的图片格式：对于能够显示WebP格式的浏览器尽量使用WebP格式。因为WebP格式具有更好的图像数据压缩算法，能带来更小的图片体积，而且拥有肉眼识别无差异的图像质量，缺点就是兼容性并不好。小图使用PNG，其实对于大部分图标这类图片，完全可以使用SVG代替照片使用JPEG。对于能够显示WebP格式的浏览器尽量使用WebP格式。因为WebP格式具有更好的图像数据压缩算法，能带来更小的图片体积，而且拥有肉眼识别无差异的图像质量，缺点就是兼容性并不好。小图使用PNG，其实对于大部分图标这类图片，完全可以使用SVG代替照片使用JPEG。其他文件优化：CSS文件放在head中，服务端开启文件压缩功能。将script标签放在body底部，因为JS文件执行会阻塞渲染。当然也可以把script标签放在任意位置然后加上defer，表示该文件会并行下载，但是会放到HTML解析完成后顺序执行。对于没有任何依赖的JS文件可以加上async，表示加载和渲染后续文档元素的过程将与JS文件的加载与执行并行无序进行。执行JS代码过长会卡住渲染，对于需要很多时间计算的代码可以考虑使用Webworker。Webworker可以让我们另开一个线程执行脚本而不影响渲染。CDN：静态资源尽量使用CDN加载，由于浏览器对于单个域名有并发请求上限，可以考虑使用多个CDN域名。对于CDN加载静态资源需要注意CDN域名要与主站不同，否则每次请求都会带上主站的Cookie。

1) 活动页为了减少请求，把一张2MB照片转成base64塞进主CSS，请求数下降了但首屏更慢；你会如何判断这次优化？

答：这是把小图内联策略错误地用于大图，图片体积会进入阻塞渲染的CSS，还失去独立资源缓存和按需加载空间。base64只适合足够小且确实值得减少请求的资源；大照片应选择合适尺寸与格式，并交给静态资源缓存或CDN。

2) 移动商品页拿到的是1920px原图，而内容区最多只有375px；设计要求清晰，后端又已有图片裁剪服务，你会怎样改造请求？

答：根据展示宽度和设备需求向图片服务请求相应裁剪尺寸，避免移动端下载远超实际像素需求的原图。照片可优先评估WebP，并为不适用的环境保留JPEG等格式；裁剪过小或压缩过度会损害清晰度，需要在体积与观感之间验收。

3) 后台管理页有80个小图标，团队在雪碧图、SVG和逐个PNG之间争论；哪些约束会改变你的选择？

答：可缩放、需要换色的图标更适合SVG，固定的小位图可考虑合并为雪碧图以减少请求，少量独立资源则未必值得增加维护复杂度。选择还要看兼容要求和更新频率；雪碧图中单个图标频繁变更会使整张图失效，独立文件则增加请求数量。

4) 线上首页的CSS已放在head，但主体仍长期空白，网络面板显示一个同步script位于文档前部且执行时间很长；你会怎么处理？

答：将非首屏脚本移到body底部或使用defer，让它并行下载并在HTML解析完成后按顺序执行。没有依赖关系的脚本才适合评估async，因为执行顺序不受保证；若真正瓶颈是长时间计算，还应拆分任务或交给Web Worker。

5) 架构师想给图片、脚本拆出多个CDN域名提升并发，安全同学担心请求携带登录Cookie；你会怎样落方案？

答：静态资源域名应与主站区分，并确保请求不携带主站Cookie，否则每次资源请求都会增加无效传输并扩大暴露面。多域名能够缓解单域并发限制，但也会增加域名解析和运维成本；资源数量不多时，不应只为理论并发继续拆域。

476. 其他

除了资源本身，构建产物和线上错误监控也是性能优化不能漏掉的两块。使用Webpack4时可启用production、基于ES6模块做tree shaking，再按路由拆包并给文件名加哈希，兼顾按需加载与浏览器缓存。线上错误可用window.onerror、Promise.catch或try catch捕获，跨域脚本还要配置crossorigin。由于生产代码通常经过压缩，构建时应生成sourceMap方便定位，并把捕获结果上传到服务端。使用 Webpack 优化项目

对于Webpack4，打包项目使用production模式，这样会自动开启代码压缩使用ES6模块来开启tree shaking，这个技术可以移除没有使用的代码优化图片，对于小图可以使用base64的方式写入文件中按照路由拆分代码，实现按需加载给打包出来的文件名添加哈希，实现浏览器缓存文件 监控 对于代码运行错误，通常的办法是使用window.onerror拦截报错。该方法能拦截到大部分的详细报错信息，但是也有例外

对于跨域的代码运行错误会显示Script error. 对于这种情况我们需要给script标签添加crossorigin属性对于某些浏览器可能不会显示调用栈信息，这种情况可以通过arguments.callee.caller来做栈递归对于异步代码来说，可以使用catch的方式捕获错误。比如Promise可以直接使用 catch 函数，async await可以使用try catch但是要注意线上运行的代码都是压缩过的，需要在打包时生成sourceMap文件便于debug。对于捕获的错误需要上传给服务器，通常可以通过img标签的src发起一个请求

1) 项目切到Webpack4的production后，负责人认为包体一定已经最优，不再做任何配置检查；你会拿什么反例追问他？

答：production会开启代码压缩，但不代表未使用代码、路由资源和图片都已按目标优化。还要确认源码使用ES6模块以支持tree shaking、路由是否按需拆分，以及小图是否适合内联；动态引用或存在副作用时，未使用代码也未必能安全移除。

2) 一个管理系统有30条路由，首屏只进入工作台；你会如何组织构建产物，并让浏览器可靠复用未变化的文件？

答：按路由拆分代码，让工作台只加载当前必需资源，其余路由在访问时再获取。输出文件名加入内容相关哈希，使未变化文件继续命中浏览器缓存、变化文件获得新地址；拆得过于细会增加请求和运行时管理成本，需要结合页面依赖调整粒度。

3) 公司要求错误监控尽量不依赖业务接口，前端负责人提出所有异常都交给window.onerror；遇到Promise和async/await异常时，你会怎样补齐？

答：window.onerror可覆盖大部分运行时错误，但异步链路仍应在Promise上使用catch，在async/await周围使用try...catch。捕获后统一整理并上报，必要时可通过图片src发起轻量请求；遗漏异步出口会形成未记录的失败。

4) 线上监控只收到Script

error，故障来自另一个域名上的脚本，本地却有完整堆栈；你先检查哪两处配置？

答：先检查跨域script是否添加了crossorigin，再确认资源服务是否允许相应的跨域访问，否则浏览器不会暴露详细错误。即使拿到文件名和行列号，线上代码仍是压缩产物，还必须保留与该版本匹配的sourceMap才能映射回源码。

5) 安全团队不允许公开部署sourceMap，研发又必须定位压缩代码报错；构建和监控之间应该怎样取舍？

答：构建阶段仍应生成与发布版本对应的sourceMap，供受控的调试或错误解析流程使用，不必把映射文件作为普通静态资源公开。监控事件还要携带可定位构建版本的信息；若映射文件与线上哈希不匹配，即使保留了文件也无法可靠还原位置。

477. 如何根据chrome的timing优化

我会先用Chrome DevTools和performance.timing拆开各阶段耗时，再针对最长的一段优化。DNS慢就减少域名并配置dns-prefetch，请求和下载慢则优先检查缓存、压缩、资源体积及并发阻塞。首屏资源可通过异步、预加载或懒加载后置，图片则压缩并选择WebP等格式。单页应用里直接依赖传统时间点不一定合适，还要结合图片onload等事件判断首屏完成。

性能优化API

Performance。performance.now()与new Date()区别，它是高精度的，且是相对时间，相对于页面加载的那一刻。但是不一定适合单页面场景window.addEventListener("load",

"");window.addEventListener("domContentLoaded", "");img的onload事件，监听首屏内的图片是否加载完成，判断首屏事件RequestAnimationFrame和RequestIdleCallbackIntersectionObserver、MutationObserver，PostMessageWeb Worker，耗时任务放在里面执行 检测工具 Chrome Dev ToolsPage SpeedJsperf 前端指标 window.onload=function(){setTimeout(function(){let t=performance.timing console.log('DNS查询耗时

:'+(t.domainLookupEnd-t.domainLookupStart).toFixed(0))console.log('TCP链接耗时

:'+(t.connectEnd-t.connectStart).toFixed(0))console.log('request请求耗时

:'+(t.responseEnd-t.responseStart).toFixed(0))console.log('解析dom树耗时

:'+(t.domComplete-t.domInteractive).toFixed(0))console.log('白屏时间

:'+(t.responseStart-t.navigationStart).toFixed(0))console.log('domready时间

:'+(t.domContentLoadedEventEnd-t.navigationStart).toFixed(0))console.log('onload时间 :'+(t.onloadEventEnd-t.navigationStart).toFixed(0))if(t=performance.memory){console.log('js内存使用占比

: '+ (t.usedJSHeapSize/t.totalJSHeapSize*100).toFixed(2)+'%')]]}}}

DNS预解析优化

dns解析是很耗时的，因此如果解析域名过多，会让首屏加载变得过慢，可以考虑dns-prefetch优化。DNS Prefetch应该尽量的放在网页的前面，推荐放在 后面。具体使用方法如下：request请求耗时 不请求，用cache（最好的方式就是尽量引用公共资源，同时设置缓存，不去重新请求资源，也可以运用PWA的离线缓存技术，可以帮助web实现离线使用）前端打包时压缩服务器上的zip压缩图片压缩（比如tiny），使用webp等高压压缩比格式把过大的包，拆分成多个较少的包，防止单个资源耗时过大同一时间针对同一域名下的请求有一定数量限制，超过限制数目的请求会被阻塞。如果资源来自于多个域下，可以增大并行请求和下载速度延迟、异步、预加载、懒加载对于非首屏的资源，可以使用 defer 或 async 的方式引入也可以按需加载，在逻辑中，只有执行到时才做请求对于多屏页面，滚动时才动态载入图片

1) 单页应用用new Date()记录按钮操作耗时，连续测量出现精度不足和系统时间调整带来的跳变；你会换成什么，并提醒什么边界？

答：短时耗时测量应使用高精度、相对页面加载时刻的performance.now()，避免依赖系统绝对时间变化。它适合计算同一页面生命周期内的时间差，但单页应用长期运行时不能把页面初始时刻直接当成每次路由的起点，应为各次操作单独记录基准。

2) 首页同时引用主站、接口、图片和分享服务四个域名，Chrome DevTools显示首次访问的域名解析拖慢首屏；你会把优化写在哪里？

答：对确定会在首屏使用的外部域名，在文档前部加入dns-prefetch，并通过x-dns-prefetch-control开启相应控制，让解析尽早发生。只应声明高概率访问的域名；预解析过多不会减少资源体积，还可能引入无意义的解析开销。

3) 瀑布图显示一个首屏包下载时间很长，团队一方主张继续拆包，另一方主张合并以减少请求；你依据哪些现象裁决？

答：先判断耗时来自单文件过大还是同域请求超过并发限制：前者适合压缩并拆出非首屏代码，后者可能需要减少首屏请求、利用缓存或调整资源域。拆包后仍应把非关键部分延迟或按需加载；仅把大包切成大量同时请求的小包，可能把下载耗时变成排队耗时。

4) 监控代码在window.onload回调里立刻读取loadEventEnd，线上偶尔得到不完整结果；你会如何修正采集时机，并区分哪些阶段？

答：应在load回调后再延迟到当前事件处理结束读取，避免loadEventEnd尚未写入。采集时分别计算DNS、连接、响应、DOM解析、白屏、DOMContentLoaded和load时间，先定位阶段再优化；单个总耗时无法说明瓶颈属于网络还是渲染。

5) 一个多屏图文页首屏资源已压缩，但用户流量仍很高；产品要求滚动顺滑，研发准备把所有图片都预加载，你会怎么取舍？

答：非首屏图片应随滚动按需加载，必要时在即将进入视口时提前触发，而不是一次预加载全部资源。首屏资源可继续使用缓存、压缩和合适图片格式；触发太晚会出现占位等待，触发太早则重新增加请求竞争与流量。

478. 移动端优化

移动端优化既要控制资源加载，也要降低脚本执行和页面渲染的开销。我一般把首屏目标控制在三秒左右，静态资源使用缓存和GZip，图片按尺寸与格式压缩，非首屏内容按需或异步加载。交互阶段要减少DOM操作、重绘和回流，高频的scroll、touchmove可配合requestAnimationFrame限制渲染频率。懒加载可能带来额外重绘，GPU加速也应合理使用，不能为了指标全部开启。

1. 概述 PC优化手段在Mobile侧同样适用在Mobile侧我们提出三秒种渲染完成首屏指标基于第二点，首屏加载3秒完成或使用Loading基于联通3G网络平均338KB/s(2.71Mb/s)，所以首屏资源不应超过1014KB Mobile侧因手机配置原因，除加载外渲

渲染速度也是优化重点基于第五点，要合理处理代码减少渲染损耗基于第二、第五点，所有影响首屏加载和渲染的代码应在处理逻辑中后置加载完成后用户交互使用时也需注意性能

2. 加载优化

加载过程是最为耗时的过程，可能会占到总耗时的80%时间，因此是优化的重点

2.1 缓存 使用缓存可以减少向服务器的请求数，节省加载时间，所以所有静态资源都要在服务器端设置缓存，并且尽量使用长Cache（长Cache资源的更新可使用时间戳）

2.2 压缩HTML、CSS、JavaScript 减少资源大小可以加快网页显示速度，所以要对HTML、CSS、JavaScript等进行代码压缩，并在服务器端设置GZip

a) 压缩（例如，多余的空格、换行符和缩进）

b) 启用GZip

2.3 无阻塞 写在HTML头部的JavaScript（无异步），和写在HTML标签中的Style会阻塞页面的渲染，因此CSS放在页面头部并使用Link方式引入，避免在HTML标签中写Style，JavaScript放在页面尾部或使用异步方式加载

2.4 使用首屏加载

首屏的快速显示，可以大大提升用户对页面速度的感知，因此应尽量针对首屏的快速显示做优化。

2.5 按需加载 将不影响首屏的资源 and 当前屏幕资源不用的资源放到用户需要时才加载，可以大大提升重要资源的显示速度和降低总体流量。

PS：按需加载会导致大量重绘，影响渲染性能

a) LazyLoad

b) 滚屏加载

c) 通过Media Query加载

2.6 预加载 大型重资源页面（如游戏）可使用增加Loading的方法，资源加载完成后显示页面。但Loading时间过长，会造成用户流失。

对用户行为分析，可以在当前页加载下一页资源，提升速度。

a) 可感知Loading

b) 不可感知的Loading（如提前加载下一页）

2.7 压缩图片 图片是最占流量的资源，因此尽量避免使用他，使用时选择最合适的格式（实现需求的前提下，以大小判断），合适的大小，然后使用智图压缩，同时在代码中用Srcset来按需显示

PS：过度压缩图片大小影响图片显示效果

a) 使用智图（<http://zhitu.tencent.com/>）

b) 使用其它方式代替图片

1. 使用CSS3

2. 使用SVG

3. 使用IconFont

c) 使用Srcset

d) 选择合适的图片

1. webP优于JPG

2. PNG8优于GIF

e) 选择合适的大小

1. 首次加载不大于1014KB2. 不宽于640（基于手机屏幕一般宽度）

2.8 减少Cookie

Cookie会影响加载速度，所以静态资源域名不使用Cookie。

2.9 避免重定向

重定向会影响加载速度，所以在服务器正确设置避免重定向。

2.10 异步加载第三方资源

第三方资源不可控会影响页面的加载和显示，因此要异步加载第三方资源

2.11 减少HTTP请求 因为手机浏览器同时响应请求为4个请求（Android支持4个，iOS5后可支持6个），所以要尽量减少页面的请求数，首次加载同时请求数不能超过4个

a) 合并CSS、JavaScript

b) 合并小图片，使用雪碧图

3. 三、脚本执行优化

脚本处理不当会阻塞页面加载、渲染，因此在使用时需当注意

CSS写在头部，JavaScript写在尾部或异步

避免图片和iFrame等的空Src，空Src会重新加载当前页面，影响速度和效率。尽量避免重设图片大小

重设图片大小是指在页面、CSS、JavaScript等中多次重置图片大小，多次重设图片大小会引发图片的多次重绘，影响性能

图片尽量避免使用DataURL，DataURL图片没有使用图片的压缩算法文件会变大，并且要解码后再渲染，加载慢耗时长

4. CSS优化 尽量避免写在HTML标签中写Style属性

4.1 css3过渡动画开启硬件加速

```
.translate3d{ -webkit-transform: translate3d(0, 0, 0); -moz-transform: translate3d(0, 0, 0); ...}
```

1) 移动活动页为了满足“三秒首屏”，在资源没完成时一直显示全屏Loading，弱网用户等很久仍看不到任何内容；你如何评价这个方案？

答：Loading只能让等待可感知，不能替代首屏资源与渲染优化，持续过久仍会造成用户流失。应先后置不影响首屏的代码和资源，尽快展示可用的首屏结构；大型重资源页面才适合等待资源完成后整体呈现，也要控制等待范围。

2) 移动首页预算约1014KB，现在一张横幅原图就占了大半，设计不允许去掉；你会怎样压缩加载成本而不直接牺牲视觉需求？

答：先按实际展示宽度生成合适尺寸，通过srcset按需选择，并在满足兼容与观感时采用WebP等压缩率更高的格式。还应使用图片压缩工具并避免超过手机所需尺寸；过度压缩会产生明显失真，最终必须以目标设备上的显示效果验收。

3) 页面首屏有10个脚本和样式请求，旧移动浏览器同时只能处理较少请求；团队又希望所有模块独立发布，你怎么处理这个冲突？

答：首屏应优先减少必要请求数，可合并关键CSS、JavaScript和小图片，并把非关键模块异步或按需加载。模块独立发布不等于必须在首屏分别请求全部产物；合并过度会扩大缓存失效范围，因此只聚合稳定且共同使用的关键资源。

4) 商品列表滚动时明显掉帧，埋点显示scroll回调频繁修改多个元素的内联style；你会先改哪一段执行链？

答：把高频scroll更新合并到requestAnimationFrame中，并增加响应间隔，避免同一帧内反复触发渲染。批量切换class，减少零散DOM操作和不必要的重绘回流；若每帧计算本身仍然过重，仅限频不能消除卡顿。

5) 动画负责人要求给页面所有卡片加translate3d强制开启GPU，测试发现低端机内存压力反而上升；你会怎么划定使用范围？

答：硬件加速应只用于确实需要平滑过渡的动画元素，并优先使用transform、opacity配合requestAnimationFrame。大量元素长期占用独立合成资源会增加设备负担，因此要在真实移动设备上验证；静态卡片或偶发变化不应统一强制提升。

6) 第三方统计脚本偶尔超时并拖住移动首页，业务方又要求不能删除；你如何隔离它与首屏渲染？

答：将第三方资源改为异步加载，并确保首屏内容不依赖其下载或执行结果，因为外部服务的时延不可控。脚本还应避免放在会阻塞解析的位置；异步化可能改变埋点初始化时序，业务事件需要在脚本就绪前暂存或允许安全丢弃。

【工程化】

479. 介绍一下 webpack 的构建流程

Webpack的构建流程本质上是读取配置，从入口递归生成依赖图，再组织为chunk并输出资源。启动时它会合并配置和命令行参数，创建全局的Compiler，加载插件并开始一次Compilation。模块经过对应loader转换后，再通过AST分析依赖，直到所有入口依赖都处理完成。最后按依赖关系组装产物并写入文件系统，插件则通过Tapable的生命周期事件介入并修改构建结果。核心概念 entry：入口。webpack是基于模块的，使用webpack首先需要指定模块解析入口(entry)，webpack从入口开始根据模块间依赖关系递归解析和处理所有资源文件。output：输出。源代码经过webpack处理之后的最终产物。loader：模块转换器。本质就是一个函数，在该函数中对接收到的内容进行转换，返回转换后的结果。因为 Webpack 只认识 JavaScript，所以 Loader 就成了翻译官，对其他类型的资源进行转译的预处理工作。plugin：扩展插件。基于事件流框架Tapable，插件可以扩展 Webpack 的功能，在 Webpack 运行的生命周期中会广播出许多事件，Plugin 可以监听这些事件，在合适的时机通过 Webpack 提供的 API 改变输出结果。module：模块。除了js范畴内的es module、commonJs、AMD等，css @import、url(...)、图片、字体等在webpack中都被视为模块。解释几个 webpack 中的术语 module：指在模块化编程中我们把应用程序分割成的独立功能的代码模块chunk：指模块间按照引用关系组合成的代码块，一个chunk中可以包含多个modulechunk group：指通过配置入口点（entry point）区分的块组，一个chunk group中可包含一到多个 chunkbundling：webpack 打包的过程asset/bundle：打包产物 webpack 的打包思想可以简化为 3 点：

一切源代码文件均可通过各种Loader转换为 JS 模块（module），模块之间可以互相引用。webpack 通过入口点（entry point）递归处理各模块引用关系，最后输出为一个或多个产物包js(bundle)文件。每一个入口点都是一个块组（chunk group），在不考虑分包的情况下，一个chunk group中只有一个chunk，该 chunk 包含递归分析后的所有模块。每一个chunk都有对应的一个打包后的输出文件（asset/bundle） 打包流程初始化参数：从配置文件和 Shell 语句中读取并合并参数，得出最终的配置参数。开始编译：从上一步得到的参数初始化Compiler对象，加载所有配置的插件，执行对象的run方法开始执行编译。确定入口：根据配置中的entry找出所有的入口文件。编译模块：从入口文件出发，调用所有配置的loader对模块进行翻译，再找出该模块依赖的模块，这个步骤是递归执行的，直至所有入口依赖的模块文件都经过本步骤的处理。完成模块编译：经过第 4 步使用 loader 翻译完所有模块后，得到了每个模块被翻译后的最终内容以及它们之间的依赖关系。输出资源：根据入口和模块之间的依赖关系，组装成一个个包含多个模块的chunk，再把每个chunk转换成一个单独的文件加入到输出列表，这一步是可以修改输出内容的最后机会。输出完成：在确定好输出内容后，根据配置确定输出的路径和文件名，把文件内容写入到文件系统。 简版 Webpack CLI 启动打包流程；载入 Webpack 核心模块，创建Compiler对象；使用Compiler对象开始编译整个项目；从入口文件开始，解析模块依赖，形成依赖关系树；递归依赖树，将每个模块交给对应的 Loader 处理；合并 Loader 处理完的结果，将打包结果输出到 dist 目录。 在以上过程中，Webpack 会在特定的时间点广播出特定的事件，插件在监听到相关事件后会执行特定的逻辑，并且插件可以调用 Webpack 提供的 API 改变 Webpack 的运行结果 构建流程核心概念：Tapable：一个基于发布订阅的事件流工具类，Compiler和Compilation对象都继承于TapableCompiler：compiler对象是一个全局单例，他负责把控整个webpack打包的构建流程。在编译初始化阶段被创建的全局单例，包含完整配置信息、loaders、plugins以及各种工具方法Compilation：代表一次 webpack 构建和生成编译资源的过程，在 watch模式下每一次文件变更触发的重新编译都会生成新的Compilation对象，包含了当前编译的模块module,...

1) 一个后台项目只有一个entry，最终却生成了入口包、公共包和异步包；同事坚持“一个入口只会对应一个文件”，你怎么纠正？

答：entry对应的是入口点和chunk group，并不保证最终只有一个输出文件。Webpack会从入口递归建立模块依赖图，再根据代码分割等规则组装多个chunk，每个chunk才通常对应一个输出asset；不考虑分包时，才可近似理解为一个入口产生一个包。

2) 团队要让Webpack支持TypeScript、Less、字体和图片，你会把这些处理分别放到构建流程的什么位置？

答：这些资源都应先作为module进入依赖图，再由匹配的loader完成转换，例如ts-loader、less-loader和文件资源加载器。Webpack从entry递归解析引用，转换完成后才汇总模块关系并生成chunk；若处理需要观察整轮构建或修改最终产物，则应交给plugin。

3) 同一套配置由开发服务器以watch模式运行，连续改动三个文件时，Compiler和Compilation会怎样变化？

答：Compiler在初始化阶段创建并贯穿整个构建生命周期，配置、插件和工具方法都由它持有。每次文件变化触发重新编译时会新建一个Compilation，记录该轮模块、资源、变更文件和依赖状态；把轮次状态长期挂在Compiler上，容易造成数据串轮或缓存失真。

4) 一个新增的loader能输出合法代码，但它引用的子模块始终没有进入产物；你会沿构建链路查哪些环节？

答：先确认入口是否实际到达该文件，以及loader的匹配范围和输出是否保留了可被解析的模块引用。随后检查转换结果生成的AST能否识别依赖，再核对依赖解析、模块编译和chunk组装结果；若依赖是在运行时动态拼接的，静态分析可能无法发现。

5) 产品要求在写入dist前改写所有资源名，同时还要转换单个源码文件；你会怎样划分loader与plugin？

答：单文件内容转换放在loader，因为它处于模块编译阶段，只需接收并返回可继续处理的内容。资源名改写应由plugin在输出资源阶段监听合适的Tapable钩子，利用Compilation操作完整产物列表；若等文件写入后再处理，会增加额外文件操作并可能破坏引用关系。

480. 介绍 Loader

Loader是Webpack的模块转换器，负责把它原本不能直接处理的资源转换成可继续构建的内容。它本质上是接收内容并返回转换结果的函数，多个loader可以按配置形成链式处理。比如babel-loader转换新语法，css-loader解析@import和url()，style-loader再把样式插入页面。编写时应保持单一职责和相互独立，否则转换链过重，也会让配置与问题定位变复杂。常用 Loader: file-loader: 加载文件资源，如 字体 / 图片 等，具有移动/复制/命名等功能；url-loader: 通常用于加载图片，可以将小图片直接转换为 Data Url，减少请求；babel-loader: 加载 js / jsx 文件，将 ES6 / ES7 代码转换成 ES5，抹平兼容性问题；ts-loader: 加载 ts / tsx 文件，编译 TypeScript；style-loader: 将 css 代码以标签的形式插入到 html 中；css-loader: 分析@import和url()，引用 css 文件与对应的资源；postcss-loader: 用于 css 的兼容性处理，具有众多功能，例如 添加前缀，单位转换等；less-loader / sass-loader: css预处理器，在 css 中新增了许多语法，提高了开发效率；编写原则: 单一原则: 每个 Loader 只做一件事；链式调用: Webpack 会按顺序链式调用每个 Loader；统一原则: 遵循 Webpack制定的设计规则和结构，输入与输出均为字符串，各个 Loader 完全独立，即插即用；

1) 一个Less文件先后经过less-loader、css-loader、style-loader，同事把三者都理解成“把样式塞进页面”，你怎么指出反例？

答：三者承担的是连续但不同的转换职责：less-loader处理预处理语法，css-loader分析@import和url()依赖，style-loader才把样式以形式注入页面。把职责合进一个loader会削弱复用和可组合性，也违背单一职责原则。

2) 项目同时有src下的业务代码和大量已编译依赖，配置babel-loader时你会怎样控制匹配范围？

答：让babel-loader只处理确实需要转换的js或jsx，通常通过test配合include或exclude缩小到业务源码。这样既避免重复转换已编译的第三方代码，也减少无效搜索；若依赖包确实发布了目标环境不支持的语法，则要针对该包单独放开范围。

3) 设计一个图片规则时，小图希望内联，大图需要复制并重命名；你如何解释url-loader与file-loader的职责边界？

答：小图片可由url-loader转成DataURL，从而减少独立资源请求；超过限定条件的文件则交给文件资源处理逻辑完成复制、命名和输出。内联会增大承载它的代码或样式体积，因此不能只追求减少请求而无条件处理所有图片。

4) 新增自定义loader后，构建报“下一个loader无法处理输入”，但单独执行转换函数结果正常，你先查什么？

答：先检查整条loader链的执行顺序，以及当前输出是否符合下一个loader可识别的内容形式。每个loader都必须保持独立且可链式组合，不能只验证自身转换结果；若某一步输出了二进制或特殊结构，还需按Webpack的接口约定明确声明和适配。

5) 团队要实现“转换每个SVG文件”和“汇总全部图片生成资源清单”，负责人想统一写成一个loader，你怎么取舍？

答：SVG到可识别模块内容的转换适合loader，因为处理对象是单个文件，输入输出边界清晰。全量资源清单依赖整次编译的输出集合，应由plugin监听生命周期并读取产物；强行放进loader会让单文件转换产生跨模块状态和执行顺序风险。

481. 介绍 plugin

Plugin是通过钩子参与Webpack整个构建生命周期、并在特定阶段修改打包结果的插件。注册时，Webpack会调用它的apply方法并传入全局唯一的Compiler，插件再借助Tapable的事件机制订阅编译钩子。每轮编译会创建新的Compilation，所以插件既能处理全局流程，也能介入单次编译。相比之下，loader主要负责转换单个模块，Plugin更适合生成页面、抽离样式或分析产物。插件系统是Webpack成功的一个关键性因素。在编译的整个生命周期中，Webpack会触发许多事件钩子，Plugin可以监听这些事件，根据需求在相应的时间点对打包内容进行定向的修改。一个最简单的plugin是这样的：

```
classPlugin{// 注册插件时，会调用 apply 方法// apply 方法接收 compiler 对象// 通过 compiler 上提供的 Api，可以对事件进行监听，执行相应的操作apply(compiler){// compilation 是监听每次编译循环// 每次文件变化，都会生成新的 compilation 对象并触发该事件compiler.plugin('compilation',function(compilation){})} // 注册插件: webpack.config.jsmodule.export={plugins:[newPlugin(options),]} // 事件流机制: Webpack
```

就像工厂中的一条产品流水线。原材料经过 Loader 与 Plugin 的一道道处理，最后输出结果。通过链式调用，按顺序串起一个个 Loader；通过事件流机制，让 Plugin 可以插入到整个生产过程中的每个步骤中；Webpack 事件流编程范式的核心是基础类 Tapable，是一种观察者模式的实现事件的订阅与广播：

```
const{SyncHook}=require("tapable")consthook=newSyncHook(['arg'])// 订阅hook.tap('event',(arg)=>{// 'event-hook'console.log(arg)})// 广播hook.call('event-hook')
```

Webpack中两个最重要的类Compiler与Compilation便是继承于Tapable，也拥有这样的事件流机制。

Compiler: 可以简单的理解为 Webpack 实例，它包含了当前 Webpack 中的所有配置信息，如 options, loaders, plugins 等信息，全局唯一，只在启动时完成初始化创建，随着生命周期逐一传递；Compilation: 可以称为编译实例。当监听到文件发生改变时，Webpack 会创建一个新的 Compilation 对象，开始一次新的编译。它包含了当前的输入资源，输出资源，变化的文件等，同时通过它提供的 api，可以监听每次编译过程中触发的事件钩子；区别:Compiler全局唯一，且从启动生存到结束；Compilation 对应每次编译，每轮编译循环均会重新创建；常用 Plugin:UglifyJsPlugin: 压缩、混淆代码；CommonsChunkPlugin: 代码分割；ProvidePlugin: 自动加载模块；html-webpack-plugin: 加载 html 文件，并引入 css / js 文件；extract-text-webpack-plugin / mini-css-extract-plugin: 抽离样式，生成 css 文件；DefinePlugin: 定义全局变量；optimize-css-assets-webpack-plugin: CSS 代码去重；webpack-bundle-analyzer: 代码分析；compression-webpack-plugin: 使用 gzip 压缩 js 和 css；happypack: 使用多进程，加速代码构建；EnvironmentPlugin: 定义环境变量；调用插件apply函数传入compiler对象通过compiler对象监听事件

Compiler: 可以简单的理解为 Webpack 实例，它包含了当前 Webpack 中的所有配置信息，如 options, loaders, plugins 等信息，全局唯一，只在启动时完成初始化创建，随着生命周期逐一传递；Compilation: 可以称为编译实例。当监听到文件发生改变时，Webpack 会创建一个新的 Compilation 对象，开始一次新的编译。它包含了当前的输入资源，输出资源，变化的文件等，同时通过它提供的 api，可以监听每次编译过程中触发的事...

1) 一个插件只修改单个.js文件源码，同事却说“只要用了apply就一定比loader合适”，你怎么反驳？

答：是否适合plugin取决于是否需要介入构建生命周期，而不是有没有apply方法。单文件、可链式的内容转换通常属于loader；只有需要观察编译事件、访问整轮资源或定向修改打包结果时，plugin的事件钩子才体现价值。

2)

你要写一个插件，在每轮构建中读取所有输出资源并生成清单，apply里应该抓住哪些对象？

答：插件注册时由Webpack调用apply(compiler)，先通过compiler订阅与编译相关的钩子，再在回调中取得当轮Compilation。资源清单应从该轮编译实例的输出资源生成，避免把上一轮数据留在全局状态中；具体钩子还要服从资源是否已完整生成的时机要求。

3) 开发服务器持续运行并触发 20

次增量编译，插件把资源列表缓存到Compiler上且从不清空，会有什么问题？

答：Compiler全局唯一并持续到进程结束，而每轮文件变化都会创建新的Compilation。把轮次资源长期累计在Compiler上，可能让已删除或改名的资源混入后续结果；缓存若必须跨轮保留，就要明确失效条件，并以当前Compilation为事实来源。

4) 自定义插件在首次构建正常，文件改动后却重复执行多次并生成重复内容，你会先排查哪里？

答：先确认钩子是否在每次compilation回调里被重复注册，以及插件实例是否被配置了多次。再区分订阅的是全局Compiler生命周期还是单轮Compilation生命周期，检查状态是否随新编译重置；错误的注册层级会在watch模式下不断叠加监听。

5) 团队要自动生成HTML并注入打包后的JS，另一个方案是在源码里手写固定文件名，你会选哪个？

答：应让类似html-webpack-plugin的插件在产物确定后生成或修改HTML，依据实际资源名完成注入。手写文件名无法可靠跟随代码分割、内容哈希和资源增删变化；代价是插件必须选择正确钩子，并避免覆盖其他插件已经完成的修改。

482. webpack 热更新实现原理

Webpack热更新本质上是重新编译变更模块，再由HMR运行时把补丁替换到当前页面中。文件变化后，文件系统通知Webpack，它只为相关模块生成新的编译结果，并通知HMR Server。服务端通过WebSocket告知浏览器存在更新，浏览器再用HTTP请求对应的更新内容。运行时能够接收更新就替换模块；如果判断无法更新，则会退化为整页刷新。HMR 的基本流程图 当修改了一个或多个文件；文件系统接收更改并通知webpack；webpack重新编译构建一个或多个模块，并通知 HMR 服务器进行更新；HMR Server使用WebSocket通知HMR runtime需要更新，HMR运行时通过HTTP请求更新jsonpHMR运行时替换更新中的模块，如果确定这些模块无法更新，则触发整个页面刷新

1) 修改一个样式模块后浏览器收到了WebSocket消息，同事便认定新代码也通过WebSocket传输，这个判断哪里不对？

答：WebSocket主要负责由HMR

Server通知浏览器端运行时存在新更新，并不等于承载完整更新代码。HMR runtime收到通知后还会通过HTTP请求更新内容，再执行模块替换；因此通知链路正常，只能证明更新事件已送达。

2) 一个包含 300

个模块的管理后台只改了其中一个文件，HMR从磁盘变化到页面更新会经过哪些关键环节？

答：文件系统先通知Webpack，Webpack针对受影响模块重新编译并把更新信息交给HMR Server。服务端经WebSocket通知浏览器，HMR runtime再通过HTTP获取更新并替换对应模块；是否能保留页面状态，还取决于更新边界能否被安全接受。

3) 公司代理允许普通HTTP请求却拦截WebSocket，页面初次加载正常，但修改文件后一直没有热更新，你如何判断影响范围？

答：初始资源仍可通过HTTP加载，但浏览器收不到HMR Server的更新通知，因此不会主动请求增量内容。应先检查连接建立、代理升级配置和服务端通知是否到达；即使更新文件可直接访问，缺少通知链路也无法自动触发替换。

4) 开发环境改文件后服务端已重新编译，浏览器也收到通知，却最终整页刷新；你按什么顺序排查？

答：先确认HMR runtime是否成功发起更新请求以及返回内容是否可执行，再检查变更模块能否被当前热更新边界接受。运行时若判断模块无法安全替换，就会退化为整页刷新；若请求本身失败，则应优先排查资源路径、服务端输出和网络代理。

5) 有人提议每次文件变化都直接刷新页面，认为这与HMR没有本质差异；在复杂表单页面里你怎么取舍？

答：整页刷新会重新加载整个应用，并丢失未持久化的界面状态；HMR只重新编译和获取受影响模块，成功时可局部替换。复杂表单更适合保留HMR，但必须接受无法更新时仍会刷新页面，并确保状态不能只依赖热更新来保护。

483. webpack 层面如何做性能优化

Webpack优化要分成构建速度和产物体积两条线，并先用数据找到真正的瓶颈。我一般用speed-measure-webpack-plugin看编译耗时，用webpack-bundle-analyzer检查包里哪些内容占空间。构建侧可以通过缓存、限制loader处理范围和减少模块搜索来提速；产物侧则依靠Tree Shaking、SplitChunksPlugin和代码压缩减少下载量。多进程、DLLPlugin或externals都有配置成本，应结合项目依赖和构建情况选择。

优化前的准备工作 准备基于时间的分析工具：我们需要一类插件，来帮助我们统计项目构建过程中在编译阶段的耗时情况。speed-measure-webpack-plugin分析插件加载的时间使用webpack-bundle-analyzer分析产物内容 代码优化: 无用代码消除，是许多编程语言都具有的优化手段，这个过程称为 DCE (dead code elimination)，即 删除不可能执行的代码；

例如我们的UglifyJs，它就会帮我们在生产环境中删除不可能被执行的代码，例如：

```
var fn=function(){return 1;}; // 下面代码便属于 不可能执行的代码； // 通过 UglifyJs (Webpack4+ 已内置) 便会进行 DCE ; var a=1;return a;}
```

摇树优化 (Tree-shaking)，这是一种形象比喻。我们把打包后的代码比喻成一棵树，这里其实表示的就是，通过工具“摇”我们打包后的 js 代码，将没有使用到的无用代码“摇”下来（删除）。即 消除那些被 引用了但未被使用的模块代码。 原理: 由于是在编译时优化，因此最基本的前提就是语法的静态分析，ES6的模块机制提供了这种可能性。不需要运行时，便可进行代码字面上的静态分析，确定相应的依赖关系。问题: 具有副作用 的函数无法被tree-shaking在引用一些第三方库，需要去观察其引入的代码量是不是符合预期；尽量写纯函数，减少函数的副作用；可使用webpack-deep-scope-plugin，可以进行作用域分析，减少此类情况的发生，但仍需要注意； 在引用一些第三方库，需要去观察其引入的代码量是不是符合预期；尽量写纯函数，减少函数的副作用；可使用webpack-deep-scope-plugin，可以进行作用域分析，减少此类情况的发生，但仍需要注意；

code-splitting: 代码分割技术，将代码分割成多份进行 懒加载 或 异步加载，避免打包成一份后导致体积过大，影响页面的首屏加载；

Webpack中使用SplitChunksPlugin进行拆分；按 页面 拆分: 不同页面打包成不同的文件；按 功能 拆分: 将类似于播放器，计算库等大模块进行拆分后再懒加载引入；提取复用的业务代码，减少冗余代码；按 文件修改频率 拆分: 将第三方库等不常修改的代码单独打包，而且不改变其文件 hash 值，能最大化运用浏览器的缓存；

将类似于播放器，计算库等大模块进行拆分后再懒加载引入；提取复用的业务代码，减少冗余代码；

scope hoisting: 作用域提升，将分散的模块划分到同一个作用域中，避免了代码的重复引入，有效减少打包后的代码体积和运行时的内存损耗；编译性能优化：升级至最新版本的webpack，能有效提升编译性能；使用dev-server/模块热替换(HMR)提升开发体验；监听文件变动忽略node_modules目录能有效提高监听时的编译效率；缩小编译范围modules: 指定模块路径，减少递归搜索；mainFields: 指定入口文件描述字段，减少搜索；noParse: 避免对非模块化文件的加载；includes/exclude: 指定搜索范围/排除不必要的搜索范围；alias: 缓存目录，避免重复寻址；babel-loader忽略node_modules，避免编译第三方库中已经被编译过的代码使用cacheDirectory，可以缓存编译结果，避免多次重复编译多进程并发webpack-parallel-uglify-plugin: 可多进程并发压缩js文件，提高压缩速度；HappyPack: 多进程并发文件的Loader解析；第三方库模块缓存:DLLPlugin和DLLReferencePlugin可以提前进行打包并缓存，避免每次都重新编译；使用分析Webpack Analyse / webpack-bundle-analyzer对打包后的文件进行分析，寻找可优化的地方配置profile: true，对各个编译阶段耗时进行监控，寻找耗时最多的地方source-map:开发:cheap-module-eval-source-map生产:hidd...

1) 产物分析显示某个库被引用却未使用，团队开启Tree Shaking后体积仍没下降；你会指出哪些反例边界？

答：Tree Shaking依赖ES6模块的静态结构来判断引用关系，并不等同于删除所有看似无用的代码。带副作用的模块或函数可能无法安全移除，应检查导入方式、第三方库实际引入量和副作用声明；强行删除会改变运行结果。

2) 一个项目构建耗时明显上升，但团队还没拿到阶段数据，你会怎样安排第一轮优化？

答：先用speed-measure-webpack-plugin或profile定位耗时阶段，再用webpack-bundle-analyzer检查产物组成，避免把构建速度和包体积混为一谈。确认瓶颈后再缩小loader范围、启用babel-loader缓存或调整解析规则；并发工具并非对所有任务都有收益。

3) 首屏包过大，但业务要求播放器只有点击后才加载，第三方框架还要尽量命中长期缓存，你怎么拆？

答：播放器适合按功能使用异步加载，避免进入首屏；复用业务代码可抽成公共块，低频变更的第三方库可单独拆分以稳定文件哈希。拆分应通过SplitChunksPlugin等规则落地，但块过多会增加加载调度成本，仍需结合实际依赖关系验证。

4) 开发构建慢，日志显示大量时间花在模块搜索和node_modules转译上，你会改哪些配置并防什么风险？

答：用test、include和exclude缩小loader搜索范围，并通过modules、mainFields、alias等解析配置减少递归寻址；babel-loader可排除无须转译的依赖并启用cacheDirectory。范围收得过窄可能漏掉含不兼容语法的第三方模块，因此要以目标运行环境验证。

5) 架构组在externals和DllPlugin之间争论如何移出大型依赖，你会依据什么选择？

答：已有稳定CDN资源且能明确全局对象、CommonJS或AMD等加载方式时，externals配置更直接。需要独立构建并缓存依赖、且业务仍会引用依赖子模块时，DllPlugin更完整，但配置和维护成本更高；两者都要防止外部依赖版本与业务构建不一致。

6) 生产包已经做了代码分割，运维又要求开启gzip，前端同事认为二者只能选一个，你怎么解释它们的关系？

答：代码分割控制哪些代码在何时加载，gzip控制传输时的压缩体积，两者解决的不是同一层问题，可以同时使用。还可配合代码压缩、样式抽离、Tree Shaking和作用域提升减少产物；最终仍需分别检查包结构、网络传输和运行时开销。

484. 介绍一下 Tree Shaking

Tree Shaking是在打包阶段分析并删除已引入但实际没有使用的代码。它依赖ES Module的import、export能被静态分析，因此工具不运行程序也能判断模块和变量的引用关系；CommonJS具有动态加载特性，就不适合这样处理。代码存在副作用时，打包器不能贸然删除，可以用package.json的sideEffects明确标记。还要避免让Babel把模块转成不利于静态分析的形式，否则优化可能无法生效。对tree-shaking的了解作用：它表示在打包的时候会去除一些无用的代码。原理：ES6的模块引入是静态分析的，所以在编译时能正确判断到底加载了哪些模块分析程序流，判断哪些变量未被使用、引用，进而删除此代码。特点：在生产模式下它是默认开启的，但是由于经过babel编译全部模块被封装成IIFE，它存在副作用无法被tree-shaking掉可以在package.json中配置sideEffects来指定哪些文件是有副作用的。它有两种值，一个是布尔类型，如果是false则表示所有文件都没有副作用；如果是一个数组的话，数组里的文件路径表示改文件有副作用。rollup和webpack中对tree-shaking的层次不同，例如对babel转译后的class，如果babel的转译是宽松模式下的话(也就是loose为true)，webpack依旧会认为它有副作用不会tree-shaking掉，而rollup会。这是因为rollup有程序流分析的功能，可以更好的判断代码是否真正会产生副作用。原理 ES6 Module引入进行静态分析，故而编译的时候正确判断到底加载了那些模块静态分析程序流，判断那些模块和变量未被使用或者引用，进而删除对应代码。依赖于import/export 通过导入所有的包后再进行条件获取。如下：

```
import foo from "foo"; import bar from "bar"; if (condition) { // foo.xxx } else { // bar.xxx }
```


ES6的import语法完美可以使用tree shaking，因为可以在代码不运行的情况下就能分析出不需要的代码。CommonJS的动态特性模块意味着tree shaking不适用。因为它是不可能确定哪些模块实际运行之前是需要的或者是不需要的。在ES6中，进入了完全静态的导入语法：import。这也意味着下面的导入是不可行的：

```
// 不可行，ES6 的 import 是完全静态的 if (condition) { myDynamicModule = require("foo"); } else { myDynamicModule = require("bar"); }
```

1) 商品页只引用了工具包导出的formatPrice，构建产物里却仍保留同文件导出的formatDate；既然用了import，为什么不能直接认定Tree Shaking一定生效？

答：import/export只是静态分析的必要条件，不等于未引用代码必然可删。打包器还要结合程序流和副作用判断；如果模块经过转译后被封装成IIFE，或相关代码可能产生副作用，就会保守地保留。

2) 团队准备发布一个包含几十个工具函数的ESM包，希望业务只打入实际引用的函数，package.json应怎样配置？

答：确认所有文件确实没有导入即执行的副作用后，可配置"sideEffects": false，帮助打包器安全删除未使用模块。若只有样式或初始化文件有副作用，应改用数组逐项声明，否则误标可能把业务依赖的执行逻辑一起删掉。

3) 同一套源码直接交给打包器时能摇掉未使用导出，经Babel处理后却全部保留，你会先检查哪一层？

答：先检查转译产物是否仍保留ESM的静态import/export，以及模块是否被包装成带潜在副作用的IIFE。Tree Shaking依赖编译期可分析的模块关系；结构被改写后，打包器无法证明代码无副作用，只能停止删除。

4) 订单页根据运行时条件在require("foo")和require("bar")之间选择，产品要求构建时移除永远不会走到的分支，Tree Shaking能完成吗？

答：仅凭这种动态CommonJS加载通常不能完成，因为构建阶段无法确定运行时究竟需要哪个模块。应把可静态确定的依赖改成顶层import，再让程序流分析处理未引用部分；若条件只能运行时确定，两份依赖都可能需要保留。

5) 一个转译后的class在webpack里未被删除，而同样产物交给Rollup却能删除，你会如何向构建负责人解释差异？

答：两者对副作用和程序流的判断能力并不完全相同，不能只凭“都支持Tree Shaking”推断结果一致。对于Babel宽松模式转译的class，webpack可能保守认定有副作用，而Rollup可借助程序流分析判断其实际影响；切换工具也要重新验证产物语义。

485. 介绍一下 webpack scope hosting

Scope Hoisting会把原本分散在多个模块函数里的代码合并到同一作用域，减少函数包裹和模块间的调用。webpack默认会为每个模块创建独立函数，模块多了会增加产物体积和运行时开销。它依赖ESM的静态导入导出关系，动态的CommonJS `require`无法完成这种提升。webpack 4+在mode: 'production'下默认开启，对应`optimization.concatenateModules`。作用域提升（Scope Hoisting），将分散的模块划分到同一个作用域中，避免了代码的重复引入，有效减少打包后的代码体积和运行时的内存损耗。原理：webpack 默认会把每个模块包裹进一个独立的函数（`(function(module, exports, require){...})`），模块越多，这类包裹函数和`__webpack_require__`调用越多，产物体积和运行时开销都变大。Scope Hoisting 会分析模块间的依赖关系，把可以合并的模块「提升」到同一个函数作用域里，减少函数包裹和模块间的`require`调用。要点：依赖 ESM 的静态结构：Scope Hoisting 只能对 ESM Module (`import/export`) 生效——因为它需要在编译期静态分析模块的导入导出关系；CommonJS 的动态`require`无法静态分析，不能提升。这也是「用 ESM 才能享受更好的打包优化」的又一个理由（同Tree Shaking）。生产模式默认开启：webpack 4+ 在mode: 'production'下默认启用（对应`optimization.concatenateModules`），开发模式默认关闭。收益：更小的产物、更少的闭包和函数调用、更快的运行时。一句话：Scope Hoisting 把多个 ESM 模块合并到同一作用域，减少函数包裹和 `require` 调用，从而减小体积、提升运行时性能；它依赖 ESM 的静态结构，生产模式默认开启。

1) 一个后台页面拆成上百个小模块后，产物里出现大量`(function(module, exports, require){...})`和`__webpack_require__`调用；Tree Shaking已经开启，为什么这些包装还在？

答：Tree Shaking负责移除未使用代码，并不负责消除仍需执行模块的包装。ScopeHoisting才会分析依赖关系，把可合并的ESM模块提升到同一函数作用域，从而减少包装函数和模块间的`require`调用。

2) 构建负责人想在生产包中启用作用域提升，webpack 4+项目需要增加什么配置，开发环境又该怎么处理？

答：在mode: "production"下通常无需额外配置，`optimization.concatenateModules`默认启用。开发模式默认关闭；若手动开启，应先确认调试体验和构建行为符合预期，因为它会改变模块在产物中的组织方式。

3) 组件库全部使用`require`和`module.exports`，业务方要求仅打开`concatenateModules`就把模块合并到一个作用域，能达到吗？

答：不能据此保证合并，因为ScopeHoisting依赖ESM的静态导入导出关系。动态`require`无法在编译期完整确定依赖图，应先评估迁移到`import/export`；否则打包器只能保留独立模块包装。

4) 生产包明明开启了`concatenateModules`，入口附近仍有很多独立函数模块，你会怎样判断是配置失效还是模块不可提升？

答：先确认实际构建使用了生产模式，并检查输入模块是否仍是可静态分析的ESM。若前置转译已改写为CommonJS，依赖关系就无法满足提升条件；此时继续调整压缩参数不会解决根因。

5) 性能评审会上，有人主张只做ScopeHoisting、不做Tree Shaking，因为两者都能缩小包体；你会怎么裁决？

答：两者解决的冗余不同，不能互相替代：Tree Shaking删除未引用代码，ScopeHoisting合并仍需保留的模块作用域。生产构建可同时利用二者，但都依赖ESM的静态结构；输入是动态模块时，两类优化都会受限。

486. Webpack Proxy工作原理？为什么能解决跨域

Webpack Proxy本质上是让本地开发服务器代替浏览器向目标服务转发请求，从而绕开浏览器的跨域限制。浏览器只请求同源的webpack-dev-server，后者通过http-proxy-middleware请求后端，再把响应转回浏览器；服务器之间通信不受浏览器同源策略约束。配置时通常关注target、pathRewrite和changeOrigin。它只用于开发阶段，生产环境仍要通过服务端代理或后端跨域配置处理。

1. 是什么 webpack proxy，即webpack提供的代理服务

基本行为就是接收客户端发送的请求后转发给其他服务器其目的是为了便于开发者在开发模式下解决跨域问题（浏览器安全策略限制）

想要实现代理首先需要有一个中间服务器，webpack中提供服务器的工具为webpack-dev-server

2. webpack-dev-server webpack-dev-server是webpack官方推出的一款开发工具，将自动编译和自动刷新浏览器等一系列对开发友好的功能全部集成在了一起

目的是为了提高开发者日常的开发效率，「只适用在开发阶段」

关于配置方面，在webpack配置对象属性中通过devServer属性提供，如下：

```
//  
./webpack.config.jsconst path=require('path')module.exports={// ...devServer:{contentBase:path.  
join(__dirname,'dist'),compress:true,port:9000,proxy:{'/api':{target:'https://api.github.com'}}// ...}
```

devServer里面proxy则是关于代理的配置，该属性为对象的形式，对象中每一个属性就是一个代理的规则匹配 属性的名称是需要被代理的请求路径前缀，一般为了辨别都会设置前缀为/api，值为对应的代理匹配规则，对应如下：

target：表示的是代理到的目标地址pathRewrite：默认情况下，我们的/api-hy也会被写入到URL中，如果希望删除，可以使用pathRewritesecure：默认情况下不接收转发到https的服务器上，如果希望支持，可以设置为falsechangeOrigin：它表示是否更新代理后请求的headers中host地址

2. 工作原理

proxy工作原理实质上是利用http-proxy-middleware这个http代理中间件，实现请求转发给其他服务器

举个例子：在开发阶段，本地地址为http://localhost:3000，该浏览器发送一个前缀带有/api标识的请求到服务端获取数据，但响应这个请求的服务器只是将请求转发到另一台服务器中

```
const express=require('express');const proxy=require('http-proxy-middleware');const app=express();app.use('/api',proxy({target:'http://www.example.org',changeOrigin:true}));app.listen(3000);//
```

http://localhost:3000/api/foo/bar -> http://www.example.org/api/foo/bar

3. 跨域 在开发阶段，webpack-dev-server会启动一个本地开发服务器，所以我们的应用在开发阶段是独立运行在localhost的一个端口上，而后端服务又是运行在另外一个地址上

所以在开发阶段中，由于浏览器同源策略的原因，当本地访问后端就会出现跨域请求的问题

通过设置webpack proxy实现代理请求后，相当于浏览器与服务端中添加一个代理者 当本地发送请求的时候，代理服务器响应该请求，并将请求转发到目标服务器，目标服务器响应数据后再将数据返回给代理服务器，最终再由代理服务器将数据响应给本地 在代理服务器传递数据给本地浏览器的过程中，两者同源，并不存在跨域行为，这时候浏览器就能正常接收数据

注意：「服务器与服务器之间请求数据并不会存在跨域行为，跨域行为是浏览器安全策略限制」

1) 本地页面运行在http://localhost:3000，直接请求另一地址的接口被浏览器拦截；把请求改成/api/users后由开发服务器转发，为什么响应就能被页面读取？

答：浏览器此时只请求同源的localhost:3000，跨地址转发由webpack-dev-server代理在服务器侧完成。同源策略限制的是浏览器中的请求，服务器之间转发不受该限制；代理收到目标响应后再以同源响应返回页面。

2) 前端请求/api/users，后端实际只接受/users，代理目标已配成https://service.example.com，你会改哪项配置？

答：应通过pathRewrite在转发时移除或改写/api前缀，使目标服务器收到它认可的路径。target只决定请求发往哪里，不会自动修正业务路径；规则写错会表现为后端路由不存在，而不是浏览器跨域。

3) 公司测试环境使用自签名的https服务，本地代理转发时证书校验失败；把secure设为false是否适合直接带到正式部署？

答：在明确是开发测试证书的前提下，可用secure: false允许代理转发到该https服务。它降低了证书校验要求，只应作为受控开发配置；webpack-dev-server本身也仅用于开发阶段，不能替代正式环境的网关。

4) 接口通过代理后仍返回错误页面，后端网关日志显示收到的Host是localhost:3000，但它只按目标域名路由，你会如何排查？

答：检查代理规则是否设置changeOrigin: true，它会把转发请求头中的Host更新为目标地址。若路径也带有/api前缀，还要同时核对pathRewrite；两者分别处理主机头和路径，不能相互替代。

5) 上线评审中有人提议把devServer.proxy配置原样用于生产跨域治理，你会接受吗？

答：不会，webpack-dev-server的代理是为本地开发效率提供的中间服务器，只适用于开发阶段。生产环境仍需部署独立的反向代理或由服务端处理跨域策略；否则前端构建结束后并不存在这台开发代理服务器。

487. 介绍一下 babel原理

Babel的核心流程是把源码解析成AST，修改这棵语法树，再生成目标代码。以ES6转ES5为例，解析器先识别源码结构，插件随后遍历AST，把需要兼容的语法节点转换成新的节点。最后，代码生成器根据转换后的AST输出ES5代码。简单来说，插件决定“改什么”，解析和生成阶段负责“读懂代码”和“重新写出代码”。babel的编译过程分为三个阶段：parsing、transforming、generating，以ES6编译为ES5作为例子：ES6代码输入；babylon进行解析得到AST；plugin用babel-traverse对AST树进行遍历编译，得到新的AST树；用babel-generator通过AST树生成ES5代码。Babel原理及其使用(opens new window)

1) 一个页面源码使用箭头函数，交给Babel后输出变成普通函数；有人说这是直接做字符串替换，你如何用编译链路反驳？

答：它不是针对源码文本做简单替换，而是先由解析器把代码转换为AST。插件遍历并改写树上的语法节点，再由生成器根据新AST输出目标代码，因此转换依据是语法结构而非字符形状。

2) 团队要写一个插件，把项目中的某类调用统一改写；在Babel的三个阶段里，核心逻辑应该放在哪里？

答：核心逻辑应位于transforming阶段，通过遍历AST定位目标节点并生成或替换相应节点。解析阶段只负责得到语法树，生成阶段负责把变换后的树输出为代码；若节点匹配过宽，可能改到语义不同的同名调用。

3) 构建工具准备把现有解析器替换掉，但仍沿用原来的遍历插件和代码生成器，你会要求验证什么约束？

答：要验证新解析结果的AST节点结构是否与遍历插件和生成器所期待的结构兼容。Babel的解析、转换、生成虽分阶段协作，但中间树是接口契约；节点类型或字段不一致时，转换可能漏掉代码或直接失败。

4) 某个Babel插件运行后生成了语法错误，源码解析本身没有报错，你会按什么顺序定位？

答：先保留解析得到的原始AST，再对比插件遍历前后的目标节点，确认错误是在转换阶段引入。随后检查生成器接收到的新树是否合法；源码能成功解析只证明parsing正常，不能证明插件改写或generating正确。

5)

两个团队争论兼容旧环境应改源码还是维护一套Babel转换插件，你会依据什么划分责任？

答：若差异能表达为稳定的语法树变换，可交给插件在AST层统一处理，并由生成器输出目标语法。若转换无法保持原语义，单靠parsing-transforming-generating流程也不能消除运行环境差异，必须另外评估实现边界。

488. 介绍一下Rollup

Rollup是面向ES Modules的打包器，通常更适合构建类库或框架。它生成的代码比较扁平、可读性好，并能自动移除未引用代码，因此类库产物通常更干净。相比之下，应用往往更依赖第三方模块、HMR和代码拆分，这些场景更适合功能更全面的Webpack。所以我一般把“应用用Webpack、库用Rollup”当作经验选择，而不是绝对规则。Rollup 是一款 ES Modules 打包器。它也可以将项目中散落的细小模块打包为整块代码，从而使得这些划分的模块可以更好地运行在浏览器环境或者 Node.js 环境。Rollup优势：输出结果更加扁平，执行效率更高；自动移除未引用代码；打包结果依然完全可读。缺点 加载非 ESM 的第三方模块比较复杂；因为模块最终都被打包到全局中，所以无法实现HMR；浏览器环境中，代码拆分功能必须使用Require.js这样的AMD库

我们发现如果我们开发的是一个应用程序，需要大量引用第三方模块，同时还需要 HMR 提升开发体验，而且应用过大就必须分包。那这些需求 Rollup 都无法满足。如果我们是开发一个 JavaScript 框架或者库，那这些优点就特别有必要，而缺点呢几乎也都可以忽略，所以在很多像 React 或者 Vue 之类的框架中都是使用的 Rollup 作为模块打包器，而并非 Webpack 总结一下：Webpack 大而全，Rollup 小而美。在对它们的选择上，我的基本原则是：应用开发使用 Webpack，类库或者框架开发使用 Rollup。不过这并不是绝对的标准，只是经验法则。因为 Rollup 也可用于构建绝大多数应用程序，而 Webpack 同样也可以构建类库或者框架。

1) 订单页提交参数缺失，接口仍返回200，只在响应体里写{"code": 400}，网关和监控都记录成功；你会接受这种约定吗？

答：不接受把所有结果都包装成200，因为HTTP层已经无法表达请求是否成功。参数错误应返回400，业务错误码再放进body细分原因；否则只观察状态码的代理、监控和调用方会得到失真的成功率。

2) 下载页要支持大文件暂停后继续，前端带上范围请求后，服务端仍返回完整文件和200；这里应该怎样落地？

答：服务端应在确认支持范围传输后返回206，并只发送客户端请求的那一段内容，而不是继续用200返回完整资源。若服务端无法正确处理范围请求，就不应伪装成断点续传，否则客户端可能重复接收或错误拼接数据。

3) 支付表单通过POST提交后需要临时跳转，产品要求“直接用302就行，还要保证下一跳仍是POST”；你会怎么选？

答：不能用302承诺保留POST，因为它通常会被实现为转向GET，但并不保证原方法得到保留。若下一跳必须继续使用POST，应选择307；若明确希望任意请求转为GET，则使用303，两者语义不能混用。

4) 用户访问管理后台得到空白页，抓包显示先返回401，登录后重试又返回403；你如何判断这两次失败分别发生了什么？

答：第一次401表示请求方尚未完成登录或身份认证，前端应进入登录流程，而不是展示无权限页面。第二次403表示身份已经确认但没有资源权限，应停止重复登录并提示授权问题；把两者统一处理会造成

成循环跳转或误导用户。

5)

线上接口经代理后返回502，运维却把它当成应用代码抛出的500；你会怎样缩小排查范围？

答：502表示充当网关或代理的服务器处理上游响应时出错，应先检查代理到源站的链路及上游响应。500才直接指向服务器内部无法完成请求；若代理等待上游超过时限，则更接近504，不能仅凭页面文案归因。

6) 新闻详情页带条件请求后收到304，前端同学认为“不是2xx就算失败”并展示错误页；这个判断错在哪里？

答：304表示资源自上次请求后未修改，客户端应继续使用已有副本，而不是把它当作业务失败。它属于重定向类状态，但承担缓存协商语义；前提是客户端确实持有可用缓存，否则无法仅靠该响应恢复正文。

489. 介绍一下Rollup

Rollup是面向ES Modules的打包器，通常更适合构建类库或框架。它生成的代码比较扁平、可读性好，并能自动移除未引用代码，因此类库产物通常更干净。相比之下，应用往往更依赖第三方模块、HMR和代码拆分，这些场景更适合功能更全面的Webpack。所以我一般把“应用用Webpack、库用Rollup”当作经验选择，而不是绝对规则。Rollup 是一款 ES Modules 打包器。它也可以将项目中散落的细小模块打包为整块代码，从而使得这些划分的模块可以更好地运行在浏览器环境或者 Node.js 环境。Rollup优势：输出结果更加扁平，执行效率更高；自动移除未引用代码；打包结果依然完全可读。缺点 加载非 ESM 的第三方模块比较复杂；因为模块最终都被打包到全局中，所以无法实现HMR；浏览器环境中，代码拆分功能必须使用Require.js这样的AMD库

我们发现如果我们开发的是一个应用程序，需要大量引用第三方模块，同时还需要 HMR 提升开发体验，而且应用过大就必须分包。那这些需求 Rollup 都无法满足。如果我们是开发一个 JavaScript 框架或者库，那这些优点就特别有必要，而缺点呢几乎也都可以忽略，所以在很多像 React 或者 Vue 之类的框架中都是使用的 Rollup 作为模块打包器，而并非 Webpack 总结一下：Webpack 大而全，Rollup 小而美。在对它们的选择上，我的基本原则是：应用开发使用 Webpack，类库或者框架开发使用 Rollup。不过这并不是绝对的标准，只是经验法则。因为 Rollup 也可用于构建绝大多数应用程序，而 Webpack 同样也可以构建类库或者框架。

1) 订单页提交参数缺失，接口仍返回200，只在响应体里写{"code": 400}，网关和监控都记录成功；你会接受这种约定吗？

答：不接受把所有结果都包装成200，因为HTTP层已经无法表达请求是否成功。参数错误应返回400，业务错误码再放进body细分原因；否则只观察状态码的代理、监控和调用方会得到失真的成功率。

2) 下载页要支持大文件暂停后继续，前端带上范围请求后，服务端仍返回完整文件和200；这里应该怎样落地？

答：服务端应在确认支持范围传输后返回206，并只发送客户端请求的那一段内容，而不是继续用200返回完整资源。若服务端无法正确处理范围请求，就不应伪装成断点续传，否则客户端可能重复接收或错误拼接数据。

3) 支付表单通过POST提交后需要临时跳转，产品要求“直接用302就行，还要保证下一跳仍是POST”；你会怎么选？

答：不能用302承诺保留POST，因为它通常会被实现为转向GET，但并不保证原方法得到保留。若下一跳必须继续使用POST，应选择307；若明确希望任意请求转为GET，则使用303，两者语义不能混用。

4) 用户访问管理后台得到空白页，抓包显示先返回401，登录后重试又返回403；你如何判断这两次失败分别发生了什么？

答：第一次401表示请求方尚未完成登录或身份认证，前端应进入登录流程，而不是展示无权限页面。第二次403表示身份已经确认但没有资源权限，应停止重复登录并提示授权问题；把两者统一处理会造成循环跳转或误导用户。

5)

线上接口经代理后返回502，运维却把它当成应用代码抛出的500；你会怎样缩小排查范围？

答：502表示充当网关或代理的服务器处理上游响应时出错，应先检查代理到源站的链路及上游响应。500才直接指向服务器内部无法完成请求；若代理等待上游超过时限，则更接近504，不能仅凭页面文案归因。

6) 新闻详情页带条件请求后收到304，前端同学认为“不是2xx就算失败”并展示错误页；这个判断错在哪里？

答：304表示资源自上次请求后未修改，客户端应继续使用已有副本，而不是把它当作业务失败。它属于重定向类状态，但承担缓存协商语义；前提是客户端确实持有可用缓存，否则无法仅靠该响应恢复正文。

【HTTP】

490. HTTP前生今世

HTTP的演进主线是不断完善能力，并逐步改善传输性能。HTTP/0.9只是能获取文本资源的简单文本协议，HTTP/1.0奠定了大部分现代技术，但并不是正式标准。HTTP/1.1功能更完善，也是目前使用最广泛的版本；HTTP/2基于SPDY，重点改善性能。再往后，HTTP/3基于QUIC，代表协议后续的发展方向。

HTTP协议始于三十年前蒂姆·伯纳斯·李的一篇论文HTTP/0.9是个简单的文本协议，只能获取文本资源；HTTP/1.0确立了大部分现在使用的技术，但它不是正式标准；HTTP/1.1是目前互联网上使用最广泛的协议，功能也非常完善；HTTP/2基于 Google 的SPDY协议，注重性能改善，但还未普及；HTTP/3基于 Google 的QUIC协议，是将来的发展方向

1) 有人看见接口返回文本，就断言它使用的是HTTP/0.9；面对现代商品详情页的请求，你会怎样反驳？

答：响应内容是文本不能证明协议版本，因为后续HTTP版本同样能够传输文本资源。HTTP/0.9的关键限制是协议非常简单且只能获取文本资源；判断现代请求所用版本，应依据实际协商或网络记录，而不是资源外观。

2) 团队维护一个同时承载页面、图片和接口的站点，架构评审只写“使用HTTP”却没有版本信息；这会遗漏什么工程判断？

答：版本不能省略，因为HTTP/1.1、HTTP/2与HTTP/3的能力基础和性能方向并不相同。应记录客户端到服务端实际采用的版本及其部署范围；仅写HTTP会让性能分析和兼容性判断缺少必要前提。

3) 旧系统负责人说HTTP/1.0已经确立了多数现用技术，因此可以把它视为完整正式标准继续扩建；你同意吗？

答：不同意把“确立多数技术”推导成“正式且适合继续扩建”，资料明确指出HTTP/1.0并非正式标准。评估旧系统时可以承认其历史贡献，但新方案仍要根据现行部署能力选型，不能只凭版本年代或名

称下结论。

4) 主站升级后仍然加载缓慢，项目经理认定“已经写了HTTP/2，协议层就不可能有问题”；你会先核实什么？

答：先核实请求链路是否真的使用HTTP/2，以及页面涉及的各个资源和服务是否都落在同一部署条件下。HTTP/2源自SPDY并侧重性能改善，但写进方案不等于已经普及或生效；协议之外的服务处理也仍可能成为瓶颈。

5) 基础设施选型会上，一方要求所有业务立即切到HTTP/3，理由是它代表未来；另一方坚持永远保留HTTP/1.1，你怎么裁决？

答：两种绝对结论都站不住：HTTP/1.1功能完善且使用广泛，HTTP/3基于QUIC，代表后续发展方向。应按客户端、服务器和中间链路的实际支持情况渐进选择，并保留可工作的兼容路径；不能把趋势直接当成现状。

6) 新人把HTTP/2和HTTP/3都说成“Google提出的同一个加速协议改名”，在技术评审里你会怎样纠正？

答：两者来源不同：HTTP/2基于Google的SPDY，而HTTP/3基于Google的QUIC。它们都关注HTTP的演进不代表底层依据相同；若把二者混为一谈，后续讨论传输基础、部署条件和故障边界都会失去准确前提。

491. HTTP世界全览

HTTP是连接浏览器、服务器和网络中间节点的应用层传输协议。浏览器和爬虫都可以作为User Agent发起请求，服务器负责应答，CDN与代理位于传输途中，可承担缓存加速、负载均衡等工作。访问资源时，DNS先把域名映射成IP，再通过由协议名、主机名和路径组成的URI定位资源。普通HTTP通常运行在TCP/IP的可靠传输之上，需要安全传输时，则使用由HTTP、SSL/TLS和TCP/IP组成的HTTPS。

互联网上绝大部分资源都使用HTTP协议传输；浏览器是HTTP协议里的请求方，即User Agent；服务器是HTTP协议里的应答方，常用的有Apache和Nginx；CDN位于浏览器和服务器之间，主要起到缓存加速的作用；爬虫是另一类User Agent，是自动访问网络资源的程序。TCP/IP是网络世界最常用的协议，HTTP通常运行在TCP/IP提供的可靠传输基础上DNS域名是IP地址的等价替代，需要用域名解析实现到IP地址的映射；URI是用来标记互联网上资源的一个名字，由“协议名 + 主机名 + 路径”构成，俗称URL；HTTPS相当于“HTTP+SSL/TLS+TCP/IP”，为HTTP套了一个安全的外壳；代理是HTTP传输过程中的“中转站”，可以实现缓存加速、负载均衡等功能

1) SEO爬虫抓取商品页后，开发说“它不是浏览器，所以不属于HTTP请求方”；你会怎么纠正这个角色判断？

答：爬虫同样属于User Agent，只是它通过程序自动访问网络资源，而不是由用户操作浏览器。服务器仍作为HTTP应答方处理请求；若按“只有浏览器才是请求方”设计策略，会漏掉爬虫这一类合法或需治理的客户端。

2) 图片站准备在浏览器与Nginx源站之间接入CDN，业务方问它除了多一跳还有什么价值；你怎样说明落地边界？

答：CDN位于浏览器和源站之间，主要价值是缓存资源并加速访问，因此适合承担可缓存内容的分发。它仍是传输链路中的中间节点，并不会替代源站处理所有请求；缓存规则不合适时，请求仍需回到服务器。

3) 内网服务迁移后保留原域名，只更换后端IP，有人认为页面里的URI没变就无需关注DNS；这成立吗？

答：不成立，域名需要经DNS解析映射到IP，后端地址变化仍要求解析结果正确指向新地址。URI负责标记资源，通常由协议名、主机名和路径构成；名称保持不变，并不意味着名称到网络地址的映射会自动正确。

4) 用户报告https://页面完全连不上，应用日志也没有请求；值班同学只检查业务路由，你会如何沿链路排查？

答：应用没有收到请求时，应先检查域名能否经DNS解析到预期IP，再核对网络传输和中间代理是否可达。HTTPS还在HTTP外增加了SSL/TLS安全层；在这些前置环节确认之前，直接归因于应用路由证据不足。

5) 高峰期需要在浏览器与多台应用服务器之间增加一层设施，团队在“只做缓存”与“同时负载均衡”之间争论；代理能承担哪些职责？

答：代理作为HTTP传输中的中转站，既可以用于缓存加速，也可以承担负载均衡，因此两种诉求并不天然冲突。具体是否同时启用取决于链路设计和代理能力；增加中转层也意味着请求路径变长，配置错误会影响源站可达性。

6)

安全评审中有人把HTTPS描述成“完全替换了HTTP和TCP/IP的新协议栈”，你会怎样校正？

答：HTTPS更准确的关系是HTTP + SSL/TLS + TCP/IP，即为HTTP增加安全外壳，而不是把HTTP与底层网络全部替换。HTTP通常仍依赖TCP/IP提供的可靠传输；混淆这些角色会导致证书、应用响应和网络故障被错误归层。

492. HTTP分层

网络分层的核心价值是职责隔离，上层使用下层能力时不必看到逐层传输的细节。TCP/IP分为四层，IP位于网际层，TCP位于传输层，HTTP位于应用层；映射到七层OSI模型时，TCP在第四层，HTTP在第七层。数据发送和接收时，会沿协议栈逐层打包再逐层拆包。日常交流常说“四层”和“七层”，可以粗略理解为操作系统负责的通常在四层或以下，而应用程序处理的通常在七层，但这不是绝对判断。第一层：物理层，TCP/IP里无对应；第二层：数据链路层，对应TCP/IP的链接层；第三层：网络层，对应TCP/IP的网际层；第四层：传输层，对应TCP/IP的传输层；第五、六、七层：统一对应到TCP/IP的应用层。总结 TCP/IP分为四层，核心是二层的IP和三层的TCP，HTTP在第四层；OSI分为七层，基本对应TCP/IP，TCP在第四层，HTTP在第七层；OSI可以映射到TCP/IP，但这期间一、五、六层消失了；日常交流的时候我们通常使用OSI模型，用四层、七层等术语；HTTP利用TCP/IP协议栈逐层打包再拆包，实现了数据传输，但下面的细节并不可见有一个辨别四层和七层比较好的（但不是绝对的）小窍门，“两个凡是”：凡是由操作系统负责处理的就是四层或四层以下，否则，凡是需要由应用程序（也就是你自己写代码）负责处理的就是七层

1) 评审会上有人说“HTTP在第四层，因为它运行在TCP/IP上”，另一人说“它在第七层”；面对这场术语冲突你怎么裁定？

答：两种说法采用了不同模型：在四层TCP/IP模型中，HTTP位于应用层，也就是第四层；在七层OSI模型中，它位于第七层。交流时必须先说明模型，否则层号相同或不同都可能造成假冲突。

2) 前端页面请求经过网卡、网络和服务后得到JSON，业务同学问数据为何能跨网络传输；你会怎样用分层解释而不陷入实现细节？

答：HTTP数据会沿TCP/IP协议栈逐层封装，经链路传输后在接收端逐层拆包，最终交给应用处理。HTTP使用底层能力，但其应用代码通常看不到下层细节；分层的价值正是隔离职责，而不是让业务直接操纵每一层。

3) 客户端从浏览器换成操作系统内置网络组件后，负责人断言所有逻辑都从七层降到了四层；这个判断为什么不可靠？

答：是否由操作系统处理只能作为辨别四层与七层的经验线索，并非绝对规则。TCP在OSI第四层，HTTP仍属于应用层；实现代码放在哪里不会自动改变协议职责，必须结合处理的实际协议内容判断。

4) 线上接口无法建立连接，但应用服务日志完全为空；开发立即修改HTTP路由，你会按分层把排查重点放在哪里？

答：应用日志没有请求时，不宜先认定是HTTP路由错误，应先检查链接层、网络层和传输层是否完成了数据送达。只有下层传输成立后，HTTP才能在应用层处理请求；不过分层只能缩小范围，最终仍需结合各层证据定位。

5) 网关团队计划只依据端口转发连接，业务团队却要求按HTTP路径分流；这两个能力分别更接近哪一层？

答：依据连接与传输信息工作的能力更接近四层，而解析HTTP路径后分流需要应用程序理解七层协议。后者能做更细的业务路由，但也承担HTTP解析和配置成本；若只做四层转发，就无法直接使用路径语义决策。

6) 新人画协议栈时把OSI的会话层、表示层逐一对应成TCP/IP的独立层，还把TCP放到了第三层；你会怎样修正？

答：OSI的第五、六、七层在TCP/IP中统一归入应用层，并不存在逐层独立对应；TCP位于OSI第四层，对应TCP/IP的传输层。映射是概念上的归并，不是编号平移，否则会误判协议职责。

493. HTTP报文是什么样子的

HTTP请求报文和响应报文结构基本一致，都由起始行、头部字段、空行和消息正文组成。起始行负责说明请求或响应的基本信息，也是两者最明显的区别；头部则用key-value形式补充细节。空行用于分隔头部和正文，正文才是实际传输的文本、图片、视频等数据，而且正文并不一定存在。 HTTP协议的请求报文和响应报文的结构基本相同，由三大部分组成 起始行（start line）：描述请求或响应的基本信息；头部字段集合（header）：使用key-value形式更详细地说明报文；消息正文（entity）：实际传输的数据，它不一定是纯文本，可以是图片、视频等二进制数据 这其中前两部分起始行和头部字段经常又合称为“请求头”或“响应头”，消息正文又称为“实体”，但与“header”对应，很多时候就直接称为“body”。一个完整的 HTTP 报文就像是下图的这个样子，注意在 header 和 body 之间有一个“空行”

1) 抓包里请求只有起始行和若干头部字段，没有JSON正文，同事因此认定它不是完整的HTTP报文；你认可吗？

答：不认可，HTTP报文由起始行、头部字段集合和可选的消息正文构成，正文并非每次都必须承载数据。没有body不能单独证明报文不完整；应继续检查起始行、header以及两者之后的报文边界是否符合格式。

2) 上传接口既要发送文件二进制，也要附带描述信息，产品担心HTTP正文只能放纯文本；你会怎样设计认知边界？

答：消息正文可以承载图片、视频等二进制数据，并不限于纯文本，因此文件能够放入body传输。头部字段负责补充说明报文，实际编码和内容解释仍需双方约定；仅证明能传二进制，不代表任意格式都能被服务端识别。

3) 团队日志把起始行和全部header统称为“请求头”，新人随后误以为起始行也是一个key-value字段；你怎么纠正？

答：工程交流中确实常把起始行与头部字段合称为“请求头”或“响应头”，但结构上两者仍不同。起始行描述请求或响应的基本信息，只有header使用key-value形式；日志展示可以合并，解析规则不能混淆。

4) 线上代理转发后，服务端把正文首行当成header，接口持续报格式错误；查看原始报文时你首先核对什么？

答：首先核对header与body之间是否保留了规定的空行，因为它承担两部分的分隔作用。若代理改写报文时丢失这一边界，接收端就可能继续按头部解析正文；还应比较代理前后的原始报文，避免只查业务字段。

5) 埋点接口要传一批事件，架构师要求把所有事件内容拆成自定义header，理由是“header和body都能携带数据”；你会接受吗？

答：不应把实际业务载荷全部塞进header，头部字段的职责是以key-value形式说明报文，事件数据更符合消息正文的定位。即使某些值技术上能写入header，也会模糊元信息与实体边界，并增加各节点解析约定的复杂度。

6) 调试工具只展示“请求头”和“响应体”两个面板，实习生据此认为请求报文与响应报文结构完全不同；你会如何串联两者？

答：请求报文和响应报文的基本结构相同，都包含起始行、头部字段集合以及可能存在的消息正文。工具只是按调试习惯拆分展示，不能替代协议结构；分析异常时仍要分别核对起始行、header、空行和body。

494. HTTP之URL

URL本质上是标记资源位置的字符串，通常由scheme、host:port、path和query组成。scheme决定用什么协议访问，主机和端口定位服务器，path定位资源，query再附加具体要求。部分结构可以省略，但像@、&、/这类特殊字符以及汉字需要正确编码，否则服务器可能无法正确解析报文。

URI是用来唯一标记服务器上资源的一个字符串，通常也称为URL；URI通常由scheme、host:port、path和query四个部分组成，有的可以省略；scheme叫“方案名”或者“协议名”，表示资源应该使用哪种协议来访问；“host:port”表示资源所在的主机名和端口号；path标记资源所在的位置；query表示对资源附加的额外要求；在URI里对“@&/”等特殊字符和汉字必须要做编码，否则服务器收到HTTP报文后会无法正确处理

1) 商品列表页把筛选条件写成/search?keyword=手机&配件&page=1，后端拿到的关键词只剩“手机”，前端同学却说汉字能显示就不用编码，你怎么判断？

答：参数边界已经被&破坏，汉字是否可见不能证明URI合法。应使用URLSearchParams，或仅对参数值调用encodeURIComponent，让特殊字符和汉字按规则编码；不要对整条URL再编码，否则scheme、host和分隔符也会失去结构含义。

2) 订单详情页要生成/orders/用户输入值?from=售后#记录，开发直接把两个动态值拼进字符串；你会要求怎样拆分和编码？

答：先区分path、query等URI组成部分，再分别处理动态数据，不能把用户输入当作结构字符直接拼接。查询参数交给URLSearchParams，路径段应单独编码；若把整段内容统一处理，输入中的/、&或#可能改变资源位置、参数边界或片段含义。

3) 搜索页既要支持同名标签tag=前端&tag=性能，又要让链接可复制分享，评审中有人主张把数组先用逗号连接，你选哪种实现？

答：优先用URLSearchParams.append保留同名多值，使每个值独立编码，复制后的URI仍能明确表达多个筛选条件。逗号拼接只有在前后端明确约定且值本身不会引起歧义时才可用，否则解析规则会混入业务层，后续扩展和兼容成本更高。

4) 线上跳转链接偶发落到错误页面，日志里出现参数值包含%252F，而原始输入只是/docs/api；你会先查哪一层？

答：%252F通常意味着%2F中的百分号又被编码了一次，应沿着表单提交、路由构造、请求封装和服务端解析逐层确认编码次数。原则是动态值在进入对应URI组件时编码一次、解析一次；重复调用编码函数或对已生成的完整URL再处理都会造成双重编码。

5) 报表页需要保存大量组合筛选，产品要求地址栏可分享，前端提议把全部条件都塞进query；你如何在URI可读性和功能完整性之间取舍？

答：少量、稳定且适合公开的筛选条件可以放入query，因为它本来就用于表达对资源的附加要求，也便于复制链接。条件复杂时应评估浏览器、代理和服务器对请求目标的实现限制，可改为服务端保存条件并在URL中放短标识；代价是分享链接依赖后端状态及权限控制。

495. HTTP实体数据

HTTP实体数据通过内容类型、压缩方式、语言和字符集来描述，客户端与服务器用对应头字段完成内容协商。客户端用Accept表示能接收的MIME type，服务器用Content-Type说明实际返回的类型；压缩则对应Accept-Encoding和Content-Encoding。多个候选项可以用;q=设置权重，例如优先返回HTML，其次返回XML。缓存场景还要关注Vary，因为同一URI可能因协商条件不同而产生多个响应版本。

1. 数据类型与编码 text：即文本格式的可读数据，我们最熟悉的应该就是text/html了，表示超文本文档，此外还有纯文本text/plain、样式表text/css等。image：即图像文件，有image/gif、image/jpeg、image/png等。audio/video：音频和视频数据，例如audio/mpeg、video/mp4等。application：数据格式不固定，可能是文本也可能是二进制，必须由上层应用程序来解释。常见的有application/json，application/javascript、application/pdf等，另外，如果实在是不知道数据是什么类型，像刚才说的“黑盒”，就会是application/octet-stream，即不透明的二进制数据 但仅有MIME type还不够，因为HTTP在传输时为了节约带宽，有时候还会压缩数据，为了不要让浏览器继续“猜”，还需要有一个“Encoding type”，告诉数据是用的什么编码格式，这样对方才能正确解压缩，还原出原始的数据。

比起MIME type来说，Encoding type就少了很多，常用的只有下面三种 gzip：GNU zip压缩格式，也是互联网上最流行的压缩格式；deflate：zlib（deflate）压缩格式，流行程度仅次于gzip；br：一种专门为HTTP优化的新压缩算法（Brotli）

2. 数据类型使用的头字段 有了MIME type和Encoding type，无论是浏览器还是服务器就都可以轻松识别出body的类型，也就能够正确处理数据了。 HTTP协议为此定义了两个Accept请求头字段和两个Content实体头字段，用于客户端和服务端进行“内容协商”。也就是说，客户端用Accept头告诉服务器希望接收什么样的数据，而服务器用Content头告诉客户端实际发送了什么样的数据 Accept字段标记的是客户端可理解的MIMEtype，可以用“,”做分隔符列出多个类型，让服务器有更多的选择余地，例如下面的这个头：

Accept: text/html,application/xml,image/webp,image/png 这就是告诉服务器：“我能够看懂 HTML、XML的文本，还有webp和png的图片，请给我这四类格式的数据”。

相应的，服务器会在响应报文里用头字段Content-Type告诉实体数据的真实类型： Content-Type: text/html Content-Type: image/png 这样浏览器看到报文里的类型是“text/html”就知道是 HTML文件，会调用排版引擎渲染出页面，看到“image/png”就知道是一个 PNG文件，就会在页面上显示出图像。

Accept-Encoding字段标记的是客户端支持的压缩格式，例如上面说的gzip、deflate 等，同样也可以用“,”列出多个，服务器可以选择其中一种来压缩数据，实际使用的压缩格式放在响应头字段Content-Encoding里 Accept-Encoding: gzip, deflate, br Content-Encoding: gzip 不过这两个字段是可以省略的，如果请求报文里没有Accept-Encoding字段，就表示客户端不支持压缩数据；如果响应报文里没有Content-Encoding字段，就表示响应数据没有被压缩

3. 语言类型使用的头字段 同样的，HTTP 协议也使用Accept请求头字段和Content实体头字段，用于客户端和服务端就语言与编码进行“内容协商”。

Accept-Language字段标记了客户端可理解的自然语言，也允许用“,”做分隔符列出多个类型，例如： Accept-Language: zh-CN, zh, en 这个请求头会告诉服务器：“最好给我zh-CN的汉语文字，如果没有

就用其他的汉语方言，如果还没有就给英文”。

相应的，服务器应该在响应报文里用头字段Content-Language告诉客户端实体数据使用的实际语言类型
Content-Language: zh-CN

字符集在HTTP里使用的请求头字段是Accept-Charset，但响应头里却没有对应的Co...

1) 下载接口实际返回application/pdf，网关却统一改成text/plain，浏览器预览区显示乱码；业务方认为文件字节没变就不影响，你同意吗？

答：不同意，实体字节未变不代表客户端能正确处理，浏览器会依据响应的Content-Type选择解析或展示方式。服务端应如实返回application/pdf；未知二进制才适合application/octet-stream，错误标成文本会导致渲染、下载行为和安全判断出现偏差。

2) 图片组件声明只接受image/webp,image/png，服务端仍返回image/jpeg；前端负责人说请求头只是提示，页面能显示就不用管，你怎么处理？

答：Accept表达客户端可理解或期望接收的媒体类型，服务端应据此协商，并用Content-Type声明最终实体类型。若服务端无法提供期望格式，应形成明确的降级或错误约定；忽略协商可能暂时可显示，但会让带宽优化、格式兼容和缓存版本失去可靠依据。

3) 静态资源请求带了Accept-Encoding: gzip, br，响应头却写Content-Encoding: br，老旧客户端解压失败；你会要求服务端怎样选择？

答：服务端只能从客户端声明支持的压缩格式中选择，并在Content-Encoding中准确标记实际采用的格式；未压缩时不应伪造该字段。若某类客户端不支持br，请求中就不应声明它，服务端也要保留gzip或无压缩回退，否则实体无法还原。

4) 国际化首页偶发把英文内容发给中文用户，CDN命中率很高，源站会根据Accept-Language返回不同实体；你从哪些响应头开始排查？

答：先核对请求的Accept-Language、响应的Content-Language，再检查缓存响应是否包含与协商维度一致的Vary: Accept-Language。同一URI可因语言协商产生多个版本，缓存若未区分该请求头就可能串内容；增加Vary会扩大缓存变体数量，需要接受命中率下降。

5) 接口团队想用Accept: application/json,text/html;q=0.8同时兼容应用和浏览器，另一个团队坚持拆成两个URL；你如何评估？

答：同一资源存在多种表示时，可以使用Accept和q值表达偏好，服务端再以Content-Type告知实际选择，并让缓存通过Vary: Accept区分版本。若两种响应的业务语义、鉴权或错误模型已经不同，拆分URI更清晰；内容协商不应掩盖接口契约差异。

496. 谈一谈HTTP协议优缺点

HTTP的优势是简单、灵活、采用请求应答模式并基于TCP/IP可靠传输，代价则是无状态、明文传输和可能出现队头阻塞。无状态让每次请求彼此独立、服务器不必天然保存会话上下文，但购物等场景又需要借助cookie和session维持状态。文本报文便于理解和扩展，也会让报文信息暴露在外。使用长连接共用一个TCP连接时，耗时较长的请求还可能阻塞后续请求。

超文本传输协议，HTTP是一个在计算机世界里专门在两点之间传输文字、图片、音频、视频等超文本数据的约定和规范。HTTP特点灵活可扩展。一个是语法上只规定了基本格式，空格分隔单词，换行分隔字段等。另外一个就是传输形式上不仅可以传输文本，还可以传输图片，视频等任意数据。请求-应答模式，通常而言，就是一方发送消息，另外一方要接受消息，或者是做出相应等。可靠传输，HTTP是基于TCP/IP，因此把这一特性继承了下来。无状态，这个分场景回答即可。HTTP 缺点无状态，有时候，需要保存信息，比如像购物系统，需要保留下顾客信息等等，另外一方面，有时候，无状态也会减少网络开销，比如类似直播行业这样子等，这个还是分场景来说。明文传输，即协议里的报文(主要指的是头部)不使用二进制数据，而是文本形式。这让HT

TP的报文信息暴露给了外界，给攻击者带来了便利。队头阻塞，当http开启长连接时，共用一个TCP连接，当某个请求时间过长时，其他的请求只能处于阻塞状态，这就是队头阻塞问题。灵活可扩展。一个是语法上只规定了基本格式，空格分隔单词，换行分隔字段等。另外一个就是传输形式上不仅可以传输文本，还可以传输图片，视频等任意数据。请求-应答模式，通常而言，就是一方发送消息，另外一方要接受消息，或者是做出相应等。可靠传输，HTTP是基于TCP/IP，因此把这一特性继承了下来。无状态，这个分场景回答即可。无状态，有时候，需要保存信息，比如像购物系统，需要保留下顾客信息等等，另外一方面，有时候，无状态也会减少网络开销，比如类似直播行业这样子等，这个还是分场景来说。明文传输，即协议里的报文(主要指的是头部)不使用二进制数据，而是文本形式。这让HTTP的报文信息暴露给了外界，给攻击者带来了便利。队头阻塞，当http开启长连接时，共用一个TCP连接，当某个请求时间过长时，其他的请求只能处于阻塞状态，这就是队头阻塞问题。http 无状态无连接 http协议对于事务处理没有记忆能力对同一个url请求没有上下文关系每次的请求都是独立的，它的执行情况和结果与前面的请求和之后的请求是无直接关系的，它不会受前面的请求应答情况直接影响，也不会直接影响后面的请求应答情况服务器中没有保存客户端的状态，客户端必须每次带上自己的状态去请求服务器人生若只如初见，请求过的资源下一次会继续进行请求 http协议无状态中的 状态 到底指的是什么？！【状态】的含义就是：客户端和服务端在某次会话中产生的数据那么对应的【无状态】就意味着：这些数据不会被保留通过增加cookie和session机制，现在的网络请求其实是有状态的在没有状态的http协议下，服务器也一定会保留你每次网络请求对数据的修改，但这跟保留每次访问的数据是不一样的，保留的只是会话产生的结果，而没有保留会话

1) 购物车接口连续两次请求同一个URL，第二次服务端却不知道第一次选了哪些商品；产品经理据此说HTTP不可靠，你会怎样纠正？

答：这体现的是HTTP无状态，不等于传输不可靠；每个请求默认没有前一次会话的上下文，而基于TCP/IP的传输可靠性是另一层概念。购物车状态应由客户端随请求携带标识，或借助cookie与session关联；代价是系统开始承担会话保存和一致性管理。

2) 直播弹幕页面每秒持续拉取内容，架构师认为所有请求都必须在服务端保存完整会话上下文；无状态在这里是缺点吗？

答：不一定，无状态能让相互独立的请求少依赖服务器会话，从而减少不必要的状态管理和网络附带信息。只有鉴权、进度或个性化确实依赖会话数据时才需要引入状态；若把所有访问上下文都持久化，会增加存储、清理和多节点共享成本。

3) 登录系统从单机扩到多个服务节点后，用户偶发被当成未登录；当前依赖本机session，你如何定位并落地修正？

答：先确认请求是否被负载均衡到不同节点，以及cookie中的会话标识是否稳定携带；本机session无法天然被其他节点读取。可采用共享会话存储或让路由保持粘性，前者增加共享存储依赖，后者影响流量调度与故障切换，都不能改变HTTP本身无状态的事实。

4) 安全评审发现内部管理页仍用明文HTTP，负责人说报文是文本格式便于排障，而且内网没有风险；你会怎么回应？

答：文本报文便于阅读也意味着传输内容更容易暴露，不能把可调试性当作安全保障。应使用HTTPS保护传输，并结合访问控制处理内网威胁；加密会引入证书和终止节点管理，但这是避免请求头与实体被直接窥视或篡改的必要成本。

5) HTTP/1.1页面共用一条长连接，其中一个慢接口迟迟不返回，后续资源也被拖住；前端想无限增加并发连接，你会如何取舍？

答：这是长连接上的队头阻塞表现：前面的请求耗时过长时，后续请求可能等待。应先定位并治理慢接口，再评估拆分关键资源、调整连接使用或升级传输方案；盲目增加连接只能缓解局部排队，还会增加客户端、服务器和网络的连接开销。

497. 说一说HTTP 的请求方法

HTTP/1.1常见方法可按读取、写入和连接诊断来理解。GET获取资源，HEAD只取响应头；POST追加数据，PUT存储或修改资源，DELETE删除资源。OPTIONS查询资源支持的方法，也用于复杂跨域请求的预检；TRACE用于测试诊断，CONNECT用于建立代理隧道。实际使用时还要看语义，例如GET通常无副作用且幂等，POST通常会产生副作用且不幂等。

HTTP1.0定义了三种请求方法：GET, POST 和 HEAD方法HTTP1.1新增了五种请求方法：OPTIONS, PUT, DELETE, TRACE 和 CONNECT
http/1.1规定了以下请求方法(注意，都是大写)： GET： 请求获取Request-URI所标识的资源POST： 在Request-URI所标识的资源后附加新的数据HEAD：

请求获取由Request-URI所标识的资源的响应消息报头PUT：

请求服务器存储一个资源，并用Request-URI作为其标识（修改数据）DELETE：

请求服务器删除对应所标识的资源TRACE：

请求服务器回送收到的请求信息，主要用于测试或诊断CONNECT：

建立连接隧道，用于代理服务器OPTIONS： 列出可对资源实行的请求方法，用来跨域请求

从应用场景角度来看，Get 多用于无副作用，幂等的场景，例如搜索关键字。Post

多用于副作用，不幂等的场景，例如注册。 options 方法有什么用 OPTIONS 请求与 HEAD

类似，一般也是用于客户端查看服务器的性能。这个方法会请求服务器返回该资源所支持的所有 HTTP

请求方法，该方法会用'*'来代替资源名称，向服务器发送 OPTIONS

请求，可以测试服务器功能是否正常。JS 的 XMLHttpRequest对象进行 CORS

跨域资源共享时，对于复杂请求，就是使用 OPTIONS

方法发送嗅探请求，以判断是否有对指定资源的访问权限。

1) 搜索页把“按关键词查询”设计成POST /search，接口没有修改任何资源；后端说只要能返回数据，请求方法无所谓，你会怎么评审？

答：该操作无副作用且期望重复执行结果语义一致，更符合GET的获取资源语义，也便于浏览器按常规方式缓存和表达查询地址。POST并非不能返回搜索结果，但会弱化方法传递的意图；只有请求条件不适合放入URI等约束成立时，才值得接受这种取舍。

2) 用户资料页用PUT /users/42提交完整资料，网络超时后客户端准备自动重试；为什么这里不能只看“都是修改数据”？

答：PUT的目标是让指定URI对应资源达到请求描述的状态，重复提交相同内容应保持相同资源状态，因此具备幂等语义。重试前仍要判断响应是否丢失以及服务端是否夹带非幂等副作用，例如重复发通知；方法语义正确不代表实现天然安全。

3) 监控平台想探测一个大文件是否存在并读取响应头，但不希望下载文件正文；有人建议先GET再由前端中断，你会选什么？

答：应优先使用HEAD，它用于获取与目标资源相关的响应报头而不传输实体正文，更符合探测元数据的目的。服务端必须正确实现其语义，并保证关键响应头与对应GET一致；若服务端不支持或实现错误，才需要受控地退回其他探测方式。

4) 跨域上传在正式环境失败，浏览器控制台显示OPTIONS返回405，但直接用命令行调用POST正常；你会沿着什么链路排查？

答：浏览器对复杂跨域请求会先发送OPTIONS预检，命令行成功不能证明浏览器具备跨域访问权限。应检查路由、网关和服务端是否允许OPTIONS，以及响应是否声明允许的来源、方法和请求头；仅放通实际POST仍会被浏览器拦截。

5) 公司代理需要访问外部HTTPS服务，网络团队提出开放CONNECT，安全团队担心它变成任意隧道；你如何界定方案？

答：CONNECT用于让客户端通过代理建立连接隧道，确实适合代理转发加密流量，但不能无条件开放。代理应限制可达目标、端口、身份和审计范围；限制过严会破坏合法访问，限制过松则可能绕过现有网络策略，这属于代理治理而非前端调用细节。

498. 谈一谈GET 和 POST 的区别

GET和POST的本质区别是语义：前者获取资源，后者提交资源。因为用途不同，GET通常是幂等的，也更容易被浏览器缓存；POST默认不缓存，重复提交还可能改变资源状态。参数一般分别放在URL和请求体中，但这不代表POST天然安全，抓包时两者都能被看到。至于报文如何拆成TCP数据包，不能简单当作两种方法的固定区别。本质上，只是语义上的区别，GET 用于获取资源，POST 用于提交资源。具体差别从缓存角度看，GET 请求后浏览器会主动缓存，POST 默认情况下不能。从参数角度来看，GET请求一般放在URL中，因此不安全，POST请求放在请求体中，相对而言较为安全，但是在抓包的情况下都是一样的。从编码角度看，GET请求只能经行URL编码，只能接受ASCII码，而POST支持更多的编码类型且不对数据类型限值。GET请求幂等，POST请求不幂等，幂等指发送 M 和 N 次请求（两者不相同且都大于1），服务器上资源的状态一致。GET请求会一次性发送请求报文，POST请求通常分为两个TCP数据包，首先发 header 部分，如果服务器响应 100(continue)，然后发 body 部分。

1) 登录页负责人说把密码从GET改成POST就已经安全，因为地址栏看不到参数；安全评审会上你会怎么反驳？

答：POST只是把数据放入请求体，并不会自动加密，抓包时仍可看到明文内容，传输安全必须依赖HTTPS。它能减少参数直接出现在URL、历史记录或常见访问日志中的机会，但服务端日志和调试工具仍可能记录请求体，敏感数据还需最小化留存。

2) 商品查询接口使用GET，筛选条件变化不频繁，前端却每次都强制绕过缓存；从方法语义和浏览器行为看，你会怎么改？

答：GET用于获取资源且具备幂等语义，浏览器默认更容易对其结果进行缓存，应结合响应缓存策略复用稳定查询结果。是否真正缓存仍由响应头和中间节点决定；若结果与用户身份或实时库存相关，就要设置正确的缓存边界，不能只凭方法放开共享缓存。

3) 报表查询条件包含嵌套对象，序列化后不适合放进URL，团队准备改成POST；产品又要求链接可收藏，你如何取舍？

答：复杂条件放入请求体时使用POST是可接受的工程折中，但会失去GET查询地址天然可分享、默认易缓存的优势。可把条件保存为服务端资源并返回短标识，再用GET访问该标识；代价是增加存储、权限、过期清理和链接生命周期管理。

4) 支付确认接口使用POST，客户端超时后自动重试，线上出现重复处理；开发认为网络层会保证一次请求只执行一次，你会如何排查？

答：POST通常具有副作用且不保证幂等，超时只能说明客户端没收到结果，不能证明服务端没有执行。应先用业务请求标识核对服务端处理记录，再在支付侧实现幂等校验；依赖传输可靠性无法消除重试造成的重复业务动作。

5) 抓包发现一个小型POST请求并未先收到100

Continue，而是请求头和请求体一起发送；同事据此判断客户端违反协议，你认可吗？

答：不认可，POST并不必然拆成两个TCP数据包，是否等待100 Continue取决于客户端是否使用相应机制、请求大小和具体实现。TCP分段也不能直接等同于HTTP语义边界；排查应查看请求头是否包含Expect: 100-continue，不能用单次抓包概括所有POST。

499. 谈一谈队头阻塞问题

队头阻塞就是同一任务队列串行处理请求时，队首请求过慢，导致后续请求只能等待。在一个域名只使用少量长连接的情况下，每条连接都像一条独立队列，因此增加并发连接可以分散阻塞风险。域名分片也是类似思路：把资源放到多个二级域名，让客户端建立更多连接并行处理。不过这类方案本质上是增加队列数量，并没有让单条队列摆脱慢请求的影响。什么是队头阻塞？对于每一个HTTP请求而言，这些任务是被放入一个任务队列中串行执行的，一旦队首任务请求太慢时，就会阻塞后面的请求处理，这就是HTTP队头阻塞问题。有什么解决办法吗 并发连接 我们知道对于一个域名而言，是允许分配多个长连接的，那么可以理解成增加了任务队列，也就是说不会导致一个任务阻塞了该任务队列的其他任务，在RFC规范中规定客户端最多并发2个连接，不过实际情况就是要比这个还要多，举个例子，Chrome中是6个。域名分片 顾名思义，我们可以在一个域名下分出多个二级域名出来，而它们最终指向的还是同一个服务器，这样的话就可以并发处理的任务队列更多，也更好的解决了队头阻塞的问题。举个例子，比如TianTian.com，可以分出很多二级域名，比如Day1.TianTian.com，Day2.TianTian.com,Day3.TianTian.com,这样子就可以有效解决队头阻塞问题。

1) 商品详情页同时请求价格、库存和推荐列表，推荐接口最先入队却响应很慢，后面的价格请求一定会被它阻塞吗？

答：不一定，只有这些请求落在同一条串行处理队列中，队首的慢请求才会挡住后续请求。若浏览器已为该域名建立多条连接，请求可能分配到不同队列；但同一连接内的排队关系仍然存在。

2) 一个HTTP/1.1页面首屏要加载几十个静态资源，瀑布图里大量请求长期处于排队状态，前端和服务端分别能做什么？

答：可以让浏览器对同一域名使用多条长连接，相当于增加并行处理队列，避免所有资源都被一个慢请求拖住。服务端还应缩短队首请求的处理时间；连接数并非无限，资源过多时仍会排队。

3) 团队准备把图片拆到img1.example.com、img2.example.com，但这些二级域名最终都指向同一台服务器，这样还能缓解阻塞吗？

答：仍可能缓解，因为浏览器按域名分配连接，多个二级域名能够扩展可并发的请求队列，即使它们最终指向同一服务器。代价是域名、连接与运维复杂度增加，后端处理能力不足时也只是把等待位置后移。

4) 线上活动页只有某个域名下的资源成批卡住，其他域名资源正常，抓包又没有发现所有接口都变慢，你会怎样定位？

答：先按域名和连接查看请求瀑布图，确认卡住的资源是否排在同一慢请求之后，再定位队首请求的服务端耗时。若换到另一连接后请求立即恢复，更符合队头阻塞；若所有连接都慢，应继续排查服务器容量或网络。

5) 架构师主张无限增加并发连接，前端负责人主张域名分片；面对大量小资源，你会怎样取舍？

答：两种方案本质上都在增加请求队列，但连接数受浏览器策略约束，不能无限扩张；域名分片则用多个二级域名换取更多并发。应先控制资源数量和慢请求，再谨慎分片，否则会引入额外连接与域名管理成本。

500. 谈一谈HTTP数据传输

HTTP/1.1传输数据时，定长内容通常用Content-Length，长度无法预先确定时使用Transfer-Encoding: chunked。Content-Length必须与实际传输长度一致，过短会截断，过长可能导致接收方继续等待；使用压缩时填写的是压缩后的长度。分块传输适合长连接中持续产生的动态内容，接收方根据各个分块判断数据边界。如果两者同时出现，应按Transfer-Encoding处理并忽略Content-Length。

大概遇到的情况就分为定长数据与不定长数据的处理吧。

定长数据

对于定长的数据包而言，发送端在发送数据的过程中，需要设置Content-Length,来指明发送数据的长度。当然了如果采用了Gzip压缩的话，Content-Length设置的就是压缩后的传输长度。

我们还需要知道的是 Content-Length如果存在并且有效的话，则必须和消息内容的传输长度完全一致，也就是说，如果过短就会截断，过长的话，就会导致超时。如果采用短链接的话，直接可以通过服务器关闭连接来确定消息的传输长度。那么在HTTP/1.0之前的版本中，Content-Length字段可有可无,因为一旦服务器关闭连接，我们就可以获取到传输数据的长度了。在HTTP/1.1版本中，如果是Keep-alive的话，chunked优先级高于Content-Length,若是非Keep-alive，跟前面情况一样，Content-Length可有可无。

那怎么来设置Content-Length

举个例子来看看

```
const server = require('http').createServer();
server.on('request', (req, res) => {
  if (req.url === '/index') {
    // 设置数据类型
    res.setHeader('Content-Type', 'text/plain');
    res.setHeader('Content-Length', 10);
    res.write("你好，使用的是Content-Length设置传输数据形式");
  }
});
server.listen(3000, () => {
  console.log("成功启动--TinaTian");
});
```

不定长数据 现在采用最多的就是HTTP/1.1版本，来完成传输数据，在保存Keep-alive状态下，当数据是不定长的时候，我们需要设置新的头部字段

Transfer-Encoding: chunked

通过chunked机制，可以完成对不定长数据的处理，当然了，你需要知道的是 如果头部信息中有Transfer-Encoding,优先采用Transfer-Encoding里面的方法来找到对应的长度。如果设置了Transfer-Encoding，那么Content-Length将被忽视。使用长连接的话，会持续的推送动态内容。

那我们来模拟一下吧

```
const server = require('http').createServer();
server.on('request', (req, res) => {
  if (req.url === '/index') {
    // 设置数据类型
    res.setHeader('Content-Type', 'text/html; charset=utf8');
    res.setHeader('Content-Length', 10);
    res.setHeader('Transfer-Encoding', 'chunked');
    res.write("你好，使用的是Transfer-Encoding设置传输数据形式");
    setTimeout(() => {
      res.write("第一次传输数据给您");
    }, 1000);
    res.write("骚等一下");
    setTimeout(() => {
      res.write("第一次传输数据给您");
      res.end();
    }, 3000);
  }
});
server.listen(3000, () => {
  console.log("成功启动--TinaTian");
});
```

上面使用的是nodejs中http模块，有兴趣的小伙伴可以去试一试，以上就是HTTP对定长数据和不定长数据传输过程中的处理手段。

1) 下载页收到响应头Content-Length:

10，服务端实际却继续写入更长的正文，前端为什么不能把多出的内容当成下一段数据？

答：有效的Content-Length必须与消息的传输长度一致，接收方会按该长度划定当前消息边界，长度过短可能造成正文截断。它不是业务分段标记，错误复用还会破坏长连接上后续消息的解析。

2) 文件服务先用Gzip压缩再返回，产品要求进度条按原文件大小计算，直接读取Content-Length能满足吗？

答：不能，启用Gzip后，Content-Length表示压缩后的实际传输长度，而不是解压后的原始大小。前端可据此展示网络传输进度；若要显示原文件进度，需要服务端另行提供可靠的原始尺寸。

3) 报表接口必须保持Keep-Alive，生成过程中又无法提前知道总长度，后端应该怎样让客户端识别响应边界？

答：在HTTP/1.1长连接下，应使用Transfer-Encoding: chunked，把动态生成的内容分块发送，并由分块机制标识结束。这样无需预先计算总长度，但客户端在响应开始时也无法仅靠标准长度字段得到最终总量。

4) 线上接口同时返回Content-Length和Transfer-Encoding:

chunked，监控发现部分客户端展示内容异常，你先按哪个字段判断？

答：应优先按Transfer-Encoding指定的方式解析，存在该字段时Content-Length会被忽略。排查时应确认服务端为何同时写入冲突头部，并检查每个分块及结束标记；保留双重声明容易造成实现差异。

5) 旧接口通过服务端写完后关闭连接来标记正文结束，迁移到长连接后偶发超时，根因可能在哪里？

答：关闭连接能够为短连接提供消息结束边界，但长连接需要继续复用，不能再依赖断开来判断正文长度。迁移后应提供正确的Content-Length，或对不定长内容使用chunked；两者都缺失时客户端可能持续等待。

501. cookie 和 session

cookie保存在浏览器，session通常保存在服务端，两者一般通过sessionId关联。登录后，服务端创建会话并把sessionId交给浏览器，后续请求再通过cookie自动携带它。简单来说，cookie是HTTP头里的字段，而session是服务端维护会话状态的一种抽象和实现。多机部署时还要共享session数据，否则请求切换到另一台机器后可能找不到登录状态。session：是一个抽象概念，开发者为了实现中断和继续等操作，将user agent和server之间一对一的交互，抽象为“会话”，进而衍生出“会话状态”，也就是session的概念cookie：它是一个广泛存在的东西，http协议中定义在header中的字段，可以认为是session的一种后端无状态实现。现在我们常说的session，是为了绕开cookie的各种限制，通常借助cookie本身和后端存储实现的，一种更高级的会话状态实现。session的常见实现要借助cookie来发送sessionId

1) 登录接口给浏览器写入一个sessionId，安全评审却说“既然用了cookie，服务端就没有session”，你会怎么纠正？

答：这个判断把载体和会话状态混为一谈了，cookie是定义在HTTP header中的机制，而常见session会借助它传递sessionId。真正的会话状态通常保存在后端，浏览器持有的只是用于关联状态的标识。

2) 客服后台要求员工关闭页面后再次进入仍能继续操作，后端准备使用session，请求链路最少要怎样串起来？

答：后端需要创建并保存会话状态，再通过cookie把对应的sessionId交给浏览器，后续请求携带它以恢复同一会话。若浏览器没有回传该标识，或后端找不到对应记录，中断后的状态就无法继续。

3)

移动端容器明确禁止持久化cookie，服务端仍坚持现有的sessionId方案，会受到什么约束？

答：常见session实现依赖客户端在后续请求中持续发送sessionId，禁止保存或回传cookie会切断这种关联。可以改用其他受控方式传递标识，但那已经改变了载体设计，仍需保证每次请求能绑定到正确会话。

4) 线上出现用户登录成功后下一次请求立即变成未登录，接口响应里已经看到Set-Cookie，你会沿哪条链路排查？

答：先确认浏览器是否实际保存该cookie，再检查后续请求是否携带同一个sessionId，最后核对后端存储中是否存在对应会话。只看到Set-Cookie不能证明链路完整，任何一段丢失都会表现为会话中断。

5) 无状态接口负责人认为只用cookie保存全部状态更简单，平台负责人坚持cookie + sessionId + 后端存储，取舍重点是什么？

答：把状态交给cookie能减少后端会话存储，但会受到客户端载体自身限制；常见session则用较小的sessionId关联后端状态。后者更适合需要中断后继续的复杂会话，代价是维护会话存储及标识映射。

502. 介绍一下HTTPS和HTTP区别

HTTPS本质上是HTTP与SSL/TLS的组合，相比明文传输的HTTP，它能加密数据并认证服务器身份。HTTP默认使用80端口，HTTPS默认使用443端口，而且后者需要配置和维护证书。加密可以降低数据被窃听或篡改的

风险，也更利于搜索引擎收录。它的取舍是会增加证书维护、握手和计算开销，不过合理优化后访问速度的影响通常可以控制。HTTPS 要比 HTTP 多了 secure 安全性这个概念，实际上，HTTPS 并不是一个新的应用层协议，它其实就是 HTTP + TLS/SSL 协议组合而成，而安全性的保证正是 SSL/TLS 所做的工作。SSL 安全套接层 (Secure Sockets Layer) TLS (传输层安全, Transport Layer Security) 现在主流的版本是 TLS/1.2, 之前的 TLS1.0、TLS1.1 都被认为是不安全的，在不久的将来会被完全淘汰。HTTPS 就是身披了一层 SSL 的 HTTP。那么区别有哪些呢 HTTP 是明文传输协议，HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，比 HTTP 协议安全。HTTPS比HTTP更加安全，对搜索引擎更友好，利于SEO,谷歌、百度优先索引HTTPS网页。HTTPS标准端口443，HTTP标准端口80。HTTPS需要用到SSL证书，而HTTP不用。我觉得记住以下两点HTTPS主要作用就行 对数据进行加密，并建立一个信息安全通道，来保证传输过程中的数据安全;对网站服务器进行真实身份认证。 HTTPS的缺点证书费用以及更新维护。HTTPS 降低一定用户访问速度（实际上优化好就不是缺点了）。HTTPS 消耗 CPU 资源，需要增加大量机器。

1) 支付页已经把端口从80改成443，运维就宣称链路具备HTTPS安全性，这个判断成立吗？

答：不成立，端口变化本身不会提供加密或身份认证，HTTPS是HTTP与TLS/SSL组合形成的安全通信。服务端还必须正确配置证书和安全协议，否则即使监听443，也不能据此证明连接可信。

2)

资讯站准备全站从HTTP迁移到HTTPS，产品只关心搜索收录，工程侧还应明确哪些直接收益？

答：迁移后传输内容由TLS/SSL建立的安全通道保护，并可通过证书认证网站服务器身份，不只是影响搜索引擎表现。实施时需要配置证书、更新维护并评估资源消耗，不能只把页面链接替换成https://。

3) 内网站点不传密码，负责人因此认为明文HTTP足够；但页面会下发业务数据，你会怎样评估？

答：不传密码并不等于没有风险，HTTP的业务数据仍以明文传输，也缺少HTTPS提供的服务器身份认证。若数据需要防止链路窃听或站点冒充，就应使用HTTPS；证书维护和计算资源仍需纳入成本。

4) 线上切换HTTPS后浏览器持续提示证书相关错误，而接口代码和响应内容都正常，你先查什么？

答：应先检查服务端是否提供了与站点匹配且可验证的证书，并确认连接确实通过TLS/SSL建立，而不是只开放了443端口。应用响应正常不能替代身份认证，证书链路失败时浏览器仍会把站点视为不可信。

5) 预算评审认为HTTPS只有证书费用，没有运行成本；基础设施团队要求扩容，你如何给出保守结论？

答：除证书采购或更新维护外，HTTPS的加密与握手还会消耗一定计算资源，并可能带来访问延迟。具体影响取决于实现和优化，不能凭题面给出固定数字；应通过实际负载评估容量，而不是直接认定成本为零。

503. HTTPS握手过程

HTTPS握手的核心，是先验证服务器证书，再安全地协商出后续通信使用的对称密钥。客户端发送支持的协议、加密方法和随机数，服务端确认参数并返回证书及自己的随机数。客户端验证证书后生成Premaster secret，用证书中的公钥加密发送，服务端再用私钥解密。双方根据这些随机数生成会话密钥，后续数据便改用效率更高的对称加密传输。第一步，客户端给出协议版本号、一个客户端生成的随机数 (Client random)，以及客户端支持的加密方法第二步，服务端确认双方使用的加密方法，并给出数字证书、以及一个服务器生成的随机数第三步，客户端确认数字证书有效，然后生成一个新的随机数 (Premaster secret)，并使用数字证书中的公钥，加密这个随机数，发给服务端第四步，服务端使用自己的私钥，获取客户端

发来的随机数（即Premaster secret）。第五步，客户端和服务端根据约定的加密方法，使用前面的三个随机数，生成“对话密钥”（session key），用来加密接下来的整个对话过程。总结：客户端发起 HTTPS 请求，服务端返回证书，客户端对证书进行验证，验证通过后本地生成用于构造对称加密算法的随机数，通过证书中的公钥对随机数进行加密传输到服务端（随机对称密钥），服务端接收后通过私钥解密得到随机对称密钥，之后的数据交互通过对称加密算法进行加解密。（既有对称加密，也有非对称加密）

1) 安全评审提议客户端首次访问时直接生成对称密钥并明文发给服务器，认为后续正文已经加密就足够，这里错在哪里？

答：明文发送会让链路中的观察者直接获得对称密钥，后续加密也就失去意义。典型握手会让客户端生成Premaster secret，使用证书中的公钥加密后传输，服务端再用私钥解开。

2) 浏览器访问登录页时，服务端已经返回数字证书和Server random，客户端接下来应完成哪些关键动作？

答：客户端应先验证数字证书是否有效，再生成Premaster secret，并用证书中的公钥加密后发送给服务端。若证书验证失败，就不应继续建立受信任会话，否则公钥来源无法得到保证。

3) 后端负责人想让整个订单响应都使用证书公钥加密，前端负责人坚持握手后改用对称加密，哪种更符合该流程？

答：握手中的非对称加密主要用于安全传递Premaster secret，后续双方根据三个随机数生成session key，再用它加密整个对话。持续使用公钥并不符合这套流程，也混淆了密钥交换与数据传输的职责。

4) 线上只有部分实例在握手阶段失败，抓包显示客户端已发出加密后的Premaster secret，但服务端无法继续，你会先排查什么？

答：应先确认故障实例是否持有与所发证书公钥匹配的私钥，因为服务端必须用该私钥解开Premaster secret。再核对双方确认的加密方法及随机数处理；私钥或协商信息不一致都会阻断会话密钥生成。

5) 网关团队认为只校验证书即可省略Client random和Server random，应用团队认为三份随机材料都要保留，如何判断？

答：按该握手流程，客户端与服务端会结合Client random、Server random和Premaster secret生成相同的session key。证书验证只确认公钥及服务端身份，不能替代会话密钥材料；省略随机数会改变既定协商过程。

504. 介绍一个HTTPS工作原理

HTTPS通过证书证明服务器身份，用非对称加密完成密钥交换，再用对称加密传输实际数据。只用对称加密会遇到密钥无法安全传递的问题，只用非对称加密又不适合持续处理大量通信，所以两者需要配合。浏览器会利用受信任认证机构的公钥验证证书和数字签名，避免服务器公钥被中间人替换。验证通过后双方取得相同的对称密钥，后续HTTP内容都用它加密和解密。我们可以把HTTPS理解成HTTPS = HTTP + SSL/TLS。SSL/TLS的功能实现主要依赖于三类基本算法：散列函数、对称加密和非对称加密，其利用非对称加密实现身份认证和密钥协商，对称加密算法采用协商的密钥对数据加密，基于散列函数验证信息的完整性。

1. 对称加密 加密和解密用同一个密钥的加密方式叫做对称加密。Client客户端和Server端共用一套密钥，这样子的加密过程似乎很让人理解，但是随之会产生一些问题。

问题一:WWW万维网有许许多多的客户端，不可能都用密钥A进行信息加密，这样子很不合理，所以解决办法就是使用一个客户端使用一个密钥进行加密。

问题二:既然不同的客户端使用不同的密钥，那么对称加密的密钥如何传输？那么解决的办法只能是一端生成一个密钥，然后通过HTTP传输给另一端，那么这样子又会产生新的问题。

问题三:这个传输密钥的过程，又如何保证加密？如果被中间人拦截，密钥也会被获取,那么你会说对密钥再进行加密，那又怎么保存对密钥加密的过程，是加密的过程？

到这里，我们似乎想明白了，使用对称加密的方式，行不通，所以我们需要采用非对称加密 2. 非对称加密 通过上面的分析，对称加密的方式行不通，那么我们来梳理一下非对称加密。采用的算法是RSA，所以在一些文章中也会看见传统RSA握手，基于现在TLS主流版本是1.2，所以接下来梳理的是TLS/1.2握手过程。

非对称加密中，我们需要明确的点是 有一对密钥，公钥和私钥。公钥加密的内容，只有私钥可以解开，私钥加密的内容，所有的公钥都可以解开，这里说的公钥都可以解开，指的是一对密钥。公钥可以发送给所有的客户端，私钥只保存在服务器端。 3. 主要工作流程 梳理起来，可以把TLS 1.2握手过程分为主要的五步

步骤一：Client发起一个HTTPS请求，连接443端口。这个过程可以理解成是请求公钥的过程。步骤二：Server端收到请求后，通过第三方机构私钥加密，会把数字证书（也可以认为是公钥证书）发送给Client。步骤三：浏览器安装后会自带一些权威第三方机构公钥，使用匹配的公钥对数字签名进行解密。根据签名生成的规则对网站信息进行本地签名生成，然后两者比对。通过比对两者签名，匹配则说明认证通过，不匹配则获取证书失败。步骤四：在安全拿到服务器公钥后，客户端Client随机生成一个对称密钥，使用服务器公钥（证书的公钥）加密这个对称密钥，发送给Server(服务器)。步骤五：Server(服务器)通过自己的私钥，对信息解密，至此得到了对称密钥，此时两者都拥有了相同的对称密钥。浏览器安装后会自带一些权威第三方机构公钥，使用匹配的公钥对数字签名进行解密。根据签名生成的规则对网站信息进行本地签名生成，然后两者比对。通过比对两者签名，匹配则说明认证通过，不匹配则获取证书失败。接下来，就可以通过该对称密钥对传输的信息加密/解密啦，从上面图举个例子 Client用户使用该对称密钥加密'明文内容B',发送给Server(服务器)Server使用该对称密钥进行解密消息，得到明文内容B。

接下来考虑一个问题，如果公钥被中间人拿到篡改怎么办呢？客户端可能拿到的公钥是假的，解决办法是什么呢？ 3. 第三方认证 客户端无法识别传回公钥是中间人的，还是服务器的，这是问题的根本，我们是不是可以通过某种规范可以让客户端和服务器都遵循某种约定呢？那就是通过第三方认证的方式 在HTTPS中，通过证书+数字签名来解决这个问题。这里唯一不同的是，假设对网站信息加密的算法是MD5，通过MD5加密后，然后通过第三方机构的私钥再次对其加密，生成数字签名。这样的话，数字证书包含有两个特别重要的信息某网站公钥+数字签名 我们再次假设中间人截取到服务器的公钥后，去替换成自己的公钥，因为有数字签名的存在，这样子客户端验证发现数字签名不匹配，这样子就防止中间人替换公钥的问题。那么客户端是如何去对比两者数字签名的呢？浏览器会去安装一些比较权威的第三方认证机构的公钥，比如VeriSign、Symantec以及GlobalSign等等。验证数字签名的时候，会直接从本地拿到相应的第三方的公钥，对私钥加密后的数字签名进行解密得到真正的签名。然后客户端利用签名生成规则进行签名生成，看两个签名是否匹配，如果匹配认证通过，不匹配则获取证书失败。 4. 数字签名作用 数字签名：将网站的信...

1) 登录页已经用HTTPS，有人却说只要拿到服务器证书里的公钥就能解密所有用户请求，你会怎么反驳？

答：证书公钥不是后续业务数据的直接解密密钥，它主要用于验证服务器身份并参与握手阶段的密钥协商。握手完成后，双方使用协商出的对称密钥加密请求与响应；服务器私钥不能公开，否则身份认证和密钥交换的安全基础都会受损。

2)

一个首页要加载HTML、接口和几十个静态资源，为什么TLS不用非对称算法加密全部内容？

答：非对称加密适合身份认证和安全协商密钥，但不适合持续承担大量业务数据的加解密。连接建立后改用同一把对称密钥处理传输，能兼顾效率与机密性；握手阶段仍需证书和数字签名，不能只发送一个裸公钥。

3) 公司内网代理把站点公钥替换成自己的公钥，但浏览器仍然显示安全锁，这一定说明证书校验失效了吗？

答：不一定，企业代理可能已把自己的根证书安装进设备信任库，再为目标域名动态签发可被本机接受的证书。浏览器仍会校验证书链、域名和签名，只是信任锚发生了变化；若设备并未授权该根证书

，替换公钥通常会触发证书错误。

4) 某域名换证后桌面浏览器正常，部分老客户端却提示证书不可信，你会沿着握手流程查什么？

答：先检查服务端是否发送了完整证书链，再核对证书域名、有效期以及客户端设备时间。桌面浏览器可能补全缺失的中间证书，而部分旧客户端不能，因此同一域名会表现不同；若处于企业网络，还要排查代理证书是否被客户端信任。

5) 团队提议为了简化部署直接使用自签名证书，面向公众的支付页能否接受？

答：公众支付页不适合直接采用未被客户端信任的自签名证书，因为浏览器无法沿受信任证书链确认服务器身份。用户手动忽略警告会削弱防中间人攻击的关键保障；自签名方案更适合已预先分发并严格管理信任根的封闭环境。

6) 证书验证已经保证了身份，为什么传输阶段还需要完整性校验，只有加密不够吗？

答：加密主要隐藏明文，并不天然等同于能够发现传输内容遭到修改。TLS还需要基于散列等机制验证数据完整性，使接收方识别被篡改的记录；身份认证、机密性和完整性分别处理不同风险，缺少任一项都不能完整替代。

505. SSL 连接断开后如何恢复

SSL连接断开后，可以通过Session ID或Session Ticket复用上次的会话信息，避免重新生成密钥。Session ID由服务端保存，客户端重连时带上编号，但请求被负载均衡到另一台服务器后可能无法恢复。Session Ticket由客户端保存，服务端解密后即可取回密钥和加密方式等信息，因此更适合多服务器场景。

一共有两种方法来恢复断开的 SSL 连接，一种是使用 session ID，一种是 session ticket。通过session ID使用 session ID 的方式，每一次的会话都有一个编号，当对话中断后，下一次重新连接时，只要客户端给出这个编号，服务器如果有这个编号的记录，那么双方就可以继续使用以前的秘钥，而不用重新生成一把。目前所有的浏览器都支持这种方法。但是这种方法有一个缺点是，session ID 只能够存在一台服务器上，如果我们的请求通过负载均衡被转移到了其他的服务器上，那么就无法恢复对话。通过session ticket另一种方式是 session ticket 的方式，session ticket 是服务器在上一次对话中发送给客户的，这个 ticket 是加密的，只有服务器能够解密，里面包含了本次会话的信息，比如对话秘钥和加密方法等。这样不管我们的请求是否转移到其他的服务器上，当服务器将 ticket 解密以后，就能够获取上次对话的信息，就不用重新生成对话秘钥了。

1) 用户关闭页面十秒后重新访问，客户端带着旧session ID，为什么服务端仍可能重新握手？

答：session ID只有在服务端仍保存对应会话记录时才能恢复，编号本身不包含完整会话状态。记录可能已过期、被清理，或请求被负载均衡到另一台没有该记录的服务器；这些情况下只能重新生成会话密钥。

2) 三台网关轮询承接TLS，上线后会话恢复命中率明显下降，你会优先调整什么？

答：若使用session ID，应让节点共享会话状态，或通过负载均衡保证后续连接落到保存记录的节点。也可改用服务端可解密的session ticket，降低对单机状态的依赖；无论选择哪种方式，都要处理过期和密钥管理。

3) 团队把网关从单机扩成二十个实例，session ID和session ticket哪个更适合横向扩容？

答：在各实例能够解密同类票据的前提下，session ticket更适合横向扩容，因为会话信息随客户端票据返回，不依赖命中特定节点。session ID则要求节点保存并找到编号对应的记录；共享状态或会话粘滞会增加系统复杂度。

4) 监控显示恢复连接全部退化为完整握手，但页面请求仍能成功，你如何区分是票据故障还是网络故障？

答：请求仍成功说明基础连接和完整握手大概率可用，应重点检查客户端是否携带票据、票据是否过期，以及当前节点能否解密。若近期轮换过票据加密密钥或新增节点，还要核对节点配置是否一致；恢复失败通常会回退握手，不一定直接报业务错误。

5) 安全团队要求频繁轮换session ticket加密密钥，性能团队担心恢复率下降，怎么取舍？

答：票据只有服务器能够解密，因此加密密钥的生命周期直接影响安全边界和可恢复窗口。轮换时可以让节点在受控期限内兼容必要的旧密钥，再逐步淘汰；保留过久会扩大密钥泄露影响，立即切断则会让旧票据全部失效。

6) 前端想靠preconnect解决移动网络频繁断连，它和会话恢复处理的是同一件事吗？

答：两者不是同一机制：preconnect是提前建立到目标源的连接，会话恢复则是在新连接中复用先前协商的信息。前者只能把连接成本前移，后者才可能避免重新生成会话密钥；若票据无效或服务端没有session ID记录，仍会完整握手。

506. 谈一谈你对HTTP/2理解

HTTP/2的核心是二进制分帧和多路复用，让多个请求、响应可以共享一条TCP连接并发传输。不同流的帧可以乱序到达，再通过Stream ID重新组装，从而缓解HTTP/1.1请求排队造成的队头阻塞。它还用HPACK压缩头部，并支持服务器推送。边界是底层仍然使用TCP，一旦丢包等待重传，同一连接中的所有流都会受到影响。首先补充一下，http 和 https 的区别，相比于 http,https 是基于 ssl 加密的 http 协议
简要概括:http2.0是基于 1999 年发布的http1.0之后的首次更新

提升访问速度(可以对于，请求资源所需时间更少，访问速度更快，相比

http1.0)允许多路复用:多路复用允许同时通过单一的 HTTP/2 连接发送多重请求-响应信息。改善了:在http1.1中，浏览器客户端在同一时间，针对同一域名下的请求有一定数量限

制(连接数量)，超过限制会被阻塞二进制分帧:HTTP2.0

会将所有的传输信息分割为更小的信息或者帧，并对他们进行二进制编码首部压缩服务器端推送 头部压缩
HTTP 1.1版本会出现User-Agent、Cookie、Accept、Server、Range等字段可能会占用几百甚至几千字节，而 Body 却经常只有几十字节，所以导致头部偏重。 HTTP 2.0 使用HPACK算法进行压缩。 多路复用
HTTP 1.x 中，如果想并发多个请求，必须使用多个 TCP 链接，且浏览器为了控制资源，还会对单个域名有6-8个的TCP链接请求限制。 HTTP2中： 同域名下所有通信都在单个连接上完成。单个连接可以承载任意数量的双向数据流。数据流以消息的形式发送，而消息又由一个或多个帧组成，多个帧之间可以乱序发送，因为根据帧首部的流标识可以重新组装，也就是Stream ID，流标识符，有了它，接收方就能从乱序的二进制帧中选择ID相同的帧，按照顺序组装成请求/响应报文。服务器推送 浏览器发送一个请求，服务器主动向浏览器推送与这个请求相关的资源，这样浏览器就不用发起后续请求。 相比较http/1.1的优势 推送资源可以由不同页面共享服务器可以按照优先级推送资源客户端可以缓存推送的资源客户端可以拒收推送过来的资源 二进制分帧 之前是明文传输，不方便计算机解析，对于回车换行符来说到底是内容还是分隔符，都需要内部状态机去识别，这样子效率低，HTTP/2采用二进制格式，全部传输01串，便于机器解码。 这样子一个报文格式就被拆分为一个个二进制帧，用Headers帧存放头部字段，Data帧存放请求体数据。这样子的话，就是一堆乱序的二进制帧，它们不存在先后关系，因此不需要排队等待，解决了HTTP队头阻塞问题。 在客户端与服务器之间，双方都可以互相发送二进制帧，这样子双向传输的序列，称为流，所以HTTP/2中以流来表示一个TCP连接上进行多个数据帧的通信，这就是多路复用概念。

那乱序的二进制帧，是如何组装成对于的报文呢？

所谓的乱序，值的是不同ID的Stream是乱序的，对于同一个Stream

ID的帧是按顺序传输的。接收方收到二进制帧后，将相同的Stream ID组装成完整的请求报文和响应报文。

二进制帧中有一些字段，控制着优先级和流量控制等功能，这样的话，就可以设置数据帧的优先级，让服务器处理重要资源，优化用户体验。HTTP2的缺点 TCP 以及 TCP+TLS建立连接的延时,HTTP/2使用TCP协议来传输的，而如果使用HTTPS的话，还需要使用TLS协议进行安全传输，而使用TLS也需要一个握手过程,在传输数据之前，导致我们需要花掉 3~4 个 RTT。TCP的队头阻塞并没有彻底解决。在HTTP/2中，多个请求是跑在一个TCP管道中的。但当HTTP/2出现丢包时，整个 TCP 都要开始等待重传，那么就会阻塞该TCP连接中的所有请求。

1) 商品页的一个大图片响应很慢，同域名的接口却还能并发返回，这能说明HTTP/2已彻底消除队头阻塞吗？

答：只能说明不同Stream ID的请求不必像HTTP/1.1那样按响应顺序等待，不能说明底层阻塞已消失。多个流仍共享一条TCP连接，一旦发生丢包，重传等待可能影响连接上的所有流；这是HTTP/2多路复用的边界。

2) 站点把接口、脚本和图片都放在同一域名，Network面板却出现一条连接上的大量并发请求，你如何解释报文不会串？

答：HTTP/2会把头部和数据体拆成二进制帧，并用Stream ID标识所属数据流。不同流的帧可以交错传输，接收端再按标识重组为各自的请求或响应；同一数据流内部仍需保持可正确组装的顺序。

3) 迁移到HTTP/2后，团队仍把静态资源拆到四个域名以突破每域名连接数限制，这个策略还合理吗？

答：该策略的收益会明显减弱，因为HTTP/2可在同域名的一条连接上承载多个双向数据流，不再依赖多条连接实现请求并发。继续分片会额外建立连接，并使多路复用和头部压缩分散到不同连接；若存在独立缓存或权限边界，仍需单独评估。

4) 线上只有弱网用户出现首页所有资源同时卡住，服务器CPU和接口耗时都正常，你会怀疑哪一层？

答：应检查共享TCP连接是否出现丢包和重传，因为HTTP/2的多个流都运行在这条连接上。某个包丢失时，底层TCP的有序交付可能阻塞所有流；还要结合连接复用情况排除资源其实来自不同域名或不同协议。

5) 一个接口响应体只有几十字节，但每次都携带较大的Cookie和User-Agent，升级HTTP/2能否替代头部治理？

答：HPACK会利用连接内的表压缩重复头字段，后续请求可减少重复传输，因此小响应通常受益明显。但首次出现的字段仍需发送，频繁变化或过大的Cookie也不会凭空消失；协议压缩不能替代删减无用头部。

6)

服务端准备主动推送首页可能需要的所有脚本，前端认为越多越快，你会接受这个结论吗？

答：不能直接接受，服务器推送可以省去浏览器发起后续请求，但客户端可能已有缓存或根本不需要这些资源。客户端能够拒收或缓存推送资源，服务端也应结合优先级选择；推送过多会占用连接带宽，反而挤压当前页面的关键响应。

507. HTTP3

HTTP/3本质上是让HTTP运行在基于UDP的QUIC上，用独立数据流解决TCP层的队头阻塞。QUIC在UDP之上补上了重传、拥塞控制和流量控制，所以仍能提供可靠传输。它同时集成TLS 1.3，连接可以通过0-RTT或1-RTT快速建立。相比HTTP/2，其价值不只是多路复用，还在于某个流受阻时不会拖住其他流。 Google 在推SPDY的时候就已经意识到了这些问题，于是就另起炉灶搞了一个基于 UDP

协议的“QUIC”协议，让HTTP跑在QUIC上而不是TCP上。主要特性如下：实现了类似TCP的流量控制、传输可靠性的功能。虽然UDP不提供可靠性的传输，但QUIC在UDP的基础之上增加了一层来保证数据可靠性传输。它提供了数据包重传、拥塞控制以及其他一些TCP中存在的特性实现了快速握手功能。由于QUIC是基于UDP的，所以QUIC可以实现使用0-RTT或者1-RTT来建立连接，这意味着QUIC可以用最快的速度来发送和接收数据。集成了TLS加密功能。目前QUIC使用的是TLS1.3，相较于早期版本TLS1.3有更多的优点，其中最重要的一点是减少了握手所花费的RTT个数。多路复用，彻底解决TCP中队头阻塞的问题。

1) 视频页切到HTTP/3后抓包仍能看到丢包，测试同学因此认定UDP会让视频数据缺失，你怎么判断？

答：看到丢包不能直接推出业务数据缺失，QUIC在UDP之上实现了确认、重传、拥塞控制和流量控制。它仍提供可靠传输，只是不依赖TCP完成这些能力；若播放异常，应继续检查重传、拥塞和应用层处理。

2)

移动首页有多个接口并发加载，为什么把HTTP/2的TCP多路复用换成QUIC仍可能改善卡顿？

答：HTTP/2的流虽然彼此独立，但都共享同一条TCP连接，丢包会触发底层有序重传等待。QUIC在传输层提供多路复用，某个流的数据丢失时不必阻塞其他流；改善幅度仍取决于实际网络质量和部署情况。

3) 团队只把HTTPS证书配置搬到新网关，却没有准备TLS 1.3，还能按预期启用HTTP/3吗？

答：不能只把它当作普通UDP服务部署，QUIC集成了TLS 1.3加密与握手能力。网关必须同时支持对应的QUIC和加密协商，客户端也要能够选择该协议；任一环节不支持时，通常需要回退到其他可用协议。

4) 灰度HTTP/3后，公司网络用户全部回退，家庭网络正常，应用接口本身没有报错，你会先查哪里？

答：应先检查企业防火墙、代理或出口策略是否允许承载QUIC的UDP流量，再确认客户端与网关是否完成协议协商。业务接口正常只说明回退路径可用，不代表HTTP/3已建立；同时要核对服务端端口和协议支持是否一致。

5)

产品要求首个支付请求使用0-RTT追求最快提交，仅依据“QUIC支持快速握手”能否批准？

答：不能仅凭快速握手特性批准，0-RTT只说明连接可更早发送数据，不代表所有带副作用的请求都适合。涉及支付时还需评估早期数据可能被重放等安全边界，并由服务端限制可接受的请求类型；保守方案是完整握手后再提交。

6) 弱网下HTTP/3已减少连接层队头阻塞，是否就不再需要控制图片和脚本体积？

答：仍然需要，QUIC改善的是连接建立、可靠传输和多路复用中的阻塞方式，并不会减少资源本身的字节数。大资源依旧占用带宽并受拥塞控制影响，脚本还要解析执行；协议升级不能替代资源压缩、缓存和加载优先级治理。

508. HTTP/1.0 HTTP1.1 HTTP2.0版本之间的差异

HTTP/1.0、HTTP/1.1和HTTP/2的差异，可以理解为不断减少重复建连、请求排队和协议冗余。HTTP/1.0通常一次请求使用一次连接，但已支持多种内容格式以及GET、POST、HEAD。HTTP/1.1默认使用持久连接，并增加管线化、缓存控制、Range断点续传和Host。HTTP/2进一步采用二进制分帧、HPACK和多路复用，不过底层TCP的队头阻塞仍然存在。

HTTP 0.9：1991年,原型版本，功能简陋，只有一个命令GET,只支持纯文本内容，该版本已过时。HTTP 1.0任何格式的内容都可以发送，这使得互联网不仅可以传输文字，还能传输图像、视频、二进制等文件。除了GET命

令，还引入了POST命令和HEAD命令。http请求和回应的格式改变，除了数据部分，每次通信都必须包括头信息（HTTP header），用来描述一些元数据。只使用 header 中的 If-Modified-Since 和 Expires 作为缓存失效的标准。不支持断点续传，也就是说，每次都会传送全部的页面和数据。通常每台计算机只能绑定一个 IP，所以请求消息中的 URL 并没有传递主机名（hostname） HTTP 1.1 http1.1是目前最为主流的http协议版本，从1999年发布至今，仍是主流的http协议版本。引入了持久连接（persistent connection），即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive。长连接的连接时长可以通过请求头中的keep-alive来设置引入了管道机制（pipelining），即在同一个TCP连接里，客户端可以同时发送多个请求，进一步改进了HTTP协议的效率。HTTP 1.1 中新增加了 E-tag, If-Unmodified-Since, If-Match, If-None-Match 等缓存控制标头来控制缓存失效。支持断点续传，通过使用请求头中的Range来实现。使用了虚拟网络，在一台物理服务器上可以存在多个虚拟主机（Multi-homed Web Servers），并且它们共享一个IP地址。新增方法：PUT、PATCH、OPTIONS、DELETE。http1.x版本问题在传输数据过程中，所有内容都是明文，客户端和服务端都无法验证对方的身份，无法保证数据的安全性。HTTP/1.1 版本默认允许复用TCP连接，但是在同一个TCP连接里，所有数据通信是按次序进行的，服务器通常在处理完一个回应后，才会继续去处理下一个，这样子就会造成队头阻塞。http/1.x 版本支持Keep-alive，用此方案来弥补创建多次连接产生的延迟，但是同样会给服务器带来压力，并且的话，对于单文件被不断请求的服务，Keep-alive会极大影响性能，因为它在文件被请求之后还保持了不必要的连接很长时间。HTTP 2.0二进制分帧这是一次彻底的二进制协议，头信息和数据体都是二进制，并且统称为"帧"：头信息帧和数据帧。头部压缩HTTP 1.1版本会出现User-Agent、Cookie、Accept、Server、Range等字段可能会占用几百甚至几千字节，而 Body 却经常只有几十字节，所以导致头部偏重。HTTP 2.0 使用HPACK算法进行压缩。多路复用复用TCP连接，在一个连接里，客户端和浏览器都可以同时发送多个请求或回应，且不用按顺序一一对应，这样子解决了队头阻塞的问题。服务器推送允许服务器未经请求，主动向客户端发送资源，即服务器推送。请求优先级可以设置数据帧的优先级，让服务端先处理重要资源，优化用户体验。任何格式的内容都可以发送，这使得互联网不仅可以传输文字，还能传输图像、视频、二进制等文件。除了GET命令，还引入了POST命令和HEAD命令。http请求和回应的格式改变，除了数据部分，每次通信都必须包括头信息（HTTP header），用来描述一些元数据。只使用 header 中的 If-Modified-Since 和 Expires 作为缓存失效的标准。不支持断点续传，也就是说，每次都会传送全部的页面和数据。通常每台计算机只能绑定一个 IP，所以请求消息中的 URL 并没有传递主机名（hostname） 引入了持久连接（persistent connection），即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive。长连接的连接时长可以通过请求头中的keep-alive来设置引入了管道机制（pipelining），即在同一个TCP连接里，客户端可以同时发送多个请求，进一步改进了HTTP协议的效率。HTTP 1.1 中新增加了 E-tag, If-Unmodified-Since, If...

1)

一个下载接口在HTTP/1.0下中断后只能从头开始，换成HTTP/1.1为什么可能支持继续下载？

答：HTTP/1.1引入了基于Range请求头的范围请求，客户端可以请求尚未完成的字节区间，从而实现断点续传。前提是服务端和资源都支持范围响应；仅升级协议标识而未实现相应处理，下载仍可能从头开始。

2) 首页连续请求十个接口，团队说开启keep-alive就等于获得HTTP/2多路复用，你会如何反驳？

答：keep-alive只是让多个请求复用同一条TCP连接，不能让HTTP/1.1的响应像HTTP/2帧那样在多个流间交错传输。前一个响应处理缓慢时仍可能造成队头阻塞；HTTP/2还包含二进制分帧、头部压缩和流标识。

3)

迁移到HTTP/1.1后，同一IP上部署多个站点不再要求每站独占地址，协议层依赖了什么变化？

答：HTTP/1.1请求会携带主机信息，使服务器能够在共享IP上区分目标虚拟主机。网关必须依据主机名正确路由，并避免默认证书或站点配置错配；缺失或错误的主机信息仍可能被导向错误服务。

4) 升级HTTP/2后接口并发能力提高，但弱网下偶尔所有请求一起停顿，你会如何定位版本收益与故障边界？

答：先确认请求是否确实复用同一条HTTP/2连接，再检查该连接上的丢包和TCP重传。二进制分帧解决了应用层按响应次序等待的问题，但多个流仍共享TCP；底层丢包造成的队头阻塞不会因协议升级彻底消失。

5)

团队准备为HTTP/2重新拆分一个超大脚本包，但担心小请求数量增加，你会依据什么取舍？

答：HTTP/2的多路复用和HPACK降低了同连接内多个请求及重复头部的成本，因此不必沿用只为减少连接并发而合并资源的策略。拆分仍要考虑资源总字节数、缓存复用和执行开销；协议能力不会自动消除过度碎片化的管理成本。

6) 安全评审认为从HTTP/1.1升到HTTP/2就能解决明文窃听，这个结论成立吗？

答：不成立，版本升级关注分帧、压缩、多路复用、优先级和服务推送，并不等同于获得传输加密。保护身份、机密性和完整性仍需使用HTTPS所依赖的TLS；若链路仍是明文，协议性能特性不能阻止监听和篡改。

509. DNS如何工作的

DNS是把域名解析成IP地址的分布式查询系统，属于应用层协议，通常使用UDP的53端口。浏览器会先检查自身缓存、操作系统缓存和hosts文件，仍未命中时再请求本地DNS服务器。客户端到本地服务器通常是递归查询，本地服务器则依次向根域名、顶级域名和权威域名服务器迭代查询。解析结果会按TTL缓存；同一域名返回多个服务器地址时，也可以借此分摊请求。DNS的作用就是通过域名查询到具体的IP。DNS协议提供的是一种主机名到IP地址的转换服务，就是我们常说的域名系统。是应用层协议，通常该协议运行在UDP协议之上，使用的是53端口号。

因为IP存在数字和英文的组合（IPv6），很不利于人类记忆，所以就出现了域名。你可以把域名看成是某个IP的别名，DNS就是去查询这个别名的真正名称是什么。

当你在浏览器中想访问www.google.com时，会通过进行以下操作：本地客户端向服务器发起请求查询IP地址查看浏览器有没有该域名的IP缓存查看操作系统有没有该域名的IP缓存查看Host文件有没有该域名的解析配置如果这时候还没得话，会通过直接去DNS根服务器查询，这一步查询会找出负责com这个一级域名的服务器然后去该服务器查询google.com这个二级域名接下来查询www.google.com这个三级域名的地址返回给DNS客户端并缓存起来我们通过一张图来看看它的查询过程吧这张图很生动的展示了DNS在本地DNS服务器是如何查询的，一般向本地DNS服务器发送请求是递归查询的本地DNS服务器向其他域名服务器请求的过程是迭代查询的过程递归查询和迭代查询递归查询指的是查询请求发出后，域名服务器代为向下一级域名服务器发出请求，最后向用户返回查询的最终结果。使用递归查询，用户只需要发出一次查询请求。迭代查询指的是查询请求后，域名服务器返回单次查询的结果。下一级的查询由用户自己请求。使用迭代查询，用户需要发出多次的查询请求。

所以一般而言，本地服务器查询是递归查询，而本地DNS服务器向其他域名服务器请求的过程是迭代查询的过程DNS缓存缓存也很好理解，在一个请求中，当某个DNS服务器收到一个DNS回答后，它能够回答中的信息缓存在本地存储器中。返回的资源记录中的TTL代表了该条记录的缓存的时间。DNS实现负载均衡它是如何实现负载均衡的呢？首先我们得清楚DNS是可以用于在冗余的服务器上实现负载均衡。

原因：这是因为一般的大型网站使用多台服务器提供服务，因此一个域名可能会对应多个服务器地址。举个例子来说，当用户发起网站域名的DNS请求的时候，DNS服务器返回这个域名所对应的服务器IP地址的集合在每个回答中，会循环这些IP地址的顺序，用户一般会选择排在前面的地址发送请求。以此将用户的请求均衡的分配到各个不同的服务器上，这样来实现负载均衡。DNS为什么使用UDP协议作为传输层协议？DNS使用UDP协议作为传输层协议的主要原因是为了避免使用TCP协议时造成的连接时延。为了得到一个域名的IP地址，往往会向多个域名服务器查询，如果使用TCP协议，那么每次请求都会存在连接时延，这样使DNS服务变得很慢。大多数的地址查询请求，都是浏览器请求页面时发出的，这样会造成网页的等待时间过长。总结DNS域名系统，是应用层协议，运行UDP协议之上，使用端口43。查询过程，本地查询是递归查询，依次通过浏览器缓存—>>本地hosts文件—>>本地DNS解析器—>>本地DNS服务器—>>其他域名服务器请求。

接下来的过程就是迭代过程。递归查询一般而言，发送一次请求就够，迭代过程需要用户发送多次请求。

1) 用户能用IP打开登录页，却不能通过域名访问，后端同事据此认定权威DNS配错了，你同意吗？

答：不能直接认定，因为域名查询可能命中浏览器、操作系统或本地hosts中的旧结果，还未访问权威服务器。应逐层检查缓存和hosts，再核对本地DNS返回的地址；直接访问IP只能说明网络与服务可能可达。

2) 新域名上线时，运维只给了权威解析记录，浏览器侧仍提示域名找不到，你会沿着哪条链路验证？

答：先检查浏览器与操作系统缓存以及本地hosts，再查询本地DNS得到的记录。若仍无结果，就继续确认解析链是否能从根域定位顶级域服务器，再找到该域名的权威服务器；任一层配置或缓存异常都可能中断查询。

3) 一个活动域名同时指向多台服务器，产品要求仅靠DNS分摊流量，并在某台机器故障后立即摘除，你会接受吗？

答：DNS可以返回多个服务器地址，并通过调整回答中的地址顺序实现较粗粒度的负载分配。但解析结果会按资源记录的TTL缓存，故障地址被删除后仍可能继续返回或被客户端使用，因此不能承诺立即摘除。

4) 发布后部分地区访问旧站、公司网络却正常，应用日志也没有异常，你怎么判断是否为解析缓存问题？

答：分别在异常网络和正常网络查询域名，比较返回的IP与预期记录，同时检查本地hosts和各级缓存。若异常侧持续得到旧地址，应结合记录的TTL判断缓存何时失效；只清浏览器缓存未必能清除操作系统或本地DNS的结果。

5) 架构师要在反向代理和DNS多地址之间选择负载均衡，前端域名切换又要求故障影响尽量小，你会提醒什么？

答：DNS多地址无需让请求先经过统一代理，但调度较粗，并会受到各级缓存和TTL的约束。反向代理能在入口处选择真实服务器，控制更集中；代价是所有请求经过该入口，其容量与可用性也必须被保障。

510. 短轮询、长轮询和 WebSocket 间的区别

短轮询是客户端定时询问，长轮询是请求挂起等待结果，WebSocket则建立可双向通信的长连接。短轮询实现最简单，但即使数据没变化也会持续建立HTTP连接，客户端和服务端都有额外开销。长轮询只在数据更新或超时后返回，减少了无效请求，不过挂起连接仍会占用资源。需要服务端主动推送且双方频繁通信时，可以使用WebSocket，代价是服务端配置更复杂。

1. 短轮询 短轮询的基本思路：

浏览器每隔一段时间向浏览器发送 http 请求，服务器端在收到请求后，不论是否有数据更新，都直接进行响应。这种方式实现的即时通信，本质上还是浏览器发送请求，服务器接受请求的一个过程，通过让客户端不断的进行请求，使得客户端能够模拟实时地收到服务器端的数据的变化。

优缺点

优点是简单，易于理解。缺点是这种方式由于需要不断的建立 http 连接，严重浪费了服务器端和客户端的资源。当用户增加时，服务器端的压力就会变大，这是很不合理的。

2. 长轮询 长轮询的基本思路：首先由客户端向服务器发起请求，当服务器收到客户端发来的请求后，服务器端不会直接进行响应，而是先将这个请求挂起，然后判断服务器端数据是否有更新。如果有更新，则进行响应，如果一直没有数据，则到达一定的时间限制才返回。客户端

JavaScript

响应处理函数会在处理完服务器返回的信息后，再次发出请求，重新建立连接。

优缺点

长轮询和短轮询比起来，它的优点是明显减少了很多不必要的

http

请求次数，相比之下节约了资源。长轮询的缺点在于，连接挂起也会导致资源的浪费

3. WebSocket

WebSocket

是

Html5

定义的一个新协议，与传统的

http

协议不同，该协议允许由服务器主动的向客户端推送信息。使用

WebSocket

协议的缺点是在服务器端的配置比较复杂。WebSocket

是一个全双工的协议，也就是通信双方是平等的，可以相互发送消息。

1) 订单列表每分钟刷新一次，候选人说短轮询不算实时通信、必须改成WebSocket，你会怎么判断？

答：短轮询仍能模拟实时更新，只是实时程度受轮询间隔限制，并非不能用于状态刷新。若业务允许分钟级延迟，它实现简单且可能足够；强行使用WebSocket会增加服务端配置复杂度，而收益未必匹配。

2) 客服工作台要等待工单状态变化，绝大多数请求期间都没有更新，你会怎样把短轮询改成长轮询？

答：客户端发起请求后，服务端先挂起，检测到状态更新时再响应，没有更新则到超时后返回。客户端处理完响应后立即建立下一次请求，这能减少短轮询产生的大量无效请求；挂起连接本身仍会占用服务端资源。

3) 在线协作页面要求浏览器和服务器都能随时发送操作，负责人仍想用长轮询统一所有通信，你会指出什么限制？

答：长轮询主要依赖客户端持续发起并重新建立HTTP请求，服务端只能借助已挂起的请求返回数据。双向交互频繁时，这种模型管理成本较高；WebSocket提供全双工通信，更符合双方都需要主动发送消息的约束。

4) 长轮询接口上线后连接数持续升高、很多请求长期处于等待状态，但业务更新很少，你会先排查哪里？

答：先检查服务端是否为挂起请求设置了明确的超时，以及客户端在响应后是否只创建一个后继请求。长轮询减少了无效请求次数，却不会消除连接占用；超时过长或重复发起请求都会放大资源压力。

5) 十万级用户的状态页在短轮询、长轮询和WebSocket之间选型，业务只说要“尽量实时”，你会要求先明确什么？

答：需要先明确更新频率、可接受延迟、通信是否双向，以及服务端能承受的请求和连接规模。短轮询简单但反复建立请求，长轮询减少无效请求却会挂起连接，WebSocket支持全双工但服务端配置更复杂；没有这些约束就无法可靠选型。

511. 说一说正向代理和反向代理

正向代理隐藏真实客户端，反向代理隐藏真实服务端，区别就在于代理替哪一侧转发请求。使用正向代理时，目标服务只看到代理服务器发出的请求，不知道后面的真实客户端是谁。使用反向代理时，客户端只访问统一入口，不需要知道请求最终落到哪台真实服务器。反向代理常用于集群负载均衡；也可以通过DNS返回多个IP分流，但缓存可能导致故障地址暂时仍被返回。

正向代理 我们常说的代理也就是指正向代理，正向代理的过程，它隐藏了真实的请求客户端，服务端不知道真实的客户端是谁，客户端请求的服务都被代理服务器代替来请求。

反向代理 这种代理模式下，它隐藏了真实的服务端，当我们向一个网站发起请求的时候，背后可能有成千上万台服务器为我们服务，具体是哪一台，我们不清楚，我们只需要知道反向代理服务器是谁就行，而且反向代理服务器会帮我们请求转发到真实的服务器那里去，一般而言反向代理服务器一般用来实现负载均衡。

负载均衡的两种实现方式？ 一种是使用反向代理的方式，用户的请求都发送到反向代理服务上，然后由反向代理服务器来转发请求到真实的服务器上，以此来实现集群的负载均衡。另一种是 DNS 的方式，DNS 可以用于在冗余的服务器上实现负载均衡。因为现在一般的大型网站使用多台服务器提供服务，因此一个域名可能会对多个服务器地址。当用户向网站域名请求的时候，DNS 服务器返回这个域名所对应的服务器 IP 地址的集合，但在每个回答中，会循环这些 IP 地址的顺序，用户一般会选排在前面的地址发送请求。以此将用户的请求均衡的分配到各个不同的服务器上，这样来实现负载均衡。这种方式有一个缺点就是，由于 DNS 服务器中存在缓存，所以有可能一个服务器出现故障后，域名解析仍然返回的是那个 IP 地址，就会造成访问的问题。

1) 浏览器通过公司代理访问外部文档站，服务端日志只看到代理地址；同事却称这是反向代理，你怎么纠正？

答：这里是正向代理，因为代理代表客户端向外部服务发起请求，外部服务看不到真实客户端。反向代理隐藏的是后方真实服务器，用户只知道统一入口；判断关键在代理代表请求方还是服务方。

2) 本地开发把/api交给devServer.proxy转发到后端，浏览器不再报跨域，你会怎样解释这条链路？

答：对浏览器而言，请求仍发往同源的开发服务器，再由该服务器转发到真实接口，因此属于反向代理场景。浏览器不知道后端实例是谁，服务器间转发也不受浏览器同源策略约束；线上若不保留同域入口，跨域约束仍会出现。

3) 一个域名后有多台应用服务器，平台团队要求用户不能感知扩缩容，你会把负载均衡放在哪里？

答：可让用户统一访问反向代理，再由代理把请求分配到真实服务器，扩缩容只需调整代理后的节点集合。这样隐藏了服务端拓扑并集中调度；反向代理自身也成为关键入口，需要具备足够容量和可用性。

4) 机房已删除故障服务器的DNS地址，部分用户却仍持续命中它，应用负责人认为删除记录应立即生效，你怎么排查？

答：先查询受影响用户实际拿到的地址，并核对各级DNS缓存及记录的TTL。缓存未过期时，客户端仍可能使用已删除的故障地址；若要求更快摘除，应评估由反向代理集中选择健康后端，而不能只依赖解析变更。

5) 流量入口评审中，一方主张DNS轮换多个地址，另一方主张所有请求经过反向代理，你如何说明取舍？

答：DNS可返回多个服务器地址并轮换顺序，部署分散，但缓存会降低故障切换的及时性。反向代理能集中转发和调整后端，更容易隐藏节点变化；相应地，它承担全部入口流量，必须额外保障自身稳定性。

512. 介绍一下Connection:keep-alive

Connection: keep-alive表示多个HTTP请求可以复用同一条TCP连接，而不是每次请求结束后立即断开。这样能减少反复建立和关闭连接带来的响应时间、CPU资源及拥堵开销，适合同一客户端连续请求多个资源的场景。在HTTP/1.0中需要显式发送Connection:

keep-alive才会启用。在HTTP/1.1中默认使用持久连接，如需关闭则发送Connection:

close，最终能否保持还取决于服务器配置。什么是keep-alive 我们知道HTTP协议采用“请求-应答”模式，当使用普通模式，即非KeepAlive模式时，每个请求/应答客户端和服务端都要新建一个连接，完成之后立即断开连接（HTTP协议为无连接的协议）；

当使用Keep-Alive模式（又称持久连接、连接重用）时，Keep-Alive功能使客户端到服务器端的连接持续有效，当出现对服务器的后继请求时，Keep-Alive功能避免了建立或者重新建立连接。

为什么要使用keep-alive

keep-alive技术的创建目的，能在多次HTTP之前重用同一个TCP连接，从而减少创建/关闭多个TCP连接的开销（包括响应时间、CPU资源、减少拥堵等），参考如下示意图 客户端如何开启在HTTP/1.0协议中，默认是关闭的，需要在http头加入"Connection:

Keep-Alive”，才能启用Keep-Alive； Connection: keep-alive http 1.1中默认启用Keep-Alive，如果加入"Connection: close “，才关闭。 Connection: close 目前大部分浏览器都是用http1.1协议，也就是说默认都会发起Keep-Alive的连接请求了，所以是否能完成一个完整的Keep-Alive连接就看服务器设置情况。

1) 商品页连续请求十个资源，候选人说keep-alive会把十个响应合成一次返回，你会怎么纠正？

答：keep-alive重用的是同一条TCP连接，并不会把多个HTTP请求或响应合并。每次交互仍遵循请求-应答模式，只是避免反复建立和关闭连接，从而减少握手、响应时间与资源开销。

2)

一个仍使用HTTP/1.0的内部接口每次请求后都断开，客户端想复用连接，应检查什么配置？

答：HTTP/1.0默认不启用持久连接，应检查请求是否携带Connection: keep-alive，以及服务器是否支持并接受连接复用。只修改客户端头部并不能保证成功，完整的持久连接还取决于服务端设置。

3) 网关从HTTP/1.0升到HTTP/1.1后，团队仍在所有请求里显式添加Connection: keep-alive，这有必要吗？

答：在HTTP/1.1中持久连接默认启用，通常无需再显式添加Connection: keep-alive。更应检查是否有人发送了Connection: close，因为它会要求请求完成后关闭连接；最终能否复用仍受服务器配置影响。

4) 接口耗时没有变化，但瀑布图里每个请求都重新建立连接，页面整体变慢，你会从哪里定位？

答：先确认请求或响应中是否出现Connection: close，再检查服务器是否允许并维持持久连接。若连接无法复用，每个请求都会承担新建与关闭TCP连接的开销；仅在前端声明keep-alive不能覆盖服务端主动断开的配置。

5)

运维认为连接复用能降低开销，因此准备让所有空闲连接永久保留，你会接受这个结论吗？

答：连接复用确实能减少重复建立和关闭TCP连接带来的时间、计算与拥塞成本，但不等于连接应永久存在。具体维持策略必须由客户端和服务端共同支持，并结合服务端资源约束设置；否则持久连接本身也可能成为负担。

513. http/https 协议总结

HTTP负责应用层通信，HTTPS则通过证书和SSL加密提升传输安全性，默认使用443端口。HTTP/1.1用长连接复用连接，并通过Host、身份认证和缓存等能力弥补HTTP/1.0的不足。HTTP/2进一步引入多路复用、二进制分帧、首部压缩和服务端推送。缓存上，未过期时走Cache-Control或Expires强缓存，过期后再用Etag或Last-Modified协商，且前者优先级更高。1.0 协议缺陷: 无法复用链接，完成即断开，重新慢启动和TCP 3次握手head of line blocking: 线头阻塞，导致请求之间互相影响 1.1 改进: 长连接(默认keep-alive)，复用host字段指定对应的虚拟站点新增功能:断点续传身份认证状态管理cache缓存Cache-ControlExpiresLast-ModifiedEtag 断点续传身份认证状态管理cache缓存Cache-ControlExpiresLast-ModifiedEtag 2.0: 多路复用二进制分帧层: 应用层和传输层之间首部压缩服务端推送 https: 较为安全的网络传输协议 证书(公钥)SSL加密端口443 TCP: 三次握手四次挥手滑动窗口: 流量控制拥塞处理慢开始拥塞避免快速重传快速恢复

慢开始拥塞避免快速重传快速恢复 缓存策略: 可分为 强缓存 和 协商缓存 Cache-Control/Expires: 浏览器判断缓存是否过期，未过期时，直接使用强缓存，Cache-Control的max-age优先级高于Expires当缓存已经过期时，使用协商缓存唯一标识方案:Etag(response携带)

&If-None-Match(request携带，上一次返回的Etag):

服务器判断资源是否被修改最后一次修改时间:Last-Modified(response) &If-Modified-Since(request，上一次返回的Last-Modified)如果一致，则直接返回 304 通知浏览器使用缓存如不一致，则服务端返回新的资源Last-Modified缺点: 周期性修改，但内容未变时，会导致缓存失效最小粒度只到s，s以内的改动无法检测到Etag的优先级高于Last-Modified

唯一标识方案:Etag(response携带) &If-None-Match(request携带，上一次返回的Etag):

服务器判断资源是否被修改最后一次修改时间:Last-Modified(response) &If-Modified-Since(request，上一次返回的Last-Modified)如果一致，则直接返回 304 通知浏览器使用缓存如不一致，则服务端返回新的资源 如果一致，则直接返回 304 通知浏览器使用缓存如不一致，则服务端返回新的资源

周期性修改，但内容未变时，会导致缓存失效最小粒度只到s，s以内的改动无法检测到

1) 首页在HTTP/1.1下开了长连接，候选人据此断言多个资源已经不会互相阻塞，你会怎么追问？

答：长连接解决的是重复建连问题，不等于消除同一连接上的请求阻塞。HTTP/2通过二进制分帧和多路复用改善多个请求相互影响，并配合首部压缩；仅看到keep-alive不能推断已获得这些能力。

2) 静态站升级协议时，网关团队问HTTP/1.1相比HTTP/1.0能带来哪些直接收益，你会抓哪些配置验收？

答：应确认默认长连接是否生效，并检查Host是否正确路由到虚拟站点，同时验收断点续传、认证、状态管理和缓存相关行为。升级协议并不保证服务端配置自动正确，连接被关闭或缓存头缺失仍会抵消收益。

3) 资源服务器已配置Expires，应用又返回了Cache-Control: max-age，两者给出的过期时间冲突时浏览器应依据哪个？

答：浏览器应优先采用Cache-Control的max-age判断强缓存是否过期，Expires的优先级更低。未过期可直接使用本地缓存；过期后还需进入协商缓存流程，不能把配置了过期时间等同于永远不请求服务器。

4) 发版后一张图片内容变了，但请求携带If-Modified-Since后仍收到304，修改发生在同一秒内，你会怎样解释并修正？

答：Last-Modified的时间粒度只能到秒，同一秒内的修改可能无法被准确识别，因此服务器可能判断资源未变化。可同时采用ETag与If-None-Match校验内容标识，而且ETag的判断优先级更高；代价是服务端需要生成和比较标识。

5) 构建任务每次都会改写文件时间，但产物内容没有变化，线上却频繁重新下载，你会选择哪种协商缓存依据？

答：仅依赖Last-Modified会把时间变化视为资源变化，即使实际内容相同也可能让缓存失效。更适合使用ETag与If-None-Match判断资源标识，未修改时返回304；服务端必须保证标识生成规则稳定且能反映内容变化。

6) 安全评审要求站点从HTTP切到HTTPS，产品却认为只要端口改成443就完成了，你会指出缺少什么？

答：HTTPS不只是更换端口，还需要证书提供公钥信息，并通过SSL建立加密传输。证书与加密配置缺失时，无法获得预期的安全通信；同时它解决的是传输安全，不能替代应用自身的认证和状态管理。

514. TCP为什么要三次握手

TCP需要三次握手，是为了确认双方都具备正常的发送和接收能力，并避免失效请求误建连接。前两次通信只能让客户端确认服务端可收可发，服务端还不知道客户端能否收到自己的确认，所以需要第三次确认。这样即使旧的连接请求延迟到达，服务端收不到最终确认，也不会一直为无效连接等待数据。第三次握手时客户端已进入ESTABLISHED状态，因此可以携带数据；断开时双方关闭发送方向的时机可能不同，所以通常需要四次挥手。客户端和服务端都需要直到各自可收发，因此需要三次握手。第一次握手成功让服务端知道了客户端具有发送能力，第二次握手成功让客户端知道了服务端具有接收和发送能力，但此时服务端并不知道客户端是否接收到了自己发送的消息。为了防止出现失效的连接请求报文段被服务端接收的情况，从而产生错误。所以第三次握手就起到了这个作用。

你可以会问，2次握手就足够了？。但其实不是，因为服务端还没有确定客户端是否准备好了。比如步骤 3 之后，服务端马上给客户端发送数据，这个时候客户端可能还没有准备好接收数据。因此还需要增加一个过程。

TCP有6种标示:SYN(建立联机) ACK(确认) PSH(传送) FIN(结束) RST(重置) URG(紧急)

举例：已失效的连接请求报文段 client发送了第一个连接的请求报文，但是由于网络不好，这个请求没有立即到达服务端，而是在某个网络节点中滞留了，直到某个时间才到达server本来这已经是一个失效的报文，但是server端接收到这个请求报文后，还是向client发出确认的报文，表示同意连接。假如不采用三次握手，那么只要server发出确认，新的建立就连接了，但其实这个请求是失效的请求，client是不会理睬server的确认信息，也不会向服务端发送确认的请求。但是server认为新的连接已经建立起来了，并一直等待client发来数据，这样，server的很多资源就没白白浪费掉了。采用三次握手就是为了防止这种情况的发生，server会因为收不到确认的报文，就知道client并没有建立连接。这就是三次握手的作用。

三次握手过程中可以携带数据吗？第一次、第二次握手不可以携带数据，因为一握二握时还没有建立连接，会让服务器容易受到攻击。而第三次握手，此时客户端已经处于ESTABLISHED (已建立连接状态)，对于客户端来说，已经建立起连接了，并且也已经知道服务器的接收、发送能力是正常的了，所以能携带数据也是没问题的。

为什么建立连接只通信了三次，而断开连接却用了四次？
客户端要求断开连接，发送一个断开的请求，这个叫作（FIN）。服务端收到请求，然后给客户端一个ACK，作为 FIN 的响应。这里你需要思考一个问题，可不可以像握手那样马上传 FIN 回去？其实这个时候服务端不能马上传

FIN，因为断开连接要处理的问题比较多，比如说服务端可能还有发送出去的消息没有得到 ACK；也有可能服务端自己有资源要释放。因此断开连接不能像握手那样操作——将两条消息合并。所以，服务端经过一个等待，确定可以关闭连接了，再发一条 FIN 给客户端。客户端收到服务端的FIN，同时客户端也可能有自己的事情需要处理完，比如客户端有发送给服务端没有收到的 ACK 的请求，客户端自己处理完成后，再给服务端发送一个 ACK。

为了确保数据能够完成传输。因为当服务端收到客户端的 FIN 报文后，发送的 ACK 报文只是用来应答的，并不表示服务端也希望立即关闭连接。

当只有服务端把所有的报文都发送完了，才会发送 FIN 报文，告诉客户端可以断开连接了，因此在断开连接时需要四次挥手。关闭连接时，当收到对方的FIN报文通知时，它仅仅表示对方没有数据发送给你了；但未必你所有的数据都全部发送给对方了所以你未必会马上关闭SOCKET,也即你可能还需要发送一些数据给对方之后，再发送FIN报文给对方来表示你同意现在可以关闭连接了，所以它这里的ACK报文和FIN报文多数情况下都是分开发送的。

1) 客户端发出SYN后，只要服务端返回SYN+ACK就算双方都具备收发能力，这个判断哪里不成立？

答：此时客户端已确认服务端能接收并发送，但服务端只能确认客户端能发送，尚不知道客户端能否收到自己的报文。客户端再发一次ACK，服务端才能确认双向收发链路均正常，因此两次握手不足以完成双方确认。

2) 一条旧的连接请求在网络节点滞留，原连接早已结束后才到达服务端；如果协议只握手两次，服务端会出现什么工程后果？

答：服务端回复确认后就可能误认为新连接已经建立，并为它保留连接状态、等待客户端发送数据。客户端知道该请求已经失效，不会响应服务端；第三次握手缺失时，服务端无法及时识别这种半开连接，资源会被无效占用。

3) 同事想把第三次握手改成“服务端返回确认后立即下载业务数据”，以减少首页等待，这个方案为什么有风险？

答：服务端收到首个请求时还没有确认客户端能接收，也不知道客户端是否仍认可这次连接，立即发送业务数据缺少可靠前提。第三次ACK到达后，服务端才确认客户端准备完成；握手规则不能由业务层为节省一次确认而随意省略。

4) 线上连接大量停留在服务端等待确认的阶段，却一直没有进入已建立状态，你会沿着哪条握手链路排查？

答：先确认客户端的首个连接请求是否到达，再检查服务端确认报文能否返回客户端，以及客户端最终ACK是否被网络设备丢弃。若服务端持续收不到第三次确认，它不会把连接视为完整建立；还应排查防火墙、链路丢包和失效请求，而不是先归因于业务数据处理。

5) 为什么第三次握手可以携带数据，而前两次通常不应携带业务数据？

答：发出第三次报文时，客户端已经确认服务端具备收发能力，并进入连接已建立状态，因此该报文具备承载数据的条件。前两次发生在连接尚未完成确认时，服务端若提前接收和处理数据，会扩大无效连接或恶意请求造成的资源风险。

6) 评审中有人认为建立连接三次、关闭连接也应该固定三次，你会如何解释关闭通常需要四次？

答：收到对方的FIN只表示对方不再发送数据，不代表本端的数据已经发送完毕，因此本端通常先单独回复ACK。待剩余报文和资源处理完成后再发送自己的FIN，对方最后确认；只有关闭确认与本端FIN恰好可合并时，报文数量才可能变化。

515. 为什么要有 WebSocket

需要WebSocket，本质上是因为HTTP的请求-应答模式不适合服务端主动推送和实时双向通信。轮询虽然能模拟实时效果，但会反复产生无效请求，浪费带宽和计算资源。WebSocket建立后支持全双工通信，并使用二进制帧传输，适合即时消息、动态页面和网络游戏。它先通过带有Connection:

Upgrade和Upgrade: websocket的HTTP GET请求握手，服务端返回101 Switching

Protocols后切换协议，但连接、缓存和状态需要应用自己管理。已经有了被广泛应用的 HTTP 协议，为什么要再出一个 WebSocket 呢？它有哪些好处呢？其实 WebSocket 与 HTTP/2 一样，都是为了解决 HTTP 某方面的缺陷而诞生的。HTTP/2 针对的是“队头阻塞”，而WebSocket 针对的是“请求 - 应答”通信模式。那么，“请求 - 应答”有什么不好的地方呢？“请求 - 应答”是一种“半双工”的通信模式，虽然可以双向收发数据，但同一时刻只能一个方向上有动作，传输效率低。更关键的一点，它是一种“被动”通信模式，服务器只能“被动”响应客户端的请求，无法主动向客户端发送数据。虽然后来的 HTTP/2、HTTP/3 新增了 Stream、Server Push 等特性，但“请求 - 应答”依然是主要的工作方式。这就导致 HTTP 难以应用在动态页面、即时消息、网络游戏等要求“实时通信”的领域。在 WebSocket 出现之前，在浏览器环境里用 JavaScript 开发实时 Web 应用很麻烦。因为浏览器是一个“受限的沙盒”，不能用 TCP，只有 HTTP 协议可用，所以就出现了很多“变通”的技术，“轮询”（polling）就是比较常用的一种。简单地说，轮询就是不停地向服务器发送 HTTP 请求，问有没有数据，有数据的话服务器就用响应报文回应。如果轮询的频率比较高，那么就可以近似地实现“实时通信”的效果。但轮询的缺点也很明显，反复发送无效查询请求耗费了大量的带宽和 CPU 资源，非常不经济。所以，为了克服 HTTP “请求 - 应答”模式的缺点，WebSocket 就“应运而生”了。WebSocket 的特点 WebSocket 是一个真正“全双工”的通信协议，与 TCP 一样，客户端和服务端都可以随时向对方发送数据。WebSocket采用了二进制帧结构，语法、语义与 HTTP 完全不兼容，但因为它的主要运行环境是浏览器，为了便于推广和应用，就不得不“搭便车”，在使用习惯上尽量向 HTTP 靠拢，这就是它名字里“Web”的含义。服务发现方面，WebSocket 没有使用 TCP 的“IP 地址 + 端口号”，而是延用了 HTTP 的 URI 格式，但开头的协议名不是“http”，引入的是两个新的名字：“ws”和“wss”，分别表示明文和加密的 WebSocket 协议。WebSocket 的默认端口也选择了 80 和 443，因为现在互联网上的防火墙屏蔽了绝大多数的端口，只对 HTTP 的 80、443 端口“放行”，所以 WebSocket 就可以“伪装”成 HTTP 协议，比较容易地“穿透”防火墙，与服务器建立连接。ws://www.chrono.com wss://www.chrono.com:8080/srv wss://www.chrono.com:445/im?user_id=xxx WebSocket 的握手和 TCP、TLS 一样，WebSocket 也要有一个握手过程，然后才能正式收发数据。这里它还是搭上了 HTTP 的“便车”，利用了 HTTP 本身的“协议升级”特性，“伪装”成 HTTP，这样就能绕过浏览器沙盒、网络防火墙等等限制，这也是 WebSocket 与 HTTP 的另一个重要关联点。WebSocket 的握手是一个标准的 HTTP GET 请求，但要带上两个协议升级的专用头字段：“Connection: Upgrade”，表示要求协议“升级”；“Upgrade: websocket”，表示要“升级”成 WebSocket 协议。另外，为了防止普通的 HTTP 消息被“意外”识别成 WebSocket，握手消息还增加了两个额外的认证用头字段（所谓的“挑战”，Challenge）：Sec-WebSocket-Key：一个 Base64 编码的 16 字节随机数，作为简单的认证密钥；Sec-WebSocket-Version：协议的版本号，当前必须是 13。服务器收到 HTTP 请求报文，看到上面的四个字段，就知道这不是一个普通的 GET 请求，而是 WebSocket 的升级请求，于是不走普通的 HTTP 处理流程，而是构造一个特殊的“101 Switching Protocols”响应。

1) 聊天页面每秒轮询一次消息接口，产品说“HTTP也能双向传数据，没必要上WebSocket”，你会指出哪个关键差异？

答：HTTP的主要工作方式仍是客户端请求、服务端应答，服务端不能在无请求时主动推送新消息。高频轮询只能近似实时，还会产生大量无新数据的请求，消耗带宽和服务端处理资源；持续双向通信才是WebSocket更匹配的场景。

2) 行情页需要服务端持续推送报价，同时客户端随时发送订阅变更；为什么全双工比“请求后等待响应”更合适？

答：WebSocket建立连接后，客户端和服务端都能随时发送数据，不必等待另一方向完成一次请求一应答。报价推送与订阅变更可以在同一连接上独立发生；代价是应用必须自行管理连接、缓存和状态，开发复杂度高于普通HTTP。

3) 公司出口防火墙只放行常见Web端口，架构师仍建议自定义一个全新端口承载实时协议；WebSocket的选择解决了什么约束？

答：WebSocket沿用兼容HTTP的URI形式，并通常使用80或443端口，更容易通过现有防火墙和浏览器环境。它先借助HTTP完成协议升级，再切换到自己的二进制帧通信；这并不意味着升级后的数据仍是普通HTTP报文。

4) 线上WebSocket请求一直返回普通页面的200，客户端没有进入连接成功状态，你会先核对哪些握手字段和响应？

答：先检查客户端是否发送HTTP GET，并包含Connection: Upgrade、Upgrade: websocket、Sec-WebSocket-Key和版本字段。服务端应识别升级请求并返回101 Switching Protocols；若返回200，通常说明请求被当成普通HTTP，还需检查代理是否转发了升级头。

5) 通知中心一天只有少量事件，团队却准备为它维护长连接、重连、缓存和状态机；你会如何做选型取舍？

答：事件稀疏且实时性要求不高时，普通请求或适度轮询可能更简单，不必仅因服务端需要通知就引入WebSocket。WebSocket更适合即时消息、游戏等持续实时通信，但连接生命周期和状态需要应用自行管理；收益不足时，复杂度会成为主要成本。

6) 安全评审看到Sec-WebSocket-Key，便认为它能替代用户登录鉴权，这个结论为什么不可靠？

答：Sec-WebSocket-Key是握手中的随机挑战值，用于避免普通HTTP消息被意外识别为WebSocket，并不是业务用户身份凭证。用户权限仍需由应用层单独校验；把协议握手校验当成登录认证，会让未授权连接进入后续通信流程。

516. UDP和TCP有什么区别

TCP面向连接并提供可靠传输，UDP无连接且不保证数据可靠到达。TCP会给数据段编号，并通过确认、窗口和流量控制来保障传输，因此协议负载相对更高。UDP不做这些连接与确认工作，机制更简单，但数据是否收到需要上层自行处理。简单来说，需要可靠性的场景更适合TCP；能够接受丢失、希望减少协议负担时，可以考虑UDP。TCP协议在传送数据段的时候要给段标号；UDP协议不TCP协议可靠；UDP协议不可靠TCP协议是面向连接；UDP协议采用无连接TCP协议负载较高，采用虚电路；UDP采用无连接TCP协议的发送方要确认接收方是否收到数据段（3次握手协议）TCP协议采用窗口技术和流控制

1) 文件上传页面要求数据完整到达，同事却说“UDP更轻，丢了让用户重传整个文件就行”，这个选型忽略了什么？

答：文件上传需要有序、可靠地交付数据，TCP会给数据段编号，并通过确认、窗口和流量控制维护传输。UDP无连接且不提供这些可靠性保证，应用若自行补齐确认和重传会增加复杂度；只追求较低协议负担不能覆盖完整性要求。

2) 浏览器调用上传接口时，抓包显示连接建立前有往返确认；这些报文如何体现TCP与UDP的工程差异？

答：TCP面向连接，发送数据前要通过握手确认通信双方的能力，后续数据段还需要确认。UDP采用无连接方式，不先建立这种虚连接；因此它的协议负担较低，但也不会自动保证接收方收到每个数据段

。

3) 实时数据面板允许丢失个别过期采样，却要求新数据尽快送达；协议约束变化后，为什么仍不能只凭“UDP更快”拍板？

答：允许少量丢失会降低对传输可靠性的要求，使无连接的UDP成为候选，但source只能支持其负担较低、缺少可靠确认。仍需由业务明确乱序、丢失和拥塞时的处理方式；若最终要求完整交付，就必须在应用层补偿或改用TCP。

4) 线上日志显示发送端已经调用成功，但接收端缺少若干报文；使用UDP时为什么不能直接判定是服务端漏处理？

答：UDP本身不要求接收方对数据段逐一确认，也不保证报文可靠到达，因此发送调用成功不等于对端已经收到。排查时要同时检查链路丢包、接收缓冲和应用处理；若业务不能容忍缺失，应增加应用层确认机制或选择可靠传输。

5) 批量接口出现接收方处理不过来的现象，评审时如何用流量控制能力比较TCP和UDP？

答：TCP采用窗口技术和流量控制，可根据接收能力约束发送过程，并配合确认维护可靠传输。UDP不提供同等级的内建机制，发送节奏需要应用自行管理；高吞吐并不等于可以忽略接收方容量，否则丢失和过载风险会转移到业务层。

6) 同事把“TCP三次握手”和“每个数据段的确认”混成一件事，你会怎样纠正这两个相邻概念？

答：三次握手用于建立面向连接的通信状态，确认双方具备相应收发能力；数据传输阶段的确认则用于判断数据段是否到达。二者都服务于可靠性，但发生阶段和目的不同；UDP既不建立这种连接，也不内建逐段可靠确认。

【9种前端常见的设计模式】

517. 外观模式

外观模式就是给一组复杂的子系统接口提供一个统一、简单的高层入口，让调用方更容易使用。比如封装浏览器事件绑定时，内部处理addEventListener、attachEvent等兼容逻辑，外部只调用一个方法。它适合用来隔离系统分层、简化持续演化的子系统，或为遗留代码提供清晰入口。代价是外观接口变化时往往需要直接修改，不太符合开闭原则。外观模式是最常见的设计模式之一，它为子系统中的一组接口提供一个统一的高层接口，使子系统更容易使用。简而言之外观设计模式就是把多个子系统中复杂逻辑进行抽象，从而提供一个更统一、更简洁、更易用的API。很多我们常用的框架和库基本都遵循了外观设计模式，比如jQuery就把复杂的原生DOM操作进行了抽象和封装，并消除了浏览器之间的兼容问题，从而提供了一个更高级更易用的版本。其实在平时工作中我们也会经常用到外观模式进行开发，只是我们不自知而已

兼容浏览器事件绑定 let addMyEvent=function(el,ev,fn){if(el.addEventListener){el.addEventListener(ev,fn,false)}elseif(el.attachEvent){el.attachEvent('on'+ev,fn)}else{el['on'+ev]=fn}}; 封装接口 let myEvent = { // ... stop: e => { e.stopPropagation(); e.preventDefault(); } }; 场景 设计初期，应该要有意识地将不同的两个层分离，比如经典的三层结构，在数据访问层和业务逻辑层、业务逻辑层和表示层之间建立外观Facade在开发阶段，子系统往往因为不断的重构演化而变得越来越复杂，增加外观Facade可以提供一个简单的接口，减少他们之间的依赖。在维护一个遗留的大型系统时，可能这个系统已经很难维护了，这时候使用外观Facade也是非常合适的，为系统开发一个外观Facade类，为设计粗糙和高度复杂的遗留代码提供比较清晰的接口，让新系统和Facade对象交互，Facade与遗留代码交互所有的复杂工

作。 优点 减少系统相互依赖。提高灵活性。提高了安全性 缺点
不符合开闭原则，如果要改东西很麻烦，继承重写都不合适。

1) 页面把addEventListener、attachEvent和onclick的兼容判断复制到二十个组件里，这只是工具函数缺失，还是适合引入外观模式？

答：这适合用外观模式收敛：对外只暴露统一的事件绑定接口，内部选择浏览器支持的实现。调用方不再依赖多个底层接口和兼容分支；但外观应保持职责集中，否则不断加入无关能力会变成新的复杂子系统。

2) 业务层同时直接调用缓存、请求和错误提示模块，重构其中一个模块时几十个页面都要修改；外观层应放在哪里？

答：可在业务层与这些子系统之间建立统一的高层接口，让页面只依赖稳定的业务操作，外观再协调底层模块。这样能减少层间依赖并提高替换灵活性；若页面仍绕过外观直接访问子系统，隔离效果就会被破坏。

3) 遗留后台有多个命名混乱的接口，新页面只需要“查询并展示用户”，你会让新代码直接理解全部旧模块吗？

答：应为遗留系统增加一个边界清晰的Facade，向新页面提供简洁接口，并在内部适配旧代码的复杂调用。这样新系统只与外观交互，维护范围更可控；外观只能隔离复杂度，不能自动消除遗留实现本身的缺陷。

4) 统一request外观升级后，某个页面的特殊配置无法传入，开发者开始绕过它直接调用底层库；这暴露了什么设计故障？

答：这说明外观提供的高层接口没有覆盖必要能力，已经从简化使用变成限制使用。应核对真实调用场景，补充明确扩展点或受控的配置透传；若把所有底层参数原样暴露，又会削弱外观减少依赖的价值。

5) 架构评审要求所有子系统能力都只能通过一个巨大Facade暴露，以获得最高“安全性”，你会接受吗？

答：不会仅为统一入口就堆成单一巨大外观，应按稳定职责和层次划分接口，限制不必要的底层暴露。外观确实能保护子系统并减少依赖，但修改集中接口可能影响大量调用方，而且其扩展并不天然符合开闭原则。

6) 有人把外观模式和代理模式都理解为“包一层”，在接口聚合页面中如何区分它们？

答：外观面向一组复杂子系统，目标是提供更统一、更易用的高层接口，调用形式通常会被重新组织。代理则通常代表某个目标对象并控制对它的访问；若代码主要在聚合多个模块，而非代替单一对象，就更接近外观模式。

518. 代理模式

代理模式是给目标对象增加一个代用品或占位对象，由它控制调用方如何访问目标对象。代理位于双方之间，可以延迟转交请求、保护目标对象，也能在不修改目标对象的情况下扩展处理逻辑，从而降低耦合。前端常见例子是事件代理：在父元素监听点击，再通过event.target识别实际触发的子元素。它不是直接访问，因此会增加一层处理，并可能带来额外开销。

是为一个对象提供一个代用品或占位符，以便控制对它的访问

假设当A

在心情好的时候收到花，小明表白成功的几率有60%，而当A

在心情差的时候收到花，小明表白的成功率无限趋近于0。小明跟A

刚刚认识两天，还无法辨别A

什么时候心情好。如果不合时宜地把花送给A，花被直接扔掉的可能性很大，这束花可是小明吃了7

天泡面换来的。但是A的朋友B却很了解A，所以小明只管把花交给B，B会监听A的心情变化，然后选择A

心情好的时候把花转交给A，代码如下：

```
let Flower = function() {}  
let xiaoming = { sendFlower: function(target) { let flower = new Flower() target.receiveFlower(flower) } }  
let B = { receiveFlower: function(flower) { A.listenGoodMood(function() { A.receiveFlower(flower) }) } }  
let A = { receiveFlower: function(flower) { console.log('收到花'+ flower) }, listenGoodMood: function(fn) { setTimeout(function() { fn() }, 1000) } }  
xiaoming.sendFlower(B)
```


场景 HTML元素事件代理

```
let ul = document.querySelector('#ul')  
ul.addEventListener('click', event => { console.log(event.target); })
```


优点 代理模式能将代理对象与被调用对象分离，降低了系统的耦合度。代理模式在客户端和目标对象之间起到一个中介作用，这样可以起到保护目标对象的作用代理对象可以扩展目标对象的功能；通过修改代理对象就可以了，符合开闭原则；
缺点 处理请求速度可能有差别，非直接访问存在开销

1) 图片组件只是把原对象的方法改了个名字，没有控制访问时机或保护目标对象，这还能算代理模式吗？

答：仅改名通常不能体现代理模式，因为代理应作为目标对象的代用品或占位符，介入并控制对目标的访问。若中间层既不延迟、不保护也不扩展访问行为，它更像普通封装；额外一层还会带来调用开销。

2) 商品列表有一万个节点，团队准备给每个条目绑定点击监听；用父容器事件代理时，代理对象和目标对象分别是什么？

答：父容器统一接收冒泡后的点击事件，相当于访问入口，再根据event.target判断实际被点击的子元素。这样无需在每个条目上分别注册处理器，降低监听关系的耦合；同时必须处理冒泡路径和非目标节点，避免误触发。

3) 送花逻辑改为由中间对象等目标状态合适时再转交，这层代理解决了什么约束变化？

答：中间对象掌握目标状态，因此能控制真正访问目标的时机，而发送方仍只调用相同的接收接口。发送方与目标状态判断被分离，后续可修改代理逻辑而不改目标；若中间状态判断失准，请求仍可能延迟或无法送达。

4) 线上点击列表项偶尔触发父级空白区域的业务动作，代码使用了事件代理，你会先检查什么？

答：先检查监听器是否只依据父容器收到click就执行，而没有核对event.target或其祖先是否为目标条目。事件代理把多个子元素的访问集中到一个入口，匹配逻辑写错会扩大误触发范围；还要确认冒泡是否被其他处理器阻止。

5) 权限模块计划为每个服务对象增加校验代码，另一方案是在访问入口设置保护代理；你会如何取舍？

答：权限属于访问控制时，保护代理能把校验与目标对象分离，让客户端先经过中介再访问真实服务。修改代理即可调整控制逻辑，也更符合对目标扩展开放、修改收敛的思路；代价是多一层转发，并可能带来处理时延和排查复杂度。

6) 评审中有人说“代理只能控制访问，绝不能扩展功能”，这与代理模式的边界一致吗？

答：不完全一致，代理的核心是代表目标并控制访问，但代理对象也可以在中介过程中扩展目标对象的功能。关键是客户端仍通过代理间接访问目标，二者保持分离；若新增能力彻底改变职责或接口，则应重新判断是否更接近其他结构型模式。

519. 工厂模式

工厂模式把对象创建过程封装到统一接口中，让调用方只关心要什么，而不用了解具体怎样实例化。当系统需要根据名称或运行环境创建行为不同的不同对象时，工厂可以隔离构造逻辑，减少调用方与具体产品的耦合。新增产品通常通过扩展产品类和工厂完成，因而更容易扩展。若对象创建本来很简单，硬套工厂

会增加抽象层、理解成本和单元测试难度，此时直接使用构造器更合适。工厂模式定义一个用于创建对象的接口，这个接口由子类决定实例化哪一个类。该模式使一个类的实例化延迟到了子类。而子类可以重写接口方法以便创建的时候指定自己的对象类型。

```
class Product { constructor(name) { this.name = name } init() { console.log('init') } fun() { console.log('fun') } } class Factory { create(name) { return new Product(name) } } // use let factory = new Factory() let p = factory.create('p1') p.init() p.fun()
```

场景 如果你不想让某个子系统与较大的那个对象之间形成强耦合，而是想运行时从许多子系统中进行挑选的话，那么工厂模式是一个理想的选择将new操作简单封装，遇到new的时候就应该考虑是否用工厂模式；需要依赖具体环境创建不同实例，这些实例都有相同的行为,这时候我们可以使用工厂模式，简化实现的过程，同时也可以减少每种对象所需的代码量，有利于消除对象间的耦合，提供更大的灵活性

优点 创建对象的过程可能很复杂，但我们只需要关心创建结果。构造函数和创建者分离，符合“开闭原则”一个调用者想创建一个对象，只要知道其名称就可以了。扩展性高，如果想增加一个产品，只要扩展一个工厂类就可以。缺点 添加新产品时，需要编写新的具体产品类,一定程度上增加了系统的复杂度考虑到系统的可扩展性，需要引入抽象层，在客户端代码中均使用抽象层进行定义，增加了系统的抽象性和理解难度

什么时候不用 当被应用到错误的问题类型上时,这一模式会给应用程序引入大量不必要的复杂性.除非为创建对象提供一个接口是我们编写的库或者框架的一个设计上目标,否则我会建议使用明确的构造器,以避免不必要的开销。由于对象的创建过程被高效的抽象在一个接口后面的事实,这也会给依赖于这个过程可能会有多复杂的单元测试带来问题。

1) 商品页只有Product一种实例，构造函数也只接收name，同事仍要求先抽象AbstractFactory、再建具体工厂；你会接受吗？

答：暂时不会接受，直接使用明确的构造器更合适，因为对象类型单一、创建过程也没有环境分支。工厂只有在隔离复杂创建过程或支持多种产品扩展时才产生价值；过早引入抽象层会增加类数量和理解成本。

2) 低代码表单根据配置中的type创建Input、Select、DatePicker，渲染层已经堆了十几个if/else；你会怎样落地工厂？

答：把控件创建集中到工厂接口中，由type决定返回具有统一行为的具体控件，页面只接收创建结果。新增产品时扩展具体产品和工厂能力，不再让渲染层了解构造细节；代价是产品类型持续增加时，相关类和注册关系也会变多。

3) 同一套上传组件在浏览器创建WebUploader，在桌面容器创建NativeUploader，两者都提供upload()；环境判断应该放在哪里？

答：环境判断应收进工厂，由工厂在运行时选择并创建实现相同行为的实例，业务层只依赖统一接口。这样能隔离具体环境与调用代码，但必须确保两种产品真正满足相同契约；若行为差异很大，强行统一会把分支转移到实例内部。

4) 线上新增video类型后，详情页拿到的产品是undefined，随后调用init()报错；你会先查哪几处？

答：先核对配置名称、工厂分支或注册项，以及具体产品是否实现了约定的init()，确认故障发生在选择还是创建阶段。工厂应对未知名称显式报错或返回受控的兜底产品；静默返回undefined会把根因推迟到调用处。

5) 插件平台允许第三方增加控件，团队在“中心化switch工厂”和“插件各自注册工厂”之间争论；你怎么选？

答：若产品集合由核心团队封闭维护，中心化工厂更直观；若第三方需要独立扩展，注册式工厂更符合开放封闭原则。后者减少核心代码改动，却需要处理名称冲突、接口校验和注册时机，不能只靠一张无约束的映射表。

520. 单例模式

单例模式保证一个类最多只有一个实例，并提供统一的访问入口。在JavaScript中，可以用IIFE形成局部作用域，再由闭包中的getInstance()缓存并返回同一个对象，同时隐藏具体构造过程。它适合登录框，以及Vuex、Redux中需要贯穿系统的store。不过单点访问会引入全局状态和模块耦合，使依赖它的代码难以单独测试，所以不需要唯一实例时应尽量避免。顾名思义，单例模式中Class的实例个数最多为1。当需要一个对象去贯穿整个系统执行某些任务时，单例模式就派上了用场。而除此之外的场景尽量避免单例模式的使用，因为单例模式会引入全局状态，而一个健康的系统应该避免引入过多的全局状态。

实现单例模式需要解决以下几个问题：如何确定Class只有一个实例？如何简便的访问Class的唯一实例？Class如何控制实例化的过程？如何将Class的实例个数限制为1？

我们一般通过实现以下两点来解决上述问题：

隐藏Class的构造函数，避免多次实例化通过暴露一个getInstance()方法来创建/获取唯一实例

Javascript中单例模式可以通过以下方式实现：

```
// 单例构造器 const FooServiceSingleton = (function () {  
  // 隐藏的Class的构造函数 function FooService() {}  
  // 未初始化的单例对象 let fooService;  
  return {  
    // 创建/获取单例对象的函数  
    getInstance: function () {  
      if (!fooService) {  
        fooService = new FooService();  
      }  
      return fooService;  
    }  
  }  
})();
```


实现的关键点有：
使用 IIFE 创建局部作用域并即时执行；
getInstance() 为一个闭包，使用闭包保存局部作用域中的单例对象并返回。
我们可以验证下单例对象是否创建成功：

```
const fooService1 = FooServiceSingleton.getInstance();  
const fooService2 = FooServiceSingleton.getInstance();  
console.log(fooService1 === fooService2); // true
```


场景例子
定义命名空间和实现分支型方法
登录框vuex 和 redux中的store
优点 划分命名空间，减少全局变量增强模块性，把自己的代码组织在一个全局变量名下，放在单一位置，便于维护且只会实例化一次。简化了代码的调试和维护
缺点 由于单例模式提供的是一种单点访问，所以它有可能导致模块间的强耦合从而不利于单元测试。无法单独测试一个调用了来自单例的方法的类，而只能把它与那个单例作为一个单元一起测试。

1) 订单列表页每打开一个筛选面板都创建独立FilterModel，评审却要求改成单例，只因为“实例越少越好”；这个理由成立吗？

答：不成立，筛选状态属于各面板时，多个实例正是合理隔离，单例反而会让面板互相覆盖状态。单例适合确实需要由一个对象贯穿系统的任务；仅为减少实例而共享可变状态，会引入隐藏耦合。

2)

后台系统的登录框只能同时存在一个，多个入口都可能触发它；你会怎样控制创建和访问？

答：隐藏具体构造过程，通过getInstance()延迟创建并返回唯一登录框，所有入口使用同一访问点。闭包或模块局部作用域可保存该实例，避免外部重复实例化；同时要定义关闭后的状态重置，否则唯一实例可能残留上次交互数据。

3) 团队把Redux的store做成模块级唯一实例，后来测试要求每个用例使用不同初始状态；还应坚持单例吗？

答：生产入口可以持有唯一store，测试侧则应依赖构造函数或可注入接口，为每个用例生成隔离实例。若业务模块直接导入全局实例，就无法单独替换依赖，只能把模块与单例一起测试；改造会增加传参或容器配置成本。

4) 线上出现两个弹窗同时展示，日志显示getInstance()被调用两次；你会如何判断是单例失效还是状态逻辑失效？

答：先比较两次返回值是否严格相等，再检查构造函数是否仍被导出、单例变量是否被放进了每次调用都会重建的作用域。若实例相同，则继续查展示状态和重复挂载；单例只保证实例数量最多为一，不自动保证一次只执行一个业务动作。

5) 公共组件库要提供通知服务，维护者在“导出唯一实例”和“导出构造器”之间选型；你倾向哪一种？

答：若库的设计目标就是提供全局唯一通知通道，可以暴露受控的getInstance()；若使用方可能需要多个容器或隔离测试，导出明确构造器更灵活。单例简化访问和维护位置，却会形成单点依赖，因此不应替调用方擅自固定生命周期。

521. 策略模式

策略模式就是把同一类行为的不同算法分别封装起来，让调用方可以按场景选择并互相替换。这样能用组合和委托减少多重条件分支，每个策略也可以独立理解、切换和扩展，更符合开放封闭原则。表单校验很适合使用它，例如把非空、长度和手机号校验做成不同策略。代价是策略对象会增多，而且调用方必须清楚各策略的差异，才能选对实现。策略模式简单描述就是：对象有某个行为，但是在不同的场景中，该行为有不同的实现算法。把它们一个个封装起来，并且使它们可以互相替换

策略模式-校验表单

用户名： 密码： 手机号码： 提交 // 策略对象

```
const strategies = {
  isEmpty: function (value, errorMsg) { if (value === '') { return errorMsg; } },
  isNotEmpty: function (value, errorMsg) { if (value.trim() === '') { return errorMsg; } },
  minLength: function (value, length, errorMsg) { if (value.trim().length < length) { return errorMsg; } },
  isMobile: function (value, errorMsg) { if (!/^(\d{3}[\d-]{3}[\d-]{3}|14[57]|\d{3}[\d-]{3}[\d-]{3}|17[7]|\d{3}[\d-]{3}[\d-]{3})\d{8}$/.test(value)) { return errorMsg; } } }
// 验证类
class Validator {
  constructor() { this.cache = [] }
  add(dom, rules) {
    for (let i = 0, rule; rule = rules[i++];) {
      let strategyAry = rule.strategy.split(':')
      let errorMsg = rule.errorMsg
      this.cache.push(() => {
        let strategy = strategyAry.shift()
        strategyAry.unshift(dom.value)
        strategyAry.push(errorMsg)
        return strategies[strategy].apply(dom, strategyAry)
      })
    }
  }
  start() {
    for (let i = 0, validatorFunc; validatorFunc = this.cache[i++];) {
      let errorMsg = validatorFunc()
      if (errorMsg) { return errorMsg }
    }
  }
}
// 调用代码
let registerForm = document.getElementById('registerForm')
let validateFunc = function () {
  let validator = new Validator()
  validator.add(registerForm.userName, [{
    strategy: 'isEmpty', errorMsg: '用户名不可为空'
  }, {
    strategy: 'isNotEmpty', errorMsg: '不允许以空白字符命名'
  }, {
    strategy: 'minLength:2', errorMsg: '用户名长度不能小于2位'
  }])
  validator.add(registerForm.password, [{
    strategy: 'minLength:6', errorMsg: '密码长度不能小于6位'
  }])
  validator.add(registerForm.phoneNumber, [{
    strategy: 'isMobile', errorMsg: '请输入正确的手机号码格式'
  }])
  return validator.start()
}
registerForm.onsubmit = function () {
  let errorMsg = validateFunc()
  if (errorMsg) { alert(errorMsg) return false }
}
// 场景例子
如果一个系统里面有许多类，它们之间的区别仅在于它们的‘行为’，那么使用策略模式可以动态地让一个对象在许多行为中选择一种行为。一个系统需要动态地在几种算法中选择一种。表单验证
```

优点

利用组合、委托、多态等技术和思想，可以有效的避免...

1) 结算页只有“满百减十”一条固定规则，同事仍拆出Context、策略接口和多个占位策略；你认为这是合理扩展吗？

答：当前只有单一且稳定的算法时，直接函数通常更清晰，完整策略体系未必有收益。策略模式适用于同一行为需要动态替换多种实现的场景；为尚不存在的变化预建对象，会增加抽象和选择成本。

2) 注册页的用户名、密码、手机号校验混在提交事件的十几段条件分支里，产品还会持续加规则；你会怎样重构？

答：把isEmpty、minLength、isMobile等校验分别封装为策略，再由验证器按字段配置选择并执行。提交主流程只负责收集规则和返回错误，从而避免继续增长条件分支；新增策略仍需明确参数格式和错误消息约定。

3) 运营要求同一优惠计算器根据会员等级在运行时切换算法，而且后续允许活动模块注入新算法；策略由谁选择？

答：由上下文根据会员等级或活动配置选择策略，具体算法只承担计算行为并保持可替换。注入新策略能减少对主流程的修改，但调用方必须了解各策略的适用条件；选错策略不会被模式本身自动纠正。

4) 线上表单把规则写成minLength:8，校验器偶尔提示错误规则不存在；你会沿着什么链路排查？

答：先检查规则字符串的解析是否修改了共享数组，再核对解析出的策略名、参数顺序和策略表注册项。验证器应对未知策略显式失败，并避免在重复执行时用shift()破坏原配置；否则第二次校验可能得到不同结果。

5) 表单校验既能写成策略表，也能为每条规则创建校验类；团队担心几十个策略对象太碎，你怎么取舍？

答：规则主要是无状态函数时，策略表更轻量，也保留动态选择和独立扩展的价值；存在独立状态或复杂依赖时，再使用策略类更合适。策略模式会增加策略数量，命名、发现和配置管理不清晰时，维护成本可能超过条件分支。

6) 控件平台里，同一个映射表既返回组件实例又存放校验函数；评审说工厂模式和策略模式没有区别，你怎么解释？

答：返回Input或Select实例的部分关注创建哪种产品，属于工厂意图；选择并执行isMobile等算法的部分关注替换行为，属于策略意图。两者都可能用映射实现，但扩展契约不同，混在一张表里会模糊生命周期和错误处理。

522. 迭代器模式

迭代器模式提供统一的顺序遍历方式，同时不暴露集合内部的数据结构。它把“怎么取下一个元素”交给迭代器，因此调用方无须了解容器的具体实现，也不用修改容器原有接口。传统实现通常提供hasNext()和next()；在ES6中，对象实现[Symbol.iterator]并返回带next()的迭代器后，就能使用for...of。例如数字区间可以自行维护当前位置，逐个返回{ value, done }。如果你看到这，ES6中的迭代器 Iterator 相信你还是有点印象的，上面第60条已经做过简单的介绍。迭代器模式简单的说就是提供一种方法顺序一个聚合对象中各个元素，而又不暴露该对象的内部表示。迭代器模式解决了以下问题：提供一致的遍历各种数据结构的方式，而不用了解数据的内部结构提供遍历容器（集合）的能力而无需改变容器的接口

一个迭代器通常需要实现以下接口：

hasNext()：判断迭代是否结束，返回Boolean
next()：查找并返回下一个元素

为Javascript的数组实现一个迭代器可以这么写：

```
const items = [1, 'red', false, 3.14];  
function Iterator(items) {  
  this.items = items;  
  this.index = 0;  
}  
Iterator.prototype = {  
  hasNext: function() { return this.index < this.items.length; },  
  next: function() { return { value: this.items[this.index++], done: false }; }  
};  
验证一下迭代器是否工作：  
const iterator = new Iterator(items);  
while (iterator.hasNext()) { console.log(iterator.next()); }  
// 输出：1, red, false, 3.14  
ES6提供了更简单的迭代循环语法 for...of，使用该语法的前提是操作对象需要实现可迭代协议（The iterable protocol），简单说就是该对象有个Key为Symbol.iterator的方法，该方法返回一个iterator对象。比如我们实现一个 Range 类用于在某个数字区间进行迭代：  
function Range(start, end) {  
  return {  
    [Symbol.iterator]: function () {  
      return {  
        next: function () {  
          if (start < end) {  
            return { value: start++, done: false };  
          }  
          return { done: true, value: end };  
        }  
      };  
    }  
  };  
}  
验证一下：  
for (num of Range(1, 5)) { console.log(num); }  
// 输出：1, 2, 3, 4
```

1)

报表页只读取固定三项数组，同事专门实现带hasNext()、next()的迭代器类；你会保留吗？

答：若数组结构不会替换且没有特殊遍历规则，直接使用现有循环更清晰，自定义迭代器收益有限。迭代器的价值在于统一不同聚合对象的遍历接口并隐藏内部表示；简单集合强行包装只会增加状态和边界处理。

2) 组件库的分页数据内部按多个批次保存，但业务方只想用统一循环顺序读取；你会怎样暴露接口？

答：让容器提供迭代器，把批次索引和元素索引封装在内部，调用方只通过next()获取下一个值。若希望接入for...of，则实现Symbol.iterator并返回符合协议的迭代器；容器结构变化时，消费代码无需同步改写。

3) 组织架构从扁平数组改成树结构，但搜索页仍要求按既定顺序消费员工节点；哪些代码应该变化？

答：应调整容器内部的迭代实现，让它按约定顺序产出树节点，搜索页继续依赖统一遍历接口。这样无需向调用方暴露树的栈、队列或层级细节；不过遍历顺序必须成为明确契约，否则结构迁移可能悄悄改变结果。

4) 自定义Range(1, 5)在线上循环时意外输出了5，而需求是只输出1到4；你会检查什么？

答：检查next()在结束分支返回的对象是否正确标记done: true，以及递增和边界判断发生的顺序。迭代协议以done决定值是否被消费，结束对象即使携带value:

5也不应进入for...of；手写消费逻辑则需同样遵守该标记。

5) 数据容器团队想只提供hasNext()和next()，页面团队坚持必须支持for...of；这两种接口如何取舍？

答：内部专用且调用点受控时，显式的hasNext()、next()已能完成顺序遍历；面向通用JavaScript消费者时，实现可迭代协议更容易接入for...of。后者需要正确提供Symbol.iterator和结束状态，但能与语言统一遍历语法协作。

523. 观察者模式

观察者模式是由被观察对象维护一组观察者，并在自身状态变化时统一通知它们。被观察对象通常提供subscribe()、unsubscribe()和fire()，分别负责订阅、取消订阅和广播通知，从而让双方依赖抽象而不是具体实现。DOM事件和Vue响应式都属于常见使用场景。它能提升扩展性和复用性，但过度使用会弱化对象间的显式联系，让调用链更难跟踪和维护。观察者模式又称发布-订阅模式（Publish/Subscribe Pattern），是我们经常接触到的设计模式，日常生活中的应用也比比皆是，比如你订阅了某个博主的频道，当有内容更新时会收到推送；又比如JavaScript中的事件订阅响应机制。观察者模式的思想用一句话描述就是：被观察对象（subject）维护一组观察者（observer），当被观察对象状态改变时，通过调用观察者的某个方法将这些变化通知到观察者。观察者模式中Subject对象一般需要实现以下API：

subscribe(): 接收一个观察者observer对象，使其订阅自己unsubscribe():

unsubscribe(): 接收一个观察者observer对象，使其取消订阅自己fire():

触发事件，通知到所有观察者

用JavaScript手动实现观察者模式：

```
// 被观察者
function Subject() { this.observers = []; }
Subject.prototype = { // 订阅
  subscribe: function (observer) { this.observers.push(observer); }, // 取消订阅
  unsubscribe: function (observerToRemove) { this.observers = this.observers.filter(observer => { return observer !== observerToRemove; }) }, // 事件触发
  fire: function () { this.observers.forEach(observer => { observer.call(); }); } }
// 验证一下订阅是否成功：
const subject = new Subject();
function observer1() { console.log('Observer 1 Firing!'); }
function observer2() { console.log('Observer 2 Firing!'); }
subject.subscribe(observer1);
subject.subscribe(observer2);
```

subject.fire(); //输出： Observer 1 Firing! Observer 2 Firing! 验证一下取消订阅是否成功：
subject.unsubscribe(observer2); subject.fire(); //输出： Observer 1 Firing! 场景 DOM事件
document.body.addEventListener('click', function() { console.log('hello world!'); });
document.body.click() vue 响应式 优点 支持简单的广播通信，自动通知所有已经订阅过的对象目标对象与观察者之间的抽象耦合关系能单独扩展以及重用增加了灵活性观察者模式所做的工作就是在解耦，让耦合的双方都依赖于抽象，而不是依赖于具体。从而使得各自的变化都不会影响到另一边的变化。 缺点 过度使用会导致对象与对象之间的联系弱化，会导致程序难以跟踪维护和理解

1) 详情页只有父组件和一个子组件，数据通过属性就能传递，同事却增加全局事件中心来同步标题；你赞成吗？

答：不赞成，明确的父子依赖用直接传递更容易追踪，全局订阅会弱化对象之间的可见联系。观察者模式适合一个状态源需要广播给多个可独立扩展的观察者；在局部单向关系中使用会增加维护难度。

2) 后台导航栏、个人中心和权限面板都要响应登录状态变化，你会怎样设计订阅接口？

答：让登录状态主体维护观察者集合，并提供subscribe()、unsubscribe()和触发通知的fire()。各模块只注册自己的响应函数，不直接依赖其他模块，从而能够独立扩展；组件销毁时必须取消订阅，避免无效回调继续存在。

3) 消息中心原来只有两个订阅者，现在插件也能动态订阅和卸载；主体通知期间有人取消订阅，你会关注什么？

答：需要明确通知时修改观察者集合的语义，例如本轮是否仍调用已进入通知序列的观察者，并让实现保持一致。可在触发前使用观察者快照，避免遍历中的增删造成漏通知或重复通知；代价是本轮变化通常到下一次触发才完全生效。

4) 单页应用运行数小时后越来越卡，性能记录显示已卸载页面的回调仍在登录事件触发时执行；你怎么定位？

答：先记录每次subscribe()与unsubscribe()的观察者身份，核对组件卸载路径是否传回同一个函数引用。再检查主体的观察者数组是否持续增长，并为重复订阅建立防护；只清理页面DOM不会自动移除手写订阅。

5) 支付结果需要通知订单页、消息角标和埋点模块，团队在直接逐个调用与观察者模式之间争论；你怎么选？

答：订阅方可能独立增删且支付模块不应了解它们时，观察者模式更利于解耦和广播；接收者固定、调用顺序与失败处理严格时，显式调用更容易控制。观察者会弱化调用链，线上排查需补充事件名称、订阅来源和触发记录。

6) 前端同事说DOM事件和Vue响应式都体现了观察者思想，但准备把所有跨模块通信都改成事件；你会怎么制止过度设计？

答：应按依赖关系选择机制，而不是因为底层具有订阅通知能力就统一事件化。DOM事件和响应式适合各自的状态传播场景，但大量匿名事件会让对象联系难以追踪；核心业务流程仍应保留清晰的数据源、契约和调用边界。

524. 中介者模式

中介者模式让多个同级对象不再直接互相调用，而是把交互统一交给中介者协调。本质上是用中介者与各对象的一对多关系，替代对象之间复杂的网状多对多关系，因此各对象可以独立调整，耦合也更松。购物车中颜色、内存和数量表单的联动，或聊天室成员间发消息，都可以由中介者转发处理。取舍是交互复杂度会集中到中介者里，它很容易变得庞大并且难以维护。在中介者模式中，中介者（Mediator）包装了一

系列对象相互作用的方式，使得这些对象不必直接相互作用，而是由中介者协调它们之间的交互，从而使它们可以松散耦合。当某些对象之间的作用发生改变时，不会立即影响其他的一些对象之间的作用，保证这些作用可以彼此独立的变化。中介者模式和观察者模式有一定的相似性，都是一对多的关系，也都是集中式通信，不同的是中介者模式是处理同级对象之间的交互，而观察者模式是处理Observer和Subject之间的交互。中介者模式有些像婚恋中介，相亲对象刚开始并不能直接交流，而是要通过中介去筛选匹配再决定谁和谁见面。场景 例如购物车需求，存在商品选择表单、颜色选择表单、购买数量表单等等，都会触发change事件，那么可以通过中介者来转发处理这些事件，实现各个事件间的解耦，仅仅维护中介者对象即可。

```
var goods = { // 手机库存 red|32G:3, red|64G:1, blue|32G:7, blue|64G:6; } // 中介者
var mediator = (function() {
    var colorSelect = document.getElementById('colorSelect');
    var memorySelect = document.getElementById('memorySelect');
    var numSelect = document.getElementById('numSelect');
    return {
        changed: function(obj) {
            switch(obj) {
                case colorSelect: // TODO break;
                case memorySelect: // TODO break;
                case numSelect: // TODO break;
            }
        },
        colorSelect: colorSelect,
        memorySelect: memorySelect,
        numSelect: numSelect
    };
})();
colorSelect.onchange = function() { mediator.changed(this); };
memorySelect.onchange = function() { mediator.changed(this); };
numSelect.onchange = function() { mediator.changed(this); };

// 聊天室里 聊天室成员类
function Member(name) {
    this.name = name;
    this.chatroom = null;
}
Member.prototype = {
    // 发送消息
    send: function(message, toMember) {
        this.chatroom.send(message, this, toMember);
    },
    // 接收消息
    receive: function(message, fromMember) {
        console.log(`${fromMember.name} to ${this.name}: ${message}`);
    }
};

// 聊天室类
function Chatroom() {
    this.members = {};
}
Chatroom.prototype = {
    // 增加成员
    addMember: function(member) {
        this.members[member.name] = member;
        member.chatroom = this;
    },
    // 发送消息
    send: function(message, fromMember, toMember) {
        toMember.receive(message, fromMember);
    }
};

// 测试一下
const chatroom = new Chatroom();
const bruce = new Member('bruce');
const frank = new Member('frank');
chatroom.addMember(bruce);
chatroom.addMember(frank);
bruce.send('Hey frank', frank);
// 输出: bruce to frank: hello frank
```

优点 使各对象之间耦合松散，而且可以独立地改变它们之间的交互。中介者和对象一对多的关系取代了对象之间的网状多对多的关系。如果对象之间的复杂耦合度导致维护很困难，而且耦合度随项目变化增速很快，就需要中介者重构代码。缺点 系统中会新增一个中介者对象...

1) 购物车页面里，颜色下拉框直接修改容量选项，容量选项又直接控制数量输入框；只有3个控件时能跑，为什么还值得改成中介者？

答：能跑不代表关系清晰，控件之间的直接调用会形成网状依赖，任一交互变化都可能牵动多个对象。让控件只上报change，由中介者决定库存、选项和数量状态，可把多对多关系收敛为一对多；控件始终固定且交互简单时，引入它反而增加层次。

2) 商品详情页有颜色、容量、数量三个表单和一张组合库存表，你会怎样设计中介者的输入与输出？

答：每个表单只把自身变化和当前值交给中介者，中介者读取完整选择并查询类似red|32G的库存键，再统一更新可选项、数量上限和购买按钮。库存判断应集中在中介者内，避免各控件复制规则；若中介者同时负责请求、渲染和埋点，职责会迅速膨胀。

3) 产品新增套餐、优惠券和配送方式后，原来的一个changed(obj)已经出现十几个case，你会继续扩展还是拆分？

答：十几个case表明交互复杂性已集中转移到中介者，应按库存、促销或履约等稳定业务边界拆分协调逻辑。各子中介者仍只处理同级对象的协作，必要时再由上层编排；拆得过细会产生中介者之间的新耦合，重新形成难追踪的调用链。

4) 线上出现“切换蓝色后购买按钮仍可点，但该容量库存为零”，你会沿哪些事件链排查？

答：先确认颜色控件是否触发change，再检查事件是否把正确对象和值交给mediator.changed，最后核对组合库存键与按钮更新分支。示例式switch(obj)很依赖对象身份和分支完整性，新增控件却漏注册或漏处理都会留下旧状态；修复后还要覆盖组合切换的回归用例。

5) 表单团队主张用观察者模式广播所有字段变化，业务团队坚持使用中介者统一决策，你如何划分两者？

答：字段变化仅需通知多个独立订阅者时，观察者的Subject—Observer关系更自然；字段彼此同级且交互结果需要协调时，中介者更合适。两者都属于集中式一对多通信，但中介者承载对象间协作规则；把所有通知都塞进去会让它沦为难维护的上帝对象。

525. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。

```
// 访问者
class Visitor {
  constructor() {}
  visitConcreteElement(ConcreteElement) {
    ConcreteElement.operation();
  }
}
```

```
// 元素类
class ConcreteElement {
  constructor() {}
  operation() {
    console.log("ConcreteElement.operation invoked");
  }
  accept(visitor) {
    visitor.visitConcreteElement(this);
  }
}
```

```
clientlet visitor = new Visitor();
let element = new ConcreteElement();
elementA.accept(visitor);
```

访问者模式的实现有以下几个要素：

- Visitor Object：访问者对象，拥有一个visit()方法
- Receiving Object：接收对象，拥有一个accept()方法
- visit(receivingObj)：用于Visitor接收一个Receiving Object
- accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力

简单的代码实现如下：

```
Receiving Object：
function Employee(name, salary) {
  this.name = name;
  this.salary = salary;
}
Employee.prototype = {
  getSalary: function () {
    return this.salary;
  },
  setSalary: function (salary) {
    this.salary = salary;
  },
  accept: function (visitor) {
    visitor.visit(this);
  }
}
Visitor Object：
function Visitor() {}
Visitor.prototype = {
  visit: function (employee) {
    employee.setSalary(employee.getSalary() * 2);
  }
}
验证一下：
const employee = new Employee('bruce', 1000);
const visitor = new Visitor();
employee.accept(visitor);
console.log(employee.getSalary());
// 输出：2000
```

场景：对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点：符合单一职责原则优秀的扩展性灵活性。缺点：具体元素对访问者公布细节，违反了迪米特原则；违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难。

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提

供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

526. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){}}visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){}}operation(){console.log("ConcreteElement.operation invoked");}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法Receiving

Object：接收对象，拥有一个 accept() 方法visit(receivingObj)：用于Visitor接收一个Receiving

Objectaccept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {

getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },

accept: function (visitor) { visitor.visit(this); }} Visitor Object： function Visitor() {} Visitor.prototype =

{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); }} 验证一下： const

employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);

console.log(employee.getSalary());//输出：2000

场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

527. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。// 访问者class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }

元素类class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }

clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving

Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving

Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },

accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype =

```
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

528. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor { constructor() {} visit ConcreteElement(ConcreteElement) { ConcreteElement.operation(); } }

```
元素类class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

```
clientlet visitor=new Visitor() let element=new ConcreteElement() elementA.accept(visitor)
```

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); //输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

529. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); //输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

530. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }
```

元素类

```
class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

```
clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素：

- Visitor Object：访问者对象，拥有一个visit()方法
- Receiving Object：接收对象，拥有一个accept()方法
- visit(receivingObj)：用于Visitor接收一个Receiving Object
- accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力

简单的代码实现如下：

```
Receiving Object：function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object：function Visitor() {} Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下：const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

531. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){}}visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}

```
元素类class ConcreteElement{constructor(){}}operation(){console.log("ConcreteElement.operation invoked");}accept(visitor){visitor.visitConcreteElement(this)}}//
```

```
clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)
```

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

532. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {

getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); }} Visitor Object： function Visitor() {} Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); }} 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); //输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作

时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

533. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者classVisitor{constructor(){visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类classConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}accept(visitor){visitor.visitConcreteElement(this)}}//

clientletvisitor=newVisitor()lelement=newConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法Receiving Object：接收对象，拥有一个 accept() 方法visit(receivingObj)：用于Visitor接收一个Receiving Objectaccept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

```
Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {
  getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },
  accept: function (visitor) { visitor.visit(this); }} Visitor Object : function Visitor() {} Visitor.prototype =
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); }} 验证一下： const
employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);
console.log(employee.getSalary()); //输出：2000
```

场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点：符合单一职责原则优秀的扩展性灵活性。缺点：具体元素对访问者公布细节，违反了迪米特原则；违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难。

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

534. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor { constructor() { visit

```
ConcreteElement(ConcreteElement){ConcreteElement.operation()}}//
```

```
元素类classConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}}accept(visitor){visitor.visitConcreteElement(this)}}//
```

```
clientletvisitor=newVisitor()lelement=newConcreteElement()elementA.accept(visitor)
```

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法Receiving

Object：接收对象，拥有一个 accept() 方法visit(receivingObj)：用于Visitor接收一个Receiving

Objectaccept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

```
Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {
```

```
getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },
```

```
accept: function (visitor) { visitor.visit(this); }} Visitor Object： function Visitor() {} Visitor.prototype =
```

```
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); }} 验证一下： const
```

```
employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);
```

```
console.log(employee.getSalary());//输出：2000 场景
```

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象

进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作

时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪

米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

535. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); //输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

536. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。

```
// 访问者
class Visitor {
  constructor() {}
  visitConcreteElement(ConcreteElement) {
    ConcreteElement.operation();
  }
}
```

```
// 元素类
class ConcreteElement {
  constructor() {}
  operation() {
    console.log("ConcreteElement.operation invoked");
  }
  accept(visitor) {
    visitor.visitConcreteElement(this);
  }
}
```

```
clientlet visitor = new Visitor();
let element = new ConcreteElement();
elementA.accept(visitor);

访问者模式的实现有以下几个要素：
Visitor Object：访问者对象，拥有一个visit()方法
Receiving Object：接收对象，拥有一个accept()方法
visit(receivingObj)：用于Visitor接收一个Receiving Object
accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力
简单的代码实现如下：
Receiving Object：
function Employee(name, salary) {
  this.name = name;
  this.salary = salary;
}
Employee.prototype = {
  getSalary: function () {
    return this.salary;
  },
  setSalary: function (salary) {
    this.salary = salary;
  },
  accept: function (visitor) {
    visitor.visit(this);
  }
}
Visitor Object：
function Visitor() {}
Visitor.prototype = {
  visit: function (employee) {
    employee.setSalary(employee.getSalary() * 2);
  }
}
验证一下：
const employee = new Employee('bruce', 1000);
const visitor = new Visitor();
employee.accept(visitor);
console.log(employee.getSalary());
//输出：2000
```

场景
对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提

供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

537. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){}}visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){}}operation(){console.log("ConcreteElement.operation invoked");}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法Receiving

Object：接收对象，拥有一个 accept() 方法visit(receivingObj)：用于Visitor接收一个Receiving

Objectaccept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {

getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },

accept: function (visitor) { visitor.visit(this); }} Visitor Object： function Visitor() {} Visitor.prototype =

{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); }} 验证一下： const

employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);

console.log(employee.getSalary());//输出：2000

场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

538. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。// 访问者class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }

元素类class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }

clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() {} Visitor.prototype =


```
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

第二部分：进阶题

【JS基础】

539. 类型及检测方式

JavaScript有 7 种基本类型和Object引用类型，检测时要根据目标选择typeof、instanceof或Object.prototype.toString.call()。typeof适合基本类型和函数，但typeof null、数组及普通对象都会得到object。instanceof本质上检查原型链，适合引用类型，却不能正确判断基本类型，而且constructor可能被修改。

需要精确区分数组、日期、正则等对象时，我一般会用Object.prototype.toString.call()。1. JS内置类型 JavaScript 的数据类型有下图所示 其中，前 7 种类型为基础类型，最后1 种（Object）为引用类型，也是你需要重点关注的，因为它在日常工作中是使用得最频繁，也是需要关注最多技术细节的数据类型 JavaScript一共有8种数据类型，其中有7种基本数据类型：Undefined、Null、Boolean、Number、String、Symbol（es6新增，表示独一无二的值）和BigInt（es10新增）；1种引用数据类型——Object（Object本质上是由一组无序的名值对组成的）。里面包含function、Array、Date等。JavaScript不支持任何创建自定义类型的机制，而所有值最终都将是上述 8 种数据类型之一。引用数据类型:对象Object（包含普通对象-Object，数组对象-Array，正则对象-RegExp，日期对象-Date，数学函数-Math，函数对象-Function）

引用数据类型:对象Object（包含普通对象-Object，数组对象-Array，正则对象-RegExp，日期对象-Date，数学函数-Math，函数对象-Function）在这里，我想先请你重点了解下面两点，因为各种 JavaScript 的数据类型最后都会在初始化之后放在不同的内存中，因此上面的数据类型大致可以分成两类来进行存储：

原始数据类型：基础类型存储在栈内存，被引用或拷贝时，会创建一个完全相等的变量；占据空间小、大小固定，属于被频繁使用数据，所以放入栈中存储。引用数据类型：引用类型存储在堆内存，存储的是地址，多个引用指向同一个地址，这里会涉及一个“共享”的概念；占据空间大、大小不固定。引用数据类型在栈中存储了指针，该指针指向堆中该实体的起始地址。当解释器寻找引用值时，会首先检索其在栈中的地址，取得地址后从堆中获得实体。JavaScript 中的数据是如何存储在内存中的？在 JavaScript 中，原始类型的赋值会完整复制变量值，而引用类型的赋值是复制引用地址。在 JavaScript 的执行过程中，主要有三种类型内存空间，分别是代码空间、栈空间、堆空间。其中的代码空间主要是存储可执行代码的，原始类型(Number、String、Null、Undefined、Boolean、Symbol、BigInt)的数据值都是直接保存在“栈”中的，引用类型(Object)的值是存放在“堆”中的。因此在栈空间中(执行上下文)，原始类型存储的是变量的值，而引用类型存储的是其在“堆空间”中的地址，当 JavaScript 需要访问该数据的时候，是通过栈中的引用地址来访问的，相当于多了一道转手流程。

在编译过程中，如果 JavaScript 引擎判断到一个闭包，也会在堆空间创建换一个“closure(fn)”的对象（这是一个内部对象，JavaScript 是无法访问的），用来保存闭包中的变量。所以闭包中的变量是存储在“堆空间”中的。JavaScript 引擎需要用栈来维护程序执行期间上下文的状态，如果栈空间大了话，所有的数据都存放在栈空间里面，那么会影响到上下文切换的效率，进而又影响到整个程序的执行效率。通常情况下，栈空间都不会设置太大，主要用来存放一些原始类型的小数据。而引用类型的数据占用的空间都比较大，所以这一类数据会被存放在堆中，堆空间很大，能存放很多大的数据，不过缺点是分配内存和回收内存都会占用一定的时间。因此需要“栈”和“堆”两种空间。

题目一：初出茅庐 let a={name:'lee',age:18}let b=a;console.log(a.name);//第一个console b.name='son';console.log(a.name);//第二个console console.log(b.name);//

第三个console 这道题比较简单，我们可以看到第一个 console 打出来 name 是 'lee'，这应该没什么疑问；但是在执行了 b.name='son' 之后，结果你会发现 a 和 b 的属性 name 都是 'son'，第二个和第三个打印结果是一样的，这里就体现了引用...

1) 埋点SDK用typeof payload === 'object'判断可展开数据，线上收到null后在Object.keys(payload)报错，判断条件该怎样收紧？

答：应先排除null，至少写成payload !== null && typeof payload === 'object'，因为typeof null的结果也是object。若只接受普通对象，还要继续排除数组、日期等引用类型；仅靠typeof无法完成这种细分。

2) 表单校验要区分数组、日期、正则和普通对象，数据量不大但来源复杂，你会统一使用哪种检测方式，返回值要注意什么？

答：可使用Object.prototype.toString.call(value)获取形如[object Array]、[object Date]的标签，再统一提取类型名。它比typeof更适合区分这些内置引用类型；返回值的大小写和格式必须固定，否则调用方容易把Array与array混用。

3) 微前端页面从另一个iframe传来数组，当前代码用value instanceof Array却返回false，为什么，应该换什么判断？

答：instanceof检查右侧构造函数的prototype是否出现在对象原型链上，不同iframe拥有不同的内建造函数和原型。判断数组应使用Array.isArray(value)，或在统一类型工具中使用Object.prototype.toString.call；跨执行环境时不要依赖当前窗口的Array.prototype。

4) 线上有个对象被库代码替换了原型，随后value.constructor === Model判断失效，你会按什么顺序排查？

答：先查看对象当前原型链以及constructor属性来自哪里，因为替换prototype或覆盖constructor都会让该判断失真。若要判断原型关系应改用合适的instanceof，若要识别数据形状则校验字段；两者语义不同，也都不能仅凭构造器名称兜底。

5) 接口字段可能是数字0、字符串"0"、null或NaN，产品要求只有后两者显示“数据异常”，能不能用if (!value)？

答：不能，!value会同时把数字0等假值判为异常，破坏合法数据。应明确写成value === null || Number.isNaN(value)；不要用全局isNaN代替，后者会进行类型转换，使非数字字符串也可能被判为NaN。

540. This

普通函数的this在调用时确定：直接调用通常走默认绑定，obj.fn()指向obj，构造调用则指向new创建的对象。多种规则同时出现时，优先级是new、显式绑定、隐式绑定、默认绑定。箭头函数没有自己的this，它会沿用最近外层普通函数的this，因此用bind也改不了。对普通函数多次调用bind时，最终绑定对象由第一次bind决定。不同情况的调用，this指向分别如何。顺带可以提一下es6中箭头函数没有this,arguments,super等，这些只依赖包含箭头函数最接近的函数

我们先来看几个函数调用的场景
function foo(){console.log(this.a)}var a=1foo()const obj={a:2,foo:foo}obj.foo()const c=new foo() 对于直接调用foo来说，不管foo函数被放在了什么地方，this一定是window对于obj.foo()来说，我们只需要记住，谁调用了函数，谁就是this，所以在这个场景下foo函数中的this就是obj对象对于new的方式来说，this被永远绑定在了c上面，不会被任何方式改变this

说完了以上几种情况，其实很多代码中的this应该就没什么问题了，下面让我们看看箭头函数中的this
function a(){return ()=>{return ()=>{console.log(this)}}}console.log(a())() 首先箭头函数其实是没有this的，箭头函数中的this只取决包裹箭头函数的第一个普通函数的this。在这个例子中，因为包裹箭头函数的

第一个普通函数是a，所以此时的this是window。另外对箭头函数使用bind这类函数是无效的。最后种情况也就是bind这些改变上下文的API了，对于这些函数来说，this取决于第一个参数，如果第一个参数为空，那么就是window。那么说到bind，不知道大家是否考虑过，如果对一个函数进行多次bind，那么上下文会是什么呢？

```

let a = {}
let fn = function() { console.log(this) }
fn.bind(a)() // => ?

```

如果你认为输出结果是a，那么你就错了，其实我们可以把上述代码转换成另一种形式

```

// fn.bind().bind(a) 等于 let fn2 = function fn1() { return function() { return fn.apply() }.apply(a) }
fn2()

```

可以从上述代码中发现，不管我们给函数bind几次，fn中的this永远由第一次bind决定，所以结果永远是window

```

let a = { name: 'poetries' }
function foo() { console.log(this.name) }
foo.bind(a)() // => 'poetries'

```

以上就是this的规则了，但是可能会发生多个规则同时出现的情况，这时候不同的规则之间会根据优先级最高的来决定this最终指向哪里。首先，new的方式优先级最高，接下来是bind这些函数，然后是obj.foo()这种调用方式，最后是foo这种调用方式，同时，箭头函数的this一旦被绑定，就不会再被任何方式所改变。

函数执行改变this 由于 JS 的设计原理：在函数中，可以引用运行环境中的变量。因此就需要一个机制来让我们可以在函数体内部获取当前的运行环境，这便是this。

因此要明白this指向，其实就是要搞清楚函数的运行环境，说人话就是，谁调用了函数。例如 obj.fn()，便是obj调用了函数，既函数中的this === objfn()，这里可以看成window.fn()，因此this === window

但这种机制并不完全能满足我们的业务需求，因此提供了三种方式可以手动修改this的指向：

```

call: fn.call(target, 1, 2)
apply: fn.apply(target, [1, 2])
bind: fn.bind(target)(1, 2)

```

1) 管理后台启用严格模式后，工具函数foo()内访问this.user报错，而旧页面里曾经指向window，你怎么解释这个差异？

答：普通函数直接调用时没有接收者，严格模式下this是undefined，因此读取this.user会报错；非严格浏览器脚本中才可能回退到window。业务代码不应依赖这种全局回退，应显式传入对象或以obj.foo()的形式调用。

2) React类组件把this.handleSave直接交给按钮，点击后this丢失；团队在构造器bind和class字段箭头函数之间怎么选？

答：两种方式都能让回调保留组件实例：构造器中执行一次bind，或把处理器定义为捕获外层this的箭头函数。选型应服从项目编译配置和代码规范；不要在每次渲染时临时bind，否则会持续产生新的函数引用。

3) 事件系统原来用handler.call(context, event)注入上下文，现在有人把处理器改成箭头函数，为什么context不再生效？

答：箭头函数没有自己的this，它从定义位置捕获外层this，所以call的第一个参数无法改写它。若事件系统契约依赖动态上下文，应保留普通函数；若处理器只依赖词法作用域，箭头函数更明确，但会放弃运行时注入能力。

4) 一段代码执行fn.bind(user).bind(admin)()，日志仍打印user.name，排查时你会如何判断是哪次绑定生效？

答：第一次bind(user)已返回一个绑定函数，后续再对该绑定函数执行bind(admin)不会替换原目标函数的绑定上下文。因此应沿函数创建链找到最早的bind；多次绑定会增加理解成本，若需要动态切换对象，应改用call、apply或显式参数。

5) 对象方法obj.run既先被bind(config)，又被new调用，代码评审中有人认为实例的this仍是config，你会怎么判断？

答：以new调用绑定函数时，构造调用创建的新实例应成为原函数的this，绑定对象不会作为实例上下文。实现手写bind时必须识别构造调用并转交原型关系；若原函数返回对象，还要遵循构造函数显式返回对象的语义，简单apply并不完整。

541. apply/call/bind 原理

call、apply和bind都能指定函数执行时的this，区别在传参形式和是否立即执行。call逐个接收参数，apply接收参数数组，两者会马上调用函数；bind则返回新函数，还能预先保存部分参数。手写call或apply时，可以把原函数临时挂到目标对象上执行，再删除该属性，使用Symbol能避免属性冲突。实现bind还要处理构造调用：通过new执行返回函数时，应让新实例成为this。

call、apply和bind是挂在Function对象上的三个方法，调用这三个方法的必须是一个函数。

func.call(thisArg, param1, param2, ...) func.apply(thisArg, [param1,param2,...]) func.bind(thisArg, param1, param2, ...) 在浏览器里，在全局范围内this 指向window对象；在函数中，this永远指向最后调用他的那个对象；构造函数中，this指向new出来的那个新的对象；call、apply、bind中的this被强绑定在指定的那个对象上；箭头函数中this比较特殊,箭头函数this为父作用域的this，不是调用时的this.要知道前四种方式,都是调用时确定,也就是动态的,而箭头函数的this指向是静态的,声明的时候就确定了下来；apply、call、bind都是js给函数内置的一些API，调用他们可以为函数指定this的执行,同时也可以传参。 let a={value:1}function getValue(name,age){console.log(name)console.log(age)console.log(this.value)}getValue.call(a,'poe','24')getValue.apply(a,['poe','24'])

bind和其他两个方法作用也是一致的，只是该方法会返回一个函数。并且我们可以通过bind实现柯里化方法的应用场景

下面几种应用场景，你多加体会就可以发现它们的理念都是“借用”方法的思路。我们来看看都有哪些。

判断数据类型

用Object.prototype.toString来判断类型是最合适的，借用它我们几乎可以判断所有类型的数据 function getType(obj){let type=typeof obj;if(type!=="object"){return type;}return Object.prototype.toString.call(obj).replace(/^\\$/,'\$1');} 类数组借用方法 类数组因为不是真正的数组，所有没有数组类型上自带的种种方法，所以我们可以利用一些方法去借用数组的方法，比如借用数组的push方法，看下面的一段代码。

```
var arrayLike={0:'java',1:'script',length:2}Array.prototype.push.call(arrayLike,'jack','lily');console.log(typeof arrayLike);// 'object'console.log(arrayLike);// {0: "java", 1: "script", 2: "jack", 3: "lily", length: 4}
```

用call的方法来借用Array 原型链上的 push方法，可以实现一个类数组的push方法，给arrayLike添加新的元素 获取数组的最大 / 最小值 我们可以用 apply 来实现数组中判断最大 / 最小值，apply直接传递数组作为调用方法的参数，也可以减少一步展开数组，可以直接使用Math.max、Math.min来获取数组的最大值 / 最小值，请看下面这段代码。 let arr=[13,6,10,11,16];const max=Math.max.apply(Math,arr);const min=Math.min.apply(Math,arr);console.log(max);// 16console.log(min);// 6

实现一个 bind 函数 对于实现以下几个函数，可以从几个方面思考 不传入第一个参数，那么默认为window改变了this指向，让新的对象可以执行该函数。那么思路是否可以变成给新的对象添加一个函数，然后在执行完以后删除？ Function.prototype.myBind=function(context){if(typeof this !== 'function'){throw new TypeError('Error')}}var _this=thisvar args=[...arguments].slice(1)// 返回一个函数return functionF...

1) 代码评审里有人说call、apply、bind都只是参数写法不同，但保存const saved = fn.bind(user, 1)后没有立即执行，你怎么反驳？

答：call和apply会立即调用函数，只是前者逐个传参、后者接收参数集合；bind返回新的绑定函数，调用可以延后并支持预置参数。把三者当成同一种立即调用API，会导致副作用时机和返回值判断错误。

2) 你手写的myCall用固定属性context.fn =

this，线上对象本来就有fn字段，调用后字段丢了，怎么修？

答：固定属性名会覆盖对象已有字段，随后delete context.fn又把原值删除，应改用临时Symbol作为键并在调用后清理。清理最好放进finally，否则目标函数抛错时临时属性会残留；此外还要处理不可扩

展对象，这种挂载模拟存在天然边界。

3) 旧工具的myCall写成context = context ||

window，传入数字0或空字符串时却把this指向全局对象，问题在哪里？

答：||会把0、空字符串和false都当成缺省值，因而错误替换合法的thisArg。实现应只按目标语义处理null和undefined，并考虑原始值装箱及严格模式差异；简单的对象挂载法无法完全复刻原生行为。

4) 批量统计页面对一个超大数组执行Math.max.apply(null, values)后抛出参数数量相关异常，你会如何改造并验证？

答：apply会把数组元素作为独立实参传入，数组足够大时可能超过引擎允许的参数数量，不能把具体上限当成稳定契约。应使用reduce逐项比较，或分块求局部最大值再合并；同时明确空数组结果，避免默认得到不符合业务预期的值。

5) 手写bind只做return (...args) => fn.apply(context, args)，测试普通调用通过，但new Bound()的实例关系失败，缺了哪些语义？

答：这种箭头包装既不能作为构造函数，也没有处理new对绑定上下文的覆盖，因此无法模拟原生bind的构造调用。返回值应使用普通函数，构造时让新实例作为this并衔接原函数原型；完整复制原生函数的所有内部行为仍不现实，应标明实现范围。

542. 变量提升

变量提升不是把代码搬到顶部，而是在创建执行环境时提前处理声明。var会先被初始化为undefined，所以声明前读取不会报错，但赋值仍留在原来的位置。let和const同样会建立绑定，只是初始化前处于TDZ，提前访问会报错。函数声明会连同函数体一起提升；同名声明出现时，后面的函数会覆盖前面的函数，而且函数声明优先于变量声明。当执行JS代码时，会生成执行环境，只要代码不是写在函数中的，就是在全局执行环境中，函数中的代码会产生函数执行环境，只此两种执行环境。 b()// call bconsole.log(a)// undefinedvara='Hello world'functionb(){console.log('call b')}想必以上的输出大家肯定都已经明白了，这是因为函数和变量提升的原因。通常提升的解释是说将声明的代码移动到了顶部，这其实没有什么错误，便于大家理解。但是更准确的解释应该是：在生成执行环境时，会有两个阶段。第一个阶段是创建的阶段，JS解释器会找出需要提升的变量和函数，并且给他们提前在内存中开辟好空间，函数的话会将整个函数存入内存中，变量只声明并且赋值为undefined，所以在第二个阶段，也就是代码执行阶段，我们可以直接提前使用在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升 b()// call bsecondfunctionb(){console.log('call b fist')}functionb(){console.log('call b second')}varb='Hello world' var会产生很多错误，所以在 ES6中引入了let。let不能在声明前使用，但是这并不是常说的let不会提升，let提升了，在第一阶段内存也已经为他开辟好了空间，但是因为这个声明的特性导致了并不能在声明前使用

1) 你接手的活动页里先执行console.log(price)，下一行才写var price = 99，页面没有报错却打印了undefined；如果同事坚持说“引擎把代码搬到了顶部”，你怎么纠正？

答：结果是undefined，但引擎并没有真的重排源码。执行环境创建阶段会登记var price并初始化为undefined，执行阶段到赋值语句才写入99；“搬到顶部”只能作为表象解释，遇到函数声明冲突时容易误判。

2) 一个旧版结算页同时声明了两个function submit()，后面又写var submit = 'disabled'，但在赋值前调用submit()执行的是第二个函数；你如何解释这段初始化过程？

答：创建阶段会处理函数声明与变量声明，同名的后一个函数声明覆盖前一个，而同名var声明不会把已登记的函数改成undefined。执行到var submit = 'disabled'时才发生赋值，此后submit()才会因值不再是函数而失败。

3) 团队把配置读取从var token改为let token，监控从静默拿到undefined变成了启动时抛出ReferenceError；这是回归还是更严格的约束？

答：这是声明语义改变带来的主动失败，不代表let完全没有提升。绑定在块级作用域创建时已经存在，但声明执行前处于暂时性死区，任何读取都会抛出ReferenceError；代价是旧代码会暴露初始化顺序问题，却能避免错误值继续传播。

4) 线上首屏偶发ReferenceError: Cannot access 'config' before initialization，同一代码块里确实写了let config；你会沿着什么方向定位，而不是把它当成变量未声明？

答：先查报错读取点是否位于let config的声明执行之前，包括条件分支、初始化表达式和间接函数调用。该名称已绑定到当前块，外层同名变量也会被遮蔽，因此不能继续沿作用域链取外层值；修复应调整初始化顺序或拆分作用域。

5) 报表页的循环用var

i注册五个延迟回调，评审中有人主张只改let，另一人要求兼容现有写法；你会怎样选择？

答：能使用块级绑定优先改为let，因为循环每轮都会形成独立绑定，回调自然取得当轮值。若必须保留var，可用IIFE把i作为参数封入独立作用域，或用setTimeout(fn, 0, i)透传实参；后两种更啰嗦，也更依赖调用形式。

543. 执行上下文

执行上下文就是JS执行一段代码时所需的运行环境，核心包含变量对象、作用域链和this。创建阶段会登记变量、函数声明和形参，因此会出现变量提升；执行阶段才逐行完成赋值和调用。全局代码、函数和eval会形成不同上下文，函数调用时，新上下文会压入执行栈，结束后再弹出。函数内部实际通过活动对象AO管理局部变量及arguments，箭头函数没有自己的arguments。

当执行JS代码时，会产生三种执行上下文：全局执行上下文、函数执行上下文、eval执行上下文。

每个执行上下文中都有三个重要的属性：变量对象（VO），包含变量、函数声明和函数的形参，该属性只能在全局上下文中访问作用域链（JS采用词法作用域，也就是说变量的作用域是在定义时就决定了）；this

对于上述代码，执行栈中有两个上下文：全局上下文和函数上下文。

stack=[globalContext,fooContext] 对于全局上下文来说，VO大概是这样的：globalContext.VO===globe globalContext.VO={a:undefined,foo:;}

对于函数foo来说，VO不能访问，只能访问到活动对象（AO）

fooContext.VO===foo.AOfooContext.AO={i:undefined,b:undefined,arguments:<>}// arguments

是函数独有的对象(箭头函数没有)// 该对象是一个伪数组，有`length`属性且可以通过下标访问元素//

该对象中的`callee`属性代表函数本身// `caller`属性代表函数的调用者 对于作用域链，可以把它理解成

包含自身变量对象和上级变量对象的列表，通过[[Scope]]属性查找上级变量 fooContext. [[Scope]]=[globalContext.VO]

fooContext.Scope=fooContext. [[Scope]]+fooContext.VOfooContext.Scope=[fooContext.VO,globalContext.VO]

接下来让我们看一个老生常谈的例子，var b();// call

bconsole.log(a)// undefinedvara='Hello world'functionb(){console.log('call b')}想必以上的输出大家肯定都已经明白了，这是因为函数和变量提升的原因。通常提升的解释是说将声明的代码移动到了顶部，这其实没有什么错误，便于大家理解。但是更准确的解释应该是：在生成执行上下文时，会有两个阶段。

第一个阶段是创建的阶段（具体步骤是创建VO），JS解释器会找出需要提升的变量和函数，并且给他们

提前在内存中开辟好空间，函数的话会将整个函数存入内存中，变量只声明并且赋值为undefined，所以在

第二个阶段，也就是代码执行阶段，我们可以直接提前使用。

在提升的过程中，相同的函数会覆盖上一个函数，并且函数优先于变量提升 b();// call b

secondfunctionb(){console.log('call b fist')}functionb(){console.log('call b second')}varb='Hello

world' var会产生很多错误，所以在ES6中引入了let。let不能在声明前使用，但是这并不是常说的let不会提升，let提升了声明但没有赋值，因为临时死区导致了并不能在声明前使用。

对于非匿名的立即执行函数需要注意以下一点 `var foo=1(function foo(){foo=10console.log(foo)}())// -> 1`
`foo() { foo = 10; console.log(foo) }` 因为当JS解释器在遇到非匿名的立即执行函数时，会创建一个辅助的特定对象，然后将函数名称作为这个对象的属性，因此函数内部才可以访问到foo，但是这个值又是只读的，所以对它的赋值并不生效，所以打印的结果还是这个函数，并且外部的值也没有发生更改。
`specialObject={};Scope=specialObject+Scope;foo=new FunctionExpression;foo.[[Scope]]=Scope;specialObject.foo=foo;// {DontDelete}, {ReadOnly}deleteScope[0];// remove specialObject from the front of scope chain` 总结 执行上下文可以简单理解为一个对象： 它包含三个部分：变量对象(VO)作用域链(词法...

1) 订单页运行foo()时，调用栈里既有全局代码又有foo，同事却说整个页面只有一个“作用域对象”；你会拿这次调用怎样反驳？

答：页面启动会建立全局执行上下文，调用foo()时还会把新的函数执行上下文压入执行栈，两者不是同一个对象。foo执行期间位于栈顶，全局上下文暂停；函数返回后其上下文出栈，控制权才交回调用方。

2) 你在调试器里检查function foo(i){ var b = 20 }，产品要求解释为什么函数内既能找到i、b，又能读取页面级变量；你会怎么说明查找路径？

答：函数上下文的活动对象会包含形参i、局部变量b和函数独有的arguments，标识符先在这里查找。未命中时再沿词法作用域链访问上级变量对象，直到全局对象；这条链由函数定义位置决定，不会因调用位置改变。

3) 一段遗留脚本把普通函数改成箭头函数后，读取arguments[0]直接失败；在不改变调用参数数量的前提下，根因是什么？

答：普通函数执行上下文具有自己的arguments伪数组，可通过下标和length读取实参；箭头函数没有自己的arguments。若业务仍需收集参数，应显式使用剩余参数，不能假定箭头函数会复制外层普通函数的活动对象。

4) 生产环境出现Maximum call stack size exceeded，调用栈显示renderNode上百层重复，而每层只创建少量局部变量；你如何把故障和执行上下文联系起来排查？

答：每次调用renderNode都会创建函数执行上下文并压栈，递归没有及时结束就会持续占用调用栈，最终溢出。先核对终止条件和输入树是否存在异常回路，再考虑改成显式栈迭代；仅减少局部变量不能消除调用层数。

5) 插件系统允许传入一段eval配置脚本，架构师认为它与普通函数执行完全相同；在执行上下文模型里，你会指出什么差异和代价？

答：eval会产生独立类型的执行上下文，不能简单等同于一次普通函数调用。它还让标识符和作用域关系在运行时才更难确定，增加理解与排查成本；若配置只需要数据，应优先采用结构化输入，避免引入动态代码执行边界。

544. 作用域

作用域决定变量在哪里可访问，而作用域链决定当前找不到变量时去哪里继续找。JS使用词法作用域，链路在函数定义时就确定，调用位置不会把它改掉；查找会从当前作用域逐层向外，最后到全局作用域。常见范围可以归为全局、函数和块级作用域，其中let、const能形成块级作用域并存在TDZ。工程中应尽量缩小变量范围，避免全局变量过多造成命名冲突。作用域：作用域是定义变量的区域，它有一套访问变量的规则，这套规则来管理浏览器引擎如何在当前作用域以及嵌套的作用域中根据变量（标识符）进行变量查找

作用域链：作用域链的作用是保证对执行环境有权访问的所有变量和函数的有序访问，通过作用域链，我们可以访问到外层环境的变量和函数。作用域链的本质是一个指向变量对象的指针列表。变量对象是一个包含了执行环境中所有变量和函数的对象。作用域链的前端始终都是当前执行上下文的变量对象。全局执行上下文的变量对象（也就是全局对象）始终是作用域链的最后一个对象。当我们查找一个变量时，如果当前执行环境中没有找到，我们可以沿着作用域链向后查找作用域链的创建过程跟执行上下文的建立有关....

作用域可以理解为变量的可访问性，总共分为三种类型，分别为：全局作用域、函数作用域、块级作用域，ES6中的let、const就可以产生该作用域。其实看完前面的闭包、this这部分内部的话，应该基本能了解作用域的一些应用。

一旦我们将这些作用域嵌套起来，就变成了另外一个重要的知识点「作用域链」，也就是JS到底是如何访问需要的变量或者函数的。首先作用域链是在定义时就被确定下来的，和箭头函数里的this一样，后续不会改变，JS会一层层往上寻找需要的内容。其实作用域链这个东西我们在闭包小结中已经看到过它的实体了：[[Scopes]]

图中的[[Scopes]]是个数组，作用域的一层层往上寻找就等同于遍历[[Scopes]]。1. 全局作用域
全局变量是挂载在window

对象下的变量，所以在网页中的任何位置你都可以使用并且访问到这个全局变量

```
varglobalName='global';functiongetName(){console.log(globalName)}//globalvarname='inner'console.log(name)//innergetName();console.log(name);//console.log(globalName);//globalfunctionsetName(){vName='setName';setName();console.log(vName);//setName
```

从这段代码中我们可以看到，globalName这个变量无论在什么地方都是可以被访问到的，所以它就是全局变量。而在getName函数中作为局部变量的name变量是不具备这种能力的当然全局作用域有相应的缺点，我们定义很多全局变量的时候，会容易引起变量命名的冲突，所以在定义变量的时候应该注意作用域的问题。2. 函数作用域
函数中定义的变量叫作函数变量，这个时候只能在函数内部才能访问到它，所以它的作用域也就是函数的内部，称为函数作用域

```
functiongetName(){varname='inner';console.log(name);//inner}getName();console.log(name);
```

除了这个函数内部，其他地方都是不能访问到它的。同时，当这个函数被执行完之后，这个局部变量也相应会被销毁。所以你会看到在getName函数外面的name是访问不到的3. 块级作用域
ES6中新增了块级作用域，最直接的表现就是新增的let关键词，使用let关键词定义的变量只能在块级作用域中被访问，有“暂时性死区”的特点，也就是说这个变量在定义之前是不能被使用的。在JS编码过程中if语句及for语句后面{...}这里面所包括的，就是块级作用域

```
console.log(a)//a is not definedif(true){leta='123'; console.log(a); // 123}console.log(a)//a is not defined
```

从这段代码可以看出，变量a是在if语句{...}中由let关键词进行定义的变量，所以它的作用域是if语句括号中的那部分，而在外面进行访问a变量是会报错的，因为这里不是它的作用域。所以在if代码块的前后输出a这个变量的结果，控制台会显示a并没有定义

1) 运营页在if (enabled) { let banner = 'A'

}后执行console.log(banner)，同事认为条件为真就应该能读取；你如何解释这个反例？

答：即使条件为真，banner的可访问范围也只在if的花括号内，块执行结束后外部读取仍会报未定义。是否执行过赋值不会扩大词法作用域；若外部确实需要该值，应在外层声明并在块内赋值。

2) 两个通过普通接入的营销组件都声明了顶层var

status，后加载的组件覆盖前者；在不引入模块系统的旧页面里你会怎样隔离？

答：可把每个组件包进IIFE，让内部变量落在各自的函数作用域中，只显式暴露必要接口。这样能减少顶层var挂到全局对象造成的命名冲突；代价是依赖和导出需要手工组织，不能获得模块系统的静态边界。

3) 代码评审里有人把函数从模块顶部移动到点击处理器内部，理由是“调用位置没变，查到的变量也不会变”；若内外层都有theme，结果会怎样变化？

答：函数的作用域链由定义位置决定，移动定义位置可能改变它捕获的theme，即使调用语句完全不变。查找会从函数自身环境逐层向外，命中最近的同名绑定即停止；因此这种重构必须核对变量遮蔽，而不能只比较调用点。

4) 线上日志出现vName is not defined，代码只在某个开关分支里执行vName = 'setName'，另一个分支随后读取它；你会先检查哪些作用域条件？

答：先确认赋值分支是否真正执行，以及脚本是否处于允许隐式创建全局变量的运行环境。未声明赋值不能作为可靠的跨分支通信方式，严格约束下还可能直接报错；应在明确的外层作用域声明状态，并保证所有读取路径都有初始化。

5) 一个页面需要让计数器只被组件内部使用，团队在“顶层全局变量”和“函数作用域封装”之间争论；你会如何取舍？

答：应选函数作用域封装，因为全局变量位于作用域链末端，页面各处都可能访问或覆盖，命名冲突面更大。封装后只返回必要操作函数，让内部计数保持不可直接访问；代价是外部调试和修改必须经过暴露的接口。

545. 闭包

闭包本质上是函数保留了对其定义时外层作用域的引用，因此外层函数结束后仍能访问相关变量。它常用于封装私有变量，定时器、事件监听和作为参数传递的回调也会自然形成这种引用关系。比如循环里用var配合setTimeout，回调执行时共享的i已变成6，所以会连续输出6。改用let、IIFE，或把当前值作为定时器参数传入，都能让每次回调拿到对应的值。闭包其实就是一个可以访问其他函数内部变量的函数。创建闭包的最常见的方式就是在一个函数内创建另一个函数，创建的函数可以访问到当前函数的局部变量。因为通常情况下，函数内部变量是无法在外部访问的（即全局变量和局部变量的区别），因此使用闭包的作用，就具备实现了能在外部访问某个函数内部变量的功能，让这些内部变量的值始终可以保存在内存中。下面我们通过代码先来看一个简单的例子

```
function fun1(){var a=1;return function(){console.log(a);}};fun1();var result=fun1();result();// 1//
```

结合闭包的概念，我们把这段代码放到控制台执行一下，就可以发现最后输出的结果是 1（即 a 变量的值）。那么可以很清楚地发现，a 变量作为一个 fun1 函数的内部变量，正常情况下作为函数内的局部变量，是无法被外部访问到的。但是通过闭包，我们最后还是可以拿到 a 变量的值。闭包有两个常用的用途。闭包的第一个用途是使我们在函数外部能够访问到函数内部的变量。通过使用闭包，我们可以通过在外部调用闭包函数，从而在外部访问到函数内部的变量，可以使用这种方法来创建私有变量。函数的另一个用途是使已经运行结束的函数上下文中的变量对象继续留在内存中，因为闭包函数保留了这个变量对象的引用，所以这个变量对象不会被回收。其实闭包的本质就是作用域链的一个特殊的应用，只要了解了作用域链的创建过程，就能够理解闭包的实现原理。

```
let a=1// fn
是闭包function fn(){console.log(a);}function fn1(){let a=1//
这里也是闭包return()=>{console.log(a);}}const fn2=fn1()fn2()
```

大家都知道闭包其中一个作用是访问私有变量，就如上述代码中的 fn2 访问到了 fn1 函数中的变量 a。但是此时 fn1 早已销毁，我们是如何访问到变量 a 的呢？不是都说原始类型是存放在栈上的么，为什么此时却没有被销毁掉？接下来笔者会根据浏览器的表现来重新理解关于原始类型存放位置的说法。先来说下数据存放的正确规则是：局部、占用空间确定的数据，一般会存放在栈中，否则就在堆中（也有例外）。

那么接下来我们可以通过 Chrome 来帮助我们验证这个说法。上图中画红框的位置我们能看到一个内部的对象[[Scopes]]，其中存放着变量 a，该对象是被存放在堆上的，其中包含了闭包、全局对象等等内容，因此我们能通过闭包访问到本该销毁的变量。另外最开始我们对

于闭包的定位是：假如一个函数能访问外部的变量，那么这个函数它就是一个闭包，因此接下来我们看看在全局下的表现是怎么样。 `leta=1varb=2// fn` 是闭包 `functionfn(){console.log(a,b);}` 从上图我们能发现全局下声明的变量，如果是 `var` 的话就直接被挂到 `globe` 上，如果是其他关键字声明的话就被挂到 `Script` 上。虽然这些内容同样还是存在 `[[Scopes]]`，但是全局变量应该是存放在静态区域的，因为全局变量无需进行垃圾回收，等需要回收的时候整个应用都没了。只有在下图的场景中，原始类型才可能是被存储在栈上。这里为什么要说可能，是因为 JS 是门动态类型语言，一个变量声明时可以是原始类型，马上又可以赋值为对象类型，然后又回到原始类型。这样频繁的在堆栈上切换存储位置，内部引擎是不是也会有什么优化手段，或者干脆全部都丢堆上？只有 `const` 声明的原始类型才一定存在栈上？当然这只是笔者的一个推测，暂时没有深究，读者可以忽略这段瞎想。因此笔者对于原始类型存储位置的理解为：局部变量才是被存储在栈上，全局变量存在静态区域上，其它都存储在堆上。当然这个理解是建立的 `Chrome` 的表现之上的，在不同的浏览器上因为引擎的不同，可能存储的方式还是有所变化的。闭包产生的原因我们在前面介绍了作用域的概念，那么你还需明白作用域链的基本概念。其实很简单，当访问一个变量时，代码解释器会首先在当前的作用域查找，如果没找到，就去父级作用域去查找，直到找到该变量或者不存在父级作用域中，这样的链路就是作用域链。需要注意的是，每一个子函数都会拷贝上级作用域，形成一个作用域的链条。那么我们还是通过下面的代码来详细说明一下作用域链 `var...`

1) 搜索页的工厂函数已经返回，但它返回的 `getKeyword` 仍能打印函数内的 `keyword`；同事据此断言“父函数根本没有结束”，你怎么纠正？

答：父函数的调用已经结束并从执行栈退出，保留下来的是内部函数对其词法环境的引用。只要 `getKeyword` 仍可达，它需要的 `keyword` 就不会被回收；这不等于整个调用过程仍在执行，也不意味着所有局部数据都必须保留。

2) 你要为支付组件保存一个不能被外部直接改写的重试次数，团队不允许把它挂到全局对象；闭包方案应怎样设计？

答：可在工厂函数内声明计数变量，并只返回读取、递增或重置它的函数，外部通过这些闭包访问状态。变量不暴露到全局，调用结束后仍因闭包引用而保留；接口一旦泄露给长期存活对象，该状态的生命周期也会随之延长。

3) 通知中心用 `var i` 给五个按钮注册延迟回调，点击后全部显示6；如果暂时不能改循环声明，怎样修复且保持输出1到5？

答：用 IIFE 在每轮把当前 `i` 作为参数传入，使每个回调引用不同的局部绑定；也可通过 `setTimeout` 的第三个参数把当轮值传给回调。两者都避开共享的全局 `i`，但前者增加一层函数，后者依赖定时器的传参接口。

4) 单页应用切换路由十次后内存持续增加，旧页面的按钮节点和定时器回调仍能在堆快照中找到；你会怎样验证是否由闭包引用造成？

答：检查未解绑的事件监听器、未清理的定时器及其回调，因为这些闭包可能继续引用页面函数作用域中的变量。沿保留引用链确认旧节点或大对象是否被回调的作用域链持有，再在卸载阶段解除监听和清除定时器；闭包本身并不等于泄漏，关键是引用是否仍有业务必要。

5) 数据面板既可用全局对象保存筛选状态，也可返回一组闭包函数维护私有状态；多人协作下你会选哪一种？

答：若状态只服务单个面板，闭包更适合限制访问范围，并能让状态在创建函数结束后继续存在。全局对象便于直接观察和跨模块共享，却更容易被任意代码覆盖；闭包的代价是生命周期不直观，需要控制返回函数被谁长期持有。

6) 异步请求回调能读取发起请求时函数里的requestId，但回调是在函数返回后才执行；这与作用域链有什么关系？

答：回调函数在定义时就形成了对外层词法环境的引用，因此稍后执行仍能沿作用域链找到requestId。异步机制只改变回调执行时机，不会把标识符改为按调用位置查找；若外层绑定被多个回调共享，读取到的也会是该共享绑定当时的值。

546. New的原理

new本质上会创建对象、连接构造函数原型、绑定this并决定最终返回值。构造函数执行时，新增的属性会写到这个实例上，使实例既能访问自身属性，也能沿原型链访问共享方法。若构造函数显式返回对象，结果会被该对象替代；返回字符串等非对象值时，仍返回原实例。手写时可用Object.create()建立原型关系，再通过apply()执行构造函数并处理返回值。 常见考点

new做了那些事？new返回不同的类型时会有什么表现？手写 new 的实现过程 new 关键词的主要作用就是执行一个构造函数、返回一个实例对象，在 new 的过程中，根据构造函数的情况，来确定是否可以接受参数的传递。下面我们通过一段代码来看一个简单的 new 的例子

```
function Person(){this.name='Jack';}var p=new Person();console.log(p.name)// Jack
```

这段代码比较容易理解，从输出结果可以看出，p 是一个通过 person 这个构造函数生成的一个实例对象，这个应该很容易理解。

new操作符可以帮助我们构建出一个实例，并且绑定上 this，内部执行步骤可大概分为以下几步：创建一个新对象对象连接到构造函数原型上，并绑定this（this 指向新对象）执行构造函数代码（为这个新对象添加属性）返回新对象

在第四步返回新对象这边有一个情况会例外：那么问题来了，如果不用new这个关键词，结合上面的代码改造一下，去掉new，会发生什么样的变化呢？我们再来看下面这段代码

```
function Person(){this.name='Jack';}var p=Person();console.log(p)// undefined
console.log(name)// Jack
console.log(p.name)// 'name' of undefined
```

从上面的代码中可以看到，我们没有使用new这个关键词，返回的结果就是undefined。其中由于JavaScript代码在默认情况下this的指向是window，那么name的输出结果就为Jack，这是一种不存在new关键词的情况。那么当构造函数中有return一个对象的操作，结果又会是什么样子呢？我们再来看一段在上面的基础上改造过的代码。

```
function Person(){this.name='Jack';return{age:18};}var p=new Person();console.log(p)// {age: 18}
console.log(p.name)// undefined
console.log(p.age)// 18
```

通过这段代码又可以看出，当构造函数最后return出来的是一个和this无关的对象时，new 命令会直接返回这个新对象，而不是通过 new 执行步骤生成的 this 对象。但是这里要求构造函数必须是返回一个对象，如果返回的不是对象，那么还是会按照 new 的实现步骤，返回新生成的对象。接下来还是在这段代码的基础之上稍微改动一下

```
function Person(){this.name='Jack';return'tom';}var p=new Person();console.log(p)// {name: 'Jack'}
console.log(p.name)// Jack
```

可以看出，当构造函数中return的不是一个对象时，那么它还是会根据new关键词的执行逻辑，生成一个新的对象（绑定了最新this），最后返回出来。因此我们总结一下：new 关键词执行之后总是会返回一个对象，要么是实例对象，要么是 return 语句指定的对象

手工实现New的过程

```
function create(fn,...args){if(typeof fn !== 'function'){throw 'fn must be a function'}}
1、用new Object() 的方式新建了一个对象obj
var obj = new Object()
2、给该对象的__proto__赋值为fn.prototype，即设置原型链
obj.__proto__ = fn.prototype
1、2步骤合并
创建一个空对象，且这个空对象继承构造函数的 prototype 属性
即实现 obj.__proto__ === constructor.prototype
var obj=Object.create(fn.prototype);
```

3、执行fn，并将obj作为内部this。使用 apply，改变构造函数 this 的指向到新建的对象，这样 obj 就可以访问到构造函数中的属性

```
var res=fn.apply(obj,args);
4、如果fn有返回值，则将其...
```

1) 用户中心执行new Person()后能读到实例属性和Person.prototype上的方法，同事却说new只是调用了一次函数；你会用对象关系怎样反驳？

答：new不只是调用函数，它还创建新对象，并把该对象连接到Person.prototype，再以新对象作为this执行构造函数。构造函数写入的是实例自身属性，原型方法则通过原型链访问；普通调用不会自动完成这些关系。

2) 遗留代码里Person()漏写new，返回值变成undefined，同时页面出现了全局name；你会怎样阻止同类污染？

答：在该非严格示例中，普通调用没有创建实例，构造函数默认返回undefined，而this.name写到了全局对象。工程上应统一构造入口并检查调用方式，或使用必须通过new调用的构造形式；仅依靠调用者记忆仍可能再次遗漏。

3) 缓存模块的构造函数执行了this.key = 'A'，随后return { key: 'B' }；调用方要求实例必须保留原型方法，实际会发生什么？

答：显式返回对象会取代原本创建的实例，因此调用方拿到{ key: 'B' }，也不会自动保留构造函数原型链上的方法。若必须返回标准实例，就不要返回无关对象，或确保返回对象具有所需原型关系；返回原始值则会被忽略。

4) 团队手写的create(Constructor, ...args)生成对象后，实例读不到原型方法；你会按new的哪几个环节定位？

答：先确认是否用Object.create(Constructor.prototype)建立了原型连接，再检查是否以新对象为this调用了构造函数并传入参数。最后核对返回逻辑：构造函数返回对象时采用该对象，否则返回创建的实例；缺少任一环节都会偏离new的行为。

5) 评审手写new时，一版用obj.__proto__ = fn.prototype，另一版用Object.create(fn.prototype)；你会选择哪版并说明边界？

答：应优先使用Object.create(fn.prototype)，它在创建对象时直接建立所需的原型关系，表达也更贴近目标。随后仍需应用apply绑定this并处理构造函数的显式返回值；只完成原型连接并不能等价实现完整的新。

6) 构造函数返回null、字符串或普通对象时，测试负责人要求列出三种调用结果；你的断言应该是什么？

答：返回普通对象时，new的结果被该对象替代；返回字符串等原始值时，结果仍是新创建的实例。null虽然历史上typeof表现特殊，但不是可替代实例的对象返回值，因此也应保留实例；手写实现不宜只靠含糊的类型判断。

547. 原型/原型链

原型链就是对象通过原型逐级连接起来的属性查找链，核心关系是实例.__proto__ === 构造函数.prototype。prototype用来保存实例共享的属性和方法，原型上的constructor通常再指回构造函数。访问属性时会先查对象自身，再沿原型向上查找，直到Object.prototype，仍未找到就得到undefined。工程中获取原型优先使用Object.getPrototypeOf()，不建议直接依赖非标准的__proto__。

__proto__和prototype关系：__proto__和constructor是对象独有的。2prototype属性是函数独有的在js中我们使用构造函数来新建一个对象的，每一个构造函数的内部都有一个prototype属性值，这个属性值是一个对象，这个对象包含了可以由该构造函数的所有实例共享的属性和方法。当我们使用构造函数新建一个对象后，在这个对象的内部将包含一个指针，这个指针指向构造函数的prototype属性对应的值，在ES5

中这个指针被称为对象的原型。一般来说我们是不应该能够获取到这个值的，但是现在浏览器中都实现了

proto 属性来让我们访问这个属性，但是我们最好不要使用这个属性，因为它不是规范中规定的。ES5 中新增了一个Object.getPrototypeOf()方法，我们可以通过这个方法来获取对象的原型。当我们访问一个对象的属性时，如果这个对象内部不存在这个属性，那么它就会去它的原型对象里找这个属性，这个原型对象又会有自己的原型，于是就这样一直找下去，也就是原型链的概念。原型链的尽头一般来说都是Object.prototype所以这就是我们新建的对象为什么能够使用toString()等方法的原因。特点：JavaScript 对象是通过引用来传递的，我们创建的每个新对象实体中并没有一份属于自己的原型副本。当我们修改原型时，与之相关的对象也会继承这一改变 原型(prototype): 一个简单的对象，用于实现对象的 属性继承。可以简单的理解成对象的爹。在Firefox和Chrome中，每个JavaScript对象中都包含一个__proto__(非标准)的属性指向它爹(该对象的原型)，可obj.__proto__进行访问。构造函数: 可以通过new来 新建一个对象的函数。实例: 通过构造函数和new创建出来的对象，便是实例。

实例通过__proto__指向原型，通过constructor指向构造函数。

以Object为例，我们常用的Object便是一个构造函数，因此我们可以通过它构建实例。 // 实例const instance=new Object() 则此时， 实例为instance, 构造函数为Object，我们知道，构造函数拥有一个prototype的属性指向原型，因此原型为: // 原型const prototype=Object.prototype 这里我们可以来看出三者的关系: 实例.__proto__ === 原型原型.constructor === 构造函数构造函数.prototype === 原型 // 这条线其实是基于原型进行获取的，可以理解成一条基于原型的映射线// 例如:// const o = new Object()// o.constructor === Object --> true// o.__proto__ = null// o.constructor === Object --> false实例.constructor===构造函数 原型链 原型链是由原型对象组成，每个对象都有__proto__属性，指向了创建该对象的构造函数的原型，__proto__将对象连接起来组成了原型链。是一个用来实现继承和共享属性的有限的对象链 属性查找机制: 当查找对象的属性时，如果实例对象自身不存在该属性，则沿着原型链往上一级查找，找到时则输出，不存在时，则继续沿着原型链往上一级查找，直至最顶级的原型对象Object.prototype，如还是没找到，则输出undefined；属性修改机制: 只会修改实例对象本身的属性，如果不存在，则进行添加该属性，如果需要修改原型的属性时，则可以用:b.prototype.x = 2；但是这样会造成所有继承于该对象的实例的属性发生改变。 js 获取原型的方法 p.prototypeObject.getPrototypeOf(p) 总结 每个函数都有prototype属性，除了Function.prototype.bind()，该属性指向原型。每个对象都有__proto__属性，指向了创建该对象的构造函数的原型。其实这个属性指向了[[pr...

1) 控制台里user.name能读到值，但Object.keys(user)和接口序列化都没有name，你会怎么解释这个反例？

答：name很可能不在user自身，而是沿原型链从原型对象上找到的。用Object.hasOwn(user, 'name')判断自有属性，再用Object.getPrototypeOf(user)逐层定位；依赖继承属性做序列化会产生遗漏。

2) 组件库想让所有实例共享一份格式化方法，你会把方法放在构造函数里，还是放到prototype上？

答：共享且无实例状态的方法应放到构造函数的prototype上，实例通过原型链复用同一个函数。若在构造函数中逐个创建方法，每个实例都会持有独立函数；但会被实例改写的方法不适合直接共享。

3) 线上热修复把Widget.prototype.render换成了新函数，已创建的几百个组件会立即使用新版吗？

答：只要实例自身没有同名render，后续查找就会命中新原型方法，因此已有实例也会受到影响。若某个实例已定义自身的render，它会遮蔽原型实现；修改共享原型属于全局影响操作，需要评估兼容性。

4) 调试时看到item.constructor !==

Item，同事据此认定对象不是Item实例，你会怎么排查？

答：不能只依赖constructor，因为它通常也是沿原型链取得的普通属性，可能被覆盖或因重设原型而丢失。应同时检查Object.getPrototypeOf(item)、Item.prototype和item instanceof Item；原型链被人为改写时，这些结果也会随之变化。

5) 旧代码大量使用obj.__proto__改继承关系，新模块要继续沿用还是改成标准API？

答：新模块应使用Object.getPrototypeOf读取原型，用Object.create建立所需原型关系，避免继续依赖__proto__。__proto__暴露的是内部[[Prototype]]的访问入口，但直接改链会让属性来源和实例判断更难追踪。

548. 继承

JavaScript继承的核心是连接原型并初始化父类属性，常用结果是寄生组合继承或class extends。单独借用构造函数无法继承原型方法，单独使用原型链又会让多个实例共享引用类型属性。组合继承解决了这两个问题，但父构造函数会执行两次；用Object.create(Parent.prototype)可避免这次多余调用，并修正constructor。class本质上仍基于原型，子类构造函数需要调用super()，静态继承还涉及构造函数本身的原型关系。

涉及面试题：原型如何实现继承？Class如何实现继承？Class本质是什么？

首先先来讲下class，其实在JS中并不存在类，class只是语法糖，本质还是函数

class Person {} Person instanceof Function // true 组合继承 组合继承是最常用的继承方式 function Parent(value) { this.val = value; } Parent.prototype.getValue = function () { console.log(this.val); } function Child(value) { Parent.call(this, value); } Child.prototype = new Parent(); const child = new Child(1); child.getValue() // 1 child instanceof Parent // true 以上继承的方式核心是在子类的构造函数中通过Parent.call(this)继承父类的属性，然后改变子类的原型为new Parent()来继承父类的函数。这种继承方式优点在于构造函数可以传参，不会与父类引用属性共享，可以复用父类的函数，但是也存在一个缺点就是在继承父类函数的时候调用了父类构造函数，导致子类的原型上多了不需要的父类属性，存在内存上的浪费 寄生组合继承 这种继承方式对组合继承进行了优化，组合继承缺点在于继承父类函数时调用了构造函数，我们只需要优化掉这点就行了 function Parent(value) { this.val = value; } Parent.prototype.getValue = function () { console.log(this.val); } function Child(value) { Parent.call(this, value); } Child.prototype = Object.create(Parent.prototype, { constructor: { value: Child, enumerable: false, writable: true, configurable: true } }); const child = new Child(1); child.getValue() // 1 child instanceof Parent // true 以上继承实现的核心就是将父类的原型赋值给了子类，并且将构造函数设置为子类，这样既解决了无用的父类属性问题，还能正确的找到子类的构造函数。 Class 继承 以上两种继承方式都是通过原型去解决的，在 ES6 中，我们可以使用 class 去实现继承，并且实现起来很简单 class Parent { constructor(value) { this.val = value; } getValue() { console.log(this.val); } } class Child extends Parent { constructor(value) { super(value); this.val = value; } } let child = new Child(1); child.getValue() // 1 child instanceof Parent // true class实现继承的核心在于使用extends表明继承自哪个父类，并且在子类构造函数中必须调用super，因为这段代码可以看成Parent.call(this, value)。 ES5 和 ES6 继承的区别： ES6 继承的子类需要调用super()才能拿到子类，ES5的话是通过apply这种绑定的方式类声明不会提升，和let这些一致 function Super() {} Super.prototype.getNumber = function () { return 1; } function Sub() {} Sub.prototype = Object.create(Super.prototype, { constructor: { value: Sub, enumerable: false, writable: true, configurable: true } }); let s = new Sub(); s.getNumber() 以下详细讲解几种常见的继承方式 1. 方式1: 借助call function Parent1() { this.name = 'paren...

1) 有人只在Child里写Parent.call(this)，实例属性都有了，却调用child.getValue()报错，缺的是哪条继承关系？

答：Parent.call(this)只把父构造函数中的实例属性初始化到子实例上，不会让Child.prototype继承Parent.prototype。应再用Object.create(Parent.prototype)建立原型链并修正constructor，否则父类原型方法不可见。

2) 表单模型有数组字段，多个Child实例修改其中一个的选项后全部联动，原型链代码该从哪里查？

答：先检查是否写了Child.prototype = new Parent()，从而把父构造函数创建的数组留在共享原型上。子构造函数中调用Parent.call(this)可让每个实例获得独立引用，再用寄生组合继承复用父类方法。

3) 父构造函数新增了日志注册逻辑后，每创建一个子实例却出现额外初始化记录，组合继承为什么会放大这个副作用？

答：传统组合继承在设置Child.prototype = new Parent()时执行一次父构造函数，创建子实例时又通过Parent.call(this)执行一次。改为Object.create(Parent.prototype)可避免为建原型而调用构造函数，也不会在子类原型上残留无用实例属性。

4) 把Child.prototype直接赋成Parent.prototype后，修复子类constructor却连父类实例的constructor也变了，原因是什么？

答：两者此时引用同一个原型对象，修改Child.prototype.constructor等同于修改Parent.prototype.constructor。应让子类拥有以父原型为原型的新对象，即Child.prototype = Object.create(Parent.prototype)，再单独恢复不可枚举的constructor。

5) 把ES5继承迁成class Child extends

Parent后，父类静态校验方法也能从Child调用，原来的寄生组合代码为什么做不到？

答：传统寄生组合继承通常只连接Child.prototype与Parent.prototype，因此只覆盖实例侧继承。extends还会建立构造函数之间的继承关系，使子类可访问父类静态成员；手写方案需要额外设置Child的原型，但会增加兼容和维护成本。

549. 面向对象

面向对象是用对象组织状态和行为，并通过封装、继承、多态与抽象控制代码关系的一种编程思想。封装可以借助闭包和局部变量隐藏实现，只向外暴露必要方法；继承则可通过原型链与父构造函数调用复用属性和行为。多态强调同一父级下的不同子对象可以呈现不同形态，抽象类还可在构造器或待实现方法中抛错，避免被直接使用。对象拷贝也要注意边界：浅拷贝会共享引用类型，确需隔离时才使用递归深拷贝。

编程思想 基本思想是使用对象，类，继承，封装等基本概念来进行程序设计优点易维护采用面向对象思想设计的结构，可读性高，由于继承的存在，即使改变需求，那么维护也只是在局部模块，所以维护起来是非常方便和较低成本的易扩展开发工作的重用性、继承性高，降低重复工作量。缩短了开发周期 易维护采用面向对象思想设计的结构，可读性高，由于继承的存在，即使改变需求，那么维护也只是在局部模块，所以维护起来是非常方便和较低成本的易扩展开发工作的重用性、继承性高，降低重复工作量。缩短了开发周期 采用面向对象思想设计的结构，可读性高，由于继承的存在，即使改变需求，那么维护也只是在局部模块，所以维护起来是非常方便和较低成本的 一般面向对象包含：继承，封装，多态，抽象 1.

对象形式的继承 浅拷贝 var Person = {name: 'poetry', age: 18, address: {home: 'home', office: 'office'}, schools: ['x', 'z']}; var programmer = {language: 'js'}; function extend(p, c) {var c = c || {}; for (var prop in p) {c[prop] = p[prop];} extend(Person, programmer); programmer.name; // poetry programmer.address.home; // home programmer.address.home = 'house'; // house Person.address.home; // house 从上面的结果看出，浅拷贝的缺陷在于修改了子对象中引用类型的值，会影响到父对象中的值，因为在浅拷贝中对引用类型的拷贝只是拷贝了地址，指向了内存中同一个副本 深拷贝 function extendDeeply(p, c) {var c = c || {}; for (var prop in p) {if (typeof p[prop] === 'object') {c[prop] = (p[prop].constructor === Array) ? [] : {};} extendDeeply(p[prop], c[prop]);} else {c[prop] = p[prop];}}

利用递归进行深拷贝，这样子对象的修改就不会影响到父对象

```
extendDeeply(Person,programer);programer.address.home='poetry';Person.address.home;//
home 利用call和apply继承 functionParent(){this.name="abc";this.address={home:"home"}};function
Child(){Parent.call(this);this.language="js";} ES5中的Object.create()
varp={name:'poetry'};varobj=Object.create(p);obj.name;// poetry
Object.create()作为new操作符的替代方案是ES5之后才出来的。我们也可以自己模拟该方法： //模拟O
bject.create()方法functionmyCreate(o){functionF(){}F.prototype=o;o=newF();returno;}varp={na
me:'poetry'};varobj=myCreate(p);obj.name;// poetry 目前，各大浏览器的最新版本（包括IE9）都部
署了这个方法。如果遇到老式浏览器，可以用下面的代码自行部署
if(!Object.create){Object.create=function(o){functionF(){}F.prototype=o;returnnewF();}} 2.
类的继承 Object.create() functionPerson(name,age){Person.prototype.headCount=1;Person.pro
totype.eat=function(){console.log('eating...');}functionProgrammer(name,age,title){Programmer
.prototype=Object.create(P...
```

1) 商品模板用浅拷贝生成两个编辑对象，修改其中一个的address.home后另一个也变了，为什么顶层字段却正常？

答：浅拷贝只复制顶层值，嵌套对象仍是同一引用，所以修改address.home会反映到所有副本。需要隔离嵌套状态时应做与数据结构匹配的深拷贝；简单递归还要谨慎处理null、特殊对象和循环引用。

2) 插件系统要求外部只能调用start，不能直接改内部令牌，ES5环境下你会怎样封装？

答：可把令牌保存在构造函数或立即执行函数的局部变量中，只通过闭包暴露start等受控方法。这样私有数据不挂在实例上，外部无法直接读写；代价是每个实例可能创建自己的闭包方法，也不便于原型复用。

3) 一个页面要创建上万个轻量模型，当前构造函数为每个实例定义save方法，你会怎样调整对象设计？

答：若save不依赖私有闭包变量，应把它移到构造函数的prototype上，让所有实例共享同一个方法。若它必须访问局部私有状态，则不能直接迁移而不改变封装方式，需要在共享方法与私有性之间取舍。

4) 重构后programmer.work()能执行，但内部调用父类eat时this.name变成了undefined，你会检查哪一行？

答：应检查父类方法是否以Person.prototype.eat.apply(this, arguments)或等价方式调用。若把方法取出后直接执行，调用上下文可能丢失，父类方法便读不到当前子实例属性；显式绑定会增加耦合，但能保持预期接收者。

5) 团队为三种检测器设计继承树，但业务只要求它们都实现detect和stop，你会坚持抽象父类吗？

答：只有共享实现或需要统一约束时，抽象父类才有明显价值，可在基类构造器或未实现的方法中抛错以阻止误用。若只是能力相同而状态与实现无关，深继承会增加耦合，组合独立对象或约定统一接口通常更易扩展。

550. 事件机制

DOM事件触发时会依次经历捕获、目标和冒泡三个阶段。捕获从根节点向目标元素传播，冒泡则从目标元素向外返回，addEventListener的第三个参数决定监听器注册在哪个阶段。事件委托本质上是利用冒泡，把多个子元素同类事件交给父元素处理，适合列表这类节点较多或动态增删的场景。需要中断传播时用stopPropagation()，若还要阻止同一元素上的其他监听器，则用stopImmediatePropagation()；取消默

认行为用preventDefault()。涉及面试题：事件的触发过程是怎么样的？知道什么是事件代理嘛？1. 简介
事件流是一个事件沿着特定数据结构传播的过程。冒泡和捕获是事件流在DOM中两种不同的传播方法
事件流有三个阶段 事件捕获阶段处于目标阶段事件冒泡阶段 事件捕获 事件捕获（event capturing）：通俗的理解就是，当鼠标点击或者触发dom事件时，浏览器会从根节点开始由外到内进行事件传播，即点击了子元素，如果父元素通过事件捕获方式注册了对应的事件的话，会先触发父元素绑定的事件 事件冒泡
事件冒泡（dubbed

bubbling）：与事件捕获恰恰相反，事件冒泡顺序是由内到外进行事件传播，直到根节点

无论是事件捕获还是事件冒泡，它们都有一个共同的行为，就是事件传播 2. 捕获和冒泡 letdiv1=document.getElementById('div1');letdiv2=document.getElementById('div2');div1.onClick=function(){alert('1')}div2.onClick=function(){alert('2');} 当点击div2时，会弹出两个弹出框。在ie8/9/10、chrome浏览器，会先弹出“2”再弹出“1”，这就是事件冒泡：事件从最底层的节点向上冒泡传播。事件捕获则跟事件冒泡相反 W3C的标准是先捕获再冒泡，addEventListener的第三个参数决定把事件注册在捕获（true）还是冒泡(false) 3. 事件对象 4. 事件流阻止 在一些情况下需要阻止事件流的传播，阻止默认动作的发生 event.preventDefault()：取消事件对象的默认动作以及继续传播。event.stopPropagation()/

event.cancelBubble = true：阻止事件冒泡。 事件的阻止在不同浏览器有不同处理 在IE下使用event.returnValue= false，在非IE下则使用event.preventDefault()进行阻止

preventDefault与stopPropagation的区别 preventDefault告诉浏览器不用执行与事件相关联的默认动作（如表单提交）stopPropagation是停止事件继续冒泡，但是对IE9以下的浏览器无效 5. 事件注册 通常我们使用addEventListener注册事件，该函数的第三个参数可以是布尔值，也可以是对象。对于布尔值useCapture参数来说，该参数默认值为false。useCapture决定了注册的事件是捕获事件还是冒泡事件一般来说，我们只希望事件只触发在目标上，这时候可以使用stopPropagation来阻止事件的进一步传播。通常我们认为stopPropagation是用来阻止事件冒泡的，其实该函数也可以阻止捕获事件。stopImmediatePropagation（DOM3级新增事件）

同样也能实现阻止事件冒泡和捕获，但是还能阻止该事件目标执行别的注册事件

stopImmediatePropagation()和stopPropagation()的区别在哪儿呢？后者只会阻止冒泡或者是捕获。

但是前者除此之外还会阻止该元素的其他事件发生，但是后者就不会阻止其他事件的发生。 node.addEventListener('click',(event)=>{event.stopImmediatePropagation()console.log('冒泡')},false);node.addEventListener('click',(event)=>{console.log('冒泡2')},false) 6. 事件委托

在js中性能优化的其中一个主要思想是减少dom操作。节省内存不需要给子节点注销事件 假设有100个li，每个li有相同的点击事件。如果为每个li都添加事件，则会造成dom访问次数过多，引起浏览器重绘与重排的次数过多，性能则会降低。使用事件委托则可以解决这样的问题 原理 实现事件委托是利用了事件的冒泡原理实现的。当我们为最外层的节点添加点击事件，那么里面的ul、li、a的点击事件都会冒泡到最外层节点上，委托...

1) 弹窗里的链接点击后既跳转页面又触发外层关闭逻辑，同事只调用了preventDefault()，为什么弹窗仍会关闭？

答：preventDefault()取消的是链接跳转等默认动作，不会停止事件沿DOM传播，因此外层监听仍会执行。若确实不应触发外层逻辑，需要调整外层判断或调用stopPropagation()；随意停止传播会破坏依赖冒泡的其他功能。

2) 商品列表有数百个动态增删的条目，每项都有同样的收藏按钮，你会把监听器挂在哪里？

答：可在稳定的列表容器上注册一个冒泡阶段监听器，根据事件目标定位对应条目并执行收藏逻辑。事件委托减少逐项绑定和注销，也能覆盖后来插入的节点；不冒泡的事件或传播被中途阻止时不能直接套用。

3) 埋点要求在子组件调用stopPropagation()后仍尽量观察点击，你会选择捕获还是冒泡阶段注册？

答：可在更外层以addEventListener的捕获选项注册，因为事件会先从根向目标传播，再进入目标和冒泡阶段。若事件在更早的捕获节点已被停止，监听仍可能收不到；捕获监听也不能替代对业务事件边界的明确设计。

4) 同一个按钮注册了两个click监听，第一个执行了stopPropagation()，第二个仍然打印日志，这是浏览器异常吗？

答：不是，stopPropagation()只阻止事件继续传播到其他节点，不会取消当前节点上其余监听器。若必须阻止同一目标的后续监听，应使用stopImmediatePropagation()；这会让监听器之间产生强顺序依赖，排查成本较高。

5) 线上委托监听偶尔拿到内部图标节点，导致用nodeName === 'li'的分支失效，你会怎样修复并验证传播链？

答：点击嵌套元素时，事件目标是实际命中的最深节点，不保证就是承载业务数据的li。应从目标向上寻找符合条件且仍位于委托容器内的节点，并检查沿途是否调用了传播阻止方法；选择器过宽还可能误匹配容器外结构。

551. 模块化

现在谈JavaScript模块化，核心就是比较ESM和CommonJS，AMD、CMD作为历史背景了解即可。ESM的依赖关系可在编译期静态分析，因此构建工具能做Tree Shaking，导入值也是实时绑定；CommonJS则通过require在运行时加载，拿到的是module.exports导出的值。加载方式上，CommonJS更适合服务端的同步读取，ESM更适合浏览器和现代构建工具。维护老项目时可能遇到require.js或sea.js，但新项目一般优先使用import、export。

版本说明（面试主线）：下面列的四种方案里，现在的主线只有两个——浏览器和构建工具用ES Module（ESM，import/export），Node

端历史上用CommonJS（require/module.exports）、现也全面支持ESM。AMD（require.js）和CMD（sea.js）是ES Module出现之前、为解决浏览器异步加载模块而生的社区方案，如今已基本退出历史舞台，了解它们主要是为理解「模块化的演进」和偶尔维护老项目。面试回答模块化，应以「ESM vs CommonJS 的区别」为主线（ESM 编译期静态分析、可 Tree Shaking、实时绑定、异步；CJS 运行时加载、值拷贝、同步），把AMD/CMD作为历史背景一笔带过即可。

js中现在比较成熟的有四种模块加载方案：第一种是CommonJS方案，它通过require来引入模块，通过module.exports定义模块的输出接口。这种模块加载方案是服务器端的解决方案，它是以同步的方式来引入模块的，因为在服务端文件都存储在本地磁盘，所以读取非常快，所以以同步的方式加载没有问题。但如果是在浏览器端，由于模块的加载是使用网络请求，因此使用异步加载的方式更加合适。第二种是AMD方案，这种方案采用异步加载的方式来加载模块，模块的加载不影响后面语句的执行，所有依赖这个模块的语句都定义在一个回调函数里，等到加载完成后再执行回调函数。require.js实现了AMD规范第三种是CMD方案，这种方案和AMD方案都是为了解决异步模块加载的问题，sea.js实现了CMD规范。它和require.js的区别在于模块定义时对依赖的处理不同和对依赖模块的执行时机的处理不同。第四种方案是ES6提出的方案，使用import和export的形式来导入导出模块

在有Babel的情况下，我们可以直接使用ES6的模块化 // file
a.jsexportfunctiona(){exportfunctionb(){}}// file
b.jsexportdefaultfunction(){import{a,b}from'./a.js'importXXXfrom'./b.js'} CommonJS
CommonJs是Node独有的规范，浏览器中使用就需要用到Browserify解析了。 //

a.jsmodule.exports={a:1}// o.exports.a=1// b.jsvarmodule=require('./a.js')module.a// -> log 1

在上述代码中，`module.exports`和`exports`很容易混淆，让我们来看看大致内部实现

```
var module = require('./a.js'); module.a //
```

这里其实就是包装了一层立即执行函数，这样就不会污染全局变量了，// 重要的是 `module`

这里，`module` 是 Node 独有的一个变量 `module.exports={a:1}`// 基本实现 `var module={exports:{}}`//

`exports` 就是个空对象// 这个是什么 `exports` 和 `module.exports`

用法相似的原因 `var exports=module.exports` `var load=function(module){//`

导出的东西 `var a=1` `module.exports=a` `return module.exports`}; 再来说说 `module.exports`和`exports`，用

法其实是相似的，但是不能对`exports`直接赋值，不会有任何效果。

对于CommonJS和ES6中的模块化的两者区别是：前者支持动态导入，也就是`require(`${path}/xx.js`)`，后

者目前不支持，但是已有提案。前者是同步导入，因为用于服务端，文件都在本地，同步导入即使卡住主线

程影响也不大。而后者是异步导入，因为用于浏览器，需要下载文件，如果也采用同步导入会对渲染有很

大影响。前者在导出时都是值拷贝，就算导出的值变了，导入的值也不会改变，所以如果想更新值，必须重

新导入一次。但是后者...

1) 计数模块导出`count`后持续自增，`require`方保存的值不再变化，而`import`方观察到更新，你怎么解释？

答：CommonJS导出通常表现为运行时取得导出值，解构或保存后不会自动形成ESM的实时绑定。ES Module的导入绑定指向导出接口，源模块更新时导入侧可观察到变化；业务代码仍不应尝试给导入绑定重新赋值。

2) 浏览器首屏直接使用同步`require`风格加载几十个远程模块，架构评审时你会否决吗？

答：浏览器模块需要经过网络获取，同步加载会阻塞后续执行，不适合作为现代页面的主线方案。应采用原生或构建工具支持的ES Module；AMD、CMD主要用于理解旧项目中的异步模块演进。

3) 插件入口路径要由租户配置在运行时决定，同事坚持所有依赖都写成静态`import`，冲突点在哪里？

答：静态`import`的接口和依赖可在解析阶段确定，利于构建工具分析，但不能直接用任意运行时字符串替代模块说明符。确需按条件加载时应采用环境支持的动态导入机制；路径范围过于开放会削弱构建可分析性并增加加载失败风险。

4) 一个CommonJS文件先写`exports.a = 1`，随后又写`exports = { b: 2 }`，调用方为什么只能拿到`a`？

答：初始时`exports`只是`module.exports`的引用，给`exports.a`赋值会修改真正的导出对象。把`exports`重新赋成新对象只改变局部变量指向，不会替换`module.exports`；要整体导出必须直接给`module.exports`赋值。

5) 老后台同时存在`require.js`、`sea.js`和新写的ESM，迁移时要不要把三者当成同等长期方案维护？

答：不应同等投入，现代浏览器与构建工具的主线应收敛到ES Module，Node侧则根据现有包体系处理CommonJS与ESM。AMD强调依赖前置，CMD强调就近依赖和不同执行时机，保留它们通常只为兼容尚未迁移的旧模块。

552. Iterator迭代器

一个对象能否被`for...of`遍历，关键看它是否实现了`[Symbol.iterator]`。这个方法需要返回迭代器对象，而迭代器通过`next()`逐项返回`{ value, done }`，两者分别负责提供遍历入口和推进遍历过程。数组、类数组对象、`Set`、`Map`已经部署了该接口，普通对象默认没有，所以不能直接使用`for...of`。如果确实需要遍历对象，可以手动规定属性顺序并实现接口，也可以用`Generator`简化实现，但顺序规则要由开发者明确。Iterat

or（迭代器）是一种接口，也可以说是一种规范。为各种不同的数据结构提供统一的访问机制。任何数据结构只要部署Iterator接口，就可以完成遍历操作（即依次处理该数据结构的所有成员）。 Iterator语法：
const obj = {[Symbol.iterator]: function() {}}

[Symbol.iterator]属性名是固定的写法，只要拥有了该属性的对象，就能够用迭代器的方式进行遍历。

迭代器的遍历方法是首先获得一个迭代器的指针，初始时该指针指向第一条数据之前，接着通过调用 next 方法，改变指针的指向，让其指向下一条数据。每一次的next都会返回一个对象，该对象有两个属性value代表想要获取的数据done布尔值，false表示当前指针指向的数据有值，true表示遍历已经结束

value代表想要获取的数据done布尔值，false表示当前指针指向的数据有值，true表示遍历已经结束

Iterator 的作用有三个： 创建一个指针对象，指向当前数据结构的起始位置。也就是说，遍历器对象本质上，就是一个指针对象。第一次调用指针对象的next方法，可以将指针指向数据结构的第一个成员。第二次调用指针对象的next方法，指针就指向数据结构的第二个成员。不断调用指针对象的next方法，直到它指向数据结构的结束位置。 每一次调用next方法，都会返回数据结构的当前成员的信息。具体来说，就是返回一个包含value和done两个属性的对象。其中，value属性是当前成员的值，done属性是一个布尔值，表示遍历是否结束。

```
let arr = [{ num: 1 }, 2, 3]; let it = arr[Symbol.iterator]()//  
获取数组中的迭代器 console.log(it.next())// { value: Object { num: 1 }, done: false }  
console.log(it.next())// { value: 2, done: false } console.log(it.next())// { value: 3, done: false }  
console.log(it.next())// { value: undefined, done: true } 对象没有布局Iterator接口，无法使用for of遍历。  
下面使得对象具备Iterator接口 一个数据结构只要有Symbol.iterator属性，就可以认为是“可遍历的”  
原型部署了Iterator接口的数据结构有三种，具体包含四种，分别是数组，类似数组的对象，Set和Map结构  
为什么对象（Object）没有部署Iterator接口呢？ 一是因为对象的哪个属性先遍历，哪个属性后遍历是不确定的，需要开发者手动指定。然而遍历遍历器是一种线性处理，对于非线性的数据结构，部署遍历器接口，就等于要部署一种线性转换对对象部署Iterator接口并不是很必要，因为Map弥补了它的缺陷，又正好有Iteraotr接口  
let obj = {id: '123', name: '张三', age: 18, gender: '男', hobby: '睡觉'}; obj[Symbol.iterator] = function() {  
let keyArr = Object.keys(obj); let index = 0; return {next() { return index < keyArr.length ? {value: {key: keyArr[index], val: obj[keyArr[index++]}} : {done: true}} }; }  
for (let key of obj) { console.log(key) }
```

1) 订单接口返回{0:'A',1:'B',length:2}，列表页执行[...data]却报is not iterable，明明它能按下标读取，问题出在哪？

答：能按下标读取只说明它是类数组，不代表实现了可迭代协议；展开语法会查找data[Symbol.iterator]，缺失时就会抛错。若只需转成数组可用Array.from(data)，若要支持统一遍历，则应显式部署迭代器。

2) 组件库有个树形结果对象，团队希望它既能保留原结构，又能被for...of按业务指定顺序遍历，你会怎样设计[Symbol.iterator]？

答：应让[Symbol.iterator]()返回独立的迭代器对象，并由闭包保存当前位置；每次next()返回{value, done}，顺序由业务规则明确决定。不要直接依赖普通对象属性的自然排列，否则遍历语义会隐含在数据结构中，后续调整字段容易改变结果。

3) 同一个分页集合需要同时启动两次for...of，如果[Symbol.iterator]()总是返回缓存的同一个迭代器，会出现什么现象？

答：两次遍历会共享指针，一个循环调用next()后会推进另一个循环的位置，最终产生跳项、错序或提前结束。每次调用[Symbol.iterator]()都应创建新的状态和指针；只有明确设计为一次性消费的数据流，才适合共享迭代状态。

4) 线上导出任务偶尔多出一条undefined，自定义迭代器结束分支返回了{value: undefined, done: false}，你会怎么定位？

答：应直接检查连续调用next()的返回序列，因为done:

false表示当前值仍是合法成员，即使value为undefined也会被消费。结束分支必须返回{done: true}，通常同时令value为undefined；还要核对索引递增是否发生在取值之前。

5) 接口字段对象要依次输出键和值，同事主张给所有普通对象的原型统一加[Symbol.iterator]，你会接受吗？

答：不建议修改所有普通对象的原型，因为对象需要先约定按键、按值还是按键值对遍历，统一实现会强加一种线性语义。更稳妥的是在具体数据结构上实现迭代器，或先用Object.keys明确顺序；通用键值集合也可考虑使用原生可迭代的Map。

553. Promise

Promise是保存异步结果的状态容器，状态只能从pending变为fulfilled或rejected，一旦改变就不可逆。每次调用then()都会返回新的Promise，因此可以把异步步骤串起来，避免多层回调嵌套；相关回调会作为微任务参与事件循环。并发任务全部成功才继续时用Promise.all()，需要保留每项结果时用Promise.allSettled()。只取首个成功结果可用Promise.any()，而超时竞争这类场景更适合Promise.race()，因为它采用最先改变的状态。这里你谈promise的时候，除了将他解决的痛点以及常用的API之外，最好进行拓展把eventloop带进来好好讲一下，microtask(微任务)、macrotask(任务)

的执行顺序，如果看过promise源码，最好可以谈一谈 原生Promise是如何实现的。Promise的关键点在于callback的两个参数，一个是resolve，一个是reject。还有就是Promise的链式调用（Promise.then()，每一个then都是一个责任人） Promise是ES6新增的语法，解决了回调地狱的问题。可以把Promise看成一个状态机。初始是pending状态，可以通过函数resolve和reject，将状态转变为resolved或者rejected状态，状态一旦改变就不能再次变化。then函数会返回一个Promise实例，并且该返回值是一个新的实例而不是之前的实例。因为Promise规范规定除了pending状态，其他状态是不可以改变的，如果返回的是一个相同实例的话，多个then调用就失去意义了。对于then来说，本质上可以把它看成是flatMap 1. Promise的基本情况 简单来说它就是一个容器，里面保存着某个未来才会结束的事件（通常是异步操作）的结果。从语法上说，Promise 是一个对象，从它可以获取异步操作的消息 一般 Promise 在执行过程中，必然会处于以下几种状态之一。 待定（pending）：初始状态，既没有被完成，也没有被拒绝。已完成（fulfilled）：操作成功完成。已拒绝（rejected）：操作失败。 待定状态的Promise对象执行的话，最后要么会通过一个值完成，要么会通过一个原因被拒绝。当其中一种情况发生时，我们用Promise的then方法排列起来的相关处理程序就会被调用。因为最后Promise.prototype.then和Promise.prototype.catch方法返回的是一个Promise，所以它们可以继续被链式调用 关于 Promise 的状态流转情况，有一点值得注意的是，内部状态改变之后不可逆，你需要在编程过程中加以注意。文字描述比较晦涩，我们直接通过一张图就能很清晰地看出 Promise 内部状态流转的情况

从上图可以看出，我们最开始创建一个新的Promise返回给p1，然后开始执行，状态是pending，当执行resolve之后状态就切换为fulfilled，执行reject之后就变为rejected的状态 2. Promise的静态方法 all 方法语法：Promise.all (iterable) 参数： 一个可迭代对象，如Array。描述：此方法对于汇总多个promise的结果很有用，在 ES6 中可以将多个Promise.all异步请求并行操作，返回结果一般有下面两种情况。当所有结果成功返回时按照请求顺序返回成功结果。当其中有一个失败方法时，则进入失败方法我们来看下业务的场景，对于下面这个业务场景页面的加载，将多个请求合并到一起，用 all 来实现可能效果会更好，请看代码片段 语法：Promise.all (iterable) 参数：一个可迭代对象，如Array。描述： 此方法对于汇总多个promise的结果很有用，在 ES6 中可以将多个Promise.all异步请求并行操作，返回结果一般有下面两种情况。当所有结果成功返回时按照请求顺序返回成功结果。当其中有一个失败方法时，则进入失败方法

当所有结果成功返回时按照请求顺序返回成功结果。当其中有一个失败方法时，则进入失败方法 // 在一个页面中需要加载获取轮播列表、获取店铺列表、获取分类列表这三个操作，页面需要同时发出请求进行页

面渲染，这样用`Promise.all`来实现，看起来更清晰、一目了然。//1.获取轮播数据列表function getBannerList(){return new Promise((resolve, reject) => {setTimeout(function(){resolve('轮播数据')}, 300)})} //2. 获取店铺列表function getStoreList(){re...

1) 支付页创建一个Promise后先调用resolve('ok')，异常分支又调用reject(error)，监控却始终看到成功，这是实现缺陷还是预期行为？

答：这是预期行为，Promise只能从pending转为fulfilled或rejected，第一次状态变化后便不可逆。排查时应找出哪个异步分支最先落定，并避免多个回调竞争；后续调用不会覆盖既有结果，也不能用来撤销已完成状态。

2) 首页要并行加载轮播、店铺和分类数据，三块全部到齐才允许渲染，团队在串行await和Promise.all之间争论，你会选哪个？

答：三个请求彼此无依赖且必须全部成功时，应使用Promise.all并行汇总，结果会按传入顺序返回，而不是按完成顺序排列。任意一个请求拒绝都会使汇总结果进入拒绝分支，因此还需在页面层明确整体失败时的降级或重试策略。

3) 批量上传 50 个附件时，产品要求成功项保留、失败项单独展示原因，原来的Promise.all一遇失败就进入catch，该怎么改？

答：应改用Promise.allSettled，等待全部任务落定后读取每项的status，成功项取value，失败项取reason。它解决的是结果汇总不短路，并不负责重试、取消或并发控制；这些策略仍需由上传层单独实现。

4) 图片详情页用Promise.race([loadImage(), timeout()])做五秒超时，界面已经提示失败，稍后图片的onload却仍然触发，你如何解释？

答：Promise.race只让最先改变状态的任务决定外层结果，不会停止其余异步操作，因此图片加载仍可能继续并触发原回调。排查时要区分“外层已拒绝”和“底层任务已取消”；若业务要求真正终止，还需要底层操作自身提供取消能力。

5) 同事实现了一个简化版Promise，只在调用then时检查当前状态；异步resolve后回调始终不执行，缺了哪部分机制？

答：调用then时若状态仍为pending，实现必须保存成功和失败处理函数，待resolve或reject改变状态后再触发。仅处理已完成状态只能覆盖同步落定场景，也无法支撑可靠链式调用；完整实现还要让每次then返回新的Promise。

6) 搜索页同时请求三个镜像源，只需要第一个成功结果；有人选择race，有人选择any，哪个更符合需求？

答：应选择Promise.any，因为它忽略先到达的拒绝，直到某个任务成功才完成；只有全部任务都拒绝时，外层才拒绝。Promise.race采用最先落定的结果，最快返回的若是失败，就会提前结束，不符合“首个成功”的约束。

554. Generator

Generator是一种可以暂停并恢复执行的函数，本质上也是Iterator接口的一种实现。调用生成器函数不会立即跑完代码，而是返回迭代器；每次调用next()，代码会执行到下一个yield并交出执行权。再次调用next(value)时，传入值会成为上一个yield表达式的结果，但第一次传参会被忽略。它适合描述分段执行流程，也是async/await演化所依赖的思路之一。Generator是ES6中新增的语法，和Promise一样，都可以用来异步编程。Generator函数可以说是Iterator接口的具体实现方式。Generator

最大的特点就是可以控制函数的执行。function*用来声明一个函数是生成器函数，它比普通的函数声明多了一个*,*的位置比较随意可以挨着function关键字，也可以挨着函数名yield产出的意思，这个关键字只能

出现在生成器函数体内，但是生成器中也可以没有yield关键字，函数遇到yield的时候会暂停，并把yield后面的表达式结果抛出去next作用是将代码的控制权交还给生成器函数

```
function*foo(x){lety=2*(yield(x+1))letz=yield(y/3)return(x+y+z)}letit=foo(5)console.log(it.next())//=> {value: 6, done: false}console.log(it.next(12))// => {value: 8, done: false}console.log(it.next(13))// => {value: 42, done: true}
```

上面这个示例就是一个Generator函数，我们来分析其执行过程：首先 Generator函数调用时它会返回一个迭代器当执行第一次 next 时，传参会被忽略，并且函数暂停在 yield (x + 1) 处，所以返回 5 + 1 = 6当执行第二次 next 时，传入的参数等于上一个 yield 的返回值，如果你不传参，yield 永远返回 undefined。此时 let y = 2 * 12，所以第二个 yield 等于 2 * 12 / 3 = 8当执行第三次 next 时，传入的参数会传递给 z，所以 z = 13, x = 5, y = 24，相加等于 42 yield实际就是暂缓执行的标示，每执行一次next()，相当于指针移动到下一个yield位置 总结一下，Generator函数是ES6提供的一种异步编程解决方案。通过yield标识位和next()方法调用，实现函数的分段执行 遍历器对象生成函数，最大的特点是可以交出函数的执行权 function关键字与函数名之间有一个星号；函数体内部使用yield表达式，定义不同的内部状态；next指针移向下一个状态 这里你可以说说Generator的异步编程，以及它的语法糖async和await，传统的异步编程。ES6之前，异步编程大致如下

回调函数事件监听发布/订阅 传统异步编程方案之一：协程，多个线程互相协作，完成异步任务。 // 使用 * 表示这是一个 Generator 函数// 内部可以通过 yield 暂停代码// 通过调用 next 恢复执行function*test(){leta=1+2;yield2;yield3;}letb=test();console.log(b.next());// > { value: 2, done: false }console.log(b.next());// > { value: 3, done: false }console.log(b.next());// > { value: undefined, done: true } 从以上代码可以发现，加上*的函数执行后拥有了next函数，也就是说函数执行后返回了一个对象。每次调用next函数可以继续执行被暂停的代码。以下是Generator函数的简单实现 // cb 也就是编译过的 test 函数functiongenerator(cb){return(function(){varobject={next:0,stop:function(){};return{next:function(){varret=cb(object);if(ret===undefined)return{value:undefined,done:true};return{value:ret,done:false};}}})();// 如果你使用 babel 编译后可以发现 test 函数变成了这样functiontest(){vara;returngenerator(function...

1) 报表代码调用buildRows()后没有生成任何数据，直到执行iterator.next()才出现第一行；同事认为函数调用已经执行了函数体，错在哪里？

答：调用Generator函数只会返回迭代器，不会像普通函数那样立即完整执行函数体；第一次next()才会运行到首个yield并暂停。若调用方忘记驱动迭代器，内部逻辑就不会推进，这也是把控制权交给外部调度者的代价。

2) 面试现场这段代码const it=foo(5); it.next(); it.next(12); it.next(13)最终返回42，第二次传入的12到底进入了哪里？

答：第二次next(12)的参数会成为上一个yield表达式的结果，因此y被计算为2 * 12，随后函数在下一个yield再次暂停。第一次next()的参数会被忽略，因为当时还没有暂停中的yield可以接收它。

3) 十万条记录需要分批格式化，主线程不能一次跑完整个循环，你会怎样利用Generator让调度器分段推进？

答：可以把处理过程写成Generator，每完成一批记录便yield当前进度，外部调度器在合适时机调用next()继续。这样执行权会在批次之间交还给调用方，但Generator本身不会自动安排时间片，批量大小和再次调度仍需额外设计。

4) 异步流水线第二步依赖第一步的请求结果，但调用方只连续执行next()没有传值，后续计算得到NaN，你会查什么？

答：应检查调用方是否把上一步完成值传入下一次next(result)，因为暂停点左侧拿到的是下一次next的参数；省略时该yield表达式结果就是undefined。同时确认每次返回的done，避免在生成器已经结束仍继续传值。

5) 团队想用Generator替代async/await编排三个串行请求，理由是它能暂停函数；这两种方案的关键取舍是什么？

答：Generator只提供通过yield暂停、由next()恢复的控制机制，异步结果何时完成以及如何继续都要由执行器负责。async/await相当于Generator配合自动执行器，语义更直接且返回Promise；只需要显式控制推进过程时，手动生成器才更有价值。

555. async/await

async/await是Generator加自动执行器思路的语法糖，让基于Promise的异步流程写起来更像同步代码。async函数会返回Promise，执行到await时会暂时交还控制权，等结果完成后再恢复后续逻辑。它比连续的then调用更直观，也更容易组织错误处理。多个任务互不依赖时，我一般不会逐个await，因为这样会失去并发性，更适合用Promise.all()。

Generator函数的语法糖。有更好的语义、更好的适用性、返回值是Promise。await和promise一样，更多的是考笔试题，当然偶尔也会问到和promise的一些区别。await相比直接使用Promise来说，优势在于处理then的调用链，能够更清晰准确的写出代码。缺点在于滥用await可能会导致性能问题，因为await会阻塞代码，也许之后的异步代码并不依赖于前者，但仍然需要等待前者完成，导致代码失去了并发性，此时更应该使用Promise.all。一个函数如果加上async

，那么该函数就会返回一个Promise async => *await => yield // 基本用法asyncfunctiontimeout(ms){awaitnewPromise((resolve)=>{setTimeout(resolve,ms)})}asyncfunctionasyncConsole(value,ms){awaittimeout(ms)console.log(value)}asyncConsole('hello async and await',1000)
下面来看一个使用await的代码。 var a=0 var b=async()=>{a=a+await 10 console.log('2',a)} // -> '2'
10 a=(await 10)+a console.log('3',a) // -> '3' 20 b() a++ console.log('1',a) // -> '1' 1
首先函数b先执行，在执行到await

10之前变量a还是0，因为在await内部实现了generators，generators会保留堆栈中东西，所以这时候a = 0被保存了下来因为await是异步操作，遇到await就会立即返回一个pending状态的Promise对象，暂时返回执行代码的控制权，使得函数外的代码得以继续执行，所以会先执行console.log('1',

a)这时候同步代码执行完毕，开始执行异步代码，将保存下来的值拿出来使用，这时候a = 10然后后面就是常规执行代码了 优缺点： async/await的优势在于处理 then

的调用链，能够更清晰准确的写出代码，并且也能优雅地解决回调地狱问题。当然也存在一些缺点，因为await将异步代码改造成了同步代码，如果多个异步代码没有依赖性却使用了await

会导致性能上的降低。 async原理 async/await语法糖就是使用Generator函数+自动执行器来运作的 // 定义了一个promise，用来模拟异步请求，作用是传入参数++functiongetNum(num){returnnewPromise((resolve,reject)=>{setTimeout(()=>{resolve(num+1)},1000)})} // 自动执行器，如果一个Generator函数没有执行完，则递归调用functionasyncFun(func){var gen=func();functionnext(data){var result=gen.next(data);if(result.done)returnresult.value;result.value.then(function(data){next(data)});}next();} // 所需要执行的Generator函数，内部的数据在执行完成一步的promise之后，再调用下一步var func=function*(){var f1=yield getNum(1);var f2=yield getNum(f1);console.log(f2);} asyncFun(func); 在运行的过程中，判断一个函数的promise是否完成，如果已经完成，将结果传入下一个函数，继续重复此步骤每一个next()方法返回值的value属性为一个Promise对象，所以我们为其添加then方法，在then方法里面接着运行next方法挪移遍历器指针，直到Generator函数运行完成

1) 用户资料页调用loadUser()后直接打印返回值，看到的是Promise而不是用户对象；函数里明明写了return user，为什么？

答：只要函数声明为async，调用结果就会被包装成Promise，函数体中的return user对应其完成值。调用方需要使用await loadUser()或注册then获取对象；把返回值当普通对象读取，会在异步结果落定前访问错误的形态。

2) 首屏循环请求 20 个互不依赖的商品详情，每个都写成await fetchOne(id)，页面等待时间明显累加，你会怎样改？

答：这些请求没有前后依赖，应先用ids.map(fetchOne)创建任务，再通过await Promise.all(...)汇总，使它们保持并发。若任一请求失败，Promise.all会进入拒绝路径；页面需要部分成功时，应改用能保留每项状态的汇总策略。

3) 订单创建必须先拿用户信息，再用用户编号查询优惠券；评审者仍要求全部放进Promise.all，你会同意吗？

答：不会，因为优惠券请求的参数依赖用户请求结果，两个操作无法在输入尚未产生时真正并行。这里顺序await更能表达依赖关系；除非接口或数据结构被重新设计，否则强行使用Promise.all只会制造无效请求或复杂的占位逻辑。

4) 线上保存按钮点击后只执行到第一个await，后续日志和成功提示都没出现，控制台显示拒绝未处理，你会从哪里排查？

答：先检查被等待的Promise是否进入rejected，因为拒绝会使当前async函数中断，并让它返回的Promise同样拒绝。应在能决定补救策略的位置使用try/catch，或由调用方接住拒绝；只加成功日志无法覆盖异常路径。

5) 同事写了一个基于Generator的asyncFun，请求成功时能串行执行，但任一步拒绝后流程悬空；自动执行器还缺什么？

答：执行器不能只对result.value注册成功回调，还要处理拒绝并把错误交回生成器或让外层Promise拒绝。每次推进也应检查done，完成时解析最终返回值；否则它只是演示版递归驱动器，达不到async/await的错误传播语义。

6) 搜索页有两个独立请求，代码为了写起来像同步流程而连续使用两个await；可读性和并发性冲突时你怎么选？

答：若后一个请求不依赖前一个，应先同时启动两个任务，再用一次Promise.all等待结果，既保留清晰的赋值结构，也避免无意义串行。await的价值是表达异步步骤，不意味着每一步都必须顺序启动；错误处理仍要覆盖汇总后的拒绝。

556. 事件循环

事件循环的核心是执行一个宏任务后清空全部微任务，再进入下一轮宏任务。同步脚本本身属于宏任务，执行期间遇到Promise.then等会登记微任务，遇到setTimeout、交互或I/O回调则进入相应的宏任务队列。当前执行栈清空后，微任务会按登记顺序持续执行，新产生的微任务也要在这一轮清空，之后浏览器才可能渲染并取下一个宏任务。Node.js还会细分事件循环阶段，并且process.nextTick的优先级高于Promise微任务。默认代码从上到下执行，执行环境通过script来执行（宏任务）在代码执行过程中，调用定时器promiseclick事件...不会立即执行，需要等待当前代码全部执行完毕给异步方法划分队列，分别存放到微任务（立即存放）和宏任务（时间到了或事情发生了才存放）到队列中script执行完毕后，会清空所有的微任务微任务执行完毕后，会渲染页面（不是每次都调用）再去宏任务队列中看有没有到达时间的，拿出来其中一个执行执行完毕后，按照上述步骤不停的循环 例子 自动执行的情况 会输出 listener1 listener2 task1 task2 如果手动点击click

会一个宏任务取出来一个个执行，先执行click的宏任务，取出微任务去执行。会输出 listener1 task1 listener2 task2 console.log(1) async function asyncFunc() { console.log(2) // await xx ==> promise.resolve(()=>{ console.log(3) }).then(() // console.log(3) 放到 promise.resolve 或立即执行 await console.log(3) // 相当于把 console.log(4) 放到了 then promise.resolve(()=>{ console.log(3) }).then(()=>{ // console.log(4) // }) // 微任务谁先注册谁先执行 console.log(4) } setTimeout(()=>{ console.log(5) }) const promise = new Promise((resolve, reject) => { console.log(6) resolve(7) }) promise.then(d=>{ console.log(d) }) asyncFunc() console.log(8) // 输出 1 6 2 3 8 7 4 5

1. 浏览器事件循环 涉及面试题：异步代码执行顺序？解释一下什么是Event Loop？JavaScript的单线程，与它的用途有关。作为浏览器脚本语言，JavaScript的主要用途是与用户互动，以及操作DOM。这决定了它只能是单线程，否则会带来很复杂的同步问题。比如，假定JavaScript同时有两个线程，一个线程在某个DOM节点上添加内容，另一个线程删除了这个节点，这时浏览器应该以哪个线程为准？所以，为了避免复杂性，从一诞生，JavaScript就是单线程，这已经成了这门语言的核心特征，将来也不会改变

js代码执行过程中会有很多任务，这些任务总的分成两类：同步任务异步任务 当我们打开网站时，网页的渲染过程就是一大堆同步任务，比如页面骨架和页面元素的渲染。而像加载图片音乐之类占用资源大耗时久的任务，就是异步任务。我们，我们用导图来说明：我们解释一下这张图：同步和异步任务分别进入不同的执行"场所"，同步的进入主线程，异步的进入Event Table并注册函数。当指定的事情完成时，Event Table会将这个函数移入Event Queue。主线程内的任务执行完毕为空，会去Event Queue读取对应的函数，进入主线程执行。上述过程会不断重复，也就是常说的Event Loop(事件循环)。那主线程执行栈何时为空呢？js引擎存在monitoring process进程，会持续不断的检查主线程执行栈是否为空，一旦为空，就会去Event Queue那里检查是否有等待被调用的函数 以上就是js运行的整体流程 面试中该如何回答呢？

下面是我个人推荐的答案：首先js 是单线程运行的，在代码执行的时候，通过将不同函数的执行上下文压入执行栈中来保证代码的有序执行在执行同步代码的时候，如果遇到了异步事件，js 引擎并不会一直等待其返回结果，而是会将这个事件挂起，继续执行执行栈中的其他任务当同步事件执行完毕后，再将异步事件对应的回调加入到与当前执行栈中不同的另一个任务队列中等待执行任务队列可以分为宏任务对列和微任务对列，当当前执行栈中的事件执行完毕后，js引擎首先会判断微任务对列中是否有任务可以执行，如果有就将微任务队首的事件压入栈中执...

1) 控制台代码先注册setTimeout(fn,0)，后注册Promise.resolve().then(task)，结果task先执行；定时器写在前面为什么没有优先？

答：当前script本身先作为一个宏任务执行，期间定时器回调进入宏任务队列，then回调进入微任务队列。同步代码结束后会先清空全部微任务，再进入下一轮取宏任务，所以注册先后不能跨越任务类型决定执行次序。

2) 按钮点击后先把状态改成loading，随后同步计算两秒，页面却直到计算结束才更新，你会怎样解释并调整？

答：点击回调作为当前任务占用主线程，同步计算未结束时，浏览器没有机会进入必要的UI渲染阶段，因此状态虽已修改却尚未绘制。应拆分长任务并主动让出执行权，或把重计算移出主线程；仅把计算包装进微任务仍可能阻塞渲染。

3) 消息列表一次追加数据后递归创建Promise.then微任务，页面长时间不重绘，明明每次回调都很短，问题在哪里？

答：一次宏任务结束后，事件循环会持续清空微任务，包括执行过程中新增的微任务；递归产生微任务可能让队列迟迟无法清空。渲染通常位于微任务检查之后，因此应限制递归链，并把后续批次交给新的任务或其他调度机制。

4) 线上日志显示同步代码正常结束，但某个`setTimeout(...,0)`偶尔延迟很久才执行，你会先排查哪些阻塞来源？

答：0只表示满足计时条件后可进入候选队列，不保证立即执行；当前执行栈、此前积压的微任务以及更早待处理的任务都会推迟回调。应检查长同步计算、连续微任务和高频事件处理，而不能仅凭定时参数判断调度异常。

5) Node服务里同时安排`process.nextTick`、`Promise.then`、`setTimeout`和`setImmediate`，同事套用浏览器顺序直接下结论，你会接受吗？

答：不能直接套用，因为Node的事件循环包含`timers`、`poll`、`check`等阶段，`setImmediate`在`check`阶段执行。`process.nextTick`的优先级还高于普通`Promise`微任务；定时器与`setImmediate`的先后需结合注册位置和所处阶段判断。

6) 数据看板想把大量DOM更新全部塞进`Promise.then`，理由是微任务比定时器更早执行，这个选型有什么风险？

答：微任务确实会在当前任务结束后优先清空，但大量DOM操作会占满微任务检查阶段，反而推迟浏览器渲染和下一轮事件处理。应按可见更新边界分批执行，并给渲染留下机会；“更早进入执行栈”不等于页面能更早呈现。

557. 垃圾回收

JavaScript垃圾回收会识别并释放程序不再使用的对象，V8则通过分代机制降低回收成本。新生代对象多数存活时间短，所以使用分成From和To空间的Scavenge算法，存活对象会被复制，满足条件时晋升到老生代。老生代对象存活更久，主要采用标记清除，并用标记压缩缓解内存碎片。由于回收会暂停应用逻辑，V8还通过增量标记把较长的老生代标记过程拆开执行。对于在JavaScript中的字符串，对象，数组是没有固定大小的，只有当对他们进行动态分配存储时，解释器就会分配内存来存储这些数据，当JavaScript的解释器消耗完系统中所有可用的内存时，就会造成系统崩溃。内存泄漏，在某些情况下，不再使用到的变量所占用内存没有及时释放，导致程序运行中，内存越占越大，极端情况下可以导致系统崩溃，服务器宕机。JavaScript有自己的一套垃圾回收机制，JavaScript的解释器可以检测到什么时候程序不再使用这个对象了（数据），就会把它所占用的内存释放掉。针对JavaScript的来及回收机制有以下两种方法（常用）：标记清除，引用计数标记清除 v8 的垃圾回收机制基于分代回收机制，这个机制又基于世代假说，这个假说有两个特点，一是新生的对象容易早死，另一个是不死的对象会活得更久。基于这个假说，v8引擎将内存分为了新生代和老生代。新创建的对象或者只经历过一次的垃圾回收的对象被称为新生代。经历过多次垃圾回收的对象被称为老生代。新生代被分为 From 和 To 两个空间，To 一般是闲置的。当 From 空间满了的时候会执行 Scavenge 算法进行垃圾回收。当我们执行垃圾回收算法的时候应用逻辑将会停止，等垃圾回收结束后再继续执行。这个算法分为三步：首先检查 From 空间的存活对象，如果对象存活则判断对象是否满足晋升到老生代的条件，如果满足条件则晋升到老生代。如果不满足条件则移动 To 空间。如果对象不存活，则释放对象的空间。最后将 From 空间和 To 空间角色进行交换。新生代对象晋升到老生代有两个条件：第一个是判断是对象否已经经过一次 Scavenge 回收。若经历过，则将对象从 From 空间复制到老生代中；若没有经历，则复制到 To 空间。第二个是 To 空间的内存使用占比是否超过限制。当对象从 From 空间复制到 To 空间时，若 To 空间使用超过 25%，则对象直接晋升到老生代中。设置 25% 的原因主要是因为算法结束后，两个空间结束后会交换位置，如果 To 空间的内存太小，会影响后续的内存分配。老生代采用了标记清除法和标记压缩法。标记清除法首先会对内存中存活的对象进行标记，标记结束后清除掉那些没有标记的对象。由于标记清除后会造成很多的内存碎片，不便于后面的内存分配。所以了解决内存碎片的问题引入了标记压缩法。由于在进行垃圾回收的时

候会暂停应用的逻辑，对于新生代方法由于内存小，每次停顿的时间不会太长，但对于老生代来说每次垃圾回收的时间长，停顿会造成很大的影响。为了解决这个问题 V8 引入了增量标记的方法，将一次停顿进行的过程分为了多步，每次执行完一小步就让运行逻辑执行一会，就这样交替运行

1) 商品瀑布流每次滚动都创建一批临时对象，内存呈锯齿状上涨后又回落；既然最终都释放了，为什么页面仍可能周期性卡顿？

答：内存能回落只说明对象最终可回收，不代表回收没有成本。新生代From空间满后会执行Scavenge，检查存活对象并复制或晋升，期间应用逻辑会暂停；高频分配会增加回收频率，因此还要减少滚动热路径中的临时对象。

2) 编辑器把一个仍被全局缓存引用的文档对象标记成“已关闭”，随后主动调用垃圾回收；引擎会因为业务上不用它而释放内存吗？

答：不会，垃圾回收依据对象是否仍可被程序引用，而不是业务状态字段。全局缓存仍保存引用时，该对象会被判断为存活；应删除缓存项或清空引用，单纯改成“已关闭”以及等待回收都不能解决占用。

3) 列表页持续创建短命数据对象，候选人建议把所有对象都长期缓存，避免频繁触发新生代回收；这个方案为什么可能适得其反？

答：长期缓存会让原本容易早死的对象继续存活，并可能在经历回收后晋升到老生代。老生代回收需要标记存活对象，还可能通过标记压缩处理碎片，停顿影响通常更明显；缓存只应保留确有复用价值且能及时淘汰的数据。

4) 监控显示一次明显长停顿发生在老生代回收阶段，同事认为标记清除结束后已有连续可用空间；你会怎样纠正并推进排查？

答：标记清除会释放未标记对象，但可能留下分散的内存碎片，并不保证形成连续空间。引擎还可能执行标记压缩来整理存活对象，因此应结合对象存活量和晋升情况排查长期引用，而不是只盯着回收后的空闲总量。

5) 动画团队要求彻底消除老生代垃圾回收停顿，并把增量标记当成保证；你会如何评估这个目标？

答：增量标记把一次较长的标记过程拆成多步，让应用逻辑在步骤之间获得执行机会，但并不等于垃圾回收零停顿。工程目标应改为控制单次影响和垃圾产生量，同时减少无意义的长期存活对象；不能把增量机制当作实时性承诺。

558. 内存泄露

内存泄露本质上是对象已经不再使用，却仍被某个引用链持有，导致垃圾回收无法释放它。常见原因可以归为两类：意外全局变量和闭包长期保留引用，以及未关闭的定时器、事件监听或已删除DOM的残留引用。排查时我一般用Chrome Timeline标记并观察内存变化，定位持续增长的异常点。开发中还要留意console.log保留的对象，避免它干扰内存回收和判断。

意外的全局变量：无法被回收定时器：未被正确关闭，导致所引用的外部变量无法被释放事件监听：没有正确销毁（低版本浏览器可能出现）闭包第一种情况是我们由于使用未声明的变量，而意外的创建了一个全局变量，而使这个变量一直留在内存中无法被回收。第二种情况是我们设置了setInterval定时器，而忘记取消它，如果循环函数有对外部变量的引用的话，那么这个变量会被一直留在内存中，而无法被回收。第三种情况是我们获取一个DOM元素的引用，而后面这个元素被删除，由于我们一直保留了对这个元素的引用，所以它也无法被回收。第四种情况是不合理的使用闭包，从而导致某些变量一直被留在内存当中。dom引用:dom元素被删除时，内存中的引用未被正确清空控制台console.log打印的东西 第一种情况是我们由于使用未声明的变量，而意外的创建了一个全局变量，而使这个变量一直留在内存中无法被回收。第二种情况是我们设置了setInterval定时器，而忘记

取消它，如果循环函数有对外部变量的引用的话，那么这个变量会被一直留在内存中，而无法被回收。第三种情况是我们获取一个DOM元素的引用，而后面这个元素被删除，由于我们一直保留了对这个元素的引用，所以它也无法被回收。第四种情况是不合理的使用闭包，从而导致某些变量一直被留在内存当中。可用chrome中的timeline进行内存标记，可视化查看内存的变化情况，找出异常点。

内存泄露排查方法(opens new window)

1) 单页后台关闭一个弹窗后，节点已经从页面移除，但堆里仍能看到整棵弹窗子树；前端同事说“删了DOM就一定会回收”，你怎么判断？

答：从文档树移除不等于失去全部引用，代码中的变量、缓存或闭包仍可能持有该节点。应沿引用关系找到保留它的对象并清空对应引用；只执行节点删除无法保证释放，残留引用还可能连带保留子节点和关联数据。

2) 行情组件每次进入页面都会启动一个setInterval，离开页面只删除界面节点；连续切换后内存上涨且旧回调仍在执行，应该改哪里？

答：组件销毁时必须取消对应的setInterval，并解除回调对外部状态的持续引用。定时器未关闭时，循环函数及其引用的变量会一直存活；只删除界面节点既阻止不了回调，也无法释放被闭包保留的数据。

3) 埋点脚本为了调试，把每次请求的完整响应都赋给未声明变量；数据量持续增加后页面占用不降，开启严格模式或改成局部变量能解决什么？

答：未声明赋值可能意外形成全局变量，使响应对象长期处于可达状态；改为受控的局部变量并在生命周期结束后解除引用，才能让数据具备回收条件。严格模式有助于尽早暴露此类赋值，但已有的全局缓存仍需显式清理。

4) 路由页反复进入退出后内存曲线持续抬升，代码里同时存在事件监听、定时器、闭包和已删除节点引用；你会怎样缩小故障范围？

答：先用Chrome的Timeline记录多轮进入、退出过程并标记内存变化，确认异常增长与哪段生命周期重合。随后逐项检查监听是否销毁、定时器是否取消、闭包和变量是否仍持有节点；单看某一次峰值不足以证明泄漏。

5) 团队想把所有闭包都改写成全局函数来治理泄漏，这个取舍合理吗？

答：不合理，闭包本身不是必然泄漏，风险来自闭包被长生命周期对象持续引用，同时又捕获了不再需要的数据。应缩小捕获范围并在定时器、监听或缓存结束时断开引用；改成全局函数反而可能引入更长生命周期的全局状态。

559. 深浅拷贝

浅拷贝只复制一层，深层对象仍共享引用；深拷贝则要为嵌套数据创建相互独立的副本。对象可用Object.assign或展开运算符浅拷贝，数组也可用slice、concat，但修改嵌套属性仍会互相影响。JSON.parse(JSON.stringify(obj))虽然简单，却会忽略undefined、函数和Symbol，还会改变Date、RegExp等类型，并且无法处理循环引用。要求完整时可递归结合Reflect.ownKeys、属性描述符和WeakMap，兼顾特殊对象、原型及循环引用。

1. 浅拷贝的原理和实现

自己创建一个新的对象，来接受你要重新复制或引用的对象值。如果对象属性是基本的数据类型，复制的就是基本类型的值给新对象；但如果属性是引用数据类型，复制的就是内存中的地址，如果其中一个对象改变了这个内存中的地址，肯定会影响到另一个对象

方法一：object.assign object.assign是ES6中object的一个方法，该方法可以用于JS对象的合并等多个用途，其中一个用途就是可以进行浅拷贝。该方法的第一个参数是拷贝的目标对象，后面的参数是拷贝的来源对象（也可以是多个来源）。object.assign的语法为：Object.assign(target, ...sources)的示例代码如下：

```

let target = {}; let source = {a: {b: 1}}; Object.assign(target, source); console.log(target); // { a: { b: 1 } };
但是使用 object.assign 方法有几点需要注意
它不会拷贝对象的继承属性；它不会拷贝对象的不可枚举的属性；可以拷贝Symbol类型的属性。 let obj1 =
{a: {b: 1}, sym: Symbol(1)}; Object.defineProperty(obj1, 'innumerable', {value: '不可枚举属性', enumerable:
false}); let obj2 = {}; Object.assign(obj2, obj1) obj1.a.b = 2; console.log('obj1', obj1); console.log('obj2',
obj2); 从上面的样例代码中可以看到，利用object.assign也可以拷贝Symbol类型的对象，但是如果到了对
象的第二层属性 obj1.a.b 这里的时候，前者值的改变也会影响后者的第二层属性的值，说明其中依旧存在
着访问共同堆内存的问题，也就是说这种方法还不能进一步复制，而只是完成了浅拷贝的功能
方法二：扩展运算符方式 我们也可以利用 JS
的扩展运算符，在构造对象的同时完成浅拷贝的功能。扩展运算符的语法为：let cloneObj = { ...obj }; /*
对象的拷贝 */ let obj = {a: 1, b: {c: 1}}; let obj2 = {...obj}; obj.a = 2; console.log(obj); // {a: 2, b: {c: 1}}
console.log(obj2); // {a: 1, b: {c: 1}} obj.b.c = 2; console.log(obj); // {a: 2, b: {c: 2}} console.log(obj2);
// {a: 1, b: {c: 2}} /* 数组的拷贝 */ let arr = [1, 2, 3]; let newArr = [...arr]; //跟arr.slice()是一样的效果 扩展运算符
和object.assign有同样的缺陷，也就是实现的浅拷贝的功能差不多，但是如果属性都是基本类型的值，使
用扩展运算符进行浅拷贝会更加方便 方法三：concat 拷贝数组 数组的concat方法其实也是浅拷贝，所以
连接一个含有引用类型的数组时，需要注意修改原数组中的元素的属性，因为它会影响拷贝之后连接的数
组。不过concat只能用于数组的浅拷贝，使用场景比较局限。代码如下所示。
let arr = [1, 2, 3]; let newArr = arr.concat(); newArr[1] = 100; console.log(arr); // [ 1, 2, 3 ]
console.log(newArr); // [ 1, 100, 3 ] 方法四：slice 拷贝数组 slice方法也比较有局限性，因为它仅仅针对
数组类型。slice方法会返回一个新的数组对象，这一对象由该方法的前两个参数来决定原数组截取的开始
和结束时间，是不会影响和改变原始数组的。 slice 的语法为：arr.slice(begin, end);
let arr = [1, 2, {val: 4}]; let newArr = arr.slice(); newArr[2].val = 1000; console.log(arr); // [ 1, 2, ...

```

1) 表单状态执行{...state}后，修改副本里的address.city，原表单也跟着变化；代码评审里有人坚持扩展运算符已经完成复制，你怎么解释？

答：扩展运算符只创建新的顶层对象，address这类嵌套引用仍指向同一块数据，因此修改其内部属性会同时影响两边。若只改顶层基本类型字段，浅拷贝足够；要求嵌套状态完全隔离时才需要继续复制对应层级或执行深拷贝。

2) 数据合并用Object.assign({}, source)，结果继承属性和不可枚举配置没有进入副本，但Symbol键被保留；这是实现故障还是方法边界？

答：这是Object.assign的边界：它不会复制继承属性和不可枚举属性，但能够复制可枚举的Symbol属性。若业务要求保留原型和完整属性特性，应结合Object.getOwnPropertyDescriptors与Object.create；代价是实现复杂度明显增加。

3) 日志快照包含Date、RegExp、undefined、NaN和循环引用，团队仍准备使用JSON.parse(JSON.stringify(data))；上线前你会拦住哪些风险？

答：这条路径会让Date变成字符串、RegExp变成空对象，忽略undefined，并把NaN与无穷值变成null；遇到循环引用还会直接失败。它只适合可安全表示为JSON的数据，不能当作通用深拷贝方案。

4) 候选人写的递归深拷贝遇到数组得到普通对象，遇到循环引用则无限递归；你要求在现有思路修正，会看哪些关键动作？

答：应按具体类型创建数组、Date、RegExp等对应实例，并在递归前用WeakMap记录“原对象到副本”的映射。再次遇到同一对象时直接返回已有副本即可闭合环；若只用普通对象承接所有引用类型，类型语义仍会丢失。

5) 权限模型依赖不可枚举字段、Symbol键和自定义原型，业务方又要求副本与原对象完全分离；普通递归赋值为什么不够？

答：普通for...in递归拿不到完整的自有键和属性描述，也不会自动保留原型链。可用Reflect.ownKeys枚举键，以Object.getOwnPropertyDescriptors获取特性，再通过Object.create保留原型；函数通常仍只能保留引用，不能假定得到独立副本。

6) 评审中一方主张所有接口结果都深拷贝，另一方只愿意用slice、concat或扩展运算符；你会依据什么决定？

答：先看调用方是否会修改嵌套引用：只需隔离数组容器或对象顶层时，slice、concat和扩展运算符成本更低，但它们都是浅拷贝。只有确实要求多层数据互不影响时才采用深拷贝，并明确特殊类型、属性描述和循环引用的支持范围。

560. 节流与防抖

防抖是在停止触发一段时间后执行，节流是在固定时间窗口内最多执行一次。防抖每次收到新事件都会清除旧定时器并重新计时，适合避免连续点击产生多次请求。节流则比较当前时间与上次执行时间，常用于降低scroll监听的调用频率。手写时返回函数不能用箭头函数替代外层普通函数，还要通过apply透传调用时的this和参数。函数防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内事件又被触发，则重新计时。这可以使用在一些点击请求的事件上，避免因为用户的多次点击向后端发送多次请求。函数节流是指规定一个单位时间，在这个单位时间内，只能有一次触发事件的回调函数执行，如果在同一个单位时间内某事件被触发多次，只有一次能生效。节流可以使用在 scroll

函数的事件监听上，通过事件节流来降低事件调用的频率。// 函数防抖的实现functiondebounce(fn,wait){var timer=null;returnfunction(){var context=this,args=arguments;//

如果此时存在定时器的话，则取消之前的定时器重新计时if(timer){clearTimeout(timer);timer=null;}}

设置定时器，使事件间隔指定事件后执行timer=setTimeout(()=>{fn.apply(context,args);},wait);})// 函数节流的实现functionthrottle(fn,delay){var preTime=Date.now();returnfunction(){var context=this,args=arguments,nowTime=Date.now();// 如果两次时间间隔超过了指定时间，则执行函数。if(nowTime-preTime>=delay){preTime=Date.now();returnfn.apply(context,args);}}

1) 搜索框输入“javascript”时连续触发十次input，产品要求用户停下后只查询一次；同事却用每300ms执行一次的节流，你会怎么改？

答：这里应使用防抖：每次输入都取消旧定时器并重新计时，只有连续一段时间没有新输入才执行查询。节流会在持续输入期间按时间窗口放行回调，仍可能产生多次请求；等待时长过大还会增加结果出现的延迟。

2) 长列表监听scroll更新吸顶位置，用户持续滚动五秒时页面必须周期性刷新；若改用只在停止滚动后执行的防抖，会出现什么现象？

答：防抖会不断重新计时，持续滚动期间回调可能一直不执行，吸顶状态因而滞后到操作结束。此处更适合节流，在规定时间内最多放行一次更新以降低频率；时间窗口过大仍会让视觉反馈不够连续。

3) 支付按钮在一秒内被连点多次，接口不能重复提交；只套用示例里的尾触发防抖，业务上还缺少什么约束？

答：尾触发防抖能把密集点击合并为一次延后调用，但它不能证明请求尚未完成时不会再次提交。按钮还应在请求期间进入不可重复触发状态，并由业务层校验提交状态；防抖只是降低调用次数，不能替代接口幂等或状态控制。

4) 组件卸载后，防抖封装里的定时器仍触发旧回调并访问已销毁状态；你会怎样定位和修复这类线上报错？

答：先确认卸载前最后一次事件是否留下待执行的setTimeout，以及回调闭包捕获了哪些组件数据。封装应暴露取消能力，并在组件销毁时清除定时器；仅移除事件监听不会自动取消已经排队的尾触发任务。

5) 同一套拖拽逻辑既要按固定频率更新坐标，又要在停止后保存最终位置，只允许选纯节流或纯防抖吗？

答：不必二选一：移动过程中用节流限制坐标更新频率，停止操作后再用防抖触发最终保存，更符合两个阶段的目标。若只用节流，最终一次状态可能未及时持久化；只用防抖则持续拖动时缺少中间反馈。

561. Proxy代理

Proxy是在目标对象外层建立统一拦截，让读取、赋值、删除等操作先经过自定义的handler。创建时传入target和handler，例如访问属性触发get，修改属性触发set，因此既能监视外部访问，也能在复杂操作前做校验或资源管理。它还支持原型、属性描述符、函数调用和构造等多类操作，不只是简单的属性读写。由于这些能力无法由Object.defineProperty完整模拟，所以现有polyfill只能覆盖部分功能。

proxy在目标对象的外层搭建了一层拦截，外界对目标对象的某些操作，必须通过这层拦截

```
var proxy = new Proxy(target, handler);
```

new Proxy()表示生成一个Proxy实例，target参数表示所要拦截的目标对象，handler参数也是一个对象，用来定制拦截行为

```
var target = {name: 'poetries'};
var logHandler = {
  get: function(target, key) {
    console.log(`${key}被读取`);
    return target[key];
  },
  set: function(target, key, value) {
    console.log(`${key}被设置为${value}`);
    target[key] = value;
  }
};
var targetWithLog = new Proxy(target, logHandler);
targetWithLog.name; // 控制台输出：name 被读取
targetWithLog.name = 'others'; // 控制台输出：name 被设置为 others
console.log(target.name); // 控制台输出：others
targetWithLog读取属性的值时，实际上执行的是logHandler.get：在控制台输出信息，并且读取被代理对象target的属性。在targetWithLog设置属性值时，实际上执行的是logHandler.set：在控制台输出信息，并且设置被代理对象target的属性的值 // 由于拦截函数总是返回35，所以访问任何属性都得到35
var proxy = new Proxy({}, {
  get: function(target, property) {
    return 35;
  }
});
proxy.time; // 35
proxy.name; // 35
proxy.title; // 35
Proxy实例也可以作为其他对象的原型对象
var proxy = new Proxy({}, {
  get: function(target, property) {
    return 35;
  }
});
let obj = Object.create(proxy);
obj.time; // 35
proxy对象是obj对象的原型，obj对象本身并没有time属性，所以根据原型链，会在proxy对象上读取该属性，导致被拦截
```

Proxy的作用

对于代理模式Proxy的作用主要体现在三个方面

拦截和监视外部对对象的访问降低函数或类的复杂度在复杂操作前对操作进行校验或对所需资源进行管理

Proxy所能代理的范围--handler 实际上 handler 本身就是ES6所新设计的一个对象.它的作用就是用来自定义代理对象的各种可代理操作

。它本身一共有13中方法,每种方法都可以代理一种操作.其13种方法如下

//

在读取代理对象的原型时触发该操作，比如在执行

Object.getPrototypeOf(proxy)

时。handler.getPrototypeOf()//

在设置代理对象的原型时触发该操作，比如在执行

Object.setPrototypeOf(proxy, null)

时。handler.setPrototypeOf()//

在判断一个代理对象是否是可扩展时触发该操作，比如在执行

Object.isExtensible(proxy)

时。handler.isExtensible()//

在让一个代理对象不可扩展时触发该操作，比如在执行

Object.preventExtensions(proxy)

时。handler.preventExtensions()//

在获取代理对象某个属性的属性描述时触发该操作，比如在执行

Object.getOwnPropertyDescriptor(proxy, "foo")

时。handler.getOwnPropertyDescriptor()//

在定义代理对象某个属性时的属性描述时触发该操作，比如在执行 Object.defineProperty(proxy, "foo", {})

时。andler.defineProperty()// 在判断代理对象是否拥有某个属性时触发该操作，比如在执行 "foo" in proxy...

1) 权限对象套上Proxy后，开发者保留了原始target，随后直接执行target.role = 'admin'，日志里没有任何拦截记录；代理失效了吗？

答：代理没有失效，只有经由代理对象发起的操作才会进入handler。直接修改原始target会绕过外层拦截，但代理读取时仍能看到目标对象的新值；若审计必须完整，就不能继续向业务代码暴露原始对象引用。

2) 配置中心希望读取任意不存在的字段都返回35，但代理同时被设为另一个对象的原型；访问子对象缺失字段时为什么也会得到35？

答：子对象自身找不到属性时会沿原型链读取代理对象，从而触发代理的get拦截。由于该拦截无条件返回35，缺失字段也得到同一结果；这种默认值策略会掩盖拼写错误，生产配置通常需要区分真实属性与兜底值。

3) 一个校验层既要拦截属性赋值和删除，又要监控in判断及键枚举；只实现get、set为什么覆盖不了需求？

答：不同对象操作对应不同陷阱：赋值使用set，删除使用deleteProperty，in判断使用has，键枚举则涉及ownKeys。只实现get、set只能监控读写属性值；选择Proxy前应列出必须覆盖的操作并配置相应handler。

4) 线上发现代理对象的普通读写有日志，但Object.getPrototypeOf(proxy)、Object.defineProperty(proxy, 'x', desc)完全没进入现有埋点；你会检查哪里？

答：应检查handler是否实现了对应的getPrototypeOf和defineProperty陷阱，而不是继续修改get、set。Proxy会按操作类型分派拦截，普通属性读写正常不能证明其他操作已覆盖；遗漏陷阱时相关行为会直接落到目标对象。

5) 项目必须运行在无法原生支持Proxy的旧环境，架构师提出用Object.defineProperty做完整垫片；你会如何做选型判断？

答：不能把Object.defineProperty视为完整的Proxy替代，因为它无法模拟全部代理语义，现有垫片也只能覆盖部分能力，例如get、set。若业务依赖删除、枚举、原型或函数调用等拦截，应调整兼容目标或改写方案，而不是承诺透明降级。

6) 安全团队想用Proxy同时包装普通对象、函数和构造器，要求分别审计读取、调用与new；应该如何映射这些行为？

答：普通属性读取进入get，直接调用函数代理进入apply，通过new构造实例则进入construct。三类行为需要分别实现和校验，单靠属性读取陷阱无法审计调用；代理层还能集中管理资源，但会增加行为边界和调试复杂度。

562. Ajax

Ajax是浏览器通过JavaScript异步发起HTTP请求，并局部更新页面而不必整页刷新的通信方式。原生实现通常用XMLHttpRequest：创建对象、配置地址、发送请求，再通过readyState和status判断响应。实际使用时可以封装成Promise，成功时resolve数据，请求失败或网络异常时reject，让调用链更清晰。

它是一种异步通信的方法，通过直接由js脚本向服务器发起http通信，然后根据服务器返回的数据，更新网页的相应部分，而不用刷新整个页面的一种方法。

面试手写（原生）：
//1：创建Ajax对象var xhr = window.XMLHttpRequest ? new XMLHttpRequest() : new ActiveXObject('Microsoft.XMLHTTP');// 兼容IE6及以下版本
//2：配置

Ajax请求地址xhr.open('get', 'index.xml', true);
//3：发送请求xhr.send(null);
// 严谨写法
//4：监听请求，接受响应xhr.onreadystatechange = function() { if (xhr.readyState == 4 && xhr.status == 200 || xhr.status == 304) console.log(xhr.responseText); }
jQuery写法 \$.ajax({ type: 'post', url: '', async: true, // async 异步 sync

```

同步 data:data, //针对post请求 dataType:'jsonp', success:function (msg) {}, error:function (error) {}
}) promise 封装实现： // promise 封装实现：functiongetJSON(url){// 创建一个 promise
对象letpromise=newPromise(function(resolve,reject){letxhr=newXMLHttpRequest();// 新建一个 http
请求xhr.open("GET",url,true);//
设置状态的监听函数xhr.onreadystatechange=function(){if(this.readyState!==4)return;//
当请求成功或失败时，改变 promise
的状态if(this.status===200){resolve(this.response);}else{reject(newError(this.statusText));};//
设置错误监听函数xhr.onerror=function(){reject(newError(this.statusText));};//
设置响应的数据类型xhr.responseType="json";//
设置请求头信息xhr.setRequestHeader("Accept","application/json");// 发送 http
请求xhr.send(null);};returnpromise;}

```

1) 订单页发起原生XHR后，服务端已经返回200，但页面偶尔在响应尚未完整时就开始渲染，你会检查哪段判断？

答：先检查回调是否同时判断了readyState === 4和成功状态码，不能只看到status === 200就消费响应。readyState === 4表示请求过程已经完成，再读取response或responseXML；条件组合还应加括号，避免&&、||优先级造成误判。

2) 老项目要把十几个读取配置的XHR调用改成Promise，调用方希望直接拿到JSON，你会怎样封装成功、失败和解析过程？

答：在封装内创建XMLHttpRequest，调用open("GET", url, true)，并把responseType设为json。状态完成且为200时用resolve(xhr.response)，其他状态和onerror都调用reject；若服务端返回的内容不是合法JSON，解析行为仍可能失败，需要由调用方处理拒绝。

3) 接口网关把成功响应从200调整为304，原来的请求封装却统一进入失败分支，你会改哪条判定？

答：当前封装若只接受status === 200，自然会把304当作失败，需要按接口约定补充允许的状态。修改前应确认304对该请求是否确实代表可用结果，并检查客户端是否有可复用内容；机械放宽为所有状态都成功，会掩盖真实的服务端错误。

4) 商品页请求一直没有进入成功回调，控制台也看不到明确异常；代码里写着xhr.onreadystatechange和xhr.readyState，你怎么定位？

答：这里首先是属性名拼写错误，正确写法是onreadystatechange和readyState，错误属性不会建立预期的状态监听。修正后再记录readyState、status和statusText，区分请求未完成、HTTP状态失败与网络错误；只盯着响应内容无法定位监听器根本没执行的情况。

5) 数据看板每分钟请求一次JSON，团队争论继续写回调还是统一封装成Promise；在不引入第三方库的前提下你怎么选？

答：保留原生XHR作为通信能力，但对外返回Promise，让调用方用统一的成功和失败链路组织刷新逻辑。封装中集中设置Accept、responseType、状态判断和onerror，可减少各页面重复代码；代价是仍要明确哪些HTTP状态算成功，不能把协议约定藏进模糊的通用封装。

563. 深入数组

理解数组方法时，我一般先看它是否修改原数组、返回什么，以及能处理哪类数据。push、splice、sort等会改变自身，concat、slice、filter等不会；map、reduce、find则分别适合映射、累计和查找。类数组只有索引和length，像arguments、HTMLCollection、NodeList，需要时可用Array.from或slice.call转成数组。一、梳理数组 API 1.Array.of Array.of用于将参数依次转化为数组中的一项，然后返回这个新数组

，而不管这个参数是数字还是其他。它基本上与 `Array` 构造器功能一致，唯一的区别就在单个数字参数的处理上 `Array.of(8.0);// [8]Array(8.0);// [empty × 8]Array.of(8.0,5);// [8, 5]Array(8.0,5);// [8, 5]Array.of('8');// ["8"]Array('8');// ["8"]` 2. `Array.from` 从语法上看，`Array.from` 拥有 3 个参数：类似数组的对象，必选；加工函数，新生成的数组会经过该函数的加工再返回；`this`作用域，表示加工函数执行时`this`的值。这三个参数里面第一个参数是必选的，后两个参数都是可选的。我们通过一段代码来看看它的用法。

```
var obj={0:'a',1:'b',2:'c',length:3};Array.from(obj,function(value,index){console.log(value,index,this,arguments.length);return value.repeat(3);//必须指定返回值，否则返回 undefined},obj);// return 的 value 重复了三遍，最后返回的数组为 ["aaa","bbb","ccc"]// 如果这里不指定 this 的话，加工函数完全可以是一个箭头函数。上述代码可以简写为如下形式。Array.from(obj,(value)=>value.repeat(3));// 控制台返回 (3) ["aaa", "bbb", "ccc"] 除了上述obj对象以外，拥有迭代器的对象还包括String、Set、Map等，Array.from 统统可以处理，请看下面的代码。 // StringArray.from('abc');// ["a", "b", "c"]// SetArray.from(new Set(['abc','def']));// ["abc", "def"]// MapArray.from(new Map([[1,'ab'],[2,'de']]));// [[1, 'ab'], [2, 'de']]
```

3. `Array` 的判断 在 ES5 提供该方法之前，我们至少有如下 5 种方式去判断一个变量是否为数组。

```
var a=[];
1.基于instanceofinstanceofArray;//
2.基于constructora.constructor===Array;//
3.基于Object.prototype.isPrototypeOfArray.prototype.isPrototypeOf(a);//
4.基于getPrototypeOfObject.getPrototypeOf(a)===Array.prototype;//
5.基于Object.prototype.toStringObject.prototype.toString.apply(a)==='[object Array]';
```

ES6 之后新增了一个`Array.isArray`方法，能直接判断数据类型是否为数组，但是如果没有，那么`Array.isArray`的 polyfill 通常可以这样写：

```
if(!Array.isArray){Array.isArray=function(arg){return Object.prototype.toString.call(arg)==='[object Array]';}}
4. 改变自身的方法 基于 ES6，会改变自身值的方法一共有9个，分别为pop、push、reverse、shift、sort、splice、unshift，以及两个 ES6 新增的方法 copyWithin 和 fill // pop方法var array=["cat","dog","cow","chicken","mouse"];var item=array.pop();console.log(array);// ["cat", "dog", "cow", "chicken"]console.log(item);// m...
```

1) 报表代码写了`Array(8)`想得到`[8]`，结果页面生成了 8 个空槽；如果参数来自用户输入，你会怎么修？

答：单个数字传给`Array`会被解释为数组长度，因此结果是包含 8 个空槽的数组，而不是`[8]`。需要把参数作为一个元素时应使用`Array.of(value)`；继续依赖构造器会让数字参数与字符串参数产生不同语义，容易形成数据类型相关的隐患。

2) 接口返回`{0:"a",1:"b",2:"c",length:3}`，列表组件既要转成数组又要把每项重复三次，你会怎样写而不再遍历一轮？

答：可以直接使用`Array.from(data, value => value.repeat(3))`，转换类数组的同时完成映射，得到`['aaa','bbb','ccc']`。映射函数必须显式返回加工后的值，否则对应位置会变成`undefined`；若回调依赖普通函数的`this`，还可通过第三个参数传入作用域。

3) 微前端页面从另一个执行环境传来一个数组，旧代码用`value instanceof Array`判断后走错分支，你会替换成什么？

答：应优先改用`Array.isArray(value)`，它直接表达数组判断，也避免把判断绑定到当前环境的`Array`构造器。需要兼容缺少该方法的旧环境时，可回退到`Object.prototype.toString.call(value) === '[object Array]'`；依赖`constructor`或原型相等会受到对象来源和原型变化影响。

4) 表格排序后发现上游缓存里的原始顺序也被改了，代码只有`const result = rows.sort()`；为什么两个变量一起变化？

答：`sort`会直接修改原数组，并返回同一个数组引用，所以`result`与`rows`指向同一对象。若上游数据必须保持不变，应先用`rows.slice()`或`rows.concat()`复制，再对副本排序；浅复制只能隔离数组结构，元素对象内部的修改仍会共享。

5) 批量删除标签时，开发者以为`splice(1,`

2)返回删除后的原数组，结果后续保存的数据只剩两个标签；你如何解释返回值约定？

答：`splice`会修改原数组，但返回的是被删除元素组成的新数组，而不是修改后的原数组。保存剩余列表应继续使用原变量，若要记录删除项再接收`splice`的返回值；这类接口同时具有副作用和返回值，混用时尤其容易覆盖业务数据。

6) 页面用`querySelectorAll`和`getElementsByClassName`各拿到一批节点，DOM更新后两边数量表现不同，你会怎样处理遍历快照？

答：两者得到的都是类数组，但`HTMLCollection`会随文档结构及时更新，而`querySelectorAll`返回的`NodeList`在该场景下适合按查询结果遍历。需要稳定快照时可用`Array.from(collection)`转成真正数组，再使用`map`、`filter`等数组方法；转换后的数组不会继续自动反映DOM变化。

【HTML】

564. meta 标签：自动刷新/跳转

`meta refresh`可以让页面在指定时间后自动刷新，或者跳转到另一个地址。例如`content="5; URL=page2.html"`表示五秒后跳转，去掉URL后，`content="60"`就表示每隔六十秒刷新。它适合简单的页面轮播或监控大屏刷新；如果还要根据业务状态决定是否跳转，单靠这个标签就不够灵活。假设要实现一个类似 PPT 自动播放的效果，你很可能会想到使用 JavaScript 定时器控制页面跳转来实现。但其实有更加简洁的实现方法，比如通过 `meta` 标签来实现：上面的代码会在 5s 之后自动跳转到同域下的 `page2.html` 页面。我们要实现 PPT 自动播放的功能，只需要在每个页面的 `meta` 标签内设置好下一个页面的地址即可。另一种场景，比如每隔一分钟就需要刷新页面的大屏幕监控，也可以通过 `meta` 标签来实现，只需去掉后面的 URL 即可：`meta viewport`相关

1) 产品把展厅页面做成多页PPT，要求每页停留 5

秒后进入同域的下一页，不想维护定时器代码，你会在页面头部写什么？

答：可以在当前页的中设置，浏览器会在 5 秒后跳到指定页面。每一页都要配置自己的下一页地址；任何一页地址错误或缺失都会中断播放链路，而且这种方式不负责保存页面运行状态。

2) 监控大屏要求每 60

秒整页刷新一次，值班同事却复制了带URL的模板，结果页面不断跳走；正确配置是什么？

答：定时刷新当前页面时应去掉目标地址，使用。带上URL表示延时跳转而非单纯刷新，因此排查时先检查`content`中是否包含分号后的地址；整页刷新会重建页面，不适合必须保留前端临时状态的界面。

3) 同一套展示页部署到不同域名，`content="5;`

`URL=page2.html"`是否需要为每个环境重写完整域名？

答：示例中的page2.html是同源相对地址，因此同样的页面结构通常不必写死完整域名。部署时仍要保证目标文件与当前路径关系一致，否则相对地址会解析到错误位置；跨域跳转或目录结构不同的环境不能直接套用这一配置。

4) 下线页设置了Refresh，测试反馈等待 5 秒后仍停在原页，你会按什么顺序检查？

答：先确认标签位于文档内，并核对http-equiv="Refresh"、秒数及URL的拼写和引号。随后直接访问目标地址，排除文件不存在或相对路径错误；如果页面还有其他跳转逻辑，也要避免多套机制同时控制导航而让现象难以判断。

5) 搜索落地页既要自动轮播，又希望搜索结果准确展示页面摘要，只有Refresh标签够吗？

答：不够，Refresh只负责延时刷新或跳转，不提供页面描述信息。还应设置语义清晰的，供搜索引擎提取和展示摘要；描述应反映当前页面内容，不能把导航控制与搜索信息混成一个标签。

565. viewport

viewport用来控制移动端页面的视口宽度、初始缩放比例，以及用户能否手动缩放。常见写法是设置width=device-width和initial-scale=1.0，让页面按设备宽度布局。minimum-scale、maximum-scale、user-scalable可以继续限制缩放范围，但要结合交互需求取舍。处理移动端细线时，也可以配合rem和transform: scale(0.5)做局部缩放。 // width 设置viewport宽度，为一个正整数，或字符串 'device-width' // device-width 设备宽度// height 设置viewport高度，一般设置了宽度，会自动解析出高度，可以不用设置// initial-scale 默认缩放比例（初始缩放比例），为一个数字，可以带小数// minimum-scale 允许用户最小缩放比例，为一个数字，可以带小数// maximum-scale 允许用户最大缩放比例，为一个数字，可以带小数// user-scalable 是否允许手动缩放 延伸提问怎样处理移动端1px被渲染成2px问题 怎样处理 移动端1px被渲染成2px问题 局部处理meta标签中的viewport属性，initial-scale设置为1rem按照设计稿标准走，外加利用transform的scale(0.5)缩小一倍即可； 全局处理mate标签中的viewport属性，initial-scale设置为0.5rem按照设计稿标准走即可

1) 设计稿标注页面宽度跟随手机屏幕，开发却把viewport的width写成固定数字，横竖屏下布局都不对；你会怎么改？

答：应将width设置为device-width，让布局视口宽度跟随设备宽度，并通常配合initial-scale=1.0。固定宽度会让浏览器按同一个布局视口缩放不同设备上的页面；改完仍需检查CSS是否另有固定宽度，否则仅调整meta不能解决全部溢出。

2) 活动页要求初始按原尺寸展示，同时允许用户在1到3倍之间缩放，你会怎样调整现有的user-scalable=no？

答：应改为允许手动缩放，并设置initial-scale=1、minimum-scale=1、maximum-scale=3。这几个值分别控制初始、最小和最大缩放比例，user-scalable=no与允许缩放的需求直接冲突；范围仍要结合页面布局验证，避免放大后关键操作区域被遮挡。

3) 移动端边框按设计稿写成1px，在高像素密度设备上视觉却像2px；只改这个组件时你会怎么处理？

答：局部处理可保持的initial-scale=1，页面的rem继续按设计稿标准计算，再对细线元素使用transform: scale(0.5)缩小。缩放会同时影响元素视觉尺寸和定位关系，因此应把变换限制在边框或专用伪元素上，避免整个业务组件一起缩小。

4) 团队想用initial-scale=0.5一次解决全站的移动端细线问题，但站内已有大量按现行rem编写的页面，你会直接合并吗？

答：不会直接合并，因为把initial-scale调为0.5属于全局视口策略，所有页面的初始缩放都会受到影响。只有在整站rem规则都按对应设计标准统一时才适合采用，并需要完整回归布局；已有页面单位体系不一致时，局部transform的影响范围更可控。

5) 线上页面在部分手机上显得异常放大，代码同时设置了initial-scale=1、minimum-scale=1和maximum-scale=1；你会先核对哪些配置含义？

答：先确认width是否为device-width，再检查三个缩放参数是否确实为合法数字，以及user-scalable是否符合产品要求。initial-scale决定初始比例，另外两个只限定允许的缩放边界，不能替代正确的视口宽度；若配置无误，还要继续检查页面自身的固定尺寸和rem基准。

566. 性能优化

HTML层面的性能优化，本质上是减少脚本阻塞，并提前完成关键资源的连接或加载。普通script会暂停解析，async在下载完成后立即执行，defer和默认的类型="module"则等文档解析完成后执行。跨域资源可按需要使用dns-prefetch、preconnect、prefetch或preload，其中preload的加载确定性更高。搜索场景还应设置meta description，方便搜索引擎提取页面描述。性能优化是前端开发中避不开的问题，性能问题无外乎两方面原因：渲染速度慢、请求时间长。性能优化虽然涉及很多复杂的原因和解决方案，但其实只要通过合理地使用标签，就可以在在一定程度上提升渲染速度以及减少请求时间

1. script 标签：调整加载顺序提升渲染速度 由于浏览器的底层运行机制，渲染引擎在解析 HTML 时，若遇到 script 标签引用文件，则会暂停解析过程，同时通知网络线程加载文件，文件加载后会切换至 JavaScript 引擎来执行对应代码，代码执行完成之后切换至渲染引擎继续渲染页面。在这一过程中可以看到，页面渲染过程中包含了请求文件以及执行文件的时间，但页面的首次渲染可能并不依赖这些文件，这些请求和执行文件的动作反而延长了用户看到页面的时间，从而降低了用户体验。为了减少这些时间损耗，可以借助 script 标签的

3 个属性来实现。 async 属性。立即请求文件，但不阻塞渲染引擎，而是文件加载完毕后阻塞渲染引擎并立即执行文件内容 defer 属性。立即请求文件，但不阻塞渲染引擎，等到解析完 HTML 之后再执行文件内容HTML5 标准 type 属性，对应值为“module”。让浏览器按照 ECMA Script 6

标准将文件当作模块进行解析，默认阻塞效果同 defer，也可以配合 async 在请求完成后立即执行。绿色的线表示执行解析 HTML，蓝色的线表示请求文件，红色的线表示执行文件 当渲染引擎解析 HTML 遇到 script 标签引入文件时，会立即进行一次渲染。所以这也就是为什么构建工具会把编译好的引用 JavaScript 代码的 script 标签放入到 body 标签底部，因为当渲染引擎执行到 body 底部时会先将已解析的内容渲染出来，然后再去请求相应的 JavaScript 文件

2. link 标签：通过预处理提升渲染速度 在我们对大型单页应用进行性能优化时，也许会用到按需懒加载的方式，来加载对应的模块，但如果能合理利用link标签的rel属性值来进行预加载，就能进一步提升渲染速度。

dns-prefetch。当link标签的rel属性值为“dns-prefetch”时，浏览器会对某个域名预先进行 DNS 解析并缓存。这样，当浏览器在请求同域名资源的时候，能省去从域名查询 IP 的过程，从而减少时间损耗。下图是淘宝网设置的 DNS 预解析preconnect。让浏览器在一个 HTTP 请求正式发给服务器前预先执行一些操作，这包括DNS 解析、TLS 协商、TCP 握手，通过消除往返延迟来为用户节省时间prefetch/preload。两个值都是让浏览器预先下载并缓存某个资源，但不同的是，prefetch 可能会在浏览器忙时被忽略，而preload

则是一定会被预先下载。prerender。浏览器不仅会加载资源，还会解析执行页面，进行预渲染 这几个属性值恰好反映了浏览器获取资源文件的过程，在这里我绘制了一个流程简图，方便你记忆。

3. 搜索优化 meta 标签：提取关键信息通过 meta

标签可以设置页面的描述信息，从而让搜索引擎更好地展示搜索结果。示例 通过 meta

标签可以设置页面的描述信息，从而让搜索引擎更好地展示搜索结果。示例

1) 首页把统计脚本直接放在，网络稍慢时正文长期不出现；浏览器遇到普通外链script时发生了什么？

答：渲染引擎解析到普通外链script时会暂停HTML解析，等待文件请求完成并切换到JavaScript引擎执行，之后才继续渲染。若首屏不依赖该脚本，这段下载和执行时间就会推迟内容出现；脚本若必须同步生成后续DOM，则不能只为提速而随意改成异步。

2) 页面有两个存在依赖关系的初始化脚本，要求下载期间不阻塞HTML，并在文档解析完成后按顺序执行，你选async还是defer？

答：这里应选defer，它会立即请求文件但不阻塞HTML解析，并在文档解析完成后执行。async会在文件下载完毕后立即打断解析并执行，多个文件的完成先后也可能不符合依赖顺序；依赖脚本使用async容易出现偶发初始化失败。

3) 团队把入口改成后又额外加了async，结果模块在HTML尚未解析完时访问节点失败；差异在哪里？

答：模块脚本默认具有类似defer的阻塞效果，会在HTML解析完成后执行；加入async后则会在请求完成时尽快执行。入口代码依赖页面节点时不应贸然添加async，或必须让代码自行等待DOM就绪；模块化并不意味着执行时机可以忽略。

4) 首屏字体来自独立CDN，性能记录显示请求前耗时主要落在域名解析、连接和TLS协商，你会配置dns-prefetch还是preconnect？

答：这种场景更适合对该CDN使用preconnect，因为它可以提前完成DNS解析、TCP握手和TLS协商。dns-prefetch只预先解析域名，覆盖不了后续连接成本；预连接应留给确定很快会访问的关键域名，过度配置会产生没有收益的连接工作。

5) 路由懒加载的下一页资源有时根本没被提前下载，代码使用的是prefetch；产品要求进入下一页前必须准备好，你会怎么取舍？

答：prefetch在浏览器繁忙时可能被忽略，因此不能把它当作必须完成的下载保证；确定当前流程必需的资源可考虑改用preload。preload会更积极地预先下载，但也会占用当前页面的网络资源，若下一页并非高概率访问，反而可能挤压首屏关键请求。

6)

搜索结果中的页面摘要混乱，开发准备靠增加预加载标签修复；你会把排查方向转到哪里？

答：预加载标签解决的是资源获取和渲染时机，不负责搜索结果摘要，应检查页面是否设置了准确的。描述内容需要提炼当前页面的关键信息，帮助搜索引擎展示结果；它与preload、preconnect属于不同优化目标，混用不会修复摘要质量。

567. 如何高效操作DOM

高效操作DOM的核心是减少频繁访问和修改，因为它既会带来引擎切换，也可能触发重排与重绘。循环中反复读取节点时，我一般先把节点引用缓存到循环外，避免重复访问document.body。批量创建元素时，可以先拼好完整的HTML，再通过一次innerHTML更新，减少多次插入。修改尺寸、边距或增删节点可能引发重排，而颜色、背景等变化通常只需重绘，成本并不完全相同。

1. 为什么说 DOM 操作耗时

1.1 线程切换 浏览器为了避免两个引擎同时修改页面而造成渲染结果不一致的情况，增加了另外一个机制，这两个引擎具有互斥性，也就是说在某个时刻只有一个引擎在运行，另一个引擎会被阻塞。操作系统在进行线程切换的时候需要保存上一个线程执行时的状态信息并读取下一个线程的状态信息，俗称上下文切换。而这个操作相对而言是比较耗时的每次 DOM 操作就会引发线程的上下文切换——从 JavaScript 引擎切换到渲染引擎执行对应操作，然后再切换回 JavaScript 引擎继续执行，这就带来了性能损耗。单次切换消耗的时间是非常少的，但是如果频繁地大量切换，那么就会产生性能问题

比如下面的测试代码，循环读取一百万次 DOM 中的 body 元素的耗时是读取 JSON 对象耗时的 10 倍。//
测试次数：一百万次const times=1000000//
缓存body元素console.time('object')let body=document.body//
循环赋值对象作为对照参考for(let i=0;i<times;i++){document.body.innerHTML+=html//
31.919921875msconsole.timeEnd('innerHTML')}

1) 商品详情页只循环读取一万次document.body，既没改样式也没增删节点，为什么仍可能明显慢于读取已缓存的普通对象？

答：瓶颈不一定来自重排，频繁访问DOM本身也需要在JavaScript执行与渲染相关能力之间反复交互。应在循环外缓存document.body或目标节点，循环内复用引用；但缓存节点只能减少重复查询，无法消除后续样式修改引起的渲染成本。

2) 数据看板循环处理一千个卡片，每轮先读offsetHeight，再写style.height，页面滚动时持续卡顿，你会怎样重构？

答：交替读写会让前一次修改影响布局，随后的尺寸读取可能迫使浏览器立即完成样式和布局计算。先批量读取高度并保存，再集中写入样式，必要时把写入安排到requestAnimationFrame；若每帧仍处理大量节点，还要减少参与渲染的元素数量。

3) 后台表格一次追加一万个，开发者在循环里反复appendChild，另一位坚持拼字符串后一次写入innerHTML，你如何判断？

答：两种写法的关键差异是提交给页面的次数，先在内存中完成构造再一次更新通常更符合批量操作原则。纯静态且内容可信时可一次赋值innerHTML，需要保留节点引用或逐项绑定行为时更适合离线创建后统一挂载；用户输入未经处理时不能直接拼入HTML。

4) 线上筛选器只改了文字颜色却出现闪烁，团队直接认定发生了重排，你会从哪里纠正并排查？

答：文字颜色变化通常只要求重绘，不能仅凭视觉闪烁断定发生重排；尺寸、边距、节点增删等才更可能改变元素排布。应在性能面板查看样式计算、布局与绘制记录，并定位同一交互中是否还读取了布局属性或修改了几何样式。

5) 消息列表从一千条增长到十万条，已经把节点创建改成一次批量插入，首屏仍然很慢，继续优化innerHTML还有意义吗？

答：批量插入只能减少操作次数，十万个节点仍会扩大样式计算、排布和绘制的工作量，继续微调字符串拼接通常不是核心矛盾。应分页或虚拟化，只保留视口附近节点；代价是滚动定位、动态高度和可访问性处理会更复杂。

【CSS基础】

568. 盒模型

盒模型从内到外由content、padding、border和margin组成，元素最终占用多大空间取决于box-sizing。默认的content-box中，声明的width只表示内容宽度，总宽度还要加上内边距、边框和外边距。使用border-box时，width已包含padding与border，因此总宽度只需再计算margin。如果设置为inherit，元素会沿用父元素的盒模型规则。 content（元素内容） + padding（内边距） + border（边框） + margin（外边距） 延伸：box-sizing content-box：默认值，总宽度

=margin+border+padding+width
border-box：盒子宽度包含padding和border，总宽度 = margin + width
inherit：从父元素继承box-sizing属性

1) 登录页输入框写了width: 200px; padding: 0 12px; border: 1px solid，验收量到的占用宽度却是226px，是哪一层计算错了？

答：默认content-box中的200px只表示内容区，左右内边距增加24px，左右边框再增加2px，所以边框盒宽度是226px。改为box-sizing: border-box后，内边距和边框被计入声明宽度，但外边距仍在盒子之外。

2) 组件库要求所有表单控件按设计稿总宽度布局，你会怎样设置box-sizing，又为什么要覆盖伪元素？

答：可在根元素声明box-sizing: border-box，再让普通元素及::before、::after通过inherit继承，统一宽度口径。伪元素也可能带内边距和边框，遗漏后会出现局部计算差异；接入依赖content-box的旧组件时需为其作用域显式恢复。

3) 一个width: 100%的侧栏在窄屏仍横向溢出，它还有左右padding、border和margin，切成border-box就一定能修好吗？

答：border-box只把内边距和边框纳入100%，左右外边距依然额外占据空间，因此仍可能超过父容器。应同时检查margin、父级可用宽度和内容最小尺寸；若内容本身不可收缩，单改盒模型也不会消除溢出。

4) 线上卡片声明width:

320px，开发者工具显示内容区不足320px，团队怀疑浏览器算错，你会核对哪些属性？

答：先看计算后的box-sizing；若为border-box，声明宽度代表边框盒，内容区会扣除左右padding与border。再检查规则是否由全局继承、组件样式或伪元素覆盖，并分别读取内容盒与边框盒尺寸，避免把不同测量口径混为一谈。

5) 设计系统在content-box和border-box之间选全局默认值，表单、栅格和第三方富文本组件同时存在，你怎么取舍？

答：自有组件按设计稿总宽度布局时，border-box能减少增加内边距或边框后尺寸意外膨胀，通常更易维护。切换全局默认值会改变既有组件的尺寸语义，应先做视觉回归，并给依赖content-box的第三方子树设置明确边界；盒模型统一不等于可以忽略外边距。

569. BFC

BFC是独立的块级布局区域，内部元素的布局不会直接影响外部元素。它可由根元素、浮动元素、绝对或固定定位，以及特定的display、非visible的overflow等方式触发。它不会与外部浮动区域重叠，计算高度时也会包含浮动子元素，所以常用于清除内部浮动或实现自适应两栏布局。需要注意，同一BFC内相邻块的垂直margin仍会重叠，要隔离它们需建立不同的BFC。块级格式化上下文，是一个独立的渲染区域，让处于BFC内部的元素与外部的元素相互隔离，使内外元素的定位不会相互影响。

IE下为Layout，可通过zoom:1触发 触发条件：根元素position: absolute/fixeddisplay: inline-block / tablefloat元素overflow != visible 规则：属于同一个BFC的两个相邻Box垂直排列属于同一个BFC的两个相邻Box的margin会发生重叠BFC中子元素的margin box的左边，与包含块 (BFC)border box的左边相接触 (子元素absolute除外)BFC的区域不会与float的元素区域重叠计算BFC的高度时，浮动子元素也参与计算文字层不会被浮动层覆盖，环绕于周围 应用：阻止margin重叠可以包含浮动元素——清除内部浮动(清除浮动的原理是两个div都位于同一个BFC区域之中)自适应两栏布局可以阻止元素被浮动元素覆盖

1) 订单页两个垂直卡片分别有margin-bottom: 20px和margin-top: 30px，实际间距只有30px，为什么简单相加不成立？

答：两个卡片处于同一BFC且垂直相邻时，上下外边距会发生重叠，因此不能按50px计算。工程上优先让间距只由一侧负责，或用父级布局统一控制；额外创建BFC虽能隔离重叠，却会改变布局边界。

2) 资讯流容器内部全是浮动卡片，父容器高度变成零，后面的页脚顶了上来，你会如何让父级重新包住内容？

答：让父容器建立BFC后，计算其高度时会把浮动子元素纳入，从而恢复对页脚的正常推开。可根据约束选择display:

flow-root，旧代码也常见非visible的overflow；后者可能裁剪阴影、菜单或其他溢出内容。

3) 两栏页面左侧头像float:

left，右侧正文背景却伸到头像下面，产品要求正文区域始终不与头像重叠，你会改哪一层？

答：可让右侧正文建立新的BFC，其区域不会与浮动元素的区域重叠，从而形成自适应剩余宽度的两栏效果。触发方式会附带不同布局语义，例如overflow: hidden可能裁剪内容，position:

absolute又会脱离常规文档流，不能只为触发而盲选。

4) 线上下拉菜单被截断，排查发现祖先为清除内部浮动设置了overflow: hidden，怎样确认并修复而不让容器再次塌陷？

答：先临时关闭该祖先的overflow，确认菜单恢复且父容器高度随之丢失，就能验证裁剪与包含浮动来自同一规则。可改用display: flow-root，或使用清除浮动的伪元素方案；若必须兼容特定旧环境，则需结合兼容范围选择替代实现。

5) 同事提议给所有模块都加overflow: hidden来隔离布局，你会接受这种全局策略吗？

答：不应把overflow: hidden当作无成本的布局重置，它虽然能建立BFC，也会改变溢出内容的可见性。只在确实需要包含浮动、阻止外边距重叠或避开浮动区域的局部容器使用，并优先选择副作用符合需求的触发方式。

570. 层叠上下文

z-index只在同一个层叠上下文中比较，并不是数值越大全局层级就越高。根元素会创建层叠上下文，定位元素以及带有transform、opacity、filter等属性的元素也可能形成新的上下文。子元素的z-index再大，也无法直接越过父级上下文与外部元素竞争。遇到弹窗被遮挡时，我一般先检查祖先元素是否创建了独立层叠上下文。元素提升为一个比较特殊的图层，在三维空间中（z轴）高出普通元素一等。触发条件根层叠上下文(html)positioncss3属性flextransformopacityfilterwill-changewebkit-overflow-scrolling flextransformopacityfilterwill-changewebkit-overflow-scrolling 层叠等级：层叠上下文在z轴上的排序在同一层叠上下文中，层叠等级才有意义z-index的优先级最高

1) 支付弹窗设置了position: fixed; z-index: 9999，仍被导航栏的z-index: 10盖住，为什么数值更大却输了？

答：z-index只在所属层叠上下文内比较，弹窗若被包在较低层级的祖先上下文中，9999无法越过祖先与导航栏竞争。应检查祖先是否由transform、opacity、filter或will-change等创建上下文，通常把弹窗挂到顶层容器更稳妥。

2) 动画组件为了启用位移给外壳加了transform:

translateZ(0)，结果内部提示气泡被相邻卡片遮住，你会怎样落地修复？

答：外壳的transform会创建新的层叠上下文，气泡的层级被限制在该上下文内，提高气泡自身的z-index未必有效。可在动画结束后移除不再需要的transform，或把气泡通过Portal挂到更高层；迁移后还要重新处理定位参照和滚动跟随。

3) 设计师要求半透明容器使用opacity:

.9, 同时要求里面的下拉层越过页面顶栏, 这两个约束冲突时怎么处理?

答: 容器的opacity会形成层叠上下文, 内部下拉层无法仅靠增大z-index跨越外部顶栏。可把透明效果放到独立背景层, 避免作用于承载下拉层的祖先, 或将下拉层移到顶层; 后者会增加坐标换算、焦点管理和事件边界成本。

4) 线上只有某个活动页的浮层被遮挡, 代码里已经写了z-index, 你会如何快速找到真正限制它的祖先?

答: 从浮层开始逐级检查祖先的计算样式, 重点查看定位与z-index, 以及transform、opacity、filter、will-change等可能创建层叠上下文的属性。再对照兄弟区域所属的上下文逐层比较, 而不是只看两个元素的数值; 临时关闭可疑规则可辅助验证。

5) 团队准备建立全站z-index数值表, 有人主张弹窗统一用999999, 这套方案能彻底避免遮挡吗?

答: 统一层级令牌只能规范同一层叠上下文内的排序, 无法突破祖先上下文的边界, 因此极大数值不能根治遮挡。更可靠的方案是规定弹层挂载位置和顶层上下文结构, 再为遮罩、弹窗、提示等分配有限层级; 集中挂载也会带来定位与可访问性管理责任。

571. 左右居中方案

左右居中要根据元素类型选择: 行内元素用text-align: center, 定宽块用margin: 0 auto, 不定宽元素可用定位配合位移。text-align应写在父元素上, 因为它控制的是内部行内内容的对齐。块级元素宽度确定后, 左右margin设为auto就能平分剩余空间。如果元素宽度未知, 可以用left: 50%加transform:

translateX(-50%), 避免手算负边距。

行内元素:	text-align:	center	定宽块状元素:
左右margin值为auto	不定宽块状元素:	table布局, position	+ transform /* 方案1
/.wrap{text-align:center}.center{display:inline;/ or *//* display: inline-block; *//* 方案2			
/.center{width:100px;margin:0 auto;}/ 方案2			
*/.wrap{position:relative;}.center{position:absolute;left:50%;transform:translateX(-50%);}			

1) 工具栏里的提示文字是行内元素, 开发者给它写了margin: 0 auto却没有居中, 页面结构不允许改, 你会改哪条规则?

答: 行内内容应由包含块设置text-align: center, margin: 0 auto不是这种场景的居中机制。该规则会同时影响容器内其他行内内容, 因此若工具栏还有左对齐按钮, 需要增加独立包装层或缩小样式作用范围。

2) 设置页有一个明确为320px宽的块级表单, 需要在父容器中水平居中, 团队在flex和自动外边距之间争论, 你选哪个?

答: 仅需居中一个定宽块时, 给表单设置width: 320px; margin: 0 auto就足够, 语义直接且不改变父级布局模式。若父级还要统一排列多个子项, flex才更有价值; 固定宽度在窄屏可能溢出, 还需配合响应式宽度约束。

3) 搜索页的按钮组宽度随文案和语言变化, 不能预先写死, 父容器又不能改成flex, 怎样保证它始终水平居中?

答: 可让按钮组采用收缩到内容宽度的布局方式, 再使用自动外边距; 也可在相对定位父级中用left: 50%配合transform: translateX(-50%)。绝对定位方案会脱离常规文档流, 父级高度和相邻内容不能再依赖它自然撑开。

4) 角标使用position: absolute; left:

50%后总是向右偏半个自身宽度, 设计稿宽度还会动态变化, 你会怎么修?

答：left: 50%只把角标左边缘放到父容器中线，因此视觉中心必然偏右。补上transform: translateX(-50%)，位移百分比按元素自身宽度计算，可适配动态宽度；同时要确认父级提供了正确的定位包含块。

5) 弹层标题用position + transform做水平居中后，内部提示层的z-index突然压不过相邻区域，你如何权衡居中方案？

答：用于居中的transform会创建层叠上下文，内部提示层可能因此无法跨越外部上下文。若标题只是普通流内内容，可改用text-align、自动外边距或父级flex居中；若必须绝对定位，则应调整提示层挂载位置，而不是无限增大其z-index。

572. 上下垂直居中方案

垂直居中主要看元素高度是否已知：定高可用外边距或定位加负边距，不定高更适合transform、flex或vertical-align。已知高度时，把元素移动到top: 50%后再回退自身高度的一半即可。高度未知时，transform: translateY(-50%)不依赖具体尺寸，父元素使用display: flex和align-items: center也更直接。兼容传统行内布局时，还可以借助inline-block和vertical-align: middle。

定高：margin，position + margin(负值)不定高：position+transform，flex，IFC + vertical-align: middle

```
/* 定高方案1 */.center{height:100px;margin:50px 0;}/* 定高方案2 */.center{height:100px;position:absolute;top:50%;margin-top:-25px;}/* 不定高方案1 */.center{position:absolute;top:50%;transform:translateY(-50%);}/* 不定高方案2 */.wrap{display:flex;align-items:center;}.center{width:100%;}/* 不定高方案3 *//* 设置 inline-block 则会在外层产生 IFC，高度设为 100% 撑开 wrap 的高度 */.wrap::before{content:"";height:100%;display:inline-block;vertical-align:middle;}.wrap{text-align:center;}.center{display:inline-block;vertical-align:middle;}
```

1) 活动页把一段可能换成三行的文案写成固定height，再用上下margin居中；运营改字后文案偏到下方，你会保留这套写法吗？

答：不会保留依赖定高的margin方案，因为内容高度变化后，原先计算的上下间距不再成立。应改用父容器display: flex; align-items: center，或用绝对定位配合top: 50%、transform: translateY(-50%)；前者更适合正常文档流。

2) 弹窗主体高度由接口返回内容决定，现有代码是position: absolute; top: 50%; margin-top: -150px，短内容正常、长内容上移，你会怎样改？

答：把固定负margin改成transform: translateY(-50%)，位移比例会依据元素自身实际高度计算，不需要预先知道高度。若弹窗还要水平居中，可同时使用left: 50%和translate(-50%, -50%)；内容超过视口时仍需另设最大高度和滚动策略。

3) 一个老专题页不能改浮动和定位结构，父容器高度固定，要求让未知高度的提示块垂直居中，你会怎样用inline-block与vertical-align落地？

答：可让父容器的::before成为高度100%的inline-block，再让它和提示块都设置vertical-align: middle。这个伪元素负责撑出可对齐的行内格式化上下文，提示块也必须是inline-block；空白字符和基线间隙可能影响细节，需要检查模板换行。

4) 列表空状态用了display: flex; align-items:

center，但上线后仍贴着容器顶部；检查发现父节点没有可用高度，你会从哪里排查？

答：先确认执行居中的.wrap是否真的具有明确高度或被上级布局拉伸，align-items: center只能在已有的交叉轴剩余空间内分配位置。再检查样式是否被覆盖以及空状态是否为该容器的直接子项；父容器高度随内容收缩时，换任何居中语法都看不出效果。

5) 表单行高度固定为100px，设计坚持用负margin居中，开发希望统一换成flex；你会如何取舍？

答：固定高度且子项高度也稳定时，position配合负margin能成立，但尺寸一改就必须同步更新偏移量。组件会复用或内容可能变化时更适合flex，只需在父项设置align-items:center；若绝对定位是为了脱离文档流，则保留定位并改用transform更稳妥。

573. 选择器权重计算方式

选择器权重可以按ID、类及属性、元素三个层级比较，先比较高层级，完全相同时后写的规则覆盖先写的。!important的优先级高于普通声明，元素的内联style也高于一般的外部规则。通配符以及后代、兄弟等关系符本身不会带来更高权重，继承样式通常也排在直接指定的规则之后。选择器匹配通常从右向左解析，这样能更早过滤不符合条件的元素。!important > 内联样式 = 外联样式 > ID选择器 > 类选择器 = 伪类选择器 = 属性选择器 > 元素选择器 = 伪元素选择器 > 通配选择器 = 后代选择器 = 兄弟选择器 属性后面加!import会覆盖页面内任何位置定义的元素样式作为style属性写在元素内的样式id选择器类选择器标签选择器通配符选择器(*) 浏览器自定义或继承 同一级别：后写的会覆盖先写的 css选择器的解析原则：选择器定位DOM元素是从右往左的方向，这样可以尽早的过滤掉一些不必要的样式规则和元素

1) 组件页中.dialog .title写在文件前面，后面的h2仍然覆盖不了字号；同事认为“后写一定赢”，你怎么指出判断漏洞？

答：后写覆盖先写只发生在同一级权重下，.dialog .title含两个类选择器，权重高于单个元素选择器h2。应先比较!important、内联样式、ID、类或伪类或属性、元素或伪元素等层级，再比较源码顺序；继承值也不能击败直接命中的声明。

2) 线上主题样式盖不住节点上的style="color:red"，维护者准备给主题选择器再加三个类名，你会接受吗？

答：继续堆类名通常仍盖不过内联样式，因为内联声明的优先级高于普通的ID、类和元素选择器。应优先移除或改造内联样式的生成位置；若确实无法控制且业务必须覆盖，只能谨慎使用!important，代价是后续覆盖会更困难。

3) 表格有一万行，样式规则写成.page .content table tbody tr td.warning；评审既担心权重失控，也担心匹配路径过长，你会怎样调整？

答：把规则收敛为靠近目标节点的稳定类名，例如.warning-cell，既减少无意义的祖先约束，也避免累积过高的类和元素权重。浏览器定位选择器时按从右向左匹配，右端应尽量有明确目标；是否形成可见性能问题仍应以页面测量为准，不能仅凭行数断言。

4) 灰度发布后只有一部分按钮变色，开发工具里两条规则都是.panel .button，一条来自主题包、一条来自业务包；你怎么定位是谁赢了？

答：两条选择器权重相同，应查看最终加载后的源码顺序，后出现的声明会覆盖先出现的声明。还要核对属性是否带!important、元素是否存在内联样式，以及选择器是否确实命中同一节点；构建产物的样式注入顺序变化也会改变结果。

5) 设计系统想让业务方容易覆盖默认样式，但业务团队又要求禁止!important；你会怎样制定选择器约束？

答：默认规则应使用短而稳定的类选择器，避免叠加ID、多层祖先和标签来制造不必要的权重。业务覆盖时保持同级或略高权重，并依靠明确的加载顺序；这样符合层叠规则，也降低双方不断加选择器的对抗成本，但仍需统一样式入口顺序。

574. 清除浮动

清除浮动的目的，是让父元素能够正常包住浮动的子元素。可以在浮动元素后添加带有clear:

both的空div，但这会引入没有实际语义的标签。也可以给父元素设置overflow:

hidden或auto来触发BFC。我一般更倾向于使用clearfix伪元素，因为它同样通过clear:

both生效，又能保持文档结构清晰；兼容IE6时再补zoom:

1。

在浮动元素后面添加clear:both的空div元素 给父元素添加overflow:hidden或者auto样式，触发BFC
.container{width:300px;background-color:#aaa;overflow:hidden;zoom:1;/*IE6*/}

使用伪元素，也是在元素末尾添加一个点并带有clear: both属性的元素实现的。 .clearfix{zoom:1;/*IE6*/}
.clearfix:after{content:".";height:0;clear:both;display:block;visibility:hidden;}

推荐使用第三种方法，不会在页面新增div，文档结构更加清晰

1) 资讯页左右两栏都设了float，父容器背景高度却变成0，有人只给父容器补一个固定高度；你会同意吗？

答：不同意用固定高度掩盖塌陷，因为浮动子项脱离普通文档流，内容增加后仍会溢出。应在末尾清除浮动，或让父元素通过overflow:

hidden、auto形成BFC；固定高度只适合内容尺寸被严格约束的少数场景。

2) 老后台几十个列表都存在父级塌陷，模板团队不允许在每个列表末尾插入空div，你会怎样统一修复？

答：给这些父容器统一挂.clearfix，使用::after生成块级伪元素并设置clear: both，即可在末尾清除浮动。它不新增无语义的真实节点，文档结构更清晰；若仍需兼容非常旧的环境，源码中的zoom:

1属于针对旧版IE的兼容处理。

3) 卡片父级用overflow:

hidden清除浮动后，子元素阴影被截断；结构不能加真实节点时，你会怎么改？

答：应撤掉用于清浮动的overflow: hidden，改用父元素的::after配合content、display: block和clear: both。overflow方案通过形成BFC包含浮动，但同时会裁剪超出边界的视觉内容；伪元素法更适合需要保留阴影或外溢效果的容器。

4) 线上页脚压到浮动商品列表上方，开发工具显示.clearfix:after存在却没有撑开父级；你会逐项检查哪些声明？

答：先检查伪元素是否实际生成，content缺失时::after不会产生可用于清除的盒子。再确认它具有display: block和clear: both，并排除规则被覆盖；若清除节点不在浮动元素所属的同一父容器末尾，也无法修复目标父级的高度。

5) 维护者在“空div清除”“父级overflow: auto”“伪元素清除”之间争论，页面还要求结构简洁且不能出现额外滚动条，你会选哪一个？

答：优先选伪元素清除，因为它不增加真实div，也不会像overflow: auto那样可能引入滚动或改变溢出表现。空节点法直观但污染结构，overflow法代码短却附带裁剪或滚动语义；只有这些副作用符合容器需求时才应采用后两者。

575. 左边定宽，右边自适应方案

左边定宽、右边自适应，可以让左侧设置固定宽度并左浮动，再由右侧占据剩余空间。常见写法是左侧width: 120px、float: left，右侧通过margin-left:

120px留出位置。另一种方式是让两边都浮动，并把右侧宽度写成calc(100%

120px)。前一种写法更直接，后一种适合确实需要明确计算右侧宽度的场景。 float + margin, float +

calc /* 方案1 */.left{width:120px;float:left;}.right{margin-left:120px;}/* 方案2
*/.left{width:120px;float:left;}.right{width:calc(100% - 120px);float:left;}

1) 管理平台左栏固定120px并浮动，右栏也写了width:

100%，结果被挤到下一行；同事说右栏本来就是“占满剩余空间”，错在哪里？

答：width: 100%按包含块宽度计算，并不会自动扣除左侧浮动的120px，两栏总宽度因此可能超过容器。可让右栏不设宽度而使用margin-left: 120px，或把右栏宽度明确写成calc(100% - 120px)并配合浮动。

2) 老门户页必须保留float，侧栏固定120px，右侧文章要随窗口伸缩；你会怎样写最少且稳定的布局规则？

答：让左栏设置width: 120px; float: left，右栏设置margin-left: 120px，即可把右侧可用区域留出来。右栏宽度由普通块级布局自动填充，通常不必再计算；若左右间还要留缝，应把间距一并计入margin-left。

3) 产品把侧栏从120px改成可配置值，但现有float + calc在多处各写了一遍宽度；你会怎样避免两边配置失配？

答：左栏宽度和右栏扣减值必须来自同一个尺寸约束，否则任一处漏改都会产生空隙或换行。可统一维护该值，再用于左栏width与右栏的calc(100% - 固定值)；如果无需显式控制右栏宽度，改用等值margin-left会减少一处计算。

4) 发布后右栏偶尔掉到左栏下方，只有容器加了边框和内边距的页面复现；你会怎样排查calc(100% - 120px)？

答：先检查两栏最终占用宽度是否还包含边框、内边距或额外外边距，因为这些尺寸可能让总和超过父容器。再核对两栏是否都成功浮动、calc是否被覆盖，以及父级可用宽度；必要时调整盒模型或把额外间距纳入计算，避免临界宽度换行。

5) 代码评审中，一方主张右栏用margin-left: 120px，另一方坚持float + calc；右栏只是普通文章流，你会选哪套？

答：普通文章流优先用左栏浮动、右栏等值margin-left，它不要求右栏也浮动，规则更少且能自然占据剩余宽度。float + calc适合确实需要明确右栏宽度或两栏都参与浮动的结构，但尺寸、间距和盒模型必须同步计算，维护成本更高。

576. 左右两边定宽，中间自适应

左右两边定宽、中间自适应，可以用浮动配合外边距，也可以直接用flex。浮动方案让左右分别靠两侧排列，中间通过margin: 0 120px留出空间；需要明确宽度时，也能用calc(100% - 240px)计算。更简洁的写法是父容器设置display: flex，左右固定为120px，中间设置flex: 1。具体选择取决于现有布局方式，维护旧浮动页面时不必强行改写。

圣杯布局（设置BFC，margin负值法），flex .wrap{width:100%;height:200px;}.wrap > div{height:100%;}/*

/.left{width:120px;float:left;}.right{float:right;width:120px;}.center{margin:0 120px;}/ 方案2

/.left{width:120px;float:left;}.right{float:right;width:120px;}.center{width:calc(100% - 240px);margin-left:120px;}/ 方案3

*/.wrap{display:flex;}.left{width:120px;}.right{width:120px;}.center{flex:1;}

1) 直播页左右栏各120px，中间栏写了width:

100%，上线后被挤到下一行；负责人认为左右浮动不会占普通流宽度，你怎么解释并修正？

答：浮动虽脱离普通文档流，但中间栏的100%仍按父容器完整宽度计算，无法与两侧共同容纳。可让左右分别float: left、float: right，中间使用margin: 0 120px；也可显式计算中间宽度，但必须把两侧总宽扣除。

2) 新闻详情页要求左右广告栏固定120px，中间正文自适应，现有float结构不能重写；你会给出什么落地代码？

答：左栏设width: 120px; float: left，右栏设width: 120px; float: right，中间正文设margin: 0 120px。中间不必固定宽度，会由块级布局占据扣除外边距后的空间；还要保证页面中的排列和清浮动规则与现有结构相容。

3) 设计把右栏改为160px、左栏仍为120px，原来的.center { margin: 0 120px }出现遮挡，你会怎样调整而不是继续套对称值？

答：中间栏应改为margin-left: 120px; margin-right: 160px，分别为两侧保留实际宽度。若采用calc，中间宽度也要扣除总计280px，不能沿用100% - 240px；侧栏还有额外间距时同样必须纳入约束。

4) 一个三栏页面在窄窗口下中间栏消失，开发工具显示.center { width: calc(100% - 240px); margin-left: 120px }，你会先查哪些尺寸关系？

答：先确认父容器实际可用宽度是否小于两侧合计240px，以及边框、内边距、外边距是否使三栏总占用继续增加。再检查左右浮动方向和中间宽度是否被其他规则覆盖；该方案没有为极窄容器定义降级行为，必要时必须另设最小宽度或重排布局。

5) 新建后台的三栏框架不受旧浏览器约束，团队在float + margin、float + calc和flex之间选型，你会推荐哪一种？

答：推荐父容器使用display: flex，左右栏固定width: 120px，中间栏设flex: 1，约束关系直接且无需负担浮动清除。旧结构难改时，float + margin比显式calc少一层宽度计算；若中间内容存在不可收缩项，仍需另外处理溢出风险。

6) 三栏容器改成flex后，中间长表格把右侧固定栏顶出屏幕；这和“中间flex: 1自适应”是否矛盾，你怎么排查？

答：不矛盾，flex: 1只声明中间项分配剩余空间，内部内容仍可能提出更大的最小尺寸需求。应检查长表格或不换行内容的溢出约束，并确保左右固定栏没有被允许意外收缩；浮动方案同样需要面对超长内容，只是故障表现可能不同。

577. CSS动画和过渡

transition适合属性状态变化时的平滑过渡，animation配合@keyframes则能描述完整的多阶段动画。transition主要控制目标属性、持续时间、速度曲线和延迟；animation还可以设置播放次数、方向及结束后的状态，并通过animationend监听完成。关键帧既能写成from到to，也能用百分比安排多个阶段。实现位移、旋转或缩放时，我会优先使用transform，简单显隐可用opacity，通常更适合连续动画。

animation / keyframes animation-name: 动画名称，对应@keyframes animation-duration: 间隔 animation-timing-function: 曲线 animation-delay: 延迟 animation-iteration-count: 次数 infinite: 循环动画 animation-direction: 方向 alternate: 反向播放 animation-fill-mode: 静止模式 forwards: 停止时，保留最后一帧 backwards: 停止时，回到第一帧 both: 同时运用 forwards / backwards 常用钩子: animationend infinite: 循环动画 alternate: 反向播放 forwards: 停止时，保留最后一帧 backwards: 停止时，回到第一帧 both: 同时运用 forwards / backwards 动画属性: 尽量使用动画属性进行动画，能拥有较好的性能表现 translate scale rotate skew opacity color transform 位移属性 translate(x, y) 旋转属性 rotate() 缩放属性 scale() 倾斜属性 skew() transition transition-property (过渡的属性的名称)。transition-duration (定义过渡效果花费的时间,默认是0)。transition-timing-function: linear(匀速) ease(慢速开始，然后变快，然后慢速结束) (规定过渡效果的时间曲线，最常用的是这两个)。transition-delay (规定过渡效果何时开始。默认是 0)

般情况下，我们都是写一起的，比如：transition: width 2s ease 1s 关键帧动画 animation

一个关键帧动画，最少包含两部分，animation

属性及属性值（动画的名称和运行方式运行时间等）。@keyframes（规定动画的具体实现过程）

animation 属性可以拆分为 animation-name规定@keyframes 动画的名称。animation-duration规定动画完成一个周期所花费的秒或毫秒。默认是0。animation-timing-function规定动画的速度曲线。默认是

“ease”，常用的还有linear，同transition。animation-delay规定动画何时开始。默认是

0。animation-iteration-count规定动画被播放的次数。默认是 1，但我们一般用infinite，一直播放 而@keyframes的使用方法，可以是from->to（等同于0%和100%），也可以是从0%->100%之间任意个的分层设置。我们通过下面一个稍微复杂点的demo来看一下，基本上用到了上面说到的大部分知识

eg:@keyframesmymove{from{top:0px;}to{top:200px;}}/* 等同于： */@keyframesmymove{0%{top:0px;}25%{top:200px;}50%{top:100px;}75%{top:200px;}100%{top:0px;}} 用css3动画使一个图片旋转
loader{display:block;position:relative;-webkit-animation:spin 2s linear infinite;animation:spin 2s linear infinite;}@-webkit-keyframesspin{0%{-webkit-transform:rotate(0deg);-ms-transform:rotate(0deg);transform:rotate(0deg);}100%{-webkit-transform:rotate(360deg);-ms-transform:rotate(360deg);transform:rotate(360deg);}}@keyframesspin{0%{-webkit-transform:rotate(0deg);...

1) 商品卡片从列表左侧滑入时，开发写了transition: left 300ms

ease，滚动过程中位移不连贯；只依据这些样式，你会先改哪一处？

答：先把位移从left改为transform: translateX()，并让transition-property指向transform。资料明确把translate、scale、rotate、skew、opacity列为更适合动画的属性，但仅凭样式不能断言卡顿一定来自布局计算，仍需结合页面负载排查。

2) 设计稿要求加载图标持续匀速旋转，另一位开发却用悬停触发的transition实现；这套方案为什么无法直接满足要求，你会怎么写？

答：持续旋转应使用animation配合@keyframes，例如从transform:

rotate(0deg)变化到rotate(360deg)，并设置linear infinite。transition依赖属性状态发生变化，本身不负责循环播放；代价是无限动画会持续运行，离开可见区域后是否暂停需要另行设计。

3) 弹窗打开后要分三段完成上移、轻微回落和停稳，产品又要求关闭时反向播放；你会怎样组织关键帧和播放方向？

答：用百分比关键帧描述三个阶段，并通过animation-duration与animation-timing-function控制整体节奏；需要往返播放时可设置animation-direction: alternate。alternate会按迭代轮次反向执行，不等于于任意时刻点击关闭就从当前进度倒放，交互中断仍需额外状态控制。

4) 线上反馈横幅动画结束后突然跳回初始位置，代码里@keyframes的100%已经是目标位置；你会优先检查哪个属性？

答：优先检查animation-fill-mode，若要结束后保留最后一帧，应使用forwards，需要同时处理开始前与结束后的样式可用both。它只影响动画执行区间外采用哪一帧，不会把关键帧结果永久写回元素原始样式，后续状态切换仍可能覆盖视觉结果。

5) 表单校验提示只需在错误出现时淡入，开发在transition和animation之间争执；你会依据什么选型？

答：若提示只是从既有状态平滑过渡到错误状态，使用transition配置属性、时长、曲线和延迟更直接。若要安排多个阶段或独立循环，则animation与@keyframes更合适；两者都应优先选择资料列出的动画属性，避免为了统一写法引入不必要的关键帧。

578. CSS3的新特性

CSS3常见的新特性可以归为视觉效果、盒模型与响应式能力、变换动画三类。视觉上有圆角、阴影和渐变，布局相关可用box-sizing，响应式页面则依赖@media。交互效果通常由transform、transition和animation完成，其中transition需要状态变化触发，animation可以按关键帧自行运行。文本场景还可以用word-break和text-overflow处理换行与溢出。transition：过渡transform: 旋转、缩放、移动或倾斜animation: 动画gradient: 渐变box-shadow: 阴影border-radius: 圆角word-break:normal|break-all|keep-all; 文字换行(默认规则|单词也可以换行|只在半角空格或连字符换行)text-overflow: 文字超出部分处理text-shadow: 水平阴影，垂直阴影，模糊的距离，以及阴影的颜色。box-sizing:content-box|border-box盒模型媒体查询@media screen and (max-width: 960px) {}还有打印print

1) 资讯卡片标题限定一行，内容超出后产品要求显示省略号；开发只加了text-overflow: ellipsis，页面仍把整段文字画出来，你会怎么判断？

答：text-overflow只规定溢出内容的呈现方式，单独设置通常不足以形成可见截断，还要确保元素确实产生受限宽度和溢出状态。资料只列出了该特性的用途，未给出完整组合，因此不能把失败归因于浏览器版本，应继续核对换行与溢出相关样式。

2) 英文订单号把移动端表格撑出屏幕，运营又要求普通英文句子尽量按单词换行；word-break应怎样选，风险是什么？

答：普通文本优先保留normal的默认换行规则；若订单号这类长串必须在任意字符处断开，可针对该字段使用break-all。keep-all只允许在半角空格或连字符等位置换行，不适合无分隔符长串；全局使用break-all则可能破坏普通单词的可读性。

3) 设计系统规定按钮视觉宽度为120px，组件同时有边框和内边距，接入方发现最终占位超过120px；你会改哪个盒模型配置？

答：可将组件设为box-sizing: border-box，让声明宽度包含内容区、内边距和边框，更容易维持设计稿给出的外部尺寸。content-box下声明宽度只约束内容区，额外边框和内边距会扩大盒子；切换盒模型后，依赖原内容区宽度的子元素可能需要重新校验。

4) 打印订单页时，网页导航和操作按钮也出现在纸面上；团队想再复制一套打印页面，你会先提出什么方案？

答：先使用@media print为打印介质覆盖样式，隐藏导航与操作区，并调整正文布局，而不是立即维护第二套页面。媒体查询也能按屏幕条件应用规则，例如max-width；但资料没有覆盖所有打印兼容细节，分页、背景和纸张尺寸仍应在目标浏览器实际验证。

5) 营销页需要圆角卡片、阴影、渐变背景和悬停放大，开发准备全部导出成图片；你会保留哪些为CSS，实现时注意什么？

答：圆角、盒阴影、渐变以及悬停缩放都可分别用border-radius、box-shadow、gradient和transform: scale()表达，通常无需把整块视觉固化成图片。这样便于尺寸和状态变化，但复杂视觉是否能完全还原仍取决于设计要求，不能仅凭特性清单承诺一致效果。

579. 列举几个css中可继承和不可继承的元素

通常，控制文字表现的属性更容易继承，而控制盒子尺寸、位置和布局的属性通常不会继承。例如color、font-family、line-height、text-align可以从父元素传给后代，visibility和cursor也可继承。相对地，display、margin、padding、width、position等不会自动继承，否则父级布局很容易意外影响子元素。遇到边界情况我会查看属性定义，而不是只靠这条经验判断。不可继承的：display、margin、border、padding、background、height、min-height、max-height、width、min-width、max-width、overflow、position、left、right、top、bottom、z-index、float、clear、table-layout、vertical-align所有元素可继承：visibility和cursor。内联元素可继承：letter-spacing、word-spacing、white-space、line-height

、color、font、font-family、font-size、font-style、font-variant、font-weight、text-decoration、text-transform、direction。终端块状元素可继承：text-indent和text-align。列表元素可继承：list-style、list-style-type、list-style-position、list-style-image。transition和animation的区别 Animation和transition大部分属性是相同的，他们都是随时间改变元素的属性值，他们的主要区别是transition需要触发一个事件才能改变属性，而animation不需要触发任何事件的情况下才会随时间改变属性值，并且transition为2帧，从from to，而animation可以一帧一帧的 ---

【浏览器】

580. 浏览器架构

现代浏览器采用多进程架构，把浏览器界面、页面渲染、网络、GPU和插件等职责隔离开。浏览器主进程负责界面、交互和子进程管理，渲染进程运行Blink与V8，把HTML、CSS和JavaScript转成可交互页面。进程隔离后，某个页面或插件崩溃通常不会拖垮整个浏览器，沙箱也能限制恶意程序获取系统权限。代价是进程会占用独立资源，但能换来更好的稳定性、安全性和多核利用。

单进程浏览器时代 单进程浏览器是指浏览器的所有功能模块都是运行在同一个进程里，这些模块包含了网络、插件、JavaScript运行环境、渲染引擎和页面等。其实早在2007年之前，市面上浏览器都是单进程的 缺点不稳定：一个插件的意外崩溃会引起整个浏览器的崩溃不流畅：所有页面的渲染模块、JavaScript执行环境以及插件都是运行在同一个线程中的，这就意味着同一时刻只能有一个模块可以执行不安全：可以通过浏览器的漏洞来获取系统权限，这些脚本获取系统权限之后也可以对你的电脑做一些恶意的事情，同样也会引发安全问题以上这些就是当时浏览器的特点，不稳定，不流畅，而且不安全

不稳定：一个插件的意外崩溃会引起整个浏览器的崩溃不流畅：所有页面的渲染模块、JavaScript执行环境以及插件都是运行在同一个线程中的，这就意味着同一时刻只能有一个模块可以执行不安全：可以通过浏览器的漏洞来获取系统权限，这些脚本获取系统权限之后也可以对你的电脑做一些恶意的事情，同样也会引发安全问题

多进程浏览器时代 由于进程是相互隔离的，所以当页面或者插件崩溃时，影响到的仅仅是当前的页面进程或者插件进程，并不会影响到浏览器和其他页面，这就完美地解决了页面或者插件的崩溃会导致整个浏览器崩溃，也就是不稳定的问题JavaScript也是运行在渲染进程中的，所以即使JavaScript阻塞了渲染进程，影响到的也只是当前的渲染页面，而并不会影响浏览器和其他页面，因为其他页面的脚本是运行在它们自己的渲染进程中的Chrome把插件进程和渲染进程锁在沙箱里面，这样即使在渲染进程或者插件进程里面执行了恶意程序，恶意程序也无法突破沙箱去获取系统权限。

最新的Chrome浏览器包括：1个浏览器（Browser）主进程、1个 GPU进程、1个网络（NetWork）进程、多个渲染进程和多个插件进程

浏览器进程。主要负责界面显示、用户交互、子进程管理，同时提供存储等功能。渲染进程。核心任务是将HTML、CSS和 JavaScript 转换为用户可以与之交互的网页，排版引擎Blink和JavaScript引擎V8都是运行在该进程中，默认情况下，Chrome会为每个Tab标签创建一个渲染进程。出于安全考虑，渲染进程都是运行在沙箱模式下。GPU进程。其实，Chrome刚开始发布的时候是没有GPU进程的。而GPU的使用初衷是为了实现3D CSS的效果，只是随后网页、Chrome的UI界面都选择采用GPU来绘制，这使得GPU成为浏览器普遍的需求。最后，Chrome在其多进程架构上也引入了GPU进程。网络进程。主要负责页面的网络资源加载，之前是作为一个模块运行在浏览器进程里面的，直至最近才独立出来，成为一个单独的进程。插件进程。主要是负责插件的运行，因插件易崩溃，所以需要通过插件进程来隔离，以保证插件进程崩溃不会对浏览器和页面造成影响

1) 客服说一个页面里的插件崩溃后整个浏览器都应该退出，因为“插件属于浏览器”；结合现代多进程架构，你会怎样反驳？

答：现代浏览器会把插件和页面等职责放入相互隔离的进程，插件崩溃通常只影响对应插件进程，不应必然带走浏览器和其他页面。单进程时代才容易因一个模块异常导致整体崩溃；进程隔离降低故障范围，但不能据此保证所有异常都绝不扩散。

2) 后台报表执行一段很长的JavaScript后当前标签页失去响应，但地址栏和其他页面仍可操作；这与哪些进程职责吻合？

答：JavaScript引擎V8与排版引擎Blink都运行在渲染进程中，长脚本阻塞该进程时，主要影响当前页面的执行和渲染。浏览器主进程负责界面、交互和子进程管理，其他页面通常还有各自的渲染进程；具体进程分配策略不宜由现象单独推断。

3) 安全评审认为把渲染进程拆开只是为了防崩溃，与恶意网页获取系统权限无关；你会指出哪处判断有问题？

答：多进程设计还配合沙箱限制渲染进程和插件进程访问系统资源，即使其中执行恶意程序，也更难直接突破边界取得系统权限。进程隔离解决的是影响范围，沙箱解决的是权限约束，两者目标相关但并不等价；沙箱也不是可以忽略浏览器漏洞的绝对安全保证。

4) 页面能显示但图片和脚本长期加载失败，另一个标签页的界面交互仍正常；排查时你会优先区分哪些进程？

答：应优先区分负责资源加载的网络进程、负责页面转换与交互的渲染进程，以及管理界面和子进程的浏览器主进程。资源普遍加载异常更应关注网络链路或网络进程，单页解析执行异常则偏向渲染进程；仅凭一个标签页现象仍不足以锁定故障点。

5) 团队讨论是否把所有浏览器能力塞回单进程以节省开销，稳定性、安全性和流畅性之间该怎么取舍？

答：单进程减少了进程边界，却会让插件、页面、网络和脚本故障更容易牵连整个浏览器，也难以借助隔离与沙箱控制影响范围。多进程的价值是稳定性、安全性及利用多核能力，代价是架构和资源管理更复杂；资料没有给出内存数据，不能虚构量化收益。

581. JavaScript单线程模型

JavaScript单线程指同一时刻只有一个调用链在执行，并不代表浏览器只有一个线程。这样设计能避免多个线程同时修改DOM时产生冲突，但长时间运行的脚本也会阻塞页面渲染。异步任务会进入任务队列，等当前任务完成、执行线程空闲后再按事件循环取出执行。计算密集型工作可以交给Web

Worker，不过它不能直接操作DOM，而且通信成本高于计算收益时并不划算。JavaScript语言的一大特点就是单线程，也就是说，同一时间只能做一件事，前面的任务没做完，后面的任务只能等着。

1. 为什么JavaScript是单线程的呢？这主要与JavaScript用途有关。它的主要用途是与用户互动，以及操作DOM。如果JavaScript是多线程的，会带来很多复杂的问题，假如JavaScript有A和B两个线程，A线程在DOM节点上添加了内容，B线程删除了这个节点，应该是哪个为准呢？

所以，为了避免复杂性，所以设计成了单线程。虽然HTML5提出了Web Worker标准。Web Worker的作用，就是为JavaScript创造多线程环境，允许主线程创建Worker线程，将一些任务分配给后者运行。但是子线程完全受主线程控制，且不得操作DOM。所以这个并没有改变JavaScript单线程的本质。一般使用Web Worker的场景是代码中有很多计算密集型或高延迟的任务，可以考虑分配给Worker线程。但是使用的时候一定要注意，worker线程是为了让你的程序跑的更快，但是如果worker线程和主线程之间通信的时间大于了你不使用worker线程的时间，结果就得不偿失了。

2. 浏览器内核中线程之间的关系 GUI渲染线程和JS引擎线程互斥js是可以操作DOM的，如果在修改这些元素的同时渲染页面（js线程和ui线程同时运行），那么渲染线程前后获得的元素数据可能就不一致了。JS阻塞页面加载js如果执行时间过长就会阻塞页面 js是可以操作DOM的，如果在修改这些元素的同时渲染页面（js线程和ui线程同时运行），那么渲染线程前后获得的元素数据可能就不一致了。

js如果执行时间过长就会阻塞页面 3. 浏览器是多进程的优点 默认新开 一个 tab 页面 新建一个进程,所以单个 tab 页面崩溃不会影响到整个浏览器。第三方插件崩溃也不会影响到整个浏览器。多进程可以充分利用现代CPU 多核的优势。方便使用沙盒模型隔离插件等进程,提高浏览器的稳定性。 4. 进程和线程又是什么呢 进程(process)和线程(thread)是操作系统的基本概念。 进程是 CPU 资源分配的最小单位(是能拥有资源和独立运行的最小单位)。线程是 CPU 调度的最小单位(是建立在进程基础上的一次程序运行单位)。 由于每个进程至少要做一件事,所以一个进程至少有一个线程。系统会给每个进程分配独立的内存,因此进程有它独立的资源。同一进程内的各个线程之间共享该进程的内存空间(包括代码段,数据集,堆等)。 进程可以理解为一个工厂不同车间,相互独立。线程是车间里的工人,可以自己做事,也可以相互配合做同一件事情。 5. 任务队列 单线程就意味着,所有任务都要排队执行,前一个任务结束,才会执行后一个任务。如果一个任务需要执行,但此时JavaScript引擎正在执行其他任务,那么这个任务就需要放到一个队列中进行等待。等到线程空闲时,就可以从这个队列中取出最早加入的任务进行执行(类似于我们去银行排队办理业务,单线程相当于说这家银行只有一个服务窗口,一次只能为一个人服务,后面到的就需要排队,而任务队列就是排队区,先到的就优先服务) 注意:如果当前线程空闲,并且队列为空,那每次加入队列的函数将立即执行。 为什么会有任务队列? 由于 JS 是单线程的,同步执行任务会造成浏览器的阻塞,所以我们将 JS 分成一个又一个的任务,通过不停的循环来执行事件队列中的任务。

1) 同事说浏览器能同时下载图片,所以页面里的JavaScript也会自动并行修改同一个DOM;你会怎样指出概念混淆?

答:浏览器内部可以有多个进程和线程,但页面主线程上的JavaScript仍按单线程模型执行,不会自动由两个线程同时修改同一DOM。这种设计避免添加节点与删除节点并发发生时的结果冲突;网络并行能力不能直接推出脚本执行模型也相同。

2) 数据看板需要在前端执行大量计算,输入期间明显卡顿;开发打算把计算搬到Web Worker,你会先确认哪些边界?

答:计算密集型或高延迟任务适合交给Web Worker,从而避免长时间占用页面主线程。Worker受主线程控制且不能操作DOM,结果必须回传后再由主线程更新界面;若任务很小或通信成本超过计算收益,迁移反而可能得不偿失。

3) 动画播放时,一段同步脚本持续处理大批数据,结果点击和页面绘制一起停住;为什么渲染线程没有绕过脚本继续工作?

答:资料指出GUI渲染线程与JS引擎线程互斥,脚本执行时间过长会阻塞页面加载和渲染。这样可避免脚本修改DOM的同时渲染线程读取到前后不一致的数据;拆分任务或移出重计算能缓解阻塞,但不能让主线程上的DOM操作并行执行。

4) 线上按钮点击后偶尔很晚才响应,监控显示事件已经进入任务队列;你会先排查事件是否丢失,还是主线程是否被占用?

答:应先检查主线程是否正在执行耗时任务,因为队列中的任务只有在线程空闲后才能按顺序取出执行。事件进入队列说明它不一定丢失,更可能是在等待前面的同步工作完成;若线程空闲且队列原本为空,新增任务才可能立即得到执行。

5) 两个标签页中,一个页面脚本死循环,另一个页面仍正常;有人据此断言JavaScript已经变成多线程,你会如何解释?

答:这反映的是浏览器多进程隔离,而不是单个页面里的JavaScript改成了多线程。不同标签页默认可由不同渲染进程承载,各自运行脚本,因此一个页面阻塞通常不影响其他页面;具体标签与进程映射可能受实现策略影响,不应把“默认”理解为绝对一一对应。

6) 架构师要在主线程分片执行和Web

Worker之间选方案，任务还需要频繁更新DOM；你会偏向哪边？

答：若核心工作必须频繁读写DOM，Worker不能直接完成这些操作，只能承担可独立计算的部分，再把结果交回主线程。计算量大且通信较少时Worker更有价值，交互步骤多且每次计算很轻时，主线程分任务可能更简单；最终取舍应比较阻塞时间与通信成本。

582. Chrome 打开一个页面需要启动多少进程？分别有哪些进程？

Chrome打开一个页面至少需要四个进程：浏览器主进程、网络进程、GPU进程和渲染进程。主进程负责界面、交互和子进程管理，网络进程加载资源，GPU进程负责绘制与合成，渲染进程则运行Blink和V8，把页面变成可交互内容。实际数量通常不止四个，因为还可能存在多个渲染进程和插件进程。渲染与插件独立运行，本质上是为了通过沙箱和进程隔离控制安全风险及崩溃影响。打开 1 个页面至少需要 1 个网络进程、1 个浏览器进程、1 个 GPU 进程以及 1 个渲染进程，共 4 个；最新的 Chrome 浏览器包括：1 个浏览器（Browser）主进程、1 个 GPU 进程、1 个网络（NetWork）进程、多个渲染进程和多个插件进程。浏览器进程：主要负责界面显示、用户交互、子进程管理，同时提供存储等功能。渲染进程：核心任务是将 HTML、CSS 和 JavaScript 转换为用户可以与之交互的网页，排版引擎 Blink 和 JavaScript 引擎 V8 都是运行在该进程中，默认情况下，Chrome 会为每个 Tab 标签创建一个渲染进程。出于安全考虑，渲染进程都是运行在沙箱模式下。GPU 进程：其实，Chrome 刚开始发布的时候是没有 GPU 进程的。而 GPU 的使用初衷是为了实现 3D CSS 的效果，只是随后网页、Chrome 的 UI 界面都选择采用 GPU 来绘制，这使得 GPU 成为浏览器普遍的需求。最后，Chrome 在其多进程架构上也引入了 GPU 进程。网络进程：主要负责页面的网络资源加载，之前是作为一个模块运行在浏览器进程里面的，直至最近才独立出来，成为一个单独的进程。插件进程：主要是负责插件的运行，因插件易崩溃，所以需要通过插件进程来隔离，以保证插件进程崩溃不会对浏览器和页面造成影响。

1) 测试同学说打开一个空白标签页只需要一个进程，因为页面里没有业务代码；你会怎么纠正这个判断？

答：即使是空白页，也不能把整个Chrome页面理解成单进程运行；按基础架构至少涉及浏览器、网络、GPU和渲染进程。它们分别承担界面与子进程管理、资源加载、绘制合成和页面解析执行，是否空白只影响工作量，不改变职责隔离。

2) 一个后台管理页同时加载HTML、样式、脚本和接口数据，负责人要求你说明这些任务分别落在哪些进程，你会怎么划分？

答：网络资源加载主要由网络进程负责，HTML、CSS和JavaScript转换成交互页面的核心工作由渲染进程完成，其中包含Blink与V8。浏览器进程负责界面、用户交互和子进程管理，最终绘制与图层合成还会使用GPU进程。

3) 产品从单标签页改成 20 个标签页，并安装了多个插件，却要求进程数仍固定为 4 个，这个约束为什么不成立？

答：4 个只是打开一个页面时所涉及的基础进程类别和最低数量描述，并不是整个浏览器会长期固定为 4 个进程。Chrome默认可能为各个Tab创建渲染进程，插件也可能运行在独立插件进程中，因此标签页和插件增加后，实际进程数通常会随之变化。

4) 线上页面突然白屏，但浏览器地址栏、菜单和其他标签页仍能操作，你会优先怀疑哪个进程，并怎样缩小范围？

答：应优先检查该页面对应的渲染进程，因为Blink、V8以及页面内容生成都运行在那里，而浏览器界面仍正常说明主进程未必失效。可用Chrome任务管理器观察对应任务是否崩溃或资源异常，再区分是脚本执行、页面渲染还是插件影响。

5) 架构评审时有人建议把插件直接放进浏览器主进程，以减少进程数量和通信成本，你会接受吗？

答：通常不应接受，因为插件更容易发生崩溃，独立插件进程的价值正是隔离故障，避免连带破坏浏览器和页面。合并可能减少部分进程与通信开销，却会扩大故障影响面；除非运行环境和插件风险受到严格控制，否则不值得牺牲隔离性。

6) 页面请求已经返回，但内容迟迟没有显示；后端认定网络进程有问题，你会怎样串联网络进程与渲染进程判断责任边界？

答：请求返回只能说明网络进程完成了资源加载，不代表页面已经完成解析、布局、绘制和合成。应继续检查渲染进程中的Blink、V8是否被脚本或渲染工作阻塞，并观察GPU绘制合成环节；仅凭接口成功或失败不能覆盖整条显示链路。

583. 渲染机制

浏览器渲染是一条从DOM、CSSOM到渲染树，再经过布局、绘制和图层合成的流水线。修改元素尺寸或位置会触发布局，也就是回流，后续通常还要重绘；只改颜色、背景等外观，一般只需重绘，所以回流成本更高。像transform、opacity这类动画可以利用独立图层，尽量减少对布局的影响，但图层也不能创建过多。我一般还会把频繁更新放进requestAnimationFrame，并避免循环读取offsetWidth、offsetHeight后又立即改样式。

1. 浏览器如何渲染网页 概述：浏览器渲染一共有五步 处理HTML并构建DOM树。处理CSS构建CSSOM树。将DOM与CSSOM合并成一个渲染树。根据渲染树来布局，计算每个节点的位置。调用GPU绘制，合成图层，显示在屏幕上

第四步和第五步是最耗时的部分，这两步合起来，就是我们通常所说的渲染 具体如下图过程如下图所示渲染

网页生成的时候，至少会渲染一次在用户访问的过程中，还会不断重新渲染重新渲染需要重复之前的第四步(重新生成布局)+第五步(重新绘制)或者只有第五步(重新绘制) 在构建CSSOM树时，会阻塞渲染，直至CSSOM树构建完成。并且构建CSSOM树是一个十分消耗性能的过程，所以应该尽量保证层级扁平，减少过度层叠，越是具体的CSS选择器，执行速度越慢当HTML解析到script标签时，会暂停构建DOM，完成后才会从暂停的地方重新开始。也就是说，如果你想首屏渲染的越快，就越不应该在首屏就加载JS文件。并且CSS也会影响JS的执行，只有当解析完样式表才会执行JS，所以也可以认为这种情况下，CSS也会暂停构建DOM

2. 浏览器渲染五个阶段 2.1 第一步：解析HTML标签，构建DOM树 在这个阶段，引擎开始解析html，解析出来的结果会成为一棵dom树dom的目的至少有2个

作为下个阶段渲染树状图的输入成为网页和脚本的交互界面。(最常用的就是getElementById等等)

当解析器到达script标签的时候，发生下面四件事情 html解析器停止解析,如果是外部脚本，就从外部网络获取脚本代码将控制权交给js引擎，执行js代码恢复html解析器的控制权 由此可以得到第一个结论1 由于标签是阻塞解析的，将脚本放在网页尾部会加速代码渲染。defer和async属性也能有助于加载外部脚本。defer使得脚本会在dom完整构建之后执行；async标签使得脚本只有在完全available才执行，并且是以非阻塞的方式进行的 2.2 第二步：解析CSS标签，构建CSSOM树 我们已经看到html解析器碰到脚本后会做的事情，接下来我们看下html解析器碰到样式表会发生的情况js会阻塞解析，因为它会修改文档(document)。css不会修改文档的结构，如果这样的话，似乎看起来css样式不会阻塞浏览器html解析。但是事实上css样式表是阻塞的。阻塞是指当cssom树建立好之后才会进行下一步的解析渲染

通过以下手段可以减轻cssom带来的影响

将script脚本放在页面底部尽可能快的加载css样式表将样式表按照media type和media query区分，这样有助于我们将css资源标记成非阻塞渲染的资源。非阻塞的资源还是会被浏览器下载，只是优先级较低

2.3 第三步：把DOM和CSSOM组合成渲染树 (render tree) 2.4

第四步：在渲染树的基础上进行布局，计算每个节点的几何结构

布局(layout)：定位坐标和大小，是否换行，各种position,overflow,z-index属性 2.5 调用 GPU

绘制，合成图层，显示在屏幕上 将渲染树的各个节点绘制到屏幕上，这一步被称为绘制painting 3.

渲染优化相关 3.1 Load 和 DOMContentLoaded 区别 Load事件触发代表页面中的DOM，CSS，JS，图片已经全部加载完毕。DOMContentLoaded事件触发代表初始的HTML被完全加载和解析，不需要等待C

SS，JS，图片加载 3.2 图层 一般来说，可以把普通文档流看成一个图层。特定的属性可以生成一个新的图层。不同的图层渲染互不影响，所以对于某些频繁需要渲染的建议单独生成一个新图层，提高性能。但也

不能生成过多的图层，会引起反作用。通过以下几个常用属性可以生成新图层 3D变换：translate3d、translateZwill-changevideo、iframe标签通过动画实现的opacity动画转换position: fixed 3.3

重绘 (Repaint) 和回流 (Reflow)

重绘和回流是渲染步骤中的一小节，但是这两个步骤对于性能影响很大

重绘是当节点需要更改外观而不会影响布局的，比如改变color就叫称为重绘回流是布局或者几何属性需...

1) 一个卡片组件只把文字颜色从黑色改成红色，开发却说一定会重新计算整页布局；你会怎样反驳？

答：单纯修改color通常只改变外观而不改变元素几何结构，因此会触发重绘，不必然触发布局回流。只有宽高、边距、字体或内容等影响位置和尺寸的变化才需要重新布局；不过重绘仍有成本，影响区域过大时也可能造成卡顿。

2) 一个 1000 行列表在循环中反复读取offsetHeight，同时逐项修改style，滚动时明显卡顿，你会怎样改造？

答：这类读写交替可能迫使浏览器反复获得最新布局结果，应先批量读取所需尺寸并缓存，再集中修改样式，避免在循环中持续查询几何属性。大量节点还可先离线或隐藏后统一修改，但display:none的切换本身会产生回流，需要权衡修改次数。

3) 动效设计要求悬浮按钮每帧修改top，视觉同学又不接受改变运动轨迹；你会选择什么实现，并说明限制？

答：应保持相同轨迹但改用transform: translate(...)表达位移，避免持续修改top引发几何布局和后续重绘。频繁动画还可使用requestAnimationFrame对齐浏览器更新节奏；若动画必须真实改变文档流位置，就无法完全绕开回流。

4) 首屏的HTML很小，但头部同步加载大型脚本和多份样式表后长时间空白，你会沿渲染链路排查什么？

答：先检查同步script是否暂停了HTML解析，以及样式表是否迟迟未完成CSSOM构建并影响后续渲染和脚本执行。可将非关键脚本后移或使用defer、async，并尽快加载必要样式、按media条件拆分；依赖执行顺序的脚本不能盲目改为async。

5) 一个长列表滚动时大量区域持续重绘，团队准备给每个列表项都加will-change，你会批准这个方案吗？

答：不应直接给所有列表项提层；独立图层可隔离频繁渲染区域，但图层过多也会产生反作用。应先确认真正高频变化的节点，只对动画或重绘热点使用will-change、translate3d等手段，并在性能面板中验证布局与绘制是否确实减少。

6) 监控显示DOMContentLoaded很早触发，但首屏图片仍未出现，产品却认为页面已经完全加载；你会如何解释并设置判断口径？

答：DOMContentLoaded只表示初始HTML已完成加载和解析，不要求图片等资源全部完成，因此不能作为完整页面就绪的唯一口径。若关注全部资源完成应观察load，若关注可交互首屏则还要结合脚本执行和实际渲染状态；两者代表的业务体验并不相同。

584. 缓存机制

浏览器缓存的关键区别是是否发送请求：强缓存命中时不请求服务器，协商缓存则要向服务器确认资源没有变化。首次响应会连同缓存头一起保存，后续浏览器根据Cache-Control或Expires判断缓存是否仍然有效；失效后再进入协商阶段。no-cache并不是完全不缓存，而是使用前必须向服务器校验，资源未变化时可以避免重新下载。真正禁止保存的是no-store，而public和private则用于控制代理缓存能否保存响应。

1. 首先得明确 http 缓存的好处 减少了冗余的数据传输，减少网费减少服务器端的压力Web缓存能够减少延迟与网络阻塞，进而减少显示某个资源所用的时间加快客户端加载网页的速度 2. 常见 http 缓存的类型 私有缓存（一般为本地浏览器缓存）代理缓存 3. 然后谈谈本地缓存 本地缓存是指浏览器请求资源时命中了浏览器本地的缓存资源，浏览器并不会发送真正的请求给服务器了。它的执行过程是 第一次浏览器发送请求给服务器时，此时浏览器还没有本地缓存副本，服务器返回资源给浏览器，响应码是200 OK，浏览器收到资源后，把资源和对应的响应头一起缓存下来第二次浏览器准备发送请求给服务器时候，浏览器会先检查上一次服务端返回的响应头信息中的Cache-Control，它的值是一个相对值，单位为秒，表示资源在客户端缓存的最大有效期，过期时间为第一次请求的时间减去Cache-Control的值，过期时间跟当前的请求时间比较，如果本地缓存资源没过期，那么命中缓存，不再请求服务器如果没有命中，浏览器就会把请求发送给服务器，进入缓存协商阶段。与本地缓存相关的头有：Cache-Control、Expires，Cache-Control有多个可选值代表不同的意义，而Expires就是一个日期格式的绝对值。 3.1 Cache-Control

Cache-Control是HTTP缓存策略中最重要的头，它是HTTP/1.1中出现的，它由如下几个值 no-cache：不使用本地缓存。需要使用缓存协商，先与服务器确认返回的响应是否被更改，如果之前的响应中存在ETag，那么请求的时候会与服务端验证，如果资源未被更改，则可以避免重新下载no-store：直接禁止浏览器缓存数据，每次用户请求该资源，都会向服务器发送一个请求，每次都会下载完整的资源public：可以被所有的用户缓存，包括终端用户和CDN等中间代理服务器。private：只能被终端用户的浏览器缓存，不允许CDN等中继缓存服务器对其缓存。max-age：从当前请求开始，允许获取的响应被重用的最长时间（秒）。must-revalidate，当缓存过期时，需要去服务端校验缓存的有效性。 例如：Cache-Control: public, max-age=1000# 表示资源可以被所有用户以及代理服务器缓存，最长时间为1000秒。

注意，虽然你可能在其他资料中看到可以使用 meta 标签来设置缓存，比如像下面的形式：<meta http-equiv="expires" content="Wed, 20 Jun 2021 22:33:00 GMT" 但在 HTML5 规范中，并不支持这种方式，所以尽量不要使用 meta 标签来设置缓存。 3.2 Expires Expires是HTTP/1.0出现的头信息，同样是用于决定本地缓存策略的头，它是一个绝对时间，时间格式是如Mon, 10 Jun 2015 21:31:12 GMT，只要发送请求时间是在Expires之前，那么本地缓存始终有效，否则就会去服务器发送请求获取新的资源。如果同时出现Cache-Control：max-age和Expires，那么max-age优先级更高。他们可以这样组合使用 Cache-Control: public Expires: Wed, Jan 10 2018 00:27:04 GMT 3.3 所谓的缓存协商当第一次请求时服务器返回的响应头中存在以下情况时 没有Cache-Control和ExpiresCache-Control和Expires过期了Cache-Control的属性设置为no-cache时 那么浏览器第二次请求时就会与服务器进行协商，询问浏览器中的缓存资源是不是旧版本，需不需要更新，此时，服务器就会做出判断，如果缓存和服务端资源的最新版本是一致的，那么就无需再次下载该资源，服务端直接返回304 Not Modified状态码，如果服务器发现浏览器中的缓存已经是旧版本了，那么服务器就会把最新资源的完整内容返回给浏览器，状态码就是200 Ok，那么服务端是根据什么来判断浏览器的缓存是不是最新的呢？其实是根据HTTP的另外两组头信息，分别是：Last-Modified/If-Modified-Since与...

1) 商品详情页第二次打开时，Network显示请求携带If-None-Match，但响应是200而不是304；有人据此断言浏览器没有使用缓存，你怎么判断？

答：不能仅凭200判断浏览器未使用缓存，因为携带If-None-Match已说明客户端正在用上次保存的ETag发起再验证。若服务端当前ETag不一致，就应返回新资源和200；只有标识一致时才返回不带资源实体的304。

2) 发版后的首页HTML已配置协商缓存，第二次请求带着If-None-Match；服务端需要怎样处理，浏览器收到响应后又怎样取资源？

答：服务端应比较请求中的If-None-Match与当前资源的ETag，一致时返回304，不再传输资源内容。浏览器随后从本地缓存读取实体；若两者不一致，则服务端返回更新后的资源，并随响应提供新的校验标识。

3) 一个高频访问的小图标和一个体积较大的JS文件都命中了本地缓存，前端同事认为它们必然进入同一种缓存位置，你会怎样纠正？

答：两者不一定进入相同位置，浏览器还会在Memory Cache与Disk Cache之间进行选择。内存缓存读取更快但随渲染进程结束而消失；较大的JS、CSS通常更倾向磁盘，而具体归属仍受浏览器策略影响。

4) 用户按普通F5后接口返回304，按Ctrl+F5却下载了完整资源；值班同学怀疑服务端随机失效缓存，你怎么解释并排查？

答：这种差异符合两种刷新方式的缓存流程：普通F5会跳过强缓存，但仍进行协商校验，所以可能收到304。Ctrl+F5会跳过强缓存和协商缓存，直接从服务端加载；排查时应对比请求头、状态码及实际传输内容。

5) 运营页更新频繁，架构师主张所有资源都只用Cache-Control: max-age，后端则主张全部依赖ETag；你如何说明两种方案的请求成本？

答：max-age命中强缓存时不会向服务器发请求，适合生命周期内无需重新确认的资源；ETag属于协商缓存，每次再验证仍会产生请求，但未变化时只返回304。选择取决于更新及时性要求，不能把免请求与免传资源视为同一件事。

6) 一个PWA页面断网后仍展示旧数据，清理普通磁盘缓存也没有恢复；同时DevTools标记资源来自Service Worker，下一步查哪里？

答：应优先检查Service Worker的离线缓存和网络代理逻辑，因为它的缓存位置优先于Memory Cache、Disk Cache和Push Cache。仅清理磁盘缓存可能绕不开其响应；还要核对注册状态与缓存更新策略，但不能直接归因于HTTP强缓存。

585. 浏览器存储

浏览器端常用的持久化方案有cookie、localStorage、sessionStorage和IndexedDB，选择时主要看数据用途、生命周期与容量。cookie通常保存身份或登录状态，约4K，请求时会自动携带；另外三者不随请求发送，更适合客户端数据。localStorage会一直保留到被清理，sessionStorage通常随页面会话结束而清理，大量结构化数据可以考虑IndexedDB。涉及登录态时还要关注HttpOnly、Secure和SameSite，分别用于限制脚本访问、安全传输以及降低跨站请求风险。

我们经常需要对业务中的一些数据进行存储，通常可以分为 短暂性存储 和 持久性存储。短暂性的时候，我们只需要将数据存在内存中，只在运行时可用持久性存储，可以分为 浏览器端 与 服务器端

存储方案	存储位置	数据范围	生命周期	容量限制
cookie	浏览器端	通常用于存储用户身份、登录状态等http中自动携带	随请求发送	体积上限为4K
localStorage	浏览器端	通常用于存储用户身份、登录状态等http中自动携带	永久保存	可自行设置过期时间
sessionStorage	浏览器端	通常用于存储用户身份、登录状态等http中自动携带	窗口关闭删除	体积限制为4~5M
IndexedDB	浏览器端	通常用于存储用户身份、登录状态等http中自动携带	永久保存	体积限制为4~5M
redis数据库	服务器端	通常用于存储用户身份、登录状态等http中自动携带	永久保存	分布式缓存

从上表可以看到，cookie已经不建议用于存储。如果

没有大量数据存储需求的话，可以使用localStorage和sessionStorage。对于不怎么改变的数据尽量使用localStorage存储，否则可以用sessionStorage存储。对于cookie，我们还需要注意安全性 Name，即该Cookie的名称。Cookie一旦创建，名称便不可更改。Value，即该Cookie的值。如果值为Unicode字符，需要为字符编码。如果值为二进制数据，则需要使用BASE64编码。Max Age，即该Cookie失效的时间，单位秒，也常和Expires一起使用，通过它可以计算出其有效时间。Max Age如果为正数，则该Cookie在Max Age秒之后失效。如果为负数，则关闭浏览器时Cookie即失效，浏览器也不会以任何形式保存该Cookie。Path，即该Cookie的使用路径。如果设置为/path/，则只有路径为/path/的页面可以访问该Cookie。如果设置为/，则本域名下的所有页面都可以访问该Cookie。Domain，即可以访问该Cookie的域名。例如如果设置为.zhihu.com，则所有以zhihu.com，结尾的域名都可以访问该Cookie。Size字段，即此Cookie的大小。Http字段，即Cookie的httponly属性。若此属性为true，则只有在HTTP Headers中会带有此 Cookie 的信息，而不能通过document.cookie来访问此 Cookie。Secure，即该Cookie是否仅被使用安全协议传输。安全协议。安全协议有HTTPS、SSL等，在网络上传输数据之前先将数据加密。默认为false。

1) 登录页把明文用户编号写进Cookie，开发认为加上HttpOnly后就绝对安全；作为安全评审你会指出什么？

答：HttpOnly只限制脚本通过document.cookie读取，并不等于明文值本身安全。登录态相关的value不应直接使用明文用户标识，还应结合Secure限制安全协议传输，并用SameSite降低跨站请求携带带来的风险。

2) 一个后台系统要保存登录态、当前窗口的筛选条件和较大规模的结构化离线数据，你会分别落到哪类存储？

答：登录态通常由Cookie承载，筛选条件若只需随当前窗口存在可放sessionStorage，较大且需持久化的数据更适合IndexedDB。Cookie约受4K限制且随请求进入请求头，localStorage、sessionStorage通常只有数兆容量。

3) 客服要求关闭页面后筛选条件消失，但产品又要求重新打开浏览器仍保留用户偏好；只允许在localStorage和sessionStorage间选，你怎么拆分？

答：筛选条件应放入sessionStorage，因为其生命周期与页面窗口相关，关闭后会被清理；用户偏好应放入localStorage，除非主动清理否则会持续存在。两者都不会自动参与服务端通信，但都不适合超出其容量边界的数据。

4) 线上接口请求头突然膨胀，排查发现同一域名下写入了大量业务Cookie；业务方却说这些数据只在个人中心页面使用，你怎么处理？

答：应把不需要服务端识别的业务数据移出Cookie，按生命周期改放localStorage、sessionStorage或更大容量的IndexedDB。Cookie会在匹配其作用范围的请求中自动进入请求头，即使接口本身不用这些值，也会持续产生传输负担。

5) 多子域系统把登录Cookie的Domain设为顶级业务域、Path设为/，便利性与隔离性发生冲突，你会怎么评审？

答：该配置会扩大Cookie的可用范围，使匹配该域及路径的更多页面和请求都可能携带它。若登录态只服务某个子域或路径，应尽量收窄Domain与Path；范围缩小会增加多系统共享登录态的设计成本。

6) 安全团队要求登录Cookie只能走HTTPS、脚本不可读取，并尽量减少跨站请求携带；你会配置哪些属性，还要提醒什么边界？

答：应配置Secure、HttpOnly与合适的SameSite：分别限制安全协议传输、阻止脚本读取并约束跨站携带。它们针对的是不同风险面，不能互相替代；同时仍需妥善处理登录态value，不能把敏感身份

直接作为明文保存。

586. 跨域方案

跨域的核心是绕开或满足浏览器同源策略，常见方案可分为服务端放行、代理转发和标签请求。接口能改服务端时优先配置CORS；开发环境可用webpack devServer.proxy或http-proxy-middleware做反向代理。只需发起GET请求时可以用JSONP，图片上报则适合img ping，但它拿不到响应正文。页面间通信可考虑iframe或postMessage，同时要留意交互安全和数据传递限制。

很多种方法，但万变不离其宗，都是为了搞定同源策略。重用的有jsonp、iframe、cors、img、HTML5 postMessage等等。其中用到html标签进行跨域的原理就是html不受同源策略影响。但只是接受Get的请求方式，这个得清楚。延伸1：img iframe script 来发送跨域请求有什么优缺点？1.iframe 优点：跨域完毕之后DOM操作和互相之间的JavaScript调用都是没有问题的缺点：1.若结果要以URL参数传递，这就意味着在结果数据量很大的时候需要分割传递，巨烦。2.还有一个是iframe本身带来的，母页面和iframe本身的交互本身就有安全性限制。

2. script 优点：可以直接返回json格式的数据，方便处理缺点：只接受GET请求方式 3. 图片ping 优点：可以访问任何url，一般用来进行点击追踪，做页面分析常用的方法缺点：不能访问响应文本，只能监听是否响应

延伸2：配合 webpack 进行反向代理？

webpack在devServer选项里面提供了一个proxy的参数供开发人员进行反向代理

```
'/api':{target:'http://www.example.com',// your target hostchangeOrigin:true,// needed for virtual hosted sitespathRewrite:{'^/api:':// rewrite path}},
```

然后再配合http-proxy-middleware插件对api请求地址进行代理

```
const express=require('express');const proxy=require('http-proxy-middleware');// proxy api requestsconst exampleProxy=proxy(options);// 这里的 options 就是 webpack 里面的 proxy 选项对应的每个选项// mount `exampleProxy` in web serverconst app=express();app.use('/api',exampleProxy);app.listen(3000);
```

然后再用nginx把允许跨域的源地地址添加到报头里面即可 说到nginx，可以再谈谈CORS配置，大致如下 location/{if(\$request_method='OPTIONS'){add_header'Access-Control-Allow-Origin'*';add_header'Access-Control-Allow-Methods'GET, POST, OPTIONS';add_header'Access-Control-Allow-Credentials'true';add_header'Access-Control-Allow-Headers'DNT, X-Mx-ReqToken, Keep-Alive, User-Agent, X-Requested-With, If-Modified-Since, Cache-Control, Content-Type';add_header'Access-Control-Max-Age'86400;add_header'Content-Type"text/plain charset=UTF-8';add_header'Content-Length'0;return200;}}

1) 埋点页面用new Image().src上报成功，产品却要求读取服务端返回的用户标签；沿用图片ping为什么行不通？

答：图片ping可以向跨域URL发起请求，但脚本不能读取响应正文，因此无法取得用户标签。它更适合点击追踪和页面分析，只能借助加载或失败事件判断是否响应；若业务依赖响应数据，就必须改用允许读取结果的受控方案。

2) 开发环境把/api配到devServer.proxy后请求正常，静态文件部署到测试环境却开始跨域；你会把代理落在哪一层？

答：devServer.proxy只属于本地开发服务器，打包后的静态资源不会携带这段转发逻辑。测试环境应由实际承载流量的服务处理，例如让Nginx按/api反向代理，或由接口端配置CORS；两边规则不一致仍会造成环境差异。

3) 旧页面用script加载跨域数据，接口负责人现在要求改成POST并提交较大的筛选条件；原方案还能继续吗？

答：依赖script的跨域方案只能发起GET，无法直接满足新的POST约束。把大量数据塞进查询参数还会受到URL长度、缓存和日志暴露等边界影响，应改由代理或CORS承载请求；迁移成本是服务端需要同步调整接口策略。

4) 线上跨域接口连真实业务日志都没有，只看到浏览器先发出OPTIONS；你会先核对Nginx的哪些响应内容？

答：真实请求未到业务服务时，应先检查预检响应是否允许当前源、方法和请求头，并确认OPTIONS能直接返回成功状态。还要核对Access-Control-Allow-Credentials与来源策略是否匹配，以及配置是否覆盖目标路径；任一项不匹配，浏览器都可能拦住后续请求。

5) 安全负责人反对放开整站CORS，前端又希望所有环境都使用统一的/api地址；你会如何取舍？

答：更稳妥的选择是让页面始终请求同域/api，再由各环境的Nginx转发到真实接口，从浏览器视角规避跨域。若必须让第三方源直接访问，再按明确来源、方法和请求头配置CORS；代理增加运维配置，开放CORS则扩大暴露边界。

6) 微前端容器需要和跨域iframe交换筛选状态，数据量持续增长；你会继续拼接URL参数还是改用什么机制？

答：大量状态通过URL参数传递会面临长度限制、拆分复杂和信息暴露，不适合作为持续通信通道。应使用postMessage传递结构化消息，并在接收端校验origin、消息类型和数据格式；它解决通信问题，但不会自动解除跨域DOM的安全限制。

587. XSS 和 CSRF

XSS是把恶意代码注入页面并执行，CSRF是借用用户已有登录态发送非本人意愿的请求。XSS分持久型和非持久型，防护重点是不信任用户输入：普通内容做转义，富文本采用白名单过滤，并用CSP限制资源加载和执行。CSRF利用了浏览器自动携带目标域名Cookie的机制，但攻击站点通常拿不到该Cookie。工程上应避免用GET修改数据，并结合验证token、验证码及SameSite等措施。

涉及面试题：什么是XSS攻击？如何防范XSS攻击？什么是CSP？XSS简单点来说，就是攻击者想尽一切办法将可以执行的代码注入到网页中。XSS可以分为多种类型，但是总体上我认为分为两类：持久型和非持久型。持久型也就是攻击的代码被服务端写入进数据库中，这种攻击危害性很大，因为如果网站访问量很大的话，就会导致大量正常访问页面的用户都受到攻击。

举个例子，对于评论功能来说，就得防范持久型XSS攻击，因为我可以在评论中输入以下内容 这种情况如果前后端没有做好防御的话，这段评论就会被存储到数据库中，这样每个打开该页面的用户都会被攻击到。非持久型相比于前者危害就小的多了，一般通过修改URL参数的方式加入攻击代码，诱导用户访问链接从而进行攻击。

举个例子，如果页面需要从URL中获取某些参数作为内容的话，不经过过滤就会导致攻击代码被执行 `alert(1) --> {{name}}` 但是对于这种攻击方式来说，如果用户使用Chrome这类浏览器的话，浏览器就能自动帮助用户防御攻击。但是我们不能因此就不防御此类攻击了，因为我不能确保用户都使用了该类浏览器。

对于XSS攻击来说，通常有两种方式可以用来防御。转义字符 首先，对于用户的输入应该是永远不信任的。最普遍的做法就是转义输入输出的内容，对于引号、尖括号、斜杠进行转义 `function escape(str){str=str.replace(/&/g,'&');str=str.replace(/>/g,'>');str=str.replace(/"/g,'"');str=str.replace(/'/g,''');str=str.replace(/`/g,'`');str=str.replace(/\\/g,'/');return str}`

通过转义可以将攻击代码 `alert(1)` 变成 `// -> <script>alert(1)</script>escape('alert(1)')` 但是对于显示富文本来讲，显然不能通过上面的办法来转义所有字符，因为这样会把需要的格式也过滤掉。

对于这种情况，通常采用白名单过滤的办法，当然也可以通过黑名单过滤，但是考虑到需要过滤的标签和标签属性实在太多，更加推荐使用白名单的方式 `const xss=require('xss') let html=xss('XSS`

```
Demoalert("xss");)// -> XSS Demo<script>alert("xss");</script>console.log(html)
```

以上示例使用了js-xss来实现，可以看到在输出中保留了h1标签且过滤了script标签 CSP CSP本质上就是建立白名单，开发者明确告诉浏览器哪些外部资源可以加载和执行。我们只需要配置规则，如何拦截是由浏览器自己实现的。我们可以通过这种方式来尽量减少XSS攻击。通常可以通过两种方式来开启 CSP：设置HTTP Header中的Content-Security-Policy设置meta标签的方式 这里以设置HTTP Header来举例只允许加载本站资源 Content-Security-Policy: default-src 'self' 只允许加载 HTTPS 协议图片 Content-Security-Policy: img-src https://* 允许加载任何来源框架 Content-Security-Policy: child-src 'none' 当然可以设置的属性远不止这些，你可以通过查阅文档(opens new window)的方式来学习，这里就不过多赘述其他的属性了。对于这种方式来说，只要开发者配置了正确的规则，那么即使网站存在漏洞，攻击者也不能执行它的攻击代码，并且CSP的兼容性也不错。 2 CSRF 跨站请求伪造（英语：Cross-site request forgery），也被称为one-click attack或者session riding，通常缩写为CSRF或者XSRF，是一种挟制用户在当前已...

1) 评论页已经把全部转义了，安全同学却用一个带恶意参数的分享链接触发了脚本；团队因此认定“转义防不了XSS”，你怎么判断？

答：不能据此否定转义，先确认参数最终进入的是文本、属性、脚本还是其他上下文，以及是否存在绕过转义的直接插入。普通文本应按输出上下文转义；富文本则需白名单净化，单靠一套字符替换无法覆盖所有注入位置。

2) 内容平台允许作者提交含标题、链接和图片的HTML，前端负责人主张提交前用正则删掉，后端认为原样入库即可；你会怎么落地？

答：不能把前端正则作为安全边界，调用接口即可绕过，而且危险标签和属性很多，黑名单容易遗漏。服务端应使用成熟净化方案按允许的标签、属性和资源规则处理，前端可同步净化改善预览，并用严格CSP限制可加载、可执行的资源。

3) 业务为了接入外部图片和统计脚本，要求把CSP放宽为允许任意来源；富文本白名单仍然保留，这时你接受吗？

答：不应直接允许任意来源，否则CSP对遗漏注入点的兜底能力会明显下降。应按资源类型分别配置可信来源，例如限制脚本、图片和子页面，并优先通过响应头下发；若业务必须放宽某类资源，要明确它不会连带放宽脚本执行。

4) 用户反馈打开一条历史评论后账号出现异常操作，数据库里发现了可执行片段，但安全人员抓不到攻击者读取Cookie的请求；你如何区分是持久型`XSS还是CSRF`？

答：评论中的代码被入库并在访问者页面执行，首先按持久型XSS排查，不以是否成功读取Cookie作为必要条件。检查渲染入口、净化结果和CSP报告；若只是外站借浏览器自动携带目标域Cookie发请求、外站拿不到其内容，才更符合CSRF。

5) 支付团队已给会话Cookie设置SameSite=Strict，因此想删除所有CSRF token；接口里还保留一个通过GET修改订阅状态的入口，你同意吗？

答：不同意把SameSite当成唯一措施，更不能保留有副作用的GET，因为图片等元素就能自动发起这类请求。应先把修改操作改为合适的方法，再附带服务端可验证的token或必要的用户验证；SameSite、接口访问限制和验证信息属于纵深防御。

588. Service Worker

Service Worker本质上是运行在网页与浏览器、网络之间的代理，可拦截请求并提供离线缓存能力。它通常在install事件中预先缓存文件，再通过fetch事件决定返回缓存还是访问网络，因此常用于缓存静态文件和改善首屏加载。它不能直接操作DOM，需要通过postMessage与受控页面通信。使用时还要注意只能运

行在本地环境或HTTPS下，并且作用范围受脚本所在路径限制。Service workers本质上充当Web应用程序与浏览器之间的代理服务器，也可以在网络可用时作为浏览器和网络间的代理。它们旨在（除其他之外）使得能够创建有效的离线体验，拦截网络请求并基于网络是否可用以及更新的资源是否驻留在服务器上采取适当的动作。他们还允许访问推送通知和后台同步API。浏览器对ServiceWorker做了很多限制。在ServiceWorker中无法直接访问DOM，但可以通过postMessage接口发送的消息来与其控制的页面进行通信。ServiceWorker只能在本地环境下或HTTPS网站中使用。ServiceWorker有作用域的限制，一个ServiceWorker脚本只能作用于当前路径及其子路径；目前该技术通常用来做缓存文件，提高首屏速度。

```
// index.jsif(navigator.serviceWorker){navigator.serviceWorker.register("sw.js").then(function(registration){console.log("service worker 注册成功");}).catch(function(err){console.log("service worker 注册失败");});}// sw.js// 监听 `install` 事件，回调中缓存所需文件self.addEventListener("install",e=>{e.waitUntil(caches.open("my-cache").then(function(cache){returncache.addAll(["./index.html","./index.js"]);});});// 拦截所有请求事件// 如果缓存中已经有请求的数据就直接用缓存，否则去请求数据self.addEventListener("fetch",e=>{e.respondWith(caches.match(e.request).then(function(response){if(response){returnresponse;}console.log("fetch source");});});};
```

打开页面，可以在开发者工具中的Application看到ServiceWorker已经启动了。在Cache中也可以发现我们所需的文件已被缓存。当我们重新刷新页面可以发现我们缓存的数据是从ServiceWorker中读取的。

1) 新闻站把资源写入Cache

API后就宣称“已经支持离线”，但断网刷新时HTML仍然白屏；这说明团队混淆了什么？

答：写入缓存不等于请求会自动命中缓存，必须由已激活且处于作用域内的Service Worker在fetch事件中返回匹配响应。还要确认入口文档确实被缓存并有离线回退；首次访问、注册失败或尚未被控制的页面仍可能无法离线。

2) 商城准备缓存index.html和index.js来提升再次访问速度，代码只在install中执行cache.addAll()；你会怎样补全最小可用链路？

答：注册脚本后，在install的event.waitUntil()中等待预缓存完成，并在fetch中用event.respondWith()返回caches.match(request)的结果。缓存未命中时必须继续请求网络或返回明确回退，否则处理器可能得不到有效响应；同时要保证站点使用HTTPS或本地环境。

3) 主站从根路径注册sw.js后缓存正常，迁到/shop/sw.js后首页请求不再被拦截，但/shop/cart正常；运维认为是浏览器缓存损坏，你怎么看？

答：更可能是作用域变化，而不是缓存损坏，因为Service Worker默认只能控制脚本所在路径及其子路径。应检查注册记录、当前页面的控制器与注册作用域；若确需覆盖首页，应调整脚本位置或在受控配置下声明合适作用域。

4) 线上显示Service Worker注册成功，Application面板也能看到它，但刷新后静态文件仍全部走网络；你按什么顺序排查？

答：先确认当前页面已被该注册实例控制，并核对页面路径是否落在其作用域内，再检查fetch事件是否触发。随后验证缓存名称、请求键和caches.match()是否一致，以及respondWith()是否真正返回了响应；只完成注册或安装并不代表页面已经从缓存读取。

5) 运营后台要求直接在Service Worker中读取表单DOM，以便断网时保存编辑内容；架构师则建议页面把数据发给它，你支持哪一方？

答：应由页面读取表单，再通过postMessage与Service Worker通信，因为后者不能直接访问DOM。离线缓存、请求拦截、推送和后台同步适合放在该代理层；页面状态采集仍属于受控页面职责，消息协议也要处理版本与校验。

589. DOM 节点操作

DOM节点操作主要包括创建、增删替换、查找和属性修改，关键是按场景选对接口。创建元素或文本可用`createElement`、`createTextNode`，批量组织节点时可先放进`createDocumentFragment`。插入、删除和替换分别可以用`appendChild`、`removeChild`、`replaceChild`或`insertBefore`。查找时精确id可用`getElementById`，复杂选择条件更适合`querySelector`或`querySelectorAll`，属性则通过`getAttribute`、`setAttribute`等接口处理。

(1) 创建新节点 `createDocumentFragment()`//创建一个DOM片段`createElement()`//创建一个具体的元素`createTextNode()`//创建一个文本节点

(2) 添加、移除、替换、插入
`appendChild(node)``removeChild(node)``replaceChild(new,old)``insertBefore(new,old)`

(3) 查找 `getElementById()`;`getElementsByName()`;`getElementsByTagName()`;`getElementsByClassName()`;`querySelector()`;`querySelectorAll()`;

(4) 属性操作
`getAttribute(key)`;`setAttribute(key,value)`;`hasAttribute(key)`;`removeAttribute(key)`;

1) 表格里有一个，同事说`querySelector('#row-7')`返回的是“动态集合”，删掉该行后变量会自动变空；你怎么纠正？

答：`querySelector()`返回首个匹配元素或空值，不是会随文档变化的集合，已保存的变量也不会因节点移除而自动清空。若要获取多个匹配项应使用`querySelectorAll()`；代码还持有节点引用时，该引用依然存在，不能把查询结果与文档结构变化混为一谈。

2) 评论列表一次插入 300 个新条目，代码在循环里逐个`appendChild()`到可见容器；只允许使用题目列出的原生节点能力，你会怎么改？

答：先用`createDocumentFragment()`作为临时容器，把`createElement()`和`createTextNode()`创建的条目依次追加进去，最后一次性挂到列表。这样组织也便于在入树前完成内容和属性设置；片段只减少零散插入，仍需控制总节点规模。

3) 设计稿要求把现有占位节点换成真实卡片，并保证它仍位于原来的兄弟节点之间；有人提议先`removeChild()`再`appendChild()`，你选什么？

答：应使用父节点的`replaceChild(newNode, oldNode)`，它直接在原位置完成替换，不会把新卡片移动到容器末尾。若只是把节点插到某个兄弟节点前，则用`insertBefore(newNode, referenceNode)`；操作前仍要确保旧节点确实属于该父节点。

4) 线上筛选器偶发报错，日志显示`removeChild`的目标节点并不在当前列表中；页面同时会重新查询并替换整块列表，你怎么定位？

答：先核对删除时使用的父节点和目标节点是否来自同一轮查询，整块替换后旧引用可能已不属于新容器。删除前检查父子关系或重新按稳定标识查找，再调用`removeChild()`；若目标可能已被其他流程删除，应把并发更新设计成幂等。

5) 组件要判断按钮是否配置`disabled`，评审中一方读取`getAttribute('disabled')`，另一方用`hasAttribute('disabled')`；这里只关心属性是否存在，该怎么选？

答：应使用`hasAttribute('disabled')`，因为需求判断的是属性存在性，而不是取得其字符串值。需要读取具体属性内容时才用`getAttribute()`，写入和清理分别用`setAttribute()`、`removeAttribute()`；不要把缺失属性和值为空混作同一业务状态。

590. 掌握页面的加载过程

页面加载就是浏览器获取并从上到下解析HTML，遇到外部资源按需下载，解析到后逐步渲染页面。通常会暂停文档解析，等脚本下载、解析和执行完再继续，所以放在里可能拖慢内容展示。工程中可以把脚本放到文档底部，或用`defer`、`async`控制加载和执行时机；前者通常等解析完成后执行，后者下载完成就执行

且顺序不可预测。判断完成状态时，DOMContentLoaded表示初始HTML已解析，load则还要等待DOM、CSS、JS和图片全部加载。

网页加载流程 当我们打开网址的时候，浏览器会从服务器中获取到 HTML 内容浏览器获取到 HTML 内容后，就开始从上到下解析 HTML 的元素元素内容会先被解析，此时浏览器还没开始渲染页面我们看到元素里有用于描述页面元数据的元素，还有一些元素涉及外部资源（如图片、CSS 样式等），此时浏览器会去获取这些外部资源。除此之外，我们还能看到元素中还包含着不少的元素，这些元素通过src属性指向外部资源当浏览器解析到这里时（步骤 3），会暂停解析并下载 JavaScript 脚本当 JavaScript 脚本下载完成后，浏览器的控制权转交给 JavaScript 引擎。当脚本执行完成后，控制权会交回给渲染引擎，渲染引擎继续往下解析HTML页面此时元素内容开始被解析，浏览器开始渲染页面

我们看到元素里有用于描述页面元数据的元素，还有一些元素涉及外部资源（如图片、CSS 样式等），此时浏览器会去获取这些外部资源。除此之外，我们还能看到元素中还包含着不少的元素，这些元素通过src属性指向外部资源

在这个过程中，我们看到中放置的元素会阻塞页面的渲染过程：把 JavaScript 放在里，意味着必须把所有 JavaScript 代码都下载、解析和解释完成后，才能开始渲染页面。如果外部脚本加载时间很长（比如一直无法完成下载），就会造成网页长时间失去响应，浏览器就会呈现“假死”状态，用户体验会变得很糟糕因此，对于对性能要求较高、需要快速将内容呈现给用户的网页，常常会将 JavaScript 脚本放在的最后面。这样可以避免资源阻塞，页面得以迅速展示。我们还可以使用defer/async/preload等属性来标记标签，来控制 JavaScript 的加载顺序

延迟加载的方式有哪些

js 的加载、解析和执行会阻塞页面的渲染过程，因此我们希望

js 脚本能够尽可能的延迟加载，提高页面的渲染速度。 几种方式是：

将 js 脚本放在文档的底部，来使 js 脚本尽可能的在后来加载执行给 js 脚本添加defer属性，这个属性会让脚本的加载与文档的解析同步解析，然后在文档解析完成后再执行这个脚本文件，这样的话就能使页面的渲染不被阻塞。多个设置了defer属性的脚本按规范来说最后是顺序执行的，但是在一些浏览器中可能不是这样给 js 脚本添加async属性，这个属性会使脚本异步加载，不会阻塞页面的解析过程，但是当脚本加载完成后立即执行 js脚本，这个时候如果文档没有解析完成的话同样会阻塞。多个async属性的脚本的执行顺序是不可预测的，一般不会按照代码的顺序依次执行动态创建DOM标签的方式，我们可以对文档的加载事件进行监听，当文档加载完成后再动态的创建script标签来引入 js 脚本

怎么判断页面是否加载完成 Load事件触发代表页面中的DOM，CSS，JS，图片已经全部加载完毕。DOMContentLoaded事件触发代表初始的HTML被完全加载和解析，不需要等待CSS，JS，图片加载

1) 活动页把同步放在，网络慢时长时间不出现；后端说HTML已经返回，所以不该白屏，你怎么解释？

答：浏览器自上而下解析到该脚本时会暂停HTML解析，等待脚本下载并交给JavaScript引擎执行，之后才继续处理。因此响应到达不等于页面可渲染；可把非关键脚本移到文档末尾，或按依赖关系选择defer、async。

2) 页面有三个存在依赖顺序的脚本，又不希望它们阻塞文档解析；一位开发选async，另一位选defer，你支持谁？

答：支持defer，它让脚本下载与文档解析并行，并在文档解析完成后执行，多个脚本按规范保持顺序。async下载完成就立即执行，执行时仍可能打断尚未完成的解析，而且多个脚本的先后不可预测，不适合存在依赖的链路。

3) 第三方统计脚本完全独立，但产品要求数据尽早上报，前端又不能让它持续阻塞首屏；你会在async、defer和动态插入之间怎么选？

答：独立且希望尽早执行时可选async，它不会等待文档解析完成，但下载完成后的执行仍可能短暂阻塞解析。若必须等文档完成再加载，可监听加载事件后动态创建script；选择取决于上报时效与渲染影响，不能假定async保证执行顺序。

4) 监控显示DOMContentLoaded已触发，但load又过了很久才发生，页面包含大图、样式和多个外部脚本；你如何判断卡在哪个资源？

答：这表示初始HTML已完成加载和解析，但页面依赖的其他资源尚未全部结束。结合网络请求检查仍在下载或失败重试的CSS、JS、图片等资源；load要等页面资源完成，不能只凭两事件间隔断定某一种资源。

5) 旧项目把所有脚本都移到末尾，新负责人要求统一改成async，理由是“两者都不会阻塞首屏”；你会接受统一替换吗？

答：不接受统一替换，末尾脚本按文档顺序遇到并执行，而async的执行顺序不可预测，下载完成时还可能打断解析。独立脚本适合async，有依赖顺序且需延后执行的脚本更适合defer；动态插入则适用于明确等待某个加载事件的场景。

591. 从输入URL到页面展示过程

输入URL后，浏览器会完成域名解析、建立连接、收发HTTP响应，再把资源渲染成可见页面。域名先经过多级缓存或DNS查询得到IP，随后通过三次握手建立TCP连接；如果是HTTPS，还要进行TLS握手来加密传输。服务器处理请求时会校验if-none-match、if-modified-since等缓存信息，缓存有效返回304，否则以200返回资源。浏览器拿到内容后构建DOM、计算样式和布局，再分层、栅格化并合成显示；大页面通常优先处理视口附近的图块，避免无意义的绘制开销。

1. DNS域名解析 根 DNS 服务器：返回顶级域 DNS 服务器的 IP 地址 顶级域 DNS 服务器：返回权威 DNS 服务器的 IP 地址 权威 DNS 服务器：返回相应主机的 IP 地址 DNS的域名查找，在客户端和浏览器，本地DNS之间的查询方式是递归查询；在本地DNS服务器与根域及其子域之间的查询方式是迭代查询；在客户端输入 URL 后，会有一个递归查找的过程，从浏览器缓存中查找->本地的hosts文件查找->找本地DNS解析器缓存查找->本地DNS服务器查找，这个过程中任何一步找到了都会结束查找流程。如果本地DNS服务器无法查询到，则根据本地DNS服务器设置的转发器进行查询。若未用转发模式，则迭代查找过程如下图：

结合起来的图表示：在查找过程中，有以下优化点：DNS存在着多级缓存，从离浏览器的距离排序的话，有以下几种：浏览器缓存，系统缓存，路由器缓存，IPS服务器缓存，根域名服务器缓存，顶级域名服务器缓存，主域名服务器缓存。在域名和IP的映射过程中，给了应用基于域名做负载均衡的机会，可以是简单的负载均衡，也可以根据地址和运营商做全局的负载均衡。

2. 建立TCP连接 首先，判断是不是https的，如果是，则HTTPS其实是HTTP + SSL / TLS 两部分组成，也就是在HTTP上又加了一层处理加密信息的模块。服务端和客户端的信息传输都会通过TLS进行加密，所以传输的数据都是加密后的数据 进行三次握手，建立TCP连接。第一次握手：建立连接。客户端发送连接请求报文段第二次握手：服务器收到SYN报文段。服务器收到客户端的SYN报文段，需要对这个SYN报文段进行确认第三次握手：客户端收到服务器的SYN+ACK报文段，向服务器发送ACK报文段 SSL握手过程 第一阶段 建立安全能力包括协议版本 会话Id 密码构件 压缩方法和初始随机数第二阶段 服务器发送证书 密钥交换数据和证书请求，最后发送请求-相应阶段的结束信号第三阶段 如果有证书请求客户端发送此证书之后客户端发送密钥交换数据 也可以发送证书验证消息第四阶段 变更密码构件和结束握手协议 完成了之后，客户端和服务端就可以开始传送数据 发送HTTP请求，服务器处理请求，返回响应结果 TCP连接建立后，浏览器就可以利用HTTP/HTTPS协议向服务器发送请求了。服务器接受到请求，就解析请求头，如果头部有缓存相关信息如if-none-match与if-modified-since，则验证缓存是否有效，若有效则返回状态码为304，若无效则重新返回资源，状态码为200

这里有发生的一个过程是HTTP缓存，是一个常考的考点，大致过程如图： 3. 关闭TCP连接 4. 浏览器渲染 按照渲染的时间顺序，流水线可分为如下几个子阶段：构建 DOM 树、样式计算、布局阶段、分层、栅格化和显示。如图：渲染进程将 HTML 内容转换为能够读懂DOM树结构。渲染引擎将 CSS 样式表转化为浏览器可以理解的styleSheets，计算出DOM节点的样式。创建布局树，并计算元素的布局信息。对布局树进行分层，并生成分层树。为每个图层生成绘制列表，并将其提

交到合成线程。合成线程将图层分图块，并栅格化将图块转换成位图。合成线程发送绘制图块命令给浏览器进程。浏览器进程根据指令生成页面，并显示到显示器上。 构建 DOM 树 转码 (Bytes -> Characters) —— 读取接收到的 HTML 二进制数据，按指定编码格式将字节转换为 HTML 字符串Tokens 化 (Characters -> Tokens) —— 解析 HTML，将 HTML 字符串转换为结构清晰的 Tokens，每个 Token 都有特殊的含义同时有自己的一套规则构建 Nodes (Tokens -> Nodes) —— 每个 Node 都添加特定的属性（或属性访问器），通过指针能够确定 Node 的父、子、兄弟关系和所属 treeScope（例如：iframe 的 treeScope 与外层页面的 treeScope 不同）构建 DOM 树 (Nodes -> DOM Tree) —— 最重要...

1) 用户输入域名后很快拿到IP，同事据此说“接下来浏览器只要发HTTP请求，不需要再建立连接”；如果地址是HTTPS，你怎么纠正？

答：拿到IP只完成了域名解析，通常还需通过三次握手建立TCP连接，并在其上完成TLS安全协商，之后才能传输加密的HTTP数据。缓存或连接复用可能缩短部分阶段，但不能把DNS命中等同于请求已经可发送。

2) 跨地区用户首次打开页面时域名解析明显偏慢，架构组准备调整权威解析和负载均衡；前端排查报告应把完整查询链路写到什么程度？

答：应先记录浏览器缓存、hosts、本地解析器缓存到本地DNS的递归查找，再说明本地DNS未命中时会向根域、顶级域和权威DNS迭代查询。域名到IP的映射还能承载按地区或运营商的负载均衡，但实际慢点需用观测数据确认。

3) 接口请求带上If-None-Match和If-Modified-Since后返回304，开发却仍等待一个完整响应体再渲染；这里的缓存流程应怎样理解？

答：304表示服务器验证缓存仍有效，浏览器应复用本地资源，而不是等待服务器重新发送完整实体。缓存无效或资源已变化时才重新返回资源并通常使用200；条件请求仍发生了网络往返，因此它与完全不发请求的本地命中不同。

4) 线上偶发“接口响应很快但页面仍白屏”，性能记录显示时间集中在样式计算和布局之后；你会沿渲染流水线怎么查？

答：响应完成后仍要经历构建DOM、样式计算、生成布局树、分层、生成绘制列表、栅格化和显示。应定位是复杂样式与布局计算、过大的图层，还是视口附近图块迟迟未完成栅格化；只看接口耗时无法解释渲染进程和合成线程的开销。

5) 长列表页面高达数十屏，产品要求首次打开就栅格化全部内容以保证滚动绝对流畅，浏览器工程师反对；你站哪边？

答：支持按视口附近图块优先栅格化，因为用户首次只能看到视口内的一小部分，提前处理全部大图层会产生不必要开销。合成线程负责分块并安排栅格化，再把绘制命令交给浏览器进程显示；过度预处理还可能拖慢首屏。

6) 面试中候选人把“构建DOM”直接说成“把字符串变成节点数组”，但线上乱码只发生在某种响应编码下；你会怎样继续追问并判断？

答：构建DOM前先要按指定编码把接收的字节转成字符，编码判断错误就可能在分词前产生乱码。随后才是字符转Token、Token转带属性和关系的节点，最终建立父子兄弟关系形成DOM树；节点数组的说法遗漏了转码和树结构。

592. 渲染引擎什么情况下才会为特定的节点创建新的图层

当节点形成独立层叠上下文，或者内容需要被裁剪时，渲染引擎会为它创建新的图层。常见触发条件可以归为定位与层级、视觉效果和渲染隔离，例如非auto的z-index、非none的transform、小于1的opacity

，以及will-change。本质上，这些属性要求节点独立参与合成或控制绘制范围。比如固定尺寸容器中的内容发生溢出时，文字裁剪区域会单独成层；如果还有滚动条，滚动条也可能成为独立图层。层叠上下文是HTML元素的三维概念，这些HTML元素在一条假想的相对于面向（电脑屏幕的）视窗或者网页的用户的z轴上延伸，HTML元素依据其自身属性按照优先级顺序占用层叠上下文的空间。

拥有层叠上下文属性的元素会被提升为单独的一层。拥有层叠上下文属性：根元素 (HTML), z-index 值不为 "auto" 的绝对/相对定位元素，position, 固定 (fixed) / 沾滞 (sticky) 定位 (沾滞定位适配所有移动设备上的浏览器，但老的桌面浏览器不支持) z-index 值不为 "auto" 的 flex 子项 (flex item)，即：父元素 display: flex | inline-flex, z-index 值不为 "auto" 的 grid 子项，即：父元素 display: grid opacity 属性值小于 1 的元素 (参考 the specification for opacity)，transform 属性值不为 "none" 的元素，mix-blend-mode 属性值不为 "normal" 的元素，filter 值不为 "none" 的元素，perspective 值不为 "none" 的元素，clip-path 值不为 "none" 的元素 mask / mask-image / mask-border 不为 "none" 的元素 isolation 属性被设置为 "isolate" 的元素在 will-change 中指定了任意 CSS 属性 (参考 这篇文章) -webkit-overflow-scrolling 属性被设置 "touch" 的元素 contain 属性值为 "layout", "paint", 或者综合值比如 "strict", "content" 需要剪裁 (clip) 的地方也会被创建为图层。这里的剪裁指的是，假如我们把 div 的大小限定为 200 * 200 像素，而 div 里面的文字内容比较多，文字所显示的区域肯定会超出 200 * 200 的面积，这时候就产生了剪裁，渲染引擎会把裁剪文字内容的一部分用于显示在 div 区域。出现这种裁剪情况的时候，渲染引擎会为文字部分单独创建一个层，如果出现滚动条，滚动条也会被提升为单独的层。

1) 商品卡片只设置了 position:

relative，没有 z-index，同事却断言它一定会生成独立图层；你会怎么纠正这个判断？

答：不能仅凭 position: relative 得出结论，源码列出的条件是绝对或相对定位元素同时满足 z-index 不为 auto。还要区分层叠上下文与渲染引擎实际创建的图层，不能把所有定位元素一概视为已经提层。

2) 长列表里有 500 个卡片，为了准备悬浮动画统一加了 will-change:

transform，上线后资源占用反而增加，你会怎么改？

答：不应给全部卡片长期声明 will-change，它会向渲染引擎提示相关属性即将变化，可能促使元素提前拥有独立层。只在动画临近开始时作用于实际运动的卡片，并在结束后移除；大量常驻图层会增加资源和合成管理成本。

3) 设计师把半透明浮层的 opacity 从 0.99 改成 1，同时去掉了 transform，你会预期它的层级行为发生什么变化？

答：opacity 小于 1 和 transform 不为 none 都属于源码列出的层叠上下文条件，两者移除后，这个浮层不再因它们获得独立层。它仍可能被 filter、isolation、定位与 z-index 等其他属性影响，因此必须结合完整样式判断。

4) 一个 200px × 200px 的滚动容器出现文字残影，现场只检查了 z-index；你会继续查哪些与分层有关的线索？

答：应继续检查容器的裁剪范围、溢出内容和滚动条，因为源码指出需要剪裁时，文字内容与滚动条可能被分别创建为图层。再核对 transform、filter、clip-path、mask 和 contain 等属性，避免把裁剪问题误判成单纯的层叠顺序错误。

5) 弹窗既能用定位配合非 auto 的 z-index，也能在父容器上设置 isolation:

isolate；评审时你会如何取舍？

答：两者都会建立新的层叠上下文，但表达的约束不同：前者直接参与定位元素的层叠排序，后者用于隔离内部元素与外部混合。应按弹窗的层级关系选择，不能只为“提层”叠加属性，否则会制造新的层叠边界并增加排查难度。

593. 定时器与requestAnimationFrame、requestIdleCallback

定时任务用setTimeout，动画更新用requestAnimationFrame，低优先级工作用requestIdleCallback。setTimeout到点只是把回调加入任务队列，主线程忙时仍要等待，并不保证准时执行。requestAnimationFrame会跟随屏幕刷新，在重绘前回调，后台标签页还会暂停，但主线程阻塞时同样可能掉帧。requestIdleCallback适合切割长任务或发送数据，不过只有当前帧有空闲时间才会执行，必要时可设置timeout。1. setTimeout setTimeout的运行机制：执行该语句时，是立即把当前定时器代码推入事件队列，当定时器在事件列表中满足设置的时间值时将传入的函数加入任务队列，之后的执行就交给任务队列负责。但是如果此时任务队列不为空，则需等待，所以执行定时器内代码的时间可能会大于设置的时间

```
setTimeout(()=>{console.log(1);},0)console.log(2);
```

 输出 2, 1;
setTimeout的第二个参数表示在执行代码前等待的毫秒数。上面代码中，设置为0，表面意思为 执行代码前等待的毫秒数为0，即立即执行。但实际上的运行结果我们也看到了，并不是表面上看起来的样子，千万不要被欺骗了。实际上，上面的代码并不是立即执行的，这是因为setTimeout有一个最小执行时间，HTML5标准规定了setTimeout()的第二个参数的最小值（最短间隔）不得低于4毫秒。当指定的时间低于该时间时，浏览器会用最小允许的时间作为setTimeout的时间间隔，也就是说即使我们把setTimeout的延迟时间设置为0，实际上可能为4毫秒后才事件推入任务队列。定时器代码在被推送到任务队列前，会先被推入到事件列表中，当定时器在事件列表中满足设置的时间值时会被推到任务队列，但是如果此时任务队列不为空，则需等待，所以执行定时器内代码的时间可能会大于设置的时间

```
setTimeout(()=>{console.log(111);},100);
```

上面代码表示100ms后执行console.log(111)，但实际上实行的时间肯定是大于100ms后的，100ms只是表示 100ms 后将任务加入到"任务队列"中，必须等到当前代码（执行栈）执行完，主线程才会去执行它指定的回调函数。要是当前代码耗时很长，有可能要等很久，所以并没有办法保证，回调函数一定会在setTimeout()指定的时间执行。2. setTimeout 和 setInterval区别 setTimeout: 指定延期后调用函数，每次setTimeout计时到后就会去执行，然后执行一段时间后才继续setTimeout,中间就多了误差，（误差多少与代码的执行时间有关）。setInterval：以指定周期调用函数，而setInterval则是每次都精确的隔一段时间推入一个事件（但是，事件的执行时间不一定就不准确，还有可能是这个事件还没执行完毕，下一个事件就来了）。btn.onclick=function(){setTimeout(function(){console.log(1);},250);} 点击按钮后，首先将onclick事件处理程序加入队列。该程序执行后才设置定时器，再有250ms后，指定的代码才被添加到队列中等待执行。如果上面代码中的onclick事件处理程序执行了300ms，那么定时器的代码至少要在定时器设置之后的300ms后才会被执行。队列中所有的代码都要等到javascript进程空闲之后才能执行，而不管它们是如何添加到队列中的。如图所示，尽管在255ms处添加了定时器代码，但这时候还不能执行，因为onclick事件处理程序仍在运行。定时器代码最早能执行的时机是在300ms处，即onclick事件处理程序结束之后。3. setInterval存在的一些问题： JavaScript中使用setInterval开启轮询。定时器代码可能在代码再次被添加到队列之前还没有完成执行，结果导致定时器代码连续运行好几次，而之间没有任何停顿。而javascript引擎对这个问题的解决是：当使用setInterval()时，仅当没有该定时器的任何其他代码实例时，才将定时器代码添加到队列中。这确保了定时器代码加入到队列中的最小时间间隔为指定间隔。

但是，这样会导致两个问题：某些间隔被跳过；多个定时器的代码执行之间的间隔可能比预期的小 假设，某个onclick事件处理程序使用setInterval()设置了200ms间隔的定时器。如果事件处理程序花了300ms多一点时间完成，同时定时器代码也花了差不多的时间，就会同时出现跳过某间隔的情况 例子中的第一个定时器是在205ms处添加到队...

594. Vue 响应式原理

Vue响应式本质上是拦截数据读写：读取时收集依赖，修改时通知Watcher更新视图。Vue 2用Object.defineProperty()递归劫持属性，数组则重写push、splice等变异方法，再通过虚拟DOM的对比局部更新真实DOM。它无法直接感知对象属性的新增和删除，需要使用Vue.set()、Vue.delete()。Vue

3改用Proxy代理对象和数组，嵌套对象可在读取时继续交给reactive处理。

Vue

的响应式原理是核心是通过 ES5 的保护对象的Object.defineProperty中的访问器属性中的 get 和 set 方法，data 中声明的属性都被添加了访问器属性，当读取 data 中的数据时自动调用 get 方法，当修改 data 中的数据时，自动调用 set 方法，检测到数据的变化，会通知观察者 Wacher，观察者 Wacher自动触发重新render 当前组件（子组件不会重新渲染），生成新的虚拟 DOM 树，Vue 框架会遍历并对比新虚拟 DOM 树和旧虚拟 DOM 树中每个节点的差别，并记录下来，最后，加载操作，将所有记录的不同点，局部修改到真实 DOM树上。

虚拟DOM (Virtual DOM): 用 js 对象模拟的，保存当前视图内所有 DOM 节点对象基本描述属性和节点间关系的树结构。用js对象，描述每个节点，及其父子关系，形成虚拟 DOM 对象树结构。因为只要在data中声明的基本数据类型的数据，基本不存在数据不响应问题，所以重点介绍数组和对象在vue中的数据响应问题，vue可以检测对象属性的修改，但无法监听数组的所有变动及对象的新增和删除，只能使用数组变异方法及\$set方法。可以看到，arrayMethods首先继承了Array，然后对数组中所有能改变数组自身的方法，如push、pop等这些方法进行重写。重写后的方法会先执行它们本身原有的逻辑，并对能增加数组长度的 3 个方法push、unshift、splice方法做了判断，获取到插入的值，然后把新添加的值变成一个响应式对象，并且再调用ob.dep.notify()手动触发依赖通知，这就很好地解释了用vm.items.splice(newLength) 方法可以检测到变化

总结：Vue

采用数据劫持结合发布—订阅模式的方法，通过Object.defineProperty()来劫持各个属性的

setter, getter，在数据变动时发布消息给订阅者，触发相应的监听回调。 Observer遍历数据对象，给所有属性加上setter和getter，监听数据的变化compile解析模板指令，将模板中的变量替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，收到通知，更新视图

Watcher订阅者是Observer和Compile之间通信的桥梁，主要做的事情在自身实例化时往属性订阅器

(dep)

里面添加自己待属性变动dep.notice()通知时，调用自身的update()方法，并触发Compile中绑定的回调

Object.defineProperty()，那么它的用法是什么，以及优缺点是什么呢？可以检测对象中数据发生的修改对于复杂的对象，层级很深的话，是不友好的，需要经行深度监听，这样子就需要递归到底，这也是它的缺点。对于一个对象中，如果你新增加属性，删除属性，**Object.defineProperty()**是不能观测到的，

那么应该如何解决呢？可以通过Vue.set()和Vue.delete()来实现。 // 模拟 Vue 中的 data 选项letdata={msg:'hello'}// 模拟 Vue 的实例letvm={}// 数据劫持:当访问或者设置 vm 中的成员的时候，做一些干预操作Object.defineProperty(vm,'msg',{//

可枚举(可遍历)enumerable:true,// 可配置(可以使用 delete 删除，可以通过 defineProperty 重新定义)configurable:true://

当获取值的时候执行get(){console.log('get: ',data.msg)returndata.msg};//

当设置值的时候执行set(newValue){console.log('set: ',newValue)if(newValue===data.msg){return}data.msg=newValue// 数据更改，更新 DOM 的值document.querySelector('#app').textContent=data.msg}}// 测试vm.msg='Hello World'...

1) Vue 2表单里执行form.newKey =

1后控制台数据已变化，但输入区没有更新；有人说是虚拟DOM的diff漏了，你同意吗？

答：不同意，根因更可能是新增属性没有被Object.defineProperty()转换成getter和setter，变化因而没有进入依赖通知链路。应使用Vue.set()或\$set添加属性；若是删除则使用Vue.delete()，而不是

先怀疑diff。

2) 一个多层配置对象初始化后会频繁替换子对象，你要用Observer、Dep和Watcher解释一次更新链路，会怎么说？

答：Observer会把已声明的多层属性转换为访问器，并在新对象被赋值时继续处理其成员；读取阶段由Dep收集对应的Watcher。写入触发通知后，Watcher更新视图并生成新的虚拟DOM，再比较差异并局部修改真实DOM。

3) 列表页直接执行items[3] = row和items.length =

2后视图不同步，但改成splice就恢复了；你如何解释并修复？

答：在Vue 2中，并非所有数组变动都能被访问器机制观察，而splice等变异方法经过重写，会处理新增值并调用依赖通知。应使用框架支持的数组变异方法或\$set；直接索引赋值和改长度不能假定会触发视图更新。

4) 把同一份深层对象从Vue 2迁到Vue

3后，评审者认为Proxy只代理第一层，所以嵌套字段不会响应；你怎么回应？

答：外层Proxy的确先拦截第一层访问，但读取结果若仍是对象，可以在Reflect.get后继续通过reactive建立代理，从而实现深度观测。它通常是按访问过程继续代理，并不等于所有嵌套对象失去响应性；具体行为仍要检查实际实现。

5) Vue 3数组一次写入触发了多次get、set日志，页面也重复更新；你会在哪些条件上收紧触发逻辑？

答：应检查写入的key是否是目标对象自身属性，并比较旧值与新值，避免没有实际变化时仍调用trigger。数组操作可能经过多个代理陷阱，不能把每次拦截都等同于有效变更；条件过宽会造成多余通知，过窄则可能漏更新。

6) 有人主张在大型对象初始化时递归给所有层级做defineProperty比Proxy更合适，因为两者都能深度监听；你如何权衡？

答：defineProperty需要遍历并改造已有属性，新增和删除还要借助Vue.set()、Vue.delete()；深层对象会放大递归处理成本。Proxy能直接监听对象和数组，并可在读取到嵌套对象时继续代理，但它属于不同的运行机制，迁移不能只替换一个API。

595. 发布订阅模式和观察者模式

两种模式的核心区别是有没有事件中心：观察者模式直接关联双方，发布订阅模式通过中间层转发。在观察者模式中，目标对象保存观察者，状态变化后直接调用其update()，所以双方存在依赖。发布订阅中，发布者只负责\$emit，订阅者只负责\$on，彼此不需要知道对方。像兄弟组件通过eventBus通信，就是典型的发布订阅场景。

1. 发布/订阅模式 发布者发布者信号中心 订阅者发布者信号中心

我们假定，存在一个"信号中心"，某个任务执行完成，就向信号中心"发布"(publish)一个信号，其他任务可以向信号中心"订阅"(subscribe)这个信号，从而知道什么时候自己可以开始执行。这就叫做"发布/订阅模式"(publish-subscribe pattern)

Vue 的自定义事件 let vm=newVue()vm.\$on('dataChange',()=>{console.log('dataChange')})vm.\$on('dataChange',()=>{console.log('dataChange1')})vm.\$emit('dataChange')

兄弟组件通信过程 // eventBus.js//

事件中心leteventHub=newVue()// ComponentA.vue// 发布者addTodo:function(){// 发布消息(事件)eventHub.\$emit('add-todo',{text:this.newTodoText})this.newTodoText=""// ComponentB.vue// 订阅者created:function(){// 订阅消息(事件)eventHub.\$on('add-todo',this.addTodo)} 模拟 Vue 自定义事件的实现

```
class EventEmitter {
  constructor() {
    this.subs = {}
  }
  $on(eventType, handler) {
    if (!this.subs[eventType]) this.subs[eventType] = []
    this.subs[eventType].push(handler)
  }
  $emit(eventType, ...args) {
    if (this.subs[eventType]) {
      this.subs[eventType].forEach(handler => handler(...args))
    }
  }
}
```



```

eventType,fn){this.subs[eventType]=this.subs[eventType]||[]this.subs[eventType].push(fn)}//
发布通知$emit(eventType){if(this.subs[eventType]){this.subs[eventType].forEach(v=>v())}}//
测试varbus=newEventEmitter()// 注册事件bus.$on('click',function(){console.log('click')})bus.$on('cli
ck',function(){console.log('click1')})// 触发事件bus.$emit('click')
2. 观察者模式 观察者(订阅者)
--Watcherupdate():当事件发生时，具体要做的事情目标(发布者) --Depsubs数组:存储所有的观察者add
Sub():添加观察者notify():当事件发生，调用所有观察者的update()方法没有事件中心
update():当事件发生时，具体要做的事情
subs数组:存储所有的观察者addSub():添加观察者notify():当事件发生，调用所有观察者的update()方法
// 目标(发布者)// DependencyclassDep{constructor(){// 存储所有的观察者this.subs=[]}
添加观察者addSub(sub){if(sub&&sub.update){this.subs.push(sub)}}//
通知所有观察者notify(){this.subs.forEach(sub=>sub.update())}
观察者(订阅者)classWatcher{update(){console.log('update')}}
测试letdep=newDep()letwatcher=newWatcher()dep.addSub(watcher)dep.notify()
3. 总结
观察者模式是由具体目标调度，比如当事件触发，Dep就会去调用观察者的方法，所以观察者模 式的订
阅者与发布者之间是存在依赖的发布/订阅模式由统一调度中心调用，因此发布者和订阅者不需要知道对方的
存在

```

1) 订单页里OrderStore直接保存三个视图观察者并逐个调用update()，同事却把它叫作发布订阅；你会怎么追问他？

答：这更符合观察者模式，因为具体目标直接持有观察者，并在状态变化时调用它们的update()，中间没有独立信号中心。应让他指出发布者与订阅者之间的调度媒介；若只能找到目标内部的订阅列表，就不能按发布订阅描述。

2) 兄弟组件通过全局eventHub.\$emit('add-todo')通信，新增页面也要监听同名事件；你会要求怎样组织注册与清理？

答：应把事件名称、载荷约定和订阅位置集中管理，并在组件不再需要监听时解除对应处理函数。发布订阅让双方通过事件中心解耦，但也隐藏了调用关系；监听增长后若缺少生命周期管理，会留下重复执行和难以追踪的订阅。

3) 需求改成一个发布者要通知多个完全不认识的业务模块，而且模块可以独立上下线；你会选直接观察还是信号中心？

答：更适合使用带信号中心的发布订阅，让发布者只发布事件，各模块自行订阅，不需要互相持有引用。代价是执行链路不再直观，必须约束事件命名、载荷和取消订阅；若参与者固定且关系紧密，直接观察反而更简单。

4) 线上一次点击打印了三遍click，代码里只有一次\$emit；你会怎样定位这个EventEmitter故障？

答：先检查同一处理函数是否被多次\$on，以及组件重复创建时旧订阅是否仍保留，因为\$emit会遍历事件类型对应的全部处理函数。再记录注册与销毁路径确认订阅数量；只在触发处防抖会掩盖重复注册，不能消除泄漏。

5) 状态库里的Dep已经能通知Watcher，架构师仍要求再套一层全局事件中心；你认为值得吗？

答：若Dep已直接维护并通知明确的Watcher，再加信号中心会把观察者链路改成更间接的调度，未必带来收益。只有确实需要让互不认识的发布者和订阅者解耦时才值得引入；否则会增加事件契约和故障定位成本。

6) 评审中有人说Watcher既然叫订阅者，Vue响应式就必然是发布订阅模式；你会如何串联两个概念？

答：“订阅者”这个角色名不足以决定模式，关键是通知由谁调度。Dep保存Watcher并直接调用其update()时属于观察者结构；只有发布者和订阅者通过独立信号中心交互，才符合这里的发布订阅模式。

596. 为什么使用 Virtual DOM

使用虚拟DOM，是为了用普通对象描述视图，并在状态变化后统一计算真实DOM应该怎样更新。它会保留上一次视图状态，通过diff找出前后差异，只把必要变化同步到页面，避免开发者手动维护复杂的DOM操作。它在复杂视图下有助于控制渲染成本，但不能简单理解为任何场景都更快。更重要的是它解耦了具体渲染平台，同一套视图描述还能用于SSR、原生应用或小程序。手动操作DOM比较麻烦，还需要考虑浏览器兼容性问题，虽然有jQuery等库简化DOM操作，但是随着项目的复杂DOM操作复杂提升为了简化DOM的复杂操作于是出现了各种MVVM框架，MVVM框架解决了视图和状态的同步问题为了简化视图的操作我们可以使用模板引擎，但是模板引擎没有解决跟踪状态变化的问题，于是Virtual DOM出现了Virtual DOM的好处是当状态改变时不需要立即更新DOM，只需要创建一个虚拟树来描述DOM，Virtual DOM内部将弄清楚如何有效(diff)的更新DOM虚拟DOM可以维护程序的状态，跟踪上一次的状态通过比较前后两次状态的差异更新真实DOM 虚拟DOM的作用 维护视图和状态的关系复杂视图情况下提升渲染性能除了渲染DOM以外，还可以实现SSR(Nuxt.js/Next.js)、原生应用(Weex/React Native)、小程序(mpvue/uni-app)等

1) 活动页只有一个计数器，候选人说引入Virtual DOM后每次更新一定比精准设置textContent更快；你认同吗？

答：不能作出“一定更快”的结论，因为Virtual DOM还要创建虚拟树并比较前后状态，精准的直接更新可能路径更短。它的核心价值是维护视图与状态关系，并由框架确定差异更新；简单节点不应仅凭性能口号选型。

2) 复杂表单有几十处状态联动，团队现在到处手写DOM查询和赋值；引入Virtual DOM能解决哪部分工程问题？

答：它能用虚拟树描述当前视图，在状态变化后比较新旧状态，只把记录到的差异更新到真实DOM。这样状态与视图的同步由统一机制维护，减少散落的手动操作；但业务状态设计不合理时，虚拟DOM本身不会消除逻辑复杂度。

3)

同一套组件既要在浏览器渲染，又计划支持SSR和原生容器；你会怎样评价虚拟树这层抽象？

答：虚拟树不只用于浏览器DOM，还可作为视图描述支持SSR、原生应用或小程序等不同渲染目标。它让状态与具体宿主操作之间多一层抽象，但各目标仍需要对应渲染实现；不能把“能描述”误解为无需适配即可运行。

4) 线上列表更新后出现旧行残留，研发只盯着真实DOM截图；按虚拟DOM链路你会依次核对什么？

答：先确认状态是否真的变化，再检查新虚拟树是否正确描述目标视图，以及新旧树比较是否记录了应有差异，最后核对差异是否应用到真实DOM。若状态源本身错误，继续调DOM更新没有意义；若有人绕过框架直接改DOM，也会破坏前后状态的一致性。

5) 可视化大屏每帧都要更新大量节点，一方坚持全部交给虚拟DOM，另一方要求全部手写；你会如何决策？

答：不能只凭技术立场决定，应先区分高频热点区域与普通状态视图，并验证比较和真实更新分别占据多少成本。复杂视图可借虚拟DOM维护状态关系，高频且更新路径固定的局部则可能需要更直接的策略；混用时必须划清所有权，避免双方同时修改同一节点。

6) 模板引擎已经能根据数据生成HTML，产品经理质疑为何还要虚拟DOM；你会怎么解释两者边界？

答：模板能简化视图生成，但仅生成内容并不等于持续跟踪状态变化。虚拟DOM会保留上一次视图状态，在新状态到来时比较两棵虚拟树并更新差异；代价是增加运行时表示与比较过程，因此静态页面未必需要这层机制。

597. VDOM：三个 part

VDOM可以拆成三个部分：用VNode描述节点、通过diff记录差异、再根据patch更新真实DOM。VNode只是保留必要信息的普通JS对象，创建成本通常低于直接创建属性繁多的真实节点。数据变化先集中反映到虚拟树，再统一修改DOM tree，可以减少零散操作引发的重绘和回流。它的价值也不只在性能，还解耦了对HTML的依赖，方便做AOT、SSR和其他平台渲染。虚拟节点类，将真实DOM节点用js对象的形式进行展示，并提供render方法，将虚拟节点渲染成真实DOM节点diff比较：对虚拟节点进行js层面的计算，并将不同的操作都记录到patch对象re-render：解析patch对象，进行re-render 补充1VDOM的必要性？ 创建真实DOM的代价高：真实的DOM节点node实现的属性很多，而vnode仅仅实现一些必要的属性，相比起来，创建一个vnode的成本比较低。触发多次浏览器重绘及回流：使用vnode，相当于加了一个缓冲，让一次数据变动所带来的所有node变化，先在vnode中进行修改，然后diff之后对所有产生差异的节点集中一次对DOM tree进行修改，以减少浏览器的重绘及回流。 补充2：vue 为什么采用vdom？ 引入Virtual DOM在性能方面的考量仅仅是一方面。性能受场景的影响是非常大的，不同的场景可能造成不同实现方案之间成倍的性能差距，所以依赖细粒度绑定及Virtual DOM哪个的性能更好还真不是一个容易下定论的问题。Vue之所以引入了Virtual DOM，更重要的原因是为了解耦HTML依赖，这带来两个非常重要的好处是：不再依赖HTML解析器进行模版解析，可以进行更多的AOT工作提高运行时效率：通过模版AOT编译，Vue的运行时体积可以进一步压缩，运行时效率可以进一步提升；可以渲染到DOM以外的平台，实现SSR、同构渲染这些高级特性，Weex等框架应用的就是这一特性。 综上，Virtual DOM在性能上的收益并不是最主要的，更重要的是它使得Vue具备了现代框架应有的高级特性。

1) 商品列表更新后页面已经变了，同事却说VDOM就是缓存的DOM字符串；结合一次更新链路，你会怎么纠正他？

答：VDOM不是HTML字符串，而是用精简的JavaScript对象描述节点。数据变化后先生成并比较虚拟节点，把差异记录为patch，再解析patch更新真实DOM；如果直接跳过比较操作真实节点，就不属于这条更新链路。

2) 一个表格组件每次筛选都直接遍历并修改多处真实DOM，评审时你为什么可能要求改成先计算vnode差异再统一更新？

答：多处真实DOM修改可能连续触发重绘和回流，而虚拟节点可充当更新缓冲。应先在JavaScript层完成diff、汇总为patch，再集中修改DOM tree；但更新范围很小且路径固定时，不能仅凭使用VDOM就断言一定更快。

3) 一个画布编辑器每帧只移动单个元素，负责人仍坚持“用了VDOM性能必然最好”，你会怎样判断？

答：不能把VDOM视为所有场景下的性能保证，它还要承担创建虚拟节点和执行diff的成本。应根据更新频率、节点规模和真实DOM操作范围验证收益；对高度可预测的细粒度更新，直接绑定或定向修改可能更合适。

4) 线上出现“状态已变化但局部视图没更新”，你沿着vnode → diff → patch → re-render会怎么定位？

答：先确认新状态是否生成了预期vnode，再检查diff是否识别出对应差异并写入patch。若patch正确，则继续查看解析和真实DOM更新阶段；逐段核对能区分响应数据、差异计算与提交渲染三类故障。

5) 团队要让同一套组件既支持浏览器页面又支持SSR和非DOM平台，为什么VDOM的价值不只在减少回流？

答：虚拟节点把组件描述与浏览器HTML、真实DOM解耦，使渲染结果能交给不同平台的渲染器处理。模板还可提前编译为render function，承载更多AOT优化；代价是增加虚拟节点、diff和平台适配层的复杂度。

598. vue 和 react技术选型

Vue和React都能做数据驱动、组件化和服务端渲染，选型关键是项目复杂度与团队匹配度。Vue提供v-model，写法直观，但底层仍是单向数据流；React更强调不可变数据和单向更新。大规模数据渲染及多人协作的复杂项目，可以考虑React配合Redux；追求快速开发的小型项目，Vue通常更顺手。开发风格上，React常用JSX，而Vue多用单文件组件组织模板、逻辑和样式。

相同点：数据驱动页面，提供响应式的视图组件都有virtual DOM,组件化的开发，通过props参数进行父子之间组件传递数据，都实现了webComponents规范数据流动单向，都支持服务器的渲染SSR都有支持native的方法，react有React native，vue有wexx

不同点：数据绑定：Vue实现了双向的数据绑定v-model，底层本质上还是单向数据流（v-model隐藏了背后:value 和 @input 的细节）数据渲染：大规模的数据渲染，react更快使用场景：React配合Redux架构适合大规模多人协作复杂项目，Vue适合小快的项目开发风格：react推荐做法jsx + inline style把html和css都写在js了 vue是采用webpack +vue-loader单文件组件格式，html, js, css同一个文件

1) 表单页用了v-model，评审同事因此认定Vue是双向数据流、React才是单向数据流，你会怎么解释？

答：v-model提供双向绑定的使用体验，但底层仍可理解为值向下传递、事件向上通知。它隐藏了类似:value与@input的配对细节，并没有推翻单向数据流；复杂表单仍要明确数据所有者，避免状态来源不清。

2) 五十人协作的大型业务平台要在Vue与React间选型，架构负责人只拿模板语法简洁度做决定，你会补充哪些判断？

答：大型多人协作项目应重点评估状态管理、代码约束和团队协作方式，来源材料更倾向React配合Redux。Vue的单文件组件适合集中组织模板、逻辑和样式，但规模结论不是绝对规则，最终仍要结合团队能力与既有生态。

3) 数据看板要渲染大规模数据，团队因为材料写着“React更快”就准备整体迁移，你会批准吗？

答：不会仅凭框架级结论批准迁移，材料只给出了大规模数据渲染中React更快的概括，未限定组件结构和更新模式。应先定位瓶颈并用当前页面验证；迁移会引入重写与协作成本，而且无法替代列表裁剪等针对性优化。

4) 新项目既要浏览器端SSR，半年后又可能扩展原生端，产品要求现在就锁定技术栈，你怎样比较两者？

答：两者都支持组件化、单向数据流和服务端渲染，因此SSR本身不足以拉开差距。原生方向可比较React

Native与Weex相关方案及团队能力；跨端只是选型约束之一，不能忽略生态成熟度和维护成本。

5) 线上页面出现父组件数据已变、子组件显示仍旧值，开发者把原因归结为“Vue和React数据流不同”，你会先查什么？

答：两者都通过props进行父子数据传递，并以单向数据流组织更新，不能先用框架差异解释故障。应检查父组件传值、子组件是否复制并滞留旧状态，以及更新是否实际触发；若使用v-model，还要核对值与事件链路。

599. nextTick

nextTick用来把回调推迟到下一轮DOM更新结束后执行，从而拿到更新后的页面节点。因为Vue不会在每次数据变化时立刻更新DOM，而是把任务放进队列并统一清空，多次调用也会先收集起来。实现时会根据环境依次尝试Promise、MutationObserver、setImmediate，最后才使用setTimeout。所以修改数据后马上读取尺寸或内容时，我一般会放到nextTick回调里。

nextTick可以让我们在下次DOM更新循环结束之后执行延迟回调，用于获得更新后的DOM

nextTick主要使用了宏任务和微任务。根据执行环境分别尝试采用

PromiseMutationObserversetImmediate如果以上都不行则采用setTimeout

定义了一个异步方法，多次调用nextTick会将方法存入队列中，通过这个异步方法清空当前队列

1) 详情页执行this.count++后立刻读取节点文本仍是旧值，同事认为响应式失效了；你会怎样解释并修正？

答：数据变化不代表真实DOM已在当前同步代码中完成更新，因此立即读取可能仍是旧内容。把读取逻辑放进nextTick，让它在下一次DOM更新循环结束后执行；它只保证等待本轮更新，不负责修复错误的数据依赖。

2)

批量导入页在一个同步循环里调用了二十次nextTick，这些回调会各自启动一次定时任务吗？

答：多次调用会先把回调保存到同一队列，再由选定的异步方法清空当前队列，而不是简单理解为每次都独立刷新。回调仍应保持轻量；如果其中执行大量计算，即使合并调度也会阻塞后续渲染。

3) 老旧运行环境没有原生Promise，业务依赖nextTick更新后聚焦输入框，它是否必然失效？

答：不必然失效，nextTick会按环境尝试Promise、MutationObserver、setImmediate，最后可退回setTimeout。不同回退方式属于微任务或宏任务，执行时机可能有差异；业务不应依赖比“更新循环结束后”更细的顺序假设。

4) 弹窗打开后偶发拿不到更新后的输入框，日志显示数据已改且nextTick已调用，你会怎样排查？

答：先确认修改数据和注册nextTick的先后关系，并检查输入框是否受条件渲染控制。随后在回调中验证节点是否已生成，排除组件尚未进入对应更新循环；若节点由后续异步数据决定，一次nextTick并不能等待那次未来更新。

5) 开发者想把所有nextTick都替换成setTimeout(fn, 0)，理由是最终都能异步执行，你会接受吗？

答：不应直接等价替换，nextTick面向的是本轮DOM更新结束后的回调队列，并会选择当前环境可用的异步机制。setTimeout只是最终回退方案，通常属于更晚的宏任务；替换会削弱与框架更新周期之间的语义关联。

600. 生命周期

Vue生命周期描述了组件从初始化、挂载、更新到销毁的完整过程，不同阶段适合处理不同事情。beforeCreate时状态还没初始化，到了created已完成数据响应性，但真实DOM尚未生成；执行渲染和挂载后才进入mounted。数据变化时会经过依赖通知、差异比较，并依次触发beforeUpdate和updated；销毁时则在对应钩子间移除节点、依赖和监听。keep-alive组件不会直接销毁，而是通过激活与失活钩子切换状态。

init initLifecycle/Event，往vm上挂载各种属性callHook: beforeCreated:

实例刚创建initInjection/initState: 初始化注入和data响应性created: 创建完成，属性已经绑定，但还未生成真实dom`进行元素的挂载：\$el / vm.\$mount()是否有template: 解析成render function*.vue文件:vue-loader会将编译成render functionbeforeMount: 模板编译/挂载之前执行render function，生成真实的dom，并替换到dom tree中mounted: 组件已挂载*.vue文件:vue-loader会将编译成render function update

执行diff算法，比对改变是否需要触发UI更新flushScheduleQueue watcher.before: 触发beforeUpdate钩子 -watcher.run(): 执行watcher中的notify，通知所有依赖项更新UI触发updated钩子: 组件已更新activated / deactivated(keep-alive): 不销毁，缓存，组件激活与失活destroybeforeDestroy: 销毁开始销毁自身且递归销毁子组件以及事件监听remove(): 删除节点watcher.teardown(): 清空依赖vm.\$off(): 解绑监听destroyed: 完成后触发钩子 beforeDestroy: 销毁开始销毁自身且递归销毁子组件以及事件监听remove(): 删除节点watcher.teardown(): 清空依赖vm.\$off(): 解绑监听destroyed: 完成后触发钩子 remove(): 删除节点watcher.teardown(): 清空依赖vm.\$off(): 解绑监听

上面是vue的声明周期的简单梳理，接下来我们直接以代码的形式来完成vue的初始化 newVue({})// 初始化Vue实例function _init(){// 挂载属性initLifeCycle(vm)// 初始化事件系统，钩子函数等initEvent(vm)// 编译slot、vnodeinitRender(vm)// 触发钩子callHook(vm,'beforeCreate')// 添加inject功能initInjection(vm)// 完成数据响应性 props/data/watch/computed/methodsinitState(vm)// 添加 provide 功能initProvide(vm)// 触发钩子callHook(vm,'created')// 挂载节点if(vm.\$options.el){vm.\$mount(vm.\$options.el)}// 挂载节点实现function mountComponent(vm){// 获取 render functionif(!this.options.render){// template to render// Vue.compile = compileToFunctionslet{render}=compileToFunctions()this.options.render=render}// 触发钩子callHook('beforeMount')// 初始化观察者// render 渲染 vdom，vdom=vm.render()// update: 根据 diff 出的 patches 挂载成真实的 domvm._update(vdom)// 触发钩子callHook(vm,'mounted')// 更新节点实现function queueWatcher(watcher){nextTick(flushScheduleQueue)}// 清空队列function flushScheduleQueue(){// 遍历队列中所有修改for(){// beforeUpdatewatcher.before()// 依赖局部更新节点watcher.update()callHook('updated')}}// 销毁实例实现Vue.proto...

1) 图表页在created中读取this.\$refs.chart得到undefined，但接口请求能正常发出；这两个现象为什么不矛盾？

答：created时数据、方法和注入等状态已经初始化，因此可以启动不依赖真实节点的逻辑。此时组件尚未挂载并生成真实DOM，所以\$refs不可用；图表初始化应放到mounted，同时考虑卸载时释放实例。

2) 后台页一次修改多项响应式数据，开发者期望每次赋值都立即触发一轮beforeUpdate和updated，你会如何纠正？

答：更新会进入调度队列，由flushScheduleQueue统一处理相关watcher。更新前触发beforeUpdate，依赖完成更新后再触发updated；不要把钩子次数与赋值次数机械对应，也不应在updated中无条件继续改状态。

3) 带keep-alive的搜索页切走后定时器仍在运行，但组件没有触发销毁钩子，你会把清理放在哪里？

答：keep-alive切换时组件只是失活并被缓存，不会走完整销毁流程，因此应在deactivated暂停定时器或其他活跃任务。再次进入时在activated恢复；真正销毁仍需在beforeDestroy或onBeforeUnmount等阶段完成最终清理。

4) 单页应用运行数小时后内存持续上涨，路由离开组件时节点消失了，你会沿销毁阶段检查哪些链路？

答：先确认是否进入beforeDestroy、执行自身及子组件移除，再核对watcher.teardown()和vm.\$off()是否完成。还要检查业务自行注册的全局监听、定时器或第三方实例；框架解绑自身事件，并不必然覆盖所有外部资源。

5) 团队把Vue 2组件迁到Vue

3，准备逐个把created改成onCreated，你会怎么调整方案？

答：Vue 3没有对应的onCreated，beforeCreate和created的初始化职责通常由setup承接。其余阶段使用onBeforeMount、onMounted、onBeforeUpdate、onUpdated、onBeforeUnmount与onUnmounted；迁移时还要重新确认逻辑对真实DOM的依赖。

601. vue-router

vue-router的本质是监听URL变化，匹配路由配置，再通过router-view渲染对应组件。hash模式监听hashchange，history模式监听popstate，区别主要在于感知地址变化的方式。代码跳转可以用this.\$router.push()，模板里则常用router-link，最终都驱动当前路由状态变化。它作为插件通过install接入Vue，注册全局组件，并用响应式的current让匹配组件自动重新渲染。mode hashhistory 跳转 this.\$router.push() 占位 vue-router源码实现 作为一个插件存在:实现VueRouter类和install方法实现两个全局组件:router-view用于显示匹配组件内容，router-link用于跳转监控url变化:监听hashchange或popstate事件响应最新url: 创建一个响应式的属性current，当它改变时获取对应组件并显示 // 我们的插件：//

1.实现一个Router类并挂载期实例//

2.实现两个全局组件router-link和router-viewlet Vue;class VueRouter{// 核心任务：//

1.监听url变化constructor(options){this.\$options=options;// 缓存path和route映射关系// 这样找组件更快this.routeMap={}this.\$options.routes.forEach(route=>{this.routeMap[route.path]=route})//

数据响应式// 定义一个响应式的current，则如果他变了，那么使用它的组件会rerenderVue.util.defineReactive(this,'current','')// 请确保onHashChange中this指向当前实例window.addEventListener('hashchange',this.onHashChange.bind(this))window.addEventListener('load',this.onHashChange.bind(this))onHashChange()//

console.log(window.location.hash);this.current=window.location.hash.slice(1)||'/'}//

插件需要实现install方法//

接收一个参数，Vue构造函数，主要用于数据响应式VueRouter.install=function(_Vue){//

保存Vue构造函数在VueRouter中使用Vue=_Vue//

任务1：使用混入来做router挂载这件事情Vue.mixin({beforeCreate()//

只有根实例才有router选项if(this.\$options.router){Vue.prototype.\$router=this.\$options.router}}//

任务2：实现两个全局组件// router-link: 生成一个a标签，在url后面添加#// aaaa//

aaaVue.component('router-link',{props:{to:{type:String,required:true}},render(h){// h(tag, props,

```
children)returnh('a',{attrs:{href:'#'+this.to}},this.$slots.default)// 使用jsx// return
{this.$slots.default}}})Vue.component('router-view',{render(h){// 根据current获取组件并render//
current怎么获取?// console.log('render',this.$router.current);// 获取要渲染的组件letcomponent=null
const{routeMap,current}=this.$routerif(routeMap[current]){component=routeMap[current].comp
onent}returnh(component)}}})exportdefaultVueRouter
```

1) 订单页点击后没有整页刷新，同事却说它只是普通链接；路由插件实际还做了什么？

答：router-link负责生成可跳转的链接，router-view根据当前地址渲染匹配组件。插件还要监听hash change或popstate，并把最新地址写入响应式的current；仅生成标签不足以完成视图联动。

2) 你要实现一个最小版hash路由，配置里有二十条静态路径，为什么先构建routeMap，地址变化后又怎样更新页面？

答：初始化时把path与路由配置缓存到routeMap，可在地址变化时直接查找对应组件。监听hashchange和首次load，更新响应式current，依赖它的router-view就会重新渲染；该简化实现未覆盖动态参数等复杂匹配。

3) 产品要求把地址从#/detail改成/detail，开发只删除了#，结果浏览器前进后退不再更新视图，你会检查哪里？

答：无#的历史模式应监听popstate，不能继续只依赖hashchange。还要确认跳转逻辑确实更新浏览器历史，并同步响应式当前地址；仅改变链接外观不会自动补齐历史模式的监听和状态维护。

4) 页面地址已经变成#/about，但仍显示首页，你按源码链路会怎么定位？

答：先检查hashchange是否触发，以及处理函数中的this是否仍指向路由实例。再核对current是否截取为/about、routeMap是否存在该路径，最后检查router-view是否读取同一个路由实例；任一环节断开都会导致视图不更新。

5) 微前端团队争论用还是业务代码统一调用this.\$router.push()，你会如何划分？

答：声明式导航适合模板中明确可点击的页面入口，能让跳转意图直接体现在视图结构里。命令式this.\$router.push()更适合校验、保存或权限判断后的跳转；两者最终都要驱动地址变化，选择代价主要在控制流程和可读性。

602. vuex

Vuex是集中管理组件共享状态的方案，核心是用固定的数据流让状态变化可预测。state保存状态，getter负责派生数据，mutations同步修改状态，actions承载异步或业务逻辑并提交mutation，复杂应用再用modules拆分。其内部会让state响应式，并通过commit、dispatch找到对应处理函数。实际使用时，我只会把跨组件共享、需要统一管理的状态放进去，局部状态仍留在组件内。

集中式存储管理应用的所有组件的状态，并以相应的规则保证状态以可预测的方式发生变化

state: 状态中心 mutations: 更改状态 actions: 异步更改状态 getters: 获取状态 modules: 状态保存应用状态
将state分成多个modules，便于管理 状态- state state保存应用状态
exportdefaultnewVueX.Store({state:{counter:0},}) 状态变更- mutations

mutations用于修改状态，store.js

exportdefaultnewVueX.Store({mutations:{add(state){state.counter++}}}) 派生状态- getters

从state派生出新状态，类似计算属性 exportdefaultnewVueX.Store({getters:{doubleCounter(state){// 计算剩余数量 return state.counter * 2;}}}) 动作- actions 加业务逻辑，类似于controller

exportdefaultnewVueX.Store({actions:{add({commit}){setTimeout(()=>{})}}}) 测试代码: counter: {{store.state.counter}}async counter:

{{store.state.counter}}double:{{store.getters.doubleCounter}} vuex原理解析 实现一个插件:声明Sto

re类，挂载\$storeStore具体实现:创建响应式的state，保存mutations、actions和getters实现commit根据用户传入type执行对应mutation实现dispatch根据用户传入type执行对应action，同时传递上下文实现getters，按照getters定义对state做派生 创建响应式的state，保存mutations、actions和getters实现commit根据用户传入type执行对应mutation实现dispatch根据用户传入type执行对应action，同时传递上下文实现getters，按照getters定义对state做派生 //

目标1：实现Store类，管理state（响应式的），commit方法和dispatch方法//

目标2：封装一个插件，使用更容易使用letVue;classStore{constructor(options){// 定义响应式的state// this.\$store.state.xx// 借鸡生蛋this._vm=newVue({data:{\$\$state:options.state}})this._mutations=options.mutationsthis._actions=options.actions//

绑定this指向this.commit=this.commit.bind(this)this.dispatch=this.dispatch.bind(this)// 只读getstate(){returnthis._vm._data.\$\$state}setstate(val){console.error('不能直接赋值呀，请换别的方式！！天王盖地虎！！');};// 实现commit方法，可以修改statecommit(type,payload){// 拿出mutations中的处理函数执行它constentry=this._mutations[type]if(!entry){console.error('未知mutation类型');return}entry(this.state,payload)}dispatch(type,payload){constentry=this._actions[type]if(!entry){console.error('未知action类型');return}}//

上下文可以传递当前store实例进去即可entry(this,payload)}}functioninstall(_Vue){Vue=_Vue// 混入stor...

1) 商品详情页把接口请求直接写进mutation，点击购买时组件用commit触发；评审同事认为“反正最后都要改state”，你会接受吗？

答：不会，mutation应承担同步、可预测的状态变更，异步流程和业务编排应放进action，完成后再commit。把请求塞进mutation会让变更发生时间难以判断，也破坏按规则追踪状态的链路；纯同步修改则没必要额外套一层action。

2) 后台有用户、权限、订单三个域，所有state、mutation和action都堆在一个store.js，新增成员经常提交错同名操作，你会怎样拆？

答：我会按业务域拆成modules，让每个模块管理自己的state、mutations、actions和getters，组件只依赖对应域的公开能力。拆分边界应跟业务职责一致，而不是按文件数量平均切割；模块过细会增加跳转和调用成本。

3) 购物车页要求“一键清空”立即完成，而结算页要求先调用接口、成功后再清空；两处都想复用名为clearCart的逻辑，你会怎样安排commit和dispatch？

答：本地立即清空应由mutation完成，结算流程则由action负责异步调用，成功后再commit同一个清空操作。这样状态写入入口保持明确，业务流程也能复用；若接口失败仍提前提交，就需要额外回滚并可能造成界面与服务端不一致。

4) 线上点击“异步加一”后计数始终不变，控制台提示未知action类型，但同步按钮正常；你会沿着Vuex的哪条执行链排查？

答：先核对dispatch的type是否存在于已注册的actions，再确认该action是否通过上下文调用了正确的commit。Vuex的dispatch只负责找到并执行动作，真正修改state仍要落到mutation；模块注册或命名不一致也会在入口处中断。

5) 团队准备写一个轻量状态库，要求支持响应式state、同步提交和异步动作，但暂时不要模块系统；最小实现需要保留哪些机制？

答：至少要用Vue的响应式能力承载state，保存操作表，并实现按type查找的commit与dispatch，同时把实例注入组件成为\$store。还要绑定方法的this，并拒绝整体替换只读state；省略getters后就不具备统一派生状态能力。

603. vue3带来的新特性/亮点

Vue 3的亮点可以归为响应式升级、代码组织改善和编译运行性能优化。响应式从Object.defineProperty换成Proxy，代理对象本身；Composition API让同一业务逻辑聚合在一起，也更利于复用和类型推导。编译阶段会标记动态节点，使虚拟DOM的更新成本更多取决于动态内容，并配合Tree-shaking减少未使用代码。组件层面还加入了Fragment、Teleport和Suspense，分别覆盖多根节点、跨容器渲染和异步等待场景。

1. 压缩包体积更小 当前最小化并被压缩的 Vue 运行时大小约为 20kB（2.6.10 版为 22.8kB）。Vue 3.0捆绑包的大小大约会减少一半，即只有10kB！

2. Object.defineProperty -> Proxy Object.defineProperty是一个相对比较昂贵的操作，因为它直接操作对象的属性，颗粒度比较小。将它替换为es6的Proxy，在目标对象之上架了一层拦截，代理的是对象而不是对象的属性。这样可以将原本对对象属性的操作变为对整个对象的操作，颗粒度变大。javascript引擎在解析的时候希望对象的结构越稳定越好，如果对象一直在变，可优化性降低，proxy不需要对原始对象做太多操作。

3. Virtual DOM 重构 vdom的本质是一个抽象层，用javascript描述界面渲染成什么样子。react用jsx，没办法检测出可以优化的动态代码，所以做时间分片，vue中足够快的话可以不用时间分片

传统vdom的性能瓶颈：虽然 Vue 能够保证触发更新的组件最小化，但在单个组件内部依然需要遍历该组件的整个 vdom 树。传统 vdom 的性能跟模版大小正相关，跟动态节点的数量无关。在一些组件整个模版内只有少量动态节点的情况下，这些遍历都是性能的浪费。JSX 和手写的 render function 是完全动态的，过度的灵活性导致运行时可以用于优化的信息不足那为什么不直接抛弃vdom呢？高级场景下手写render function获得更强的表达力生成的代码更简洁兼容2.x

传统vdom的性能瓶颈：虽然 Vue 能够保证触发更新的组件最小化，但在单个组件内部依然需要遍历该组件的整个 vdom 树。传统 vdom 的性能跟模版大小正相关，跟动态节点的数量无关。在一些组件整个模版内只有少量动态节点的情况下，这些遍历都是性能的浪费。JSX 和手写的 render function 是完全动态的，过度的灵活性导致运行时可以用于优化的信息不足 那为什么不直接抛弃vdom呢？高级场景下手写render function获得更强的表达力生成的代码更简洁兼容2.x

vue的特点是底层为Virtual DOM，上层包含有大量静态信息的模版。为了兼容手写 render function，最大化利用模版静态信息，vue3.0采用了动静结合的解决方案，将vdom的操作颗粒度变小，每次触发更新不再以组件为单位进行遍历，主要更改如下 将模版基于动态节点指令切割为嵌套的区块每个区块内部的节点结构是固定的每个区块只需要以一个Array追踪自身包含的动态节点

vue3.0将 vdom 更新性能由与模版整体大小相关提升为与动态内容的数量相关

Vue 3.0 动静结合的 Dom diff Vue3.0提出动静结合的 DOM diff 思想，动静结合的 DOM diff其实是在预编译阶段进行了优化。之所以能够做到预编译优化，是因为 Vue core 可以静态分析 template，在解析模版时，整个 parse 的过程是利用正则表达式顺序解析模板，当解析到开始标签、闭合标签和文本的时候都会分别执行对应的回调函数，来达到构造 AST 树的目的。借助预编译过程，Vue 可以做到的预编译优化就很强大了。比如在预编译时标记出模版中可能变化的组件节点，再次进行渲染前 diff

时就可以跳过“永远不会变化的节点”，而只需要对比“可能会变化的动态节点”。这也就是动静结合的 DOM diff 将 diff 成本与模版大小正相关优化到与动态节点正相关的理论依据。

4. Performance vue3在性能方面比vue2快了2倍。

重写了虚拟DOM的实现运行时编译update性能提高SSR速度提高

5. Tree-shaking support vue3中的核心api都支持了tree-shaking，这些api都是通过包引入的方式而不是直接在实例化时就注入，只会对使用到的功能或特性进行打包（按需打包），这意味着更多的功能和更小的体积。

6. Composition API vue2中，我们一般会采用mixin来复用逻辑代码，用倒是挺好用的，不过也存在一些问题：例如代码来源不清晰、方法属性等...

1) 一个展示页只有标题会变化，模板里还有上百个静态节点；同事说Vue3更新时仍必须完整比较整棵Virtual DOM，你怎么判断？

答：这个判断忽略了Vue3 的编译期优化：模板会被分析为结构稳定的区块，并追踪其中可能变化的动态节点。更新成本因此更接近动态内容数量，而非模板总体大小；手写render function或高度动态的结构无法同等利用静态信息。

2) 组件库要同时提供表单、动画和状态能力，但业务项目只用了其中一部分，构建负责人担心Vue3 功能越多包越大；你会关注什么？

答：应关注这些核心API是否以可导入、可摇树的形式使用，让构建工具只保留实际引用的功能。Vue 3 的tree-shaking支持能减少未使用能力进入产物，但最终体积仍受业务代码、第三方依赖和构建配置影响，不能只凭框架特性下结论。

3) 管理后台频繁给响应式对象新增字段，技术负责人认为Vue3 仍要像基于Object.defineProperty的方案那样逐属性处理，你会如何解释选型变化？

答：Vue3 以Proxy在对象外层建立拦截，代理对象操作，而不是预先逐个改造已有属性。它减少了对原对象属性结构的直接操作，也更适合对象结构变化的场景；代价是运行环境必须具备相应的Proxy能力，不能把它当作旧机制的无条件替换。

4) 弹窗放在有overflow: hidden和复杂z-index的卡片组件里，线上始终被裁切；产品又要求弹窗逻辑仍归卡片维护，你会用Vue3 的什么能力？

答：可用Teleport把弹窗实际渲染到卡片外的目标DOM容器，同时保留它在原组件中的声明和逻辑归属。这样能绕开祖先容器的裁切或层叠限制；必须确保目标节点存在，并继续处理焦点、遮罩和关闭等交互职责。

5) 订单页的异步子组件加载前会短暂空白，团队在“父组件手写加载状态”和Vue3 内建组件之间争论，你会怎么取舍？

答：若等待发生在异步组件渲染阶段，可用Suspense提供default与fallback内容，把加载占位集中在边界上。简单请求或需要精细控制错误、重试与局部状态时，显式状态可能更清楚；Suspense解决等待展示，不会自动替代完整的异常处理。

6) 一个大型组件把搜索、分页和鼠标坐标逻辑分散在data、computed、methods与生命周期中，维护者找不到同一业务链；Vue3 哪项变化最直接？

答：Composition API允许按业务逻辑聚合相关状态、计算、监听和生命周期，并把鼠标等逻辑提取为可复用组合函数。它还能改善来源识别与类型推导；简单组件继续使用Options API也成立，强行组合反而可能增加抽象层次。

604. Composition api

Composition API是按业务功能组织和复用组件逻辑的一套组合式接口。Options API会把同一功能拆到data、computed、methods等区域，组件变大后阅读和维护都更费劲；组合式写法则能把相关状态与操作放在一起。相比mixin，它通过显式调用组合函数返回数据，来源更清楚，也降低了属性命名冲突和类型推断困难。它基于Vue响应式系统自动收集依赖，调用也不依赖固定顺序，但中小型、逻辑简单的组件并不一定必须改写。Composition API也叫组合式API，是Vue3.x的新特性。通过创建Vue 组件，我们可以将接口的可重复部分及其功能提取到可重用的代码段中。仅此一项就可以使我们的应用程序在可维护性和灵活性方面走得更远。然而，我们的经验已经证明，光靠这一点可能是不够的，尤其是当你的应用程序变得非常大的时候——想想几百个组件。在处理如此大的应用程序时，共享和重用代码变得尤为重要。Vue2.0中，随着功能的增加，组件变得越来越复杂，越来越难维护，而难以维护的根本原因是Vue的API设计迫使开发者使用watch，computed，methods选项组织代码，而不是实际的业务逻辑。另外Vue2.0缺少一种较为简洁的低成本的机制来完成逻辑复用，虽然可以mixin完成逻辑复用，但是当mixin变多的时候，会使得难以找到对应的data、computed或者method来源于哪个mixin，使得类型推断难以进行。所以Composition API的出现，主要是为了解决Options API带来的问题，第一个是代码组织问题，Composition API可

以让开发者根据业务逻辑组织自己的代码，让代码具备更好的可读性和可扩展性，也就是说当下一个开发者接触这一段不是他自己写的代码时，他可以更好的利用代码的组织反推出实际的业务逻辑，或者根据业务逻辑更好的理解代码。第二个是实现代码的逻辑提取与复用，当然mixin也可以实现逻辑提取与复用，但是像前面所说的，多个mixin作用在同一个组件时，很难看出property是来源于哪个mixin，来源不清楚，另外，多个mixin的property存在变量命名冲突的风险。而Composition API刚好解决了这两个问题。通俗的讲：没有Composition API之前vue相关业务的代码需要配置到option的特定的区域，中小型项目是没有问题的，但是在大型项目中会导致后期的维护性比较复杂，同时代码可复用性不高。Vue3.x中的composition-api就是为了解决这个问题而生的。composition api提供了以下几个函数：setup、ref、reactive、watch、effect、computed、toRefs、生命周期钩子。都说Composition API与React Hook很像，说说区别。从React Hook的实现角度看，React Hook是根据useState调用的顺序来确定下一次重渲染时的state是来源于哪个useState，所以出现了以下限制：不能在循环、条件、嵌套函数中调用Hook，必须确保总是在你的React函数的顶层调用Hook。useEffect、useMemo等函数必须手动确定依赖关系。而Composition API是基于Vue的响应式系统实现的，与React Hook的相比，声明在setup函数内，一次组件实例化只调用一次setup，而React Hook每次重渲染都需要调用Hook，使得React的GC比Vue更有压力，性能也相对于Vue来说也较慢。Composition API的调用不需要顾虑调用顺序，也可以在循环、条件、嵌套函数中使用响应式系统自动实现了依赖收集，进而组件的部分的性能优化由Vue内部自己完成，而React Hook需要手动传入依赖，而且必须保证依赖的顺序，让useEffect、useMemo等函数正确的捕获依赖变量，否则会由于依赖不正确使得组件性能下降。虽然Composition API看起来比React Hook好用，但是其设计思想也是借鉴React Hook的。

1) 一个表格组件用了五个mixin，模板里的load和rows找不到来源，另一个mixin又声明了同名属性；你会怎样重构？

答：应把各业务能力提取为显式调用的组合函数，例如由useTable返回rows和load，组件能直接看出来来源。需要重命名时可在解构或返回结构上主动处理；组合函数过度拆碎也会让调用关系分散，因此边界应围绕完整业务逻辑。

2) 搜索页的筛选、分页、请求取消逻辑分别散在watch、computed、methods和卸载钩子里，新成员改一次要跳四处；setup中你会怎么组织？

答：我会让同一搜索流程的ref、computed、watch和生命周期钩子相邻放置，再把稳定可复用的部分提取成useSearch。这种组织方式按业务关系而非选项类型聚合代码；若组合函数同时承担多个领域，仍会退化成难维护的大函数。

3) 权限组合函数只在管理员分支中需要，评审者套用React规则，禁止在条件分支调用它；在Vue的Composition API下这个限制成立吗？

答：不应机械套用React Hook的调用顺序规则，Vue的setup对每个组件实例执行一次，组合式调用不靠固定序号匹配状态。它可以出现在条件、循环或嵌套函数中；但分支调用产生的生命周期与资源清理仍要清晰，否则工程上依然容易遗漏。

4) 线上筛选条件变化后副作用没有执行，同事刚把相关读取移出watchEffect，却认为响应式系统会自动追踪文件里所有变量；你会怎样排查？

答：先确认响应式值是否在该副作用执行期间被实际读取，因为依赖收集来自运行中的访问关系，而不是静态扫描整个文件。再检查值是否仍由ref或reactive持有；脱离响应式容器或不在执行链中读取，都不会建立预期依赖。

5) 团队要在watch和watchEffect间选型：风控页必须明确监听账户和金额，并记录变更前值；哪一个更合适？

答：这里更适合watch，因为依赖目标需要显式表达，并且业务要使用新旧值进行审计或判断。watchEffect更适合让系统自动收集副作用中读取的依赖；自动收集虽简洁，但依赖边界不够显眼，复杂风控逻辑会更难审查。

6) React开发者说组合式函数就是换名后的Hook，并准备给每个副作用手写依赖数组；你会指出哪些实现差异？

答：Vue的组合式能力建立在响应式依赖收集上，watchEffect等机制可根据实际读取自动建立依赖，并不依靠Hook调用顺序。setup通常按组件实例执行一次，而ReactHook随重渲染再次调用；两者思想相近，但不能照搬依赖数组和顶层调用约束。

605. computed 的实现原理

computed本质上是带缓存的惰性观察者，通过dirty标记决定是否重新求值。首次读取时它会执行计算并缓存结果；依赖变化后，若当前没有订阅者，通常只把dirty设为true，等下次读取再计算。存在订阅者时，它会重新计算并比较新旧结果，最终值确实变化才通知渲染观察者，从而减少无效更新。因此，同一派生结果会被多次读取时适合用computed，单纯执行动作则不必依赖这层缓存。

computed本质是一个惰性求值的观察者computed

watcher。其内部通过this.dirty属性标记计算属性是否需要重新求值。当computed的依赖状态发生改变时，就会通知这个惰性的 watcher，computed watcher通过this.dep.subs.length判断有没有订阅者，有的话，会重新计算，然后对比新旧值，如果变化了，会重新渲染。(Vue

想确保不仅仅是计算属性依赖的值发生变化，而是当计算属性最终计算的值发生变化时才会触发渲染

watcher重新渲染，本质上是一种优化。)没有的话，仅仅把this.dirty = true(当计算属性依赖于其他数据时，属性并不会立即重新计算，只有之后其他地方需要读取属性的时候，它才会真正计算，即具备lazy（懒计算）特性。)

1) 报表页把totalPrice写成普通方法，模板在十几个位置调用，任意按钮触发重渲染都会重新遍历订单；改成computed为什么有意义？

答：computed由惰性的computed watcher管理，依赖未变化时可复用已有结果，不必在每次渲染中重复执行遍历。依赖变化后它才进入需要重新求值的状态；若计算本身极轻且只调用一次，缓存带来的收益可能并不明显。

2) 商品列表有一个昂贵的派生筛选值，但折叠面板关闭时没有任何模板或其他逻辑读取它；源数据更新后会马上重新计算吗？

答：在没有订阅者时，依赖变化通常只会把计算观察者的dirty标记设为true，不会立刻执行筛选。等面板展开并再次读取该值时才真正求值，这就是惰性计算；若其他观察者仍订阅它，就不能假设始终无人触发。

3) 一个computed同时依赖价格和折扣，价格变化后被连续读取三次；面试现场让你描述dirty从变化到复用的过程，你会怎么说？

答：依赖变化会通知对应的情性观察者，使其进入需要求值的状态；首次读取时重新执行计算并产出结果，随后读取可继续使用缓存。直到相关依赖再次变化前，不应重复计算；前提是这些依赖确实在求值过程中被响应式系统收集。

4) 线上日志显示源字段频繁变化，但依赖某个computed的视图并非每次都重渲染；你会先判断这是故障还是优化？

答：先比较计算属性最终的新旧值，而不是只看底层依赖是否变化；最终值未变时，不触发渲染观察者属于预期优化。若最终值已变化仍不更新，再检查依赖是否具有响应性、计算属性是否被订阅，以及读取链路是否真正建立。

5) 库存页需要在数量变化后写日志并请求补货，同事想把这些副作用塞进computed以利用缓存；你会如何取舍？

答：不应把日志和请求作为计算属性的核心职责，computed更适合根据响应式状态产生可缓存的派生值。副作用应放到侦听器或明确的业务动作中，以便控制触发和失败处理；否则惰性读取会让请求时机取决于“是否有人访问”。

606. watch 的理解

watch本质上是观察数据变化，并在变化后执行回调，它本身没有缓存能力。当监听对象内部的属性时，可以开启deep: true，让对象中的每一项都被监听。深度监听覆盖得更全面，但也会带来额外的性能开销。明确知道要观察哪个属性时，我一般会使用字符串形式监听，缩小监听范围。watch没有缓存性，更多的是观察的作用，可以监听某些数据执行回调。当我们需要深度监听对象中的属性时，可以打开deep: true选项，这样便会对对象中的每一项进行监听。这样会带来性能问题，优化的话可以使用字符串形式监听
注意：Watcher：观察者对象，实例分为渲染 watcher(render watcher),计算属性 watcher(computed watcher),侦听器 watcher (user watcher) 三种

1) 个人资料页把完整用户对象做成computed，只为了姓名变化时调用埋点；同事认为computed和watch都能依赖收集，你会接受吗？

答：不接受，埋点属于数据变化后的副作用，应使用watch执行回调，而computed应保留给派生值和缓存。混用会让读取动作夹带外部行为，触发时机也更难判断；若只是展示拼接姓名，才适合计算属性。

2) 配置中心监听一个包含大量嵌套字段的表单，任意字段改动都触发自动保存；直接设置deep: true有什么工程风险？

答：深度监听会遍历对象中的各项以建立监听，结构越大，初始化与变更处理的负担越值得警惕。更稳妥的是只监听确实影响保存的字段，或拆成多个边界明确的对象；若需求就是捕获任意深层修改，精确监听会增加维护清单。

3) 订单页原来只监听status，产品临时要求地址任一层级变化也重新校验；你会继续写单字段监听，还是改成深度监听？

答：若地址结构稳定且只有少数字段参与校验，优先精确监听这些路径，避免无关字段也触发回调。只有规则明确要求捕获整个地址对象的任意深层变化时才考虑deep: true；范围扩大后必须评估回调频率和对象规模。

4) 线上修改profile.contact.phone后校验没有执行，但替换整个profile对象时会执行；当前只监听了对象本身，你会怎么定位？

答：先确认侦听器是否只观察了对象引用，因为深层属性变化不等于引用被替换。随后核对目标字段路径，并选择精确的字符串路径监听或按需求开启deep；开启深度监听虽能覆盖该变化，也可能把无关字段纳入回调。

5) 审计模块既要计算账户风险等级，又要在等级变化时发送告警；架构师想全部交给一个侦听器，你会怎样拆分？

答：风险等级应由computed watcher负责派生和缓存，告警则由user watcher观察等级变化后执行回调。组件渲染仍由render watcher响应最终展示变化；拆分能区分计算、用户副作用和渲染职责，但要避免告警回调反向修改依赖造成循环。

607. vue 渲染过程

Vue的渲染过程是先把模板编译成render函数，再生成VNode，最后通过patch更新真实DOM。编译时，parse把模板转成AST，optimize标记静态节点，generate再生成render函数字符串。数据变化会触发渲染Watcher，重新执行render得到新的VNode。随后通过DOM

diff对比新旧节点，只对真实DOM做必要的添加、修改或删除。调用compile函数生成render函数字符串，编译过程如下:parse

使用大量的正则表达式对template字符串进行解析，将标签、指令、属性等转化为抽象语法树AST。模板 -> AST（最消耗性能）optimize 遍历AST，找到其中的一些静态节点并进行标记，方便在页面重渲染的时候进行diff比较时，直接跳过这一些静态节点，优化runtime的性能generate 将最终的AST转化为render函数字符串调用new

Watcher函数,监听数据的变化,当数据发生变化时，Render 函数执行生成 vnode 对象调用patch方法,对比新旧 vnode 对象,通过 DOM diff 算法,添加、修改、删除真正的 DOM 元素 parse

使用大量的正则表达式对template字符串进行解析，将标签、指令、属性等转化为抽象语法树AST。模板 -> AST（最消耗性能）optimize 遍历AST，找到其中的一些静态节点并进行标记，方便在页面重渲染的时候进行diff比较时，直接跳过这一些静态节点，优化runtime的性能generate 将最终的AST转化为render函数字符串

1) 商品详情页只修改购物车数量，为什么不能理解成Vue会重新创建整棵真实DOM？

答：数据变化会触发对应的Watcher，重新执行render生成新的VNode，但不会直接重建整棵真实DOM。随后patch对比新旧VNode，只对需要变化的真实节点执行添加、修改或删除；若节点结构或标识发生变化，复用范围才会缩小。

2) 后台表格有 3000

行，筛选条件每输入一个字符就明显卡顿，你会把耗时拆到渲染链路的哪些阶段？

答：先区分数据变化后执行render的脚本耗时，以及patch后真实DOM更新带来的布局和绘制耗时。若生成大量VNode已经很慢，应减少参与渲染的数据；若主要慢在真实节点更新，则应分页或缩小一次更新的节点规模。

3) 运营平台允许用户在线修改template，而普通业务组件在构建时已有render，两类组件的渲染链路有什么关键差异？

答：在线模板必须先经过parse、optimize和generate，得到render后才能进入创建VNode与patch的流程。构建阶段已经生成render的组件可跳过运行时编译；动态模板若频繁变化，编译成本会重新出现，不能只优化后续diff。

4) 线上页面数据已更新，控制台也能看到新值，但视图保持旧内容，你会沿着哪条链路定位断点？

答：先确认变化是否触发负责渲染的Watcher，再检查render是否生成了符合预期的新VNode，最后观察patch是否更新了真实DOM。若新旧VNode本身没有差异，应回查渲染依赖；若差异存在但节点未变，再检查节点匹配与补丁过程。

5) 内容团队要求首屏模板高度动态，性能团队要求彻底移除运行时编译，你会如何划分方案边界？

答：固定结构的组件应在构建阶段完成模板编译，运行时直接执行render，避免承担最耗性能的模板到AST转换。真正由用户配置结构的区域才保留动态编译，并限制其范围和触发次数；完全动态与完全预编译无法同时满足同一块模板。

608. 说一说keep-alive实现原理

keep-alive的核心是缓存组件对应的VNode，再次命中时直接从缓存中取出。它通过include和exclude按组件的name控制哪些需要缓存，参数可以是字符串、正则或数组。max用来限制缓存数量，超过上限后会删除最先进入的缓存项。这个淘汰过程采用LRU思路，让最近访问过的内容更有机会继续保留。

keep-alive组件接受三个属性参数：include、exclude、max

include指定需要缓存的组件name集合，参数格式支持String, RegExp, Array。当为字符串的时候，多个组件名称以逗号隔开。exclude指定不需要缓存的组件name集合，参数格式和include一样。max指定最多可缓存组件的数量,超过数量删除第一个。参数格式支持String、Number。原理

keep-alive实例会缓存对应组件的VNode,如果命中缓存，直接从缓存对象返回对应VNode LRU (Least recently used) 算法根据数据的历史访问记录来进行淘汰数据，其核心思想是“如果数据最近被访问过，那么将来被访问的几率也更高”。(墨菲定律：越担心的事情越会发生)

1) 订单列表被keep-alive包裹后，从详情页返回仍保留筛选条件，这是否说明它把页面真实DOM截图后直接恢复了？

答：不是，keep-alive的核心是缓存对应组件的VNode，再次命中时从缓存对象返回该VNode。保留页面状态来自组件实例及其虚拟节点被复用，而不是保存一份页面截图；未命中缓存时仍需按正常组件流程创建。

2) 移动端有商品、订单、消息三类列表，只希望订单页返回时保留状态，你会怎样配置include和组件名称？

答：给订单页设置稳定的组件name，并让include只匹配该名称，商品页和消息页便不会进入缓存。include支持String、RegExp或Array，字符串包含多个名称时用逗号分隔；名称不匹配会导致看似配置成功却始终无法命中。

3) 路由扩展到 20

个详情页，产品要求全部缓存，客户端负责人又要求限制缓存数量，你会怎么处理max？

答：设置max限制缓存组件数量，超出后按最近使用情况淘汰长期未访问的缓存项。max支持String或Number，但它只能控制数量，不能判断业务重要性；关键页面仍应结合include、exclude明确缓存范围。

4) 线上反馈用户返回列表时状态偶尔丢失，但同一路由有时又能恢复，你会优先核对哪些缓存条件？

答：先核对组件name是否稳定且确实命中include，再检查是否被exclude排除或因超过max而被淘汰。随后确认访问顺序，因为LRU会改变淘汰对象；若组件从未命中缓存，问题不在恢复阶段，而在缓存筛选条件。

5) 中后台有 30

个路由，团队争论是全部套keep-alive，还是只缓存高频表单页，你会支持哪种方案？

答：应只缓存确实需要保留编辑状态或页面上下文的组件，并用include或exclude明确名单，同时设置合理的max。全量缓存会让缓存集合持续扩大并触发淘汰，命中结果更依赖访问历史；选择性缓存更可控，但未缓存页面需要自行恢复状态。

609. 为什么访问data属性不需要带data

访问data属性时不需要再写data，是因为Vue把这些属性代理到了实例上。简单来说，读取this.xxx时，代理的get会转发到内部对象对应的属性；给this.xxx赋值时，set也会把新值写回去。这个代理通过Object.defineProperty实现，并将属性设置为可枚举、可配置，因此模板和实例代码都能用更简洁的方式访问数据。vue中访问属性代理this.data.xxx转换this.xxx的实现 /** 将 某一个对象的属性 访问 映射到对象的某一个属性成员上 */function proxy(target, prop, key){Object.defineProperty(target, key, {enum


```
erable:true,configurable:true,get(){return target[prop][key];},set(newVal){target[prop][key]=newVal;}};}
```

1) 组件里写了data: { count: 1

}，模板和方法都能访问this.count，这是否意味着count被复制到实例上了？

答：不是复制，而是在实例的count属性上通过Object.defineProperty建立访问代理。读取this.count时，get返回this.data.count；写入时，set再把新值交给对应的数据属性，因此两处不会维护两份独立状态。

2) 表单组件有 80

个字段，团队想手写this.data.xxx来减少一层代理，你会如何评价这个改动？

答：现有代理让模板和实例方法统一使用this.xxx，手写完整路径会改变组件的访问约定，却不会改变底层数据仍位于data的事实。是否值得改动应以实际测量为依据；仅凭多一次get转发，无法断言它就是表单卡顿来源。

3) 基础组件实例上已经存在status，业务又在data中声明同名字段并希望通过this.status访问，代理实现需要满足什么约束？

答：代理要求在目标实例上为该key定义可配置的访问属性，因此同名成员会产生覆盖或定义冲突风险。初始化阶段应检测并避免名称碰撞，必要时重命名业务字段；否则this.status指向哪一层会变得不可靠。

4) 线上出现this.total = 20后this.data.total仍是旧值，你会怎样验证代理是否被破坏？

答：先用属性描述符检查实例total是否仍具有代理的get和set，再确认它们是否指向预期的data.total。若该属性后来被重新定义，赋值就可能不再转发；若代理完整，则继续核对实际代理容器和字段名是否一致。

5) 代码评审中有人建议用一次性赋值this.count =

this.data.count代替访问器代理，两种方案的行为差异是什么？

答：一次性赋值只同步初始化时的值，之后修改实例属性或data.count都不会自动反映到另一处。访问器代理则让每次读写都落到同一个底层字段；代价是实例属性必须保留这组get、set，不能随意改成普通值属性。

610. template预编译是什么

模板预编译，就是在项目构建阶段把template转换成render function。如果留到运行时处理，组件实例化时需要先完成一次模板编译，之后才能执行渲染，这部分工作会带来运行时损耗。提前编译后，组件运行时可以直接使用生成的渲染函数，从而跳过模板编译。它优化的是组件启动和渲染前的准备过程，并不是让模板在运行时反复编译。对于Vue组件来说，模板编译只会在组件实例化的时候编译一次，生成渲染函数之后在也不会进行编译。因此，编译对组件的 runtime 是一种性能损耗。

而模板编译的目的仅仅是将template转化为render

function，这个过程，正好可以在项目构建的过程中完成，这样可以让实际组件在 runtime 时直接跳过模板渲染，进而提升性能，这个在项目构建的编译template的过程，就是预编译。

1) 一个组件只实例化一次，负责人因此认为运行时编译template没有成本，你会怎样反驳？

答：模板即使只在组件实例化时编译一次，也必须完成从template到render function的转换，这仍是一次运行时性能损耗。预编译把这段工作移到项目构建过程，使实际组件运行时直接执行渲染函数；实例少只能降低影响，不能让成本消失。

2) 营销页有 40 个固定组件模板，构建负责人问是否值得做预编译，你会给出什么落地判断？

答：这些模板在发布前已经确定，适合在构建阶段统一转换为render function，运行时无需再次处理模板。落地时应确认产物包含可直接执行的渲染函数；若仍把原始template留到浏览器处理，就没有获得预编译的核心收益。

3) 低代码平台的模板由运营发布后即时生效，架构组仍要求所有模板构建期预编译，这个约束为何无法直接满足？

答：构建期预编译只能处理构建时已经存在的模板，而运营发布的模板在运行时才确定，无法被先前构建产物完整覆盖。可把固定外壳预编译，将动态区域限制在必要范围；若动态结构也必须即时变化，就仍需承担相应的模板转换成本。

4) 线上首开页面卡顿，火焰图显示组件实例化阶段出现明显模板编译工作，你会检查构建链路的什么结果？

答：先检查发布产物中的组件是否已经携带render function，以及运行时是否仍接收到原始template。若模板没有在构建阶段转换，实例化时就会再次承担编译；若已有渲染函数，则应继续排查render和后续真实节点更新，而非归因于预编译。

5) 插件系统在“构建期预编译”和“运行时接收模板”之间选型，产品强调灵活性，性能团队强调启动速度，你会怎样定边界？

答：核心页面和稳定插件应优先预编译，让运行时直接进入渲染流程；必须动态下发结构的插件才保留运行时模板转换。前者牺牲发布后的即时结构变化，后者增加实例化阶段的编译工作，边界应由模板是否能在构建时确定来划分。

611. 介绍一下Vue中的Diff算法

Vue的Diff算法用于比较新旧虚拟DOM，找出同层节点中真正需要更新的部分。它先判断节点是否相同，不同就删除旧节点并创建新节点，相同则通过patchVnode继续处理子节点。双方都有多个子节点时，才进入updateChildren做核心比较，匹配到相同子节点后再递归处理。它不做跨层级比较，因此把原本可能达到 $O(n^3)$ 的复杂度降低到了 $O(n)$ 。在新老虚拟DOM对比时 首先，对比节点本身，判断是否为同一节点，如果不为相同节点，则删除该节点重新创建节点进行替换如果为相同节点，进行patchVnode，判断如何对该节点的子节点进行处理，先判断一方有子节点一方没有子节点的情况(如果新的children没有子节点，将旧的子节点移除)比较如果都有子节点，则进行updateChildren，判断如何对这些新老节点的子节点进行操作(diff核心)。匹配时，找到相同的子节点，递归比较子节点 在diff中，只对同层的子节点进行比较，放弃跨级的节点比较，使得时间复杂从 $O(n^3)$ 降低值 $O(n)$ ，也就是说，只有当新旧children都为多个子节点时才需要用核心的Diff算法进行同层级比较。

1) 资讯列表把一个卡片从第三层容器移到相邻栏目，内容完全相同，为什么不能期待diff跨层找到并直接复用？

答：该算法只比较同层子节点，主动放弃跨级匹配，因此相同内容移动到不同层级时不会按跨层节点处理。旧位置可能被删除，新位置再创建对应节点；这项约束换来了线性的同层比较范围，代价是跨层重排无法直接复用。

2) 商品列表更新 1000

条数据时，只有价格字段变化，你会如何解释从根节点到真实DOM的处理路径？

答：先比较新旧虚拟节点本身，若判定为同一节点，就进入patchVnode处理其子节点。双方都有多个子节点时进入updateChildren，找到相同子节点后递归比较，最终只把确定的差异落实到真实DOM；若节点不相同，则会删除并重新创建。

3) 空状态页从“无订单”切换为包含 20 条订单的列表，和两个列表各有 20 条数据时，是否都会进入核心updateChildren？

答：不会，只有新旧双方都具有多个子节点时，才需要进入核心的同层子节点比较。由无子节点变为有子节点时可直接添加，新的一方没有子节点时则移除旧子节点；因此应先判断子节点存在形态，再决定是否执行列表diff。

4) 线上更新一条记录后整行输入状态丢失，你怀疑节点被替换，会沿着diff的哪个判断排查？

答：先检查新旧行节点是否被判定为同一节点，因为一旦不相同，旧节点会被删除并创建新节点，内部状态也无法沿用。若节点相同，再进入patchVnode检查子节点处理；排查重点是生成的虚拟节点结构与身份条件，而不只是数据值。

5) 设计器团队希望支持任意节点跨层拖拽，同时要求每次更新都保持现有diff的低复杂度，你会如何说明取舍？

答：现有算法通过只比较同层子节点，把原本可能达到 $O(n^3)$ 的一般树比较收敛到 $O(n)$ 范围。任意跨层匹配会扩大搜索空间，不能直接沿用这一复杂度承诺；可把拖拽结果视为旧层删除、新层创建，或由业务层额外维护映射。

6) 代码评审时有人把patchVnode和updateChildren当成同一层逻辑，你会用“新节点没有子节点”的场景怎样区分？

答：patchVnode负责同一节点内部该如何更新，其中先判断新旧双方是否存在子节点。只有双方都有多个子节点时才调用updateChildren做核心同层比较；若新节点没有子节点，直接移除旧子节点即可，无需进入列表匹配。

612. 说说Vue2.0和Vue3.0有什么区别

Vue 3相比Vue 2，核心变化集中在响应式、代码组织和编译运行效率上。响应式从Object.defineProperty改为Proxy，可以直接监听数组变化和属性增删，也不必预先遍历对象的每个属性。它新增Composition API，让相关逻辑更容易组织和复用；同时重构虚拟DOM，对静态节点、slot和内联事件做编译优化。代码结构也更利于Tree shaking，并使用TypeScript替代Flow。 重构响应式系统，使用Proxy替换Object.defineProperty，使用Proxy优势：可直接监听数组类型的数据变化监听的目标为对象本身，不需要像Object.defineProperty一样遍历每个属性，有一定的性能提升可拦截apply、ownKeys、has等13种方法，而Object.defineProperty不行直接实现对象属性的新增/删除新增Composition API，更好的逻辑复用和代码组织重构Virtual DOM模板编译时的优化，将一些静态节点编译成常量slot优化，将slot编译为lazy函数，将slot的渲染的决定权交给子组件模板中内联事件的提取并重用（原本每次渲染都重新生成内联函数）代码结构调整，更便于Tree shaking，使得体积更小使用Typescript替换Flow 可直接监听数组类型的数据变化监听的目标为对象本身，不需要像Object.defineProperty一样遍历每个属性，有一定的性能提升可拦截apply、ownKeys、has等13种方法，而Object.defineProperty不行直接实现对象属性的新增/删除 模板编译时的优化，将一些静态节点编译成常量slot优化，将slot编译为lazy函数，将slot的渲染的决定权交给子组件模板中内联事件的提取并重用（原本每次渲染都重新生成内联函数）

1) 商品编辑页给一个已有响应式对象动态增加discount字段，Vue 2页面不更新，迁到Vue 3后却正常了；你怎么解释这种差异，又会提醒团队别得出什么错误结论？

答：差异来自响应式实现：Vue 2的Object.defineProperty主要在初始化时逐项劫持属性，后续新增或删除属性不能被直接感知；Vue 3的Proxy代理对象本身，可以拦截这类操作。不能因此认定所有更新都会更快，组件划分、依赖数量和渲染成本仍会决定实际表现。

2) 一个Vue 2后台有几十种动态表单，字段和数组项频繁增删；如果准备迁移到Vue 3，你会优先验证哪些响应式代码，而不是只确认页面能启动？

答：应优先检查依赖Vue.set、Vue.delete、数组变更补丁以及深层对象初始化的代码，因为迁移后这些路径会由Proxy直接跟踪。再用新增字段、删除字段和数组增删等真实操作做回归，确认计算属性

与视图同步；响应式改善并不能替代业务状态和表单边界测试。

3) 一个多人维护的复杂业务组件在Vue 2中把请求、筛选、分页都塞进mixins，升级时有人主张只换响应式实现、不引入Composition API；你会如何取舍？

答：只完成兼容迁移是更低风险的路径，Composition API并不是升级后必须立即采用。若组件长期存在逻辑分散、同名覆盖或复用困难，可按业务能力抽成组合函数并逐段验证；一次性同时改框架、状态结构和组织方式，会扩大回归范围，也更难定位问题。

4) 迁移后列表数据没变，但性能面板显示重复创建了大量模板内联事件函数；你会从Vue 3的编译与虚拟DOM优化角度检查什么？

答：先确认生产构建确实使用了对应的Vue 3模板编译器，并检查模板是否让静态节点、内联事件和slot获得编译期优化。Vue 3可提升静态内容、复用内联事件并将slot编译为惰性函数；若大量内容本身是动态的，这些优化空间会自然缩小。

5) 团队争论升级收益时，一方只谈Proxy，另一方只谈包体积；你会怎样给出更完整的技术判断？

答：应把收益拆成运行时、开发组织和构建产物三类：Proxy改善对象与数组变化追踪，Composition API改善逻辑复用，编译器和虚拟DOM重构减少部分重复工作。代码结构也更利于Tree Shaking，且实现语言由Flow转向TypeScript；最终收益仍受依赖兼容和业务代码形态约束。

【React】

613. 对虚拟DOM的理解

虚拟DOM的核心价值不是比手写DOM操作更快，而是让声明式更新和视图抽象成为可能。它作为局部更新链路的一环，用更轻量的数据结构承载视图，使新旧视图能够执行Diff，避免每次都做全量更新。直接、精准地操作真实DOM通常仍可能更快，因为人为已经省掉了比较过程。工程上的取舍是接受一定抽象成本，换取state = UI的开发方式以及更好的跨平台能力。虚拟dom从来不是用来和直接操作dom对比的，它们俩最终殊途同归。虚拟dom只不过是局部更新的一个环节而已，整个环节的对比对象是全量更新。虚拟dom对于state = UI的意义是，虚拟dom使diff成为可能（理论上也可以直接用dom对象diff，但是太臃肿），促进了新的开发思想，又不至于性能太差。但是性能再好也不可能好过直接操作dom，人脑连diff都省了。还有一个很重要的意义是，对视图抽象，为跨平台助力。其实我最终希望你明白的事情只有一件：虚拟DOM的价值不在性能，而在别处。因此想要从性能角度来把握虚拟DOM的优势，无异于南辕北辙。偏偏在面试场景下，10个人里面有9个都走这条歧路，最后9个人里面自然没有一个能自圆其说，实在让人惋惜。真正理解虚拟DOM(opens new window)

1) 活动页只有一个计数器，工程师说直接写textContent肯定比虚拟DOM快，所以虚拟DOM没价值；你会怎样回应这个反例？

答：单点且更新路径已知时，直接操作真实DOM通常少了一层创建和比较，虚拟DOM并不保证更快。它的价值是让开发者用state = UI描述界面，再通过差异计算完成通用的局部更新；若把比较对象限定成一次精准手工操作，结论本身就失去了公平性。

2) 一个表格页面每次筛选都重新生成整棵虚拟节点树，但最终只改了几个单元格；这层虚拟DOM在工程上解决了什么，又没有解决什么？

答：它提供了轻量的视图描述，使新旧结果能够执行diff，再把必要变更提交到真实DOM，而不必由业务代码维护每个更新路径。它没有消除生成对象和比较树的成本，也不会自动修复不合理的全量状态更新；组件边界和状态粒度仍需工程控制。

3) 同一套业务组件要同时运行在浏览器和原生容器中，产品要求交互逻辑复用但宿主控件完全不同；虚拟DOM在这里承担哪一层职责？

答：虚拟DOM抽象的是视图结构与变化，而不是某一种宿主节点，因此同一套组件描述可以交给不同渲染器落地。浏览器渲染器操作DOM，其他平台可映射到自身视图对象；跨端仍需处理平台能力、布局 and 交互差异，不能理解为业务代码百分之百无需适配。

4) 线上长列表滚动卡顿，监控只显示主线程繁忙，团队立即把责任归到diff；你会怎样验证，而不是直接移除虚拟DOM？

答：先区分耗时发生在状态计算、虚拟树创建与比较，还是最终布局、绘制和真实DOM操作，并用性能轨迹定位长任务。虚拟DOM只是局部更新链路中的一环，卡顿也可能来自全量重渲染或节点规模过大；没有分段证据就改渲染模型，往往只会转移成本。

5) 后台搭建器需要支持大量动态条件和组件插槽，一方主张手写DOM获得极致性能，另一方坚持虚拟DOM；你会如何做选型？

答：动态结构复杂且更新路径难以人工维护时，虚拟DOM能用统一声明式模型换取可组合性和可维护性，通常更适合作为默认方案。只有热点路径稳定、性能瓶颈已被证明确实位于通用比较时，才值得局部采用更直接的更新方式；代价是手工同步状态与视图的复杂度会上升。

614. 谈谈你对React的理解

React本质上是一个组件化的视图层框架，用声明式方式把状态映射成界面。声明式让界面逻辑更直观，组件化便于拆分和复用，也更容易做到高内聚、低耦合。借助虚拟DOM，同一套组件思想还能用于Web、Native和VR等场景。它只负责视图层，大型应用仍要结合社区方案处理路由和状态管理，因此会增加选型与学习成本。

React 是一个网页 UI 框架，通过组件化的方式解决视图层开发复用的问题，本质是一个组件化框架。

它的核心设计思路有三点，分别是声明式、组件化与通用性。声明式的优势在于直观与组合。组件化的优势在于视图的拆分与模块复用，可以更容易做到高内聚低耦合。通用性在于一次学习，随处编写。比如 React Native，React 360 等，这里主要靠虚拟 DOM 来保证实现。这使得 React 的适用范围变得足够广，无论是 Web、Native、VR，甚至 Shell 应用都可以进行开发。这也是 React 的优势。但作为一个视图层的框架，React 的劣势也十分明显。它并没有提供完整的一揽子解决方案，在开发大型前端应用时，需要向社区寻找并整合解决方案。虽然一定程度上促进了社区的繁荣，但也为开发者在技术选型和学习适用上造成了一定的成本。承接在优势后，可以再谈一下自己对于 React 优化的看法、对虚拟 DOM 的看法

1) 一个静态营销页只有几处按钮交互，候选人说用了React就一定比原生实现更先进；作为技术负责人你会怎么判断？

答：React的优势不在于天然更先进，而在于用声明式和组件化组织会持续变化、需要复用的视图。若页面结构简单且长期稳定，引入组件运行时与配套工具未必带来足够收益；选型应看状态复杂度、复用需求和团队维护成本，而不是框架标签。

2) 订单后台把筛选栏、表格和详情弹窗拆成二十多个组件，但改一个字段仍要跨层修改；为什么“拆成组件”没有自动实现高内聚低耦合？

答：组件化只提供拆分与组合机制，边界若按标签数量而非业务职责划分，依赖仍会散落在多层。应把相关状态、行为和视图收拢到稳定能力中，并明确输入输出；拆得过细会增加数据传递和理解成本

，拆得过粗又会削弱复用与隔离。

3) 同一支团队要把Web端的内容流迁到原生端，管理层认为React Native意味着整套页面直接复制；你会如何校正预期？

答：可复用的核心是声明式组件思维和部分业务逻辑，虚拟DOM所代表的视图抽象让不同渲染器成为可能。浏览器DOM与原生视图的能力、样式和交互并不相同，因此宿主相关代码仍要适配；“一次学习，随处编写”不等于“一份界面代码，所有平台无差别运行”。

4) 大型React项目上线前，路由、数据请求和状态管理分别来自不同社区库，升级时出现版本冲突；这是偶发管理失误，还是框架定位带来的成本？

答：这与React主要聚焦视图层的定位直接相关，它没有为大型应用提供完整的一揽子方案。社区生态带来选择空间，也把选型、集成和升级责任交给团队；应通过统一技术基线、封装边界和升级策略控制成本，但无法把这部分复杂度完全消除。

5) 架构评审中，一方看重声明式与跨端能力，另一方担心生态拼装成本；什么项目条件下你仍会选择React？

答：当界面状态复杂、组件复用价值高，或团队需要在多种渲染目标间共享开发模型时，React的声明式、组件化和通用性更有吸引力。若项目更需要开箱即用的全套约束，团队又缺少整合生态的能力，则其视图层定位可能成为负担；选择必须连同路由、数据和工程规范一起评估。

615. 如何避免React生命周期中的坑

避免生命周期的坑，关键是让渲染阶段保持纯净，把副作用和清理放到合适的提交阶段。Fiber让渲染可以暂停或重来，因此不要在已废弃的`componentWillMount`、`componentWillReceiveProps`和`componentWillUpdate`中请求数据或绑定事件。异步请求一般放在`componentDidMount`，更新前读取DOM可结合`getSnapshotBeforeUpdate`与`componentDidUpdate`。卸载时还要取消定时器和事件监听，并用错误边界避免渲染异常直接白屏。

16.3版本

>=16.4版本

在线查看：<https://projects.wojtekmaj.pl/react-lifecycle-methods-diagram>(opens new window)

避免生命周期中的坑需要做好两件事：不在恰当的时候调用了不该调用的代码；在需要调用时，不要忘了调用。那么主要有这么 7 种情况容易造成生命周期的坑`getDerivedStateFromProps`容易编写反模式代码，使受控组件与非受控组件区分模糊`componentWillMount`在 React 16.3 起被标记弃用、React 19 已移除（仅存`UNSAFE_componentWillMount`），不要在新代码中使用，主要原因是新的异步渲染架构会导致它被多次调用。所以网络请求及事件绑定代码应移至`componentDidMount`中。`componentWillReceiveProps`同样被标记弃用、React 19 已移除，被`getDerivedStateFromProps`所取代，主要原因是性能问题`shouldComponentUpdate`通过返回`true`或者`false`来确定是否需要触发新的渲染。主要用于性能优化`componentWillUpdate`同样是由于新的异步渲染机制而被标记废弃、React 19 已移除，不要在新代码中使用，原先的逻辑可结合`getSnapshotBeforeUpdate`与`componentDidUpdate`改造使用。如果在`componentWillUnmount`函数中忘记解除事件绑定，取消定时器等清理操作，容易引发

bug如果没有添加错误边界处理，当渲染发生异常时，用户将会看到一个无法操作的白屏，所以一定要添加`getDerivedStateFromProps`容易编写反模式代码，使受控组件与非受控组件区分模糊`componentWillMount`在 React 16.3 起被标记弃用、React 19 已移除（仅存`UNSAFE_componentWillMount`），不要在新代码中使用，主要原因是新的异步渲染架构会导致它被多次调用。所以网络请求及事件绑定代码应移至`componentDidMount`中。`componentWillReceiveProps`同样被标记弃用、React 19 已移除，被`getDerivedStateFromProps`所取代，主要原因是性能问题`shouldComponentUpdate`通过返回`true`或者`false`来确定是否需要触发新的渲染。主要用于性能优化`componentWillUpdate`同样是由于新的异步渲染机制而被标记废弃、React 19 已移除，不要在新代码中使用，原先的逻辑可结合`getSnapshotBeforeUpdate`与`componentDidUpdate`改造使用。如果在`componentWillUnmount`函数中忘记解除事件绑定，取消定时器

等清理操作，容易引发

bug如果没有添加错误边界处理，当渲染发生异常时，用户将会看到一个无法操作的白屏，所以一定要添加“React 的请求应该放在哪里，为什么？”这也是经常会被追问的问题。你可以这样回答。对于异步请求，应该放在componentDidMount中去操作。从时间顺序来看，除了componentDidMount还可以有以下选择：
constructor：可以放，但从设计上而言不推荐。constructor 主要用于初始化 state 与函数绑定，并不承载业务逻辑。而且随着类属性的流行，constructor 已经很少使用了
componentWillMount：已被标记废弃、React 19 已移除，在新的异步渲染架构下会触发多次渲染，容易引发 Bug，不利于未来 React 升级后的代码维护。所以 React 的请求放在componentDidMount 里是最好的选择。
透过现象看本质：React 16 缘何两次求变？Fiber 架构简析
Fiber 是 React 16 对 React 核心算法的一次重写。你只需要 get 到这一个点：Fiber 会使原本同步的渲染过程变成异步的。在 React 1...

1) 一个类组件在componentWillMount里发起订单请求，开发环境偶尔出现两笔相同请求；为什么把去重逻辑塞进去仍不是可靠修复？

答：componentWillMount属于可能因异步渲染而重复执行的旧生命周期，且在React 19已移除，仅遗留UNSAFE_形式用于迁移。请求应移到提交后的componentDidMount，并按业务需要处理取消或幂等；仅在旧钩子内部去重会继续绑定不稳定的执行时机。

2) 资料编辑页收到新userId后，用getDerivedStateFromProps把全部表单状态覆盖一遍，结果用户刚输入的内容频繁丢失；你会怎样重构？

答：这里混淆了受控数据和组件内部草稿，任何父组件更新都可能触发错误覆盖。应先明确表单由父级完全控制，还是仅在身份变化时重新初始化，再缩小需要派生的状态；getDerivedStateFromProps必须保持纯净，但纯净并不能消除状态双源带来的反模式。

3) 聊天页卸载后仍持续收到消息并触发更新，线上表现为内存上涨和重复提示；你会从哪个生命周期切入排查？

答：应检查componentWillUnmount是否对事件订阅、定时器及其他长期资源执行了对称清理，并确认绑定与解绑使用的是同一引用。仅阻止卸载后的状态更新不能终止资源本身；若组件反复挂载，遗漏清理还会累积监听器，使故障越来越明显。

4) 发布后某个商品卡片渲染异常导致整个内容区白屏，但网络和接口都正常；生命周期层面应补什么保护，它又保护不了什么？

答：应在合适的组件边界增加错误边界，把渲染异常限制在局部并提供降级界面，避免整棵相关视图不可操作。边界位置要兼顾隔离范围与恢复体验；它不是所有错误的通用捕获器，网络失败和事件回调等场景仍需各自处理。

5) 滚动列表更新前必须记录容器高度，DOM更新后再恢复滚动位置；有人准备把读取和写入都放进componentDidUpdate，你会怎么拆？

答：更新前需要读取旧DOM状态时，应使用getSnapshotBeforeUpdate获取快照，再把结果交给componentDidUpdate完成更新后的DOM操作。这样对应pre-commit可读DOM、commit执行副作用的阶段划分；若读写时机混在提交后，旧布局信息可能已经丢失。

616. React Fiber架构

Fiber是对React核心协调算法的重构，本质上把不可中断的递归遍历改成可暂停、可恢复的任务单元遍历。每个Fiber节点通过child、sibling和return连接，使协调工作能够拆分，并按优先级让高优任务先执行。可中断的reconciliation阶段负责比较并标记变化，commit阶段则同步更新真实DOM。它不一定缩短总计

算时间，主要价值是避免长任务持续占用主线程，让交互保持响应。最主要的思想就是将任务拆分。DOM需要渲染时暂停，空闲时恢复。window.requestIdleCallbackReact内部实现的机制 React 追求的是“快速响应”，那么，“快速响应”的制约因素都有什么呢 CPU的瓶颈：当项目变得庞大、组件数量繁多、遇到大计算量的操作或者设备性能不足使得页面掉帧，导致卡顿。IO的瓶颈：发送网络请求后，由于需要等待数据返回才能进一步操作导致不能快速响应。fiber架构主要就是用来解决CPU和网络的问题，这两个问题一直也是最影响前端开发体验的地方，一个会造成卡顿，一个会造成白屏。为此 react 为前端引入了两个新概念：Time Slicing时间分片和Suspense。

1. React 都做过哪些优化

React渲染页面的两个阶段调度阶段（reconciliation）：在这个阶段 React 会更新数据生成新的Virtual DOM，然后通过Diff算法，快速找出需要更新的元素，放到更新队列中去，得到新的更新队列。渲染阶段（commit）：这个阶段 React 会遍历更新队列，将其所有的变更一次性更新到DOM上React 15 架构React15架构可以分为两层Reconciler（协调器）——

负责找出变化的组件；Renderer（渲染器）—— 负责将变化的组件渲染到页面上；

调度阶段（reconciliation）：在这个阶段 React 会更新数据生成新的Virtual DOM，然后通过Diff算法，快速找出需要更新的元素，放到更新队列中去，得到新的更新队列。渲染阶段（commit）：这个阶段 React 会遍历更新队列，将其所有的变更一次性更新到DOM上

React15架构可以分为两层Reconciler（协调器）—— 负责找出变化的组件；Renderer（渲染器）—— 负责将变化的组件渲染到页面上； Reconciler（协调器）—— 负责找出变化的组件；Renderer（渲染器）—— 负责将变化的组件渲染到页面上；

在React15及以前，Reconciler采用递归的方式创建虚拟DOM，递归过程是不能中断的。如果组件树的层级很深，递归会占用线程很多时间，递归更新时间超过了16ms，用户交互就会卡顿。为了解决这个问题，React16将递归的无法中断的更新重构为异步的可中断更新，由于曾经用于递归的虚拟DOM数据结构已经无法满足需要。于是，全新的Fiber架构应运而生。React 16 架构为了解决同步更新长时间占用线程导致页面卡顿的问题，也为了探索运行时优化的更多可能，React开始重构并一直持续至今。重构的目标是实现Concurrent Mode（并发模式）。从v15到v16，React团队花了两年时间将源码架构中的Stack Reconciler重构为Fiber ReconcilerReact16架构可以分为三层：Scheduler（调度器）——

调度任务的优先级，高优任务优先进入Reconciler；Reconciler（协调器）—— 负责找出变化的组件：更新工作从递归变成了可以中断的循环过程。Reconciler内部采用了Fiber的架构；Renderer（渲染器）—— 负责将变化的组件渲染到页面上。React 17 优化使用Lane来管理任务的优先级。Lane用二进制位表示任务的优先级，方便优先级的计算（位运算），不同优先级占用不同位置的“赛道”，而且存在批的概念，优先级越低，“赛道”越多。高优先级打断低优先级，新建的任务需要赋予什么优先级等问题都是Lane所要解决的问题。Concurrent Mode的目的是实现一套可中断/恢复的更新机制。其由两部分组成：一套协程架构：Fiber Reconciler基于协程架构的启发式更新算法：控制协程架构工作方式的算法 React 16 架构 为了解决同步更新长时间占用线程导致页面卡顿的问题，也为了探索运行时优化的更多可能，React开始重构并一直持续至今。重构的目标是实现Concurrent Mode（并发模式）。从v15到v16，React团队花了两年时间将源码架构中的Stack Reconciler重构为Fiber ReconcilerReact16架构可以分为三层：Scheduler（调度器）——

调度任务的优先级，高优...

1) 搜索页一次更新要遍历上千个组件，接入Fiber后总计算时间没有明显下降，团队据此认为架构改造失败；这个判断哪里有问题？

答：Fiber的主要目标不是减少总工作量，而是把协调阶段拆成可暂停、恢复或放弃的小任务，避免长时间独占主线程。总计算甚至可能没有下降，但输入等高优先级工作能更早响应；如果瓶颈是单个不可分割的同步任务，任务拆分也难以发挥作用。

2) 输入框联动一个大型结果列表，用户连续键入时旧列表还没算完；Scheduler、Reconciler和Renderer分别应承担什么职责？

答：Scheduler根据优先级安排工作，使更紧急的输入更新先进入处理；采用Fiber的Reconciler可中断地找出视图变化，Renderer再把确定的变更提交给宿主环境。若所有更新都被赋予相同优先级，架构具备中断能力也不等于产品自然获得理想调度。

3) 低优先级报表渲染被新的交互更新打断，日志显示前面一部分协调工作后来没有提交；这是数据丢失还是预期行为？

答：这是可中断协调阶段允许出现的预期行为：更高优先级任务到来后，未完成的低优先级工作可能被放弃并在之后重新计算。用户尚未看到这些中间结果，因此不会产生半裸界面；这也要求协调阶段保持纯净，不能把不可重复的副作用放进去。

4) 线上拖拽仍会掉帧，性能记录显示一次commit中有大量真实DOM更新；为什么Fiber没把这段工作切到多帧完成？

答：可拆分的是生成和比较更新结果的reconciliation阶段，commit要同步应用变更，以避免数据与可见UI长时间处于不一致状态。应继续排查本次提交的节点规模与DOM成本，而不是期待提交阶段被任意暂停；过大的commit仍可能造成明显卡顿。

5) 有人建议直接用浏览器的requestIdleCallback实现全部React调度，理由是它能返回剩余时间；你会指出哪些工程约束？

答：requestIdleCallback提供了空闲时间提示，但兼容性和触发频率并不稳定，切换标签页等情况还可能显著降低调用频率。React因此实现自己的Scheduler，并额外支持多种任务优先级；单纯等待空闲时间也可能让持续被高优任务挤压的工作长期得不到执行。

6) 代码评审发现类组件在协调阶段的旧will生命周期里注册事件，而团队正启用可中断更新；这和Fiber的哪条约束直接冲突？

答：协调阶段可能暂停、终止或重新开始，因此其中的生命周期存在重复执行的可能，注册事件会产生不可安全重放的副作用。副作用应移到同步提交阶段，并在卸载时清理；这正是旧componentWillMount、componentWillUpdate等生命周期被弃用并最终移除的重要背景。

617. createElement过程

React.createElement会根据类型、属性和子节点创建一个描述界面的React元素。JSX编译后本质上就是这类调用，第一个参数是标签或组件类型，第二个参数是props，后续参数表示子节点。返回结果只是普通的界面描述，并不会在这一步直接操作真实DOM，实际更新要经过协调和提交。React18

起挂载通常使用createRoot(container).render(element)，但不影响createElement的参数和作用。

React.createElement()：根据指定的第一个参数创建一个React元素

React.createElement(type,[props],[...children]) 第一个参数是必填，传入的是似HTML标签名称，eg: ul, li第二个参数是选填，表示的是属性，eg: className第三个参数是选填，子节点，eg:

要显示的文本内容 下面示例里的ReactDOM.render是历史写法：React 18

起改用ReactDOM.createRoot(container).render(element)，React 19

已移除ReactDOM.render。示例本身讲的是createElement的参数形式，挂载 API

换成createRoot后结论不变。 //写法一：var child1=React.createElement('li',null,'one');var child2=React.createElement('li',null,'two');var content=React.createElement('ul',{className:'teststyle'},child1,child2);// 第三个参数可以分开也可以写成一个数组ReactDOM.render(content,document.getElementById('example'));//写法二：var child1=React.createElement('li',null,'one');var child2=React.createElement('li',null,'two');var content=React.createElement('ul',{className:'teststyle'},[child1,child2]);ReactDOM.render(content,document.getElementById('example'));

1) 商品列表页把`React.createElement('li', { className: 'item' }, sku)`当成已经插入页面的节点，随后立刻查询`item`却找不到，误判在哪里？

答：误判是把创建React元素与挂载到页面混为一谈；这次调用只按type、属性和子节点构造待渲染内容。还要把返回结果交给根节点渲染，`ReactDOM.render(element, container)`，否则页面中不会出现对应节点。

2) 后台菜单有 200 项，开发者把子项数组作为第三个参数传给`React.createElement('ul', null, children)`；评审认为必须展开成`...children`，你怎么判断？

答：两种传法在该参数形式下都成立：子节点既能作为后续参数逐个传入，也能以数组形式放在第三个参数。工程上应统一团队写法并保证数组内容确实是合法子节点；不要仅因没有展开数组，就把渲染异常归因于`createElement`。

3) 老管理台仍用`ReactDOM.render(content, container)`展示`createElement`示例，升级到React18后负责人与维护者对是否重写有分歧，你会怎么处理？

答：示例所讲的三个参数形式不需要改变，但挂载代码应迁移为`ReactDOM.createRoot(container).render(content)`。`ReactDOM.render`起旧写法已经是历史用法，`ReactDOM.createRoot`更移除了`ReactDOM.render`；继续保留会把元素创建和版本相关的挂载接口混在一起。

4) 营销页生成的列表没有样式，代码是`React.createElement('ul', { classname: 'teststyle' }, items)`，DOM结构和文本都正常，你先查哪里？

答：先核对第二个参数中的属性名，`React`示例使用的是`className`，而不是`classname`。第三个参数已经能正常产生文本和列表结构，说明排查重点应落在属性传递及样式选择器；改挂载API不能修复属性拼写错误。

5) 低代码渲染器需要同时生成两个，有人主张先各自调用`createElement`，另一人主张直接传配置对象数组；哪种边界必须守住？

答：传给父元素的子项应当是已经创建好的React子节点，例如两个`React.createElement('li', ...)`的结果，再逐个或以数组传入。普通配置对象不能仅凭形状替代子节点；若选择配置驱动，仍需先把配置映射为元素，代价是多一层转换与校验。

618. 调和阶段 `setState`内部干了什么

调用`setState`后，`React`会合并或排队保存状态更新，并启动调和流程。调和时会根据新状态构建新的`React`元素树，再与上一次的元素树做diff，找出真正发生变化的部分。这样最终只更新必要的UI，而不是每次都重建整个页面；状态计算和界面更新也不一定在调用处立即完成。当调用`setState`时，`React`会做的第一件事情是将传递给`setState`的对象合并到组件的当前状态这将启动一个称为和解（`reconciliation`）的过程。和解（`reconciliation`）的最终目标是以最有效的方式，根据这个新的状态来更新UI。

为此，`React`将构建一个新的`React`元素树（您可以将其视为UI的对象表示）一旦有了这个树，为了弄清UI如何响应新的状态而改变，`React`会将这个新树与上一个元素树相比较（`diff`）通过这样做，`React`将会知道发生的确切变化，并且通过了解发生什么变化，只需在绝对必要的情况下进行更新即可最小化UI的占用空间

1) 订单页调用`setState({ status: 'paid' })`后，开发者认为`React`会直接把按钮文本改成“已支付”，这个判断漏掉了什么过程？

答：`setState`触发的不是一次面向具体DOM文本的直接修改，而是基于新状态重新构建`React`元素树。`React`还要把新树与上一棵树比较，确定真正发生变化的位置后再更新UI；因此不能把状态对象与某

个DOM操作简单画等号。

2) 看板一次收到 300

条状态变化，产品要求只更新发生变化的卡片；你如何用调和过程解释实现判断？

答：React会依据新状态生成新的元素树，再与上一次的元素树进行比较，从而找出需要更新的部分。组件仍应让状态准确表达每张卡片的结果，不能手工假定只有某个节点变化；错误的状态建模会让生成的新树本身就不符合预期。

3) 同一份组件升级后改用并发能力，负责人仍要求“每次调用setState后立刻完成整棵树比较”，这个约束合理吗？

答：不应把setState理解为调用返回前必须完成全部UI更新；可靠结论是它会发起状态更新与后续调和。具体调度时机不由本节的树比较模型保证，业务若依赖立即可见结果，就把实现绑到了未承诺的执行时序上。

4) 库存页状态已经变成0，页面却仍显示“有货”；值班同学只盯着接口响应，你会沿哪条链路排查？

答：先确认传入setState的新状态是否正确，再检查该状态生成的新元素树是否真的改变了展示分支。随后核对树比较后对应UI是否提交；若新旧树表达相同内容，调和没有可更新之处，问题通常在状态到元素的映射而非接口本身。

5) 团队想绕过React，监听状态后直接改DOM，以避免元素树比较；在频繁更新的仪表盘里你如何取舍？

答：直接改DOM会绕开“新状态生成元素树、与旧树比较、只更新必要部分”的一致更新链路。除非该区域明确由外部系统独占管理，否则React后续调和可能覆盖手工结果；省下一次声明式计算的同时，会增加状态与页面失配的风险。

619. setState

setState的核心是排队和批量更新，不能简单理解成一个异步函数。对象写法连续读取旧state时，同名字段可能互相覆盖；依赖前一次结果时，应使用函数式写法，让更新按顺序计算。React18

的createRoot和React19 会在定时器、原生事件等场景自动批处理，React17

及以前则可能同步刷新。需要在提交后读取结果，可以使用回调；确需强制同步时可用flushSync。

先看版本结论（React 17 / 18 / 19 三档）

本节余下的Transaction（事务）与isBatchingUpdates分析描述的是React 16 / 17 的内部实现，属于历史对照：现在 Fiber 走的是 lane 优先级调度，不再依赖单个全局布尔标志。这些内容仍然值得读——它解释了「为什么当年定时器里会变成同步」——但不要当作 React 18 及以后的现状。

下面凡是出现「在原生事件和 setTimeout 中 setState 是同步的」这类表述，都请补上前提：React 17 及以前，或 React 18 的ReactDOM.render兼容模式。在了解setState之前，我们先来简单了解下 React

一个包装结构：Transaction：事务（Transaction）是 React

中的一个调用结构，用于包装一个方法，结构为：initialize - perform(method) -

close。通过事务，可以统一管理一个方法的开始与结束；处于事务流中，表示进程正在执行一些操作

setState：React 中用于修改状态，更新视图。它具有以下特点：

异步与同步:setState并不是单纯的异步或同步，这其实与调用时的环境相关：

在合成事件和生命周期钩子(除 componentDidMount) 中，setState是"异步"的；原因：

因为在setState的实现中，有一个判断：当更新策略正在事务流的执行中时，该组件更新会被推入dirtyComponents队列中等待执行；否则，开始执行batchedUpdates队列更新；在生命周期钩子调用中，更新策略都处于更新之前，组件仍处于事务流中，而componentDidUpdate是在更新之后，此时组件已经不在事务流中了，因此则会同步执行；在合成事件中，React 是基于 事务流完成的事件委托机制

实现，也是处于事务流中；问题：无法在setState后马上从this.state上获取更新后的值。解决：如果需要马上同步去获取新值，setState其实是可以传入第二个参数的。setState(updater, callback)，在回调中即可获取最新值；在原生事件和setTimeout中，setState是同步的，可以马上获取更新后的值（仅限 React 17 及以前 / React 18 兼容模式；React 18 createRoot起这些场景同样批量合并）；原因：原生事件是浏览器本身的实现，与事务流无关，自然是同步；而setTimeout是放置于定时器线程中延后执行，此时事务流已结束，因此也是同步；批量更新：在合成事件和生命周期钩子中，setState更新队列时，存储的是合并状态(Object.assign)。因此前面设置的key值会被后面所覆盖，最终只会执行一次更新；函数式：由于Fiber及合并的问题，官方推荐可以传入函数的形式。setState(fn)，在fn中返回新的state对象即可，例如this.setState((state, props) => newState)；使用函数式，可以用于避免setState的批量更新的逻辑，传入的函数将会被顺序调用；原因：因为在setState的实现中，有一个判断：当更新策略正在事务流的执行中时，该组件更新会被推入dirtyComponents队列中等待执行；否则，开始执行batchedUpdates队列更新；在生命周期钩子调用中，更新策略都处于更新之前，组件仍处于事务流中，而componentDidUpdate是在更新之后，此时组件已经不在事务流中了，因此则会同步执行；在合成事件中，React是基于事务流完成的事件委托机制实现，也是处于事务流中；问题：无法在setState后马上从this.state上获取更新后的值。解决：如果需要马上同步去获取新值，setState其实是可以传入第二个参数的。setState(updater, callback)，在回调中即可获取最新值；在生命周期钩子调用中，更新策略都处于更新之前，组件仍处于事务流中，而componentDidUpdate是在更新之后，此时组件已经不在事务流中了，因此则会同步执行；在...

1) 类组件计数器在一次点击里连续执行两次this.setState({ count: this.state.count + 1 })，两个回调都打印1；候选人说队列丢了一次更新，你同意吗？

答：不同意，两次对象式更新读取的都是当时相同的this.state.count，同一批次里同名字段会被后一次结果覆盖。改成this.setState(state => ({ count: state.count + 1}))才会顺序消费前一次结果；两个提交后回调看到的是合并完成的最终值。

2) React19 的搜索页在Promise.then里连续更新筛选条件和页码，产品担心产生两次渲染；你如何落地并验证？

答：React19 会对该异步回调中的更新进行批处理，不应沿用旧版“异步回调逐次同步刷新”的判断。把相互依赖的状态更新写成函数式形式，并用渲染日志或Profiler验证实际提交；批处理只减少提交次数，不会替你修正错误的状态依赖。

3) 遗留系统一部分用React18的ReactDOM.render，新页面使用createRoot，同样在setTimeout中两次setState却表现不同，怎么解释？

答：React18 的兼容根沿用React17 行为，定时器中的更新会逐次同步刷新；createRoot则启用automaticbatching，将这些更新一起处理。排查时必须先确认根节点创建方式，不能只看React主版本；迁移根节点还可能改变依赖同步读值的旧代码。

4) 线上弹窗执行setState后马上读取this.state.open，在React18createRoot下仍是旧值，开发者准备套一层setTimeout，你会怎么改？

答：不要依赖定时器碰运气，因为automaticbatching已覆盖setTimeout，回调中的读取也未必对应预想的提交边界。类组件可使用setState(updater, callback)在更新完成后读取，或在componentDidUpdate中响应变化；若新值依赖旧值，应同时改用函数式更新。

5) 展开面板后必须立刻测量新DOM高度，架构师禁止所有同步刷新，动画负责人坚持使用flushSync；你支持哪边？

答：该场景确实需要在状态更新后立即读取已提交的DOM，可把最小范围的更新包在flushSync中。它是React18createRoot与React19打破批处理的出口，但会牺牲批处理收益；应限制在必要测量点，不能作为普通状态更新的默认写法。

6) 组件卸载后请求回调仍调用setState，监控出现警告；有人提议每个组件都维护isUnmounted，另一人要把数据提升到父层，你怎么选？

答：先让异步任务在卸载时停止回写；若数据生命周期确实长于组件，再考虑放到父级或外部状态中通过props传入。isUnmounted判断是局部防线，会在大量组件中形成重复状态；无论选哪种方案，都不能让已销毁实例继续进入更新队列。

620. setState原理分析

setState本质上会把状态变更放进更新队列，再由React在合适的时机统一计算和渲染。React16、17的旧实现通过isBatchingUpdates和dirtyComponents控制批处理，React18起则改用Fiber的lane调度 and 自动批处理。直接修改this.state不会进入这套更新流程，因此不能可靠触发重新渲染。若新值依赖旧值，应使用函数式更新；提交后的值可在回调或componentDidUpdate中读取。本节是 React 16 / 17 内部实现的历史对照：下面的pre/post钩子、isBatchingUpdates、dirtyComponents属于旧的Transaction批处理实现。React 18起改为Fiber的lane优先级调度 + automatic batching，「是否在React调用栈里」不再决定批处理与否。版本结论见上一节开头的三档对照表。

1. setState异步更新 我们都知道，React通过this.state来访问state，通过this.setState()方法来更新state。当this.setState()方法被调用的时候，React会重新调用render方法来重新渲染UI首先如果直接在setState后面获取state的值是获取不到的。在React内部机制能检测到的地方，setState就是异步的；在React检测不到的地方，例如setInterval, setTimeout, setState就是同步更新的（React 17 及以前；React 18createRoot起这些场景同样批量合并，React 19 无法退回旧行为）

因为setState是可以接受两个参数的，一个state，一个回调函数。因此我们可以在回调函数里面获取值，setState方法通过一个队列机制实现state更新，当执行setState的时候，会将需要更新的state合并之后放入状态队列，而不会立即更新this.state如果我们不使用setState而是使用this.state.key来修改，将不会触发组件的re-render。如果将this.state赋值给一个新的对象引用，那么其他不在对象上的state将不会被放入状态队列中，当下次调用setState并对状态队列进行合并时，直接造成了state丢失

1.1 setState批量更新的过程 在react生命周期和合成事件执行前后都有相应的钩子，分别是pre钩子和post钩子，pre钩子会调用batchedUpdate方法将isBatchingUpdates变量置为true，开启批量更新，而post钩子会将isBatchingUpdates置为false isBatchingUpdates变量置为true，则会走批量更新分支，setState的更新会被存入队列中，待同步代码执行完后，再执行队列中的state更新。isBatchingUpdates为true，则把当前组件（即调用了setState的组件）放入dirtyComponents数组中；否则batchUpdate所有队列中的更新而在原生事件和异步操作中，不会执行pre钩子，或者生命周期的中的异步操作之前执行了pre钩子，但是pos钩子也在异步操作之前执行完了，isBatchingUpdates必定为false，也就不会进行批量更新 enqueueUpdate包含了React避免重复render的逻辑。mountComponent和updateComponent方法在执行的最开始，会调用到batchedUpdates进行批处理更新，此时会将isBatchingUpdates设置为true，也就是将状态标记为现在正处于更新阶段了。isBatchingUpdates为true，则把当前组件（即调用了setState的组件）放入dirtyComponents数组中；否则batchUpdate所有队列中的更新

1.2 为什么直接修改this.state无效 要知道setState本质是通过一个队列机制实现state更新的。执行setState时，会将需要更新的state合并后放入状态队列，而不会立刻更新state，队列机制可以批量更新state。如果不通过setState而直接修改this.state，那么这个state不会放入状态队列中，下次调用setState时对状态队列进行合并时，会忽略之前直接被修改的state，这样我们就无法合并了，而且实际也没有把你想要的state更新上去

1.3 什么是批量更新 Batch Update 在一些mv*框架中，就是将一段时间内对model的修改批量更新到view的机制。比如那前端比较火的React、vue（next...）

1) 资料编辑页直接执行`this.state.profile.name =`

`value`，调试器里值已变化但页面没重渲染；开发者认为React的diff失效了，你怎么纠正？

答：直接修改`this.state`没有通过`setState`进入状态更新队列，因此不能据此期待React重新调用`render`。应把状态视为不可直接修改的数据，通过`setState`提交新值；否则后续队列合并还可能忽略这次私改，造成页面与内存观察结果不一致。

2) 表格一次编辑三个字段，代码连续调用对象式`setState`，评审担心每次都重绘整页；你会怎样实现？

答：这些更新会先进入状态队列，并在合适时机批量合并后更新视图，而不是把每次调用都等同于一次立即DOM修改。若字段互不覆盖，可提交各自的部分状态；若新值依赖旧状态，则使用函数式更新，避免对象式更新在队列中覆盖结果。

3) 老代码用`isBatchingUpdates`和`dirtyComponents`解释React19

的所有更新时序，面试中你会指出什么版本边界？

答：这套`pre`、`post`、`isBatchingUpdates`与`dirtyComponents`描述的是React16、17的`Transaction`实现，只适合作为历史对照。React18起采用Fiber的`lane`优先级调度与`automatic batching`，触发点是否位于React调用栈内已不再决定是否批处理。

4)

支付页调用`setState`后日志仍是旧金额，值班同学认定更新失败；你要求他按什么顺序定位？

答：先确认日志是否紧跟在`setState`后，因为状态更新入队并不保证当场改写`this.state`。再把观察点移到`setState`回调或`componentDidUpdate`，确认提交后的值；若那里仍异常，继续检查对象式更新是否被后续同字段更新覆盖。

5) 类组件需要在价格更新完成后上报一次埋点，有人放在`render`，有人放进`setState`第二个回调；你怎么取舍？

答：不要在`render`中上报，因为重新渲染不等于这次价格更新的专属完成通知，容易产生重复副作用。针对单次类组件更新可使用`setState`的第二个回调，跨多次价格变化的统一响应则放在`componentDidUpdate`并比较前后值；两者都要防止回调再次无条件更新状态。

6) 遗留组件在`shouldComponentUpdate`中调用`setState`修正数据，线上出现持续更新和内存上涨；根因与改法是什么？

答：该生命周期本就在判断一次待处理更新，内部再次调用`setState`会继续向待处理队列追加更新，可能形成循环调用。应把数据修正移到事件入口、状态计算阶段或受条件约束的更新位置；仅加日志或扩大内存不能消除循环更新链路。

621. React事务机制

React事务机制是Stack架构时期用来包裹更新流程、实现批处理的一套机制。它会在目标方法执行前运行`initialize`，执行完成后再运行`close`，从而统一开启和结束批量更新。处于事务中的多次`setState`会先被收集，所以调用后看起来没有立即生效。需要注意，这属于旧版实现；Fiber架构下已经由`lane`优先级调度等机制接替。

1) 遗留后台在React合成事件里连续调用三次`setState`，开发者用“每次调用都立刻刷新一次”解释页面行为；结合事务图你会怎么反驳？

答：旧实现会在受React管理的调用过程外包一层事务，在开始阶段进入批量更新状态，结束阶段再统一处理队列。因而三次调用不能简单等同于三次即时提交；但该解释属于旧版内部机制，不应直接套到React18、19的`lane`调度上。

2) 维护者准备把旧Transaction图直接写进React19

的故障手册，用它判断setTimeout一定逃离批处理；你批准吗？

答：不批准，这组图更适合解释旧实现中事务边界与批处理的关系，不能证明现代版本的定时器更新仍逐次刷新。React18createRoot起已有automaticbatching，React19也保持批处理；手册必须先标注根节点和版本边界。

3) React17 页面在原生事件回调里两次更新状态，迁移createRoot后日志时序改变；值班人员只检查业务代码，漏了什么？

答：他漏查了根节点迁移带来的批处理边界变化：旧事务机制无法覆盖的原生回调，在React18createRoot下也会进入automaticbatching。应对照迁移前后的根节点、触发来源和提交日志；不能继续用旧图中的事务包裹范围推断新版本时序。

4) 数据大屏要求更新状态后立即读取新DOM，团队想修改事务开关强制提交；你会给出什么工程方案？

答：不要操作内部事务状态，这些标志属于旧实现细节，也不是稳定公共接口。现代React应在确有同步DOM读取需求时，以最小范围使用flushSync包住更新；它会打破批处理并增加同步工作，因此只适合作为受控逃生口。

5) 培训中有人把Transaction、automaticbatching和reconciliation说成同一件事，导致新人认为合并更新就是跳过树比较；你如何串起来？

答：Transaction描述旧版如何包裹一段执行并在边界处理更新，automaticbatching描述现代版本如何合并一批状态更新；它们都不等于reconciliation。批次形成后，React仍需根据新状态构建元素树并与旧树比较，再决定必要的UI更新。

622. React组件和渲染更新过程

React组件的渲染分为计算差异的render阶段和提交变更的commit阶段，前者可中断，后者不可中断。JSX本质上会调用createElement生成虚拟节点，组件再根据props和state执行render()，最终把结果更新到页面。调用setState后，相关组件会被标记为待更新，重新生成新节点并与旧节点比较，因此副作用应留到提交阶段处理。

渲染和更新过程

jsx如何渲染为页面setState之后如何更新页面面试考察全流程

JSX本质和vdom

JSX即createElement函数执行生成vnodepatch(elem,vnode)和patch(vnode,newNode)

组件渲染过程 props staterender()生成vnodepatch(elem, vnode) 组件更新过程

setState-->dirtyComponents(可能有子组件)render生成newVnodepatch(vnode, newVnode)

1) 商品详情页只修改了购物车数量，父组件和多个商品推荐子组件的render都执行了，但DOM几乎没变；你会把这次更新描述成整页重绘吗？

答：不会，组件重新执行render只是在依据新的props、state生成newVnode，不等于重新创建整页真实DOM。随后patch(vnode, newVnode)会找出变化并更新对应节点；若只看到函数执行次数，不能直接判定真实DOM操作过多。

2) 后台表格点击“刷新状态”后调用了setState，面试官要求你从这一行代码一直讲到页面文字变化，中间链路怎么说？

答：setState会使相关组件进入待更新集合，可理解为被标记到dirtyComponents，其中也可能涉及子组件。React随后重新执行组件渲染得到newVnode，再与旧vnode做patch，最终把差异落实到真实DOM；它不是直接修改页面节点。

3) 一个组件首次挂载和后续调用setState时都执行了render，前端同学认为两条链路完全相同，你会指出哪处关键差异？

答：首次挂载是由props、state生成vnode，再执行类似patch(elem, vnode)的过程，把描述落实到容器。更新阶段已有旧vnode，所以重点变成生成newVnode并执行patch(vnode, newVnode)；两者都渲染，但比较基准不同。

4) 订单页调用setState后数据已经变了，界面却仍显示旧金额；你会按哪几层定位，而不是直接怀疑浏览器缓存？

答：先确认更新是否让目标组件进入待更新流程，再检查render生成的newVnode是否真的包含新金额。若虚拟节点正确，再沿patch(vnode, newVnode)检查差异是否落到对应真实节点；若渲染输入仍旧，问题通常在状态或属性传递链路，而非DOM补丁阶段。

5) 团队想绕过虚拟节点，在库存变化时直接找到节点改textContent，理由是只有一个数字变化；你如何评估这种取舍？

答：直接改DOM可能立即显示结果，但会绕开由props、state生成vnode再执行patch的一致更新链路。后续组件再次渲染时，声明结果可能覆盖手工修改，也会增加状态与页面不一致的排查成本；除非明确隔离该节点，否则不宜作为常规方案。

623. 如何解释 React 的渲染流程

React 的渲染流程可以概括为触发更新、调度任务、计算差异和提交结果。React 16以前的Stack Reconciler以递归和事务为核心，执行期间容易持续占用主线程；之后的Fiber Reconciler把节点组织成可遍历的Fiber结构，并支持按优先级让出执行权。render阶段构造workInProgress树，可中断且不应产生副作用；commit阶段同步更新DOM、执行生命周期和处理ref，不可中断。普通管理后台里两者差距未必明显，但动画、画布或手势场景更容易体现Fiber调度的价值。

React 16为分界线，分为Stack Reconciler和Fiber Reconciler。这里的协调从狭义上来讲，特指 React 的 diff 算法，广义上来讲，有时候也指 React 的reconciler模块，它通常包含了diff算法和一些公共逻辑。回到Stack Reconciler中，Stack Reconciler的核心调度方式是递归。调度的基本处理单位是事务，它的事务基类是Transaction，这里的事务是 React 团队从后端开发中加入的概念。在 React 16 以前，挂载主要通过 ReactDOM 模块完成，更新通过ReactUpdate模块完成，模块之间相互分离，落脚执行点也是事务。在React 16及以后，协调改为了Fiber Reconciler。它的调度方式主要有两个特点，第一个是协作式多任务模式，在这个模式下，线程会定时放弃自己的运行权利，交还给主线程，通过requestIdleCallback实现。第二个特点是策略优先级，调度任务通过标记tag的方式分优先级执行，比如动画，或者标记为high的任务可以优先执行。Fiber Reconciler的基本单位是Fiber，Fiber基于过去的React Element提供了二次封装，提供了指向父、子、兄弟节点的引用，为diff工作的双链表实现提供了基础。在新的架构下，整个生命周期被划分为Render 和 Commit 两个阶段。Render 阶段的执行特点是可中断、可停止、无副作用，主要是通过构造workInProgress树计算出diff。以current树为基础，将每个Fiber作为一个基本单位，自下而上逐个节点检查并构造 workInProgress 树。这个过程不再是递归，而是基于循环来完成在执行上通过requestIdleCallback来调度执行每组任务，每组中的每个计算任务被称为work，每个work完成后确认是否有优先级更高的work需要插入，如果有就让位，没有就继续。优先级通常是标记为动画或者high的会先处理。每完成一组后，将调度权交回主线程，直到下一次requestIdleCallback调用，再继续构建workInProgress树在commit阶段需要处理effect列表，这里的effect列表包含了根据diff 更新 DOM 树、回调生命周期、响应 ref等。但一定要注意，这个阶段是同步执行的，不可中断暂停，所以不要在componentDidMount、componentDidUpdate、componentWillUnmount中去执行重度消耗算力的任务如果只是一般的应用场景，比如管理后台、H5 展示页等，两者性能差距并不大，但在动画、画布及手势等场景下，Stack Reconciler的设计会占用主线程，造成卡顿，而fiber reconciler的设计则能带来高性能的表现。

1) 动画面板更新时主线程连续被占用，同事说React只是在递归整棵组件树；放到React 16之后的架构里，这个说法哪里不准确？

答：React 16之后使用Fiber Reconciler，基本处理单位是带父、子、兄弟引用的Fiber，工作过程不再等同于旧架构的连续递归。协调工作可被拆分、暂停和恢复，并结合任务优先级让位；但后续commit仍是同步且不可中断的。

2) 画布编辑器同时收到拖拽更新和低优先级列表刷新，产品要求拖拽优先响应；渲染流程中的哪一层负责体现这种优先级？

答：优先级体现在Fiber Reconciler的调度与render工作阶段，任务会被标记并按策略安排，高优先级工作可插入并获得执行机会。它改善的是协调过程对主线程的占用方式，不代表所有工作都会异步；真正应用变更的commit仍要同步完成。

3) 管理后台只有少量表单和列表，架构评审却把从Stack Reconciler到Fiber Reconciler当成必然的大幅性能收益；你会接受吗？

答：不会直接接受，一般管理后台或H5展示页中，两种协调方式的体验差距未必明显。Fiber的优势更容易在动画、画布、手势或大批量更新等主线程敏感场景体现；是否有收益仍要结合更新规模和交互特征判断。

4) 线上弹窗打开时页面长时间不响应，日志显示节点差异已经算完，随后生命周期回调里执行了重计算；你会把故障定位到哪个阶段？

答：应重点定位commit阶段，因为它会同步处理effect列表，包括更新DOM、生命周期回调和ref。该阶段不可中断，在componentDidMount、componentDidUpdate或卸载回调中执行重计算会直接延长阻塞；把渲染阶段做成可中断并不能抵消这类开销。

5) 面试官让你用current树、workInProgress树和真实DOM串起一次更新，不能只背“渲染与提交”，你会怎么讲？

答：render阶段以current树为基础，逐个处理Fiber并构造workInProgress树，在此期间计算差异且应避免副作用。完成后进入同步commit，按照effect信息更新真实DOM、触发生命周期并处理ref；前一阶段可调度，后一阶段不能暂停。

624. diff算法是怎么运作

React的diff通过只比较同层节点、按组件类型判断复用，并用key识别同层元素，把树比较从 $O(n^3)$ 简化到 $O(n)$ 。节点类型不同会直接删除旧子树并创建新子树，类型相同才继续比较属性和子节点，所以跨层移动通常也会被当成删除与新增。列表更新时会优先处理可复用节点，剩余节点再执行插入、删除或移动。key应保持唯一和稳定，否则在头部插入或顺序变化时，可能让后续节点失去正确的复用关系；用索引作key也有同样风险。每一种节点类型有自己的属性，也就是prop，每次进行diff的时候，react会先比较该节点类型，假如节点类型不一样，那么react会直接删除该节点，然后直接创建新的节点插入到其中，假如节点类型一样，那么会比较prop是否有更新，假如有prop不一样，那么react会判定该节点有更新，那么重渲染该节点，然后在对其子节点进行比较，一层一层往下，直到没有子节点。把树形结构按照层级分解，只比较同级元素。给列表结构的每个单元添加唯一的key属性，方便比较。React只会匹配相同class的component（这里的class指的是组件的名字）合并操作，调用component的setState方法的时候，React将其标记为 -dirty。到每一个事件循环结束，React检查所有标记dirty的component重新绘制。选择性子树渲染。开发人员可以重写shouldComponentUpdate提高diff的性能

优化

为了降低算法复杂度，React的diff会预设三个限制：只对同级元素进行Diff。如果一个DOM节点在前后两次更新中跨越了层级，那么React不会尝试复用他。两个不同类型的元素会产生出不同的树。如果元素由div变为p，React会销毁div及其子孙节点，并新建p及其子孙节点。开发者可以通过key prop来暗示哪些子元素在不同的渲染下能保持稳定。考虑如下例子：

Diff的思路

该如何设计算法呢？如果让我设计一个Diff算法，我首先想到的方案是：判断当前节点的更新属于哪种情况如果是新增，执行新增逻辑如果是删除，执行删除逻辑如果是更新，执行更新逻辑 按这个方案，其实有个隐含的前提——不同操作的优先级是相同的但是React团队发现，在日常开发中，相较于新增和删除，更新组件发生的频率更高。所以Diff会优先判断当前节点是否属于更新。

基于以上原因，Diff算法的整体逻辑会经历两轮遍历：

第一轮遍历：处理更新的节点。第二轮遍历：处理剩下的不属于更新的节点。 diff算法的作用

计算出Virtual DOM中真正变化的部分，并只针对该部分进行原生DOM操作，而非重新渲染整个页面。

传统diff算法 通过循环递归对节点进行依次对比，算法复杂度达到 $O(n^3)$ ，n是树的节点数，这个有多可怕呢？——如果要展示1000个节点，得执行上亿次比较。。即便是CPU快能执行30亿条命令，也很难在一秒

内计算出差异。React的diff算法 什么是调和？将Virtual DOM树转换成actual DOM树的最少操作的过程称为 调和 。 什么是React diff算法？ diff算法是调和的具体实现。 diff策略 React用 三大策略

将 $O(n^3)$ 复杂度 转化为 $O(n)$ 复杂度 策略一（tree diff）： Web UI中DOM节点跨层级的移动操作特别少，可以忽略不计。 策略二（component diff）：

拥有相同类的两个组件 生成相似的树形结构，拥有不同类的两个组件 生成不同的树形结构。

策略三（element diff）： 对于同一层级的一组子节点，通过唯一id区分。 tree diff

React通过updateDepth对Virtual DOM树进行层级控制。对树分层比较，两棵树 只对同一层次节点 进行比较。如果该节点不存在时，则该节点及其子节点会被完全删除，不会进一步比较。只需遍历一次，就能完成整棵DOM树的比较。 那么问题来了，如果DOM节点出现了跨层级操作,diff会咋办呢？

答：diff只简单考虑同层级的节点位置变换，如果是跨层级的话，只有创建节点和删除节点的操作。

如上图所示，以A为根节点的整棵树会被重新创建，而不是移动，因此 官方建议不要进行DOM节点跨层级操作，可以通过CSS隐藏、显示节点，而不是真正地移除、添加DOM节点 component diff

React对不同的组件间的比较，有三种策略 同一类型的两个组件，按原策略（层级比较）继续比较Virtual DOM树即可。同一类型的两个组件，组件A变化为组件B时，可能Virtual

DOM没有任何变化，如果知道这点（变换的过程中，Virtual

DOM没有改变），可节省大量计算时间，所以 用户 可以通过shouldComponentUpdate...

1) 消息列表头部插入一条新消息，开发者没有提供稳定key，结果插入点后的多行都被当成变化；为什么diff没识别出只是新增一项？

答：同层子节点缺少稳定标识时，React主要按位置判断对应关系，头部插入会改变后续节点的位置映射。为每条消息使用唯一且稳定的业务key，才能帮助element diff区分插入、删除和移动；key只在一组兄弟节点中参与匹配。

2) 可拖拽看板把一个带输入状态的卡片从A列移到B列，业务希望组件实例和状态原样保留；仅让两边使用相同key能做到吗？

答：不能保证，因为React只在同一层级、同一组子节点中利用key建立对应关系，不会把跨父节点移动当作可复用的同一次移动。旧子树会被删除，新位置会创建节点，局部状态可能随之丢失；需要保持层级不变并调整展示，或把需保留的状态移到更稳定的位置。

3) 设计系统把列表项根节点从div改成p，内部子组件和属性看起来完全一致；为什么上线后仍出现整段节点重建？

答：根元素类型不同会触发不同树的假设，React会销毁旧div及其子孙节点，再创建新的p子树，而不是继续比较内部结构。只有类型相同时才会比较prop并向下处理子节点；结构相似并不足以跨类型复用。

4) 千行表格更新一行后出现大量节点操作，你拿到新旧虚拟树时会先查哪些破坏复用关系的线索？

答：先核对同层列表是否有唯一、稳定的key，以及更新前后元素或组件类型是否被意外替换。再检查节点是否跨父级移动，因为这会表现为旧子树删除和新子树创建；这些线索比单纯统计组件执行次数更能解释异常补丁规模。

5) 前端负责人建议实现完整的跨层树匹配，认为能减少任何移动场景的节点创建；你如何回应复杂度与收益冲突？

答：完整树比较的传统复杂度可达到 $O(n^3)$ ，React通过只比较同层节点、按组件类型判断、用key区分同组元素，把常见比较约束到 $O(n)$ 。支持任意跨层复用会破坏这些前提并显著增加成本；更稳妥的是调整页面结构，减少跨层移动。

6) 一个类组件渲染代价较高，但其输入在多数更新中不变；shouldComponentUpdate与diff的关系该怎样向评审解释？

答：shouldComponentUpdate可在已知组件输出无需变化时阻止继续计算其子树，从而减少不必要的diff。它依赖开发者正确判断props与state是否影响输出，判断错误会让页面漏更新；因此应针对明确稳定的子树使用，而不是无条件阻断。

625. 合成事件原理

合成事件本质上是React对原生事件的统一封装，事件通常委托到顶层容器，而不是直接绑定在每个子元素上。原生事件冒泡到委托节点后，React会创建SyntheticEvent，找到对应组件，并按组件树分发处理函数。这样既能屏蔽浏览器差异，也便于统一管理事件并减少大量监听器带来的内存开销。需要注意版本边界：React

17以前主要委托到document，之后改为各自的根容器；混用原生事件时也要留意两套传播路径的差别。为了解决跨浏览器兼容性问题，React会将浏览器原生事件（Browser Native Event）封装为合成事件（SyntheticEvent）传入设置的事件处理器中。这里的合成事件提供了与原生事件相同的接口，不过它们屏蔽了底层浏览器的细节差异，保证了行为的一致性。另外有意思的是，React并没有直接将事件附着到子元素上，而是以单一事件监听器的方式将所有的事件发送到顶层进行处理。这样React在更新DOM的时候就不需要考虑如何去处理附着在DOM上的事件监听器，最终达到优化性能的目的

所有的事件挂在document上，DOM事件触发后冒泡到document；React找到对应的组件，造出一个合成事件出来；并按组件树模拟一遍事件冒泡。event不是原生的，是SyntheticEvent合成事件对象和Vue事件不同，和DOM事件也不同

React 17 之前的事件冒泡流程图

所以这就造成了，在一个页面中，只能有一个版本的

React。如果有多个版本，事件就乱套了。值得一提的是，这个问题在React 17中得到了解决，事件委托不再挂在document上，而是挂在DOM容器上，也就是ReactDOM.Render所调用的节点上。

React 17 后的事件冒泡流程图

那到底哪些事件会被捕获生成合成事件呢？可以从React的源码测试文件中一探究竟。下面的测试快照中罗列了大量的事件名，也只有在这份快照中的事件，才会被捕获生成合成事件。// react/packages/react-dom/src/__tests__/__snapshots__/ReactTestUtils-test.js.snapArray["abort","animationEnd","animationIteration","animationStart","auxClick","beforeInput","blur","canPlay","canPlayThrough","cancel","change","click","close","compositionEnd","compositionStart","compositionUpdate","contextMenu","copy","cut","doubleClick","drag","dragEnd","dragEnter","dragExit","dragLeave","dragOver","dragStart","drop","durationChange","emptied","encrypted","ended","error","focus","gotPointerCapture","input","invalid","keyDown","keyPress","keyUp","load","loadStart","loadedData","loadedMetadata","lostPointerCapture","mouseDown","mouseenter","mouseleave","mousemove","mouseout","mouseover","mouseup","paste","pause","play","playing","pointerCancel","pointerDown","pointerEnter","pointerLeave","pointerMove","pointerOut","pointerOver","pointerUp","progress","rateChange","reset","scroll","seeked","seeking","select","stalled","submit","suspend","timeUpdate","toggle","touchCancel",

"touchEnd","touchMove","touchStart","transitionEnd","volumeChange","waiting","wheel",] 如果DOM上绑定了过多的事件处理函数,整个页面响应以及内存占用可能都会受到影响。React为了避免这类DOM事件滥用,同时屏蔽底层不同浏览器之间的事件系统的差异,实现了一个...

1) 列表里有一千个按钮,实习生认为每个onClick都会对应一个直接挂在按钮上的原生监听器;结合React的事件链路,你会如何纠正?

答: React不会按这种理解把每个处理器直接附着到对应子节点,而是统一监听支持的事件,再在事件到达委托层后定位组件并分发。传给业务处理器的是SyntheticEvent,这种中间层减少了逐节点管理监听器的负担,也统一了浏览器差异。

2) 同一页面嵌入两个独立React应用,旧方案都把事件交给document,团队担心事件互相影响;React 17的委托位置变化解决了什么?

答: React 17将事件委托从document下移到各自的React根容器,也就是渲染所使用的容器节点。这样多个应用或版本能在各自边界内处理合成事件,降低顶层事件系统相互干扰的风险;跨容器的原生冒泡规则仍需单独考虑。

3) 埋点SDK用原生addEventListener,业务组件使用onClick,一次点击出现顺序与组件树直觉不一致;排查时为什么不能只看JSX嵌套?

答: 业务onClick经过React的统一监听、合成和dispatchEvent分发,并非直接绑定在JSX对应节点上。应同时确认原生监听器所在节点、捕获或冒泡阶段,以及React版本对应的委托层;混用两套事件系统时,原生传播路径与组件树模拟分发不能混为一谈。

4) 线上只有部分事件能进入组件处理器,click正常但某个自定义事件名称毫无响应;你会怎样判断是不是合成事件系统的问题?

答: 先确认该事件是否属于React支持并会捕获生成SyntheticEvent的事件集合,不能假设任意事件名都会被统一分发。若不在支持范围,应改用合适的原生监听或封装桥接;若属于支持事件,再沿原生冒泡、根容器监听和dispatchEvent定位中断点。

5) 基础设施团队主张全部改用原生事件,理由是SyntheticEvent只是多包了一层;你会保留它的哪些工程价值?

答: 合成事件统一了不同浏览器的事件接口,并让React集中管理监听和分发,避免组件更新时反复处理大量节点监听器。全部改用原生事件会获得更直接的底层控制,但需要自行承担兼容性、解绑和与React委托边界协作的成本;两套体系混用还会增加传播顺序的复杂度。

626. JSX语法糖本质

JSX本质上是JavaScript的语法扩展,也是创建ReactElement的语法糖。经典转换中,Babel会先把它解析成AST,再转换为React.createElement调用,最终得到描述节点类型、属性和子元素的普通对象。这个对象组成虚拟DOM树,后续经过对比和更新渲染到页面。实际开发选择JSX,是因为它比手写嵌套函数更直观,也不需要额外引入模板指令。JSX是语法糖,通过babel转成React.createElement函数,在babel官网上可以在线把JSX转成React的JS语法 首先解析出来的话,就是一个createElement函数然后这个函数执行完后,会返回一个vnode通过vdom的patch或者是其他的一个方法,最后渲染一个页面script标签中不添加text/babel解析jsx语法的情况下 const ele=React.createElement("h2",null,"Hello React!");// 历史写法: React 18 起用 ReactDOM.createRoot(...).render(ele), React 19 已移除 ReactDOM.renderReactDOM.render(ele,document.getElementById("app"));

JSX的本质是React.createElement()函数 createElement函数返回的对象是ReactElement对象。createElement的写法如下 class App extends React.Component{constructor(){super()}this.state={}}render(){return React.createElement("div",null,/*第一个子元素, header*/React.createElement("div",{

```
className:"header"},React.createElement("h1",{title:"\u6807\u9898"},"\u6211\u662F\u6807\u9898"))/*第二个子元素，content*/React.createElement("div",{className:"content"},React.createElement("h2",null,"\u6211\u662F\u9875\u9762\u7684\u5185\u5BB9"),React.createElement("button",null,"\u6309\u94AE"),React.createElement("button",null,"+1"),React.createElement("a",{href:"http://www.baidu.com"},"\u767E\u5EA6\u4E00\u4E0B"))/*第三个子元素，footer*/React.createElement("div",{className:"footer"},React.createElement("p",null,"\u6211\u662F\u5C3E\u90E8\u7684\u5185\u5BB9")));};// 历史写法：React 18 起用 ReactDOM.createRoot(document.getElementById("app")).render()ReactDOM.render(,document.getElementById("app"));
```

实际开发中不会使用createElement来创建ReactElement的，一般都是使用JSX的形式开发。

ReactElement在程序中打印一下

```
render(){let ele=(我是标题我是页面的内容按钮+1 百度一下我是尾部的内容)console.log(ele);return ele;}
```

react通过babel把JSX转成createElement函数，生成ReactElement对象，然后通过ReactDOM.render函数把ReactElement渲染成真实的DOM元素

为什么 React 使用 JSX 在回答问题之前，我首先解释下什么是 JSX 吧。JSX 是一个JavaScript的语法扩展，结构类似 XML。JSX 主要用于声明React元素，但 React 中并不强制使用JSX。即使使用了JSX，也会在构建过程中，通过Babel 插件编译为React.createElement。所以 JSX 更像是React.createElement的一种语法糖接下来与 以外的三种技术方案进行对比首先是模板，React

团队认为模板不应该是开发过程中的关注点，因为引入了模板语法、模板指令等...

1) 代码评审里有人说会在浏览器里直接创建一个名为UserCard的DOM标签；你会用编译产物怎样反驳？

答：JSX会先经Babel转换为类似React.createElement(UserCard, {name: "Ada"})的调用，其中type是组件本身而不是浏览器标签字符串。该调用组装并返回ReactElement描述，之后才由React的渲染流程把它落实到真实DOM。

2) 低代码平台不能使用JSX，只允许执行普通JavaScript；要生成一个带两个li的ul，最少需要向createElement提供哪些信息？

答：需要提供节点type、保存属性的config，以及从第三个参数开始传入的一个或多个children。例如父节点类型为"ul"，子项分别是嵌套的React.createElement("li", {key: "..."}, "...")；返回值仍是元素描述，不会在调用时直接操作DOM。

3) 调试台打印JSX变量时只看到带type、key、ref、props的普通对象，开发者以为渲染失败了；这个现象正常吗？

答：正常，createElement的职责主要是整理参数并组装ReactElement，所以JSX表达式的结果本来就是JavaScript对象形式的节点描述。真实节点要等后续渲染与补丁流程处理；仅打印元素对象看不到浏览器DOM，不能据此判断失败。

4) 组件传入key、ref、className和多个子节点后，业务代码发现props的形状与原始配置对象不同；createElement做了哪些整理？

答：createElement会单独提取key、ref等保留字段，把普通配置项放入props，并将子节点整理到props.children。一个子节点通常直接保存，多个子节点会组成数组；因此不能把所有JSX属性都假设为组件可读取的普通props。

5) 团队准备自己写Babel插件，把内部XML风格语法转换成React元素；最核心的AST转换动作是什么？

答：插件需要在Babel解析得到AST后访问JSXElement，提取标签、属性和子节点，再把该节点替换为React.createElement调用表达式。难点不只是替换函数名，还要完整保留组件类型、属性表达式和

嵌套子节点；只处理字符串标签会在组件或复杂属性场景失效。

6) 模板方案和JSX方案选型时，有人认为JSX是运行时模板解释器；基于其工作链路，你会怎样说明取舍？

答：JSX是JavaScript的语法扩展，通常在构建阶段转换为React.createElement，并不依赖浏览器把它当模板解释。它减少了额外模板指令概念，并有利于编辑器提示，但仍需要编译工具处理；若运行环境没有对应转换，就必须直接编写普通JavaScript调用。

627. 为什么 React 元素有一个 \$\$typeof 属性

React元素上的\$\$typeof是身份标记，用来区分可信的ReactElement与可能从外部混入的普通对象。它的值使用Symbol，而Symbol不能被JSON序列化，因此服务端或数据库返回的JSON很难直接伪造成合法元素。React检查不到这个标记时会拒绝按元素处理，从而降低旧式元素注入导致XSS的风险。它是一层安全护栏，但不能替代对dangerouslySetInnerHTML内容本身的谨慎处理。目的是为了防止 XSS 攻击。因为Symbol 无法被序列化，所以 React 可以通过有没有 \$\$typeof 属性来断出当前的 element 对象是从数据库来的还是自己生成的。如果没有 \$\$typeof 这个属性，react 会拒绝处理该元素。在 React 的古老版本中，下面的写法会出现 XSS 攻击：

```
// 服务端允许用户存储 JSON let expectedTextButGotJSON = { type: 'div', props: { dangerouslySetInnerHTML: { __html: '/* 把你想的搁着 */' }, }, // ... }; let message = { text: expectedTextButGotJSON }; // React 0.13 中有风险 {message.text}
```

1) 评论页原本只显示后端返回的文本，但接口把一段带type、props和dangerouslySetInnerHTML的JSON对象交给了{message.text}；没有\$\$typeof校验时会发生什么？

答：旧版React可能把这个普通对象误认成元素，并按其中的props创建节点，最终让攻击者控制的HTML进入页面。\$\$typeof相当于元素身份标记；缺少该标记的对象不会作为合法元素处理，但普通文本输出是否安全仍取决于具体渲染入口。

2) 消息中心既接收服务端JSON，又允许业务代码通过React.createElement动态生成提示组件，你会怎样划分两类数据的信任边界？

答：服务端JSON只能作为文本或经过校验的业务数据，不能直接充当待渲染的元素对象；动态组件则应在前端通过JSX或React.createElement创建。这样生成的元素带有React认可的\$\$typeof，但组件类型和危险属性仍不能由用户输入任意决定。

3) 如果运行环境支持把Symbol转成自定义字符串并写入数据库，攻击者能否仅靠伪造同名的"\$\$typeof"字段绕过检查？

答：仅写入同名字符串通常不能等价于React内部使用的Symbol值，因为标准JSON无法表达Symbol。不过若应用在反序列化后主动把外部标记转换为真实标记，就重新打开了信任通道；因此不要为用户数据补造元素身份。

4) 一次发布后，富文本详情页仍出现脚本注入，而监控显示所有可疑对象都没有通过元素身份校验，你会先排查哪些渲染入口？

答：应先搜索dangerouslySetInnerHTML以及把外部内容交给组件类型或属性的路径，确认注入是否绕开了伪造元素这一入口。\$\$typeof只阻止普通JSON冒充React元素，不能替代HTML净化、URL协议校验或服务端输入治理。

5) 安全评审有人主张删掉\$\$typeof，改为递归检查对象是否只有type、props、children，这种方案为什么不可靠？

答：字段白名单只能判断对象的形状，攻击者仍可构造完全符合形状约束的JSON，并把危险内容放进允许的属性中。使用JSON无法序列化的Symbol作为身份标记，能区分前端创建的元素和数据库对象

；代价是它只提供来源识别，不负责内容安全。

628. Virtual DOM 的工作原理是什么

虚拟DOM的工作流程可以概括为用对象描述界面、比较前后差异，再把patch应用到真实DOM。JSX经编译并执行后会生成包含节点类型、props和children的普通对象，这些对象共同组成树。状态变化时，框架对新旧树执行diff，只根据差异更新真实页面，从而减少人为操作DOM的风险。它还便于服务端渲染和跨平台复用，但会占用额外内存，也不意味着所有高性能场景都更合适。虚拟DOM的工作原理是通过JS对象模拟DOM的节点。在Facebook构建React初期时，考虑到要提升代码抽象能力、避免人为的DOM操作、降低代码整体风险等因素，所以引入了虚拟DOM。虚拟DOM在实现上通常是Plain Object，以React为例，在render函数中写的JSX会在Babel插件的作用下，编译为React.createElement执行JSX中的属性参数React.createElement执行后会返回一个Plain Object，它会描述自己的tag类型、props属性以及children情况等。这些Plain Object通过树形结构组成一棵虚拟DOM树。当状态发生变更时，将变更前后的虚拟DOM树进行差异比较，这个过程称为diff，生成的结果称为patch。计算之后，会渲染Patch完成对真实DOM的操作。虚拟DOM的优点主要有三点：改善大规模DOM操作的性能、规避XSS风险、能以较低的成本实现跨平台开发。虚拟DOM的缺点在社区中主要有两点内存占用较高，因为需要模拟整个网页的真实DOM高性能应用场景存在难以优化的情况，类似像Google Earth一类的高性能前端应用在技术选型上往往不会选择React。除了渲染页面，虚拟DOM还有哪些应用场景？这个问题考验面试者的想象力。通常而言，我们只是将虚拟DOM与渲染绑定在一起，但实际上虚拟DOM的应用更为广阔。比如，只要你记录了真实DOM变更，它甚至可以应用于埋点统计与数据记录等。SSR原理借助虚拟dom,服务器中没有dom概念的，react巧妙的借助虚拟dom，然后可以在服务器中nodejs可以运行起来react代码。

1) 商品页只更新购物车角标，有人却说Virtual DOM会直接把整页真实DOM重建一遍；你会怎样纠正这个判断？

答：状态变化后会先得到新的虚拟DOM树，再与旧树执行diff，计算出描述差异的patch，最后把必要变更应用到真实DOM。虚拟树本身通常是描述标签、props和子节点的普通JS对象，但比较和保存这些对象也有计算与内存成本。

2) 你接手一个用JSX写成的活动页，评审要求画出从render到浏览器节点更新的链路，关键步骤应该是什么？

答：JSX会经编译转换为创建元素的调用，执行后产生描述节点的普通对象，并按子节点关系组成虚拟DOM树。状态更新时生成新树，diff得出patch，渲染阶段再据此操作真实DOM；这条链路把声明式描述与具体DOM操作隔开。

3) 同一套资讯卡片既要在浏览器渲染，也计划输出到非DOM宿主环境，虚拟DOM在这个约束变化下提供了什么能力？

答：虚拟DOM是与真实DOM分离的对象描述，因此上层组件逻辑不必直接依赖浏览器节点，底层可以针对不同平台实现对应渲染过程。跨平台并非自动免费获得，各宿主仍需处理属性、事件和生命周期差异，平台特有功能也需要适配。

4) 一个包含大量节点的地图式页面持续掉帧，Profiler显示JS比较和对象分配都很重，但真实DOM更新并不多，你会如何解释并推进排查？

答：瓶颈可能位于虚拟树创建和diff，而不是最终的真实DOM写入；虚拟DOM不保证所有高性能场景都更快。应缩小参与更新的树和更新频率，并评估局部直接绘制或更专用的渲染技术；类似高性能地

图场景可能并不适合由React主导全部渲染。

5) 服务端没有浏览器DOM，却要为文章详情页生成首屏HTML，虚拟DOM为什么仍能参与这条链路？

答：组件先生成与浏览器DOM无关的虚拟节点描述，因此相关代码可以在Node.js环境运行，再由服务端渲染器把描述转换为HTML。它解决的是渲染描述与宿主解耦，浏览器接管后的事件绑定和状态衔接仍需另一阶段完成。

629. React有哪些优化性能的手段

React性能优化的核心是避免无意义的重渲染，并把暂时用不到的代码延后加载。类组件可以用PureComponent或shouldComponentUpdate控制更新，函数组件则常用React.memo；局部计算和回调再按需要交给useMemo、useCallback。列表更新时要使用稳定且唯一的key，避免用数组下标造成错误复用。体积较大的页面还可以通过lazy配合Suspense懒加载，但这些手段应在确有重复渲染或加载成本时使用。

类组件中的优化手段 使用纯组件PureComponent作为基类。使用shouldComponentUpdate生命周期函数来自定义渲染逻辑。 方法组件中的优化手段 使用React.memo高阶函数包装组件，React.memo可以实现类似于shouldComponentUpdate或者PureComponent的效果使用useMemo使用React.useMemo精细化的管控，useMemo 控制的则是是否需要重复执行某一段逻辑，而React.memo控制的是是否需要重渲染一个组件使用useCallback。 使用React.useMemo精细化的管控，useMemo控制的则是是否需要重复执行某一段逻辑，而React.memo 控制的是是否需要重渲染一个组件 其他方式在列表需要频繁变动时，使用唯一 id 作为 key，而不是数组下标。必要时通过改变 CSS 样式隐藏显示组件，而不是通过条件判断显示隐藏组件。使用Suspense和 lazy 进行懒加载，例如：

```
import React, { lazy, Suspense } from "react";
export default class CallingLazyComponent extends React.Component {
  render() {
    var ComponentToLazyLoad = null;
    if (this.props.name === "Mayank") {
      ComponentToLazyLoad = lazy(() => import("./mayankComponent"));
    } else if (this.props.name === "Anshul") {
      ComponentToLazyLoad = lazy(() => import("./anshulComponent"));
    }
    return (
      This is the Base User: {this.state.name} Loading...
    );
  }
}
```

1) 订单列表父组件每次输入筛选词都会创建新数组，团队给所有子组件套了React.memo，但渲染次数没有下降；你会指出哪里判断错了？

答：React.memo控制的是组件在属性可比较为未变化时跳过重渲染，新数组会破坏引用稳定性，使包装失去预期效果。应先确认筛选计算和传入属性是否需要useMemo，但缓存本身也有维护成本，简单计算或持续变化的依赖未必受益。

2) 后台表格有数千条可编辑记录，某个单元格更新会带动整页刷新；在不改业务功能的前提下，你会怎样落地优化？

答：先用React.memo或类组件的PureComponent隔离无关行，并保证列表使用稳定且唯一的业务id作为key。对确实昂贵的派生计算使用useMemo，对需要稳定传递的回调用useCallback；若属性始终变化，这些优化仍会失效。

3) 仪表盘组件切换后必须保留内部输入和滚动位置，产品同时要求隐藏期间减少开销；你会选择条件卸载还是CSS隐藏？

答：需要保留组件实例和内部状态时，可以通过CSS切换显示状态，避免反复卸载与挂载。代价是隐藏组件仍占用DOM和内存，也可能继续响应状态更新；若组件长期不用或资源昂贵，条件卸载通常更合适。

4) 一次重构把列表key从业务id换成数组下标，排序后出现输入框内容跟着错误行移动；你会如何定位并修复？

答：排序改变了下标与业务记录的对应关系，React会按旧key复用节点，从而把局部状态保留在错误条目上。恢复稳定且唯一的业务id，并检查新增、删除和排序链路；只有列表顺序和成员永久不变时，下标风险才相对有限。

5) 首屏包体过大，设置页中的两个低频模块只有用户进入特定分支才需要，你会怎样使用lazy和Suspense，又要防什么问题？

答：可用lazy(() => import(...))延迟加载分支组件，并在外层用Suspense提供加载期间的占位内容。这样把非首屏代码推迟到需要时下载，但首次进入会产生等待；加载边界、失败处理以及组件引用是否有效都需要单独设计。

630. Redux实现原理解析

Redux本质上用单向数据流和唯一的store，把状态变化收口到dispatch(action)与纯函数reducer。createStore内部维护state和监听器，派发后调用reducer生成新状态，再通知订阅者更新视图。异步请求、日志等副作用由中间件拦截action，通过逐层包装dispatch扩展流程。使用时不能直接修改state，否则状态变化会变得不可预测。

在 Redux 的整个工作过程中，数据流是严格单向的。这一点一定要背下来，面试的时候也一定要记得说为什么用redux 在React中，数据在组件中是单向流动的，数据从一个方向父组件流向子组件（通过props），所以，两个非父子组件之间通信就相对麻烦，redux的出现就是为了解决state里面的数据问题

Redux设计理念 Redux是将整个应用状态存储到一个地方上称为store,里面保存着一个状态树store tree,组件可以派发(dispatch)行为(action)给store,而不是直接通知其他组件，组件内部通过订阅store中的状态state来刷新自己的视图 如果你想对数据进行修改，只有一种途径：派发 action。action 会被 reducer 读取，进而根据 action 内容的不同对数据进行修改、生成新的 state（状态），这个新的 state 会更新到 store 对象里，进而驱动视图层面做出对应的改变。

Redux三大原则 唯一数据源 整个应用的state都被存储到一个状态树里面，并且这个状态树，只存在于唯一的store中 保持只读状态 state是只读的，唯一改变state的方法就是触发action，action是一个用于描述以发生时间的普通对象

数据改变只能通过纯函数来执行

使用纯函数来执行修改，为了描述action如何改变state的，你需要编写reducers 从编码的角度理解

Redux workflow 使用createStore 来完成 store 对象的创建 // 引入 reduximport{createStore}from'redux'// 创建 storeconststore=createStore(reducer,initial_state,applyMiddleware(middleware1,middleware2,...)); createStore 方法是一切的开始，它接收三个入参： reducer；初始状态内容；指定中间件 reducer 的作用是将新的 state 返回给 store 一个 reducer 一定是一个纯函数，它可以有各种各样的内在逻辑，但它最终一定要返回一个 state：

constreducer=(state,action)=>{// 此处是各种样的 state处理逻辑returnnew_state} 当我们基于某个 reducer 去创建 store 的时候，其实就是给这个 store 指定了一套更新规则： // 更新规则全都写在 reducer 里conststore=createStore(reducer) action 的作用是通知 reducer “让改变发生” 要想让 state 发生改变，就必须用正确的 action 来驱动这个改变。

constaction={type:"ADD_ITEM",payload:'text'} action 对象中允许传入的属性有多个，但只有 type 是必传的。type 是 action 的唯一标识，reducer 正是通过不同的 type 来识别出需要更新的不同的 state，由此才能够实现精准的“定向更新”。 派发 action，靠的是 dispatch action 本身只是一个对象，要想让 reducer 感知到 action，还需要“派发 action”这个动作，这个动作是由 store.dispatch 完成的。这里我简单地示范一下：

```
import{createStore}from'redux'// 创建 reducerconstreducer=(state,action)=>{// 此处是各种样的 state处理逻辑returnnew_state}// 基于 reducer 创建 stateconststore=createStore(reducer)// 创建一个 action，这个 action 用 "ADD_ITEM" 来标识constaction={type:"ADD_ITEM",payload:'text'}// 使用 dispatch 派发 action，action 会进入到 reducer 里触发对应的更新store.dispatch(action)
```

以上这段代码，是从编码角度对 Redux ...

1) 购物车页面有人绕过dispatch，直接修改从store.getState()取得的数组，视图偶尔更新、调试记录却没有对应事件；为什么这违背了Redux的工作流？

答：Redux规定状态只读，变更必须由普通action经store.dispatch送入reducer，再生成新的state。直接修改既绕开了订阅通知和行为记录，也破坏了可预测性；即使页面偶然显示正确，时间回溯和问题定位也会失真。

2) 请你在代码评审中补全一个最小createStore：它要支持读取状态、派发行为和订阅更新，dispatch内部的执行顺序应是什么？

答：闭包保存state和监听函数集合，getState返回当前状态，subscribe登记监听者。dispatch(action)先执行reducer(state, action)得到新状态，再依次通知监听者；初始化时还需触发一次reducer，使默认状态得以建立。

3) 支付流程要求dispatch接受函数来发起异步请求，但现有reducer只能消费带type的普通对象；你会把扩展点放在哪里？

答：应通过中间件包装dispatch，让函数类型的action先被类似redux-thunk的逻辑拦截，并在异步完成后派发普通action。reducer继续保持纯函数，只负责根据当前状态和action生成新状态；请求等副作用进入reducer会破坏结果可复现性。

4) 线上接入两个日志与异步中间件后，执行顺序和配置顺序相反，排查applyMiddleware时你会关注哪段函数组合？

答：中间件本质是加工dispatch的工厂，形态可表示为store => next => action，每层把包装后的函数交给下一层。组合时常通过反向构造形成嵌套，最终得到的最外层会最先执行；若某层未调用next(action)，后续中间件和reducer都会被截断。

5) 大型表单拆成多个领域reducer后，团队打算让每个reducer都返回完整状态树；combineReducers应怎样避免这种耦合？

答：combineReducers可遍历reducer映射，把各自对应的state[attr]和同一个action传入，再将返回值组装成新的总状态。每个reducer只管理自己的状态切片，边界更清晰；切片命名或状态结构调整会影响选择器和持久化数据，需要同步迁移。

6) 同事认为action自己会修改状态，因此在actionCreator里直接操作store；你会怎样区分action、reducer和store的职责？

答：action只是描述已发生行为的普通对象，至少需要用于识别行为的type；actionCreator只是生成该对象。store.dispatch负责把它交给reducer，后者计算新状态并由store保存、通知订阅者；把修改逻辑藏进actionCreator会破坏单向数据流。

631. 谈谈你对状态管理的理解

状态管理的核心，是用明确的数据流统一保存和修改跨组件状态，让变化可预测、可追踪。Flux通过Action、Dispatcher、Store到视图的单向流动，解决了数据流向不清的问题；Redux则进一步用单一数据源、只读状态和纯函数reducer约束更新。异步请求等副作用通常交给中间件处理。若更看重少量样板代码和自动响应更新，也可以选择Mobx。

首先介绍 Flux，Flux 是一种使用单向数据流的形式来组合 React 组件的应用架构。Flux 包含了 4 个部分，分别是Dispatcher、Store、View、Action。Store存储了视图层所有的数据，当Store变化后会引起 View 层的更新。如果在视图层触发一个Action，就会使当前的页面数据值发生变化。Action 会被 Dispatcher 进行统一的收发处理，传递给 Store 层，Store 层已经注册过相关 Action 的处理逻辑，处理对应的内部状态变化后，触发 View 层更新。Flux 的优点是单向数据流，解决了 MVC 中

数据流向不清的问题，使开发者可以快速了解应用行为。从项目结构上简化了视图层设计，明确了分工，数据与业务逻辑也统一存放管理，使在大型架构的项目中更容易管理、维护代码。其次是 Redux，Redux 本身是一个 JavaScript 状态容器，提供可预测化状态的管理。社区通常认为 Redux 是 Flux 的一个简化设计版本，它提供的状态管理，简化了一些高级特性的实现成本，比如撤销、重做、实时编辑、时间旅行、服务端同构等。Redux 的核心设计包含了三大原则：单一数据源、纯函数 Reducer、State 是只读的。Redux 中整个数据流的方案与 Flux 大同小异。Redux 中的另一大核心点是处理“副作用”，AJAX 请求等异步工作，或不是纯函数产生的第三方的交互都被认为是“副作用”。这就造成在纯函数设计的 Redux 中，处理副作用变成了一件至关重要的事情。社区通常有两种解决方案：第一类是在 Dispatch 的时候会有一个 middleware 中间件层，拦截分发的 Action 并添加额外的复杂行为，还可以添加副作用。第一类方案的流行框架有 Redux-thunk、Redux-Promise、Redux-Observable、Redux-Saga 等。第二类是允许 Reducer 层中直接处理副作用，采取该方案的有 React Loop，React Loop 在实现中采用了 Elm 中分形的思想，使代码具备更强的组合能力。除此以外，社区还提供了更为工程化的方案，比如 rematch 或 dva，提供了更详细的模块架构能力，提供了拓展插件以支持更多功能。Redux 的优点很多：结果可预测；代码结构严格易维护；模块分离清晰且小函数结构容易编写单元测试；Action 触发的方式，可以在调试器中使用时间回溯，定位问题更简单快捷；单一数据源使服务端同构变得更为容易；社区方案多，生态也更为繁荣。最后是 Mobx，Mobx 通过监听数据的属性变化，可以直接在数据上更改触发 UI 的渲染。在使用上更接近 Vue，比起 Flux 与 Redux 的手动挡的体验，更像开自动挡的汽车。Mobx 的响应式实现原理与 Vue 相同，以 Mobx 5 为分界点，5 以前采用 Object.defineProperty 的方案，5 及以后使用 Proxy 的方案。它的优点是样板代码少、简单粗暴、用户学习快、响应式自动更新数据让开发者的心智负担更低。Mobx 在开发项目时简单快速，但应用 Mobx 的场景，其实完全可以用 Vue 取代。如果纯用 Vue，体积还会更小巧。第一类是在 Dispatch 的时候会有一个 middleware 中间件层，拦截分发的 Action 并添加额外的复杂行为，还可以添加副作用。第一类方案的流行框架有 Redux-thunk、Redux-Promise、Redux-Observable、Redux-Saga 等。第二类是允许 Reducer 层中直接处理副作用，采取该方案的有 React Loop，React Loop 在实现中采用了 Elm 中分形的思想，使代码具备更强的组合能力。除此以外，社区还提供了更为工程化的方案，比如 rematch 或 dva，提供了更详细的模块架构能力，提供了拓展插件以支持更多功能。结果可预测；代码结构严格易维护；模块分离清晰且小函数结构容易编写单元测试；Action 触发的方式，可以在调试器中使用时间回溯，定位问题更简单快捷；单一数据源使服务端同构变得更为容易；社区方案多，生态也更为繁荣。

1) 一个小型资料页只有父子组件传值，架构师却要求所有输入都先经过 Dispatcher 和全局 Store；你会如何评价这种选型？

答：Flux 的价值在于用 Action、Dispatcher、Store 和 View 建立清晰的单向数据流，尤其适合状态关系复杂的应用。简单父子传值强行全局化会增加结构和行为跳转成本；是否引入应由共享范围、变更复杂度和维护需求决定。

2) 协作编辑页面需要统一管理文档、选区和工具栏状态，并希望每次变化都能定位到触发行为；你会怎样组织 Flux 数据流？

答：视图触发 Action，由 Dispatcher 统一分发给已注册处理逻辑的 Store，Store 更新内部状态后通知 View 刷新。这样数据沿单一方向流动，行为入口容易追踪；若把所有局部交互都塞入同一 Store，模块边界仍可能变得模糊。

3) 产品新增撤销、重做和服务端同构要求，现有响应式状态允许任意位置直接赋值；为什么团队可能转向 Redux？

答：Redux 以单一数据源、只读状态和纯函数 Reducer 约束变化，使同一组 action 更容易重放，因此更适合撤销、重做、时间回溯和同构等需求。代价是更新路径更严格，副作用不能随意混入 reducer，

还需额外的中间件或工程封装。

4) 线上偶发状态无法复现，日志中同一个action重放后得到不同结果，而reducer内部读取了时间并发起AJAX；你会怎样修正？

答：应把时间读取和AJAX等副作用移出纯函数reducer，通过dispatch前的中间件处理，再把确定结果封装成普通action。Reducer只依据传入的state与action计算新状态；否则时间旅行、单元测试和故障重放都会失去可信度。

5) 团队在Redux与Mobx之间争论：大型权限后台强调变更审计，原型团队更看重少样板和直接赋值，你会怎样取舍？

答：强调可预测、严格结构和行为追踪时，Redux的action与纯reducer更契合，也便于测试和时间回溯。追求快速开发和自动响应更新时，Mobx的属性监听与直接修改更轻量；代价是隐式依赖增多后，变化来源通常不如显式action清晰。

6) 维护一个旧项目时发现Mobx的响应式层依赖属性拦截，新模块却使用Proxy；这与它和Vue的相邻原理有什么关系？

答：Mobx与Vue的响应式思路相近，都是监听数据属性变化后驱动UI更新；资料所述的分界是Mobx 5以前使用Object.defineProperty，之后使用Proxy。两种机制的能力边界不同，混合维护时应以项目实际版本和构建环境为准，不能只按写法推断。

632. connect组件原理分析

connect是连接React组件与Redux store的高阶函数，负责订阅状态并把数据和派发能力映射成组件的props。Provider先通过context向下传递store，connect再用mapStateToProps和mapDispatchToProps计算组件真正需要的内容。状态或组件自身props变化时，它会重新计算并判断是否需要渲染。组件卸载时还要取消订阅，避免遗留监听。

1. connect用法 作用：连接React组件与Redux store
connect([mapStateToProps],[mapDispatchToProps],[mergeProps],[options])// 这个函数允许我们将store 中的数据作为 props 绑定到组件上const mapStateToProps=(state)=>{return{count:state.count}}这个函数的第一个参数就是Redux的store，我们从中摘取了count属性。你不必将state中的数据原封不动地传入组件，可以根据state中的数据，动态地输出组件需要的（最小）属性函数的第二个参数ownProps，是组件自己的props 当state变化，或者ownProps变化的时候，mapStateToProps都会被调用，计算出一个新的stateProps，（在与ownProps merge后）更新给组件

mapDispatchToProps(dispatch,ownProps):dispatchProps connect的第二个参数是mapDispatchToProps，它的功能是，将action作为props绑定到组件上，也会成为MyComp的 `props`

2. 原理解析 首先connect之所以会成功，是因为Provider组件 在原应用组件上包裹一层，使原来整个应用成为Provider的子组件接收Redux的store作为props，通过context对象传递给子孙组件上的connect

connect做了些什么 它真正连接Redux和React，它包在我们的容器组件的外一层，它接收上面Provider提供的store里面的state和dispatch，传给一个构造函数，返回一个对象，以属性形式传给我们的容器组件

3. 源码 connect是一个高阶函数，首先传入mapStateToProps、mapDispatchToProps，然后返回一个生产Component的函数(wrapWithConnect)，然后再将真正的Component作为参数传入wrapWithConnect，这样就生产出一个经过包裹的Connect组件，该组件具有如下特点

通过props.store获取祖先Component的store props包括stateProps、dispatchProps、parentProps,合并在一起得到nextState，作为props传给真正的ComponentcomponentDidMount时，添加事件this.store.subscribe(this.handleChange)，实现页面交互shouldComponentUpdate时判断是否有避免进行渲染，提升页面性能，并得到nextStatecomponentWillUnmount时移除注册的事件this.handleChange

// 主要逻辑export default function connect(mapStateToProps,mapDispatchToProps,mergeProps,options={}){return function wrapWithConnect(WrappedComponent){class Connect extends Compo

```

nent{constructor(props,context){// 从祖先Component处获得storethis.store=props.store||context.s
torethis.stateProps=computeStateProps(this.store,props)this.dispatchProps=computeDispatchPro
ps(this.store,props)this.state={storeState:null}// 对stateProps、dispatchProps、parentProps进行合
并this.updateState())shouldComponentUpdate(nextProps,nextState){// 进行判断，当数据发生改变
时，Component重新渲染if(propsChanged||mapStateProducedChang...

```

1) 商品列表页把store直接作为一个data属性传给展示组件，团队说这样比写mapStateToProps省事；当无关的登录状态变化时，列表也收到新属性，这种接法的问题在哪？

答：connect的职责不是原样转发整个state，而是通过mapStateToProps摘取组件真正需要的最小数据。传递范围过大会扩大组件与全局状态的耦合，也让无关状态变化更容易进入更新链路；代价是后续拆分、测试和定位渲染来源都会更困难。

2) 订单容器既要读取state.orders，又要根据父组件传入的userId过滤数据，还要暴露reload操作；你会怎样组织connect的几个参数？

答：在mapStateToProps(state, ownProps)中结合userId计算订单属性，在mapDispatchToProps(dispatch, ownProps)中封装需要派发的操作。最后由默认或自定义mergeProps合并stateProps、dispatchProps与父级属性；合并规则越复杂，越要防止同名属性被意外覆盖。

3) 同一个详情组件要挂在两个不同的Provider下，父组件还可能通过props.store显式传入另一个store；此时包装层应该从哪里取状态，更新订阅又跟着谁？

答：按给出的实现，包装层优先读取props.store，否则才从context获取祖先Provider提供的store。状态读取、dispatch和subscribe必须绑定同一个实例，否则页面可能展示一个仓库的数据却监听另一个仓库；显式覆盖因此需要非常谨慎。

4) 线上出现“ReduxDevTools里状态已经变化，但被connect包装的库存组件没有刷新”，你会沿着包装组件的哪条链路排查？

答：先确认Provider传入的store是否正确，再检查挂载时是否执行了store.subscribe，订阅回调有没有通过setState触发包装层更新。随后核对mapStateToProps是否产出变化，以及shouldComponentUpdate是否错误拦截；卸载过早或订阅错误实例也会造成同样现象。

5) 团队想把所有mapStateToProps、mapDispatchToProps和父级属性都手工塞进一个对象再传下去，认为这样更可控；什么时候值得写mergeProps，什么时候应保持默认合并？

答：只有属性需要重命名、组合或根据父级参数形成特定接口时，才值得提供mergeProps。普通场景保留默认合并更容易追踪数据来源，也减少每次更新时的额外计算；自定义合并若持续产生不稳定结果，还会削弱包装层避免无效渲染的能力。

6) 类组件页面不能直接读取context中的Redux数据，架构师建议每个组件都手写订阅；connect相比这种方案补齐了哪些生命周期责任？

答：connect把取store、计算状态与派发属性、订阅变化、判断更新和向真实组件传参集中在高阶组件中。组件挂载时注册订阅，卸载时还应移除订阅，避免遗留监听；手写方案若漏掉清理或更新判断，很容易产生泄漏或冗余渲染。

633. React Hooks

React Hooks让函数组件能够使用状态和副作用，并把相关逻辑抽成可复用的自定义Hook。它解决了类组件中生命周期拆散业务逻辑、this使用复杂，以及HOC或render props容易造成嵌套的问题。由于内部按调用顺序保存Hook，只能在函数组件或自定义Hook顶层调用，不能放进条件、循环或嵌套函数。普通副作用优先用useEffect，需要同步读取或调整DOM、避免闪烁时再用useLayoutEffect。

代码逻辑聚合，逻辑复用HOC嵌套地狱代替class React 中通常使用 类定义 或者 函数定义 创建组件：在类定义中，我们可以使用到许多 React 特性，例如 state、各种组件生命周期钩子等，但是在函数定义中，我们却无能为力，因此 React 16.8 版本推出了一个新功能 (React Hooks)，通过它，可以更好的在函数定义组件中使用 React 特性。

函数组件与类组件的对比：无关“优劣”，只谈“不同” 类组件需要继承 class，函数组件不需要；类组件可以访问生命周期方法，函数组件不能；类组件中可以获取到实例化后的 this，并基于这个 this 做各种各样的事情，而函数组件不可以；类组件中可以定义并维护 state（状态），而函数组件不可以；但是类组件它太重了，对于解决许多问题来说，编写一个类组件实在是一个过于复杂的姿势。复杂的姿势必然带来高昂的理解成本，这也是我们所不想看到的 react hooks的好处：跨组件复用：其实 render props / HOC 也是为了复用，相比于它们，Hooks 作为官方的底层 API，最为轻量，而且改造成本小，不会影响原来的组件层次结构和传说中的嵌套地狱；类定义更为复杂 不同的生命周期会使逻辑变得分散且混乱，不易维护和管理；时刻需要关注this的指向问题；代码复用代价高，高阶组件的使用经常会使整个组件树变得臃肿； 状态与UI隔离： 正是由于 Hooks 的特性，状态逻辑会变成更小的粒度，并且极容易被抽象成一个自定义 Hooks，组件中的状态和 UI 变得更为清晰和隔离。 注意：避免在 循环/条件判断/嵌套函数 中调用 hooks，保证调用顺序的稳定；只有 函数定义组件 和 hooks 可以调用 hooks，避免在 类组件 或者 普通函数 中调用；不能在useEffect中使用useState，React 会报错提示；类组件不会被替换或废弃，不需要强制改造类组件，两种方式能并存； 重要钩子 状态钩子 (useState)： 用于定义组件的 State，其到类定义中this.state的功能； // useState 只接受一个参数：初始状态// 返回的是组件名和更改该组件对应的函数const[flag,setFlag]=useState(true);// 修改状态setFlag(false)// 上面的代码映射到类定义中:this.state={flag:true}constflag=this.state.flagconstsetFlag=(bool)=>{this.setState({flag:bool,})} 生命周期钩子 (useEffect)： 类定义中有许多生命周期函数，而在 React Hooks 中也提供了一个相应的函数 (useEffect)，这里可以看做componentDidMount、componentDidUpdate和componentWillUnmount的结合。

useEffect(callback, [source])接受两个参数 callback： 钩子回调函数；source： 设置触发条件，仅当 source 发生改变时才会触发；useEffect钩子在没有传入[source]参数时，默认在每次 render 时都会优先调用上次保存的回调中返回的函数，后再重新调用回调； useEffect(()=>{// 组件挂载后执行事件绑定console.log('on')addEventListener()// 组件 update 时会执行事件解绑return()=>{console.log('off')removeEventListener()},{source]);// 每次 source 发生改变时，执行结果(以类定义的生命周期，便于大家理解):// --- DidMount ---// 'on'// --- DidUpdate ---// 'off'// 'on'// --- DidUpdate ---// 'off'// 'on'// --- WillUnmount ---// 'off' 通过第二个参数，我们便可模拟出几个常用的生命周期：componentDidMount: 传入[]时...

1) 筛选面板只有展开时才执行useEffect订阅快捷键，收起时跳过；测试发现后面的useState偶尔读到别的状态，为什么不能把Hook放进if？

答：Hook依赖每次渲染中稳定的调用顺序，条件分支会使内部链表的取值位置发生错位。应始终调用Hook，把条件放到副作用内部，或把展开区域拆成独立组件；循环、嵌套函数中的条件调用同样存在风险。

2) 一个报表页把订阅、标题更新和请求状态全塞进类组件的多个生命周期，产品又要求三个页面复用；你会怎样用自定义Hook重组？

答：把彼此关联的状态和副作用分别抽成useSubscription、useTitle等自定义Hook，让每段业务逻辑在一个位置维护并跨组件复用。自定义Hook内仍可调用useState、useEffect，但自身也必须遵守顶层调用规则；拆得过碎会增加依赖关系的理解成本。

3) 聊天室的useEffect原来依赖[roomId]，有人为减少请求改成[]；用户在同一页面切换房间后仍收到旧房间消息，你会如何修正执行与清理时机？

答：订阅确实随roomId变化，因此依赖项应保留[roomId]。每次切换时，React会先执行上一次回调返回的退订函数，再建立新订阅，卸载时也会清理；改成[]只适用于整个挂载期间参数不会变化的副作用。

4) 线上弹窗打开时偶发绑定两次滚动监听，关闭后仍触发回调；相关代码用了没有依赖数组的useEffect，你会先查什么？

答：先检查副作用是否每次渲染都重新绑定，以及回调是否返回了与本次绑定对应的移除函数。没有依赖数组时，每次渲染会先清理旧副作用再执行新副作用；若清理缺失、函数引用不匹配或绑定逻辑不对称，就会残留监听。

5) 可视化页面需要读取更新后的DOM尺寸并立即调整样式，useEffect下会短暂闪动；同事提议所有副作用统一换成useLayoutEffect，你怎么取舍？

答：这类必须在DOM变更后同步测量并调整样式的逻辑适合useLayoutEffect，可减少绘制后的视觉闪动。普通订阅、日志和数据副作用仍优先使用useEffect；useLayoutEffect会同步阻塞后续绘制，放入耗时计算会拖慢页面呈现。

6) 编辑器把回调传给多个依赖组件，父组件每次渲染都创建新函数；评审同时提出useCallback、useMemo和useRef，三者在这里分别解决什么？

答：需要稳定回调引用时使用useCallback，需要缓存计算结果或传入属性时使用useMemo，访问真实节点或保存可变引用则使用useRef。它们用途不同，不能互相替代；缓存本身也增加依赖维护成本，应围绕明确的重复渲染或计算问题使用。

634. 受控组件和非受控组件

受控组件的值由React管理，非受控组件的值则由DOM自己保存。同时传入value和onChange时，组件的输入与更新都由外部控制；只传defaultValue，通常就是给非受控组件设置初始值。受控方式更方便统一管理和同步数据，非受控方式脱离了组件自身的state或props控制，一般不建议作为常规方案。// 上面的是受控组件 下面的是非受控组件 当你一个组件同时传递一个value以及onChange事件时，它就是一个受控组件，输入输出都是我控制的。第二个只是传递了默认的初时值，并没有传onChange事件，非受控组件是一种反模式，它的值不受组件自身的state或props控制

1) 登录页给却没有传onChange，测试说输入框怎么敲都不变；开发认为删掉value就算修好，你怎么看？

答：传入value意味着显示值由外部控制，没有对应的onChange更新来源时，用户输入无法形成新的受控值。应补上变更回调并更新状态；若改用defaultValue，则只是交给组件内部维护，父级也就失去持续控制能力。

2) 后台编辑页有姓名、部门和角色三个字段，保存按钮必须随任一字段变化更新可用状态；你会让输入组件接收哪些属性？

答：这类实时联动适合受控方式：父级持有字段值，向每个输入传入value和onChange，变更后统一计算保存条件。数据入口和出口都由父级掌握，校验与提交更直接；代价是父级需要维护完整的状态更新链路。

3) 搜索框原来只需要一个初始关键词，改版后路由参数变化时必须立即覆盖输入内容；继续使用defaultValue={keyword}能满足吗？

答：defaultValue只表达初始值，后续路由参数变化不应依赖它持续驱动输入框。需要外部参数随时覆盖显示内容时，应改为value={keyword}并提供onChange更新来源；切换模式时还要避免同时混

用两套值来源。

4) 线上表单从接口数据未返回时的undefined，切换成返回后的字符串，控制台提示输入组件控制方式发生变化；你会怎样定位？

答：先检查首屏是否没有传value，数据返回后才开始传入，导致组件从内部维护切到外部控制。若决定受控，应从首次渲染就提供稳定值，例如空字符串，并始终配套onChange；若决定非受控，则只传defaultValue，不要中途切换。

5) 内容发布页只在点击提交时读取标题，产品没有实时校验、联动或外部重置要求；受控和非受控怎么选？

答：仅需初始值且提交时再读取的场景，可以采用defaultValue，让输入值不受父级state或props持续控制。若随后加入实时校验、跨字段联动或外部覆盖，就更适合受控方式；选择取决于谁必须成为稳定的数据源。

635. 如何避免ajax数据请求重新获取

避免重复请求的核心是缓存数据并对相同请求去重，通常优先交给React Query或SWR处理。它们能按请求键复用缓存、合并并发请求，并在数据过期后重新验证，比把服务端数据塞进Redux或Zustand更省维护成本。组件里还要正确设置useEffect依赖，并用AbortController取消过期请求；不常变化的接口也可以配合Cache-Control或ETag。

核心思路是缓存

+

请求去重——把已经拿到的数据缓存起来，相同的请求不重复发。

用数据请求库管理缓存（现代主线）：React

Query

/

SWR内置了缓存、请求去重（同一时刻多个组件请求同一 key 只发一次）、后台重新验证（stale-while-revalidate）、失效控制。这是现在处理「服务端状态」的标准做法——避免重复请求本就是它们的核心能力，不用自己造轮子。放全局状态（旧做法）：把数据存进 Redux/Zustand，组件先读缓存、没有再请求。这是过去常见的做法，但要自己处理缓存失效、去重、更新，样板多——现在更推荐用 React Query管理服务端状态、全局 store 只放真正的客户端状态。请求去重（in-flight dedup）：对「正在进行中」的相同请求做去重——用一个 Map 缓存进行中的 Promise，相同 key 直接返回同一个 Promise，避免并发的重复请求。HTTP 缓存：对可缓存的接口配Cache-Control/ETag，让浏览器层面命中缓存（适合不常变的数据）。组件层面：用useEffect的依赖数组控制只在必要时请求、用AbortController取消过期请求避免竞态。一句话：优先用 React Query/SWR（自带缓存 + 去重 + 重验证），它们专门解决「服务端状态」的重复获取；全局 store 只放客户端状态；再配合请求去重和HTTP 缓存。

1) 商品详情页三个区域同时请求/product?id=42，抓包看到首屏并发三次；只把最后一次结果存进缓存，为什么仍挡不住重复请求？

答：结果缓存只有请求完成后才能命中，三个调用在完成前仍会各自发出请求。请求层应按包含参数的key把进行中的Promise放进Map，后续调用复用它；结束后还要按缓存策略清理或转存结果，避免把失败请求永久保留。

2) 列表页返回时产品要求先展示旧数据、再静默校准库存，团队准备自己维护过期时间、重试和并发合并；你会怎么落地？

答：优先使用React Query或SWR，利用缓存先呈现已有数据，再通过后台重新验证更新库存。它们也覆盖请求去重与失效控制，能减少自建状态机；团队仍需定义合理的key和数据失效边界，否则不同筛选条件可能误用同一缓存。

3) 新闻接口从“几乎不变”改成“运营每分钟修订”，后端却设置了长时间Cache-Control；前端继续依赖浏览器缓存会有什么风险，怎么调整？

答：浏览器缓存适合不常变化的数据，更新频率提高后，过长的缓存时间会让用户持续看到旧内容。应由服务端调整Cache-Control，或使用ETag让浏览器重新验证；前端请求库的失效策略也要同步，否则两层缓存会产生不同的新鲜度判断。

4) 用户快速切换status筛选，慢的旧请求最后返回并覆盖了新结果；页面还偶发重复请求，你会按什么顺序排查？

答：先核对useEffect依赖是否准确反映status，避免无关渲染重复触发或漏掉必要请求。再用AbortController取消已经过期的请求，防止旧响应覆盖新状态；请求去重只能合并相同key，不能替代不同筛选条件间的竞态处理。

5) 架构评审争论把所有接口结果放进Redux，还是引入React

Query；订单数据需要缓存、失效、重试，主题和侧栏状态只在客户端存在，你怎么划分？

答：订单数据属于服务端状态，优先交给React Query或SWR管理缓存、去重、重验证与失效。主题和侧栏这类客户端状态再放入全局store；把接口数据统一塞进Redux也能工作，但团队必须自行补齐更新和缓存生命周期。

6) 接口已经支持ETag，前端又用了SWR，同事认为两者重复只能留一个；在一次返回列表页的流程里，它们各自拦截什么？

答：SWR在应用层按key复用数据和进行中的请求，并可先展示旧数据后重新验证；ETag则让重新访问服务器时验证资源是否变化，可能避免重复传输完整响应。两层可以协作，但缓存时效要协调，否则应用层不发请求时不会触发HTTP验证。

636. 组件之间通信

组件通信通常按关系远近选择：父子之间用props和回调，跨多层用Context，复杂公共状态再考虑状态管理库。父组件可以把状态传给子组件，子组件则通过回调把输入或事件交回父组件，兄弟组件一般把状态提升到共同父级。主题、语言这类跨层信息适合Context；组件树两侧共享的复杂数据可用Redux等方案，但会增加学习和结构成本。需要父组件直接调用子组件方法时，也可以通过ref获取实例，不过这是比较特殊的通信方式。

父子组件通信自定义事件redux和context context如何运用

父组件向其下所有子孙组件传递信息如一些简单的信息：主题、语言复杂的公共信息用redux

在跨层级通信中，主要分为一层或多层的情况 如果只有一层，那么按照 React 的树形结构进行分类的话，主要有以下三种情况：父组件向子组件通信，子组件向父组件通信以及平级的兄弟组件间互相通信。在父与子的情况下，因为 React 的设计实际上就是传递Props即可。那么场景体现在容器组件与展示组件之间，通过Props传递state，让展示组件受控。在子与父的情况下，有两种方式，分别是回调函数与实例函数。

回调函数，比如输入框向父级组件返回输入内容，按钮向父级组件传递点击事件等。实例函数的情况有些特别，主要是在父组件中通过

React 的 ref API 获取子组件的实例，然后通过实例调用子组件的实例函数。这种方式在过去常见于 Modal 框的显示与隐藏

多层级间的数据通信，有两种情况。第一种是一个容器中包含了多层子组件，需要最底部的子组件与顶部组件进行通信。在这种情况下，如果不断透传

Props 或回调函数，不仅代码层级太深，后续也很不好维护。第二种是两个组件不相关，在整个

React 的组件树的两侧，完全不相交。那么基于多层级间的通信一般有三个方案。第一个是使用

React 的Context API，最常见的用途是做语言包国际化第二个是使用全局变量与事件。第三个是使用状态管理框架，比如

Flux、Redux 及 Mobx。优点是由于引入了状态管理，使得项目的开发模式与代码结构得以约束，缺点是学习成本相对较高

第一个是使用 React 的Context API，最常见的用途是做语言包国际化第二个是使用全局变量与事件。第三个是使用状态管理框架，比如

Mobx。优点是引入了状态管理，使得项目的开发模式与代码结构得以约束，缺点是学习成本相对较高

1) 商品卡片的收藏按钮点击后要通知父列表更新计数，开发准备用ref调子组件实例再反向取值；这条只有一层的通信为什么不合适？

答：一层子到父通信应由父组件传入回调，子组件在点击时把结果作为参数回传，数据流更清晰。通过ref获取实例并调用方法属于更特殊的命令式场景，过去常见于弹窗显隐；用它传普通业务数据会增加父子实现耦合。

2) 结算页的优惠金额由父组件计算，价格展示组件只负责渲染，数量组件还要把修改结果传回去；你会怎样设计属性边界？

答：父组件通过props把优惠金额和数量传给展示组件，再把数量变更回调传给数量组件。子组件触发回调后由父级更新状态并重新下发，形成单向数据流；不要让两个子组件私下同步副本，否则容易出现价格与数量不一致。

3) 主题值要从应用根节点传到第六层按钮，中间五层都不用它；设计师又要求切换主题后整棵页面同步，你会选Context还是逐层props？

答：这种面向一批子孙组件的简单公共信息适合Context，根节点提供主题，深层按钮直接消费，避免无意义的逐层透传。若信息演变为复杂且频繁变化的业务状态，则应重新评估状态管理框架；Context不等于所有全局状态的默认容器。

4) 两个位于组件树两侧的面板通过全局事件同步筛选条件，线上偶尔出现页面已卸载但监听仍执行；你会查哪里并考虑怎样替换？

答：先检查事件监听是否在挂载时注册、卸载时解除，以及同一组件是否重复注册。对于跨层级且互不相交的复杂共享状态，可改用Redux、Mobx等状态管理框架，让更新路径受到统一约束；全局事件虽直接，但生命周期和数据来源更难追踪。

5) 小型后台只有语言和主题两个公共值，架构师要求一开始就上Redux，另一位坚持所有层级都传props；你如何裁决？

答：语言和主题属于Context的典型用途，可由上层统一提供并让后代消费，不必为了简单信息引入完整状态管理框架。逐层props在层级很浅时仍最直观；只有公共状态复杂度和协作约束明显上升时，Redux等方案的结构化收益才更突出。

6) 父组件需要控制第三方弹窗的打开和关闭，但该弹窗只暴露实例方法；团队规定所有子到父通信都必须用回调，这里是否需要例外？

答：若第三方弹窗确实只提供命令式实例方法，父组件可通过React的ref获取实例并调用显示或隐藏方法。它适用于这类特殊控制，不应扩展成常规数据通信机制；能通过props和回调表达的状态，仍应优先保持声明式传递。

637. 类组件与函数组件有什么区别呢？

类组件和函数组件在展示及现代浏览器中的性能没有明显差异，真正不同的是开发时的心智模型。类组件围绕面向对象、this和生命周期组织代码，函数组件则更强调不可变数据、减少副作用，并通过Hooks组织和复用逻辑。性能优化时，类组件常用shouldComponentUpdate，函数组件可用React.memo；随着Hooks出现，函数组件已经能覆盖原先依赖生命周期的场景。组件设计也更推崇组合而非继承，因此函数组件通常更符合后续的并发与细粒度逻辑复用方向。作为组件而言，类组件与函数组件在使用与呈现上没有任何不同，性能上在现代浏览器中也不会有明显差异它们在开发时的心智模型上却存在巨大的差异。类组件是基于面向对象编程的，它主打的是继承、生命周期等核心概念；而函数组件内核是函数式编程，主打的是immutable、没有副作用、引用透明等特点。之前，在使用场景上，如果存在需要使用生命周期的组

件，那么主推类组件；设计模式上，如果需要使用继承，那么主推类组件。但现在由于 React Hooks 的推出，生命周期概念的淡出，函数组件可以完全取代类组件。其次继承并不是组件最佳的设计模式，官方更推崇“组合优于继承”的设计概念，所以类组件在这方面的优势也在淡出。性能优化上，类组件主要依靠 `shouldComponentUpdate` 阻断渲染来提升性能，而函数组件依靠 `React.memo` 缓存渲染结果来提升性能。从上手程度而言，类组件更容易上手，从未来趋势上看，由于 React Hooks 的推出，函数组件成了社区未来主推的方案。类组件在未来时间切片与并发模式中，由于生命周期带来的复杂度，并不易于优化。而函数组件本身轻量简单，且在 Hooks 的基础上提供了比原先更细粒度的逻辑组织与复用，更能适应 React 的未来发展。

1) 商品详情页把类组件改成函数组件后，产品认为页面表现和性能都会明显提升，你会怎样纠正这个判断？

答：单纯改写组件形态通常不会带来可感知的展示或性能提升，两者作为组件时的使用结果并无本质差异。迁移价值主要来自 Hooks 带来的细粒度逻辑组织与复用，以及更适合后续并发能力的模型；若现有代码稳定且没有复用诉求，不应把重写包装成性能优化。

2) 一个类组件同时处理请求、订阅和表单校验，多个生命周期里散落着同一业务逻辑，准备迭代时你会怎样落地改造？

答：适合在本次业务迭代中改成函数组件，并按业务关注点拆出自定义 Hook，让请求、订阅和校验各自聚合相关状态与清理逻辑。不要机械地把每个生命周期翻译成一个 `useEffect`，否则只是换了语法；依赖项和副作用边界不清时，迁移后仍会难以维护。

3) 后台管理系统大量使用基类继承权限校验，新模块坚持继续继承，另一位开发主张全面改成组合，你会支持哪一边？

答：新模块应优先使用组合，把权限能力做成包装组件、Context 或自定义 Hook，而不是继续扩大组件继承层级。React 的组件复用更适合“组合优于继承”，函数组件也因此削弱了类组件的传统优势；旧基类可渐进兼容，但一次性替换会扩大回归范围。

4) 函数组件接入实时行情后偶发重复订阅，类组件版本却正常；日志显示每次渲染后订阅数量都增加，你先查哪里？

答：先检查副作用是否写在渲染过程，或 `useEffect` 是否缺少正确依赖与清理函数，确保每次重新订阅前都会解除旧订阅。函数组件强调渲染无副作用，副作用应集中在对应的 Hook 中；若依赖对象每次都重新创建，还需稳定引用，否则清理正确也可能频繁重订阅。

5) 数据表格频繁刷新，类组件方案要写 `shouldComponentUpdate`，函数组件方案准备套 `React.memo`，你会如何决定并验证？

答：两种方案都能阻断不必要的渲染：类组件使用 `shouldComponentUpdate`，函数组件通常使用 `React.memo` 缓存渲染结果。应先确认瓶颈来自重复渲染，再保证传入的对象和回调引用足够稳定；盲目增加比较逻辑也有成本，且无法修复组件内部昂贵计算。

638. 如何设计 React 组件

设计 React 组件的核心是单一职责、接口精简，并根据业务边界合理分层。我一般把纯展示和通用能力放进基础组件，把状态与业务逻辑留给容器组件或页面，避免为了复用而过度抽象。组件之间优先通过 `props`、`children` 等方式组合，接口命名要有语义，也要考虑受控与非受控场景。基础组件还应补齐 TypeScript 类型、无障碍能力、单测和 Storybook 文档。React 组件应从设计与工程实践两个方向进行探讨从设计上而言，社区主流分类的方案是展示组件与灵巧组件 展示组件内部没有状态管理，仅仅用于最简单的展示表达。展示组件中最基础的一类组件称作代理组件。代理组件常用于封装常用属性、减少重复代码。很经典的场景就是引入 `Antd` 的 `Button` 时，你再自己封一层。如果未来需要替换掉 `Antd`

或者需要在所有的 Button 上添加一个属性，都会非常方便。基于代理组件的思想还可以继续分类，分为样式组件与布局组件两种，分别是将样式与布局内聚在自己组件内部。从工程实践而言，通过文件夹划分的方式切分代码。我初步常用的分割方式是将页面单独建立一个目录，将复用性略高的 components 建立一个目录，在下面分别建立 basic、container 和 hoc 三类。这样可以保证无法复用的业务逻辑代码尽量留在 Page 中，而可以抽象复用的部分放入 components 中。其中 basic 文件夹放展示组件，由于展示组件本身与业务关联性较低，所以可以使用 Storybook 进行组件的开发管理，提升项目的工程化管理能力

1) 订单页把请求、权限判断和几十行表格结构都写进一个所谓“展示组件”，评审时你会指出什么设计矛盾？

答：它已经不是纯展示组件，因为数据获取和权限判断属于业务协调或状态管理，应该由页面或容器层承担。展示组件只接收明确数据并表达界面，才能低成本复用和独立验证；如果强行追求绝对无状态，也可能把紧密相关的交互拆得过散。

2) 六个页面都直接使用Antd

Button，现在设计要求统一补埋点并可能更换组件库，你会怎样调整目录和依赖？

答：应增加项目自己的按钮代理组件，集中封装常用属性、样式和埋点，再让业务页面依赖这层稳定接口。通用展示组件可放入components/basic并用Storybook管理；代理层若暴露过多底层专有属性，未来替换组件库仍会产生大面积修改。

3) 团队规定所有组件必须放进公共components，结果两百多个只服务单一页面的组件也被共享，你会如何重新划分？

答：无法复用且高度依赖业务流程的代码应留在对应Page目录，只有抽象稳定、复用性较高的内容才进入公共组件目录。可进一步按basic、container和hoc等职责分类，使依赖方向清楚；过早公共化会固化尚未稳定的接口，并增加跨页面修改风险。

4) 改动会员卡样式后十几个页面错位，排查发现每页都复制了相同布局和间距，你会怎样止损并定位抽象边界？

答：先把重复的样式与布局收敛为样式组件或布局组件，再逐页替换，避免继续复制并分别修补。组件内部应内聚稳定的视觉规则，页面只提供业务内容；若不同页面的结构差异已经超过共同点，硬合并会制造大量条件属性，应保留少量明确变体。

5) 搜索页组件已经拆成三十个文件，阅读一次交互要跨越多层props，但架构师仍要求继续细拆，你会用什么标准取舍？

答：拆分应围绕职责、复用价值和封装边界，而不是文件数量或代码行数。稳定的展示单元可进入basic，负责数据与状态协调的部分留在容器或页面层；粒度过细会让父子关系和数据传递更难理解，粒度过粗则会把业务逻辑与展示再次绑死。

639. 组件的协同及（不）可控组件

组件协同本质上是组织父子关系和复用公共逻辑，而可控与不可控的区别在于状态由谁管理。父组件可以通过嵌套和props统一管理子组件，也可以抽离多个组件共有的方法，减少重复代码，但抽象过多会让逻辑分散、维护难度上升。可控组件的值来自外部状态，符合React数据流，也更方便读取和处理。不可控组件通常用defaultValue初始化，后续需要借助ref获取实际值。为什么要进行组件的协同 我们在实际的开发项目的时候，不会只用几个组件，有时候遇到大型的项目，可能会有成千上百的组件，难免会遇到有功能重复的组件。要进行修改，就会修改大部分的文件。所以我们需要进行组件的协同开发。

什么是组件的协同使用？ 组件的协同本质上是对组件的一种组织、管理的方式。目的：逻辑清晰：这是组件与组件之间的逻辑代码模块化封装细节：像面向对象一样将常用的方法以及数据封装起来提高代码的复

用性：因为是组件，相当于一个封装好的东西，用的时候直接调用 逻辑清晰：这是组件与组件之间的逻辑
代码模块化封装细节：像面向对象一样将常用的方法以及数据封装起来提高代码的复用性：因为是组件，
相当于一个封装好的东西，用的时候直接调用 如何实现组件的协同使用 第一种：增加一个父组件，将其他
的组件进行嵌套，更多的是实现代码的封装第二种：通过一些操作从后台获取数据，React中的Mixin，更
多的是实现代码的复用 组件嵌套的含义 组件嵌套的本质是父子关系 组件嵌套的优缺点 优点：逻辑清晰：
父子关系类似于人类中的父子关系模块化开发：每个模块对应一个功能，不同的模块可以同步开发封装细
节：开发者必须要关注组件的功能，不需要了解细节缺点：编写难度高：父子组件的关系需要经过深思熟
虑，贸然编写可能导致关系混乱，代码难以维护无法掌握所有细节：使用者只知道组件的用法，不知道实
现细节，遇到问题难以修复 逻辑清晰：父子关系类似于人类中的父子关系模块化开发：每个模块对应一个
功能，不同的模块可以同步开发封装细节：开发者必须要关注组件的功能，不需要了解细节 编写难度高：
父子组件的关系需要经过深思熟虑，贸然编写可能导致关系混乱，代码难以维护无法掌握所有细节：使用
者只知道组件的用法，不知道实现细节，遇到问题难以修复 Mixin Mixin的含义
Mixin=一组方法。他的目的是横向抽离出组件的相似代码，把组件的共同作用以及效果的代码提出来
Mixin的优缺点 优点代码复用：抽离出通用的代码，减少开发成本，提高开发效率即插即用：可以使用许多
现有的Mixin来开发自己的代码适应性强：改动一次代码，影响多个组件缺点编写难度高：Mixin可能被用在
各种环境中，想要兼容多种环境就需要更多的 - 码与逻辑，通用的代价是提高复杂度降低代码的可读性：组
件的优势在于将逻辑与是界面直接结合在一起，Mixin本质上会分散逻辑，理解起来难度大 代码复用：抽离
出通用的代码，减少开发成本，提高开发效率即插即用：可以使用许多现有的Mixin来开发自己的代码适应
性强：改动一次代码，影响多个组件
编写难度高：Mixin可能被用在各种环境中，想要兼容多种环境就需要更多的 - 码与逻辑，通用的代价是提
高复杂度降低代码的可读性：组件的优势在于将逻辑与是界面直接结合在一起，Mixin本质上会分散逻辑，
理解起来难度大 不可控组件 上图：defaultValue的值是固定的，这就是一个不可控组件如果要获取input
的value值，只有使用ref获取节点来获取值 可控组件 defaultValue的值是根据状态确定了，只需要拿到thi
s.state.value的值就可以了这里需要注意一下：使用value的值是不可修改的，defaultValue的值是可以
修改的 可控组件的优点 符合React的数据流数据存储在state中，便于获取便于处理数据

1) 注册页的手机号输入框传了defaultValue，用户编辑后父组件更新默认值，界面却没有跟着变化，为什么？

答：defaultValue只负责非受控输入框的初始值，挂载后输入内容由DOM自己维护，父组件随后修改默认值不会持续驱动界面。若业务要求外部状态始终决定内容，应改用value配合onChange；这样也意味着父组件必须及时更新状态，否则输入框会表现为不可编辑。

2) 结算页要求地址输入变化后立即联动运费、校验和提交按钮状态，你会选受控还是非受控组件？

答：应优先选择受控组件，把输入值保存在React的state中，并通过value和变更回调形成单向数据流。这样联动逻辑能读取同一份确定数据，也便于统一校验；字段数量很大或变更非常频繁时，需要控制状态作用域，避免无关区域跟随渲染。

3) 一个文件上传控件需要读取原生节点状态，团队却要求所有表单项都必须用value控制，你会怎样处理这组选型冲突？

答：不必把所有输入都强行设计成受控模式，无法由普通value完整驱动或更适合读取DOM状态的控件可使用非受控方案，并通过ref获取节点值。组件应封装这些实现细节，对外暴露清晰事件；代价是数据不再始终位于React状态中，联动和即时校验会更困难。

4) 筛选面板上线后偶发出现“输入框显示值与提交参数不一致”，代码同时通过ref读DOM，又在state中保存一份值，你先怎么排查？

答：先确认界面的唯一数据源，检查渲染使用的是value、defaultValue还是DOM当前值，并对照提交路径读取了哪一份数据。受控和非受控机制混用容易产生两个不同步的状态源；应统一为state驱动，或明确只在提交时通过ref读取，避免双向同步。

5) 十几个组件都要复用请求和日志逻辑，旧方案准备继续追加Mixin，新方案主张父组件协调，你会如何取舍？

答：组件间的结构协作可由父组件嵌套和组织，使父子关系、数据来源与封装边界保持清晰。Mixin能横向复用一组方法，但会把逻辑分散到组件之外，兼容场景越多复杂度越高；遗留代码可维护，新增逻辑则应优先选择边界更显式的组合方式。

640. React-Router 的实现原理及工作方式分别是什么

React Router的实现原理是监听地址变化并匹配对应组件，工作时则通过上下文连接路由容器、匹配组件和导航组件。Hash模式依赖哈希变化，History模式使用pushState、replaceState等浏览器API，后者还要求服务端配置historyApiFallback。浏览器端使用History API，原生端可以使用基于数组维护记录的内存历史。像Router提供上下文，Route负责匹配，Link则完成平台相关的导航。React

Router路由的基础实现原理分为两种，如果是切换

Hash的方式，那么依靠浏览器Hash变化即可；如果是切换网址中的Path，就要用到HTML5 History API中的pushState、replaceState等。在使用这个方式时，还需要在服务端完成historyApiFallback配置在React Router内部主要依靠history库完成，这是由React Router自己封装的库，为了实现跨平台运行的特性，内部提供两套基础history，一套是直接使用浏览器的History API，用于支持react-router-dom；另一套是基于内存实现的版本，这是自己做的一个数组，用于支持react-router-native。React Router的工作方式可以分为设计模式与关键模块两个部分。从设计模式的角度出发，在架构上通过Monorepo进行库的管理。Monorepo具有团队间透明、迭代便利的优点。其次在整体的数据通信上使用了Context API完成上下文传递。在关键模块上，主要分为三类组件：第一类是Context容器，比如Router与MemoryRouter；第二类是消费者组件，用以匹配路由，主要有Route、Redirect、Switch等；第三类是与平台关联的功能组件，比如Link、NavLink、DeepLinking等。React router原理分析(opens new window)

1) 运营把商品详情从/product/42改成#/product/42后，认为两种地址只是写法不同，你会怎样解释底层差异？

答：Hash路由依赖地址中散列部分的变化，而普通Path路由依赖HTML5HistoryAPI的pushState、replaceState等能力。两者都能驱动客户端匹配组件，但服务器参与程度不同；选择Path后若没有回退配置，直接访问深层地址可能由服务器返回404。

2) 内容站准备从HashRouter切到正常路径，上线前除了替换路由容器，你会要求服务端完成什么配置？

答：服务端需要配置historyApiFallback或等价回退规则，让未知的前端路径仍返回应用入口，再由ReactRouter完成匹配。还要排除静态资源和真实接口路径，不能把所有请求无条件重写；否则深层链接虽能打开，资源请求或后端错误却可能被错误地返回HTML。

3) 同一套路由配置既要跑浏览器端管理后台，又要跑不具备浏览器HistoryAPI的测试环境，历史记录层怎么选？

答：浏览器环境使用基于HistoryAPI的实现，非浏览器场景可使用内存历史记录，以数组维护位置变化。ReactRouter通过不同的history基础实现支持跨平台，上层匹配逻辑可以保持接近；内存实现不会天然改变真实地址栏，不能把测试结果直接等同于浏览器导航行为。

4)

用户直接打开/orders/1001得到服务器404，但先访问首页再点Link却正常，你会如何定位？

答：现象说明客户端路由匹配大概率正常，故障位于服务器对深层路径的处理，应先检查入口回退配置和部署平台重写规则。Link在应用内通过HistoryAPI切换，因此不会重新请求对应文档；修复时需保留API、资源和真实文件的优先匹配，避免过度回退。

5) 微前端页面需要共享当前位置，团队争论是层层传location，还是让路由组件直接读取，你会怎么设计？

答：路由容器应通过Context API传递位置和导航上下文，负责匹配的消费者组件按需读取，避免每层手工转发。整体可区分Router、MemoryRouter等容器，路由匹配组件，以及Link、NavLink等平台功能组件；多个应用根并存时仍要明确各自上下文边界。

641. React 17 带来了哪些改变

React 17的关键变化是更新JSX转换、重构事件系统，并引入更细致的Lane优先级模型。新转换会从react/jsx-runtime自动引入解析能力，因此使用JSX时不再必须手动导入React，编译输出也有简化空间。事件委托从全局document下移到各自的根容器，让多个React应用之间的影响更可控，同时取消事件池，异步访问事件不再需要e.persist()。源码调度方面，Lane用二进制位表达优先级，比原来的expirationTime能覆盖更多边界情况。最重要的是以下三点：新的JSX转换逻辑事件系统重构Lane模型的引入

1. 重构 JSX 转换逻辑 在过去，如果我们在 React 项目中写入下面这样的代码：

```
function MyComponent() { return 这是我的组件 }
```

React 是会报错的，原因是 React 中对 JSX 代码的转换依赖的是React.createElement这个函数。因此但凡我们在代码中包含了

JSX，那么就必须在文件中引入 React，像下面这样：

```
import React from 'react'; function MyComponent() { return 这是我的组件 }
```

而React 17则允许我们在不引入 React 的情况下直接使用 JSX。这是因为在 React 17 中，编译器会自动帮我们引入 JSX 的解析器，也就是说像下面这样一段逻辑：

```
function MyComponent() { return 这是我的组件 }
```

会被编译器转换成这个样子：

```
import { jsx as _jsx } from 'react/jsx-runtime'; function MyComponent() { return _jsx('p', { children: '这是我的组件' }); }
```

react/jsx-runtime中的 JSX 解析器将取代React.createElement完成JSX的编译工作，这个过程对开发者而言是自动化、无感知的。因此，新的

JSX 转换逻辑带来的最显著的改变就是降低了开发者的学习成本。react/jsx-runtime中的 JSX 解析器看上去似乎在调用姿势上和React.createElement区别不大，那么它是否只是React.createElement换了个马甲呢？

当然不是，它在内部实现了React.createElement无法做到的性能优化和简化。在一定情况下，它可能会略微改善编译输出内容的大小

2. 事件系统重构 事件系统在 React 17 中的重构要从以下两个方面来看：

- 2.1 卸掉历史包袱：放弃利用 document 来做事件的中心化管理 React 16.13.x 版本中的事件系统会通过将所有事件冒泡到 document 来实现对事件的中心化管理 这样的做法虽然看上去已经足够巧妙，但仍然有它不聪明的地方——document

是整个文档树的根节点，操作 document 带来的影响范围实在是太大了，这将会使事情变得更加不可控 在 React 17 中，React 团队终于正面解决了这个问题：事件的中心化管理不会再全部依赖document，管控相关的逻辑被转移到了每个 React 组件自己的容器 DOM 节点中。比如说我们在 ID 为 root 的 DOM 节点下挂载了一个 React 组件，像下面代码这样：

```
const rootElement = document.getElementById("root"); // 这里沿用 React 17 的写法说明事件挂载位置的变化；React 18 起写作 // ReactDOM.createRoot(rootElement).render(), 事件同样挂在这个容器节点上 ReactDOM.render(rootElement);
```

那么事件管控相关的逻辑就会被安装到root 节点上去。这样一来，React 组件就能够自己玩自己的，再也无法对全局的事件流构成威胁了

- 2.2 拥抱新的潮流：放弃事件池 在 React 17 之前，合成事件对象会被放进一个叫作“事件池”的地方统一管理。这样做的目的是能够实现事件对象

的复用，进而提高性能：每当事件处理函数执行完毕后，其对应的合成事件对象内部的所有属性都会被置空，意为为下一次被复用做准备。这也就意味着事件逻辑一旦执行完毕，我们就拿不到事件对象了，React官方给出的这个例子就很可能说明问题，请看下面这个代码 `function handleChange(e){ // This won't work because the event object gets reused. setTimeout(()=>{ console.log(e.target.value); }, ...`

1) 一个只写的文件删掉 `import React from`

'react'后仍能编译，同事认为浏览器已经原生支持JSX，你会怎样纠正？

答：不是浏览器直接执行JSX，而是新的JSX转换由编译器自动引入 `react/jsx-runtime`，再生成相应的函数调用。它因此不再要求每个JSX文件显式导入 `React`，并可能简化编译输出；若构建工具尚未启用新转换，删除导入仍可能导致编译失败。

2) 大型页面里同时挂载两个独立React根节点，升级到React17

后事件隔离为什么比旧版本更容易控制？

答：React17 将事件的中心化管理从全局 `document` 移到各自的根容器节点，因此不同React根对全局事件流的影响更小。工程上仍要检查原生事件监听、冒泡边界和第三方脚本；容器级委托降低了相互干扰，并不代表所有跨根事件冲突都会自动消失。

3) 表单代码在事件回调结束后通过 `setTimeout` 读取 `e.target.value`，React16

需要 `e.persist()`，升级到React17 后要怎么处理？

答：React17 放弃了合成事件池，异步回调可以继续访问事件对象，因此不再需要为此调用 `e.persist()`。迁移时可清理依赖事件复用机制的兼容代码，但仍应注意目标DOM是否已被移除；取消事件池只保证对象不被统一置空，不保证相关节点永远存在。

4) 升级React17 后点击事件完全不触发，控制台也没有业务异常；页面由脚本动态替换过 `root` 容器，你先查什么？

答：先确认React实际挂载的根容器是否仍在文档中，以及动态替换节点后是否重新完成挂载，因为事件委托现在安装在该容器上。再检查容器附近的原生监听器是否阻止传播；继续只排查 `document` 上的React事件监听会偏离方向，但全局第三方监听仍可能造成干扰。

5) 调度模块准备继续用单一过期时间表达所有任务优先级，源码维护者建议切换到Lane模型，你如何说明取舍？

答：React17 引入的Lane模型使用二进制位表示不同优先级，并通过位运算组合和处理任务，比单一 `expirationTime` 能表达更细腻的优先级关系。它为更多调度边界提供了空间，但属于内部复杂度的增加；业务代码不应依赖具体位值，也不能把升级直接等同于页面必然变快。

【性能】

642. DNS 预解析

DNS预解析就是提前把域名解析成对应的IP，减少真正请求资源时等待解析的时间。浏览器访问外部域名之前通常需要先完成解析，因此预先处理可以让后续请求更早发出。比如页面确定会访问 `blog.poetries.top`，可以通过提前解析。DNS解析也是需要时间的，可以通过预解析的方式来预先获得域名所对应的IP

1) 商品页接入地图、客服、埋点和两个CDN，产品要求给全部第三方域名都加 `dns-prefetch`，你会直接照做吗？

答：不会直接给所有域名都加，而是只保留当前页面较可能请求的域名。dns-prefetch只是提前完成域名到IP的解析，若用户根本不会触发客服或地图，请求前的准备工作就可能白做；候选域名过多时，还应通过实际请求链路筛选。

2) 首屏图片来自img.example.com，你准备怎样在页面里落地DNS预解析，并确认写法指向正确域名？

答：可在文档头部加入，让浏览器提前解析该图片域名。配置必须写资源实际访问的主机名，不能只写主站域名；若图片后来改到其他域名，这条提示不会替新域名完成解析。

3) 同一套站点内网环境直接访问固定IP，公网环境则通过多个域名加载资源，平台团队想统一保留dns-prefetch，是否都有收益？

答：固定IP的请求不存在域名到IP的解析过程，因此dns-prefetch对它没有对应收益；公网域名请求才可能减少后续等待DNS的时间。统一模板可以保留条件配置，但不能把该提示当成所有网络环境都有效的加速手段。

4) 活动页已经加了dns-prefetch，线上瀑布图里接口请求仍长时间等待，开发因此认定预解析失效，你会先查什么？

答：先确认中的主机名是否与接口最终请求域名一致，并区分耗时究竟发生在DNS、连接还是服务端响应阶段。dns-prefetch只预先获得域名对应的IP，无法消除连接建立、重定向或接口处理耗时；若DNS本身不慢，整体变化可能并不明显。

5) 支付页只有一个首屏接口域名，但架构师希望用dns-prefetch解决整条请求链路延迟，你如何界定这个方案？

答：它只能针对请求前的DNS解析环节，适合域名确定且稍后会访问的场景，不能替代接口、网络连接或服务端优化。应先按请求阶段定位瓶颈，再决定是否加入预解析；若主要耗时不在DNS，继续堆叠该标签只会掩盖真正原因。

643. 缓存

浏览器缓存主要分为强缓存和协商缓存，区别在于资源复用前是否还要向服务器发请求。强缓存命中时直接使用本地资源，常用Cache-Control，其优先级高于Expires；协商缓存则携带ETag或修改时间验证，未变化时返回304。频繁变化的资源可用no-cache配合ETag，代码文件通常设置长期缓存并通过文件指纹更新，不应缓存的资源则用no-store。缓存对于前端性能优化来说是个很重要的点，良好的缓存策略可以降低资源的重复加载提高网页的整体加载速度通常浏览器缓存策略分为两种：强缓存和协商缓存 强缓存 实现强缓存可以通过两种响应头实现：Expires和Cache-Control。强缓存表示在缓存期间不需要请求，state code为200 Expires: Wed, 22 Oct 2018 08:41:00 GMT Expires是HTTP / 1.0的产物，表示资源会在Wed, 22 Oct 2018 08:41:00 GMT后过期，需要再次请求。并且Expires受限于本地时间，如果修改了本地时间，可能会造成缓存失效 Cache-control: max-age=30 Cache-Control出现于HTTP / 1.1，优先级高于Expires。该属性表示资源会在30秒后过期，需要再次请求 协商缓存 如果缓存过期了，我们就可以使用协商缓存来解决问题。协商缓存需要请求，如果缓存有效会返回304协商缓存需要客户端和服务端共同实现，和强缓存一样，也有两种实现方式 Last-Modified和If-Modified-Since Last-Modified表示本地文件最后修改日期，If-Modified-Since会将Last-Modified的值发送给服务器，询问服务器在该日期后资源是否有更新，有更新的话就会将新的资源发送回来但是如果本地打开缓存文件，就会造成Last-Modified被修改，所以在HTTP / 1.1出现了ETag ETag和If-None-Match ETag类似于文件指纹，If-None-Match会将当前ETag发送给服务器，询问该资源 ETag 是否变动，有变动的话就将新的资源发送回来。并且ETag优先级比Last-Modified高选择合适的缓存策略 对于大部分的场景都可以使用强缓存配合协商缓存解决，但是在一些特殊的地方可能

需要选择特殊的缓存策略

对于某些不需要缓存的资源，可以使用Cache-control: no-store，表示该资源不需要缓存对于频繁变动的资源，可以使用Cache-Control: no-cache并配合ETag使用，表示该资源已被缓存，但是每次都会发送请求询问资源是否更新。对于代码文件来说，通常使用Cache-Control: max-age=31536000并配合策略缓存使用，然后对文件进行指纹处理，一旦文件名变动就会立刻下载新的文件

1) 运营后台的配置接口每天变更多次，后端却给它设置一年max-age，前端说再配ETag就能保证实时，你认可吗？

答：不认可，因为强缓存有效期内浏览器通常不会发起请求，也就没有机会用ETag校验。频繁变动且仍允许保存副本的资源更适合Cache-Control: no-cache配合ETag；若内容根本不应落入缓存，则应使用no-store，代价是每次都要完整请求。

2) 构建后的脚本文件名带内容指纹，首页文档仍可能更新，你会怎样分别配置两类资源的缓存？

答：带指纹的代码文件可设置Cache-Control: max-age=31536000，内容变化后通过新文件名触发重新下载。入口文档若频繁变化，可采用no-cache并配合ETag做协商校验；若入口也长期强缓存，用户可能持续拿到引用旧资源的文档。

3) 公司代理仍会经过只理解部分旧缓存规则的链路，服务端同时返回Expires和Cache-Control，浏览器应以谁为准？

答：在支持HTTP/1.1缓存语义的客户端中，Cache-Control的优先级高于Expires，应以其指令和max-age为主要依据。Expires依赖绝对时间且受本地时间影响，可作为兼容信息保留；两者冲突会增加排查成本，配置时应保持意图一致。

4) 用户反馈刚发布的app.js仍是旧内容，网络面板显示200且请求没有发到服务器，你会怎样定位？

答：这更像资源命中了未过期的强缓存，应先检查响应中的Cache-Control、Expires、实际URL和文件名是否变化。强缓存期间不会发起校验请求，状态表现仍可能是200；若固定URL配了长max-age，应改用内容指纹，而不是期待ETag立即纠正。

5) 图片接口返回内容很大但一周才变化一次，团队在Last-Modified与ETag之间争论，你会如何选择？

答：两者都能用于协商缓存，但ETag更像资源指纹，且校验优先级高于Last-Modified，更适合需要准确判断内容是否变化的资源。Last-Modified依赖修改时间，本地打开缓存文件还可能改变该时间；采用ETag会增加服务端生成和维护标识的工作。

6) 安全团队要求登录后的敏感响应不能保存，性能团队却想用no-cache减少传输，你如何裁决？

答：若要求是不允许存储响应，应使用Cache-Control: no-store，因为no-cache仍允许缓存，只是每次使用前都要向服务器验证。性能团队可把协商缓存留给允许保存且频繁变化的普通资源；敏感数据场景需要接受重复传输带来的成本。

644. 使用 HTTP / 2.0

HTTP/2的核心改进是多路复用，让多个请求可以共用同一个TCP连接。在HTTP/1.1下，请求受浏览器并发限制，连接建立和断开会消耗多个RTT，大文件还会受到TCP慢启动影响。HTTP/2通过复用连接减少这类等待，同时使用头部压缩降低请求数据量，因此更适合资源请求较多的页面。

因为浏览器会有并发请求限制，在HTTP / 1.1时代，每个请求都需要建立和断开，消耗了好几个RTT时间，并且由于TCP慢启动的原因，加载体积大的文件会需要更多的时间在HTTP / 2.0中引入了多路复用，能够让多

个请求使用同一个TCP链接，极大的加快了网页的加载速度。并且还支持Header压缩，进一步的减少了请求的数据大小

1) 新闻首页把二十个接口迁到HTTP/2后，开发认为每个请求从此都不再有网络成本，你会怎样纠正？

答：HTTP/2的收益在于多个请求可复用同一条TCP连接，并通过头部压缩减少传输数据，并不意味着请求本身没有成本。接口响应体、服务端处理和网络往返仍然存在；若瓶颈不在连接并发或请求头，迁移后的体感提升可能有限。

2) 一个首屏要并发加载十个脚本和六张图片，你会用什么证据确认线上确实按HTTP/2复用了连接？

答：应在浏览器网络面板或抓包结果中核对协议，并观察同一主机的多个请求是否共享连接，而不是只看服务器配置文件。只有客户端与服务端实际使用HTTP/2，多路复用和头部压缩才会生效；若链路中间环节回落，页面仍会表现为旧协议路径。

3) 站点只有一个体积较大的下载文件，没有并发小请求，负责人仍把升级HTTP/2当作唯一优化项，你会同意吗？

答：不会把它视为唯一方案，因为多路复用主要改善多个请求共享连接的场景，单个大文件仍受传输速率和TCP慢启动等因素影响。升级协议可能保留其他收益，但还应检查文件体积与传输链路；否则协议变化不一定解决主要耗时。

4) 网关宣称已启用HTTP/2，但页面仍出现大量连接建立时间和并发等待，你会按什么顺序排查？

答：先核对浏览器实际协商出的协议，再检查资源是否分散在不同主机，因为连接复用依赖实际使用的连接范围。随后区分等待来自旧协议并发限制、连接建立还是服务端响应；仅网关支持不代表每个资源请求都已通过同一条HTTP/2连接。

5) 老项目为绕过HTTP/1.1并发限制，把静态资源拆到多个域名；迁移HTTP/2后是否应立即合并？

答：多域名会形成不同连接，无法让所有资源共享同一条TCP连接，因此应重新评估拆分是否仍有价值。是否合并还要结合部署、缓存和故障隔离约束验证，不能只改域名；若资源仍跨主机，多路复用的覆盖范围自然会受限。

645. 预加载

预加载适合那些暂时不用、但希望浏览器尽早获取的重要资源。它本质上是一种声明式的fetch，通过强制浏览器提前发起请求，而且不会阻塞onload事件。合理使用可以降低首屏加载时间，例如提前获取后续会用到的重要文件；不过需要留意浏览器兼容性，不能把它当成所有环境都可靠的默认方案。在开发中，可能会遇到这样的情况。有些资源不需要马上用到，但是希望尽早获取，这时候就可以使用预加载预加载其实是声明式的fetch，强制浏览器请求资源，并且不会阻塞onload事件，可以使用以下代码开启预加载 预加载可以一定程度上降低首屏的加载时间，因为可以将一些不影响首屏但重要的文件延后加载，唯一缺点就是兼容性不好

1) 详情页有一个用户可能点击后才播放的视频，产品要求首屏用preload强制下载，理由是它不会阻塞onload，你接受吗？

答：不能仅凭不阻塞onload就预加载，因为preload会强制浏览器请求资源，用户不播放时可能产生无效下载。它更适合当前页面确定需要且希望尽早取得的资源；对低概率交互内容，应结合使用概率和资源成本再决定。

2) 首屏确定要执行的脚本发现得很晚，你准备如何用HTML声明预加载，并避免链接指向错误资源？

答：可在文档中加入，让浏览器更早发起该资源请求。落地时必须保证href与页面最终使用的资源一致，并在目标浏览器中验证兼容性；地址不一致会形成额外请求，兼容性不足时也不能依赖它保证效果。

3) 内嵌WebView的浏览器版本分布复杂，业务负责人仍要求把首屏达标完全押在preload上，你如何评估？

答：不应把结果完全押在preload上，因为其兼容性存在限制，部分运行环境可能无法获得预期收益。应先验证目标WebView是否支持并正确执行，再保留正常资源发现与加载路径；预加载只能作为提前请求的优化，不能成为资源可用性的唯一前提。

4)

上线后网络面板出现两次相同脚本请求，页面还加了对应preload，你会先核对哪些地方？

答：先核对预加载的href与脚本实际URL是否完全一致，包括协议、主机、路径和参数，再确认页面最终确实使用该资源。preload是一次声明式请求，若声明目标与实际目标不同，就可能产生两份下载；还应检查是否有重复标签或重定向。

5) 团队把下一页可能使用的模块也全部标成preload，架构师建议改成prefetch，你站哪一边？

答：下一页未来可能使用的资源更符合prefetch，而preload面向当前页面所需的脚本、样式等资源。把大量不确定资源设为preload会强制提前请求，可能浪费传输资源；若下一页并非高概率访问，连prefetch也应谨慎使用。

6) 页面把非首屏但确定重要的文件延后加载，开发担心这会拖慢首次呈现，preload与async、defer分别解决什么？

答：preload解决资源尽早获取的问题，本身不会阻塞onload；async和defer描述脚本异步读取后的执行方式。defer保持脚本顺序依赖，async则加载完成后执行；不能用执行属性替代资源优先发现，也不能用预加载替代脚本执行控制。

646. 预渲染

预渲染就是让浏览器提前在后台渲染用户接下来大概率会访问的页面，从而缩短真正打开时的等待时间。可以通过指定目标页面，本质上是用当前的网络和计算资源换取后续访问速度。这个优化只适合用户几乎确定会打开的页面，否则提前下载和渲染的内容没有被使用，反而会白白消耗资源。

可以通过预渲染将下载的文件预先在后台渲染，可以使用以下代码开启预渲染。预渲染虽然可以提高页面的加载速度，但是要确保该页面百分百会被用户在之后打开，否则就白白浪费资源去渲染。总结 defer和async在网络读取的过程中都是异步解析defer是有顺序依赖的，async只要脚本加载完后就会执行preload可以对当前页面所需的脚本、样式等资源进行预加载prefetch加载的资源一般不是用于当前页面的，是未来很可能用到的这样一些资源

1) 电商首页列出一百个商品链接，运营想给每个详情页都加，说这样点击一定更快，你会批准吗？

答：不会批准批量预渲染，因为它会在后台提前渲染目标页面，用户未访问的页面会白白消耗资源。预渲染只适合之后几乎确定会打开的页面；面对一百个候选目标，应缩小到确定性极高的路径，否则加速收益建立在大量浪费之上。

2) 支付成功页之后固定进入订单详情页，你会怎样写预渲染声明，并说明它何时值得保留？

答：可在当前文档中加入，让目标页面预先在后台渲染。只有用户后续几乎必然进入该地址时才值得保留；若流程允许关闭页面、跳转其他入口或目标URL不确定，就应重新评估。

3) 同一流程从“付款后必进订单页”改成“付款后可能去抽奖、订单页或首页”，原有prerender还应保留吗？

答：原有预渲染的成立条件已经变弱，不应继续按必达路径处理。它的代价不仅是提前获取，还包括后台渲染目标页面；当三个去向都可能发生时，应根据真实导航确定性缩减或移除，避免为未打开页面消耗资源。

4) 上线预渲染后，低端设备在当前页出现额外资源消耗，用户却很少进入目标页，你会怎样确认根因？

答：先检查页面中的prerender声明及其目标地址，再对照用户实际后续导航，确认目标页是否真的概率打开。若大多数用户没有访问，后台渲染就是直接浪费；应移除或只在路径高度确定时声明，不能用当前页加载完成来证明配置合理。

5) 结算页的下一步几乎确定，但团队在prefetch和prerender之间争论，你如何选择？

答：若只想提前取得未来可能使用的资源，prefetch更贴近需求；若目标页面之后确定会打开，并希望连后台渲染也提前完成，才考虑prerender。后者工作更重，对导航确定性要求更高；一旦用户改走其他路径，已完成的渲染就会浪费。

6) 首屏脚本、下一页资源和下一页完整页面都想提前处理，评审会上有人建议统一使用preload，你会怎么拆分？

答：当前页面所需的脚本或样式使用preload，未来很可能用到的资源对应prefetch，几乎确定会访问的完整页面才适合prerender。三者提前处理的对象和成本不同，统一使用preload会混淆当前页资源与未来导航，也无法等同于后台渲染。

647. 懒执行与懒加载

懒执行是推迟计算逻辑，懒加载则是推迟非关键资源的下载，两者优化的对象并不一样。耗时逻辑如果首屏用不到，可以等定时器或用户事件触发后再执行；图片、视频等资源则可以等进入指定区域时再加载。图片场景里常先展示占位图，把真实地址放在自定义属性中，满足加载条件后再赋给src，但要确保触发逻辑可靠。懒执行 懒执行就是将某些逻辑延迟到使用时再计算。该技术可以用于首屏优化，对于某些耗时逻辑并不需要在首屏就使用的，就可以使用懒执行。懒执行需要唤醒，一般可以通过定时器或者事件的调用来唤醒 懒加载 懒加载就是将不关键的资源延后加载 懒加载的原理就是只加载自定义区域（通常是可视区域，但也可以是即将进入可视区域）内需要加载的东西。对于图片来说，先设置图片标签的src属性为一张占位图，将真实的图片资源放入一个自定义属性中，当进入自定义区域时，就将自定义属性替换为src属性，这样图片就会去下载资源，实现了图片懒加载

懒加载不仅可以用于图片，也可以使用在别的资源上。比如进入可视区域才开始播放视频等

1) 商品详情页把价格计算、推荐列表和首屏主图都改成进入视口后执行，结果页面先出现空壳，用户还没滚动就无法下单；这里把“懒”用错在哪了？

答：首屏成交链路不应被统一延后，价格、主图等当前区域必需内容应立即执行或加载。懒执行适合暂时不用的耗时逻辑，懒加载适合非关键资源；判断标准是当前区域与当前操作是否依赖它，延后关键内容只会把成本变成可感知等待。

2) 一个长列表有500张商品图，接口一次返回了全部图片地址；你会怎样落地图片懒加载，同时避免未加载区域把页面高度撑乱？

答：先为图片保留明确尺寸并展示占位图，把真实地址放在自定义属性中，再用IntersectionObserver监听图片进入或接近可视区域时替换src。加载完成后应取消观察，失败时保留占位或展示重试入口；

没有预留尺寸会在图片出现时引发布局跳动。

3) 资讯页的下一个视频刚进入可视区域才开始下载并播放，用户在弱网下总看到黑屏；产品又不接受进入页面就加载全部视频，你怎么调整触发边界？

答：把触发区域从“已经可见”扩大为“即将进入可视区域”，提前加载下一段视频，但仍只播放当前可见内容。预加载距离应结合滚动速度、资源体积和网络状况控制；范围过大会退化为批量加载，范围过小则无法掩盖下载等待。

4) 图片懒加载上线后，部分图片滚到屏幕内仍保持占位图，而手动刷新又恢复；你会按什么顺序定位？

答：先确认观察器是否绑定到真实图片节点，以及列表更新后新增节点是否被继续观察，再检查真实地址写入和src替换逻辑。随后查看图片请求是否发出、是否失败，并检查容器滚动场景下观察区域是否正确；只监听初始节点会让动态列表稳定复现漏加载。

5) 筛选面板包含一段耗时统计逻辑：用setTimeout延后执行，还是等用户点击“展开”再执行？如果结果还要用于首屏角标，你怎么选？

答：若统计只服务于展开后的面板，应由展开事件唤醒，避免定时器在用户从不使用时仍消耗主线程。若首屏角标依赖同一结果，就不能整体懒执行，应拆出角标所需的最小计算，其余逻辑继续延后；拆分会增加状态与缓存一致性的维护成本。

648. 文件优化

文件优化的核心是减少资源体积、避免阻塞渲染，并把静态资源更高效地送到用户手里。图片应按设备尺寸请求裁剪版本，照片可用JPEG，小图可用PNG或base64，图标优先考虑SVG，支持时也可选压缩效果更好的WebP。CSS放在head，脚本按依赖选择defer或async，耗时计算可交给Webworker。服务端还应开启压缩并使用不同于主站的CDN域名，避免静态资源请求携带主站Cookie。

图片优化对于如何优化图片，有2个思路：减少像素点、减少每个像素点能够显示的颜色。图片加载优化：不用图片。很多时候会使用到很多修饰类图片，其实这类修饰图片完全可以用CSS去代替。对于移动端来说，屏幕宽度就那么点，完全没有必要去加载原图浪费带宽。一般图片都用CDN加载，可以计算出适配屏幕的宽度，然后去请求相应裁剪好的图片。小图使用base64格式将多个图标文件整合到一张图片中（雪碧图）。选择正确的图片格式：对于能够显示WebP格式的浏览器尽量使用WebP格式。因为WebP格式具有更好的图像数据压缩算法，能带来更小的图片体积，而且拥有肉眼识别无差异的图像质量，缺点就是兼容性并不好。小图使用PNG，其实对于大部分图标这类图片，完全可以使用SVG代替照片使用JPEG。对于能够显示WebP格式的浏览器尽量使用WebP格式。因为WebP格式具有更好的图像数据压缩算法，能带来更小的图片体积，而且拥有肉眼识别无差异的图像质量，缺点就是兼容性并不好。小图使用PNG，其实对于大部分图标这类图片，完全可以使用SVG代替照片使用JPEG。其他文件优化：CSS文件放在head中，服务端开启文件压缩功能。将script标签放在body底部，因为JS文件执行会阻塞渲染。当然也可以把script标签放在任意位置然后加上defer，表示该文件会并行下载，但是会放到HTML解析完成后顺序执行。对于没有任何依赖的JS文件可以加上async，表示加载和渲染后续文档元素的过程将与JS文件的加载与执行并行无序进行。执行JS代码过长会卡住渲染，对于需要很多时间计算的代码可以考虑使用Webworker。Webworker可以让我们另开一个线程执行脚本而不影响渲染。CDN：静态资源尽量使用CDN加载，由于浏览器对于单个域名有并发请求上限，可以考虑使用多个CDN域名。对于CDN加载静态资源需要注意CDN域名要与主站不同，否则每次请求都会带上主站的Cookie。

1) 活动页为了减少请求，把一张2MB照片转成base64塞进主CSS，请求数下降了但首屏更慢；你会如何判断这次优化？

答：这是把小图内联策略错误地用于大图，图片体积会进入阻塞渲染的CSS，还失去独立资源缓存和按需加载空间。base64只适合足够小且确实值得减少请求的资源；大照片应选择合适尺寸与格式，并交给静态资源缓存或CDN。

2) 移动商品页拿到的是1920px原图，而内容区最多只有375px；设计要求清晰，后端又已有图片裁剪服务，你会怎样改造请求？

答：根据展示宽度和设备需求向图片服务请求相应裁剪尺寸，避免移动端下载远超实际像素需求的原图。照片可优先评估WebP，并为不适用的环境保留JPEG等格式；裁剪过小或压缩过度会损害清晰度，需要在体积与观感之间验收。

3) 后台管理页有80个小图标，团队在雪碧图、SVG和逐个PNG之间争论；哪些约束会改变你的选择？

答：可缩放、需要换色的图标更适合SVG，固定的小位图可考虑合并为雪碧图以减少请求，少量独立资源则未必值得增加维护复杂度。选择还要看兼容要求和更新频率；雪碧图中单个图标频繁变更会使整张图失效，独立文件则增加请求数量。

4) 线上首页的CSS已放在head，但主体仍长期空白，网络面板显示一个同步script位于文档前部且执行时间很长；你会怎么处理？

答：将非首屏脚本移到body底部或使用defer，让它并行下载并在HTML解析完成后按顺序执行。没有依赖关系的脚本才适合评估async，因为执行顺序不受保证；若真正瓶颈是长时间计算，还应拆分任务或交给Web Worker。

5) 架构师想给图片、脚本拆出多个CDN域名提升并发，安全同学担心请求携带登录Cookie；你会怎样落方案？

答：静态资源域名应与主站区分，并确保请求不携带主站Cookie，否则每次资源请求都会增加无效传输并扩大暴露面。多域名能够缓解单域并发限制，但也会增加域名解析和运维成本；资源数量不多时，不应只为理论并发继续拆域。

649. 其他

除了资源本身，构建产物和线上错误监控也是性能优化不能漏掉的两块。使用Webpack4时可启用production、基于ES6模块做tree shaking，再按路由拆包并给文件名加哈希，兼顾按需加载与浏览器缓存。线上错误可用window.onerror、Promise.catch或try catch捕获，跨域脚本还要配置crossorigin。由于生产代码通常经过压缩，构建时应生成sourceMap方便定位，并把捕获结果上传到服务端。使用 Webpack 优化项目

对于Webpack4，打包项目使用production模式，这样会自动开启代码压缩使用ES6模块来开启tree shaking，这个技术可以移除没有使用的代码优化图片，对于小图可以使用base64的方式写入文件中按照路由拆分代码，实现按需加载给打包出来的文件名添加哈希，实现浏览器缓存文件 监控 对于代码运行错误，通常的办法是使用window.onerror拦截报错。该方法能拦截到大部分的详细报错信息，但是也有例外

对于跨域的代码运行错误会显示Script error. 对于这种情况我们需要给script标签添加crossorigin属性对于某些浏览器可能不会显示调用栈信息，这种情况可以通过arguments.callee.caller来做栈递归对于异步代码来说，可以使用catch的方式捕获错误。比如Promise可以直接使用 catch 函数，async await可以使用try catch但是要注意线上运行的代码都是压缩过的，需要在打包时生成sourceMap文件便于debug。对于捕获的错误需要上传给服务器，通常可以通过img标签的src发起一个请求

1) 项目切到Webpack4的production后，负责人认为包体一定已经最优，不再做任何配置检查；你会拿什么反例追问他？

答：production会开启代码压缩，但不代表未使用代码、路由资源和图片都已按目标优化。还要确认源码使用ES6模块以支持tree shaking、路由是否按需拆分，以及小图是否适合内联；动态引用或存在副作用时，未使用代码也未必能安全移除。

2) 一个管理系统有30条路由，首屏只进入工作台；你会如何组织构建产物，并让浏览器可靠复用未变化的文件？

答：按路由拆分代码，让工作台只加载当前必需资源，其余路由在访问时再获取。输出文件名加入内容相关哈希，使未变化文件继续命中浏览器缓存、变化文件获得新地址；拆得过于细会增加请求和运行时管理成本，需要结合页面依赖调整粒度。

3) 公司要求错误监控尽量不依赖业务接口，前端负责人提出所有异常都交给window.onerror；遇到Promise和async/await异常时，你会怎样补齐？

答：window.onerror可覆盖大部分运行时错误，但异步链路仍应在Promise上使用catch，在async/await周围使用try...catch。捕获后统一整理并上报，必要时可通过图片src发起轻量请求；遗漏异步出口会形成未记录的失败。

4) 线上监控只收到Script

error，故障来自另一个域名上的脚本，本地却有完整堆栈；你先检查哪两处配置？

答：先检查跨域script是否添加了crossorigin，再确认资源服务是否允许相应的跨域访问，否则浏览器不会暴露详细错误。即使拿到文件名和行列号，线上代码仍是压缩产物，还必须保留与该版本匹配的sourceMap才能映射回源码。

5) 安全团队不允许公开部署sourceMap，研发又必须定位压缩代码报错；构建和监控之间应该怎样取舍？

答：构建阶段仍应生成与发布版本对应的sourceMap，供受控的调试或错误解析流程使用，不必把映射文件作为普通静态资源公开。监控事件还要携带可定位构建版本的信息；若映射文件与线上哈希不匹配，即使保留了文件也无法可靠还原位置。

650. 如何根据chrome的timing优化

我会先用Chrome DevTools和performance.timing拆开各阶段耗时，再针对最长的一段优化。DNS慢就减少域名并配置dns-prefetch，请求和下载慢则优先检查缓存、压缩、资源体积及并发阻塞。首屏资源可通过异步、预加载或懒加载后置，图片则压缩并选择WebP等格式。单页应用里直接依赖传统时间点不一定合适，还要结合图片onload等事件判断首屏完成。

性能优化API

Performance。performance.now()与new Date()区别，它是高精度的，且是相对时间，相对于页面加载的那一刻。但是不一定适合单页面场景window.addEventListener("load",

"");window.addEventListener("domContentLoaded", "");img的onload事件，监听首屏内的图片是否加载完成，判断首屏事件RequestAnimationFrame和RequestIdleCallbackIntersectionObserver、MutationObserver，PostMessageWeb Worker，耗时任务放在里面执行 检测工具 Chrome Dev ToolsPage SpeedJsperf 前端指标 window.onload=function(){setTimeout(function(){let t=performance.timing console.log('DNS查询耗时

:'+(t.domainLookupEnd-t.domainLookupStart).toFixed(0))console.log('TCP链接耗时

:'+(t.connectEnd-t.connectStart).toFixed(0))console.log('request请求耗时

:'+(t.responseEnd-t.responseStart).toFixed(0))console.log('解析dom树耗时

:'+(t.domComplete-t.domInteractive).toFixed(0))console.log('白屏时间

:'+(t.responseStart-t.navigationStart).toFixed(0))console.log('domready时间

:'+(t.domContentLoadedEventEnd-t.navigationStart).toFixed(0))console.log('onload时间 :'+(t.onloadEventEnd-t.navigationStart).toFixed(0))if(t=performance.memory){console.log('js内存使用占比

: '+ (t.usedJSHeapSize/t.totalJSHeapSize*100).toFixed(2)+'%')]]}]}

DNS预解析优化

dns解析是很耗时的，因此如果解析域名过多，会让首屏加载变得过慢，可以考虑dns-prefetch优化。DNS Prefetch应该尽量的放在网页的前面，推荐放在 后面。具体使用方法如下：request请求耗时 不请求，用cache（最好的方式就是尽量引用公共资源，同时设置缓存，不去重新请求资源，也可以运用PWA的离线缓存技术，可以帮助web实现离线使用）前端打包时压缩服务器上的zip压缩图片压缩（比如tiny），使用webp等高压压缩比格式把过大的包，拆分成多个较少的包，防止单个资源耗时过大同一时间针对同一域名下的请求有一定数量限制，超过限制数目的请求会被阻塞。如果资源来自于多个域下，可以增大并行请求和下载速度延迟、异步、预加载、懒加载对于非首屏的资源，可以使用 defer 或 async 的方式引入也可以按需加载，在逻辑中，只有执行到时才做请求对于多屏页面，滚动时才动态载入图片

1) 单页应用用new Date()记录按钮操作耗时，连续测量出现精度不足和系统时间调整带来的跳变；你会换成什么，并提醒什么边界？

答：短时耗时测量应使用高精度、相对页面加载时刻的performance.now()，避免依赖系统绝对时间变化。它适合计算同一页面生命周期内的时间差，但单页应用长期运行时不能把页面初始时刻直接当成每次路由的起点，应为各次操作单独记录基准。

2) 首页同时引用主站、接口、图片和分享服务四个域名，Chrome DevTools显示首次访问的域名解析拖慢首屏；你会把优化写在哪里？

答：对确定会在首屏使用的外部域名，在文档前部加入dns-prefetch，并通过x-dns-prefetch-control开启相应控制，让解析尽早发生。只应声明高概率访问的域名；预解析过多不会减少资源体积，还可能引入无意义的解析开销。

3) 瀑布图显示一个首屏包下载时间很长，团队一方主张继续拆包，另一方主张合并以减少请求；你依据哪些现象裁决？

答：先判断耗时来自单文件过大还是同域请求超过并发限制：前者适合压缩并拆出非首屏代码，后者可能需要减少首屏请求、利用缓存或调整资源域。拆包后仍应把非关键部分延迟或按需加载；仅把大包切成大量同时请求的小包，可能把下载耗时变成排队耗时。

4) 监控代码在window.onload回调里立刻读取loadEventEnd，线上偶尔得到不完整结果；你会如何修正采集时机，并区分哪些阶段？

答：应在load回调后再延迟到当前事件处理结束读取，避免loadEventEnd尚未写入。采集时分别计算DNS、连接、响应、DOM解析、白屏、DOMContentLoaded和load时间，先定位阶段再优化；单个总耗时无法说明瓶颈属于网络还是渲染。

5) 一个多屏图文页首屏资源已压缩，但用户流量仍很高；产品要求滚动顺滑，研发准备把所有图片都预加载，你会怎么取舍？

答：非首屏图片应随滚动按需加载，必要时在即将进入视口时提前触发，而不是一次预加载全部资源。首屏资源可继续使用缓存、压缩和合适图片格式；触发太晚会出现占位等待，触发太早则重新增加请求竞争与流量。

651. 移动端优化

移动端优化既要控制资源加载，也要降低脚本执行和页面渲染的开销。我一般把首屏目标控制在三秒左右，静态资源使用缓存和GZip，图片按尺寸与格式压缩，非首屏内容按需或异步加载。交互阶段要减少DOM操作、重绘和回流，高频的scroll、touchmove可配合requestAnimationFrame限制渲染频率。懒加载可能带来额外重绘，GPU加速也应合理使用，不能为了指标全部开启。

1. 概述

PC优化手段在Mobile侧同样适用在Mobile侧我们提出三秒种渲染完成首屏指标基于第二点，首屏加载3秒完成或使用Loading基于联通3G网络平均338KB/s(2.71Mb/s)，所以首屏资源不应超过1014KB Mobile侧因手机配置原因，除加载外渲

渲染速度也是优化重点基于第五点，要合理处理代码减少渲染损耗基于第二、第五点，所有影响首屏加载和渲染的代码应在处理逻辑中后置加载完成后用户交互使用时也需注意性能

2. 加载优化

加载过程是最为耗时的过程，可能会占到总耗时的80%时间，因此是优化的重点

2.1 缓存 使用缓存可以减少向服务器的请求数，节省加载时间，所以所有静态资源都要在服务器端设置缓存，并且尽量使用长Cache（长Cache资源的更新可使用时间戳）

2.2 压缩HTML、CSS、JavaScript 减少资源大小可以加快网页显示速度，所以要对HTML、CSS、JavaScript等进行代码压缩，并在服务器端设置GZip

a) 压缩（例如，多余的空格、换行符和缩进）

b) 启用GZip

2.3 无阻塞 写在HTML头部的JavaScript（无异步），和写在HTML标签中的Style会阻塞页面的渲染，因此CSS放在页面头部并使用Link方式引入，避免在HTML标签中写Style，JavaScript放在页面尾部或使用异步方式加载

2.4 使用首屏加载

首屏的快速显示，可以大大提升用户对页面速度的感知，因此应尽量针对首屏的快速显示做优化。

2.5 按需加载 将不影响首屏的资源 and 当前屏幕资源不用的资源放到用户需要时才加载，可以大大提升重要资源的显示速度和降低总体流量。

PS：按需加载会导致大量重绘，影响渲染性能

a) LazyLoad

b) 滚屏加载

c) 通过Media Query加载

2.6 预加载 大型重资源页面（如游戏）可使用增加Loading的方法，资源加载完成后显示页面。但Loading时间过长，会造成用户流失。

对用户行为分析，可以在当前页加载下一页资源，提升速度。

a) 可感知Loading

b) 不可感知的Loading（如提前加载下一页）

2.7 压缩图片 图片是最占流量的资源，因此尽量避免使用他，使用时选择最合适的格式（实现需求的前提下，以大小判断），合适的大小，然后使用智图压缩，同时在代码中用Srcset来按需显示

PS：过度压缩图片大小影响图片显示效果

a) 使用智图（<http://zhitu.tencent.com/>）

b) 使用其它方式代替图片

1. 使用CSS3

2. 使用SVG

3. 使用IconFont

c) 使用Srcset

d) 选择合适的图片

1. webP优于JPG

2. PNG8优于GIF

e) 选择合适的大小

1. 首次加载不大于1014KB

2. 不宽于640（基于手机屏幕一般宽度）

2.8 减少Cookie

Cookie会影响加载速度，所以静态资源域名不使用Cookie。

2.9 避免重定向

重定向会影响加载速度，所以在服务器正确设置避免重定向。

2.10 异步加载第三方资源

第三方资源不可控会影响页面的加载和显示，因此要异步加载第三方资源

2.11 减少HTTP请求 因为手机浏览器同时响应请求为4个请求（Android支持4个，iOS5后可支持6个），所以要尽量减少页面的请求数，首次加载同时请求数不能超过4个

a) 合并CSS、JavaScript

b) 合并小图片，使用雪碧图

3. 三、脚本执行优化

脚本处理不当会阻塞页面加载、渲染，因此在使用时需当注意

CSS写在头部，JavaScript写在尾部或异步

避免图片和iFrame等的空Src，空Src会重新加载当前页面，影响速度和效率。尽量避免重设图片大小

重设图片大小是指在页面、CSS、JavaScript等中多次重置图片大小，多次重设图片大小会引发图片的多次重绘，影响性能

图片尽量避免使用DataURL，DataURL图片没有使用图片的压缩算法文件会变大，并且要解码后再渲染，加载慢耗时长

4. CSS优化 尽量避免写在HTML标签中写Style属性

4.1 css3过渡动画开启硬件加速

```
.translate3d{ -webkit-transform: translate3d(0, 0, 0); -moz-transform: translate3d(0, 0, 0); ...}
```

1) 移动活动页为了满足“三秒首屏”，在资源没完成时一直显示全屏Loading，弱网用户等很久仍看不到任何内容；你如何评价这个方案？

答：Loading只能让等待可感知，不能替代首屏资源与渲染优化，持续过久仍会造成用户流失。应先后置不影响首屏的代码和资源，尽快展示可用的首屏结构；大型重资源页面才适合等待资源完成后整体呈现，也要控制等待范围。

2) 移动首页预算约1014KB，现在一张横幅原图就占了大半，设计不允许去掉；你会怎样压缩加载成本而不直接牺牲视觉需求？

答：先按实际展示宽度生成合适尺寸，通过srcset按需选择，并在满足兼容与观感时采用WebP等压缩率更高的格式。还应使用图片压缩工具并避免超过手机所需尺寸；过度压缩会产生明显失真，最终必须以目标设备上的显示效果验收。

3) 页面首屏有10个脚本和样式请求，旧移动浏览器同时只能处理较少请求；团队又希望所有模块独立发布，你怎么处理这个冲突？

答：首屏应优先减少必要请求数，可合并关键CSS、JavaScript和小图片，并把非关键模块异步或按需加载。模块独立发布不等于必须在首屏分别请求全部产物；合并过度会扩大缓存失效范围，因此只聚合稳定且共同使用的关键资源。

4) 商品列表滚动时明显掉帧，埋点显示scroll回调频繁修改多个元素的内联style；你会先改哪一段执行链？

答：把高频scroll更新合并到requestAnimationFrame中，并增加响应间隔，避免同一帧内反复触发渲染。批量切换class，减少零散DOM操作和不必要的重绘回流；若每帧计算本身仍然过重，仅限频不能消除卡顿。

5) 动画负责人要求给页面所有卡片加translate3d强制开启GPU，测试发现低端机内存压力反而上升；你会怎么划定使用范围？

答：硬件加速应只用于确实需要平滑过渡的动画元素，并优先使用transform、opacity配合requestAnimationFrame。大量元素长期占用独立合成资源会增加设备负担，因此要在真实移动设备上验证；静态卡片或偶发变化不应统一强制提升。

6) 第三方统计脚本偶尔超时并拖住移动首页，业务方又要求不能删除；你如何隔离它与首屏渲染？

答：将第三方资源改为异步加载，并确保首屏内容不依赖其下载或执行结果，因为外部服务的时延不可控。脚本还应避免放在会阻塞解析的位置；异步化可能改变埋点初始化时序，业务事件需要在脚本就绪前暂存或允许安全丢弃。

【工程化】

652. 介绍一下 webpack 的构建流程

Webpack的构建流程本质上是读取配置，从入口递归生成依赖图，再组织为chunk并输出资源。启动时它会合并配置和命令行参数，创建全局的Compiler，加载插件并开始一次Compilation。模块经过对应loader转换后，再通过AST分析依赖，直到所有入口依赖都处理完成。最后按依赖关系组装产物并写入文件系统，插件则通过Tapable的生命周期事件介入并修改构建结果。核心概念 entry：入口。webpack是基于模块的，使用webpack首先需要指定模块解析入口(entry)，webpack从入口开始根据模块间依赖关系递归解析和处理所有资源文件。output：输出。源代码经过webpack处理之后的最终产物。loader：模块转换器。本质就是一个函数，在该函数中对接收到的内容进行转换，返回转换后的结果。因为 Webpack 只认识 JavaScript，所以 Loader 就成了翻译官，对其他类型的资源进行转译的预处理工作。plugin：扩展插件。基于事件流框架Tapable，插件可以扩展 Webpack 的功能，在 Webpack 运行的生命周期中会广播出许多事件，Plugin 可以监听这些事件，在合适的时机通过 Webpack 提供的 API 改变输出结果。module：模块。除了js范畴内的es module、commonJs、AMD等，css @import、url(...)、图片、字体等在webpack中都被视为模块。解释几个 webpack 中的术语 module：指在模块化编程中我们把应用程序分割成的独立功能的代码模块chunk：指模块间按照引用关系组合成的代码块，一个chunk中可以包含多个modulechunk group：指通过配置入口点（entry point）区分的块组，一个chunk group中可包含一到多个 chunkbundling：webpack 打包的过程asset/bundle：打包产物 webpack 的打包思想可以简化为 3 点：

一切源代码文件均可通过各种Loader转换为 JS 模块（module），模块之间可以互相引用。webpack 通过入口点（entry point）递归处理各模块引用关系，最后输出为一个或多个产物包js(bundle)文件。每一个入口点都是一个块组（chunk group），在不考虑分包的情况下，一个chunk group中只有一个chunk，该 chunk 包含递归分析后的所有模块。每一个chunk都有对应的一个打包后的输出文件（asset/bundle） 打包流程初始化参数：从配置文件和 Shell 语句中读取并合并参数，得出最终的配置参数。开始编译：从上一步得到的参数初始化Compiler对象，加载所有配置的插件，执行对象的run方法开始执行编译。确定入口：根据配置中的entry找出所有的入口文件。编译模块：从入口文件出发，调用所有配置的loader对模块进行翻译，再找出该模块依赖的模块，这个步骤是递归执行的，直至所有入口依赖的模块文件都经过本步骤的处理。完成模块编译：经过第 4 步使用 loader 翻译完所有模块后，得到了每个模块被翻译后的最终内容以及它们之间的依赖关系。输出资源：根据入口和模块之间的依赖关系，组装成一个个包含多个模块的chunk，再把每个chunk转换成一个单独的文件加入到输出列表，这一步是可以修改输出内容的最后机会。输出完成：在确定好输出内容后，根据配置确定输出的路径和文件名，把文件内容写入到文件系统。 简版 Webpack CLI 启动打包流程；载入 Webpack 核心模块，创建Compiler对象；使用Compiler对象开始编译整个项目；从入口文件开始，解析模块依赖，形成依赖关系树；递归依赖树，将每个模块交给对应的 Loader 处理；合并 Loader 处理完的结果，将打包结果输出到 dist 目录。 在以上过程中，Webpack 会在特定的时间点广播出特定的事件，插件在监听到相关事件后会执行特定的逻辑，并且插件可以调用 Webpack 提供的 API 改变 Webpack 的运行结果 构建流程核心概念：Tapable：一个基于发布订阅的事件流工具类，Compiler和Compilation对象都继承于TapableCompiler：compiler对象是一个全局单例，他负责把控整个webpack打包的构建流程。在编译初始化阶段被创建的全局单例，包含完整配置信息、loaders、plugins以及各种工具方法Compilation：代表一次 webpack 构建和生成编译资源的过程，在 watch模式下每一次文件变更触发的重新编译都会生成新的Compilation对象，包含了当前编译的模块module,...

1) 一个后台项目只有一个entry，最终却生成了入口包、公共包和异步包；同事坚持“一个入口只会对应一个文件”，你怎么纠正？

答：entry对应的是入口点和chunk group，并不保证最终只有一个输出文件。Webpack会从入口递归建立模块依赖图，再根据代码分割等规则组装多个chunk，每个chunk才通常对应一个输出asset；不考虑分包时，才可近似理解为一个入口产生一个包。

2) 团队要让Webpack支持TypeScript、Less、字体和图片，你会把这些处理分别放到构建流程的什么位置？

答：这些资源都应先作为module进入依赖图，再由匹配的loader完成转换，例如ts-loader、less-loader和文件资源加载器。Webpack从entry递归解析引用，转换完成后才汇总模块关系并生成chunk；若处理需要观察整轮构建或修改最终产物，则应交给plugin。

3) 同一套配置由开发服务器以watch模式运行，连续改动三个文件时，Compiler和Compilation会怎样变化？

答：Compiler在初始化阶段创建并贯穿整个构建生命周期，配置、插件和工具方法都由它持有。每次文件变化触发重新编译时会新建一个Compilation，记录该轮模块、资源、变更文件和依赖状态；把轮次状态长期挂在Compiler上，容易造成数据串轮或缓存失真。

4) 一个新增的loader能输出合法代码，但它引用的子模块始终没有进入产物；你会沿构建链路查哪些环节？

答：先确认入口是否实际到达该文件，以及loader的匹配范围和输出是否保留了可被解析的模块引用。随后检查转换结果生成的AST能否识别依赖，再核对依赖解析、模块编译和chunk组装结果；若依赖是在运行时动态拼接的，静态分析可能无法发现。

5) 产品要求在写入dist前改写所有资源名，同时还要转换单个源码文件；你会怎样划分loader与plugin？

答：单文件内容转换放在loader，因为它处于模块编译阶段，只需接收并返回可继续处理的内容。资源名改写应由plugin在输出资源阶段监听合适的Tapable钩子，利用Compilation操作完整产物列表；若等文件写入后再处理，会增加额外文件操作并可能破坏引用关系。

653. 介绍 Loader

Loader是Webpack的模块转换器，负责把它原本不能直接处理的资源转换成可继续构建的内容。它本质上是接收内容并返回转换结果的函数，多个loader可以按配置形成链式处理。比如babel-loader转换新语法，css-loader解析@import和url()，style-loader再把样式插入页面。编写时应保持单一职责和相互独立，否则转换链过重，也会让配置与问题定位变复杂。常用 Loader: file-loader: 加载文件资源，如 字体 / 图片 等，具有移动/复制/命名等功能；url-loader: 通常用于加载图片，可以将小图片直接转换为 Data Url，减少请求；babel-loader: 加载 js / jsx 文件，将 ES6 / ES7 代码转换成 ES5，抹平兼容性问题；ts-loader: 加载 ts / tsx 文件，编译 TypeScript；style-loader: 将 css 代码以标签的形式插入到 html 中；css-loader: 分析@import和url()，引用 css 文件与对应的资源；postcss-loader: 用于 css 的兼容性处理，具有众多功能，例如 添加前缀，单位转换等；less-loader / sass-loader: css预处理器，在 css 中新增了许多语法，提高了开发效率；编写原则: 单一原则: 每个 Loader 只做一件事；链式调用: Webpack 会按顺序链式调用每个 Loader；统一原则: 遵循 Webpack制定的设计规则和结构，输入与输出均为字符串，各个 Loader 完全独立，即插即用；

1) 一个Less文件先后经过less-loader、css-loader、style-loader，同事把三者都理解成“把样式塞进页面”，你怎么指出反例？

答：三者承担的是连续但不同的转换职责：less-loader处理预处理语法，css-loader分析@import和url()依赖，style-loader才把样式以形式注入页面。把职责合进一个loader会削弱复用和可组合性，也违背单一职责原则。

2) 项目同时有src下的业务代码和大量已编译依赖，配置babel-loader时你会怎样控制匹配范围？

答：让babel-loader只处理确实需要转换的js或jsx，通常通过test配合include或exclude缩小到业务源码。这样既避免重复转换已编译的第三方代码，也减少无效搜索；若依赖包确实发布了目标环境不支持的语法，则要针对该包单独放开范围。

3) 设计一个图片规则时，小图希望内联，大图需要复制并重命名；你如何解释url-loader与file-loader的职责边界？

答：小图片可由url-loader转成DataURL，从而减少独立资源请求；超过限定条件的文件则交给文件资源处理逻辑完成复制、命名和输出。内联会增大承载它的代码或样式体积，因此不能只追求减少请求而无条件处理所有图片。

4) 新增自定义loader后，构建报“下一个loader无法处理输入”，但单独执行转换函数结果正常，你先查什么？

答：先检查整条loader链的执行顺序，以及当前输出是否符合下一个loader可识别的内容形式。每个loader都必须保持独立且可链式组合，不能只验证自身转换结果；若某一步输出了二进制或特殊结构，还需按Webpack的接口约定明确声明和适配。

5) 团队要实现“转换每个SVG文件”和“汇总全部图片生成资源清单”，负责人想统一写成一个loader，你怎么取舍？

答：SVG到可识别模块内容的转换适合loader，因为处理对象是单个文件，输入输出边界清晰。全量资源清单依赖整次编译的输出集合，应由plugin监听生命周期并读取产物；强行放进loader会让单文件转换产生跨模块状态和执行顺序风险。

654. 介绍 plugin

Plugin是通过钩子参与Webpack整个构建生命周期、并在特定阶段修改打包结果的插件。注册时，Webpack会调用它的apply方法并传入全局唯一的Compiler，插件再借助Tapable的事件机制订阅编译钩子。每轮编译会创建新的Compilation，所以插件既能处理全局流程，也能介入单次编译。相比之下，loader主要负责转换单个模块，Plugin更适合生成页面、抽离样式或分析产物。插件系统是Webpack成功的一个关键性因素。在编译的整个生命周期中，Webpack会触发许多事件钩子，Plugin可以监听这些事件，根据需求在相应的时间点对打包内容进行定向的修改。一个最简单的plugin是这样的：

```
classPlugin{// 注册插件时，会调用 apply 方法// apply 方法接收 compiler 对象// 通过 compiler 上提供的 Api，可以对事件进行监听，执行相应的操作apply(compiler){// compilation 是监听每次编译循环// 每次文件变化，都会生成新的 compilation 对象并触发该事件compiler.plugin('compilation',function(compilation){})}注册插件:// webpack.config.jsmodule.export={plugins:[newPlugin(options),]}事件流机制:Webpack就像工厂中的一条产品流水线。原材料经过 Loader 与 Plugin 的一道道处理，最后输出结果。通过链式调用，按顺序串起一个个 Loader；通过事件流机制，让 Plugin 可以插入到整个生产过程中的每个步骤中；Webpack 事件流编程范式的核心是基础类 Tapable，是一种观察者模式的实现事件的订阅与广播：
```

```
const{SyncHook}=require("tapable")consthook=newSyncHook(['arg'])// 订阅hook.tap('event',(arg)=>{// 'event-hook'console.log(arg)})// 广播hook.call('event-hook')Webpack中两个最重要的类Compiler与Compilation便是继承于Tapable，也拥有这样的事件流机制。Compiler: 可以简单的理解为 Webpack 实例，它包含了当前 Webpack 中的所有配置信息，如 options，loaders, plugins 等信息，全局唯一，只在启动时完成初始化创建，随着生命周期逐一传递；Compilation: 可以称为编译实例。当监听到文件发生改变时，Webpack 会创建一个新的 Compilation 对象，开始一次新的编译。它包含了当前的输入资源，输出资源，变化的文件等，同时通过它提供的 api，可以监听每次编译过程中触发的事件钩子；区别:Compiler全局唯一，且从启动生存到结束；Compilation 对应每次编译，每轮编译循环均会重新创建；常用 Plugin:UglifyJsPlugin: 压缩、混淆代码；CommonsChunkPlugin: 代码分割；ProvidePlugin: 自动加载模块；html-webpack-plugin: 加载 html 文件，并引入 css / js 文件；extract-text-webpack-plugin / mini-css-extract-plugin: 抽离样式，生成 css 文件；DefinePlugin: 定义全局变量；optimize-css-assets-webpack-plugin: CSS 代码去重；webpack-bundle-analyzer: 代码分析；compression-webpack-plugin: 使用 gzip 压缩 js 和 css；happypack: 使用多进程，加速代码构建；EnvironmentPlugin: 定义环境变量；调用插件apply函数传入compiler对象通过compiler对象监听事件 Compiler: 可以简单的理解为 Webpack 实例，它包含了当前 Webpack 中的所有配置信息，如 options，loaders, plugins 等信息，全局唯一，只在启动时完成初始化创建，随着生命周期逐一传递；Compilation: 可以称为编译实例。当监听到文件发生改变时，Webpack 会创建一个新的 Compilation 对象，开始一次新的编译。它包含了当前的输入资源，输出资源，变化的文件等，同时通过它提供的 api，可以监听每次编译过程中触发的事...
```

1) 一个插件只修改单个.js文件源码，同事却说“只要用了apply就一定比loader合适”，你怎么反驳？

答：是否适合plugin取决于是否需要介入构建生命周期，而不是有没有apply方法。单文件、可链式的内容转换通常属于loader；只有需要观察编译事件、访问整轮资源或定向修改打包结果时，plugin的事件钩子才体现价值。

2)

你要写一个插件，在每轮构建中读取所有输出资源并生成清单，apply里应该抓住哪些对象？

答：插件注册时由Webpack调用apply(compiler)，先通过compiler订阅与编译相关的钩子，再在回调中取得当轮Compilation。资源清单应从该轮编译实例的输出资源生成，避免把上一轮数据留在全局状态中；具体钩子还要服从资源是否已完整生成的时机要求。

3) 开发服务器持续运行并触发 20

次增量编译，插件把资源列表缓存到Compiler上且从不清空，会有什么问题？

答：Compiler全局唯一并持续到进程结束，而每轮文件变化都会创建新的Compilation。把轮次资源长期累计在Compiler上，可能让已删除或改名的资源混入后续结果；缓存若必须跨轮保留，就要明确失效条件，并以当前Compilation为事实来源。

4) 自定义插件在首次构建正常，文件改动后却重复执行多次并生成重复内容，你会先排查哪里？

答：先确认钩子是否在每次compilation回调里被重复注册，以及插件实例是否被配置了多次。再区分订阅的是全局Compiler生命周期还是单轮Compilation生命周期，检查状态是否随新编译重置；错误的注册层级会在watch模式下不断叠加监听。

5) 团队要自动生成HTML并注入打包后的JS，另一个方案是在源码里手写固定文件名，你会选哪个？

答：应让类似html-webpack-plugin的插件在产物确定后生成或修改HTML，依据实际资源名完成注入。手写文件名无法可靠跟随代码分割、内容哈希和资源增删变化；代价是插件必须选择正确钩子，并避免覆盖其他插件已经完成的修改。

655. webpack 热更新实现原理

Webpack热更新本质上是重新编译变更模块，再由HMR运行时把补丁替换到当前页面中。文件变化后，文件系统通知Webpack，它只为相关模块生成新的编译结果，并通知HMR Server。服务端通过WebSocket告知浏览器存在更新，浏览器再用HTTP请求对应的更新内容。运行时能够接收更新就替换模块；如果判断无法更新，则会退化为整页刷新。HMR的基本流程图 当修改了一个或多个文件；文件系统接收更改并通知webpack；webpack重新编译构建一个或多个模块，并通知 HMR 服务器进行更新；HMR Server使用WebSocket通知HMR runtime需要更新，HMR运行时通过HTTP请求更新jsonpHMR运行时替换更新中的模块，如果确定这些模块无法更新，则触发整个页面刷新

1) 修改一个样式模块后浏览器收到了WebSocket消息，同事便认定新代码也通过WebSocket传输，这个判断哪里不对？

答：WebSocket主要负责由HMR

Server通知浏览器端运行时存在新更新，并不等于承载完整更新代码。HMR runtime收到通知后还会通过HTTP请求更新内容，再执行模块替换；因此通知链路正常，只能证明更新事件已送达。

2) 一个包含 300

个模块的管理后台只改了其中一个文件，HMR从磁盘变化到页面更新会经过哪些关键环节？

答：文件系统先通知Webpack，Webpack针对受影响模块重新编译并把更新信息交给HMR Server。服务端经WebSocket通知浏览器，HMR runtime再通过HTTP获取更新并替换对应模块；是否能保留页面状态，还取决于更新边界能否被安全接受。

3) 公司代理允许普通HTTP请求却拦截WebSocket，页面初次加载正常，但修改文件后一直没有热更新，你如何判断影响范围？

答：初始资源仍可通过HTTP加载，但浏览器收不到HMR Server的更新通知，因此不会主动请求增量内容。应先检查连接建立、代理升级配置和服务端通知是否到达；即使更新文件可直接访问，缺少通知链路也无法自动触发替换。

4) 开发环境改文件后服务端已重新编译，浏览器也收到通知，却最终整页刷新；你按什么顺序排查？

答：先确认HMR runtime是否成功发起更新请求以及返回内容是否可执行，再检查变更模块能否被当前热更新边界接受。运行时若判断模块无法安全替换，就会退化为整页刷新；若请求本身失败，则应优先排查资源路径、服务端输出和网络代理。

5) 有人提议每次文件变化都直接刷新页面，认为这与HMR没有本质差异；在复杂表单页面里你怎么取舍？

答：整页刷新会重新加载整个应用，并丢失未持久化的界面状态；HMR只重新编译和获取受影响模块，成功时可局部替换。复杂表单更适合保留HMR，但必须接受无法更新时仍会刷新页面，并确保状态不能只依赖热更新来保护。

656. webpack 层面如何做性能优化

Webpack优化要分成构建速度和产物体积两条线，并先用数据找到真正的瓶颈。我一般用speed-measure-webpack-plugin看编译耗时，用webpack-bundle-analyzer检查包里哪些内容占空间。构建侧可以通过缓存、限制loader处理范围和减少模块搜索来提速；产物侧则依靠Tree Shaking、SplitChunksPlugin和代码压缩减少下载量。多进程、DLLPlugin或externals都有配置成本，应结合项目依赖和构建情况选择。

优化前的准备工作 准备基于时间的分析工具：我们需要一类插件，来帮助我们统计项目构建过程中在编译阶段的耗时情况。speed-measure-webpack-plugin分析插件加载的时间使用webpack-bundle-analyzer分析产物内容 代码优化: 无用代码消除，是许多编程语言都具有的优化手段，这个过程称为 DCE (dead code elimination)，即 删除不可能执行的代码；

例如我们的UglifyJs，它就会帮我们在生产环境中删除不可能被执行的代码，例如：

```
var fn=function(){return 1; // 下面代码便属于 不可能执行的代码； // 通过 UglifyJs (Webpack4+ 已内置) 便会进行 DCE ; var a=1; return a;}
```

摇树优化 (Tree-shaking)，这是一种形象比喻。我们把打包后的代码比喻成一棵树，这里其实表示的就是，通过工具“摇”我们打包后的 js 代码，将没有使用到的无用代码“摇”下来（删除）。即 消除那些被 引用了但未被使用的模块代码。 原理: 由于是在编译时优化，因此最基本的前提就是语法的静态分析，ES6的模块机制提供了这种可能性。不需要运行时，便可进行代码字面上的静态分析，确定相应的依赖关系。问题: 具有副作用 的函数无法被tree-shaking在引用一些第三方库，需要去观察其引入的代码量是不是符合预期；尽量写纯函数，减少函数的副作用；可使用webpack-deep-scope-plugin，可以进行作用域分析，减少此类情况的发生，但仍需要注意； 在引用一些第三方库，需要去观察其引入的代码量是不是符合预期；尽量写纯函数，减少函数的副作用；可使用webpack-deep-scope-plugin，可以进行作用域分析，减少此类情况的发生，但仍需要注意；

code-splitting: 代码分割技术，将代码分割成多份进行 懒加载 或 异步加载，避免打包成一份后导致体积过大，影响页面的首屏加载；

Webpack中使用SplitChunksPlugin进行拆分；按 页面 拆分: 不同页面打包成不同的文件；按 功能 拆分: 将类似于播放器，计算库等大模块进行拆分后再懒加载引入；提取复用的业务代码，减少冗余代码；按 文件修改频率 拆分: 将第三方库等不常修改的代码单独打包，而且不改变其文件 hash 值，能最大化运用浏览器的缓存；

将类似于播放器，计算库等大模块进行拆分后再懒加载引入；提取复用的业务代码，减少冗余代码；

scope hoisting: 作用域提升，将分散的模块划分到同一个作用域中，避免了代码的重复引入，有效减少打包后的代码体积和运行时的内存损耗；编译性能优化：升级至最新版本的webpack，能有效提升编译性能；使用dev-server/模块热替换(HMR)提升开发体验；监听文件变动忽略node_modules目录能有效提高监听时的编译效率；缩小编译范围modules: 指定模块路径，减少递归搜索；mainFields: 指定入口文件描述字段，减少搜索；noParse: 避免对非模块化文件的加载；includes/exclude: 指定搜索范围/排除不必要的搜索范围；alias: 缓存目录，避免重复寻址；babel-loader忽略node_modules，避免编译第三方库中已经被编译过的代码使用cacheDirectory，可以缓存编译结果，避免多次重复编译多进程并发webpack-parallel-uglify-plugin: 可多进程并发压缩js文件，提高压缩速度；HappyPack: 多进程并发文件的Loader解析；第三方库模块缓存:DLLPlugin和DLLReferencePlugin可以提前进行打包并缓存，避免每次都重新编译；使用分析Webpack Analyse / webpack-bundle-analyzer对打包后的文件进行分析，寻找可优化的地方配置profile: true，对各个编译阶段耗时进行监控，寻找耗时最多的地方source-map:开发:cheap-module-eval-source-map生产:hidd...

1) 产物分析显示某个库被引用却未使用，团队开启Tree Shaking后体积仍没下降；你会指出哪些反例边界？

答：Tree Shaking依赖ES6模块的静态结构来判断引用关系，并不等同于删除所有看似无用的代码。带副作用的模块或函数可能无法安全移除，应检查导入方式、第三方库实际引入量和副作用声明；强行删除会改变运行结果。

2) 一个项目构建耗时明显上升，但团队还没拿到阶段数据，你会怎样安排第一轮优化？

答：先用speed-measure-webpack-plugin或profile定位耗时阶段，再用webpack-bundle-analyzer检查产物组成，避免把构建速度和包体积混为一谈。确认瓶颈后再缩小loader范围、启用babel-loader缓存或调整解析规则；并发工具并非对所有任务都有收益。

3) 首屏包过大，但业务要求播放器只有点击后才加载，第三方框架还要尽量命中长期缓存，你怎么拆？

答：播放器适合按功能使用异步加载，避免进入首屏；复用业务代码可抽成公共块，低频变更的第三方库可单独拆分以稳定文件哈希。拆分应通过SplitChunksPlugin等规则落地，但块过多会增加加载调度成本，仍需结合实际依赖关系验证。

4) 开发构建慢，日志显示大量时间花在模块搜索和node_modules转译上，你会改哪些配置并防什么风险？

答：用test、include和exclude缩小loader搜索范围，并通过modules、mainFields、alias等解析配置减少递归寻址；babel-loader可排除无须转译的依赖并启用cacheDirectory。范围收得过窄可能漏掉含不兼容语法的第三方模块，因此要以目标运行环境验证。

5) 架构组在externals和DllPlugin之间争论如何移出大型依赖，你会依据什么选择？

答：已有稳定CDN资源且能明确全局对象、CommonJS或AMD等加载方式时，externals配置更直接。需要独立构建并缓存依赖、且业务仍会引用依赖子模块时，DllPlugin更完整，但配置和维护成本更高；两者都要防止外部依赖版本与业务构建不一致。

6) 生产包已经做了代码分割，运维又要求开启gzip，前端同事认为二者只能选一个，你怎么解释它们的关系？

答：代码分割控制哪些代码在何时加载，gzip控制传输时的压缩体积，两者解决的不是同一层问题，可以同时使用。还可配合代码压缩、样式抽离、Tree Shaking和作用域提升减少产物；最终仍需分别检查包结构、网络传输和运行时开销。

657. 介绍一下 Tree Shaking

Tree Shaking是在打包阶段分析并删除已引入但实际没有使用的代码。它依赖ES Module的import、export能被静态分析，因此工具不运行程序也能判断模块和变量的引用关系；CommonJS具有动态加载特性，就不适合这样处理。代码存在副作用时，打包器不能贸然删除，可以用package.json的sideEffects明确标记。还要避免让Babel把模块转成不利于静态分析的形式，否则优化可能无法生效。对tree-shaking的了解作用：它表示在打包的时候会去除一些无用的代码。原理：ES6的模块引入是静态分析的，所以在编译时能正确判断到底加载了哪些模块分析程序流，判断哪些变量未被使用、引用，进而删除此代码。特点：在生产模式下它是默认开启的，但是由于经过babel编译全部模块被封装成IIFE，它存在副作用无法被tree-shaking掉可以在package.json中配置sideEffects来指定哪些文件是有副作用的。它有两种值，一个是布尔类型，如果是false则表示所有文件都没有副作用；如果是一个数组的话，数组里的文件路径表示改文件有副作用。rollup和webpack中对tree-shaking的层次不同，例如对babel转译后的class，如果babel的转译是宽松模式下的话(也就是loose为true)，webpack依旧会认为它有副作用不会tree-shaking掉，而rollup会。这是因为rollup有程序流分析的功能，可以更好的判断代码是否真正会产生副作用。原理 ES6 Module引入进行静态分析，故而编译的时候正确判断到底加载了那些模块静态分析程序流，判断那些模块和变量未被使用或者引用，进而删除对应代码。依赖于import/export 通过导入所有的包后再进行条件获取。如下：
`import foo from "foo"; import bar from "bar"; if (condition) { // foo.xxx } else { // bar.xxx }`
ES6的import语法完美可以使用tree shaking，因为可以在代码不运行的情况下就能分析出不需要的代码。CommonJS的动态特性模块意味着tree shaking不适用。因为它是不可能确定哪些模块实际运行之前是需要的或者是不需要的。在ES6中，进入了完全静态的导入语法：import。这也意味着下面的导入是不可行的：
`// 不可行，ES6 的import是完全静态的 if (condition) { myDynamicModule = require("foo"); } else { myDynamicModule = require("bar"); }`

1) 商品页只引用了工具包导出的formatPrice，构建产物里却仍保留同文件导出的formatDate；既然用了import，为什么不能直接认定Tree Shaking一定生效？

答：import/export只是静态分析的必要条件，不等于未引用代码必然可删。打包器还要结合程序流和副作用判断；如果模块经过转译后被封装成IIFE，或相关代码可能产生副作用，就会保守地保留。

2) 团队准备发布一个包含几十个工具函数的ESM包，希望业务只打入实际引用的函数，package.json应怎样配置？

答：确认所有文件确实没有导入即执行的副作用后，可配置"sideEffects": false，帮助打包器安全删除未使用模块。若只有样式或初始化文件有副作用，应改用数组逐项声明，否则误标可能把业务依赖的执行逻辑一起删掉。

3) 同一套源码直接交给打包器时能摇掉未使用导出，经Babel处理后却全部保留，你会先检查哪一层？

答：先检查转译产物是否仍保留ESM的静态import/export，以及模块是否被包装成带潜在副作用的IIFE。Tree Shaking依赖编译期可分析的模块关系；结构被改写后，打包器无法证明代码无副作用，只能停止删除。

4) 订单页根据运行时条件在require("foo")和require("bar")之间选择，产品要求构建时移除永远不会走到的分支，Tree Shaking能完成吗？

答：仅凭这种动态CommonJS加载通常不能完成，因为构建阶段无法确定运行时究竟需要哪个模块。应把可静态确定的依赖改成顶层import，再让程序流分析处理未引用部分；若条件只能运行时确定，两份依赖都可能需要保留。

5) 一个转译后的class在webpack里未被删除，而同样产物交给Rollup却能删除，你会如何向构建负责人解释差异？

答：两者对副作用和程序流的判断能力并不完全相同，不能只凭“都支持Tree Shaking”推断结果一致。对于Babel宽松模式转译的class，webpack可能保守认定有副作用，而Rollup可借助程序流分析判断其实际影响；切换工具也要重新验证产物语义。

658. 介绍一下 webpack scope hosting

Scope Hoisting会把原本分散在多个模块函数里的代码合并到同一作用域，减少函数包裹和模块间的调用。webpack默认会为每个模块创建独立函数，模块多了会增加产物体积和运行时开销。它依赖ESM的静态导入导出关系，动态的CommonJS `require`无法完成这种提升。webpack 4+在mode: 'production'下默认开启，对应`optimization.concatenateModules`。作用域提升（Scope Hoisting），将分散的模块划分到同一个作用域中，避免了代码的重复引入，有效减少打包后的代码体积和运行时的内存损耗。原理：webpack默认会把每个模块包裹进一个独立的函数（`(function(module, exports, require){...})`），模块越多，这类包裹函数和`__webpack_require__`调用越多，产物体积和运行时开销都变大。Scope Hoisting会分析模块间的依赖关系，把可以合并的模块「提升」到同一个函数作用域里，减少函数包裹和模块间的`require`调用。要点：依赖ESM的静态结构：Scope Hoisting只能对ES Module（`import/export`）生效——因为它需要在编译期静态分析模块的导入导出关系；CommonJS的动态`require`无法静态分析，不能提升。这也是「用ESM才能享受更好的打包优化」的又一个理由（同Tree Shaking）。生产模式默认开启：webpack 4+在mode: 'production'下默认启用（对应`optimization.concatenateModules`），开发模式默认关闭。收益：更小的产物、更少的闭包和函数调用、更快的运行时。一句话：Scope Hoisting把多个ESM模块合并到同一作用域，减少函数包裹和`require`调用，从而减小体积、提升运行时性能；它依赖ESM的静态结构，生产模式默认开启。

1) 一个后台页面拆成上百个小模块后，产物里出现大量`(function(module, exports, require){...})`和`__webpack_require__`调用；Tree Shaking已经开启，为什么这些包装还在？

答：Tree Shaking负责移除未使用代码，并不负责消除仍需执行模块的包装。ScopeHoisting才会分析依赖关系，把可合并的ESM模块提升到同一函数作用域，从而减少包装函数和模块间的`require`调用。

2) 构建负责人想在生产包中启用作用域提升，webpack 4+项目需要增加什么配置，开发环境又该怎么处理？

答：在mode: "production"下通常无需额外配置，`optimization.concatenateModules`默认启用。开发模式默认关闭；若手动开启，应先确认调试体验和构建行为符合预期，因为它会改变模块在产物中的组织方式。

3) 组件库全部使用`require`和`module.exports`，业务方要求仅打开`concatenateModules`就把模块合并到一个作用域，能达到吗？

答：不能据此保证合并，因为ScopeHoisting依赖ESM的静态导入导出关系。动态`require`无法在编译期完整确定依赖图，应先评估迁移到`import/export`；否则打包器只能保留独立模块包装。

4) 生产包明明开启了`concatenateModules`，入口附近仍有很多独立函数模块，你会怎样判断是配置失效还是模块不可提升？

答：先确认实际构建使用了生产模式，并检查输入模块是否仍是可静态分析的ESM。若前置转译已改写为CommonJS，依赖关系就无法满足提升条件；此时继续调整压缩参数不会解决根因。

5) 性能评审会上，有人主张只做ScopeHoisting、不做Tree Shaking，因为两者都能缩小包体；你会怎么裁决？

答：两者解决的冗余不同，不能互相替代：Tree Shaking删除未引用代码，ScopeHoisting合并仍需保留的模块作用域。生产构建可同时利用二者，但都依赖ESM的静态结构；输入是动态模块时，两类优化都会受限。

659. Webpack Proxy工作原理？为什么能解决跨域

Webpack Proxy本质上是让本地开发服务器代替浏览器向目标服务转发请求，从而绕开浏览器的跨域限制。浏览器只请求同源的webpack-dev-server，后者通过http-proxy-middleware请求后端，再把响应转回浏览器；服务器之间通信不受浏览器同源策略约束。配置时通常关注target、pathRewrite和changeOrigin。它只用于开发阶段，生产环境仍要通过服务端代理或后端跨域配置处理。

1. 是什么 webpack proxy，即webpack提供的代理服务

基本行为就是接收客户端发送的请求后转发给其他服务器其目的是为了便于开发者在开发模式下解决跨域问题（浏览器安全策略限制）

想要实现代理首先需要有一个中间服务器，webpack中提供服务器的工具为webpack-dev-server

2. webpack-dev-server webpack-dev-server是webpack官方推出的一款开发工具，将自动编译和自动刷新浏览器等一系列对开发友好的功能全部集成在了一起

目的是为了提高开发者日常的开发效率，「只适用在开发阶段」

关于配置方面，在webpack配置对象属性中通过devServer属性提供，如下：

```
//  
./webpack.config.jsconst path=require('path')module.exports={// ...devServer:{contentBase:path.  
join(__dirname,'dist'),compress:true,port:9000,proxy:{'/api':{target:'https://api.github.com'}}// ...}
```

devServer里面proxy则是关于代理的配置，该属性为对象的形式，对象中每一个属性就是一个代理的规则匹配 属性的名称是需要被代理的请求路径前缀，一般为了辨别都会设置前缀为/api，值为对应的代理匹配规则，对应如下：

target：表示的是代理到的目标地址pathRewrite：默认情况下，我们的/api-hy也会被写入到URL中，如果希望删除，可以使用pathRewritesecure：默认情况下不接收转发到https的服务器上，如果希望支持，可以设置为falsechangeOrigin：它表示是否更新代理后请求的headers中host地址

2. 工作原理

proxy工作原理实质上是利用http-proxy-middleware这个http代理中间件，实现请求转发给其他服务器

举个例子：在开发阶段，本地地址为http://localhost:3000，该浏览器发送一个前缀带有/api标识的请求到服务端获取数据，但响应这个请求的服务器只是将请求转发到另一台服务器中

```
const express=require('express');const proxy=require('http-proxy-middleware');const app=express();app.use('/api',proxy({target:'http://www.example.org',changeOrigin:true}));app.listen(3000);//
```

http://localhost:3000/api/foo/bar -> http://www.example.org/api/foo/bar

3. 跨域 在开发阶段，webpack-dev-server会启动一个本地开发服务器，所以我们的应用在开发阶段是独立运行在localhost的一个端口上，而后端服务又是运行在另外一个地址上

所以在开发阶段中，由于浏览器同源策略的原因，当本地访问后端就会出现跨域请求的问题

通过设置webpack proxy实现代理请求后，相当于浏览器与服务端中添加一个代理者 当本地发送请求的时候，代理服务器响应该请求，并将请求转发到目标服务器，目标服务器响应数据后再将数据返回给代理服务器，最终再由代理服务器将数据响应给本地 在代理服务器传递数据给本地浏览器的过程中，两者同源，并不存在跨域行为，这时候浏览器就能正常接收数据

注意：「服务器与服务器之间请求数据并不会存在跨域行为，跨域行为是浏览器安全策略限制」

1) 本地页面运行在http://localhost:3000，直接请求另一地址的接口被浏览器拦截；把请求改成/api/users后由开发服务器转发，为什么响应就能被页面读取？

答：浏览器此时只请求同源的localhost:3000，跨地址转发由webpack-dev-server代理在服务器侧完成。同源策略限制的是浏览器中的请求，服务器之间转发不受该限制；代理收到目标响应后再以同源响应返回页面。

2) 前端请求/api/users，后端实际只接受/users，代理目标已配成https://service.example.com，你会改哪项配置？

答：应通过pathRewrite在转发时移除或改写/api前缀，使目标服务器收到它认可的路径。target只决定请求发往哪里，不会自动修正业务路径；规则写错会表现为后端路由不存在，而不是浏览器跨域。

3) 公司测试环境使用自签名的https服务，本地代理转发时证书校验失败；把secure设为false是否适合直接带到正式部署？

答：在明确是开发测试证书的前提下，可用secure: false允许代理转发到该https服务。它降低了证书校验要求，只应作为受控开发配置；webpack-dev-server本身也仅用于开发阶段，不能替代正式环境的网关。

4) 接口通过代理后仍返回错误页面，后端网关日志显示收到的Host是localhost:3000，但它只按目标域名路由，你会如何排查？

答：检查代理规则是否设置changeOrigin: true，它会把转发请求头中的Host更新为目标地址。若路径也带有/api前缀，还要同时核对pathRewrite；两者分别处理主机头和路径，不能相互替代。

5) 上线评审中有人提议把devServer.proxy配置原样用于生产跨域治理，你会接受吗？

答：不会，webpack-dev-server的代理是为本地开发效率提供的中间服务器，只适用于开发阶段。生产环境仍需部署独立的反向代理或由服务端处理跨域策略；否则前端构建结束后并不存在这台开发代理服务器。

660. 介绍一下 babel原理

Babel的核心流程是把源码解析成AST，修改这棵语法树，再生成目标代码。以ES6转ES5为例，解析器先识别源码结构，插件随后遍历AST，把需要兼容的语法节点转换成新的节点。最后，代码生成器根据转换后的AST输出ES5代码。简单来说，插件决定“改什么”，解析和生成阶段负责“读懂代码”和“重新写出代码”。babel的编译过程分为三个阶段：parsing、transforming、generating，以ES6编译为ES5作为例子：ES6代码输入；babylon进行解析得到AST；plugin用babel-traverse对AST树进行遍历编译，得到新的AST树；用babel-generator通过AST树生成ES5代码。Babel原理及其使用(opens new window)

1) 一个页面源码使用箭头函数，交给Babel后输出变成普通函数；有人说这是直接做字符串替换，你如何用编译链路反驳？

答：它不是针对源码文本做简单替换，而是先由解析器把代码转换为AST。插件遍历并改写树上的语法节点，再由生成器根据新AST输出目标代码，因此转换依据是语法结构而非字符形状。

2) 团队要写一个插件，把项目中的某类调用统一改写；在Babel的三个阶段里，核心逻辑应该放在哪里？

答：核心逻辑应位于transforming阶段，通过遍历AST定位目标节点并生成或替换相应节点。解析阶段只负责得到语法树，生成阶段负责把变换后的树输出为代码；若节点匹配过宽，可能改到语义不同的同名调用。

3) 构建工具准备把现有解析器替换掉，但仍沿用原来的遍历插件和代码生成器，你会要求验证什么约束？

答：要验证新解析结果的AST节点结构是否与遍历插件和生成器所期待的结构兼容。Babel的解析、转换、生成虽分阶段协作，但中间树是接口契约；节点类型或字段不一致时，转换可能漏掉代码或直接失败。

4) 某个Babel插件运行后生成了语法错误，源码解析本身没有报错，你会按什么顺序定位？

答：先保留解析得到的原始AST，再对比插件遍历前后的目标节点，确认错误是在转换阶段引入。随后检查生成器接收到的新树是否合法；源码能成功解析只证明parsing正常，不能证明插件改写或generating正确。

5)

两个团队争论兼容旧环境应改源码还是维护一套Babel转换插件，你会依据什么划分责任？

答：若差异能表达为稳定的语法树变换，可交给插件在AST层统一处理，并由生成器输出目标语法。若转换无法保持原语义，单靠parsing-transforming-generating流程也不能消除运行环境差异，必须另外评估实现边界。

661. 介绍一下Rollup

Rollup是面向ES Modules的打包器，通常更适合构建类库或框架。它生成的代码比较扁平、可读性好，并能自动移除未引用代码，因此类库产物通常更干净。相比之下，应用往往更依赖第三方模块、HMR和代码拆分，这些场景更适合功能更全面的Webpack。所以我一般把“应用用Webpack、库用Rollup”当作经验选择，而不是绝对规则。Rollup 是一款 ES Modules 打包器。它也可以将项目中散落的细小模块打包为整块代码，从而使得这些划分的模块可以更好地运行在浏览器环境或者 Node.js 环境。Rollup优势：输出结果更加扁平，执行效率更高；自动移除未引用代码；打包结果依然完全可读。缺点 加载非 ESM 的第三方模块比较复杂；因为模块最终都被打包到全局中，所以无法实现HMR；浏览器环境中，代码拆分功能必须使用Require.js这样的AMD库

我们发现如果我们开发的是一个应用程序，需要大量引用第三方模块，同时还需要 HMR 提升开发体验，而且应用过大就必须分包。那这些需求 Rollup 都无法满足。如果我们是开发一个 JavaScript 框架或者库，那这些优点就特别有必要，而缺点呢几乎也都可以忽略，所以在很多像 React 或者 Vue 之类的框架中都是使用的 Rollup 作为模块打包器，而并非 Webpack 总结一下：Webpack 大而全，Rollup 小而美。在对它们的选择上，我的基本原则是：应用开发使用 Webpack，类库或者框架开发使用 Rollup。不过这并不是绝对的标准，只是经验法则。因为 Rollup 也可用于构建绝大多数应用程序，而 Webpack 同样也可以构建类库或者框架。

1) 订单页提交参数缺失，接口仍返回200，只在响应体里写{"code": 400}，网关和监控都记录成功；你会接受这种约定吗？

答：不接受把所有结果都包装成200，因为HTTP层已经无法表达请求是否成功。参数错误应返回400，业务错误码再放进body细分原因；否则只观察状态码的代理、监控和调用方会得到失真的成功率。

2) 下载页要支持大文件暂停后继续，前端带上范围请求后，服务端仍返回完整文件和200；这里应该怎样落地？

答：服务端应在确认支持范围传输后返回206，并只发送客户端请求的那一段内容，而不是继续用200返回完整资源。若服务端无法正确处理范围请求，就不应伪装成断点续传，否则客户端可能重复接收或错误拼接数据。

3) 支付表单通过POST提交后需要临时跳转，产品要求“直接用302就行，还要保证下一跳仍是POST”；你会怎么选？

答：不能用302承诺保留POST，因为它通常会被实现为转向GET，但并不保证原方法得到保留。若下一跳必须继续使用POST，应选择307；若明确希望任意请求转为GET，则使用303，两者语义不能混用。

4) 用户访问管理后台得到空白页，抓包显示先返回401，登录后重试又返回403；你如何判断这两次失败分别发生了什么？

答：第一次401表示请求方尚未完成登录或身份认证，前端应进入登录流程，而不是展示无权限页面。第二次403表示身份已经确认但没有资源权限，应停止重复登录并提示授权问题；把两者统一处理会造成

成循环跳转或误导用户。

5)

线上接口经代理后返回502，运维却把它当成应用代码抛出的500；你会怎样缩小排查范围？

答：502表示充当网关或代理的服务器处理上游响应时出错，应先检查代理到源站的链路及上游响应。500才直接指向服务器内部无法完成请求；若代理等待上游超过时限，则更接近504，不能仅凭页面文案归因。

6) 新闻详情页带条件请求后收到304，前端同学认为“不是2xx就算失败”并展示错误页；这个判断错在哪里？

答：304表示资源自上次请求后未修改，客户端应继续使用已有副本，而不是把它当作业务失败。它属于重定向类状态，但承担缓存协商语义；前提是客户端确实持有可用缓存，否则无法仅靠该响应恢复正文。

662. 介绍一下Rollup

Rollup是面向ES Modules的打包器，通常更适合构建类库或框架。它生成的代码比较扁平、可读性好，并能自动移除未引用代码，因此类库产物通常更干净。相比之下，应用往往更依赖第三方模块、HMR和代码拆分，这些场景更适合功能更全面的Webpack。所以我一般把“应用用Webpack、库用Rollup”当作经验选择，而不是绝对规则。Rollup 是一款 ES Modules 打包器。它也可以将项目中散落的细小模块打包为整块代码，从而使得这些划分的模块可以更好地运行在浏览器环境或者 Node.js 环境。Rollup优势：输出结果更加扁平，执行效率更高；自动移除未引用代码；打包结果依然完全可读。缺点 加载非 ESM 的第三方模块比较复杂；因为模块最终都被打包到全局中，所以无法实现HMR；浏览器环境中，代码拆分功能必须使用Require.js这样的AMD库

我们发现如果我们开发的是一个应用程序，需要大量引用第三方模块，同时还需要 HMR 提升开发体验，而且应用过大就必须分包。那这些需求 Rollup 都无法满足。如果我们是开发一个 JavaScript 框架或者库，那这些优点就特别有必要，而缺点呢几乎也都可以忽略，所以在很多像 React 或者 Vue 之类的框架中都是使用的 Rollup 作为模块打包器，而并非 Webpack 总结一下：Webpack 大而全，Rollup 小而美。在对它们的选择上，我的基本原则是：应用开发使用 Webpack，类库或者框架开发使用 Rollup。不过这并不是绝对的标准，只是经验法则。因为 Rollup 也可用于构建绝大多数应用程序，而 Webpack 同样也可以构建类库或者框架。

1) 订单页提交参数缺失，接口仍返回200，只在响应体里写{"code": 400}，网关和监控都记录成功；你会接受这种约定吗？

答：不接受把所有结果都包装成200，因为HTTP层已经无法表达请求是否成功。参数错误应返回400，业务错误码再放进body细分原因；否则只观察状态码的代理、监控和调用方会得到失真的成功率。

2) 下载页要支持大文件暂停后继续，前端带上范围请求后，服务端仍返回完整文件和200；这里应该怎样落地？

答：服务端应在确认支持范围传输后返回206，并只发送客户端请求的那一段内容，而不是继续用200返回完整资源。若服务端无法正确处理范围请求，就不应伪装成断点续传，否则客户端可能重复接收或错误拼接数据。

3) 支付表单通过POST提交后需要临时跳转，产品要求“直接用302就行，还要保证下一跳仍是POST”；你会怎么选？

答：不能用302承诺保留POST，因为它通常会被实现为转向GET，但并不保证原方法得到保留。若下一跳必须继续使用POST，应选择307；若明确希望任意请求转为GET，则使用303，两者语义不能混用。

4) 用户访问管理后台得到空白页，抓包显示先返回401，登录后重试又返回403；你如何判断这两次失败分别发生了什么？

答：第一次401表示请求方尚未完成登录或身份认证，前端应进入登录流程，而不是展示无权限页面。第二次403表示身份已经确认但没有资源权限，应停止重复登录并提示授权问题；把两者统一处理会造成循环跳转或误导用户。

5)

线上接口经代理后返回502，运维却把它当成应用代码抛出的500；你会怎样缩小排查范围？

答：502表示充当网关或代理的服务器处理上游响应时出错，应先检查代理到源站的链路及上游响应。500才直接指向服务器内部无法完成请求；若代理等待上游超过时限，则更接近504，不能仅凭页面文案归因。

6) 新闻详情页带条件请求后收到304，前端同学认为“不是2xx就算失败”并展示错误页；这个判断错在哪里？

答：304表示资源自上次请求后未修改，客户端应继续使用已有副本，而不是把它当作业务失败。它属于重定向类状态，但承担缓存协商语义；前提是客户端确实持有可用缓存，否则无法仅靠该响应恢复正文。

【HTTP】

663. HTTP前生今世

HTTP的演进主线是不断完善能力，并逐步改善传输性能。HTTP/0.9只是能获取文本资源的简单文本协议，HTTP/1.0奠定了大部分现代技术，但并不是正式标准。HTTP/1.1功能更完善，也是目前使用最广泛的版本；HTTP/2基于SPDY，重点改善性能。再往后，HTTP/3基于QUIC，代表协议后续的发展方向。

HTTP协议始于三十年前蒂姆·伯纳斯·李的一篇论文HTTP/0.9是个简单的文本协议，只能获取文本资源；HTTP/1.0确立了大部分现在使用的技术，但它不是正式标准；HTTP/1.1是目前互联网上使用最广泛的协议，功能也非常完善；HTTP/2基于 Google 的SPDY协议，注重性能改善，但还未普及；HTTP/3基于 Google 的QUIC协议，是将来的发展方向

1) 有人看见接口返回文本，就断言它使用的是HTTP/0.9；面对现代商品详情页的请求，你会怎样反驳？

答：响应内容是文本不能证明协议版本，因为后续HTTP版本同样能够传输文本资源。HTTP/0.9的关键限制是协议非常简单且只能获取文本资源；判断现代请求所用版本，应依据实际协商或网络记录，而不是资源外观。

2) 团队维护一个同时承载页面、图片和接口的站点，架构评审只写“使用HTTP”却没有版本信息；这会遗漏什么工程判断？

答：版本不能省略，因为HTTP/1.1、HTTP/2与HTTP/3的能力基础和性能方向并不相同。应记录客户端到服务端实际采用的版本及其部署范围；仅写HTTP会让性能分析和兼容性判断缺少必要前提。

3) 旧系统负责人说HTTP/1.0已经确立了多数现用技术，因此可以把它视为完整正式标准继续扩建；你同意吗？

答：不同意把“确立多数技术”推导成“正式且适合继续扩建”，资料明确指出HTTP/1.0并非正式标准。评估旧系统时可以承认其历史贡献，但新方案仍要根据现行部署能力选型，不能只凭版本年代或名

称下结论。

4) 主站升级后仍然加载缓慢，项目经理认定“已经写了HTTP/2，协议层就不可能有问题”；你会先核实什么？

答：先核实请求链路是否真的使用HTTP/2，以及页面涉及的各个资源和服务是否都落在同一部署条件下。HTTP/2源自SPDY并侧重性能改善，但写进方案不等于已经普及或生效；协议之外的服务处理也仍可能成为瓶颈。

5) 基础设施选型会上，一方要求所有业务立即切到HTTP/3，理由是它代表未来；另一方坚持永远保留HTTP/1.1，你怎么裁决？

答：两种绝对结论都站不住：HTTP/1.1功能完善且使用广泛，HTTP/3基于QUIC，代表后续发展方向。应按客户端、服务器和中间链路的实际支持情况渐进选择，并保留可工作的兼容路径；不能把趋势直接当成现状。

6) 新人把HTTP/2和HTTP/3都说成“Google提出的同一个加速协议改名”，在技术评审里你会怎样纠正？

答：两者来源不同：HTTP/2基于Google的SPDY，而HTTP/3基于Google的QUIC。它们都关注HTTP的演进不代表底层依据相同；若把二者混为一谈，后续讨论传输基础、部署条件和故障边界都会失去准确前提。

664. HTTP世界全览

HTTP是连接浏览器、服务器和网络中间节点的应用层传输协议。浏览器和爬虫都可以作为User Agent发起请求，服务器负责应答，CDN与代理位于传输途中，可承担缓存加速、负载均衡等工作。访问资源时，DNS先把域名映射成IP，再通过由协议名、主机名和路径组成的URI定位资源。普通HTTP通常运行在TCP/IP的可靠传输之上，需要安全传输时，则使用由HTTP、SSL/TLS和TCP/IP组成的HTTPS。

互联网上绝大部分资源都使用HTTP协议传输；浏览器是HTTP协议里的请求方，即User Agent；服务器是HTTP协议里的应答方，常用的有Apache和Nginx；CDN位于浏览器和服务器之间，主要起到缓存加速的作用；爬虫是另一类User Agent，是自动访问网络资源的程序。TCP/IP是网络世界最常用的协议，HTTP通常运行在TCP/IP提供的可靠传输基础上DNS域名是IP地址的等价替代，需要用域名解析实现到IP地址的映射；URI是用来标记互联网上资源的一个名字，由“协议名 + 主机名 + 路径”构成，俗称URL；HTTPS相当于“HTTP+SSL/TLS+TCP/IP”，为HTTP套了一个安全的外壳；代理是HTTP传输过程中的“中转站”，可以实现缓存加速、负载均衡等功能

1) SEO爬虫抓取商品页后，开发说“它不是浏览器，所以不属于HTTP请求方”；你会怎么纠正这个角色判断？

答：爬虫同样属于User Agent，只是它通过程序自动访问网络资源，而不是由用户操作浏览器。服务器仍作为HTTP应答方处理请求；若按“只有浏览器才是请求方”设计策略，会漏掉爬虫这一类合法或需治理的客户端。

2) 图片站准备在浏览器与Nginx源站之间接入CDN，业务方问它除了多一跳还有什么价值；你怎样说明落地边界？

答：CDN位于浏览器和源站之间，主要价值是缓存资源并加速访问，因此适合承担可缓存内容的分发。它仍是传输链路中的中间节点，并不会替代源站处理所有请求；缓存规则不合适时，请求仍需回到服务器。

3) 内网服务迁移后保留原域名，只更换后端IP，有人认为页面里的URI没变就无需关注DNS；这成立吗？

答：不成立，域名需要经DNS解析映射到IP，后端地址变化仍要求解析结果正确指向新地址。URI负责标记资源，通常由协议名、主机名和路径构成；名称保持不变，并不意味着名称到网络地址的映射会自动正确。

4) 用户报告https://页面完全连不上，应用日志也没有请求；值班同学只检查业务路由，你会如何沿链路排查？

答：应用没有收到请求时，应先检查域名能否经DNS解析到预期IP，再核对网络传输和中间代理是否可达。HTTPS还在HTTP外增加了SSL/TLS安全层；在这些前置环节确认之前，直接归因于应用路由证据不足。

5) 高峰期需要在浏览器与多台应用服务器之间增加一层设施，团队在“只做缓存”与“同时负载均衡”之间争论；代理能承担哪些职责？

答：代理作为HTTP传输中的中转站，既可以用于缓存加速，也可以承担负载均衡，因此两种诉求并不天然冲突。具体是否同时启用取决于链路设计和代理能力；增加中转层也意味着请求路径变长，配置错误会影响源站可达性。

6)

安全评审中有人把HTTPS描述成“完全替换了HTTP和TCP/IP的新协议栈”，你会怎样校正？

答：HTTPS更准确的关系是HTTP + SSL/TLS + TCP/IP，即为HTTP增加安全外壳，而不是把HTTP与底层网络全部替换。HTTP通常仍依赖TCP/IP提供的可靠传输；混淆这些角色会导致证书、应用响应和网络故障被错误归层。

665. HTTP分层

网络分层的核心价值是职责隔离，上层使用下层能力时不必看到逐层传输的细节。TCP/IP分为四层，IP位于网际层，TCP位于传输层，HTTP位于应用层；映射到七层OSI模型时，TCP在第四层，HTTP在第七层。数据发送和接收时，会沿协议栈逐层打包再逐层拆包。日常交流常说“四层”和“七层”，可以粗略理解为操作系统负责的通常在四层或以下，而应用程序处理的通常在七层，但这不是绝对判断。第一层：物理层，TCP/IP里无对应；第二层：数据链路层，对应TCP/IP的链接层；第三层：网络层，对应TCP/IP的网际层；第四层：传输层，对应TCP/IP的传输层；第五、六、七层：统一对应到TCP/IP的应用层。总结 TCP/IP分为四层，核心是二层的IP和三层的TCP，HTTP在第四层；OSI分为七层，基本对应TCP/IP，TCP在第四层，HTTP在第七层；OSI可以映射到TCP/IP，但这期间一、五、六层消失了；日常交流的时候我们通常使用OSI模型，用四层、七层等术语；HTTP利用TCP/IP协议栈逐层打包再拆包，实现了数据传输，但下面的细节并不可见有一个辨别四层和七层比较好的（但不是绝对的）小窍门，“两个凡是”：凡是由操作系统负责处理的就是四层或四层以下，否则，凡是需要由应用程序（也就是你自己写代码）负责处理的就是七层

1) 评审会上有人说“HTTP在第四层，因为它运行在TCP/IP上”，另一人说“它在第七层”；面对这场术语冲突你怎么裁定？

答：两种说法采用了不同模型：在四层TCP/IP模型中，HTTP位于应用层，也就是第四层；在七层OSI模型中，它位于第七层。交流时必须先说明模型，否则层号相同或不同都可能造成假冲突。

2) 前端页面请求经过网卡、网络和服务后得到JSON，业务同学问数据为何能跨网络传输；你会怎样用分层解释而不陷入实现细节？

答：HTTP数据会沿TCP/IP协议栈逐层封装，经链路传输后在接收端逐层拆包，最终交给应用处理。HTTP使用底层能力，但其应用代码通常看不到下层细节；分层的价值正是隔离职责，而不是让业务直接操纵每一层。

3) 客户端从浏览器换成操作系统内置网络组件后，负责人断言所有逻辑都从七层降到了四层；这个判断为什么不可靠？

答：是否由操作系统处理只能作为辨别四层与七层的经验线索，并非绝对规则。TCP在OSI第四层，HTTP仍属于应用层；实现代码放在哪里不会自动改变协议职责，必须结合处理的实际协议内容判断。

4) 线上接口无法建立连接，但应用服务日志完全为空；开发立即修改HTTP路由，你会按分层把排查重点放在哪里？

答：应用日志没有请求时，不宜先认定是HTTP路由错误，应先检查链接层、网络层和传输层是否完成了数据送达。只有下层传输成立后，HTTP才能在应用层处理请求；不过分层只能缩小范围，最终仍需结合各层证据定位。

5) 网关团队计划只依据端口转发连接，业务团队却要求按HTTP路径分流；这两个能力分别更接近哪一层？

答：依据连接与传输信息工作的能力更接近四层，而解析HTTP路径后分流需要应用程序理解七层协议。后者能做更细的业务路由，但也承担HTTP解析和配置成本；若只做四层转发，就无法直接使用路径语义决策。

6) 新人画协议栈时把OSI的会话层、表示层逐一对应成TCP/IP的独立层，还把TCP放到了第三层；你会怎样修正？

答：OSI的第五、六、七层在TCP/IP中统一归入应用层，并不存在逐层独立对应；TCP位于OSI第四层，对应TCP/IP的传输层。映射是概念上的归并，不是编号平移，否则会误判协议职责。

666. HTTP报文是什么样子的

HTTP请求报文和响应报文结构基本一致，都由起始行、头部字段、空行和消息正文组成。起始行负责说明请求或响应的基本信息，也是两者最明显的区别；头部则用key-value形式补充细节。空行用于分隔头部和正文，正文才是实际传输的文本、图片、视频等数据，而且正文并不一定存在。 HTTP协议的请求报文和响应报文的结构基本相同，由三大部分组成 起始行（start line）：描述请求或响应的基本信息；头部字段集合（header）：使用key-value形式更详细地说明报文；消息正文（entity）：实际传输的数据，它不一定是纯文本，可以是图片、视频等二进制数据 这其中前两部分起始行和头部字段经常又合称为“请求头”或“响应头”，消息正文又称为“实体”，但与“header”对应，很多时候就直接称为“body”。一个完整的 HTTP 报文就像是下图的这个样子，注意在 header 和 body 之间有一个“空行”

1) 抓包里请求只有起始行和若干头部字段，没有JSON正文，同事因此认定它不是完整的HTTP报文；你认可吗？

答：不认可，HTTP报文由起始行、头部字段集合和可选的消息正文构成，正文并非每次都必须承载数据。没有body不能单独证明报文不完整；应继续检查起始行、header以及两者之后的报文边界是否符合格式。

2) 上传接口既要发送文件二进制，也要附带描述信息，产品担心HTTP正文只能放纯文本；你会怎样设计认知边界？

答：消息正文可以承载图片、视频等二进制数据，并不限于纯文本，因此文件能够放入body传输。头部字段负责补充说明报文，实际编码和内容解释仍需双方约定；仅证明能传二进制，不代表任意格式都能被服务端识别。

3) 团队日志把起始行和全部header统称为“请求头”，新人随后误以为起始行也是一个key-value字段；你怎么纠正？

答：工程交流中确实常把起始行与头部字段合称为“请求头”或“响应头”，但结构上两者仍不同。起始行描述请求或响应的基本信息，只有header使用key-value形式；日志展示可以合并，解析规则不能混淆。

4) 线上代理转发后，服务端把正文首行当成header，接口持续报格式错误；查看原始报文时你首先核对什么？

答：首先核对header与body之间是否保留了规定的空行，因为它承担两部分的分隔作用。若代理改写报文时丢失这一边界，接收端就可能继续按头部解析正文；还应比较代理前后的原始报文，避免只查业务字段。

5) 埋点接口要传一批事件，架构师要求把所有事件内容拆成自定义header，理由是“header和body都能携带数据”；你会接受吗？

答：不应把实际业务载荷全部塞进header，头部字段的职责是以key-value形式说明报文，事件数据更符合消息正文的定位。即使某些值技术上能写入header，也会模糊元信息与实体边界，并增加各节点解析约定的复杂度。

6) 调试工具只展示“请求头”和“响应体”两个面板，实习生据此认为请求报文与响应报文结构完全不同；你会如何串联两者？

答：请求报文和响应报文的基本结构相同，都包含起始行、头部字段集合以及可能存在的消息正文。工具只是按调试习惯拆分展示，不能替代协议结构；分析异常时仍要分别核对起始行、header、空行和body。

667. HTTP之URL

URL本质上是标记资源位置的字符串，通常由scheme、host:port、path和query组成。scheme决定用什么协议访问，主机和端口定位服务器，path定位资源，query再附加具体要求。部分结构可以省略，但像@、&、/这类特殊字符以及汉字需要正确编码，否则服务器可能无法正确解析报文。

URI是用来唯一标记服务器上资源的一个字符串，通常也称为URL；URI通常由scheme、host:port、path和query四个部分组成，有的可以省略；scheme叫“方案名”或者“协议名”，表示资源应该使用哪种协议来访问；“host:port”表示资源所在的主机名和端口号；path标记资源所在的位置；query表示对资源附加的额外要求；在URI里对“@&/”等特殊字符和汉字必须要做编码，否则服务器收到HTTP报文后会无法正确处理

1) 商品列表页把筛选条件写成/search?keyword=手机&配件&page=1，后端拿到的关键词只剩“手机”，前端同学却说汉字能显示就不用编码，你怎么判断？

答：参数边界已经被&破坏，汉字是否可见不能证明URI合法。应使用URLSearchParams，或仅对参数值调用encodeURIComponent，让特殊字符和汉字按规则编码；不要对整条URL再编码，否则scheme、host和分隔符也会失去结构含义。

2) 订单详情页要生成/orders/用户输入值?from=售后#记录，开发直接把两个动态值拼进字符串；你会要求怎样拆分和编码？

答：先区分path、query等URI组成部分，再分别处理动态数据，不能把用户输入当作结构字符直接拼接。查询参数交给URLSearchParams，路径段应单独编码；若把整段内容统一处理，输入中的/、&或#可能改变资源位置、参数边界或片段含义。

3) 搜索页既要支持同名标签tag=前端&tag=性能，又要让链接可复制分享，评审中有人主张把数组先用逗号连接，你选哪种实现？

答：优先用URLSearchParams.append保留同名多值，使每个值独立编码，复制后的URI仍能明确表达多个筛选条件。逗号拼接只有在前后端明确约定且值本身不会引起歧义时才可用，否则解析规则会混入业务层，后续扩展和兼容成本更高。

4) 线上跳转链接偶发落到错误页面，日志里出现参数值包含%252F，而原始输入只是/docs/api；你会先查哪一层？

答：%252F通常意味着%2F中的百分号又被编码了一次，应沿着表单提交、路由构造、请求封装和服务端解析逐层确认编码次数。原则是动态值在进入对应URI组件时编码一次、解析一次；重复调用编码函数或对已生成的完整URL再处理都会造成双重编码。

5) 报表页需要保存大量组合筛选，产品要求地址栏可分享，前端提议把全部条件都塞进query；你如何在URI可读性和功能完整性之间取舍？

答：少量、稳定且适合公开的筛选条件可以放入query，因为它本来就用于表达对资源的附加要求，也便于复制链接。条件复杂时应评估浏览器、代理和服务器对请求目标的实现限制，可改为服务端保存条件并在URL中放短标识；代价是分享链接依赖后端状态及权限控制。

668. HTTP实体数据

HTTP实体数据通过内容类型、压缩方式、语言和字符集来描述，客户端与服务器用对应头字段完成内容协商。客户端用Accept表示能接收的MIME type，服务器用Content-Type说明实际返回的类型；压缩则对应Accept-Encoding和Content-Encoding。多个候选项可以用;q=设置权重，例如优先返回HTML，其次返回XML。缓存场景还要关注Vary，因为同一URI可能因协商条件不同而产生多个响应版本。

1. 数据类型与编码 text：即文本格式的可读数据，我们最熟悉的应该就是text/html了，表示超文本文档，此外还有纯文本text/plain、样式表text/css等。image：即图像文件，有image/gif、image/jpeg、image/png等。audio/video：音频和视频数据，例如audio/mpeg、video/mp4等。application：数据格式不固定，可能是文本也可能是二进制，必须由上层应用程序来解释。常见的有application/json，application/javascript、application/pdf等，另外，如果实在是不知道数据是什么类型，像刚才说的“黑盒”，就会是application/octet-stream，即不透明的二进制数据 但仅有MIME type还不够，因为HTTP在传输时为了节约带宽，有时候还会压缩数据，为了不要让浏览器继续“猜”，还需要有一个“Encoding type”，告诉数据是用的什么编码格式，这样对方才能正确解压缩，还原出原始的数据。

比起MIME type来说，Encoding type就少了很多，常用的只有下面三种 gzip：GNU zip压缩格式，也是互联网上最流行的压缩格式；deflate：zlib（deflate）压缩格式，流行程度仅次于gzip；br：一种专门为HTTP优化的新压缩算法（Brotli）

2. 数据类型使用的头字段 有了MIME type和Encoding type，无论是浏览器还是服务器就都可以轻松识别出body的类型，也就能够正确处理数据了。 HTTP协议为此定义了两个Accept请求头字段和两个Content实体头字段，用于客户端和服务端进行“内容协商”。也就是说，客户端用Accept头告诉服务器希望接收什么样的数据，而服务器用Content头告诉客户端实际发送了什么样的数据 Accept字段标记的是客户端可理解的MIMEtype，可以用“,”做分隔符列出多个类型，让服务器有更多的选择余地，例如下面的这个头：

Accept: text/html,application/xml,image/webp,image/png 这就是告诉服务器：“我能够看懂 HTML、XML的文本，还有webp和png的图片，请给我这四类格式的数据”。

相应的，服务器会在响应报文里用头字段Content-Type告诉实体数据的真实类型： Content-Type: text/html Content-Type: image/png 这样浏览器看到报文里的类型是“text/html”就知道是 HTML文件，会调用排版引擎渲染出页面，看到“image/png”就知道是一个 PNG文件，就会在页面上显示出图像。

Accept-Encoding字段标记的是客户端支持的压缩格式，例如上面说的gzip、deflate 等，同样也可以用“,”列出多个，服务器可以选择其中一种来压缩数据，实际使用的压缩格式放在响应头字段Content-Encoding里 Accept-Encoding: gzip, deflate, br Content-Encoding: gzip 不过这两个字段是可以省略的，如果请求报文里没有Accept-Encoding字段，就表示客户端不支持压缩数据；如果响应报文里没有Content-Encoding字段，就表示响应数据没有被压缩

3. 语言类型使用的头字段 同样的，HTTP 协议也使用Accept请求头字段和Content实体头字段，用于客户端和服务端就语言与编码进行“内容协商”。

Accept-Language字段标记了客户端可理解的自然语言，也允许用“,”做分隔符列出多个类型，例如： Accept-Language: zh-CN, zh, en 这个请求头会告诉服务器：“最好给我zh-CN的汉语文字，如果没有

就用其他的汉语方言，如果还没有就给英文”。

相应的，服务器应该在响应报文里用头字段Content-Language告诉客户端实体数据使用的实际语言类型
Content-Language: zh-CN

字符集在HTTP里使用的请求头字段是Accept-Charset，但响应头里却没有对应的Co...

1) 下载接口实际返回application/pdf，网关却统一改成text/plain，浏览器预览区显示乱码；业务方认为文件字节没变就不影响，你同意吗？

答：不同意，实体字节未变不代表客户端能正确处理，浏览器会依据响应的Content-Type选择解析或展示方式。服务端应如实返回application/pdf；未知二进制才适合application/octet-stream，错误标成本会导致渲染、下载行为和安全判断出现偏差。

2) 图片组件声明只接受image/webp,image/png，服务端仍返回image/jpeg；前端负责人说请求头只是提示，页面能显示就不用管，你怎么处理？

答：Accept表达客户端可理解或期望接收的媒体类型，服务端应据此协商，并用Content-Type声明最终实体类型。若服务端无法提供期望格式，应形成明确的降级或错误约定；忽略协商可能暂时可显示，但会让带宽优化、格式兼容和缓存版本失去可靠依据。

3) 静态资源请求带了Accept-Encoding: gzip, br，响应头却写Content-Encoding: br，老旧客户端解压失败；你会要求服务端怎样选择？

答：服务端只能从客户端声明支持的压缩格式中选择，并在Content-Encoding中准确标记实际采用的格式；未压缩时不应伪造该字段。若某类客户端不支持br，请求中就不应声明它，服务端也要保留gzip或无压缩回退，否则实体无法还原。

4) 国际化首页偶发把英文内容发给中文用户，CDN命中率很高，源站会根据Accept-Language返回不同实体；你从哪些响应头开始排查？

答：先核对请求的Accept-Language、响应的Content-Language，再检查缓存响应是否包含与协商维度一致的Vary: Accept-Language。同一URI可因语言协商产生多个版本，缓存若未区分该请求头就可能串内容；增加Vary会扩大缓存变体数量，需要接受命中率下降。

5) 接口团队想用Accept: application/json,text/html;q=0.8同时兼容应用和浏览器，另一个团队坚持拆成两个URL；你如何评估？

答：同一资源存在多种表示时，可以使用Accept和q值表达偏好，服务端再以Content-Type告知实际选择，并让缓存通过Vary: Accept区分版本。若两种响应的业务语义、鉴权或错误模型已经不同，拆分URI更清晰；内容协商不应掩盖接口契约差异。

669. 谈一谈HTTP协议优缺点

HTTP的优势是简单、灵活、采用请求应答模式并基于TCP/IP可靠传输，代价则是无状态、明文传输和可能出现队头阻塞。无状态让每次请求彼此独立、服务器不必天然保存会话上下文，但购物等场景又需要借助cookie和session维持状态。文本报文便于理解和扩展，也会让报文信息暴露在外。使用长连接共用一个TCP连接时，耗时较长的请求还可能阻塞后续请求。

超文本传输协议，HTTP是一个在计算机世界里专门在两点之间传输文字、图片、音频、视频等超文本数据的约定和规范。HTTP特点灵活可扩展。一个是语法上只规定了基本格式，空格分隔单词，换行分隔字段等。另外一个就是传输形式上不仅可以传输文本，还可以传输图片，视频等任意数据。请求-应答模式，通常而言，就是一方发送消息，另外一方要接受消息，或者是做出相应等。可靠传输，HTTP是基于TCP/IP，因此把这一特性继承了下来。无状态，这个分场景回答即可。HTTP 缺点无状态，有时候，需要保存信息，比如像购物系统，需要保留下顾客信息等等，另外一方面，有时候，无状态也会减少网络开销，比如类似直播行业这样子等，这个还是分场景来说。明文传输，即协议里的报文(主要指的是头部)不使用二进制数据，而是文本形式。这让HT

TP的报文信息暴露给了外界，给攻击者带来了便利。队头阻塞，当http开启长连接时，共用一个TCP连接，当某个请求时间过长时，其他的请求只能处于阻塞状态，这就是队头阻塞问题。 灵活可扩展。一个是语法上只规定了基本格式，空格分隔单词，换行分隔字段等。另外一个就是传输形式上不仅可以传输文本，还可以传输图片，视频等任意数据。请求-应答模式，通常而言，就是一方发送消息，另外一方要接受消息，或者是做出相应等。可靠传输，HTTP是基于TCP/IP，因此把这一特性继承了下来。无状态，这个分场景回答即可。 无状态，有时候，需要保存信息，比如像购物系统，需要保留下顾客信息等等，另外一方面，有时候，无状态也会减少网络开销，比如类似直播行业这样子等，这个还是分场景来说。明文传输，即协议里的报文(主要指的是头部)不使用二进制数据，而是文本形式。这让HTTP的报文信息暴露给了外界，给攻击者带来了便利。队头阻塞，当http开启长连接时，共用一个TCP连接，当某个请求时间过长时，其他的请求只能处于阻塞状态，这就是队头阻塞问题。 http 无状态无连接 http协议对于事务处理没有记忆能力对同一个url请求没有上下文关系每次的请求都是独立的，它的执行情况和结果与前面的请求和之后的请求是无直接关系的，它不会受前面的请求应答情况直接影响，也不会直接影响后面的请求应答情况服务器中没有保存客户端的状态，客户端必须每次带上自己的状态去请求服务器人生若只如初见，请求过的资源下一次会继续进行请求 http协议无状态中的 状态 到底指的是什么？！ 【状态】的含义就是：客户端和服务端在某次会话中产生的数据那么对应的【无状态】就意味着：这些数据不会被保留通过增加cookie和session机制，现在的网络请求其实是有状态的在没有状态的http协议下，服务器也一定会保留你每次网络请求对数据的修改，但这跟保留每次访问的数据是不一样的，保留的只是会话产生的结果，而没有保留会话

1) 购物车接口连续两次请求同一个URL，第二次服务端却不知道第一次选了哪些商品；产品经理据此说HTTP不可靠，你会怎样纠正？

答：这体现的是HTTP无状态，不等于传输不可靠；每个请求默认没有前一次会话的上下文，而基于TCP/IP的传输可靠性是另一层概念。购物车状态应由客户端随请求携带标识，或借助cookie与session关联；代价是系统开始承担会话保存和一致性管理。

2) 直播弹幕页面每秒持续拉取内容，架构师认为所有请求都必须在服务端保存完整会话上下文；无状态在这里是缺点吗？

答：不一定，无状态能让相互独立的请求少依赖服务器会话，从而减少不必要的状态管理和网络附带信息。只有鉴权、进度或个性化确实依赖会话数据时才需要引入状态；若把所有访问上下文都持久化，会增加存储、清理和多节点共享成本。

3) 登录系统从单机扩到多个服务节点后，用户偶发被当成未登录；当前依赖本机session，你如何定位并落地修正？

答：先确认请求是否被负载均衡到不同节点，以及cookie中的会话标识是否稳定携带；本机session无法天然被其他节点读取。可采用共享会话存储或让路由保持粘性，前者增加共享存储依赖，后者影响流量调度与故障切换，都不能改变HTTP本身无状态的事实。

4) 安全评审发现内部管理页仍用明文HTTP，负责人说报文是文本格式便于排障，而且内网没有风险；你会怎么回应？

答：文本报文便于阅读也意味着传输内容更容易暴露，不能把可调试性当作安全保障。应使用HTTPS保护传输，并结合访问控制处理内网威胁；加密会引入证书和终止节点管理，但这是避免请求头与实体被直接窥视或篡改的必要成本。

5) HTTP/1.1页面共用一条长连接，其中一个慢接口迟迟不返回，后续资源也被拖住；前端想无限增加并发连接，你会如何取舍？

答：这是长连接上的队头阻塞表现：前面的请求耗时过长时，后续请求可能等待。应先定位并治理慢接口，再评估拆分关键资源、调整连接使用或升级传输方案；盲目增加连接只能缓解局部排队，还会增加客户端、服务器和网络的连接开销。

670. 说一说HTTP 的请求方法

HTTP/1.1常见方法可按读取、写入和连接诊断来理解。GET获取资源，HEAD只取响应头；POST追加数据，PUT存储或修改资源，DELETE删除资源。OPTIONS查询资源支持的方法，也用于复杂跨域请求的预检；TRACE用于测试诊断，CONNECT用于建立代理隧道。实际使用时还要看语义，例如GET通常无副作用且幂等，POST通常会产生副作用且不幂等。

HTTP1.0定义了三种请求方法：GET, POST 和 HEAD方法HTTP1.1新增了五种请求方法：OPTIONS, PUT, DELETE, TRACE 和 CONNECT
http/1.1规定了以下请求方法(注意，都是大写)： GET： 请求获取Request-URI所标识的资源POST： 在Request-URI所标识的资源后附加新的数据HEAD：

请求获取由Request-URI所标识的资源的响应消息报头PUT：

请求服务器存储一个资源，并用Request-URI作为其标识（修改数据）DELETE：

请求服务器删除对应所标识的资源TRACE：

请求服务器回送收到的请求信息，主要用于测试或诊断CONNECT：

建立连接隧道，用于代理服务器OPTIONS： 列出可对资源实行的请求方法，用来跨域请求

从应用场景角度来看，Get 多用于无副作用，幂等的场景，例如搜索关键字。Post

多用于副作用，不幂等的场景，例如注册。 options 方法有什么用 OPTIONS 请求与 HEAD

类似，一般也是用于客户端查看服务器的性能。这个方法会请求服务器返回该资源所支持的所有 HTTP

请求方法，该方法会用'*'来代替资源名称，向服务器发送 OPTIONS

请求，可以测试服务器功能是否正常。JS 的 XMLHttpRequest对象进行 CORS

跨域资源共享时，对于复杂请求，就是使用 OPTIONS

方法发送嗅探请求，以判断是否有对指定资源的访问权限。

1) 搜索页把“按关键词查询”设计成POST /search，接口没有修改任何资源；后端说只要能返回数据，请求方法无所谓，你会怎么评审？

答：该操作无副作用且期望重复执行结果语义一致，更符合GET的获取资源语义，也便于浏览器按常规方式缓存和表达查询地址。POST并非不能返回搜索结果，但会弱化方法传递的意图；只有请求条件不适合放入URI等约束成立时，才值得接受这种取舍。

2) 用户资料页用PUT /users/42提交完整资料，网络超时后客户端准备自动重试；为什么这里不能只看“都是修改数据”？

答：PUT的目标是让指定URI对应资源达到请求描述的状态，重复提交相同内容应保持相同资源状态，因此具备幂等语义。重试前仍要判断响应是否丢失以及服务端是否夹带非幂等副作用，例如重复发通知；方法语义正确不代表实现天然安全。

3) 监控平台想探测一个大文件是否存在并读取响应头，但不希望下载文件正文；有人建议先GET再由前端中断，你会选什么？

答：应优先使用HEAD，它用于获取与目标资源相关的响应报头而不传输实体正文，更符合探测元数据的目的。服务端必须正确实现其语义，并保证关键响应头与对应GET一致；若服务端不支持或实现错误，才需要受控地退回其他探测方式。

4) 跨域上传在正式环境失败，浏览器控制台显示OPTIONS返回405，但直接用命令行调用POST正常；你会沿着什么链路排查？

答：浏览器对复杂跨域请求会先发送OPTIONS预检，命令行成功不能证明浏览器具备跨域访问权限。应检查路由、网关和服务端是否允许OPTIONS，以及响应是否声明允许的来源、方法和请求头；仅放通实际POST仍会被浏览器拦截。

5) 公司代理需要访问外部HTTPS服务，网络团队提出开放CONNECT，安全团队担心它变成任意隧道；你如何界定方案？

答：CONNECT用于让客户端通过代理建立连接隧道，确实适合代理转发加密流量，但不能无条件开放。代理应限制可达目标、端口、身份和审计范围；限制过严会破坏合法访问，限制过松则可能绕过现有网络策略，这属于代理治理而非前端调用细节。

671. 谈一谈GET 和 POST 的区别

GET和POST的本质区别是语义：前者获取资源，后者提交资源。因为用途不同，GET通常是幂等的，也更容易被浏览器缓存；POST默认不缓存，重复提交还可能改变资源状态。参数一般分别放在URL和请求体中，但这不代表POST天然安全，抓包时两者都能被看到。至于报文如何拆成TCP数据包，不能简单当作两种方法的固定区别。本质上，只是语义上的区别，GET 用于获取资源，POST 用于提交资源。具体差别从缓存角度看，GET 请求后浏览器会主动缓存，POST 默认情况下不能。从参数角度来看，GET请求一般放在URL中，因此不安全，POST请求放在请求体中，相对而言较为安全，但是在抓包的情况下都是一样的。从编码角度看，GET请求只能经行URL编码，只能接受ASCII码，而POST支持更多的编码类型且不对数据类型限值。GET请求幂等，POST请求不幂等，幂等指发送 M 和 N 次请求（两者不相同且都大于1），服务器上资源的状态一致。GET请求会一次性发送请求报文，POST请求通常分为两个TCP数据包，首先发 header 部分，如果服务器响应 100(continue)，然后发 body 部分。

1) 登录页负责人说把密码从GET改成POST就已经安全，因为地址栏看不到参数；安全评审会上你会怎么反驳？

答：POST只是把数据放入请求体，并不会自动加密，抓包时仍可看到明文内容，传输安全必须依赖HTTPS。它能减少参数直接出现在URL、历史记录或常见访问日志中的机会，但服务端日志和调试工具仍可能记录请求体，敏感数据还需最小化留存。

2) 商品查询接口使用GET，筛选条件变化不频繁，前端却每次都强制绕过缓存；从方法语义和浏览器行为看，你会怎么改？

答：GET用于获取资源且具备幂等语义，浏览器默认更容易对其结果进行缓存，应结合响应缓存策略复用稳定查询结果。是否真正缓存仍由响应头和中间节点决定；若结果与用户身份或实时库存相关，就要设置正确的缓存边界，不能只凭方法放开共享缓存。

3) 报表查询条件包含嵌套对象，序列化后不适合放进URL，团队准备改成POST；产品又要求链接可收藏，你如何取舍？

答：复杂条件放入请求体时使用POST是可接受的工程折中，但会失去GET查询地址天然可分享、默认易缓存的优势。可把条件保存为服务端资源并返回短标识，再用GET访问该标识；代价是增加存储、权限、过期清理和链接生命周期管理。

4) 支付确认接口使用POST，客户端超时后自动重试，线上出现重复处理；开发认为网络层会保证一次请求只执行一次，你会如何排查？

答：POST通常具有副作用且不保证幂等，超时只能说明客户端没收到结果，不能证明服务端没有执行。应先用业务请求标识核对服务端处理记录，再在支付侧实现幂等校验；依赖传输可靠性无法消除重试造成的重复业务动作。

5) 抓包发现一个小型POST请求并未先收到100

Continue，而是请求头和请求体一起发送；同事据此判断客户端违反协议，你认可吗？

答：不认可，POST并不必然拆成两个TCP数据包，是否等待100 Continue取决于客户端是否使用相应机制、请求大小和具体实现。TCP分段也不能直接等同于HTTP语义边界；排查应查看请求头是否包含Expect: 100-continue，不能用单次抓包概括所有POST。

672. 谈一谈队头阻塞问题

队头阻塞就是同一任务队列串行处理请求时，队首请求过慢，导致后续请求只能等待。在一个域名只使用少量长连接的情况下，每条连接都像一条独立队列，因此增加并发连接可以分散阻塞风险。域名分片也是类似思路：把资源放到多个二级域名，让客户端建立更多连接并行处理。不过这类方案本质上是增加队列数量，并没有让单条队列摆脱慢请求的影响。什么是队头阻塞？对于每一个HTTP请求而言，这些任务是被放入一个任务队列中串行执行的，一旦队首任务请求太慢时，就会阻塞后面的请求处理，这就是HTTP队头阻塞问题。有什么解决办法吗 并发连接 我们知道对于一个域名而言，是允许分配多个长连接的，那么可以理解成增加了任务队列，也就是说不会导致一个任务阻塞了该任务队列的其他任务，在RFC规范中规定客户端最多并发2个连接，不过实际情况就是要比这个还要多，举个例子，Chrome中是6个。域名分片 顾名思义，我们可以在一个域名下分出多个二级域名出来，而它们最终指向的还是同一个服务器，这样的话就可以并发处理的任务队列更多，也更好的解决了队头阻塞的问题。举个例子，比如TianTian.com，可以分出很多二级域名，比如Day1.TianTian.com，Day2.TianTian.com,Day3.TianTian.com,这样子就可以有效解决队头阻塞问题。

1) 商品详情页同时请求价格、库存和推荐列表，推荐接口最先入队却响应很慢，后面的价格请求一定会被它阻塞吗？

答：不一定，只有这些请求落在同一条串行处理队列中，队首的慢请求才会挡住后续请求。若浏览器已为该域名建立多条连接，请求可能分配到不同队列；但同一连接内的排队关系仍然存在。

2) 一个HTTP/1.1页面首屏要加载几十个静态资源，瀑布图里大量请求长期处于排队状态，前端和服务端分别能做什么？

答：可以让浏览器对同一域名使用多条长连接，相当于增加并行处理队列，避免所有资源都被一个慢请求拖住。服务端还应缩短队首请求的处理时间；连接数并非无限，资源过多时仍会排队。

3) 团队准备把图片拆到img1.example.com、img2.example.com，但这些二级域名最终都指向同一台服务器，这样还能缓解阻塞吗？

答：仍可能缓解，因为浏览器按域名分配连接，多个二级域名能够扩展可并发的请求队列，即使它们最终指向同一服务器。代价是域名、连接与运维复杂度增加，后端处理能力不足时也只是把等待位置后移。

4) 线上活动页只有某个域名下的资源成批卡住，其他域名资源正常，抓包又没有发现所有接口都变慢，你会怎样定位？

答：先按域名和连接查看请求瀑布图，确认卡住的资源是否排在同一慢请求之后，再定位队首请求的服务端耗时。若换到另一连接后请求立即恢复，更符合队头阻塞；若所有连接都慢，应继续排查服务器容量或网络。

5) 架构师主张无限增加并发连接，前端负责人主张域名分片；面对大量小资源，你会怎样取舍？

答：两种方案本质上都在增加请求队列，但连接数受浏览器策略约束，不能无限扩张；域名分片则用多个二级域名换取更多并发。应先控制资源数量和慢请求，再谨慎分片，否则会引入额外连接与域名管理成本。

673. 谈一谈HTTP数据传输

HTTP/1.1传输数据时，定长内容通常用Content-Length，长度无法预先确定时使用Transfer-Encoding: chunked。Content-Length必须与实际传输长度一致，过短会截断，过长可能导致接收方继续等待；使用压缩时填写的是压缩后的长度。分块传输适合长连接中持续产生的动态内容，接收方根据各个分块判断数据边界。如果两者同时出现，应按Transfer-Encoding处理并忽略Content-Length。

大概遇到的情况就分为定长数据与不定长数据的处理吧。

定长数据

对于定长的数据包而言，发送端在发送数据的过程中，需要设置Content-Length,来指明发送数据的长度。当然了如果采用了Gzip压缩的话，Content-Length设置的就是压缩后的传输长度。

我们还需要知道的是 Content-Length如果存在并且有效的话，则必须和消息内容的传输长度完全一致，也就是说，如果过短就会截断，过长的话，就会导致超时。如果采用短链接的话，直接可以通过服务器关闭连接来确定消息的传输长度。那么在HTTP/1.0之前的版本中，Content-Length字段可有可无,因为一旦服务器关闭连接，我们就可以获取到传输数据的长度了。在HTTP/1.1版本中，如果是Keep-alive的话，chunked优先级高于Content-Length,若是非Keep-alive，跟前面情况一样，Content-Length可有可无。

那怎么来设置Content-Length

举个例子来看看

```
const server = require('http').createServer(); server.on('request', (req, res) => { if (req.url === '/index') { // 设置数据类型 res.setHeader('Content-Type', 'text/plain'); res.setHeader('Content-Length', 10); res.write("你好，使用的是Content-Length设置传输数据形式"); } }) server.listen(3000, () => { console.log("成功启动--Tina Tian"); })
```

不定长数据 现在采用最多的就是HTTP/1.1版本，来完成传输数据，在保存Keep-alive状态下，当数据是不定长的时候，我们需要设置新的头部字段

Transfer-Encoding: chunked

通过chunked机制，可以完成对不定长数据的处理，当然了，你需要知道的是 如果头部信息中有Transfer-Encoding, 优先采用Transfer-Encoding里面的方法来找到对应的长度。如果设置了Transfer-Encoding，那么Content-Length将被忽视。使用长连接的话，会持续的推送动态内容。

那我们来模拟一下吧

```
const server = require('http').createServer(); server.on('request', (req, res) => { if (req.url === '/index') { // 设置数据类型 res.setHeader('Content-Type', 'text/html; charset=utf8'); res.setHeader('Content-Length', 10); res.setHeader('Transfer-Encoding', 'chunked'); res.write("你好，使用的是Transfer-Encoding设置传输数据形式"); setTimeout(() => { res.write("第一次传输数据给您"); }, 1000); res.write("骚等一下"); setTimeout(() => { res.write("第一次传输数据给您"); res.end(); }, 3000); }) server.listen(3000, () => { console.log("成功启动--Tina Tian"); })
```

上面使用的是nodejs中http模块，有兴趣的小伙伴可以去试一试，以上就是HTTP对定长数据和不定长数据传输过程中的处理手段。

1) 下载页收到响应头Content-Length:

10，服务端实际却继续写入更长的正文，前端为什么不能把多出的内容当成下一段数据？

答：有效的Content-Length必须与消息的传输长度一致，接收方会按该长度划定当前消息边界，长度过短可能造成正文截断。它不是业务分段标记，错误复用还会破坏长连接上后续消息的解析。

2) 文件服务先用Gzip压缩再返回，产品要求进度条按原文件大小计算，直接读取Content-Length能满足吗？

答：不能，启用Gzip后，Content-Length表示压缩后的实际传输长度，而不是解压后的原始大小。前端可据此展示网络传输进度；若要显示原文件进度，需要服务端另行提供可靠的原始尺寸。

3) 报表接口必须保持Keep-Alive，生成过程中又无法提前知道总长度，后端应该怎样让客户端识别响应边界？

答：在HTTP/1.1长连接下，应使用Transfer-Encoding: chunked，把动态生成的内容分块发送，并由分块机制标识结束。这样无需预先计算总长度，但客户端在响应开始时也无法仅靠标准长度字段得到最终总量。

4) 线上接口同时返回Content-Length和Transfer-Encoding:

chunked，监控发现部分客户端展示内容异常，你先按哪个字段判断？

答：应优先按Transfer-Encoding指定的方式解析，存在该字段时Content-Length会被忽略。排查时应确认服务端为何同时写入冲突头部，并检查每个分块及结束标记；保留双重声明容易造成实现差异。

5) 旧接口通过服务端写完后关闭连接来标记正文结束，迁移到长连接后偶发超时，根因可能在哪里？

答：关闭连接能够为短连接提供消息结束边界，但长连接需要继续复用，不能再依赖断开来判断正文长度。迁移后应提供正确的Content-Length，或对不定长内容使用chunked；两者都缺失时客户端可能持续等待。

674. cookie 和 session

cookie保存在浏览器，session通常保存在服务端，两者一般通过sessionId关联。登录后，服务端创建会话并把sessionId交给浏览器，后续请求再通过cookie自动携带它。简单来说，cookie是HTTP头里的字段，而session是服务端维护会话状态的一种抽象和实现。多机部署时还要共享session数据，否则请求切换到另一台机器后可能找不到登录状态。session：是一个抽象概念，开发者为了实现中断和继续等操作，将user agent和server之间一对一的交互，抽象为“会话”，进而衍生出“会话状态”，也就是session的概念cookie：它是一个广泛存在的东西，http协议中定义在header中的字段，可以认为是session的一种后端无状态实现。现在我们常说的session，是为了绕开cookie的各种限制，通常借助cookie本身和后端存储实现的，一种更高级的会话状态实现 session的常见实现要借助cookie来发送sessionId

1) 登录接口给浏览器写入一个sessionId，安全评审却说“既然用了cookie，服务端就没有session”，你会怎么纠正？

答：这个判断把载体和会话状态混为一谈了，cookie是定义在HTTP header中的机制，而常见session会借助它传递sessionId。真正的会话状态通常保存在后端，浏览器持有的只是用于关联状态的标识。

2) 客服后台要求员工关闭页面后再次进入仍能继续操作，后端准备使用session，请求链路最少要怎样串起来？

答：后端需要创建并保存会话状态，再通过cookie把对应的sessionId交给浏览器，后续请求携带它以恢复同一会话。若浏览器没有回传该标识，或后端找不到对应记录，中断后的状态就无法继续。

3)

移动端容器明确禁止持久化cookie，服务端仍坚持现有的sessionId方案，会受到什么约束？

答：常见session实现依赖客户端在后续请求中持续发送sessionId，禁止保存或回传cookie会切断这种关联。可以改用其他受控方式传递标识，但那已经改变了载体设计，仍需保证每次请求能绑定到正确会话。

4) 线上出现用户登录成功后下一次请求立即变成未登录，接口响应里已经看到Set-Cookie，你会沿哪条链路排查？

答：先确认浏览器是否实际保存该cookie，再检查后续请求是否携带同一个sessionId，最后核对后端存储中是否存在对应会话。只看到Set-Cookie不能证明链路完整，任何一段丢失都会表现为会话中断。

5) 无状态接口负责人认为只用cookie保存全部状态更简单，平台负责人坚持cookie + sessionId + 后端存储，取舍重点是什么？

答：把状态交给cookie能减少后端会话存储，但会受到客户端载体自身限制；常见session则用较小的sessionId关联后端状态。后者更适合需要中断后继续的复杂会话，代价是维护会话存储及标识映射。

675. 介绍一下HTTPS和HTTP区别

HTTPS本质上是HTTP与SSL/TLS的组合，相比明文传输的HTTP，它能加密数据并认证服务器身份。HTTP默认使用80端口，HTTPS默认使用443端口，而且后者需要配置和维护证书。加密可以降低数据被窃听或篡改的

风险，也更利于搜索引擎收录。它的取舍是会增加证书维护、握手和计算开销，不过合理优化后访问速度的影响通常可以控制。HTTPS 要比 HTTP 多了 secure 安全性这个概念，实际上，HTTPS 并不是一个新的应用层协议，它其实就是 HTTP + TLS/SSL 协议组合而成，而安全性的保证正是 SSL/TLS 所做的工作。SSL 安全套接层 (Secure Sockets Layer) TLS (传输层安全, Transport Layer Security) 现在主流的版本是 TLS/1.2, 之前的 TLS1.0、TLS1.1 都被认为是不安全的，在不久的将来会被完全淘汰。HTTPS 就是身披了一层 SSL 的 HTTP。那么区别有哪些呢 HTTP 是明文传输协议，HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，比 HTTP 协议安全。HTTPS比HTTP更加安全，对搜索引擎更友好，利于SEO,谷歌、百度优先索引HTTPS网页。HTTPS标准端口443，HTTP标准端口80。HTTPS需要用到SSL证书，而HTTP不用。我觉得记住以下两点HTTPS主要作用就行 对数据进行加密，并建立一个信息安全通道，来保证传输过程中的数据安全;对网站服务器进行真实身份认证。 HTTPS的缺点证书费用以及更新维护。HTTPS 降低一定用户访问速度（实际上优化好就不是缺点了）。HTTPS 消耗 CPU 资源，需要增加大量机器。

1) 支付页已经把端口从80改成443，运维就宣称链路具备HTTPS安全性，这个判断成立吗？

答：不成立，端口变化本身不会提供加密或身份认证，HTTPS是HTTP与TLS/SSL组合形成的安全通信。服务端还必须正确配置证书和安全协议，否则即使监听443，也不能据此证明连接可信。

2)

资讯站准备全站从HTTP迁移到HTTPS，产品只关心搜索收录，工程侧还应明确哪些直接收益？

答：迁移后传输内容由TLS/SSL建立的安全通道保护，并可通过证书认证网站服务器身份，不只是影响搜索引擎表现。实施时需要配置证书、更新维护并评估资源消耗，不能只把页面链接替换成https://。

3) 内网站点不传密码，负责人因此认为明文HTTP足够；但页面会下发业务数据，你会怎样评估？

答：不传密码并不等于没有风险，HTTP的业务数据仍以明文传输，也缺少HTTPS提供的服务器身份认证。若数据需要防止链路窃听或站点冒充，就应使用HTTPS；证书维护和计算资源仍需纳入成本。

4) 线上切换HTTPS后浏览器持续提示证书相关错误，而接口代码和响应内容都正常，你先查什么？

答：应先检查服务端是否提供了与站点匹配且可验证的证书，并确认连接确实通过TLS/SSL建立，而不是只开放了443端口。应用响应正常不能替代身份认证，证书链路失败时浏览器仍会把站点视为不可信。

5) 预算评审认为HTTPS只有证书费用，没有运行成本；基础设施团队要求扩容，你如何给出保守结论？

答：除证书采购或更新维护外，HTTPS的加密与握手还会消耗一定计算资源，并可能带来访问延迟。具体影响取决于实现和优化，不能凭题面给出固定数字；应通过实际负载评估容量，而不是直接认定成本为零。

676. HTTPS握手过程

HTTPS握手的核心，是先验证服务器证书，再安全地协商出后续通信使用的对称密钥。客户端发送支持的协议、加密方法和随机数，服务端确认参数并返回证书及自己的随机数。客户端验证证书后生成Premaster secret，用证书中的公钥加密发送，服务端再用私钥解密。双方根据这些随机数生成会话密钥，后续数据便改用效率更高的对称加密传输。第一步，客户端给出协议版本号、一个客户端生成的随机数 (Client random)，以及客户端支持的加密方法第二步，服务端确认双方使用的加密方法，并给出数字证书、以及一个服务器生成的随机数第三步，客户端确认数字证书有效，然后生成一个新的随机数 (Premaster secret)，并使用数字证书中的公钥，加密这个随机数，发给服务端第四步，服务端使用自己的私钥，获取客户端

发来的随机数（即Premaster secret）。第五步，客户端和服务端根据约定的加密方法，使用前面的三个随机数，生成“对话密钥”（session key），用来加密接下来的整个对话过程。总结：客户端发起 HTTPS 请求，服务端返回证书，客户端对证书进行验证，验证通过后本地生成用于构造对称加密算法的随机数，通过证书中的公钥对随机数进行加密传输到服务端（随机对称密钥），服务端接收后通过私钥解密得到随机对称密钥，之后的数据交互通过对称加密算法进行加解密。（既有对称加密，也有非对称加密）

1) 安全评审提议客户端首次访问时直接生成对称密钥并明文发给服务器，认为后续正文已经加密就足够，这里错在哪里？

答：明文发送会让链路中的观察者直接获得对称密钥，后续加密也就失去意义。典型握手会让客户端生成Premaster secret，使用证书中的公钥加密后传输，服务端再用私钥解开。

2) 浏览器访问登录页时，服务端已经返回数字证书和Server random，客户端接下来应完成哪些关键动作？

答：客户端应先验证数字证书是否有效，再生成Premaster secret，并用证书中的公钥加密后发送给服务端。若证书验证失败，就不应继续建立受信任会话，否则公钥来源无法得到保证。

3) 后端负责人想让整个订单响应都使用证书公钥加密，前端负责人坚持握手后改用对称加密，哪种更符合该流程？

答：握手中的非对称加密主要用于安全传递Premaster secret，后续双方根据三个随机数生成session key，再用它加密整个对话。持续使用公钥并不符合这套流程，也混淆了密钥交换与数据传输的职责。

4) 线上只有部分实例在握手阶段失败，抓包显示客户端已发出加密后的Premaster secret，但服务端无法继续，你会先排查什么？

答：应先确认故障实例是否持有与所发证书公钥匹配的私钥，因为服务端必须用该私钥解开Premaster secret。再核对双方确认的加密方法及随机数处理；私钥或协商信息不一致都会阻断会话密钥生成。

5) 网关团队认为只校验证书即可省略Client random和Server random，应用团队认为三份随机材料都要保留，如何判断？

答：按该握手流程，客户端与服务端会结合Client random、Server random和Premaster secret生成相同的session key。证书验证只确认公钥及服务端身份，不能替代会话密钥材料；省略随机数会改变既定协商过程。

677. 介绍一个HTTPS工作原理

HTTPS通过证书证明服务器身份，用非对称加密完成密钥交换，再用对称加密传输实际数据。只用对称加密会遇到密钥无法安全传递的问题，只用非对称加密又不适合持续处理大量通信，所以两者需要配合。浏览器会利用受信任认证机构的公钥验证证书和数字签名，避免服务器公钥被中间人替换。验证通过后双方取得相同的对称密钥，后续HTTP内容都用它加密和解密。我们可以把HTTPS理解成HTTPS = HTTP + SSL/TLS。SSL/TLS的功能实现主要依赖于三类基本算法：散列函数、对称加密和非对称加密，其利用非对称加密实现身份认证和密钥协商，对称加密算法采用协商的密钥对数据加密，基于散列函数验证信息的完整性。

1. 对称加密 加密和解密用同一个密钥的加密方式叫做对称加密。Client客户端和Server端共用一套密钥，这样子的加密过程似乎很让人理解，但是随之会产生一些问题。

问题一:WWW万维网有许许多多的客户端，不可能都用密钥A进行信息加密，这样子很不合理，所以解决办法就是使用一个客户端使用一个密钥进行加密。

问题二:既然不同的客户端使用不同的密钥，那么对称加密的密钥如何传输？那么解决的办法只能是一端生成一个密钥，然后通过HTTP传输给另一端，那么这样子又会产生新的问题。

问题三:这个传输密钥的过程，又如何保证加密？如果被中间人拦截，密钥也会被获取,那么你会说对密钥再进行加密，那又怎么保存对密钥加密的过程，是加密的过程？

到这里，我们似乎想明白了，使用对称加密的方式，行不通，所以我们需要采用非对称加密 2. 非对称加密 通过上面的分析，对称加密的方式行不通，那么我们来梳理一下非对称加密。采用的算法是RSA，所以在一些文章中也会看见传统RSA握手，基于现在TLS主流版本是1.2，所以接下来梳理的是TLS/1.2握手过程。非对称加密中，我们需要明确的点是 有一对密钥，公钥和私钥。公钥加密的内容，只有私钥可以解开，私钥加密的内容，所有的公钥都可以解开，这里说的公钥都可以解开，指的是一对密钥。公钥可以发送给所有的客户端，私钥只保存在服务器端。

3. 主要工作流程 梳理起来，可以把TLS 1.2握手过程分为主要的五步 步骤一：Client发起一个HTTPS请求，连接443端口。这个过程可以理解成是请求公钥的过程。步骤二：Server端收到请求后，通过第三方机构私钥加密，会把数字证书（也可以认为是公钥证书）发送给Client。步骤三：浏览器安装后会自带一些权威第三方机构公钥，使用匹配的公钥对数字签名进行解密。根据签名生成的规则对网站信息进行本地签名生成，然后两者比对。通过比对两者签名，匹配则说明认证通过，不匹配则获取证书失败。步骤四：在安全拿到服务器公钥后，客户端Client随机生成一个对称密钥，使用服务器公钥（证书的公钥）加密这个对称密钥，发送给Server(服务器)。步骤五：Server(服务器)通过自己的私钥，对信息解密，至此得到了对称密钥，此时两者都拥有了相同的对称密钥。浏览器安装后会自带一些权威第三方机构公钥，使用匹配的公钥对数字签名进行解密。根据签名生成的规则对网站信息进行本地签名生成，然后两者比对。通过比对两者签名，匹配则说明认证通过，不匹配则获取证书失败。接下来，就可以通过该对称密钥对传输的信息加密/解密啦，从上面图举个例子 Client用户使用该对称密钥加密'明文内容B',发送给Server(服务器)Server使用该对称密钥进行解密消息，得到明文内容B。

接下来考虑一个问题，如果公钥被中间人拿到篡改怎么办呢？客户端可能拿到的公钥是假的，解决办法是什么呢？ 3. 第三方认证 客户端无法识别传回公钥是中间人的，还是服务器的，这是问题的根本，我们是不是可以通过某种规范可以让客户端和服务器都遵循某种约定呢？那就是通过第三方认证的方式 在HTTPS中，通过证书+数字签名来解决这个问题。这里唯一不同的是，假设对网站信息加密的算法是MD5，通过MD5加密后，然后通过第三方机构的私钥再次对其加密，生成数字签名。这样的话，数字证书包含有两个特别重要的信息某网站公钥+数字签名 我们再次假设中间人截取到服务器的公钥后，去替换成自己的公钥，因为有数字签名的存在，这样子客户端验证发现数字签名不匹配，这样子就防止中间人替换公钥的问题。那么客户端是如何去对比两者数字签名的呢？浏览器会去安装一些比较权威的第三方认证机构的公钥，比如VeriSign、Symantec以及GlobalSign等等。验证数字签名的时候，会直接从本地拿到相应的第三方的公钥，对私钥加密后的数字签名进行解密得到真正的签名。然后客户端利用签名生成规则进行签名生成，看两个签名是否匹配，如果匹配认证通过，不匹配则获取证书失败。

4. 数字签名作用 数字签名：将网站的信...

1) 登录页已经用HTTPS，有人却说只要拿到服务器证书里的公钥就能解密所有用户请求，你会怎么反驳？

答：证书公钥不是后续业务数据的直接解密密钥，它主要用于验证服务器身份并参与握手阶段的密钥协商。握手完成后，双方使用协商出的对称密钥加密请求与响应；服务器私钥不能公开，否则身份认证和密钥交换的安全基础都会受损。

2)

一个首页要加载HTML、接口和几十个静态资源，为什么TLS不用非对称算法加密全部内容？

答：非对称加密适合身份认证和安全协商密钥，但不适合持续承担大量业务数据的加解密。连接建立后改用同一把对称密钥处理传输，能兼顾效率与机密性；握手阶段仍需证书和数字签名，不能只发送一个裸公钥。

3) 公司内网代理把站点公钥替换成自己的公钥，但浏览器仍然显示安全锁，这一定说明证书校验失效了吗？

答：不一定，企业代理可能已把自己的根证书安装进设备信任库，再为目标域名动态签发可被本机接受的证书。浏览器仍会校证书链、域名和签名，只是信任锚发生了变化；若设备并未授权该根证书

，替换公钥通常会触发证书错误。

4) 某域名换证后桌面浏览器正常，部分老客户端却提示证书不可信，你会沿着握手流程查什么？

答：先检查服务端是否发送了完整证书链，再核对证书域名、有效期以及客户端设备时间。桌面浏览器可能补全缺失的中间证书，而部分旧客户端不能，因此同一域名会表现不同；若处于企业网络，还要排查代理证书是否被客户端信任。

5) 团队提议为了简化部署直接使用自签名证书，面向公众的支付页能否接受？

答：公众支付页不适合直接采用未被客户端信任的自签名证书，因为浏览器无法沿受信任证书链确认服务器身份。用户手动忽略警告会削弱防中间人攻击的关键保障；自签名方案更适合已预先分发并严格管理信任根的封闭环境。

6) 证书验证已经保证了身份，为什么传输阶段还需要完整性校验，只有加密不够吗？

答：加密主要隐藏明文，并不天然等同于能够发现传输内容遭到修改。TLS还需要基于散列等机制验证数据完整性，使接收方识别被篡改的记录；身份认证、机密性和完整性分别处理不同风险，缺少任一项都不能完整替代。

678. SSL 连接断开后如何恢复

SSL连接断开后，可以通过Session ID或Session Ticket复用上次的会话信息，避免重新生成密钥。Session ID由服务端保存，客户端重连时带上编号，但请求被负载均衡到另一台服务器后可能无法恢复。Session Ticket由客户端保存，服务端解密后即可取回密钥和加密方式等信息，因此更适合多服务器场景。

一共有两种方法来恢复断开的 SSL 连接，一种是使用 session ID，一种是 session ticket。通过session ID使用 session ID 的方式，每一次的会话都有一个编号，当对话中断后，下一次重新连接时，只要客户端给出这个编号，服务器如果有这个编号的记录，那么双方就可以继续使用以前的秘钥，而不用重新生成一把。目前所有的浏览器都支持这一种方法。但是这种方法有一个缺点是，session ID 只能够存在一台服务器上，如果我们的请求通过负载均衡被转移到了其他的服务器上，那么就无法恢复对话。通过session ticket另一种方式是 session ticket 的方式，session ticket 是服务器在上一次对话中发送给客户的，这个 ticket 是加密的，只有服务器能够解密，里面包含了本次会话的信息，比如对话秘钥和加密方法等。这样不管我们的请求是否转移到其他的服务器上，当服务器将 ticket 解密以后，就能够获取上次对话的信息，就不用重新生成对话秘钥了。

1) 用户关闭页面十秒后重新访问，客户端带着旧session ID，为什么服务端仍可能重新握手？

答：session ID只有在服务端仍保存对应会话记录时才能恢复，编号本身不包含完整会话状态。记录可能已过期、被清理，或请求被负载均衡到另一台没有该记录的服务器；这些情况下只能重新生成会话密钥。

2) 三台网关轮询承接TLS，上线后会话恢复命中率明显下降，你会优先调整什么？

答：若使用session ID，应让节点共享会话状态，或通过负载均衡保证后续连接落到保存记录的节点。也可改用服务端可解密的session ticket，降低对单机状态的依赖；无论选择哪种方式，都要处理过期和密钥管理。

3) 团队把网关从单机扩成二十个实例，session ID和session ticket哪个更适合横向扩容？

答：在各实例能够解密同类票据的前提下，session ticket更适合横向扩容，因为会话信息随客户端票据返回，不依赖命中特定节点。session ID则要求节点保存并找到编号对应的记录；共享状态或会话粘滞会增加系统复杂度。

4) 监控显示恢复连接全部退化为完整握手，但页面请求仍能成功，你如何区分是票据故障还是网络故障？

答：请求仍成功说明基础连接和完整握手大概率可用，应重点检查客户端是否携带票据、票据是否过期，以及当前节点能否解密。若近期轮换过票据加密密钥或新增节点，还要核对节点配置是否一致；恢复失败通常会回退握手，不一定直接报业务错误。

5) 安全团队要求频繁轮换session ticket加密密钥，性能团队担心恢复率下降，怎么取舍？

答：票据只有服务器能够解密，因此加密密钥的生命周期直接影响安全边界和可恢复窗口。轮换时可以让节点在受控期限内兼容必要的旧密钥，再逐步淘汰；保留过久会扩大密钥泄露影响，立即切断则会让旧票据全部失效。

6) 前端想靠preconnect解决移动网络频繁断连，它和会话恢复处理的是同一件事吗？

答：两者不是同一机制：preconnect是提前建立到目标源的连接，会话恢复则是在新连接中复用先前协商的信息。前者只能把连接成本前移，后者才可能避免重新生成会话密钥；若票据无效或服务端没有session ID记录，仍会完整握手。

679. 谈一谈你对HTTP/2理解

HTTP/2的核心是二进制分帧和多路复用，让多个请求、响应可以共享一条TCP连接并发传输。不同流的帧可以乱序到达，再通过Stream ID重新组装，从而缓解HTTP/1.1请求排队造成的队头阻塞。它还用HPACK压缩头部，并支持服务器推送。边界是底层仍然使用TCP，一旦丢包等待重传，同一连接中的所有流都会受到影响。首先补充一下，http 和 https 的区别，相比于 http,https 是基于 ssl 加密的 http 协议
简要概括:http2.0是基于 1999 年发布的http1.0之后的首次更新

提升访问速度(可以对于，请求资源所需时间更少，访问速度更快，相比

http1.0)允许多路复用:多路复用允许同时通过单一的 HTTP/2 连接发送多重请求-响应信息。改善了:在http1.1中，浏览器客户端在同一时间，针对同一域名下的请求有一定数量限

制(连接数量)，超过限制会被阻塞二进制分帧:HTTP2.0

会将所有的传输信息分割为更小的信息或者帧，并对他们进行二进制编码首部压缩服务器端推送 头部压缩
HTTP 1.1版本会出现User-Agent、Cookie、Accept、Server、Range等字段可能会占用几百甚至几千字节，而 Body 却经常只有几十字节，所以导致头部偏重。 HTTP 2.0 使用HPACK算法进行压缩。 多路复用
HTTP 1.x 中，如果想并发多个请求，必须使用多个 TCP 链接，且浏览器为了控制资源，还会对单个域名有6-8个的TCP链接请求限制。 HTTP2中： 同域名下所有通信都在单个连接上完成。单个连接可以承载任意数量的双向数据流。数据流以消息的形式发送，而消息又由一个或多个帧组成，多个帧之间可以乱序发送，因为根据帧首部的流标识可以重新组装，也就是Stream ID，流标识符，有了它，接收方就能从乱序的二进制帧中选择ID相同的帧，按照顺序组装成请求/响应报文。 服务器推送 浏览器发送一个请求，服务器主动向浏览器推送与这个请求相关的资源，这样浏览器就不用发起后续请求。 相比较http/1.1的优势 推送资源可以由不同页面共享服务器可以按照优先级推送资源客户端可以缓存推送的资源客户端可以拒收推送过来的资源 二进制分帧 之前是明文传输，不方便计算机解析，对于回车换行符来说到底是内容还是分隔符，都需要内部状态机去识别，这样子效率低，HTTP/2采用二进制格式，全部传输01串，便于机器解码。 这样子一个报文格式就被拆分为一个个二进制帧，用Headers帧存放头部字段，Data帧存放请求体数据。这样子的话，就是一堆乱序的二进制帧，它们不存在先后关系，因此不需要排队等待，解决了HTTP队头阻塞问题。 在客户端与服务器之间，双方都可以互相发送二进制帧，这样子双向传输的序列，称为流，所以HTTP/2中以流来表示一个TCP连接上进行多个数据帧的通信，这就是多路复用概念。

那乱序的二进制帧，是如何组装成对于的报文呢？

所谓的乱序，值的是不同ID的Stream是乱序的，对于同一个Stream

ID的帧是按顺序传输的。接收方收到二进制帧后，将相同的Stream ID组装成完整的请求报文和响应报文。

二进制帧中有一些字段，控制着优先级和流量控制等功能，这样的话，就可以设置数据帧的优先级，让服务器处理重要资源，优化用户体验。HTTP2的缺点 TCP 以及 TCP+TLS建立连接的延时,HTTP/2使用TCP协议来传输的，而如果使用HTTPS的话，还需要使用TLS协议进行安全传输，而使用TLS也需要一个握手过程,在传输数据之前，导致我们需要花掉 3~4 个 RTT。TCP的队头阻塞并没有彻底解决。在HTTP/2中，多个请求是跑在一个TCP管道中的。但当HTTP/2出现丢包时，整个 TCP 都要开始等待重传，那么就会阻塞该TCP连接中的所有请求。

1) 商品页的一个大图片响应很慢，同域名的接口却还能并发返回，这能说明HTTP/2已彻底消除队头阻塞吗？

答：只能说明不同Stream ID的请求不必像HTTP/1.1那样按响应顺序等待，不能说明底层阻塞已消失。多个流仍共享一条TCP连接，一旦发生丢包，重传等待可能影响连接上的所有流；这是HTTP/2多路复用的边界。

2) 站点把接口、脚本和图片都放在同一域名，Network面板却出现一条连接上的大量并发请求，你如何解释报文不会串？

答：HTTP/2会把头部和数据体拆成二进制帧，并用Stream ID标识所属数据流。不同流的帧可以交错传输，接收端再按标识重组为各自的请求或响应；同一数据流内部仍需保持可正确组装的顺序。

3) 迁移到HTTP/2后，团队仍把静态资源拆到四个域名以突破每域名连接数限制，这个策略还合理吗？

答：该策略的收益会明显减弱，因为HTTP/2可在同域名的一条连接上承载多个双向数据流，不再依赖多条连接实现请求并发。继续分片会额外建立连接，并使多路复用和头部压缩分散到不同连接；若存在独立缓存或权限边界，仍需单独评估。

4) 线上只有弱网用户出现首页所有资源同时卡住，服务器CPU和接口耗时都正常，你会怀疑哪一层？

答：应检查共享TCP连接是否出现丢包和重传，因为HTTP/2的多个流都运行在这条连接上。某个包丢失时，底层TCP的有序交付可能阻塞所有流；还要结合连接复用情况排除资源其实来自不同域名或不同协议。

5) 一个接口响应体只有几十字节，但每次都携带较大的Cookie和User-Agent，升级HTTP/2能否替代头部治理？

答：HPACK会利用连接内的表压缩重复头字段，后续请求可减少重复传输，因此小响应通常受益明显。但首次出现的字段仍需发送，频繁变化或过大的Cookie也不会凭空消失；协议压缩不能替代删减无用头部。

6)

服务端准备主动推送首页可能需要的所有脚本，前端认为越多越快，你会接受这个结论吗？

答：不能直接接受，服务器推送可以省去浏览器发起后续请求，但客户端可能已有缓存或根本不需要这些资源。客户端能够拒收或缓存推送资源，服务端也应结合优先级选择；推送过多会占用连接带宽，反而挤压当前页面的关键响应。

680. HTTP3

HTTP/3本质上是让HTTP运行在基于UDP的QUIC上，用独立数据流解决TCP层的队头阻塞。QUIC在UDP之上补上了重传、拥塞控制和流量控制，所以仍能提供可靠传输。它同时集成TLS 1.3，连接可以通过0-RTT或1-RTT快速建立。相比HTTP/2，其价值不只是多路复用，还在于某个流受阻时不会拖住其他流。 Google 在推SPDY的时候就已经意识到了这些问题，于是就另起炉灶搞了一个基于 UDP

协议的“QUIC”协议，让HTTP跑在QUIC上而不是TCP上。主要特性如下：实现了类似TCP的流量控制、传输可靠性的功能。虽然UDP不提供可靠性的传输，但QUIC在UDP的基础之上增加了一层来保证数据可靠性传输。它提供了数据包重传、拥塞控制以及其他一些TCP中存在的特性实现了快速握手功能。由于QUIC是基于UDP的，所以QUIC可以实现使用0-RTT或者1-RTT来建立连接，这意味着QUIC可以用最快的速度来发送和接收数据。集成了TLS加密功能。目前QUIC使用的是TLS1.3，相较于早期版本TLS1.3有更多的优点，其中最重要的一点是减少了握手所花费的RTT个数。多路复用，彻底解决TCP中队头阻塞的问题。

1) 视频页切换到HTTP/3后抓包仍能看到丢包，测试同学因此认定UDP会让视频数据缺失，你怎么判断？

答：看到丢包不能直接推出业务数据缺失，QUIC在UDP之上实现了确认、重传、拥塞控制和流量控制。它仍提供可靠传输，只是不依赖TCP完成这些能力；若播放异常，应继续检查重传、拥塞和应用层处理。

2)

移动首页有多个接口并发加载，为什么把HTTP/2的TCP多路复用换成QUIC仍可能改善卡顿？

答：HTTP/2的流虽然彼此独立，但都共享同一条TCP连接，丢包会触发底层有序重传等待。QUIC在传输层提供多路复用，某个流的数据丢失时不必阻塞其他流；改善幅度仍取决于实际网络质量和部署情况。

3) 团队只把HTTPS证书配置搬到新网关，却没有准备TLS 1.3，还能按预期启用HTTP/3吗？

答：不能只把它当作普通UDP服务部署，QUIC集成了TLS 1.3加密与握手能力。网关必须同时支持对应的QUIC和加密协商，客户端也要能够选择该协议；任一环节不支持时，通常需要回退到其他可用协议。

4) 灰度HTTP/3后，公司网络用户全部回退，家庭网络正常，应用接口本身没有报错，你会先查哪里？

答：应先检查企业防火墙、代理或出口策略是否允许承载QUIC的UDP流量，再确认客户端与网关是否完成协议协商。业务接口正常只说明回退路径可用，不代表HTTP/3已建立；同时要核对服务端端口和协议支持是否一致。

5)

产品要求首个支付请求使用0-RTT追求最快提交，仅依据“QUIC支持快速握手”能否批准？

答：不能仅凭快速握手特性批准，0-RTT只说明连接可更早发送数据，不代表所有带副作用的请求都适合。涉及支付时还需评估早期数据可能被重放等安全边界，并由服务端限制可接受的请求类型；保守方案是完整握手后再提交。

6) 弱网下HTTP/3已减少连接层队头阻塞，是否就不再需要控制图片和脚本体积？

答：仍然需要，QUIC改善的是连接建立、可靠传输和多路复用中的阻塞方式，并不会减少资源本身的字节数。大资源依旧占用带宽并受拥塞控制影响，脚本还要解析执行；协议升级不能替代资源压缩、缓存和加载优先级治理。

681. HTTP/1.0 HTTP1.1 HTTP2.0版本之间的差异

HTTP/1.0、HTTP/1.1和HTTP/2的差异，可以理解为不断减少重复建连、请求排队和协议冗余。HTTP/1.0通常一次请求使用一次连接，但已支持多种内容格式以及GET、POST、HEAD。HTTP/1.1默认使用持久连接，并增加管线化、缓存控制、Range断点续传和Host。HTTP/2进一步采用二进制分帧、HPACK和多路复用，不过底层TCP的队头阻塞仍然存在。

HTTP 0.9：1991年,原型版本，功能简陋，只有一个命令GET,只支持纯文本内容，该版本已过时。HTTP 1.0任何格式的内容都可以发送，这使得互联网不仅可以传输文字，还能传输图像、视频、二进制等文件。除了GET命

令，还引入了POST命令和HEAD命令。http请求和回应的格式改变，除了数据部分，每次通信都必须包括头信息（HTTP header），用来描述一些元数据。只使用 header 中的 If-Modified-Since 和 Expires 作为缓存失效的标准。不支持断点续传，也就是说，每次都会传送全部的页面和数据。通常每台计算机只能绑定一个 IP，所以请求消息中的 URL 并没有传递主机名（hostname） HTTP 1.1 http1.1是目前最为主流的http协议版本，从1999年发布至今，仍是主流的http协议版本。引入了持久连接（persistent connection），即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive。长连接的连接时长可以通过请求头中的keep-alive来设置引入了管道机制（pipelining），即在同一个TCP连接里，客户端可以同时发送多个请求，进一步改进了HTTP协议的效率。HTTP 1.1 中新增加了 E-tag, If-Unmodified-Since, If-Match, If-None-Match 等缓存控制标头来控制缓存失效。支持断点续传，通过使用请求头中的Range来实现。使用了虚拟网络，在一台物理服务器上可以存在多个虚拟主机（Multi-homed Web Servers），并且它们共享一个IP地址。新增方法：PUT、PATCH、OPTIONS、DELETE。http1.x版本问题在传输数据过程中，所有内容都是明文，客户端和服务端都无法验证对方的身份，无法保证数据的安全性。HTTP/1.1 版本默认允许复用TCP连接，但是在同一个TCP连接里，所有数据通信是按次序进行的，服务器通常在处理完一个回应后，才会继续去处理下一个，这样子就会造成队头阻塞。http/1.x 版本支持Keep-alive，用此方案来弥补创建多次连接产生的延迟，但是同样会给服务器带来压力，并且的话，对于单文件被不断请求的服务，Keep-alive会极大影响性能，因为它在文件被请求之后还保持了不必要的连接很长时间。HTTP 2.0二进制分帧这是一次彻底的二进制协议，头信息和数据体都是二进制，并且统称为"帧"：头信息帧和数据帧。头部压缩HTTP 1.1版本会出现User-Agent、Cookie、Accept、Server、Range等字段可能会占用几百甚至几千字节，而 Body 却经常只有几十字节，所以导致头部偏重。HTTP 2.0 使用HPACK算法进行压缩。多路复用复用TCP连接，在一个连接里，客户端和浏览器都可以同时发送多个请求或回应，且不用按顺序一一对应，这样子解决了队头阻塞的问题。服务器推送允许服务器未经请求，主动向客户端发送资源，即服务器推送。请求优先级可以设置数据帧的优先级，让服务端先处理重要资源，优化用户体验。任何格式的内容都可以发送，这使得互联网不仅可以传输文字，还能传输图像、视频、二进制等文件。除了GET命令，还引入了POST命令和HEAD命令。http请求和回应的格式改变，除了数据部分，每次通信都必须包括头信息（HTTP header），用来描述一些元数据。只使用 header 中的 If-Modified-Since 和 Expires 作为缓存失效的标准。不支持断点续传，也就是说，每次都会传送全部的页面和数据。通常每台计算机只能绑定一个 IP，所以请求消息中的 URL 并没有传递主机名（hostname） 引入了持久连接（persistent connection），即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive。长连接的连接时长可以通过请求头中的keep-alive来设置引入了管道机制（pipelining），即在同一个TCP连接里，客户端可以同时发送多个请求，进一步改进了HTTP协议的效率。HTTP 1.1 中新增加了 E-tag, If-Unmodified-Since, If...

1)

一个下载接口在HTTP/1.0下中断后只能从头开始，换成HTTP/1.1为什么可能支持继续下载？

答：HTTP/1.1引入了基于Range请求头的范围请求，客户端可以请求尚未完成的字节区间，从而实现断点续传。前提是服务端和资源都支持范围响应；仅升级协议标识而未实现相应处理，下载仍可能从头开始。

2) 首页连续请求十个接口，团队说开启keep-alive就等于获得HTTP/2多路复用，你会如何反驳？

答：keep-alive只是让多个请求复用同一条TCP连接，不能让HTTP/1.1的响应像HTTP/2帧那样在多个流间交错传输。前一个响应处理缓慢时仍可能造成队头阻塞；HTTP/2还包含二进制分帧、头部压缩和流标识。

3)

迁移到HTTP/1.1后，同一IP上部署多个站点不再要求每站独占地址，协议层依赖了什么变化？

答：HTTP/1.1请求会携带主机信息，使服务器能够在共享IP上区分目标虚拟主机。网关必须依据主机名正确路由，并避免默认证书或站点配置错配；缺失或错误的主机信息仍可能被导向错误服务。

4) 升级HTTP/2后接口并发能力提高，但弱网下偶尔所有请求一起停顿，你会如何定位版本收益与故障边界？

答：先确认请求是否确实复用同一条HTTP/2连接，再检查该连接上的丢包和TCP重传。二进制分帧解决了应用层按响应次序等待的问题，但多个流仍共享TCP；底层丢包造成的队头阻塞不会因协议升级彻底消失。

5)

团队准备为HTTP/2重新拆分一个超大脚本包，但担心小请求数量增加，你会依据什么取舍？

答：HTTP/2的多路复用和HPACK降低了同连接内多个请求及重复头部的成本，因此不必沿用只为减少连接并发而合并资源的策略。拆分仍要考虑资源总字节数、缓存复用和执行开销；协议能力不会自动消除过度碎片化的管理成本。

6) 安全评审认为从HTTP/1.1升到HTTP/2就能解决明文窃听，这个结论成立吗？

答：不成立，版本升级关注分帧、压缩、多路复用、优先级和服务推送，并不等同于获得传输加密。保护身份、机密性和完整性仍需使用HTTPS所依赖的TLS；若链路仍是明文，协议性能特性不能阻止监听和篡改。

682. DNS如何工作的

DNS是把域名解析成IP地址的分布式查询系统，属于应用层协议，通常使用UDP的53端口。浏览器会先检查自身缓存、操作系统缓存和hosts文件，仍未命中时再请求本地DNS服务器。客户端到本地服务器通常是递归查询，本地服务器则依次向根域名、顶级域名和权威域名服务器迭代查询。解析结果会按TTL缓存；同一域名返回多个服务器地址时，也可以借此分摊请求。DNS的作用就是通过域名查询到具体的IP。DNS协议提供的是一种主机名到IP地址的转换服务，就是我们常说的域名系统。是应用层协议，通常该协议运行在UDP协议之上，使用的是53端口号。

因为IP存在数字和英文的组合（IPv6），很不利于人类记忆，所以就出现了域名。你可以把域名看成是某个IP的别名，DNS就是去查询这个别名的真正名称是什么。

当你在浏览器中想访问www.google.com时，会通过进行以下操作：本地客户端向服务器发起请求查询IP地址查看浏览器有没有该域名的IP缓存查看操作系统有没有该域名的IP缓存查看Host文件有没有该域名的解析配置如果这时候还没得话，会通过直接去DNS根服务器查询，这一步查询会找出负责com这个一级域名的服务器然后去该服务器查询google.com这个二级域名接下来查询www.google.com这个三级域名的地址返回给DNS客户端并缓存起来我们通过一张图来看看它的查询过程吧这张图很生动的展示了DNS在本地DNS服务器是如何查询的，一般向本地DNS服务器发送请求是递归查询的本地DNS服务器向其他域名服务器请求的过程是迭代查询的过程递归查询和迭代查询递归查询指的是查询请求发出后，域名服务器代为向下一级域名服务器发出请求，最后向用户返回查询的最终结果。使用递归查询，用户只需要发出一次查询请求。迭代查询指的是查询请求后，域名服务器返回单次查询的结果。下一级的查询由用户自己请求。使用迭代查询，用户需要发出多次的查询请求。

所以一般而言，本地服务器查询是递归查询，而本地DNS服务器向其他域名服务器请求的过程是迭代查询的过程DNS缓存缓存也很好理解，在一个请求中，当某个DNS服务器收到一个DNS回答后，它能够回答中的信息缓存在本地存储器中。返回的资源记录中的TTL代表了该条记录的缓存的时间。DNS实现负载均衡它是如何实现负载均衡的呢？首先我们得清楚DNS是可以用于在冗余的服务器上实现负载均衡。

原因：这是因为一般的大型网站使用多台服务器提供服务，因此一个域名可能会对应多个服务器地址。举个例子来说，当用户发起网站域名的DNS请求的时候，DNS服务器返回这个域名所对应的服务器IP地址的集合在每个回答中，会循环这些IP地址的顺序，用户一般会选择排在前面的地址发送请求。以此将用户的请求均衡的分配到各个不同的服务器上，这样来实现负载均衡。DNS为什么使用UDP协议作为传输层协议？DNS使用UDP协议作为传输层协议的主要原因是为了避免使用TCP协议时造成的连接时延。为了得到一个域名的IP地址，往往会向多个域名服务器查询，如果使用TCP协议，那么每次请求都会存在连接时延，这样使DNS服务变得很慢。大多数的地址查询请求，都是浏览器请求页面时发出的，这样会造成网页的等待时间过长。总结DNS域名系统，是应用层协议，运行UDP协议之上，使用端口43。查询过程，本地查询是递归查询，依次通过浏览器缓存—>>本地hosts文件—>>本地DNS解析器—>>本地DNS服务器—>>其他域名服务器请求。

接下来的过程就是迭代过程。递归查询一般而言，发送一次请求就够，迭代过程需要用户发送多次请求。

1) 用户能用IP打开登录页，却不能通过域名访问，后端同事据此认定权威DNS配错了，你同意吗？

答：不能直接认定，因为域名查询可能命中浏览器、操作系统或本地hosts中的旧结果，还未访问权威服务器。应逐层检查缓存和hosts，再核对本地DNS返回的地址；直接访问IP只能说明网络与服务可能可达。

2) 新域名上线时，运维只给了权威解析记录，浏览器侧仍提示域名找不到，你会沿着哪条链路验证？

答：先检查浏览器与操作系统缓存以及本地hosts，再查询本地DNS得到的记录。若仍无结果，就继续确认解析链是否能从根域定位顶级域服务器，再找到该域名的权威服务器；任一层配置或缓存异常都可能中断查询。

3) 一个活动域名同时指向多台服务器，产品要求仅靠DNS分摊流量，并在某台机器故障后立即摘除，你会接受吗？

答：DNS可以返回多个服务器地址，并通过调整回答中的地址顺序实现较粗粒度的负载分配。但解析结果会按资源记录的TTL缓存，故障地址被删除后仍可能继续返回或被客户端使用，因此不能承诺立即摘除。

4) 发布后部分地区访问旧站、公司网络却正常，应用日志也没有异常，你怎么判断是否为解析缓存问题？

答：分别在异常网络和正常网络查询域名，比较返回的IP与预期记录，同时检查本地hosts和各级缓存。若异常侧持续得到旧地址，应结合记录的TTL判断缓存何时失效；只清浏览器缓存未必能清除操作系统或本地DNS的结果。

5) 架构师要在反向代理和DNS多地址之间选择负载均衡，前端域名切换又要求故障影响尽量小，你会提醒什么？

答：DNS多地址无需让请求先经过统一代理，但调度较粗，并会受到各级缓存和TTL的约束。反向代理能在入口处选择真实服务器，控制更集中；代价是所有请求经过该入口，其容量与可用性也必须被保障。

683. 短轮询、长轮询和 WebSocket 间的区别

短轮询是客户端定时询问，长轮询是请求挂起等待结果，WebSocket则建立可双向通信的长连接。短轮询实现最简单，但即使数据没变化也会持续建立HTTP连接，客户端和服务端都有额外开销。长轮询只在数据更新或超时后返回，减少了无效请求，不过挂起连接仍会占用资源。需要服务端主动推送且双方频繁通信时，可以使用WebSocket，代价是服务端配置更复杂。

1. 短轮询 短轮询的基本思路：

浏览器每隔一段时间向浏览器发送 http 请求，服务器端在收到请求后，不论是否有数据更新，都直接进行响应。这种方式实现的即时通信，本质上还是浏览器发送请求，服务器接受请求的一个过程，通过让客户端不断的进行请求，使得客户端能够模拟实时地收到服务器端的数据的变化。

优缺点

优点是简单，易于理解。缺点是这种方式由于需要不断的建立 http 连接，严重浪费了服务器端和客户端的资源。当用户增加时，服务器端的压力就会变大，这是很不合理的。

2. 长轮询 长轮询的基本思路：首先由客户端向服务器发起请求，当服务器收到客户端发来的请求后，服务器端不会直接进行响应，而是先将这个请求挂起，然后判断服务器端数据是否有更新。如果有更新，则进行响应，如果一直没有数据，则到达一定的时间限制才返回。客户端

JavaScript

响应处理函数会在处理完服务器返回的信息后，再次发出请求，重新建立连接。

优缺点

长轮询和短轮询比起来，它的优点是明显减少了很多不必要的

http

请求次数，相比之下节约了资源。长轮询的缺点在于，连接挂起也会导致资源的浪费

3. WebSocket

WebSocket

是

Html5

定义的一个新协议，与传统的

http

协议不同，该协议允许由服务器主动的向客户端推送信息。使用

WebSocket

协议的缺点是在服务器端的配置比较复杂。WebSocket

是一个全双工的协议，也就是通信双方是平等的，可以相互发送消息。

1) 订单列表每分钟刷新一次，候选人说短轮询不算实时通信、必须改成WebSocket，你会怎么判断？

答：短轮询仍能模拟实时更新，只是实时程度受轮询间隔限制，并非不能用于状态刷新。若业务允许分钟级延迟，它实现简单且可能足够；强行使用WebSocket会增加服务端配置复杂度，而收益未必匹配。

2) 客服工作台要等待工单状态变化，绝大多数请求期间都没有更新，你会怎样把短轮询改成长轮询？

答：客户端发起请求后，服务端先挂起，检测到状态更新时再响应，没有更新则到超时后返回。客户端处理完响应后立即建立下一次请求，这能减少短轮询产生的大量无效请求；挂起连接本身仍会占用服务端资源。

3) 在线协作页面要求浏览器和服务器都能随时发送操作，负责人仍想用长轮询统一所有通信，你会指出什么限制？

答：长轮询主要依赖客户端持续发起并重新建立HTTP请求，服务端只能借助已挂起的请求返回数据。双向交互频繁时，这种模型管理成本较高；WebSocket提供全双工通信，更符合双方都需要主动发送消息的约束。

4) 长轮询接口上线后连接数持续升高、很多请求长期处于等待状态，但业务更新很少，你会先排查哪里？

答：先检查服务端是否为挂起请求设置了明确的超时，以及客户端在响应后是否只创建一个后继请求。长轮询减少了无效请求次数，却不会消除连接占用；超时过长或重复发起请求都会放大资源压力。

5) 十万级用户的状态页在短轮询、长轮询和WebSocket之间选型，业务只说要“尽量实时”，你会要求先明确什么？

答：需要先明确更新频率、可接受延迟、通信是否双向，以及服务端能承受的请求和连接规模。短轮询简单但反复建立请求，长轮询减少无效请求却会挂起连接，WebSocket支持全双工但服务端配置更复杂；没有这些约束就无法可靠选型。

684. 说一说正向代理和反向代理

正向代理隐藏真实客户端，反向代理隐藏真实服务端，区别就在于代理替哪一侧转发请求。使用正向代理时，目标服务只看到代理服务器发出的请求，不知道后面的真实客户端是谁。使用反向代理时，客户端只访问统一入口，不需要知道请求最终落到哪台真实服务器。反向代理常用于集群负载均衡；也可以通过DNS返回多个IP分流，但缓存可能导致故障地址暂时仍被返回。

正向代理 我们常说的代理也就是指正向代理，正向代理的过程，它隐藏了真实的请求客户端，服务端不知道真实的客户端是谁，客户端请求的服务都被代理服务器代替来请求。

反向代理 这种代理模式下，它隐藏了真实的服务端，当我们向一个网站发起请求的时候，背后可能有成千上万台服务器为我们服务，具体是哪一台，我们不清楚，我们只需要知道反向代理服务器是谁就行，而且反向代理服务器会帮我们请求转发到真实的服务器那里去，一般而言反向代理服务器一般用来实现负载均衡。

负载均衡的两种实现方式？ 一种是使用反向代理的方式，用户的请求都发送到反向代理服务上，然后由反向代理服务器来转发请求到真实的服务器上，以此来实现集群的负载均衡。另一种是 DNS 的方式，DNS 可以用于在冗余的服务器上实现负载均衡。因为现在一般的大型网站使用多台服务器提供服务，因此一个域名可能会对多个服务器地址。当用户向网站域名请求的时候，DNS 服务器返回这个域名所对应的服务器 IP 地址的集合，但在每个回答中，会循环这些 IP 地址的顺序，用户一般会选择排在前面的地址发送请求。以此将用户的请求均衡的分配到各个不同的服务器上，这样来实现负载均衡。这种方式有一个缺点就是，由于 DNS 服务器中存在缓存，所以有可能一个服务器出现故障后，域名解析仍然返回的是那个 IP 地址，就会造成访问的问题。

1) 浏览器通过公司代理访问外部文档站，服务端日志只看到代理地址；同事却称这是反向代理，你怎么纠正？

答：这里是正向代理，因为代理代表客户端向外部服务发起请求，外部服务看不到真实客户端。反向代理隐藏的是后方真实服务器，用户只知道统一入口；判断关键在代理代表请求方还是服务方。

2) 本地开发把/api交给devServer.proxy转发到后端，浏览器不再报跨域，你会怎样解释这条链路？

答：对浏览器而言，请求仍发往同源的开发服务器，再由该服务器转发到真实接口，因此属于反向代理场景。浏览器不知道后端实例是谁，服务器间转发也不受浏览器同源策略约束；线上若不保留同域入口，跨域约束仍会出现。

3) 一个域名后有多台应用服务器，平台团队要求用户不能感知扩缩容，你会把负载均衡放在哪里？

答：可让用户统一访问反向代理，再由代理把请求分配到真实服务器，扩缩容只需调整代理后的节点集合。这样隐藏了服务端拓扑并集中调度；反向代理自身也成为关键入口，需要具备足够容量和可用性。

4) 机房已删除故障服务器的DNS地址，部分用户却仍持续命中它，应用负责人认为删除记录应立即生效，你怎么排查？

答：先查询受影响用户实际拿到的地址，并核对各级DNS缓存及记录的TTL。缓存未过期时，客户端仍可能使用已删除的故障地址；若要求更快摘除，应评估由反向代理集中选择健康后端，而不能只依赖解析变更。

5) 流量入口评审中，一方主张DNS轮换多个地址，另一方主张所有请求经过反向代理，你如何说明取舍？

答：DNS可返回多个服务器地址并轮换顺序，部署分散，但缓存会降低故障切换的及时性。反向代理能集中转发和调整后端，更容易隐藏节点变化；相应地，它承担全部入口流量，必须额外保障自身稳定性。

685. 介绍一下Connection:keep-alive

Connection: keep-alive表示多个HTTP请求可以复用同一条TCP连接，而不是每次请求结束后立即断开。这样能减少反复建立和关闭连接带来的响应时间、CPU资源及拥堵开销，适合同一客户端连续请求多个资源的场景。在HTTP/1.0中需要显式发送Connection:

keep-alive才会启用。在HTTP/1.1中默认使用持久连接，如需关闭则发送Connection:

close，最终能否保持还取决于服务器配置。什么是keep-alive 我们知道HTTP协议采用“请求-应答”模式，当使用普通模式，即非KeepAlive模式时，每个请求/应答客户端和服务端都要新建一个连接，完成之后立即断开连接（HTTP协议为无连接的协议）；

当使用Keep-Alive模式（又称持久连接、连接重用）时，Keep-Alive功能使客户端到服务器端的连接持续有效，当出现对服务器的后继请求时，Keep-Alive功能避免了建立或者重新建立连接。

为什么要使用keep-alive

keep-alive技术的创建目的，能在多次HTTP之前重用同一个TCP连接，从而减少创建/关闭多个TCP连接的开销（包括响应时间、CPU资源、减少拥堵等），参考如下示意图 客户端如何开启在HTTP/1.0协议中，默认是关闭的，需要在http头加入"Connection:

Keep-Alive”，才能启用Keep-Alive； Connection: keep-alive http 1.1中默认启用Keep-Alive，如果加入"Connection: close “，才关闭。 Connection: close 目前大部分浏览器都是用http1.1协议，也就是说默认都会发起Keep-Alive的连接请求了，所以是否能完成一个完整的Keep-Alive连接就看服务器设置情况。

1) 商品页连续请求十个资源，候选人说keep-alive会把十个响应合成一次返回，你会怎么纠正？

答：keep-alive重用的是同一条TCP连接，并不会把多个HTTP请求或响应合并。每次交互仍遵循请求-应答模式，只是避免反复建立和关闭连接，从而减少握手、响应时间与资源开销。

2)

一个仍使用HTTP/1.0的内部接口每次请求后都断开，客户端想复用连接，应检查什么配置？

答：HTTP/1.0默认不启用持久连接，应检查请求是否携带Connection: keep-alive，以及服务器是否支持并接受连接复用。只修改客户端头部并不能保证成功，完整的持久连接还取决于服务端设置。

3) 网关从HTTP/1.0升到HTTP/1.1后，团队仍在所有请求里显式添加Connection: keep-alive，这有必要吗？

答：在HTTP/1.1中持久连接默认启用，通常无需再显式添加Connection: keep-alive。更应检查是否有人发送了Connection: close，因为它会要求请求完成后关闭连接；最终能否复用仍受服务器配置影响。

4) 接口耗时没有变化，但瀑布图里每个请求都重新建立连接，页面整体变慢，你会从哪里定位？

答：先确认请求或响应中是否出现Connection: close，再检查服务器是否允许并维持持久连接。若连接无法复用，每个请求都会承担新建与关闭TCP连接的开销；仅在前端声明keep-alive不能覆盖服务端主动断开的配置。

5)

运维认为连接复用能降低开销，因此准备让所有空闲连接永久保留，你会接受这个结论吗？

答：连接复用确实能减少重复建立和关闭TCP连接带来的时间、计算与拥塞成本，但不等于连接应永久存在。具体维持策略必须由客户端和服务端共同支持，并结合服务端资源约束设置；否则持久连接本身也可能成为负担。

686. http/https 协议总结

HTTP负责应用层通信，HTTPS则通过证书和SSL加密提升传输安全性，默认使用443端口。HTTP/1.1用长连接复用连接，并通过Host、身份认证和缓存等能力弥补HTTP/1.0的不足。HTTP/2进一步引入多路复用、二进制分帧、首部压缩和服务端推送。缓存上，未过期时走Cache-Control或Expires强缓存，过期后再用Etag或Last-Modified协商，且前者优先级更高。1.0 协议缺陷: 无法复用链接，完成即断开，重新慢启动和TCP 3次握手head of line blocking: 线头阻塞，导致请求之间互相影响 1.1 改进: 长连接(默认keep-alive)，复用host字段指定对应的虚拟站点新增功能:断点续传身份认证状态管理cache缓存Cache-ControlExpiresLast-ModifiedEtag 断点续传身份认证状态管理cache缓存Cache-ControlExpiresLast-ModifiedEtag 2.0: 多路复用二进制分帧层: 应用层和传输层之间首部压缩服务端推送 https: 较为安全的网络传输协议 证书(公钥)SSL加密端口443 TCP: 三次握手四次挥手滑动窗口: 流量控制拥塞处理慢开始拥塞避免快速重传快速恢复

慢开始拥塞避免快速重传快速恢复 缓存策略: 可分为 强缓存 和 协商缓存 Cache-Control/Expires: 浏览器判断缓存是否过期，未过期时，直接使用强缓存，Cache-Control的max-age优先级高于Expires当缓存已经过期时，使用协商缓存唯一标识方案:Etag(response携带)

&If-None-Match(request携带，上一次返回的Etag):

服务器判断资源是否被修改最后一次修改时间:Last-Modified(response) &If-Modified-Since(request，上一次返回的Last-Modified)如果一致，则直接返回 304 通知浏览器使用缓存如不一致，则服务端返回新的资源Last-Modified缺点: 周期性修改，但内容未变时，会导致缓存失效最小粒度只到s，s以内的改动无法检测到Etag的优先级高于Last-Modified

唯一标识方案:Etag(response携带) &If-None-Match(request携带，上一次返回的Etag):

服务器判断资源是否被修改最后一次修改时间:Last-Modified(response) &If-Modified-Since(request，上一次返回的Last-Modified)如果一致，则直接返回 304 通知浏览器使用缓存如不一致，则服务端返回新的资源 如果一致，则直接返回 304 通知浏览器使用缓存如不一致，则服务端返回新的资源

通知浏览器使用缓存如不一致，则服务端返回新的资源

周期性修改，但内容未变时，会导致缓存失效最小粒度只到s，s以内的改动无法检测到

1) 首页在HTTP/1.1下开了长连接，候选人据此断言多个资源已经不会互相阻塞，你会怎么追问？

答：长连接解决的是重复建连问题，不等于消除同一连接上的请求阻塞。HTTP/2通过二进制分帧和多路复用改善多个请求相互影响，并配合首部压缩；仅看到keep-alive不能推断已获得这些能力。

2) 静态站升级协议时，网关团队问HTTP/1.1相比HTTP/1.0能带来哪些直接收益，你会抓哪些配置验收？

答：应确认默认长连接是否生效，并检查Host是否正确路由到虚拟站点，同时验收断点续传、认证、状态管理和缓存相关行为。升级协议并不保证服务端配置自动正确，连接被关闭或缓存头缺失仍会抵消收益。

3) 资源服务器已配置Expires，应用又返回了Cache-Control: max-age，两者给出的过期时间冲突时浏览器应依据哪个？

答：浏览器应优先采用Cache-Control的max-age判断强缓存是否过期，Expires的优先级更低。未过期可直接使用本地缓存；过期后还需进入协商缓存流程，不能把配置了过期时间等同于永远不请求服务器。

4) 发版后一张图片内容变了，但请求携带If-Modified-Since后仍收到304，修改发生在同一秒内，你会怎样解释并修正？

答：Last-Modified的时间粒度只能到秒，同一秒内的修改可能无法被准确识别，因此服务器可能判断资源未变化。可同时采用ETag与If-None-Match校验内容标识，而且ETag的判断优先级更高；代价是服务端需要生成和比较标识。

5) 构建任务每次都会改写文件时间，但产物内容没有变化，线上却频繁重新下载，你会选择哪种协商缓存依据？

答：仅依赖Last-Modified会把时间变化视为资源变化，即使实际内容相同也可能让缓存失效。更适合使用ETag与If-None-Match判断资源标识，未修改时返回304；服务端必须保证标识生成规则稳定且能反映内容变化。

6) 安全评审要求站点从HTTP切到HTTPS，产品却认为只要端口改成443就完成了，你会指出缺少什么？

答：HTTPS不只是更换端口，还需要证书提供公钥信息，并通过SSL建立加密传输。证书与加密配置缺失时，无法获得预期的安全通信；同时它解决的是传输安全，不能替代应用自身的认证和状态管理。

687. TCP为什么要三次握手

TCP需要三次握手，是为了确认双方都具备正常的发送和接收能力，并避免失效请求误建连接。前两次通信只能让客户端确认服务端可收可发，服务端还不知道客户端能否收到自己的确认，所以需要第三次确认。这样即使旧的连接请求延迟到达，服务端收不到最终确认，也不会一直为无效连接等待数据。第三次握手时客户端已进入ESTABLISHED状态，因此可以携带数据；断开时双方关闭发送方向的时机可能不同，所以通常需要四次挥手。客户端和服务端都需要直到各自可收发，因此需要三次握手。第一次握手成功让服务端知道了客户端具有发送能力，第二次握手成功让客户端知道了服务端具有接收和发送能力，但此时服务端并不知道客户端是否接收到了自己发送的消息。为了防止出现失效的连接请求报文段被服务端接收的情况，从而产生错误。所以第三次握手就起到了这个作用。

你可以会问，2次握手就足够了？。但其实不是，因为服务端还没有确定客户端是否准备好了。比如步骤 3 之后，服务端马上给客户端发送数据，这个时候客户端可能还没有准备好接收数据。因此还需要增加一个过程。

TCP有6种标示:SYN(建立联机) ACK(确认) PSH(传送) FIN(结束) RST(重置) URG(紧急)

举例：已失效的连接请求报文段 client发送了第一个连接的请求报文，但是由于网络不好，这个请求没有立即到达服务端，而是在某个网络节点中滞留了，直到某个时间才到达server本来这已经是一个失效的报文，但是server端接收到这个请求报文后，还是向client发出确认的报文，表示同意连接。假如不采用三次握手，那么只要server发出确认，新的建立就连接了，但其实这个请求是失效的请求，client是不会理睬server的确认信息，也不会向服务端发送确认的请求。但是server认为新的连接已经建立起来了，并一直等待client发来数据，这样，server的很多资源就没白白浪费掉了。采用三次握手就是为了防止这种情况的发生，server会因为收不到确认的报文，就知道client并没有建立连接。这就是三次握手的作用。

三次握手过程中可以携带数据吗？第一次、第二次握手不可以携带数据，因为一握二握时还没有建立连接，会让服务器容易受到攻击。而第三次握手，此时客户端已经处于ESTABLISHED (已建立连接状态)，对于客户端来说，已经建立起连接了，并且也已经知道服务器的接收、发送能力是正常的了，所以能携带数据也是没问题的。

为什么建立连接只通信了三次，而断开连接却用了四次？
客户端要求断开连接，发送一个断开的请求，这个叫作（FIN）。服务端收到请求，然后给客户端一个ACK，作为 FIN 的响应。这里你需要思考一个问题，可不可以像握手那样马上传 FIN 回去？其实这个时候服务端不能马上传

FIN，因为断开连接要处理的问题比较多，比如说服务端可能还有发送出去的消息没有得到 ACK；也有可能服务端自己有资源要释放。因此断开连接不能像握手那样操作——将两条消息合并。所以，服务端经过一个等待，确定可以关闭连接了，再发一条 FIN 给客户端。客户端收到服务端的FIN，同时客户端也可能有自己的事情需要处理完，比如客户端有发送给服务端没有收到的 ACK 的请求，客户端自己处理完成后，再给服务端发送一个 ACK。

为了确保数据能够完成传输。因为当服务端收到客户端的 FIN 报文后，发送的 ACK 报文只是用来应答的，并不表示服务端也希望立即关闭连接。

当只有服务端把所有的报文都发送完了，才会发送 FIN 报文，告诉客户端可以断开连接了，因此在断开连接时需要四次挥手。关闭连接时，当收到对方的 FIN 报文通知时，它仅仅表示对方没有数据发送给你了；但未必你所有的数据都全部发送给对方了所以你未必会马上关闭 SOCKET，也即你可能还需要发送一些数据给对方之后，再发送 FIN 报文给对方来表示你同意现在可以关闭连接了，所以它这里的 ACK 报文和 FIN 报文多数情况下都是分开发送的。

1) 客户端发出 SYN 后，只要服务端返回 SYN+ACK 就算双方都具备收发能力，这个判断哪里不成立？

答：此时客户端已确认服务端能接收并发送，但服务端只能确认客户端能发送，尚不知道客户端能否收到自己的报文。客户端再发一次 ACK，服务端才能确认双向收发链路均正常，因此两次握手不足以完成双方确认。

2) 一条旧的连接请求在网络节点滞留，原连接早已结束后才到达服务端；如果协议只握手两次，服务端会出现什么工程后果？

答：服务端回复确认后就可能误认为新连接已经建立，并为它保留连接状态、等待客户端发送数据。客户端知道该请求已经失效，不会响应服务端；第三次握手缺失时，服务端无法及时识别这种半开连接，资源会被无效占用。

3) 同事想把第三次握手改成“服务端返回确认后立即下载业务数据”，以减少首页等待，这个方案为什么有风险？

答：服务端收到首个请求时还没有确认客户端能接收，也不知道客户端是否仍认可这次连接，立即发送业务数据缺少可靠前提。第三次 ACK 到达后，服务端才确认客户端准备完成；握手规则不能由业务层为节省一次确认而随意省略。

4) 线上连接大量停留在服务端等待确认的阶段，却一直没有进入已建立状态，你会沿着哪条握手链路排查？

答：先确认客户端的首个连接请求是否到达，再检查服务端确认报文能否返回客户端，以及客户端最终 ACK 是否被网络设备丢弃。若服务端持续收不到第三次确认，它不会把连接视为完整建立；还应排查防火墙、链路丢包和失效请求，而不是先归因于业务数据处理。

5) 为什么第三次握手可以携带数据，而前两次通常不应携带业务数据？

答：发出第三次报文时，客户端已经确认服务端具备收发能力，并进入连接已建立状态，因此该报文具备承载数据的条件。前两次发生在连接尚未完成确认时，服务端若提前接收和处理数据，会扩大无效连接或恶意请求造成的资源风险。

6) 评审中有人认为建立连接三次、关闭连接也应该固定三次，你会如何解释关闭通常需要四次？

答：收到对方的 FIN 只表示对方不再发送数据，不代表本端的数据已经发送完毕，因此本端通常先单独回复 ACK。待剩余报文和资源处理完成后再发送自己的 FIN，对方最后确认；只有关闭确认与本端 FIN 恰好可合并时，报文数量才可能变化。

688. 为什么要有 WebSocket

需要 WebSocket，本质上是因为 HTTP 的请求-应答模式不适合服务端主动推送和实时双向通信。轮询虽然能模拟实时效果，但会反复产生无效请求，浪费带宽和计算资源。WebSocket 建立后支持全双工通信，并使用二进制帧传输，适合即时消息、动态页面和网络游戏。它先通过带有 Connection:

Upgrade 和 Upgrade: websocket 的 HTTP GET 请求握手，服务端返回 101 Switching

Protocols后切换协议，但连接、缓存和状态需要应用自己管理。已经有了被广泛应用的 HTTP 协议，为什么要再出一个 WebSocket 呢？它有哪些好处呢？其实 WebSocket 与 HTTP/2 一样，都是为了解决 HTTP 某方面的缺陷而诞生的。HTTP/2 针对的是“队头阻塞”，而WebSocket 针对的是“请求 - 应答”通信模式。那么，“请求 - 应答”有什么不好的地方呢？“请求 - 应答”是一种“半双工”的通信模式，虽然可以双向收发数据，但同一时刻只能一个方向上有动作，传输效率低。更关键的一点，它是一种“被动”通信模式，服务器只能“被动”响应客户端的请求，无法主动向客户端发送数据。虽然后来的 HTTP/2、HTTP/3 新增了 Stream、Server Push 等特性，但“请求 - 应答”依然是主要的工作方式。这就导致 HTTP 难以应用在动态页面、即时消息、网络游戏等要求“实时通信”的领域。在 WebSocket 出现之前，在浏览器环境里用 JavaScript 开发实时 Web 应用很麻烦。因为浏览器是一个“受限的沙盒”，不能用 TCP，只有 HTTP 协议可用，所以就出现了很多“变通”的技术，“轮询”（polling）就是比较常用的一种。简单地说，轮询就是不停地向服务器发送 HTTP 请求，问有没有数据，有数据的话服务器就用响应报文回应。如果轮询的频率比较高，那么就可以近似地实现“实时通信”的效果。但轮询的缺点也很明显，反复发送无效查询请求耗费了大量的带宽和 CPU 资源，非常不经济。所以，为了克服 HTTP “请求 - 应答”模式的缺点，WebSocket 就“应运而生”了。WebSocket 的特点 WebSocket 是一个真正“全双工”的通信协议，与 TCP 一样，客户端和服务端都可以随时向对方发送数据。WebSocket采用了二进制帧结构，语法、语义与 HTTP 完全不兼容，但因为它的主要运行环境是浏览器，为了便于推广和应用，就不得不“搭便车”，在使用习惯上尽量向 HTTP 靠拢，这就是它名字里“Web”的含义。服务发现方面，WebSocket 没有使用 TCP 的“IP 地址 + 端口号”，而是延用了 HTTP 的 URI 格式，但开头的协议名不是“http”，引入的是两个新的名字：“ws”和“wss”，分别表示明文和加密的 WebSocket 协议。WebSocket 的默认端口也选择了 80 和 443，因为现在互联网上的防火墙屏蔽了绝大多数的端口，只对 HTTP 的 80、443 端口“放行”，所以 WebSocket 就可以“伪装”成 HTTP 协议，比较容易地“穿透”防火墙，与服务器建立连接 ws://www.chrono.com ws://www.chrono.com:8080/srv wss://www.chrono.com:445/im?user_id=xxx WebSocket 的握手和 TCP、TLS 一样，WebSocket 也要有一个握手过程，然后才能正式收发数据。这里它还是搭上了 HTTP 的“便车”，利用了 HTTP 本身的“协议升级”特性，“伪装”成 HTTP，这样就能绕过浏览器沙盒、网络防火墙等等限制，这也是 WebSocket 与 HTTP 的另一个重要关联点。WebSocket 的握手是一个标准的 HTTP GET 请求，但要带上两个协议升级的专用头字段：“Connection: Upgrade”，表示要求协议“升级”；“Upgrade: websocket”，表示要“升级”成 WebSocket 协议。另外，为了防止普通的 HTTP 消息被“意外”识别成 WebSocket，握手消息还增加了两个额外的认证用头字段（所谓的“挑战”，Challenge）：Sec-WebSocket-Key：一个 Base64 编码的 16 字节随机数，作为简单的认证密钥；Sec-WebSocket-Version：协议的版本号，当前必须是 13。服务器收到 HTTP 请求报文，看到上面的四个字段，就知道这不是一个普通的 GET 请求，而是 WebSocket 的升级请求，于是不走普通的 HTTP 处理流程，而是构造一个特殊的“101 Switching Protocols”响应。

1) 聊天页面每秒轮询一次消息接口，产品说“HTTP也能双向传数据，没必要上WebSocket”，你会指出哪个关键差异？

答：HTTP的主要工作方式仍是客户端请求、服务端应答，服务端不能在无请求时主动推送新消息。高频轮询只能近似实时，还会产生大量无新数据的请求，消耗带宽和服务端处理资源；持续双向通信才是WebSocket更匹配的场景。

2) 行情页需要服务端持续推送报价，同时客户端随时发送订阅变更；为什么全双工比“请求后等待响应”更合适？

答：WebSocket建立连接后，客户端和服务端都能随时发送数据，不必等待另一方向完成一次请求一应答。报价推送与订阅变更可以在同一连接上独立发生；代价是应用必须自行管理连接、缓存和状态，开发复杂度高于普通HTTP。

3) 公司出口防火墙只放行常见Web端口，架构师仍建议自定义一个全新端口承载实时协议；WebSocket的选择解决了什么约束？

答：WebSocket沿用兼容HTTP的URI形式，并通常使用80或443端口，更容易通过现有防火墙和浏览器环境。它先借助HTTP完成协议升级，再切换到自己的二进制帧通信；这并不意味着升级后的数据仍是普通HTTP报文。

4) 线上WebSocket请求一直返回普通页面的200，客户端没有进入连接成功状态，你会先核对哪些握手字段和响应？

答：先检查客户端是否发送HTTP GET，并包含Connection: Upgrade、Upgrade: websocket、Sec-WebSocket-Key和版本字段。服务端应识别升级请求并返回101 Switching Protocols；若返回200，通常说明请求被当成普通HTTP，还需检查代理是否转发了升级头。

5) 通知中心一天只有少量事件，团队却准备为它维护长连接、重连、缓存和状态机；你会如何做选型取舍？

答：事件稀疏且实时性要求不高时，普通请求或适度轮询可能更简单，不必仅因服务端需要通知就引入WebSocket。WebSocket更适合即时消息、游戏等持续实时通信，但连接生命周期和状态需要应用自行管理；收益不足时，复杂度会成为主要成本。

6) 安全评审看到Sec-WebSocket-Key，便认为它能替代用户登录鉴权，这个结论为什么不可靠？

答：Sec-WebSocket-Key是握手中的随机挑战值，用于避免普通HTTP消息被意外识别为WebSocket，并不是业务用户身份凭证。用户权限仍需由应用层单独校验；把协议握手校验当成登录认证，会让未授权连接进入后续通信流程。

689. UDP和TCP有什么区别

TCP面向连接并提供可靠传输，UDP无连接且不保证数据可靠到达。TCP会给数据段编号，并通过确认、窗口和流量控制来保障传输，因此协议负载相对更高。UDP不做这些连接与确认工作，机制更简单，但数据是否收到需要上层自行处理。简单来说，需要可靠性的场景更适合TCP；能够接受丢失、希望减少协议负担时，可以考虑UDP。TCP协议在传送数据段的时候要给段标号；UDP协议不TCP协议可靠；UDP协议不可靠TCP协议是面向连接；UDP协议采用无连接TCP协议负载较高，采用虚电路；UDP采用无连接TCP协议的发送方要确认接收方是否收到数据段（3次握手协议）TCP协议采用窗口技术和流控制

1) 文件上传页面要求数据完整到达，同事却说“UDP更轻，丢了让用户重传整个文件就行”，这个选型忽略了什么？

答：文件上传需要有序、可靠地交付数据，TCP会给数据段编号，并通过确认、窗口和流量控制维护传输。UDP无连接且不提供这些可靠性保证，应用若自行补齐确认和重传会增加复杂度；只追求较低协议负担不能覆盖完整性要求。

2) 浏览器调用上传接口时，抓包显示连接建立前有往返确认；这些报文如何体现TCP与UDP的工程差异？

答：TCP面向连接，发送数据前要通过握手确认通信双方的能力，后续数据段还需要确认。UDP采用无连接方式，不先建立这种虚连接；因此它的协议负担较低，但也不会自动保证接收方收到每个数据段

。

3) 实时数据面板允许丢失个别过期采样，却要求新数据尽快送达；协议约束变化后，为什么仍不能只凭“UDP更快”拍板？

答：允许少量丢失会降低对传输可靠性的要求，使无连接的UDP成为候选，但source只能支持其负担较低、缺少可靠确认。仍需由业务明确乱序、丢失和拥塞时的处理方式；若最终要求完整交付，就必须在应用层补偿或改用TCP。

4) 线上日志显示发送端已经调用成功，但接收端缺少若干报文；使用UDP时为什么不能直接判定是服务端漏处理？

答：UDP本身不要求接收方对数据段逐一确认，也不保证报文可靠到达，因此发送调用成功不等于对端已经收到。排查时要同时检查链路丢包、接收缓冲和应用处理；若业务不能容忍缺失，应增加应用层确认机制或选择可靠传输。

5) 批量接口出现接收方处理不过来的现象，评审时如何用流量控制能力比较TCP和UDP？

答：TCP采用窗口技术和流量控制，可根据接收能力约束发送过程，并配合确认维护可靠传输。UDP不提供同等级的内建机制，发送节奏需要应用自行管理；高吞吐并不等于可以忽略接收方容量，否则丢失和过载风险会转移到业务层。

6) 同事把“TCP三次握手”和“每个数据段的确认”混成一件事，你会怎样纠正这两个相邻概念？

答：三次握手用于建立面向连接的通信状态，确认双方具备相应收发能力；数据传输阶段的确认则用于判断数据段是否到达。二者都服务于可靠性，但发生阶段和目的不同；UDP既不建立这种连接，也不内建逐段可靠确认。

【9种前端常见的设计模式】

690. 外观模式

外观模式就是给一组复杂的子系统接口提供一个统一、简单的高层入口，让调用方更容易使用。比如封装浏览器事件绑定时，内部处理addEventListener、attachEvent等兼容逻辑，外部只调用一个方法。它适合用来隔离系统分层、简化持续演化的子系统，或为遗留代码提供清晰入口。代价是外观接口变化时往往需要直接修改，不太符合开闭原则。外观模式是最常见的设计模式之一，它为子系统中的一组接口提供一个统一的高层接口，使子系统更容易使用。简而言之外观设计模式就是把多个子系统中复杂逻辑进行抽象，从而提供一个更统一、更简洁、更易用的API。很多我们常用的框架和库基本都遵循了外观设计模式，比如jQuery就把复杂的原生DOM操作进行了抽象和封装，并消除了浏览器之间的兼容问题，从而提供了一个更高级更易用的版本。其实在平时工作中我们也会经常用到外观模式进行开发，只是我们不自知而已

兼容浏览器事件绑定 let addMyEvent=function(el,ev,fn){if(el.addEventListener){el.addEventListener(ev,fn,false)}elseif(el.attachEvent){el.attachEvent('on'+ev,fn)}else{el['on'+ev]=fn}}; 封装接口 let myEvent = { // ... stop: e => { e.stopPropagation(); e.preventDefault(); } }; 场景 设计初期，应该要有意识地将不同的两个层分离，比如经典的三层结构，在数据访问层和业务逻辑层、业务逻辑层和表示层之间建立外观Facade在开发阶段，子系统往往因为不断的重构演化而变得越来越复杂，增加外观Facade可以提供一个简单的接口，减少他们之间的依赖。在维护一个遗留的大型系统时，可能这个系统已经很难维护了，这时候使用外观Facade也是非常合适的，为系统开发一个外观Facade类，为设计粗糙和高度复杂的遗留代码提供比较清晰的接口，让新系统和Facade对象交互，Facade与遗留代码交互所有的复杂工

作。 优点 减少系统相互依赖。提高灵活性。提高了安全性 缺点
不符合开闭原则，如果要改东西很麻烦，继承重写都不合适。

1) 页面把addEventListener、attachEvent和onclick的兼容判断复制到二十个组件里，这只是工具函数缺失，还是适合引入外观模式？

答：这适合用外观模式收敛：对外只暴露统一的事件绑定接口，内部选择浏览器支持的实现。调用方不再依赖多个底层接口和兼容分支；但外观应保持职责集中，否则不断加入无关能力会变成新的复杂子系统。

2) 业务层同时直接调用缓存、请求和错误提示模块，重构其中一个模块时几十个页面都要修改；外观层应放在哪里？

答：可在业务层与这些子系统之间建立统一的高层接口，让页面只依赖稳定的业务操作，外观再协调底层模块。这样能减少层间依赖并提高替换灵活性；若页面仍绕过外观直接访问子系统，隔离效果就会被破坏。

3) 遗留后台有多个命名混乱的接口，新页面只需要“查询并展示用户”，你会让新代码直接理解全部旧模块吗？

答：应为遗留系统增加一个边界清晰的Facade，向新页面提供简洁接口，并在内部适配旧代码的复杂调用。这样新系统只与外观交互，维护范围更可控；外观只能隔离复杂度，不能自动消除遗留实现本身的缺陷。

4) 统一request外观升级后，某个页面的特殊配置无法传入，开发者开始绕过它直接调用底层库；这暴露了什么设计故障？

答：这说明外观提供的高层接口没有覆盖必要能力，已经从简化使用变成限制使用。应核对真实调用场景，补充明确扩展点或受控的配置透传；若把所有底层参数原样暴露，又会削弱外观减少依赖的价值。

5) 架构评审要求所有子系统能力都只能通过一个巨大Facade暴露，以获得最高“安全性”，你会接受吗？

答：不会仅为统一入口就堆成单一巨大外观，应按稳定职责和层次划分接口，限制不必要的底层暴露。外观确实能保护子系统并减少依赖，但修改集中接口可能影响大量调用方，而且其扩展并不天然符合开闭原则。

6) 有人把外观模式和代理模式都理解为“包一层”，在接口聚合页面中如何区分它们？

答：外观面向一组复杂子系统，目标是提供更统一、更易用的高层接口，调用形式通常会被重新组织。代理则通常代表某个目标对象并控制对它的访问；若代码主要在聚合多个模块，而非代替单一对象，就更接近外观模式。

691. 代理模式

代理模式是给目标对象增加一个代用品或占位对象，由它控制调用方如何访问目标对象。代理位于双方之间，可以延迟转交请求、保护目标对象，也能在不修改目标对象的情况下扩展处理逻辑，从而降低耦合。前端常见例子是事件代理：在父元素监听点击，再通过event.target识别实际触发的子元素。它不是直接访问，因此会增加一层处理，并可能带来额外开销。

是为一个对象提供一个代用品或占位符，以便控制对它的访问

假设当A

在心情好的时候收到花，小明表白成功的几率有60%，而当A

在心情差的时候收到花，小明表白的成功率无限趋近于0。小明跟A

刚刚认识两天，还无法辨别A

什么时候心情好。如果不合时宜地把花送给A，花被直接扔掉的可能性很大，这束花可是小明吃了7

天泡面换来的。但是A的朋友B却很了解A，所以小明只管把花交给B，B会监听A的心情变化，然后选择A

心情好的时候把花转交给A，代码如下：

```
let Flower = function() {}  
let xiaoming = {  
  sendFlower: function(target) {  
    let flower = new Flower()  
    target.receiveFlower(flower)  
  }  
}  
let B = {  
  receiveFlower: function(flower) {  
    A.listenGoodMood(function() {  
      A.receiveFlower(flower)  
    })  
  }  
}  
let A = {  
  receiveFlower: function(flower) {  
    console.log('收到花'+ flower)  
  },  
  listenGoodMood: function(fn) {  
    setTimeout(function() {  
      fn()  
    }, 1000)  
  }  
}  
xiaoming.sendFlower(B)
```


场景 HTML元素事件代理

```
let ul = document.querySelector('#ul')  
ul.addEventListener('click', event => {  
  console.log(event.target)  
})
```


优点 代理模式能将代理对象与被调用对象分离，降低了系统的耦合度。代理模式在客户端和目标对象之间起到一个中介作用，这样可以起到保护目标对象的作用代理对象可以扩展目标对象的功能；通过修改代理对象就可以了，符合开闭原则；
缺点 处理请求速度可能有差别，非直接访问存在开销

1) 图片组件只是把原对象的方法改了个名字，没有控制访问时机或保护目标对象，这还能算代理模式吗？

答：仅改名通常不能体现代理模式，因为代理应作为目标对象的代用品或占位符，介入并控制对目标的访问。若中间层既不延迟、不保护也不扩展访问行为，它更像普通封装；额外一层还会带来调用开销。

2) 商品列表有一万个节点，团队准备给每个条目绑定点击监听；用父容器事件代理时，代理对象和目标对象分别是什么？

答：父容器统一接收冒泡后的点击事件，相当于访问入口，再根据event.target判断实际被点击的子元素。这样无需在每个条目上分别注册处理器，降低监听关系的耦合；同时必须处理冒泡路径和非目标节点，避免误触发。

3) 送花逻辑改为由中间对象等目标状态合适时再转交，这层代理解决了什么约束变化？

答：中间对象掌握目标状态，因此能控制真正访问目标的时机，而发送方仍只调用相同的接收接口。发送方与目标状态判断被分离，后续可修改代理逻辑而不改目标；若中间状态判断失准，请求仍可能延迟或无法送达。

4) 线上点击列表项偶尔触发父级空白区域的业务动作，代码使用了事件代理，你会先检查什么？

答：先检查监听器是否只依据父容器收到click就执行，而没有核对event.target或其祖先是否为目标条目。事件代理把多个子元素的访问集中到一个入口，匹配逻辑写错会扩大误触发范围；还要确认冒泡是否被其他处理器阻止。

5) 权限模块计划为每个服务对象增加校验代码，另一方案是在访问入口设置保护代理；你会如何取舍？

答：权限属于访问控制时，保护代理能把校验与目标对象分离，让客户端先经过中介再访问真实服务。修改代理即可调整控制逻辑，也更符合对目标扩展开放、修改收敛的思路；代价是多一层转发，并可能带来处理时延和排查复杂度。

6) 评审中有人说“代理只能控制访问，绝不能扩展功能”，这与代理模式的边界一致吗？

答：不完全一致，代理的核心是代表目标并控制访问，但代理对象也可以在中介过程中扩展目标对象的功能。关键是客户端仍通过代理间接访问目标，二者保持分离；若新增能力彻底改变职责或接口，则应重新判断是否更接近其他结构型模式。

692. 工厂模式

工厂模式把对象创建过程封装到统一接口中，让调用方只关心要什么，而不用了解具体怎样实例化。当系统需要根据名称或运行环境创建行为不同的不同对象时，工厂可以隔离构造逻辑，减少调用方与具体产品的耦合。新增产品通常通过扩展产品类和工厂完成，因而更容易扩展。若对象创建本来很简单，硬套工厂

会增加抽象层、理解成本和单元测试难度，此时直接使用构造器更合适。工厂模式定义一个用于创建对象的接口，这个接口由子类决定实例化哪一个类。该模式使一个类的实例化延迟到了子类。而子类可以重写接口方法以便创建的时候指定自己的对象类型。

```
class Product { constructor(name) { this.name = name } init() { console.log('init') } fun() { console.log('fun') } } class Factory { create(name) { return new Product(name) } } // use let factory = new Factory() let p = factory.create('p1') p.init() p.fun()
```

场景 如果你不想让某个子系统与较大的那个对象之间形成强耦合，而是想运行时从许多子系统中进行挑选的话，那么工厂模式是一个理想的选择将new操作简单封装，遇到new的时候就应该考虑是否用工厂模式；需要依赖具体环境创建不同实例，这些实例都有相同的行为,这时候我们可以使用工厂模式，简化实现的过程，同时也可以减少每种对象所需的代码量，有利于消除对象间的耦合，提供更大的灵活性

优点 创建对象的过程可能很复杂，但我们只需要关心创建结果。构造函数和创建者分离，符合“开闭原则”一个调用者想创建一个对象，只要知道其名称就可以了。扩展性高，如果想增加一个产品，只要扩展一个工厂类就可以。缺点 添加新产品时，需要编写新的具体产品类,一定程度上增加了系统的复杂度考虑到系统的可扩展性，需要引入抽象层，在客户端代码中均使用抽象层进行定义，增加了系统的抽象性和理解难度

什么时候不用 当被应用到错误的问题类型上时,这一模式会给应用程序引入大量不必要的复杂性.除非为创建对象提供一个接口是我们编写的库或者框架的一个设计上目标,否则我会建议使用明确的构造器,以避免不必要的开销。由于对象的创建过程被高效的抽象在一个接口后面的事实,这也会给依赖于这个过程可能会有多复杂的单元测试带来问题。

1) 商品页只有Product一种实例，构造函数也只接收name，同事仍要求先抽象AbstractFactory、再建具体工厂；你会接受吗？

答：暂时不会接受，直接使用明确的构造器更合适，因为对象类型单一、创建过程也没有环境分支。工厂只有在隔离复杂创建过程或支持多种产品扩展时才产生价值；过早引入抽象层会增加类数量和理解成本。

2) 低代码表单根据配置中的type创建Input、Select、DatePicker，渲染层已经堆了十几个if/else；你会怎样落地工厂？

答：把控件创建集中到工厂接口中，由type决定返回具有统一行为的具体控件，页面只接收创建结果。新增产品时扩展具体产品和工厂能力，不再让渲染层了解构造细节；代价是产品类型持续增加时，相关类和注册关系也会变多。

3) 同一套上传组件在浏览器创建WebUploader，在桌面容器创建NativeUploader，两者都提供upload()；环境判断应该放在哪里？

答：环境判断应收进工厂，由工厂在运行时选择并创建实现相同行为的实例，业务层只依赖统一接口。这样能隔离具体环境与调用代码，但必须确保两种产品真正满足相同契约；若行为差异很大，强行统一会把分支转移到实例内部。

4) 线上新增video类型后，详情页拿到的产品是undefined，随后调用init()报错；你会先查哪几处？

答：先核对配置名称、工厂分支或注册项，以及具体产品是否实现了约定的init()，确认故障发生在选择还是创建阶段。工厂应对未知名称显式报错或返回受控的兜底产品；静默返回undefined会把根因推迟到调用处。

5) 插件平台允许第三方增加控件，团队在“中心化switch工厂”和“插件各自注册工厂”之间争论；你怎么选？

答：若产品集合由核心团队封闭维护，中心化工厂更直观；若第三方需要独立扩展，注册式工厂更符合开放封闭原则。后者减少核心代码改动，却需要处理名称冲突、接口校验和注册时机，不能只靠一张无约束的映射表。

693. 单例模式

单例模式保证一个类最多只有一个实例，并提供统一的访问入口。在JavaScript中，可以用IIFE形成局部作用域，再由闭包中的getInstance()缓存并返回同一个对象，同时隐藏具体构造过程。它适合登录框，以及Vuex、Redux中需要贯穿系统的store。不过单点访问会引入全局状态和模块耦合，使依赖它的代码难以单独测试，所以不需要唯一实例时应尽量避免。顾名思义，单例模式中Class的实例个数最多为1。当需要一个对象去贯穿整个系统执行某些任务时，单例模式就派上了用场。而除此之外的场景尽量避免单例模式的使用，因为单例模式会引入全局状态，而一个健康的系统应该避免引入过多的全局状态。

实现单例模式需要解决以下几个问题：如何确定Class只有一个实例？如何简便的访问Class的唯一实例？Class如何控制实例化的过程？如何将Class的实例个数限制为1？

我们一般通过实现以下两点来解决上述问题：

隐藏Class的构造函数，避免多次实例化通过暴露一个getInstance()方法来创建/获取唯一实例

Javascript中单例模式可以通过以下方式实现：
`// 单例构造器 const FooServiceSingleton = (function () {
 // 隐藏的Class的构造函数 function FooService() {}
 // 未初始化的单例对象 let fooService;
 return {
 // 创建/获取单例对象的函数
 getInstance: function () {
 if (!fooService) {
 fooService = new FooService();
 }
 return fooService;
 }
 }
})();`
实现的关键点有：使用 IIFE 创建局部作用域并即时执行；getInstance() 为一个闭包，使用闭包保存局部作用域中的单例对象并返回。我们可以验证下单例对象是否创建成功：
`const fooService1 = FooServiceSingleton.getInstance();
const fooService2 = FooServiceSingleton.getInstance();
console.log(fooService1 === fooService2);` // true
场景例子
定义命名空间和实现分支型方法登录框vuex 和 redux中的store
优点 划分命名空间，减少全局变量增强模块性，把自己的代码组织在一个全局变量名下，放在单一位置，便于维护且只会实例化一次。简化了代码的调试和维护
缺点 由于单例模式提供的是一种单点访问，所以它有可能导致模块间的强耦合从而不利于单元测试。无法单独测试一个调用了来自单例的方法的类，而只能把它与那个单例作为一个单元一起测试。

1) 订单列表页每打开一个筛选面板都创建独立FilterModel，评审却要求改成单例，只因为“实例越少越好”；这个理由成立吗？

答：不成立，筛选状态属于各面板时，多个实例正是合理隔离，单例反而会让面板互相覆盖状态。单例适合确实需要由一个对象贯穿系统的任务；仅为减少实例而共享可变状态，会引入隐藏耦合。

2)

后台系统的登录框只能同时存在一个，多个入口都可能触发它；你会怎样控制创建和访问？

答：隐藏具体构造过程，通过getInstance()延迟创建并返回唯一登录框，所有入口使用同一访问点。闭包或模块局部作用域可保存该实例，避免外部重复实例化；同时要定义关闭后的状态重置，否则唯一实例可能残留上次交互数据。

3) 团队把Redux的store做成模块级唯一实例，后来测试要求每个用例使用不同初始状态；还应坚持单例吗？

答：生产入口可以持有唯一store，测试侧则应依赖构造函数或可注入接口，为每个用例生成隔离实例。若业务模块直接导入全局实例，就无法单独替换依赖，只能把模块与单例一起测试；改造会增加传参或容器配置成本。

4) 线上出现两个弹窗同时展示，日志显示getInstance()被调用两次；你会如何判断是单例失效还是状态逻辑失效？

答：先比较两次返回值是否严格相等，再检查构造函数是否仍被导出、单例变量是否被放进了每次调用都会重建的作用域。若实例相同，则继续查展示状态和重复挂载；单例只保证实例数量最多为一，不自动保证一次只执行一个业务动作。

3) 运营要求同一优惠计算器根据会员等级在运行时切换算法，而且后续允许活动模块注入新算法；策略由谁选择？

答：由上下文根据会员等级或活动配置选择策略，具体算法只承担计算行为并保持可替换。注入新策略能减少对主流程的修改，但调用方必须了解各策略的适用条件；选错策略不会被模式本身自动纠正。

4) 线上表单把规则写成minLength:8，校验器偶尔提示错误规则不存在；你会沿着什么链路排查？

答：先检查规则字符串的解析是否修改了共享数组，再核对解析出的策略名、参数顺序和策略表注册项。验证器应对未知策略显式失败，并避免在重复执行时用shift()破坏原配置；否则第二次校验可能得到不同结果。

5) 表单校验既能写成策略表，也能为每条规则创建校验类；团队担心几十个策略对象太碎，你怎么取舍？

答：规则主要是无状态函数时，策略表更轻量，也保留动态选择和独立扩展的价值；存在独立状态或复杂依赖时，再使用策略类更合适。策略模式会增加策略数量，命名、发现和配置管理不清晰时，维护成本可能超过条件分支。

6) 控件平台里，同一个映射表既返回组件实例又存放校验函数；评审说工厂模式和策略模式没有区别，你怎么解释？

答：返回Input或Select实例的部分关注创建哪种产品，属于工厂意图；选择并执行isMobile等算法的部分关注替换行为，属于策略意图。两者都可能用映射实现，但扩展契约不同，混在一张表里会模糊生命周期和错误处理。

695. 迭代器模式

迭代器模式提供统一的顺序遍历方式，同时不暴露集合内部的数据结构。它把“怎么取下一个元素”交给迭代器，因此调用方无须了解容器的具体实现，也不用修改容器原有接口。传统实现通常提供hasNext()和next()；在ES6中，对象实现[Symbol.iterator]并返回带next()的迭代器后，就能使用for...of。例如数字区间可以自行维护当前位置，逐个返回{ value, done }。如果你看到这，ES6中的迭代器 Iterator 相信你还是有点印象的，上面第60条已经做过简单的介绍。迭代器模式简单的说就是提供一种方法顺序一个聚合对象中各个元素，而又不暴露该对象的内部表示。迭代器模式解决了以下问题：提供一致的遍历各种数据结构的方式，而不用了解数据的内部结构提供遍历容器（集合）的能力而无需改变容器的接口

一个迭代器通常需要实现以下接口：

hasNext()：判断迭代是否结束，返回Boolean
next()：查找并返回下一个元素

为Javascript的数组实现一个迭代器可以这么写：

```
const items = [1, 'red', false, 3.14];  
function Iterator(items) {  
  this.items = items;  
  this.index = 0;  
}  
Iterator.prototype = {  
  hasNext: function() { return this.index < this.items.length; },  
  next: function() { return { value: this.items[this.index++], done: false }; }  
};  
验证一下迭代器是否工作：  
const iterator = new Iterator(items);  
while (iterator.hasNext()) { console.log(iterator.next()); }  
// 输出：1, red, false, 3.14  
ES6提供了更简单的迭代循环语法 for...of，使用该语法的前提是操作对象需要实现可迭代协议（The iterable protocol），简单说就是该对象有个Key为Symbol.iterator的方法，该方法返回一个iterator对象。比如我们实现一个 Range 类用于在某个数字区间进行迭代：  
function Range(start, end) {  
  return {  
    [Symbol.iterator]: function () {  
      return {  
        next: function () {  
          if (start < end) {  
            return { value: start++, done: false };  
          }  
          return { done: true, value: end };  
        }  
      };  
    }  
  };  
}  
验证一下：  
for (num of Range(1, 5)) { console.log(num); }  
// 输出：1, 2, 3, 4
```

1)

报表页只读取固定三项数组，同事专门实现带hasNext()、next()的迭代器类；你会保留吗？

答：若数组结构不会替换且没有特殊遍历规则，直接使用现有循环更清晰，自定义迭代器收益有限。迭代器的价值在于统一不同聚合对象的遍历接口并隐藏内部表示；简单集合强行包装只会增加状态和边界处理。

2) 组件库的分页数据内部按多个批次保存，但业务方只想用统一循环顺序读取；你会怎样暴露接口？

答：让容器提供迭代器，把批次索引和元素索引封装在内部，调用方只通过next()获取下一个值。若希望接入for...of，则实现Symbol.iterator并返回符合协议的迭代器；容器结构变化时，消费代码无需同步改写。

3) 组织架构从扁平数组改成树结构，但搜索页仍要求按既定顺序消费员工节点；哪些代码应该变化？

答：应调整容器内部的迭代实现，让它按约定顺序产出树节点，搜索页继续依赖统一遍历接口。这样无需向调用方暴露树的栈、队列或层级细节；不过遍历顺序必须成为明确契约，否则结构迁移可能悄悄改变结果。

4) 自定义Range(1, 5)在线上循环时意外输出了5，而需求是只输出1到4；你会检查什么？

答：检查next()在结束分支返回的对象是否正确标记done: true，以及递增和边界判断发生的顺序。迭代协议以done决定值是否被消费，结束对象即使携带value:

5也不应进入for...of；手写消费逻辑则需同样遵守该标记。

5) 数据容器团队想只提供hasNext()和next()，页面团队坚持必须支持for...of；这两种接口如何取舍？

答：内部专用且调用点受控时，显式的hasNext()、next()已能完成顺序遍历；面向通用JavaScript消费者时，实现可迭代协议更容易接入for...of。后者需要正确提供Symbol.iterator和结束状态，但能与语言统一遍历语法协作。

696. 观察者模式

观察者模式是由被观察对象维护一组观察者，并在自身状态变化时统一通知它们。被观察对象通常提供subscribe()、unsubscribe()和fire()，分别负责订阅、取消订阅和广播通知，从而让双方依赖抽象而不是具体实现。DOM事件和Vue响应式都属于常见使用场景。它能提升扩展性和复用性，但过度使用会弱化对象间的显式联系，让调用链更难跟踪和维护。观察者模式又称发布-订阅模式（Publish/Subscribe Pattern），是我们经常接触到的设计模式，日常生活中的应用也比比皆是，比如你订阅了某个博主的频道，当有内容更新时会收到推送；又比如JavaScript中的事件订阅响应机制。观察者模式的思想用一句话描述就是：被观察对象（subject）维护一组观察者（observer），当被观察对象状态改变时，通过调用观察者的某个方法将这些变化通知到观察者。观察者模式中Subject对象一般需要实现以下API：

subscribe(): 接收一个观察者observer对象，使其订阅自己unsubscribe():

unsubscribe(): 接收一个观察者observer对象，使其取消订阅自己fire():

触发事件，通知到所有观察者

用JavaScript手动实现观察者模式：

```
// 被观察者
function Subject() { this.observers = []; }
Subject.prototype = { // 订阅
  subscribe: function (observer) { this.observers.push(observer); }, // 取消订阅
  unsubscribe: function (observerToRemove) { this.observers = this.observers.filter(observer => { return observer !== observerToRemove; }) }, // 事件触发
  fire: function () { this.observers.forEach(observer => { observer.call(); }); } }
// 验证一下订阅是否成功：
const subject = new Subject();
function observer1() { console.log('Observer 1 Firing!'); }
function observer2() { console.log('Observer 2 Firing!'); }
subject.subscribe(observer1);
subject.subscribe(observer2);
```

subject.fire(); //输出： Observer 1 Firing! Observer 2 Firing! 验证一下取消订阅是否成功：
subject.unsubscribe(observer2); subject.fire(); //输出： Observer 1 Firing! 场景 DOM事件
document.body.addEventListener('click', function() { console.log('hello world!'); });
document.body.click() vue 响应式 优点 支持简单的广播通信，自动通知所有已经订阅过的对象目标对象与观察者之间的抽象耦合关系能单独扩展以及重用增加了灵活性观察者模式所做的工作就是在解耦，让耦合的双方都依赖于抽象，而不是依赖于具体。从而使得各自的变化都不会影响到另一边的变化。 缺点 过度使用会导致对象与对象之间的联系弱化，会导致程序难以跟踪维护和理解

1) 详情页只有父组件和一个子组件，数据通过属性就能传递，同事却增加全局事件中心来同步标题；你赞成吗？

答：不赞成，明确的父子依赖用直接传递更容易追踪，全局订阅会弱化对象之间的可见联系。观察者模式适合一个状态源需要广播给多个可独立扩展的观察者；在局部单向关系中使用会增加维护难度。

2) 后台导航栏、个人中心和权限面板都要响应登录状态变化，你会怎样设计订阅接口？

答：让登录状态主体维护观察者集合，并提供subscribe()、unsubscribe()和触发通知的fire()。各模块只注册自己的响应函数，不直接依赖其他模块，从而能够独立扩展；组件销毁时必须取消订阅，避免无效回调继续存在。

3) 消息中心原来只有两个订阅者，现在插件也能动态订阅和卸载；主体通知期间有人取消订阅，你会关注什么？

答：需要明确通知时修改观察者集合的语义，例如本轮是否仍调用已进入通知序列的观察者，并让实现保持一致。可在触发前使用观察者快照，避免遍历中的增删造成漏通知或重复通知；代价是本轮变化通常到下一次触发才完全生效。

4) 单页应用运行数小时后越来越卡，性能记录显示已卸载页面的回调仍在登录事件触发时执行；你怎么定位？

答：先记录每次subscribe()与unsubscribe()的观察者身份，核对组件卸载路径是否传回同一个函数引用。再检查主体的观察者数组是否持续增长，并为重复订阅建立防护；只清理页面DOM不会自动移除手写订阅。

5) 支付结果需要通知订单页、消息角标和埋点模块，团队在直接逐个调用与观察者模式之间争论；你怎么选？

答：订阅方可能独立增删且支付模块不应了解它们时，观察者模式更利于解耦和广播；接收者固定、调用顺序与失败处理严格时，显式调用更容易控制。观察者会弱化调用链，线上排查需补充事件名称、订阅来源和触发记录。

6) 前端同事说DOM事件和Vue响应式都体现了观察者思想，但准备把所有跨模块通信都改成事件；你会怎么制止过度设计？

答：应按依赖关系选择机制，而不是因为底层具有订阅通知能力就统一事件化。DOM事件和响应式适合各自的状态传播场景，但大量匿名事件会让对象联系难以追踪；核心业务流程仍应保留清晰的数据源、契约和调用边界。

697. 中介者模式

中介者模式让多个同级对象不再直接互相调用，而是把交互统一交给中介者协调。本质上是用中介者与各对象的一对多关系，替代对象之间复杂的网状多对多关系，因此各对象可以独立调整，耦合也更松。购物车中颜色、内存和数量表单的联动，或聊天室成员间发消息，都可以由中介者转发处理。取舍是交互复杂度会集中到中介者里，它很容易变得庞大并且难以维护。在中介者模式中，中介者（Mediator）包装了一

系列对象相互作用的方式，使得这些对象不必直接相互作用，而是由中介者协调它们之间的交互，从而使它们可以松散耦合。当某些对象之间的作用发生改变时，不会立即影响其他的一些对象之间的作用，保证这些作用可以彼此独立的变化。中介者模式和观察者模式有一定的相似性，都是一对多的关系，也都是集中式通信，不同的是中介者模式是处理同级对象之间的交互，而观察者模式是处理Observer和Subject之间的交互。中介者模式有些像婚恋中介，相亲对象刚开始并不能直接交流，而是要通过中介去筛选匹配再决定谁和谁见面。场景 例如购物车需求，存在商品选择表单、颜色选择表单、购买数量表单等等，都会触发change事件，那么可以通过中介者来转发处理这些事件，实现各个事件间的解耦，仅仅维护中介者对象即可。

```
var goods = { // 手机库存 red|32G:3, red|64G:1, blue|32G:7, blue|64G:6; } // 中介者
var mediator = (function() {
    var colorSelect = document.getElementById('colorSelect');
    var memorySelect = document.getElementById('memorySelect');
    var numSelect = document.getElementById('numSelect');
    return {
        changed: function(obj) {
            switch(obj) {
                case colorSelect: // TODO break;
                case memorySelect: // TODO break;
                case numSelect: // TODO break;
            }
        },
        colorSelect: colorSelect,
        memorySelect: memorySelect,
        numSelect: numSelect
    };
})();
colorSelect.onchange = function() { mediator.changed(this); };
memorySelect.onchange = function() { mediator.changed(this); };
numSelect.onchange = function() { mediator.changed(this); };

// 聊天室里 聊天室成员类
function Member(name) {
    this.name = name;
    this.chatroom = null;
}
Member.prototype = {
    // 发送消息
    send: function(message, toMember) {
        this.chatroom.send(message, this, toMember);
    },
    // 接收消息
    receive: function(message, fromMember) {
        console.log(`${fromMember.name} to ${this.name}: ${message}`);
    }
};

// 聊天室类
function Chatroom() {
    this.members = {};
}
Chatroom.prototype = {
    // 增加成员
    addMember: function(member) {
        this.members[member.name] = member;
        member.chatroom = this;
    },
    // 发送消息
    send: function(message, fromMember, toMember) {
        toMember.receive(message, fromMember);
    }
};

// 测试一下
const chatroom = new Chatroom();
const bruce = new Member('bruce');
const frank = new Member('frank');
chatroom.addMember(bruce);
chatroom.addMember(frank);
bruce.send('Hey frank', frank);
// 输出: bruce to frank: hello frank
```

优点 使各对象之间耦合松散，而且可以独立地改变它们之间的交互。中介者和对象一对多的关系取代了对象之间的网状多对多的关系。如果对象之间的复杂耦合度导致维护很困难，而且耦合度随项目变化增速很快，就需要中介者重构代码。缺点 系统中会新增一个中介者对象...

1) 购物车页面里，颜色下拉框直接修改容量选项，容量选项又直接控制数量输入框；只有3个控件时能跑，为什么还值得改成中介者？

答：能跑不代表关系清晰，控件之间的直接调用会形成网状依赖，任一交互变化都可能牵动多个对象。让控件只上报change，由中介者决定库存、选项和数量状态，可把多对多关系收敛为一对多；控件始终固定且交互简单时，引入它反而增加层次。

2) 商品详情页有颜色、容量、数量三个表单和一张组合库存表，你会怎样设计中介者的输入与输出？

答：每个表单只把自身变化和当前值交给中介者，中介者读取完整选择并查询类似red|32G的库存键，再统一更新可选项、数量上限和购买按钮。库存判断应集中在中介者内，避免各控件复制规则；若中介者同时负责请求、渲染和埋点，职责会迅速膨胀。

3) 产品新增套餐、优惠券和配送方式后，原来的一个changed(obj)已经出现十几个case，你会继续扩展还是拆分？

答：十几个case表明交互复杂性已集中转移到中介者，应按库存、促销或履约等稳定业务边界拆分协调逻辑。各子中介者仍只处理同级对象的协作，必要时再由上层编排；拆得过细会产生中介者之间的新耦合，重新形成难追踪的调用链。

4) 线上出现“切换蓝色后购买按钮仍可点，但该容量库存为零”，你会沿哪些事件链排查？

答：先确认颜色控件是否触发change，再检查事件是否把正确对象和值交给mediator.changed，最后核对组合库存键与按钮更新分支。示例式switch(obj)很依赖对象身份和分支完整性，新增控件却漏注册或漏处理都会留下旧状态；修复后还要覆盖组合切换的回归用例。

5) 表单团队主张用观察者模式广播所有字段变化，业务团队坚持使用中介者统一决策，你如何划分两者？

答：字段变化仅需通知多个独立订阅者时，观察者的Subject—Observer关系更自然；字段彼此同级且交互结果需要协调时，中介者更合适。两者都属于集中式一对多通信，但中介者承载对象间协作规则；把所有通知都塞进去会让它沦为难维护的上帝对象。

698. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。

```
// 访问者
class Visitor {
  constructor() {}
  visitConcreteElement(ConcreteElement) {
    ConcreteElement.operation()
  }
}
```

```
// 元素类
class ConcreteElement {
  constructor() {}
  operation() {
    console.log("ConcreteElement.operation invoked");
  }
  accept(visitor) {
    visitor.visitConcreteElement(this)
  }
}
```

```
clientlet visitor = new Visitor()
let element = new ConcreteElement()
elementA.accept(visitor)

访问者模式的实现有以下几个要素：
Visitor Object：访问者对象，拥有一个visit()方法
Receiving Object：接收对象，拥有一个accept()方法
visit(receivingObj)：用于Visitor接收一个Receiving Object
accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力
简单的代码实现如下：
Receiving Object：
function Employee(name, salary) {
  this.name = name;
  this.salary = salary;
}
Employee.prototype = {
  getSalary: function () {
    return this.salary;
  },
  setSalary: function (salary) {
    this.salary = salary;
  },
  accept: function (visitor) {
    visitor.visit(this);
  }
}
Visitor Object：
function Visitor() {}
Visitor.prototype = {
  visit: function (employee) {
    employee.setSalary(employee.getSalary() * 2);
  }
}
验证一下：
const employee = new Employee('bruce', 1000);
const visitor = new Visitor();
employee.accept(visitor);
console.log(employee.getSalary());
//输出：2000
```

场景
对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提

供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

699. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor { constructor() {} visit ConcreteElement(ConcreteElement) { ConcreteElement.operation(); } }

元素类class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }

clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving

Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving

Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {

getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },

accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() {} Visitor.prototype =

{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const

employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);

console.log(employee.getSalary()); // 输出：2000

场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

700. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }
```

元素类

```
class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

client

```
let visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素：

Visitor	Object
访问者对象，拥有一个visit()方法	Receiving Object
接收对象，拥有一个accept()方法	visit(receivingObj)：用于Visitor接收一个Receiving Object
accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力	简单的代码实现如下：

```
Receiving Object: function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object: function Visitor() {} Visitor.prototype =
```

```
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

701. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。

```
// 访问者class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation(); } }  
元素类class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

```
clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() {} Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000 场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

702. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法Receiving Object：接收对象，拥有一个 accept() 方法visit(receivingObj)：用于Visitor接收一个Receiving Objectaccept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary());//输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作"污染"这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

703. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }
```

元素类

```
class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

```
clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素：

- Visitor Object：访问者对象，拥有一个visit()方法
- Receiving Object：接收对象，拥有一个accept()方法
- visit(receivingObj)：用于Visitor接收一个Receiving Object
- accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力

简单的代码实现如下：

```
Receiving Object：function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object：function Visitor() {} Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下：const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

704. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } } // 元素类
```

```
class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } } // client
```

```
let visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object

accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object：

```
function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() {} Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000 场景
```

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

705. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }
```

元素类

```
class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

client

```
let visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素：

- Visitor Object：访问者对象，拥有一个visit()方法
- Receiving Object：接收对象，拥有一个accept()方法
- visit(receivingObj)：用于Visitor接收一个Receiving Object
- accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力

简单的代码实现如下：

```
Receiving Object：function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object：function Visitor() {} Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下：const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作

时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

706. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor {
    constructor() {}
    visitConcreteElement(ConcreteElement) {
        ConcreteElement.operation()
    }
}
```

元素类

```
class ConcreteElement {
    constructor() {}
    operation() {
        console.log("ConcreteElement.operation invoked");
    }
    accept(visitor) {
        visitor.visitConcreteElement(this)
    }
}
```

client

```
let visitor = new Visitor()
let element = new ConcreteElement()
element.accept(visitor)
```

访问者模式的实现有以下几个要素：

- Visitor Object：访问者对象，拥有一个visit()方法
- Receiving Object：接收对象，拥有一个 accept() 方法visit(receivingObj)：用于Visitor接收一个Receiving Object
- accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力

简单的代码实现如下：

```
Receiving      Object：      function
```

```
Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {
  getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },
  accept: function (visitor) { visitor.visit(this); }} Visitor Object : function Visitor() {} Visitor.prototype =
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); }} 验证一下： const
employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);
console.log(employee.getSalary()); //输出：2000
```

场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点：符合单一职责原则优秀的扩展性灵活性。缺点：具体元素对访问者公布细节，违反了迪米特原则；违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难。

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

707. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor { constructor() { visit

```
ConcreteElement(ConcreteElement){ConcreteElement.operation()}}//
```

```
元素类class ConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}}accept(visitor){visitor.visitConcreteElement(this)}}//
```

```
clientlet visitor=new Visitor()let element=new ConcreteElement()elementA.accept(visitor)
```

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving

Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving

Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

```
Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = {
```

```
getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },
```

```
accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype =
```

```
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const
```

```
employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);
```

```
console.log(employee.getSalary()); //输出：2000 场景
```

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象

进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作

时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪

米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

708. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor{constructor(){visitConcreteElement(ConcreteElement){ConcreteElement.operation()}}//

元素类class ConcreteElement{constructor(){operation(){console.log("ConcreteElement.operation invoked");}}accept(visitor){visitor.visitConcreteElement(this)}}//

clientlet visitor=new Visitor()lelement=new ConcreteElement()elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit() 为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() { } Visitor.prototype = { visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); //输出：2000 场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

709. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。

```
// 访问者
class Visitor {
  constructor() {}
  visitConcreteElement(ConcreteElement) {
    ConcreteElement.operation();
  }
}
```

```
// 元素类
class ConcreteElement {
  constructor() {}
  operation() {
    console.log("ConcreteElement.operation invoked");
  }
  accept(visitor) {
    visitor.visitConcreteElement(this);
  }
}
```

```
clientlet visitor = new Visitor();
let element = new ConcreteElement();
elementA.accept(visitor);

访问者模式的实现有以下几个要素：
Visitor Object：访问者对象，拥有一个visit()方法
Receiving Object：接收对象，拥有一个accept()方法
visit(receivingObj)：用于Visitor接收一个Receiving Object
accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力
简单的代码实现如下：
Receiving Object：
function Employee(name, salary) {
  this.name = name;
  this.salary = salary;
}
Employee.prototype = {
  getSalary: function () {
    return this.salary;
  },
  setSalary: function (salary) {
    this.salary = salary;
  },
  accept: function (visitor) {
    visitor.visit(this);
  }
};
Visitor Object：
function Visitor() {}
Visitor.prototype = {
  visit: function (employee) {
    employee.setSalary(employee.getSalary() * 2);
  }
};
验证一下：
const employee = new Employee('bruce', 1000);
const visitor = new Visitor();
employee.accept(visitor);
console.log(employee.getSalary());
//输出：2000
```

场景
对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要对一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提

供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

710. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式 是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者class Visitor { constructor() {} visit ConcreteElement(ConcreteElement) { ConcreteElement.operation(); } }

元素类class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }

clientlet visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)

访问者模式的实现有以下几个要素： Visitor Object：访问者对象，拥有一个visit()方法 Receiving

Object：接收对象，拥有一个 accept() 方法 visit(receivingObj)：用于Visitor接收一个Receiving

Object accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的 visit()

为其提供获取Receiving Object数据的能力 简单的代码实现如下： Receiving Object： function

Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; },

accept: function (visitor) { visitor.visit(this); } } Visitor Object： function Visitor() {} Visitor.prototype =

{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const

employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor);

console.log(employee.getSalary()); // 输出：2000

场景

对象结构中对对象对应的类很少改变，但经常需要在此对象结构上定义新的操作 需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。 优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。

7.11. 访问者模式

访问者模式把操作逻辑从对象结构中拆出来，让对象不改结构也能增加新操作。接收对象通过accept(visitor)把自己交给访问者，访问者再用visit()读取数据或执行逻辑。本质上它适合对象类型较稳定、操作经常变化的场景，也能避免无关逻辑污染元素类。代价是元素要暴露实现细节，而且新增或调整具体元素时，访问者往往也要跟着修改。访问者模式是一种将算法与对象结构分离的设计模式，通俗点讲就是：访问者模式让我们能够在不改变一个对象结构的前提下能够给该对象增加新的逻辑，新增的逻辑保存在一个独立的访问者对象中。访问者模式常用于拓展一些第三方的库和工具。 // 访问者

```
class Visitor { constructor() {} visitConcreteElement(ConcreteElement) { ConcreteElement.operation() } }
```

元素类

```
class ConcreteElement { constructor() {} operation() { console.log("ConcreteElement.operation invoked"); } accept(visitor) { visitor.visitConcreteElement(this); } }
```

client

```
let visitor = new Visitor(); let element = new ConcreteElement(); elementA.accept(visitor)
```

访问者模式的实现有以下几个要素：

- Visitor Object：访问者对象，拥有一个visit()方法
- Receiving Object：接收对象，拥有一个accept()方法
- visit(receivingObj)：用于Visitor接收一个Receiving Object
- accept(visitor)：用于Receiving Object接收一个Visitor，并通过调用Visitor的visit()为其提供获取Receiving Object数据的能力

简单的代码实现如下：

```
Receiving Object: function Employee(name, salary) { this.name = name; this.salary = salary; } Employee.prototype = { getSalary: function () { return this.salary; }, setSalary: function (salary) { this.salary = salary; }, accept: function (visitor) { visitor.visit(this); } } Visitor Object: function Visitor() {} Visitor.prototype =
```



```
{ visit: function (employee) { employee.setSalary(employee.getSalary() * 2); } } 验证一下： const employee = new Employee('bruce', 1000); const visitor = new Visitor(); employee.accept(visitor); console.log(employee.getSalary()); // 输出：2000
```

场景

对象结构中对象对应的类很少改变，但经常需要在此对象结构上定义新的操作。需要在一个对象结构中的对象进行很多不同的并且不相关的操作，而需要避免让这些操作“污染”这些对象的类，也不希望在增加新操作时修改这些类。优点 符合单一职责原则优秀的扩展性灵活性 缺点 具体元素对访问者公布细节，违反了迪米特原则违反了依赖倒置原则，依赖了具体类，没有依赖抽象。具体元素变更比较困难

1) 工资管理页只有一个Employee类，却为了“薪资翻倍”新增accept(visitor)和Visitor，这一定比直接写方法好吗？

答：不一定；只有一个元素类型和一次操作时，直接方法更短，也不会暴露额外协作接口。访问者的价值在于对象结构较稳定而新操作持续增加，可把薪资调整等逻辑移出Employee；若操作不会扩展，这套分派结构只是额外成本。

2) 你接手一个不能随意修改的第三方节点库，需要增加校验、导出和统计三种处理，访问者怎样落地？

答：为每种处理建立独立访问者，由接收对象通过accept(visitor)把自身传给对应的visit方法，使算法不混入节点类。这样新增第四种操作主要增加一个访问者，而不是改写全部节点行为；前提是库能提供接收入口或可控适配层，否则无法机械套用。

3) 编辑器节点结构半年不变，但导出格式从一种增加到五种；另一个团队却每周新增节点类型，哪边更适合访问者？

答：导出操作持续增加而节点类型稳定的编辑器更适合，每种格式可作为独立访问者扩展。频繁新增节点类型的一方代价更高，因为已有访问者通常都要识别新元素；访问者优化的是“新增操作”，并不擅长“频繁改变对象结构”。

4) 线上新增VideoNode后，HTML导出正常，统计结果却漏掉视频数量，你会先查哪里？

答：先检查VideoNode.accept是否调用了正确的访问方法，再逐个核对统计访问者是否实现了对应处理。具体元素变化困难正是该模式的边界，新类型容易让部分访问者遗漏分支；应通过类型覆盖检查或遍历用例验证所有访问者，而不能只测一个导出结果。

5) 架构评审中，一方要把所有操作放回节点类以保持封装，另一方要用访问者避免污染节点，你怎么取舍？

答：若操作与节点核心职责紧密且数量有限，留在节点类能减少对象细节外泄；若存在许多互不相关且持续增加的操作，访问者更符合单一职责。代价是具体元素需向访问者提供数据并依赖具体分派，封装性、迪米特原则和依赖倒置都会承受压力。